Kalman and Bayesian Filters in Python

Python 卡尔曼滤波与贝叶斯滤波

审校修订版

原著：Roger R. Labbe Jr.

14 章正文 · 前言 · 原仓库全部 7 个附录

2026 年 10 月

版本、来源与使用说明

本版在英中对照审校及页面校样基础上，完成一轮全书错别字、漏字与语句检查。公式采用常规字重；动画在静态 PDF 中以完整画面呈现。全书 22 个笔记本的 608 个代码单元已在 Python 3.12 的 Jupyter 内核中逐章执行通过。

本版为非官方中文译本。正文及附录 E、G 参考 pleb631 的中英对照版本，并按所选英文原稿逐句校订；前言和附录 A、B、D、H、I 根据英文原稿补译。数学公式以英文 LaTeX 为基础校订排版，程序代码保留原有算法，并修复已失效的绘图库参数和接口；少量图表按原算法重绘并改善标注布局；代码注释、变量名、运行输出及图中英文标签保留，便于复现。相关数学定义、假设与技术说明融入正文。中文翻译与审校使用 AI 辅助完成，未获原作者或出版社审定。

本版收录所选仓库根目录的前言、第 1—14 章和全部 Appendix 笔记本；辅助实验与动画生成笔记本不另列章节。原仓库没有附录 C、F。附录 D、G、H 含作者尚未完成的内容，附录 I 只有标题，均保留原稿的完成状态；其中附录 G 的草稿推导不宜直接用作工程计算依据。PDF 为静态阅读版，交互演示须在 Jupyter 中运行。

原作者：Roger R. Labbe Jr.；原作：Kalman and Bayesian Filters in Python。 所选英文仓库：Penghui-Zhang/Kalman-and-Bayesian-Filters-in-Python 。 中文参考：pleb631/Kalman-and-Bayesian-Filters-in-Python-chinese（提交 6867aff40a76662af98aa89e20739a9b6a25b3f4）。参考项目及其贡献者的工作在此致谢。本次补译与 PDF 整理使用 AI 辅助完成。

许可说明：原仓库 README 声明书籍为 CC BY 4.0，前言却保留了较早的 CC BY-NC-SA 4.0 声明。本版同时保留来源及历史声明，并以署名—非商业性使用—相同方式共享 4.0 条件提供本次翻译整理。软件代码适用原 MIT 许可证。本版进行了翻译、术语整理及版式调整；不表示原作者或参考译本作者认可这些修改。

阅读建议：先读前言和第 1—4 章，建立预测与更新的直觉，再进入多元、非线性和粒子滤波。使用目录链接可以直接跳转。公式中的符号含义、适用条件及实现细节随正文解释。

目录

前言

第 1 章：g-h 滤波器

第 2 章：离散贝叶斯滤波

第 3 章：高斯分布

第 4 章：一维卡尔曼滤波器

第 5 章：多元高斯分布

第 6 章：多元卡尔曼滤波器

第 7 章：卡尔曼滤波的数学基础

第 8 章：卡尔曼滤波器的设计

第 9 章：非线性滤波

第 10 章：无迹卡尔曼滤波器

第 11 章：扩展卡尔曼滤波器

第 12 章：粒子滤波

第 13 章：平滑

第 14 章：自适应滤波

附录 A：安装

附录 B：符号与记法

附录 D：H∞ 滤波器

附录 E：集合卡尔曼滤波器

附录 G：非线性卡尔曼滤波器的设计

附录 H：最小二乘滤波器

附录 I：性能的解析评估

前言

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

这是一本介绍卡尔曼滤波与贝叶斯滤波的入门教材。本书使用 Jupyter Notebook 编写，因此你既可以在浏览器中阅读，也可以运行、修改代码，并直接在书中看到结果。还有什么比这更好的学习方式呢?

卡尔曼滤波与贝叶斯滤波

传感器存在噪声。世界上有许多我们希望测量和跟踪的数据与事件，但不能指望传感器提供完美的信息。我的汽车 GPS 会报告海拔；每次经过道路上的同一个位置，读数都会略有不同。用厨房秤给同一个物体称两次重量，也会得到不同的读数。

在简单情况下，解决办法很明显。如果秤的读数只是略有差异，多测几次再取平均值就可以，或者换一台更精确的秤。但如果传感器噪声很大，或者环境让数据采集变得困难，该怎么办？我们可能要跟踪低空飞机，为无人机设计自动驾驶仪，或确保农用拖拉机播种覆盖整片田地。我从事计算机视觉工作，需要跟踪图像中的运动物体，而计算机视觉算法产生的结果往往噪声很大，并不可靠。

本书教你解决这类滤波问题。我会使用许多不同的算法，但它们都建立在*贝叶斯概率*之上。简单地说，贝叶斯概率根据过去的信息判断什么更可能为真。

如果我问你此刻汽车的航向，你大概毫无头绪，只能在 1° 到 360° 之间猜一个数，猜中的概率是 1/360。现在假设我告诉你，两秒前航向是 243°。两秒之内汽车不可能转向太多，因此你能做出准确得多的预测。你正在利用过去的信息，更准确地推断现在或未来的情况。

世界本身也有噪声。这样的预测有助于改善估计，但仍然受噪声影响。我可能刚刚为躲避一只狗而刹车，或者为绕过坑洞而转向。强风和路面结冰也是影响行驶路径的外部因素。在控制领域，我们把它们称为*噪声*，尽管你平时未必这样称呼。

贝叶斯概率远不止这些，但你已经掌握了基本思想：知识是不确定的，我们应根据证据的强弱调整信念。卡尔曼滤波和贝叶斯滤波把关于系统行为的、有噪声且有限的知识，与有噪声且有限的传感器读数结合起来，尽可能准确地估计系统状态。我们的原则是不要丢弃信息。

假设正在跟踪一个物体，传感器报告它突然改变了方向。它真的转向了，还是数据存在噪声？这取决于具体情况。如果目标是喷气式战斗机，我们很可能相信突然机动的报告；如果它是直轨上的货运列车，我们就会降低这一报告的可信度。传感器的精度也会进一步影响判断。我们的信念取决于历史、对被跟踪系统的了解，以及传感器的特性。

鲁道夫·埃米尔·卡尔曼（Rudolf Emil Kálmán）发明了卡尔曼滤波，以数学上最优的方式解决这类问题。它早期的重要应用包括阿波罗登月任务，此后应用遍及各个领域：飞机、潜艇、巡航导弹，华尔街的市场跟踪，机器人、物联网传感器和实验室仪器；化工厂用它控制和监测反应，医学成像和心脏信号去噪也用得到它。只要涉及传感器或时间序列数据，通常就会涉及卡尔曼滤波或其近亲。

本书的写作动机

我是一名软件工程师，在航空航天领域工作了近二十年，因此总是与卡尔曼滤波打交道，却一直没有亲手实现过。它素以难学著称。理论很优美，但如果没有信号处理、控制理论、概率统计以及制导与控制等方面的扎实训练，就很难掌握。后来我开始用计算机视觉解决跟踪问题，自己实现滤波器的需要变得迫切。

这个领域有许多优秀教材，例如 Grewal 和 Andrews 的《Kalman Filtering》。但缺少必要背景时，坐下来阅读这些书会令人沮丧。通常，开头几章就匆匆扫过本科数年的数学内容，轻描淡写地让你去查伊藤微积分教材，并用短短几段介绍一学期的统计学。它们适合高年级本科生或研究生，也是研究人员和专业人士的重要参考资料，但普通读者读起来十分吃力。符号未作解释就被引入，不同教材为同一概念使用不同术语和变量名，例子和完整解题过程又很少。我常常能够读懂文字、理解定义中的数学，却不知道这些文字和数学试图描述什么现实现象。我反复问自己：“但这到底*意味着什么*？”下面就是曾经困扰我的典型例子：

$$\hat{x}_k = \Phi_k\hat{x}_{k-1} + G_k u_{k-1} + K_k[z_k - H\Phi_k\hat{x}_{k-1} - HG_k u_{k-1}]$$
$$\mathbf{P}_{k|k} = (I - \mathbf{K}_k\mathbf{H}_k)\mathrm{cov}(\mathbf{x}_k - \hat{\mathbf{x}}_{k|k-1})(I - \mathbf{K}_k\mathbf{H}_k)^{\mathbf{T}} + \mathbf{K}_k\mathrm{cov}(\mathbf{v}_k)\mathbf{K}_k^{\mathbf{T}}$$

然而，当我终于开始理解卡尔曼滤波时，发现其基本概念其实相当直接。只要掌握几个简单的概率规则，并对如何融合不确定知识有一些直觉，就能够理解它。卡尔曼滤波虽然以困难闻名，但去掉大量形式化术语后，这门学科及其数学之美逐渐显现，我也爱上了这个主题。

理解数学和理论之后，又出现了其他困难。书籍或论文会提出某个结论，再画一幅图作为证明，但我并不清楚结论为何成立，或者无法复现图形。我也可能想问：“R=0 时还成立吗？”有的作者提供过于抽象的伪代码，让人看不出如何实现；有的书提供 Matlab 代码，但我没有这个昂贵软件的许可证。许多书在章末列出很有用的习题，想自己实现卡尔曼滤波就必须掌握，却不提供答案。课堂教学中这或许可以接受，对独立读者却很糟糕。我厌恶作者为了防止课堂上的学生“作弊”而拒绝提供这些信息。

这些都会妨碍学习。我想跟踪屏幕上的图像，或为 Arduino 项目编程；想知道书中的图怎样绘制，并尝试作者没有选择的参数；想运行仿真，向信号加入更多噪声，看看滤波器的表现。日常编程有成千上万种使用卡尔曼滤波的机会，但这样一个相当直观的主题，却仿佛成了火箭科学家和学者的专属领域。

我写这本书，就是为了满足这些需要。如果你要设计军用雷达，这本书不能作为唯一教材；你需要在优秀的理工科院校获得硕士或博士层次的训练。本书面向爱好者、好奇的读者，以及需要对数据进行滤波或平滑的工程师。对爱好者来说，它应当提供所需的全部基础；如果希望深入研究卡尔曼滤波，还需要更多知识。我的目标是介绍足够的概念与数学，让你能够读懂专业教材和论文。

本书具有交互性。虽然可以把它当作静态网页阅读，我仍强烈建议按设计的方式使用它。Jupyter Notebook 让我能够把文字、数学、Python 和程序输出放在同一处。本书每幅图、每项数据，都是由笔记本里的 Python 代码生成的。想把某个参数加倍？改掉参数值，按下 CTRL-ENTER，新的图形或输出就会出现。

本书有习题，也有答案。我信任你。如果只是需要答案，就直接阅读；如果想真正掌握知识，请先尝试完成习题。由于本书可交互，你能直接在书中输入并运行自己的解答，不必切换环境，也不必先导入一大堆东西。

本书免费。我在卡尔曼滤波书籍上花了几千美元，很难相信经济困难地区的读者或拮据的学生能够负担这些开销。我从 Python 这样的免费软件，以及 Allen B. Downey 的免费书籍 [1] 中受益良多，现在该回报了。因此，本书免费，托管在 GitHub 的免费服务器上，并且只使用 IPython、MathJax 等自由开放的软件。

在线阅读

GitHub

本书托管在 GitHub，点击章节名称即可阅读。GitHub 会静态渲染 Jupyter Notebook；你可以阅读全部内容，但不能运行或修改代码。

项目地址：

https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

binder

binder 在线提供可交互笔记本，无需下载本书或安装 Jupyter，就能在浏览器中运行和修改代码。入口：

http://mybinder.org/repo/rlabbe/Kalman-and-Bayesian-Filters-in-Python

nbviewer

nbviewer 能把任意 Notebook 渲染为静态页面。我认为效果略好于 GitHub，但使用稍麻烦。它直接访问 GitHub，因此会渲染我提交到 GitHub 的内容。

本书入口：

http://nbviewer.ipython.org/github/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/table_of_contents.ipynb

PDF 版本

我会定期把笔记本生成为 PDF，可以从以下地址获取：

https://drive.google.com/file/d/0By_SW19c1BfhSVFzNHc0SjduNzg/view?usp=sharing

下载和运行本书

本书的设计目标是交互式使用，我也建议这样阅读。虽然配置稍费功夫，但值得。安装 IPython 及相关库、克隆本书后，你就能运行所有代码，做实验，观察滤波器如何应对不同数据，以及不同滤波器处理相同数据时的差异。我认为这种即时反馈非常重要，也令人振奋。不必只问“如果这样会怎样”，直接试一试！

安装说明见附录 A：安装。

软件安装后，进入本书所在目录，在命令行运行：

```
jupyter notebook
```

浏览器会打开，显示根目录内容。本书按章组织，各章文件名为 *xx-name*.ipynb，其中 *xx* 是章节编号，.ipynb 是 Notebook 扩展名。章节笔记本直接放在仓库根目录中，打开相应文件即可。有时还会有用于生成章节动画等内容的辅助笔记本；它们不要求读者阅读，但好奇时可以查看动画的实现方法。

诚然，用这种界面阅读一本书有些笨拙。我沿用了其他一些把 Jupyter Notebook 用来生成整本书的项目的做法。但这种小小的不便换来的收益很大：你无需下载单独的代码库，也不必一边看书一边在 IDE 中运行，所有文字和代码都在同一处。你可以修改代码，立刻观察结果；发现问题后可以修复并向仓库提交改进。更重要的是，你不会遇到传统图书常见的麻烦——书和代码版本不同步，令人困惑不知该信哪一个。

Jupyter

先介绍一下在本书中使用 Jupyter Notebook 的注意事项。本书是交互式的；要运行示例代码，特别是查看动画图形，就需要执行代码单元。我无法在这里讲完 Jupyter 的全部用法，但有几个地方经常让读者困惑。详细文档见 http://jupyter.org/ 。

首先，一定要运行最上方带有 `#format the book` 注释的代码单元。它就在上方。它不仅设置格式，还会加载必要模块，并设置绘图和打印相关的全局选项。因此，除非只是静态阅读，否则请始终运行这个单元。

```
%matplotlib inline
```

这行代码使图形显示在笔记本内部。Matplotlib 是后面会介绍的绘图库。出于我不明白的原因，Jupyter Notebook 默认可能把图形画在外部窗口中。

`%matplotlib` 的百分号表示 IPython 的*魔法命令*：这些命令让内核执行 Python 语言本身之外的操作。还有许多实用的魔法命令，参见：http://ipython.readthedocs.io/en/stable/interactive/magics.html 。

运行一个单元很简单：点击它使其获得焦点（外面会出现边框），然后按 CTRL-Enter。

其次，必须按顺序执行单元。我会把一个问题拆成多个单元；如果直接跳到第十个代码单元运行，几乎肯定不行。如果尚未运行任何内容，从 **Cell** 菜单选择 *Run All Above*，便可方便地保证前面的代码全部执行。

运行过单元之后，有时可以跳转并以不同顺序重新执行，但并非总可以。我正在努力改善这一点，不过需要权衡。例如，第 10 个单元定义了一个变量，第 11、12 个单元修改它；若随后重新执行第 11 个单元，变量已经是第 12 个单元设置的值，而代码预期的却是第 10 个单元的值。因此乱序执行偶尔会产生奇怪结果。我

的建议是往前回退一些，再按顺序运行，恢复正确状态。虽然烦人，交互式 Notebook 的优势足以弥补。更好的做法是在 GitHub 提交问题，让我知道并修复！

最后，有读者报告某些浏览器的动画绘图有问题，我尚无法复现。本书部分位置使用 `%matplotlib notebook` 开启交互绘图。若无法显示，可以改成 `%matplotlib inline`。这会失去动画效果，但似乎能在所有平台和浏览器上工作。

SciPy、NumPy 与 Matplotlib

SciPy 是开源数学软件集合。相关生态包括提供数组、线性代数、随机数等功能的 NumPy，以及绘制 NumPy 数组图形的 Matplotlib。SciPy 的模块与 NumPy 有部分功能重叠，并增加了优化、图像处理等能力。

为了让本书的工作量可控，我假定你已经会使用 Python 编程，并且熟悉这些包。不过，我仍会花一点篇幅展示它们的部分功能；更详细的知识需要查阅其他资料。SciPy 主页 https://scipy.org 是很好的起点，之后你可能希望查找相关教程或视频。

NumPy、SciPy 和 Matplotlib 并不随默认 Python 发行版提供。如果尚未安装，请参阅*安装*附录。

本书会大量使用 NumPy 数组，因此现在先来学习。我会介绍足够的入门知识；若要精通，请阅读 NumPy 文档。

`numpy.array` 实现一维或多维数组。它的类型是 `numpy.ndarray`，下文简称 ndarray。可以用任意类似列表的对象创建数组。下面从列表构建一个一维数组：

```
import numpy as np
x = np.array([1, 2, 3])
print(type(x))
x
```

```
<class 'numpy.ndarray'>
```

```
array([1, 2, 3])
```

使用 `import numpy as np` 已经成为业界惯例。

也可以使用元组：

```
x = np.array((4,5,6))
x
```

```
array([4, 5, 6])
```

用嵌套方括号创建多维数组：

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
print(x)
```

```
[[1 2 3]
 [4 5 6]]
```

也能创建三维或更高维数组，但这里用不到，因此不再展开。

默认情况下，数组沿用列表元素的数据类型；若存在多种类型，就选择能尽量准确表示全部数值的类型。例如，列表同时含有 `int` 和 `float` 时，数组类型会是 `float`。可以用 `dtype` 参数覆盖默认选择。

```
x = np.array([1, 2, 3], dtype=float)
print(x)
```

```
[1. 2. 3.]
```

使用下标访问数组元素：

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])

print(x[1,2])
```

```
6
```

可以用切片获取一列或一行。下标中的冒号 `:` 表示该行或该列的全部数据。因此，`x[:,0]` 返回第一列的全部数据（0 表示第一列）：

```
x[:, 0]
```

```
array([1, 4])
```

获取第二行：

```
x[1, :]
```

```
array([4, 5, 6])
```

获取第二行的最后两个元素：

```
x[1, 1:]
```

```
array([5, 6])
```

与 Python 的 `list` 一样，负索引可从数组末尾计数，-1 表示最后一个位置。因此，获取第二行（也就是最后一行）最后两个元素的另一种方式是：

```
x[-1, -2:]
```

```
array([5, 6])
```

矩阵加法可以使用 + 运算符；矩阵乘法则需要 dot 方法或函数。* 运算符执行逐元素相乘，**不是**线性代数中的矩阵乘法。

```
x = np.array([[1., 2.],
              [3., 4.]])
print('addition:\n', x + x)
print('\nelement-wise multiplication\n', x * x)
print('\nmultiplication\n', np.dot(x, x))
print('\ndot is also a member of np.array\n', x.dot(x))
```

```
addition:
 [[2. 4.]
 [6. 8.]]

element-wise multiplication
 [[ 1.  4.]
 [ 9. 16.]]

multiplication
 [[ 7. 10.]
 [15. 22.]]

dot is also a member of np.array
 [[ 7. 10.]
 [15. 22.]]
```

Python 3.5 引入了用于矩阵乘法的 @ 运算符。

```
x @ x
```

```
array([[ 7., 10.],
       [15., 22.]])
```

这需要 Python 3.5 或更高版本。本书要求 3.6 或更高版本，所以我会尽量使用它。注意，运算符两侧都必须是数组。因此，`x @ 3.` 会引发 ValueError，而 `np.dot(X, 3.)` 可以正常工作。

用 `.T` 获取转置，用 `numpy.linalg.inv` 求逆。SciPy 也提供求逆函数。

```
import scipy.linalg as linalg
print('transpose\n', x.T)
print('\nNumPy ninverse\n', np.linalg.inv(x))
print('\nSciPy inverse\n', linalg.inv(x))
```

```
transpose
 [[1. 3.]
 [2. 4.]]

NumPy ninverse
 [[-2.   1. ]
 [ 1.5 -0.5]]

SciPy inverse
 [[-2.   1. ]
 [ 1.5 -0.5]]
```

还有一些辅助函数：`zeros` 创建全零矩阵，`ones` 创建全一矩阵，`eye` 创建单位矩阵。如果要创建多维数组，可以用元组指定形状。

```
print('zeros\n', np.zeros(7))
print('\nzeros(3x2)\n', np.zeros((3, 2)))
print('\neye\n', np.eye(3))
```

```
zeros
 [0. 0. 0. 0. 0. 0. 0.]

zeros(3x2)
 [[0. 0.]
 [0. 0.]
 [0. 0.]]

eye
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

也有创建等间隔数据的函数。`arange` 与 Python 的 `range` 很相似，只是返回 NumPy 数组。`linspace` 略有不同，调用方式是 `linspace(start, stop, num)`，其中 `num` 是所需数组的长度。

```
np.arange(0, 2, 0.1)
```

```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. , 1.1,
       1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9])
```

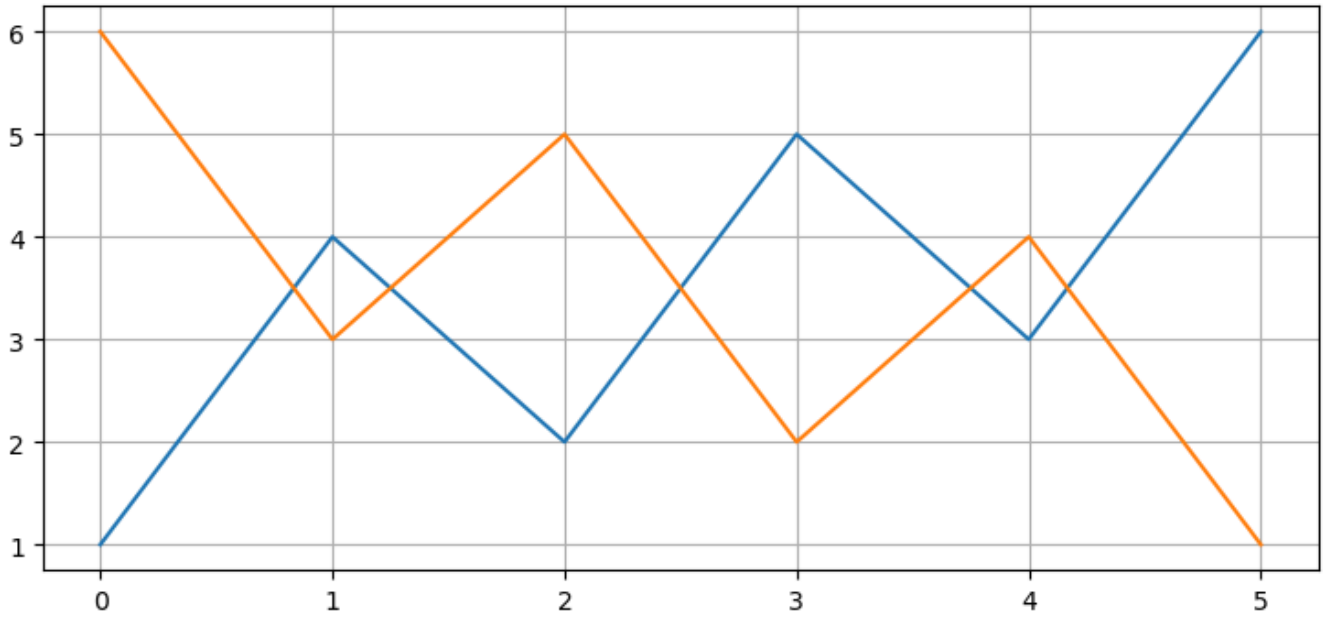
```
np.linspace(0, 2, 20)
```

```
array([0.   , 0.105, 0.211, 0.316, 0.421, 0.526, 0.632, 0.737, 0.842,
       0.947, 1.053, 1.158, 1.263, 1.368, 1.474, 1.579, 1.684, 1.789,
       1.895, 2.   ])
```

现在画一些数据。大多数情况下很简单。Matplotlib 包含名为 `pyplot` 的绘图库，惯例是将它导入为 `plt`。导入后，把数字列表或数组传给 `plt.plot` 即可绘图。多次调用会画出多条数据序列，并使用不同颜色。

```
import matplotlib.pyplot as plt
a = np.array([6, 3, 5, 2, 4, 1])
plt.plot([1, 4, 2, 5, 3, 6])
plt.plot(a)
```

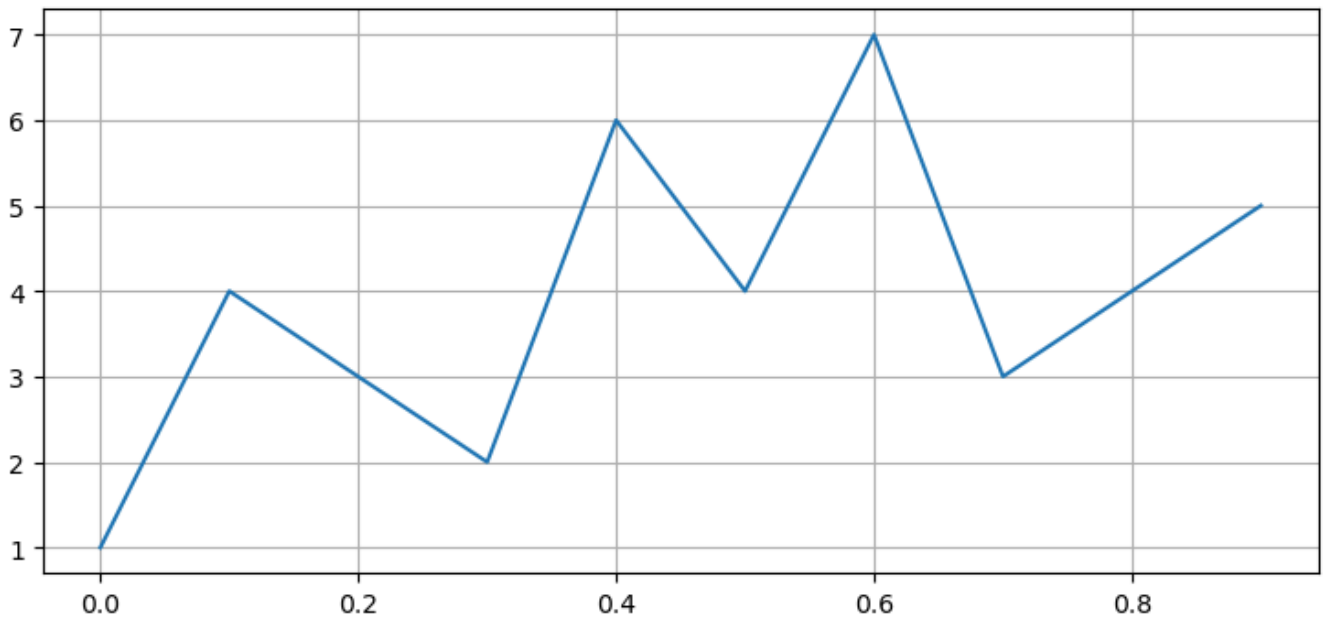
```
[<matplotlib.lines.Line2D at 0x1ecdb844b80>]
```



输出 `[<matplotlib.lines.Line2D at 0x2ba160bed68>]` 是因为 `plt.plot` 返回刚创建的对象。通常不希望显示它，因此我会在最后一条绘图语句后加上分号 `;` 来抑制输出。

默认情况下，`plot` 假定 x 序列每次递增 1。同时传入 x 和 y，就可以使用自己的 x 序列。

```
plt.plot(np.arange(0,1, 0.1), [1,4,3,2,6,4,7,3,4,5]);
```



本书还会使用这些包的许多其他功能。通常我不会逐一解释，而是相信你能够根据上下文理解，或者上网查找说明。与往常一样，如果不确定，就在 Notebook 中新建一个单元，或打开 Python 控制台亲手试验！

练习：创建数组

请创建一个含有 10 个元素的 NumPy 数组，每个元素都等于 1/10。有多种方法，试着实现尽可能多的方案。

```
# your solution
```

解答

下面有三种方法。第一种是我希望你掌握的：用 / 运算符把数组的所有元素除以 10。稍后会用同样的方式把数组中的单位从米换算成千米。

```
print(np.ones(10) / 10.)
print(np.array([.1, .1, .1, .1, .1, .1, .1, .1, .1, .1]))
print(np.array([.1] * 10))
```

```
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

下面还有一种尚未介绍的方法。`numpy.asarray()` 会在参数不是 ndarray 时，将其转换为 ndarray；若已经是 ndarray，则数据不变。这让函数可以方便地同时接收 Python 列表和 ndarray，而当输入本来就是 ndarray 时，无需创建新对象，效率很高。

```
def one_tenth(x):
    x = np.asarray(x)
    return x / 10.

print(one_tenth([1, 2, 3]))            # I work!
print(one_tenth(np.array([4, 5, 6])))  # so do I!
```

```
[0.1 0.2 0.3]
[0.4 0.5 0.6]
```

配套软件

我正在编写一个名为 FilterPy 的开源贝叶斯滤波 Python 库，安装方法见附录 A。

FilterPy 托管在 GitHub：https://github.com/rlabbe/filterpy ，不过通过 `pip` 安装的版本通常已经足够。

本书专用代码与书放在一起，位于 **/kf_book** 子目录。里面包含书籍格式化代码，以及形如 *xxx*_internal.py 的 Python 文件。这些文件保存对特定章节有用的函数，让我能够把不太值得逐行阅读的代码隐藏起来。例如，生成图表时，我希望你关注图表的内容，而非 Python 绘图的具体步骤。如果对此感兴趣，可以直接阅读源代码。

某些章节引入的函数会在后文继续使用。这些函数最初在 Notebook 中定义，同时也保存在 **/kf_book** 的 Python 文件里，供后续章节按需导入。我会在首次定义时说明这一点，但相关工作仍在推进。我尽量避免这样做，因为目录中的代码可能与书中的代码不同步。不过，Jupyter Notebook 没有提供跨笔记本引用代码单元的方法，这是我目前知道的共享功能的唯一办法。

另有一个没有文档的 **/experiments** 目录，用来编写和测试准备放入书中的代码，其中有一些有趣的内容，可以自由查看。随着本书发展，我计划创建示例和项目，许多材料会移入其中；小型试验则可能删除。如果只是读书，可以放心忽略该目录。

/kf_book 中还有定义本书外观的 CSS 样式文件。IPython Notebook 默认外观较朴素，这方面仍在改进。我借鉴了 Probabilistic Programming and Bayesian Methods for Hackers [2] 等项目，也深受 Lorena Barba 教授优秀工作的影响：CFDPython [3]。本书的外观设计得益于这些项目。

关于 Python 和数学代码的想法

大多数卡尔曼滤波及工程教材由数学家或学者编写。即使提供软件（这并不常见），通常也没有达到生产级质量。以 Paul Zarchan 的《Fundamentals of Kalman Filtering》为例：这是一本值得收藏的好书，也是少数为每个例子和图表提供完整源代码的教材之一。但代码使用 Fortran，除了 `MATMUL` 等函数调用外，几乎没有子程序。全书反复实现卡尔曼滤波器，同一段程序把仿真与滤波混在一起，难以区分。某些章节用略有差异的方式实现同一个滤波器，再用粗体突出改动的几行。如果需要龙格—库塔方法，就把实现直接嵌入代码，甚至没有注释。

有更好的方式。如果我需要龙格—库塔积分，就调用 `ode45`，而不是把实现嵌在自己的代码中。我不想多次实现同一算法并反复调试。若发现错误，只要修复一次，就能确信所有不同项目都得到修复；代码也更易读。大多数时候，我并不关心龙格—库塔算法内部如何实现。

这是一本卡尔曼滤波教材，你也许会说，我们确实关心卡尔曼滤波器的实现。这没错，但真正执行滤波的代码只有大约 10 行，数学部分的实现相当简单。使用卡尔曼滤波器的大部分工作，其实是设计输入数学计算核心的那些矩阵。

潜在的缺点是：滤波方程被封装到函数背后，似乎损害了教学价值。我持相反看法。我希望你学会在真实世界的项目中使用卡尔曼滤波，而不应到处复制粘贴已经成熟的算法。

我使用 Python 类，主要是为了组织滤波器需要的数据，而非实现继承等面向对象特性。例如，`KalmanFilter` 类保存名为 `x`、`P`、`R`、`Q`、`S`、`y`、`K` 的矩阵与向量。我见过过程式的卡尔曼滤波库，要求程序员自己维护这些矩阵。在小程序中也许还好，但实现一组卡尔曼滤波器时，手动管理所有矩阵及相关数据会非常麻烦。我在自己的工作中偶尔也会继承这些类，觉得很方便；不过我知道很多人不喜欢面向对象，因此不想强迫所有人采用它。

许可证

Roger Labbe 的《Kalman Filters and Random Signals in Python》前言列出的许可为 Creative Commons 署名—非商业性使用—相同方式共享 4.0 国际许可协议（CC BY-NC-SA 4.0）。

http://creativecommons.org/licenses/by-nc-sa/4.0/

基于以下项目的工作：

https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

仓库 README 另列有 CC BY 4.0 声明。为兼顾两处许可信息，本中文译本按署名—非商业性使用—相同方式共享 4.0 条件提供；软件代码继续遵循 MIT 许可证。

联系方式

rlabbejr@gmail.com

参考资源

- [1] http://www.greenteapress.com/
- [2] http://nbviewer.ipython.org/github/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers/blob/master/Chapter1_Introduction/Chapter1.ipynb
- [3] https://github.com/barbagroup/CFDPython

第 1 章：g-h 滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

在开始之前，请确保你已经了解如何使用 Jupyter Notebooks，并熟悉 SciPy、NumPy 和 Matplotlib 这些软件包。因为本书全程都会用到这些工具。前言部分对这些软件包进行了简要介绍。

通过思维实验培养直觉能力

想象一下，我们生活在一个没有秤的世界里——也就是没有那种用来称体重的仪器。有一天，一位同事跑过来告诉你，她发明了一种“秤”。听完她的介绍后，你立刻站到上面称重。结果显示：“172 磅”。你欣喜若狂——有生以来，你终于知道了自己的体重。更重要的是，一想到可以把这种设备卖给世界各地的减肥中心，你的眼睛里就充满了兴奋的光芒！真是太棒了！

另一位同事听到喧闹声后过来询问发生了什么事。你向他解释了这项发明的原理，然后再次站上体重秤，自豪地宣布了结果：“161 磅。”但随后，你又犹豫了，显得有些困惑。

“几秒钟前还显示 172 磅呢，”你向同事抱怨道。

“我从没说过那是准确的，”她回答道。

传感器的精度并不高。这也是为什么有大量研究致力于解决滤波问题。而解决这个问题，正是本书要探讨的主题。我本可以直接介绍过去半个世纪中人们所提出的各种解决方案，但这些方案都是通过深入探讨我们所知道的事物及其认知方式而得出的。在开始进行数学计算之前，让我们先回顾一下这一探索过程，看看它能否帮助我们更深入地理解滤波的原理。

试试另一个秤吧。

有没有什么办法可以改进这个测量结果呢？最显而易见的办法就是使用精度更高的传感器。不过，你的同事告诉你，她已经制作了 10 台秤，而它们的测量精度大致相同。你让她再拿出一台秤来，你分别用这两台秤称了自己的体重。第一台秤的显示结果是“160 磅”，第二台秤的显示结果是“170 磅”。那么，关于你的体重，我们能得出什么结论呢？

那么，我们有哪些选择呢？

- 我们可以选择只相信 A 的说法，然后将 160 磅作为我们的体重估算值。
- 我们可以选择只相信 B 的说法，那么我们的体重就应该是 170 磅。
- 我们可以选择一个小于 A 和 B 的数字。
- 我们可以选择一个大于 A 和 B 的数。
- 我们可以在 A 和 B 之间选择一个数字。

前两种选择都有一定道理，但我们没有理由偏好其中某一种。我们为什么要选择 A 而不是 B 呢？没有任何理由支持这种选择。第三和第四种选择则是不合理的。虽然这两种测量工具的准确性不高，但完全没有理由选择超出它们所测量范围之外的数值。最后一种选择才是合理的。如果两种测量工具都不准确，那么它们得出的结果很可能高于或低于我的实际体重。在这种情况下，答案很可能介于 A 和 B 之间。

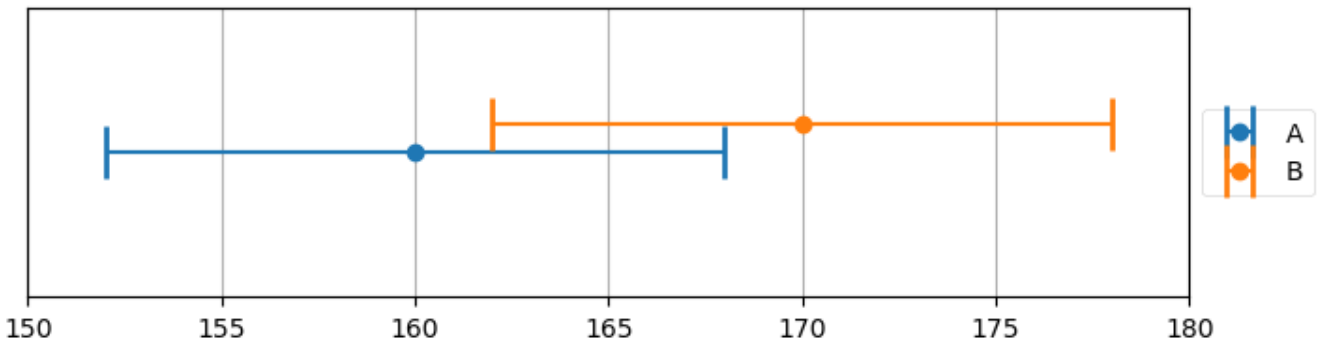
在数学中，这个概念可用期望值形式化，后面会详细介绍。先想一想：如果测量一百万次，会发生什么？有时两台秤都偏低，有时都偏高，其余时候两个读数分别位于真值两侧。若读数跨在真值两侧，就应在 A、B 之间估计；若没有跨在两侧，我们不知道它们是共同偏高还是共同偏低，但在两者之间选择，至少能减弱较差读数的影响。例如真值为 180 磅，160 磅的误差很大；取 160—170 磅之间的值，就比取 160 磅更接近真值。两台秤都偏高时也有类似道理。

我们稍后会更正式地探讨这个问题，但目前希望大家都能明白：我们的最佳估算值其实是 A 和 B 的平均值。

$$\frac{160 + 170}{2} = 165$$

我们可以用图表来直观地理解这一点。我根据假设的误差范围 ±8 磅，绘制了 A 和 B 的测量值。从图表可以看出，测量值介于 160 磅到 170 磅之间。因此，合理的重量数值必然位于 160 磅到 170 磅的范围内。

```
import kf_book.book_plots as book_plots
from kf_book.book_plots import plot_errorbars
plot_errorbars([(160, 8, 'A'), (170, 8, 'B')], xlims=(150, 180))
```



关于我如何生成这个图表的说明：我从 `kf_book` 子目录中的 book_plots 模块中导入代码来生成图表。生成这个图表需要编写大量的 Python 代码，这些代码相当繁琐，阅读起来也不有趣。在本书中，我经常采用这种方式来生成图表。当相关代码被执行时， `plot_errorbars()` 会被调用，从而将图表插入到书中。

如果你是第一次使用 Jupyter Notebook，那么上面的代码就位于一个单元格中。文本“In [2]:”表示这是一个可以输入代码的单元格。括号中的数字表示该单元格是第二个被执行的单元格。要运行某个单元格，用鼠标点击该单元格使其成为焦点，然后按 CTRL+ENTER 键。在后续操作中，你可以修改单元格中的代码并重新运行它们。试着把“160”、“170”和“8”这些数值换成其他数字，然后再运行该单元格。根据你输入的数值，输出结果也会相应发生变化。

如果你想查看 plot_errorbars 函数的代码，可以在编辑器中打开该文件，或者新建一个单元格，输入函数名后加上两个问号。按下 Ctrl+Enter 键，浏览器就会打开一个窗口，显示该函数的源代码。这是 Jupyter Notebooks 的独特功能。如果你只想查看该函数的文档，只需在函数名后加上一个问号即可。

```
plot_errorbars??
```

或者

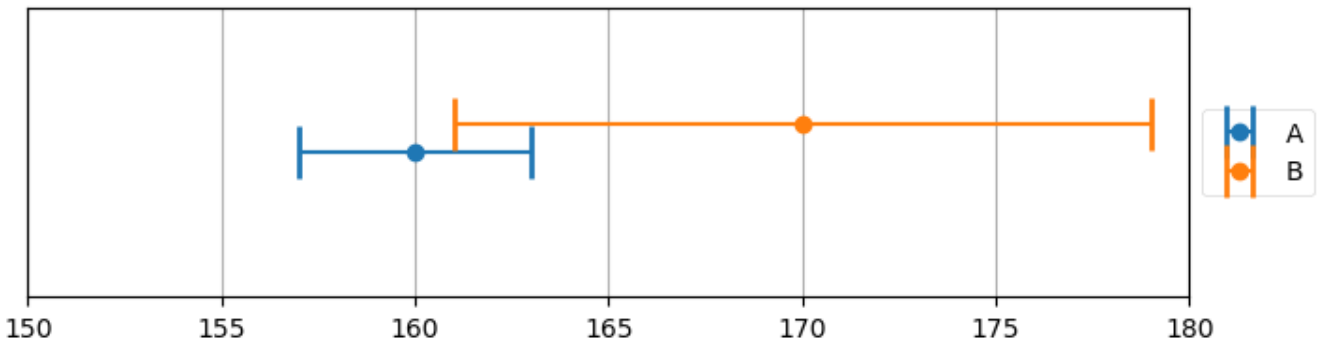
```
plot_errorbars?
```

因此，165 磅似乎是一个合理的估计值。不过，这里还有其他一些信息可以加以利用。只有那些位于 A 和 B 的误差范围之间的数值才是可行的。例如，161 磅是不可能的，因为 B 秤的最大误差为 8 磅，所以它不可能显示出 170 磅的读数。同样，169 磅也是不可能的，因为 A 秤的最大误差为 8 磅，所以它不可能显示出 160 磅的读数。在这个例子中，只有 162 到 168 磅这个范围内的数值才是可行的。

这还不能给出更好的体重估计。再假设一下：若 A 的测量误差只有 B 的三分之一呢？回顾前面五个选项，在 A、B 两个读数所界定范围之外取值仍然没有依据，因此不考虑这两项。直接选 A 似乎更有吸引力，因为它更准确。但加入较差的 B，能否比只使用 A 获得更多信息？

答案也许违反直觉：是的，确实可以。先看同样的测量值 A=160、B=170，但这次 A 的误差为 ±3 磅，而 B 的误差是它的 3 倍，即 ±9 磅。

```
plot_errorbars([(160, 3, 'A'), (170, 9, 'B')], xlims=(150, 180))
```

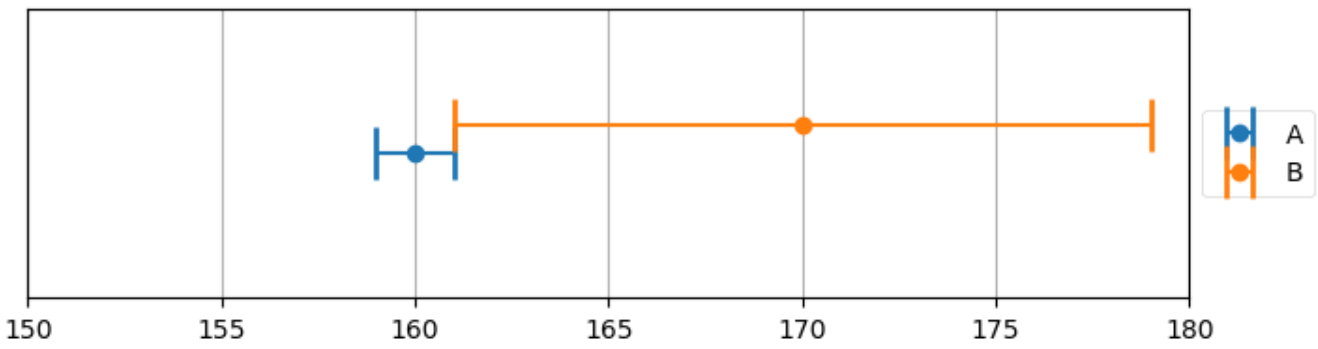


A 和 B 的误差范围之间的重叠区域，才代表了唯一可能的真实重量值。这一重叠区域比 A 单独的误差还要小。更重要的是，从图中可以看出，这一重叠区域并不包括 160 磅或 165 磅这个数值。如果我们只使用 A 的测量结果，因为它比 B 更准确，那么得到的估计值将是 160 磅。而如果我们将 A 和 B 的测量结果取平均

值，那么得到的估计值就是 165 磅。但根据我们对秤的准确度的了解，这两种数值都是不可能的。通过同时考虑 A 和 B 的测量结果，我们可以得到 161 磅到 163 磅之间的估计值，这一范围正好是两个误差范围的交集所在。

让我们把这种情况推向极端。假设秤 A 的精确度为 1 磅。也就是说，如果我们真正的体重是 170 磅，那么秤 A 的显示结果可能是 169 磅、170 磅或 171 磅。同时，我们知道秤 B 的精确度为 9 磅。如果我们用这两台秤分别称重，得到的结果是：秤 A 显示 160 磅，秤 B 显示 170 磅。那么，我们应该将自己的体重估计为多少呢？我们可以通过图表来直观地了解这个问题。

```
plot_errorbars([(160, 1, 'A'), (170, 9, 'B')], xlims=(150, 180))
```



从这里可以看出，唯一的合理重量数值就是 161 磅。这是一个非常重要的结果。尽管使用了两个精度不太高的传感器，我们仍然能够得到极其精确的测量结果。

所以，即使两个传感器中有一个的精度较低，但总比只有一个传感器要好。在本书的剩余部分，我都会反复强调这一点。我们绝不能放弃任何信息，无论其质量有多差。我们将开发出相应的数学模型和算法，以便能够利用所有可能的信息来源，从而得出最准确的估算结果。

不过，我们偏离了主题。没有哪个客户会愿意购买多台秤来使用。另外，我们最初假设所有秤的测量精度都差不多。这种“无论精度如何，都把所有测量数据都考虑进来”的思路后来会发挥重要作用，所以请不要忘记这一点。

如果我只有一台秤，但用它来称重很多次会怎么样呢？我们得出结论：如果使用两台精度相同的秤来称重，那么应该将两次测量的结果取平均值。那么，如果我用同一台秤称重 10,000 次呢？我们已经知道，这台秤给出过高或过低的重量数值的概率是相等的。不难证明，多次测量的平均值会非常接近真实的体重。不过，我们现在先来进行一次模拟吧。我会使用 NumPy 来进行计算，NumPy 是 SciPy 生态系统中的数值计算工具。

```
import numpy as np
measurements = np.random.uniform(160, 170, size=10000)
mean = measurements.mean()
print(f'Average of measurements is {mean:.4f}')
```

```
Average of measurements is 164.9353
```

实际打印出来的数字会因随机数生成器的不同而有所差异，但应该会非常接近 165。

这段代码有个通常不符合实际的假设：真实体重为 165 磅时，读出 160 磅与读出 165 磅同样可能。实际传感器通常更容易给出接近真值的读数，偏离越远越少见。高斯分布一章会详细解释。目前先使用 numpy.random.normal()：它会产生更多接近 165 磅的数值，远离它的数值更少。暂且把它看作一种近似真实体重秤的带噪测量模型。

```
mean = np.random.normal(165, 5, size=10000).mean()
print(f'Average of measurements is {mean:.4f}')
```

```
Average of measurements is 164.9762
```

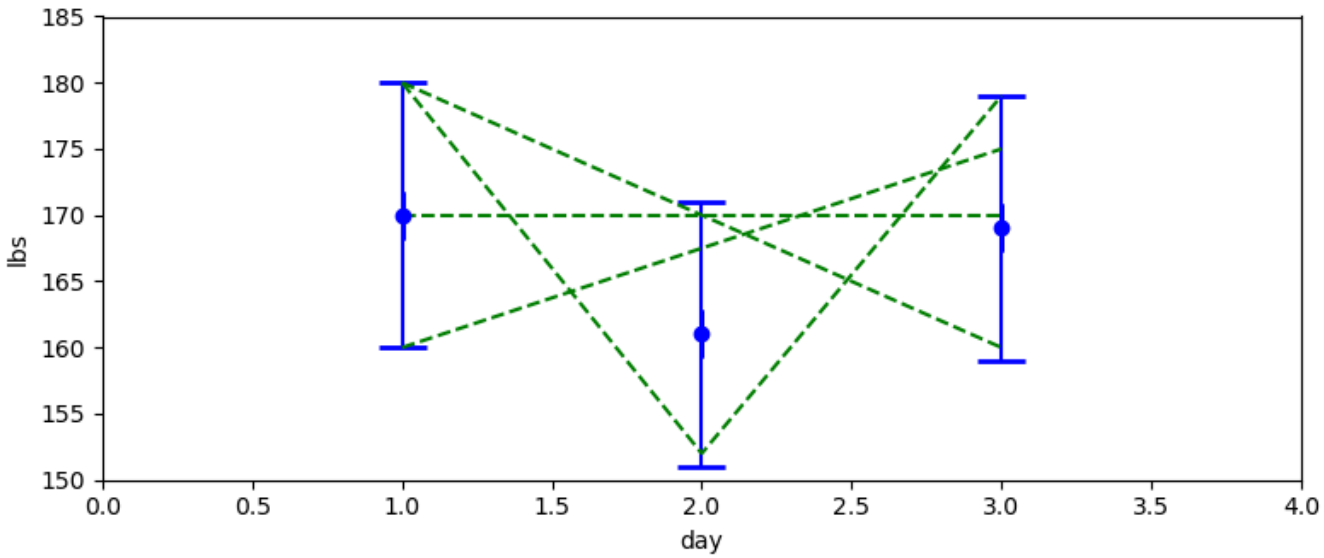
答案仍然非常接近 165。

很好，我们似乎解决了传感器的问题，但方法并不实用。谁也没有耐心为了称一次体重，重复一万次，甚至十来次。

那么，我们来玩个“假如”的游戏吧。假如你每天测量一次体重，得到的数值分别是 170、161 和 169，那你到底是胖了、瘦了，还是这些数值只是测量上的误差而已呢?

我们真的无法确定。第一次测量结果是 170 磅，最后一次则是 169 磅，这意味着体重减少了 1 磅。不过，如果秤的精确度只有 10 磅的话，这种误差也是可以理解的。实际上，我的体重可能有所增加；也许第一天我的体重是 165 磅，而第三天则上升到了 172 磅。这样的体重变化完全有可能被秤的误差所掩盖。秤显示我的体重在下降，但实际上我的体重却在上升！让我们用图表来仔细看看吧。我把各项测量结果以及误差范围都标了出来，同时用绿色虚线标出了根据这些测量结果可能出现的体重增减情况。

```
import kf_book.gh_internal as gh
gh.plot_hypothesis1()
```

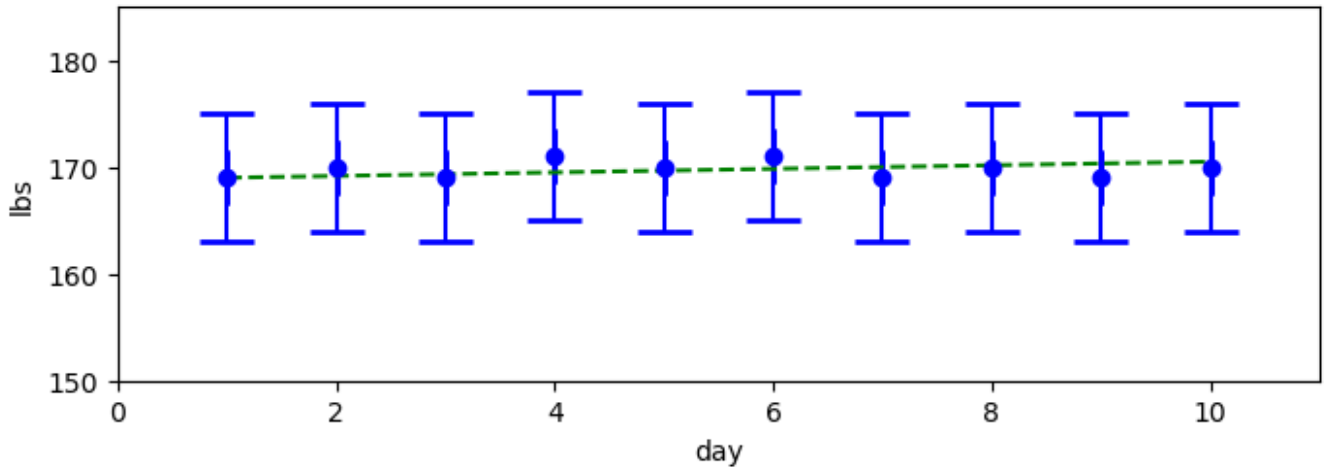


正如我们所看到的，这些测量结果所能解释的体重变化范围其实非常大。实际上，有无数种可能的体重变化情况。那我们是否应该放弃呢？我可不会！请记住，我们这里讨论的是人类的体重。一个人不可能在第一天体重为 180 磅，第三天却只有 160 磅；也不可能一天内减掉 30 磅的体重，第二天又把体重恢复过来（当然，我们假设这个人没有接受截肢手术，也没有遭受其他损伤）。

我们所测量的物理系统的行为，会影响到我们对测量结果的解读。如果我们每天都在称量同一块石头，那么我们会把所有的波动都视为偶然的误差。但如果我们称量的是一个通过雨水补给、用于家庭用途的蓄水池，那么我们可能会认为这种重量变化是真实存在的。

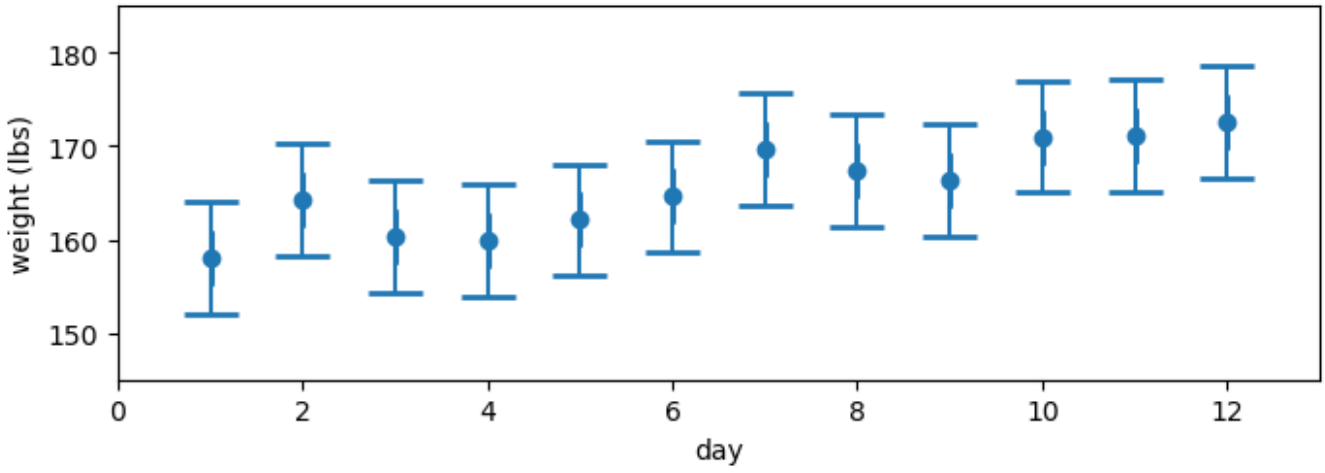
假设我使用不同的测量工具进行测量，得到的数据如下：169、170、169、171、170、171、169、170、169、170。凭直觉来看，这种情况可能意味着什么呢？比如，有可能你每天体重增加 1 磅，但由于测量误差，看起来你的体重没有变化。当然，也有可能你每天体重减少 1 磅，但测量结果仍然显示体重没有变化。不过，这种可能性有多大呢？抛硬币 10 次都正面朝上的概率有多大呢？显然不大。虽然我们无法仅凭这些数据来证明什么，但看起来我的体重很可能保持稳定。在下图中，我用误差条标出了测量结果，同时用虚线绿色线条表示了可能的真实体重。这条虚线并非所谓的“正确答案”，而只是根据测量结果得出的合理推测而已。

```
gh.plot_hypothesis2()
```



再假设一种情况：如果测量结果分别是 158.0、164.2、160.3、159.9、162.1、164.6、169.6、167.4、166.4、171.0 会怎么样呢？让我们先看一下对应的图表，然后再来回答一些问题。

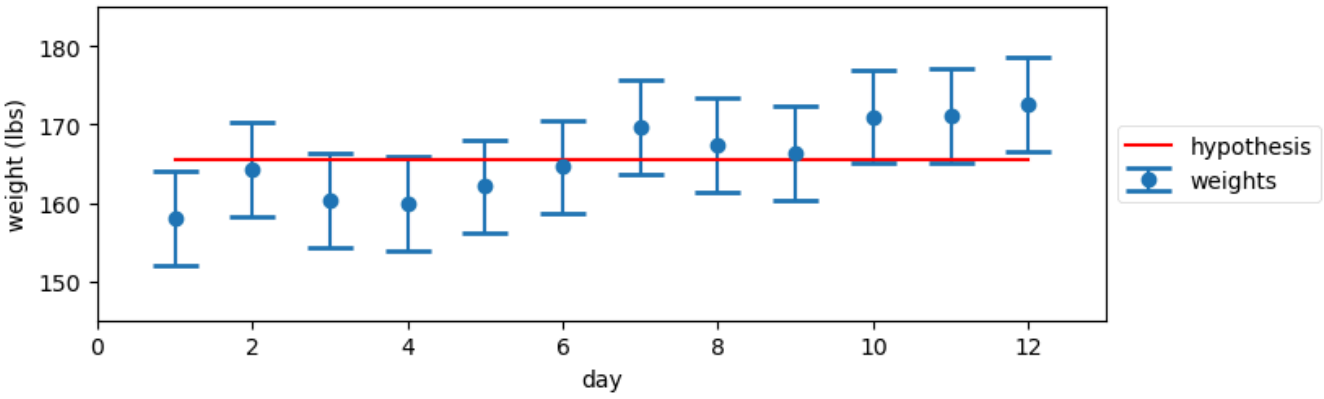
```
gh.plot_hypothesis3()
```



我觉得自己有可能减重了，而这些数据其实只是随机波动而已，这种可能性大吗？其实不大。那我觉得自己的体重可能保持不变，这种可能性呢？同样不大。从数据趋势来看，体重是呈上升趋势的；虽然不是均匀上升，但总体上是上升的。我们无法确定，但看起来像是体重增加了，而且增加的幅度相当可观。让我们用更多的图表来验证这个假设吧。通常来说，通过图表来分析数据比用表格更直观。

那么，让我们来考虑两种假设。首先，假设我们的体重没有发生变化。为了得出这个数值，我们同意应该对各项测量结果进行平均处理。让我们来具体看看吧。

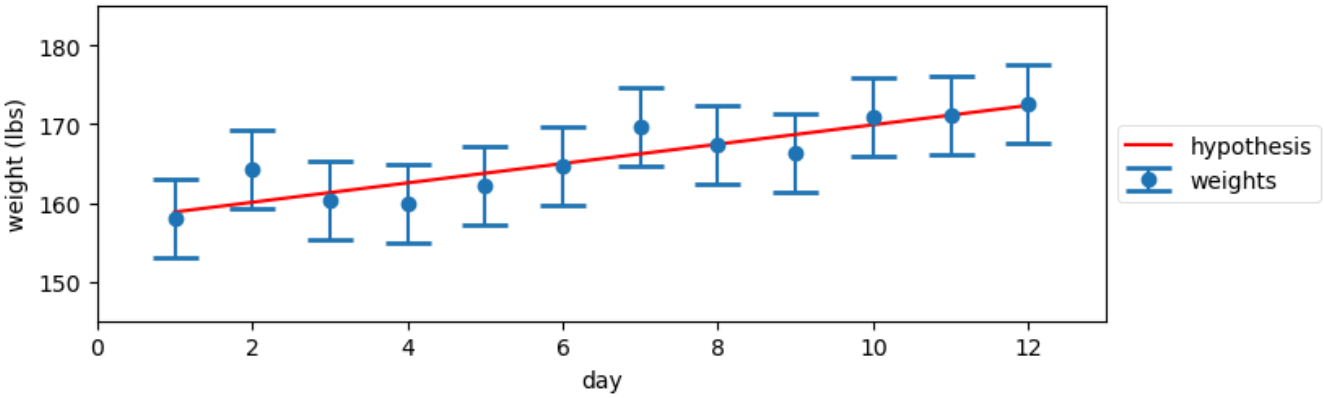
```
gh.plot_hypothesis4()
```



这看起来并不令人信服。事实上，我们可以看到，根本不存在一条能够完全位于所有误差范围之内的水平线。

现在，假设我们的体重增加了。增加了多少呢？我不知道，但 NumPy 知道！我们希望用一条直线来表示这些测量数据，这条直线看起来要“尽可能”符合实际情况。NumPy 有相应的函数可以实现这一点，其原理被称为“最小二乘法”。我们不必太担心具体的计算过程（如果你感兴趣，这里使用的是 `polyfit()` 函数）。只要把计算结果绘制出来就行了。

```
gh.plot_hypothesis5()
```



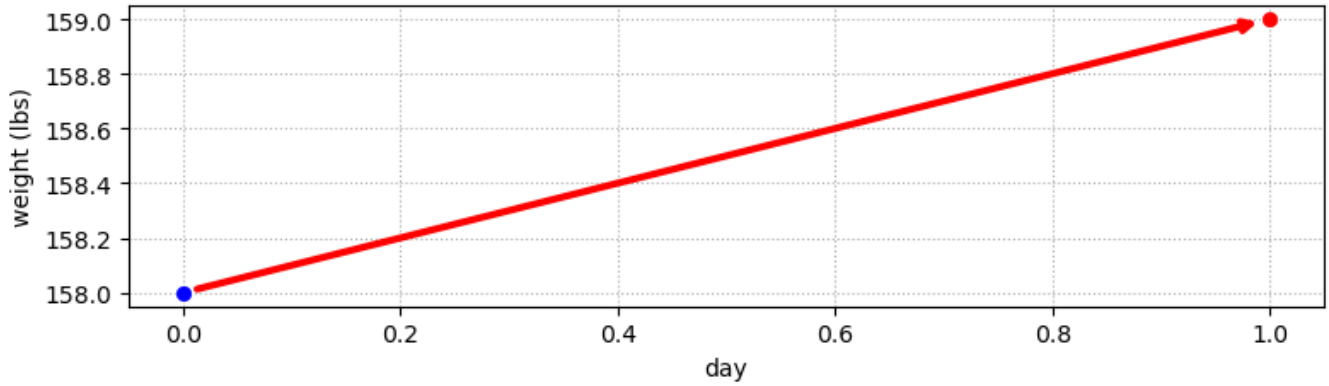
至少在我看来，这样看起来要好得多。现在可以看到，假设值与各个测量值都非常接近，而在之前的图表中，假设值往往与测量值相差甚远。看来，我确实长了体重的可能性要大得多，而不是完全没有长体重。我真的长了 13 磅吗？谁知道呢？这似乎是无法回答的问题。

“但那真的不可能吗？”一位同事插话道。

我们来试试某种“疯狂”的假设吧。假设我知道自己每天大约会增重一磅。现在不必管我是怎么知道这一点的，只要假设这个数值大致正确即可。也许我每天摄入 6000 卡路里的热量，这才导致了这样的体重增加。当然，也可能还有其他方法来估算体重变化。这只是一个思维实验而已，细节并不重要。让我们看看，如果拥有这些信息的话，我们能否加以利用。

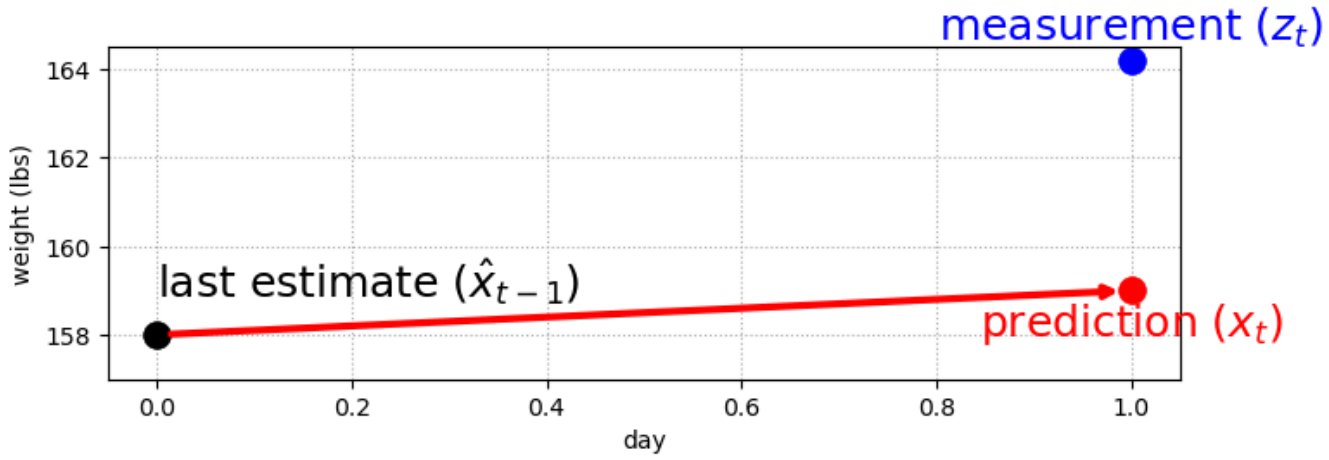
第一次测量得到的数值是 158。我们无法确定还有没有其他可能的数值，所以就以 158 作为我们的估算值吧。如果今天我们的体重是 158，那么明天会是多少呢？我们认为自己的体重每天增加 1 磅，所以预计明天的体重会是 159。具体来说就是：

```
gh.plot_estimate_chart_1()
```



不过，这有什么实际用处？我们可以假定每天增重 1 磅，再预测未来十天；但若完全不采用读数，还要秤做什么？因此看看下一次测量：再次站上秤，读数为 164.2 磅。

```
gh.plot_estimate_chart_2()
```



我们遇到了一个问题：我们的预测结果与实际测量结果并不一致。不过，这其实也是我们预料之中的吧？如果预测结果总是与测量结果完全一致的话，那它就毫无用处了，因为它无法为“滤波器”提供任何有用的信息。当然，既然我们的预测已经完美无缺，那也就没有必要再进行测量了。

> **整本书的关键见解就在下一段中。请仔细阅读！**

那我们该怎么办呢？如果我们仅根据测量结果来做出估计，那么预测结果就不会对最终结果产生影响。如果我们仅根据预测结果来做出估计，那么测量结果就会被忽略。要想让这个方法奏效，我们就需要将预测结果和测量结果结合起来使用（我已经把关键点用粗体标出来了）。

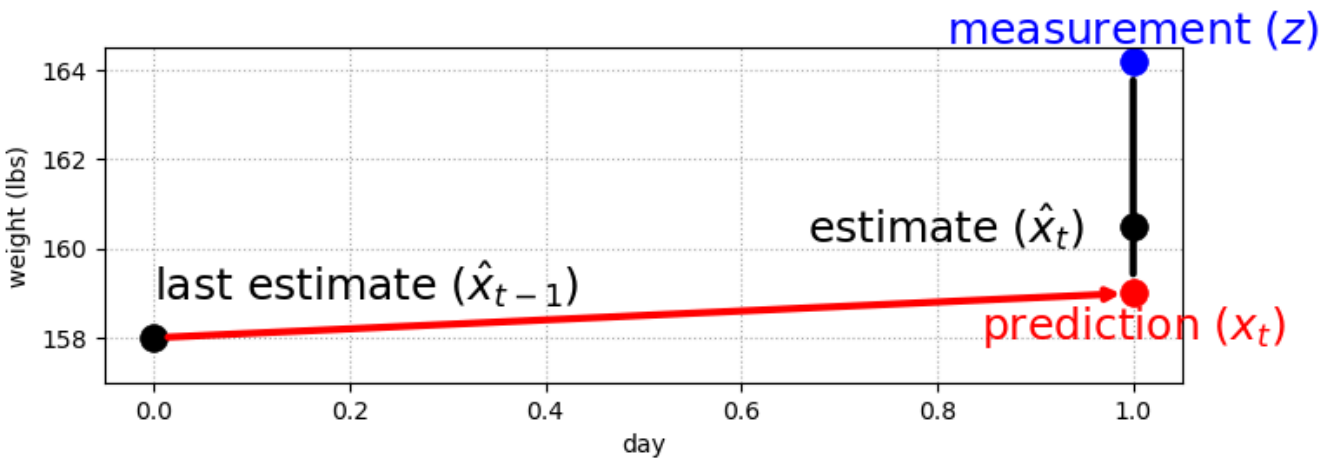
融合两个值，很像前面的两台秤问题。按相同思路，应在预测值和测量值之间取一个估计。例如取 165 或 157 都没有根据；这里估计应在 159（预测）与 164.2（测量）之间。

再强调一次，这一点非常重要。我们之前已经达成共识：当面对两个有误差的数值时，我们应该取这两个数值之间的中间值作为估算结果。这些数值的来源并不重要。在本章的开头，我们有两个测量结果；但现在，我们只有一个测量结果和一个预测值。无论哪种情况，其推理过程和数学计算方式都是相同的。我们绝不能丢弃任何信息。我是认真的。我见过很多商业软件会丢弃那些有误差的数据。千万不要这么做！虽然我们对体重增加的预测可能不太准确，但只要还有哪怕一点信息，我们都应该加以利用。

我必须坚持要求你停下来，认真思考一下这个问题。我所做的只不过是用基于人体生理学的预测结果来替代了那些不准确的测量结果而已。本质上，这仍然属于数据而已。数学并不关心这些数据是从测量仪器得到的，还是通过预测得出的。我们手头有两组带有某些误差的数据，现在需要把它们结合起来。在本书的后续内容中，我们将运用一些复杂的数学公式来进行这些计算。但数学并不关心数据的来源，它只关心数据的数值及其准确性。

估计值应该恰好位于预测与测量的正中间吗？未必。我们可能知道预测与测量的相对精度，二者一般并不相同。回忆 A 秤明显比 B 秤准确时，我们让融合结果更接近 A。下面用图说明。

```
gh.plot_estimate_chart_3()
```



现在任意选取一个缩放系数 $\frac{4}{10}$。估计由测量值的十分之四与预测值的十分之六组成。也就是说，我们表达了这样的信念：预测比测量稍可信一些。计算方法如下：

$$\textbf{estimate} = \textbf{prediction} + \frac{4}{10}(\textbf{measurement} - \textbf{prediction})$$

测量值减去预测值所得的差称为残差，上图用黑色竖线表示。后面还会反复用到它，因为它直接反映测量与预测之间的差异。在其他条件相同时，残差较小通常说明二者更一致。但残差较小本身不能保证估计更准确，还需要结合测量噪声、模型偏差以及滤波器对测量的依赖程度来判断。

下面编写代码，并用前面那组体重读数测试结果。还需要计入时间：增重率的单位是磅／单位时间，因此，为使算法具有一般性，引入时间步长 $\boldsymbol{t}$，这里取为 1 天。

我手动设置了这些体重数据：起始体重为 160 磅，之后每天体重增加 1 磅。也就是说，第一天时，实际体重为 160 磅；第二天时，实际体重为 161 磅，以此类推。

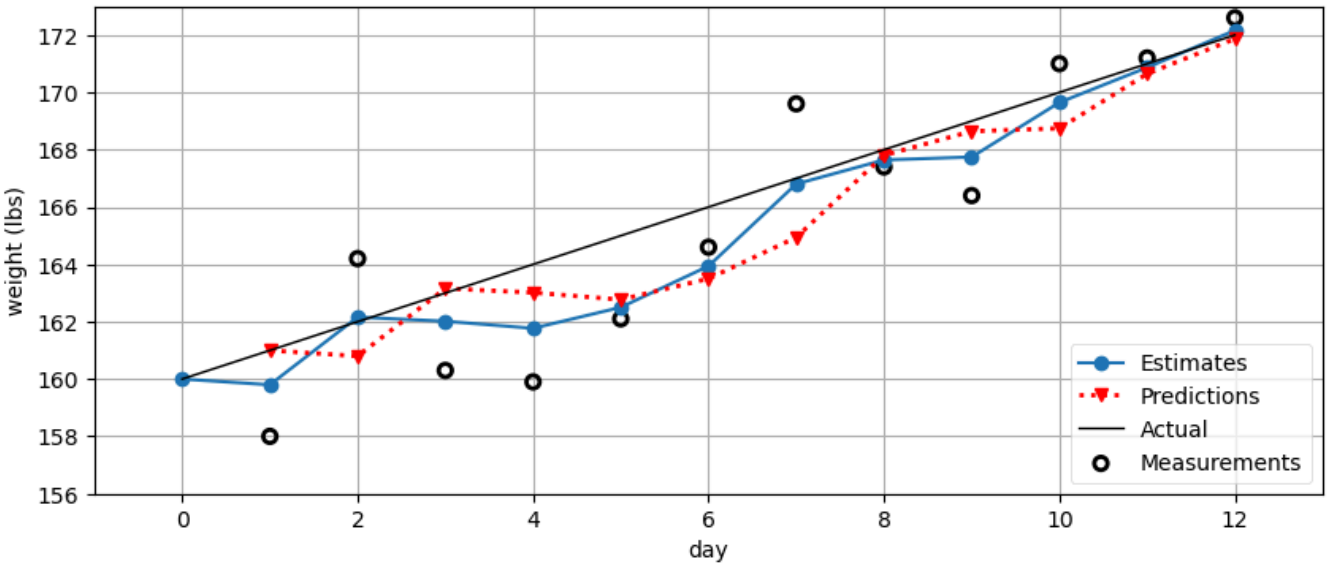
我们需要先对初始重量进行估算。现在讨论初始化策略还为时过早，所以我暂时假设初始重量为 160 磅。

```
from kf_book.book_plots import figsize
import matplotlib.pyplot as plt

weights = [158.0, 164.2, 160.3, 159.9, 162.1, 164.6,
           169.6, 167.4, 166.4, 171.0, 171.2, 172.6]

time_step = 1.0  # day
scale_factor = 4.0/10

def predict_using_gain_guess(estimated_weight, gain_rate, do_print=False):
    # storage for the filtered results
    estimates, predictions = [estimated_weight], []

    # most filter literature uses 'z' for measurements
    for z in weights:
        # predict new position
        predicted_weight = estimated_weight + gain_rate * time_step

        # update filter
        estimated_weight = predicted_weight + scale_factor * (z - predicted_weight)

        # save and log
        estimates.append(estimated_weight)
        predictions.append(predicted_weight)
        if do_print:
            gh.print_results(estimates, predicted_weight, estimated_weight)

    return estimates, predictions

initial_estimate = 160.
estimates, predictions = predict_using_gain_guess(
    estimated_weight=initial_estimate, gain_rate=1, do_print=True)
```

```
previous estimate: 160.00, prediction: 161.00, estimate 159.80
previous estimate: 159.80, prediction: 160.80, estimate 162.16
previous estimate: 162.16, prediction: 163.16, estimate 162.02
previous estimate: 162.02, prediction: 163.02, estimate 161.77
previous estimate: 161.77, prediction: 162.77, estimate 162.50
previous estimate: 162.50, prediction: 163.50, estimate 163.94
previous estimate: 163.94, prediction: 164.94, estimate 166.80
previous estimate: 166.80, prediction: 167.80, estimate 167.64
previous estimate: 167.64, prediction: 168.64, estimate 167.75
previous estimate: 167.75, prediction: 168.75, estimate 169.65
previous estimate: 169.65, prediction: 170.65, estimate 170.87
previous estimate: 170.87, prediction: 171.87, estimate 172.16
```

```
# plot results
book_plots.set_figsize(10)
gh.plot_gh_results(weights, estimates, predictions, [160, 172])
weights
```

```
[158.0,
 164.2,
 160.3,
 159.9,
 162.1,
 164.6,
 169.6,
 167.4,
 166.4,
 171.0,
 171.2,
 172.6]
```



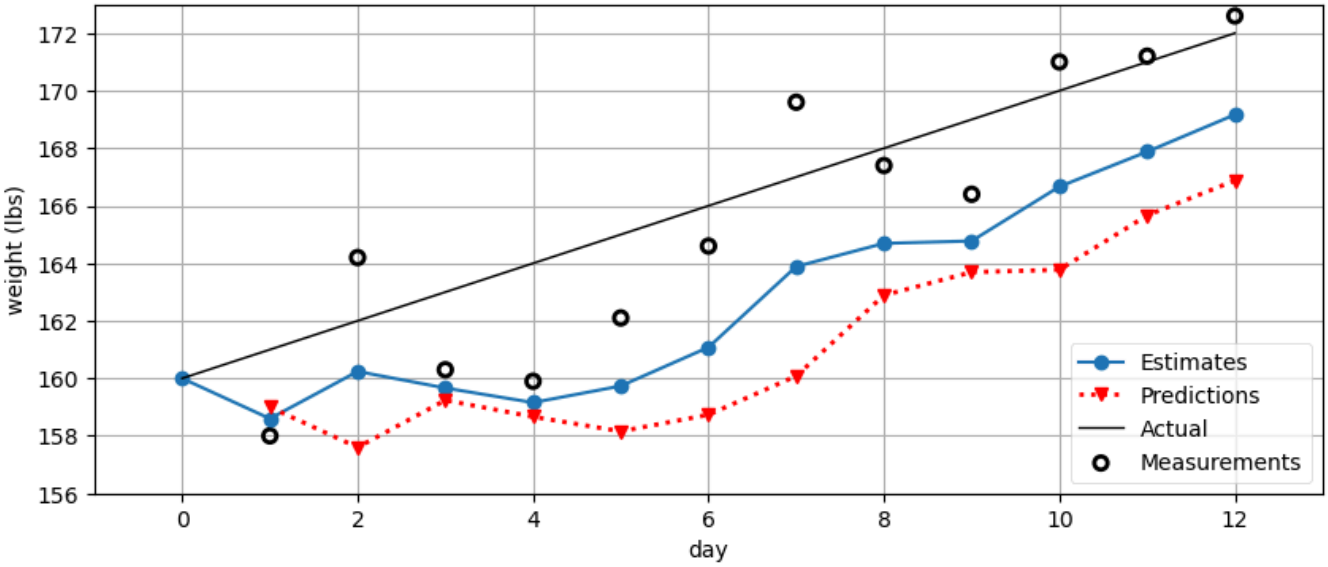
结果不错。粗蓝线是滤波器估计，从第 0 天的初值 160 磅开始。红线是根据前一天估计作出的预测：第 1 天，上一估计为 160 磅，每天增重 1 磅，因此预测为 161 磅。再与测量融合，第 1 天的估计为 159.8 磅，位于两者之间。图下表格列出每一天的上一估计、预测值与新估计。细黑线表示被称重者的真实体重变化。

请逐天仔细查看这些数据，确保自己明白在每个步骤中，预测值和估算值是如何得出的。请注意，估算值始终位于测量值和预测值之间。

这些估算值并非呈直线分布，但相比实际测量数据来说，其趋势更为平缓，且与我们所绘制的趋势线较为接近。此外，随着时间的推移，这些估算值似乎还有所改善。

你可能觉得这很可笑：如果预先假定每天增重约 1 磅，当然能得到好看的结果！如果最初对变化率的猜测错了呢？下面让滤波器错误地预测每天减重 1 磅，观察结果。注意，这只是错误的模型假设，并没有改变仿真中的真实增重。

```
e, p = predict_using_gain_guess(initial_estimate, -1.)
gh.plot_gh_results(weights, e, p, [160, 172])
```



那并没有什么了不起的。各种估算结果与实际测量值相差甚远。显然，那种需要我们准确预测变化率的滤波器没什么用处。即便我们的初始猜测是正确的，但一旦变化率发生改变，该滤波器就会失效。如果我不再暴饮暴食，该滤波器也很难适应这种变化。请注意，它其实是在尝试调整的！尽管我们告诉它我们每天都在减重 1 磅，但估算结果却仍在上升。只是它的调整速度不够快而已。

但是，“要是那样会怎样呢？”如果我们不是简单地把体重增加量定为最初的 1 磅（或其它数值），而是根据现有的测量数据和估算结果来计算呢？第一天时，我们对体重的估算结果是：

$$(160 + 1) + \frac{4}{10}(158 - 161) = 159.8$$

第二天，我们测得的体重为 164.2 磅。这意味着体重增加了 4.4 磅（因为 164.2 - 159.8 = 4.4），而不是 1 磅。我们能利用这些数据吗？看起来是可行的。毕竟，体重测量结果本身就是基于现实中的体重数值，因此具有实际意义。虽然我们对体重增加量的估算可能并不完全准确，但总比随便猜测 1 磅要好。数据总比猜测更可靠，哪怕这些数据存在一定的误差。

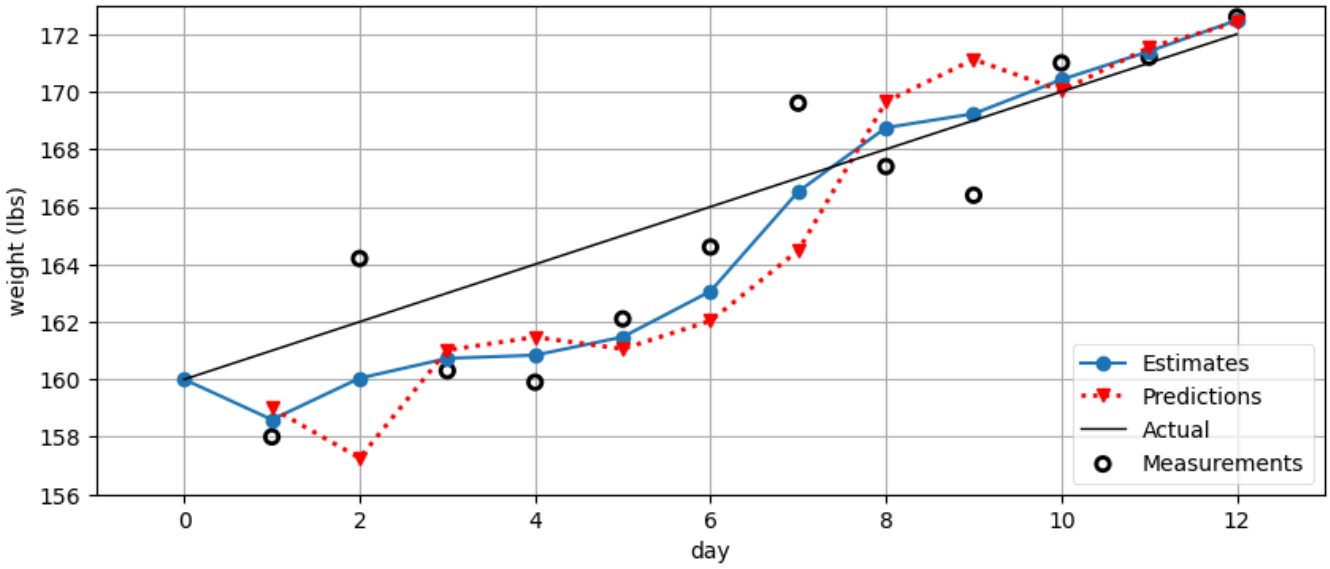
在这一点上，人们往往会犹豫不决，所以一定要确保自己同意这个做法。通过两次称重得到的数值，我们可以估算出体重的增减情况。如果称重结果不准确，那么估算出的数值也会不准确。不过，这种计算方式仍然能提供一些有用的信息。比如，如果我们用精度为 1 磅的秤来称一头牛的体重，结果显示牛的体重增加了 10 磅。实际上，由于测量误差，牛的增重可能在 8 至 12 磅之间。不过，我们至少可以知道牛的体重确实增加了，而且大致增加了多少。这就是有用的信息。那我们该如何利用这些信息呢？千万不要把它们浪费掉！

回到我的饮食计划上。我们应该把每天的增重估计设为 4.4 磅吗？昨天我们认为每天增重 1 磅，今天则根据测量估计为 4.4 磅。我们有两个不同的数值，需要把它们结合起来。这又是同一个问题，所以仍使用之前的方法：在两者之间选择一个估计值。这次另取一个任意选定的缩放系数 $\frac{1}{3}$，用它来加权融合两个增重率。计算公式与估计体重时相同，只是还需要计入时间，因为这里估计的是变化率，即每天增加的体重。

$$\textbf{new gain} = \textbf{old gain} + \frac{1}{3}\frac{\textbf{measurement - predicted weight}}{\textbf{1 day}}$$

```
weight = 160.  # initial guess
gain_rate = -1.0  # initial guess
```

```
time_step = 1.
weight_scale = 4./10
gain_scale = 1./3
estimates = [weight]
predictions = []

for z in weights:
    # prediction step
    weight = weight + gain_rate*time_step
    gain_rate = gain_rate
    predictions.append(weight)

    # update step
    residual = z - weight

    gain_rate = gain_rate + gain_scale   * (residual/time_step)
    weight    = weight    + weight_scale * residual

    estimates.append(weight)

gh.plot_gh_results(weights, estimates, predictions, [160, 172])
```



我觉得现在的结果相当不错。由于最初对体重变化的估计值为-1，导致滤波器需要几天时间才能准确预测体重。但一旦能够准确预测后，它就能持续准确地记录体重变化了。我们在确定参数 $\frac{4}{10}$ 和 $\frac{1}{3}$ 的值时并没有采用任何特定的方法（实际上，这些数值并不适合用来解决这个问题）。不过，其余的数学计算都是基于合理的假设得出的。如果你想查看数值变化的详细过程，可以将参数 `time_step` 的值调大，然后重新运行该程序。

在继续之前，还有最后一点需要说明。在预测步骤中，我写了这样一行代码。

```
gain_rate = gain_rate
```

这一赋值显然没有改变数值，可以删除。保留它是为了强调：预测步骤必须预测全部变量，包括 weight 和 gain_rate。这里先假设每日增重率不变，推广算法时就会放宽这一假设。

G-H 滤波器

这个算法称为 g-h 滤波器，也称 α-β 滤波器。g 和 h 是前面使用的两个缩放因子：g 用于测量量，本例中是体重；h 用于测量量随时间的变化率，本例中是每天增加的磅数。α 和 β 只是这两个因子的另一套名称。

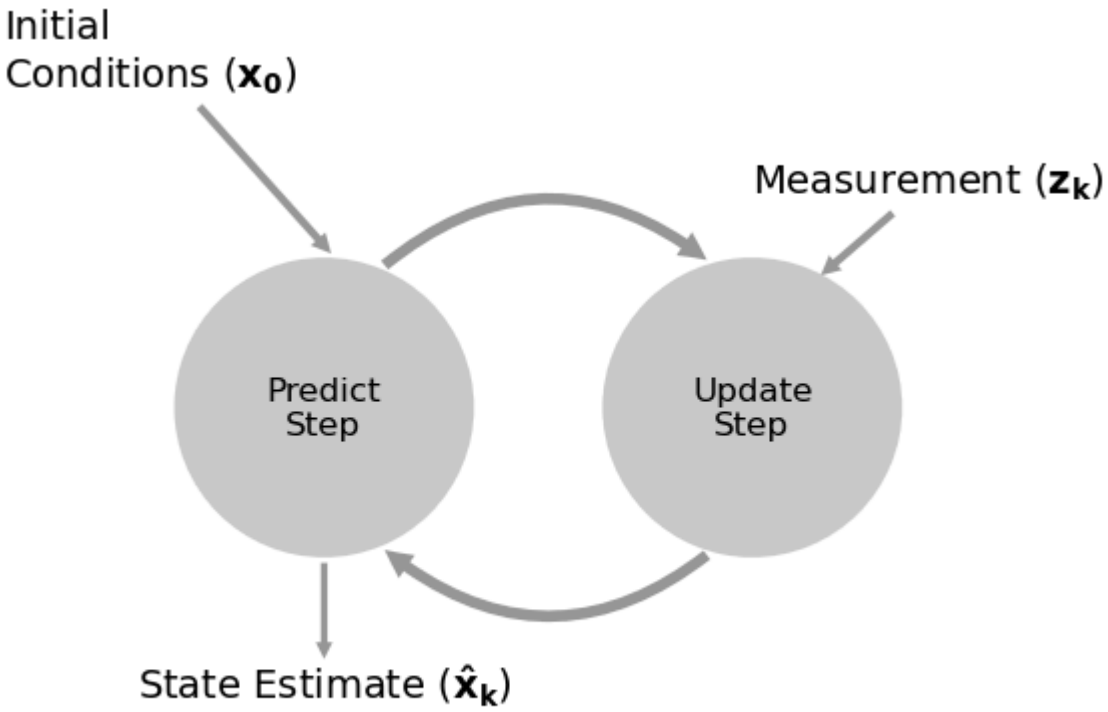
这种滤波器是众多滤波器的基础，包括卡尔曼滤波器。换句话说，卡尔曼滤波器其实是一种特殊的 g-h 滤波器形式，我将在本书后面证明这一点。最小二乘滤波器也是如此，你可能听说过它；而本尼迪克特-博德纳滤波器则可能不太为人所知。每种滤波器在为 g 和 h 赋值的方式上有所不同，但算法本身是相同的。例如，本尼迪克特-博德纳滤波器会为 g 和 h 分配固定值，但这些值必须符合一定的范围限制。而像卡尔曼滤波器这样的滤波器，则会在每个时间步动态地调整 g 和 h 的值。

让我把那些要点再重复一遍，因为它们非常重要。如果你不理解这些要点，那么你就无法理解这本书的其余内容。而如果你理解了这些要点，那么这本书的其余部分就会变得容易理解了——因为那些内容其实都是针对我们关于 g 和 h 的各种“假设”而提出的数学推导而已。虽然数学表达方式可能有所不同，但算法本身是完全相同的。

- 多个数据点总比一个数据点更准确。因此，无论某个数据点的准确性如何，都不要放弃它。
- 为了得到更准确的估算结果，应始终选择位于两个数据点之间的数值作为计算依据。
- 根据当前的估算值，以及我们认为该数值还会发生多大的变化，来预测接下来的测量结果和变化率。
- 新的估算值则是根据预测值和下一次测量结果的准确性来确定的，具体来说，就是将预测值与下一次测量结果按各自的准确性进行加权平均后得到的数值。

让我们来看看该算法的可视化表现形式吧。

```
book_plots.predict_update_chart()
```



让我来介绍一些更正式的术语。所谓“系统”，指的就是我们想要进行评估的对象。在本章中，所谓“系统”指的就是我们试图对其进行衡量的任何事物。有些文献将之称为“被控对象”。这一术语源自控制系统理论。https://en.wikipedia.org/wiki/Plant_(control_theory)

系统的“状态”指的是该系统当前所具有的配置或数值。我们只关心与重量相关的数值。如果我在秤上放置 100 公斤的重物，那么该系统的状态就是 100 公斤。我们根据对自己来说重要的信息来定义系统的状态。秤的颜色对我们来说并不重要，因此不会将其纳入系统的状态中。不过，作为制造商的质量控制工程师，可能会将秤的颜色也纳入系统的状态中，以便于对生产过程进行监控和控制。

该测量值实际上是系统被测得的数值。由于测量可能存在误差，因此该数值可能与系统的真实状态并不完全一致。

状态估计指的是滤波器对系统状态的估算值。例如，对于 100 公斤的重量，由于传感器误差，我们的估算值可能会是 99.327 公斤。这一数值通常简称为“估计值”，我在本章中也是这么处理的。

换句话说，状态应理解为系统的真实数值，它通常不能被直接获知。如果我站上秤，你就得到一次可直接读取的测量。真实体重与秤的读数不同：读数含有误差，我们只能借助测量推断真实体重。

这种“隐藏状态”与“可观测信息”的概念非常重要。在所有的估算问题中，都需要通过可观测的数值来推断出隐藏状态。如果你阅读相关文献，就会发现这些术语在描述各种问题时都被经常使用。因此，你必须熟练掌握这些概念。

我们使用过程模型来对系统进行数学描述。在本章中，我们所采用的过程模型是这样的：今天的体重等于昨天的体重加上昨天体重增加的数值。该过程模型并未考虑传感器的因素。另一个例子是汽车的过程模型：距离等于速度乘以时间。不过，这种模型并不完美，因为汽车的速度会随时间而变化，轮胎也可能会在路面上打

滑等等。系统误差或过程误差指的就是这种模型中的偏差。我们永远无法准确知道这个数值；如果能够知道的话，我们就可以改进模型，使其误差为零。有些文献会使用“系统模型”或“系统误差”这样的术语，其实它们都指的是同一种概念。

预测步骤被称为“系统传播”。该过程利用过程模型来生成新的状态估计值。由于存在过程误差，这种估计值并不完美。如果我们是在对随时间变化的数据进行跟踪，那么就可以说我们把系统的状态“传播”到了未来。有些文献将这一过程称为“系统演化”。

更新步骤称为测量更新。一次系统传播加一次测量更新，构成一次滤波迭代，也称一个时间步（epoch）。

现在换几个应用领域，进一步理解这个算法。考虑跟踪轨道上的火车：轨道把位置约束在很明确的区域内；火车体积大、运动变化慢，显著加速或减速需要好几分钟。因此，如果知道火车在时刻 t 位于 23 千米标记处，以 18 千米/小时行驶，就能很有把握地预测 t+1 秒时的位置。这为什么重要？假设位置测量精度只有 ±250 米。18 千米/小时等于每秒 5 米，所以 t+1 秒时火车应位于 23.005 千米处，而读数可能落在 22.755—23.255 千米之间。若下一次读数是 23.4 千米，就知道它不准确。即使司机在时刻 t 猛踩刹车，火车仍会非常接近 23.005 千米处，因为它不可能在一秒内大幅减速。如果要为这个问题设计滤波器——本章稍后就会这样做——我们应赋予预测值远高于测量值的权重。

现在来考虑追踪被抛出的球的问题。我们知道，当物体处于重力场中时，其在真空中的运动轨迹是抛物线形。但在地球上，由于空气阻力的影响，球的实际运动轨迹并非完美的抛物线。棒球投手正是利用了这一原理来投出曲线球。假设我们使用计算机视觉技术在体育场内追踪球的运动轨迹——这也是我工作中经常做的事情。虽然计算机视觉技术的精确度可能不高，但假设球沿抛物线轨迹运动来预测其未来位置的方法，其精确度也并不高。在这种情况下，我们或许应该设计一种算法，让测量结果和预测结果具有大致相同的权重。

现在，试想一下在飓风中追踪一个氦气气球的情况吧。我们没有任何可靠的模型能够用来预测气球的运动轨迹，除非是在极短的时间范围内（比如，我们知道气球不可能在一秒钟内移动 10 英里）。在这种情况下，我们可以设计一种滤波器，让那些基于实际观测得到的数据比预测结果更受重视。

本书的大部分内容都致力于用数学方式来表达最后三段文字中所表达的观念。这样一来，我们就能找到一个“最优解”（从数学意义上来说）。在本章中，我们将以一种更直观的方式给 $\boldsymbol{g}$ 和 $\boldsymbol{h}$ 赋予不同的数值，这种方式虽然不是最优的，但基本思路是：将那些不太准确的测量结果与那些对系统行为描述不太准确的模型结合起来，从而得到一个比单独使用其中任何一种信息来源都要好的估算结果。

我们可以将其表述为一个算法：

初始化

1. 初始化滤波器的状态。
2. 初始化对该状态的信念（不确定性）。

预测

1. 根据系统行为预测下一时间步的状态。
2. 调整状态信念，计入预测的不确定性。

更新

1. 获取测量值及其精度信息。
2. 计算测量与预测状态之间的残差。
3. 沿残差方向，在预测与测量之间形成新估计。

在整本书中，我们都会使用同样的算法，不过会做一些必要的修改。

符号约定

下面开始介绍文献中常用的记号与变量名，其中有些已经出现在前面的图中。测量通常记为 $\boldsymbol{z}$，本书也采用这一约定；另一些文献记为 $\boldsymbol{y}$。下标 $\boldsymbol{k}$ 表示时间步的编号，因此 $\boldsymbol{z_k}$ 是该步的测量数据。目前只考虑一个传感器和一个测量量；一般情况下，也可能有 n 个传感器和 n 个测量分量。$\mathbf{x}$ 表示状态向量，在称重例子中同时包含初始体重和初始增重率，如下所示。向量、矩阵与标量的区别可根据它们的定义和维数确定。

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

这里采用牛顿点记法，在 x 上加点表示它对时间的一阶导数。在位置跟踪问题中，这个导数就是速度；在体重问题中，它表示体重变化率。例如，体重为 62 kg、每天增加 0.3 kg 时，有：

$$\mathbf{x} = \begin{bmatrix} 62 \\ 0.3 \end{bmatrix}$$

该算法其实很简单。首先，将状态初始化为初始值 $\mathbf{x_0}$。接着，进入循环：根据时间点 $\boldsymbol{k-1}$ 的数值来预测时间点 $\boldsymbol{k}$ 的状态。之后，获取实际测量值 $\boldsymbol{z_k}$，并在该测量值与预测值之间选择一个中间值，从而得到新的估计值 $\mathbf{x_k}$。

练习：编写通用算法

在上面的例子中，我特意这样编写代码，为的就是解决我们在本章中一直在讨论的称重问题。例如，各个变量的名称被分别设为“weight_scale”、“gain”等等。我这么做是为了让算法更易于理解——这样就能清楚地看出，我们确实正确地实现了算法的每一步。不过，这段代码只是为了解决某个特定问题而编写的。而实际上，对于任何问题来说，算法都是相同的。因此，让我们重新编写代码，使其具有通用性，从而能够适用于各种问题。请使用如下函数格式来编写代码：

```
def g_h_filter(data, x0, dx, g, h, dt):
 """
    Performs g-h filter on 1 state variable with a fixed g and h.
    'data' contains the data to be filtered.
    'x0' is the initial value for our state variable
    'dx' is the initial change rate for our state variable
    'g' is the g-h's g scale factor
    'h' is the g-h's h scale factor
    'dt' is the length of the time step
    """
```

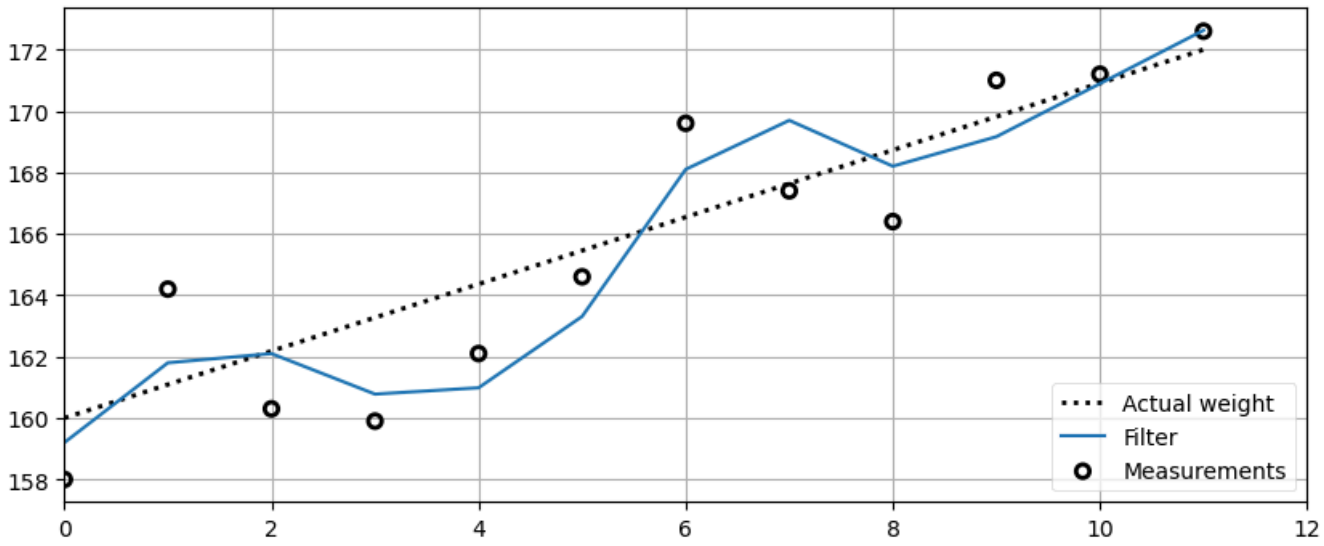
返回 NumPy 数组而不是列表。把前面同样的体重测量数据传入，画出结果，并目测确认实现正常。

```
from kf_book.gh_internal import plot_g_h_results
def g_h_filter(data, x0, dx, g, h, dt):
    pass #  your solution here

# uncomment to run the filter and plot the results
#book_plots.plot_track([0, 11], [160, 172], label='Actual weight')
#data = g_h_filter(data=weights, x0=160., dx=1., g=6./10, h=2./3, dt=1.)
#plot_g_h_results(weights, data)
```

解答与讨论

```
import matplotlib.pylab as pylab

def g_h_filter(data, x0, dx, g, h, dt=1.):
    x_est = x0
    results = []
    for z in data:
        # prediction step
        x_pred = x_est + (dx*dt)
        dx = dx

        # update step
        residual = z - x_pred
        dx = dx + h * (residual) / dt
        x_est = x_pred + g * residual
        results.append(x_est)
    return np.array(results)

book_plots.plot_track([0, 11], [160, 172], label='Actual weight')
data = g_h_filter(data=weights, x0=160., dx=1., g=6./10, h=2./3, dt=1.)
plot_g_h_results(weights, data)
print(weights)
print(data)
```

```
[158.0, 164.2, 160.3, 159.9, 162.1, 164.6, 169.6, 167.4, 166.4, 171.0, 171.2, 172.6]
[159.2   161.8   162.1   160.78  160.985 163.311 168.1   169.696
 168.204 169.164 170.892 172.629]
```



这应该很直接：用通用名称 x0、dx 等替换体重变化示例中的专用变量名，其他部分不需要改变。

g 和 h 的选择

G-H 滤波器并非单一的滤波器类型，而是一类滤波器的总称。Eli Brookner 在《Tracking and Kalman Filtering Made Easy》一书中列出了 11 种此类滤波器，实际上肯定还有更多。此外，每种滤波器又包含多种子类型。各种滤波器的区别在于如何确定 g 和 h 的值。因此，这里无法给出适用于所有情况的通用建议。有些滤波器将 g 和 h 设置为固定值，而另一些则让这些值动态变化。卡尔曼滤波器就是在每一步都动态调整这些值。有些滤波器允许 g 和 h 在某个范围内取任意值，而另一些则要求这两个值通过某种函数 $f(), \text{where } g = f(h)$ 相互关联。

本书的主题并非 G-H 滤波器的全部内容；更重要的是，我们感兴趣的是这些滤波器的贝叶斯方面，而这一点我尚未详细阐述。因此，我不会深入探讨 g 和 h 的选择问题。关于这一主题，《Tracking and Kalman Filtering Made Easy》是一本非常好的参考书。如果你觉得我的这种观点有些奇怪，请记住：卡尔曼滤波器的典型实现方式其实根本不涉及 g 和 h。卡尔曼滤波器可以被看作是一种 G-H 滤波器，因为它在数学上可以转化为这种算法。在设计卡尔曼滤波器时，我们会使用一些设计准则，这些准则在数学上可以转化为 g 和 h，不过，使用卡尔曼滤波器的形式来思考问题通常更为有效。如果现在还不太明白的话，不必担心，等我们进一步了解卡尔曼滤波器的理论后，就会明白的。

值得研究的是， g 和 h 的不同取值会对结果产生怎样的影响。因此，我们将通过几个例子来进行分析。这将有助于我们深入了解这类滤波器的基本优点和局限性，同时也有助于我们理解更为复杂的卡尔曼滤波器的工作原理。

练习：创建测量函数

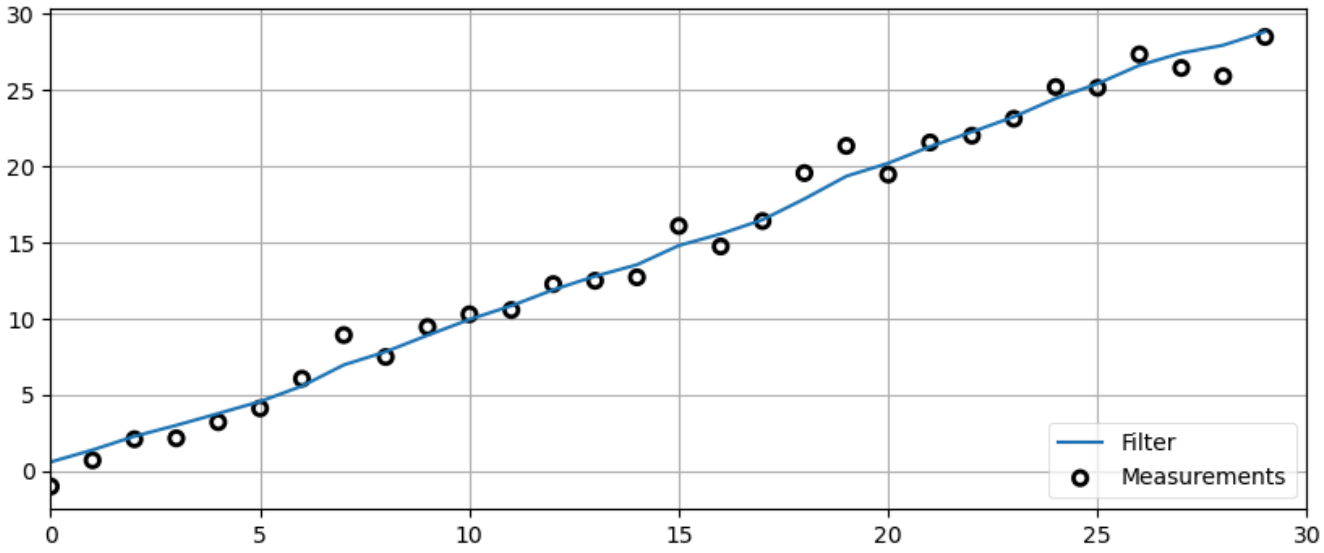
现在编写带噪数据生成函数。本书将带噪信号建模为真实信号加上白噪声。有关统计学的完整定义后面再讲；直觉上，可以把它看作围绕信号随机上下变化、没有时间相关模式的扰动。这里假设它是时间上互不相关、均值为零、方差有限的随机序列。高斯分布一章之后就会更容易理解。若没有统计学基础，暂时做不出练习也没关系，可以先阅读解答和讨论。

白噪声可以由 `numpy.random.randn()` 生成。我们需要一个函数，该函数接受起始值、每步的变动幅度、步数以及所需的噪声量作为输入参数。该函数应返回一个包含所有计算结果的列表。可以通过生成 30 个数据点，再利用 `g_h_filter()` 对数据进行滤波，最后用 `plot_g_h_results()` 将结果绘制成图表来测试该函数。

```
# your code here
```

解答

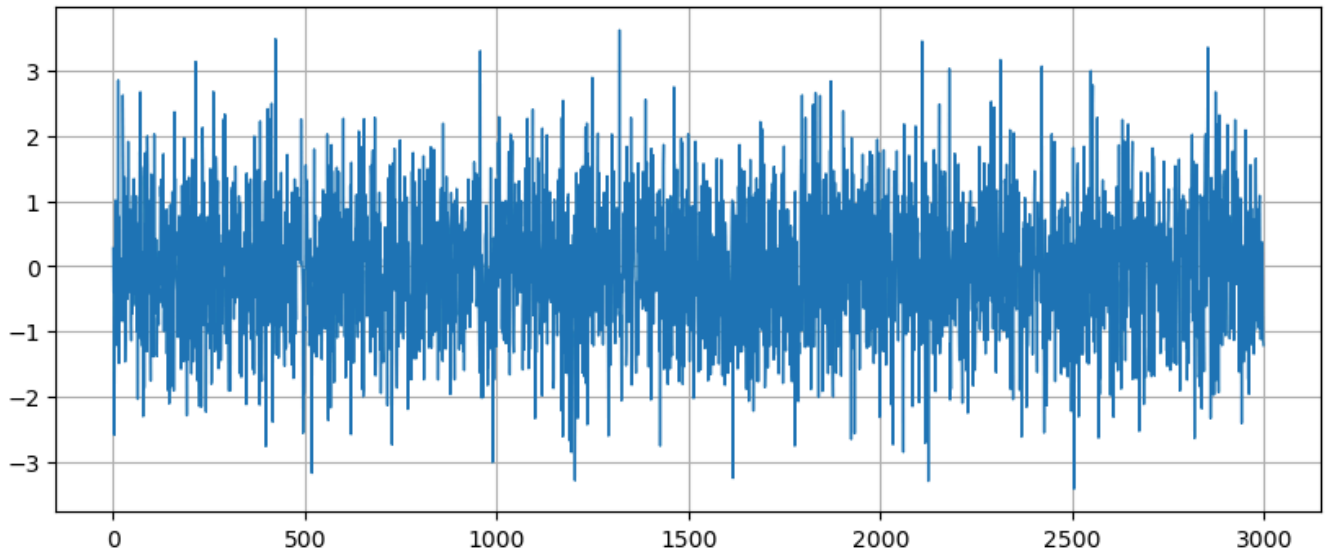
```
from numpy.random import randn
def gen_data(x0, dx, count, noise_factor):
    return [x0 + dx*i + randn()*noise_factor for i in range(count)]

measurements = gen_data(0, 1, 30, 1)
data = g_h_filter(data=measurements, x0=0., dx=1., dt=1., g=.2, h=0.02)
plot_g_h_results(measurements, data)
```



讨论

randn() 返回均值为 0、标准差为 1 的正态随机数，大于零与小于零的概率相同。这里的“标准差为 1”不表示数值只能在 -1 到 1 之间。下面绘出 3000 次调用的结果：数据以零为中心，大部分在中心附近，偶尔也会明显偏离。暂时不理解标准差也不要紧，后面会解释。

```
plt.plot([randn() for _ in range(3000)], lw=1);
```



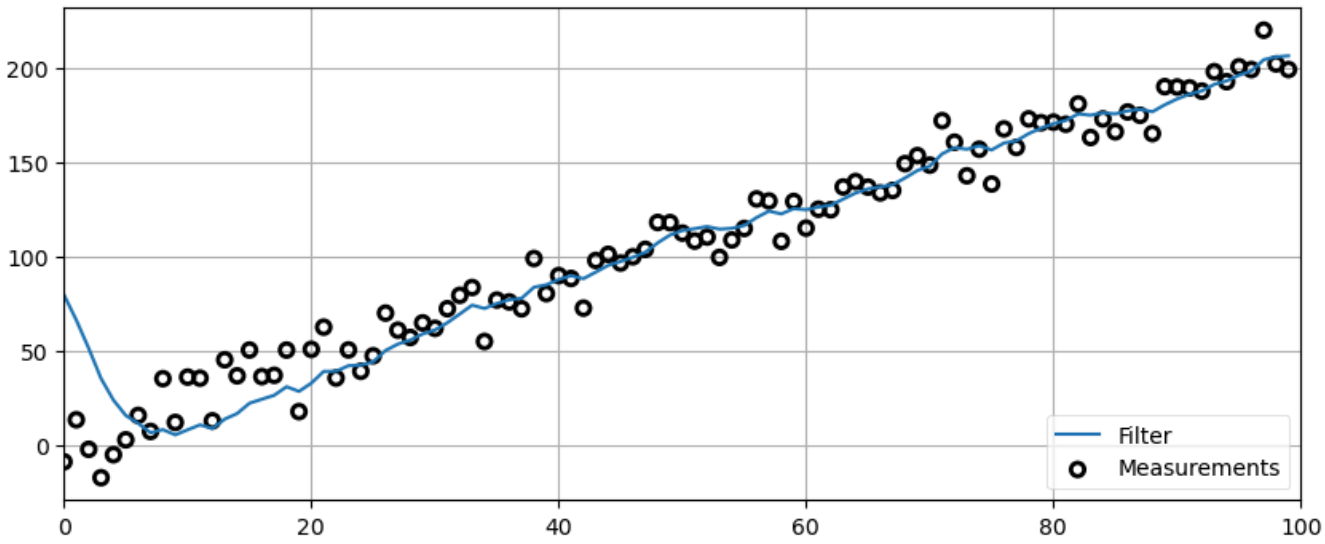
练习：不良的初始条件

编写代码，使用 gen_data 与 g_h_filter，对 100 个数据点滤波。数据起始值为 5，导数为 2，噪声缩放系数为 10，滤波参数取 g=0.2、h=0.02；滤波器对 x 的初始猜测设为 100。

```
# your code here
```

解答与讨论

```
zs = gen_data(x0=5., dx=2., count=100, noise_factor=10)
data = g_h_filter(data=zs, x0=100., dx=2., dt=1., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



由于初始值被设定为 100 这个不准确的数值，滤波器在开始时的预测结果与实际测量数据相差甚远。可以看到，滤波器在调整到与实际数据相符的状态之前，会呈现出一种“振荡”现象。所谓“振荡”，指的是信号以正弦波的形式超出或低于实际数据。这是滤波器中非常常见的现象。在滤波器设计中，人们通常会努力减少这种振荡现象。不过，这个话题我们暂时还不打算深入探讨，我只是想让大家了解这种现象而已。

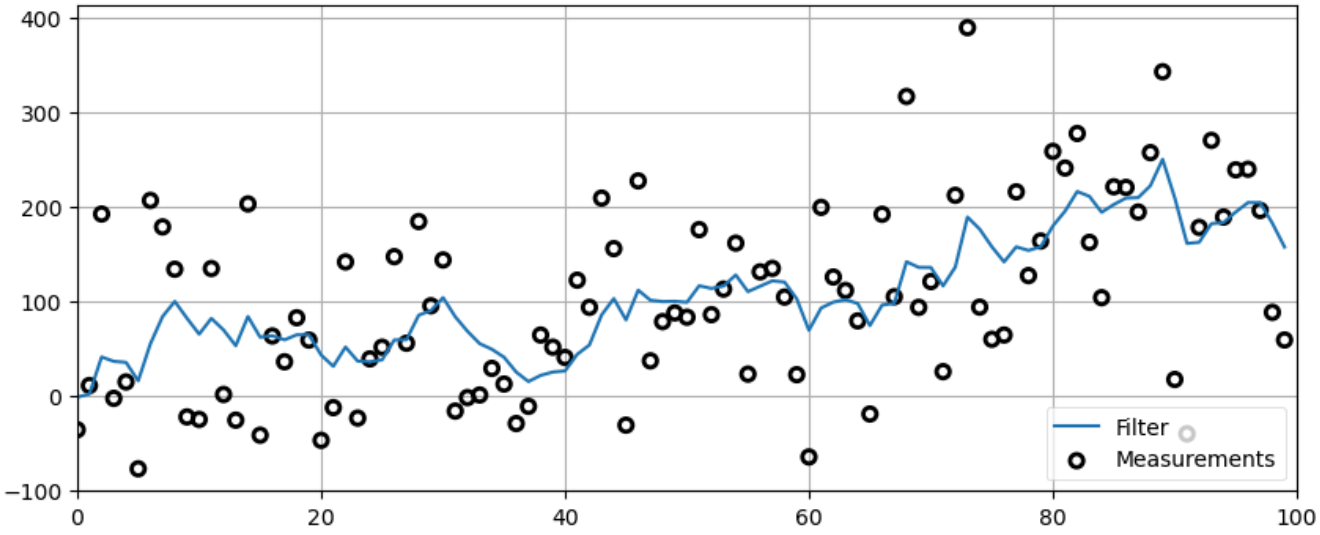
练习：极端噪声

重新进行同样的测试，但这次将噪声系数设置为 100。为了消除初始条件带来的干扰，将初始值从 100 改为 5。

```
# your code here
```

解答与讨论

```
zs = gen_data(x0=5., dx=2., count=100, noise_factor=100)
data = g_h_filter(data=zs, x0=5., dx=2., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



在我看来，这并不怎么样。可以看出，经过滤波后的信号变化幅度可能小于有噪声的信号，但它远远不是一条直线。如果只展示滤波后的结果，没人能猜出该信号的起始值为 5，并且每个时间点上数值都会增加 2。虽然在该滤波器起作用的情况下，噪声确实有所减少，但在其他地方，信号的数值却出现了超调或欠调的情况。

目前，我们还不足以对这一点做出准确的判断。我们加入了太多不必要的“噪声”；也许，这已经算是滤波效果最好的结果了。不过，既然还有许多章节存在，那就说明我们完全有可能做得更好。

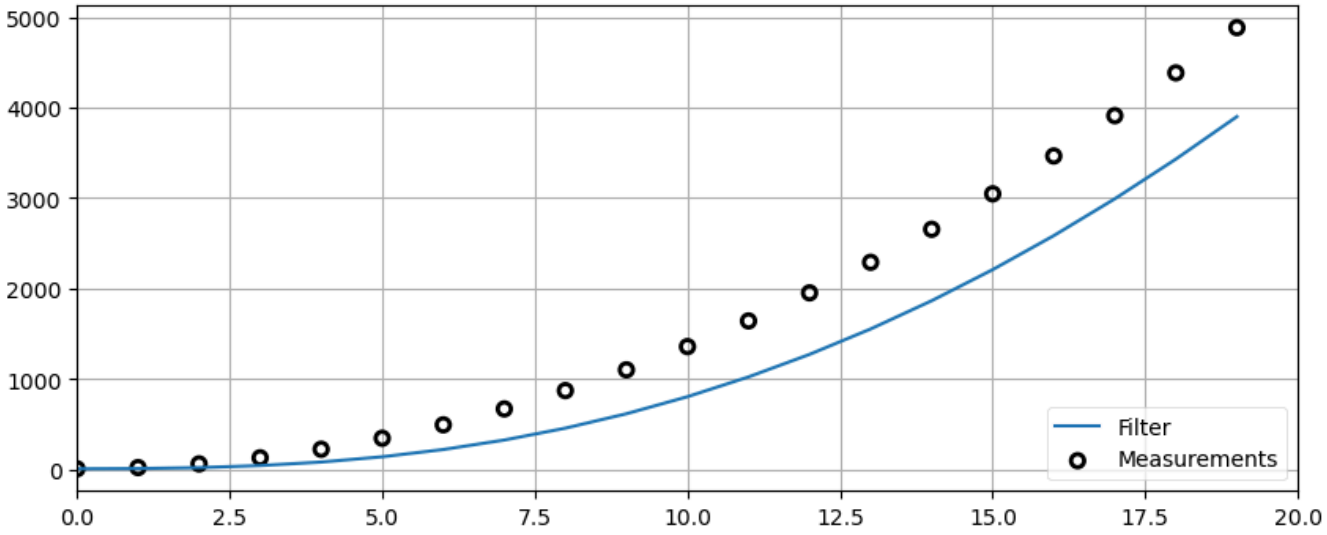
练习：加速度的影响

编写新的数据生成函数，给每个数据点加入恒定加速度。也就是说，每次计算数据点时递增 dx，使速度 dx 不断增加。将噪声设为 0，取 $g = 0.2$、$h = 0.02$，用 plot_g_h_results 或自己的绘图函数画出结果。试验不同的加速度和时间步长，并解释观察到的现象。

```
# your code here
```

解答与讨论

```
def gen_data(x0, dx, count, noise_factor, accel=0.):
    zs = []
    for i in range(count):
        zs.append(x0 + accel * (i**2) / 2 + dx*i + randn()*noise_factor)
        dx += accel
    return zs

predictions = []
zs = gen_data(x0=10., dx=0., count=20, noise_factor=0, accel=9.)
data = g_h_filter(data=zs, x0=10., dx=0., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



每一次预测值都会落后于实际数值。仔细想想，这是合乎逻辑的。我们的模型假设速度是恒定的。g-h 滤波器会计算 $\boldsymbol{x}$ 的一阶导数（我们用 $\dot{\boldsymbol{x}}$ 来表示导数），而不计算二阶导数 $\ddot{\boldsymbol{x}}$。因此，我们实际上假设 $\ddot{\boldsymbol{x}} = \mathbf{0}$ 为恒定值。在每次预测时，我们将 x 的新值预测为 $\boldsymbol{x} + \dot{\boldsymbol{x}} * \boldsymbol{t}$。但由于加速度的存在，预测值必然会落后于实际数值。接着，我们试图重新计算 $\dot{\boldsymbol{x}}$ 的值，但由于 $\boldsymbol{h}$ 的影响，我们只能部分地调整 $\dot{\boldsymbol{x}}$ 以适应新的速度。在下一次迭代中，预测值仍会再次落后于实际数值。

请注意，我们无法对 $\boldsymbol{g}$ 或 $\boldsymbol{h}$ 进行任何调整来纠正这个问题。这被称为系统的滞后误差或系统误差。这是 g-h 滤波器所固有的特性。也许你已经想到了解决这个问题的方法或变通方案。正如你所预期的，许多研究都致力于解决这个问题，我们在本书中将会介绍各种解决方案。

> 归根结底，滤波器的性能好坏，完全取决于用来描述该系统的数学模型的质量如何。

练习：改变 g 的值

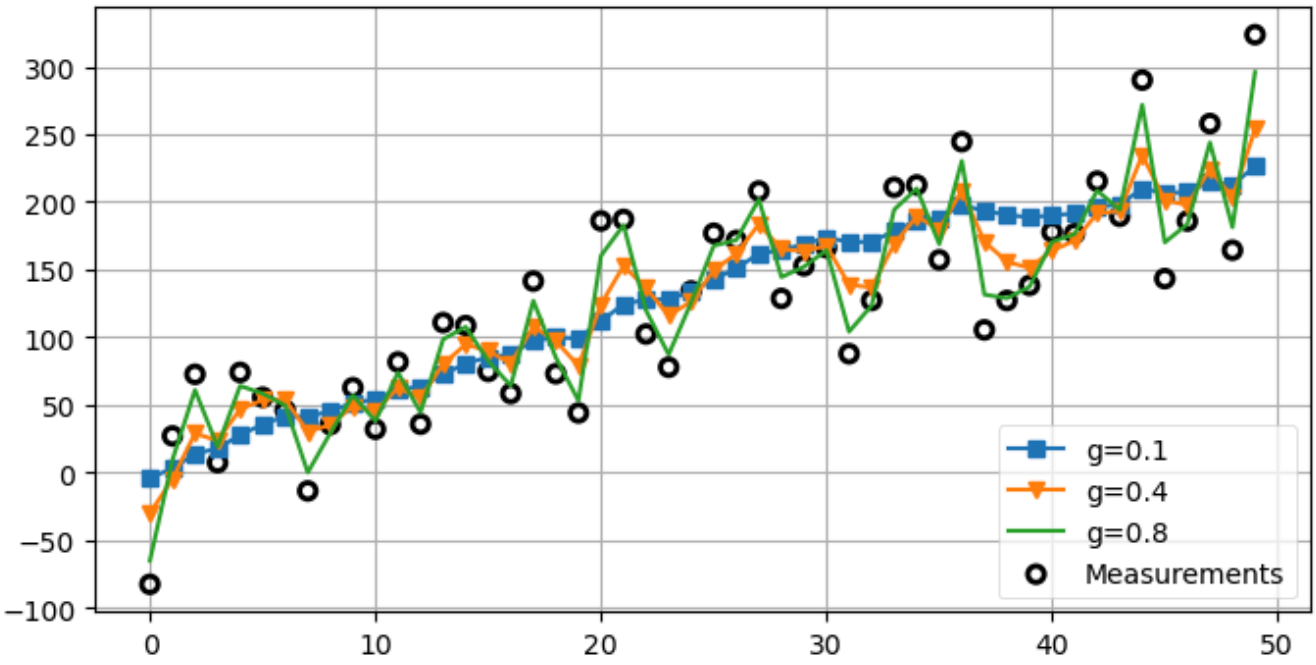
下面考察 $\boldsymbol{g}$ 的取值对结果的影响。开始之前，先回忆：$\boldsymbol{g}$ 是控制测量与预测相对权重的缩放系数。你认为较大的 $\boldsymbol{g}$ 会产生什么影响？较小的取值又会怎样？

现在设置 noise_factor=50、dx=5，分别绘制 $\boldsymbol{g} = \mathbf{0.1, 0.4, \text{ and } 0.8}$ 时的滤波结果。

```
# your code here
```

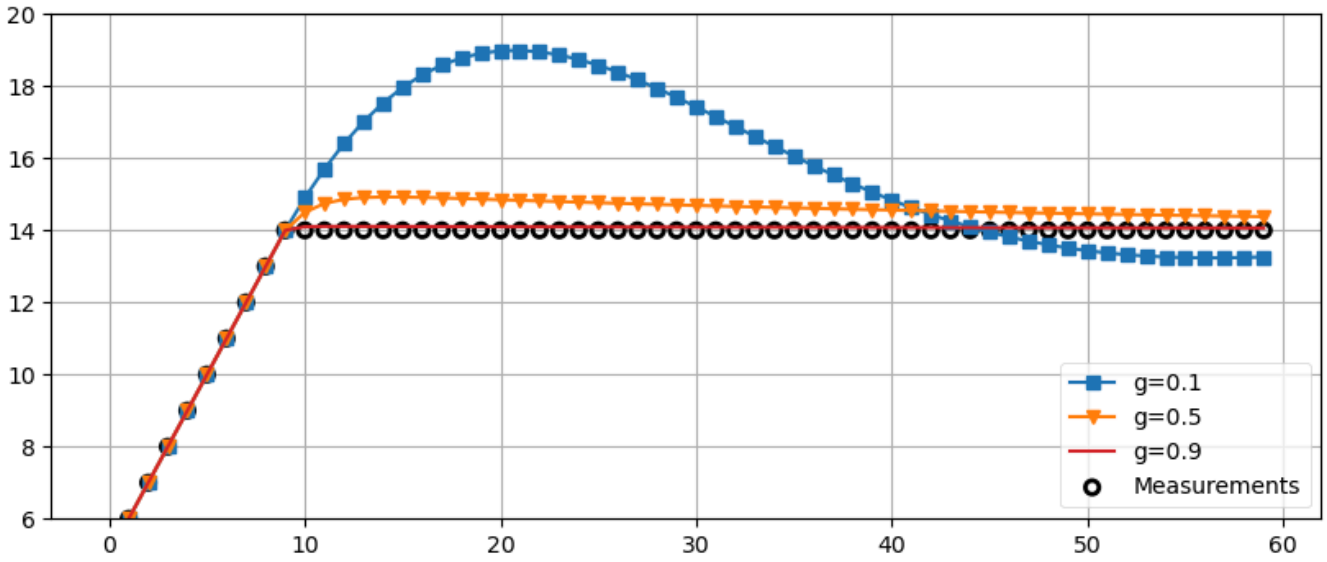
解答与讨论

```
np.random.seed(100)
zs = gen_data(x0=5., dx=5., count=50, noise_factor=50)
data1 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.1, h=0.01)
data2 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.4, h=0.01)
data3 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.8, h=0.01)

with book_plots.figsize(y=4):
    book_plots.plot_measurements(zs, color='k')
    book_plots.plot_filter(data1, label='g=0.1', marker='s', c='C0')
    book_plots.plot_filter(data2, label='g=0.4', marker='v', c='C1')
    book_plots.plot_filter(data3, label='g=0.8', c='C2')
    plt.legend(loc=4)
```



显然，当 g 的数值越大时，我们就会更倾向于参考实际测量结果，而非预测值。当 $g = 0.8$ 时，我们几乎完全按照信号来行事，几乎不会忽略任何噪声信号。人们可能会天真地认为，为了最大限度地减少噪声干扰，g 的值应该始终很小。但实际上，这样做的后果就是我们几乎完全忽略了实际测量结果，而只依赖预测值。那么，当信号的变化并非由噪声引起，而是由实际状态的变化所导致时，又会怎样呢？让我们来仔细看看。我会创建这样一些数据：在 9 个步骤内，$\dot{x} = 1$ 的数值保持不变，之后才变为 $\dot{x} = 0$。

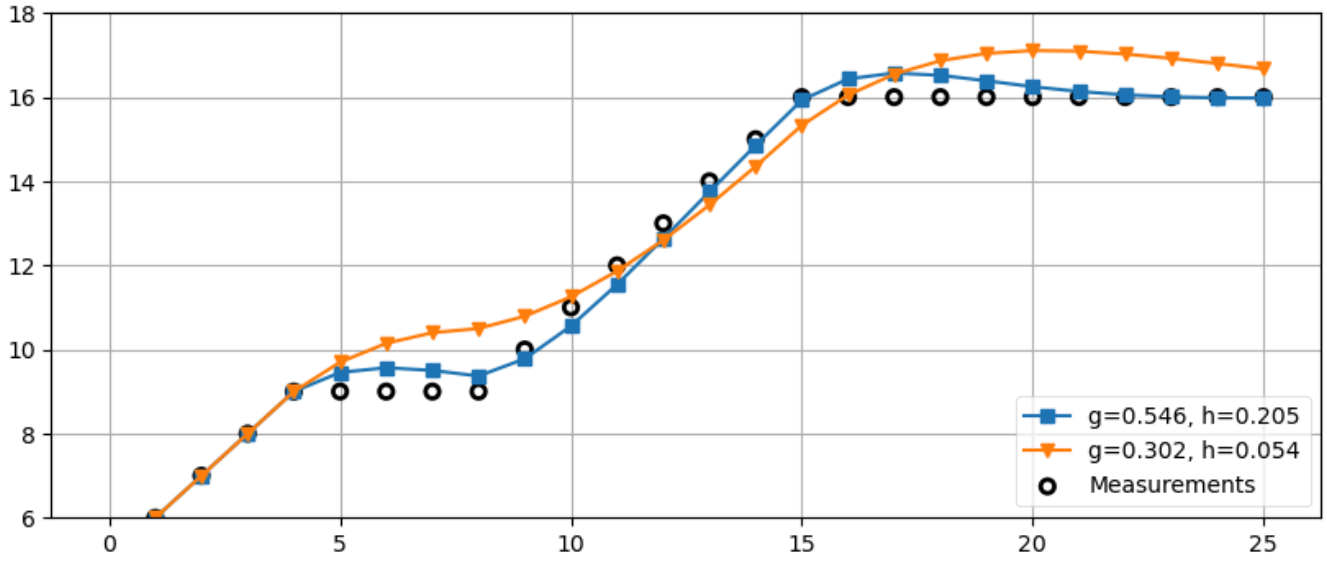
```
zs = [5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
for i in range(50):
    zs.append(14)

data1 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.1, h=0.01)
data2 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.5, h=0.01)
data3 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.9, h=0.01)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data1, label='g=0.1', marker='s', c='C0')
book_plots.plot_filter(data2, label='g=0.5', marker='v', c='C1')
book_plots.plot_filter(data3, label='g=0.9', c='C3')
plt.legend(loc=4)
plt.ylim([6, 20]);
```



在这里，我们可以看到忽略该信号所带来的后果。我们不仅滤除了噪声，同时也把信号中那些正常的、有意义的变动给滤除了。

也许我们需要一种“恰到好处的”滤波机制：让 g 的值既不太大也不太小，而是处于恰到好处的水平。不过，其实并非完全如此。正如前面所提到的，不同的滤波机制会根据问题的数学特性来选择不同的 g 和 h 值。例如，Benedict-Bordner 滤波机制就是为了最小化这种例子中的瞬态误差而设计的，其中 $\dot{x}$ 的值会发生突变。我们在本书中不会详细讨论这种滤波机制，不过这里有两组不同的 g 和 h 组合所对应的曲线图。这种滤波机制能够有效减少 $\dot{x}$ 发生突变时的瞬态误差，但代价是它并不适合处理 $\dot{x}$ 的其他类型的变化。

```
zs = [5,6,7,8,9,9,9,9,9,10,11,12,13,14,
      15,16,16,16,16,16,16,16,16,16,16,16]
data1 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=.302, h=.054)
data2 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=.546, h=.205)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data2, label='g=0.546, h=0.205', marker='s', c='C0')
book_plots.plot_filter(data1, label='g=0.302, h=0.054', marker='v', c='C1')
plt.legend(loc=4)
plt.ylim([6, 18]);
```

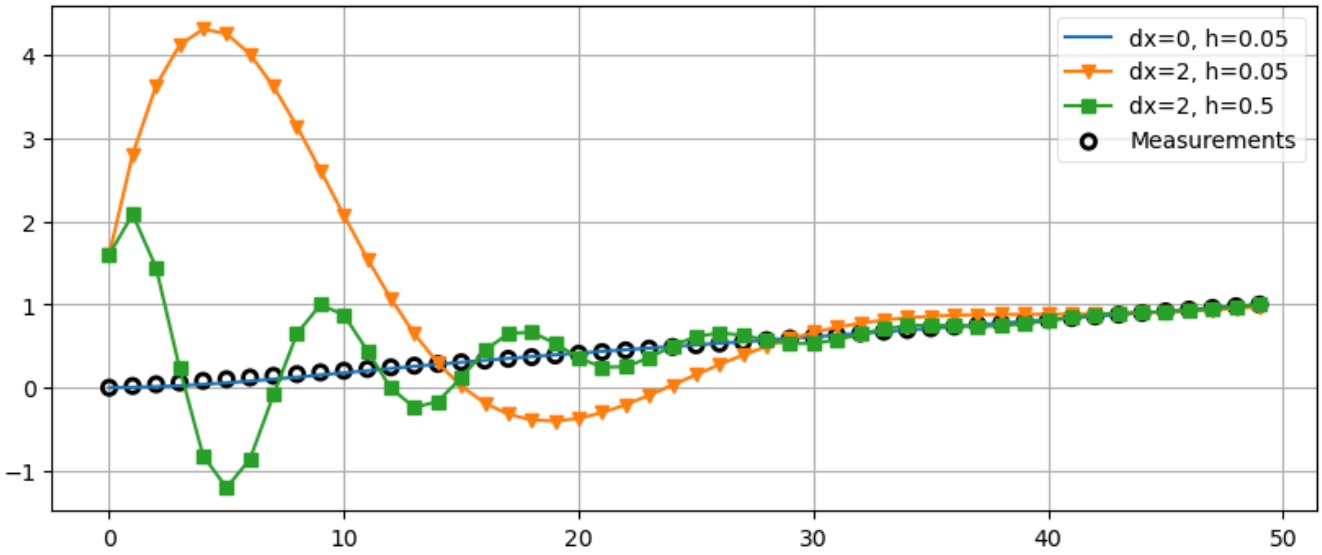


改变 h 的值

现在，让我们保持 g 不变，来研究修改 h 所带来的影响。我们知道， h 会影响我们倾向于使用 $\dot{x}$ 的测量结果还是自己的预测结果。但这到底意味着什么呢？如果我们的信号变化很大（相对于滤波器的时间步长来说变化很快），那么较大的 h 值会让我们更快地对那些瞬时变化做出反应。而较小的 h 值则会导致我们反应得更慢。

来看三个例子。无噪声信号在 50 个时间步内从 0 缓慢增加到 1。第一个滤波器采用接近正确值的 $\dot{x}$ 初始估计，以及较小的 h，输出与信号十分接近。第二个滤波器采用偏差很大的初始估计 $\dot{x} = 2$，输出出现振铃，随后才稳定并跟上信号。第三个例子与第二个条件相同，但改用 $h = 0.5$。此时振铃幅度明显减小，频率却更高；稳定所需的时间也稍短，但差别不大。

```
zs = np.linspace(0, 1, 50)

data1 = g_h_filter(data=zs, x0=0, dx=0., dt=1., g=.2, h=0.05)
data2 = g_h_filter(data=zs, x0=0, dx=2., dt=1., g=.2, h=0.05)
data3 = g_h_filter(data=zs, x0=0, dx=2., dt=1., g=.2, h=0.5)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data1, label='dx=0, h=0.05', c='C0')
book_plots.plot_filter(data2, label='dx=2, h=0.05', marker='v', c='C1')
book_plots.plot_filter(data3, label='dx=2, h=0.5',  marker='s', c='C2')
plt.legend(loc=1);
```



交互式示例

对于那些在 Jupyter Notebook 中运行该代码的用户，我编写了该滤波器的交互式版本。这样，你就可以实时查看当 $\dot{x}$ 、 g 和 h 的值发生变化时，会产生什么效果。当你调整 $\dot{x}$ 、 g 和 h 的数值时，数据会重新被滤波处理，相应的结果也会被绘制出来供你查看。

如果你真的想考验一下自己，那就请阅读下一段文字，试着在移动滑块之前预测出结果吧。

可以尝试以下设置：将 g 和 h 设置为最小值。看看滤波器对数据的追踪效果如何！之所以能实现完美的追踪，是因为我们成功预测了重量的变化情况。将 $\dot{x}$ 的值调整为大于或小于 5。此时，滤波器会偏离数据点，而无法再次与数据点重合。然后试着逐步增加 g 或 h 的值，观察滤波器是如何重新回到数据点附近的。比较仅添加 g 与仅添加 h 时的效果差异。你能解释这种差异的原因吗？接着，尝试将 g 的值设置为大于 1。你能解释此时的结果吗？将 g 的值恢复为合理的数值（比如 0.1），同时将 h 的值设置得很大。你能解释这些结果吗？最后，将 g 和 h 都设置为最大值。

如果你想进一步探索这个现象，可以将数组 `zs` 的值更改为上述图表中使用的数值，然后重新运行该单元格，查看结果。

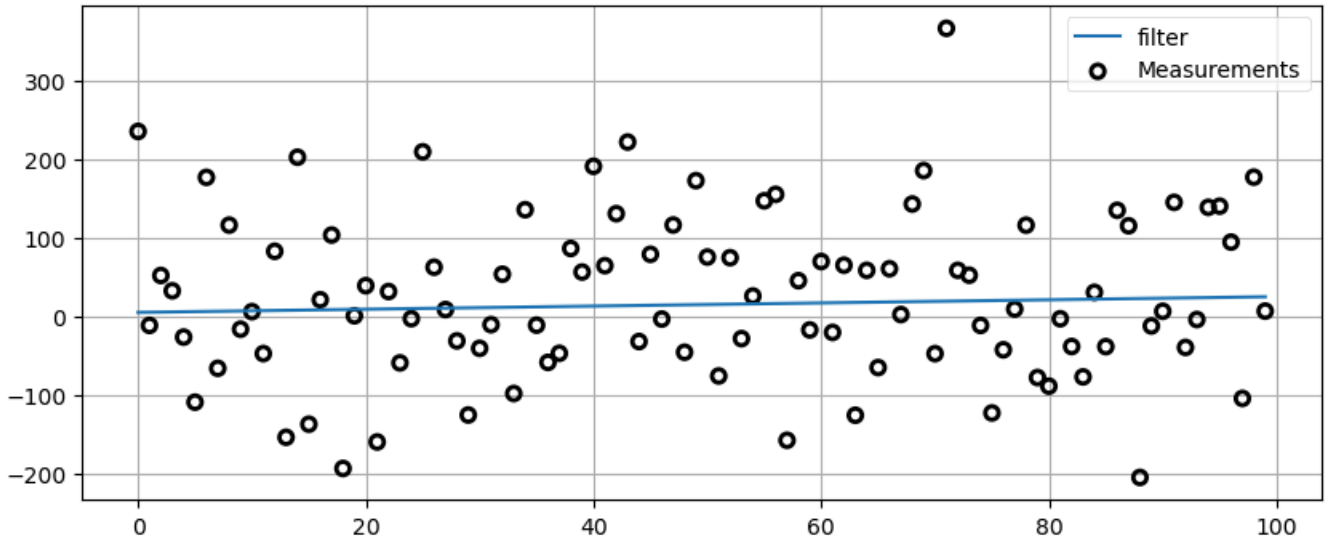
```
from ipywidgets import interact

# my FloatSlider returns an ipywidgets.FloatSlider with
# continuous_update=False. Filtering code runs too slowly
# to instantly react to slider changes.
from kf_book.book_plots import FloatSlider

zs1 = gen_data(x0=5, dx=5., count=100, noise_factor=50)

fig = None
def interactive_gh(x, dx, g, h):
    global fig
    if fig is not None: plt.close(fig)
    fig = plt.figure()
    data = g_h_filter(data=zs1, x0=x, dx=dx, g=g, h=h)
    plt.scatter(range(len(zs1)), zs1, edgecolor='k',
                facecolors='none', marker='o', lw=1)
    plt.plot(data, color='b')
    plt.show()

interact(interactive_gh,
         x=FloatSlider(value=0, min=-200, max=200),
         dx=FloatSlider(value=5, min=-50, max=50),
         g=FloatSlider(value=.1, min=.01, max=2, step=.02),
         h=FloatSlider(value=.02, min=.0, max=.5, step=.01));
```

```
interactive(children=(FloatSlider(value=0.0, continuous_update=False, description='x', max=200.0,
min=-200.0),…
```

不要对滤波系统说谎

你可以随意将 g 和 h 设置为任意数值。这个滤波器即使在有强烈噪声的情况下也能保持出色的滤波效果。

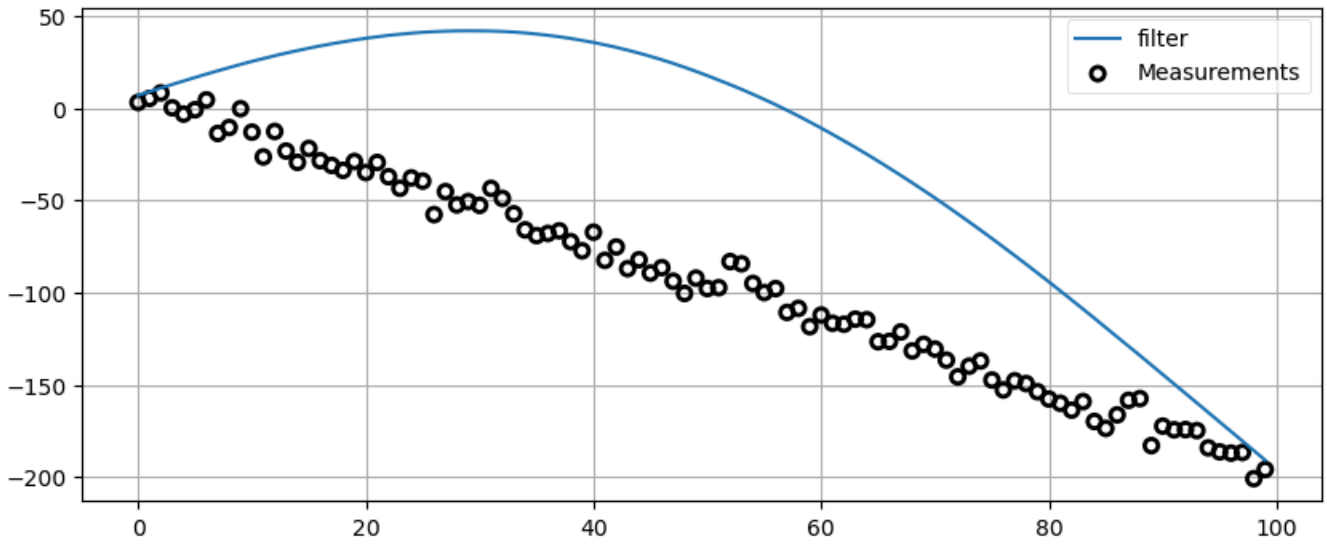
```
zs = gen_data(x0=5., dx=.2, count=100, noise_factor=100)
data = g_h_filter(data=zs, x0=5., dx=.2, dt=1., g=0., h=0.)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data, label='filter')
plt.legend(loc=1);
```



我成功地从充满噪声的数据中提取出了一条直线！也许我现在还不应该试图去争取数学界的菲尔兹奖。我之所以能做到这一点，是因为将 g 和 h 都设为 0。这样做有什么效果呢？它使得滤波器忽略掉所有的测量数据。因此，在每次更新时，系统都会以 $x + \Delta x \Delta t$ 作为新的位置值。当然，如果我们忽略所有的测量数据，最终得到的自然就是一条直线了。

那些忽略实际测量数据的滤波器毫无用处。我知道，你绝不会把 g 和 h 都设为零——因为那样做需要非凡的智慧，而这种智慧只有我具备。但我必须提醒你：如果不小心的话，你很可能会把这两个数值设置得过低。当然，你可以利用测试数据得到看似完美的结果。但当你把同样的滤波器应用到真实数据上时，你一定会失望的，因为你所调整的参数都是针对特定数据集而设定的。 g 和 h 应该反映的是你所要处理的系统的真实行为，而不是某个特定数据集的行为。在后面的章节中，我们会详细学习如何正确设置这些参数。现在我只能再说一次：请务必小心行事。否则，虽然你用测试数据能得到“完美”的结果，但一旦使用真实数据，结果就会大相径庭了。

```
zs = gen_data(x0=5, dx=-2, count=100, noise_factor=5)
data = g_h_filter(data=zs, x0=5., dx=2., dt=1., g=.005, h=0.001)
book_plots.plot_measurements(zs)
book_plots.plot_filter(data, label='filter')
plt.legend(loc=1);
```



追踪火车的行进轨迹

我们准备通过一个具体的例子来加以说明。在本章的前面部分，我们讨论了如何追踪火车的行踪。火车重量大、速度慢，因此无法快速改变速度。由于火车行驶在铁轨上，它只能通过减速停车后再掉头来改变方向。因此，如果我们知道火车的大致位置和速度，那么我们就可以相当准确地预测它未来的位置。火车在一两秒钟内是无法显著改变其速度的。

下面为火车编写滤波器。位置表示相对于轨道上某个固定点的距离，规定该点为 0 千米。例如，位置为 1 表示火车距固定点 1 千米。速度单位为米/秒，每秒测量一次位置，误差为 ±500 米。应该如何实现这个滤波器?

首先，让我们模拟一下没有滤波器的情况。假设火车当前位于 23 公里处，以 15 米/秒的速度行驶。我们可以这样来编码实现这一情况:

```
pos = 23*1000
vel = 15
```

现在，我们可以计算出在未来的某个时刻，列车的位置。当然，这是假设列车的速度保持不变的情况下。

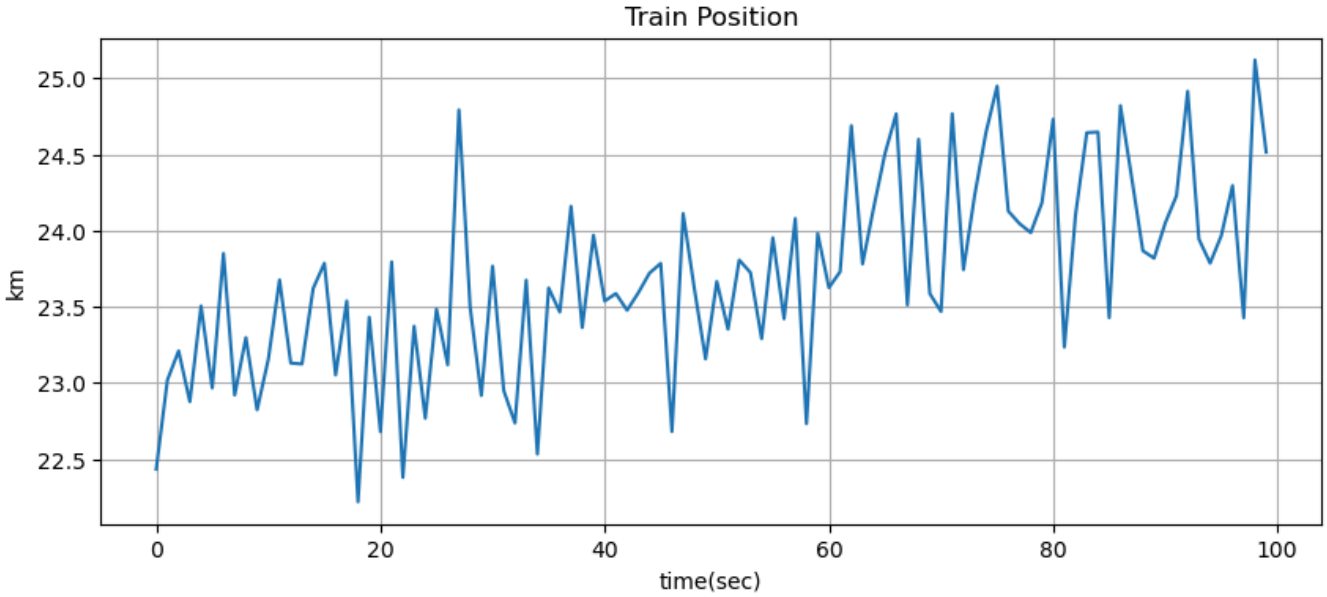
```
def compute_new_position(pos, vel, dt=1):
return pos + (vel * dt)
```

可以在位置真值上加随机噪声来模拟测量。下面把高斯测量噪声的标准差设为 500 米，代码如下:

```
def measure_position(pos):
return pos + random.randn()*500
```

把上述代码放入一个代码单元，画出 100 秒仿真的结果。使用 NumPy 的 asarray 将数据转换为数组，便可用 / 运算符一次性对所有元素做除法。

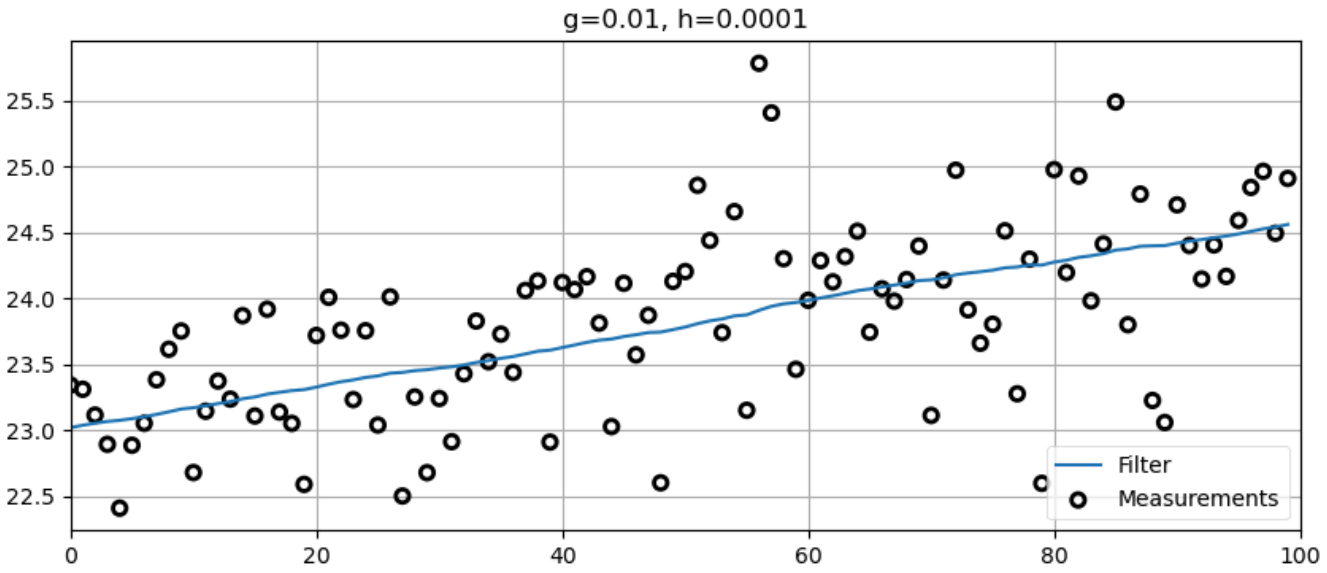
```
from numpy.random import randn

def compute_new_position(pos, vel, dt=1.):
    """ dt is the time delta in seconds."""
    return pos + (vel * dt)

def measure_position(pos):
    return pos + randn()*500

def gen_train_data(pos, vel, count):
    zs = []
    for t in range(count):
        pos = compute_new_position(pos, vel)
        zs.append(measure_position(pos))
    return np.asarray(zs)

pos, vel = 23.*1000, 15.
zs = gen_train_data(pos, vel, 100)

plt.plot(zs / 1000.)  # convert to km
book_plots.set_labels('Train Position', 'time(sec)', 'km')
```



从图表中可以看出，这些测量数据糟糕透顶。没有任何一列真正的火车能够以这样的方式行驶。

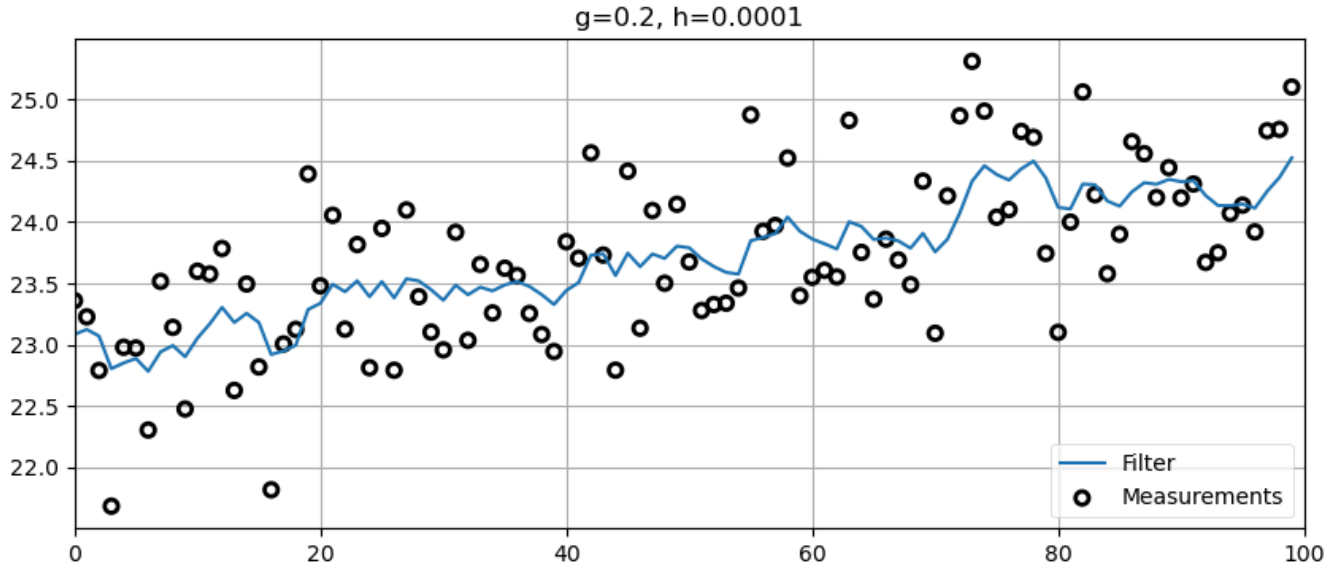
那么，如果我们想滤波这些数据，应该将 g 和 h 设置为多少呢？虽然我们还没有为此制定出具体的理论依据，但我们可以凭直觉来尝试找出一个合理的数值。我们知道，这些测量数据的准确性很低，因此我们不想给它们太大的权重。为此，我们需要将 g 设置得非常小。另外，由于火车无法快速加速或减速，所以 h 也应该设置得非常小。例如：

```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.0001')
```



作为初步的猜测，这个结果相当不错。让我们把 g 的值调大一些，看看会有什么效果。

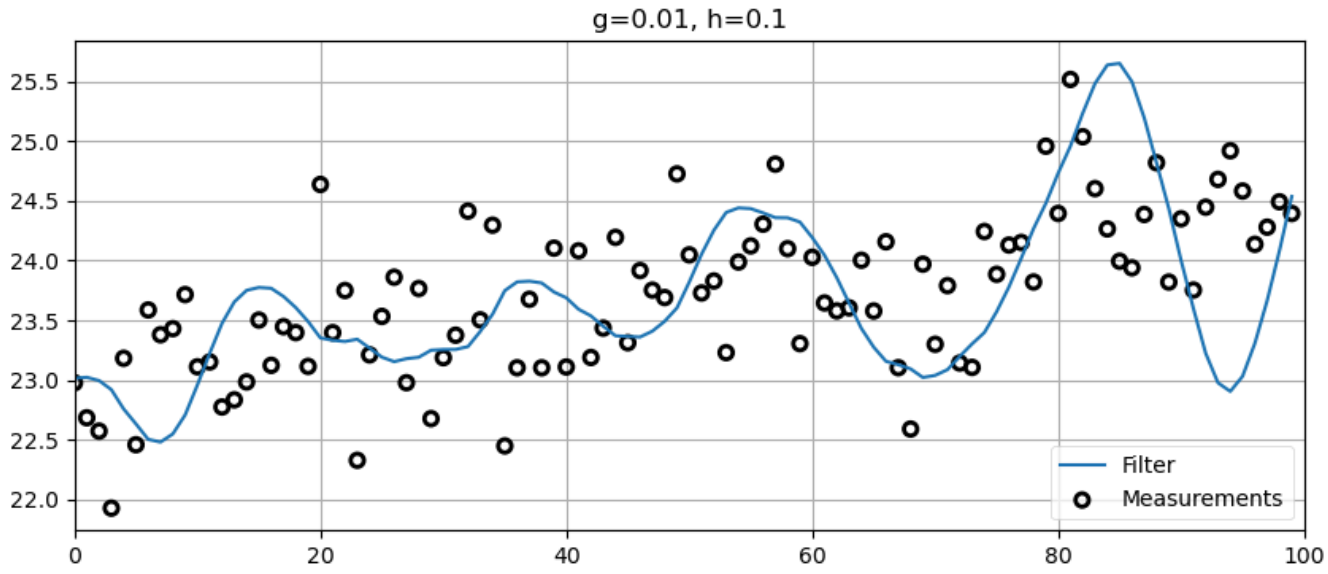
```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.2, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.2, h=0.0001')
```



我们得到了 g=0.2 的结果。可以看出，虽然列车的位置被平滑处理过了，但估计出的位置（进而速度）在极短的时间间隔内会有很大的波动，这种波动幅度远远超过了真实列车的正常波动范围。因此，从实际经验来看，我们更倾向于使用 g<<0.2 。

现在，让我们来看看对 h 做出错误选择会带来怎样的后果吧。

```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=0.01, h=0.1)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.1')
```

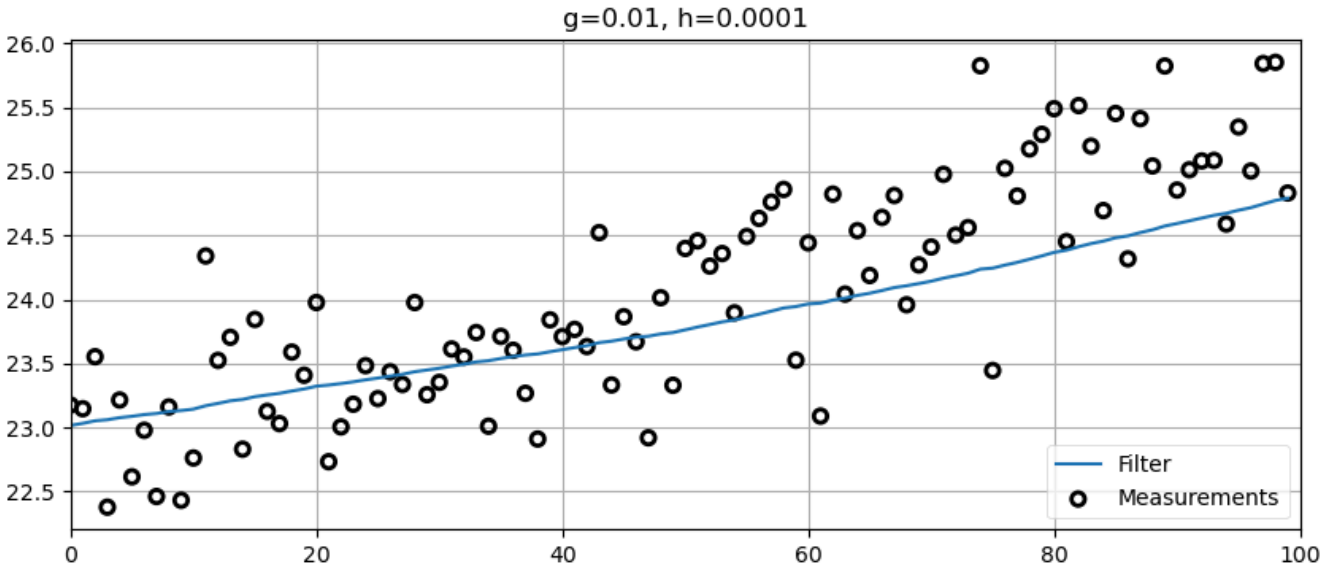


在这里，由于 g 的数值较小，因此位置的变动较为平滑。但 h 的数值较大，导致滤波器对各种测量数据的反应过于敏感。这是因为在几秒钟的时间里，数据的快速变化意味着速度也有很大的变动。而较大的 h 数值则迫使滤波器迅速对这些变化做出反应。不过，火车无法如此快速地改变速度，因此滤波器在处理数据时效果不佳——滤波器反应的速度远远快过了火车的反应速度。

最后，让我们给这列火车加上一些加速度吧。我不知道火车的实际加速度有多大，但假设它的加速度为 0.2 米/秒^{2}吧。

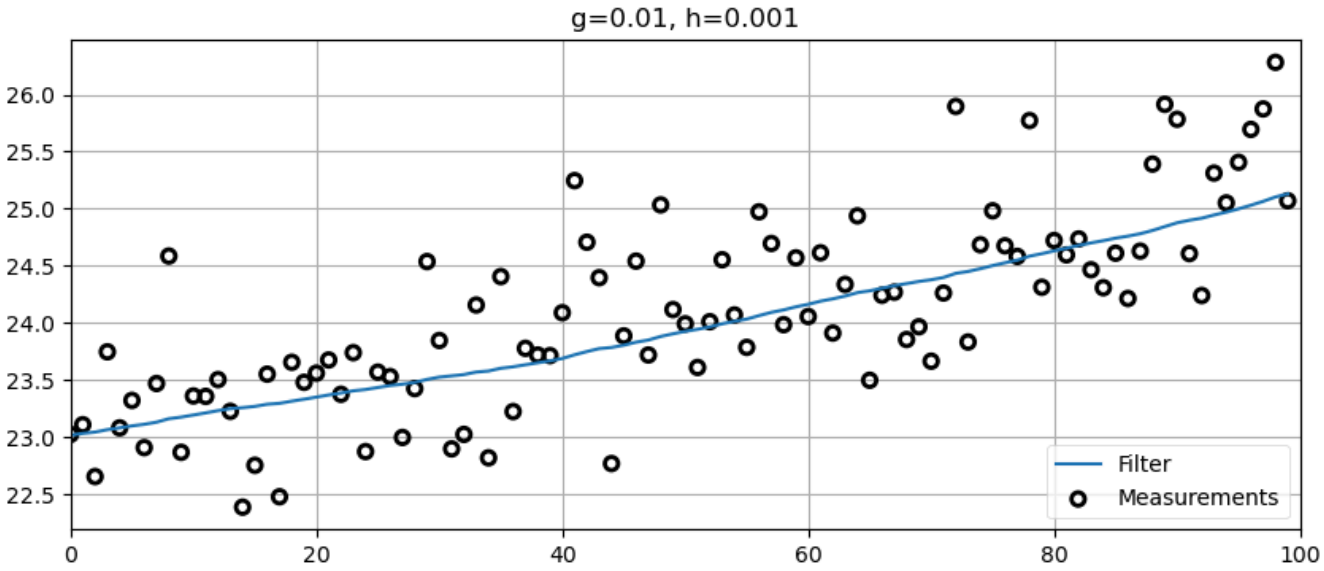
```
def gen_train_data_with_acc(pos, vel, count):
    zs = []
    for t in range(count):
        pos = compute_new_position(pos, vel)
        vel += 0.2
        zs.append(measure_position(pos))
    return np.asarray(zs)
```

```
zs = gen_train_data_with_acc(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.0001')
```



可以看到，由于加速度的作用，滤波器已经无法准确跟踪列车的运动了。我们可以通过调整 h 的参数来改善这一情况，不过这样一来，滤波后的结果就会不够平滑。

```
zs = gen_train_data_with_acc(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.001')
```



这里有两个值得借鉴的教训。首先，可以利用 $\boldsymbol{h}$ 这个参数来应对那些未被纳入模型中的速度变化。但更重要的是，如何在快速、准确地响应行为变化与确保系统在稳定状态时能够输出理想结果之间找到平衡。如果列车的速度始终不变，我们可以将 $\boldsymbol{h}$ 的值设得极小，从而避免滤波后的数值受到测量噪声的过度影响。但在现实情况中，系统的状态几乎总是在变化，我们需要尽快对这些变化做出反应。不过，反应越快，传感器中的噪声带来的影响就越大。

我还可以继续说下去，但我的目的并非要详细阐述 G-H 滤波器的相关理论，而是想深入理解：如何通过结合各种测量数据和预测结果来得到最终的滤波结果。关于如何为这类问题选择合适的参数 $\boldsymbol{g}$ 和 $\boldsymbol{h}$，已有大量研究文献。为了实现不同的目标，也有相应的最佳选择方法。正如我之前所解释的，使用这样的测试数据进行实验时，很容易让滤波器“产生错误判断”。在后面的章节中，我们将了解卡尔曼滤波器是如何以类似的方式解决这个问题的，只不过它所使用的数学方法要复杂得多。

使用 FilterPy 实现的 g-h 滤波器

FilterPy 是我自己编写的开源滤波库。它包含了本书中提到的所有滤波功能，此外还有更多其他功能。虽然自己编写 g-h 滤波器相当简单，但随着学习的深入，我们会越来越依赖 FilterPy。作为简要介绍，让我们先来看看 FilterPy 中的 g-h 滤波器吧。

如果你还没有安装 FilterPy，请在命令行中输入以下命令即可。

```
pip install filterpy
```

如需了解如何从 GitHub 安装或下载 FilterPy 的更多详细信息，请参阅附录 A。

要使用 g-h 滤波器，需先将其导入，并通过 `GHFilter` 类来创建该滤波器的实例。

```
from filterpy.gh import GHFilter
f = GHFilter(x=0., dx=0., dt=1., g=.8, h=.2)
```

在创建该对象时，需要指定信号的初始值和变化率（即 `x` 和 `dx`）、更新之间的时间间隔（ `dt` ），以及两个滤波器参数的值（ `g` 和 `h` ）。 `dx` 的单位必须与 `x` / `dt` 的单位相同——如果 `x` 的单位是米， `dt` 的单位是秒，那么 `dx` 的单位就必须是米每秒。

要运行该滤波器，需调用 `update` 函数，并将测量结果作为参数 `z` 传递进去。需要提醒的是， `z` 在相关文献中是用于表示测量结果的标准名称。

```
f.update(z=1.2)
```

```
(0.96, 0.24)
```

`update()` 会以元组的形式返回 `x` 和 `dx` 的新值，不过你也可以直接通过该对象来访问它们。

```
print(f.x, f.dx)
```

```
0.96 0.24
```

你可以动态地更改 g 和 h 的值。

```
print(f.update(z=2.1, g=.85, h=.15))
```

```
(1.965, 0.375)
```

可以使用批量接口对一串测量值依次进行滤波。

```
print(f.batch_filter([3., 4., 5.]))
```

```
[[1.965 0.375]
 [2.868 0.507]
 [3.875 0.632]
 [4.901 0.731]]
```

可以对多个相互独立的变量分别滤波。如果要跟踪飞机，就需要跟踪三维空间中的运动。用 NumPy 数组表示 x、dx 和测量值。

```
x_0  = np.array([1., 10., 100.])
dx_0 = np.array([10., 12., .2])

f_air = GHFilter(x=x_0, dx=dx_0, dt=1., g=.8, h=.2)
f_air.update(z=np.array((2., 11., 102.)))
print(' x =', f_air.x)
print('dx =', f_air.dx)
```

```
 x = [  3.8   13.2  101.64]
dx = [8.2  9.8  0.56]
```

GHFilterOrder 这个类允许你创建 0 阶、1 阶或 2 阶的滤波器。g-h 滤波器属于 1 阶滤波器。至于我们尚未讨论过的 g-h-k 滤波器，它则能够跟踪加速度。这两种滤波器都具备实际应用中所需的功能，比如计算方差降低因子（VRF），不过这一点我们在这章中没有详细讨论。关于 g-h 滤波器的理论及其应用，完全可以写成一本书。不过，本书还有其他目标需要实现。如果你感兴趣的话，可以仔细研究 FilterPy 中的相关代码，并进一步阅读相关资料。

FilterPy 的文档地址为：https://filterpy.readthedocs.org/。

本章小结

我建议你尝试使用这个滤波器，从而更好地了解它的运作方式。不用试太多次，你就会发现，对 g 和 h 的随意选择往往不会带来好的效果。某个特定的选择在某种情况下可能效果不错，但在另一种情况下则可能适得其反。即便你理解了 g 和 h 的作用，要选择合适的数值也并非易事。事实上，你很难为 g 和 h 选出在特定问题上最理想的数值。滤波器是经过精心设计的，而不是随意挑选出来的。

关于如何选择 g 和 h，还有很多可以讨论的内容。不过，本书的目标并不是完整介绍这种形式的 g-h 滤波器。设计卡尔曼滤波器时，需要确定若干参数；它们与 g 和 h 有间接联系，但设计过程中通常并不直接使用这两个记号。此外，g 和 h 还会以不那么直观的方式随时间步变化。

我们还几乎没有讨论这些滤波器的另一个方面：贝叶斯统计。“贝叶斯”出现在书名中并非偶然。接下来先暂时离开尚未充分展开的 g 和 h，学习一种有力的概率滤波推理方法。之后，相同的 g-h 滤波算法会重新出现，这一次，它将置于完整的数学框架之中，使我们能融合多个传感器的信息，准确估计结果的不确定性，并用于控制机器人。

第 2 章：离散贝叶斯滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

卡尔曼滤波器属于贝叶斯滤波器这一类滤波器。在大多数关于卡尔曼滤波器的教材中，都会介绍贝叶斯公式，也会说明该公式是如何被运用到卡尔曼滤波器方程中的。不过，这些介绍大多停留在非常抽象的层面。

这种方法需要读者对多个数学领域有相当深入的理解。不过，对于如何理解并直观地把握相关情况，大部分工作仍需由读者自己来完成。

我会用另一种方式来探讨这个主题。在这一点上，我深受 Dieter Fox 和 Sebastian Thrun 的启发。其核心思路是通过让物体在走廊中移动来帮助人们理解贝叶斯统计的原理——他们使用机器人来模拟物体移动，而我则用狗来代替。我喜欢狗，不过狗的行为比机器人更难以预测，这就给数据处理带来了挑战。据我所知，关于这一方法的最早研究文献是 Fox 在 1999 年发表的论文[1]，更详细的介绍则见于 Fox 在 2003 年的论文[2]。Sebastian Thrun 也在他那门精彩的 Udacity 课程《人工智能与机器人技术》中采用了这种方法[3]。如果你喜欢观看视频，我强烈建议先暂停阅读本书，先学习该课程的前几节课，然后再回来仔细阅读本书，以便更深入地理解这个主题。

现在，让我们来做一个简单的思维实验，就像我们之前对 g-h 滤波器所做的那样。通过这个实验，我们可以了解如何利用概率来进行数据的滤波和追踪处理。

追踪一只狗

先从一个简单问题开始。我们的工作场所允许带狗上班，但狗有时会离开办公室，在走廊中游荡，我们希望跟踪它们。一次黑客马拉松中，有人发明了可装在狗项圈上的声纳传感器：它发出信号、接收回波，根据回波返回所需的时间判断狗是否位于敞开的门口。传感器还能检测狗的行走及移动方向，通过 Wi-Fi 联网，每秒发送一次更新。

我想追踪我的狗西蒙的行踪。于是，我把该设备安装在西蒙的项圈上，然后启动 Python 编程语言，准备编写代码来追踪西蒙在楼内的位置。乍看之下，这似乎是不可能的。如果我监听西蒙项圈上的传感器数据，可能会听到“门”、“走廊”之类的信息。那我该如何利用这些信息来确定西蒙的具体位置呢?

为了便于处理，我们会假设走廊上只有 10 个位置，这些位置分别用 0 到 9 来编号，其中 1 号位置位于 0 号位置的右侧。出于稍后会解释的原因，我们还会假设走廊是圆形或矩形的。如果你从 9 号位置向右移动，就会回到 0 号位置。

当我开始监听传感器的信号时，我没有理由认为西蒙一定位于走廊的某个特定位置。在我看来，他有可能处于走廊的任何一个位置。走廊总共有 10 个位置，因此他处于任意一个位置的概率都是 1/10。

用一个 NumPy 数组表示我们对狗的位置的信念分布。也可以用 Python 列表，但 NumPy 提供了稍后会用到的功能。

```
import numpy as np
belief = np.array([1/10]*10)
print(belief)
```

```
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

在贝叶斯统计中，这称为先验概率，即纳入当前测量或其他新信息之前的概率。更完整的名称是先验概率分布，它列出各种可能结果及各自的概率。本章讨论的离散概率分布包含所有可能的位置，因此各项概率之和必须为 1。

你一定用过概率，例如“今天下雨的概率为 30%”。上一段也在谈概率，但贝叶斯统计的一个重要特点，是把概率解释为对某个具体事件的信念程度。举例说，公平硬币大量反复抛掷，正反面的长期频率各趋近 50%。这种依据事件发生频率的解释称为频率学派解释，用来区别于贝叶斯解释。

我再抛一次硬币，然后让它自然落地。我觉得硬币会落在哪一面呢？频率主义概率论对此无能为力；它只会说，50%的抛硬币结果会是正面。从某种意义上说，给硬币的当前状态赋予概率是没有意义的。硬币要么是正面，要么是反面，我们只是不知道到底是哪一面而已。贝叶斯理论则将此视为对某一事件的“信念”——也就是说，我认为这次抛硬币结果是正面的概率是 50%。不过，有些人反对使用“信念”这个词；因为“信念”可能意味着在没有证据的情况下仍然认为某事为真。在这本书中，“信念”始终指的是我们对某事的了解程度。我们会在后续的内容中进一步了解这一点。

贝叶斯统计会把过去的信息，即先验，考虑在内。假如观察到每 100 天有 4 天下雨，仅据这一频率，可能会说明天下雨的概率为 1/25。但天气预测不会只这样做：如果今天正在下雨，而且锋面停滞不前，明天继续下雨就很有可能。这体现了利用已有信息更新判断的贝叶斯思想。

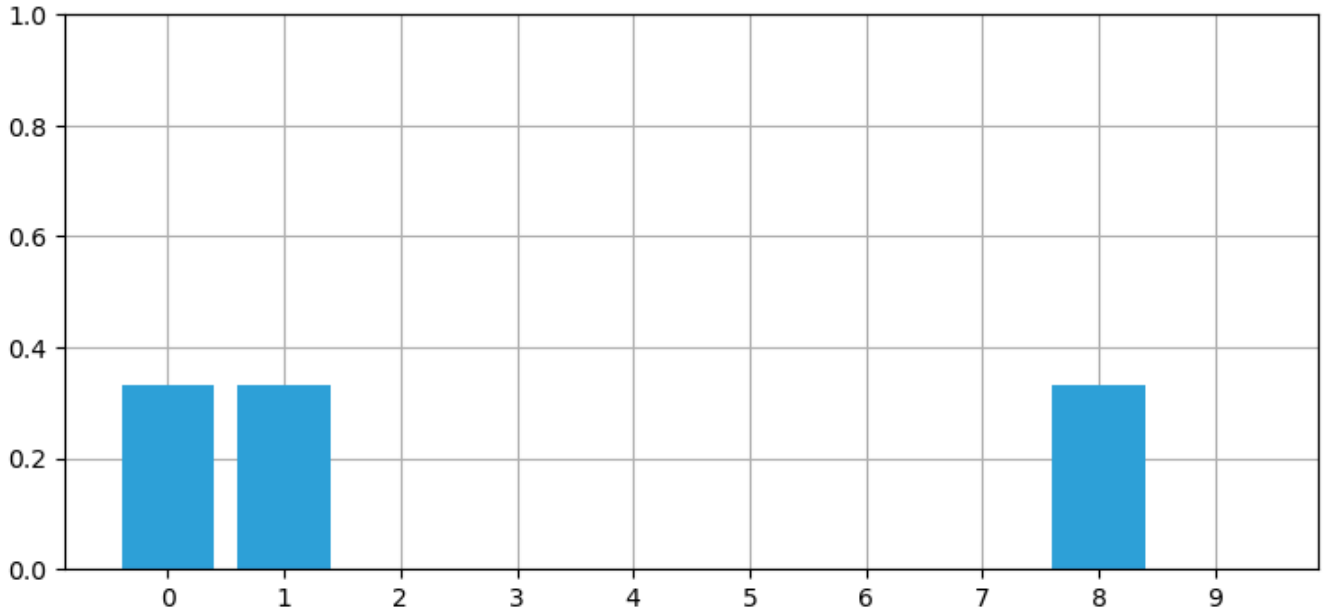
在实践中，统计学家会结合使用频率主义和贝叶斯主义的方法。有时候，确定先验概率很困难或根本不可能，这时就需要采用频率主义方法。而在本书中，我们能够确定先验概率。当我谈论某件事的概率时，我指的是在已知过去事件的情况下，该事件发生的概率。在这种情况下，我采用的是贝叶斯主义方法。

现在，让我们来绘制一下走廊的地图吧。我们将把前两扇门放得靠得很近，然后再放一扇门，距离稍远一些。我们用 1 来表示门，用 0 来表示墙壁：

```
hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])
```

我开始收听西蒙通过网络发送的信息。从传感器那里得到的第一组数据是“门”。暂时假设传感器总是能给出正确的答案。由此可以推断，他现在正站在某扇门的前面，但到底是哪扇门呢？我没有理由认为他站在第一扇、第二扇或第三扇门的前面。我能做的就是给每扇门分配一个概率值。由于三扇门的可能性都相等，所以每扇门被分配的概率都是 1/3。

```
import kf_book.book_plots as book_plots
from kf_book.book_plots import figsize, set_figsize
import matplotlib.pyplot as plt

belief = np.array([1/3, 1/3, 0, 0, 0, 0, 0, 0, 1/3, 0])
book_plots.bar_plot(belief)
```



这种分布称为分类分布，是描述 n 种可能结果之概率的离散分布。由于我们对狗的位置有多个候选判断，它又是一种多峰分布。这并不是说狗同时位于三个地方，而是说已将位置的不确定范围缩小到其中之一。我的贝叶斯信念是：狗在 0 号门的概率为 33.3%，在 1 号门的概率为 33.3%，在 8 号门的概率也为 33.3%。

信息在两个方面改善了判断：一批走廊位置被排除，而剩余候选位置各自的概率从 10% 增加到约 33%。随着有效信息继续积累，分布可能越来越集中，某个位置的概率可能接近 100%。不过，这并不意味着每次观测都会提高定位的确定程度；单次观测也可能使后验分布更分散。

关于分布的众数，简单来说：给定一组数字，比如{1, 2, 2, 2, 3, 3, 4}，众数就是出现次数最多的那个数字。在这个例子中，众数是 2。一个分布可以有多个众数。例如，{1, 2, 2, 2, 3, 3, 4, 4, 4}这个集合的众数是 2 和 4，因为这两个数字都出现了三次。我们把前者称为单峰分布，后者则称为多峰分布。

这种分布形态也被称为直方图。直方图以图形方式展示了某一组数据的分布情况。上图就是一幅直方图。

我在上面的代码中手动编写了 `belief` 数组的相关处理。那么，我们该如何在代码中实现这一功能呢？我们可以用 1 来表示门，用 0 来表示墙。因此，我们需要将“走廊”对应的数值乘以相应的百分比，具体操作如下：

```
belief = hallway * (1/3)
print(belief)
```

```
[0.333 0.333 0.    0.    0.    0.    0.    0.    0.333 0.   ]
```

从传感器读数中提取信息

先暂且不谈 Python，让我们仔细思考一下这个问题。假设我们从西蒙的传感器中读取到了以下数据：

- 门
- 向右移动
- 门

我们能推断出西蒙的位置吗？当然可以！根据走廊的布局，只有唯一一个位置能够产生这样的传感器读数，那就是走廊的左端。因此，我们可以确定西蒙现在位于第二道门的前方。如果还不明白的话，不妨假设西蒙是从第二道或第三道门开始移动的。如果他那样做的话，他的传感器会检测到“墙壁”信号。但这与实际的传感器读数不符，所以可以排除这种可能性。用同样的逻辑来分析其他所有可能的起始位置后，我们可以得出结论：西蒙现在肯定位于第二道门的前方。我们的结论是：

```
belief = np.array([0., 1., 0., 0., 0., 0., 0., 0., 0., 0.])
```

我设计了走廊的布局以及传感器的检测方式，以便能快速得到准确的答案。现实中的问题往往没那么简单明了。不过，这个例子应该能帮助你理解其中的原理：最初，第一次传感器读数表明西蒙可能位于几个位置，但概率都很低（0.333）。但在经过位置更新和再次检测后，我们就能更准确地确定他的位置了。你可以合理地推测，如果走廊很长，门的数量也很多，那么经过多次检测和位置更新后，我们要么能确定西蒙的具体位置，要么至少能把可能的位置范围缩小到几个选项之内。当所有的传感器检测结果都只与一两个初始位置相符时，这种情况就有可能发生。

我们现在就可以实施这个解决方案，不过，让我们先来考虑一下现实中可能出现的各种复杂情况吧。

噪声较大的传感器

完美的传感器很少见。西蒙坐在门前挠痒，或者没有面朝走廊时，传感器都可能误判。因此，即使收到“门”的读数，也不能把每扇门对应的位置都赋予 1/3 的概率；应让这些概率略小于 1/3，并给各个墙壁位置保留少量概率。例如：

```
[.31, .31, .01, .01, .01, .01, .01, .01, .31, .01]
```

起初，这似乎是无法克服的难题。如果传感器的信号有噪声，那么所有数据都不可信。如果我们始终无法确定数据的准确性，又如何能得出任何结论呢？

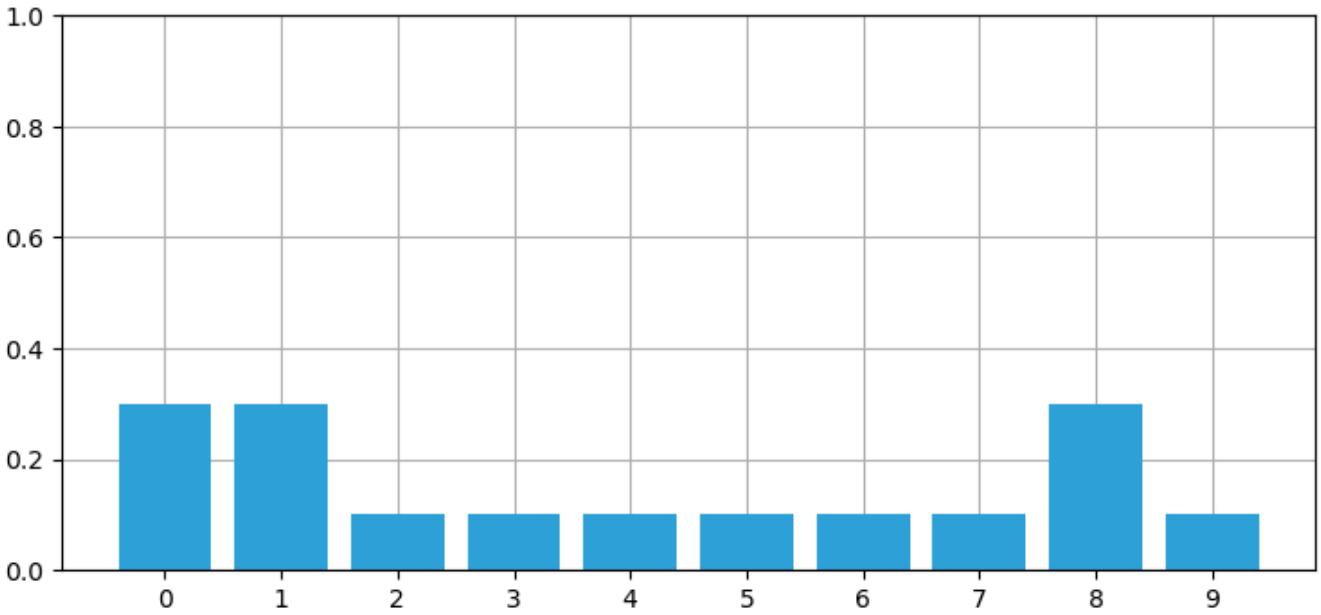
与上述问题一样，答案也与概率有关。我们已经能够为狗的位置赋予相应的概率值；现在，我们需要把传感器噪声带来的额外不确定性也考虑进去。

假设收到“门”的读数，测试表明传感器判断正确的概率是错误的三倍。那么，只把与“门”相对应位置的先验概率乘以 3。这样得到的数组暂时不再是归一化的概率分布，稍后再处理归一化。

让我们用 Python 代码来演示一下。在这里，我使用变量 `z` 来表示测量结果。在相关文献中， `z` 或 `y` 也是常用的变量名。作为程序员，我更喜欢使用有意义的变量名。不过，为了方便你阅读相关文献或其它滤波代码，我现在开始使用这些缩写形式。

```
def update_belief(hall, belief, z, correct_scale):
    for i, val in enumerate(hall):
        if val == z:
            belief[i] *= correct_scale

belief = np.array([0.1] * 10)
reading = 1 # 1 is 'door'
update_belief(hallway, belief, z=reading, correct_scale=3.)
print('belief:', belief)
print('sum =', sum(belief))
plt.figure()
book_plots.bar_plot(belief)
```

```
belief: [0.3 0.3 0.1 0.1 0.1 0.1 0.1 0.1 0.3 0.1]
sum = 1.6000000000000003
```



这并不是一种概率分布，因为它各元素的和并不等于 1.0。不过，代码的实现方式大体上是正确的：各个门的概率值被设置为 0.3，这个数值是墙壁概率值 0.1 的 3 倍。我们所需要做的，就是对结果进行归一化处理，使得所有概率值之和等于 1.0。归一化的方法是将列表中每个元素都除以该列表中所有元素的总和。使用 NumPy 的话，这一操作非常简单。

```
belief / sum(belief)
```

```
array([0.188, 0.188, 0.062, 0.062, 0.062, 0.062, 0.062, 0.062, 0.188,
       0.062])
```

FilterPy 通过 `normalize` 函数来实现这一功能：

```
from filterpy.discrete_bayes import normalize
normalize(belief)
```

说“正确的概率是错误的 3 倍”这种说法有点奇怪。我们处理的是概率问题，所以应该明确指出传感器判断正确的概率，再据此计算出相应的比例系数。相关的计算公式如下：

$$scale = \frac{prob_{correct}}{prob_{incorrect}} = \frac{prob_{correct}}{1 - prob_{correct}}$$

此外， `for` 循环的使用也很麻烦。一般来说，最好避免在 NumPy 代码中使用 `for` 循环。NumPy 是用 C 和 Fortran 语言实现的，因此，如果不用 for 循环，程序的执行速度通常会比使用 for 循环快 100 倍以上。

我们该如何摆脱这个 `for` 循环呢？NumPy 允许使用布尔数组来对数组进行索引操作。我们可以利用逻辑运算符来创建布尔数组。这样，我们就能找到走廊里所有的门了：

```
hallway == 1
```

```
array([ True,  True, False, False, False, False, False, False,  True,
       False])
```

当使用布尔数组作为另一个数组的索引时，该操作只会返回索引值为 `True` 的元素。因此，我们可以用这种方式来替代 `for` 循环。

```
belief[hall==z] *= scale
```

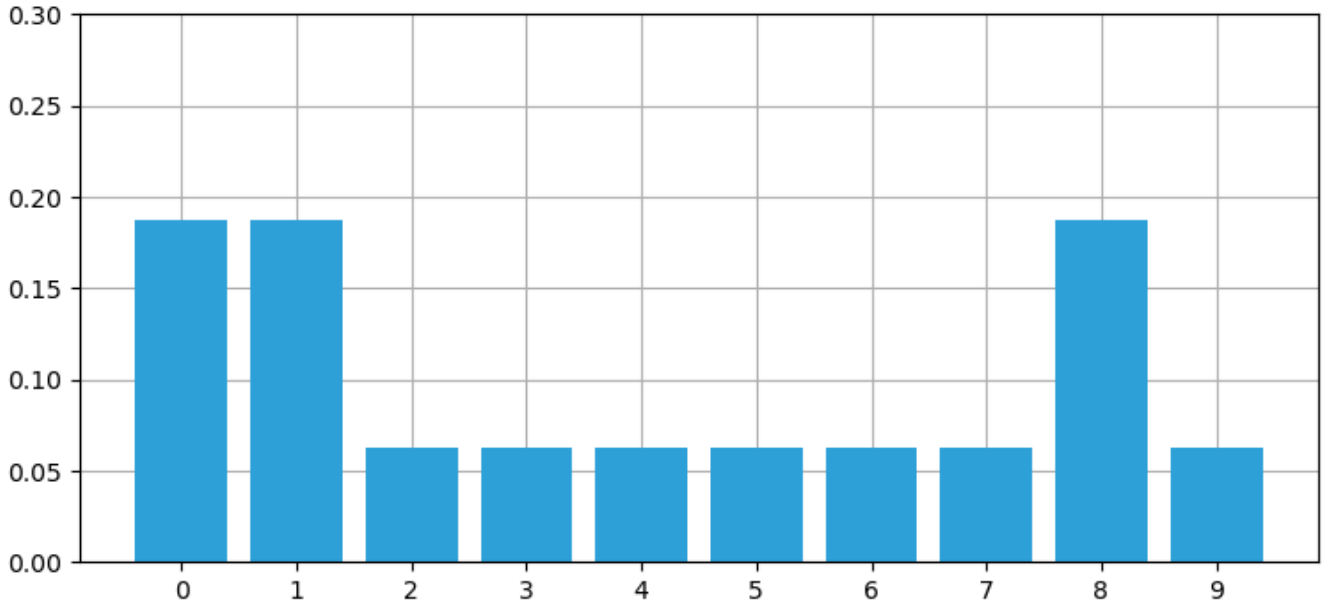
这里只把 hall 中等于 z 的位置所对应的 belief 元素乘以 scale。

教授 NumPy 的使用方法超出了本书的范畴。我会使用 NumPy 中的常用语法，并在首次介绍时加以说明。如果你是 NumPy 的新手，网上有很多文章和视频介绍如何高效且恰当地使用 NumPy。

这是我们改进后的版本：

```
from filterpy.discrete_bayes import normalize

def scaled_update(hall, belief, z, z_prob):
    scale = z_prob / (1. - z_prob)
    belief[hall==z] *= scale
    normalize(belief)

belief = np.array([0.1] * 10)
scaled_update(hallway, belief, z=1, z_prob=.75)

print('sum =', sum(belief))
print('probability of door =', belief[0])
print('probability of wall =', belief[2])
book_plots.bar_plot(belief, ylim=(0, .3))
```

```
sum = 1.0
probability of door = 0.1875
probability of wall = 0.06249999999999999
```



从输出结果可以看出，现在的总和为 1.0。同时，门与墙的概率之比仍然是 3:1。这一结果符合我们的直觉：门的概率肯定小于 0.333，而墙的概率则肯定大于 0.0。此外，这也符合我们的另一个直觉：目前我们还没有任何信息可以用来区分不同的门或墙的位置。因此，所有门的概率都应该相同，墙的概率也应当如此。

这个结果被称为“后验概率分布”。所谓“后验概率分布”，指的是在纳入了测量数据之后的概率分布。简单来说，就是把测量数据考虑进去之后的概率分布情况。回顾一下，所谓“先验概率分布”，则是指在还没有纳入测量数据之前的概率分布。

另一个术语是似然，即给定状态时得到当前测量的概率或概率密度。对每个候选位置，计算假定狗位于该处时，当前传感器读数出现的可能性；把这些似然值乘到先验上，再归一化，就得到后验。作为状态位置的函数，似然不要求各位置取值之和等于 1，因此不应与给定测量后的状态概率分布混淆。

将这些因素结合起来，就得到了这个方程式。

$$\textbf{posterior} = \frac{\textbf{likelihood} \times \textbf{prior}}{\textbf{normalization}}$$

当我们谈论滤波器的输出时，通常把预测后的状态称为“先验状态”或“预测值”；而把更新后的状态则称为“后验状态”或“估计状态”。

学习并牢记这些术语非常重要，因为大多数文献中都会大量使用它们。

`scaled_update()` 会执行这个计算吗？它会。让我把它重新表述成这样：

```
def scaled_update(hall, belief, z, z_prob):
    scale = z_prob / (1. - z_prob)
    likelihood = np.ones(len(hall))
    likelihood[hall==z] *= scale
    return normalize(likelihood * belief)
```

这个函数并非具有完全的通用性。它包含了与走廊结构相关的信息，以及如何将各种测量数据与走廊结构相对应的方法。我们始终致力于编写具有通用性的函数。在这里，我们将把似然的计算部分从函数中分离出来，要求调用者自行进行可能性的计算。

以下是该算法的完整实现代码：

```
def update(likelihood, prior):
return normalize(likelihood * prior)
```

似然的计算方式因具体问题而异。例如，传感器可能不会只返回 1 或 0，而是返回 0 到 1 之间的数值，以此表示物体位于门前的概率。传感器也可能利用计算机视觉技术来识别物体的形状，然后将其与门进行匹配。或者，传感器可以使用声纳来测量距离。无论哪种情况，似然的计算方式都不尽相同。在本书中，我们将看到许多这样的例子，并学习如何进行这些计算。

FilterPy 实现了 `update` 功能。以下是该功能的通用实现方式：

```
from filterpy.discrete_bayes import update

def lh_hallway(hall, z, z_prob):
    """ compute likelihood that a measurement matches
    positions in the hallway."""

    try:
        scale = z_prob / (1. - z_prob)
    except ZeroDivisionError:
        scale = 1e8

    likelihood = np.ones(len(hall))
    likelihood[hall==z] *= scale
    return likelihood

belief = np.array([0.1] * 10)
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
update(likelihood, belief)
```

```
array([0.188, 0.188, 0.062, 0.062, 0.062, 0.062, 0.062, 0.062, 0.188,
       0.062])
```

纳入运动信息

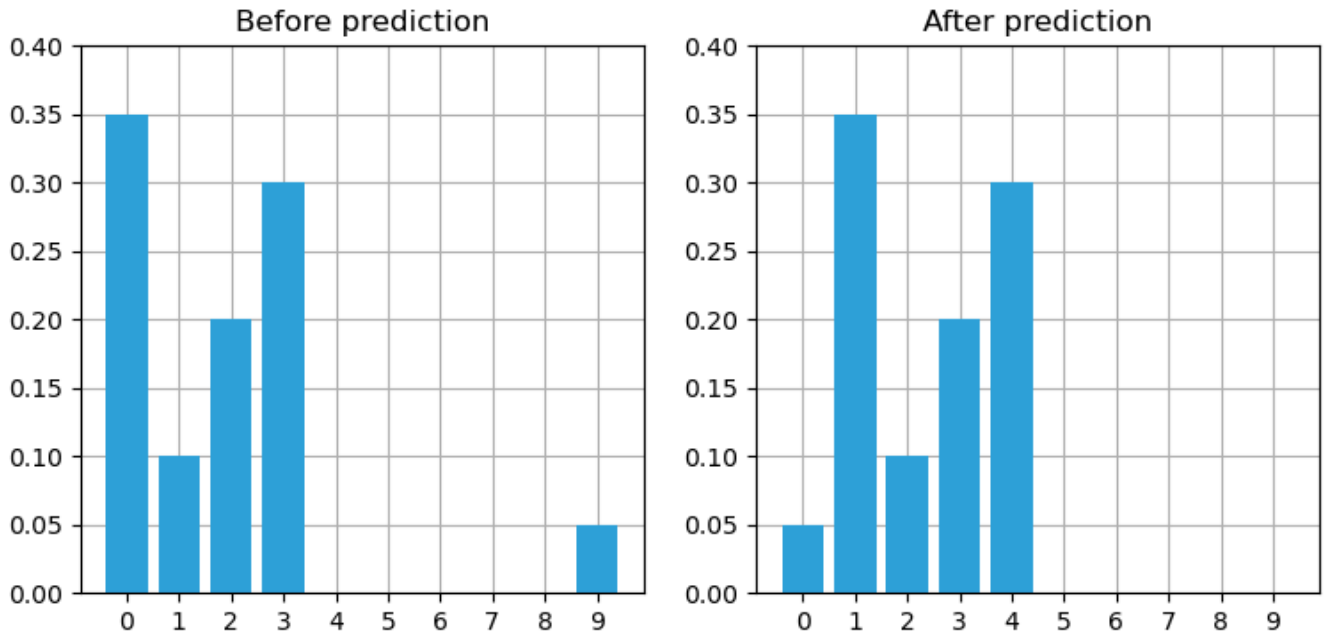
回想一下，当我们把各种测量数据和运动信息都考虑进来时，我们能够多快找到精确的解决方案。不过，那是在一个拥有完美传感器的理想化世界中才有可能实现的。那么，如果使用有噪声的传感器，我们还能找到精确的解决方案吗？

不幸的是，答案是否定的。即便传感器的读数与极其复杂的走廊布局完全吻合，我们也无法 100%确定狗的确切位置——毕竟，所有传感器的读数都可能出错。当然，在正常情况下，大多数传感器的读数都是正确的，我们可以有接近 100%的把握来判断狗的位置，但永远无法做到 100%确定。这听起来可能很复杂，不过，我们还是来编写相关的数学公式吧。

首先，我们先来处理这个简单的情况：假设运动传感器非常精确，它报告说狗向右移动了一个单位距离。那么，我们应该如何修改 `belief` 数组呢？

希望经过一番思考后，大家能明白：我们应该把所有的数值都向右移动一个位置。如果我们之前认为西蒙位于第 3 个位置的概率是 50%，那么在他向右移动一个位置后，我们应认为他位于第 4 个位置的概率也是 50%。由于走廊是环形的，因此我们需要使用取模运算来进行这种移动。

```
def perfect_predict(belief, move):
    """ move the position by `move` spaces, where positive is
    to the right, and negative is to the left
    """
    n = len(belief)
    result = np.zeros(n)
    for i in range(n):
        result[i] = belief[(i-move) % n]
    return result

belief = np.array([.35, .1, .2, .3, 0, 0, 0, 0, 0, .05])
plt.subplot(121)
book_plots.bar_plot(belief, title='Before prediction', ylim=(0, .4))

belief = perfect_predict(belief, 1)
plt.subplot(122)
book_plots.bar_plot(belief, title='After prediction', ylim=(0, .4))
```



我们可以看到，我们已经正确地将所有数值向右移动了一位。数值的移动是循环进行的，即从数组的末尾移动到开头。

下一个动画会展示这一过程，这样你就能亲眼看到其运作方式了。可以使用滑块来控制时间的推进或回溯。该模拟展示了西蒙在走廊里不断行走的情景。由于目前还没有加入新的测量数据，因此概率分布的形状不会发生变化，只有位置会改变而已。

```
from ipywidgets import interact, IntSlider

belief = np.array([.35, .1, .2, .3, 0, 0, 0, 0, 0, .05])
perfect_beliefs = []

for _ in range(20):
    # Simon takes one step to the right
    belief = perfect_predict(belief, 1)
    perfect_beliefs.append(belief)

def simulate(time_step):
    book_plots.bar_plot(perfect_beliefs[time_step], ylim=(0, .4))
    plt.show()

interact(simulate, time_step=IntSlider(value=0, max=len(perfect_beliefs)-1));
```

```
interactive(children=(IntSlider(value=0, description='time_step', max=19), Output()),
_dom_classes=('widget-in…
```

术语回顾

让我们暂停一下，回顾一下相关术语。我在上一章中已经介绍了这些术语，但现在再花点时间来巩固一下大家的理解吧。

我们所试图建模或处理的，其实就是这个“系统”。在这里，这个“系统”指的就是我们的狗。而“状态”则指的是狗当前的形态或状况。在本章中，狗的位置就代表了它的状态。我们通常无法知道狗的真实状态，因此可以说，我们的滤波器实际上只是对系统状态的估算而已。在实践中，这种估算结果通常也被视为系统的“状态”。所以，请务必根据上下文来理解相关概念。

一次预测与一次测量更新构成一个滤波周期。相关文献还常用状态演化、系统演化或系统传播等术语，描述系统状态随时间的变化，也就是时间演化 [7]。在滤波计算中，时间通常按离散步推进，例如每步 1 秒。本例的系统状态是狗的位置，状态演化模型描述经过一个时间步之后的位置。

我们用过程模型来描述系统的行为。在我们的模型中，狗在每个时间步都会移动一个或多个位置。不过，这种模型并不能准确反映狗的真实行为。模型中的误差被称为系统误差或过程误差。

这一预测就构成了我们的新“先验信息”。时间不断向前推进，而我们在做出预测时，并没有那些测量数据作为参考。

我们来看一个例子。目前，那只狗的位置是 17 米处。我们的时间间隔为 2 秒，而那只狗的移动速度为 15 米/秒。那么，我们能预测出两秒后它会在哪里吗?

显然，

$$\begin{aligned}\bar{x} &= 17 + (15 * 2) \\ &= 47\end{aligned}$$

我在变量上方加横线，表示它们是先验值，也就是预测值。过程模型的方程可以写成：

$$\bar{x}_{k+1} = f_x(\bullet) + x_k$$

x_k 代表当前的位置或状态。如果狗位于 17 米处，那么 $x_k = 17$ 就是该位置的信息。

$f_x(\bullet)$ 是 x 的状态传播函数。它描述了 x_k 在一个时间步长内会发生多大的变化。在我们的例子中，该函数执行的是 $15 \cdot 2$ 这样的计算，因此我们可以将其定义为

$$f_x(v_x, t) = v_k t$$

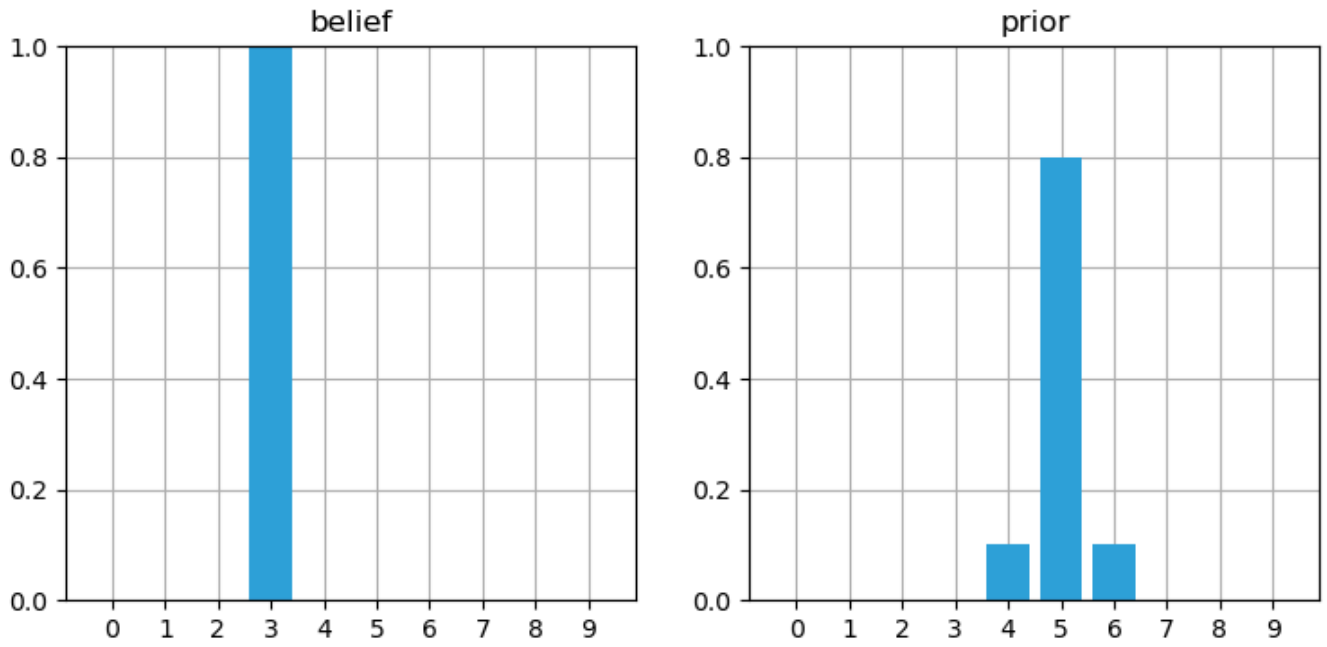
在预测中加入不确定性

perfect_predict() 假设运动测量完全准确，但实际传感器都有噪声。如果传感器报告狗移动了一个位置，实际上却移动了两个位置，或者根本没动，会怎样？这似乎很难处理，下面试着建模。

假设该传感器的移动测量结果有 80%的概率是正确的，10%的概率会向右偏移一个单位，10%的概率会向左偏移一个单位。也就是说，如果测量结果显示为 4（即向右移动 4 个单位），那么狗实际上有 80%的概率真的向右移动了 4 个单位，10%的概率向右移动了 3 个单位，10%的概率向右移动了 5 个单位。

现在，数组中的每个结果都需要包含三种不同情况下的概率。例如，以数字 2 所代表的移动情况来说：如果我们确定狗是从位置 3 开始移动的，那么它位于位置 5 的概率为 80%，而位于位置 4 或 6 的概率则各为 10%。让我们试着把这一点用代码体现出来吧：

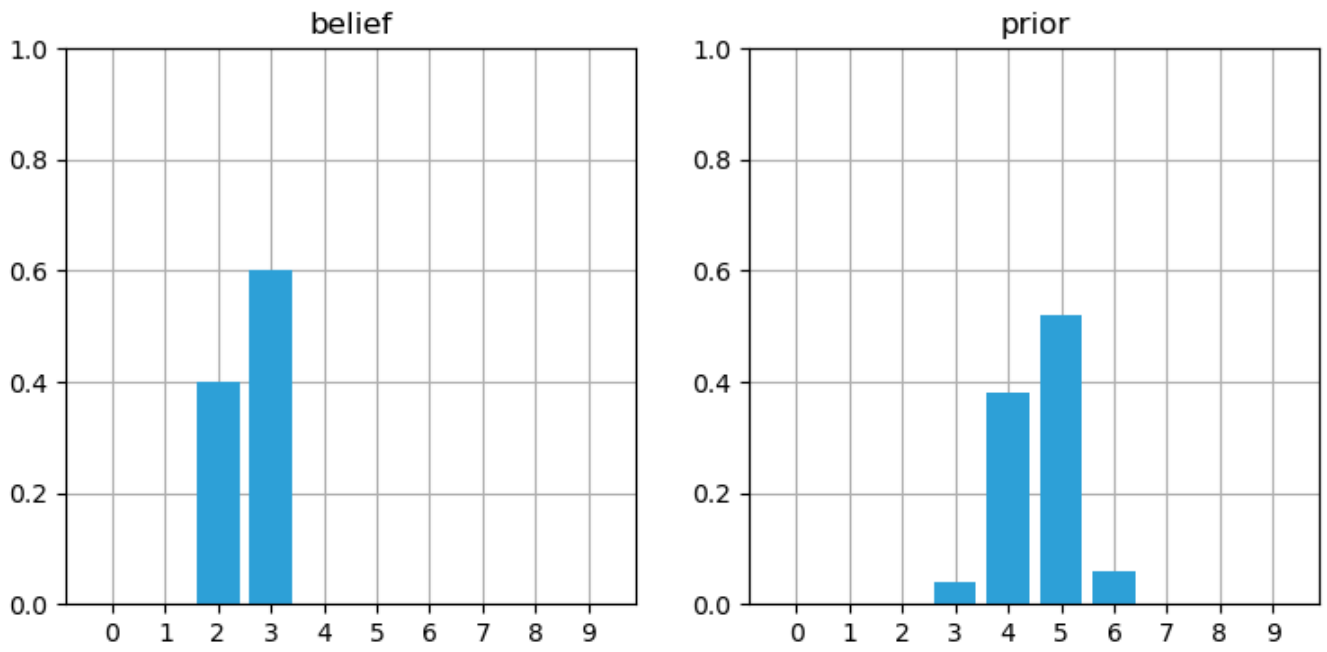
```
def predict_move(belief, move, p_under, p_correct, p_over):
    n = len(belief)
    prior = np.zeros(n)
    for i in range(n):
        prior[i] = (
            belief[(i-move) % n]   * p_correct +
            belief[(i-move-1) % n] * p_over +
            belief[(i-move+1) % n] * p_under)
    return prior

belief = [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]
prior = predict_move(belief, 2, .1, .8, .1)
book_plots.plot_belief_vs_prior(belief, prior)
```



看起来它运作正常。那么，当我们的判断并非百分之百确定时，又会怎样呢？

```
belief = [0, 0, .4, .6, 0, 0, 0, 0, 0, 0]
prior = predict_move(belief, 2, .1, .8, .1)
book_plots.plot_belief_vs_prior(belief, prior)
prior
```

```
array([0.  , 0.  , 0.  , 0.04, 0.38, 0.52, 0.06, 0.  , 0.  , 0.  ])
```



这里的结果较复杂，但仍可以心算。0.04 来自信念值为 0.4 的那一部分少移动一个位置的可能性。0.38 则由两部分组成：移动两个位置的概率为 80%，对应 0.4×0.8；少移动一个位置的概率为 10%，对应 0.6×0.1。多移动一个位置在这里没有贡献，因为那样 0.4 和 0.6 对应的位置都会越过当前讨论的位置。强烈建议多算几个例子，直到完全理解，因为后续大量内容都依赖这一步。

查看预测后的分布，也许会感到失望：最初两个位置的概率分别为 0.4 和 0.6，经过运动预测后，概率峰值降低，分布扩散到更多位置。

在本例的环形状态空间和运动转移模型下，只进行带有运动不确定性的预测而不加入新观测，会让位置分布逐步扩散。如果反复预测，最终会接近在 belief 数组上均匀分布，失去最初的定位信息。这一结果依赖本例的模型，并非任意系统都如此。下面进行 100 次迭代；在交互式笔记本中，可以用滑块改变迭代次数。

```
belief = np.array([1.0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
predict_beliefs = []

for i in range(100):
    belief = predict_move(belief, 1, .1, .8, .1)
    predict_beliefs.append(belief)

print('Final Belief:', belief)

# make interactive plot
def show_prior(step):
    book_plots.bar_plot(predict_beliefs[step-1])
    plt.title(f'Step {step}')
    plt.show()

interact(show_prior, step=IntSlider(value=1, max=len(predict_beliefs)));
```

```
Final Belief: [0.104 0.103 0.101 0.099 0.097 0.096 0.097 0.099 0.101 0.103]
```

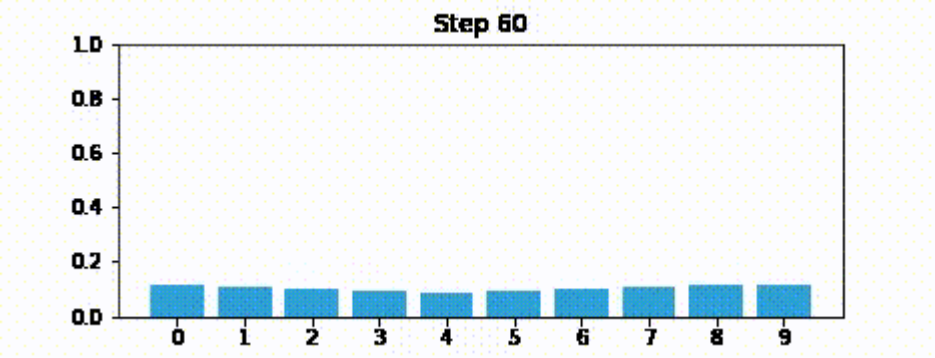
```
interactive(children=(IntSlider(value=1, description='step'), Output()), _dom_classes=('widget-
interact',))
```

```
print('Final Belief:', belief)
```

```
Final Belief: [0.104 0.103 0.101 0.099 0.097 0.096 0.097 0.099 0.101 0.103]
```

经过 100 次迭代后，我们几乎丢失了所有信息，尽管我们确信自己是从位置 0 开始迭代的。你可以试着改变迭代次数，看看不同迭代次数会对结果产生什么影响。例如，经过 100 次迭代后，还剩下少量信息；经过 50 次迭代后，还能保留较多信息；但到了 200 次迭代后，几乎所有信息都消失了。

另外，如果你是在线查看此内容的话，这里还有该结果的动画演示。



在本书的剩余部分中，我不会再单独生成这些动画了。关于如何在网上免费运行本书中的代码，或如何在电脑上安装 IPython，请参阅前言中的说明。这样，你就可以运行所有的代码片段并查看相应的动画效果了。请务必亲自尝试运行这些代码，而不仅仅是被动地阅读文字说明。

通过卷积进行泛化处理

我们假设，移动误差最多为一个位置单位。但实际上，误差也可能为两个、三个或更多位置单位。作为程序员，我们总是希望让代码具有通用性，从而能够适用于所有情况。

这可以用卷积自然地解决。这里把位置概率分布与运动误差分布进行卷积。前面的 predict_move() 实际上已经实现了卷积，只是尚未使用这个名称。对于因果函数，连续卷积可以写成下面积分限为 0 到 t 的形式。一般连续卷积在整个实数轴上积分，而本章代码采用的是有限环形状态空间上的离散循环卷积。

$$(f * g)(t) = \int_0^t f(\tau)\, g(t-\tau)\, \mathrm{d}\tau$$

其中，$f * g$ 表示将 f 与 g 进行卷积运算。这并不意味着相乘。

积分是用于连续函数的，但我们这里处理的是离散函数。因此，我们用求和来代替积分，同时用方括号来代替圆括号。

$$(f*g)[t] = \sum_{\tau=0}^{t} f[\tau]\, g[t-\tau]$$

比较结果显示， `predict_move()` 正在计算这个公式——它实际上是在计算一系列乘法运算的结果之和。

可汗学院[4]对卷积的概念有很好的讲解，而维基百科上也有一些关于卷积的精彩动画演示[5]。不过，其基本原理其实很容易理解。所谓卷积，就是将一个被称为“核”的数组在另一个数组上滑动，同时将当前位置周围的数值与第二个数组中的相应数值相乘。在上面的例子中，我们将 0.8 设为移动到正确位置的概率，0.1 设为移动位置偏小的概率，0.1 则设为移动位置偏大的概率。我们用数组 `[0.1, 0.8, 0.1]` 来表示这个“核”。我们所需要做的，就是编写一个循环，遍历数组中的每个元素，将其与“核”中的数值相乘，然后把所有结果相加。为了强调“信念”其实是一种概率分布，我将其命名为 `pdf` 。

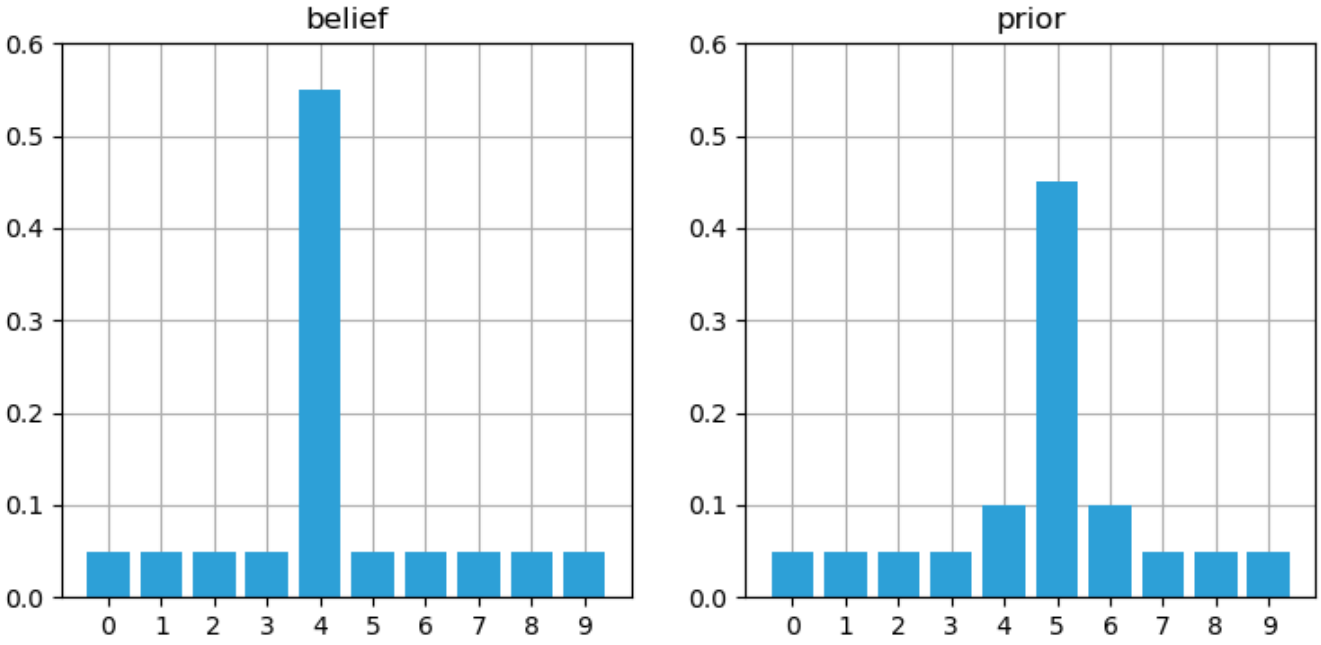
```
def predict_move_convolution(pdf, offset, kernel):
    N = len(pdf)
    kN = len(kernel)
    width = int((kN - 1) / 2)

    prior = np.zeros(N)
    for i in range(N):
        for k in range (kN):
            index = (i + (width-k) - offset) % N
            prior[i] += pdf[index] * kernel[k]
    return prior
```

上面的代码便于理解，但运行较慢。SciPy 在 ndimage 模块中提供 convolve()。卷积前先用 np.roll() 把概率数组 pdf 循环平移 offset 个位置，整个移动预测就可写成一行：

```
convolve(np.roll(pdf, offset), kernel, mode='wrap')
```

FilterPy 通过 discrete_bayes 模块中的 predict() 函数实现这一功能。

```
from filterpy.discrete_bayes import predict

belief = [.05, .05, .05, .05, .55, .05, .05, .05, .05, .05]
prior = predict(belief, offset=1, kernel=[.1, .8, .1])
book_plots.plot_belief_vs_prior(belief, prior, ylim=(0,0.6))
```



除了中间的元素外，其他所有元素的值都保持不变。第 4 位和第 6 位的数值应该分别是……

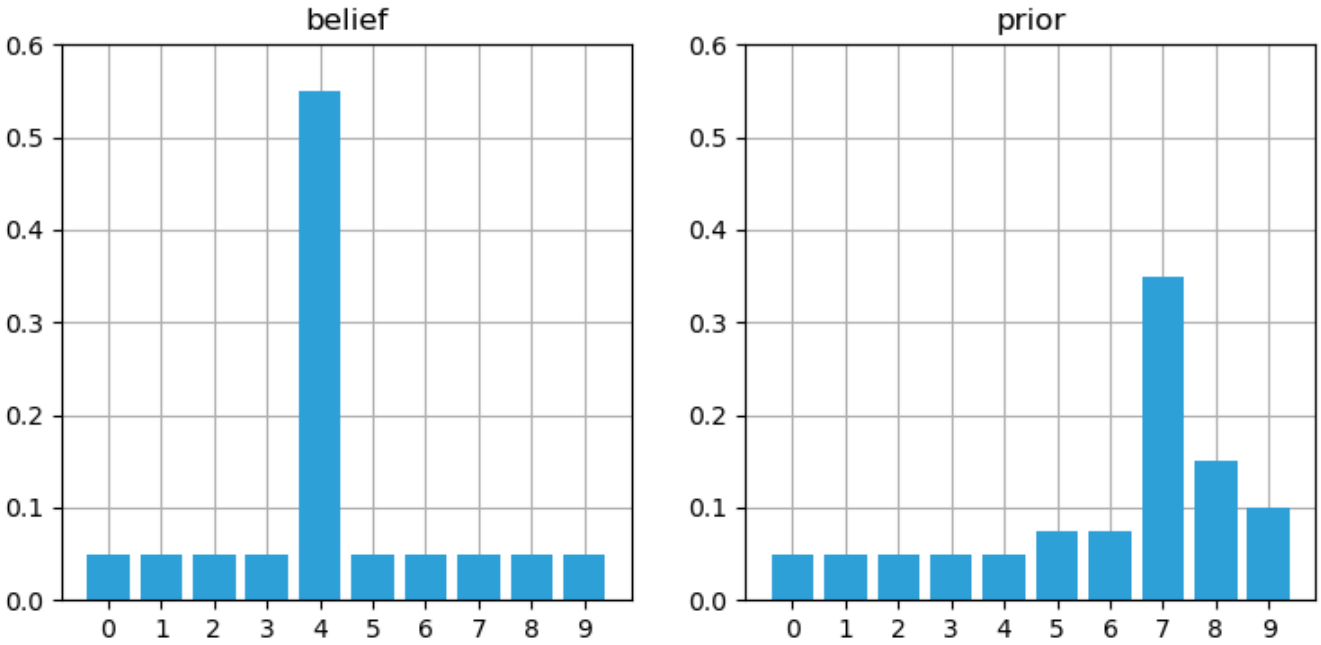
$$(0.1 \times 0.05) + (0.8 \times 0.05) + (0.1 \times 0.55) = 0.1$$

第 5 个位置应该填入……

$$(0.1 \times 0.05) + (0.8 \times 0.55) + (0.1 \times 0.05) = 0.45$$

让我们确保：当移动距离大于一时，或者使用非对称的核函数时，该算法能够正确地调整各元素的排列位置。

```
prior = predict(belief, offset=3, kernel=[.05, .05, .6, .2, .1])
book_plots.plot_belief_vs_prior(belief, prior, ylim=(0,0.6))
```



该位置的调整是正确的，调整了 3 个单位。同时，我们更重视“超调”情况的可能性，而非“欠调”情况。因此，这个结果看起来是正确的。

务必理解正在做什么：先预测狗会怎样移动，再把当前位置的概率分布与运动概率分布进行卷积，得到下一时刻的先验分布。

如果我们不使用概率的话，那就用我之前给出的那个公式吧：

$$\bar{x}_{k+1} = x_k + f_x(\bullet)$$

先验就是对狗下一时刻位置的预测。在确定性的例子中，用当前位置加上移动距离：狗原来在 10 米处，向前移动 5 米，就到了 15 米处。这里我们用概率分布描述这个过程，因此方程变为：

$$\bar{\mathbf{x}}_{k+1} = \mathbf{x}_k * f_{\mathbf{x}}(\bullet)$$

把当前位置分布与狗的移动距离分布进行卷积，就得到预测位置的分布。概念与确定性位置相加相同，但概率计算采用卷积，而不是逐元素相乘。$\mathbf{x}$ 表示一组数值，即描述位置概率分布的数组。

将测量数据与运动状态更新信息相结合

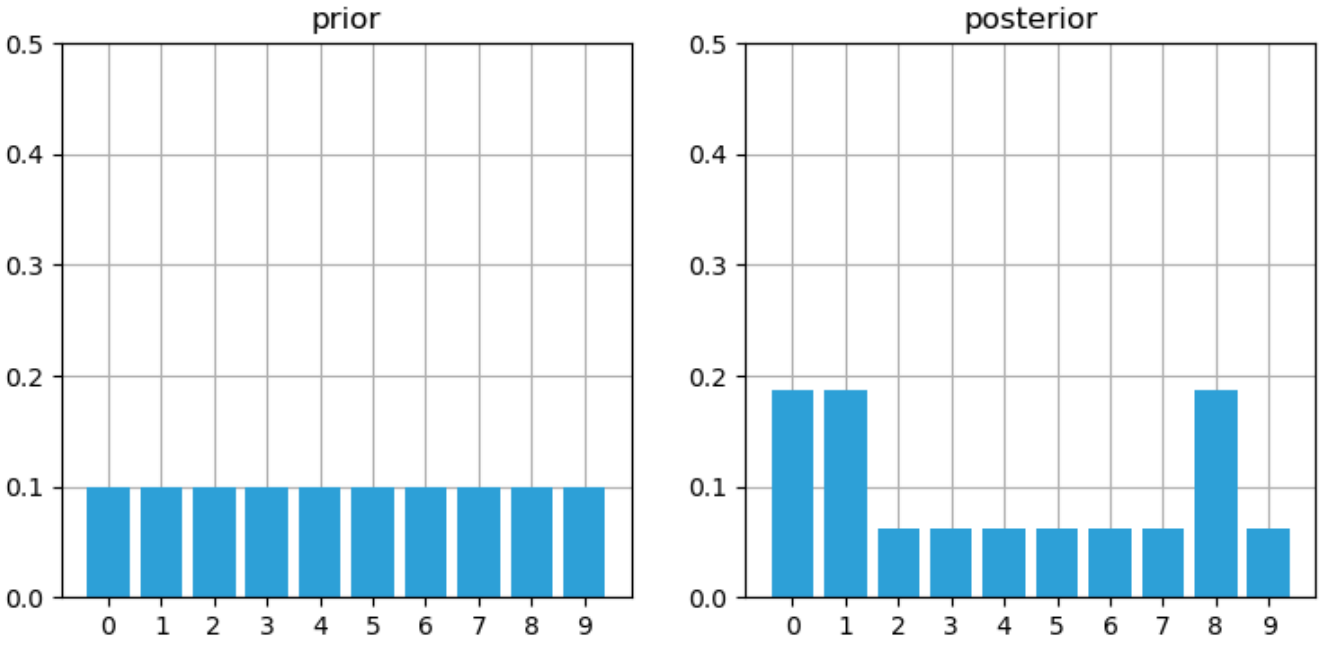
在预测过程中，信息可能会丢失，这似乎意味着我们的系统会很快变得一无所知。但实际上，每次预测之后都会进行更新，我们将新的测量结果纳入到估算中。这样的更新有助于提升我们的认知水平。更新后的结果会被用于下一次预测。虽然预测会降低我们的确定性，但这一结果又会作为输入用于下一次更新，从而再次提升确定性。

凭直觉想一想：最初狗坐着不动，每次预测都认为它仍在原位，滤波器很快得到准确的位置估计。随后厨房里的微波炉启动，狗飞奔过去。你并不知道这一变化，下一次仍预测它停在原位；测量却呈现另一种情况。随着不断纳入测量，位置的信念分布会沿走廊向厨房方向延展。每经过一个周期，你对它仍然静止的信念就减弱，对它正在快速奔向厨房的信念则增强。

直觉就是这么告诉我们的。那么，数学又告诉我们什么呢?

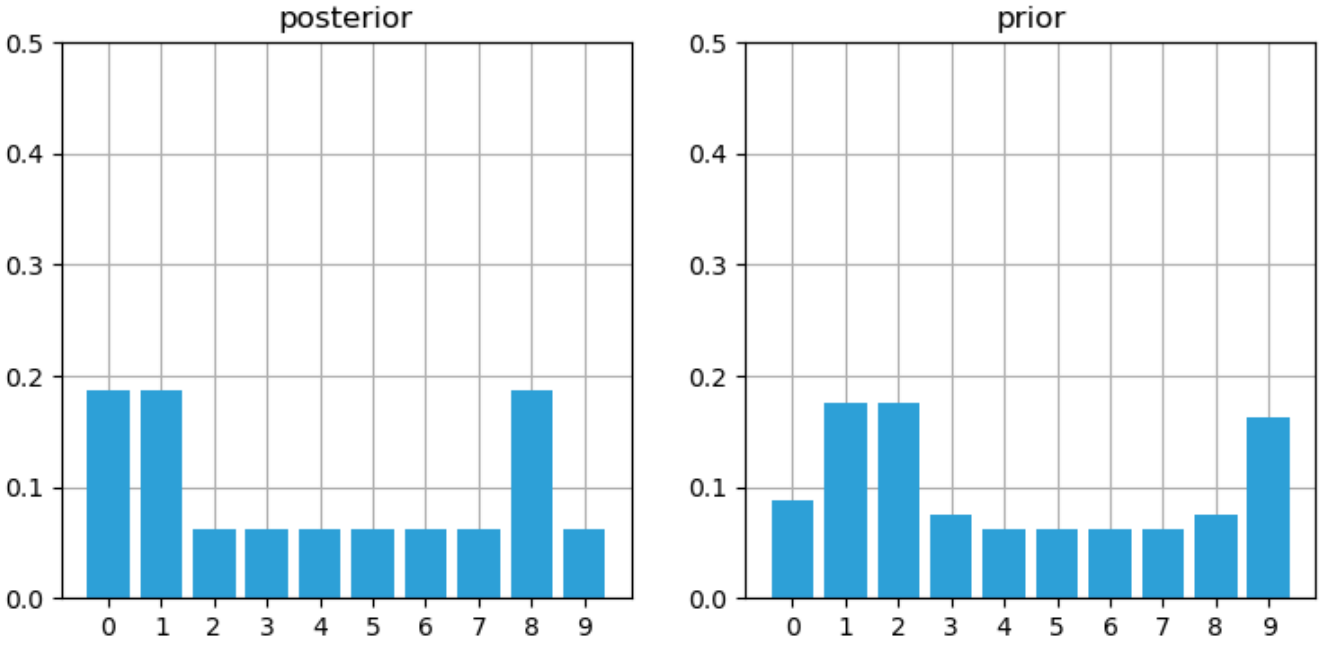
我们已经完成了更新和预测的相关程序编写。现在只需将其中一个步骤的输出结果作为输入传递给另一个步骤即可。这样，我们就成功实现了狗的追踪功能！！！让我们看看它的实际表现如何吧。我们会假设狗从位置 0 开始移动，每个时间单位都向右移动一个单位。与现实世界的情况类似，一开始我们不会知道狗的具体位置，因此会给予所有位置相同的概率。

```
from filterpy.discrete_bayes import update

hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])
prior = np.array([.1] * 10)
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



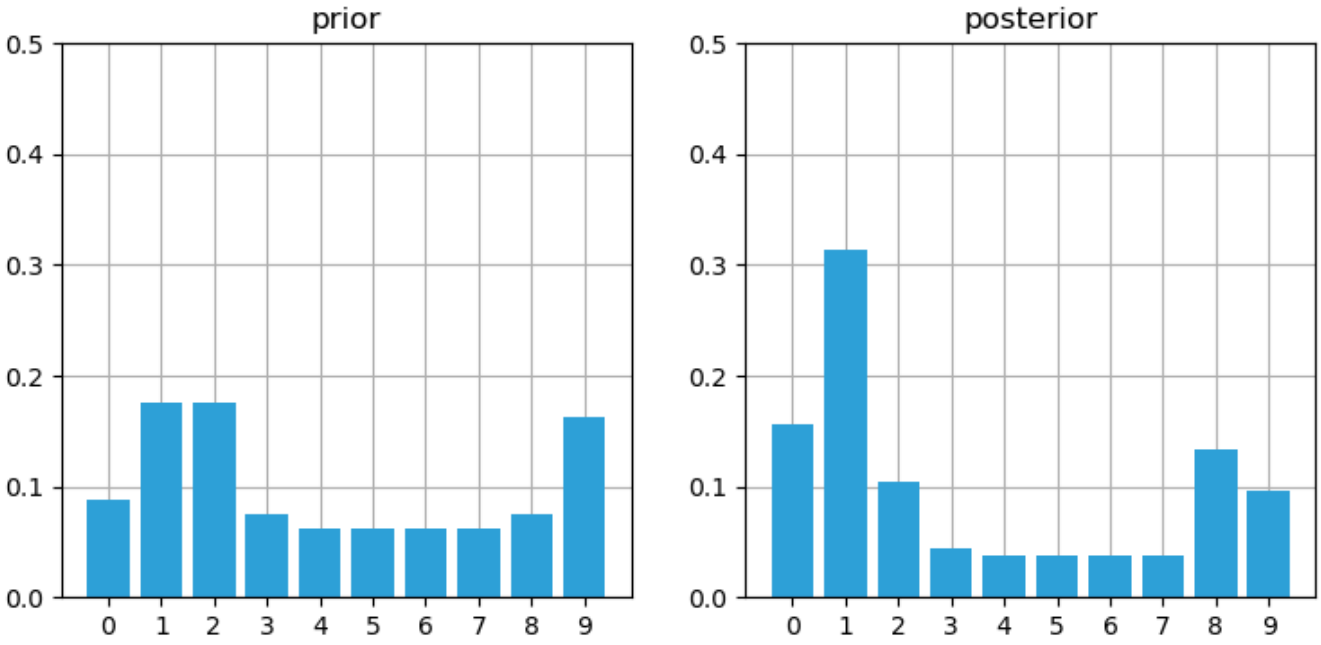
在第一次更新之后，我们给每个门的位置分配了较高的概率，而给每个墙的位置分配了较低的概率。

```
kernel = (.1, .8, .1)
prior = predict(posterior, 1, kernel)
book_plots.plot_prior_vs_posterior(prior, posterior, True, ylim=(0,.5))
```



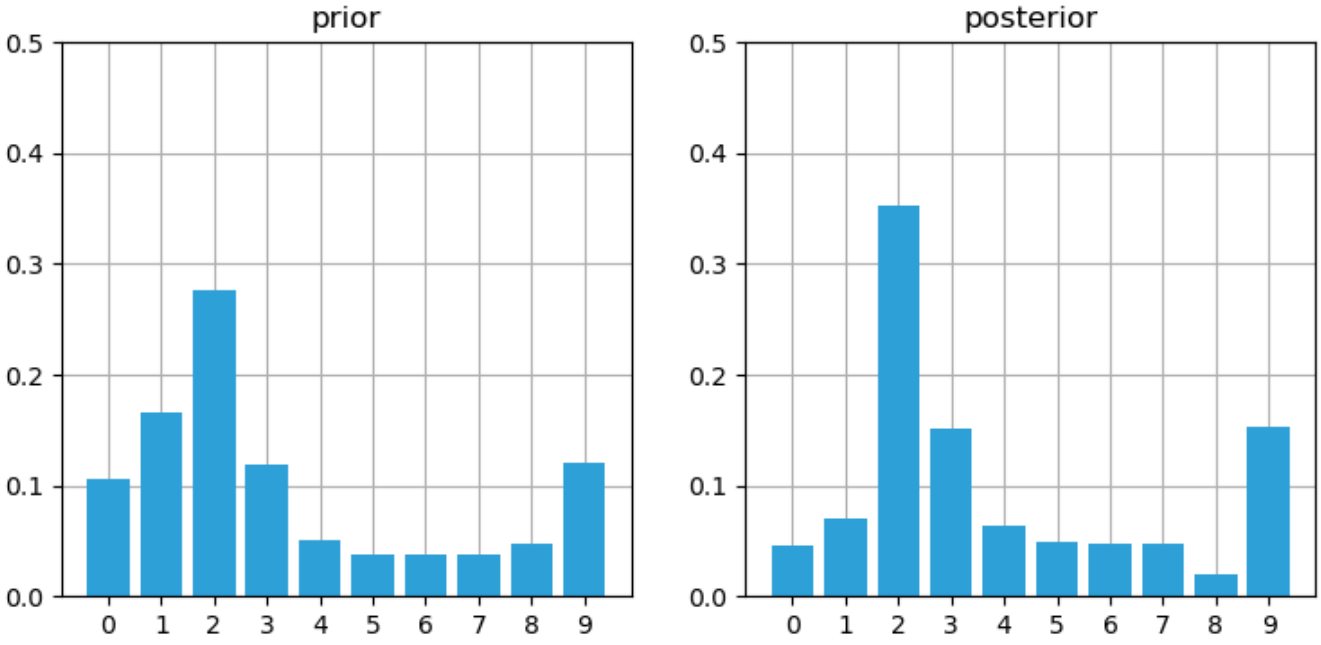
在预测步骤中，这些概率值向右移动了些，分布也变得稍微模糊了一些。现在，让我们看看在下一个处理步骤中会发生什么。

```
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



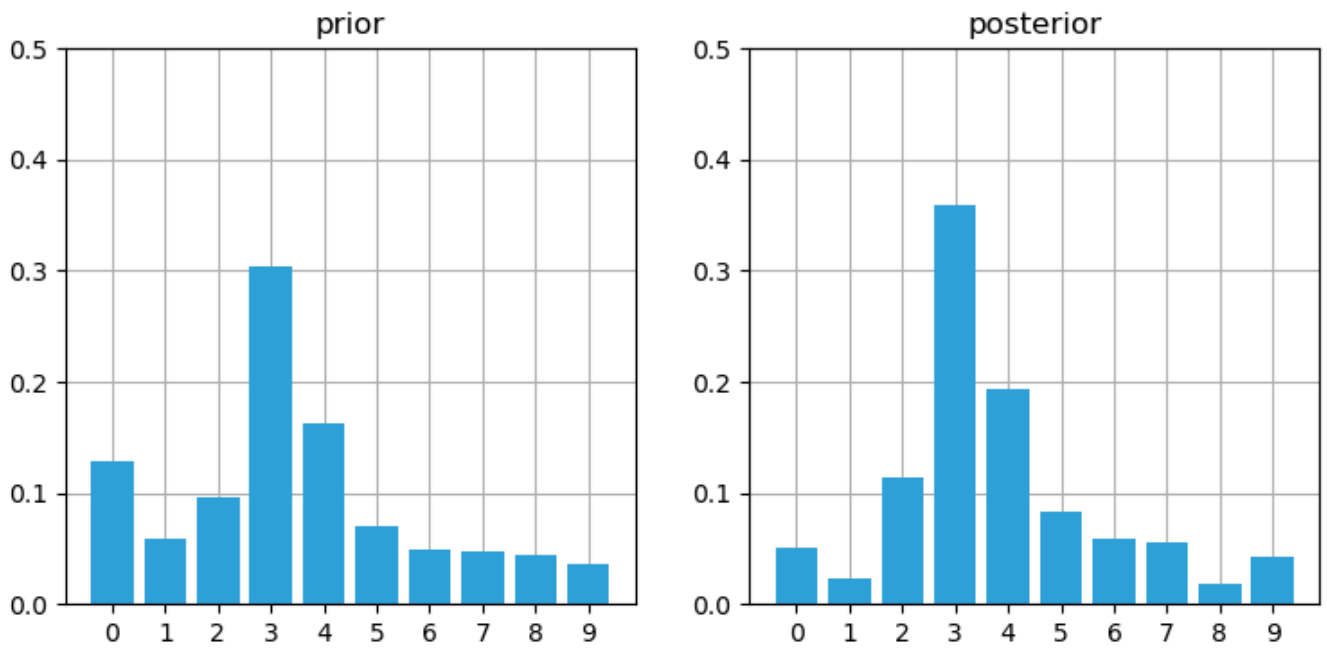
请注意，位置 1 处有个较高的条形图。这种情况对应于从位置 0 开始、检测到一扇门后向右移动 1 个单位、然后再检测到另一扇门的正确流程。没有其他位置能产生与这一系列观察结果相符的结果。现在，我们将进行一次更新，然后再检测墙壁的位置。

```
prior = predict(posterior, 1, kernel)
likelihood = lh_hallway(hallway, z=0, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



真是令人兴奋！在位置 2 上，有一个非常突出的条形图，其数值约为 35%。这一数值是图中其他所有条形图数值的两倍多。与上一次的统计结果相比，这一数值还高出约 4%；上一次统计中，数值最高的条形图数值约为 31%。让我们再观察一个周期吧。

```
prior = predict(posterior, 1, kernel)
likelihood = lh_hallway(hallway, z=0, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



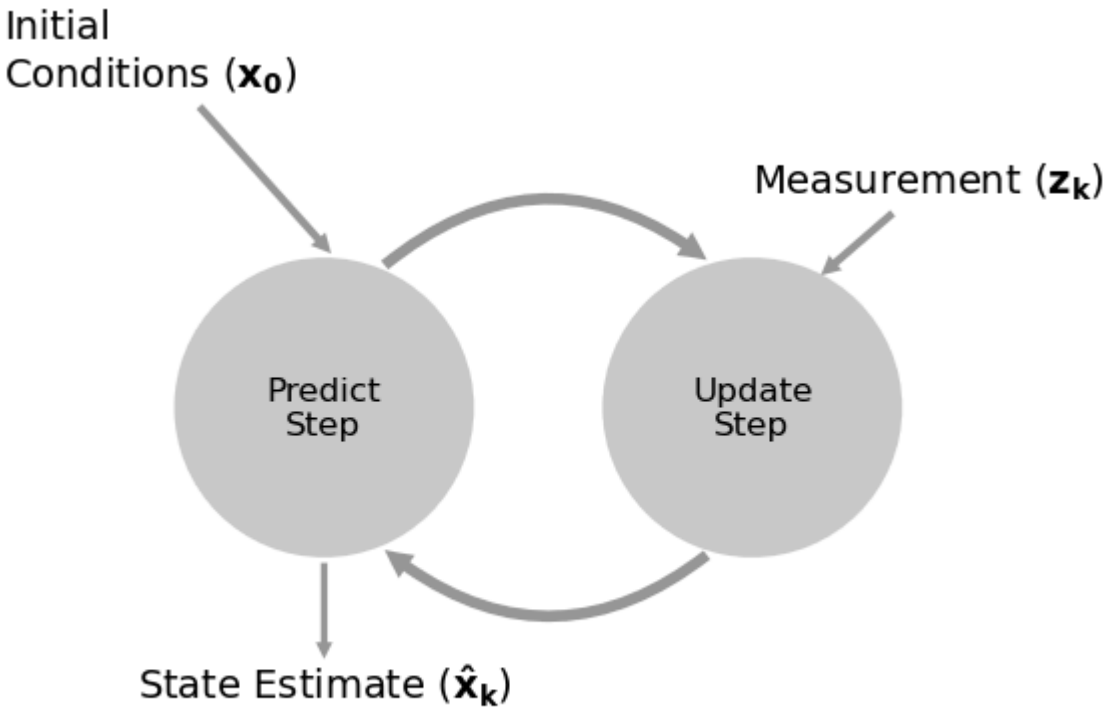
我忽略了一个重要问题。之前，我以为在预测步骤中使用了运动传感器来获取数据；后来，在讨论狗和微波炉的问题时，我又以为你并不知道狗突然开始奔跑这一情况。我提到，随着时间的推移，你会越来越认为狗正在奔跑，但我没有给出相关的代码实现。简而言之，如果我们无法直接测量某个过程，那要如何检测或估算该过程中的变化呢?

目前，我打算先忽略这个问题。在后面的章节中，我们会学习与这种估算方法相关的数学原理。不过，目前来说，学会这个算法就已经足够了。虽然解决这个问题非常重要，但我们目前还缺乏必要的数学工具来处理它。因此，在本章的剩余部分，我们将假设自己拥有能够感知运动的传感器，从而暂时忽略这个问题。

离散贝叶斯算法

该图表展示了该算法的运作方式：

```
book_plots.predict_update_chart()
```



这个滤波器同样采用预测－更新结构，从这一点看与 g-h 滤波器相似。g-h 滤波器用 g 和 h 控制预测与测量的融合，而离散贝叶斯滤波器直接传播和更新整个概率分布。这里的相似性主要体现在算法结构上；任意多峰的离散贝叶斯分布，一般不能精确化成只用两个标量增益表示的 g-h 滤波器。

滤波器方程如下：

$$\begin{aligned}
\bar{\mathbf{x}} &= \mathbf{x} * f_{\mathbf{x}}(\bullet) && \textbf{Predict Step} \\
\mathbf{x} &= \|\mathcal{L} \cdot \bar{\mathbf{x}}\| && \textbf{Update Step}
\end{aligned}$$

$\mathcal{L}$ 通常表示似然函数，因此这里采用这一记法。$\|\|\|$ 表示归一化操作：将似然与先验的逐元素乘积除以各分量之和，从而保证 $\boldsymbol{x}$ 是总和为 1 的概率分布。归一化与计算向量范数是不同的操作。

我们可以用伪代码来表示这一点。

初始化

1. 初始化对状态的信念。

预测

1. 根据系统行为预测下一时间步的状态。
2. 调整状态信念，计入预测的不确定性。

更新

1. 获取测量值及其精度信息。
2. 计算各候选状态对应的测量似然。
3. 利用似然更新对状态的信念。

在讲解卡尔曼滤波器时，我们会使用完全相同的算法；只不过在具体的计算细节上会有所不同而已。

这类算法有时称为预测—校正算法：先作预测，再根据新测量校正。

下面把这一过程做成动画。先编写执行滤波及绘制每一步结果的函数。图中黑色表示门的位置，橙色表示先验分布，蓝色表示后验分布，粗竖线表示 Simon 的真实位置。这条线不是滤波器输出；之所以知道真值，只是因为这里在仿真他的运动。

```
def discrete_bayes_sim(prior, kernel, measurements, z_prob, hallway):
    posterior = np.array([.1]*10)
    priors, posteriors = [], []
    for i, z in enumerate(measurements):
        prior = predict(posterior, 1, kernel)
        priors.append(prior)

        likelihood = lh_hallway(hallway, z, z_prob)
        posterior = update(likelihood, prior)
        posteriors.append(posterior)
    return priors, posteriors


def plot_posterior(hallway, posteriors, i):
    plt.title('Posterior')
    book_plots.bar_plot(hallway, c='k')
    book_plots.bar_plot(posteriors[i], ylim=(0, 1.0))
    plt.axvline(i % len(hallway), lw=5)
    plt.show()

def plot_prior(hallway, priors, i):
    plt.title('Prior')
    book_plots.bar_plot(hallway, c='k')
    book_plots.bar_plot(priors[i], ylim=(0, 1.0), c='#ff8015')
    plt.axvline(i % len(hallway), lw=5)
    plt.show()

def animate_discrete_bayes(hallway, priors, posteriors):
    def animate(step):
        step -= 1
        i = step // 2
        if step % 2 == 0:
            plot_prior(hallway, priors, i)
        else:
            plot_posterior(hallway, posteriors, i)

    return animate
```

让我们运行这个滤波器，并让它以动画形式展示出来吧。

```
# change these numbers to alter the simulation
kernel = (.1, .8, .1)
z_prob = 1.0
hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])

# measurements with no noise
zs = [hallway[i % len(hallway)] for i in range(50)]

priors, posteriors = discrete_bayes_sim(prior, kernel, zs, z_prob, hallway)
interact(animate_discrete_bayes(hallway, priors, posteriors), step=IntSlider(value=1,
max=len(zs)*2));
```

```
interactive(children=(IntSlider(value=1, description='step'), Output()), _dom_classes=('widget-
interact',))
```

现在能看到：运动预测使分布平移，并因运动不确定性而扩散；随后测量更新在同一位置坐标系中融合新信息，使分布通常更集中。用代码行 z_prob=1.0 设置了完美的门墙测量，下一节再讨论错误测量。注意，“同一位置坐标系”不表示整个后验概率分布保持不变；更新会改变各位置的概率。

另一点需要注意的是：当我们处于门的前方时，我们的定位精度会很高；而当我们在走廊的中间时，定位精度则会下降。这一点其实很容易理解。由于门的数量有限，当传感器检测到我们位于某扇门的前方时，我们就能更准确地确定自己的位置。而如果周围没有门的话，我们的定位精度就会下降。

不良传感器数据的影响

你可能会对上述结果产生怀疑，因为我始终将正确的传感器数据传递给了相关函数。不过，我们声称这段代码其实起到了滤波的作用——它应该能够滤除那些有问题的传感器测量数据。那么，它真的做到了这一点吗?

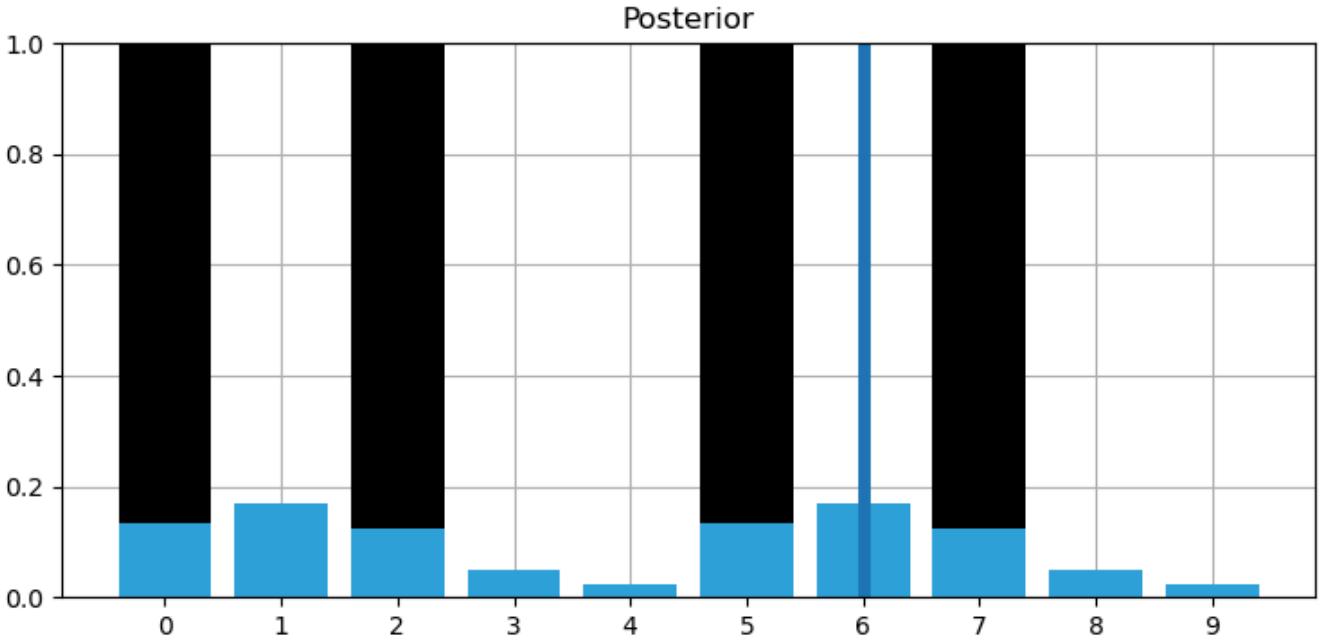
为了便于编程和可视化处理，我会将走廊的布局调整为以交替出现的门和走廊为主。同时，我会使用 6 组正确的测量数据来运行该算法。

```
hallway = np.array([1, 0, 1, 0, 0]*2)
kernel = (.1, .8, .1)
prior = np.array([.1] * 10)
zs = [1, 0, 1, 0, 0, 1]
z_prob = 0.75
priors, posteriors = discrete_bayes_sim(prior, kernel, zs, z_prob, hallway)
interact(animate_discrete_bayes(hallway, priors, posteriors), step=IntSlider(value=12,
max=len(zs)*2));
```

```
interactive(children=(IntSlider(value=12, description='step', max=12), Output()), _dom_classes=
('widget-intera…
```

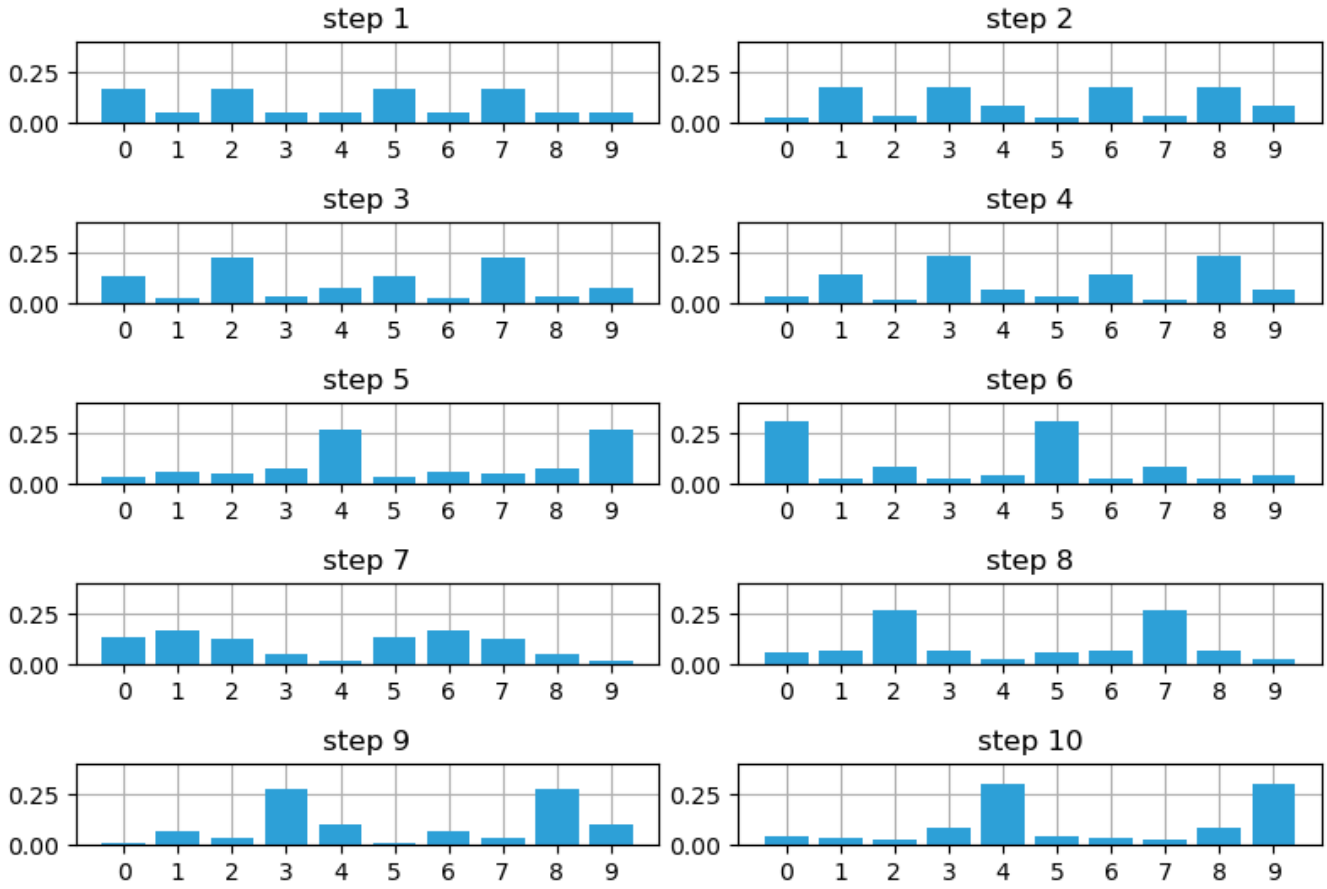
我们已经确定了那些可能从位置 0 或 5 开始的情况。因为我们观察到了这样的门和墙的排列顺序：1,0,1,0,0。现在，我故意输入了一个错误的测量值。按照正常情况，下一次的测量值应该是 0，但实际上却得到了 1。

```
measurements = [1, 0, 1, 0, 0, 1, 1]
priors, posteriors = discrete_bayes_sim(prior, kernel, measurements, z_prob, hallway);
plot_posterior(hallway, posteriors, 6)
```



这一条错误测量明显削弱了我们对位置的确定程度。现在继续输入一串正确测量。

```
with figsize(y=5.5):
    measurements = [1, 0, 1, 0, 0, 1, 1, 1, 0, 0]
    for i, m in enumerate(measurements):
        likelihood = lh_hallway(hallway, z=m, z_prob=.75)
        posterior = update(likelihood, prior)
        prior = predict(posterior, 1, kernel)
        plt.subplot(5, 2, i+1)
        book_plots.bar_plot(posterior, ylim=(0, .4), title=f'step {i+1}')
    plt.tight_layout()
```



我们迅速滤除了那些有问题的传感器数据，从而确定了狗狗最可能所在的位置。

缺点与局限性

不要因为例子简单，就低估这个滤波器。它具备完整而实用的功能，可用于真实问题。如果需要处理多峰分布、状态空间又适合离散化，这种方法就能发挥作用。

不过，这种滤波器并不常被使用，因为它存在不少局限性。克服这些局限性，正是本书后续各章节要探讨的内容。

第一个问题是数据量的处理问题。在解决狗狗定位问题时，我们只使用了一个变量 pos 来表示狗狗的位置。但实际上，大多数情况下，我们需要跟踪更大范围内的多个信息。至少，我们需要知道狗狗的 (x, y) 坐标，可能还需要知道它的速度 $(\dot{x}, \dot{y})$。我们还没有考虑多维情况，但实际上，我们可以用多维网格来存储每个位置上的概率值，而不是使用数组。每进行一次 `update()` 或 `predict()` 操作时，都需要更新网格中的所有数值。因此，对于一个包含四个变量的简单问题来说，每个时间步长所需的计算时间就是 $O(n^4)$。而实际上，复杂的系统可能需要跟踪 10 个或更多的变量，这将导致极高的计算成本。

第二个问题是滤波器离散，而现实世界连续。直方图要求把状态表示为一组离散位置。若要将一条 100 米长的走廊划分到 1 厘米的分辨率，就需要 10,000 个位置；每次更新和预测都要处理相应的概率。维度增加时，网格数量呈指数增长。例如，面积为 $100 \times 100\ \mathrm{m}^2$ 的庭院，要达到 1 厘米的分辨率，就需要 100,000,000 个网格单元。

第三个问题是后验可能有多个峰。上一例中，狗位于位置 4 或 9 的可能性都很高。这未必是缺点；后面介绍的粒子滤波也能表示多峰分布，常因这一能力而被采用。不过，若车载 GPS 告诉你：有 40% 的概率位于 D 街、30% 的概率位于 Willow 大道，驾驶决策就会遇到困难。

第四个问题是，需要测量状态的变化情况。我们必须使用运动传感器来检测狗的移动程度。虽然有办法解决这个问题，但那样会使得本章的阐述变得复杂。鉴于上述问题，我就不再详细讨论了。

也就是说，如果我遇到这种技术能够解决的小问题，我肯定会选择使用它。这种技术的实现、调试和理解都相当简单，这些都是它的优点。

跟踪与控制

我们一直在被动地跟踪那些能够自主移动的物体。不过，再考虑一个类似的问题吧。我正在对仓库进行自动化改造，希望利用机器人来收集客户订单中的所有物品。也许最简单的方法就是让机器人沿着轨道移动。我希望能够为机器人指定目的地，让它自动前往那里。但是，轨道和机器人的电机都不完美。车轮打滑以及电机性能不佳意味着，机器人很难准确到达指定的位置。此外，还有多台机器人需要同时操作，我们必须知道每台机器人的位置，以避免它们相互碰撞。

因此，我们可以添加传感器来辅助定位。也许可以在轨道上每隔几英尺就安装一个磁铁，然后使用霍尔传感器来统计经过了多少个磁铁。如果统计到 10 个磁铁，那么机器人应该位于第 10 个磁铁的位置。当然，也有可能漏掉某个磁铁或重复计数某个磁铁，因此我们必须考虑到一定程度的误差。我们可以利用上一节中的代码来追踪机器人的位置，因为磁铁的计数方式与检测门框的位置很相似。

但我们还没有完成。我们明白了永远不要浪费任何信息。只要掌握了某种信息，就应该利用它来改进自己的预测。我们还遗漏了哪些信息呢？我们知道，在每一时刻，我们都在向机器人的各个部件发送控制指令。比如，假设我们每秒向机器人发送一次移动指令：向左移动 1 个单位、向右移动 1 个单位，或者保持不动。当我发出“向左移动 1 个单位”的指令后，我预计 1 秒钟后，机器人会位于当前位置的左侧 1 个单位处。当然，这是一种简化的处理方式，因为我没有考虑加速度的因素。不过，我并不是想在这里讲解控制理论。车轮和电机本身并不完美，所以机器人实际移动的距离可能会是 0.9 个单位，也有可能是 1.2 个单位。

现在，整个解决方案已经很清晰了。我们假设狗会继续沿原来的方向移动。不过，对于我的狗来说，这种假设并不成立！机器人的行为则容易预测得多。与其基于某种假设来做出不确定的预测，不如直接将我们发给机器人的指令输入进去！换句话说，当我们调用 `predict()` 时，我们会把给机器人的指令以及描述该指令执行可能性的参数一起传递过去。

模拟列车的运行行为

先模拟一个存在运动与测量误差的列车系统：执行移动指令时，实际位移可能略有偏差，传感器也可能报告错误读数。

```
class Train(object):

    def __init__(self, track_len, kernel=[1.], sensor_accuracy=.9):
        self.track_len = track_len
        self.pos = 0
        self.kernel = kernel
        self.sensor_accuracy = sensor_accuracy
```

```
    def move(self, distance=1):
        """ move in the specified direction
        with some small chance of error"""

        self.pos += distance
        # insert random movement error according to kernel
        r = random.random()
        s = 0
        offset = -(len(self.kernel) - 1) / 2
        for k in self.kernel:
            s += k
            if r <= s:
                break
            offset += 1
        self.pos = int((self.pos + offset) % self.track_len)
        return self.pos

    def sense(self):
        pos = self.pos
         # insert random sensor error
        if random.random() > self.sensor_accuracy:
            if random.random() > 0.5:
                pos += 1
            else:
                pos -= 1
        return pos
```

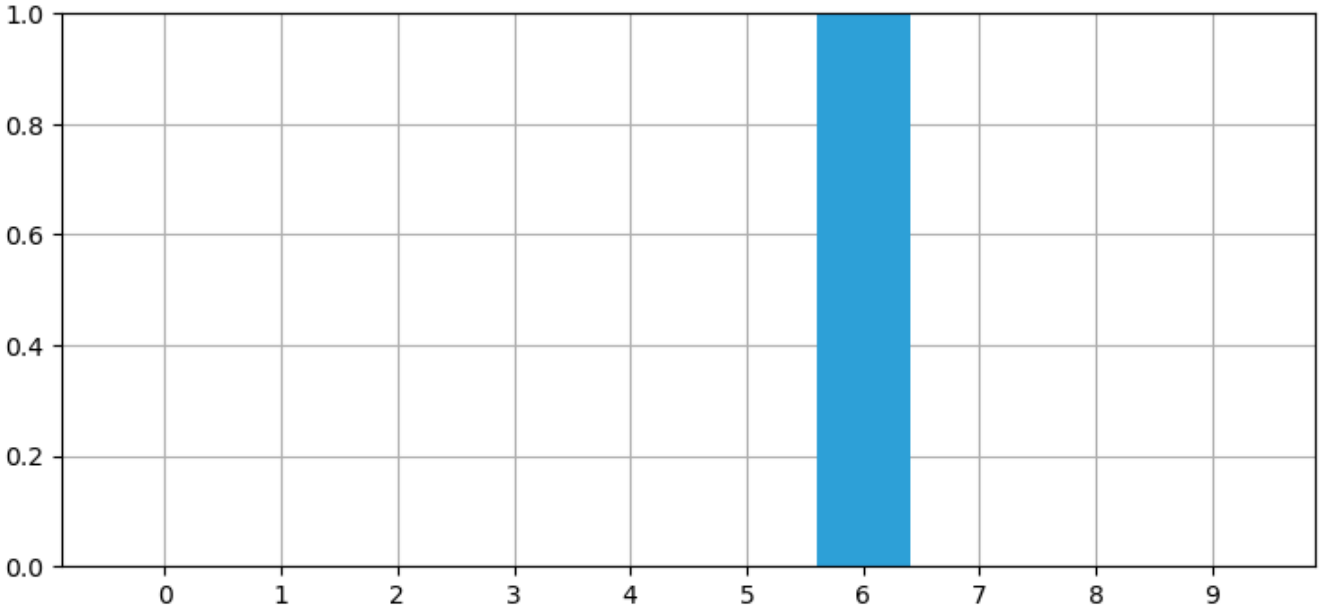
现在，我们可以开始编写这个滤波器了。我们会将其放入一个函数中，这样就可以根据不同的假设来运行该函数。我假设机器人总是从轨道的起点开始移动。该轨道被设定为 10 个单位长，但实际上，你可以将其想象成长度为 10,000 个单位的轨道，只不过磁铁图案每隔 10 个单位就会重复出现一次。将轨道长度设为 10 个单位，便于我们进行绘制和检查。

```
def train_filter(iterations, kernel, sensor_accuracy,
                 move_distance, do_print=True):
    track = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    prior = np.array([.9] + [0.01]*9)
    posterior = prior[:]
    normalize(prior)

    robot = Train(len(track), kernel, sensor_accuracy)
    for i in range(iterations):
        # move the robot and
        robot.move(distance=move_distance)

        # peform prediction
        prior = predict(posterior, move_distance, kernel)

        #  and update the filter
        m = robot.sense()
        likelihood = lh_hallway(track, m, sensor_accuracy)
        posterior = update(likelihood, prior)
        index = np.argmax(posterior)

        if do_print:
            print(f'time {i}: pos {robot.pos}, sensed {m}, at position {track[robot.pos]}')
            conf = posterior[index] * 100
            print(f'        estimated position is {index} with confidence {conf:.4f}%:')

    book_plots.bar_plot(posterior)
    if do_print:
        print()
        print('final position is', robot.pos)
        index = np.argmax(posterior)
        conf = posterior[index]*100
        print(f'Estimated position is {index} with confidence {conf:.4f}')
```

仔细阅读代码，确保理解。先在传感器读数无误差、执行运动也无误差的条件下运行；如果代码正确，应能无误差地定位机器人。输出虽然较长，若还不清楚预测—更新循环，请逐步阅读以巩固理解。

```
import random

random.seed(3)
np.set_printoptions(precision=2, suppress=True, linewidth=60)
train_filter(4, kernel=[1.], sensor_accuracy=.999,
             move_distance=4, do_print=True)
```

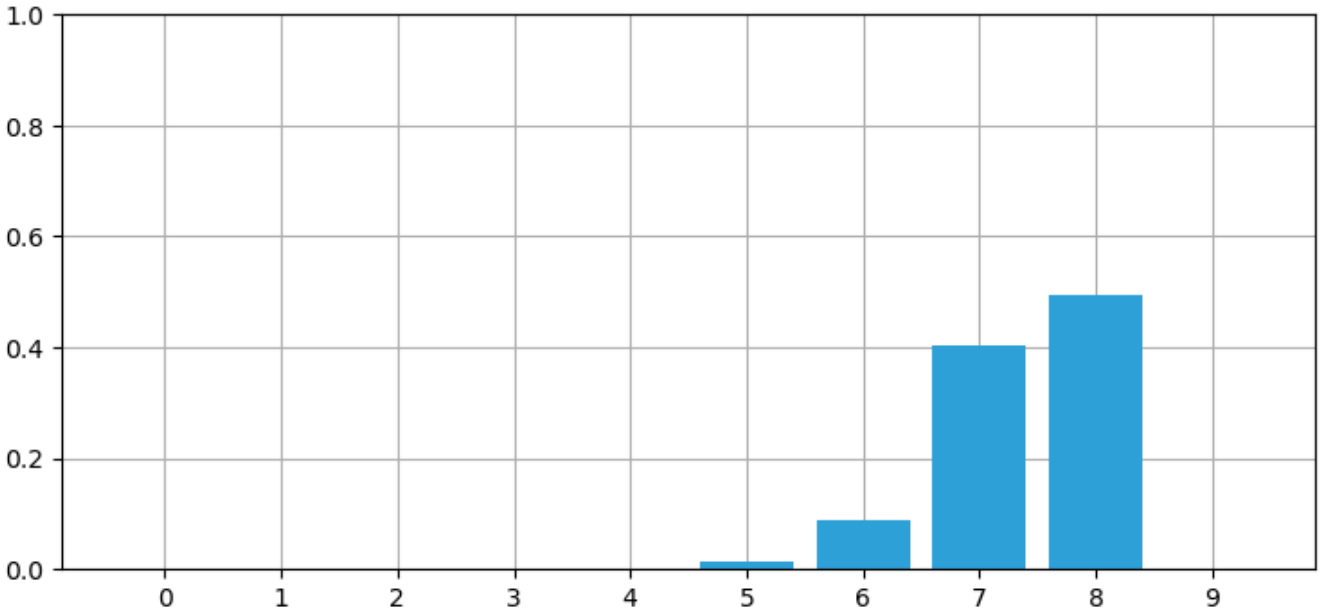
```
time 0: pos 4, sensed 4, at position 4
        estimated position is 4 with confidence 99.9900%:
time 1: pos 8, sensed 8, at position 8
        estimated position is 8 with confidence 100.0000%:
time 2: pos 2, sensed 2, at position 2
        estimated position is 2 with confidence 100.0000%:
time 3: pos 6, sensed 6, at position 6
        estimated position is 6 with confidence 100.0000%:

final position is 6
Estimated position is 6 with confidence 100.0000
```



可以看到，代码准确跟踪了机器人，因此对实现已有一定信心。再观察存在误差时的表现。

```
random.seed(5)
train_filter(4, kernel=[.1, .8, .1], sensor_accuracy=.9,
             move_distance=4, do_print=True)
```

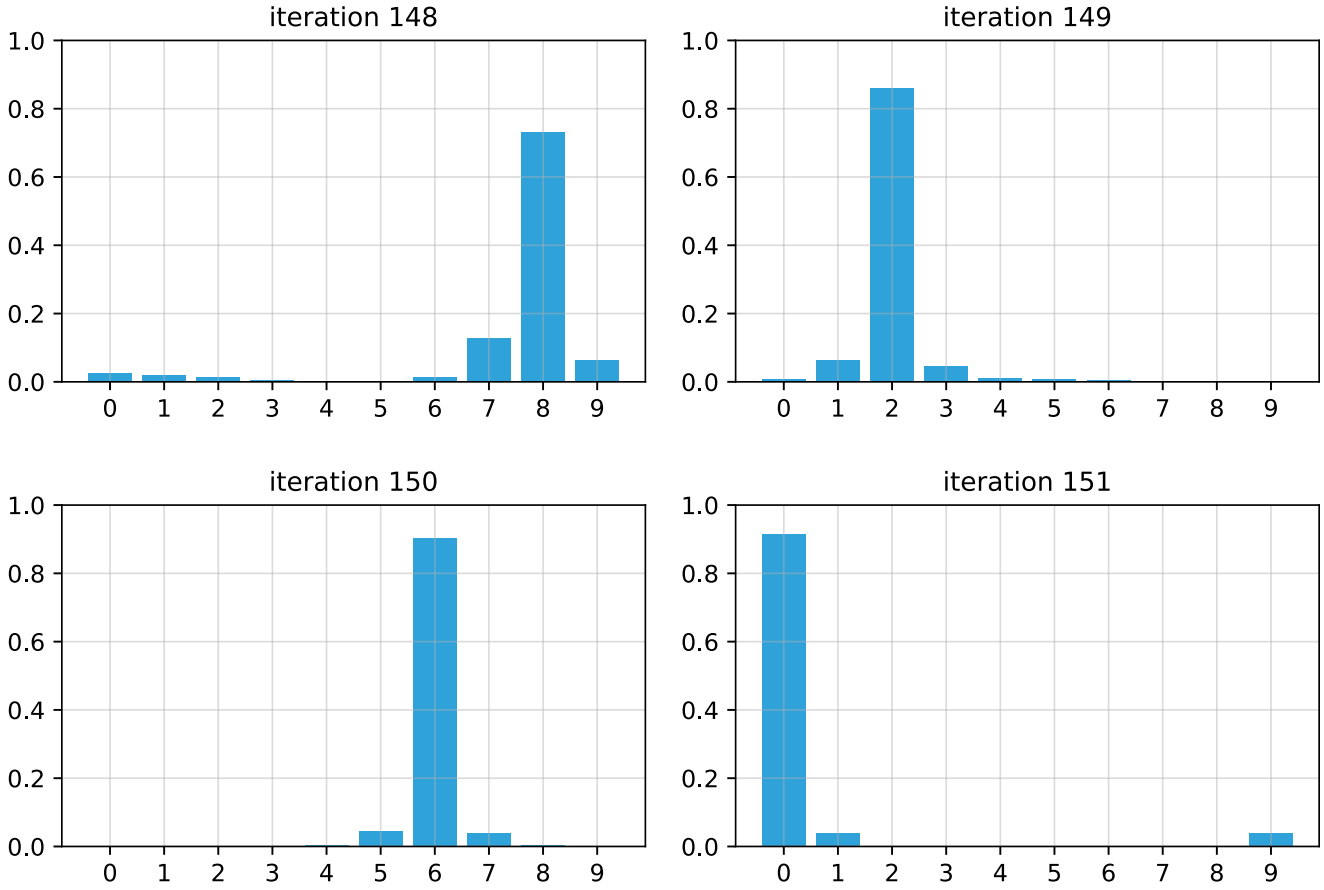
```
time 0: pos 4, sensed 4, at position 4
        estimated position is 4 with confidence 96.0390%:
time 1: pos 8, sensed 9, at position 8
        estimated position is 9 with confidence 52.1180%:
time 2: pos 3, sensed 3, at position 3
        estimated position is 3 with confidence 88.3993%:
time 3: pos 7, sensed 8, at position 7
        estimated position is 8 with confidence 49.3174%:

final position is 7
Estimated position is 8 with confidence 49.3174
```



在时间点 1 时出现了测量误差，不过我们对自己的定位仍然相当有信心。

现在，让我们进行一次长时间的模拟测试，看看该滤波器在面对错误时会有怎样的反应。

```
with figsize(y=5.5):
    for i in range (4):
        random.seed(3)
        plt.subplot(221+i)
        train_filter(148+i, kernel=[.1, .8, .1],
                     sensor_accuracy=.8,
                     move_distance=4, do_print=False)
        plt.title (f'iteration {148 + i}')
    plt.tight_layout(pad=1.5, h_pad=2.0)
```



我们可以看到，在第 149 次迭代时出现了问题，置信度有所下降。不过，经过几次迭代后，滤波器能够自我修正，重新获得对估计位置的信任。

贝叶斯定理与全概率定理

在本章中，我们完全是通过对所掌握的信息进行推理来推导出相关的数学公式。在这个过程中，我们发现了贝叶斯定理和全概率定理。

贝叶斯定理告诉我们，如何根据先前的信息来计算某个事件的概率。

我们通过这种概率计算方式来实现 `update()` 函数：

$$\mathbf{posterior} = \frac{\mathbf{likelihood} \times \mathbf{prior}}{\mathbf{normalization\,factor}}$$

我们还没有建立起用于阐述贝叶斯定理的数学体系，但这就是贝叶斯定理。本书中提到的每一个滤波机制，其实都是贝叶斯定理的具体体现。在下一章中，我们会进一步发展相关的数学理论，不过这样做反而会让人忽略这个公式所表达的简单道理。

$$\textit{updated knowledge} = \left\| \textit{likelihood of new knowledge} \times \textit{prior knowledge} \right\|$$

其中，$\|\cdot\|$ 表示对该项进行归一化处理。

我们是通过一个简单的例子来理解这个原理的：想象一只狗在走廊里行走。不过，正如我们将看到的，同样的原理也适用于各种滤波问题。在后面的每一章中，我们都会用到这个原理。

同样，predict() 通过对各种可能的前一状态及其转移概率求和，计算预测概率。这利用了统计学中的全概率定理。下一章介绍必要的数学基础后，会进一步讨论。

目前，你需要明白：贝叶斯定理其实是一种将新信息纳入现有信息中的公式。

本章小结

代码很短，效果却很好：我们实现了一种贝叶斯滤波器，学会从未知初始位置出发，利用带噪传感器逐步定位。相较实际中许多正确率高于 80% 的传感器，本章设定的读数相当不可靠，但滤波仍能快速集中到狗最可能的位置。预测传播运动不确定性，测量更新又带来新信息，两者交替形成有效估计。

这本书主要介绍卡尔曼滤波器的相关内容。虽然其中所使用的数学公式有所不同，但其逻辑原理与本章所阐述的完全一致。卡尔曼滤波器运用贝叶斯推理，结合各种测量数据和过程模型来得出估算结果。

理解本章，就具备了理解和实现卡尔曼滤波的重要基础。若有不清楚之处，请重读并动手改代码。后面的内容以这里的算法为基础。关键是：测量更新时，把先验与似然相乘并归一化；运动预测时，对位置分布进行平移与卷积。掌握这两步，再补充后面的数学知识，就能实现卡尔曼滤波。

参考文献

- [1] D. Fox、W. Burgard 和 S. Thrun。“蒙特卡洛定位：一种高效的移动机器人位置估计方法。”载于《人工智能研究杂志》，1999 年。

  http://www.cs.cmu.edu/afs/cs/project/jair/pub/volume11/fox99a-html/jair-localize.html

- [2] Dieter Fox 等人。“用于位置估计的贝叶斯滤波器”。载于《IEEE 普适计算》杂志，2003 年 9 月。

  http://swarmlab.unimaas.nl/wp-content/uploads/2012/07/fox2003bayesian.pdf

- [3] 塞巴斯蒂安·特伦。《用于机器人的人工智能》。

  https://www.udacity.com/course/cs373

- [4] 可汗学院。“卷积入门”

  https://www.khanacademy.org/math/differential-equations/laplace-transform/convolution-integral/v/introduction-to-the-convolution

- [5] 维基百科。“卷积”

  http://en.wikipedia.org/wiki/Convolution

- [6] 维基百科。“全概率定理”

  http://en.wikipedia.org/wiki/Law_of_total_probability

- [7] Wikipedia. "Time Evolution"

  https://en.wikipedia.org/wiki/Time_evolution

- [8] We need to rethink how we teach statistics from the ground up

  http://www.statslife.org.uk/opinion/2405-we-need-to-rethink-how-we-teach-statistics-from-the-ground-up

第 3 章：高斯分布

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

上一章最后讨论了离散贝叶斯滤波的一些缺点。许多跟踪问题希望用连续、单峰的分布表示状态：连续表示不再只使用固定网格，单峰表示分布只有一个主要的概率集中区域。例如，希望用纬度、经度和高度 (12.34,-95.54,2389.5) 及其不确定性描述飞机位置，而不是给出两个相距很远的候选区域。多峰问题不仅可能计算昂贵，也会给导航决策带来困难。这里的连续性是概率模型的性质，不等同于计算机使用浮点数；单峰也不等于只输出一个估计值。

我们希望用计算高效的连续、单峰分布来表达适合当前问题的概率模型。高斯分布具备这些特点。

均值、方差和标准差

你们中的大多数人应该都接触过统计学相关的内容，不过我还是想再详细讲解一下。即便你们觉得自己已经很熟悉这些内容了，也请务必仔细阅读。我这么做有两个原因：首先，我希望我们能以相同的方式理解各种术语；其次，我希望大家能建立起对统计学的直观理解，这对后面章节的学习会有很大帮助。很多人上完统计学课程后，只记得那些公式和计算方法，却不清楚自己所学内容的实际意义。

随机变量

每次掷骰子时，结果都在 1 到 6 之间。如果我们用一个公平的骰子掷一百万次，那么预计约有六分之一的次数掷出 1 点。因此，我们说得到 1 点的概率是 1/6。同样地，如果我问你下一次掷骰子得到 1 点的概率是多少，你也会回答 1/6。

这些数值及其对应的概率共同构成了一个随机变量。这里所说的“随机”，并不意味着该过程是不可预测的，而只是表示我们无法确切知道最终的结果。掷骰子的结果其实是可确定的，但我们缺乏足够的信息来计算出具体结果。我们只能以概率的方式来了解可能的结果。

再介绍一个术语：试验的所有可能结果构成样本空间。骰子的样本空间为 {1,2,3,4,5,6}，硬币的样本空间为 {正面,反面}。“空间”在数学中指具有某种结构的集合。骰子的这些结果是自然数集中的一个有限子集。

大学学生的身高是另一个随机变量例子。这里的可能取值是由生物学条件限定的上下界之间的一段实数范围。

像抛硬币或掷骰子这样的随机变量属于离散随机变量。这意味着，它们的样本空间由有限个数值或可数无限个数值构成，比如自然数。而人的身高则属于连续随机变量，因为人的身高可以取两个极限值之间的任意实数。

不要将随机变量的测量结果与其实际数值混为一谈。如果我们只能将一个人的身高精确到 0.1 米，那么我们只能记录下 0.1、0.2、0.3……2.7 这些数值，也就是 27 个离散的值。不过，一个人的身高实际上可以取这些刻度之间的实数值，因此身高属于连续型随机变量。

统计学通常用大写字母表示随机变量，且常选取字母表后半部分的字母。例如，用 $\boldsymbol{X}$ 表示掷骰子的结果，用 $\boldsymbol{Y}$ 表示一年级诗歌课学生的身高。后面用线性代数解决问题时，则遵循小写表示向量、大写表示矩阵的惯例。两种惯例可能冲突，因此要根据上下文判断，并结合定义和维数区分随机变量、标量、向量与矩阵。

概率分布

概率分布描述了随机变量在样本空间中取各个值的概率。例如，对于一个公平的六面骰子来说：

取值	概率
1	1/6
2	1/6
3	1/6
4	1/6
5	1/6
6	1/6

我们用小写字母 p 来表示这种分布：$\boldsymbol{p(x)}$。用普通的函数表示法来写，就是：

$$\boldsymbol{P(X{=}4) = p(4) = \frac{1}{6}}$$

这意味着，骰子掷出 4 的概率为 $\frac{1}{6}$。$\boldsymbol{P(X{=}x_k)}$ 实际上表示“ $\boldsymbol{X}$ 为 $\boldsymbol{x_k}$ 的概率”。请注意这种细微的符号差异：大写的 $\boldsymbol{P}$ 表示单个事件的概率，而小写的 $\boldsymbol{p}$ 则代表概率分布函数。如果不仔细注意的话，很容易搞混这些概念。有些教材会用 $\boldsymbol{Pr}$ 来代替 $\boldsymbol{P}$，以避免这种混淆。

另一个例子就是公平的硬币。它的样本空间为{正面, 反面}。由于硬币是公平的，因此出现正面的概率为 50%，出现反面的概率也为 50%。我们可以将其表示为：

$$\boldsymbol{P(X{=}H) = 0.5}$$
$$\boldsymbol{P(X{=}T) = 0.5}$$

样本空间并不唯一。对于骰子来说，一个有效的样本空间可以是{1, 2, 3, 4, 5, 6}。另一个有效的样本空间可以是{偶数, 奇数}。再比如，{所有角上有点，所有角上没有点}也是一种有效的样本空间。只要样本空间能够涵盖所有可能性，并且每个事件都只能由样本空间中的一个元素来表示，那么这个样本空间就是有效的。{偶数, 1, 3, 4, 5}不是骰子的有效样本空间，因为数字 4 既属于“偶数”，也属于“4”。

离散概率分布描述离散随机变量各个可能取值的概率；连续概率分布则描述连续随机变量落入各个取值区域的概率。离散情况下，各项概率之和为 1；连续概率密度则以积分归一化。

构成概率分布时，各个可能取值 x_i 所对应的概率必须非负，即 $p(x_i) \geq 0$。对于离散分布，所有可能取值的概率之和还必须为 1。例如，正面的概率为 70%，反面的概率就为 30%。概率非负并不限制随机变量本身的取值；随机变量完全可以取负数。

$$\sum_u P(X=u) = 1$$

对于离散分布而言，情况也是如此。

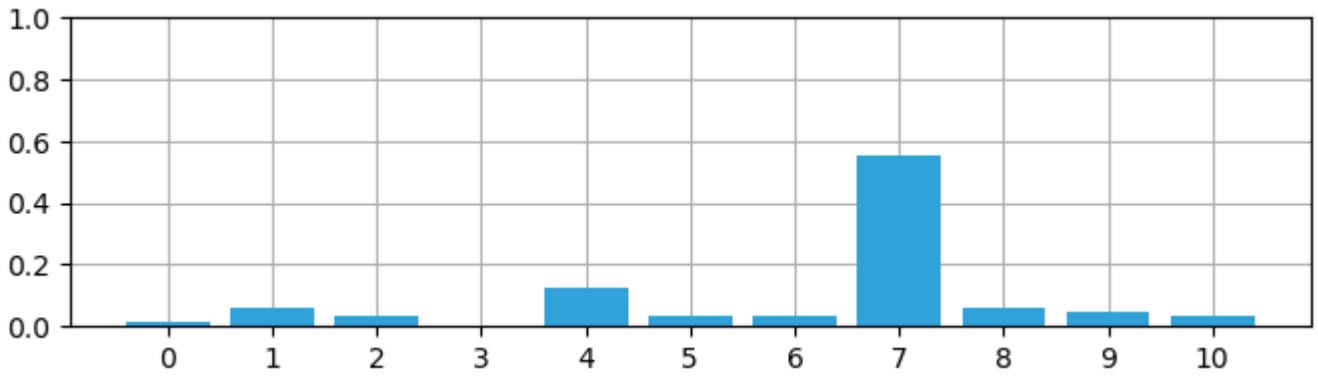
$$\int_{-\infty}^{\infty} p(u)\, du = 1$$

对于连续分布，应对概率密度积分，如上式所示。概率密度在整个取值范围上的积分必须为 1；对于具有密度的连续随机变量，取任一精确数值的单点概率均为零。

在上一章中，我们利用概率分布来估算狗在走廊中的位置。例如：

```
import numpy as np
import kf_book.book_plots as book_plots

belief = np.array([1, 4, 2, 0, 8, 2, 2, 35, 4, 3, 2])
belief = belief / np.sum(belief)
with book_plots.figsize(y=2):
    book_plots.bar_plot(belief)
print('sum = ', np.sum(belief))
```

```
sum =  1.0
```



每个位置对应的概率都在 0 到 1 之间，所有概率之和等于 1。因此，这实际上是一种概率分布。由于这里的位置取值是离散的，我们可以更准确地将其称为离散概率分布。在实际应用中，除非有特殊需要，否则我们通常不会刻意区分“离散”和“连续”这两个概念。

随机变量的均值、中位数和众数

给定一组数据，我们通常希望知道该组的代表性数值或平均值。衡量这一数值的方法有很多，这一概念被称为“集中趋势的度量”。例如，我们可能想知道某个班级里学生的平均身高。我们都知道如何计算一组数据的平均值，不过让我再详细说明一下，以便引入更正式的符号和术语。平均值的另一个说法是“均值”。我们通过将所有数值相加，再除以数值的个数来计算均值。如果以米为单位来表示学生的身高的话……

$$X = \{1.8, 2.0, 1.7, 1.9, 1.6\}$$

我们将平均值计算为

$$\mu = \frac{1.8 + 2.0 + 1.7 + 1.9 + 1.6}{5} = 1.8$$

按照惯例，通常用符号 μ（μ）来表示平均值。

我们可以用方程式来正式表达这一计算过程。

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

NumPy 提供了 `numpy.mean()` 这个函数来计算平均值。

```
x = [1.8, 2.0, 1.7, 1.9, 1.6]
np.mean(x)
```

```
1.8
```

为方便使用，NumPy 数组提供了 `mean()` 这个方法。

```
x = np.array([1.8, 2.0, 1.7, 1.9, 1.6])
x.mean()
```

```
1.8
```

一组数字中出现次数最多的数称为众数。若只有一个最高频数对应的数，则称单峰；若两个或更多数都以同样的最高频数出现，则称多峰。例如 {1,2,2,2,3,4,4,4} 的众数为 2、4，{5,7,7,13} 的众数只有 7。书中更常使用连续或离散概率分布的单峰、多峰概念：比如狗的位置分布有若干主要概率峰时，就称为多峰。不能仅因为不同位置概率不同，就认定分布多峰。

最后，一组数字的中位数就是该组数字的中间值。也就是说，有一半的数字小于中位数，另一半数字大于中位数。这里的“大于”和“小于”都是相对于已排序的数字而言的。如果该组数字的个数是偶数，则取中间的两个数字的平均值作为中位数。

NumPy 提供了 numpy.median() 这个函数来计算中位数。可以看到，{1.8, 2.0, 1.7, 1.9, 1.6}这个数列的中位数是 1.8，因为在排序后，1.8 是该数列的第三个元素。在这个例子中，中位数等于平均值，但通常情况下并非如此。

```
np.median(x)
```

```
1.8
```

随机变量的期望值

随机变量的期望，可以直观地理解为不断重复抽样时，样本均值所趋近的总体平均值。假设可能取值为 $\boldsymbol{x} = [1, 3, 5]$，各值出现的概率相同，那么 $\boldsymbol{x}$ 的平均值应是多少？

当然，这就是 1、3 和 5 的平均值，也就是 3。这很合理：我们预期 1、3 和 5 出现的次数应该相等。所以，$(1 + 3 + 5)/3 = 3$ 显然就是这一系列无穷多个样本的平均值。换句话说，这里的期望值其实就是样本空间的平均值。

现在假设各个值出现的概率不同：1 的概率为 80%，3 的概率为 15%，5 的概率为 5%。这时，计算期望值的方法是把 $\boldsymbol{x}$ 的各个可能值乘以各自出现的概率，再将结果相加。在这个例子中，可以计算：

$$\mathbb{E}[X] = (1)(0.8) + (3)(0.15) + (5)(0.05) = 1.5$$

这里用 $\mathbb{E}[X]$ 表示 $\boldsymbol{x}$ 的期望，有些文献记作 $\boldsymbol{E(x)}$。把 1.5 作为 $\boldsymbol{x}$ 的期望是符合直觉的，因为 $\boldsymbol{x}$ 取 1 的概率远大于取 3 或 5 的概率，取 3 的概率也大于取 5 的概率。注意，1.5 是期望值，不是这个随机变量本身可能出现的取值。

可把上述计算写得更正式：令 $\boldsymbol{x_i}$ 为随机变量 $\boldsymbol{X}$ 的第 $\boldsymbol{i^{th}}$ 个可能取值，$\boldsymbol{p_i}$ 为该值出现的概率，于是有：

$$\mathbb{E}[X] = \sum_{i=1}^{n} p_i x_i$$

简单的代数运算就能证明：如果所有概率都相等，那么期望值就等于平均值。

$$\mathbb{E}[X] = \sum_{i=1}^{n} p_i x_i = \frac{1}{n} \sum_{i=1}^{n} x_i = \mu_x$$

如果 $\boldsymbol{x}$ 是连续随机变量，就把求和替换为积分：

$$\mathbb{E}[X] = \int_{a}^{b} x f(x)\, dx$$

其中 $\boldsymbol{f(x)}$ 是连续随机变量 $\boldsymbol{x}$ 的概率密度函数。暂时不使用这一积分公式，下一章会用到。

我们可以用一些 Python 代码来模拟这个过程。在这里，我选取了 1,000,000 个样本，然后计算出我们通过分析方法所得到的概率分布的期望值。

```
total = 0
N = 1000000
for r in np.random.rand(N):
    if r <= .80: total += 1
    elif r < .95: total += 3
    else: total += 5

total / N
```

```
1.50053
```

可以看出，计算得到的数值与通过分析方法得出的数值非常接近。不过，这两个数值并不完全相同，因为要得到精确的数值，需要无限大的样本量。

练习

掷骰子的期望值是多少?

解答

骰子的每一面出现的可能性相同，各点数的概率均为 1/6。因此，

$$\begin{aligned}\mathbb{E}[X] &= 1/6 \times 1 + 1/6 \times 2 + 1/6 \times 3 + 1/6 \times 4 + 1/6 \times 5 + 1/6 \times 6 \\ &= 1/6(1+2+3+4+5+6) \\ &= 3.5\end{aligned}$$

练习

在均匀连续分布的条件下

$$f(x) = \frac{1}{b-a}$$

计算 $a = 0$ 和 $b = 20$ 的期望值。

解答

$$\begin{aligned}\mathbb{E}[X] &= \int_0^{20} x\frac{1}{20}\,dx \\ &= \left[\frac{x^2}{40}\right]_0^{20} \\ &= 10 - 0 \\ &= 10\end{aligned}$$

随机变量的方差

上述计算结果给出了学生的平均身高，但并未告诉我们所有想要了解的信息。例如，假设我们有三个班级，分别记为 X、Y 和 Z，这些班级学生的身高分别为：

```
X = [1.8, 2.0, 1.7, 1.9, 1.6]
Y = [2.2, 1.5, 2.3, 1.7, 1.3]
Z = [1.8, 1.8, 1.8, 1.8, 1.8]
```

使用 NumPy 进行分析后我们发现，每个班级的平均身高都是相同的。

```
print(np.mean(X), np.mean(Y), np.mean(Z))
```

```
1.8 1.8 1.8
```

这三个班级的平均身高均为 1.8 米。不过，第二个班级的身高差异远大于第一个班级，而第三个班级的学生身高完全相同。

平均值能让我们了解一些关于数据的信息，但并不能完全反映实际情况。我们需要知道学生的身高之间存在多大的差异。原因可能有很多。比如，某个学区需要订购 5,000 张课桌，他们当然希望所购买的课桌尺寸能够适应所有学生的身高差异。

统计学用“标准差”和“方差”来定量描述这种差异。计算方差的公式为：

$$VAR(X) = \mathbb{E}[(X - \mu)^2]$$

先看偏差，也就是随机变量 X 与均值 μ 之差，记作 $X - \mu$)。方差是这些偏差的平方的期望。偏差本身的期望为零，因此不能省略平方；平方项的作用稍后再解释。根据期望公式 $\mathbb{E}[X] = \sum_{i=1}^{n} p_i x_i$，就可以把前面的定义展开如下。

$$VAR(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

让我们来计算这三个班级的方差，看看会得到什么数值，同时也能更好地理解这个概念。

X 的平均值为 1.8（ $\mu_x = 1.8$ ），因此我们进行如下计算：

$$\begin{aligned} VAR(X) &= \frac{(1.8 - 1.8)^2 + (2 - 1.8)^2 + (1.7 - 1.8)^2 + (1.9 - 1.8)^2 + (1.6 - 1.8)^2}{5} \\ &= \frac{0 + 0.04 + 0.01 + 0.01 + 0.04}{5} \\ VAR(X) &= 0.02\,m^2 \end{aligned}$$

NumPy 提供了 `var()` 函数来计算方差：

```
print(f"{np.var(X):.2f} meters squared")
```

```
0.02 meters squared
```

这可能有点难以理解。身高是以米为单位的，而方差则是以米的平方为单位的。因此，我们使用更常见的度量标准——标准差。标准差的定义是方差的平方根。

$$\sigma = \sqrt{VAR(X)} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

通常，用 σ 表示标准差，用 σ^2 表示方差。在本书的大部分内容中，我会使用 σ^2 来表示方差，而不是 $VAR(X)$；其实这两个符号表示的是同一种概念。

第一个班级的标准差计算如下。

$$\begin{aligned}\sigma_x &= \sqrt{\frac{(1.8-1.8)^2 + (2-1.8)^2 + (1.7-1.8)^2 + (1.9-1.8)^2 + (1.6-1.8)^2}{5}} \\ &= \sqrt{\frac{0 + 0.04 + 0.01 + 0.01 + 0.04}{5}} \\ \sigma_x &= 0.1414\end{aligned}$$

我们可以使用 NumPy 中的相关函数来验证这个计算结果。该函数用于计算标准差。`std`是标准差的常用缩写形式。

```
print(f"std {np.std(X):.4f}")
print(f"var {np.std(X)**2:.4f}")
```

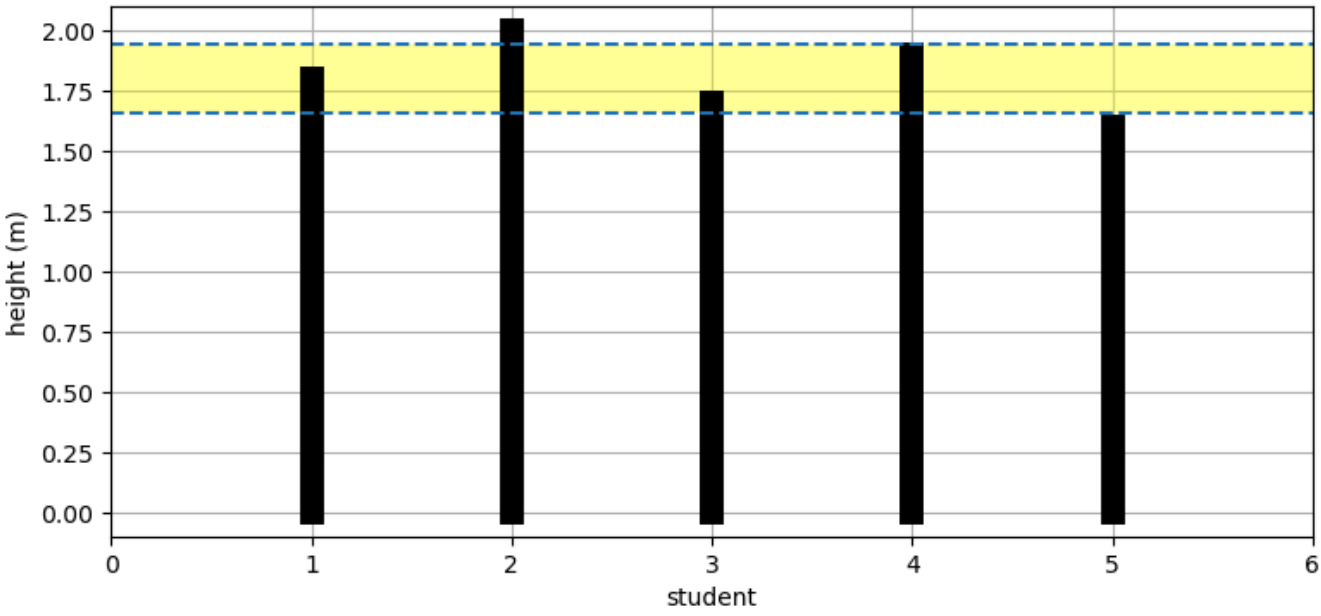
```
std 0.1414
var 0.0200
```

当然，$0.1414^2 = 0.02$ 的结果也与我们之前计算出的方差一致。

标准差描述取值相对均值的离散程度。后面引入高斯分布时，会看到更具体的覆盖率解释。若身高近似服从均值 1.8 米、标准差 0.1414 米的高斯分布，则约 68% 的总体取值位于 1.66—1.94 米之间。注意，68% 是高斯假设下的近似结论，不适用于任意分布，也不保证有限样本中恰有这一比例。

我们可以通过图表来直观地了解这一点：

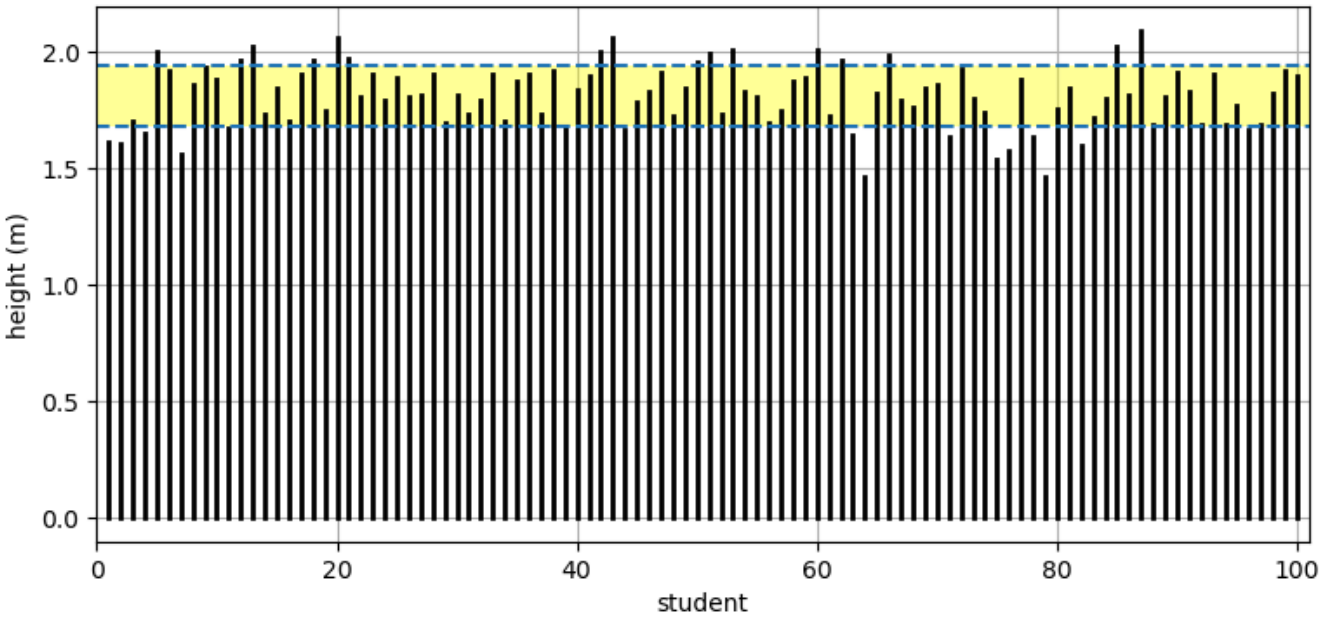
```
from kf_book.gaussian_internal import plot_height_std
import matplotlib.pyplot as plt

plot_height_std(X)
```



只有 5 名学生时，落在一个标准差内的比例显然很难恰好为 68%。这里有 3 人落在均值的 $\pm 1\sigma$ 范围内，即 60%；只有 5 个样本时，这已经是最接近 68% 的可能比例。再来看一个有 100 名学生的班级。

> 我们将一个标准差记作 1σ，读作“一个标准差”，而不是“一个西格玛”。两个标准差则记作 2σ，依此类推。

```
from numpy.random import randn
data = 1.8 + randn(100)*.1414
mean, std = data.mean(), data.std()

plot_height_std(data, lw=2)
print(f'mean = {mean:.3f}')
print(f'std  = {std:.3f}')
```



```
mean = 1.808
std  = 0.133
```

目测大约 68% 的身高落在均值 1.8 的 $\pm 1\sigma$ 范围内，也可以用代码验证。

```
np.sum((data > mean-std) & (data < mean+std)) / len(data) * 100.
```

```
68.0
```

我们稍后会更深入地探讨这个问题。目前，让我们先来计算一下标准差吧。

$$Y = [2.2, 1.5, 2.3, 1.7, 1.3]$$

Y 的平均值为 $\mu = 1.8$ 米。

$$\begin{aligned}\sigma_y &= \sqrt{\frac{(2.2-1.8)^2 + (1.5-1.8)^2 + (2.3-1.8)^2 + (1.7-1.8)^2 + (1.3-1.8)^2}{5}} \\ &= \sqrt{0.152} = 0.39\,m\end{aligned}$$

我们将使用 NumPy 来验证这一点。

```
print(f'std of Y is {np.std(Y):.2f} m')
```

```
std of Y is 0.39 m
```

这与我们的预期相符。 Y 的身高差异更大，标准差也更高。

最后，让我们来计算 Z 的标准差。由于各个数值之间没有差异，因此标准差应该为零。

$$\begin{aligned}\sigma_z &= \sqrt{\frac{(1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2}{5}} \\ &= \sqrt{\frac{0+0+0+0+0}{5}} \\ \sigma_z &= 0.0\,m\end{aligned}$$

```
print(np.std(Z))
```

```
0.0
```

继续之前要指出，我暂时忽略了男性平均身高通常高于女性这一因素。一般来说，只有男性或只有女性的班级，身高方差可能小于两者混合的班级。其他因素也会影响总体分布，例如营养状况和人群构成。例如，营养良好儿童的平均身高往往高于营养不良儿童，斯堪的纳维亚人群的平均身高高于意大利人群。统计学家设计实验时需要考虑这些因素。这里讨论的是群体分布，不能理解为每个男性都高于每个女性。

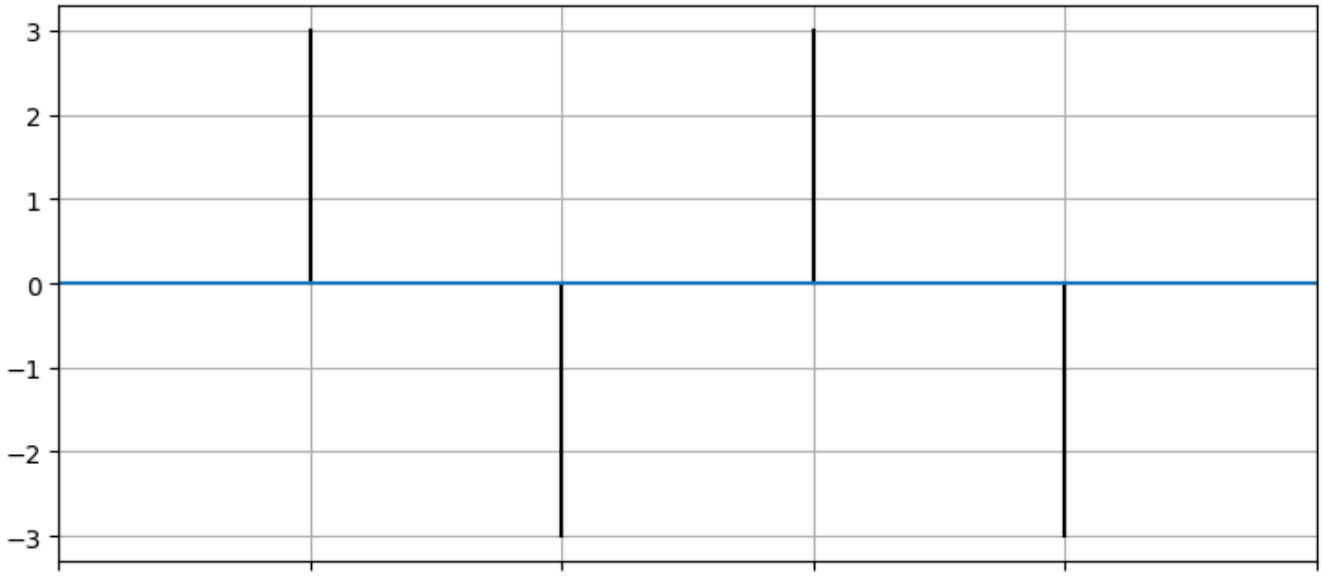
前面我提到，进行这项分析可能是为了给某个学区订购课桌。对于每个年龄组来说，很可能会出现两个不同的平均身高值：一个接近女性学生的平均身高，另一个则接近男性学生的平均身高。整个班级的平均身高则介于这两个数值之间。如果我们按照所有学生的平均身高来订购课桌，那么最终得到的课桌很可能既不适合男生，也不适合女生使用！

在本书中，我们不会探讨这些问题。如果你需要了解处理这些问题的方法，可以参考任何标准的概率论教材。

为什么是差值的平方呢?

为什么要用偏差的平方计算方差？先不作复杂推导，来看一个简单例子：令 $\boldsymbol{X} = [3, -3, 3, -3]$，下图把 $\boldsymbol{X}$ 的各个取值与其均值作比较。

```
X = [3, -3, 3, -3]
mean = np.average(X)
for i in range(len(X)):
    plt.plot([i ,i], [mean, X[i]], color='k')
plt.axhline(mean)
plt.xlim(-1, len(X))
plt.tick_params(axis='x', labelbottom=False)
```



如果我们不计算差值的平方，那么各种正负号就会相互抵消，结果为零：

$$\frac{(3-0)+(-3-0)+(3-0)+(-3-0)}{4} = 0$$

这显然是错误的，因为数据中的方差肯定大于 0。

也许可以使用离均值差的绝对值？观察可得 $\boldsymbol{12/4 = 3}$，每个值与均值的距离都是 3。但若取 $\boldsymbol{Y} = [6, -2, -3, 1]$，同样得到 $\boldsymbol{12/4 = 3}$。$\boldsymbol{Y}$ 显然比 $\boldsymbol{X}$ 更分散，平均绝对偏差却相同。若用平方差计算，$\boldsymbol{Y}$ 的均值为 0.5，总体方差为 12.25，标准差为 3.5；X 的总体方差为 9。因此，方差可以区分这两组数据的离散程度。

这并不能作为该方法的正确性的证明。事实上，这一方法的发明者卡尔·弗里德里希·高斯也意识到，这种处理方式其实有些任意性。如果数据中存在异常值，那么对差异进行平方处理会使得这些异常值的权重过大。例如，让我们来看看以下情况会怎样：

```
X = [1, -1, 1, -2, -1, 2, 1, 2, -1, 1, -1, 2, 1, -2, 100]
print(f'Variance of X with outlier    = {np.var(X):6.2f}')
print(f'Variance of X without outlier = {np.var(X[:-1]):6.2f}')
```

```
Variance of X with outlier    = 621.45
Variance of X without outlier =   2.03
```

这样“正确”吗？由你来判断吧。如果不把那个值为 100 的异常值考虑在内，我们得到的结果就是 $\sigma^2 = 2.03$，这一结果准确地反映了在没有该异常值的情况下 X 的变动情况。而那个异常值却严重影响了整体数据的方差计算结果。我们是想通过这种方式来确认存在异常值，还是希望能够妥善处理这个异常值，同时得到一个接近于没有异常值时的估计值呢？当然，这取决于你的具体问题。

我不会继续沿着这条道路走了；如果你感兴趣的话，可以参考詹姆斯·伯杰在贝叶斯稳健性领域的相关研究成果，或者阅读彼得·J·胡贝尔关于稳健统计学的优秀著作[4]。在本书中，我们将始终按照高斯所定义的方式来使用方差和标准差。

从这一点可以看出，这些汇总统计量永远无法完整地反映数据的真实情况。以高斯所定义的方差为例，它无法告诉我们数据中是否存在某个异常值。不过，汇总统计量仍然是一种非常有用的工具，因为它让我们用寥寥几个数字就能概括整个数据集的情况。如果我们有 10 亿个数据点，我们肯定不想手动去查看图表或逐个查看数字列表；汇总统计量则为我们提供了一种有效的方式来描述数据的特征。

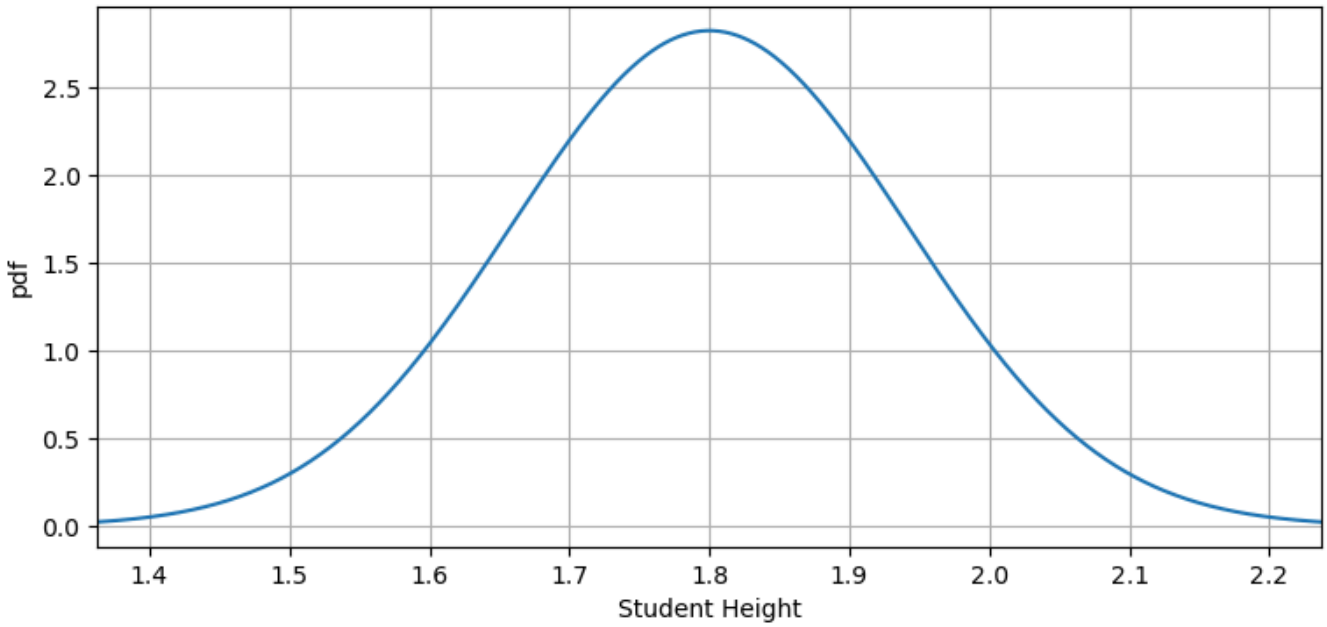
高斯分布

我们现在准备开始学习高斯函数的相关知识了。让我们先回顾一下本章的学习目的吧。

> 我们需要一种连续、单峰、计算高效的概率表示方法，在适用的建模假设下描述现实系统。

让我们看一下高斯分布的图表，以便更好地理解我们正在讨论的内容。

```
from filterpy.stats import plot_gaussian_pdf
plot_gaussian_pdf(mean=1.8, variance=0.1414**2,
                  xlabel='Student Height', ylabel='pdf');
```



这条曲线称为概率密度函数，简称 PDF，表示随机变量在各个值附近的相对可能性。图中，身高在 1.8 米附近比在 1.7 米附近稍常见，在 1.9 米附近则比在 1.4 米附近常见得多。换句话说，许多学生身高接近 1.8 米，接近 1.4 米或 2.2 米的很少。还要注意，曲线中心位于均值 1.8 米处。概率密度不是某个单点的概率；连续变量取单个精确值的概率为零。

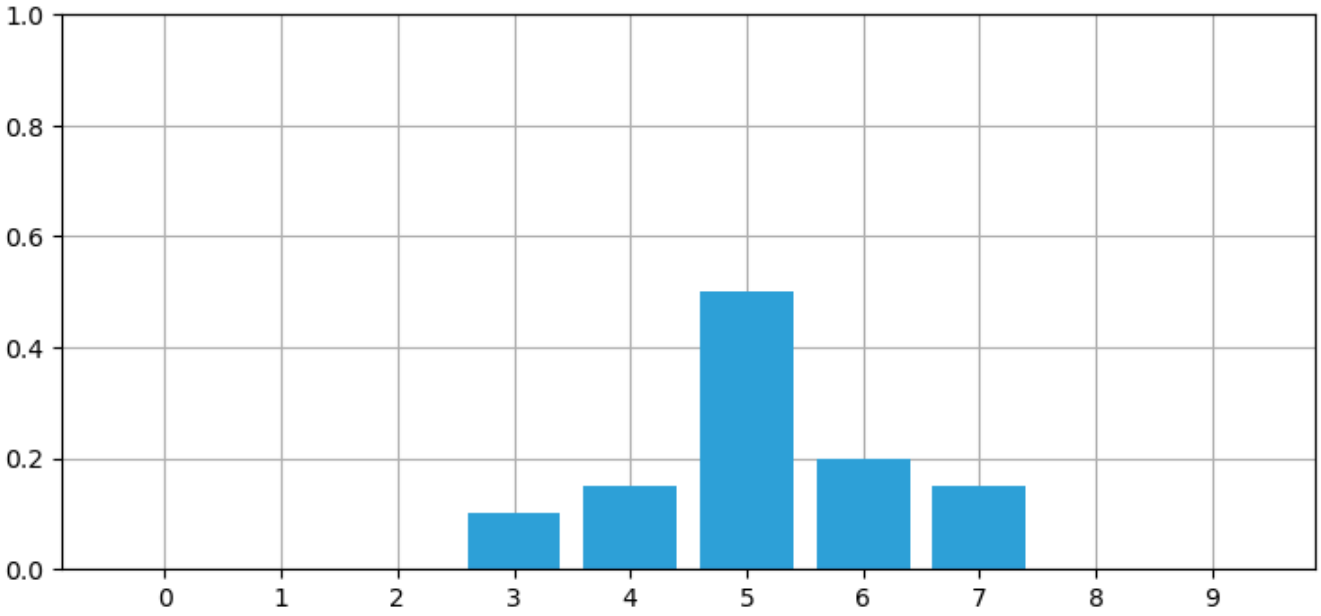
> 我在“Supporting_Notebooks”文件夹中的“Computing_and_Plotting_PDFs”文档里详细介绍了如何绘制高斯曲线图以及更多相关内容。你可以在这里在线阅读该文档[1]。

你可能认出这就是通常所说的钟形曲线。现实中许多观测近似呈现这种形状，因此它很常见。但我不会把钟形曲线直接当作高斯分布的同义词，因为其他概率分布也可以呈钟形。非数学资料的用词未必严格，遇到没有定义的“钟形曲线”时，不要据此断定它一定是正态分布。

这种分布并非身高独有，许多自然现象和滤波传感器误差也可以近似用它建模。它具有所需特点：用单峰概率分布表示不确定性，取值连续，而且计算高效。稍后还会发现它的其他优良性质。

为了进一步激励你，不妨回想一下在“离散贝叶斯”那一章中提到的各种概率分布的形状吧：

```
import kf_book.book_plots as book_plots
belief = [0., 0., 0., 0.1, 0.15, 0.5, 0.2, .15, 0, 0]
book_plots.bar_plot(belief)
```

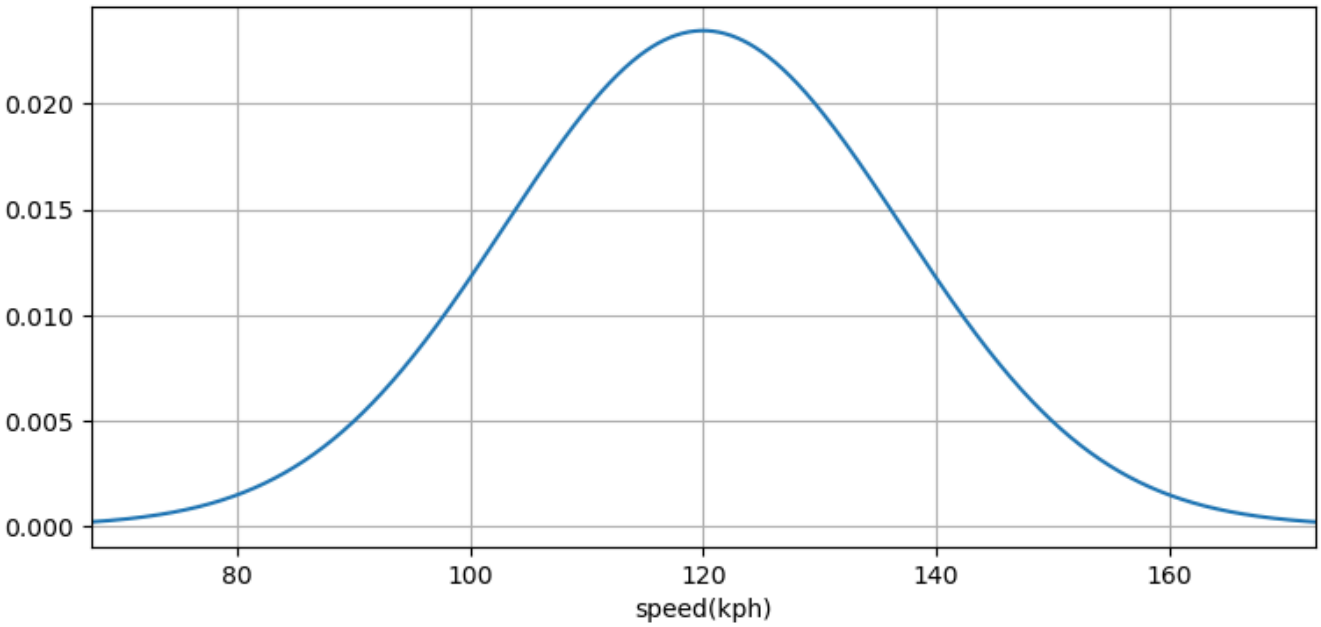


它们并非完美的高斯曲线，但大致类似。在后续的计算中，我们将用高斯曲线来替代那一章中使用的离散概率值！

术语说明

继续之前，先说明一个术语：这幅图表示随机变量在区间 $(-\infty, \infty)$ 上的概率密度。它是什么意思？设想在一段公路上对车速进行无限次、精度无限高的测量，再绘出各个速度附近的车辆相对频数。如果平均速度为 120 千米/小时，图形可能如下。

```
plot_gaussian_pdf(mean=120, variance=17**2, xlabel='speed(kph)');
```



纵轴表示概率密度，可理解为相应速度附近、每单位速度区间内的相对车辆频数。下一节将进一步解释。

高斯模型并不完美。虽然图中没有完整显示，分布的两条尾部会一直延伸到无穷远。尾部是曲线两端数值最低的部分。当然，人的身高或汽车速度不可能小于零，更不可能取 $-\infty$ 或 ∞。“地图不等于疆域”这句话同样适用于贝叶斯滤波与统计学。上面的高斯分布对测得的车速分布进行建模，但模型必然不完美。模型与现实的差异会在这些滤波器中反复出现。高斯分布广泛用于数学的多个分支，并非因为它能完美描述现实，而是因为它比其他精度相当的选择更容易处理。不过，即使在本书中，高斯分布也会遇到无法合理描述现实的情况，迫使我们采用计算代价更大的替代方法。

这些分布既称为高斯分布，也称为正态分布。在这一语境中，两种名称指的是同一种分布，可以互换使用。不同资料会采用不同名称，所以本书也会交替使用，让你熟悉这两种叫法。此外，人们还常把“高斯分布”简称为“高斯”，把“正态分布”简称为“正态”。

高斯分布

让我们来了解一下高斯函数的工作原理。高斯函数只需两个参数就能完全描述：均值（ μ ）和方差（ σ^2 ）。其定义如下：

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

$\exp[x]$ 其实指的是 e^x 。

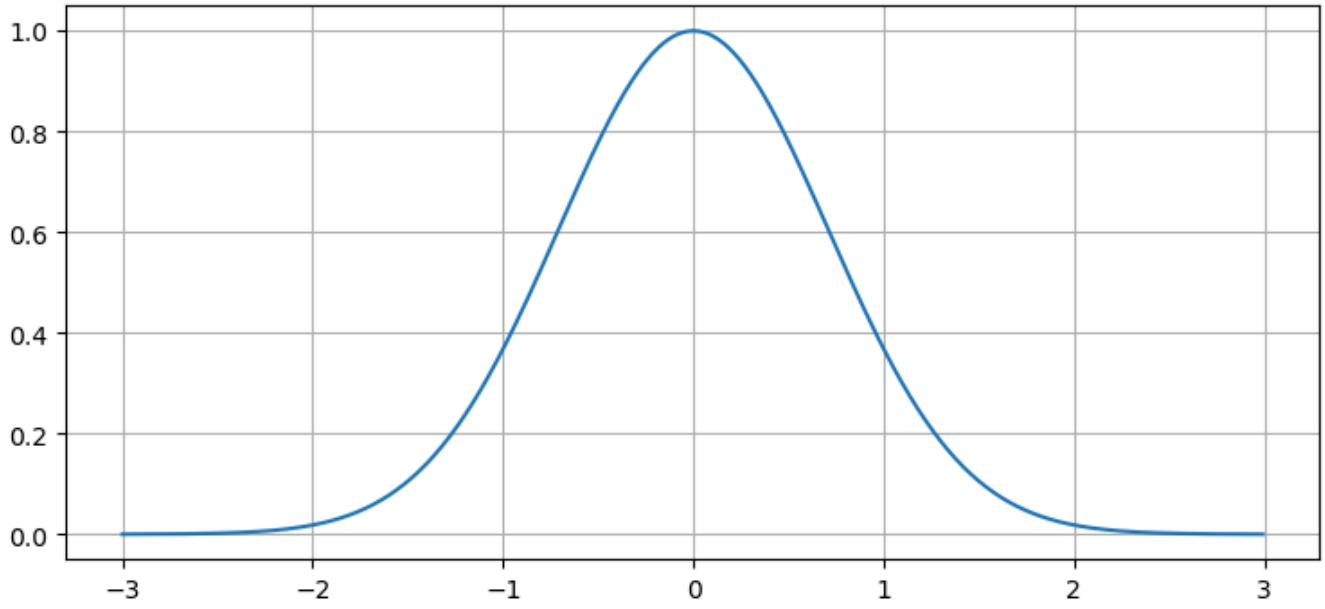
如果你之前没见过这个公式，不必因此而却步；你无需记住它或对其进行复杂的运算。该函数的计算过程已存储在`stats.py`文件中，函数名为`gaussian(x, mean, var, normed=True)`。

去掉那些常数后，就可以看出这其实是一个简单的指数函数而已：

$$f(x) \propto e^{-x^2}$$

呈现出人们所熟悉的钟形曲线形状。

```
x = np.arange(-3, 3, .01)
plt.plot(x, np.exp(-x**2));
```

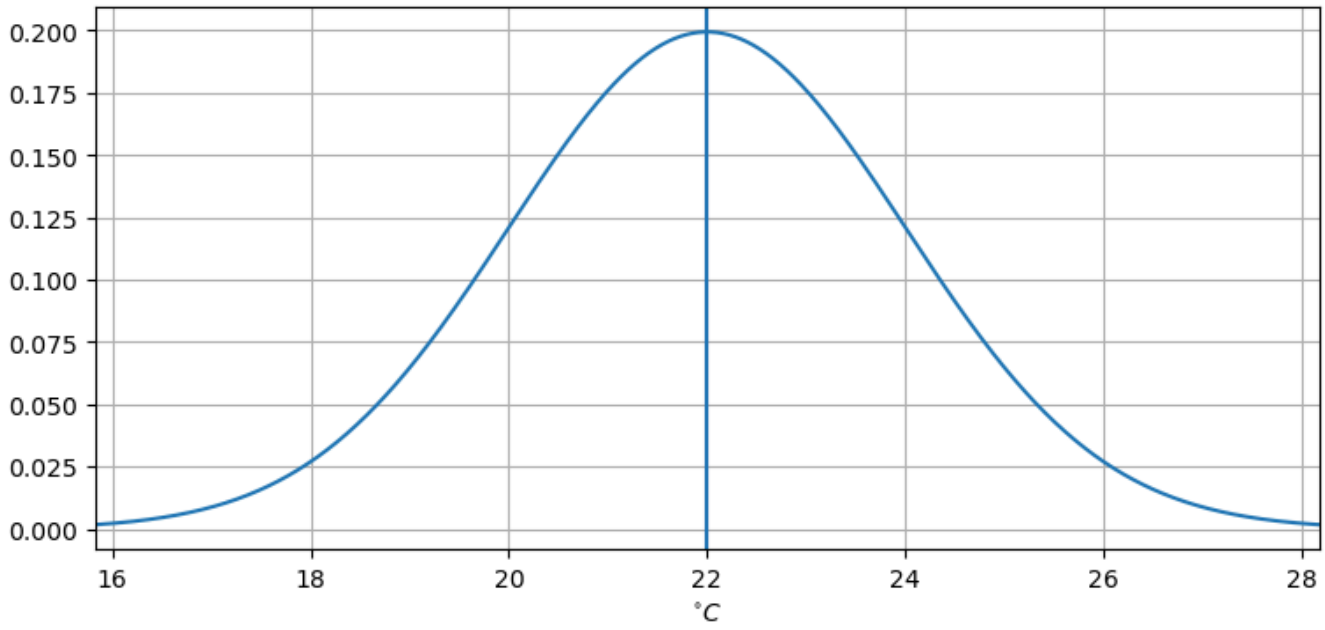


让我们回顾一下如何查看函数的代码。在某个单元格中，输入函数名称，后跟两个问号，然后按 CTRL+ENTER 键。这样就会弹出一个窗口，显示该函数的源代码。请取消下一个单元格中的注释，然后试着运行一下吧。

```
from filterpy.stats import gaussian
#gaussian??
```

下面画出均值为 22 $(\mu = 22)$、方差为 4 $(\sigma^2 = 4)$ 的高斯分布。

```
plot_gaussian_pdf(22, 4, mean_line=True, xlabel='$^{\circ}C$');
```



这条曲线意味着什么？假设真实温度为 22°C，温度计读数因测量误差而在它附近变化。如果误差可合理建模为高斯噪声，那么图中的密度曲线描述各个读数附近的相对可能性。采用这一模型需要依据误差的特性，而不能仅凭测量次数多就断定误差服从高斯分布。中心极限定理在相应条件下描述大量随机量之和或均值的标准化分布，并不保证单个传感器的原始读数或误差都服从高斯分布。

高斯分布是连续分布。对于具有概率密度的连续随机变量，任意单个精确数值的概率均为零，因此图中恰好读到 2°C 的概率为零。这与读数落在某个非零宽度区间内的概率不同。

这条曲线究竟是什么呢？其实，它就是我们所说的概率密度函数。曲线下面积的大小，代表了相应数值出现的概率。例如，如果计算 20 到 22 之间曲线的面积，那么得到的结果就是温度值位于这两个数值之间的概率。

还可以借助物质密度来理解。岩石或海绵的密度是什么？密度是单位体积内物质的质量。岩石密度较高，海绵密度较低。如果想知道一块岩石的质量但没有秤，可以测量它的体积，再乘以密度。实际物体的密度未必处处相同，因此，更一般的做法是对整个岩石体积内的局部密度进行积分，从而得到总质量。

$$M = \iiint_R p(x, y, z)\, dV$$

对于概率密度函数，我们也是用同样的方法来处理。如果你想知道温度是否在 20°C 到 21°C 之间，就需要将上述曲线在 20 到 21 这个区间内进行积分。众所周知，曲线的积分值代表了曲线下面积。由于这里涉及的是概率密度函数，因此其积分值就是相应的概率。

温度恰好为 22°C 的概率是多少呢？凭直觉来说，应该是 0。这些都是实数，所以，22°C 与 22.00000000000017°C 之间的差异其实微乎其微。从数学上讲，如果对 22 到 22 之间的区间进行积分，结果也是 0。

再想想那块岩石吧：岩石上某一个单独的点的重量是多少呢？一个微小的点当然没有重量。询问某个单独点的重量是没有意义的；同样，询问连续随机变量取某个特定值的概率也是没有意义的。这两者的答案显然都是零。

实际传感器的精度并非无限高，因此 22°C 的读数隐含了一个范围，例如 22 ±0.1°C。对 21.9 到 22.1 积分，就能计算该范围内的概率。

可以从贝叶斯或频率学派的角度理解这幅图。按贝叶斯的解释，如果温度计恰好显示 22°C，曲线表示我们对真实系统温度的信念：真实温度接近 22°C 的可信程度高，接近 18°C 的可信程度低。按频率学派的解释，如果真实温度保持在精确的 22°C，对系统重复测量十亿次，则测得读数的直方图会近似这条曲线。注意，保持 22°C 的是被测系统的真实温度，并不是每次读数都恰好等于 22°C。

如何计算概率或曲线下的面积呢？需要对高斯函数的方程进行积分。

$$\int_{x_0}^{x_1} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(x-\mu)^2/\sigma^2} dx$$

这类积分可借助累积分布函数（CDF）计算。CDF 表示从负无穷到某个上界的概率密度积分；上式给出的有限区间概率，等于两个端点处 CDF 值之差。

我编写了 `filterpy.stats.norm_cdf`，它可以帮你计算积分。例如，我们可以这样来计算：

```
from filterpy.stats import norm_cdf
print('Cumulative probability of range 21.5 to 22.5 is {:.2f}%'.format(
      norm_cdf((21.5, 22.5), 22,4)*100))
print('Cumulative probability of range 23.5 to 24.5 is {:.2f}%'.format(
      norm_cdf((23.5, 24.5), 22,4)*100))
```

```
Cumulative probability of range 21.5 to 22.5 is 19.74%
Cumulative probability of range 23.5 to 24.5 is 12.10%
```

均值（μ）表示分布的平均位置，也就是各个可能取值按其概率加权的平均。由于高斯曲线对称，均值也位于曲线最高点。温度计读数为 22°C，因此这里将均值设为 22°C。

对于随机变量 $\boldsymbol{X}$ 而言，正态分布的表示方式为 $\boldsymbol{X} \sim \mathcal{N}(\mu, \sigma^2)$，其中 $\sim$ 表示“按照某种分布规律来分布”。这意味着，我可以将温度计的测量结果用这种方式来表示。

$$\textbf{temp} \sim \mathcal{N}(22, 4)$$

这是一个极其重要的成果。高斯函数让我只需用两个数值就能描述无数种可能的数值情况！只要知道 $\mu = 22$ 和 $\sigma^2 = 4$ 这两个数值，我就能计算出任意范围内各数值的分布情况。

有些资料使用 $\mathcal{N}(\mu, \sigma)$ 来代替 $\mathcal{N}(\mu, \sigma^2)$。两种方式都可以，都属于常见的惯例。如果你看到像 $\mathcal{N}(22, 4)$ 这样的符号，就需要弄清楚到底使用的是哪种形式。在这本书中，我一直使用 $\mathcal{N}(\mu, \sigma^2)$，所以在这个例子中，也使用 $\sigma = 2$ 和 $\sigma^2 = 4$。

方差与信念

作为概率密度函数，曲线在整个实数轴下的面积必须等于 1，因为这覆盖了所有可能结果。可以编写代码验证；也可以对高斯密度从 $-\infty$ 到 ∞ 积分。注意，连续密度的归一化是积分等于 1，不是把各点密度直接相加等于 1。

```
print(norm_cdf((-1e8, 1e8), mu=0, var=4))
```

```
1.0
```

由此得到一个重要直觉：方差小，曲线较窄；为了使总面积保持 1，峰值就较高。方差大，曲线较宽；为了保持同样面积，峰值就较低。

让我们用图表来直观地看一下吧。我们将使用之前提到的 `filterpy.stats.gaussian`，它既可以接受单个数值，也可以接受数值数组作为输入。

```
from filterpy.stats import gaussian

print(gaussian(x=3.0, mean=2.0, var=1))
print(gaussian(x=[3.0, 2.0], mean=2.0, var=1))
```

```
0.24197072451914337
[0.378 0.622]
```

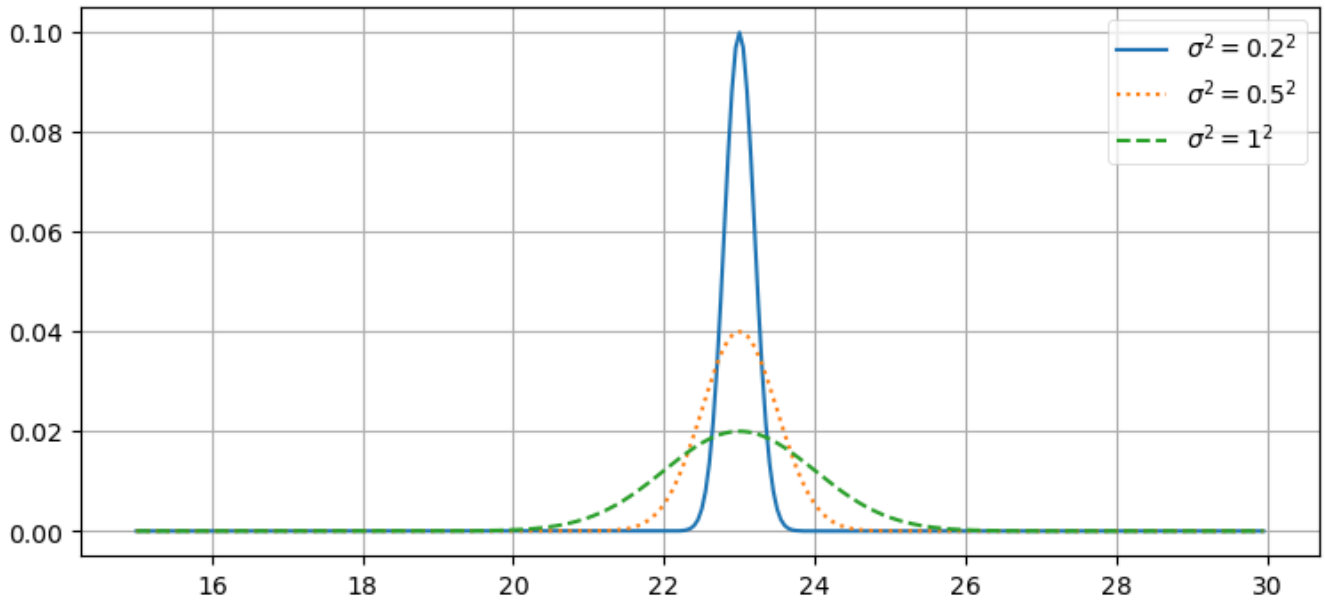
默认情况下，当输入为一组取值时，FilterPy 的 gaussian 会把返回的密度值再除以它们的总和，使离散输出之和为 1。可用 normed 参数控制这一行为。若需要计算连续高斯分布的原始概率密度值，应设置 normed=False；这与连续密度在实数轴上积分为 1 的归一化不同。

```
print(gaussian(x=[3.0, 2.0], mean=2.0, var=1, normed=False))
```

```
[0.242 0.399]
```

一般来说，若一个高斯形函数尚未归一化为积分等于 1，就只能称为高斯函数，而不是概率密度。这里还要区分：gaussian 的 normed 参数控制的是数组输出是否按离散和归一化，并不控制连续高斯密度公式中的归一化常数。

```
xs = np.arange(15, 30, 0.05)
plt.plot(xs, gaussian(xs, 23, 0.2**2), label='$\sigma^2=0.2^2$')
plt.plot(xs, gaussian(xs, 23, .5**2), label='$\sigma^2=0.5^2$', ls=':')
plt.plot(xs, gaussian(xs, 23, 1**2), label='$\sigma^2=1^2$', ls='--')
plt.legend();
```



这说明了什么？方差为 $\sigma^2 = 0.2^2$ 的高斯分布很窄，表示我们相信 $x = 23$，并且很有把握，标准差仅为 0.2，对应一个标准差的范围 ± 0.2。相比之下，方差为 $\sigma^2 = 1^2$ 的高斯分布同样以 $x = 23$ 为中心，但确定程度低得多。我们对 $x = 23$ 的信心降低，认为 x 可能取的值分布得更宽；例如，$x = 20$ 或 $x = 26$ 都更有可能。方差为 $\sigma^2 = 0.2^2$ 的分布已使 22 或 24 几乎不可能，而方差为 $\sigma^2 = 1^2$ 时，这两个值的可能性仍接近 23。

如果我们回想一下温度计的原理，就可以把这三条曲线看作是三支不同温度计的测量结果。$\sigma^2 = 0.2^2$ 对应的曲线代表的是非常精确的温度计的测量结果；而 $\sigma^2 = 1^2$ 对应的曲线则代表的是精度较低的温度计的测量结果。需要注意的是，高斯分布有一个非常重要的优点：我们只需两个数值——均值和方差——就能完整地描述温度计的测量结果以及其误差。

高斯分布还可以写为 $\mathcal{N}(\mu, 1/\tau)$，其中 μ 是均值，τ 是精度，精度等于方差的倒数，因此 $1/\tau = \sigma^2$。本书不采用这一参数化，但它突出说明方差与精度的反向关系：方差小，精度大；方差大，精度小。应熟悉这些等价观点：高斯分布可以描述信念、测量精度以及测量的离散程度。注意，是 τ 而不是 1/τ 表示方差的倒数。

我有点过于超前了，但在接下来的章节中，我们将利用高斯分布来描述我们对各种数值的估计，比如我们所追踪的物体的位置，以及我们所使用的传感器的精确度。

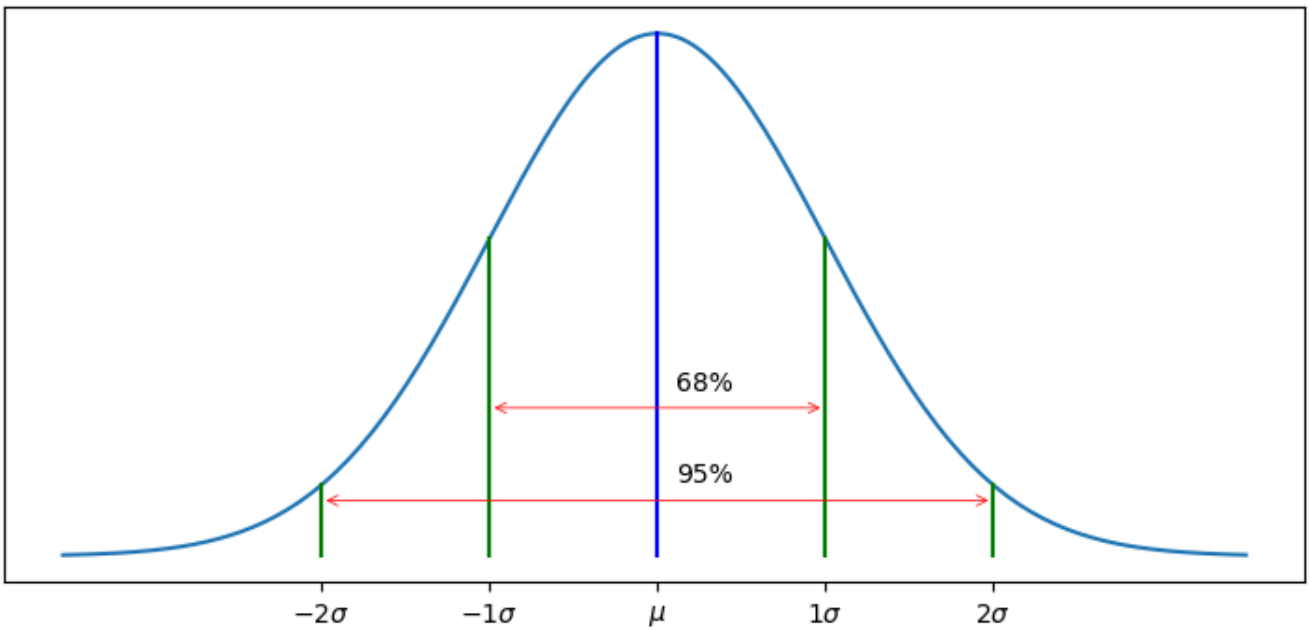
68-95-99.7 法则

这里值得多谈几句标准差。标准差衡量数据偏离均值的程度。对于高斯分布，大约 68% 的数据位于均值的一个标准差（$\pm 1\sigma$）范围内，95% 位于两个标准差（$\pm 2\sigma$）范围内，99.7% 位于三个标准差（$\pm 3\sigma$）范围内。这常被称为 68—95—99.7 法则。若某班考试成绩均值为 71、标准差为 9.4，且成绩服从正态分布，则约 95% 的学生成绩应在 52.2—89.8 之间，计算方法是 $71 \pm (2 * 9.4)$。

这些参数有明确单位：若位置均值为 $\mu = 22$ 米，标准差也以米计。$\sigma = 0.2$ 表示约 68% 的高斯分布取值位于 21.8—22.2 米之间。方差是标准差的平方，因此为 $\sigma^2 = .04$ 米^2，即平方米。写成 $\sigma^2 = 0.2^2$ 有时更直观，因为 0.2 与原数据使用相同单位。

下图展示了标准差与正态分布之间的关系。

```
from kf_book.gaussian_internal import display_stddev_plot
display_stddev_plot()
```



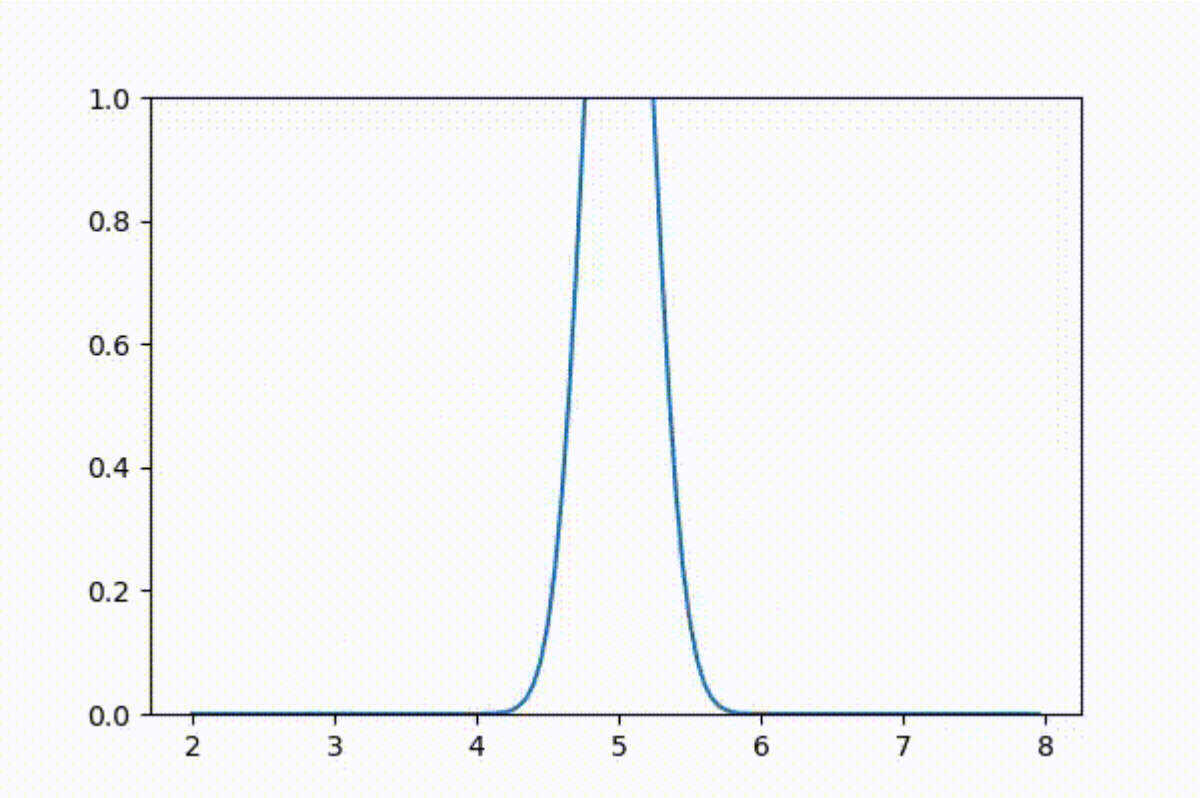
交互式高斯分布

对于那些在 Jupyter Notebook 中阅读此内容的人士，这里有高斯曲线的交互式版本。可以使用滑块来调整 μ 和 σ^2 的值。调整 μ 会改变图形的左右位置，因为这相当于在调整平均值；而调整 σ^2 则会使高斯曲线的形状变得更粗或更细。

```
import math
from ipywidgets import interact, FloatSlider

def plt_g(mu,variance):
    plt.figure()
    xs = np.arange(2, 8, 0.01)
    ys = gaussian(xs, mu, variance)
    plt.plot(xs, ys)
    plt.ylim(0, 0.04)
    plt.show()

interact(plt_g, mu=FloatSlider(value=5, min=3, max=7),
         variance=FloatSlider(value = .03, min=.01, max=1.));
```

```
interactive(children=(FloatSlider(value=5.0, description='mu', max=7.0, min=3.0),
FloatSlider(value=0.03, desc…
```

最后，如果你是通过在线方式阅读这篇文章的，那么这里有一个高斯分布的动画演示。首先，将均值向右移动；接着，将均值重新置于 $\mu = 5$ 位置，并调整方差值。



正态分布随机变量的计算性质

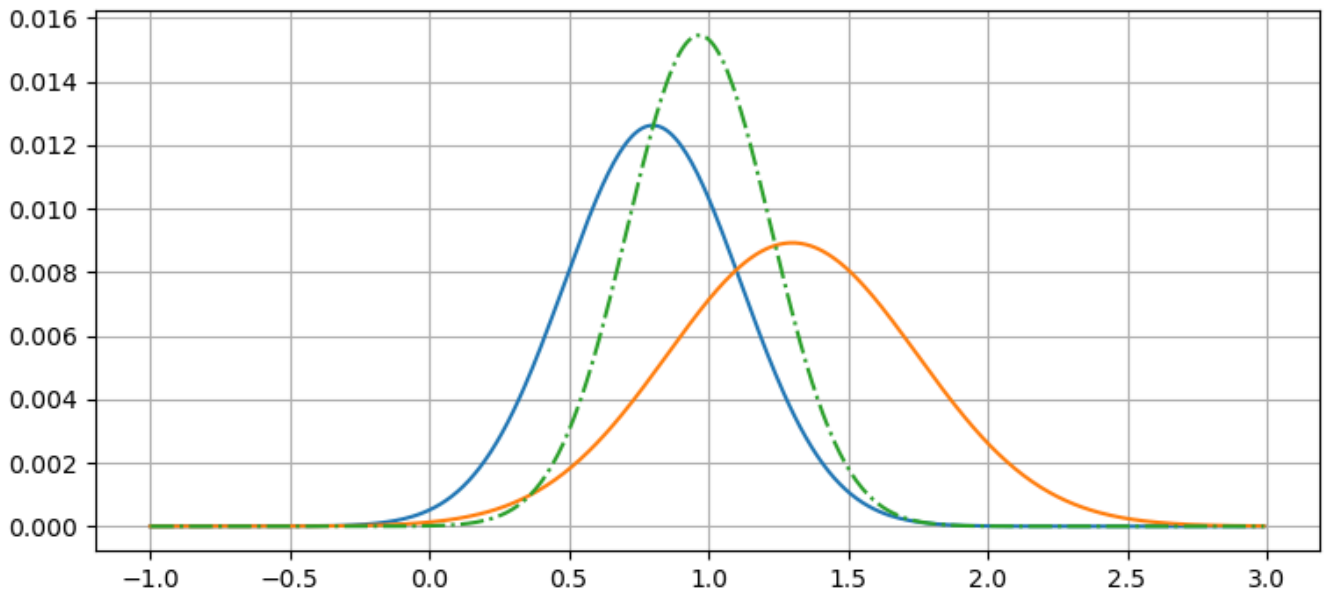
离散贝叶斯滤波对概率数组进行逐元素相乘、归一化和卷积等运算。卡尔曼滤波以高斯分布表示状态不确定性，预测—更新的基本逻辑不变。因此，需要研究独立高斯随机变量相加，以及高斯概率密度相乘的性质。二者是不同运算，不能把密度函数的乘积误当作随机变量本身的乘积。

高斯分布具有重要的封闭性质：两个独立高斯随机变量之和仍服从高斯分布；两个高斯密度函数逐点相乘，结果正比于另一个高斯密度，归一化后成为合法高斯分布。注意，两个独立高斯随机变量本身的乘积，一般既不服从高斯分布，也不能说“正比于高斯分布”。这里后半部分讨论的是密度函数相乘。

维基百科上有关于这个性质的详细介绍，我也在本章末尾对此进行了验证。
https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables

在开始计算之前，我们先来从视觉上验证一下吧。

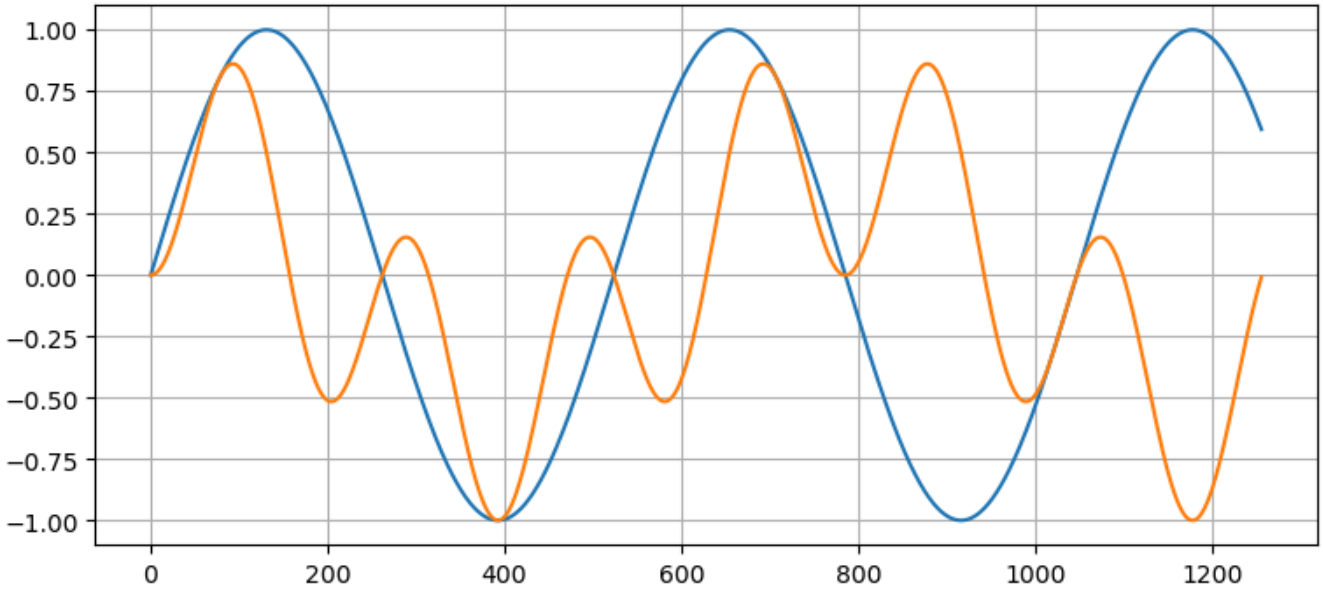
```
x = np.arange(-1, 3, 0.01)
g1 = gaussian(x, mean=0.8, var=.1)
g2 = gaussian(x, mean=1.3, var=.2)
plt.plot(x, g1, x, g2)

g = g1 * g2  # element-wise multiplication
g = g / sum(g)  # normalize
plt.plot(x, g, ls='-.');
```



在这里，我创建了两个高斯函数，分别为 g1= $\mathcal{N}(0.8, 0.1)$ 和 g2= $\mathcal{N}(1.3, 0.2)$ ，然后把它们绘制出来。接着，我将这两个高斯函数相乘，再对结果进行归一化处理。如图所示，最终得到的结果呈现出高斯分布的形态。

高斯函数属于非线性函数。通常情况下，当对非线性方程进行相乘运算时，得到的结果会是另一种类型的函数。例如，两个正弦函数的乘积，其形状与 `sin(x)` 完全不同。

```
x = np.arange(0, 4*np.pi, 0.01)
plt.plot(np.sin(1.2*x))
plt.plot(np.sin(1.2*x) * np.sin(2*x));
```



但是，两个高斯分布相乘的结果仍然是一个高斯函数。这也是卡尔曼滤波器在计算上可行的关键原因。换句话说，卡尔曼滤波器之所以使用高斯分布，是因为高斯分布在计算上更为简便。

两个高斯概率密度的乘积，经归一化后可表示为：

$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

两个独立高斯随机变量的和可表示为：

$$\mu = \mu_1 + \mu_2$$
$$\sigma^2 = \sigma_1^2 + \sigma_2^2$$

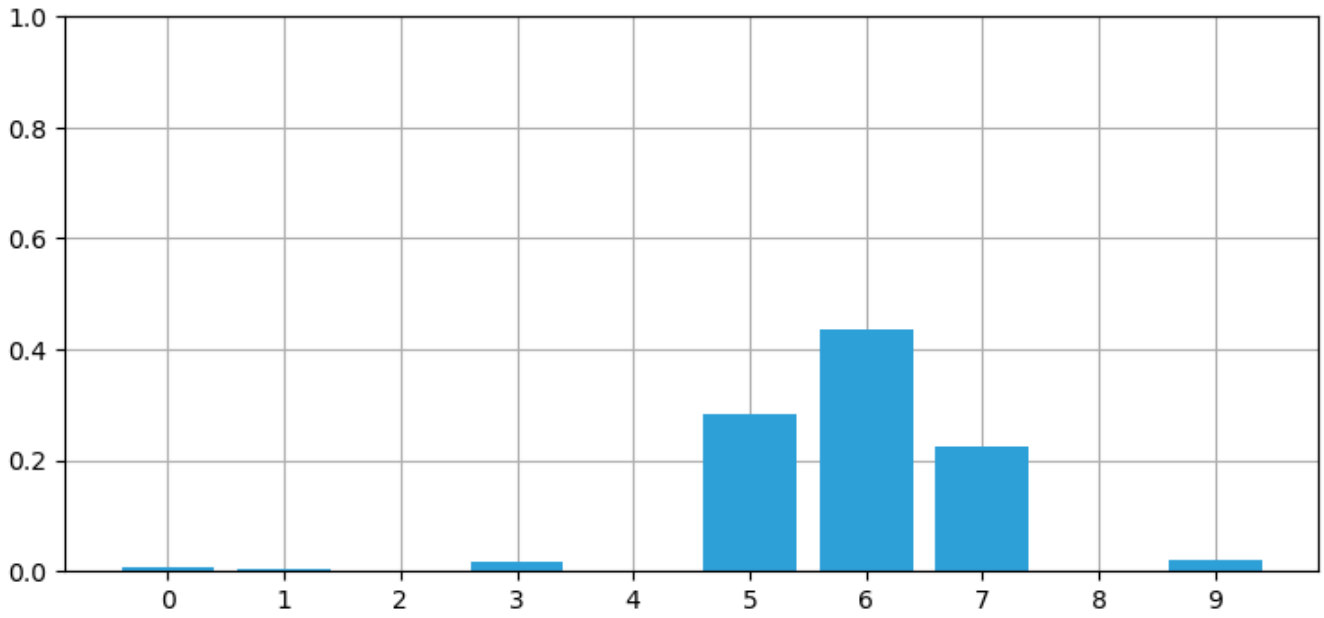
在本章的末尾，我推导出了这些方程式。不过，理解这些推导过程其实并不那么重要。

综合应用

现在，我们准备来探讨如何利用高斯函数来进行滤波处理。在下一章中，我们将实际实现一个基于高斯函数的滤波器。在此，我会先解释一下，为什么我们要使用高斯函数来进行滤波处理。

在上一章中，我们用数组来表示概率分布。为了更新概率分布，我们需要将该分布与另一个表示各点上测量结果可能性的分布进行逐元素相乘，具体操作如下：

```
def normalize(p):
    return p / sum(p)

def update(likelihood, prior):
    return normalize(likelihood * prior)

prior =      normalize(np.array([4, 2, 0, 7, 2, 12, 35, 20, 3, 2]))
likelihood = normalize(np.array([3, 4, 1, 4, 2, 38, 20, 18, 1, 16]))
posterior = update(likelihood, prior)
book_plots.bar_plot(posterior)
```

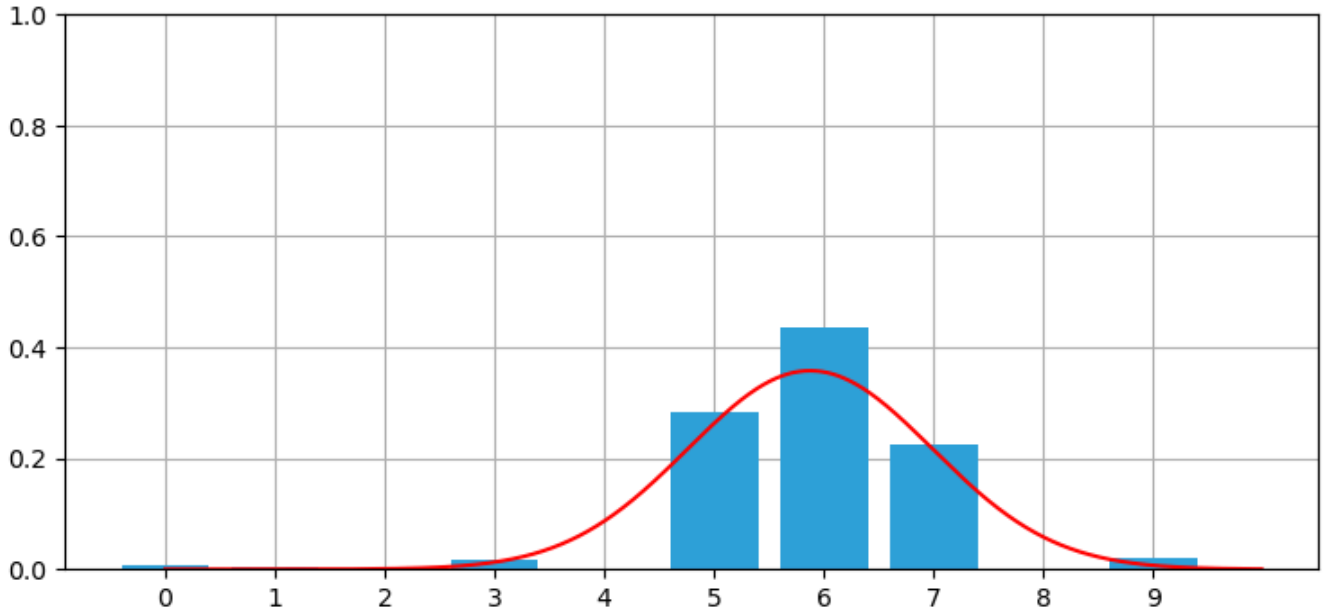


换句话说，为了得到这个结果，我们必须进行 10 次乘法运算。而对于那些具有多维大规模数组的实时滤波器来说，所需的乘法运算次数将达到数十亿次，同时还需要大量的内存来存储数据。

不过，这种分布看起来更像是高斯分布。如果我们用高斯分布来代替原来的数组结构会怎么样呢？我会计算后验分布的均值和方差，然后把结果与条形图一起展示出来。

```
xs = np.arange(0, 10, .01)

def mean_var(p):
    x = np.arange(len(p))
    mean = np.sum(p * x,dtype=float)
    var = np.sum((x - mean)**2 * p)
    return mean, var

mean, var = mean_var(posterior)
book_plots.bar_plot(posterior)
plt.plot(xs, gaussian(xs, mean, var, normed=False), c='r');
print('mean: %.2f' % mean, 'var: %.2f' % var)
```

```
mean: 5.88 var: 1.24
```



这个结果很有启发：如果分布可以合理地用高斯分布近似，就只需均值和方差两个数描述这一近似。这里只有 10 个数，优势还不明显；真实问题中，原来的数组可能有数百万项。这一简化依赖高斯近似，不能用两个参数精确表示任意形状的分布。

接下来，请记住，我们的滤波器是通过以下方式来执行更新功能的：

```
def update(likelihood, prior):
return normalize(likelihood * prior)
```

如果数组中包含一百万个元素，那就需要进行一百万次乘法运算。不过，如果我们用高斯函数来替代这些数组，那么所需的计算次数就会大大减少。

$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

也就是三次乘法和两次除法。

贝叶斯定理

在上一章中，我们通过分析某一时刻所掌握的信息来构建算法，这些信息被表示为离散概率分布。在这个过程中，我们发现了贝叶斯定理。贝叶斯定理告诉我们，如何根据先验信息来计算某个事件的概率。

我们通过这种概率计算方式来实现 `update()` 函数：

$$\mathbf{posterior} = \frac{\mathbf{likelihood} \times \mathbf{prior}}{\mathbf{normalization}}$$

原来，这就是贝叶斯定理。稍后我会详细阐述其中的数学原理，但这么做反而可能掩盖这个公式所表达的简单含义。我们可以这样理解这个公式：

$$updated\ knowledge = \|likelihood\ of\ new\ knowledge \times prior\ knowledge\|$$

其中，$\|\cdot\|$ 表示对该项进行归一化处理。

我们是通过关于“狗在走廊里行走”的简单推理得出了这个结论。不过，正如我们将会看到的，同样的原理也适用于各种滤波问题。在后面的每一章中，我们都会用到这个原理。

回顾：先验是纳入本次测量之前的状态概率分布；似然描述给定状态时得到该测量的可能性；后验是结合先验与测量信息并归一化后得到的状态分布。

贝叶斯定理是

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

$P(A \mid B)$ 称为条件概率，即在 B 已发生的条件下，A 发生的概率。例如，如果昨天下雨，今天下雨的可能性会高于通常的一天，因为降雨天气系统通常持续不止一天。这个概率可写成 P（今天下雨 | 昨天下雨）。

还有一个重要区别：上面的程序处理的是一整组概率，也就是概率分布，而不只是单个事件概率。刚写出的贝叶斯定理针对事件概率，但它同样可以用相应的概率质量函数或密度形式表达。这里用小写 p 表示这些函数。

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)}$$

在上述公式中，B 代表证据，$p(A)$ 代表先验概率，$p(B \mid A)$ 代表似然值，$p(A \mid B)$ 则代表后验概率。将这些数学符号对应的含义代入后，可以看出贝叶斯定理其实与我们所说的“更新公式”是一致的。让我们用与问题相关的术语来重新表述这个公式。我们用 x_i 表示某一点的位置，用 z 表示在该点的测量结果。因此，我们想要知道的是 $P(x_i \mid z)$，也就是在得到测量结果 z 的条件下，狗位于 x_i 的概率。

那么，让我们把这个值代入方程中，然后求解吧。

$$p(x_i \mid z) = \frac{p(z \mid x_i) p(x_i)}{p(z)}$$

右边各项其实并不复杂。首先，$p(z \mid x_i)$ 是似然，即假定狗位于各个候选位置 x_i 时，得到本次测量的概率。$p(x_i)$ 是先验，表示纳入这次测量之前的位置判断。将二者相乘，就是 update() 中尚未归一化的乘法：

```
def update(likelihood, prior):
posterior = prior * likelihood # p(z|x) * p(x)
return normalize(posterior)
```

最后考虑分母 $p(z)$。它是对所有可能位置按先验加权后，得到测量 z 的边缘概率，常称为证据或边缘似然。代码中，对尚未归一化的乘积向量 x 求和，即 sum(belief)，得到归一化常数。不是对原先已归一化的先验直接求和；后者恒为 1。update() 正是在计算这一贝叶斯更新。

在文献中，这些方程通常以积分的形式呈现。毕竟，积分本质上就是对连续函数进行求和。因此，你可能会看到贝叶斯定理被写成这样的形式：

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{\int p(B \mid A_j) p(A_j)\, \mathrm{d}A_j}.$$

分母中的积分通常没有容易求出的解析解，即便能够解析求解，过程也可能十分复杂。一篇英国皇家统计学会的评论文章把它形容为“一团乱麻”[8]。不要只因看到积分就望而却步：在前面的有限离散例子中，我们已用求和归一化处理了相同的问题。“粒子滤波”一章还会介绍其他近似方法。阅读公式时，不妨先问：为什么要累加这些量？为什么要除以这一项？将积分理解为连续情形的加权累积，往往有助于看清含义，但这并不意味着一般高维积分或归一化计算都容易。

贝叶斯定理的力量或许还未完全显现。我们希望求 $p(x_i \mid Z)$，即给定测量之后各个候选状态的后验概率。一般情况下，这并不容易。相同形式还可用于根据癌症检查结果判断患癌概率，或根据传感器读数判断下雨概率。直接计算这些后验，乍看可能无从下手。

但贝叶斯定理允许我们通过反向的条件概率 $p(Z \mid x_i)$ 来计算它，而后者通常更容易求出：

$$p(x_i \mid Z) \propto p(Z \mid x_i)\, p(x_i)$$

也就是说，要计算给定传感器读数时下雨的后验概率，可以先求“已经下雨时，出现这些读数的似然”，再结合下雨的先验并归一化。通常这个条件方向更容易建模。天气预测当然仍然困难，但贝叶斯定理把问题组织成了可处理的形式。

同样地，正如你在“离散贝叶斯”那一章中所看到的，我们是通过计算：当西蒙位于位置 x 时，各个传感器所检测到的数值出现的概率，从而来确定西蒙处于走廊上某个特定位置的概率。这样一来，原本复杂的问题就变得简单了。

全概率定理

update() 背后的数学已明确，那么 predict() 呢？它应用全概率定理。时刻 t 位于位置 i 的概率记为 $P(X_i^t)$。计算时，对上一时刻 $t-1$ 的每个候选位置，将其概率 $P(X_j^{t-1})$ 乘以从 x_j 转移到 x_i 的概率，再对所有起点求和。不能漏掉最后这一步求和。

$$P(X_i^t) = \sum_j P(X_j^{t-1}) P(x_i|x_j)$$

那个公式被称为“全概率定理”。引用维基百科[6]的说法：“它描述了可以通过多个不同事件来实现的结果的总概率。”我本可以直接把那个公式告诉你，并演示其计算过程，但你理解该公式为何有效的概率其实很低。作为提醒，以下是用于计算该公式的代码。

```
for i in range(N):
for k in range (kN):
index = (i + (width-k) - offset) % N
result[i] += prob_dist[index] * kernel[k]
```

使用 scipy.stats 计算概率

本章使用 FilterPy 的代码计算和绘制高斯分布，以便读者查看实现。不过，Python 的科学计算生态还提供 scipy.stats，其中包含丰富的统计函数。下面介绍如何用它计算统计量与概率。SciPy 是第三方库，不是 Python 标准库的内置模块。

`scipy.stats` 模块包含了许多可用于计算各种概率分布特性的函数。该模块的完整文档请参见：http://docs.scipy.org/doc/scipy/reference/stats.html。我们重点关注 norm 这个函数，它用于实现正态分布的计算。下面来看一段使用 `scipy.stats.norm` 来计算高斯分布的代码，并将其结果与 FilterPy 中的 `gaussian()` 函数所返回的结果进行比较。

```
from scipy.stats import norm
import filterpy.stats
print(norm(2, 3).pdf(1.5))
print(filterpy.stats.gaussian(x=1.5, mean=2, var=3*3))
```

```
0.13114657203397997
0.13114657203397995
```

norm(2, 3) 这个调用会创建出 SciPy 中所说的“冻结分布”（frozen distribution），即参数已固定的分布。它会生成一个均值为 2、标准差为 3 的对象。之后，你可以多次使用这个对象来获取不同数值对应的概率密度，具体操作如下：

```
n23 = norm(2, 3)
print('pdf of 1.5 is       %.4f' % n23.pdf(1.5))
print('pdf of 2.5 is also  %.4f' % n23.pdf(2.5))
print('pdf of 2 is         %.4f' % n23.pdf(2))
```

```
pdf of 1.5 is       0.1311
pdf of 2.5 is also  0.1311
pdf of 2 is         0.1330
```

[2]中的文档还列出了许多其他功能。例如，我们可以利用该函数从该分布中生成 n 个样本。

```
np.set_printoptions(precision=3, linewidth=50)
print(n23.rvs(size=15))
```

```
[ 2.269  2.485 -0.469  1.991  2.468  1.667  1.231
  4.577  2.56   0.224 -3.851  2.604  3.583 -1.623
  2.348]
```

还可以计算累积分布函数（CDF），即从该分布中抽取的随机变量小于或等于 x 的概率。

```
# probability that a random value is less than the mean 2
print(n23.cdf(2))
```

```
0.5
```

我们可以了解该分布的多种特性：

```
print('variance is', n23.var())
print('standard deviation is', n23.std())
print('mean is', n23.mean())
```

```
variance is 9.0
standard deviation is 3.0
mean is 2.0
```

用高斯函数来模拟世界的局限性

前面提到中心极限定理。一个经典版本说明：在独立同分布、有限均值和有限正方差等条件下，随机变量之和经过适当中心化和尺度归一化，其分布随项数增大而趋近标准正态分布。它不要求单个变量本身正态，但也不意味着任意数量、任意独立变量的和都严格正态。自然界许多扰动由大量小因素叠加，这使高斯近似常有用；其适用条件仍须判断。

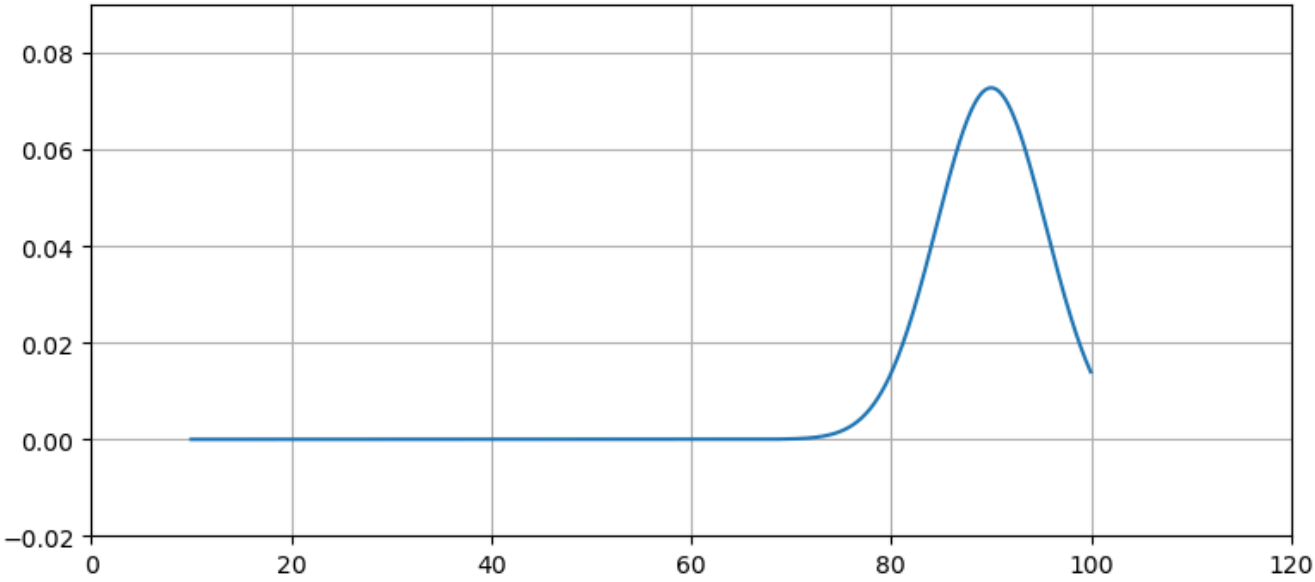
关键在于“在某些条件下”。真实系统未必满足这些理想化条件。例如，实际厨房秤的显示值可能被限制为非负；高斯测量模型的尾部却延伸到负无穷，会给负读数分配非零概率。这就与这种秤的实际输出范围不同。

这是一个范围很广的主题，我无法面面俱到地加以阐述。

考虑一个简单例子。我们常把考试成绩等事物视为正态分布。如果教授曾经按曲线调整你的成绩，你就接触过这一假设。但考试成绩当然不可能严格服从正态分布，因为正态分布对任意远离均值的有限区间，都赋予非零概率。例如，均值为 90、标准差为 13 时，得 90 分的可能性很大，得 40 分的可能性较小；但模型还意味着得 -10 分或 150 分也有极小可能。它甚至给 -10^{300} 或 10^{32986} 附近的分数赋予极其微小的概率。高斯分布的尾部无限延伸。

但在测试中，情况并非如此。如果不考虑额外加分的话，成绩不可能低于 0 分，也不可能高于 100 分。让我们用正态分布来描绘这个数值范围，从而看看这种模型在描述实际测试成绩分布时的准确性有多差。

```
xs = np.arange(10, 100, 0.05)
ys = [gaussian(x, 90, 30) for x in xs]
plt.plot(xs, ys, label='var=0.2')
plt.xlim(0, 120)
plt.ylim(-0.02, 0.09);
```



如果仅保留正态曲线在 0—100 分范围内的部分，其面积小于 1，必须重新归一化才能构成合法的截断概率密度。实际成绩分布还可能因满分上限、评分规则等在高分端堆积，或在某些位置聚集，因此未必接近原先的高斯模型。

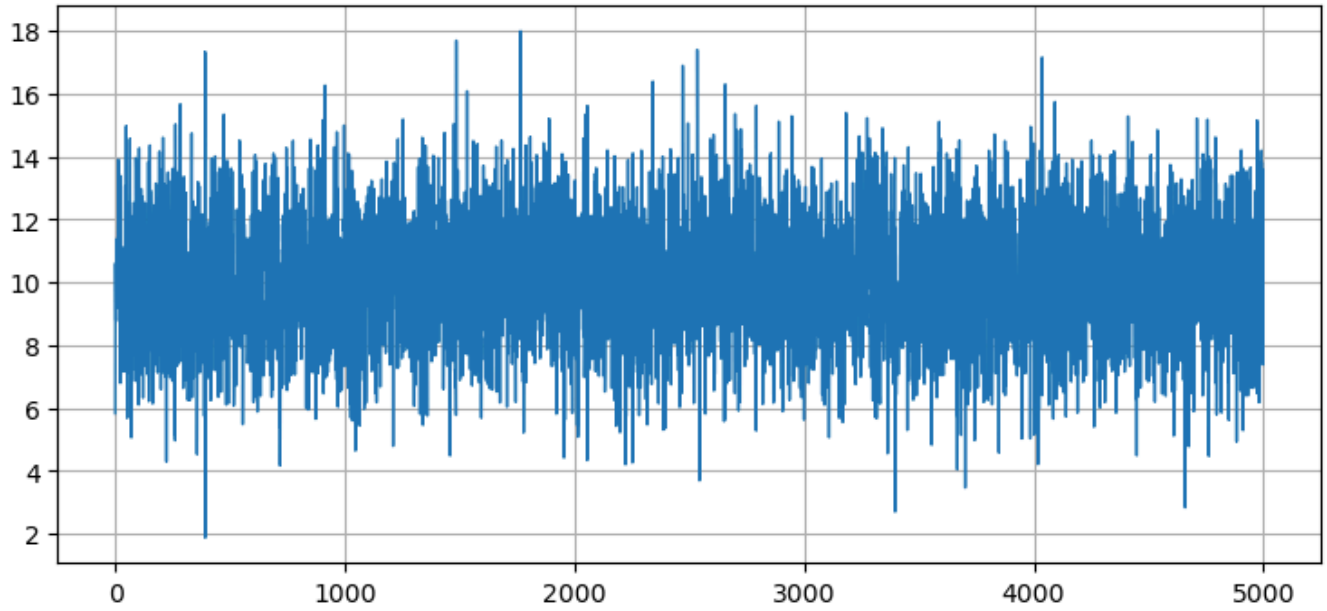
传感器测量现实世界，而测量误差很少真正服从高斯分布。现在讨论这会给卡尔曼滤波设计者带来哪些困难还为时过早，但请记住：卡尔曼滤波的数学建立在理想化的世界模型之上。这里先介绍一段后文会使用的代码，用它构造分布，以模拟不同过程与传感器。这种分布称为 Student t 分布。

假设我想要对一个输出中包含白噪声的传感器进行建模。为简化问题，假设该信号的值为恒定的 10，而噪声的标准差为 2。我们可以使用 `numpy.random.randn()` 函数来生成均值为 0、标准差为 1 的随机数。我可以通过以下方式来模拟这一过程：

```
from numpy.random import randn
def sense():
    return 10 + randn()*2
```

让我们把那个信号画出来，看看它到底是什么样子的。

```
zs = [sense() for i in range(5000)]
plt.plot(zs, lw=1);
```

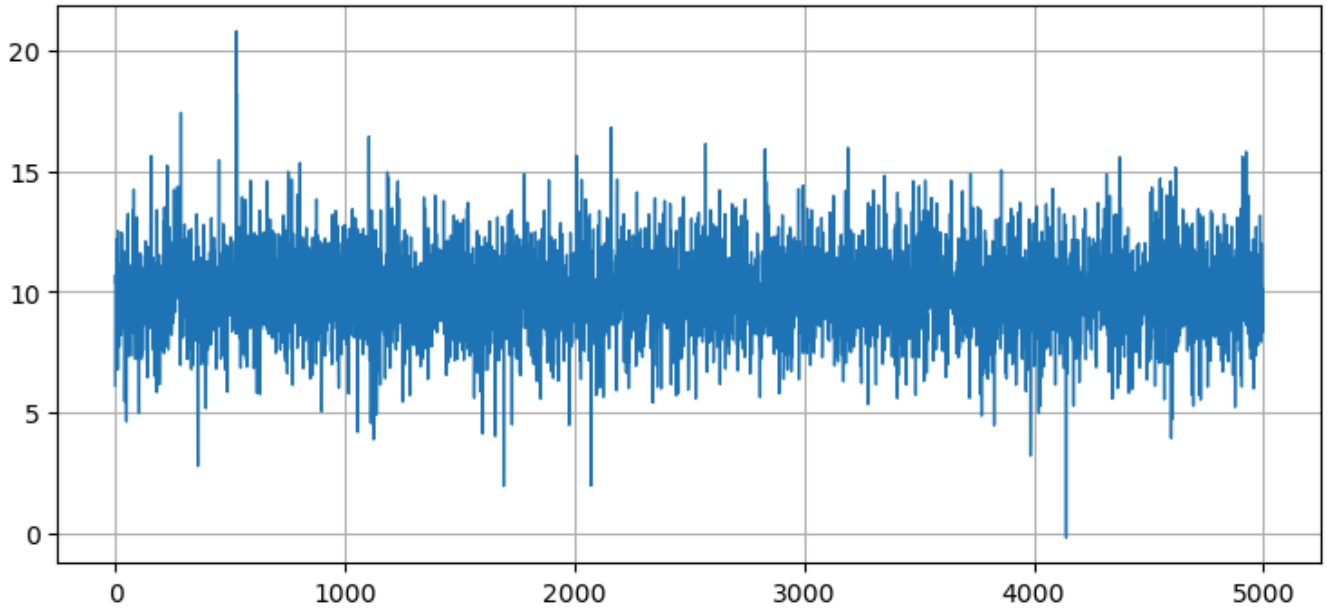


这看起来符合预期：信号以 10 为中心，标准差为 2，意味着约 68% 的测量值位于 10 ± 2 内，约 99.7% 位于 10 ± 6 内，图中也大致如此。

现在看看 Student t 分布生成的数据。这里不展开数学推导，只给出源代码，再用它画出一个分布。

```
import random
import math

def rand_student_t(df, mu=0, std=1):
    """return random number distributed by Student's t
    distribution with `df` degrees of freedom with the
    specified mean and standard deviation.
    """
    x = random.gauss(0, std)
    y = 2.0*random.gammavariate(0.5*df, 2.0)
    return x / (math.sqrt(y / df)) + mu
```

```
def sense_t():
    return 10 + rand_student_t(7)*2

zs = [sense_t() for i in range(5000)]
plt.plot(zs, lw=1);
```



从图中可以看到，整体形状与正态分布相似，但存在偏离中心较远的取值，表现出较重的尾部。这里 t 型生成函数中的 std 是尺度参数，不能直接当作最终分布的标准差，因此不能直接套用正态分布的三倍标准差覆盖率来解释它。

Student t 分布未必能精确描述 GPS、多普勒传感器等设备的误差，本书也不是物理系统建模教材。但这类重尾模拟数据有助于测试滤波器面对偏离高斯假设的噪声时的表现。后续仿真和测试还会使用类似分布。

这并非毫无根据的担忧。卡尔曼滤波方程假设噪声呈正态分布，如果这一假设不成立，滤波效果就会大打折扣。对于那些对任务成败至关重要的滤波器来说，比如航天器上的滤波器，设计人员必须充分了解有关传感器性能的各种理论和实际数据。例如，我在 NASA 的一个项目报告中看到，虽然从理论上讲，应该使用 3 个标准差来区分噪声和有效测量值，但实际上他们不得不使用 5 到 6 个标准差才能达到理想效果。这是通过实验得出的结论。

rand_student_t 的实现包含在 filterpy.stats 中，可按下面方式导入：

```
from filterpy.stats import rand_student_t
```

统计学还有其他描述分布形状的指标。正态分布关于均值对称；偏度衡量分布的不对称性。峰度与标准化四阶矩有关，反映尾部等形状特征；常用的超额峰度以正态分布为零基准。scipy.stats.describe 可以计算这些统计量。

```
import scipy
scipy.stats.describe(zs)
```

```
DescribeResult(nobs=5000, minmax=(-0.19143741858944452, 20.804030812682086),
mean=9.996901102331591, variance=2.7388116003856884, skewness=0.015276194207011526,
kurtosis=1.622567142788295)
```

让我们考察从正态分布中抽取的两组样本：一组样本量较小，另一组较大。

```
print(scipy.stats.describe(np.random.randn(10)))
print()
print(scipy.stats.describe(np.random.randn(300000)))
```

```
DescribeResult(nobs=10, minmax=(-1.2037202953168122, 0.894366248899801),
mean=-0.18007426896541692, variance=0.43204122396032946, skewness=-0.08295117992362264,
kurtosis=-0.9197042651911951)

DescribeResult(nobs=300000, minmax=(-4.75206637131407, 4.178833173976851),
mean=-0.0004743681087834309 6, variance=0.9973760542573228, skewness=-0.0038868831924059035,
kurtosis=-0.005797621180390955)
```

小样本得到的偏度与超额峰度可能明显偏离零，均值和方差也可能偏离理论值 0 与 1。大样本的这些统计量通常更接近理论值。这里 describe 默认报告超额峰度：正态总体的超额峰度为 0，普通峰度为 3，两者不要混淆。

高斯函数的乘积（可选）

阅读这一部分并不重要。在这里，我推导了两个高斯函数相乘时的公式。

把两个高斯密度函数相乘并合并各项，就可以得到这个结果，但代数运算比较繁琐。这里改用贝叶斯定理来推导。问题可以表述为：设先验分布为 $N(\bar{\mu}, \bar{\sigma}^2)$，测量模型为 $z \mid x \sim \mathcal{N}(x, \sigma_z^2)$。给定测量结果 z 后，状态 x 的后验分布是什么？

将后验概率表示为 $p(x \mid z)$。现在，我们可以利用贝叶斯定理来进一步推导了。

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)}$$

$p(z)$ 是归一化常数，因此可以写出下面的正比关系：

$$p(x \mid z) \propto p(z|x)p(x)$$

现在，我们将高斯函数的方程代入其中，这些方程如下：

$$p(z \mid x) = \frac{1}{\sqrt{2\pi\sigma_z^2}} \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2}\Big]$$

$$p(x) = \frac{1}{\sqrt{2\pi\bar{\sigma}^2}} \exp\Big[-\frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big]$$

我们可以忽略那些常数项，这样就能得到：

$$\begin{aligned}
p(x \mid z) &\propto \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2}\Big] \exp\Big[-\frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big] \\
&\propto \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2} - \frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big] \\
&\propto \exp\Big[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}[\bar{\sigma}^2(z-x)^2 + \sigma_z^2(x-\bar{\mu})^2]\Big]
\end{aligned}$$

展开平方项，再按照状态变量 x 的幂次合并各项。

$$
\begin{aligned}
p(x \mid z) &\propto \exp\left[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}\left[\bar{\sigma}^2(z^2-2xz+x^2)+\sigma_z^2(x^2-2x\bar{\mu}+\bar{\mu}^2)\right]\right] \\
&\propto \exp\left[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}\left[x^2(\bar{\sigma}^2+\sigma_z^2)-2x(\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z)+(\bar{\sigma}^2 z^2+\sigma_z^2\bar{\mu}^2)\right]\right]
\end{aligned}
$$

最后一个括号不含状态变量 x，因此相对于它是常数，可以并入尚未确定的归一化系数。

$$
p(x \mid z) \propto \exp\left[-\frac{1}{2}\frac{x^2(\bar{\sigma}^2+\sigma_z^2)-2x(\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z)}{\sigma_z^2\bar{\sigma}^2}\right]
$$

将分子和分母同时除以 $\bar{\sigma}^2+\sigma_z^2$，即可得到结果。

$$
p(x \mid z) \propto \exp\left[-\frac{1}{2}\frac{x^2-2x(\frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2})}{\frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}}\right]
$$

我们讨论的是正比关系，因此可以把与 x 无关的非零因子吸收到比例常数中，据此整理为：

$$
p(x \mid z) \propto \exp\left[-\frac{1}{2}\frac{(x-\frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2})^2}{\frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}}\right]
$$

回顾高斯分布的形式：

$$
N(\mu,\sigma^2) \propto \exp\left[-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right]
$$

由此可见，$p(x \mid z)$ 的平均值为

$$
\mu_{\text{posterior}} = \frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2}
$$

后验方差为：

$$
\sigma^2_{\text{posterior}} = \frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}
$$

前面省略了常数，因此当前表达式不是已经归一化的正态密度，而是与某个正态密度成正比。贝叶斯公式中的 $p(z)$ 充当归一化分母，使密度积分为 1。滤波测量更新也需要这种归一化；这里得到高斯形式，是因为前面指数项已具有相应的二次形式，并非任意函数一经归一化就都会变成高斯。

$$
\mathcal{N}_1 = \|\mathcal{N}_2 \cdot \mathcal{N}_3\|
$$

独立高斯随机变量的和（可选）

本节推导两个独立高斯随机变量之和的分布。结果是：它们的和仍服从高斯分布，均值相加，方差相加。

该数值由以下方式得出：

$$\mu = \mu_1 + \mu_2$$
$$\sigma^2 = \sigma_1^2 + \sigma_2^2$$

关于这一点，有几种证明方法。由于在上一章中，我们已经在概率直方图的计算中使用了卷积运算，因此在这里我也会继续使用卷积运算来证明这一点。

设两个独立高斯随机变量为 p 和 z，它们的和记为 x。要得到 x 的概率密度，需要对各个可能分解取值下的密度乘积积分，即计算卷积，而不是把两个密度函数逐点相加。

$$p(x) = \int_{-\infty}^{\infty} f_p(x-z) f_z(z)\, dz$$

将两个高斯密度代入卷积公式，再对指数项配方。为缩短表达式，将两方差之和记为 S，将配方后关于积分变量的高斯密度的均值和方差分别记为 m、v，如下所示。

$$p(x) = \int_{-\infty}^{\infty} f_p(x-x_1) f_z(x_1)\, dx_1$$
$$p(x) = \frac{1}{2\pi\sigma_p\sigma_z} \int_{-\infty}^{\infty} \exp\left[-\frac{(x-z-\mu_p)^2}{2\sigma_p^2} - \frac{(z-\mu_z)^2}{2\sigma_z^2}\right] dz$$

$$S = \sigma_p^2 + \sigma_z^2, \qquad v = \frac{\sigma_p^2\sigma_z^2}{S},$$
$$m = \frac{\sigma_z^2(x-\mu_p) + \sigma_p^2\mu_z}{S},$$
$$p(x) = \int_{-\infty}^{\infty} \frac{\exp\left[-\frac{(x-\mu_p-\mu_z)^2}{2S}\right]}{\sqrt{2\pi S}} \frac{\exp\left[-\frac{(z-m)^2}{2v}\right]}{\sqrt{2\pi v}}\, dz$$

$$p(x) = \frac{\exp\left[-\frac{(x-\mu_p-\mu_z)^2}{2S}\right]}{\sqrt{2\pi S}} \int_{-\infty}^{\infty} \frac{\exp\left[-\frac{(z-m)^2}{2v}\right]}{\sqrt{2\pi v}}\, dz$$

最后一个积分中的函数，是关于积分变量 z 的归一化正态密度，其均值为 m、方差为 v，因此在整个实数轴上的积分为 1。留下的因子就是和的概率密度：

$$p(x) = \frac{1}{\sqrt{2\pi(\sigma_p^2 + \sigma_z^2)}} \exp\left[-\frac{(x-\mu_p-\mu_z)^2}{2(\sigma_p^2+\sigma_z^2)}\right]$$

这是正态分布的形式，其中：

$$\mu_x = \mu_p + \mu_z$$
$$\sigma_x^2 = \sigma_z^2 + \sigma_p^2 \square$$

总结与要点

这一章对统计学的介绍相当不够全面。我所讲述的内容仅仅是为了在本书的后续部分中使用高斯分布而必需的概念而已。如果你打算深入研究卡尔曼滤波的相关知识，那么我所讲的内容远远不够用。如果你是第一次接触统计学，我建议你阅读专门的统计学教材。我个人很喜欢 Schaum 系列的教材，Allen B. Downey 所著的《Think Stats》[5]也是一本非常好的书，而且可以在网上免费阅读。

在继续之前，你必须先理解以下几点：

- 正态分布体现了连续的概率分布特性。
- 它们完全由两个参数来描述：均值（μ）和方差（σ^2）。
- μ 是可能取值按其概率加权的平均。
- 方差 σ^2 反映了我们的测量结果与平均值的偏离程度。
- 标准差（σ）是方差（σ^2）的平方根。
- 自然界中的许多现象都近似于正态分布，但这种数学模型并不完美。
- 在滤波问题中，计算 $p(x \mid z)$ 几乎是不可能的，而计算 $p(z \mid x)$ 则相对简单。贝叶斯定理让我们能够利用 $p(z \mid x)$ 来计算 $p(x \mid z)$。

在接下来的几章中，我们将利用高斯分布和贝叶斯定理来帮助进行数据滤波。正如上一节所提到的，有时候高斯分布并不能很好地描述现实世界的情况。本书的后半部分将专门介绍那些即使在噪声或系统行为极不符合高斯分布的情况下也能有效工作的滤波算法。

参考文献

[1] https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/Supporting_Notebooks/Computing_and_plotting_PDFs.ipynb

[2] http://docs.scipy.org.cn/doc/scipy/reference/generated/scipy.stats.norm.html

[3] http://docs.scipy.org.cn/doc/scipy/reference/tutorial/stats.html

[4] 休伯，彼得·J.《稳健统计方法》，第二版。工业与应用数学学会，1996 年。

[5] Allen B. Downey 著。《Think Stats》第二版。O’Reilly Media 出版社。

https://github.com/AllenDowney/ThinkStats2

http://greenteapress.com/thinkstats/

有用的维基百科链接

https://en.wikipedia.org/wiki/Probability_distribution

https://en.wikipedia.org/wiki/Random_variable

https://en.wikipedia.org/wiki/Sample_space

https://en.wikipedia.org/wiki/Central_tendency

https://en.wikipedia.org/wiki/Expected_value

https://en.wikipedia.org/wiki/Standard_deviation

https://en.wikipedia.org/wiki/Variance

https://en.wikipedia.org/wiki/Probability_density_function

https://en.wikipedia.org/wiki/Central_limit_theorem

https://en.wikipedia.org/wiki/68%E2%80%9395%E2%80%9399.7_rule

https://en.wikipedia.org/wiki/Cumulative_distribution_function

https://en.wikipedia.org/wiki/Skewness

https://en.wikipedia.org/wiki/Kurtosis

第 4 章：一维卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

既然我们已经了解了离散贝叶斯滤波器和高斯分布，那么现在就可以开始实现卡尔曼滤波器了。我们的实现方式将与处理离散贝叶斯滤波器时完全相同：不会从方程式开始，而是通过逐步分析问题来编写代码。

本章“一维”表示只估计一个状态变量，例如 x 轴位置。后续会学习更一般的多维形式，能够同时估计位置、速度和加速度等多个变量。回忆 g-h 滤波器：把速度纳入模型，比只估计位置能获得更好的结果。卡尔曼滤波也是如此。

那么，为什么不直接采用多维形式的滤波器呢？说实话，相关的数学计算相当复杂，我试图用直观的方法来设计滤波器，但这种方法已经不再适用了。这些复杂的数学公式掩盖了卡尔曼滤波器运作所依据的那些简单原理。

本章学习如何用高斯分布实现贝叶斯滤波器。卡尔曼滤波器正是一种利用高斯分布的贝叶斯滤波器。下一章将转向多维形式，展现它的完整能力。

问题描述

与在“离散贝叶斯滤波器”那一章中类似，我们在这里也要追踪工作场所里走廊中移动的物体。假设在最近的黑客马拉松活动中，有人开发出了一个 RFID 追踪器，能够相当准确地确定狗的位置。该传感器会反馈狗距离走廊左端的距离，单位为米。因此，23.4 米的数值意味着狗距离走廊左端 23.4 米远。

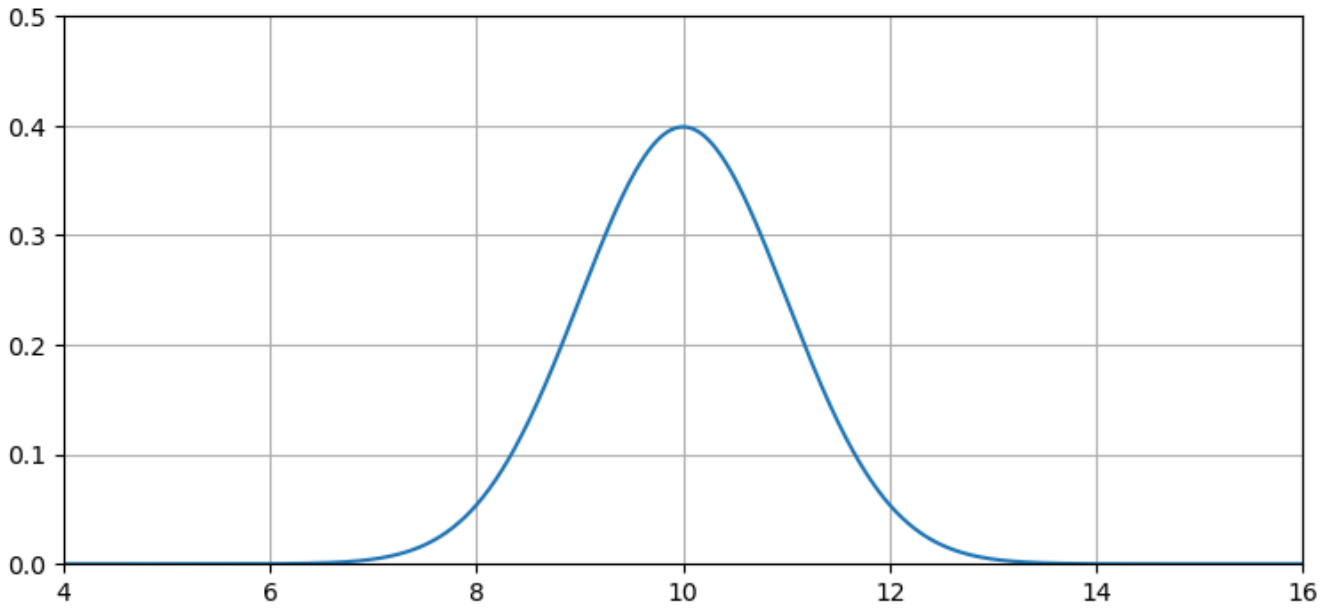
传感器并不完美。读数为 23.4 米时，狗的真实位置可能是 23.7 或 23.0 米，但不大可能远至 47.6 米。测试表明，传感器相当准确，误差较小；误差在真值两侧大致对称，例如真实位置为 23 米时，读到 22.9 与 23.1 米的可能性相近。这里说的是两侧对称，不是误差服从均匀分布。或许可以采用高斯误差模型。

我们预测这只狗正在移动。不过，这种预测并不完美。有时候，我们的预测会偏高；有时候则偏低。通常情况下，预测值会稍微偏高或偏低一点，而不会偏差太大。或许，我们可以用高斯函数来模拟这一现象。

将信念视为高斯分布

可以用高斯分布表达我们对狗的位置的信念。例如，认为狗位于 10 米处，而这一判断的方差为 $1\ \mathrm{m}^2$，即 $\mathcal{N}(10, 1)$。其概率密度图如下。

```
import filterpy.stats as stats
stats.plot_gaussian_pdf(mean=10., variance=1.,
                        xlim=(4, 16), ylim=(0, .5));
```

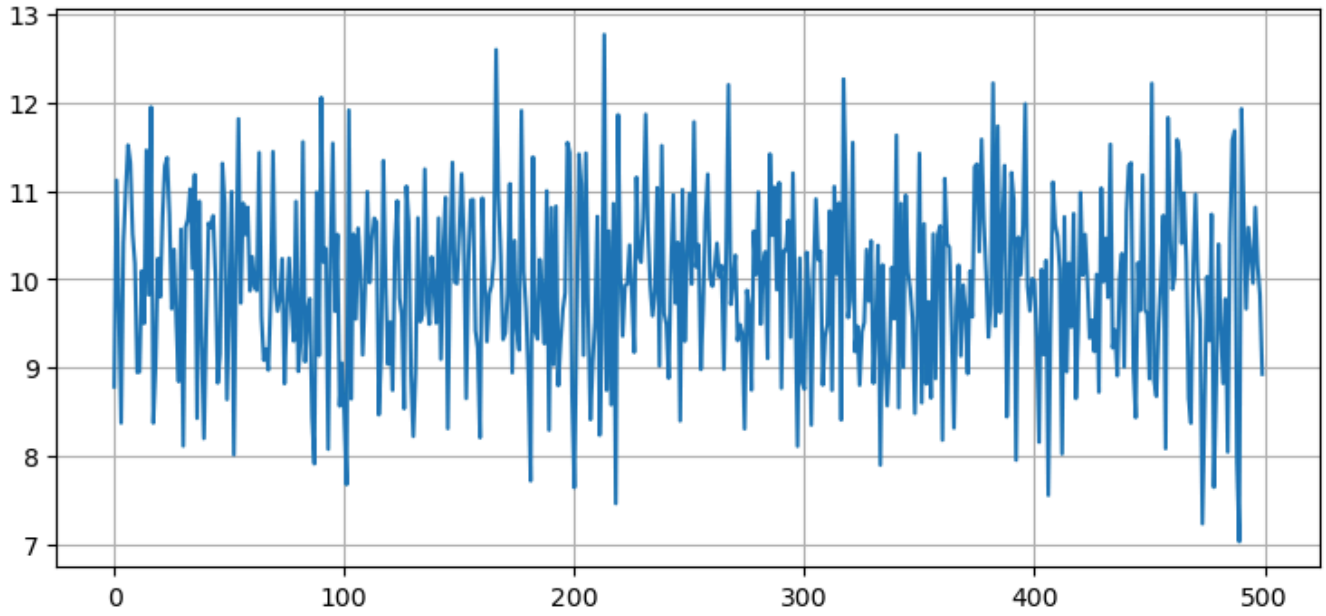


该图表反映了我们对狗所在位置的不确定性。这其实是一种相当不精确的估计。虽然我们认为狗很可能位于 10 米处，但 9 米到 11 米之间的任何位置也都有可能。假设狗静止不动，我们再次询问传感器。这次，传感器显示的距离是 10.2 米。那么，我们能否利用这一额外信息来更准确地估计狗的位置呢?

直觉告诉我们，我们是可以这么做的。试想一下：如果我们读取传感器的数值 500 次，而每次得到的数值都在 8 到 12 之间，且数值都接近 10，那么我们就有充分的理由相信，那只狗确实就在 10 附近。当然，也有可能有其他解释。也许那只狗只是随机地来回移动，其运动轨迹恰好符合正态分布的随机抽样规律。但这种可能性极低——我从未见过狗会有这样的行为。让我们来看看 $\mathcal{N}(10,1)$ 的 500 次测量结果吧:

```
import numpy as np
from numpy.random import randn
import matplotlib.pyplot as plt

xs = range(500)
ys = randn(500)*1. + 10.
plt.plot(xs, ys)
print(f'Mean of readings is {np.mean(ys):.3f}')
```

```
Mean of readings is 9.936
```



目测证实了直觉：狗不会这样运动，但带噪传感器数据完全可能如此。读数均值几乎正好为 10。假设狗静止不动，可以把位置建模为均值 10、方差 1 的分布。

基于高斯概率的跟踪算法

这种离散贝叶斯滤波器利用概率直方图来追踪狗的位置。直方图中的每个区间代表一个位置，而该区间的数值则代表了狗处于该位置的概率。

跟踪过程是通过一系列的预测和更新来实现的。我们使用了相应的数学公式来进行计算。

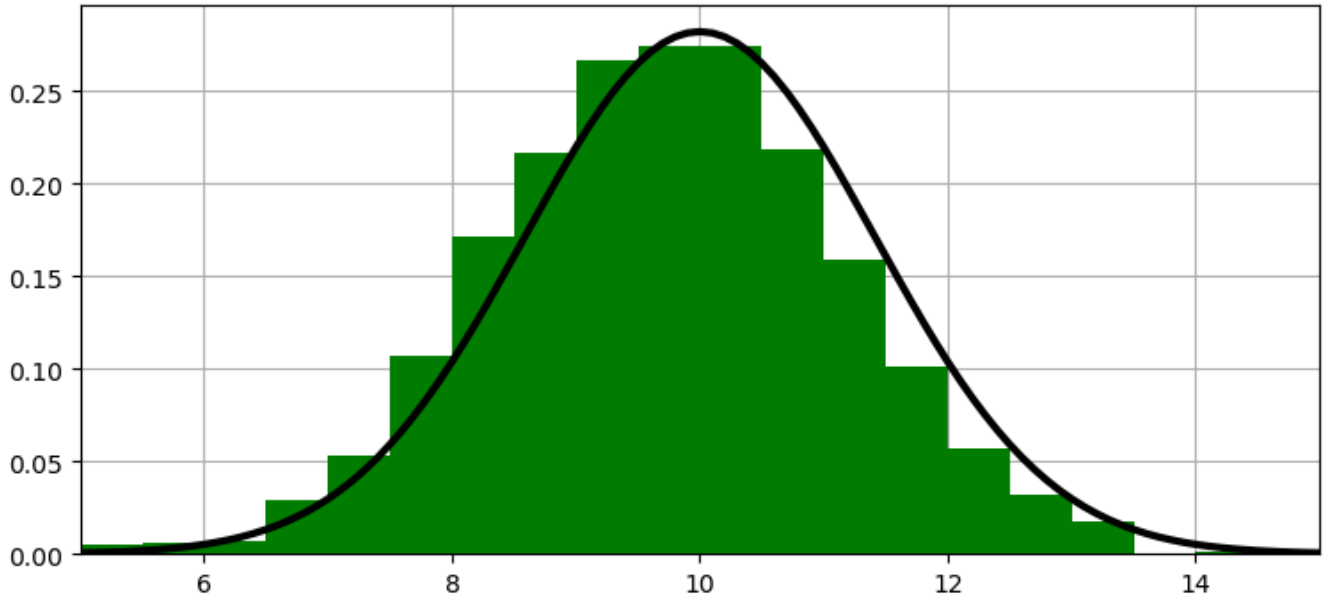
$$\begin{aligned} \bar{\mathbf{x}} &= \mathbf{x} * f_{\mathbf{x}}(\bullet) && \textbf{Predict} \\ \mathbf{x} &= \mathcal{L} \cdot \bar{\mathbf{x}} && \textbf{Update} \end{aligned}$$

以此计算新的概率分布。$\bar{\mathbf{x}}$ 是先验，$\mathcal{L}$ 是本次测量的似然，用来与先验 $\bar{\mathbf{x}}$ 结合；$f_{\mathbf{x}}(\bullet)$ 是过程模型，$*$ 表示卷积。$\mathbf{x}$ 表示一组直方图数值，即向量。

这种方法有效，但直方图可能对几个彼此分离的位置同时赋予较高概率，表示多个候选判断，而不是说狗在物理上同时位于几个地方。对于大规模问题，计算还会很慢。

我们能把直方图 $\mathbf{x}$ 替换为高斯分布 $\mathcal{N}(x, \sigma^2)$ 吗？当然可以！我们已经知道如何用高斯分布来表示概率。高斯分布其实只是一个数字对 $\mathcal{N}(\mu, \sigma^2)$,，它完全可以替代整个概率直方图。

```
import kf_book.kf_internal as kf_internal
kf_internal.gaussian_vs_histogram()
```



希望你能明白其中的妙处。我们可以用一对数字 $x = \mathcal{N}(\mu, \sigma^2)$ 来替代成百上千个数字。

高斯分布的两条尾部都延伸到无穷远，因此它相当于包含了任意多个直方图柱。若用它表示狗在走廊中的位置，这一个分布就覆盖了整条走廊，甚至覆盖该坐标轴上的整个实数范围。我们认为狗很可能在 10 附近，但也可能在 8 或 14 附近；甚至在 10^{80} 附近，模型也给出极其微小的概率。

在本章中，我们用高斯分布来替代直方图：

discrete Bayes	Gaussian	Step
$\bar{\mathbf{x}} = \mathbf{x} * f(\mathbf{x})$	$\bar{x}_\mathcal{N} = x_\mathcal{N} \oplus f_{x_\mathcal{N}}(\bullet)$	Predict
$\mathbf{x} = \|\mathcal{L}\bar{\mathbf{x}}\|$	$x_\mathcal{N} = L \otimes \bar{x}_\mathcal{N}$	Update

其中，$\oplus$ 和 $\otimes$ 表示对高斯函数施加的某种未知运算。在本书的其余部分，我不会再使用这种表示方法。不过，下标表明 $x_\mathcal{N}$ 确实是一个高斯函数。

离散贝叶斯滤波器在预测中使用卷积。根据全概率定理，卷积给出两个独立随机变量之和的分布，因此可以尝试用独立高斯随机变量相加来表示预测。测量更新则把似然与先验相乘，所以也可以尝试将高斯密度函数相乘。真的会这么简单吗？

$$\bar{x} \overset{?}{=} x + f_x(\bullet)$$
$$x \overset{?}{=} \mathcal{L} \cdot \bar{x}$$

这一方案要求：预测时独立高斯随机变量相加仍为高斯，更新时高斯密度相乘并归一化后仍为高斯。否则第一个时间步之后，x 就不再属于高斯分布，这种二参数表示便无法继续保持。

高斯模型预测结果

我们利用牛顿运动定律，根据当前速度和先前的位置来计算出当前位置。

$$\begin{aligned}\bar{x}_k &= x_{k-1} + v_k\Delta t \\ &= x_{k-1} + f_x\end{aligned}$$

为了使方程更简洁明了，我不再使用 $f_x(\bullet)$ 这个符号，而改用 f_x 。

如果狗位于 10 米处，其速度为 15 米/秒，且时间间隔为 2 秒，那么我们可以得到：

$$\begin{aligned}f_x &= v\Delta t = 15 \cdot 2 \\ \bar{x}_k &= 10 + (15 \cdot 2) = 40\end{aligned}$$

我们无法确定他目前的所在位置和速度，因此这种表示方式不可行。我们需要用高斯函数来表示这种不确定性。

定位其实很简单。我们将 x 定义为高斯分布。如果我们认为狗位于 10 米处，而我们的不确定性标准差为 0.2 米，那么就能得出 $x = \mathcal{N}(10, 0.2^2)$ 的值。

如何表示对狗的运动的不确定性？将一步位移 f_x 建模为高斯分布。若速度为 15 米/秒，时间间隔为 1 秒，速度不确定性的标准差为 0.7 米/秒，就得到 $f_x = \mathcal{N}(15, 0.7^2)$。这里每一步恰为 1 秒，因此位移均值和标准差以米计时，与速度参数的数值相同。

先验概率的公式为：

$$\bar{x} = x + f_x$$

两个独立高斯随机变量之和服从什么分布？上一章已给出：

$$\begin{aligned}\mu &= \mu_1 + \mu_2 \\ \sigma^2 &= \sigma_1^2 + \sigma_2^2\end{aligned}$$

这是一个非常有用的性质：两个独立高斯随机变量之和仍服从高斯分布！

从数学上来说，这个公式是成立的。但是，从直观上来看，这真的合理吗？试着想象一下这个抽象方程在现实世界中的体现吧。我们有……

$$\begin{aligned}x &= \mathcal{N}(10, 0.2^2) \\ f_x &= \mathcal{N}(15, 0.7^2)\end{aligned}$$

如果把它们加起来，就得到：

$$\begin{aligned}\bar{x} &= \mu_x + \mu_{f_x} = 10 + 15 &&= 25 \\ \bar{\sigma}^2 &= \sigma_x^2 + \sigma_{f_x}^2 = 0.2^2 + 0.7^2 &&= 0.53\end{aligned}$$

预测位置等于先前位置加上位移，直觉上很清楚。方差的变化稍难理解。回忆离散贝叶斯滤波的 predict()：运动存在不确定性时，预测后的分布会扩散。这里的 $\sigma_{f_x}^2$ 刻画位移预测带来的额外不确定性，因此要把它加到已有的位置方差上。

利用 Python 的 collections 模块提供的 namedtuple，构造一个描述高斯分布的对象。也可以直接用普通元组：例如 $\mathcal{N}(10, 0.04)$ 对应 g=(10.,0.04)，用 g[0] 访问均值，g[1] 访问方差。

namedtuple 的用法与元组类似，只不过需要指定类型名称和各个字段的名称。虽然理解这一点并非必要，但我修改了 __repr__ 方法，使其能按照本章所介绍的格式来显示其值。

```
from collections import namedtuple
gaussian = namedtuple('Gaussian', ['mean', 'var'])
gaussian.__repr__ = lambda s: f'𝒩(μ={s[0]:.3f}, σ²={s[1]:.3f})'
```

现在可以创建并打印一个高斯分布参数对象：

```
g1 = gaussian(3.4, 10.1)
g2 = gaussian(mean=4.5, var=0.2**2)
print(g1)
print(g2)
```

```
𝒩(μ=3.400, σ²=10.100)
𝒩(μ=4.500, σ²=0.040)
```

我们可以通过下标或字段名称来获取均值和方差：

```
g1.mean, g1[0], g1[1], g1.var
```

```
(3.4, 3.4, 10.1, 10.1)
```

以下是我们对“predict”函数的实现方式。其中， `pos` 和 `movement` 都是高斯元组，其形式为(μ , σ^2)。

```
def predict(pos, movement):
    return gaussian(pos.mean + movement.mean, pos.var + movement.var)
```

我们来测试一下。如果初始位置服从高斯分布 $\mathcal{N}(10, 0.2^2)$ ，而运动过程也服从高斯分布 $\mathcal{N}(15, 0.7^2)$ ，那么先验概率是多少呢？

```
pos = gaussian(10., .2**2)
move = gaussian(15., .7**2)
predict(pos, move)
```

```
𝒩(μ=25.000, σ²=0.530)
```

先验位置分布的均值为 25 米，方差为 0.53 m^2，与手工计算结果一致。

高斯函数的更新情况

离散贝叶斯滤波把位置判断编码为概率直方图。这是离散分布，并且可能多峰：它可以同时给两个位置较高概率，表示两种候选位置，而非断言狗同时位于两个地方。

我们建议用高斯分布来替代直方图。所使用的离散贝叶斯滤波器正是利用这一公式来计算后验概率的。

```
def update(likelihood, prior):
posterior = likelihood * prior
return normalize(posterior)
```

这是该方程式的具体实现方式：

$$x = \|\mathcal{L}\bar{x}\|$$

我们刚刚证明了可以用高斯分布来表示先验概率。那么，似然呢？似然指的是在当前状态下，进行某次测量的概率。我们已经知道如何用高斯分布来表示各种测量结果。例如，假设我们的传感器检测到狗位于 23 米处，标准差为 0.4 米。那么，用似然来表示的话，就是 $z = \mathcal{N}(23, 0.16)$ 。

似然和先验都用高斯分布建模。高斯分布能相乘吗？两个高斯密度相乘并归一化之后，仍然是高斯密度吗？

前者当然成立，后者也几乎成立！在上一章中，我已经证明：两个高斯函数的乘积与另一个高斯函数成正比。

$$\begin{aligned}\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2},\\ \sigma^2 &= \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}\end{aligned}$$

可以立即推断：乘积归一化后得到另一个高斯分布。一项代表似然，另一项代表先验；融合后的均值是两者均值的加权和，方差由两项方差共同确定。在当前的线性高斯公式中，更新后的方差不取决于输入均值。

我们可以用贝叶斯术语来表达这一点：

$$\begin{aligned}\mathcal{N}(\mu, \sigma^2) &= \|prior \cdot likelihood\| \\ &= \|\mathcal{N}(\bar{\mu}, \bar{\sigma}^2) \cdot \mathcal{N}(\mu_z, \sigma_z^2)\| \\ &= \mathcal{N}(\frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}, \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2})\end{aligned}$$

如果我们在函数 `gaussian_multiply()` 中实现这一点，那么就可以将滤波器的更新步骤也一并实现出来。

```
def gaussian_multiply(g1, g2):
    mean = (g1.var * g2.mean + g2.var * g1.mean) / (g1.var + g2.var)
    variance = (g1.var * g2.var) / (g1.var + g2.var)
    return gaussian(mean, variance)

def update(prior, likelihood):
    posterior = gaussian_multiply(likelihood, prior)
    return posterior

# test the update function
predicted_pos = gaussian(10., .2**2)
measured_pos = gaussian(11., .1**2)
estimated_pos = update(predicted_pos, measured_pos)
estimated_pos
```

```
𝒩(μ=10.800, σ²=0.008)
```

如果我们使用更具体的名称来表述，或许会更容易理解：

```
def update_dog(dog_pos, measurement):
estimated_pos = gaussian_multiply(measurement, dog_pos)
return estimated_pos
```

这样的名称不那么抽象，也许有助于理解，却不是理想的编程方式。我们希望滤波器能够用于各种问题，而不只用于跟踪走廊里的狗，因此不应在通用变量名中写入 dog。此外，这种写法掩盖了运算本质：将似然与先验相乘。

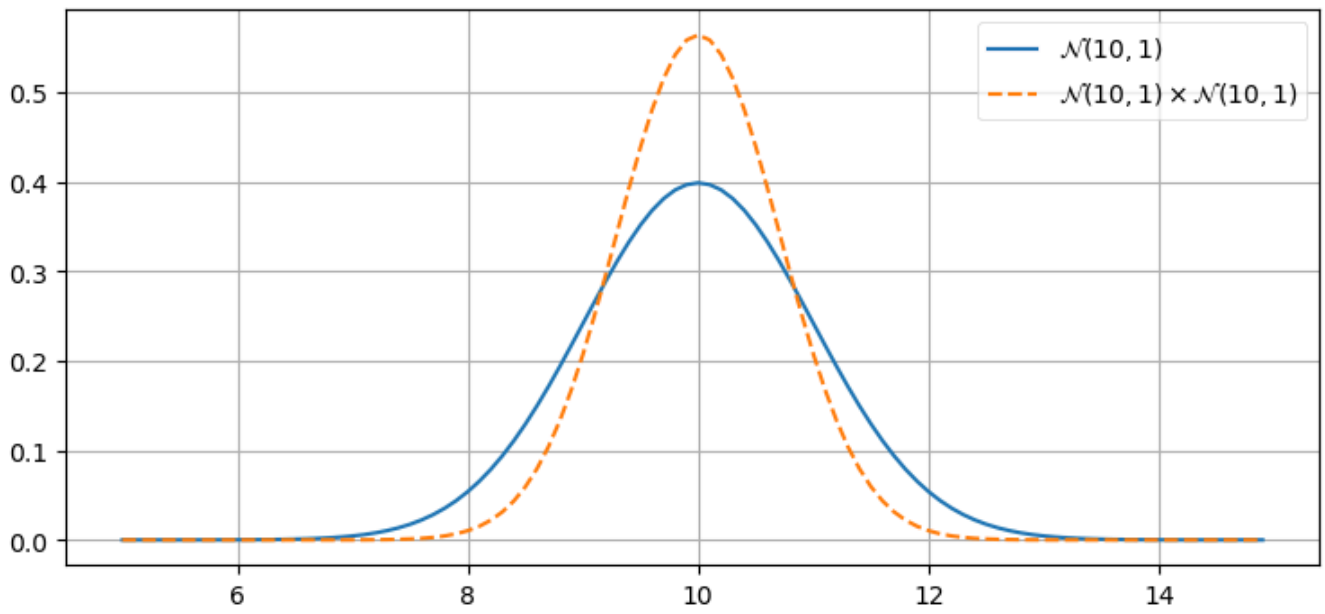
我们已经实现了大部分的滤波功能，但我觉得这一步还是有点复杂。我确认过，高斯函数是可以相乘的，而且这种操作确实能够正确地完成更新步骤。但为什么会是这样呢？让我们花点时间来仔细研究一下高斯函数的相乘运算吧。

理解高斯乘法运算

让我们来绘制 $\mathcal{N}(10, 1) \times \mathcal{N}(10, 1)$ 的概率密度函数吧。不用查看结果，你能判断出它的形状吗？新的均值应该是多少呢？这条曲线会比 $\mathcal{N}(10, 1)$ 的更宽、更窄，还是保持不变呢？

```
z = gaussian(10., 1.)  # Gaussian N(10, 1)

product = gaussian_multiply(z, z)

xs = np.arange(5, 15, 0.1)
ys = [stats.gaussian(x, z.mean, z.var) for x in xs]
plt.plot(xs, ys, label='$\mathcal{N}(10,1)$')

ys = [stats.gaussian(x, product.mean, product.var) for x in xs]
plt.plot(xs, ys, label='$\mathcal{N}(10,1) \\times \mathcal{N}(10,1)$', ls='--')
plt.legend()
print(product)
```

```
𝒩(μ=10.000, σ²=0.500)
```



乘法运算后的结果比原始的高斯分布更细长，但均值保持不变。这一结果符合你的直觉吗?

可以把高斯分布看作是两次测量结果。如果我进行了两次测量，每次得到的数值都是 10 米，那么我应该认为该长度接近 10 米。因此，平均值应该是 10 米。认为该长度实际上是 11 米或 9.5 米是没有道理的。此外，两次测量的结果比一次测量的结果更可靠，所以测量结果的方差应该更小。

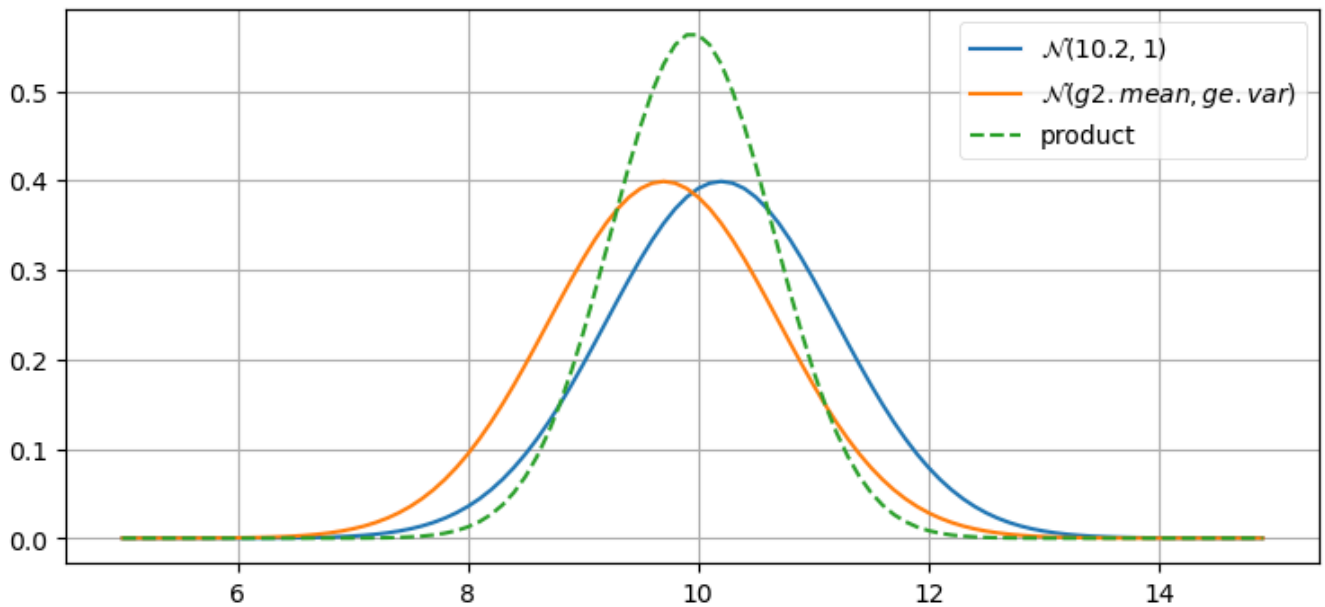
“量两次，切一次”是一句常见的木工格言。高斯信息融合为重复测量能够提升确定性的做法提供了数学描述。

我不太可能连续两次得到相同的测量结果。现在，让我们来绘制 $\mathcal{N}(10.2,\ 1) \times \mathcal{N}(9.7,\ 1)$ 的概率密度函数吧。你认为结果会怎样呢? 先好好思考一下，然后再看看图表吧。

```
m = 3.3
f'$({m},{m})$'
```

```
'$(3.3,3.3)$'
```

```
def plot_products(g1, g2):
    plt.figure()
    product = gaussian_multiply(g1, g2)

    xs = np.arange(5, 15, 0.1)
    ys = [stats.gaussian(x, g1.mean, g1.var) for x in xs]
    plt.plot(xs, ys, label='$\mathcal{N}$' + f'$({g1.mean},{g1.var})$')

    ys = [stats.gaussian(x, g2.mean, g2.var) for x in xs]
    plt.plot(xs, ys, label='$\mathcal{N}$' + '$({g2.mean},{ge.var})$')

    ys = [stats.gaussian(x, product.mean, product.var) for x in xs]
    plt.plot(xs, ys, label='product', ls='--')
    plt.legend();
    plt.show()

z1 = gaussian(10.2, 1)
z2 = gaussian(9.7, 1)

plot_products(z1, z2)
```



如果你让两个人来测量桌子到墙的距离，其中一个人测得 10.2 米，另一个人测得 9.7 米，那么如果你对两人的测量能力都相当信任的话，最合理的估计应该是两者的平均值，即 9.95 米。

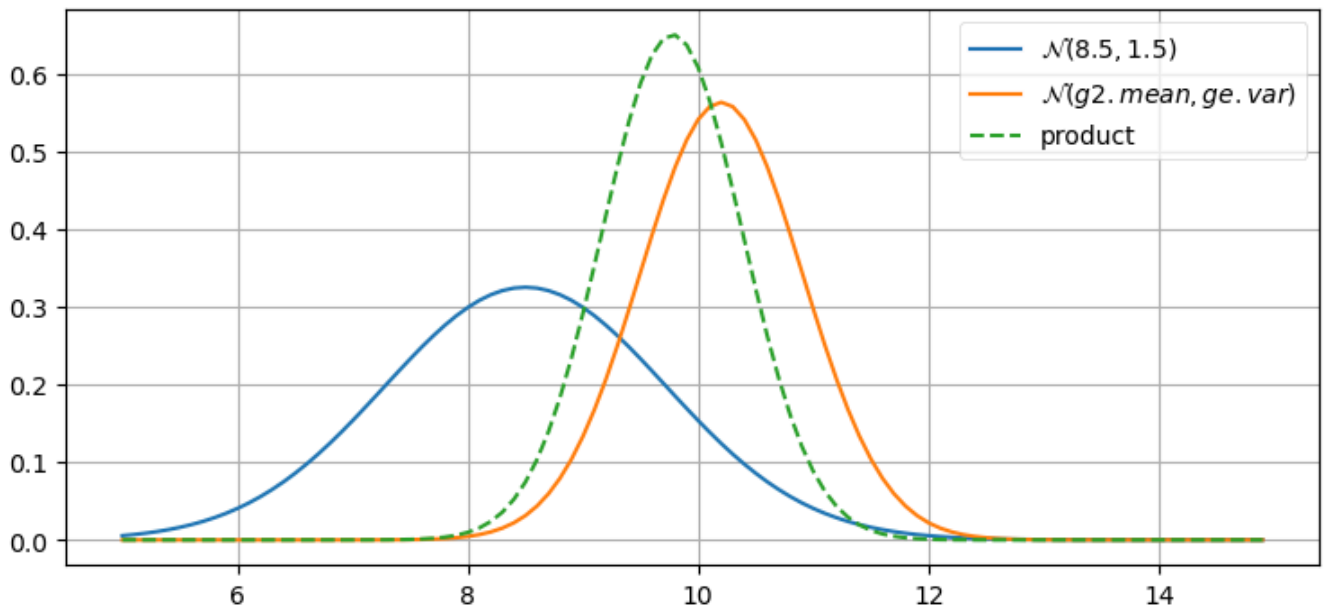
回忆 g-h 滤波中的两台秤：一台读 160 磅，另一台读 170 磅，若二者精度相同，融合估计为 165 磅。在所用的独立、无偏误差模型下，两次测量共同支持这个估计，因此比只使用一次测量更有把握。

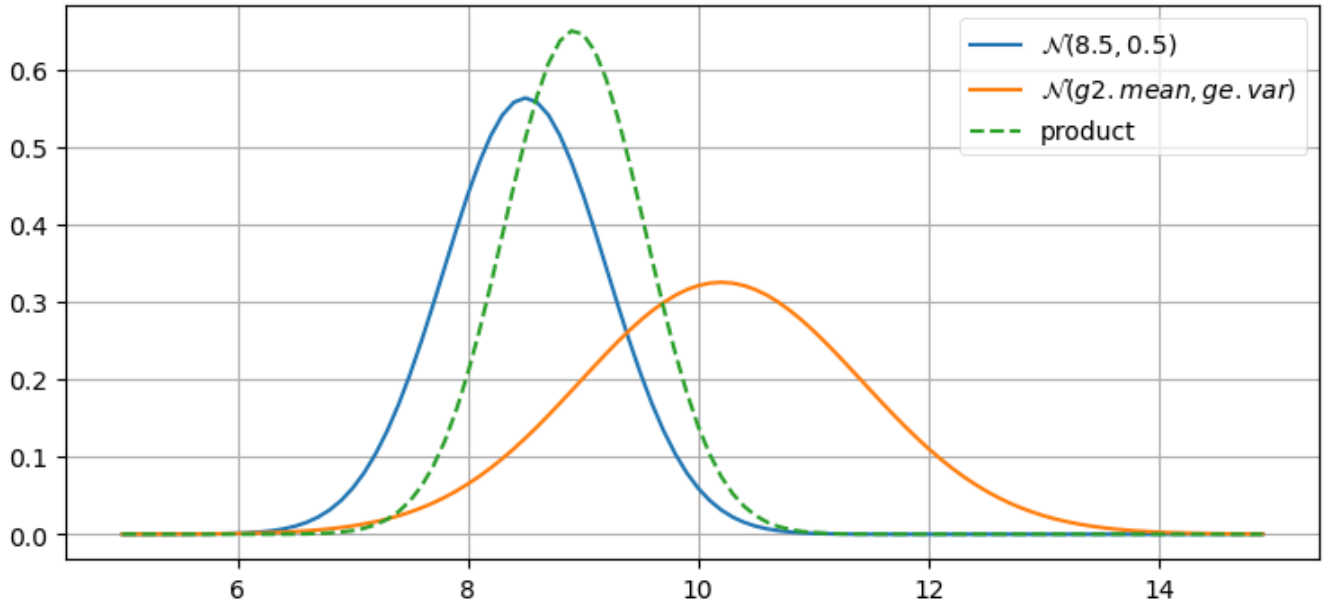
在更复杂的情况下，这种推理方式就显得不合常理了。让我们进一步思考一下。或许，更合理的假设是：其中一个人犯了错误，所以真实的距离应该是 10.2 或 9.7，而不可能是 9.95。当然，这种可能性是存在的。但是，由于我们的测量结果都存在误差，我们没有理由认为其中某个测量结果是完全准确的，或者认为某个人犯了严重的错误，从而可以忽略他的测量结果。根据所有可用的信息，最合理的估计值应该是 9.95。

在卡尔曼滤波的更新步骤中，融合的是一次测量与先验，即纳入这次测量之前的估计，而不是简单把两次测量相加。g-h 滤波器中已经讨论过这一逻辑。无论融合两次测量，还是融合一次测量与一次预测，其所用的高斯信息融合数学关系相同。

先取较不确定的先验 $\mathcal{N}(8.5, 1.5)$，以及更确定的测量似然 $\mathcal{N}(10.2, 0.5)$.。这里“更准确”表示测量噪声方差比先验方差小，不表示我已经知道真实位置更接近 10.2 而不是 8.5。接着交换不确定程度：使用更确定的先验 $\mathcal{N}(8.5, 0.5)$，与较不确定的测量 $\mathcal{N}(10.2, 1.5)$，比较融合结果。

```
prior, z = gaussian(8.5, 1.5), gaussian(10.2, 0.5)
plot_products(prior, z)

prior, z = gaussian(8.5, 0.5), gaussian(10.2, 1.5)
plot_products(prior, z)
```





融合后的高斯密度曲线，其峰值比两条输入曲线都高。这符合直觉：纳入了更多信息，方差因而减小。还可以看到，融合后的均值更接近方差较小的那个输入均值。我们对这一输入更有信心，因此赋予它更高权重是合理的。

看起来这个方法确实有效，但是否真的正确呢？关于这一点还有更多需要探讨的。不过，我现在想先让这个滤波器能够正常运行起来，这样你们就能亲自体验它的效果了。之后，我们再重新探讨高斯乘法的原理，弄清楚为什么这种方法是正确的。

交互式示例

这段交互式代码提供了滑块，可用于调整两个高斯函数的均值和方差。当你移动滑块时，图表会随之重新绘制。请将光标置于代码单元格内，然后按 CTRL+Enter 键来执行该代码。

```
from ipywidgets import interact

def interactive_gaussian(m1, m2, v1, v2):
    g1 = gaussian(m1, v1)
    g2 = gaussian(m2, v2)
    plot_products(g1, g2)

interact(interactive_gaussian,
         m1=(5, 10., .5), m2=(10, 15, .5),
         v1=(.1, 2, .1), v2=(.1, 2, .1));
```

```
interactive(children=(FloatSlider(value=7.5, description='m1', max=10.0, min=5.0, step=0.5),
FloatSlider(value…
```

卡尔曼滤波器初探

让我们回到具体的实现上，来运用卡尔曼滤波器吧。我们已经实现了 `update()` 和 `predict()` 这两个函数。现在只需要编写一些通用代码来模拟狗的行为并生成相应的测量数据即可。我把 `DogSimulation` 类放在了 `kf_internal` 中，这样就不会因为处理这个任务而分心了。

这段样板代码通过定义各种参数、计算方差以及生成模拟的狗的移动轨迹，从而确立了整个问题的框架。

```
import kf_book.kf_internal as kf_internal
from kf_book.kf_internal import DogSimulation

np.random.seed(13)

process_var = 1. # variance in the dog's movement
sensor_var = 2. # variance in the sensor

x = gaussian(0., 20.**2)  # dog's position, N(0, 20**2)
velocity = 1
dt = 1. # time step in seconds
process_model = gaussian(velocity*dt, process_var) # displacement to add to x

# simulate dog and get measurements
dog = DogSimulation(
    x0=x.mean,
    velocity=process_model.mean,
    measurement_var=sensor_var,
    process_var=process_model.var)

# create list of measurements
zs = [dog.move_and_sense() for _ in range(10)]
```

这就是卡尔曼滤波器。

```
print('PREDICT\t\t\tUPDATE')
print('     x      var\t\t  z\t    x      var')

# perform Kalman filter on measurement z
for z in zs:
    prior = predict(x, process_model)
    likelihood = gaussian(z, sensor_var)
    x = update(prior, likelihood)

    kf_internal.print_gh(prior, x, z)

print()
print(f'final estimate:        {x.mean:10.3f}')
print(f'actual final position: {dog.x:10.3f}')
```

```
PREDICT         UPDATE
     x      var       z     x      var
  1.000  401.000    1.354     1.352   1.990
  2.352    2.990    1.882     2.070   1.198
  3.070    2.198    4.341     3.736   1.047
  4.736    2.047    7.156     5.960   1.012
  6.960    2.012    6.939     6.949   1.003
  7.949    2.003    6.844     7.396   1.001
  8.396    2.001    9.847     9.122   1.000
 10.122    2.000    12.553   11.338   1.000
 12.338    2.000    16.273   14.305   1.000
 15.305    2.000    14.800   15.053   1.000

final estimate:            15.053
actual final position:     14.838
```

以下是该滤波器的动画演示。预测结果以红色三角形表示。在每次预测之后，滤波器会接收下一个测量值，该测量值以黑色圆圈表示。随后，滤波器会在这两个数值之间得出一个估算值。

```
from kf_book import book_plots as book_plots
from ipywidgets.widgets import IntSlider
```

```
# save output in these lists for plotting
xs, predictions = [], []

process_model = gaussian(velocity, process_var)

# perform Kalman filter
x = gaussian(0., 20.**2)
for z in zs:
    prior = predict(x, process_model)
    likelihood = gaussian(z, sensor_var)
    x = update(prior, likelihood)

    # save results
    predictions.append(prior.mean)
    xs.append(x.mean)

def plot_filter(step):
    plt.cla()
    step -= 1
    i = step // 3 + 1

    book_plots.plot_predictions(predictions[:i])
    if step % 3 == 0:
        book_plots.plot_measurements(zs[:i-1])
        book_plots.plot_filter(xs[:i-1])
    elif step % 3 == 1:
        book_plots.plot_measurements(zs[:i])
        book_plots.plot_filter(xs[:i-1])
    else:
        book_plots.plot_measurements(zs[:i])
        book_plots.plot_filter(xs[:i])

    plt.xlim(-1, 10)
    plt.ylim(0, 20)
    plt.legend(loc=2);
    plt.show()

interact(plot_filter, step=IntSlider(value=1, min=1, max=len(predictions)*3));
```

```
interactive(children=(IntSlider(value=1, description='step', max=30, min=1), Output()),
_dom_classes=('widget-…
```

图中画出了先验（标为 prediction）、测量值与滤波输出。每次循环都先形成先验，获取一次测量，根据测量建立似然，再将似然与先验结合得到后验。

从图表中可以看出，滤波后的估计值始终位于测量值和预测值之间。回想一下，对于 g-h 滤波器来说，我们曾经指出，估计值必须始终处于测量值和先验值之间。选择超出这两个数值所界定范围的估计值是没有意义的。如果我预测自己处于 10 的位置，但实际测量结果却是 9，那么认为自己应该处于 8 或 11 的位置就显得很荒谬了。

代码详解

现在让我们来仔细看看这段代码。

```
process_var = 1.
sensor_var = 2.
```

这些是过程模型和传感器的方差。传感器方差的含义很直接：每次测量有多大的离散程度。过程方差则表示过程模型存在多少误差。我们预测狗在每个时间步向前走一米，但狗很少完全按预期行动，山坡或松鼠的气味都可能改变它的进度。如果是按数字指令行动的机器人，表现通常会好得多，方差也许可以取 $\sigma^2 = .05$。这些并非随意的“魔法数字”；方差的平方根以米为单位，刻画距离误差的典型尺度。随便填入数字，就可能让卡尔曼滤波运行起来，但数字若不符合现实，性能就会很差。

```
x = gaussian(0., 20.**2)
```

这是用高斯分布表示狗的初始位置。位置为 0 米，方差为 400 m^2，对应标准差 20 米。可以把它理解为：“以约 99.7% 的概率，位置在 0 米上下 60 米的范围内。”因为高斯分布约有 99.7% 的取值落在均值的 $\pm 3\sigma$ 范围内。

```
process_model = gaussian(velocity, process_var)
```

这就是所谓的“过程模型”——也就是我们对狗的移动方式所做出的描述。那我怎么知道狗的速度呢？这有什么神奇的吗？其实可以把这看作是一种预测而已。或者，也许我们还有别的方式来测量速度。如果这是一台机器人，那么这些数据就可以作为机器人的控制输入信号。在后面的章节中，我们将学习如何处理没有速度传感器或相关输入信号的情况。所以，请暂时接受这种简化后的解释吧。

接下来，我们初始化模拟过程，并进行 10 次测量：

```
dog = DogSimulation(
x0=x.mean,
velocity=process_model.mean,
measurement_var=sensor_var,
process_var=process_model.var)
zs = [dog.move_and_sense() for _ in range(10)]
```

现在，我们进入 `predict() ... update()` 循环。

```
for z in zs:
prior = predict(x, process_model)
likelihood = gaussian(z, sensor_var)
x = update(prior, likelihood)
```

在循环的第一次迭代中， `prior` 的值为 `(1.0, 401.0)` ，这一点可以从打印出的表格中看出。在预测之后，我们认为当前的值应为 1.0，而方差则从 400 上升到了 401。方差有所增加，这在预测过程中是不可避免的，因为预测过程本身就会导致信息的丢失。

然后，我们以 `prior` 作为当前位置，调用更新函数。

因此，我得到的结果是： `pos = (1.352, 1.990), z = 1.354` 。

这里发生了什么？先验位置预测为 1.0，传感器读数为 1.354，二者存在残差。先验方差为 401 m^2，表示位置预测很不确定；因此，滤波器把估计 1.352 放在非常接近测量值的位置。这里的 1.0 是先验预测，实际位置还会受到仿真中过程噪声的影响。

再看方差：现在为 1.99 m^2，比原先的 401 m^2 大幅减小。原因是 RFID 测量的方差只有 2.0 m^2，因此比先验可靠得多。不过，先验仍含有少量有用信息，融合后的方差便略小于 2.0。

程序继续循环，依次调用 predict() 和 update()。最终估计位置为 15.053，实际位置为 14.838，方差收敛到 1.0 m^2。

图中黑色圆点表示带噪测量，蓝色实线表示滤波估计，红色三角形表示先验预测。测量和估计都可能波动，但估计明显更平滑。在当前标量直接测量模型中，更新后的估计总位于先验与测量之间。

滤波功能的实现仅需要几行代码而已。大部分代码用于初始化、数据存储、模拟狗的移动以及结果输出。负责执行滤波操作的代码非常简洁：

```
prior = predict(x, process_model)
likelihood = gaussian(z, sensor_var)
x = update(prior, likelihood)
```

如果我们不使用 `predict` 和 `update` 函数，那么代码可能会是这样的：

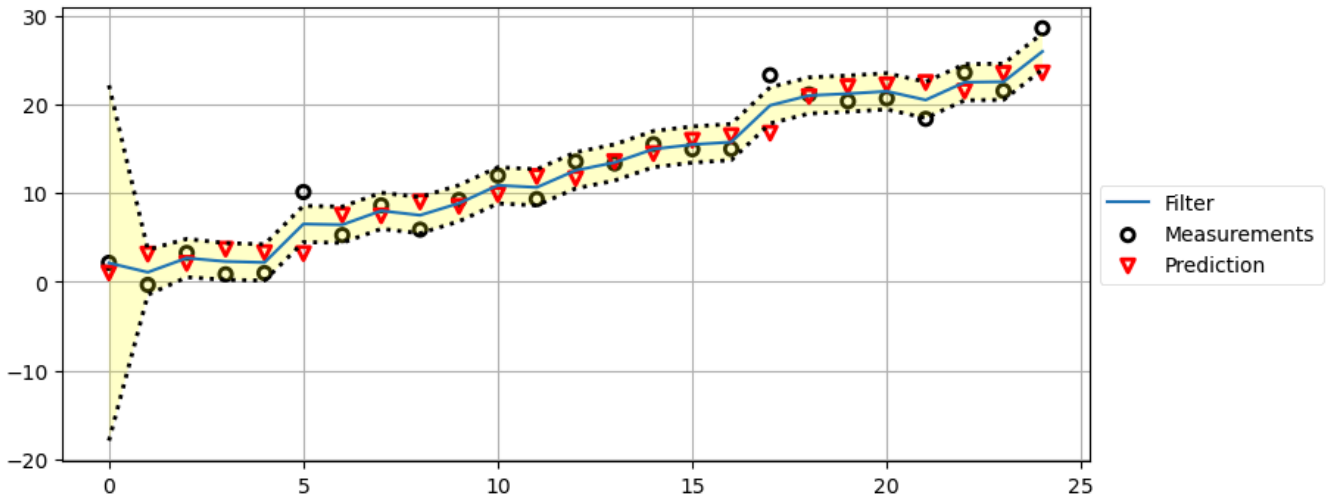
```
for z in zs:
# predict
dx = velocity*dt
pos = pos + dx
var = var + process_var
# update
pos = (var*z + sensor_var*pos) / (var + sensor_var)
var = (var * sensor_var) / (var + sensor_var)
```

只需要 5 行非常简单的数学公式，就能实现整个滤波过程！

这里只画了 10 个点，以免打印输出过长。下面增加数据量，观察滤波效果。虚线之间的浅黄色区域表示估计均值两侧的一个标准差范围。为便于观察，下面增大了过程噪声与传感器噪声方差；实际设计时，这些参数应根据模型与传感器特性设定，而不能随意更改。

```
process_var = 2.
sensor_var = 4.5
x = gaussian(0., 400.)
process_model = gaussian(1., process_var)
N = 25

dog = DogSimulation(x.mean, process_model.mean, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]

xs, priors = np.zeros((N, 2)), np.zeros((N, 2))
for i, z in enumerate(zs):
    prior = predict(x, process_model)
    x = update(prior, gaussian(z, sensor_var))
    priors[i] = prior

    xs[i] = x

book_plots.plot_measurements(zs)
book_plots.plot_filter(xs[:, 0], var=priors[:, 1])
book_plots.plot_predictions(priors[:, 0])
book_plots.show_legend()
kf_internal.print_variance(xs)
```

```
    4.4502 2.6507 2.2871 2.1955 2.1712
    2.1647 2.1629 2.1625 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
```



可以看到，方差约在 9 步后收敛到 2.1623，对应标准差 $\sigma = 1.47$ 米，而传感器的标准差为 $\sigma = 2.12$ 米。起初不确定的是状态估计，因为初始位置未知；并不是传感器前几次测量的噪声性质发生了变化。滤波很快得到比单次测量更小的估计方差。

这段代码完整实现了卡尔曼滤波器。如果你尝试过阅读相关文献，可能会感到惊讶：因为这里的实现方式与那些书中冗长的数学公式完全不同。只要我们专注于使用这些公式，而不用去推导它们，那么这个概念就很容易理解了。此外，我相信你也能感觉到，自己对整个过程的直觉理解相当不错。我们用高斯分布来表示各种概率值，随着时间的推移，这些概率值会越来越准确，因为更多的测量数据意味着我们有更多的信息可以用来进行计算。

练习：修改方差数值

修改 `process_var` 和 `sensor_var` 的值，观察其对滤波效果以及方差的影响。其中，哪一个因素对方差收敛的影响更大呢？也就是说，哪一个方案能使得方差更小呢？

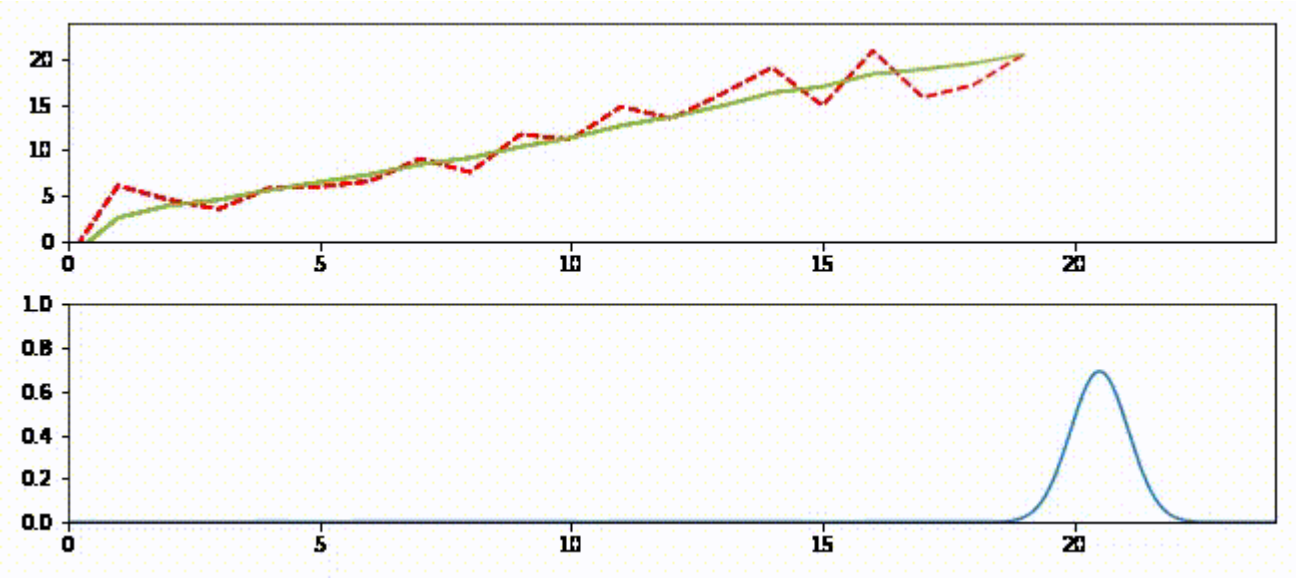
```
process_var = 40
sensor_var = 2
```

或者：

```
process_var = 2
sensor_var = 40
```

KF 动画

如果你是通过浏览器阅读这段文字，那么你会看到一个动画，展示该滤波器是如何跟踪那只狗的。就在这句话的正下方。



上图以绿色线条显示了滤波器的处理结果，红色虚线则标出了测量值。下图则展示了每一步处理后的高斯分布情况。

当测量开始时，可以看到实际数值与最初的预测值有很大差异。此时，高斯曲线的幅度较小（曲线较平缓），因此滤波器对自身的预测结果不太信任。相应地，滤波器会大幅调整自己的估算值。随着滤波器不断更新其估算结果，高斯曲线会变得更高，这意味着估计的确定性有所提高，滤波器的输出也越接近直线。在 x = 15 及更高的数值范围内，虽然测量结果中存在大量噪声，但与第一次遇到噪声时的情况相比，滤波器的反应并不明显。

卡尔曼增益

滤波器已经能够运行。现在回到数学，看看其中的计算。后验分布 $\boldsymbol{x}$ 由似然与先验的乘积 $\mathcal{L}\bar{x}$ 归一化得到；本例中，二者都是高斯形式。

因此，后验分布的均值为：

$$\mu = \frac{\bar{\sigma}^2\,\mu_z + \sigma_z^2\,\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$$

我用下标 $\boldsymbol{z}$ 来表示该测量值。我们可以将其重新表示为：

$$\mu = \left(\frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}\right)\mu_z + \left(\frac{\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2}\right)\bar{\mu}$$

从这个形式可以看出，我们是通过权重来调整测量值和先验值的。

$$\mu = W_1\mu_z + W_2\bar{\mu}$$

各权重之和为一，因为分母是一个用于归一化的项。我们引入了一个新项 $K = W_1$，从而得到：

$$\begin{aligned}\mu &= K\mu_z + (1-K)\bar{\mu} \\ &= \bar{\mu} + K(\mu_z - \bar{\mu})\end{aligned}$$

其中：

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

K 是卡尔曼增益，也是这个标量滤波器的核心缩放系数。它决定更新后的均值在 μ_z 与 $\bar{\mu}$ 之间的位置。

来看几个例子。若测量的精度是先验的九倍，也就是其方差只有先验的九分之一，则有 $\bar{\sigma}^2 = 9\sigma_z^2$，因此：

$$\begin{aligned}\mu &= \frac{9\sigma_z^2\mu_z + \sigma_z^2\bar{\mu}}{9\sigma_z^2 + \sigma_z^2} \\ &= \left(\frac{9}{10}\right)\mu_z + \left(\frac{1}{10}\right)\bar{\mu}\end{aligned}$$

所以 $K = \frac{9}{10}$。后验均值是测量值的十分之九，加上先验均值的十分之一。

如果测量和先验同样准确，则 $\bar{\sigma}^2 = \sigma_z^2$，因而有：

$$\begin{aligned}\mu &= \frac{\sigma_z^2(\bar{\mu} + \mu_z)}{2\sigma_z^2} \\ &= \left(\frac{1}{2}\right)\bar{\mu} + \left(\frac{1}{2}\right)\mu_z\end{aligned}$$

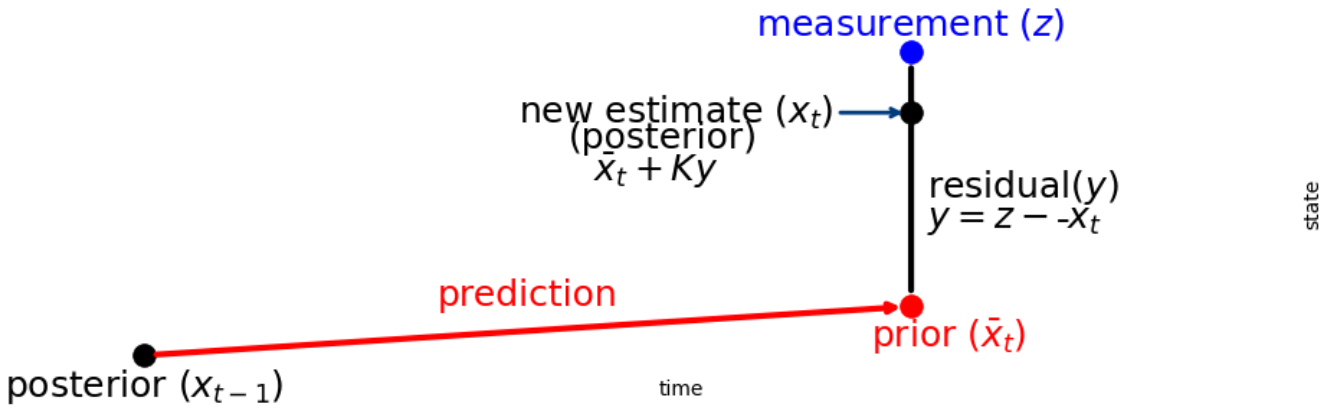
这是两个平均值的平均值。从直觉上来说，对两个同样准确的数值取平均值是合理的。

我们还可以用卡尔曼增益来表示方差：

$$\begin{aligned}\sigma^2 &= \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2} \\ &= K\sigma_z^2 \\ &= (1-K)\bar{\sigma}^2\end{aligned}$$

我们可以通过查看这张图表来理解这一点：

```
import kf_book.book_plots as book_plots
book_plots.show_residual_chart()
```



卡尔曼增益 $\boldsymbol{K}$ 缩放已经算出的残差，决定从先验沿残差方向修正多少。因此，update() 与 predict() 可以采用另一种等价写法：

```
def update(prior, measurement):
    x, P = prior        # mean and variance of prior
    z, R = measurement  # mean and variance of measurement

    y = z - x        # residual
    K = P / (P + R)  # Kalman gain

    x = x + K*y      # posterior
    P = (1 - K) * P  # posterior variance
    return gaussian(x, P)

def predict(posterior, movement):
    x, P = posterior # mean and variance of posterior
    dx, Q = movement # mean and variance of movement
    x = x + dx
    P = P + Q
    return gaussian(x, P)
```

为什么改写为这种形式，并采用这些不直观的变量名？多数教材和论文正是这样表示卡尔曼滤波；贝叶斯推导虽然并不少见，但其他常用推导也会自然得到这种形式。此外，多元卡尔曼滤波方程与它几乎一模一样，因此必须学习和理解。

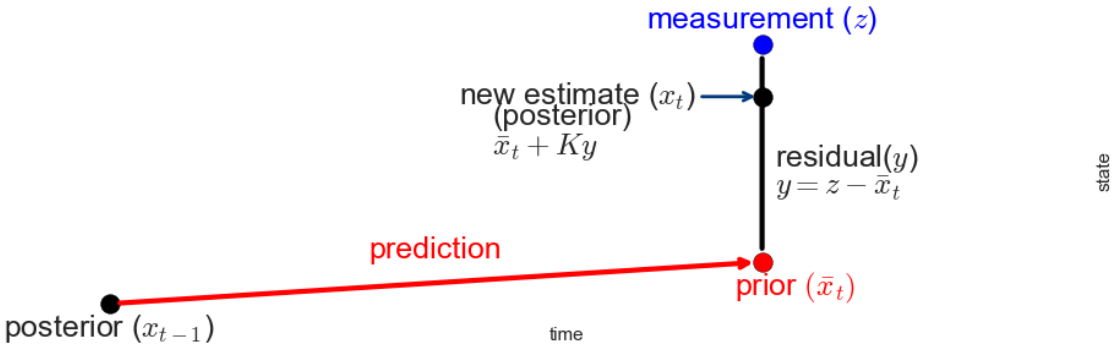
z、P、Q、R 这些名称来自文献中的常见约定。本章的标量模型中，$\boldsymbol{R}$ 表示测量噪声方差，$\boldsymbol{Q}$ 表示过程噪声方差，$\boldsymbol{P}$ 表示状态估计误差方差。$\boldsymbol{z}$ 则通常表示测量值。后面的多元模型会将这些方差推广为协方差矩阵。并非所有文献都采用完全相同的记号，但这组名称非常常见。

这种形式也有助于理解滤波，与 g-h 滤波的思路相同：先计算残差 $\boldsymbol{y = \mu_z - \bar{\mu}}$，再依据先验方差及测量噪声方差计算增益 $\boldsymbol{K = P/(P+R)}$，最后把修正量 $\boldsymbol{Ky}$ 加到先验均值上，得到后验均值。增益是先验方差占二者方差之和的比例。

这种形式不再直观显示贝叶斯关系，也不直接显示似然与先验相乘的过程。但两种观点在数学上等价。我选择从贝叶斯方法入手，是因为它能直观而深入地解释概率推理。采用 $\boldsymbol{K}$ 的另一种形式，则有助于理解正交投影方法。卡尔曼最初推导这种滤波器时采用的就是正交投影，而非贝叶斯推理。接下来的几章将进一步说明。

该算法的完整描述

请回想一下我们用来描述 g-h 滤波器的那个图表：



在这一章中，我们所做的其实都是一样的。卡尔曼滤波器会先进行预测，再获取实际测量值，最后根据这两个数值来得出一个新的估计值。

这一点非常重要：本书中的每一个滤波器都采用了相同的算法，只不过在数学细节上有所区别而已。后面的章节中，相关的数学公式可能会比较复杂，但核心思想其实很容易理解。

重要的是，不要只盯着某个特定滤波器所对应的方程式的细节，而应理解这些方程式究竟在计算什么，以及为何要这样计算。世界上有无数种滤波器，它们虽然都使用不同的数学公式来实现相同的算法，但所选的数学公式会影响到最终的结果质量以及该滤波器能够处理的问题类型。不过，这些数学公式的选择并不会影响滤波器的基本原理。

以下是该算法的通用流程：

初始化

1. 初始化滤波器的状态。
2. 初始化对该状态的信念（不确定性）。

预测

1. 根据系统行为预测下一时间步的状态。
2. 调整状态信念，计入预测的不确定性。

更新

1. 获取测量值及其精度信息。
2. 计算测量与预测状态之间的残差。
3. 根据测量与预测的相对精度计算增益。
4. 按增益加权，在预测与测量之间形成状态估计。
5. 依据测量的确定程度更新状态不确定性。

很难找到一种不符合这种模式的贝叶斯滤波算法。有些滤波器会忽略某些因素，比如预测误差；而另一些则采用极其复杂的计算方法。但归根结底，所有贝叶斯滤波算法都是如此运作的。

单变量卡尔曼滤波器的方程如下：

预测

Equation	Implementation	Kalman Form
$\bar{x} = x + f_x$	$\bar{\mu} = \mu + \mu_{f_x}$ $\bar{\sigma}^2 = \sigma^2 + \sigma^2_{f_x}$	$\bar{x} = x + dx$ $\bar{P} = P + Q$

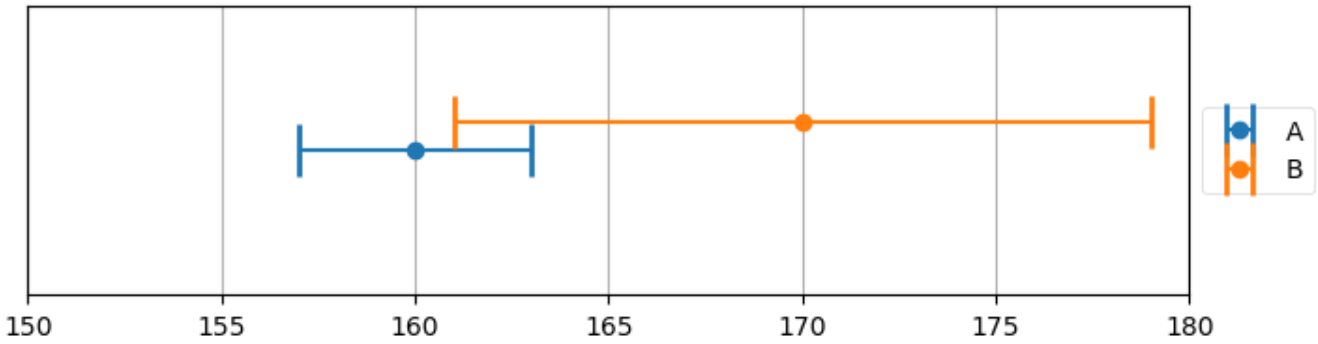
更新

Equation	Implementation	Kalman Form
$x = \|\mathcal{L}\bar{x}\|$	$y = z - \bar{\mu}$ $K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}$ $\mu = \bar{\mu} + Ky$ $\sigma^2 = \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2+\sigma_z^2}$	$y = z - \bar{x}$ $K = \frac{\bar{P}}{\bar{P}+R}$ $x = \bar{x} + Ky$ $P = (1-K)\bar{P}$

与 g-h 滤波器及离散贝叶斯滤波器的比较

现在正是了解这三种滤波器在处理误差方式上的差异的好时机。对于 g-h 滤波器，我们按照下图所示的方式对测量数据进行了处理：

```
book_plots.plot_errorbars([(160, 3, 'A'), (170, 9, 'B')], xlims=(150, 180))
```

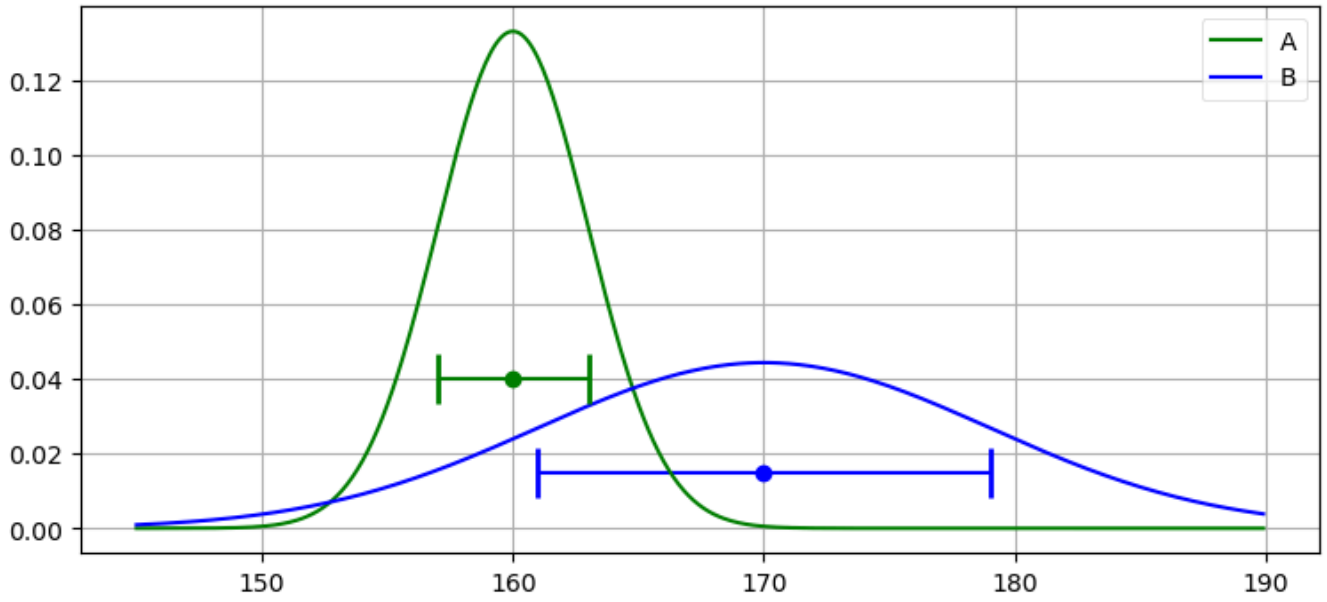


传感器 A 读数为 160，B 读数为 170。误差条表示各自读数允许的真值范围：依据 A，真值可在 157—163 之间；依据 B，真值可在 161—179 之间。读数本身已经固定，变化的是可能的真实值，不是“实际测量值”。

当时没有明确命名，但这个误差模型采用的是均匀分布：在给定范围内，等宽区间具有相同概率。按这个模型，157、160、163 附近同样宽的区间具有相同概率，范围以外的概率为零。这里比较的是概率密度或等宽小区间的概率；对于连续分布，任意单个精确数值的概率均为零。

我们可以用高斯函数来模拟这种情况。我用 $\mathcal{N}(160, 3^2)$ 表示传感器 A，用 $\mathcal{N}(170, 9^2)$ 表示传感器 B。下图中，我还标出了均匀分布的误差范围，以便进行比较。

```
xs = np.arange(145, 190, 0.1)
ys = [stats.gaussian(x, 160, 3**2) for x in xs]
plt.plot(xs, ys, label='A', color='g')

ys = [stats.gaussian(x, 170, 9**2) for x in xs]
plt.plot(xs, ys, label='B', color='b')
plt.legend();
plt.errorbar(160, [0.04], xerr=[3], fmt='o', color='g', capthick=2, capsize=10)
plt.errorbar(170, [0.015], xerr=[9], fmt='o', color='b', capthick=2, capsize=10);
```

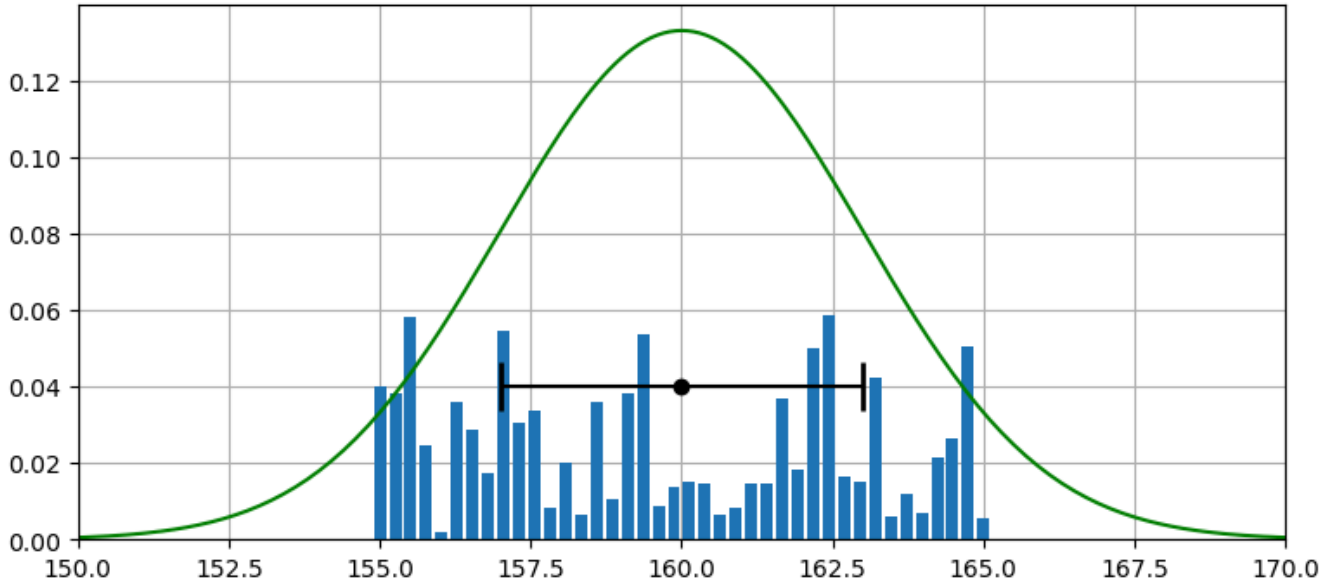


选择使用均匀分布或高斯分布其实是一种建模上的取舍。这两种分布都无法完全反映现实情况。在大多数情况下，高斯分布更符合实际情况。大多数传感器所测得的数值往往接近于被测量的真实值，而很少会测得与真实值相差很大的数值。高斯分布正好能够体现这种趋势。相比之下，均匀分布则假设在某个范围内，任何数值被测得的可能性都是相等的。

现在，让我们来看看在离散贝叶斯滤波器中使用的离散分布。该模型将所有可能的数值范围划分为若干个离散的区间，并为每个区间分配一个概率值。只要这些概率值之和等于 1，这种分配方式完全可以由人为决定。

让我们用均匀分布、高斯分布和离散分布来分别绘制某个传感器的数据曲线吧。

```
from random import random
xs = np.arange(145, 190, 0.1)
ys = [stats.gaussian(x, 160, 3**2) for x in xs]
belief = np.array([random() for _ in range(40)])
belief = belief / sum(belief)

x = np.linspace(155, 165, len(belief))
plt.gca().bar(x, belief, width=0.2)
plt.plot(xs, ys, label='A', color='g')
plt.errorbar(160, [0.04], xerr=[3], fmt='o', color='k', capthick=2, capsize=10)
plt.xlim(150, 170);
```



我使用随机数来构建离散分布，以此说明该模型能够用来模拟任何形式的概率分布。这一特性使得该模型具有极大的实用性。只要拥有足够多的离散区间，我们就能模拟出任何复杂传感器的误差特性。不过，这种强大功能也带来了数学上的复杂性：对高斯分布进行运算时，只需执行两步数学操作，结果依然是高斯分布。这种规律性让我们能够轻松分析滤波器的性能和行为。但对于离散分布来说，情况则完全不同：对其进行运算需要遍历所有数据，而且我们很难准确描述运算结果。基于离散分布来分析滤波器的性能特性，几乎是不可能的。

这里并没有所谓的“正确”选择。在本书的后面部分，我们将介绍使用离散分布的粒子滤波算法。这是一种极其强大的技术，因为它能够处理各种复杂的情形。不过，这种方法的缺点是计算速度较慢，且难以进行数学分析。

目前，我们先忽略这些问题，继续在接下来的几章中使用高斯函数。随着学习的深入，你将会了解在数学模型中使用高斯函数的优点和局限性。

滤波器设计入门

到目前为止，我们已经为位置传感器开发了相应的滤波器。对于这类问题，我们已经很熟悉了；不过，要为其他类型的问题实现卡尔曼滤波器，可能仍会觉得准备不足。说实话，目前的描述中还缺少不少信息。接下来的章节会补充这些遗漏的内容。不过，我们不妨先试着为温度计设计并实现一个卡尔曼滤波器，从而了解其原理。温度计的传感器会输出与所测温度相对应的电压值。我们已经查阅了该传感器的制造商规格书，得知该传感器产生的噪声属于白噪声，其标准差为 0.13 伏特。

我们可以用下面的函数模拟温度传感器的测量结果：

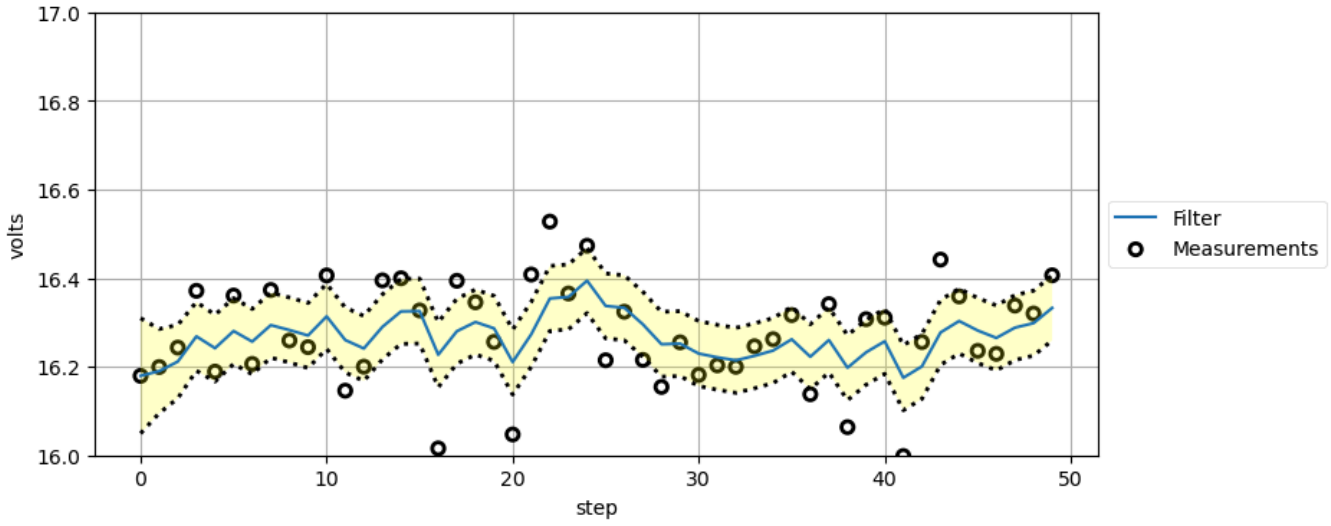
```
def volt(voltage, std):
    return voltage + (randn() * std)
```

现在，我们需要编写卡尔曼滤波器的处理流程。与之前的问题类似，我们需要进行预测和更新的操作。测量步骤其实很简单：调用 volt() 来获取测量结果，然后将结果传递给 update() 方法。但是，预测步骤该如何处理呢？我们没有传感器来检测电压中的“变化”，而且，在短时间内，我们预期电压会保持不变。那我们该如何处理这种情况呢?

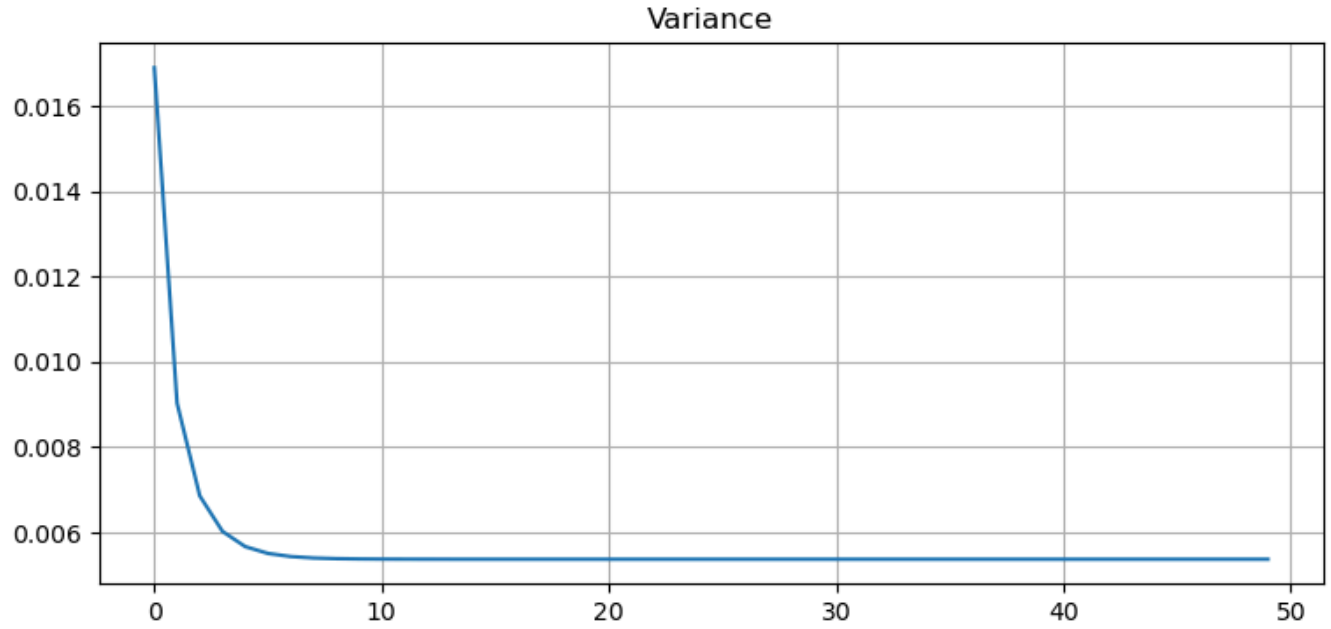
按数学模型处理：没有已知的平均电压变化，就把每步平均增量设为零。但若同时假设变化完全没有不确定性，模型就会把温度视为永久不变，累积测量后变得过度自信，甚至忽略真实温度变化造成的新读数。这称为过度自信或自满的滤波器，应当避免。因此在预测中加入适量过程噪声，允许电压随时间变化。代码设置 process_var=.05**2，表示每个时间步电压随机增量的方差，不是增量本身，也不是标准差。这里只为展示预测与更新中不确定性的变化；真实传感器应按预期变化来设定。例如，室内环境温度变化慢，所需过程方差通常较小；化学反应室中的热电偶可能需要较大数值。后文会继续讨论选择方法。

让我们看看会发生什么吧。

```
temp_change = 0
voltage_std = .13
process_var = .05**2
actual_voltage = 16.3

x = gaussian(25., 1000.) # initial state
process_model = gaussian(0., process_var)

N = 50
zs = [volt(actual_voltage, voltage_std) for i in range(N)]
ps = []
estimates = []

for z in zs:
    prior = predict(x, process_model)
    x = update(prior, gaussian(z, voltage_std**2))

    # save for latter plotting
    estimates.append(x.mean)
    ps.append(x.var)

# plot the filter output and the variance
book_plots.plot_measurements(zs)
book_plots.plot_filter(estimates, var=np.array(ps))
book_plots.show_legend()
plt.ylim(16, 17)
book_plots.set_labels(x='step', y='volts')
plt.show()

plt.plot(ps)
plt.title('Variance')
print(f'Variance converges to {ps[-1]:.3f}')
```




```
Variance converges to 0.005
```



第一张图表展示了各个传感器的测量值与滤波器输出值之间的关系。尽管传感器产生的噪声很大，但我们仍然能够大致确定传感器的电压值。在作者进行测试时，滤波器输出的最后一个电压值为 **16.213**，这一数值与 `volt()` 函数所使用的 **16.4** 数值非常接近。在其他测试中，得到的数值则有所差异。

规格书给出性能规定，但器件会因正常制造差异而表现不同。某些参数给出保证的最大误差，昂贵仪器还可能附有该台设备的实测报告；低价传感器则可能只经过抽样检验。关键应用必须弄清：参数是最大误差，还是 3σ 对应的统计误差范围？是否逐个测试？测量误差是否近似正态，还是服从其他分布？此外，产品也可能有缺陷而不满足规格。

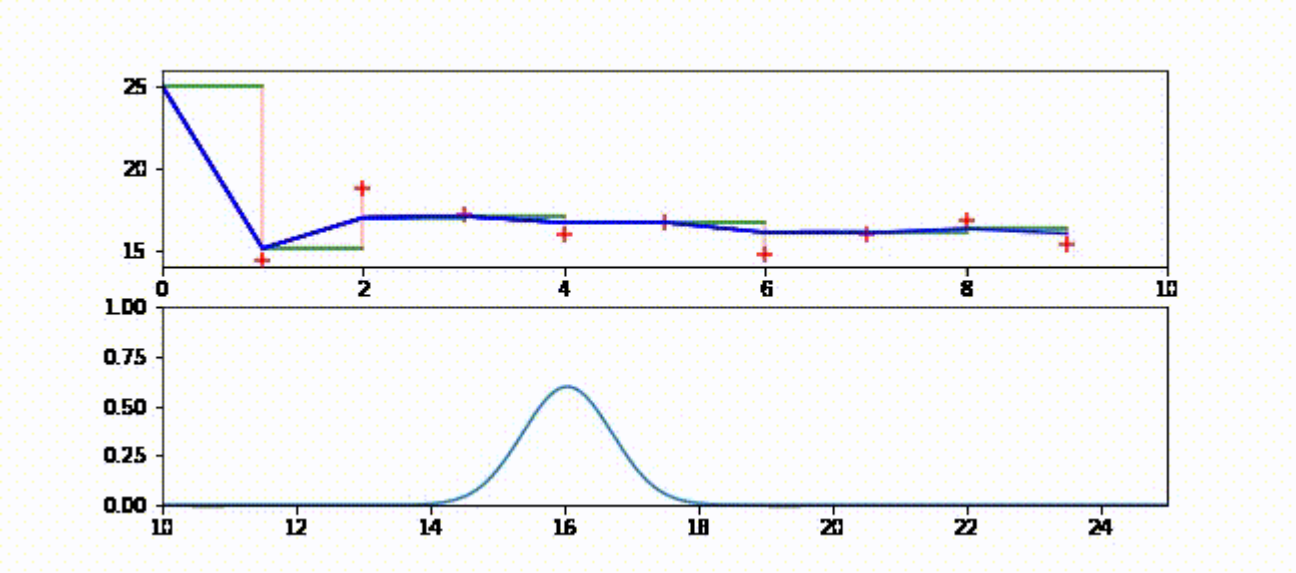
例如，我正在查看一款气流传感器的数据手册。其中有一个名为“重复性”的参数，其数值为 $\pm 0.50\%$。这个数值是否服从高斯分布呢？是否存在偏差呢？比如，低温时，重复性可能接近 0.0%；而高温时则始终接近 $+0.50\%$。电气元件的数据手册中通常会有一节名为“典型性能特性”的内容。这些内容用于描述那些无法通过表格轻易呈现的信息。例如，我看到的这张图表展示了 LM555 定时器的输出电压与电流之间的关

系。图表中有三条曲线，分别对应不同温度下的性能表现。理想情况下，输出电压应呈线性关系，但实际上这三条曲线都是弯曲的。这说明，该芯片的输出电压误差很可能不服从高斯分布——在高温环境下，输出电压会降低；而当输入电流很大时，输出电压的非线性程度也很高。

正如你可能猜到的那样，对传感器的性能进行建模，是构建一个性能良好的卡尔曼滤波器过程中较为困难的部分之一。

动画

对于那些在浏览器中阅读此内容的人士，这里有一个动画，展示了该滤波器的工作原理。如果你不是通过浏览器来查看，请访问 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/animations/05_volt_animate.gif 来查看该动画。



在动画中，最上方的图表首先用绿色线条表示预测出的下一电压值，然后用红色“+”号表示实际测量值。接着，用浅红色线条表示误差值，最后用蓝色线条表示滤波器的输出值。可以看出，当滤波器开始进行修正时，修正幅度相当大；但经过几次调整后，即使测量值与预测值相差很大，滤波器对输出值的调整幅度也变得很小了。

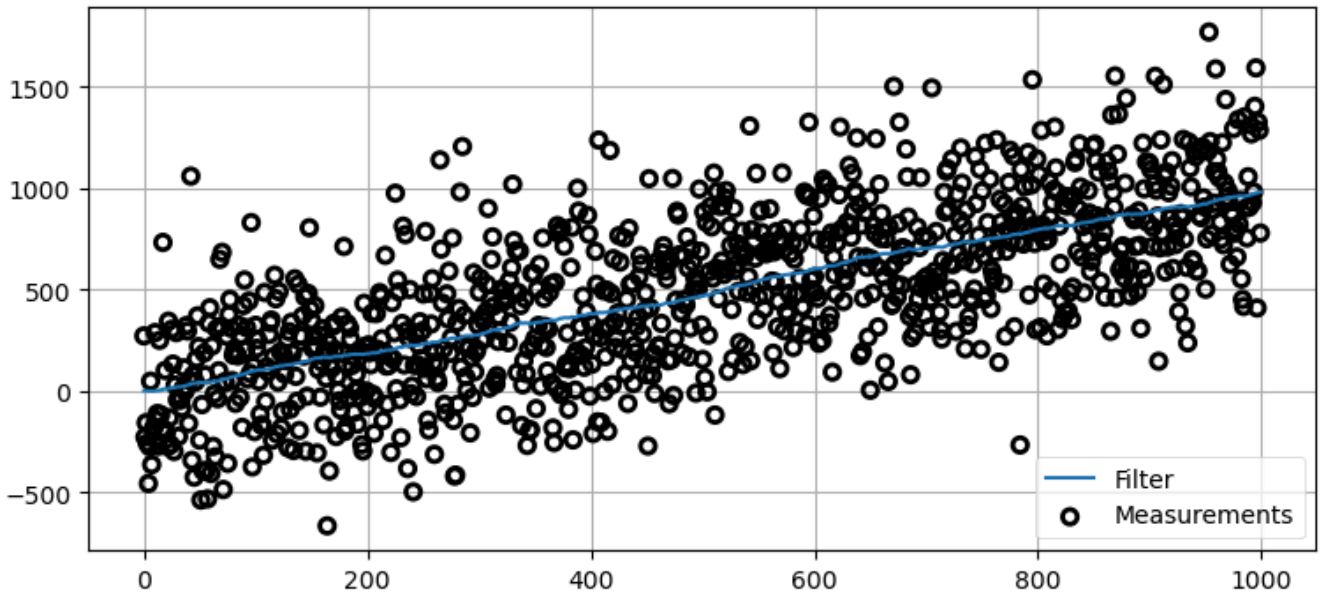
下图展示了随着滤波器不断更新，高斯分布形态的变化情况。在滤波器开始工作时，高斯曲线的中心位于 25 处，这正是我们对电压的初始估计值。由于初始时存在不确定性，高斯曲线既宽又短。但随着滤波器不断更新，高斯曲线的中心迅速移至 16.0 左右，同时曲线也变得更陡峭，这反映出滤波器对电压估计值的信心越来越强。另外，你还会注意到高斯曲线的高度会有些波动。仔细观察会发现，在预测阶段，高斯曲线会变得更短且更分散；而当滤波器再次获取新的测量数据后，高斯曲线又会变陡峭且更窄。

从 g-h 滤波器的角度理解这段动画：每一步先预测，再测量，计算残差——测量与预测之间的差异——最后根据缩放系数 g，在残差所对应的线段上选择一个位置。卡尔曼滤波做的完全一样，只是缩放系数 g 随时间变化。随着滤波器对自身状态越来越有信心，缩放系数会使结果更偏向预测，而非测量。

示例：极高的噪声水平

前面跟踪狗的滤波器中，信号噪声不多，我还“猜定”狗的初始位置为 0。它在现实条件下表现如何？现在向 RFID 传感器加入更多噪声，过程方差仍保持 2 m^2。这次噪声极大，看起来几乎淹没真实测量。如果传感器标准差为 300 米，你直觉上认为滤波器会表现怎样？也就是说，实际位置为 1.0 米，读数却可能是 287.9 米、-589.6 米，或大致这个范围内的其他值。在往下阅读前先想一想。

```
sensor_var = 300.**2
process_var = 2.
process_model = gaussian(1., process_var)
pos = gaussian(0., 500.)
N = 1000
dog = DogSimulation(pos.mean, 1., sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for i in range(N):
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(zs[i], sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```



在这个例子中，噪声非常大，但滤波器处理后的输出却几乎呈直线状！这一结果真是令人惊讶！你认为造成这种现象的原因可能是什么？

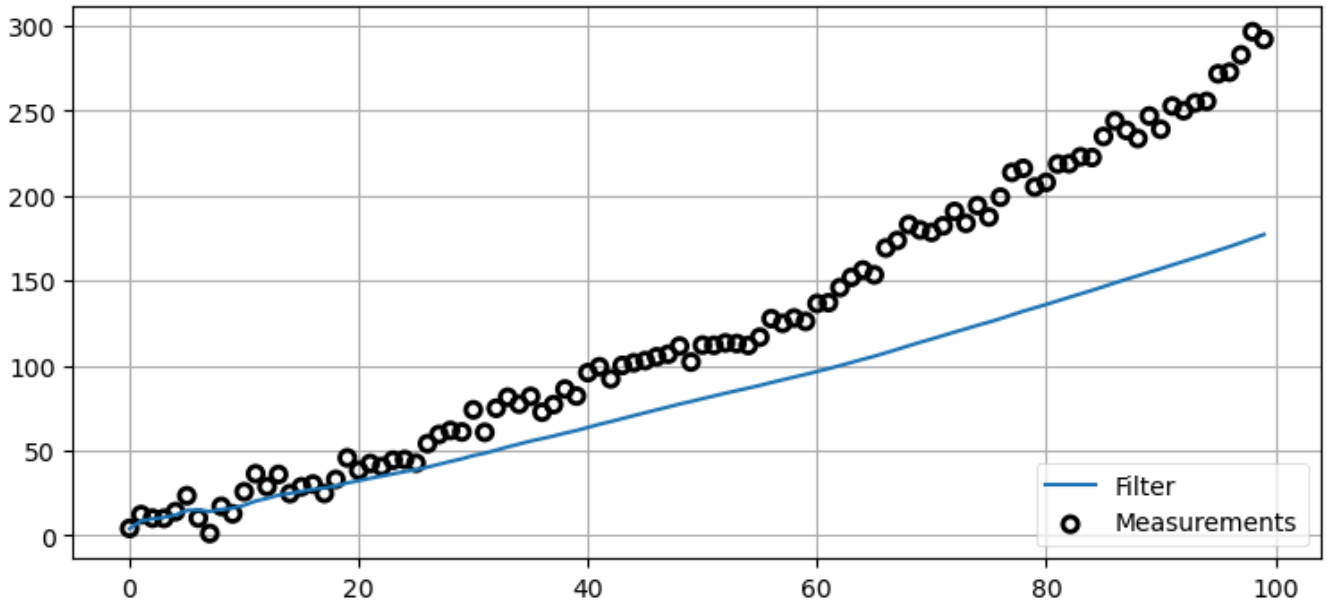
结果接近直线，是因为设定的过程误差较小：滤波器因此高度信任直线运动预测，输出也接近直线。

示例：不合适的过程方差

刚才的滤波结果看起来很好！那么，为什么不干脆把过程方差设得极小，让结果始终平直而光滑呢？

过程方差告诉滤波器系统随时间变化的程度。如果人为把它设得很小，相当于向滤波器提供错误信息，它就无法响应真实变化。现在让狗的速度在每个时间步都稍有增加，看看过程方差为 $0.001\ m^2$ 时滤波器如何表现。

```
sensor_var = 20.
process_var = .001
process_model = gaussian(1., process_var)
pos = gaussian(0., 500.)
N = 100
dog = DogSimulation(pos.mean, 1, sensor_var, process_var*10000)
zs, ps = [], []
for _ in range(N):
    dog.velocity += 0.04
    zs.append(dog.move_and_sense())

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```

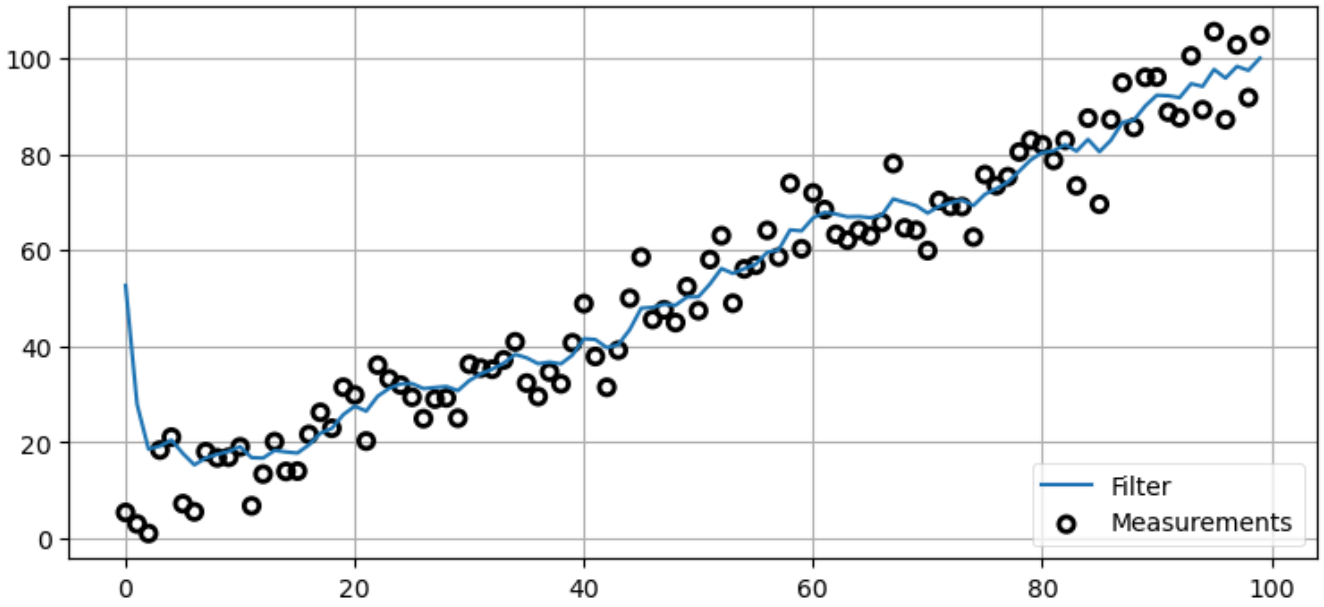


很容易看出，滤波器没有正确响应测量。测量清楚地表明狗的速度正在变化，但模型却告诉滤波器预测几乎完美，因此它基本忽略了测量。建议你同时改变狗的运动变化程度和过程方差，比较结果，后文还会深入讨论。关键在于，方差必须正确描述系统。滤波器不会自行“察觉”与测量的偏离并自动修复错误模型；它依据先验和测量的方差计算卡尔曼增益，按两者的相对可靠程度形成融合估计，而不是简单二选一。

示例：不准确的初始估计

下面考察初始位置估计错误时的结果。为使效果容易看清，将传感器方差减小到 30，但把初始位置估计设为 1000 米。滤波器能从这 1000 米的初始误差中恢复吗?

```
sensor_var = 5.**2
process_var = 2.
pos = gaussian(1000., 500.)
process_model = gaussian(1., process_var)
N = 100
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```

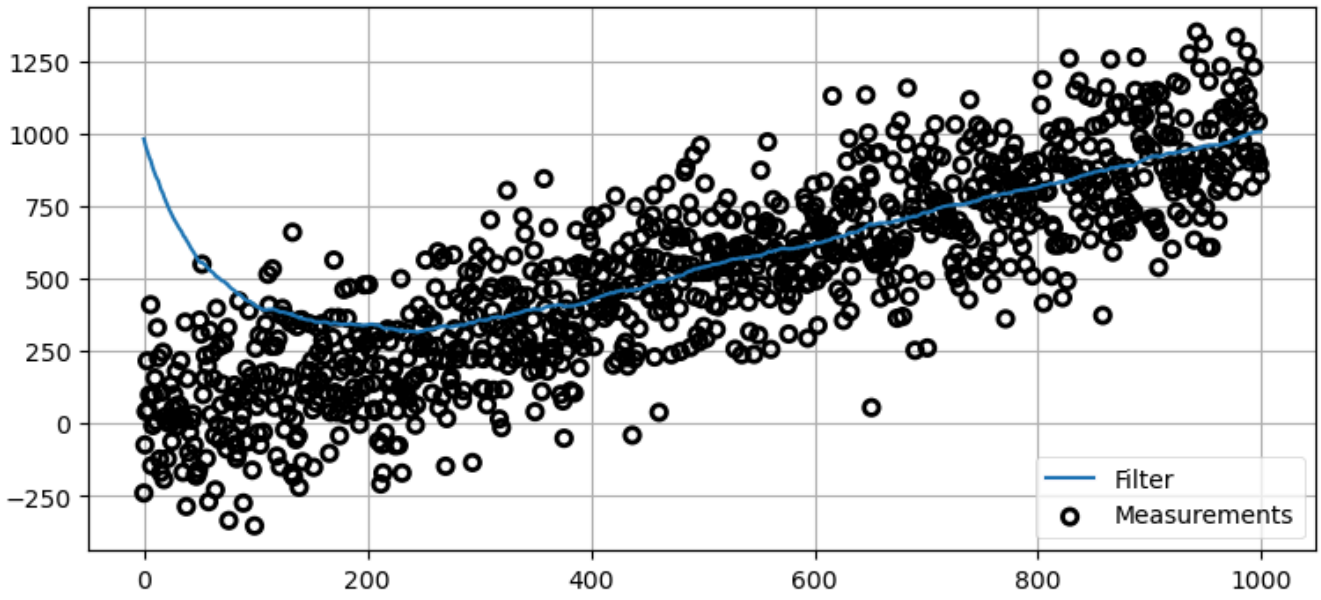


答案仍然是可以！由于我们对传感器比较有信心（$\sigma^2 = 5^2$），仅经过第一步，位置估计就从 1000 米变成约 50 米；再经过 5—10 次测量，就收敛到正确值。这就是绕开初值选择这一“先有鸡还是先有蛋”问题的方法。实际中，通常把传感器第一次测量作为初值。但即使初始条件猜得离谱，只要滤波器使用的方差符合实际过程和测量方差，卡尔曼滤波仍能收敛。

示例：巨大的噪声以及不准确的初始估计值

那如果同时出现最糟糕的情况呢：既有巨大的噪声，又有不准确的初始估计值?

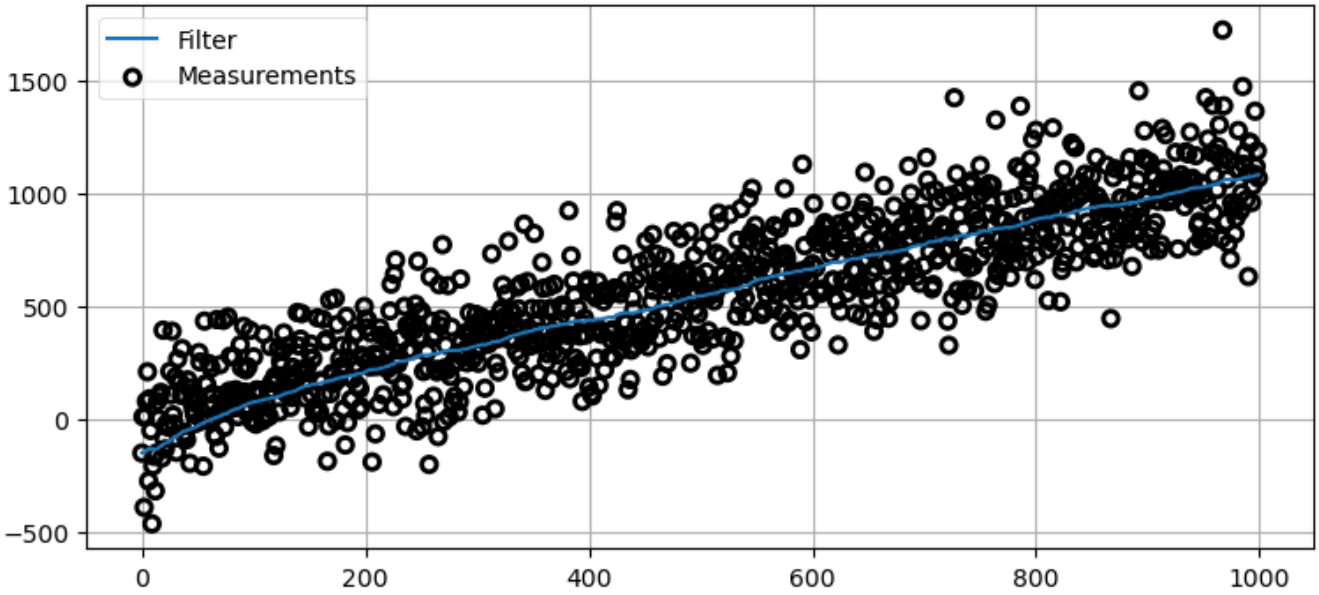
```
sensor_var = 30000.
process_var = 2.
pos = gaussian(1000., 500.)
process_model = gaussian(1., process_var)

N = 1000
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```



这一次滤波器表现吃力。前例只有 100 次更新，本例用了 1000 次。目测约 400 次迭代后估计才比较准确，超过 600 次后效果才更好。卡尔曼滤波很有用，但并不能创造奇迹；本例同时存在极大测量噪声和很差的初始条件，因此收敛较慢。

最后，让我们按照建议，将第一次测量得到的数值作为初始位置吧。

```
sensor_var = 30000.
process_var = 2.
process_model = gaussian(1., process_var)
N = 1000
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]

pos = gaussian(zs[0], 500.)
ps = []
for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc='best');
```



这一简单的改动显著提升了算法的运行效果。在某些情况下，需要大约 200 次迭代才能得到满意的解决方案；而在其他情况下，算法则能很快收敛到正确结果。这一切都取决于第一次测量时存在的噪声程度。如果噪声较大，那么初始估计值就会与狗的实际位置相差甚远。

200 次迭代看起来很多，但我们加入的噪声确实极大。现实中的温度计、激光测距仪、GPS、计算机视觉等传感器，通常不会有这些示例中如此巨大的误差。廉价温度计的合理方差也许是 0.2 $(°C)^2$，而这里的代码使用了 30,000 $(°C)^2$。这里的方差单位是摄氏度的平方。

练习：交互式图表

利用 Jupyter Notebook 的动画功能来实现卡尔曼滤波器。这样，你就可以通过滑块实时调整各种参数值。具体操作方法请参考“高斯分布”章节中的“交互式高斯函数”部分。你需要使用 `interact()` 函数来调用相关的计算和绘图功能。传递给 `interact()` 的每个参数都会自动对应一个滑块。我已经编写好了相关的模板代码，你需要自行填写所需的代码即可。

```
from ipywidgets import interact
from kf_book.book_plots import FloatSlider

def plot_kalman_filter(start_pos,
                       sensor_noise,
                       velocity,
                       process_noise):
    plt.figure();
    # your code goes here

interact(plot_kalman_filter,
         start_pos=(-10, 10),
         sensor_noise=FloatSlider(value=5, min=0, max=100),
         velocity=FloatSlider(value=1, min=-2., max=2.),
         process_noise=FloatSlider(value=5, min=0, max=100.));
```

```
interactive(children=(IntSlider(value=0, description='start_pos', max=10, min=-10),
FloatSlider(value=5.0, con…
```

解答

以下是一种可能的解决方案。我们可以使用滑块来调整起始位置、传感器中的噪声水平、每个时间步长内的移动量以及移动过程中的误差大小。其中，“过程噪声”这个参数比较难以理解——它实际上描述了狗在每个时间步长内偏离预定路径的程度。因此，我们需要将这一数值加到狗在每个时间步长后的位置上。我设置了随机数生成器的种子，这样每次重新计算时都会使用相同的随机数，从而便于我们比较在不同滑块设置下得到的结果。

```
from numpy.random import seed
from ipywidgets import interact

def plot_kalman_filter(start_pos,
                       sensor_noise,
                       velocity,
                       process_noise):
    N = 20
    zs, ps = [], []
    seed(303)
    dog = DogSimulation(start_pos, velocity, sensor_noise, process_noise)
    zs = [dog.move_and_sense() for _ in range(N)]
    pos = gaussian(0., 1000.) # mean and variance
    process_model = gaussian(velocity, process_noise)

    for z in zs:
        pos = predict(pos, process_model)
        pos = update(pos, gaussian(z, sensor_noise))
        ps.append(pos.mean)

    plt.figure()
    plt.plot(zs, c='k', marker='o', linestyle='', label='measurement')
    plt.plot(ps, c='#004080', alpha=0.7, label='filter')
    plt.legend(loc=4);
    plt.show()

interact(plot_kalman_filter,
         start_pos=(-10, 10),
         sensor_noise=FloatSlider(value=5, min=0., max=100),
         velocity=FloatSlider(value=1, min=-2., max=2.),
         process_noise=FloatSlider(value=.1, min=0, max=40));
```

```
interactive(children=(IntSlider(value=0, description='start_pos', max=10, min=-10),
FloatSlider(value=5.0, con…
```

练习——非线性系统

卡尔曼滤波器的方程都是线性的：

$$\mathcal{N}(\bar{\mu}, \bar{\sigma}^2) = \mathcal{N}(\mu, \sigma^2) + \mathcal{N}(\mu_{\mathbf{move}}, \sigma^2_{\mathbf{move}})$$
$$\mathcal{N}(\mu, \sigma^2) = \mathcal{N}(\bar{\mu}, \bar{\sigma}^2) \times \mathcal{N}(\mu_{\mathbf{z}}, \sigma^2_{\mathbf{z}})$$

你认为，这种滤波器在处理非线性系统时，效果如何呢？效果好还是差呢？

实现一个卡尔曼滤波器，该滤波器使用以下公式来生成测量值。

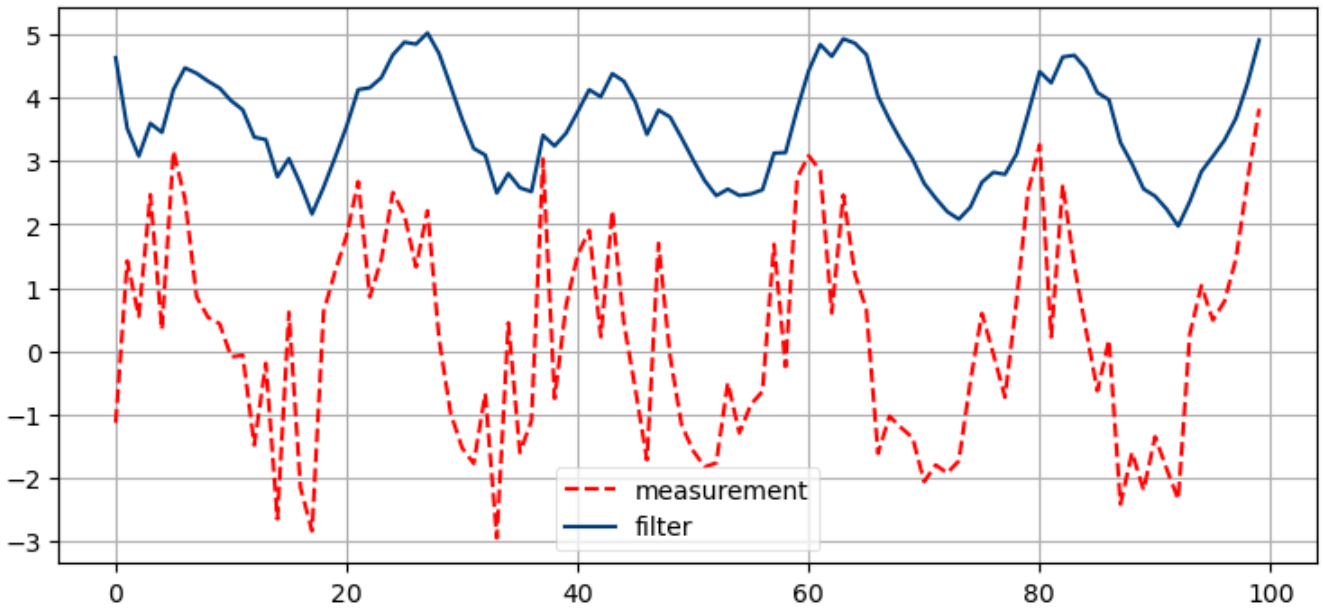
```
for i in range(100):
z = math.sin(i/3.) * 2
```

调整方差和初始值，看看会产生什么效果。比如，如果初始猜测非常糟糕，会得到怎样的结果呢?

```
#enter your code here.
```

解答

```
import math

sensor_var = 30.
process_var = 2.
pos = gaussian(100., 500.)
process_model = gaussian(1., process_var)

zs, ps = [], []

for i in range(100):
    pos = predict(pos, process_model)

    z = math.sin(i/3.)*2 + randn()*1.2
    zs.append(z)

    pos = update(pos, gaussian(z, sensor_var))
    ps.append(pos.mean)

plt.plot(zs, c='r', linestyle='dashed', label='measurement')
plt.plot(ps, c='#004080', label='filter')
plt.legend(loc='best');
```



讨论

这太糟糕了！输出信号根本不像是正弦波，其形态与正弦波相差甚远。在线性系统中，我们可以在信号中加入大量噪声，但仍然能得到相当准确的结果。而在这里，哪怕是少量的噪声也会导致极其糟糕的结果。

回忆 g-h 滤波器，就能理解本例的问题：更新根据预测与测量形成加权估计，而所用的常速过程模型无法准确描述正弦信号的变化。因此，为抑制噪声而平滑输出时，会出现明显滞后。问题在于本例的过程模型与信号不匹配；为正弦信号选择合适的状态模型后，线性滤波器同样可以用于跟踪。

在从业者开始使用卡尔曼滤波器后不久，他们就发现该滤波器在处理非线性系统时效果不佳。于是，他们开始寻求解决这一问题的方法。后面的章节将专门探讨这个问题。

固定增益滤波器

嵌入式计算机的处理器性能通常极为有限。许多嵌入式计算机甚至没有浮点运算功能。这些简单的计算公式可能会给芯片带来很大的负担。不过，随着技术的进步，这种情况有所改善。但千万不要低估了这样一个事实：当需要购买数百万个处理器时，每个处理器多节省一美元的成本也是相当可观的。

在上面的例子中，滤波器的方差最终会收敛到一个固定值。只要测量值和过程中的方差保持恒定，这种情况必然会发生。你可以利用这一特性，通过模拟来确定方差最终会收敛到什么数值。然后，你可以将该数值直接写入滤波器中。只要给滤波器一个合理的初始值（我建议使用第一次测量得到的数值作为初始值），滤波器就能发挥出很好的性能。例如，狗的跟踪滤波器就可以简化为这样的形式：

```
def update(x, z):
K = .13232 # experimentally derived Kalman gain
y = z - x # residual
x = x + K*y # posterior
return x
def predict(x):
return x + vel*dt
```

这里用卡尔曼增益形式的更新，是为了说明：在已确定稳态增益后，在线计算可以不再递归更新方差。若估计方差收敛到固定值，卡尔曼增益也会收敛到它自己的固定值；并不是说方差和增益数值相同。

FilterPy 的实现方式

FilterPy 实现了 predict() 和 update() 功能。这些功能不仅适用于本章所讨论的单变量情况，也适用于我们在后续章节中将要学习的多元情形。正因如此，它们的接口略有不同。FilterPy 不会将高斯分布参数作为元组来处理，而是将它们视为两个分别命名的变量来处理。

`predict()` 需要多个参数，但我们只需要使用其中的四个即可：

```
predict(x, P, u=u, Q=Q)
```

x 是状态估计，P 是对应的估计误差方差，u 表示控制或运动输入，Q 是过程噪声方差。调用 predict() 时，建议显式写 u=u、Q=Q 这样的关键字参数，以免混淆参数位置；函数的第三个形参是 F，并不是 u。

这些名称听起来实在糟糕透顶。确实如此！正如我之前所说，这些名称源自控制理论的悠久历史。你读过的每一篇论文或每一本书都会使用这些名称。所以，我们只能习惯它们。如果拒绝记住这些名称，那你将永远无法理解那些学术文献的内容。

用状态分布 $\mathcal{N}(10, 3)$ 和位移分布 $\mathcal{N}(1, 4)$ 进行测试。预计预测位置的均值为 11（10+1），方差为 7（3+4）。

```
import filterpy.kalman as kf
kf.predict(x=10., P=3., u=1., Q=4.)
```

```
(11.0, 7.0)
```

update 也接受多个参数，但目前你只需关注以下四个参数即可：

```
update(x, P, z, R)
```

与之前一样， x 和 P 分别代表系统的状态和方差。 z 代表测量结果， R 则代表测量结果的方差。让我们先执行最后的预测操作，以得到先验信息，然后再进行更新。

```
x, P = kf.predict(x=10., P=3., u=1., Q=2.**2)
print(f'{x:.3f}')

x, P = kf.update(x=x, P=P, z=12., R=3.5**2)
print(f'{x:.3f} {P:.3f}')
```

```
11.000
11.364 4.455
```

这里给出的是一次噪声较大、方差较大的测量，因此更新后的估计仍接近先验均值 11。

最后一点。在预测结果的输出上，我没有使用变量名 prior 。在本书的其余部分，我也不会使用这个变量名。卡尔曼滤波方程中使用的变量名是 $\mathbf{x}$ 。所谓“先验值”和“后验值”，其实都是对系统状态的估计值：前者是在考虑测量结果之前的估计值，后者则是在考虑了测量结果之后的估计值。

本章小结

本章所介绍的卡尔曼滤波器，其实是我们接下来要学习的更通用滤波器的一种特殊形式。大多数教材都不会详细讨论这种一维形式的滤波器。不过，我认为它是一个非常重要的基础。我们在本书的开头学习了 g-h 滤波器，接着又实现了离散贝叶斯滤波器，现在则实现了这一维卡尔曼滤波器。我试图向大家说明：这些滤波器其实都使用着相同的算法和原理。我们稍后要学习的卡尔曼滤波器的数学原理相当复杂，因此很难理解其背后的简单性。不过，这种复杂性也带来了巨大的优势：这种通用型滤波器的性能将远远优于本章所介绍的那些滤波器。

消化本章需要时间，可能需要反复学习几遍。建议修改代码中的常量，观察结果。先通过试验理解：高斯分布为什么能很好地描述对狗的位置、飞机的位置或化学反应室温度的单峰信念；再确认，将先验高斯密度与新测量的似然相乘并归一化，确实得到融合后的新信念；最后理解，预测运动时，独立高斯随机变量相加如何传播位置分布。测量更新中的密度相乘，与运动预测中的随机变量相加，不能混为一谈。

最重要的是，务必花足够的时间仔细阅读“该算法的完整描述”部分，确保自己理解了该算法的原理，以及它与 g-h 滤波器和离散贝叶斯滤波器之间的关系。这里有个“技巧”：需要选择一个介于预测值和测量值之间的数值。虽然各种算法在具体计算方式上有所不同，但所遵循的逻辑其实是一样的。

第 5 章：多元高斯分布

多维度下的不确定性建模

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

上一章中介绍的各种技术虽然非常有效，但它们只能处理单个变量或维度的数据。这些技术无法用来表示多维数据，比如一只狗在田野中的位置和速度。位置和速度之间存在着相互关联的关系。正如我们在 g-h 章节中学到的，我们绝不能丢弃任何有用的信息。在这一章中，我们将学习如何用概率论的方式来描述这种相互关系。通过这一重要思路，我们将能够显著提升滤波器的性能。

多元正态分布

对于标量随机变量，我们一直使用高斯分布来描述，其数学表达式为 $\mathcal{N}(\mu, \sigma^2)$。更正式的称谓是“单变量正态分布”，其中“单变量”指的是只有一个变量。高斯分布所对应的概率分布，就被称为单变量正态分布。

什么是多元正态分布？“多元”表示多个变量，我们希望用它描述多维的正态分布。这里的维度不一定是空间维度：若跟踪飞机在 x、y、z 三个方向上的位置、速度和加速度，就是九维问题。二维情形可以是机器人的 x、y 坐标、狗在 x 轴上的位置与速度，也可以是奶牛的产奶量与饲料投喂量，具体含义并不重要。对于 N 个维度，需要 N 个均值，将它们排成如下列矩阵，也就是列向量：

$$\mu = \begin{bmatrix}\mu_1 \\ \mu_2 \\ \vdots \\ \mu_n\end{bmatrix}$$

假设我们认为 $x = 2$、$y = 17$，则均值向量为：

$$\mu = \begin{bmatrix}2 \\ 17\end{bmatrix}$$

下一步是表示各个变量的方差。乍看之下，我们可能会认为，对于 N 个维度来说，需要 N 个方差值。比如，我们可以说 x 的方差是 10，y 的方差是 4。

$$\sigma^2 = \begin{bmatrix}10\\4\end{bmatrix}$$

这个分析并不完整，因为它没有考虑到更一般的情况。在高斯分布的相关内容中，我们计算了学生身高的方差。方差实际上反映了学生们身高之间的差异程度。如果所有学生的身高都相同，那么方差为 0；而如果学生们的身高差异很大，那么方差就会很大。

身高和体重之间也存在一定的关系。一般来说，身高较高的人，体重也相对较重。身高和体重是相互关联的。我们希望找到一种方式来描述身高和体重的差异程度，以及它们之间的关联强度。换句话说，我们想知道体重是如何随着身高的变化而变化的。我们把这种关系称为协方差。

在理解多元正态分布之前，我们有必要先掌握与相关系数和协方差相关的数学知识。

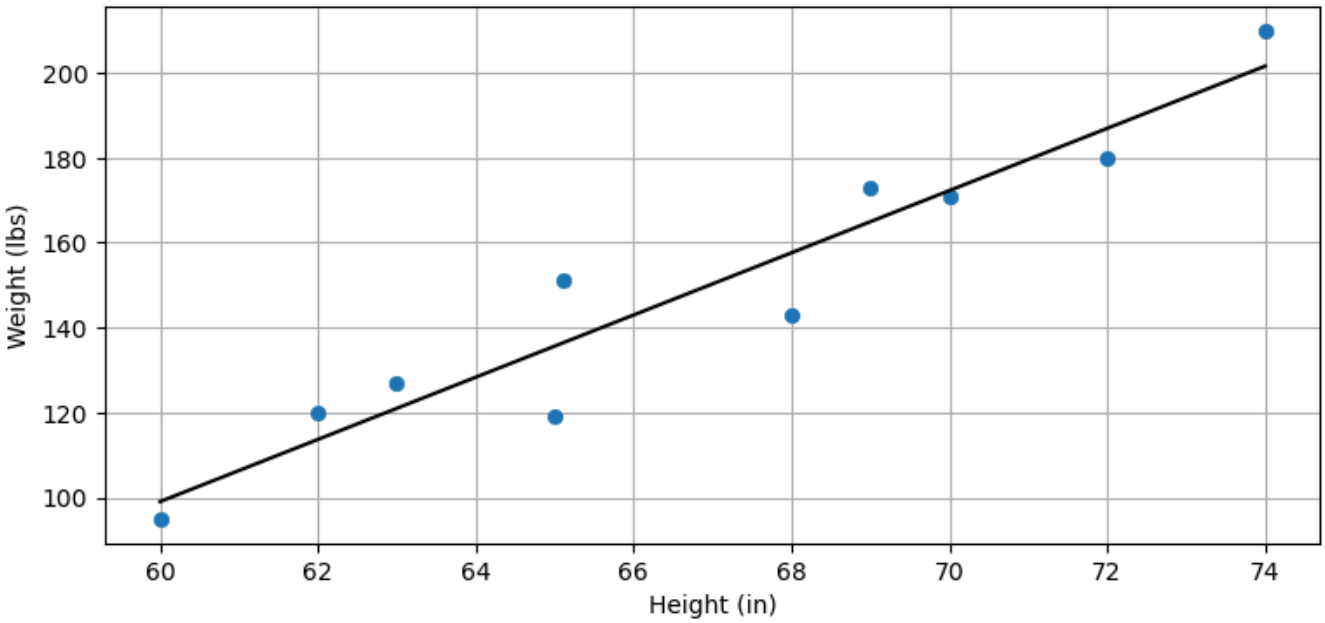
相关性与协方差

协方差描述两个变量共同变化的程度。身高较高的人通常体重也较大，可表现为正相关；室外温度降低、取暖费用增加，可表现为负相关。茶价与狗摇尾巴次数在所设想模型中没有关系，可作为独立的例子。需要严格区分不相关与独立：存在有限二阶矩时，独立蕴含不相关，反过来通常不成立；只有联合高斯等特殊条件下，零协方差才蕴含独立。

相关性有助于进行预测。如果你比我高很多，那么我可以推测你的体重也比我重。随着冬天的到来，我预计自己需要花费更多钱来供暖。不过，如果我的狗摇尾巴的频率增加了，我并不会因此就认为茶叶的价格会发生变化。

例如，下图显示了该校田径队中各学生的身高和体重情况。如果某学生的身高为 68 英寸，那么可以推测他的体重约为 160 磅。不过，由于两者之间的关联并非完全精确，因此我的预测也未必完全准确。

```
from kf_book.gaussian_internal import plot_correlated_data

height = [60, 62, 63, 65, 65.1, 68, 69, 70, 72, 74]
weight = [95, 120, 127, 119, 151, 143, 173, 171, 180, 210]
plot_correlated_data(height, weight, 'Height (in)', 'Weight (lbs)', False)
```



在这本书中，我们只考虑线性相关性。我们假设各变量之间的关系是线性的。也就是说，用直线来描述这些数据是最合适的。我在上图中用直线来拟合了这些数据。虽然也存在非线性相关性的情况，但我们在本书中不会涉及这一概念。

$\boldsymbol{X}$ 与 $\boldsymbol{Y}$ 之间的协方差公式为

$$\boldsymbol{COV(X,Y) = \sigma_{xy} = \mathbb{E}\big[(X-\mu_x)(Y-\mu_y)\big]}$$

其中，$\mathbb{E}[X]$ 表示 X 的期望值，其定义如下：

$$\mathbb{E}[X] = \begin{cases} \sum_{i=1}^{n} p_i x_i & \textbf{discrete} \\ \int_{-\infty}^{\infty} f(x)\, x dx & \textbf{continuous} \end{cases}$$

我们假设每个数据点出现的可能性都是相等的，因此每个数据点被选中的概率都是 $\frac{1}{N}$ 。

$$\mathbb{E}[X] = \frac{1}{N}\sum_{i=1}^{n} x_i$$

以上是我们即将使用的离散情形。

将协方差公式与方差公式进行比较。可以看出，两者非常相似：

$$\boldsymbol{VAR(X) = \sigma_x^2 = \mathbb{E}[(X-\mu)^2]}$$
$$\boldsymbol{COV(X,Y) = \sigma_{xy} = \mathbb{E}\big[(X-\mu_x)(Y-\mu_y)\big]}$$

具体来说，如果你计算 $\boldsymbol{COV(X,X)}$ ，就能得到 $\boldsymbol{VAR(X)}$ 的公式。这一结果进一步印证了我的观点：方差实际上反映了随机变量自身的变动情况。

我们用协方差矩阵来表示多元正态分布中的各变量之间的协方差关系。其形式如下：

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_n^2 \end{bmatrix}$$

对角线上的数值表示各个变量的方差，而对角线以外的数值则表示第 i^{th} 个变量与第 j^{th} 个变量之间的协方差。因此， σ_3^2 表示第三个变量的方差，而 σ_{13} 则表示第一个变量与第三个变量之间的协方差。

协方差为 0 表示两者之间没有相关性。如果 x 的方差为 10， y 的方差为 4，且 x 与 y 之间不存在线性相关性，那么我们可以这样表示：

$$\Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix}$$

如果 x 和 y 之间存在轻微的正相关关系的话，那或许……

$$\Sigma = \begin{bmatrix} 10 & 1.2 \\ 1.2 & 4 \end{bmatrix}$$

其中，1.2 是 x 与 y 的协方差。判断相关强弱时，应将协方差除以两者标准差之积，得到无量纲的相关系数。本例相关系数为 $1.2/\sqrt{40} \approx 0.19$，表示较弱的线性相关。

如果 x 与 y 高度负相关，非对角协方差可以为较大的负数，例如

$$\Sigma = \begin{bmatrix} 10 & -6 \\ -6 & 4 \end{bmatrix}$$

。协方差矩阵必须为半正定矩阵：对角元为 10、4 时，非对角元的绝对值不能超过 $\sqrt{40} \approx 6.325$。本例取 -6，满足这一约束。

协方差矩阵是对称的。毕竟， x 与 y 之间的协方差始终等于 y 与 x 之间的协方差。也就是说，对于任意的 x 和 y，都有 $\sigma_{xy} = \sigma_{yx}$ 成立。

如果前面的概念有些抽象，下面用一个例子说明。高斯分布一章中，有五名学生的身高 H=[1.8,2.0,1.7,1.9,1.6] 米，我们已计算：

$$\begin{aligned} VAR(H) &= E[(H - \mu_H)^2] \\ &= \frac{1}{N}\sum_{i=1}^{n}(H_i - \mu_H)^2 \\ &= \frac{1}{5}\left[(1.8-1.8)^2 + (2-1.8)^2 + (1.7-1.8)^2 + (1.9-1.8)^2 + (1.6-1.8)^2\right] \\ &= 0.02 \end{aligned}$$

很简单，对吧？如果我们称量这些学生的体重，可能会得到如下数值：W = [70.1, 91.2, 59.5, 93.2, 53.5]。那么，我们能否利用协方差公式来计算出协方差矩阵呢？当然可以。计算出来的协方差矩阵如下：

$$\Sigma = \begin{bmatrix} \sigma_H^2 & \sigma_{H,W} \\ \sigma_{W,H} & \sigma_W^2 \end{bmatrix}$$

我们刚刚计算出了身高的方差，该数值会出现在矩阵的左上角。右下角则包含体重的方差。利用同样的公式，我们可以得到：

$$
\begin{aligned}
\mu_W &= \frac{1}{5}(70.1 + 91.2 + 59.5 + 93.2 + 53.5) = 73.5 \\
\sigma_W^2 &= \frac{1}{5}\left[(70.1 - 73.5)^2 + (91.2 - 73.5)^2 + (59.5 - 73.5)^2 + (93.2 - 73.5)^2 + (53.5 - 73.5)^2\right] \\
&= 261.8
\end{aligned}
$$

现在来计算协方差。利用上述公式，我们可以得到：

$$
\begin{aligned}
\sigma_{H,W} &= \mathbb{E}\left[(H - \mu_H)(W - \mu_W)\right] \\
&= \frac{1}{N}\sum_{i=1}^{n}(H_i - \mu_H)(W_i - \mu_W) \\
&= \frac{1}{5}[(1.8 - 1.8)(70.1 - 73.5) + (2 - 1.8)(91.2 - 73.5) + (1.7 - 1.8)(59.5 - 73.5) + \\
&\quad (1.9 - 1.8)(93.2 - 73.5) + (1.6 - 1.8)(53.5 - 73.5)] \\
&= 2.18
\end{aligned}
$$

那虽然很繁琐，但其实也挺简单的。我们再也不会这么做啦，因为 NumPy 当然会帮我们完成这项计算工作。

```
import numpy as np

W = [70.1, 91.2, 59.5, 93.2, 53.5]
H = [1.8, 2.0, 1.7, 1.9, 1.6]
np.cov(H, W)
```

```
array([[  0.025,   2.727],
       [  2.727, 327.235]])
```

这与我们的计算结果不符！到底哪里出错了？其实没什么问题。NumPy 在处理小样本量时会有相应的修正措施：它使用 $\frac{1}{N-1}$ 作为标准化参数，而不是 $\frac{1}{N}$ 。

这里需要区分抽样误差与估计偏差。假设班级有 200 人，只测了其中 5 人，所得统计量通常与整个班级的描述统计量不同，这属于抽样误差。无偏的含义是：在同一抽样机制下反复抽样，估计量的期望等于所估计的总体参数，并不要求每个样本都没有误差。

在独立同分布抽样、总体均值未知并用样本均值替代的条件下，计算样本协方差通常除以 N-1，而不是 N；其倒数就是 $\frac{1}{N-1}$，这给出总体协方差的无偏估计。np.cov 默认采用 N-1。若只想描述手中这一整组有限数据，以 N 为分母，可设 bias=1；这与从样本估计未知总体协方差是不同的统计目标。

```
np.cov(H, W, bias=1)
```

```
array([[  0.02 ,   2.182],
       [  2.182, 261.788]])
```

这与手工按 N 为分母的计算一致，因为本例把列出的学生当作所描述的完整总体。在后续从随机过程取样来估计未知总体协方差时，通常使用默认的样本修正；应根据具体统计目标决定，而不是把 bias=1 一概称为无偏选项。

这个矩阵告诉我们什么信息呢？它表明，身高的方差为 0.02 m^2 ，而体重的方差则为 261.788 kg^2 。此外，该矩阵还显示，体重和身高之间存在正相关关系——也就是说，随着身高的增加，体重也会相应增加。

让我们来创建完全相关的数据吧。所谓“完全相关”，指的是数据点完全沿着一条直线分布——没有任何偏离这条直线的现象。

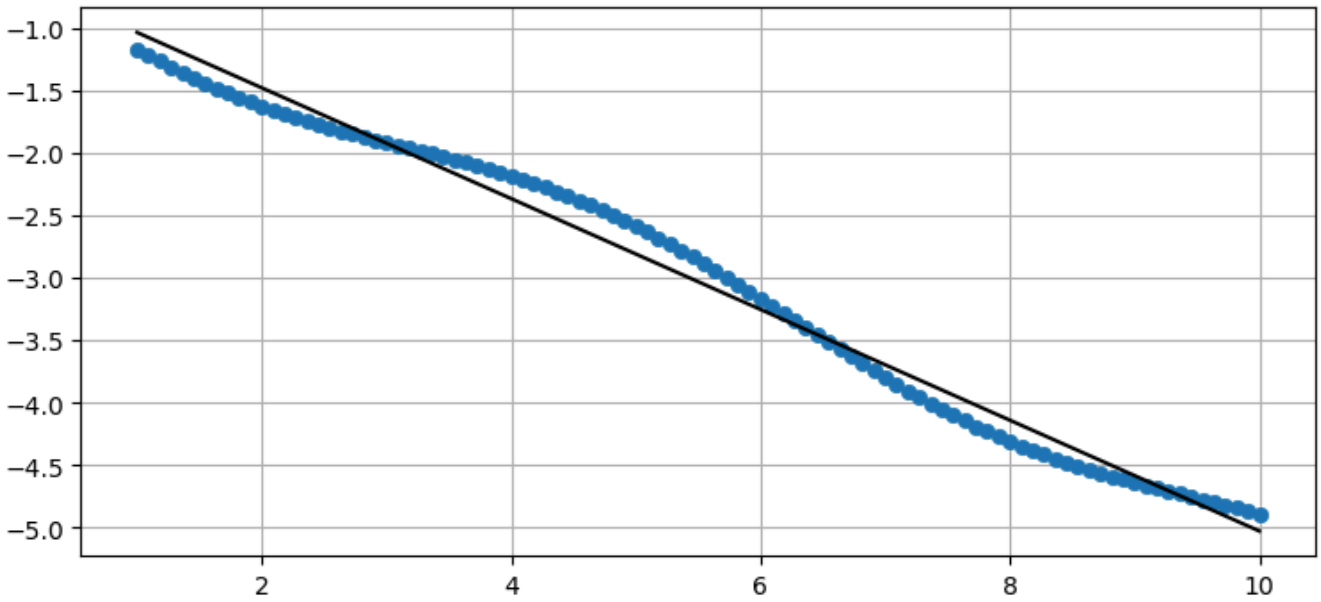
```
X = np.linspace(1, 10, 100)
Y = np.linspace(1, 10, 100)
np.cov(X, Y)
```

```
array([[6.956, 6.956],
       [6.956, 6.956]])
```

从协方差矩阵可以看出，x 和 y 的协方差实际上等于各自的方差。

现在，让我们对其中一个变量添加一些“噪声”，这样它们就不再完全相关了。我会把 Y 的值设为负数，从而产生负相关关系。

```
X = np.linspace(1, 10, 100)
Y = -(np.linspace(1, 5, 100) + np.sin(X)*.2)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```

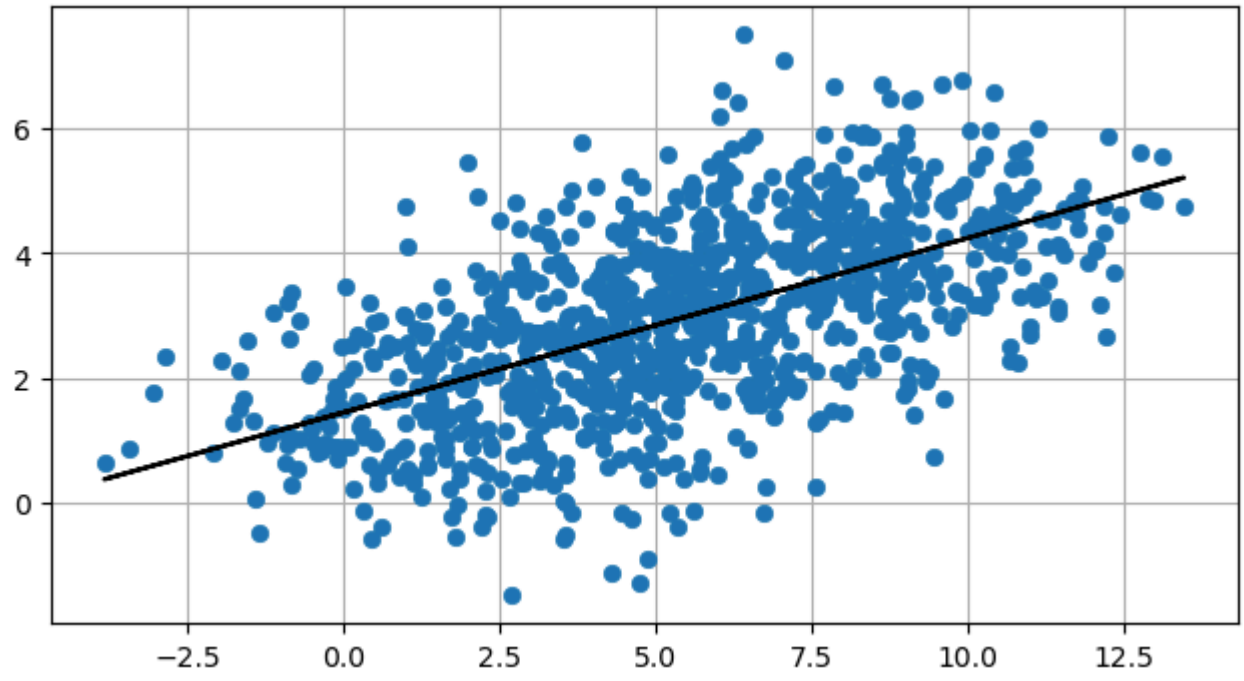



```
[[ 6.956 -3.084]
 [-3.084  1.387]]
```

这些数据不再呈直线形态。协方差值为 $\sigma_{xy} = -3.08$ 。与 σ_x^2 和 σ_y^2 的数值相比，该值远非零，因此可以确定两者之间仍然存在高度相关性。我们可以通过查看图表来验证这一点：数据点大致呈直线分布。

现在，我会在一条直线上添加随机噪声。

```
from numpy.random import randn
X = np.linspace(1, 10, 1000) + randn(1000)*2
Y = np.linspace(1, 5, 1000) + randn(1000)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```

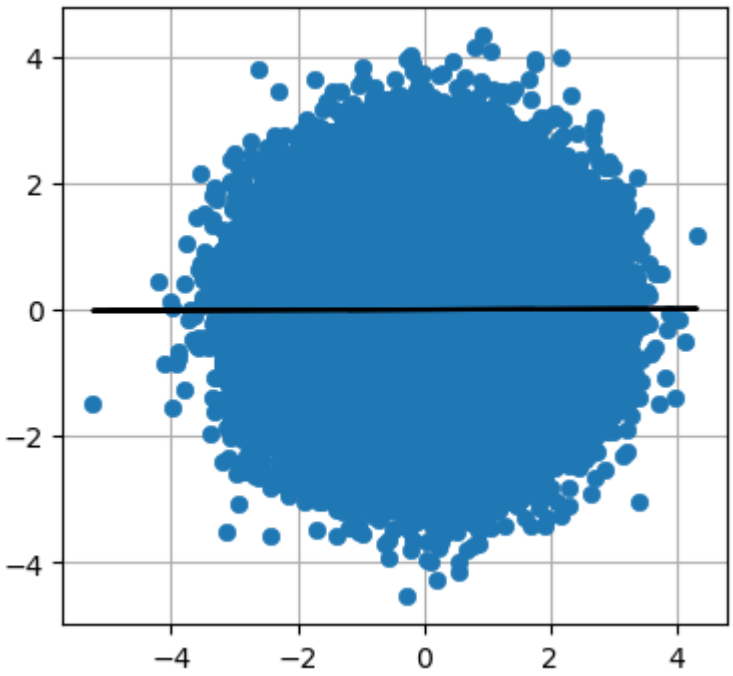



```
[[10.295  2.875]
 [ 2.875  2.299]]
```

我们发现，协方差相对于方差来说要小，这反映出 X 与 Y 之间的相关性较低。我们仍然可以用直线来拟合这些数据，但数据的波动幅度要大得多。

最后，这是完全随机数据之间的协方差。

```
X = randn(100000)
Y = randn(100000)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```




```
[[1.003 0.003]
 [0.003 1.001]]
```

这里的协方差值几乎为零。从图表中可以看出，根本无法画出一条能够准确描述数据的直线。无论是垂直线还是水平线，都同样无法有效描述这些数据。

多元正态分布方程

请回想一下高斯分布相关章节中关于正态分布的公式：

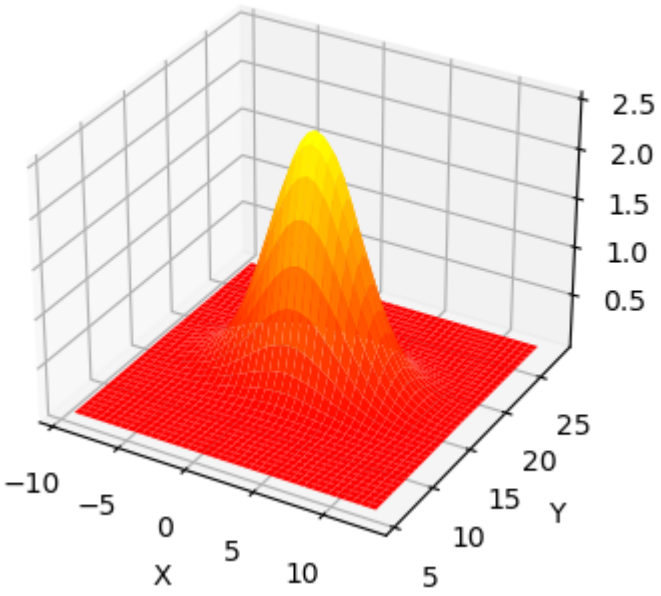
$$f(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left[-\frac{1}{2}(x-\mu)^2/\sigma^2\right]$$

以下是 $\boldsymbol{n}$ 维空间中的多元正态分布。

$$f(\mathbf{x}, \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^n|\Sigma|}}\exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^\mathsf{T}\Sigma^{-1}(\mathbf{x}-\mu)\right]$$

多元版本只不过是用矩阵来替代了单变量方程中的标量而已。如果你对线性代数有一定的了解，那么这个方程应该不难理解。如果你不太熟悉线性代数，也不用担心，FilterPy 和 SciPy 都提供了相应的函数来帮你计算。暂且先不考虑具体的计算过程，我们先来绘制一下这个方程的图像，看看它到底是什么样子的。

```
import kf_book.mkf_internal as mkf_internal

mean = [2., 17.]
cov = [[10., 0.],
       [0., 4.]]

mkf_internal.plot_3d_covariance(mean, cov)
```



这幅多元高斯分布图的均值为 $\mu = \begin{bmatrix}2\\17\end{bmatrix}$，协方差为 $\Sigma = \begin{bmatrix}10 & 0\\0 & 4\end{bmatrix}$。对于任意平面位置 (X, Y)，曲面在 z 轴上的高度表示概率密度。我将 x、y 两个方向的边缘分布投影到图的侧壁，可以看到它们都是高斯钟形曲线。X 对应的曲线比 Y 宽，因为 $\sigma_x^2 = 10$、$\sigma_y^2 = 4$。三维曲面的最高点位于 X 与 Y 的均值对应的位置。

多元高斯分布的密度曲面具有这样的钟形。如果把它看作狗的位置分布，那么各个 (X, Y) 位置上曲面的 z 值，就是狗位于该处的概率密度。严格说，这叫联合概率密度函数，稍后会定义。因此，狗最可能出现在 (2,17) 附近，在 (5,14) 附近的可能性较低，在 (10,10) 附近则很低。与一元情形一样，这里表示的是概率密度，而不是概率。连续分布有无限多个可能取值，恰好位于 (2,17) 或任意单独一点的概率都为零。对曲面下方体积积分，才能得到落在某个区域内的概率。

FilterPy [2] 在 filterpy.stats 模块中通过 multivariate_gaussian() 函数实现这一公式。SciPy 的 stats 模块则通过 multivariate_normal() 实现多元正态分布的计算。后者支持“冻结”分布：只需设定一次均值和协方差，之后便可以多次调用，对任意数量的 x 值计算概率密度。我将自己的函数命名为 multivariate_gaussian()，以免与 SciPy 的版本混淆。

> `scipy.stats` 模块的教程[1]详细介绍了如何“冻结”数据分布，以及该模块的其他实用功能。

```
from filterpy.stats import gaussian, multivariate_gaussian
```

我会先演示如何使用它，然后再介绍一些更有趣的内容。

首先，假设我们的狗位于点(2, 7)，其中 x 的方差为 8，y 的方差为 3。那么，我们就可以计算出狗位于点(2.5, 7.3)时的概率密度。

首先，将 x 设置为(2.5, 7.3)。你可以使用元组、列表或 NumPy 数组来表示这个数值。

```
x = [2.5, 7.3]
```

接下来，我们确定信念的均值：

```
mu = [2.0, 7.0]
```

最后定义协方差矩阵。问题没有指定 $\boldsymbol{x}$ 与 $\boldsymbol{y}$ 相关，因此这里假设不相关；直觉上，狗可以在 $\boldsymbol{x}$ 方向运动，而不必改变 $\boldsymbol{y}$ 坐标，反之亦然。使用变量名 P，因为卡尔曼滤波惯例以 $\mathbf{P}$ 表示状态估计协方差矩阵。

```
P = [[8., 0.],
     [0., 3.]]
```

现在调用该函数吧。

```
%precision 4
multivariate_gaussian(x, mu, P)
```

```
0.0315
```

我们也可以通过 `scipy.stats` 模块得到同样的结果。

```
import scipy
from scipy.stats import multivariate_normal
print(f'{multivariate_normal(mu, P).pdf(x):.4f}')
```

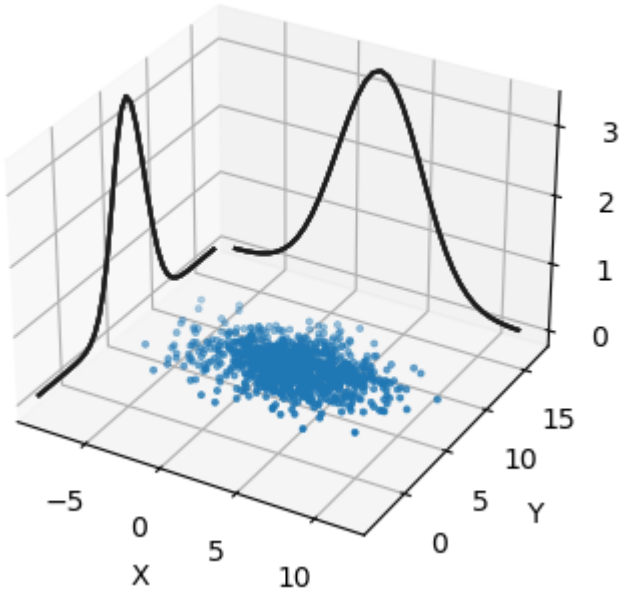
```
0.0315
```

是时候定义一些术语了。联合概率，用 $\boldsymbol{P(x, y)}$ 表示，指的是 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 同时发生的概率。例如，如果你掷两个骰子，$\boldsymbol{P(2, 5)}$ 就是第一个骰子掷出 2、第二个骰子掷出 5 的概率。假设这两个骰子都是六面体且公平无偏，那么该概率为 $\boldsymbol{P(2, 5) = \frac{1}{6} \times \frac{1}{6} = \frac{1}{36}}$。上方的三维图表展示了联合概率密度函数。

边缘分布描述某个变量，而不要求同时指定其他变量的取值。上图左侧的曲线是 $\boldsymbol{Y}$ 的边缘概率密度，表示只考虑狗在 $\boldsymbol{Y}$ 方向的位置、对 $\boldsymbol{X}$ 的全部可能值积分后的分布。前面所说投影到图侧壁的曲线，正是 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 的边缘密度。多元高斯分布还有一个计算上的优点：其边缘分布仍是高斯分布。

换一种方式观察：不画概率密度曲面，而是从给定均值、协方差矩阵为 $\left[\begin{smallmatrix}8 & 0\\0 & 3\end{smallmatrix}\right]$ 的多元高斯分布中生成 1000 个样本点。

```
mkf_internal.plot_3d_sampled_covariance(mu, P)
```



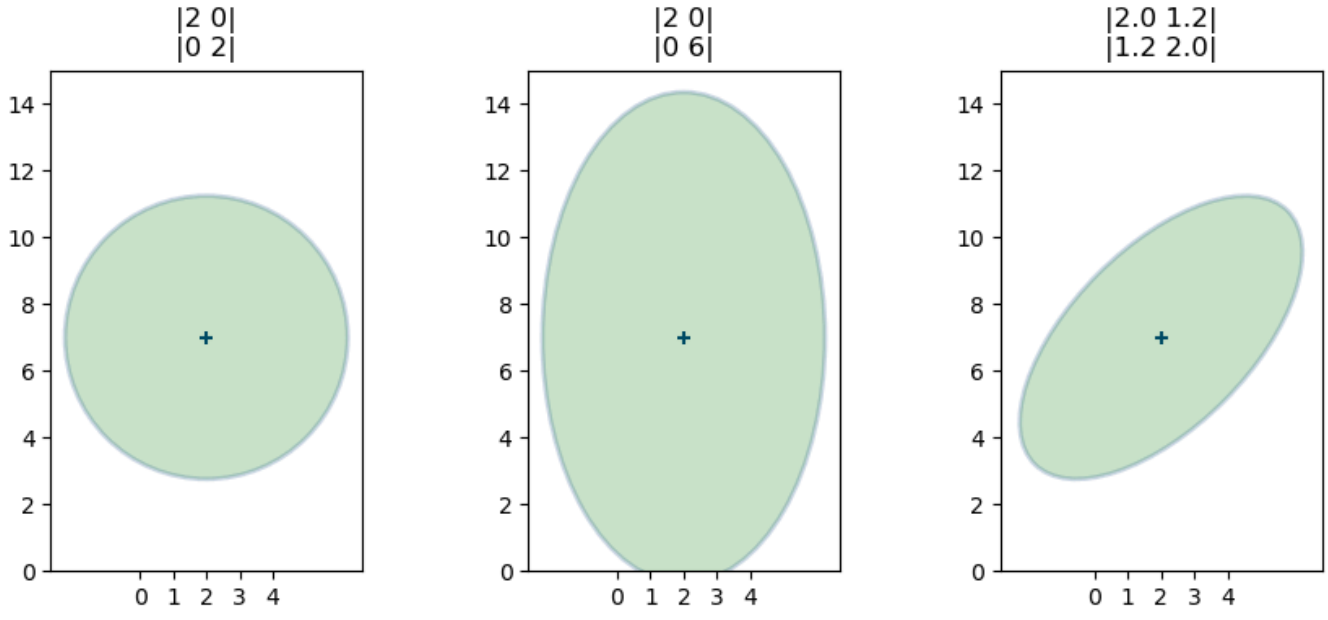
这些样本点可看作在给定均值与协方差下，狗可能出现的位置。侧边的曲线表示 $\boldsymbol{X}$ 与 $\boldsymbol{Y}$ 的边缘概率密度。(2,7) 附近样本密集，说明落在其附近区域的概率较大；(-5,5) 附近样本稀少，相应概率较小。

虽然这些图表看起来很美观，但很难从中获取有用的信息。例如，很难判断 $\boldsymbol{X}$ 和 $\boldsymbol{Y}$ 是否具有相同的方差，以及它们之间的相关程度如何。在本书的大部分内容中，我都会用等高线图来表示高斯分布。

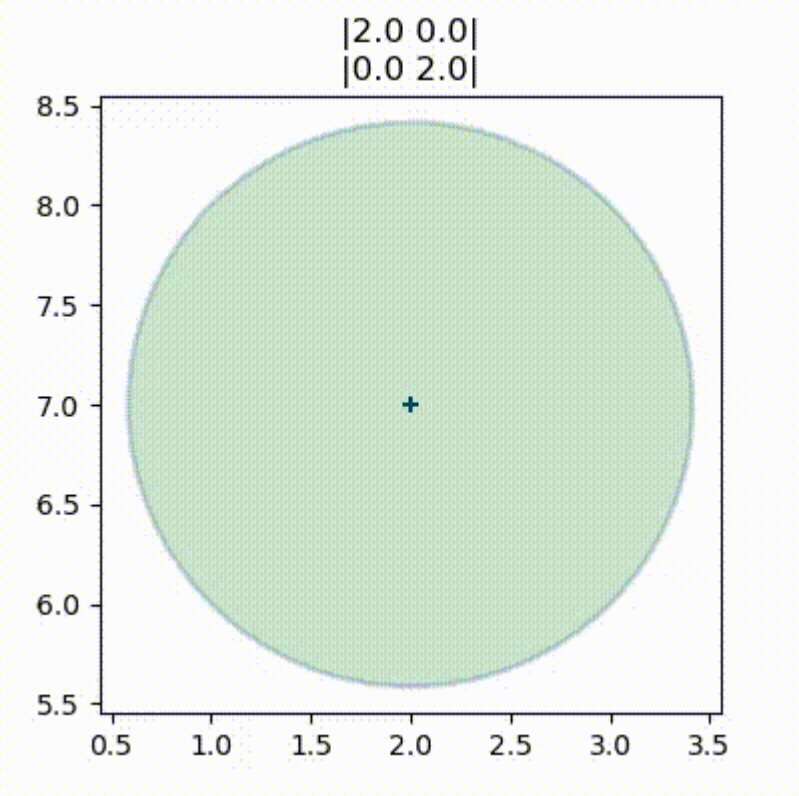
等高线图展示了在特定的标准差下，多元高斯分布所取值的范围。这就好比从三维图中切下水平的一层而已。

这些图表展示了在 3 个标准差范围内的数据分布情况。

```
mkf_internal.plot_3_covariances()
```



对于那些通过在线方式或使用电脑上的 Jupyter Notebook 来观看此内容的人士，这里有一个动画，展示了在方差保持不变的情况下，协方差如何发生变化。

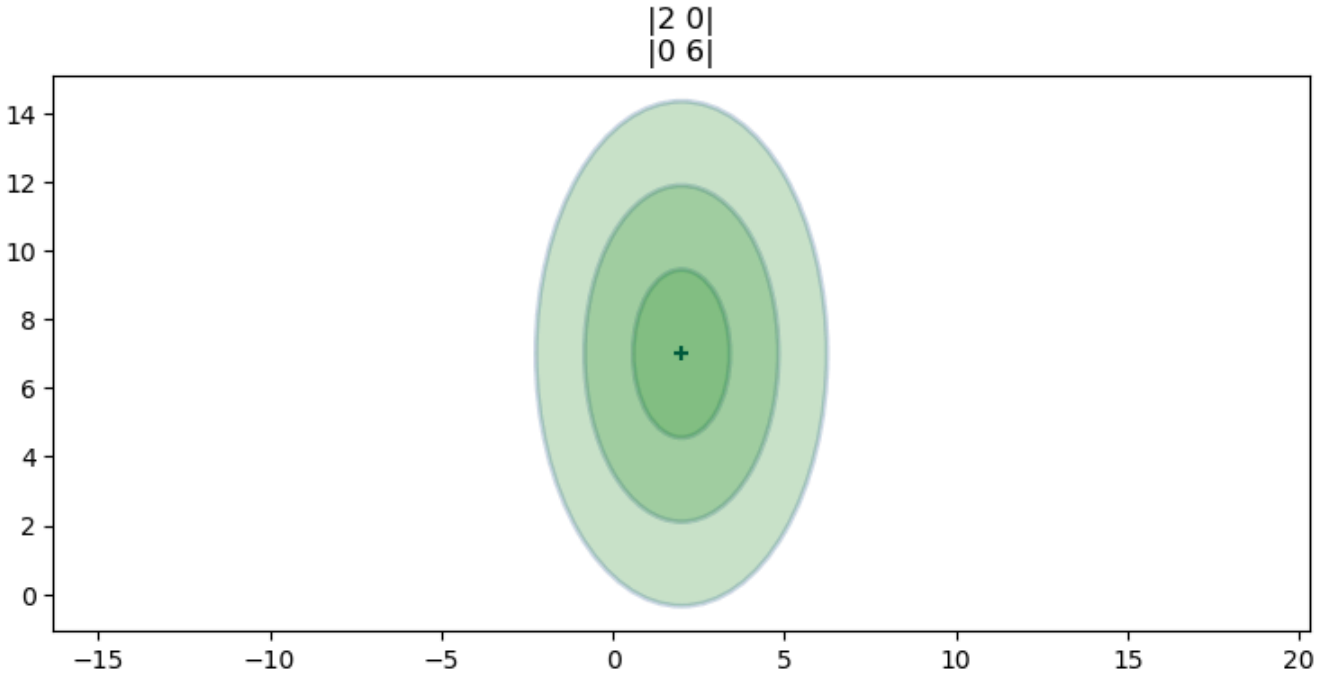


(来源：http://git.io/vqxLS)

这些等密度轮廓为圆或椭圆，也常称为协方差椭圆、误差椭圆；指定概率覆盖率后，可据相应尺度构造置信椭圆。它们对应密度曲面的水平切片边界，而不是任意方向截面都必为椭圆。不同维数下，同样的标准差倍数也不对应同一覆盖概率。

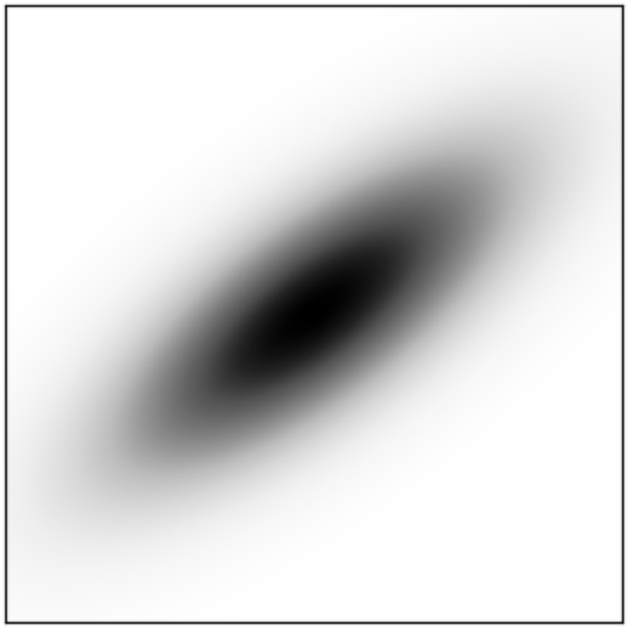
代码使用 filterpy.stats 的 plot_covariance_ellipse()，默认画出一倍标准差的协方差椭圆。可以用 variance 或 std 参数控制尺度：variance=3**2 或 std=3 画出三倍标准差椭圆；variance=[1,4,9] 或 std=[1,2,3] 则同时画出一、二、三倍标准差椭圆。

```
from filterpy.stats import plot_covariance_ellipse
import matplotlib.pyplot as plt

P = [[2, 0], [0, 6]]
plot_covariance_ellipse((2, 7), P, fc='g', alpha=0.2,
                        std=[1, 2, 3],
                        title='|2 0|\n|0 6|')
plt.gca().grid(visible=False);
```



图中的纯色区域可能让人误以为，相邻标准差轮廓之间的概率密度是常数。实际上并非如此，前面的高斯三维图已展示了密度变化。下面用二维灰度图表示协方差矩阵为 $\begin{pmatrix} 2 & 1.2 \\ 1.2 & 1.3 \end{pmatrix}$ 的概率分布；灰色越深，概率密度越高。

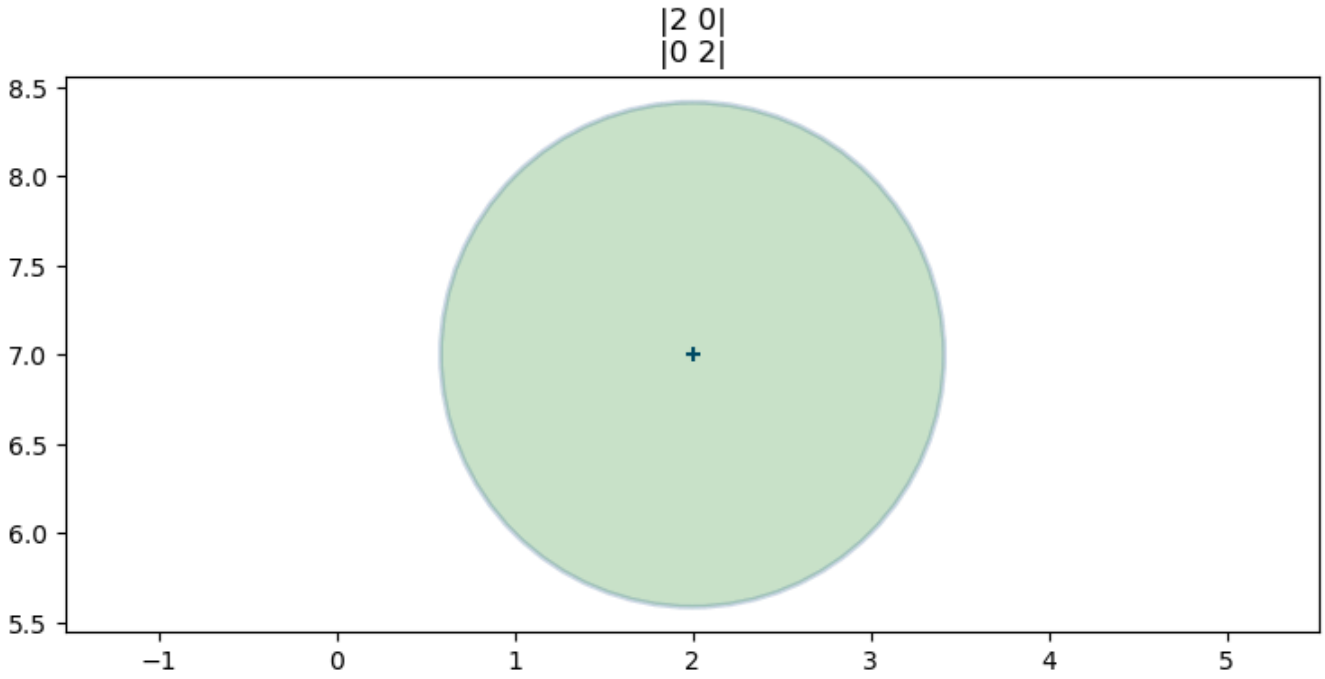
```
from kf_book.nonlinear_plots import plot_cov_ellipse_colormap
plot_cov_ellipse_colormap(cov=[[2, 1.2], [1.2, 1.3]]);
```



仔细思考这些图表的物理意义后，就能更清楚地理解它们的含义了。第一个图表的均值和协方差分别为……

$$\mu = \begin{bmatrix} 2 \\ 7 \end{bmatrix},\ \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

```
x = [2, 7]
P = [[2, 0], [0, 2]]
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                        title='|2 0|\n|0 2|')
plt.gca().grid(visible=False)
```

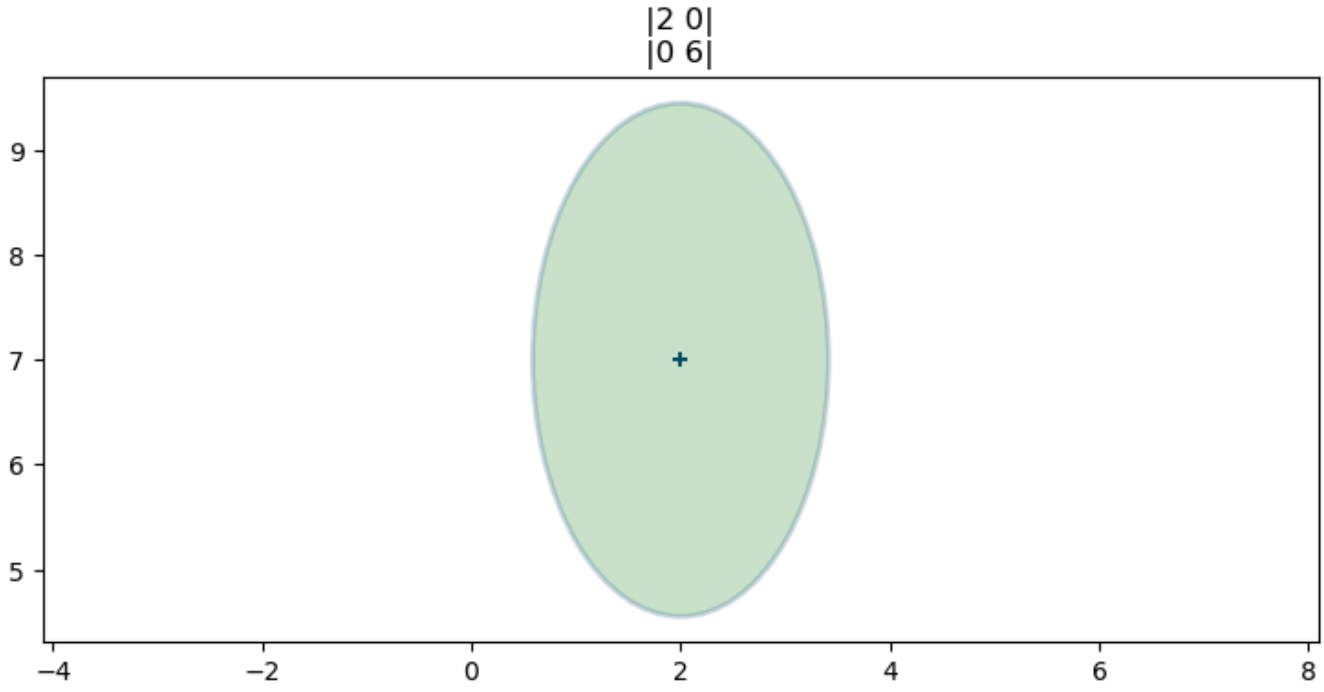


从贝叶斯角度，椭圆显示信念的不确定性：尺度很小的圆表示不确定性小，尺度很大的圆表示不确定性大。形状反映 $\boldsymbol{X}$、$\boldsymbol{Y}$ 两个方向误差的几何关系；本例为圆，表示 $\boldsymbol{X}$、$\boldsymbol{Y}$ 方向具有相同方差且没有相关性。

第二个图形的均值和协方差分别为：

$$\mu = \begin{bmatrix}2\\7\end{bmatrix},\ \Sigma = \begin{bmatrix}2 & 0\\0 & 6\end{bmatrix}$$

```
x = [2, 7]
P = [[2, 0], [0, 6]]
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                        title='|2 0|\n|0 6|')
plt.gca().grid(visible=False)
```

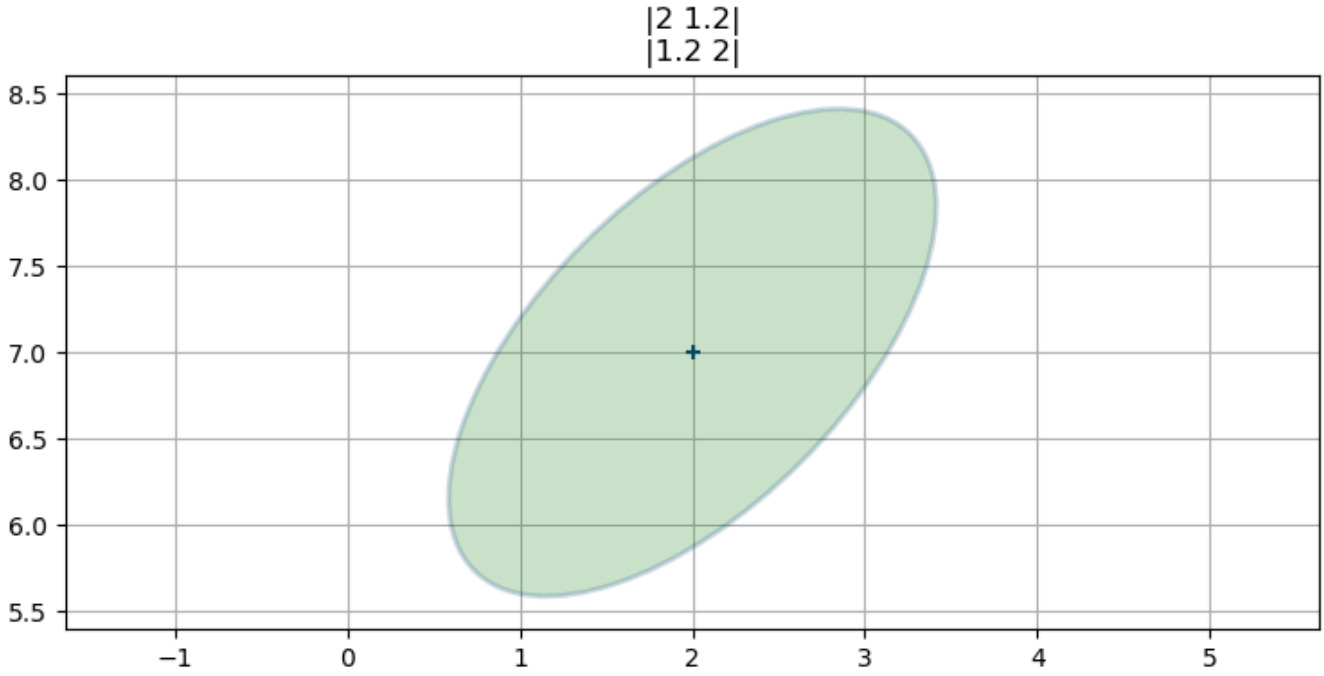


这一次，我们对 $\boldsymbol{X}$ ($\boldsymbol{\sigma_x^2 = 2}$)和 $\boldsymbol{Y}$ ($\boldsymbol{\sigma_y^2 = 6}$)使用了不同的方差值。结果得到的是一个又高又窄的椭圆形。可以看出， $\boldsymbol{Y}$ 的不确定性要远远大于 $\boldsymbol{X}$ 的不确定性。在这两种情况下，我们都认为狗的位置在(2, 7)，但两者之间的不确定性程度不同。

第三幅图表展示了平均值和协方差。

$$\mu = \begin{bmatrix}2\\7\end{bmatrix},\ \Sigma = \begin{bmatrix}2 & 1.2\\1.2 & 2\end{bmatrix}$$

```
x = [2, 7]
P = [[2, 1.2], [1.2, 2]]
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                        title='|2 1.2|\n|1.2 2|')
```



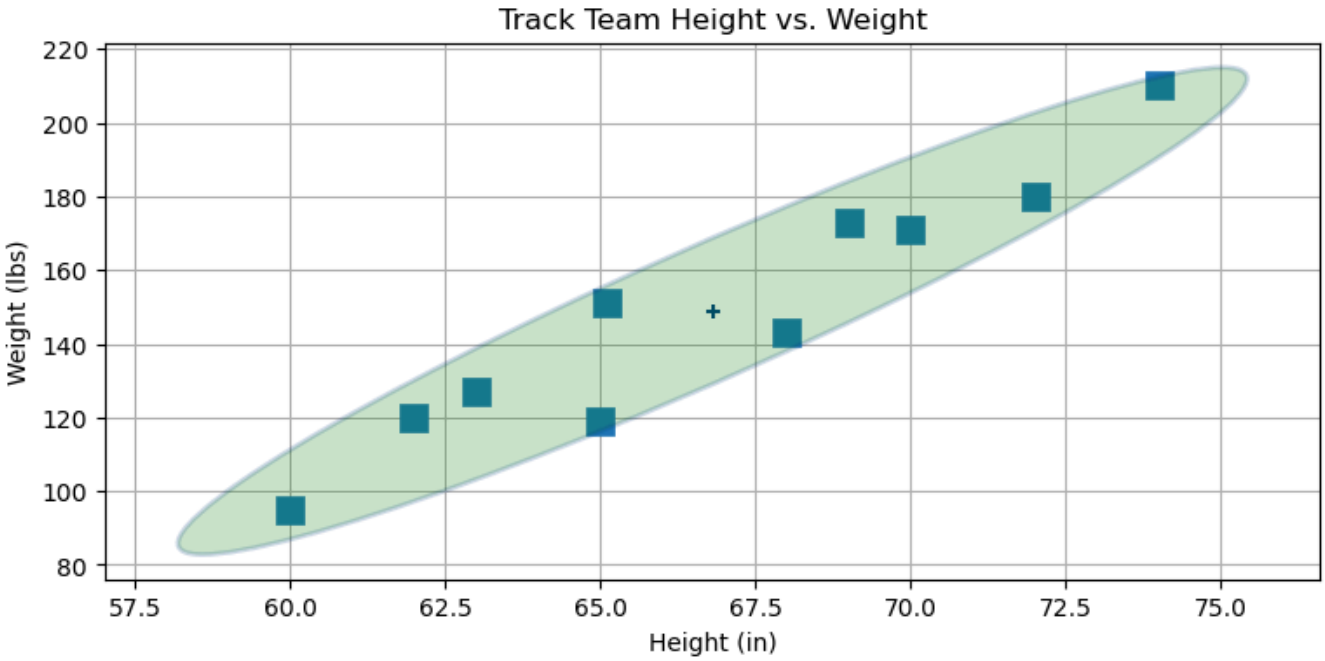
这是第一个在协方差矩阵的非对角线元素上也有数值的等高线图。同时，这也是第一个包含倾斜椭圆的等高线图。这并非巧合。这两个现象其实都在告诉我们同样的道理：倾斜的椭圆表明，x 和 y 这两个数值之间存在某种关联。协方差矩阵中的非对角线元素不为零，这也进一步说明了两者之间存在相关性。

回忆身高与体重的散点图，数据点形成了倾斜的点云。把两个或更多变量放入二维数组后，就可以用 NumPy 的 cov() 计算协方差。下面先计算，再把 2σ 协方差椭圆叠加到数据上。这里的数据被视为整个总体，而非总体的一个样本，因此使用 bias=1。

```
cov_hw = np.cov(np.vstack((height, weight)), bias=1)
cov_hw
```

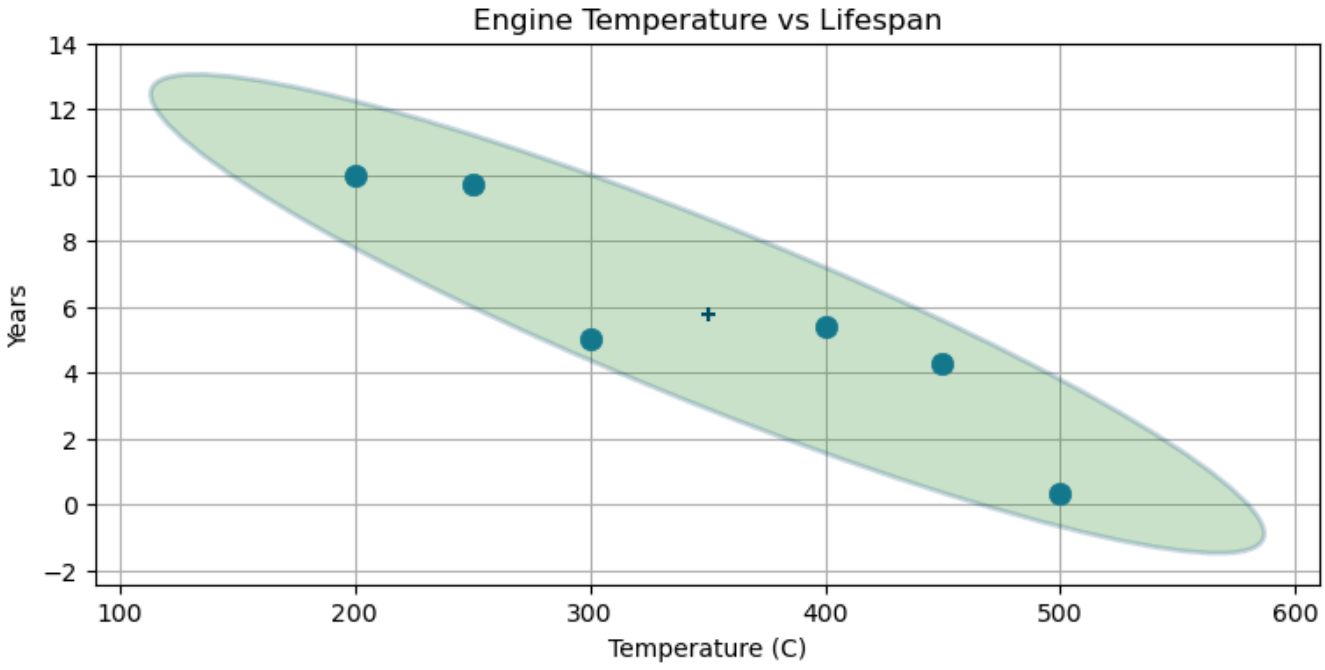
```
array([[  18.5249,  135.701 ],
       [ 135.701 , 1092.29  ]])
```

```
plt.scatter(height, weight, s=120, marker='s')
plt.title('Track Team Height vs. Weight')
plt.xlabel('Height (in)'); plt.ylabel('Weight (lbs)')
plot_covariance_ellipse((np.mean(height), np.mean(weight)), cov_hw, fc='g',
                        alpha=0.2, axis_equal=False, std=2)
```



协方差椭圆显示两个变量如何共同散布。在这个身高一体重例子中，点云沿一条斜线聚集：给定身高后，体重的可能范围比总体范围窄得多，因此二者相关性很强。椭圆从左下向右上倾斜，表示正相关，即 x 增大时 y 通常也增大；反向倾斜则表示负相关。判断相关强弱时还须考虑坐标尺度，不能仅凭椭圆细长就判定强相关；两个不相关、方差差别很大的变量也会形成细长椭圆。

```
max_temp = [200, 250, 300, 400, 450, 500]
lifespan = [10, 9.7, 5, 5.4, 4.3, 0.3]

plt.scatter(max_temp, lifespan, s=80)
cov = np.cov(np.vstack((max_temp, lifespan)))
plot_covariance_ellipse((np.mean(max_temp), np.mean(lifespan)), cov, fc='g',
            alpha=0.2, axis_equal=False, std=2)
plt.title('Engine Temperature vs Lifespan')
plt.xlabel('Temperature (C)'); plt.ylabel('Years');
```



通过直观观察，很难理解方差和协方差之间的关系。因此，这里有一个交互式图表来帮助理解。如果你是以静态文本形式阅读此内容，请点击此处了解如何在线查看该图表：https://git.io/vza7b

```
from ipywidgets import interact
from kf_book.book_plots import figsize, FloatSlider

fig = None
def plot_covariance(var_x, var_y, cov_xy):
    global fig
    if fig: plt.close(fig)
    fig = plt.figure(figsize=(4,4))
    P1 = [[var_x, cov_xy], [cov_xy, var_y]]

    plot_covariance_ellipse((10, 10), P1, axis_equal=False,
                            show_semiaxis=True)

    plt.xlim(4, 16)
    plt.gca().set_aspect('equal')
    plt.ylim(4, 16)


with figsize(y=6):
    interact (plot_covariance,
          var_x=FloatSlider(5, min=0, max=20),
          var_y=FloatSlider(5, min=0, max=20),
          cov_xy=FloatSlider(1.5, min=0, max=50, step=.2));
```

```
interactive(children=(FloatSlider(value=5.0, continuous_update=False, description='var_x',
max=20.0), FloatSli…
```

皮尔逊相关系数

在本书中，我们不会使用这个系数。不过，你可能会在其他地方看到它。如果你不感兴趣的话，可以直接跳过这一部分。

两个变量之间的相关性可以用皮尔逊相关系数来表示为一个数值。其定义如下：

$$\rho_{xy} = \frac{COV(X,Y)}{\sigma_x \sigma_y}$$

这个值介于 -1 和 1 之间。协方差为 0 时，$\rho = 0$。大于 0 表示正相关，小于 0 表示负相关。接近 -1 或 1 表示相关程度很强，接近 0 则表示线性相关程度很弱。

协方差与相关系数密切相关。协方差的单位是两个变量单位之积，皮尔逊相关系数则是无量纲比率。例如，狗的 x、y 坐标均以米计，协方差 σ_{xy} 的单位就是平方米。

我们可以使用 `scipy.stats.pearsonr` 函数来计算皮尔逊系数。该函数会返回一个包含皮尔逊系数和双尾 p 值的元组。不过，在本书中，我们不会使用双尾 p 值。这里，我们以学生运动员的身高与体重为例来计算 ρ 值：

```
from scipy.stats import pearsonr
pearsonr(height, weight)[0]
```

```
0.9539731096080194
```

在这里，我们计算了发动机温度与使用寿命之间的相关性。

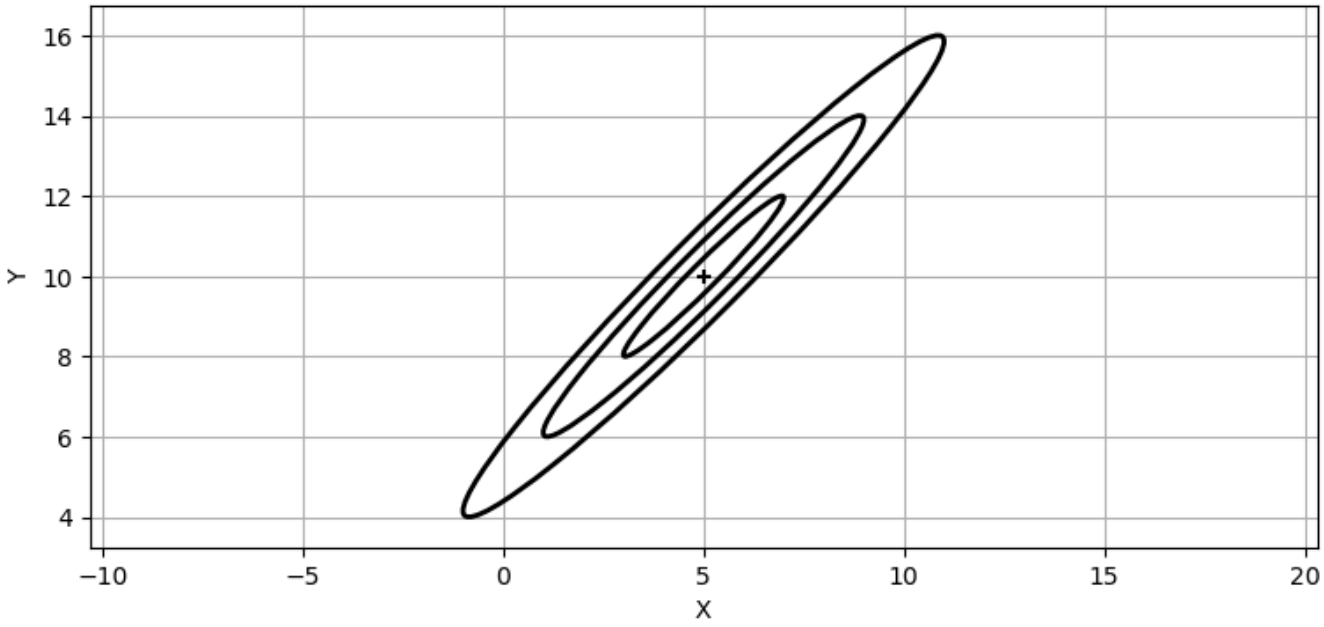
```
pearsonr(max_temp, lifespan)[0]
```

```
-0.9178223453527254
```

利用相关性来提高估算精度

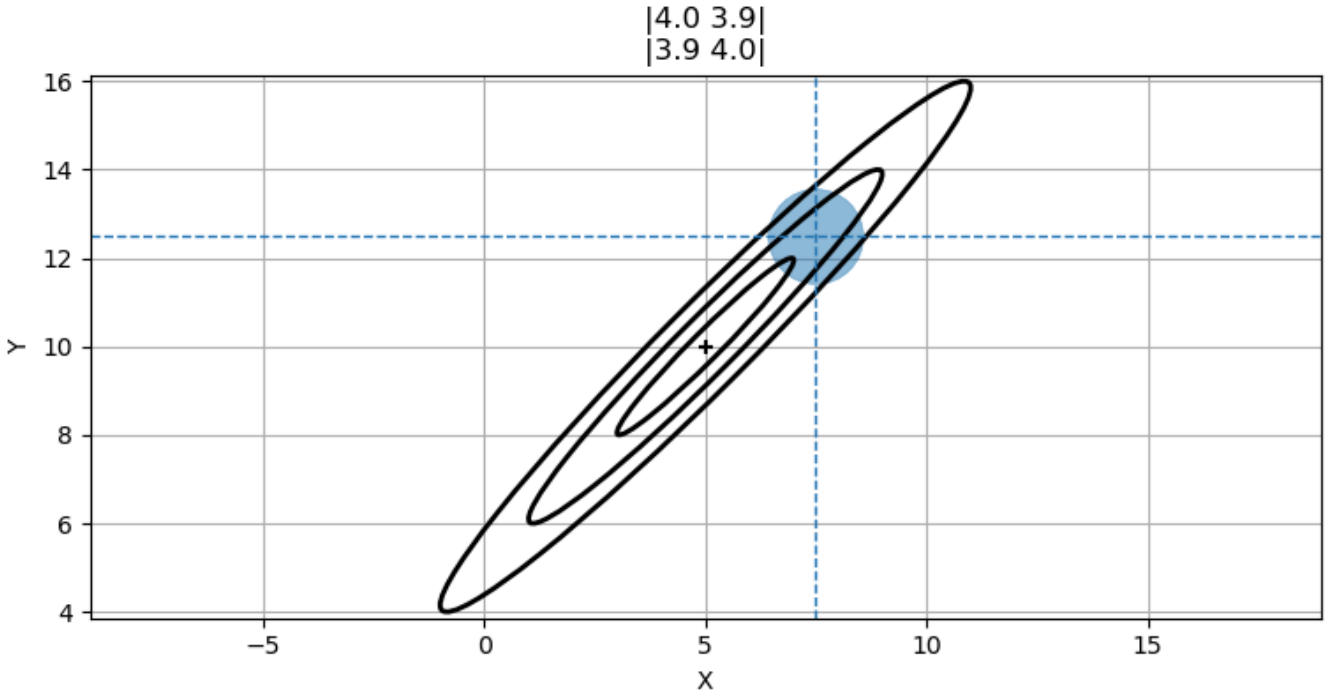
假设我们认为，我们的狗位于坐标点(5, 10)处，且该位置的协方差值已知。如果 x 轴和 y 轴上的标准差均为 2 米，但两者之间存在很强的相关性，那么协方差等高线就会呈现类似这样的形状。

```
P = [[4, 3.9], [3.9, 4]]

plot_covariance_ellipse((5, 10), P, ec='k', std=[1, 2, 3])
plt.xlabel('X')
plt.ylabel('Y');
```



现在，假设我告诉你，我们知道 $\boldsymbol{x = 7.5}$ 的值。那么，关于 $\boldsymbol{y}$ 的值，我们能推断出什么呢？ $\boldsymbol{y}$ 的值极有可能位于 3 个 $\boldsymbol{\sigma}$ 所构成的协方差椭圆范围内。由于 x 和 y 之间存在相关性，我们可以利用协方差矩阵来推断 y 的值。我用蓝色实心圆标出了 y 的可能取值范围。

```
mkf_internal.plot_correlation_covariance()
```



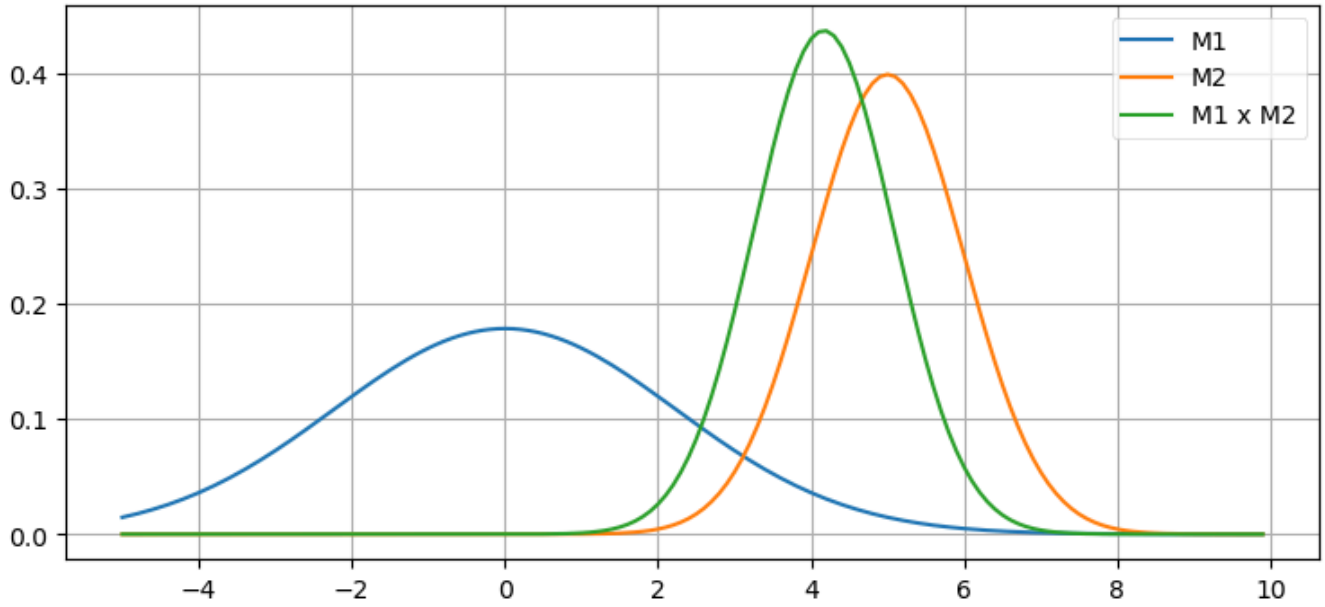
这个圆在数学上并不完全准确，但大致能表达出我们的意图。我们将在下一节中详细探讨其中的数学问题。目前可以确定的是，y 的值很可能接近 12。而 $y = -10$ 的值则几乎不可能出现。

再谈谈相关性与独立性。变量独立时，其取值可分别变化。例如在开阔田野中，可以沿 x 方向（东西）、y 方向（南北），或它们的组合方向行走。独立变量在有关矩存在时必然不相关，反过来除特定情形外并不成立：变量可以不相关，却仍然相互依赖。考虑 $y = x^2$。相关系数只反映线性关联，在关于零对称且有关矩存在的 x 分布下，x 与 y 不相关，但 y 显然依赖 x。

多维高斯函数的乘法运算

在上一章中，我们通过将具有不确定性的测量结果和估计值的高斯分布相乘，来处理这些不确定性。这样得到的结果仍然是高斯分布，只不过其方差更小了。如果两段不确定的信息能够相互印证，那么我们得出的结论就会更可靠。相关图表如下所示：

```
mkf_internal.plot_gaussian_multiply()
```



将测量结果 1 和 2 结合起来后，结果的确定性会更高。因此，新的高斯分布曲线会更高、更窄——方差也变小了。在多维情况下，多元高斯分布也是如此。

下面给出多元高斯分布相乘的公式。大写希腊字母 Sigma（Σ）表示矩阵而非标量，具体来说就是协方差矩阵：

$$\mu = \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}\mu_1 + \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\mu_2$$
$$\Sigma = \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\Sigma_2$$

将先验和测量似然对应的多元高斯密度代入贝叶斯定理，就能推导这些公式。高斯分布一章已经给出了一元情形的代数推导。

你无需记住这些方程式，因为它们会由卡尔曼滤波公式来计算，这些公式稍后会详细介绍。在 FilterPy 中，也可以使用 `multivariate_multiply()` 方法来进行这些计算，你可以通过 `filterpy.stats` 来导入该函数。

为了让你更直观地理解这一点，不妨回想一下关于单变量高斯分布相乘的公式：

$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2},$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

这与前面一元情形的公式相似。如果注意到 -1 次幂表示矩阵求逆，而 $\boldsymbol{AA^{-1}} = \boldsymbol{I}$，就更容易看出联系：求逆与标量取倒数有相似之处。下面暂时把求逆写成类似除法的形式，便于比较。注意，这不是严格的矩阵运算写法，因为矩阵除法并未在这里定义。

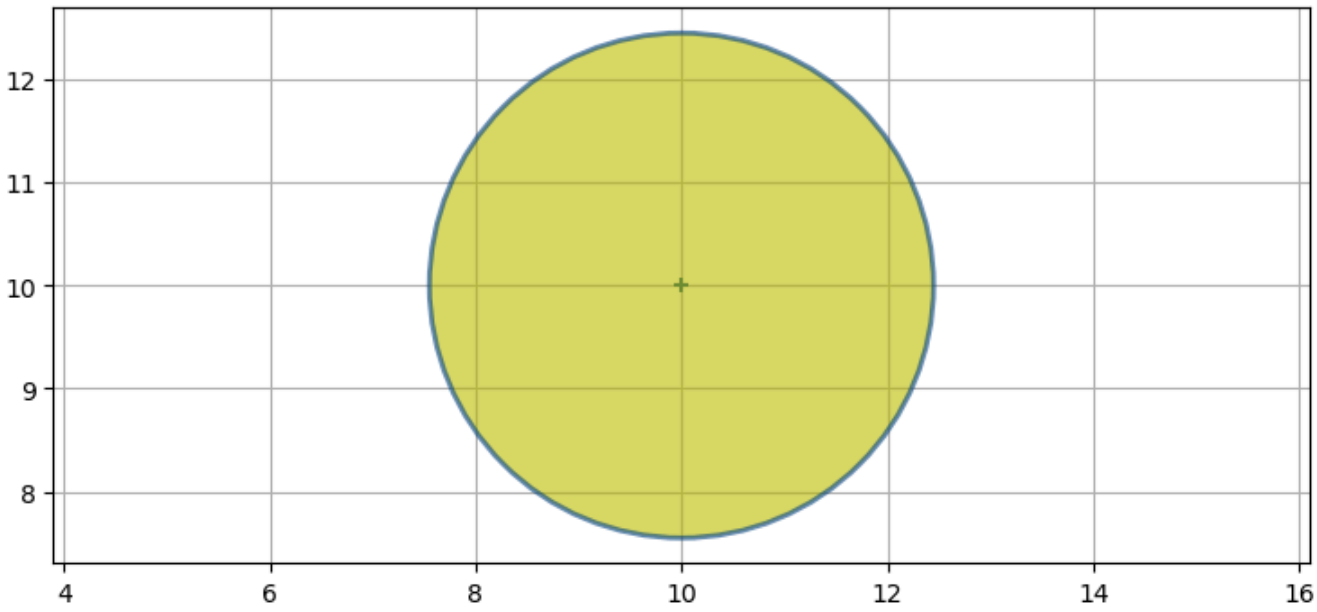
$$\mu \approx \frac{\Sigma_2\mu_1 + \Sigma_1\mu_2}{\Sigma_1 + \Sigma_2}$$

$$\Sigma \approx \frac{\Sigma_1\Sigma_2}{(\Sigma_1 + \Sigma_2)}$$

在这种形式下，单变量方程与多变量方程之间的关系就一目了然了。

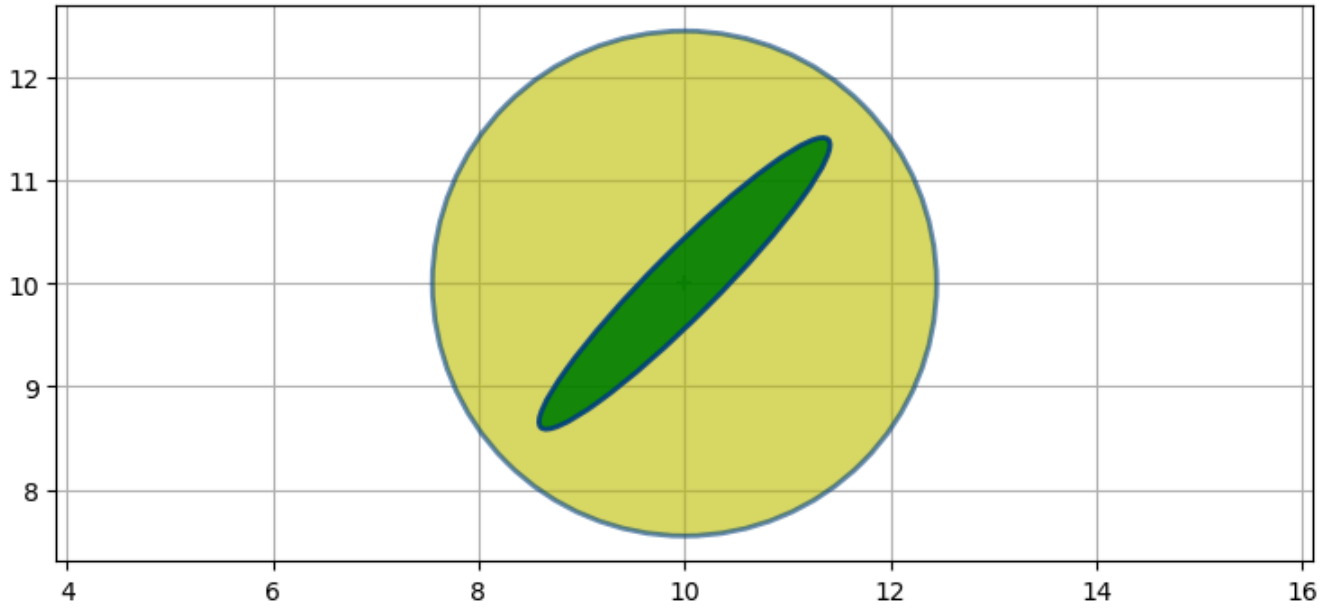
现在，让我们通过一个具体的例子来了解多元高斯分布。假设我们使用两个雷达系统来追踪一架飞机的位置。为了简化问题，我们不考虑飞机的飞行高度，因此可以使用二维坐标图来表示飞机的位置。雷达可以提供目标到探测器的距离和方位角。由于我们最初并不清楚飞机的确切位置，因此表示位置不确定性的协方差可能如下所示。在贝叶斯统计学中，这被称为“先验概率”。

```
P0 = [[6, 0], [0, 6]]
plot_covariance_ellipse((10, 10), P0, fc='y', alpha=0.6)
```



现在，假设在飞机的左下方有一个雷达。进一步假设，该雷达的方位测量是准确的，但距离测量则不准确。那么，这些测量误差的协方差可能如下所示（以绿色线条显示在黄色的基准线上方）：

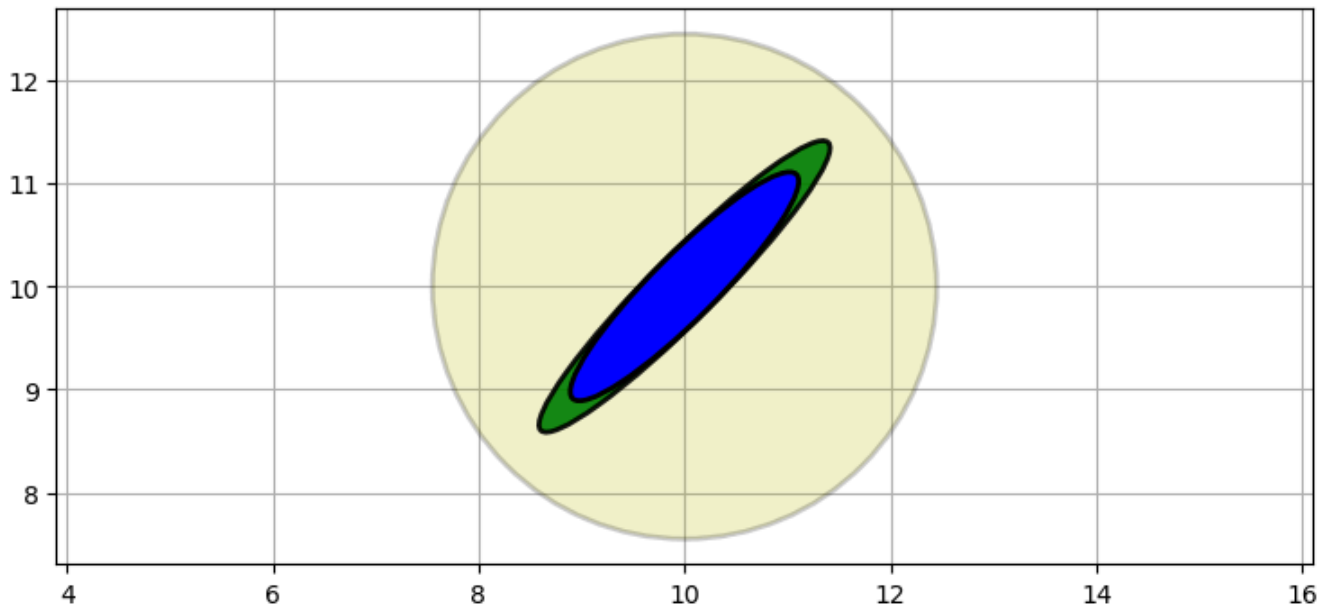
```
P1 = [[2, 1.9], [1.9, 2]]
plot_covariance_ellipse((10, 10), P0, fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P1, fc='g', alpha=0.9)
```



请记住，贝叶斯统计学将此称为“证据”。椭圆的中心指向雷达所探测到的位置。由于测距不准确，椭圆的长度相当大；实际上，飞机可能与测得的距离有相当大的偏差。不过，由于方位角的测量非常精确，因此椭圆的宽度很小，这意味着飞机很可能就在该方位角所指示的位置附近。

我们希望求出后验分布，也就是纳入这次证据之后的均值与协方差。与前面相同，将测量似然与先验密度相乘并归一化，就完成信息融合。

```
from filterpy.stats import multivariate_multiply

P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]
plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.9)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')
```

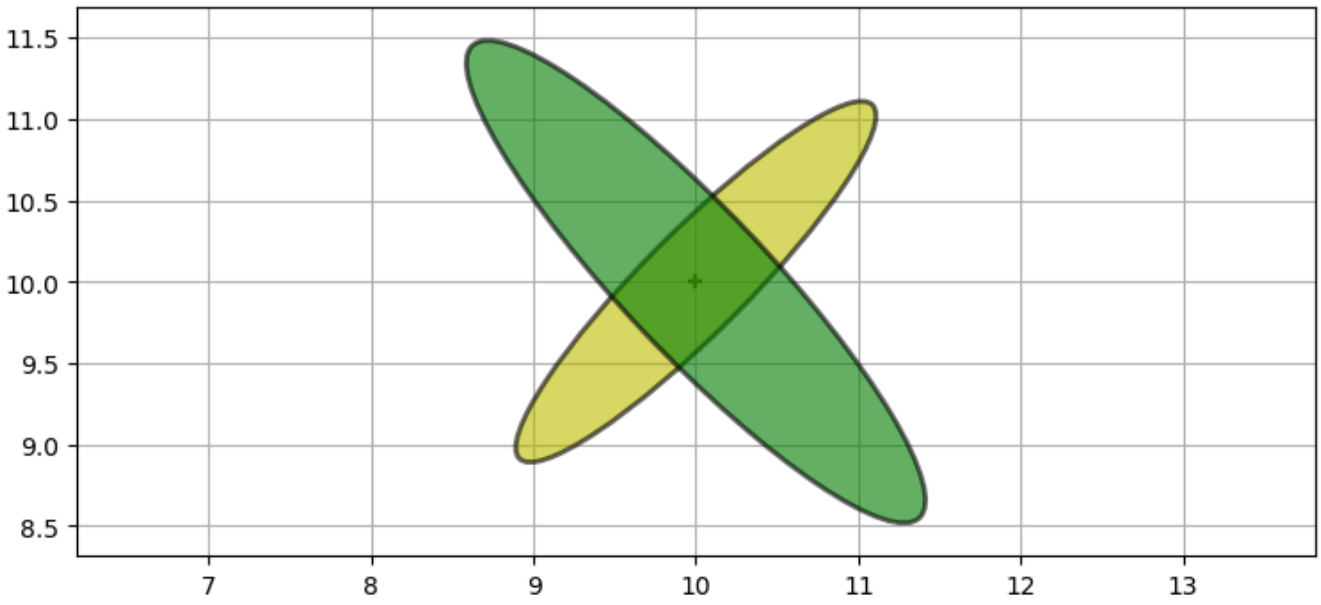


我用透明的黄色标出了最初的估算值（先验值），用绿色标出了雷达测得的数值（证据），再用蓝色标出了最终的估算值（后验值）。

后验椭圆基本保留了雷达测量分布的形状和位置，但范围更小。一元高斯中也有相同现象：融合独立信息时，高斯密度相乘并归一化，使估计方差减小，不确定性降低。还要注意，协方差椭圆的形状反映了飞机与雷达系统的几何布局，下一步会看到这一点的重要性。

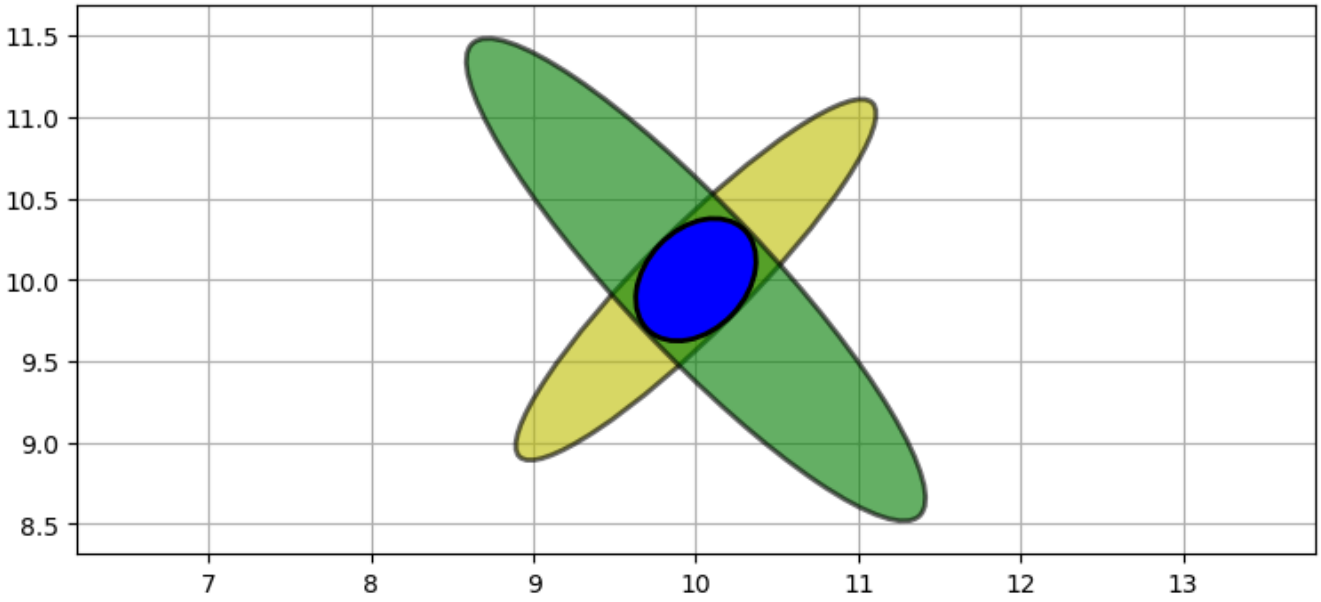
现在，假设我们从另一个雷达那里获得了测量数据，也就是右下角的那个雷达。上一步得到的后验概率值就变成了新的先验概率值，我用黄色来表示这个新的先验概率值。新的测量数据则用绿色来表示。

```
P3 = [[2, -1.9], [-1.9, 2.2]]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
```



我们通过将高斯函数相乘来整合这些信息：

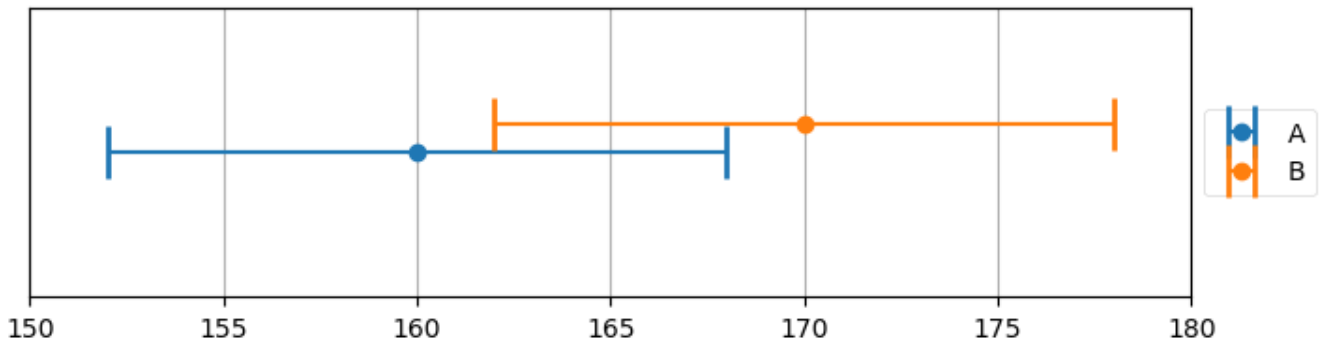
```
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```



飞机最可能出现在两个分布共同支持的区域。把先验密度与测量似然相乘并归一化，就得到新的高斯分布。它的形状体现了观测几何关系，因此自然产生类似三角定位的信息增益。我们没有另写求两条椭圆边界交点的代码；结果来自概率密度的融合，而不是边界曲线的几何求交。

回想一下关于 g-h 滤波器的那一章内容：我们当时展示了用秤称量两次所得结果的误差范围。估计值应该位于这两个误差范围所覆盖的区域内。在这里，估计值应该在 161 磅到 163 磅之间。

```
import kf_book.book_plots as book_plots
book_plots.plot_errorbars([(160, 8, 'A'), (170, 8, 'B')], xlims=(150, 180))
```

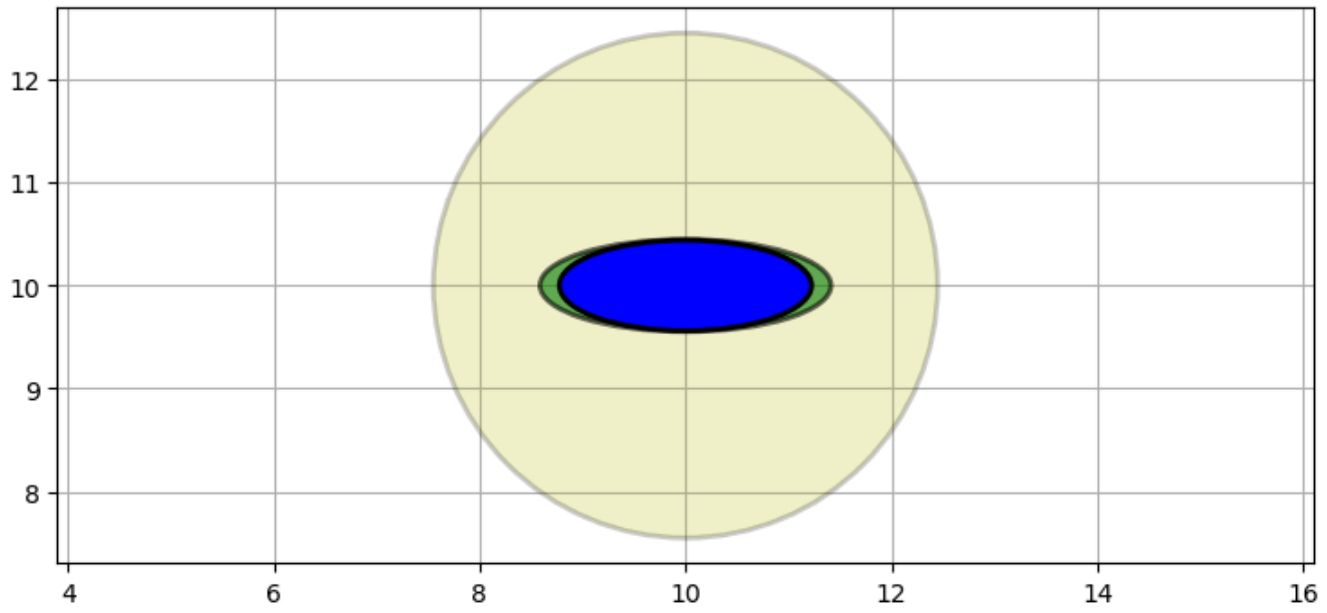


让我们考虑另一种布局。假设第一台雷达位于飞机的正左侧。我可以用某种方式来模拟这种测量误差。

$$\Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 0.2 \end{bmatrix}$$

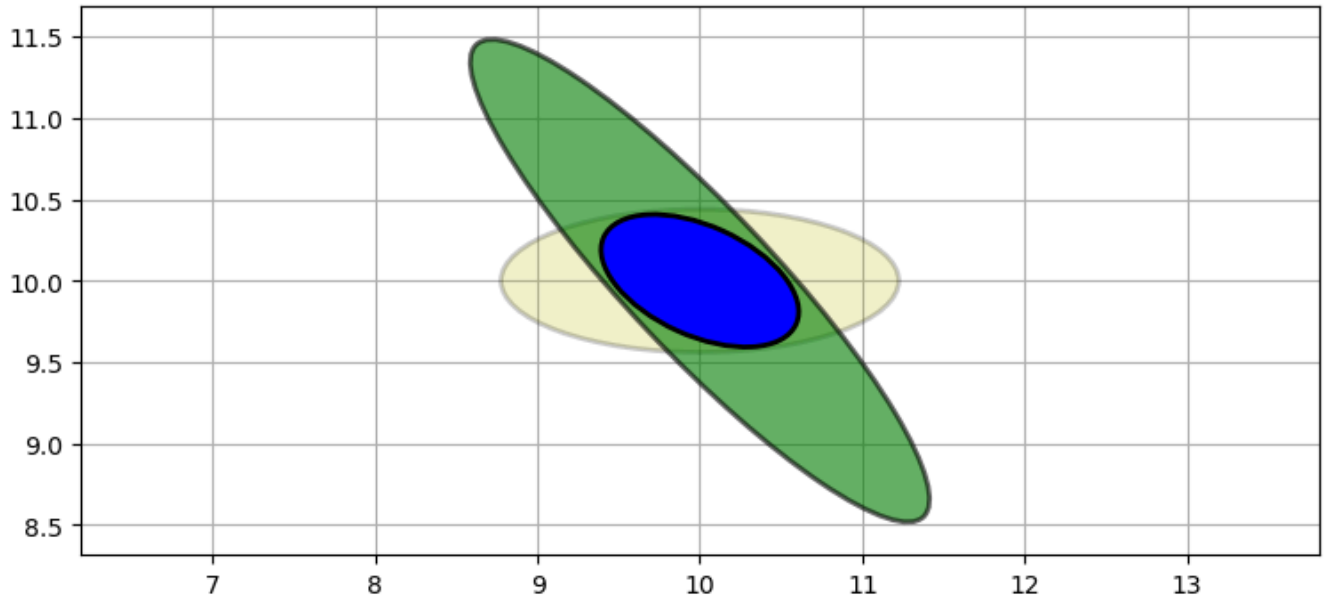
这里我们看到了将先验概率与测量结果相乘后的结果。

```
P1 = [[2, 0], [0, .2]]
P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]
plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')
```



现在，我们可以把第二个雷达系统的测量数据也纳入考虑范围了。该雷达系统仍保持与之前相同的位置。

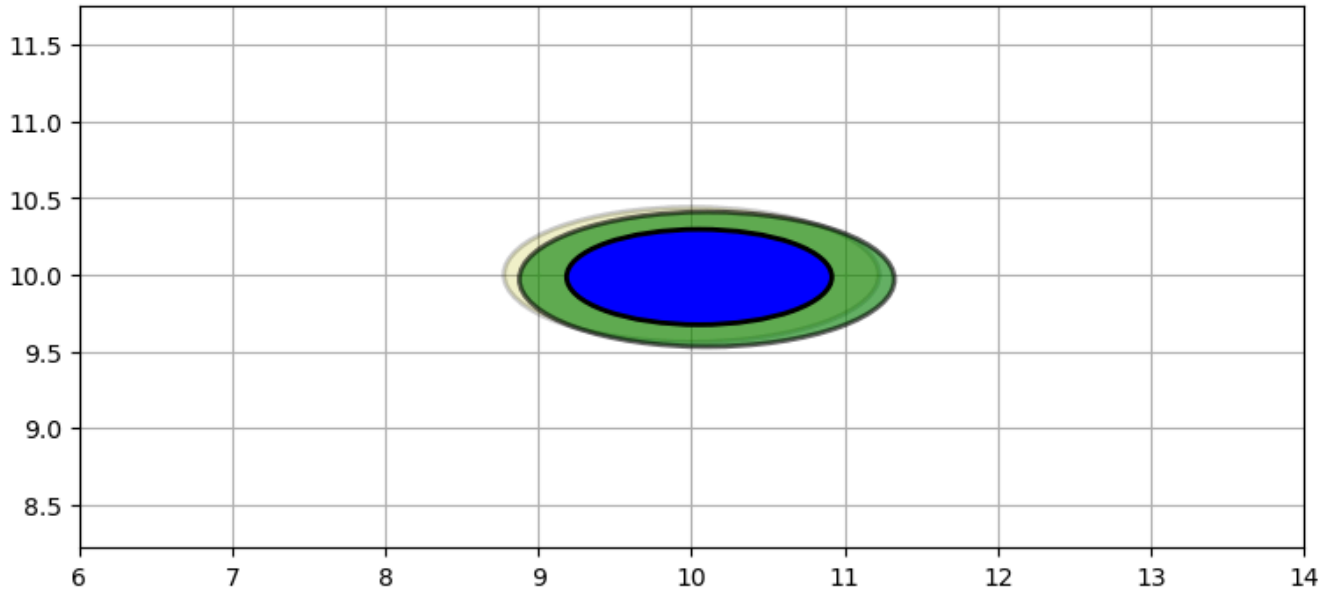
```
P3 = [[2, -1.9], [-1.9, 2.2]]
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```



我们的估算结果不如前面的例子准确。由于这两个雷达站相对于飞机的位置不再处于正交状态，因此三角测量结果的准确性也就大打折扣了。

最后，设在很短间隔内用同一雷达测量两次，两条协方差椭圆的长轴方向几乎相同，重叠很多，融合后沿不精确方向仍可能留下较大不确定性。这体现了相似观测方向的信息增益不如相互补充的观测方向。重复测量若存在相关误差，还应在融合模型中处理。

```
P5 = multivariate_multiply((10,10), P2, (10.1, 9.97), P2)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10.1, 9.97), P2, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse(P5[0], P5[1], ec='k', fc='b')
plt.xlim(6, 14);
```

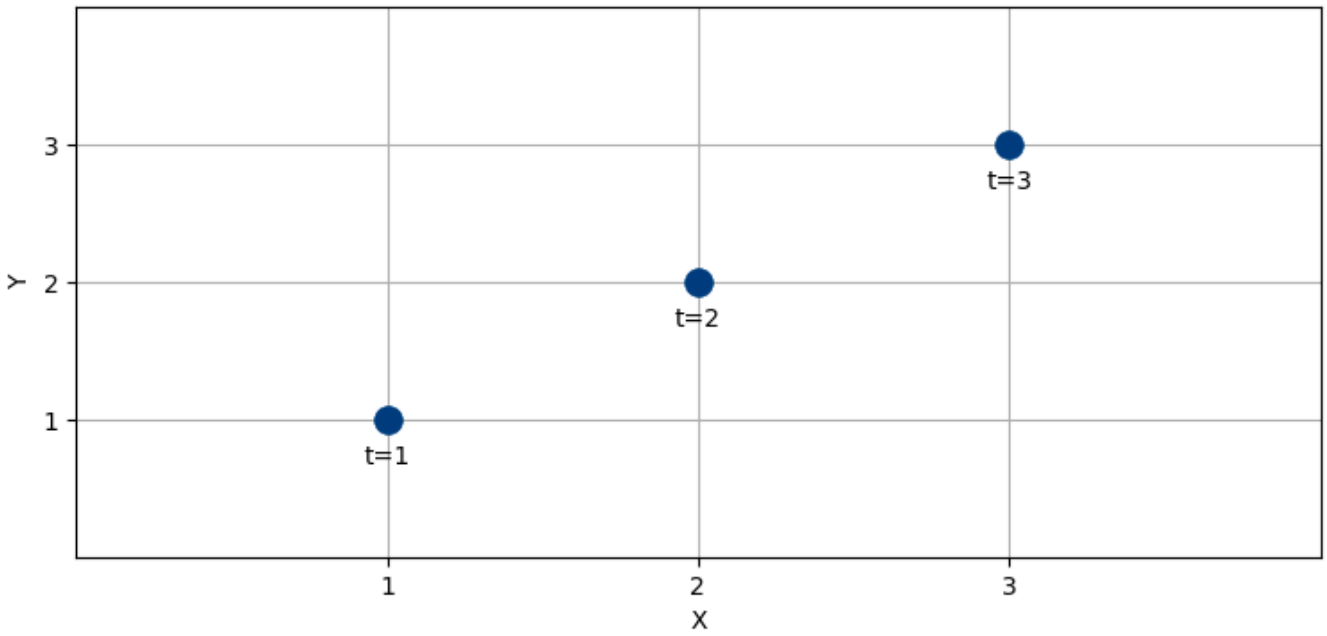


隐变量

你现在应该明白，为什么多元卡尔曼滤波器的性能会优于单变量卡尔曼滤波器了。各变量之间的相关性能够显著提升我们的估算精度。我们还可以进一步探讨这个问题。本节包含了本章的核心思想，请仔细阅读。

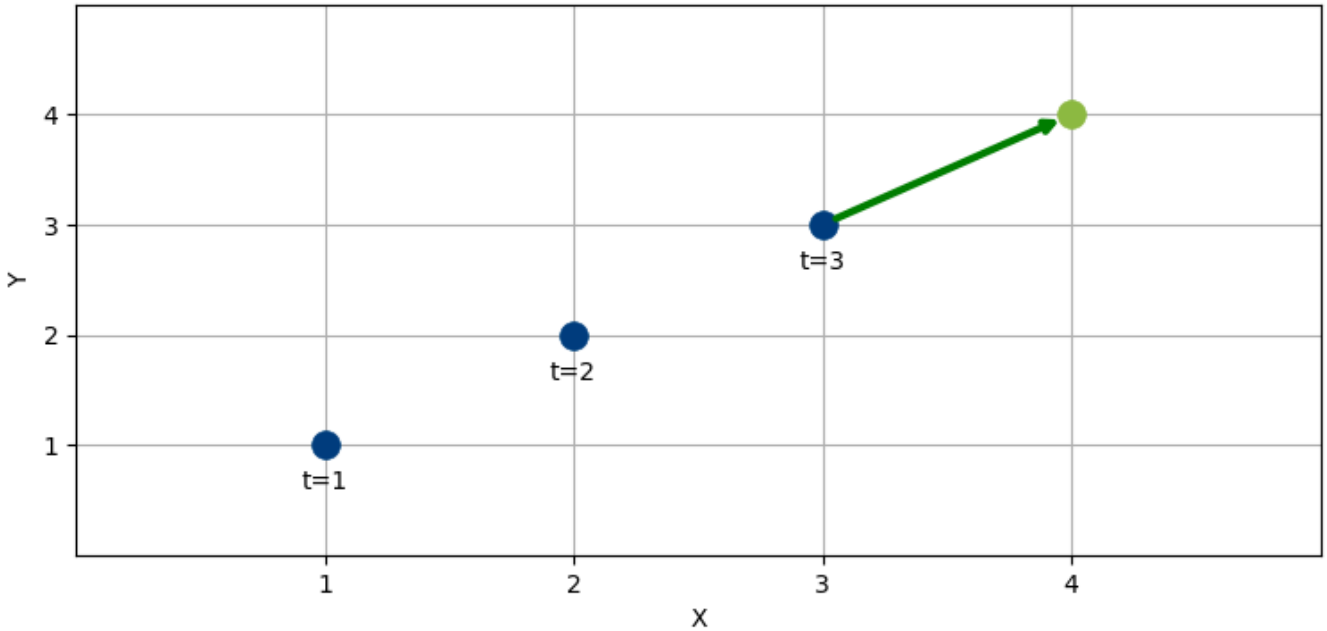
假设我们正在追踪一架飞机，我们在时间点 t 为 1 秒、2 秒和 3 秒时，分别得到了 x 和 y 坐标的数据。那么，凭直觉来看，当时间点 t 为 4 秒时， x 的数值会是多少呢?

```
mkf_internal.show_position_chart()
```



看起来，该飞机是沿直线飞行的。我们知道，飞机不可能瞬间改变飞行方向。最合理的推测是：在 t =4 时，飞机的位置应该在(4,4)处。我会用绿色箭头来标示这个位置。

```
mkf_internal.show_position_prediction_chart()
```



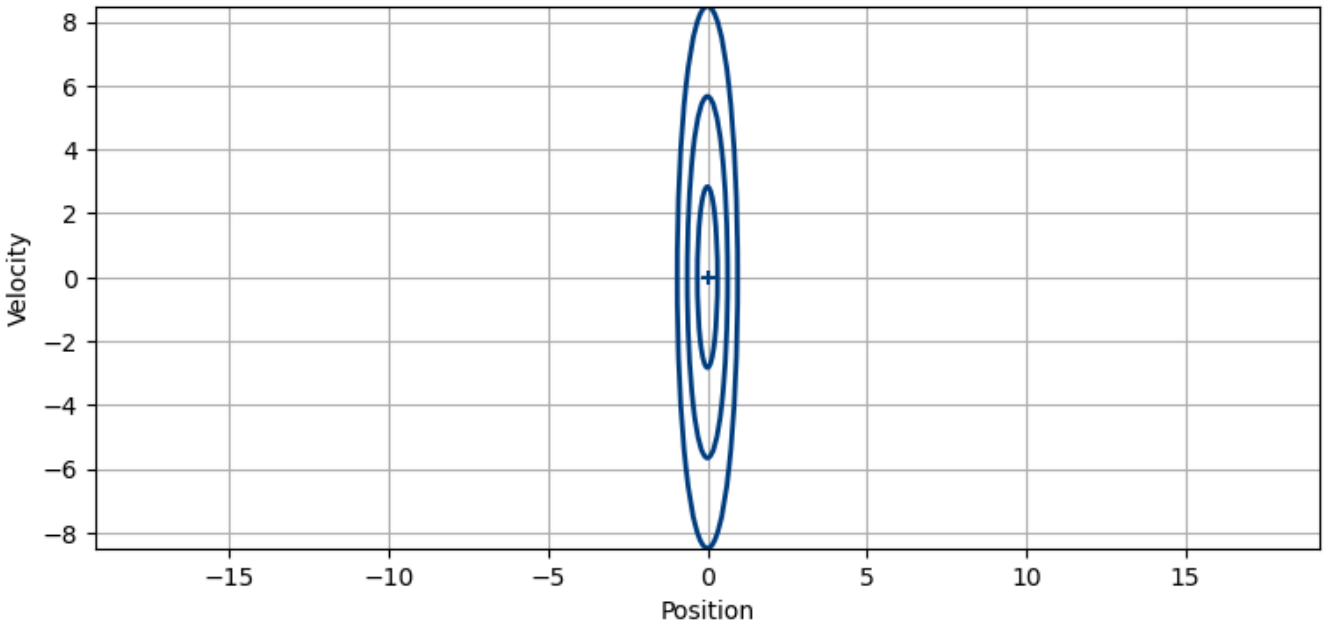
你得出这个结论，是因为你假设飞机的速度是恒定的。合理的假设应该是：在每个时间单位内，飞机在 x 轴和 y 轴上各移动一个单位长度。

回想一下在 g-h 滤波器那一章的内容吧。当时，我们试图改进那些受噪声干扰的体重测量结果。我们将体重变化纳入了计算公式中，因为这样能更准确地预测第二天的体重数值。g-h 滤波器利用 g 参数来调整对当前体重测量的重视程度，而 h 参数则用于调整对体重变化的重视程度。

我们对卡尔曼滤波器也会采用同样的处理方式。毕竟，卡尔曼滤波器本质上也是一种 g-h 滤波器。在这里，我们跟踪的是飞机，因此需要估计位置和速度，而不是体重及其变化率。体重变化率是体重对时间的导数，速度则是位置对时间的导数。很难在图中同时展示 x、y 及其各自速度构成的四维状态并直观理解，因此我们先只考虑 $\boldsymbol{x}$ 方向；同样的数学原理可以推广到更多维度。

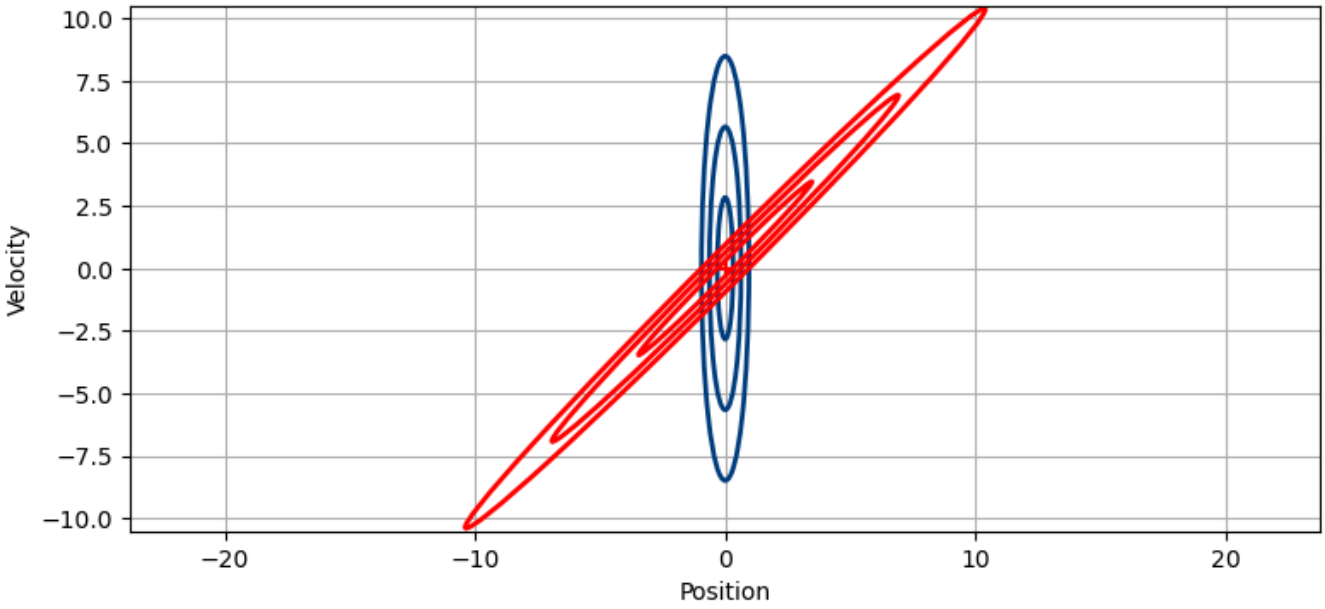
初始时刻，我们可能很确定位置 x=0，却不知道速度。可以用如下协方差椭圆表示：位置方向较窄，表示位置不确定性小；速度方向较高，表示速度不确定性大。

```
mkf_internal.show_x_error_chart(1)
```



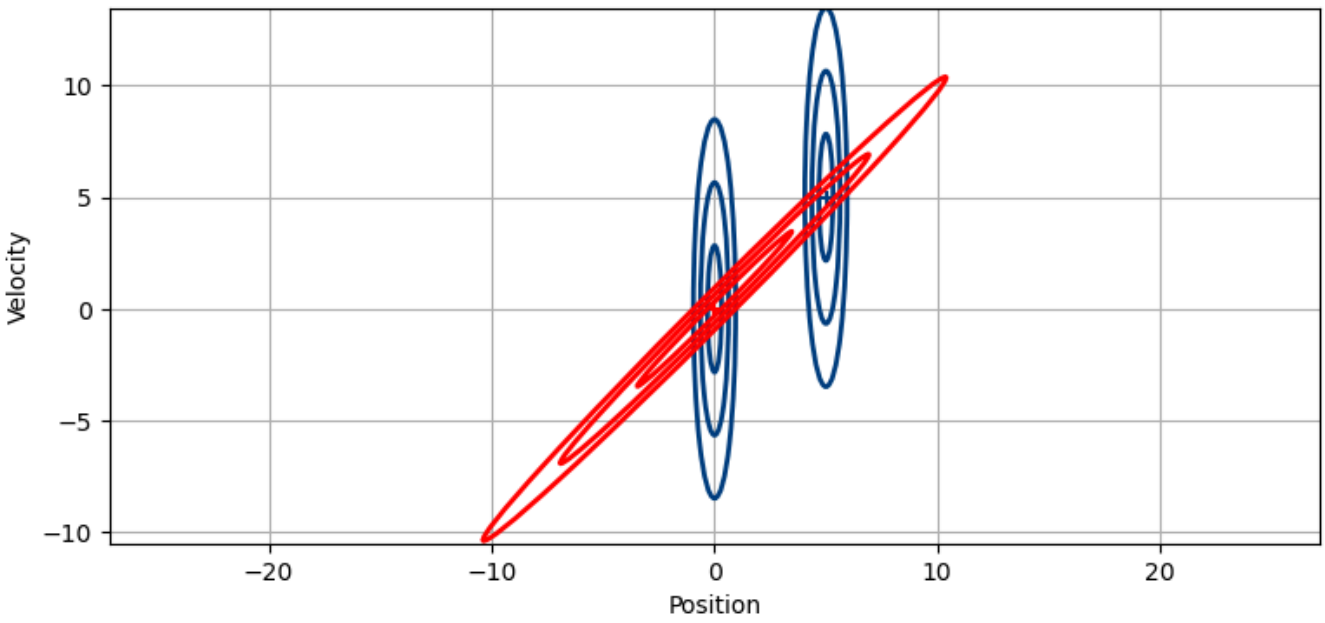
不过，位置和速度是相互关联的。如果速度为 5 米/秒，那么 1 秒钟后，位置就是 5 米。如果速度为-10 米/秒，那么 1 秒钟后，位置就是-10 米。我们可以通过在坐标系中对角线上绘制速度相关图来直观地理解这一点。

```
mkf_internal.show_x_error_chart(2)
```



此时，知道速度的方差仍不足以帮助我们预测新的位置，因为我们不知道速度是多少。不过，一秒钟后，我们得到了新的位置测量：x=5。

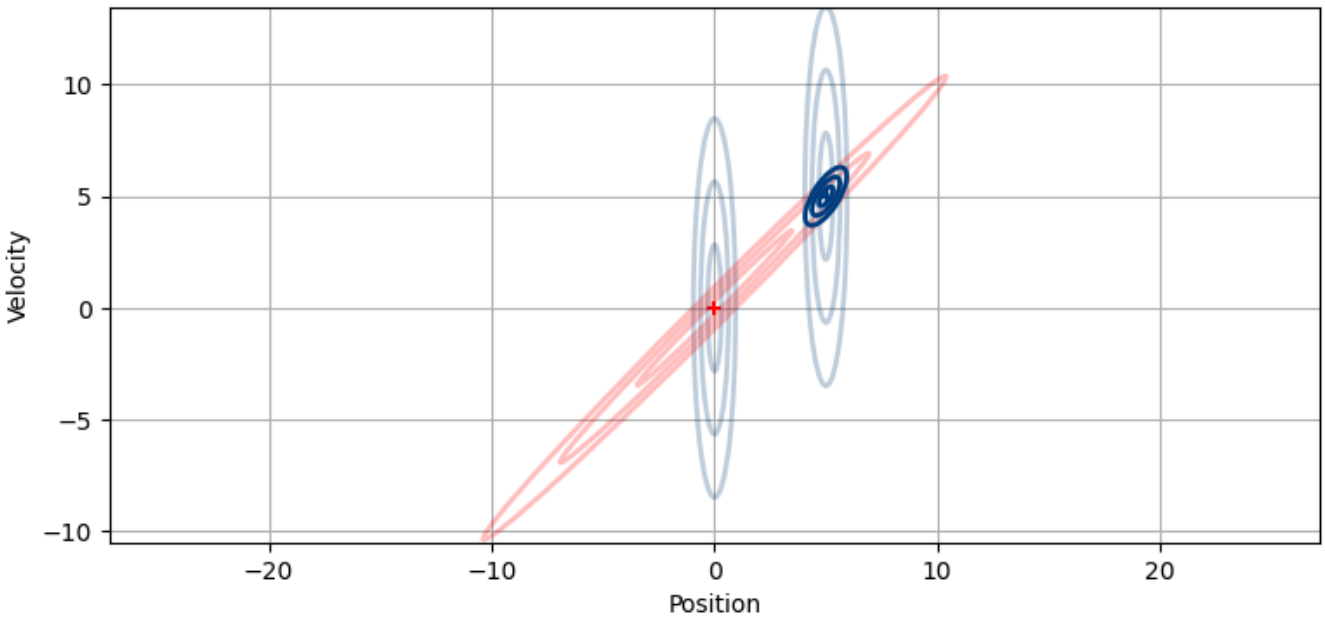
```
mkf_internal.show_x_error_chart(3)
```



具体计算下一章再讲，先理解几何含义。由于速度未知，预测位置沿水平轴的不确定性很大；速度本身也很不确定，所以纵向范围同样大。但位置与速度相关：经过一秒，5 米/秒对应约 5 米的位移，10 米/秒对应约 10 米的位移。因此椭圆沿斜线方向伸展，垂直于长轴的方向很窄。

把两种不确定性叠加到同一图中，就能看出信息融合的关键。收到位置约为 5 的新测量时，合理估计位于运动预测分布与位置测量分布共同支持的区域。运用上一节公式，把两个高斯概率密度相乘并归一化。从贝叶斯角度，这就是先验乘以测量似然，得到后验。这里相乘的是概率密度，而不是两个协方差矩阵。

```
mkf_internal.show_x_error_chart(4)
```



后验协方差椭圆位于预测分布与位置测量分布共同支持的区域。它略微倾斜，表示位置与速度的估计误差仍有一定相关性。更重要的是，它的范围比任一输入分布都小。上一章每次 update() 都将先验与新测量的似然相乘并归一化，从而减小方差；这里也是如此，但利用位置—速度之间的关联，改善尤为明显。即使位置和速度各自只知道个大概，只要知道它们的相关关系，就能得到精确得多的联合估计。从图上看，后验椭圆在 x 轴方向的跨度小于 t=1 时仅使用位置测量得到的跨度。因此，加入速度不仅改善速度估计，也能使位置估计比只使用位置测量时更确定。

这是一个关键点。雷达只直接测量飞机的位置，这里称它为观测变量。根据位置的时间序列和运动模型，可以估计速度；由于没有专门的速度传感器，这里把速度称为隐变量。利用位置与速度的相关关系，仍可以准确估计它。未直接测量并不等于在系统意义上不可观测。

还要区分未直接测量与不可观测。某些状态没有专门传感器，却可凭动态模型及位置序列间接估计，例如速度和地面运动方向；另一些量在当前模型和观测下不可辨识，例如飞机颜色或飞行员姓名，加入滤波状态也不能凭空恢复。是否可观测取决于模型和观测关系，不能笼统说所有未测量变量都不可观测。

为什么能够做到这一点？若将另一架飞机的速度分布叠加到这架飞机的位置图上，二者没有相应的动态联系，融合也就不能提供有用信息。相反，同一架飞机的速度决定其运动方向与快慢；只要速度没有变化，就能据此预测下一位置。速度估计即使只偏差一点，经过时间传播，也可能与位置测量明显不符。改变速度会以可预测的方式改变后续位置；若位置读数不符合这种动态关系，就应对它的可靠性有所保留。正是这种由运动模型建立的联系，使位置与速度可以互相提供约束。

重要的是要明白：我们利用了速度与位置之间存在关联这一事实。我们通过两次测量之间的距离和时间来大致估算速度，然后再利用贝叶斯定理，在进行少量观测后就能得到非常精确的估计值。如果你有任何疑问，请重新阅读这一部分。如果不理解这一点，那么你将很难理解后续章节的内容。

只看位置时，一次更新带来的改善可能显得不大。但下一章将看到，多次更新后确定性会显著提高：测量方差仍可能很大，位置估计方差却很小。每次把速度相关的预测信息与位置测量融合，位置方向的不确定性都可能进一步收缩。

更高维度

到目前为止，我向你们展示了二维高斯分布，但实际上，数学原理并不局限于二维空间。在后面的章节中，我们将研究九维或十二维的空间。如果你从事天气预报等领域的工作，那么你可能会面对成千上万的维度。

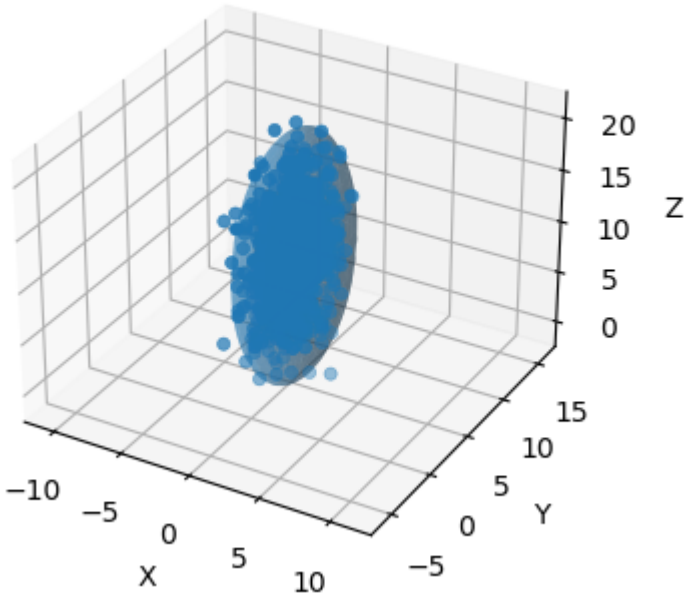
这些高维空间究竟“长什么样”呢？其实，二维高斯分布可以用误差椭圆来表示，因此，三维高斯分布自然可以用三维误差椭球来表示。这里就不详细讨论数学原理了，但事实证明确实如此。 `FilterPy` 提供了一个用于绘制这种椭球的函数。

首先，让我们生成一些具有特定协方差的随机数据，这样我们就可以把它们绘制在椭球体内部了。

```
from filterpy.stats import plot_3d_covariance

mu = [0.3, 5., 10.]
C = np.array([[1.0, .03, .2],
              [.03,  4.0, .0],
              [.2,  .0, 16.1]])

sample = np.random.multivariate_normal(mu, C, size=1000)
```

现在，我们使用 `FilterPy` 函数 `plot_3d_covariance` 来绘制椭球体，然后再将各个样本点以散点图的形式展示出来。

```
ax = plot_3d_covariance(mu, C, alpha=.4, std=3, limit_xyz=True)
ax.scatter(sample[:, 0], sample[:, 1], zs=sample[:, 2],);
```



图中的点大部分落在所画椭球内。不同维数下，相同马氏距离对应的联合覆盖概率不同。对于一元高斯分布，均值两侧三倍标准差的区间覆盖约 99.7% 的概率质量；对于三维高斯分布，马氏距离不超过 3 的椭球覆盖约 97.07%，二维对应概率约为 98.89%。

九个维度？我还没想好如何在二维屏幕上画出九维椭球体，所以这里不再绘图。不过概念相同：分布在给定标准差尺度下的误差范围，可以用九维椭球体描述。

本章小结

我们充分利用了该系统的几何特性和各种相关性，从而得到了非常精确的估算结果。在数学上，我们不必关心所处理的究竟是两个数值、一个数值与一个相关联的速度，还是其他类型的空间参数。如果房屋面积与房价之间存在相关性，那么就可以用卡尔曼滤波器来追踪房价变化。同样地，如果年龄与疾病发病率相关，也可以用卡尔曼滤波器来监测疾病趋势。如果僵尸数量与猎枪的数量呈负相关，那么同样可以用卡尔曼滤波器来追踪僵尸数量的变化。我之前用几何概念来解释这一点，还提到了三角测量法。这么做只是为了帮助大家更好地理解这个原理。其实，即使某些状态变量没有明确的几何表现形式，也可以用卡尔曼滤波器来处理，比如股票价格或牛奶产量等数据（我甚至收到过有人用卡尔曼滤波器来追踪牛奶产量的邮件！）试着把这些看作是具有相关性的高斯分布吧。如果我们能把各种不确定性表示为多维高斯分布，那么就可以将先验概率与似然值相乘，从而得到更精确的结果。

参考文献

- [1] http://docs.scipy.org.cn/doc/scipy/reference/tutorial/stats.html
- [2] `FilterPy` 库。Roger Labbe 开发。https://github.com/rlabbe/filterpy

第 6 章：多元卡尔曼滤波器

多个随机变量的滤波处理

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

现在可以研究并实现完整的多元卡尔曼滤波器了。上一章介绍了如何用多元高斯分布表示多个随机变量之间的相关性，例如飞机位置与速度之间的相关性。我们还看到，利用变量之间的相关性，可以显著改善后验估计。即使对位置和速度的了解都很粗略，只要利用二者的相关性，新的估计仍可能十分准确。

我希望通过几个完整例子，帮助你建立滤波器如何工作的直觉，因此会暂时略去一些细节。有些方法只适用于特殊情形，有些结果则暂不解释推导过程。如果一开始就给出严格的一般公式，你可能会难以理解各项的含义，也不知道怎样用于自己的问题。后续章节将补充更严谨的数学基础，修正本章采用的近似，或说明这里暂未涉及的条件。

为了实现这一目标，我们将把研究范围限制在那些可以用牛顿运动方程来描述的问题的子集上。这类滤波器被称为离散化连续时间运动学滤波器。在卡尔曼滤波器的数学原理部分，我们将探讨如何处理非牛顿系统的相关数学问题。

牛顿运动方程

牛顿运动方程告诉我们：系统以恒定速度 $\boldsymbol{v}$ 运动时，经过时间 $\boldsymbol{t}$ 后的位置 $\boldsymbol{x}$ 可由下式计算：

$$\boldsymbol{x} = \boldsymbol{v}\boldsymbol{t} + \boldsymbol{x}_0$$

例如，如果我们从位置 13 开始移动，速度为 10 米/秒，经过 12 秒后，我们的最终位置将是 133（$\boldsymbol{10 \times 12 + 13}$）。

我们可以用这个公式来表示匀加速运动。

$$\boldsymbol{x} = \frac{1}{2}\boldsymbol{a}\boldsymbol{t}^2 + \boldsymbol{v}_0\boldsymbol{t} + \boldsymbol{x}_0$$

如果进一步假设加加速度（加速度对时间的导数）恒定，则有：

$$\boldsymbol{x} = \frac{1}{6}\boldsymbol{j}\boldsymbol{t}^3 + \frac{1}{2}\boldsymbol{a}_0\boldsymbol{t}^2 + \boldsymbol{v}_0\boldsymbol{t} + \boldsymbol{x}_0$$

这些方程是通过积分微分方程得到的。只要给定一个恒定的速度 v，我们就可以利用该方程来计算物体随时间所行驶的距离。

$$x = vt + x_0$$

我们可以通过以下方式推导出来：

$$\begin{aligned}
v &= \frac{dx}{dt} \\
dx &= v\,dt \\
\int_{x_0}^{x} dx &= \int_0^t v\,dt \\
x - x_0 &= vt - 0 \\
x &= vt + x_0
\end{aligned}$$

设计卡尔曼滤波器时，首先建立描述系统动力学的微分方程组。大多数方程并不容易直接积分。这里从牛顿运动方程出发，是因为它们可以积分得到闭式解，使滤波器设计更容易。另一个好处是，它们恰好适用于运动目标跟踪，而这正是卡尔曼滤波的主要用途之一。

卡尔曼滤波算法

该算法与我们在前几章中使用的贝叶斯滤波算法相同。更新步骤稍微复杂一些，不过我们会在具体讲解时详细说明原因。

初始化

1. 初始化滤波器的状态。
2. 初始化对该状态的信念（不确定性）。

预测

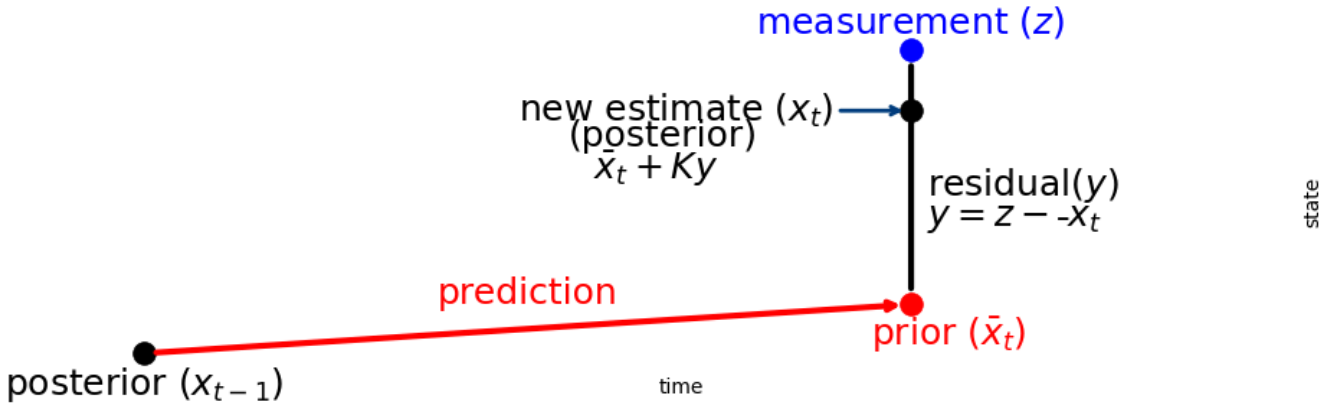
1. 根据系统行为预测下一时间步的状态。
2. 调整状态信念，计入预测的不确定性。

更新

1. 获取测量值及其精度信息。
2. 计算测量与预测状态之间的残差。
3. 根据测量与预测的相对精度计算增益。
4. 按增益加权，在预测与测量之间形成状态估计。
5. 依据测量的确定程度更新状态不确定性。

作为提醒，以下是该算法的图形化展示：

```
import kf_book.book_plots as book_plots
book_plots.show_residual_chart()
```



单变量卡尔曼滤波器用单变量高斯分布来描述系统的状态。显然，多元卡尔曼滤波器则使用多元高斯分布来描述系统的状态。在上一章中我们了解到，多元高斯分布用向量来表示均值，用矩阵来表示协方差。这意味着，卡尔曼滤波器在进行状态估计时需要运用线性代数的知识。

我不希望你们死记硬背这些方程式，不过我把单变量和多变量方程式都列在下面了。它们其实很相似。

预测

Univariate	Univariate (Kalman form)	Multivariate
$\bar{\mu} = \mu + \mu_{f_x}$ $\bar{\sigma}^2 = \sigma_x^2 + \sigma_{f_x}^2$	$\bar{x} = x + dx$ $\bar{P} = P + Q$	$\bar{\mathbf{x}} = \mathbf{Fx} + \mathbf{Bu}$ $\bar{\mathbf{P}} = \mathbf{FPF}^\mathsf{T} + \mathbf{Q}$

无需过于纠结于线性代数的具体细节，我们可以看出：

$\mathbf{x}$, $\mathbf{P}$ 分别代表该状态的均值和协方差。它们对应于 x 和 σ^2 。

$\mathbf{F}$ 是状态转移函数。将它与 $\mathbf{x}$ 相乘，即可得到先验状态均值。

$\mathbf{Q}$ 是过程噪声协方差，对应于单变量情形中的 $\sigma^2_{f_x}$。

$\mathbf{B}$ 和 $\mathbf{u}$ 对我们来说都是新的概念。它们让我们能够对系统的控制输入进行建模。

更新

Univariate	Univariate (Kalman form)	Multivariate
	$y = z - \bar{x}$	$\mathbf{y} = \mathbf{z} - \mathbf{H}\bar{\mathbf{x}}$
	$K = \frac{\bar{P}}{\bar{P}+R}$	$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}$
$\mu = \frac{\bar{\sigma}^2 \mu_z + \sigma_z^2 \bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$	$x = \bar{x} + Ky$	$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{Ky}$
$\sigma^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2}$	$P = (1 - K)\bar{P}$	$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$

$\mathbf{H}$ 是一种测量函数。我们在本书中还没有见过这种函数，稍后我会详细解释。如果你在心中把 $\mathbf{H}$ 从这些方程式中去掉，就会发现这些方程式其实是非常相似的。

$\mathbf{z}$, $\mathbf{R}$ 分别表示测量向量和测量噪声协方差，对应一元滤波器中的 z 和 σ_z^2。（为使两种情形的符号尽可能一致，这里将一元方程中的 μ 改写为 x。）

$\mathbf{y}$ 和 $\mathbf{K}$ 分别代表残差和卡尔曼增益。

与单变量滤波器相比，其具体实现方式会有所不同，因为这里处理的是向量和矩阵。不过，其核心概念是完全相同的。

- 用高斯分布来表示我们对状态和误差的估计。
- 用高斯分布来表示测量值及其误差。
- 使用高斯分布来表示该过程模型。
- 使用过程模型预测下一时刻状态，得到先验。
- 在先验预测与测量之间，按各自不确定性形成融合估计。

作为设计者，你需要确定状态 $(\mathbf{x}, \mathbf{P})$、过程模型 $(\mathbf{F}, \mathbf{Q})$、测量 $(\mathbf{z}, \mathbf{R})$ 以及测量函数 $\mathbf{H}$。如果系统具有控制输入，例如机器人系统，还需要确定 $\mathbf{B}$ 和 $\mathbf{u}$。

我已经将卡尔曼滤波器的计算公式编写进了 FilterPy 中的 `predict` 和 `update` 函数中。你可以通过以下方式来导入这些函数：

```
from filterpy.kalman import predict, update
```

追踪一只狗

让我们回到熟悉而可靠的狗的位置跟踪问题。这次要运用上一章的关键认识，利用隐变量改善估计。我可以先介绍数学推导，不过这里选择先实现一个滤波器，边做边学。表面上，这里的数学形式不同，可能也比前几章复杂，但基本思想完全相同：仍然是在对高斯分布进行乘法和加法运算。

我们首先为这只狗编写模拟程序。该模拟会进行 `count` 步操作，每步狗会向前移动大约 1 米。在每一步中，狗的速度会根据过程方差 `process_var` 而发生变化。在更新了狗的位置之后，我们会根据假设的传感器方差 `z_var` 来计算相应的测量值。该函数会返回一个包含狗位置的 NumPy 数组，以及另一个包含测量值的数组。

```
import math
import numpy as np
from numpy.random import randn

def compute_dog_data(z_var, process_var, count=1, dt=1.):
    "returns track, measurements 1D ndarrays"
    x, vel = 0., 1.
    z_std = math.sqrt(z_var)
    p_std = math.sqrt(process_var)
    xs, zs = [], []
    for _ in range(count):
        v = vel + (randn() * p_std)
        x += v*dt
        xs.append(x)
        zs.append(x + randn() * z_std)
    return np.array(xs), np.array(zs)
```

预测步骤

在进行预测时，我们需要确定状态和协方差、过程模型以及过程噪声，必要时还要确定控制输入。我们将依次来处理这些问题。

设计状态变量

此前，我们用高斯分布跟踪狗在一维空间中的位置。均值 (μ) 表示最可能的位置，方差 σ^2 则刻画位置分布的离散程度。位置是系统的状态，我们将 μ 称为状态变量。

在这个问题中，我们需要同时追踪狗的位置和速度。为此，我们需要使用多元高斯分布来描述这些数据，该分布由状态向量 $\mathbf{x}$ 及其对应的协方差矩阵 $\mathbf{P}$ 来表示。

状态变量可以是观测变量，即传感器直接测量的量；也可以是隐变量，即需要根据观测变量推断的量。在狗的位置跟踪问题中，传感器只测量位置，因此位置是观测变量，速度是隐变量。稍后将介绍如何跟踪隐变量。

需要明白的是，对位置和速度的追踪其实是一种设计上的选择，而这种选择背后涉及各种假设和影响，我们目前还来不及深入探讨。例如，我们也可以追踪加速度，甚至是加速度的变化率。先回想一下，上一章中我们提到，如果在协方差矩阵中加入速度这一项，那么位置的方差会大大减小。在本章的后面部分，我们将了解卡尔曼滤波器是如何计算隐变量值的。

在单变量分析中，我们用一个标量值来表示狗的位置（例如 $\mu = 3.27$）。在上一章中，我们学习了如何使用多元高斯分布来处理多个变量。例如，如果我们想指定狗的位置为 10.0 米，速度为 4.5 米/秒，那么我们可以这样表示：

$$\mu = \begin{bmatrix}10.0\\4.5\end{bmatrix}$$

卡尔曼滤波用线性代数实现。我们用一个 $n \times 1$ 矩阵，也就是列向量，存储 n 个状态变量。在狗的跟踪问题中，用 x 表示位置，用 x 的一阶导数 $\dot{x}$ 表示速度。这里采用牛顿点记法：$\dot{x}$ 表示 x 对时间 t 的一阶导数，即 $\dot{x} = \frac{dx}{dt}$。卡尔曼滤波方程用 $\mathbf{x}$ 表示状态，因此把 $\mathbf{x}$ 定义为：

$$\mathbf{x} = \begin{bmatrix}x\\\dot{x}\end{bmatrix}$$

这里用 $\mathbf{x}$ 代替 μ 表示均值；应记住，它仍是多元高斯分布的均值向量。

另一种表示方法是 $\mathbf{x} = \begin{bmatrix}x & \dot{x}\end{bmatrix}^\mathsf{T}$，因为行向量的转置就是列向量。这种表示法在文本中更易于使用，因为它占用的垂直空间更少。

状态向量 $\mathbf{x}$ 与位置 x 恰好使用同一个字母。如果沿 y 轴跟踪狗，应写作 $\mathbf{x} = \begin{bmatrix}y & \dot{y}\end{bmatrix}^\mathsf{T}$，而不是 $\mathbf{y} = \begin{bmatrix}y & \dot{y}\end{bmatrix}^\mathsf{T}$。在卡尔曼滤波文献中，$\mathbf{x}$ 是状态变量的通用记号，我们不会为使名称更直观而随意改变它。统一记号有助于与同行交流。

让我们来编写代码吧。 x 的初始化过程非常简单。

```
x = np.array([[10.0],
              [4.5]])
x
```

```
array([[10. ],
       [ 4.5]])
```

在我的代码中，我经常使用转置操作将行矩阵转换为列向量。我觉得这样处理起来既方便输入，也便于阅读。

```
x = np.array([[10., 4.5]]).T
x
```

```
array([[10. ],
       [ 4.5]])
```

不过，NumPy 将一维数组视为向量处理，因此我可以简化这行代码，直接使用一维数组即可。

```
x = np.array([10.0, 4.5])
x
```

```
array([10. ,  4.5])
```

NumPy 数值数组中的元素具有统一的数据类型，通常为整数或浮点数。在这里的示例中，列表全部由整数构成时，数组采用整数类型；混合整数与浮点数时，数组会采用浮点类型。我常利用这一点，只把其中一个数写成浮点数：

```
np.array([1., 0, 0, 0, 0, 0])
```

```
array([1., 0., 0., 0., 0., 0.])
```

以下是一些例子。

```
A = np.array([[1, 2], [3, 4]])
x = np.array([[10.0], [4.5]])

# matrix multiply
print(np.dot(A, x))
print()
```

```
[[19.]
 [48.]]
```

Python 3.5 及更高版本提供矩阵乘法运算符 @；对于这里的数组运算，np.dot(A, B) == A @ B。不过，它可能没有你想象的那么方便，因为 A 和 B 都必须是数组。本书的数学运算允许某些变量取标量值，因此经常无法直接使用 @。

```
# alternative matrix multiply)
print(A @ x)
print()

x = np.array([[10.0, 4.5]]).T
print(A @ x)
print()

x = np.array([10.0, 4.5])
print(A @ x)
```

```
[[19.]
 [48.]]

[[19.]
 [48.]]

[19. 48.]
```

最后一种写法返回一维数组，但我编写的卡尔曼滤波器类也能处理这种形式。现在回头看，这种兼容设计可能会引起混淆，不过它确实能够工作。

设计状态协方差

状态高斯分布的另一半则对应于协方差矩阵 $\mathbf{P}$ 。在单变量卡尔曼滤波器中，我们需要为 σ^2 指定一个初始值，之后随着新测量数据的加入，滤波器会自动更新该数值。多元卡尔曼滤波器也是如此：我们为 $\mathbf{P}$ 指定初始值，然后滤波器会在每个时间步长中不断更新该数值。

我们需要将方差设为合理的值。例如，如果对初始位置很不确定，可以取 $\sigma^2_{\mathtt{pos}} = 500m^2$。狗的最高速度约为 21 m/s，因此在没有其他速度信息时，可以设 $3\sigma_{\mathtt{vel}} = 21$，即 $\sigma^2_{\mathtt{vel}} = 7^2 = 49$。

上一章说明了位置与速度存在相关性。但是，对这只狗来说，它们的相关性究竟有多强？我并不知道。后面会看到，滤波器会在计算中更新它们的协方差，所以这里把位置与速度之间的初始互协方差设为零；对角线上的位置方差和速度方差仍按前面的设定保留。当然，如果你知道初始互协方差，就应该使用已知值。

请记住，协方差矩阵的对角线元素分别代表各个变量的方差，而非对角线元素则代表各变量之间的协方差。因此，我们有：

$$\mathbf{P} = \begin{bmatrix} 500 & 0 \\ 0 & 49 \end{bmatrix}$$

我们可以使用 `numpy.diag` 来创建对角矩阵。对角矩阵的特点是：非对角线上的元素都为零。这一点在线性代数中有所介绍。

```
P = np.diag([500., 49.])
P
```

```
array([[500.,   0.],
       [  0.,  49.]])
```

我本可以这样写：

```
P = np.array([[500., 0.],
              [0., 49.]])
P
```

```
array([[500.,   0.],
       [  0.,  49.]])
```

我们完成了。我们已经将滤波器的状态表示为多元高斯分布，并在代码中实现了这一模型。

设计过程模型

下一步是设计过程模型。这是一种用于描述系统行为的数学模型。该模型被滤波器用来预测在经过一个离散时间步长之后的系统状态。我们通过一组描述系统动态的方程来实现这一目标。

在单变量卡尔曼滤波一章中，我们用下式描述狗的运动：

$$x = v\Delta t + x_0$$

我们的实现方式如下：

```
def predict(pos, movement):
return gaussian(pos.mean + movement.mean,
pos.var + movement.var)
```

在本章中，我们将采用同样的方法，只不过是用多元高斯分布来代替单变量高斯分布。你可以这样想象这种实现方式：

$$\mathbf{x} = \begin{bmatrix}5.4\\4.2\end{bmatrix},\ \dot{\mathbf{x}} = \begin{bmatrix}1.1\\0.\end{bmatrix}$$
$$\mathbf{x} = \dot{\mathbf{x}}t + \mathbf{x}$$

不过，我们需要将这种做法推广。卡尔曼滤波方程适用于一般线性系统，并不限于牛顿运动系统。例如，要滤波的对象可能是化工厂的管道系统，其中某根管道的流量由不同阀门设置的线性组合决定。

$$\mathbf{pipe}_1 = 0.134(\mathbf{valve}_1) + 0.41(\mathbf{valve}_2 - \mathbf{valve}_3) + 1.34$$

$$\mathbf{pipe}_2 = 0.210(\mathbf{valve}_2) - 0.62(\mathbf{valve}_1 - \mathbf{valve}_5) + 1.86$$

线性代数提供了一种处理方程组的有效方法。以这个方程组为例吧。

$$\begin{cases}2x + 3y = 8\\4x - y = 2\end{cases}$$

我们可以将其表示为矩阵形式，即：

$$\begin{bmatrix}2 & 3\\4 & -1\end{bmatrix}\begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}8\\2\end{bmatrix}$$

如果按照这个方程式进行矩阵乘法运算，得到的结果就是上面那两个方程式。在线性代数中，我们会用 $\mathbf{Ax} = \mathbf{B}$ 来表示这个运算。

$$\mathbf{A} = \begin{bmatrix}2 & 3 \\ 4 & -1\end{bmatrix}, \mathbf{x} = \begin{bmatrix}x \\ y\end{bmatrix}, \mathbf{B} = \begin{bmatrix}8 \\ 2\end{bmatrix}$$

然后，我们可以利用 SciPy 的 `linalg` 包来求解 $\mathbf{x}$ 的值：

```
from scipy.linalg import solve
A = np.array([[2, 3],[4, -1]])
b = np.array([[8], [2]])
x = solve(A, b)
x
```

```
array([[1.],
       [2.]])
```

使用过程模型进行状态预测，因为这些方程告诉我们给定当前状态时，下一时刻状态会怎样。卡尔曼滤波用以下线性方程实现，其中 $\bar{\mathbf{x}}$ 为先验或预测状态。状态预测完成后，将实际测量减去预测测量，得到新息（innovation），也称预测残差或更新前的测量残差。若使用更新后的状态计算测量残差，则称为更新后残差。新息与表示 information 的“信息”是不同的概念。

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$$

我们可以将其明确表示为：

$$\begin{bmatrix}\bar{x} \\ \bar{\dot{x}}\end{bmatrix} = \begin{bmatrix}? & ? \\ ? & ?\end{bmatrix} \begin{bmatrix}x \\ \dot{x}\end{bmatrix}$$

作为卡尔曼滤波器设计者，我们需要确定 $\mathbf{F}$，使 $\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$ 能够完成系统的状态预测。为此，每个状态变量都需要一个方程。本例的状态为 $\mathbf{x} = \begin{bmatrix}x & \dot{x}\end{bmatrix}^\mathsf{T}$，因此需要一个方程计算位置 x，另一个方程计算速度 $\dot{x}$。位置预测方程已经知道：

$$\bar{x} = x + \dot{x}\Delta t$$

速度方程是什么？我们没有预测狗的速度如何随时间变化的模型，因此假设相邻预测时刻之间速度保持恒定。当然，实际并非完全如此；但只要每个时间步内速度变化不太大，滤波器仍能表现良好。因此：

$$\bar{\dot{x}} = \dot{x}$$

这就构成了我们系统的过程模型。

$$\begin{cases}\bar{x} = x + \dot{x}\Delta t \\ \bar{\dot{x}} = \dot{x}\end{cases}$$

这组方程为每个状态变量各提供了一个方程，并将待预测的变量单独放在等号左边。我们需要把它们写成 $\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$ 的形式。重新排列各项，就更容易看出如何构造矩阵。

$$\begin{cases}\bar{x} = 1x + & \Delta t\,\dot{x} \\ \bar{\dot{x}} = 0x + & 1\,\dot{x}\end{cases}$$

我们可以将其以矩阵形式重新表示为：

$$\begin{bmatrix}\bar{x} \\ \bar{\dot{x}}\end{bmatrix} = \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}\begin{bmatrix}x \\ \dot{x}\end{bmatrix}$$
$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$$

$\mathbf{F}$ 称为状态转移函数或状态转移矩阵。在后面的章节中，状态转移将由一般函数而非矩阵表示，因此“函数”这一称呼更具一般性。

```
dt = 0.1
F = np.array([[1, dt],
              [0, 1]])
F
```

```
array([[1. , 0.1],
       [0. , 1. ]])
```

让我们来测试一下吧！FilterPy 有一个名为 `predict` 的方法，该方法通过计算 $\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$ 来进行预测。我们先调用这个方法，看看会发生什么。我们已经将位置设置为 10.0 米，速度设置为 4.5 米/秒。同时，我们还定义了 `dt = 0.1`，这意味着时间步长为 0.1 秒。因此，我们预计在经过一次计算后，新的位置应该是 10.45 米。而速度应该保持不变。

```
from filterpy.kalman import predict

x = np.array([10.0, 4.5])
P = np.diag([500, 49])
F = np.array([[1, dt], [0, 1]])

# Q is the process noise
x, P = predict(x=x, P=P, F=F, Q=0)
print('x =', x)
```

```
x = [10.45  4.5 ]
```

这个方法有效。如果我们连续多次调用 `predict()`，其值会每次都被更新。

```
for _ in range(4):
    x, P = predict(x=x, P=P, F=F, Q=0)
    print('x =', x)
```

```
x = [10.9  4.5]
x = [11.35  4.5 ]
x = [11.8  4.5]
x = [12.25  4.5 ]
```

predict() 计算预测状态的均值与协方差。下面是连续进行五次预测后 $\mathbf{P}$ 的值；在卡尔曼滤波方程中，预测协方差记为 $\bar{\mathbf{P}}$。

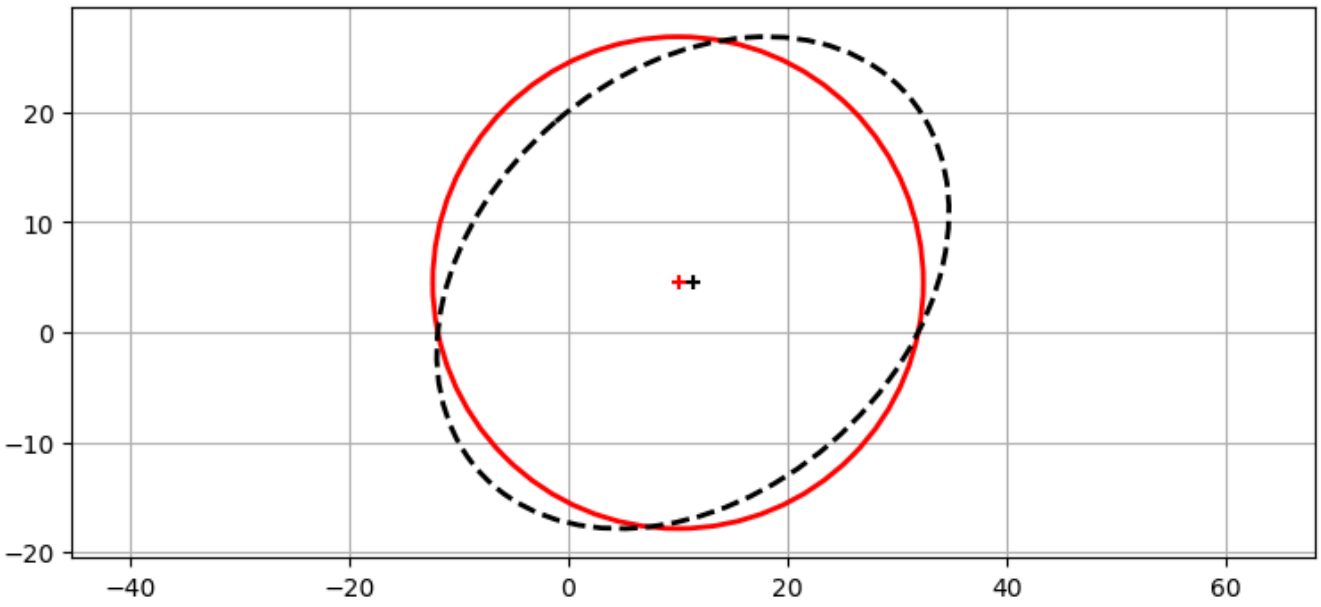
```
print(P)
```

```
[[512.25  24.5 ]
 [ 24.5   49.  ]]
```

通过观察对角线上的数值，我们可以发现位置方差有所增大。我们进行了五次预测，但并未进行任何测量，因此不确定性也增加了。非对角线上的元素不再为零——卡尔曼滤波器检测到了位置和速度之间的相关性！不过，速度的方差则没有发生变化。

在这里，我绘制了预测前后协方差的变化情况。初始值用实线红色表示，而预测结果则用虚线黑色表示。为了更清楚地展示这种变化，我还调整了协方差和时间步长。

```
from filterpy.stats import plot_covariance_ellipse

dt = 0.3
F = np.array([[1, dt], [0, 1]])
x = np.array([10.0, 4.5])
P = np.diag([500, 500])
plot_covariance_ellipse(x, P, edgecolor='r')
x, P = predict(x, P, F, Q=0)
plot_covariance_ellipse(x, P, edgecolor='k', ls='dashed')
```



可以看到，位置变化使椭圆中心从 10 移到 11.35；椭圆同时被拉长，反映了位置与速度的相关性。滤波器如何计算新的 $\bar{\mathbf{P}}$？这里每次都将过程噪声 Q 设为零，所以变化并不是额外噪声造成的。预测把速度的不确定性传播到位置上，从而增大位置的边缘不确定性。在本例无噪声且状态转移可逆的情况下，这并不意味着联合状态信息必然损失。后面将给出推导。

设计过程噪声

简单回顾一下过程噪声。假设一辆汽车开启定速巡航，在道路上行驶，那么它的速度应该保持不变。我们用 $\bar{x}_k = \dot{x}_k \Delta t + x_{k-1}$ 来建立模型。不过，实际行驶会受到许多未知因素影响，定速巡航系统无法完美地保持恒定速度。风、坡道和坑洼都会影响汽车的运动；乘客摇下窗户，也会改变汽车的空气阻力。

我们可以用微分方程来描述这个系统。

$$\dot{\mathbf{x}} = f(\mathbf{x}) + w$$

其中，$f(\mathbf{x})$ 描述状态转移，w 是白过程噪声。

在“卡尔曼滤波的数学原理”一章中，我们将学习如何从微分方程组得到卡尔曼滤波所需的矩阵。本章则直接利用牛顿运动方程。将过程噪声协方差矩阵 $\mathbf{Q}$ 加到协方差 $\mathbf{P}$ 上，就能计入系统噪声的影响。这里另行假设过程噪声的均值为零，因此不需要向 $\mathbf{x}$ 添加噪声均值项，$\mathbf{x}$ 也不会因该项改变。白性本身并不保证零均值，二者是不同的假设。

单变量卡尔曼滤波器使用 `variance = variance + process_noise` 来计算预测结果的方差。多元卡尔曼滤波器的处理方式类似，本质上也是 `P = P + Q`。我之所以说“本质上类似”，是因为在协方差方程中，还有其他与噪声无关的项，我们稍后会详细讨论。

过程噪声矩阵的推导可能相当复杂，留待“卡尔曼滤波的数学原理”一章讨论。在这里的零均值假设下，$\mathbf{Q}$ 是过程白噪声 $\boldsymbol{w}$ 的协方差，即噪声外积的期望：$\mathbf{Q} = \mathbb{E}[\mathbf{w}\mathbf{w}^\mathsf{T}]$。本章重点建立直观认识：改变这个矩阵，会怎样改变滤波器的行为。

FilterPy 提供了计算本章运动学问题中 $\mathbf{Q}$ 的函数。这里使用 Q_discrete_white_noise 的三个主要参数：dim 指定矩阵的维数，dt 指定以秒为单位的时间步长，var 指定噪声方差。简而言之，该函数在一定假设下，将给定时间间隔内的噪声离散化；这些假设会在后文讨论。下面的代码计算噪声方差为 2.35、时间步长为 1 秒时的 $\mathbf{Q}$：

```
from filterpy.common import Q_discrete_white_noise
Q = Q_discrete_white_noise(dim=2, dt=1., var=2.35)
print(Q)
```

```
[[0.588 1.175]
 [1.175 2.35 ]]
```

设计控制函数

卡尔曼滤波器不仅仅用于数据的滤波处理，它还能帮助我们把机器人、飞机等系统的控制指令也纳入滤波过程之中。假设我们正在控制一个机器人。在每个时间点上，我们都会根据机器人当前的位置与期望位置之间的关系，向机器人发送相应的控制信号。卡尔曼滤波器的算法会将这些信息纳入计算公式中，从而根据当前的速度以及驱动电机所接收到的控制指令来预测机器人的位置。请记住，我们绝不会丢弃任何有用的信息。

对于线性系统而言，控制输入所产生的效果可以用一组线性方程来描述。用线性代数的方式来表达，就是：

$$\Delta\mathbf{x} = \mathbf{B}\mathbf{u}$$

这里，$\mathbf{u}$ 是控制输入，而 $\mathbf{B}$ 则是控制输入的数学模型或控制函数。例如，$\mathbf{u}$ 可能是一个电压值，它决定了轮毂电机转动的速度；将 $\mathbf{B}$ 乘以 $\mathbf{u}$ 后，得到的结果就是 $\Delta\begin{bmatrix}x\\ \dot{x}\end{bmatrix}$。换句话说，需要计算出由于控制输入而使得 $\mathbf{x}$ 发生多少变化。

因此，关于先验均值的完整卡尔曼滤波方程为：

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$$

而这就是当你调用 `KalmanFilter.predict()` 时所计算出的公式。

你的狗可能已经被训练成能够对语音指令做出反应。不过，所有现有证据都表明，我的狗完全无法对任何指令做出反应。因此，我将 $\mathbf{B}$ 的值设为零。在 Python 中，我们可以这样编写代码：

```
B = 0.  # my dog doesn't listen to me!
u = 0
x, P = predict(x, P, F, Q, B, u)
print('x =', x)
print('P =', P)
```

```
x = [12.7  4.5]
P = [[680.587 301.175]
 [301.175 502.35 ]]
```

无控制输入时，不必显式将 $\mathbf{B}$ 和 $\mathbf{u}$ 都设为零：FilterPy 的独立 predict 函数默认 B=1、u=0，控制项为零；KalmanFilter 类中，未设置的 B 为 None，默认 u=0，同样不会添加控制作用。

```
predict(x, P, F, Q)[0] == predict(x, P, F, Q, B, u)[0]
```

```
array([ True,  True])
```

```
predict(x, P, F, Q)[1] == predict(x, P, F, Q, B, u)[1]
```

```
array([[ True,  True],
       [ True,  True]])
```

预测：总结

作为设计师，你的职责就是确定相关的矩阵参数。

- $\mathbf{x}$、$\mathbf{P}$：状态与协方差
- $\mathbf{F}$、$\mathbf{Q}$：过程模型与噪声协方差
- $\mathbf{B}, \mathbf{u}$：可选的控制输入模型与控制输入。

更新步骤

现在我们可以开始执行该滤波器的更新步骤了。你只需要再提供两个矩阵即可，这两个矩阵的含义相当简单易懂。

设计测量函数

卡尔曼滤波器在测量空间中完成更新计算。在单变量一章中，为避免增加复杂性，我们基本没有讨论这个问题。当时，我们用直接报告狗的位置的传感器跟踪它的位置。残差很容易计算：用测量位置减去滤波器预测的位置。

$$\textbf{residual} = \textbf{measured position} - \textbf{predicted position}$$

计算残差后，用卡尔曼增益将其映射为状态修正量，再把修正量加到先验状态均值上，得到新的估计。

假设我们用一个输出电压的温度计来跟踪温度，其电压与温度读数相对应，会发生什么？直接套用上面的残差公式便没有意义，因为不能用电压减去温度。

$$\mathbf{residual} = \mathbf{voltage} - \mathbf{temperature} \quad (NONSENSE!)$$

我们需要将温度转换为电压，这样才能进行减法运算。对于温度计来说，我们可以这样表示：

```
CELSIUS_TO_VOLTS = 0.21475
residual = voltage - (CELSIUS_TO_VOLTS * predicted_temperature)
```

为统一处理这类问题，卡尔曼滤波器要求你提供一个测量函数，将状态映射到测量空间。

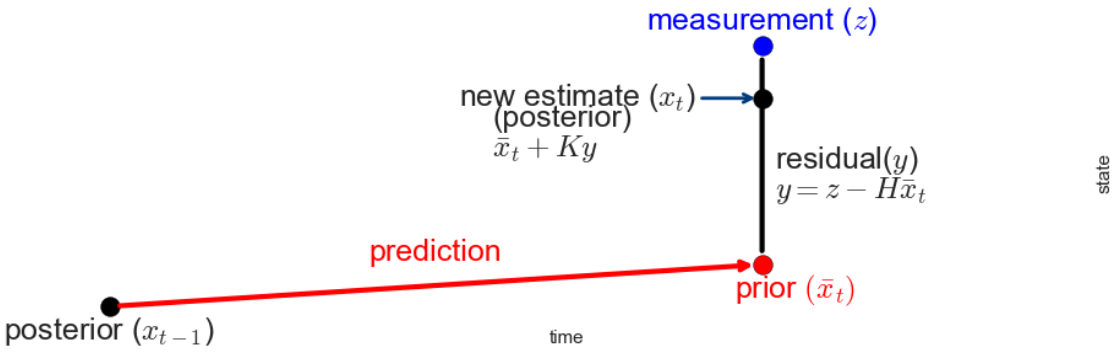
为什么在测量空间中计算？为什么不先把电压转换为温度，在状态空间中运算，使残差直接表示温度差呢？

一般不能通过测量函数的逆映射恢复完整状态。例如，跟踪状态中含有隐变量 $\dot{x}$，单次位置测量不足以同时确定速度；但从包含位置与速度的状态中提取预测位置却很容易。因此，在测量空间中计算残差最为自然。

测量值 $\mathbf{z}$ 和状态值 $\mathbf{x}$ 都是向量，因此我们需要使用矩阵来进行转换。执行这一操作的卡尔曼滤波方程如下：

$$\mathbf{y} = \mathbf{z} - \mathbf{H\bar{x}}$$

其中，$\mathbf{y}$ 是残差，$\mathbf{\bar{x}}$ 是先验状态，$\mathbf{z}$ 是测量值，$\mathbf{H}$ 是测量函数。先将先验状态左乘 $\mathbf{H}$，映射为预测测量；再用实际测量减去这个预测测量。得到的便是测量空间中的残差。



我们需要对 $\mathbf{H}$ 进行设计，以便 $\mathbf{H\bar{x}}$ 能够产生可测量的数值。对于这个问题，我们有一个用于测量位置的传感器，因此 $\mathbf{z}$ 将是一个一维向量。

$$\mathbf{z} = \begin{bmatrix} z \end{bmatrix}$$

残差方程的形式如下：

$$\mathbf{y} = \mathbf{z} - \mathbf{H\bar{x}}$$

$$\begin{bmatrix} y \end{bmatrix} = \begin{bmatrix} z \end{bmatrix} - \begin{bmatrix} ? & ? \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

为了让 $\mathbf{Hx}$ 成为 1x1 矩阵，$\mathbf{H}$ 必须是一个 1x2 矩阵。请记住，将矩阵 $m \times n$ 与 $n \times p$ 相乘后，会得到一个 $m \times p$ 矩阵。

我们需要将位置 x 乘以 1，从而得到该位置的对应数值。由于计算该数值时不需要用到速度，因此我们将 $\dot{x}$ 乘以 0。

$$\begin{aligned}\mathbf{y} &= \mathbf{z} - \begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}x \\ \dot{x}\end{bmatrix} \\ &= [z] - [x]\end{aligned}$$

因此，对于我们的卡尔曼滤波器，我们进行了如下设置：

$$\mathbf{H} = \begin{bmatrix}1 & 0\end{bmatrix}$$

```
H = np.array([[1., 0.]])
```

我们已经设计了卡尔曼滤波器的大部分组件。现在只需对传感器中的噪声进行建模即可。

设计测量模型

测量由测量向量 $\mathbf{z}$ 及测量噪声协方差 $\mathbf{R}$ 表示。

$\mathbf{z}$ 很简单。它以向量的形式来表示各项测量数值。因为我们只有一项测量数据，所以表达方式如下：

$$\mathbf{z} = [z]$$

如果我们有两个传感器或测量数据，那么就会有：

$$\mathbf{z} = \begin{bmatrix}z_1 \\ z_2\end{bmatrix}$$

测量噪声矩阵用协方差描述传感器噪声。实际中，复杂系统可能具有多个传感器，噪声之间的相关性不明确，分布也未必是严格高斯。例如，温度升高可能使传感器读数产生正偏差；其他因素还可能导致偏斜或其他非高斯特征。后面将学习如何处理这些问题。存在偏差并不必然意味着分布不对称，二者应加以区分。

卡尔曼滤波用协方差矩阵 $\mathbf{R}$ 表示测量噪声，其维数为 $m \times m$，其中 m 是测量向量的分量数；当每个传感器只报告一个标量时，它也等于传感器数量。非对角元描述不同测量噪声之间的互协方差。本例只有一个标量位置测量，因此 $\mathbf{R}$ 为：

$$\mathbf{R} = [\sigma_z^2]$$

如果方差 σ_z^2 为 5 平方米，则有 $\mathbf{R} = [5]$。

若有两个位置传感器，第一个的测量噪声方差为 5 m^2，第二个为 3 m^2，则可以写成：

$$\mathbf{R} = \begin{bmatrix}5 & 0 \\ 0 & 3\end{bmatrix}$$

我们将方差放在对角线上，因为这是一个协方差矩阵。在协方差矩阵中，方差位于对角线上，而协方差（如果有的话）则位于对角线以外的元素位置。这里我们假设两个传感器产生的噪声之间不存在相关性，因此协方差值为 0。

对于我们的问题来说，我们只有一个传感器，因此可以这样来实现。

```
R = np.array([[5.]])
```

我们通过调用 `update` 来执行更新操作。

```
from filterpy.kalman import update
z = 1.
x, P = update(x, P, z, R, H)
print('x =', x)
```

```
x = [ 1.085 -0.64 ]
```

要同时跟踪所有这些变量相当麻烦，因此 FilterPy 还提供了 `KalmanFilter` 这个类来帮助实现滤波功能。在本书的后续内容中，我将会继续使用这个类。不过，我觉得还是有必要让你们了解这些函数的实现方式，因为我知道，有些人并不喜欢面向对象编程。

实现卡尔曼滤波器

我已经把与滤波器相关的所有代码都提供给了你们，现在让我们把它们集中到一处吧。首先，我们需要创建一个 `KalmanFilter` 对象。我们必须使用 `dim_x` 参数来指定状态中变量的数量，用 `dim_z` 参数来指定测量值的数量。在这个例子中，状态中有两个随机变量和一项测量值，因此我们这样编写代码：

```
from filterpy.kalman import KalmanFilter
dog_filter = KalmanFilter(dim_x=2, dim_z=1)
```

这样就创建了一个对象，其中包含了卡尔曼滤波器各矩阵的默认值。

```
from filterpy.kalman import KalmanFilter
dog_filter = KalmanFilter(dim_x=2, dim_z=1)
print('x = ', dog_filter.x.T)
print('R = ', dog_filter.R)
print('Q = \n', dog_filter.Q)
# etc...
```

```
x =  [[0. 0.]]
R =  [[1.]]
Q =
 [[1. 0.]
 [0. 1.]]
```

现在，我们用适合本问题的数值来初始化滤波器的各种矩阵和向量。我把这个过程放入了一个函数中，这样用户就可以为 `R`、 `P` 和 `Q` 指定不同的初始值。我们会创建并使用许多这样的滤波器，这样做可以大大节省我们的工作量。

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

def pos_vel_filter(x, P, R, Q=0., dt=1.0):
    """ Returns a KalmanFilter which implements a
    constant velocity model for a state [x dx].T
    """

    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.x = np.array([x[0], x[1]]) # location and velocity
    kf.F = np.array([[1., dt],
                     [0.,  1.]])  # state transition matrix
    kf.H = np.array([[1., 0]])    # Measurement function
    kf.R *= R                     # measurement uncertainty
    if np.isscalar(P):
        kf.P *= P                 # covariance matrix
    else:
        kf.P[:] = P               # [:] makes deep copy
    if np.isscalar(Q):
        kf.Q = Q_discrete_white_noise(dim=2, dt=dt, var=Q)
    else:
        kf.Q[:] = Q
    return kf
```

KalmanFilter 将 R 、 P 和 Q 初始化为单位矩阵。因此， kf.P *= P 是一种将所有对角线元素都设置为相同标量值的简便方法。现在我们来创建这个滤波器：

```
dt = .1
x = np.array([0., 0.])
kf = pos_vel_filter(x, P=500, R=5, Q=0.1, dt=dt)
```

你可以在命令行中输入相应的变量，从而查看该滤波器所有属性的当前值。

```
kf
```

```
KalmanFilter object
dim_x = 2
dim_z = 1
dim_u = 0
x = [0. 0.]
P = [[500.   0.]
     [  0. 500.]]
x_prior = [[0. 0.]].T
P_prior = [[1. 0.]
           [0. 1.]]
x_post = [[0. 0.]].T
P_post = [[1. 0.]
          [0. 1.]]
F = [[1.  0.1]
     [0.  1. ]]
Q = [[0.    0.   ]
     [0.    0.001]]
R = [[5.]]
H = [[1. 0.]]
K = [[0. 0.]].T
y = [[0.]]
S = [[0.]]
SI = [[0.]]
M = [[0.]]
B = None
z = [[None]]
log-likelihood = -708.3964185322641
likelihood = 2.2250738585072014e-308
mahalanobis = 0.0
```

```
alpha = 1.0
inv = <function inv at 0x00000268CA22DB80>
```

现在要做的，就是编写代码来运行卡尔曼滤波器了。

```
from kf_book.mkf_internal import plot_track

def run(x0=(0.,0.), P=500, R=0, Q=0, dt=1.0,
        track=None, zs=None,
        count=0, do_plot=True, **kwargs):
    """
    track is the actual position of the dog, zs are the
    corresponding measurements.
    """

    # Simulate dog if no data provided.
    if zs is None:
        track, zs = compute_dog_data(R, Q, count)

    # create the Kalman filter
    kf = pos_vel_filter(x0, R=R, P=P, Q=Q, dt=dt)

    # run the kalman filter and store the results
    xs, cov = [], []
    for z in zs:
        kf.predict()
        kf.update(z)
        xs.append(kf.x)
        cov.append(kf.P)

    xs, cov = np.array(xs), np.array(cov)
    if do_plot:
        plot_track(xs[:, 0], track, zs, cov, **kwargs)
    return xs, cov
```

这就是滤波器的完整代码，其中大部分是常规的样板代码。为支持本章中的多种用途，我使它具备一定灵活性，因此略显冗长。下面逐行分析。

首先，程序会检查你是否在 `zs` 中提供了必要的测量数据。如果没有，程序会使用我们之前编写的 `compute_dog_data` 函数来生成这些数据。

接下来的几行代码使用了我们的辅助函数来构建卡尔曼滤波器。

```
# create the Kalman filter
kf = pos_vel_filter(x0, R=R, P=P, Q=Q, dt=dt)
```

对每次测量，我们只需依次执行卡尔曼滤波器的预测和更新步骤。KalmanFilter 类提供了 predict() 和 update() 两个方法来完成这些操作。update() 执行测量更新，因此需要以传感器测量值作为输入。

如果不进行结果存储的操作，该循环将执行如下操作：

```
 for z in zs:
kf.predict()
kf.update(z)
```

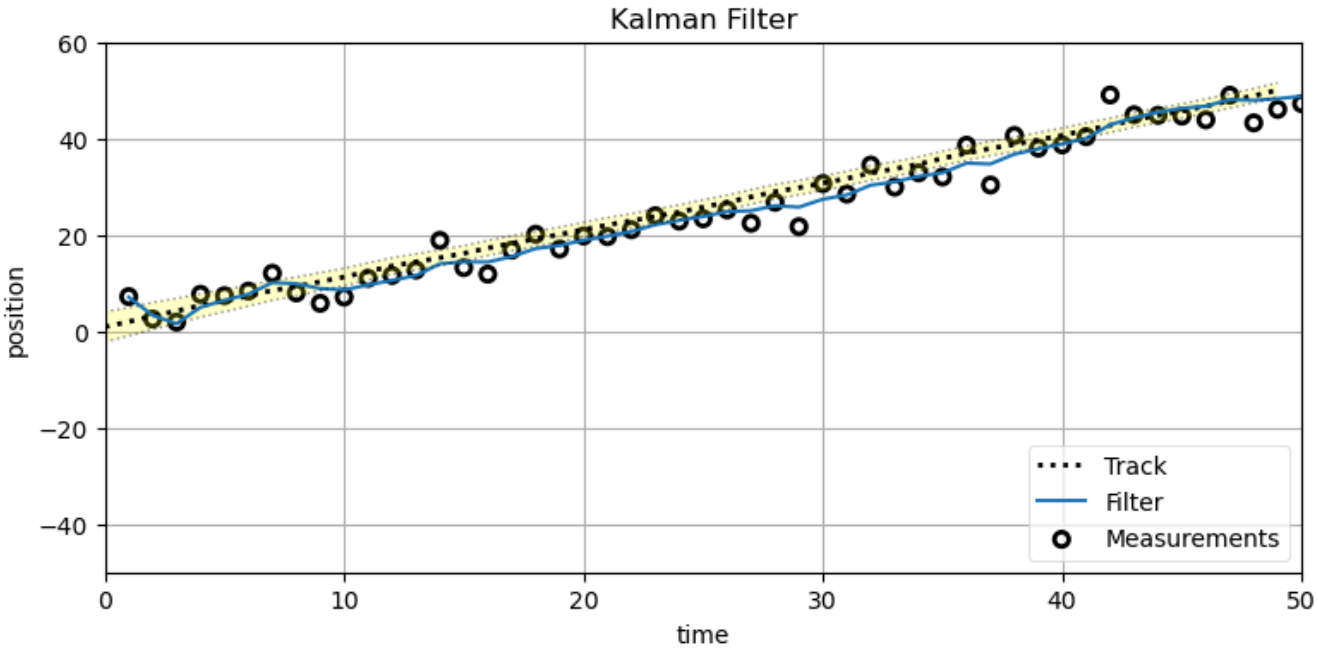
每次对 `predict` 和 `update` 的调用都会改变状态变量 `x` 和 `P` 的值。因此，在调用 `predict` 之后， `kf.x` 中保存的是先验值。在调用更新函数之后， `kf.x` 中则保存了后验值。这些状态值和协方差则分别保存在 `xs` 和 `cov` 中。

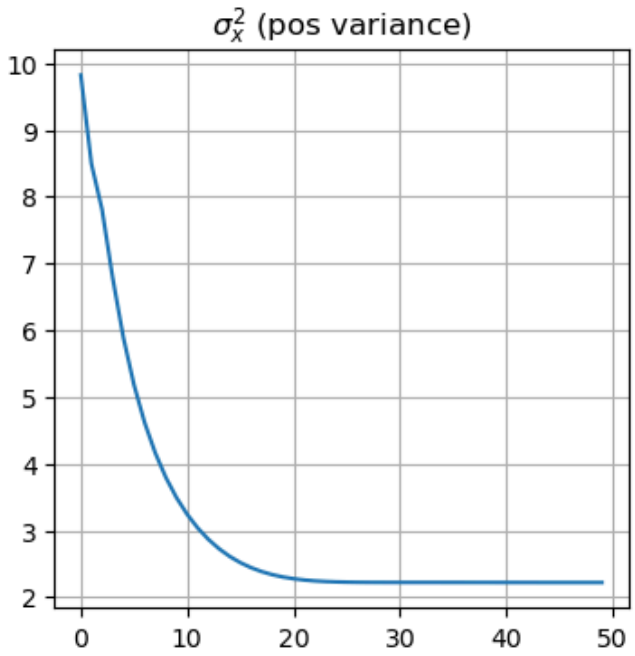
真的没有比这更简单的了。在处理更复杂的问题时，这段代码的基本结构依然保持不变；所有的工作都集中在对 `KalmanFilter` 矩阵的设置上；而执行滤波操作则非常简单。

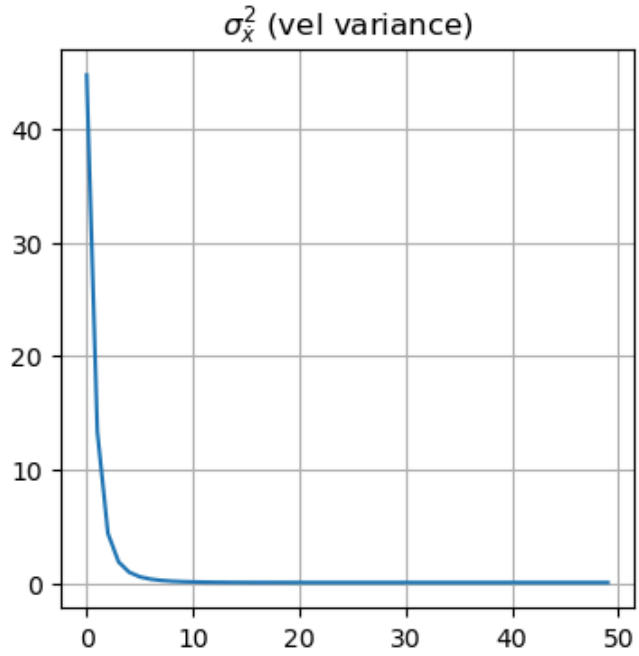
其余的代码会根据需要绘制出结果图，然后返回已保存的状态值和协方差矩阵。

让我们来运行一下吧。我们有 50 组测量数据，其中噪声的方差为 10，而过程本身的方差则为 0.01。

```
P = np.diag([500., 49.])
Ms, Ps = run(count=50, R=10, Q=0.01, P=P)
```







虽然还有很多需要学习的地方，但我们已经成功实现了卡尔曼滤波器，所使用的理论和公式都与鲁道夫·卡尔曼所提出的完全一致！类似这样的代码被广泛应用于 GPS、客机、机器人等各种设备中。

第一张图表展示了卡尔曼滤波器的输出结果，以及狗狗的测量值和实际位置（标记为“Track”）。在经过最初的稳定期后，该滤波器应该能够非常精确地追踪狗狗的位置。黑色虚线之间的黄色阴影区域代表了滤波器误差的 1 个标准差，具体解释请见下一段。

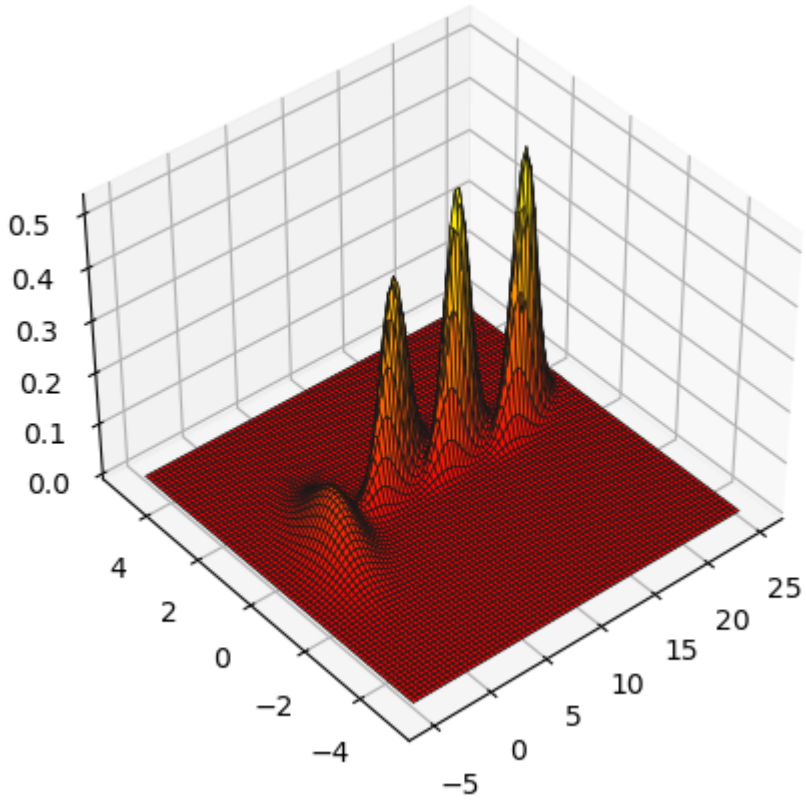
接下来的两个图表分别展示了 $\boldsymbol{x}$ 和 $\dot{\boldsymbol{x}}$ 的方差。我还绘制了 $\mathbf{P}$ 的对角线值随时间的变化情况。需要记住的是，协方差矩阵的对角线元素分别代表各个状态变量的方差。因此，$\mathbf{P}[0,0]$ 实际上就是 $\boldsymbol{x}$ 的方差，而 $\mathbf{P}[1,1]$ 则是 $\dot{\boldsymbol{x}}$ 的方差。可以看出，这两种情况的方差都很快趋近于较小的数值。

协方差矩阵 $\mathbf{P}$ 描述了模型及噪声假设成立时滤波器的理论性能。标准差是方差的平方根；一维高斯分布约有 68% 的概率质量位于均值两侧一个标准差之内。如果真实状态约有 68% 的时间落在估计均值两侧一个标准差的区间内，就表明滤波器的不确定性估计可能与实际相符。上图中，两条虚线之间的黄色阴影表示这一范围。目测真实轨迹落在范围外的情形似乎略多，因此滤波器可能还需要调参。覆盖率并非越高越好；区间过宽也会得到过高的覆盖率，应同时检查估计精度和不确定性的一致性。

在单变量一章中，我们用比上面简单得多的代码滤除了强噪声。不过，这里也只是一个很简单的例子：一个物体在一维空间中运动，只有一个传感器。那已接近前面单变量代码所能处理的范围。相比之下，只需改变滤波器变量的设置，本章代码就能实现复杂的多维滤波器。例如，可以跟踪金融模型中的 100 个维度；也可以融合飞机上的 GPS、惯性导航系统（INS）、战术空中导航系统（TACAN）、雷达高度计、气压高度计和空速计，以预测三维空间中的位置、速度与加速度。本章代码能够处理这类问题。

为帮助你直观理解高斯分布如何随时间变化，下图以三维形式展示了每隔 7 个时间步的分布。这样的间隔足以将各个分布分开，便于逐个观察。最左边的是 $\boldsymbol{t} = \mathbf{0}$ 时的第一个高斯分布。

```
from kf_book.book_plots import set_figsize, figsize
from kf_book.nonlinear_plots import plot_gaussians

P = np.diag([3., 1.])
np.random.seed(3)
Ms, Ps = run(count=25, R=10, Q=0.01, P=P, do_plot=False)
with figsize(x=9, y=5):
    plot_gaussians(Ms[::7], Ps[::7], (-5,25), (-5, 5), 75)
```



Saver 类

在 run() 方法中，我编写了用于保存滤波结果的样板代码。

```
 xs, cov = [], []
for z in zs:
kf.predict()
kf.update(z)
xs.append(kf.x)
cov.append(kf.P)
xs, cov = np.array(xs), np.array(cov)
```

避免这些重复代码有个简单办法：filterpy.common 提供了 Saver 类。每次调用 Saver.save()，它都会保存卡尔曼滤波器对象各属性的当前值。先看一个例子，再进一步讨论。

```
from filterpy.common import Saver
kf = pos_vel_filter([0, .1], R=R, P=P, Q=Q, dt=1.)
s = Saver(kf)
for i in range(1, 6):
    kf.predict()
    kf.update([i])
    s.save()  # save the current state
```

Saver 对象现在包含了 KalmanFilter 对象的所有属性信息。 kf.x 表示该滤波器当前的状态估计值。而 s.x 则保存了在循环过程中计算得到的状态估计值。

```
s.x
```

```
[array([0.531, 0.304]),
 array([1.555, 0.763]),
 array([2.784, 1.036]),
 array([3.944, 1.105]),
 array([5.015, 1.086])]
```

你可以通过 keys 属性查看所有可用的属性。

```
s.keys
```

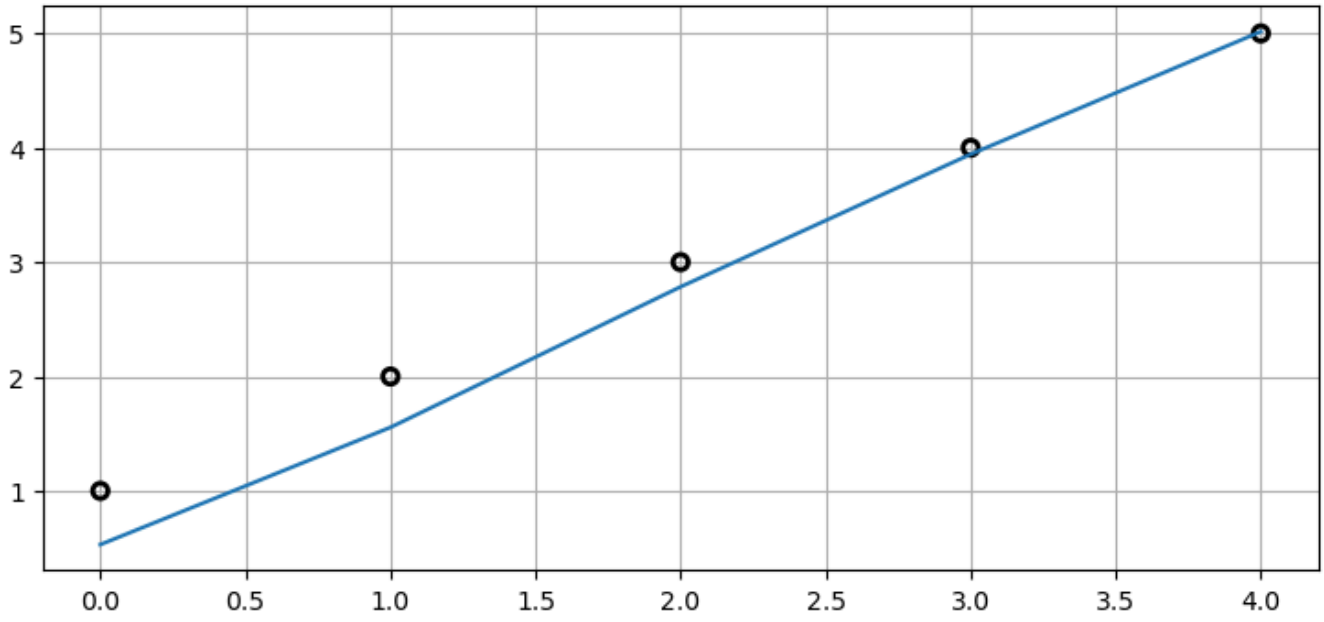
```
['alpha',
 'likelihood',
 'log_likelihood',
 'mahalanobis',
 'dim_x',
 'dim_z',
 'dim_u',
 'x',
 'P',
 'Q',
 'B',
 'F',
 'H',
 'R',
 '_alpha_sq',
 'M',
 'z',
 'K',
 'y',
 'S',
 'SI',
 '_I',
 'x_prior',
 'P_prior',
 'x_post',
 'P_post',
 '_log_likelihood',
 '_likelihood',
 '_mahalanobis',
 'inv']
```

其中还有许多属性我们尚未讨论，不过大多数应该都是大家所熟悉的。

此时可以编写代码绘制这些属性，但通常将列表转换成 NumPy 数组会更方便。调用 Saver.to_array() 即可完成转换。需要注意：若某个属性在运行过程中改变了形状，列表中的各项就可能无法组成规则数值数组，从而使转换失败。

再查看一次 keys，就会发现其中有 z。这说明测量值 z 也已保存下来。下面将它与估计值画在一起比较。

```
import matplotlib.pyplot as plt

s.to_array()
book_plots.plot_measurements(s.z);
plt.plot(s.x[:, 0]);
```



虽然这里使用 KalmanFilter 类演示，但 Saver 适用于 FilterPy 实现的所有滤波器类。它通过检查对象来获取属性名，因此通常也适用于你自己编写的类。本书将使用 Saver 使代码保持简洁、易读。Saver 在内部需要执行许多操作，因而会降低运行速度；不过，在学习和探索时，它带来的便利很有价值。

卡尔曼滤波方程

我们现在准备了解 `predict()` 和 `update()` 是如何进行计算的。

先说明一下记号。我是一名程序员，习惯使用下面这样的代码：

```
x = x + 1
```

这不是数学等式，因为两边并不相等，而是一条赋值语句。若要用数学记号表达同一操作，应写作

$$x_k = x_{k-1} + 1$$

。

卡尔曼滤波方程中到处都是下标和上标，这些符号的存在是为了确保方程在数学上的一致性。不过，我觉得这反而让方程变得极其难以理解。在本书中，我尽量不使用下标来表示各种变量。作为一名程序员，你应该明白：我所展示的这些代码其实代表了某种算法的实现方式，而这种算法需要一步步来执行。等我们有了具体的例子后，我会再详细解释这一点。

预测方程式

卡尔曼滤波器利用这些方程来计算系统的“先验值”——也就是对系统下一状态的预测值。具体来说，就是计算系统的先验均值（$\bar{\mathbf{x}}$）和协方差（$\bar{\mathbf{P}}$）。

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$$
$$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$$

Mean

$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$

需要提醒的是，线性方程 $\mathbf{A}\mathbf{x} = \mathbf{b}$ 实际上代表了一组方程式。其中， $\mathbf{A}$ 表示这些方程式的系数； $\mathbf{x}$ 则是各个变量的向量。通过执行乘法运算 $\mathbf{A}\mathbf{x}$，可以得出该组方程式的右侧数值，这些数值由 $\mathbf{b}$ 表示。

如果 $\mathbf{F}$ 包含了某个特定时间步的状态转移信息，那么 $\mathbf{F}\mathbf{x}$ 则计算出该转换之后的状态。很简单吧！同样地， $\mathbf{B}$ 代表控制函数， $\mathbf{u}$ 代表控制输入，因此 $\mathbf{B}\mathbf{u}$ 则计算了各种控制因素对转换后状态的贡献。最后， $\bar{\mathbf{x}}$ 的值则是 $\mathbf{F}\mathbf{x}$ 与 $\mathbf{B}\mathbf{u}$ 之和。

相应的单变量方程为

$$\bar{\mu} = \mu + \mu_{move}$$

如果你执行矩阵乘法 $\mathbf{F}\mathbf{x}$，就会得到关于 $\boldsymbol{x}$ 的这个方程。

让我们把这一点说清楚。回想一下上一章中 $\mathbf{F}$ 的值：

$$\mathbf{F} = \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}$$

因此， $\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$ 对应于以下线性方程组：

$$\begin{cases}\bar{x} = 1x + & \Delta t\,\dot{x} \\ \bar{\dot{x}} = 0x + & 1\,\dot{x}\end{cases}$$

Covariance

$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$

这个方程式不太容易理解，因此我们需要花更多时间来研究它。

该方程的单变量形式为：

$$\bar{\sigma}^2 = \sigma^2 + \sigma^2_{move}$$

我们将这种“运动过程中的不确定性”所导致的方差，加到我们的估计值方差上，以此来反映知识上的损失。在这里，我们也需要做同样的处理，只不过，对于多元高斯分布来说，这并不那么容易实现。

不能简单地写成 $\bar{\mathbf{P}} = \mathbf{P} + \mathbf{Q}$。多元状态的各个分量会通过过程模型相互影响。例如，我们并不完全知道速度，却仍需要用它推算下一时刻位置：

$$\bar{x} = \dot{x}\Delta t + x$$

由于我们无法准确知道 $\dot{\boldsymbol{x}}$ 的值，因此 $\bar{\boldsymbol{x}} = \dot{\boldsymbol{x}}\Delta \boldsymbol{t} + \boldsymbol{x}$ 的值也就具有不确定性。由于位置和速度之间存在相关性，我们无法简单地将它们的协方差矩阵相加。例如，如果 $\mathbf{P}$ 和 $\mathbf{Q}$ 都是对角矩阵，那么它们的和也仍是对角矩阵。但由于位置与速度相关，对角线以外的元素必然不为零。

正确的方程式是

$$\bar{\mathbf{P}} = \mathbf{FPF}^\mathsf{T} + \mathbf{Q}$$

$\mathbf{ABA}^\mathsf{T}$ 形式的表达式在线性代数中很常见。可以把它直观理解为：用两侧的变换对中间的协方差进行映射。本书后面还会多次用到这种形式。我承认，这个方程目前对你来说可能仍有些“神奇”。下面具体看看。

若将 $\mathbf{P}$ 初始化为下式：

$$\mathbf{P} = \begin{bmatrix}\sigma_x^2 & 0 \\ 0 & \sigma_v^2\end{bmatrix}$$

$\mathbf{FPF}^\mathsf{T}$ 的值为：

$$\begin{aligned}\mathbf{FPF}^\mathsf{T} &= \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}\begin{bmatrix}\sigma_x^2 & 0 \\ 0 & \sigma_v^2\end{bmatrix}\begin{bmatrix}1 & 0 \\ \Delta t & 1\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2 & \sigma_v^2\Delta t \\ 0 & \sigma_v^2\end{bmatrix}\begin{bmatrix}1 & 0 \\ \Delta t & 1\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2 + \sigma_v^2\Delta t^2 & \sigma_v^2\Delta t \\ \sigma_v^2\Delta t & \sigma_v^2\end{bmatrix}\end{aligned}$$

$\mathbf{P}$ 的初始值中，位置和速度之间不存在协方差。位置是通过 $\dot{\boldsymbol{x}}\Delta t + \boldsymbol{x}$ 来计算的，因此位置和速度之间存在相关性。通过 $\mathbf{FPF}^\mathsf{T}$ 进行运算后，可以得到 $\sigma_v^2\Delta t$ 这个协方差值。具体的数值并不重要；只要明白 $\mathbf{FPF}^\mathsf{T}$ 利用了该模型来自动计算位置和速度之间的协方差即可！

还可以从 $\mathbf{Fx}$ 的乘法来理解：这一运算将 $\mathbf{x}$ 向时间上推进。$\mathbf{FP}$ 看起来像是相应的协方差运算，但 $\mathbf{P}$ 是矩阵，而 $\mathbf{x}$ 是向量。右侧的 $\mathbf{F}^\mathsf{T}$ 确保状态转移矩阵 $\mathbf{F}$ 同时作用于协方差矩阵的两个维度。在上面 $\mathbf{FPF}^\mathsf{T}$ 的推导中，第二行已经算出了 $\mathbf{FP}$；此时得到的是上三角矩阵，因为尚未乘上右侧的转置因子，$\mathbf{F}$ 对协方差的变换还未完成。

如果你了解一些线性代数和统计学，下面的推导可能会有帮助。预测产生的协方差可写为预测误差与其转置的外积的期望，如下式所示：

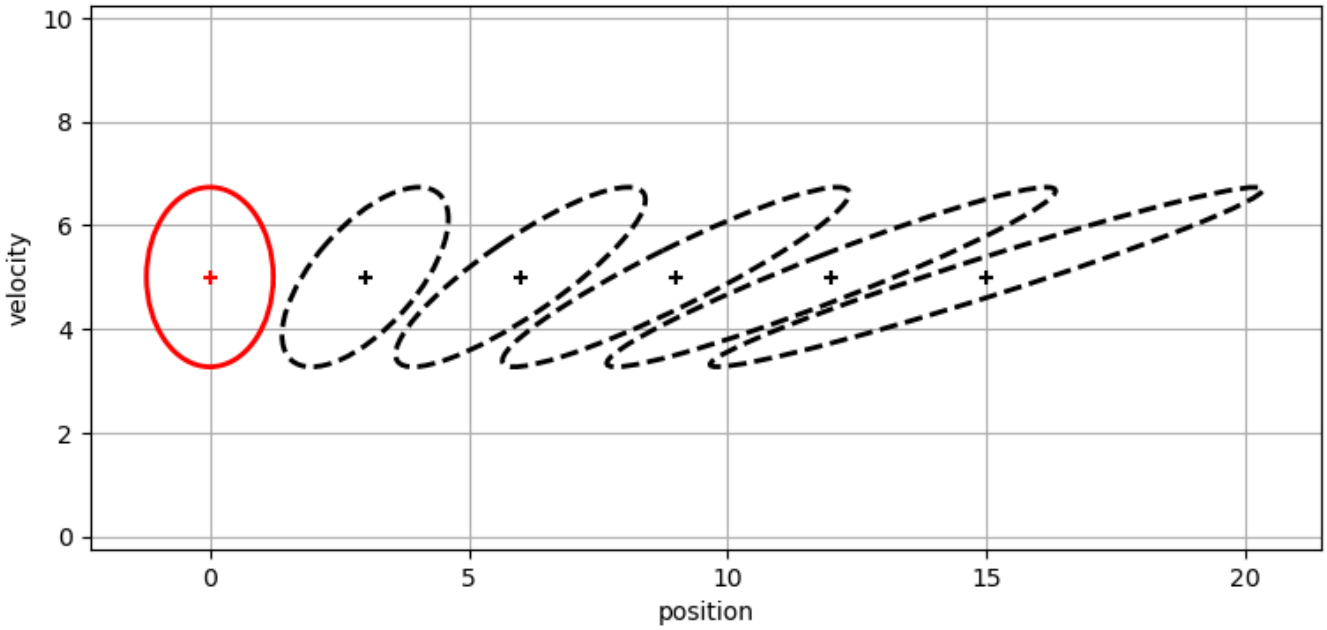
$$\begin{aligned}\bar{\mathbf{P}} &= \mathbb{E}[(\mathbf{Fx} - \mathbf{F}\bar{\boldsymbol{\mu}})(\mathbf{Fx} - \mathbf{F}\bar{\boldsymbol{\mu}})^\mathsf{T}] \\ &= \mathbf{F}\,\mathbb{E}[(\mathbf{x} - \bar{\boldsymbol{\mu}})(\mathbf{x} - \bar{\boldsymbol{\mu}})^\mathsf{T}]\,\mathbf{F}^\mathsf{T}\end{aligned}$$

当然，$\mathbb{E}[(\mathbf{x} - \bar{\boldsymbol{\mu}})(\mathbf{x} - \bar{\boldsymbol{\mu}})^\mathsf{T}]$ 其实也就是 $\mathbf{P}$ 而已，这没什么好奇怪的。

$$\bar{\mathbf{P}} = \mathbf{FPF}^\mathsf{T}$$

让我们来看看它的效果。这里，我使用了我们滤波器中的 $\mathbf{F}$，并将状态向前推进了 6/10 秒。我重复了这个操作五次，这样你就能看到 $\bar{\mathbf{P}}$ 是如何持续变化的了。

```
dt = 0.6
x = np.array([0., 5.])
F = np.array([[1., dt], [0, 1.]])
P = np.array([[1.5, 0], [0, 3.]])
plot_covariance_ellipse(x, P, edgecolor='r')

for _ in range(5):
    x = F @ x
    P = F @ P @ F.T
    plot_covariance_ellipse(x, P, edgecolor='k', ls='dashed')
book_plots.set_labels(x='position', y='velocity')
```



可以看到，速度为 5 时，每个 0.6 秒的时间步内，位置恰好前进 3 个单位。椭圆在位置方向的跨度逐步增大，表明每次向 x 加上 $\dot{\boldsymbol{x}}\Delta t$ 都会引入位置不确定性。速度方向的跨度没有变化：系统模型假设速度恒定，因此关于速度的概率分布保持不变。随着时间推移，椭圆越来越倾斜，这表示位置与速度之间的相关性。$\mathbf{F}$ 通过 $\bar{\boldsymbol{x}} = \dot{\boldsymbol{x}}\Delta t + \boldsymbol{x}$ 将 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 线性联系起来，$\mathbf{FPF}^{\mathsf{T}}$ 则把这种相关性正确地纳入协方差矩阵。

下面的交互动画可以改变 $\mathbf{F}$，观察 $\mathbf{P}$ 的形状变化。F00 滑块控制 F[0, 0]，covar 控制位置与速度之间的初始协方差，可写为 $\boldsymbol{\rho}_{\boldsymbol{x},\dot{\boldsymbol{x}}}\boldsymbol{\sigma}_{\boldsymbol{x}}\boldsymbol{\sigma}_{\dot{\boldsymbol{x}}}$，即相关系数与两个标准差的乘积。建议至少思考以下问题。

- 如果 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 之间没有相关性会怎样呢？（将 F01 设置为 0，其余参数保持默认值。）
- 若 $\boldsymbol{x} = \mathbf{2}\dot{\boldsymbol{x}}\Delta t + \boldsymbol{x}_{\mathbf{0}}$，会怎样？（将 F01 设为 2，其余保持默认值。）
- 若 $\boldsymbol{x} = \dot{\boldsymbol{x}}\Delta t + \mathbf{2}\boldsymbol{x}_{\mathbf{0}}$，会怎样？（将 F00 设为 2，其余保持默认值。）
- 若 $\boldsymbol{x} = \dot{\boldsymbol{x}}\Delta t$，会怎样？（将 F00 设为 0，其余保持默认值。）

```
from ipywidgets import interact
from kf_book.book_plots import IntSlider, FloatSlider

def plot_FPFT(F00, F01, F10, F11, covar):
    plt.figure()
    dt = 1.
    x = np.array((0, 0.))
    P = np.array(((1, covar), (covar, 2)))
    F = np.array(((F00, F01), (F10, F11)))
    plot_covariance_ellipse(x, P)
```

```
    plot_covariance_ellipse(x, F @ P @ F.T, ec='r')
    plt.gca().set_aspect('equal')
    plt.xlim(-4, 4)
    plt.ylim(-4, 4)
    #plt.title(str(F))
    plt.xlabel('position')
    plt.ylabel('velocity')
    plt.show()

interact(plot_FPFT,
         F00=IntSlider(value=1, min=0, max=2),
         F01=FloatSlider(value=1, min=0, max=2, description='F01(dt)'),
         F10=FloatSlider(value=0, min=0, max=2),
         F11=FloatSlider(value=1, min=0, max=2),
         covar=FloatSlider(value=0, min=0, max=1));
```

```
interactive(children=(IntSlider(value=1, continuous_update=False, description='F00', max=2),
FloatSlider(value…
```

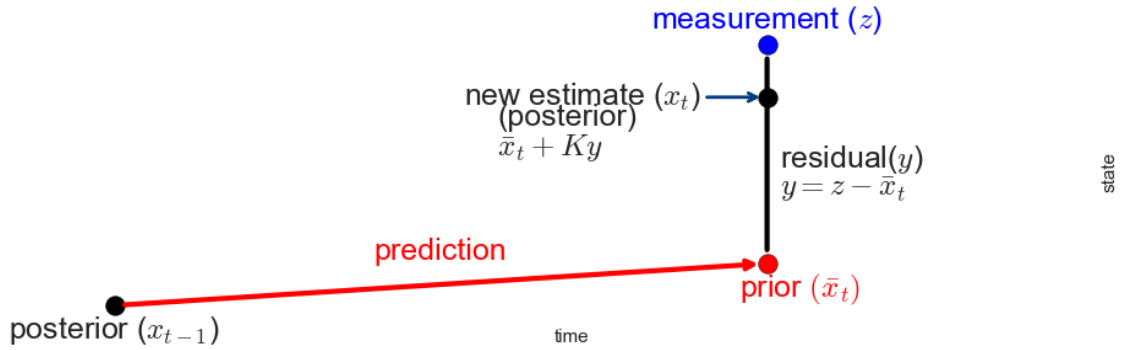
（如果你是以静态文本形式阅读此内容：在线运行该程序的说明请点击此处：https://git.io/vza7b）。或者，使用下面的链接进入 Binder 平台，再从那里打开该笔记本。

http://mybinder.org/repo/rlabbe/Kalman-and-Bayesian-Filters-in-Python

更新方程式

更新方程看起来比预测方程复杂，主要是因为卡尔曼滤波器在测量空间中计算更新。测量映射通常不可逆：例如，某个传感器只报告到目标的距离，而圆周上的无数个位置都对应同一个距离，因此仅凭距离无法唯一确定目标的位置。反过来，已知位置（状态），就可以计算相应的距离（测量）。

在继续之前，请记住，我们试图做的其实是一件非常简单的事情：在测量值和预测值之间选择一个合适的数值作为新的估计值，就像这张图表所展示的那样：



由于该系统具有多个维度，因此相关方程会相当复杂。但这就是我们需要做的操作。不要让这些复杂的方程式掩盖了这个概念本身的简单性。

System Uncertainty

$$\mathbf{S} = \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}$$

为了在测量空间中进行运算，卡尔曼滤波器需要将协方差矩阵投影到测量空间中。相关的数学公式为 $\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T}$，其中 $\bar{\mathbf{P}}$ 表示先验协方差，$\mathbf{H}$ 则代表测量函数。

你应该能认出这种 $\mathbf{ABA}^\mathsf{T}$ 形式——它其实就是 $\mathbf{FPF}^\mathsf{T}$ 中用来通过状态转移函数来更新 $\mathbf{P}$ 的预测步骤。在这里，我们同样使用这种形式，只不过是用测量函数来更新 $\mathbf{P}$ 而已。线性代数实际上起到了为我们调整坐标系的作用。

将先验协方差投影到测量空间后，还必须计入传感器噪声。做法是加上测量噪声协方差。所得矩阵通常称为系统不确定性或新息协方差。

如果忽略 $\mathbf{H}$ 项，那么这个方程就相当于卡尔曼增益的单变量方程中的分母部分。

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

比较该系统不确定性及其协方差的计算公式。

$$\mathbf{S} = \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}$$
$$\bar{\mathbf{P}} = \mathbf{FPF}^\mathsf{T} + \mathbf{Q}$$

在这两个方程中，$\mathbf{P}$ 分别通过函数 $\mathbf{H}$ 或 $\mathbf{F}$ 映射到相应空间，然后加上该空间对应的噪声协方差矩阵。

Kalman Gain

$$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}$$

再来看看残差图。一旦我们得到了预测值和测量值，就需要在这两个数值之间选择一个估计值。如果我们对测量值的把握更大，那么估计值就会更接近测量值；相反，如果我们对预测值的把握更大，那么估计值就会更接近预测值。

在单变量一章中，我们用下式对均值进行加权：

$$\mu = \frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$$

我们将其简化为：

$$\mu = (1-K)\bar{\mu} + K\mu_z$$

这让我们得到了……

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

在当前标量、直接位置测量的例子中，K 是卡尔曼增益，取值介于 0 与 1 之间。例如 K=0.9，表示更新后的均值含有 90% 的测量与 10% 的预测。一般的多维卡尔曼增益是矩阵，其各元素并不必然介于 0 与 1 之间，不能将这一标量结论推广到所有模型。

在多元卡尔曼滤波中，$\mathbf{K}$ 不再是标量；本例只有一个测量分量，因此它是列向量，一般情况下则是矩阵。再看方程 $\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}$。它是否仍可理解为一种“比值”？矩阵求逆可以类比于标量求倒数。虽然矩阵并没有通常意义下的除法，这种类比仍有助于直观理解。因此，可以把 $\mathbf{K}$ 的方程理解为：

$$\mathbf{K} \approx \frac{\bar{\mathbf{P}}\mathbf{H}^\mathsf{T}}{\mathbf{S}}$$
$$\mathbf{K} \approx \frac{\textbf{uncertainty}_{\textbf{prediction}}}{\textbf{uncertainty}_{\textbf{measurement}}}\mathbf{H}^\mathsf{T}$$

卡尔曼增益公式是根据我们对预测结果和测量结果的信任程度来计算出一个比例值。在之前的每一章中，我们都采用了同样的方法。虽然该公式因为需要用矩阵来处理多维数据而显得比较复杂，但其核心概念其实很简单。 $\mathbf{H}^\mathsf{T}$ 这个项的含义不太清楚，我稍后会解释。如果忽略这个项，卡尔曼增益的公式就与单变量情况下的公式相同：只需将先验信息的不确定性除以先验信息和测量结果不确定性的总和即可。

Residual

$$\mathbf{y} = \mathbf{z} - \mathbf{H}\bar{\mathbf{x}}$$

这很简单，因为在设计测量函数 $\mathbf{H}$ 时，我们已经处理过这个方程式了。请记住，测量函数的作用是将某种状态转移为可测量的数值。因此， $\mathbf{Hx}$ 会将 $\mathbf{x}$ 转换为等效的测量值。完成这一步后，我们只需用测量值 $\mathbf{z}$ 减去这个转换后的数值，就能得到残差——也就是测量值与预测值之间的差异。

该单变量方程为：

$$y = z - \bar{x}$$

显然，它计算出的结果也是一样的，只不过是在一维空间中进行的计算而已。

State Update

$$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{Ky}$$

卡尔曼增益将测量残差映射为状态修正量。$\mathbf{Ky}$ 完成这种缩放与映射；$\mathbf{K}$ 中的 $\mathbf{H}^\mathsf{T}$ 项参与把测量空间的信息传回状态空间。将修正量加到先验状态，便得到 $\mathbf{x} = \bar{\mathbf{x}} + \mathbf{Ky}$。展开 $\mathbf{K}$，可看清整个计算：

$$\begin{aligned}
\mathbf{x} &= \bar{\mathbf{x}} + \mathbf{Ky} \\
&= \bar{\mathbf{x}} + \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}\mathbf{y} \\
&\approx \bar{\mathbf{x}} + \frac{\textbf{uncertainty}_{\textbf{prediction}}}{\textbf{uncertainty}_{\textbf{measurement}}}\mathbf{H}^\mathsf{T}\mathbf{y}
\end{aligned}$$

或许，理解这一比例的更好方式是重新表述一下那个估算公式：

$$\begin{aligned}
\mathbf{x} &= \bar{\mathbf{x}} + \mathbf{Ky} \\
&= \bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}}) \\
&= (\mathbf{I} - \mathbf{KH})\bar{\mathbf{x}} + \mathbf{Kz}
\end{aligned}$$

这与单变量形式的相似性应该是显而易见的：

$$\mu = (1 - K)\bar{\mu} + K\mu_{\mathbf{z}}$$

Covariance Update

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$$

$\mathbf{I}$ 是单位矩阵，在多维运算中起到类似标量 $\mathbf{1}$ 的作用。$\mathbf{H}$ 是固定的测量矩阵。若只作标量类比，可将更新直观写成 $\mathbf{P} = (\mathbf{1} - c\mathbf{K})\mathbf{P}$，把 $\mathbf{K}$ 看作调节融合比例的增益。$\mathbf{K}$ 较大时，$(\mathbf{1} - \mathbf{cK})$ 较小，$\mathbf{P}$ 相对先验缩小得更多；$\mathbf{K}$ 较小时，$(\mathbf{1} - \mathbf{cK})$ 更接近 1，$\mathbf{P}$ 则保留较多先验不确定性。这是直观类比；一般矩阵没有统一的逐元素大小关系，不能把标量结论直接推广。

这个方程在数值计算中可能不稳定，因此 FilterPy 不使用这种形式。浮点运算中的减法可能破坏协方差矩阵的对称性，误差也可能随时间累积。后面将介绍更复杂但数值更稳定的形式。

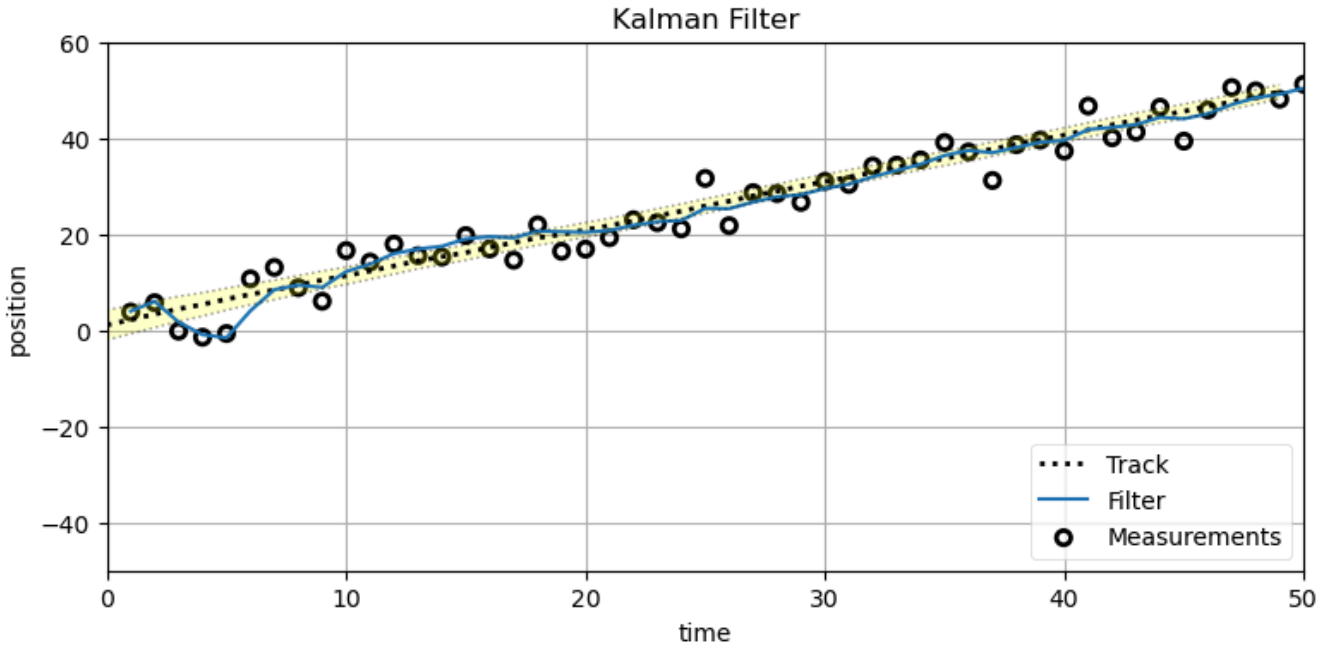
一个不使用 FilterPy 的示例

FilterPy 将实现细节隐藏了起来。通常情况下，这种处理方式很方便。不过，让我们试着不使用 FilterPy 来实现最后一个滤波器吧。为此，我们需要将相关的矩阵定义为变量，然后手动计算卡尔曼滤波所需的方程式。

在这里，我们对矩阵进行初始化：

```
dt = 1.
R_var = 10
Q_var = 0.01
x = np.array([[10.0, 4.5]]).T
P = np.diag([500, 49])
F = np.array([[1, dt],
              [0,  1]])
H = np.array([[1., 0.]])
R = np.array([[R_var]])
Q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_var)
```

```
from scipy.linalg import inv

count = 50
track, zs = compute_dog_data(R_var, Q_var, count)
xs, cov = [], []
for z in zs:
    # predict
    x = F @ x
    P = F @ P @ F.T + Q

    #update
    S = H @ P @ H.T + R
    K = P @ H.T @ inv(S)
    y = z - H @ x
    x += K @ y
    P = P - K @ H @ P

    xs.append(x)
    cov.append(P)

xs, cov = np.array(xs), np.array(cov)
plot_track(xs[:, 0], track, zs, cov, plot_P=False)
```



其结果与 FilterPy 版本完全一致。你可以选择自己更喜欢的版本。我个人不喜欢在命名空间中使用 `x`、`P` 之类的变量；对我来说，`dog_filter.x` 的格式更易读。

更重要的是，这个例子要求你牢记并正确编写卡尔曼滤波器的相关公式。迟早你会犯错。而使用 FilterPy 的话，就能确保代码的准确性。另一方面，如果你在定义时出错，比如把 $\mathbf{H}$ 设为列向量而非行向量，那么使用 FilterPy 时出现的错误信息将比使用这种显式编码方式时更难调试。

FilterPy 的 KalmanFilter 类还提供了许多其他功能，比如数据平滑处理、批量处理、基于衰减因子的记忆滤波、最大似然函数的计算等等。无需自行编写代码，即可使用这些功能。

本章小结

我们已经学过卡尔曼滤波方程，下面将它们汇总，便于复习。内容虽然不少，但希望你在学习每个方程时，都能看出它与单变量滤波方程的联系。在“卡尔曼滤波的数学原理”一章中，我将说明：把 $\mathbf{x}$ 的维数设为 1，这些方程就会退化为单变量滤波方程。它们与单变量滤波器的关系并非仅仅是“相似”，而正是同一滤波器的多维实现。

Predict Step

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$$
$$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$$

Update Step

$$\mathbf{S} = \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}$$
$$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}$$
$$\mathbf{y} = \mathbf{z} - \mathbf{H}\bar{\mathbf{x}}$$
$$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$$
$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}$$

我想分享一下在文献中常见的方程式表示方式。虽然存在多种不同的符号体系，但这种方式能让你大致了解方程式的表现形式。

$$
\begin{aligned}
\hat{\mathbf{x}}_{k|k-1} &= \mathbf{F}_k\hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k\mathbf{u}_k \\
\mathbf{P}_{k|k-1} &= \mathbf{F}_k\mathbf{P}_{k-1|k-1}\mathbf{F}_k^\mathsf{T} + \mathbf{Q}_k \\
\tilde{\mathbf{y}}_k &= \mathbf{z}_k - \mathbf{H}_k\hat{\mathbf{x}}_{k|k-1} \\
\mathbf{S}_k &= \mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^\mathsf{T} + \mathbf{R}_k \\
\mathbf{K}_k &= \mathbf{P}_{k|k-1}\mathbf{H}_k^\mathsf{T}\mathbf{S}_k^{-1} \\
\hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\tilde{\mathbf{y}}_k \\
\mathbf{P}_{k|k} &= (I - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1}
\end{aligned}
$$

这种记法采用贝叶斯条件符号 $a \mid b$，表示给定 b 的证据时对 a 的判断。帽号表示估计。因此，$\hat{\mathbf{x}}_{k|k}$ 是利用第 k 步的证据（竖线右侧的第二个 k），估计第 k 步的状态 $\mathbf{x}$（竖线左侧的第一个 k），也就是后验。$\hat{\mathbf{x}}_{k|k-1}$ 则表示根据第 $k-1$ 步的信息，对第 k 步的状态 $\mathbf{x}$ 进行估计，也就是先验。

这种记号法是从维基百科[1]上借鉴来的。它能让数学家更精确地表达自己的想法。在发表新研究成果的正式文献中，这种精确性是不可或缺的。不过，作为程序员，我觉得这种记号法很难理解。我习惯于将变量视为在程序运行过程中会不断变化的值，不会为每次计算都使用不同的变量名。由于学术界没有统一的记号规范，所以每位作者都会采用不同的方式来表示变量。我觉得在阅读不同的书籍和论文时，很难快速切换这种记号方式。因此，我选择了自己认为更简单的记号法。虽然数学家们可能会给我发愤怒的邮件，但我希望程序员和学生能喜欢我这种简化的记号法。

“符号说明附录”列出了不同作者所使用的符号表示法。但这又带来了另一个问题：不同的作者会使用不同的变量名称。$\mathbf{x}$ 这个符号比较通用，但除此之外，就很难确定其他符号的含义了。例如，我通常用 $\mathbf{F}$ 来表示某个量，而其他作者则可能用 $\mathbf{A}$ 来表示同一个量。因此，读者必须仔细阅读文本，并希望作者能明确说明各个变量的含义（不过，很多时候作者并不会这么做）。

如果你是一名程序员，试图理解论文中的数学公式，我建议先去掉所有的上标、下标和特殊符号，把它们都替换为一个字母。对于那些每天都要处理这类公式的人来说，这个建议可能没什么用处。但对我来说，记住“ P ”代表上一步计算出的 P 的更新值，要比试图记住 $P_{k-1}(+)$ 的含义、它与 $P_k(-)$ 的关系，以及这些符号与 5 分钟前读到的论文中的符号之间的联系容易得多。

练习：展示隐变量的影响

在我们的模型中，速度是一个隐变量。如果我们不在模型中考虑速度这一因素，那么滤波器的性能会如何呢?

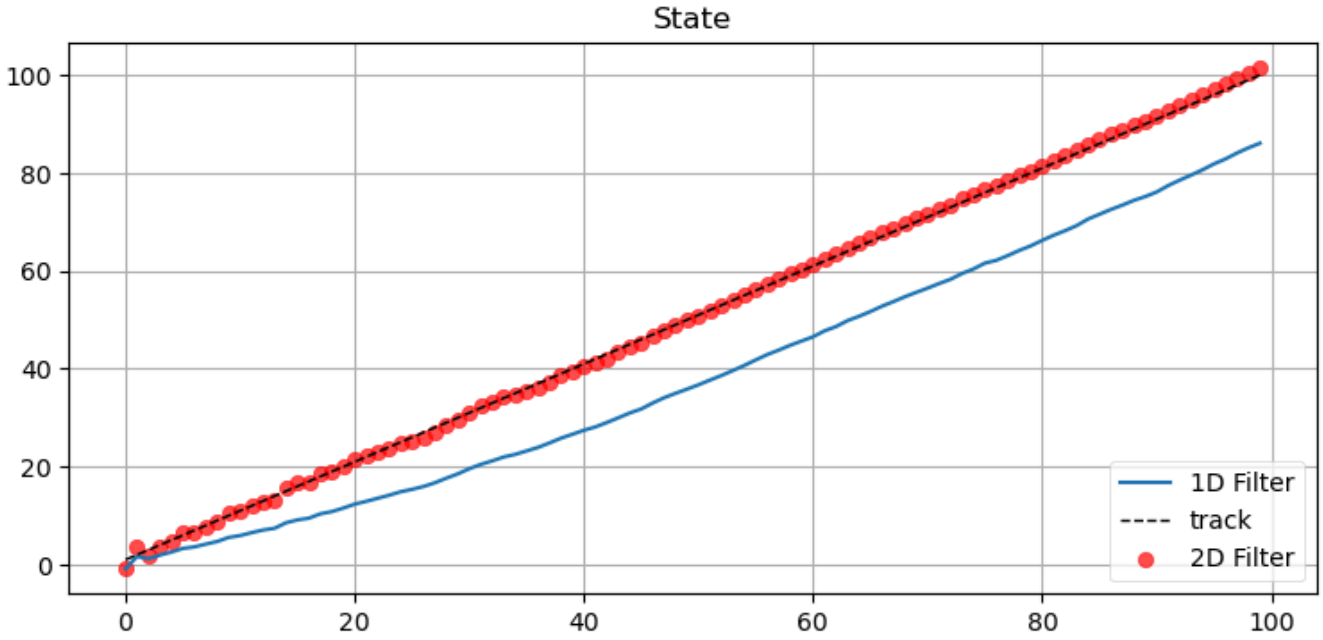
编写一个使用状态量 $\mathbf{x} = [x]$ 的卡尔曼滤波器，并将其与使用状态量 $\mathbf{x} = \begin{bmatrix}x & \dot{x}\end{bmatrix}^\mathsf{T}$ 的滤波器进行比较。

```
# your code here
```

解答

我们已经为位置和速度的计算实现了卡尔曼滤波器。因此，我会直接给出代码，不做太多解释，然后再绘制出相应的图表。

```
from math import sqrt
from numpy.random import randn

def univariate_filter(x0, P, R, Q):
    f = KalmanFilter(dim_x=1, dim_z=1, dim_u=1)
    f.x = np.array([[x0]])
    f.P *= P
    f.H = np.array([[1.]])
    f.F = np.array([[1.]])
    f.B = np.array([[1.]])
    f.Q *= Q
    f.R *= R
    return f

def plot_1d_2d(xs, xs1d, xs2d):
    plt.plot(xs1d, label='1D Filter')
    plt.scatter(range(len(xs2d)), xs2d, c='r', alpha=0.7, label='2D Filter')
    plt.plot(xs, ls='--', color='k', lw=1, label='track')
    plt.title('State')
    plt.legend(loc=4)
    plt.show()

def compare_1D_2D(x0, P, R, Q, vel, u=None):
    # storage for filter output
    xs, xs1, xs2 = [], [], []

    # 1d KalmanFilter
    f1D = univariate_filter(x0, P, R, Q)

    #2D Kalman filter
    f2D = pos_vel_filter(x=(x0, vel), P=P, R=R, Q=0)
    if np.isscalar(u):
        u = [u]
    pos = 0 # true position
    for i in range(100):
        pos += vel
        xs.append(pos)

        # control input u - discussed below
        f1D.predict(u=u)
        f2D.predict()

        z = pos + randn()*sqrt(R) # measurement
        f1D.update(z)
        f2D.update(z)

        xs1.append(f1D.x[0])
        xs2.append(f2D.x[0])
    plt.figure()
    plot_1d_2d(xs, xs1, xs2)

compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=1.)
```



讨论

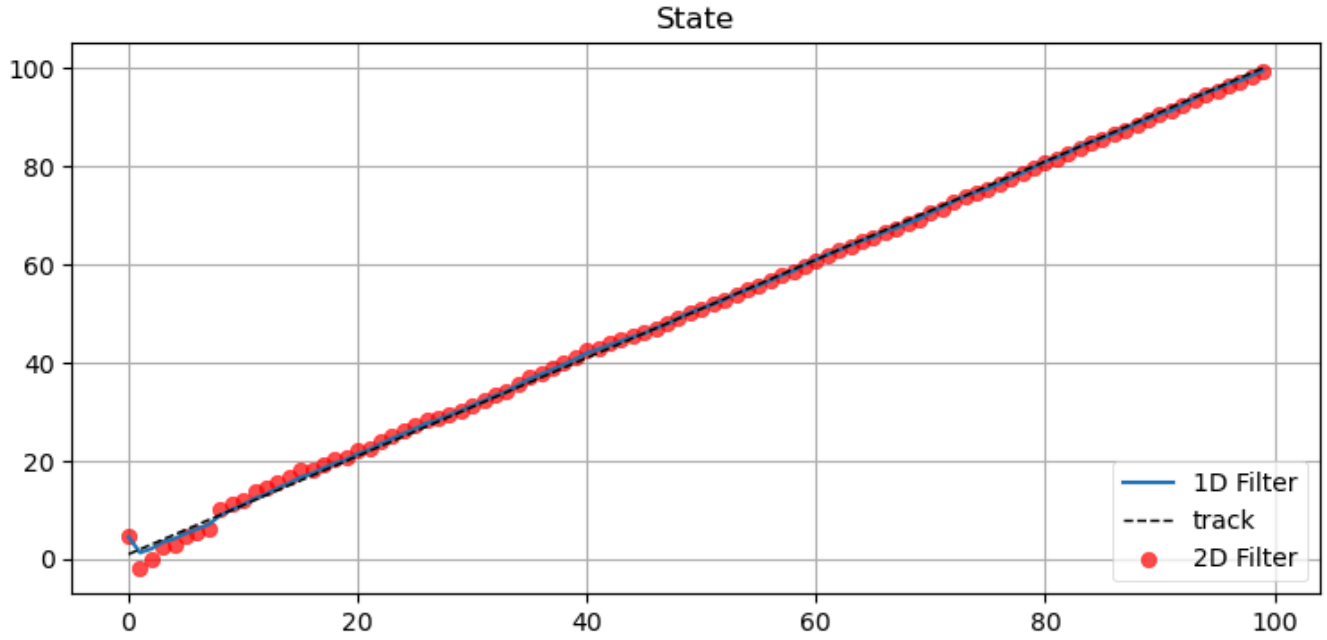
把速度纳入状态的滤波器，估计效果明显好于仅跟踪位置的滤波器。单变量滤波器无法估计速度或位置变化，因此估计轨迹会滞后于目标。

在单变量卡尔曼滤波器的相关章节中，我们在预测方程中加入了控制输入 u 。

```
 def predict(self, u=0.0):
self.x += u
self.P += self.Q
```

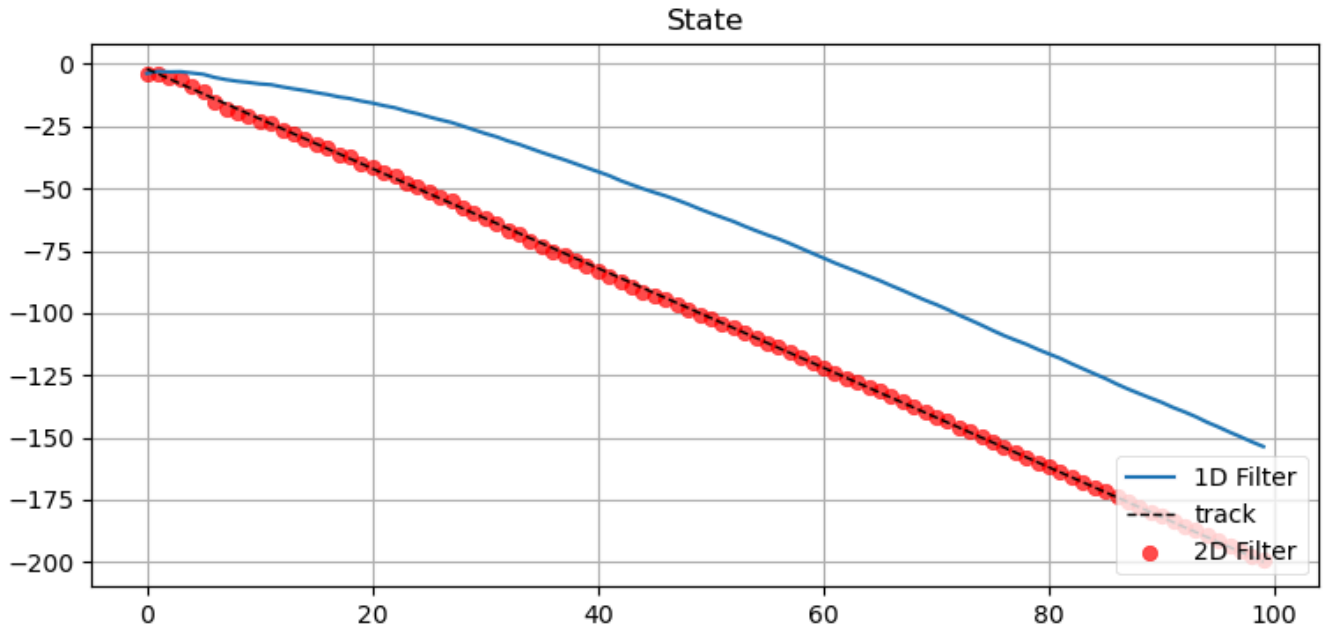
让我们试着指定一下控制输入吧：

```
compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=1., u=1.)
```



在这里，两种滤波器的性能相当，也许单变量滤波器的跟踪效果更佳。不过，让我们看看当实际速度 `vel` 与控制输入 `u` 不同时会发生什么：

```
compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=-2., u=1.)
```



如果我们正在控制某个机器人，那么使用单变量滤波器就能很好地完成跟踪任务，因为控制输入有助于滤波器做出准确的预测。但如果只是被动地跟踪机器人，那么控制输入就没什么用处了，除非我们能够事先准确预测出机器人的速度。不过，这种情况很少发生。

速度是如何计算出来的

我还没有解释该滤波器是如何计算速度的，也没有说明其中所涉及的任何隐变量。如果我们把为滤波器的各个矩阵计算出的数值代入其中，就能明白具体发生了什么。

首先，我们需要计算系统的不确定性。

$$\begin{aligned}\mathbf{S} &= \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R} \\ &= \begin{bmatrix}1 & 0\end{bmatrix} \begin{bmatrix}\sigma_x^2 & \sigma_{xv} \\ \sigma_{xv} & \sigma_v^2\end{bmatrix} \begin{bmatrix}1 \\ 0\end{bmatrix} + \begin{bmatrix}\sigma_z^2\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2 & \sigma_{xv}\end{bmatrix} \begin{bmatrix}1 \\ 0\end{bmatrix} + \begin{bmatrix}\sigma_z^2\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2 + \sigma_z^2\end{bmatrix}\end{aligned}$$

现在我们有了 $\mathbf{S}$，就可以计算出卡尔曼增益的值了：

$$\begin{aligned}\mathbf{K} &= \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1} \\ &= \begin{bmatrix}\sigma_x^2 & \sigma_{xv} \\ \sigma_{xv} & \sigma_v^2\end{bmatrix} \begin{bmatrix}1 \\ 0\end{bmatrix} \begin{bmatrix}\frac{1}{\sigma_x^2+\sigma_z^2}\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2 \\ \sigma_{xv}\end{bmatrix} \begin{bmatrix}\frac{1}{\sigma_x^2+\sigma_z^2}\end{bmatrix} \\ &= \begin{bmatrix}\sigma_x^2/(\sigma_x^2+\sigma_z^2) \\ \sigma_{xv}/(\sigma_x^2+\sigma_z^2)\end{bmatrix}\end{aligned}$$

换句话说， $\boldsymbol{x}$ 的卡尔曼增益为

$$K_x = \frac{VAR(x)}{VAR(x) + VAR(z)}$$

这一点在单变量情况下应该对你来说很熟悉吧。

速度 $\dot{\boldsymbol{x}}$ 的卡尔曼增益为

$$K_{\dot{x}} = \frac{COV(x, \dot{x})}{VAR(x) + VAR(z)}$$

这会带来什么影响呢？请记住，我们是这样来计算状态的：

$$\begin{aligned}\mathbf{x} &= \bar{\mathbf{x}} + \mathbf{K}(z - \mathbf{H}\mathbf{x}) \\ &= \bar{\mathbf{x}} + \mathbf{K}y\end{aligned}$$

在这里，残差 $\boldsymbol{y}$ 是一个标量。因此，它需要与 $\mathbf{K}$ 中的每个元素相乘。这样一来，我们就得到了……

$$\begin{bmatrix}x \\ \dot{x}\end{bmatrix} = \begin{bmatrix}\bar{x} \\ \bar{\dot{x}}\end{bmatrix} + \begin{bmatrix}K_x \\ K_{\dot{x}}\end{bmatrix} y$$

这就得到了如下方程组：

$$x = \bar{x} + yK_x$$
$$\dot{x} = \bar{\dot{x}} + yK_{\dot{x}}$$

预测位置 $\bar{x}$ 由 $x + \dot{x}\Delta t$ 计算。如果预测完全准确，忽略测量噪声时就有 $y = 0$，速度估计无需改变。反之，速度估计很差会使位置预测也很差，导致残差的绝对值很大，即 $|y| \gg 0$。此时，用 $yK_{\dot{x}}$ 修正速度估计。$K_{\dot{x}}$ 与 $COV(x, \dot{x})$ 成正比，因此速度修正量等于位置残差乘以一个与位置—速度协方差成正比的系数。在其他条件相同的情况下，协方差的绝对值越大，修正幅度越大。

为了使这个过程完整起来，$COV(x, \dot{x})$ 实际上就是 $\mathbf{P}$ 中的非对角线元素。请记住，这些数值是通过 $\mathbf{FPF}^\mathsf{T}$ 计算得出的。因此，位置和速度的协方差是在预测阶段计算出来的。速度的卡尔曼增益与这个协方差成正比，我们会根据上一个时间点上速度估计的准确性来调整速度值，调整的幅度则与这个协方差成正比。

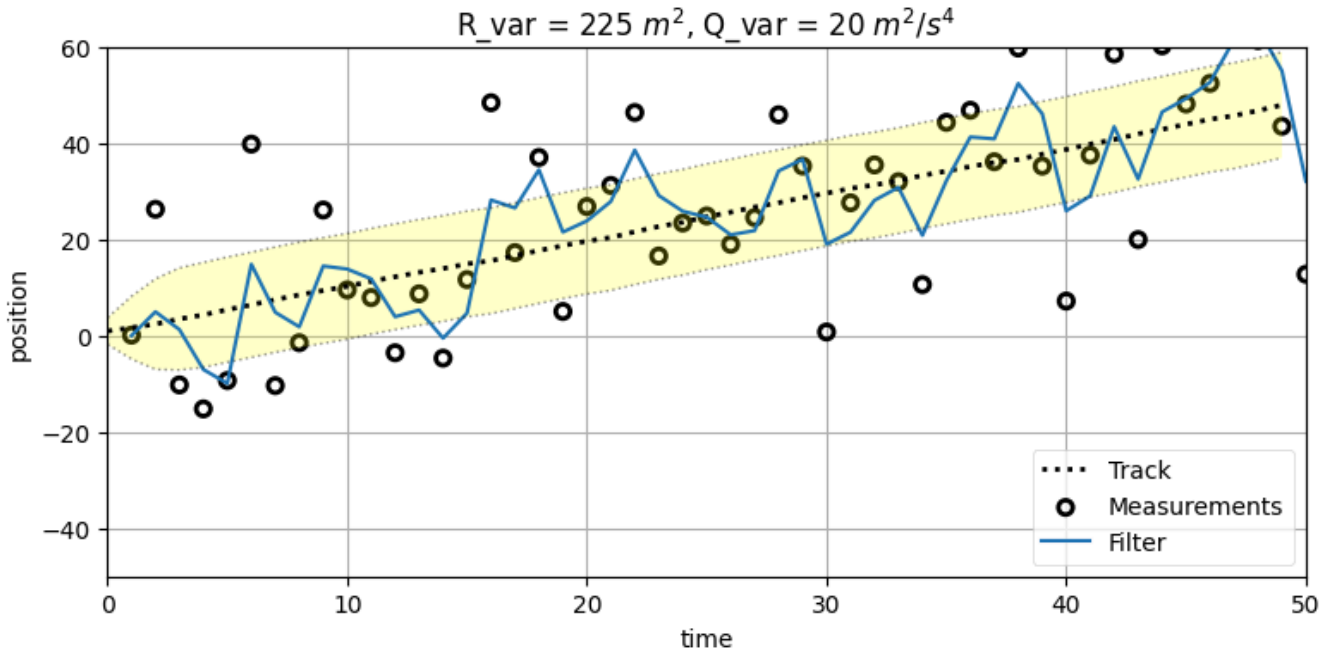
总的来说，这些线性代数方程可能让你觉得有些陌生，但实际上计算过程非常简单。本质上，这与我们在 g-h 滤波器中使用的计算方法是一样的。在本章中，由于我们需要考虑过程模型和传感器中的噪声因素，所以所使用的常数有所不同，但数学原理其实是一样的。

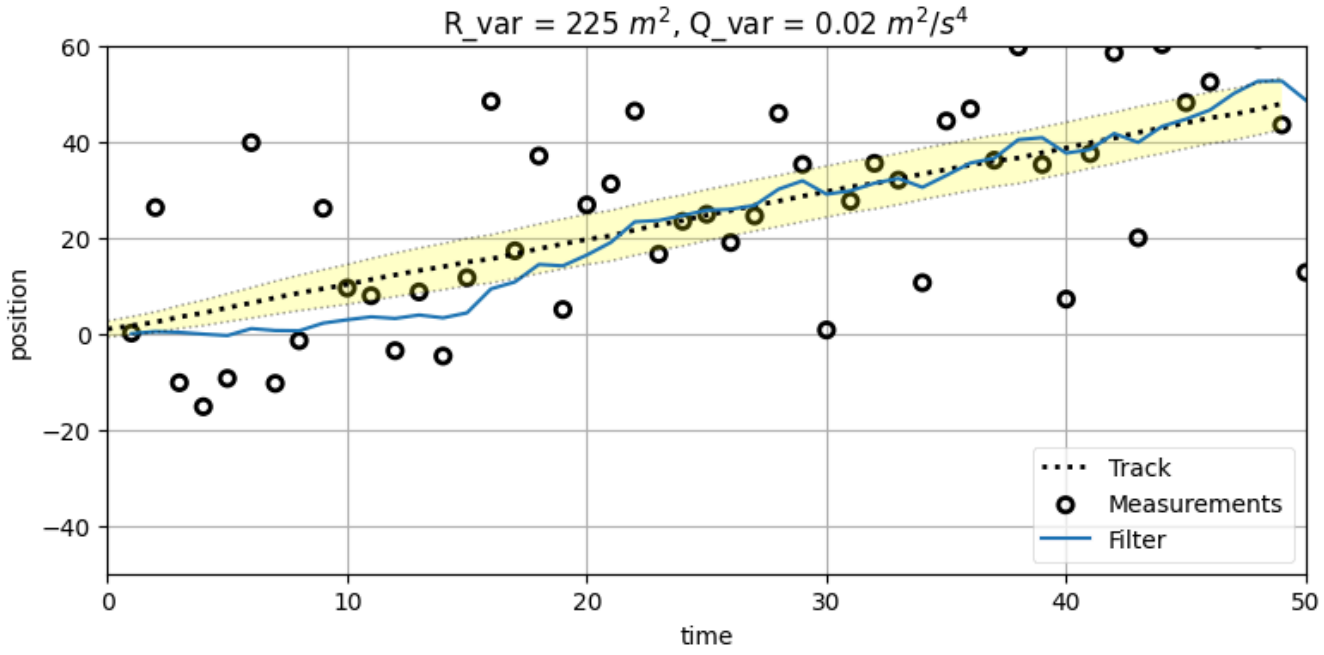
调整滤波器

让我们开始调整各种参数，看看不同调整所带来的效果。对于卡尔曼滤波器来说，这是很常规的操作。实际上，要精确地模拟传感器的性能是非常困难的，甚至是不可能的。模型不完善，那么滤波器的输出也会不准确。工程师们需要花费大量时间来调整卡尔曼滤波器，使其能够更好地适应现实世界中的传感器。现在，我们就来仔细研究这些调整所带来的效果。通过了解每项调整的效果，你就能逐渐掌握如何设计卡尔曼滤波器。设计卡尔曼滤波器既需要科学的方法，也需要艺术性的思维。我们用数学来模拟物理系统，但模型本身永远不可能做到完美。

下面考察测量噪声协方差 $\mathbf{R}$ 与过程噪声协方差 $\mathbf{Q}$ 的影响。为看清 $\mathbf{R}$ 和 $\mathbf{Q}$ 不同设置的效果，我把测量噪声方差设为 225 平方米。这个值很大，但能放大图中不同设计的差异，便于观察。第一个实验保持 $\mathbf{R}$ 不变，只改变 $\mathbf{Q}$。

```
from numpy.random import seed
seed(2)
trk, zs = compute_dog_data(z_var=225, process_var=.02, count=50)

run(track=trk, zs=zs, R=225, Q=20, P=P, plot_P=False,
    title='R_var = 225 $m^2$, Q_var = 20 $m^2/s^4$')
run(track=trk, zs=zs, R=225, Q=.02, P=P, plot_P=False,
    title='R_var = 225 $m^2$, Q_var = 0.02 $m^2/s^4$');
```





第一个图中的滤波结果应当紧跟带噪声的测量值。第二个图中，滤波结果应与测量值有较大差异，而且比第一个图更接近直线。为什么 $\mathbf{Q}$ 会产生这样的影响?

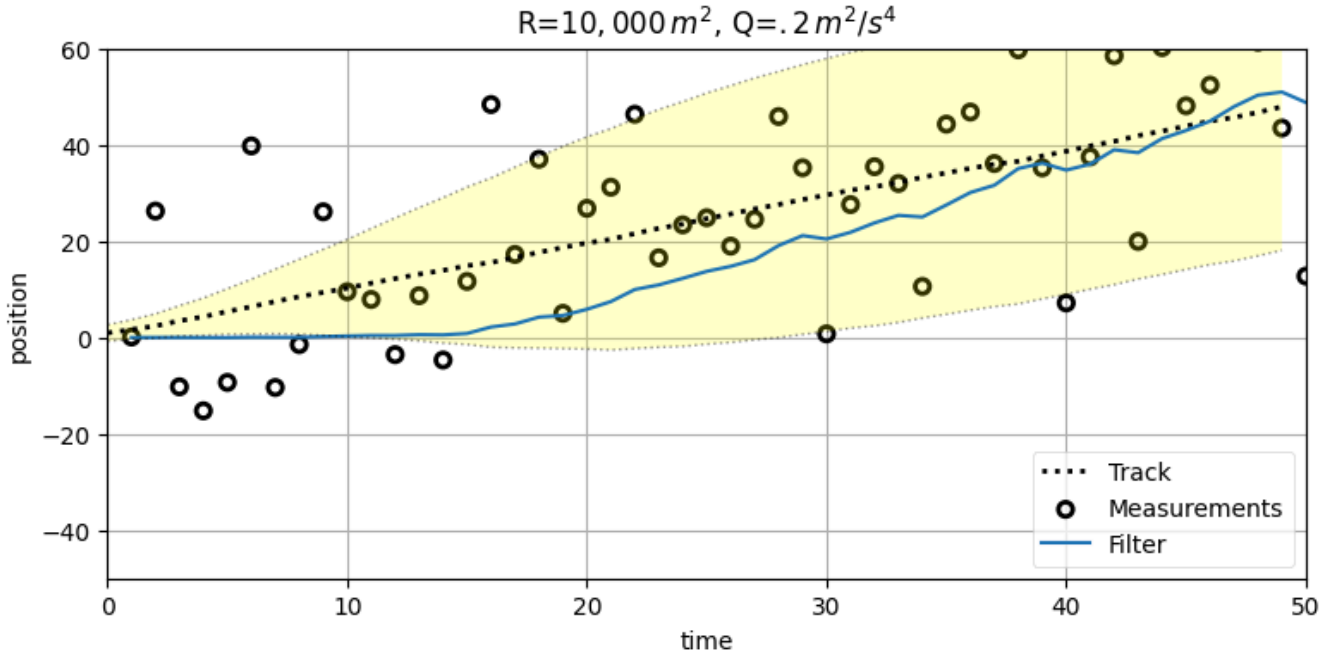
让我们先回顾一下“过程不确定性”这个概念的含义。以追踪球体的运动为例：在真空中，我们可以用数学模型来精确描述球体的运动轨迹。但是，一旦有风、空气密度变化、温度差异，或者球体表面不光滑，我们的数学模型就无法准确反映现实情况了。

第一个例子将 Q_var 设为 20，表示假设存在较强的过程扰动。换言之，$\mathbf{F}$ 未描述的外部变化较大，因此不应过度信任运动预测。滤波器仍估计速度 $\dot{\boldsymbol{x}}$，但更新时会更多依赖位置测量。Q_var 是 Q_discrete_white_noise 的扰动方差参数，不是速度估计方差；其单位应与扰动模型及时间单位一致。

第二个例子将 Q_var 设为 0.02，意味着假设过程扰动较弱，因而更信任运动预测。滤波器不会紧跟测量中的上下波动，因为这些变化与可信的运动模型不符。本例以米和秒为单位，加速度扰动方差的单位为 $\mathbf{m^2/s^4}$。

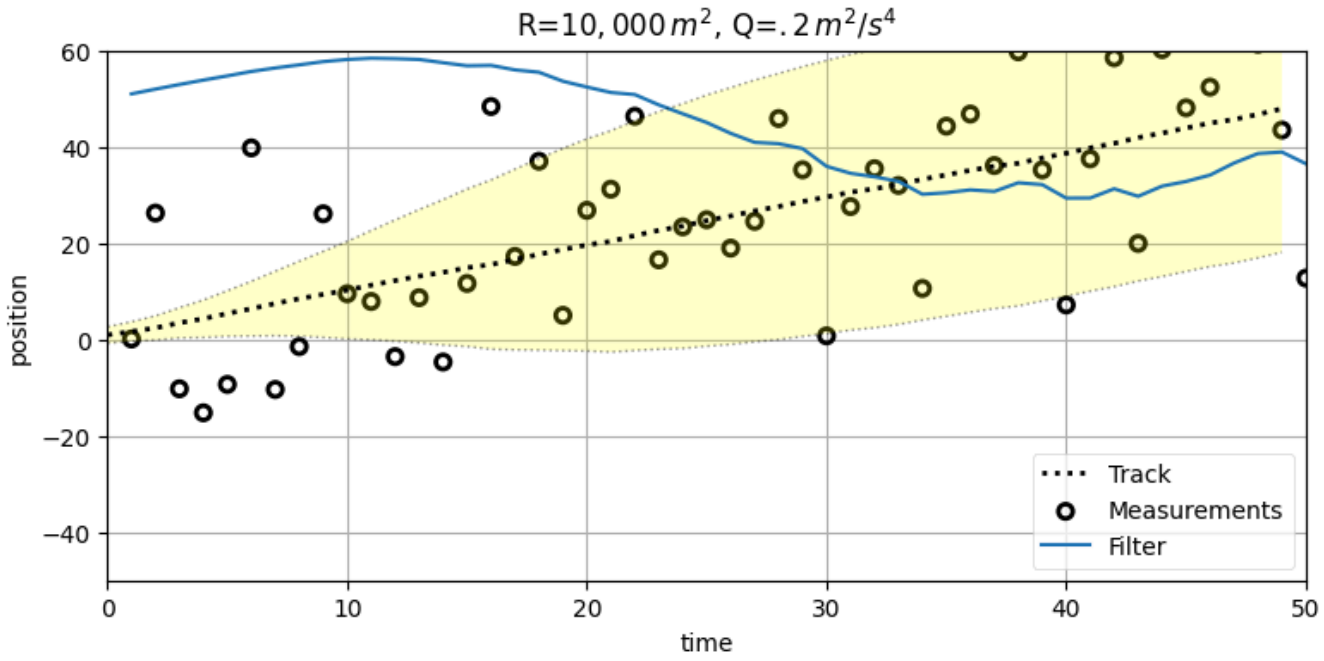
现在，让我们把 `Q_var` 的值设置为 $\mathbf{0.2}\,m^2$，同时把 `R_var` 的值提高到 $\mathbf{10,000}\,m^2$。这样做的目的是告诉滤波器：测量过程中的噪声非常大。

```
run(track=trk, zs=zs, R=10000, Q=.2, P=P, plot_P=False,
    title='R=$10,000\, m^2$, Q=$.2\, m^2/s^4$');
```



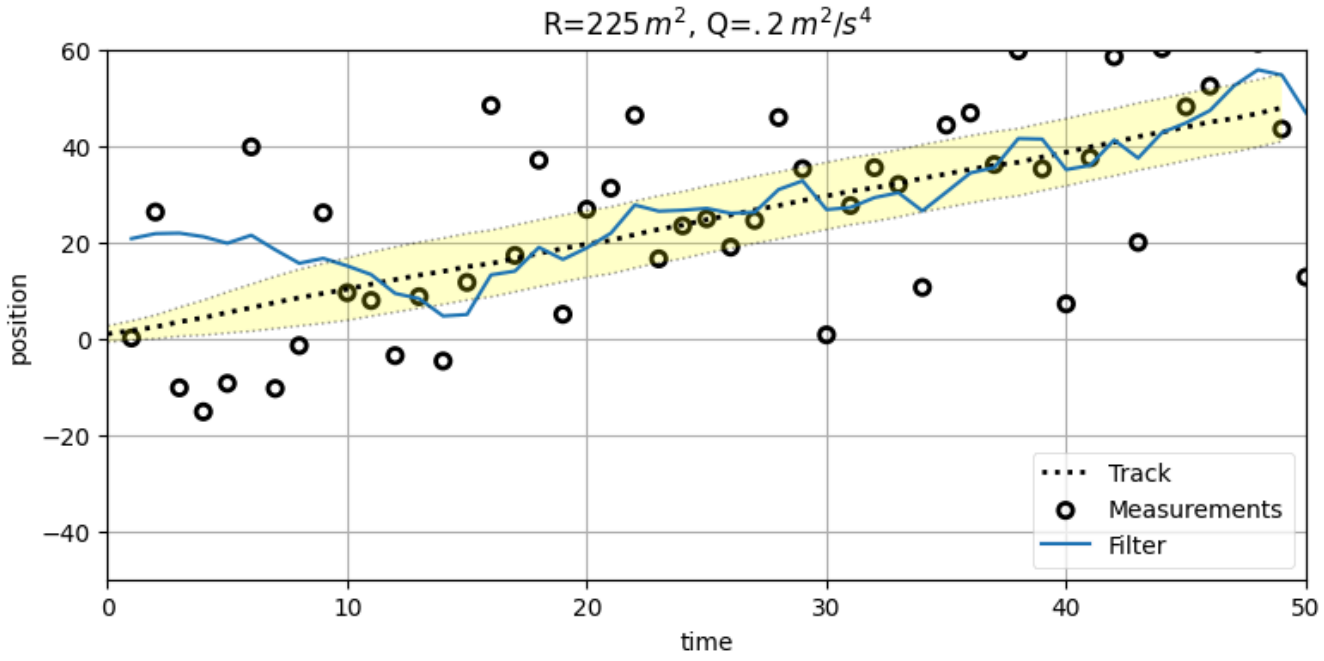
这种效应可能不太明显。我们之所以使用了效果不佳的滤波器，是因为实际的测量噪声方差为 225 m^2，而非 10,000 m^2。将滤波器的噪声方差设置得如此之高，会迫使滤波器更倾向于采用预测值而非实际测量值。这样一来，虽然得到的结果看起来很平滑、很理想，但实际上并不可靠。在上图中，这条轨迹看起来非常完美，因为它与理想路径极为接近。不过，刚开始时的“完美表现”其实值得怀疑——因为滤波器尚未收敛（$\mathbf{P}$ 的值仍然很大），所以它不可能如此精确地反映实际位置。从图中可以看出，$\mathbf{P}$ 的值仍未收敛，因为整个图表都用黄色背景来表示 $\mathbf{P}$ 的大小。让我们试着用错误的初始值来测试一下：将初始位置设为 50 米，初始速度设为 1 米/秒，看看会得到什么结果。

```
run(track=trk, zs=zs, R=10000, Q=.2, P=P, plot_P=False,
    x0=np.array([50., 1.]),
    title='R=$10,000\, m^2$, Q=$.2\, m^2/s^4$');
```



这里可以看到，滤波器无法建立正确的目标跟踪。尽管测量值相当可靠，滤波器却认为它们很差，因此每一步仍从错误的位置向前预测。如果你怀疑，错误初始位置在较小的测量噪声设定下也会造成同样的结果，那么下面将测量噪声方差恢复为正确的 225 m^2。

```
run(track=trk, zs=zs, R=225, Q=.2, P=P, plot_P=False,
    x0=np.array([20., 1.]),
    title='R=$225\, m^2$, Q=$.2\, m^2/s^4$');
```



滤波器最初经过几次迭代才建立跟踪，之后便能准确跟踪狗。这已接近最优表现：我们尚未最优地设计 $\mathbf{Q}$，但 $\mathbf{R}$ 已采用正确的噪声参数。构造 $\mathbf{Q}$ 时，一种经验做法是将加速度扰动的标准差取为 $\frac{1}{2}\Delta a$ 至 Δa，其中 Δa 表示相邻采样周期之间加速度的最大变化量。这针对分段恒定加速度模型：每个周期内加速度恒定，

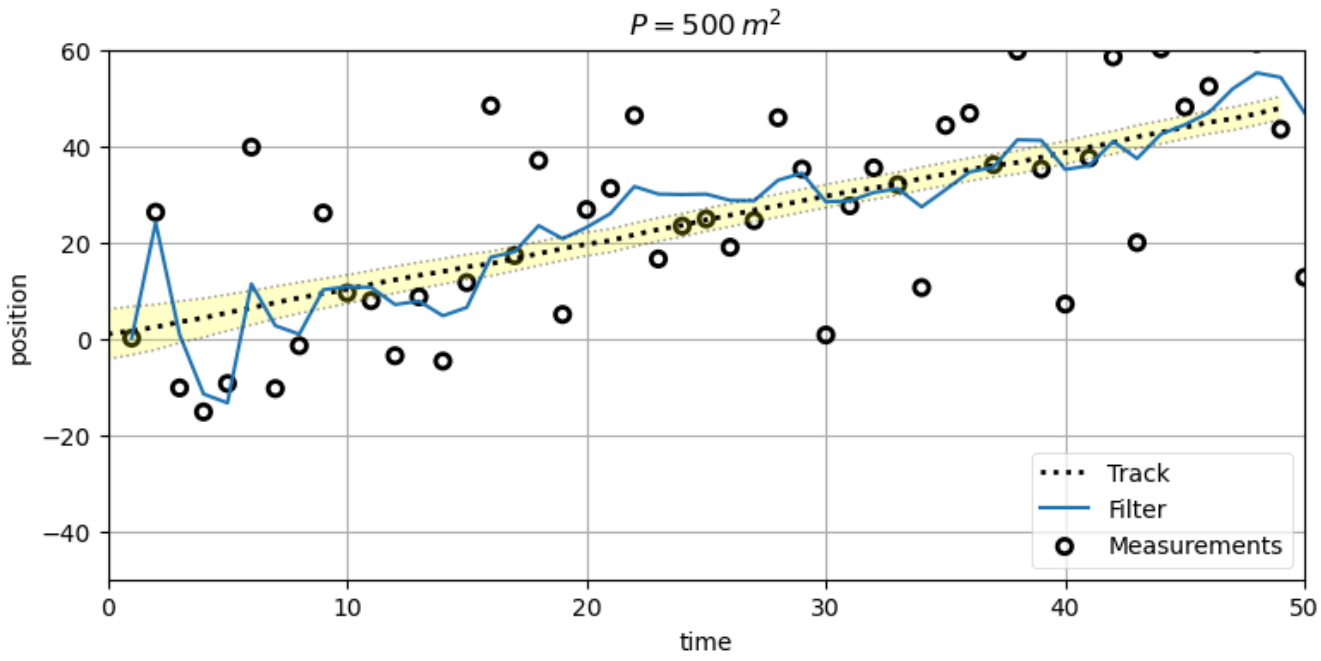
不同周期的扰动互不相关。“卡尔曼滤波的数学原理”一章将讨论几种 $\mathbf{Q}$ 设计方法。使用 Q_discrete_white_noise 时，var 参数应传入扰动方差，即标准差的平方，并依据采样间隔构造矩阵。

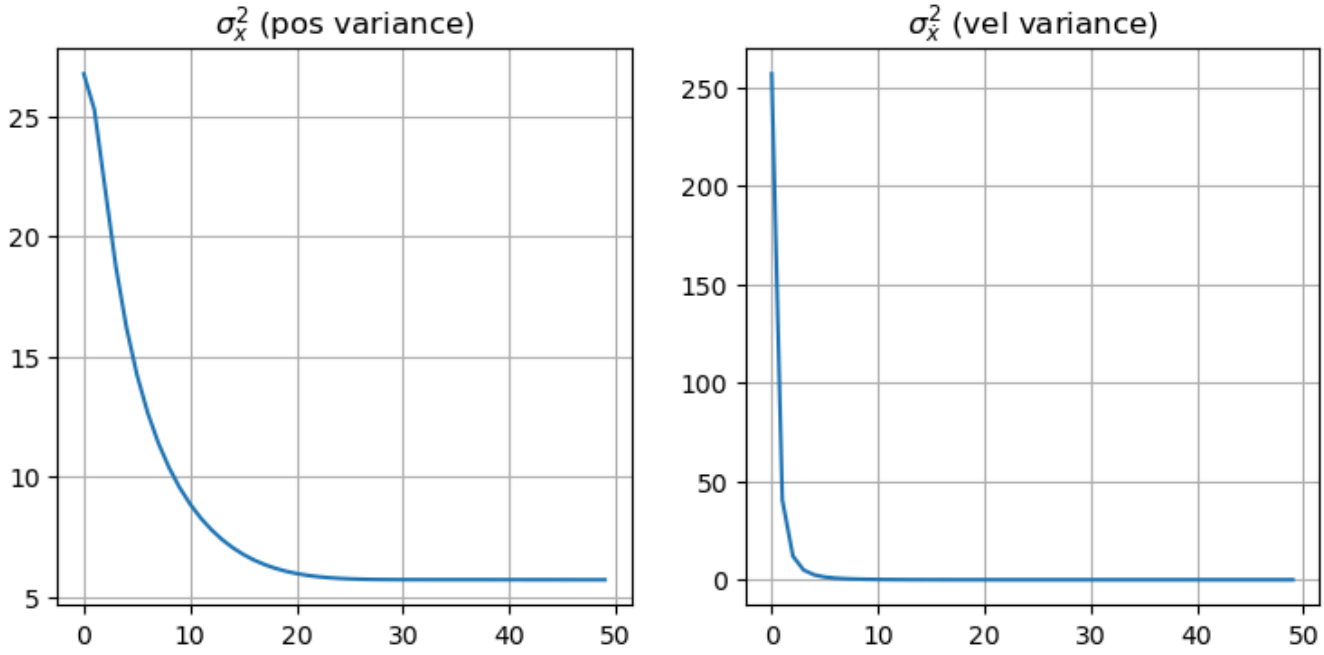
在一定程度上，改变 $\mathbf{R}$ 或 $\mathbf{Q}$ 都可能得到相似的输出曲线。但不要仅凭曲线是否好看来反复调参。应始终考虑这些设置的物理含义，根据对系统的了解选择 $\mathbf{R}$ 和／或 $\mathbf{Q}$，再用充分的仿真实验及真实数据试验验证。

深入分析协方差矩阵

我们先来重新回顾一下如何绘制曲线图。我把数据和噪声都硬编码到了 zs_var_275 中，这样就不受随机数生成器的影响了。因为随机数生成器可能会产生无法准确反映我想要表达的内容的数据。我会从 P=500 开始。

```
import kf_book.mkf_internal as mkf_internal

var = 27.5
data = mkf_internal.zs_var_275()
run(track=trk, zs=zs, R=var, Q=.02, P=500., plot_P=True,
    title='$P=500\, m^2$');
```





初始输出出现很大的波动。这里设 $\mathbf{P} = 500\,\mathbf{I_2}$，即两个对角元均为 500 的 2×2 对角矩阵。左上角对应位置方差 $\boldsymbol{\sigma_x^2}$，因此位置标准差为 $\sqrt{500}$，约 22.36 m。在一维高斯假设下，均值两侧 3σ 的区间包含约 99.7% 的概率质量，所以 $\boldsymbol{\sigma_x^2} = 500$ 对应约正负 67 m 的三倍标准差范围。这是较大的初始不确定性；测量出现明显变化时，滤波器就会较多采纳测量。随后 $\mathbf{P}$ 逐渐收敛。该区间是概率范围，并不是误差的绝对上限。

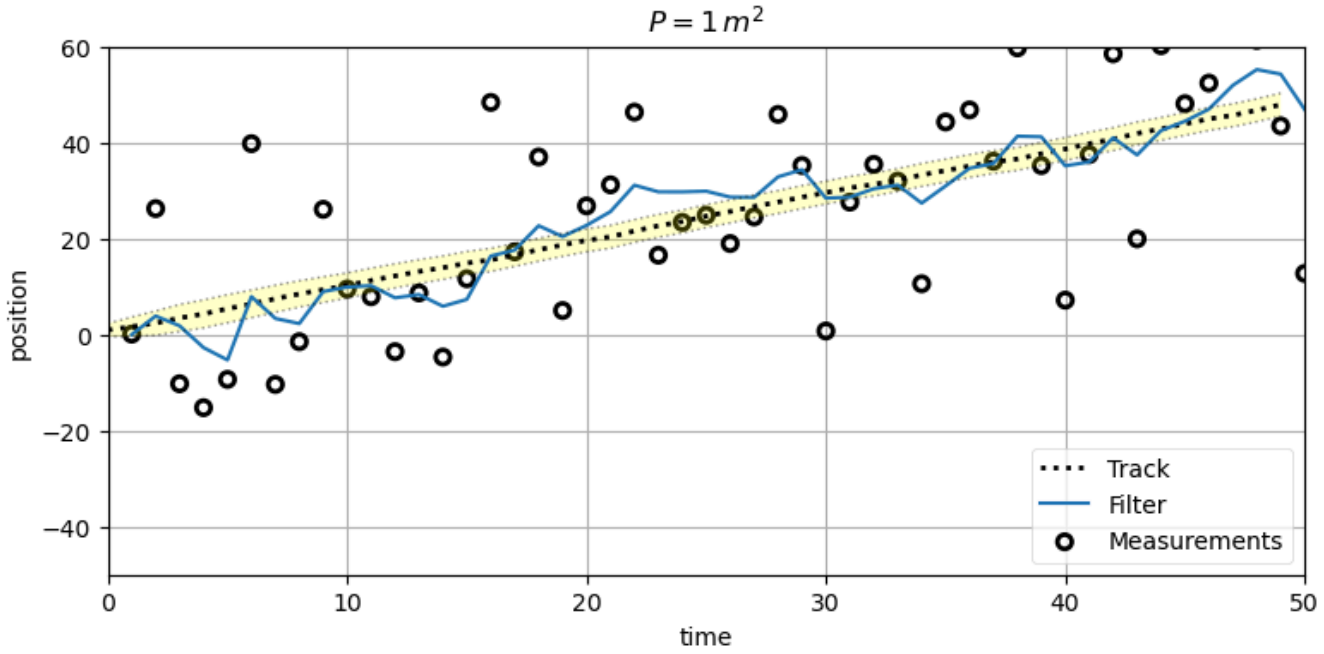
让我们来看看其中的数学原理。卡尔曼增益的公式为：

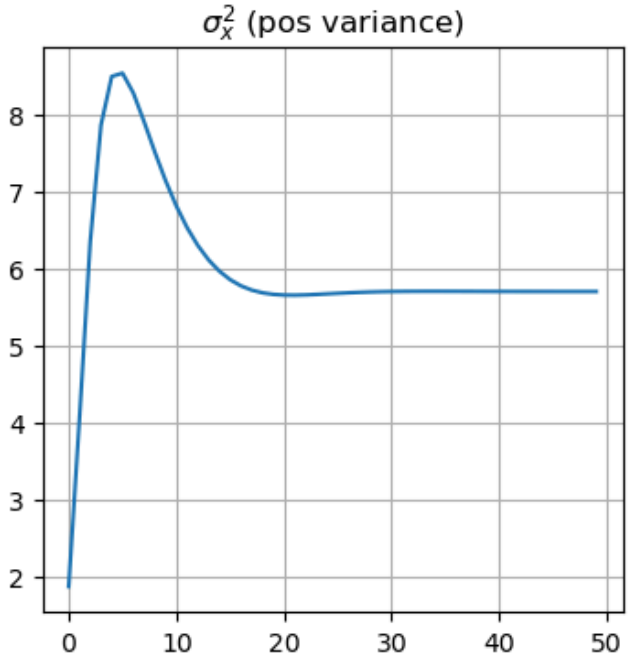
$$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1} \approx \frac{\bar{\mathbf{P}}\mathbf{H}^\mathsf{T}}{\mathbf{S}} \approx \frac{\mathbf{uncertainty}_{\mathbf{prediction}}}{\mathbf{uncertainty}_{\mathbf{measurement}}}\mathbf{H}^\mathsf{T}$$

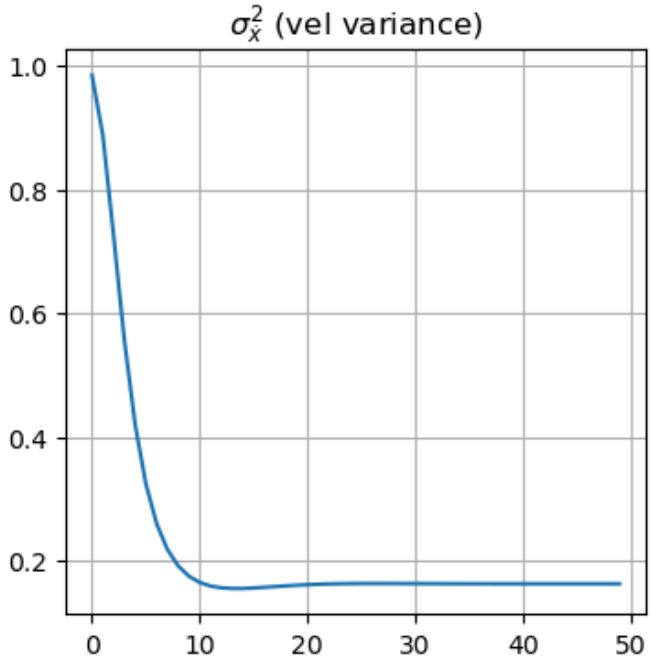
卡尔曼增益体现了预测与测量不确定性的相对作用。本例中预测不确定性很大，因此 $\mathbf{K}$ 在位置更新方向上较大；若是直接观测的标量问题，其值会接近 1。$\mathbf{K}$ 与残差 $\mathbf{y} = \mathbf{z} - \mathbf{H}\bar{\mathbf{x}}$ 相乘，因此较大的 $\mathbf{K}$ 会使更新更偏向测量。当 $\mathbf{P}$ 相对于 $\mathbf{R}$ 较大时，估计会更多依赖测量。这反映模型赋予的相对权重，并不保证实际测量一定更准确。

现在，让我们来看看 $\mathbf{P} = 1.0\,\mathbf{I_2}$ 的初始值较小时会产生怎样的效果。

```
run(track=trk, zs=zs, R=var, Q=.02, P=1., plot_P=True,
    title='$P=1\, m^2$');
```





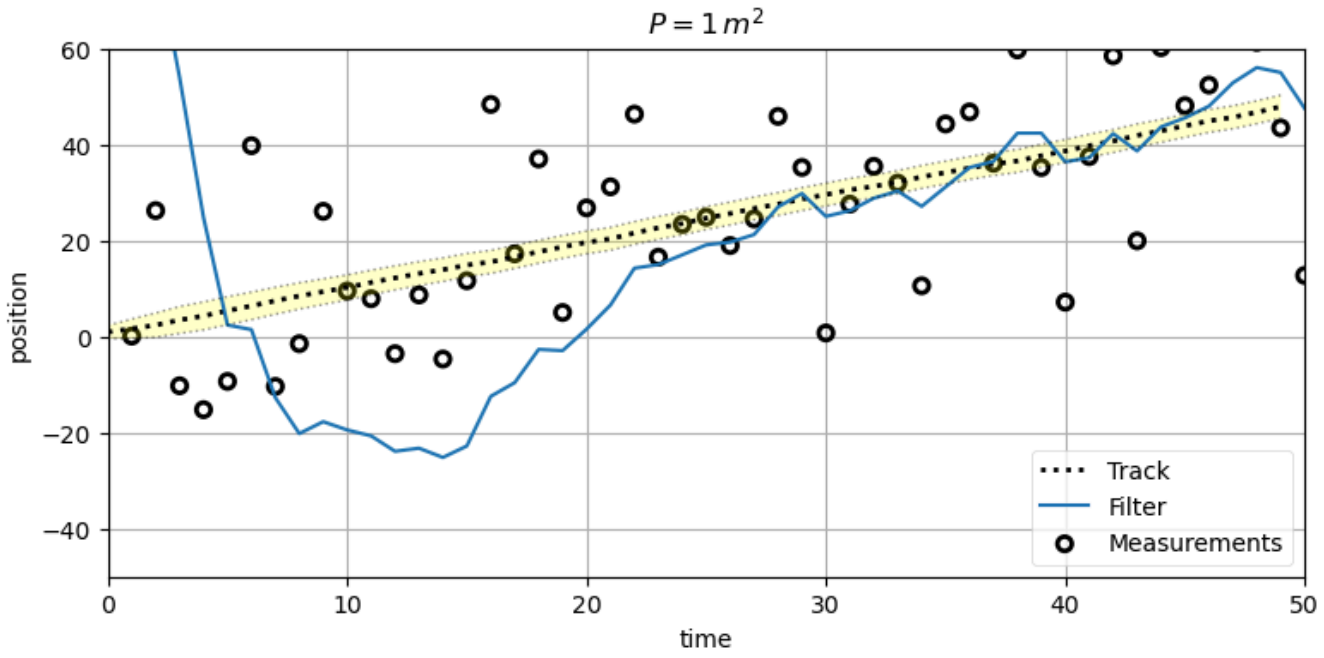


乍看之下，结果不错：曲线没有前一幅图中的尖峰，滤波器很快就开始跟踪测量值，似乎不需要过渡过程。然而，观察 P 的曲线，会发现位置方差起初出现尖峰，随后也没有明显收敛。设计不当会延长收敛时间，并产生次优结果。

因此，尽管滤波器的跟踪效果与实际信号非常接近，但我们仍不能断定，使用较小的 $\mathbf{P}$ 值就是解决问题的“秘诀”。没错，这样做确实可以避免卡尔曼滤波器需要较长时间才能准确跟踪信号。但如果我们对初始测量值本身就不确定，那么这种做法反而会导致滤波器产生糟糕的结果。如果我们要跟踪某个移动中的物体，那么在开始跟踪之前，我们很可能无法准确知道它的位置。而如果我们只是对温度计的测量结果进行滤波处理，那么第一次测量得到的数值与第 1000 次测量得到的数值具有同样的可靠性。为了让卡尔曼滤波器能够正常工作，必须将 $\mathbf{P}$ 的值设定为能够真实反映我们对数据的了解的数值。

下面看看错误初始估计与过小的 $\mathbf{P}$ 同时出现时会怎样。令初始位置估计 x = 100 m，而狗实际从 0 m 出发；同时设 P = 1 m^2。显然，$\mathbf{P}$ 的设置不合理：初始估计已偏离 100 m，却告诉滤波器 3σ 的误差范围只有 3 m。

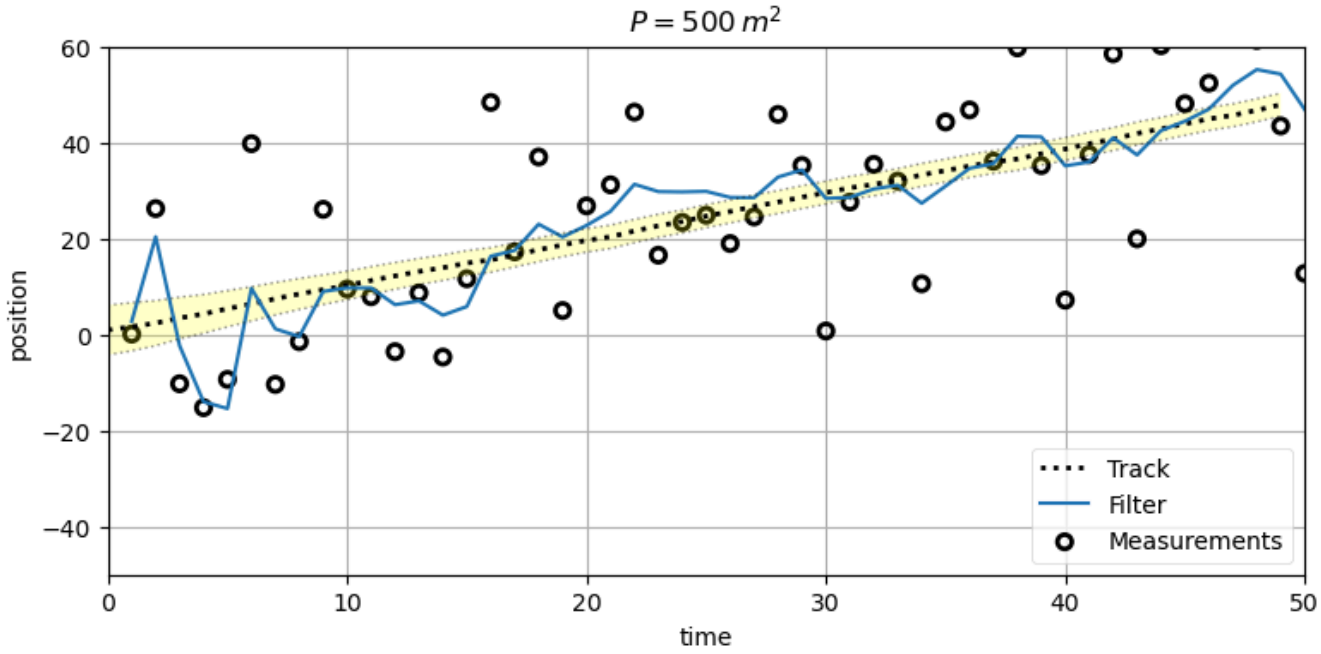
```
x = np.array([100., 0.])
run(track=trk, zs=zs, R=var, Q=.02, P=1., x0=x,
    plot_P=False, title='$P=1\, m^2$');
```



我们可以看到，最初的估计结果相当糟糕，而且卡尔曼滤波器需要很长时间才能开始收敛到正确的信号值。这是因为我们告诉卡尔曼滤波器，我们非常相信 100 米的初始估计值，但实际上这种判断是错误的。

现在，让我们给 P 一个更合理的数值，然后看看效果有何不同。

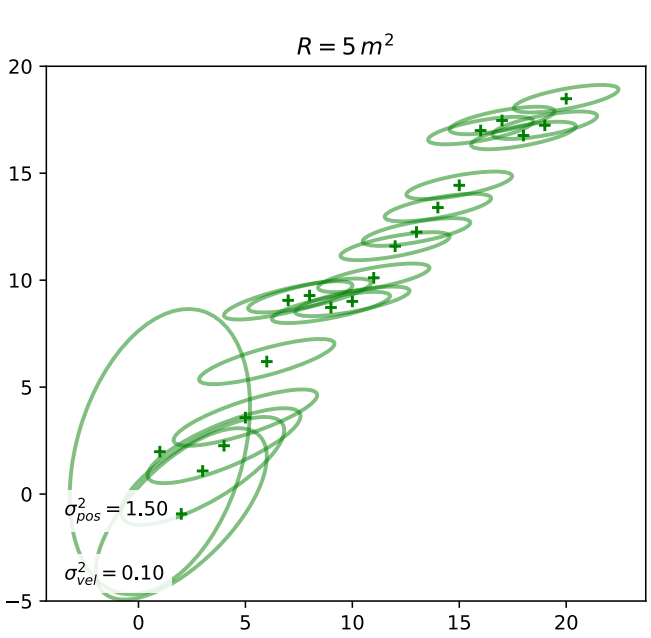
```
x = np.array([100., 0.])
run(track=trk, zs=zs, R=var, Q=.02, P=500., x0=x,
   plot_P=False, title='$P=500\, m^2$');
```

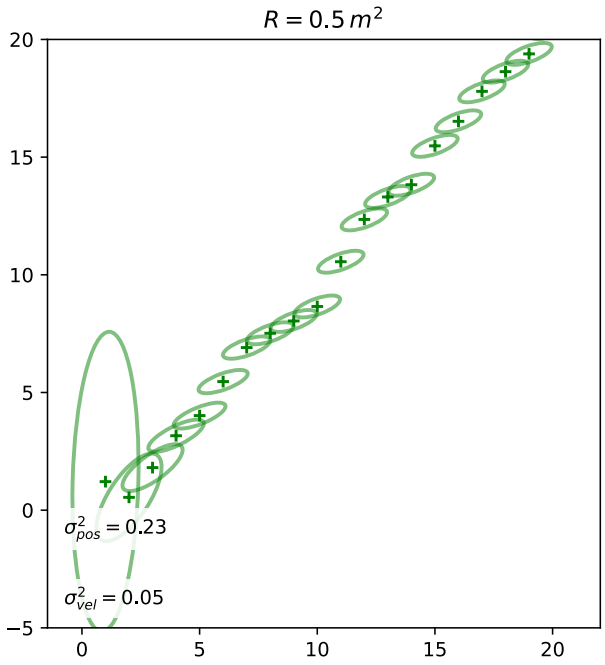


在这种情况下，卡尔曼滤波器对初始状态的估计相当不确定，因此它能够更快地收敛到正确的信号值。只需 5 到 6 个周期后，该滤波器就能输出较为准确的数值。根据我们目前所发展的理论，这已经算是最好的结果了。不过，这种情形其实有点人为制造出来的；如果我们不知道在开始追踪时物体位于何处，就不应该将滤波器的初始值设定为某个任意数值，比如 0 米或 100 米。我将在下文的“滤波器初始化”部分详细阐述这一点。

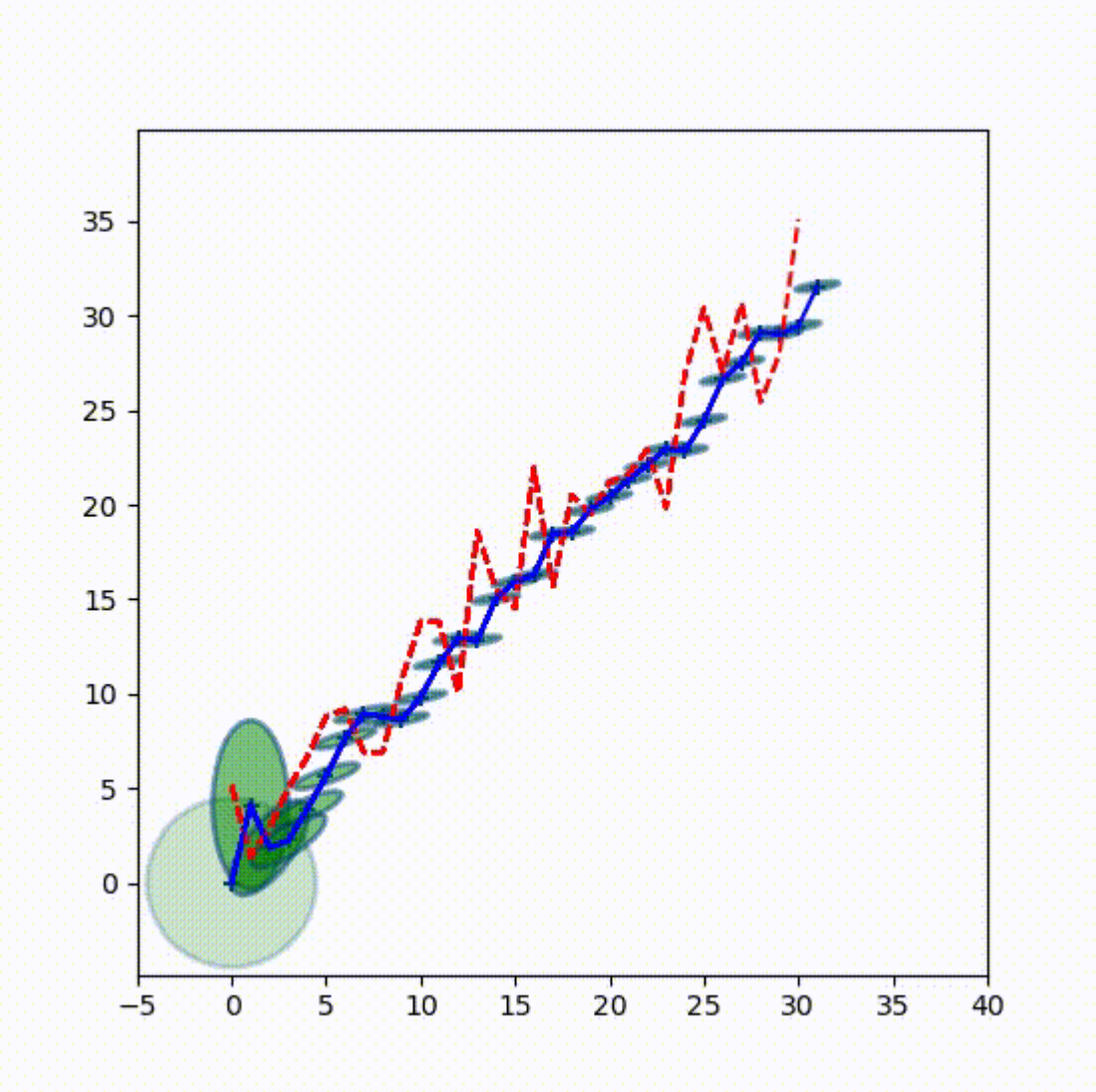
让我们再为我们的“狗”应用一次卡尔曼滤波算法。这次，把协方差椭圆与狗的位置一起绘制在同一个图表上吧。

```
from kf_book.mkf_internal import plot_track_ellipses

def plot_covariances(count, R, Q=0, P=20., title=''):
    track, zs = compute_dog_data(R, Q, count)
    f = pos_vel_filter(x=(0., 0.), R=R, Q=Q, P=P)

    xs, cov = [], []
    for z in zs:
        f.predict()
        f.update(z)

        xs.append(f.x[0])
        cov.append(f.P)
    plot_track_ellipses(count, zs, xs, cov, title)

plt.figure(figsize=(10,6))
plt.subplot(121)
plot_covariances(R=5, Q=.02, count=20, title='$R = 5\, m^2$')
plt.subplot(122)
plot_covariances(R=.5, Q=.02, count=20, title='$R = 0.5\, m^2$')
```





如果你是在 Jupyter Notebook 或网页上查看此内容，那么这里有一个展示该滤波器如何处理数据的动画。我已经调整了滤波器的参数，这样就能清楚地看到随着滤波器不断运行， $\mathbf{P}$ 的值是如何变化的。



这些图表的显示效果有点混乱，但你应该能够理解其中所反映的情况。在两个图表中，我们都绘制了每个点的协方差矩阵。首先，我们看到的是协方差 $\mathbf{P} = \begin{pmatrix}20 & 0\\ 0 & 20\end{pmatrix}$，这表明我们对初始状态的估计存在很大的不确定性。在接收到第一组测量数据后，卡尔曼滤波器会更新这一估计值，因此方差也就不再那么大了。在上面的图表中，最左边的那个椭圆应该稍微有些变形。随着滤波器不断处理更多的测量数据，协方差椭圆的形状会逐渐稳定下来，最终变成一个细长的椭圆，且其方向与物体的运动方向一致。

图中横轴是位置、纵轴是速度，因此椭圆在两个方向的跨度分别反映这两个分量的不确定性。纵向更高表示速度分量在所选单位下的数值标准差更大；横向更宽则相反。倾斜反映非零互协方差。位置与速度量纲不同，椭圆外观还取决于单位及坐标轴显示比例，不能仅凭长宽比较二者的物理不确定性。

第一个图中，R=5 m^2 时，最终椭圆的宽度大于高度。为便于比较，图中标出了最后一个椭圆对应的位置方差和速度方差。

相比之下，第二个图表中，以 R=0.5 m^2 表示的椭圆，其高度大于宽度。第二个图表中的所有椭圆都比第一个图表中的椭圆小得多。这是合乎逻辑的，因为较小的 $\mathbf{R}$ 意味着测量中的噪声较小。噪声越小，预测就越准确，因此我们对所处位置的判断也就越有信心。

问题：请解释椭圆的种种差异。

为什么 $\mathbf{R} = 5m^2$ 所对应的椭圆比 $\mathbf{R} = 0.5m^2$ 所对应的椭圆更接近水平方向呢？提示：请从这些椭圆在物理意义上的含义来思考这个问题，而不仅仅是从数学角度来分析。如果你不确定答案，可以试着将 $\mathbf{R}$ 替换为极大或极小的数值，比如 $100\,m^2$ 和 $0.1\,m^2$，观察由此产生的变化，再思考其中的含义。

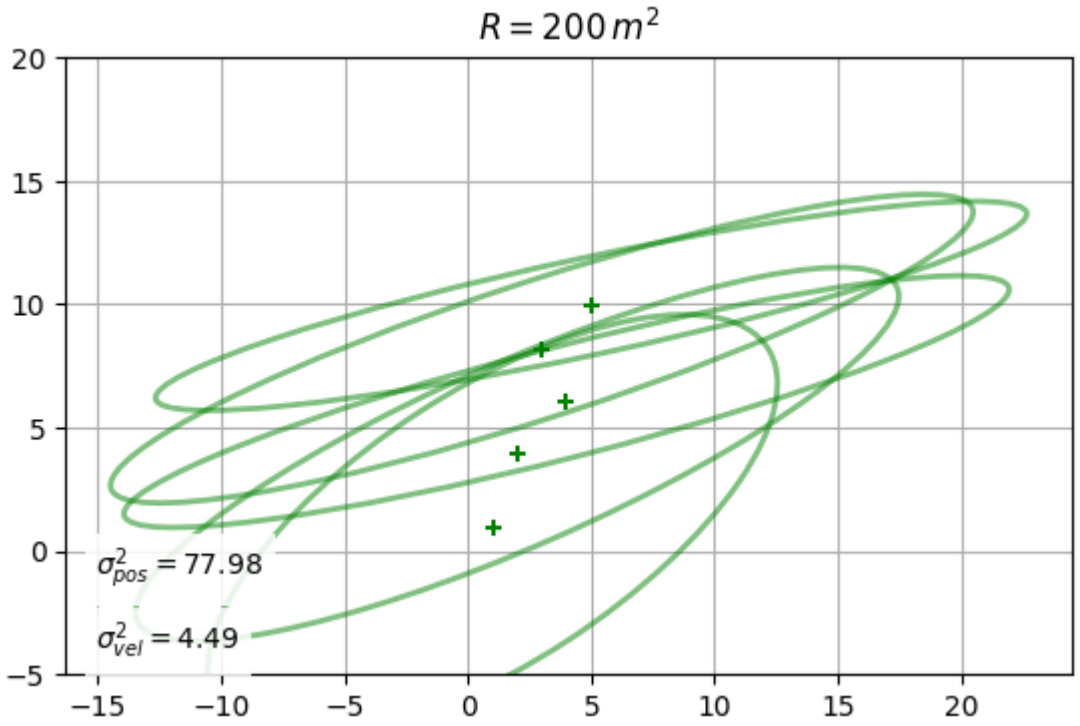
解答

横轴表示位置，纵轴表示速度。椭圆主轴与坐标轴平行时，互协方差为零；主轴倾斜则反映非零互协方差，但倾角本身并不是相关系数。这里描述的是滤波估计误差，而非物理状态固有的关系。当 $\mathbf{R}$ 很大时，$\mathbf{K}$ 倾向于更信任预测；位置预测又依赖速度，因此 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 的估计误差会产生相关性。反之，$\mathbf{R}$ 较小时，测量较可信，$\mathbf{K}$ 会更多采纳位置测量，本例中误差相关性也相应改变。

这是必须理解的关键：卡尔曼滤波器是现实系统的数学模型。协方差矩阵所反映的是估计误差的线性相关性，而不等于物理系统中各状态量之间固有的相关性。很小的估计误差相关性，并不说明物理系统中的位置与速度没有关系。测量和预测在更新中的相对作用，会影响这种误差相关性。

下面将测量噪声协方差设为很大的 $\mathbf{R} = 200\,m^2$，进一步说明这一点。看图之前，先想一想结果会是什么样。

```
plot_covariances(R=200., Q=.2, count=5, title='$R = 200\, m^2$')
```



希望得到的结果符合你的预期。该椭圆很快变得非常宽，而高度则很低。出现这种情况的原因是：卡尔曼滤波器在生成滤波结果时，主要依据的是预测值而非实际测量值。我们还可以看出，该滤波器在获取轨迹信息方面反应较慢。卡尔曼滤波器认为测量值中包含大量噪声，因此它需要很长时间才能更新对 $\dot{\boldsymbol{x}}$ 的估计值。

请反复观察，直到理解怎样解释协方差矩阵 $\mathbf{P}$。面对 $\mathbf{9{\times}9}$ 矩阵时，81 个数可能令人无从下手，但可以拆开来看：对角元是各状态变量的方差，非对角元是相应两个变量的标准差之积，再乘以相关系数 $\boldsymbol{p}$。高维 $\mathbf{9{\times}9}$ 协方差难以直接画成几何图形，因此先在简单的二维情形中培养直觉。

> 绘制协方差椭圆时，应使用 ax.set_aspect('equal') 或 plt.axis('equal')，使两条坐标轴上单位长度的显示比例相同；前者还允许设置 xlim 与 ylim。若两轴比例不同，椭圆的形状便会失真。例如，实际横向更宽的椭圆，图中却可能显得纵向更高。

滤波器初始化

有多种初始化滤波器的方法。在大多数情况下，以下方法效果不错。按照这种方法，你需要等到获得第一个测量值 $\mathbf{z}_0$ 之后，才开始对滤波器进行初始化。根据这个测量值，你可以利用 $\mathbf{x}_0 = \mathbf{z}_0$ 来计算 $\mathbf{x}$ 的初始值。如果 $\mathbf{z}$ 的尺寸、类型和单位与 $\mathbf{x}$ 不一致——这种情况很常见——我们可以按照以下方式来使用我们的测量函数。

由测量模型可知：

$$\mathbf{z} = \mathbf{H}\mathbf{x}$$

因此，

$$\mathbf{H}^{-1}\mathbf{H}\mathbf{x} = \mathbf{H}^{-1}\mathbf{z}$$
$$\mathbf{x} = \mathbf{H}^{-1}\mathbf{z}$$

普通矩阵求逆要求矩阵为方阵，但 $\mathbf{H}$ 通常不是方阵。可以用 SciPy 的 scipy.linalg.pinv 计算 Moore–Penrose 伪逆，代码如下。它给出满足最小二乘与最小范数准则的初始化，但不能恢复测量中未包含的隐状态信息；这些分量仍需额外先验或后续观测。

```
from scipy.linalg import pinv

H = np.array([[1, 0.]])
z0 = 3.2
x = np.dot(pinv(H), z0)
print(x)
```

```
[[3.2]
 [0. ]]
```

领域知识可能使你采用不同的计算方法，上述方法只是其中一种。例如，若状态含有速度，可以取前两次位置测量之差，再除以两次测量之间的时间间隔，作为初始速度估计。

还需要确定初始协方差 $\mathbf{P}$。对直接由测量初始化的状态分量，可以参考测量噪声协方差 $\mathbf{R}$；对尚未直接观测的分量，则依据领域知识给出合理且足够宽的先验方差。例如，本章以位置与速度为状态，而测量只有位置时，可以将 $\mathbf{P}$ 初始化为：

$$\mathbf{P} = \begin{bmatrix} \mathbf{R}_0 & 0 \\ 0 & vel^2_{max} \end{bmatrix}$$

$\mathbf{P}$ 的对角元是各状态分量的方差，必须赋予合理值。位置方差可参考 $\mathbf{R}_0$；速度未知时，最大可能速度的平方可以作为一种较宽的先验方差尺度。平方体现了方差 σ^2 的量纲，但这一具体数值仍是建模选择，不是普遍公式。

你确实需要充分了解自己所工作的领域，并根据最可靠的信息来初始化滤波器。例如，假设我们试图追踪赛马比赛中的马匹位置。最初的测量结果可能相当不准确，导致得到的马匹位置与实际起点相去甚远。我们知道，马匹必须从起点开始奔跑；如果以最初的测量结果来初始化滤波器，那么得到的结果肯定不会理想。在这种情况下，我们应该始终以马匹的起点位置来初始化卡尔曼滤波器。

批量处理

卡尔曼滤波器是递推算法：新测量到来时，立即形成新的估计。对于已经采集完成的一组数据，可以使用批量接口。KalmanFilter.batch_filter() 内部仍依次遍历测量，执行预测与更新，再将状态和协方差保存为数组。这样既简化代码，也便于汇总输出。我把它放在本章末尾，是希望你先熟悉基本的预测—更新循环。

首先，把各项测量数据收集到一个数组或列表中。这些数据可能保存在 CSV 文件里。

```
zs = read_altitude_from_csv('altitude_data.csv')
```

也可以用代码生成这些数据，例如使用下面的列表推导式：

```
zs = [some_func(i) for i in range(1000)]
```

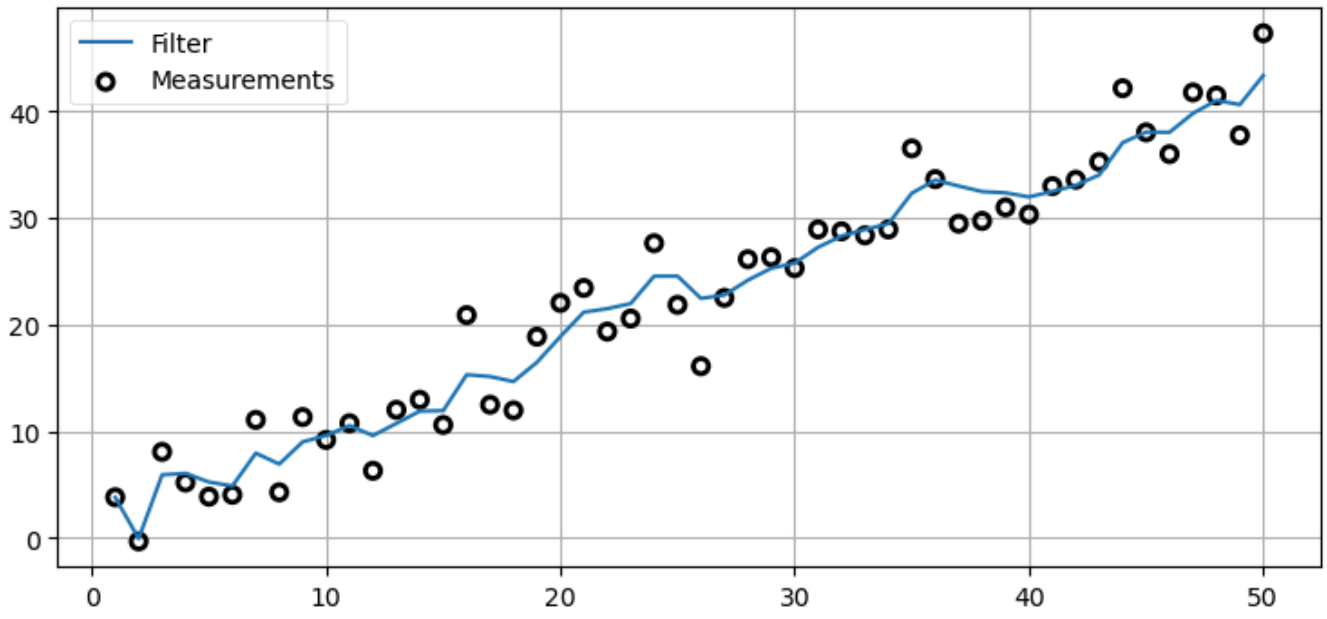
然后调用 batch_filter() 方法。

```
Xs, Ps, Xs_prior, Ps_prior = kfilter.batch_filter(zs)
```

该函数接收测量值列表，完成滤波，并返回四个 NumPy 数组：状态估计 Xs、协方差矩阵 Ps，以及相应的先验状态均值 Xs_prior 和先验协方差 Ps_prior。

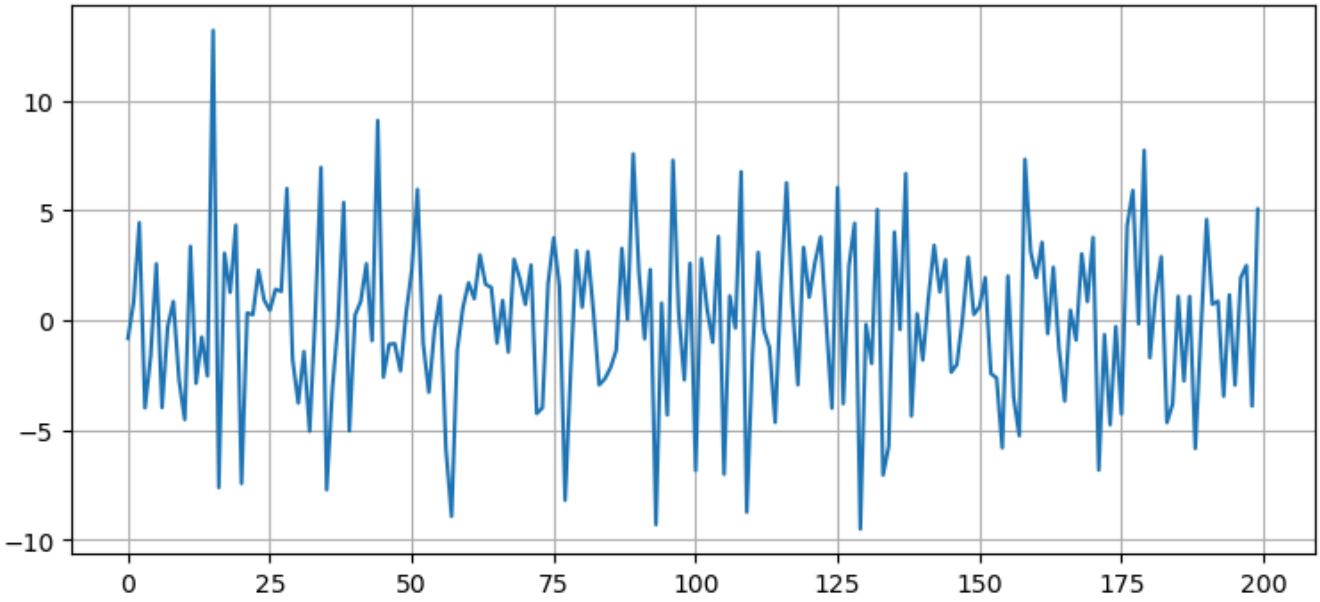
以下是一个完整的示例。

```
count = 50
track, zs = compute_dog_data(10, .2, count)
P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
xs, _, _, _ = f.batch_filter(zs)

book_plots.plot_measurements(range(1, count + 1), zs)
book_plots.plot_filter(range(1, count + 1), xs[:, 0])
plt.legend(loc='best');
```



批量滤波可以接收可选的 filterpy.common.Saver 对象，保存滤波器的各个属性，便于检查状态和协方差之外的量。这里绘制测量残差，观察它是否大致表现为以零为中心的噪声。这是检查模型的一种直观方法。若残差持续偏离零或具有明显结构，说明模型、噪声假设或参数可能不合适，需要进一步诊断；仅凭图形不能直接断定过程是否高斯。后续章节会详细讨论，这里先演示 Saver 的用法。

```
track, zs = compute_dog_data(10, .2, 200)
P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
s = Saver(f)
xs, _, _, _ = f.batch_filter(zs, saver=s)
s.to_array()
plt.plot(s.y);
```



对结果进行平滑处理

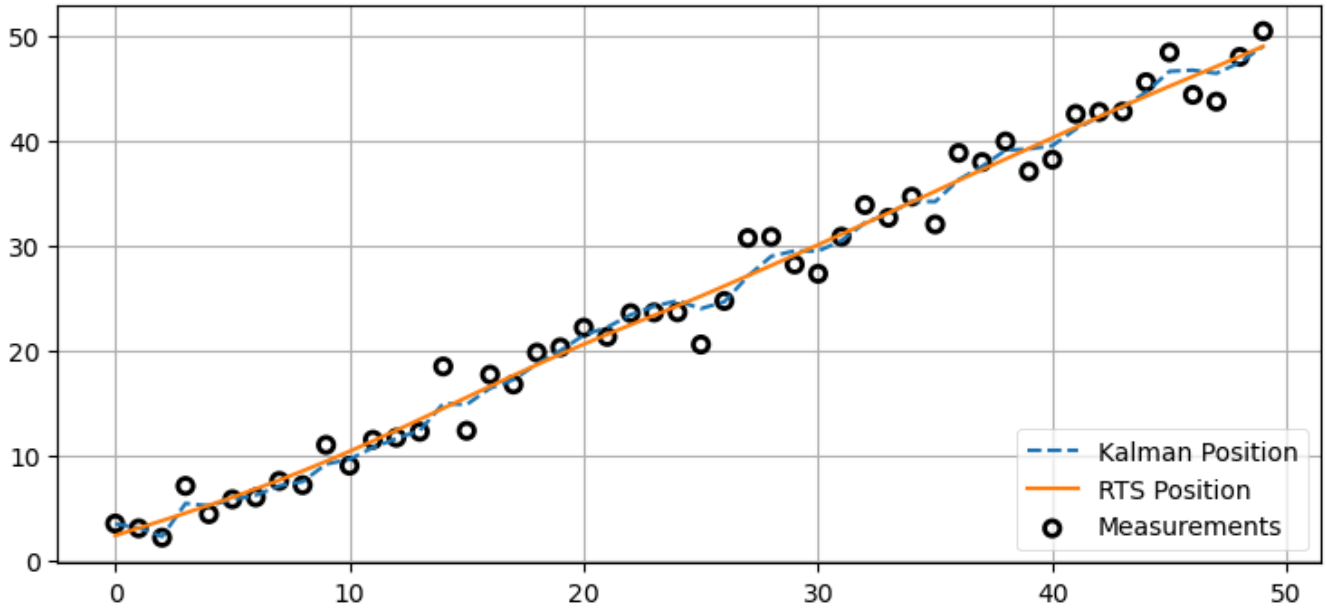
这本书中有一章专门介绍如何对处理结果进行平滑处理；这里就不再重复这些内容了。不过，这个工具的使用非常简单，而且能显著提升处理效果。我会用几个例子来简单演示一下。关于平滑处理的那一章并不难理解，你现在应该已经有足够的准备来阅读它了。

假设我们正在追踪一辆沿直线行驶的汽车。我们得到的测量结果显示，该汽车开始向左转弯。卡尔曼滤波器会调整状态估计值，使其更接近该测量结果。不过，卡尔曼滤波器无法判断：这一测量结果是由于噪声干扰产生的，还是汽车确实开始转弯了。

有了后续测量，就能更好地判断先前是否开始转弯。若后续测量持续显示左转，就有更充分的依据认为此前确实发生了转向；若后续仍沿直线，则说明先前那个偏离的测量值更可能受噪声影响，应降低其权重。利用后续观测，可以重新调整先前估计中对测量与预测的信任程度。RTS 平滑通过连续的概率加权调整估计，而不是简单地完全采纳或丢弃某个测量。

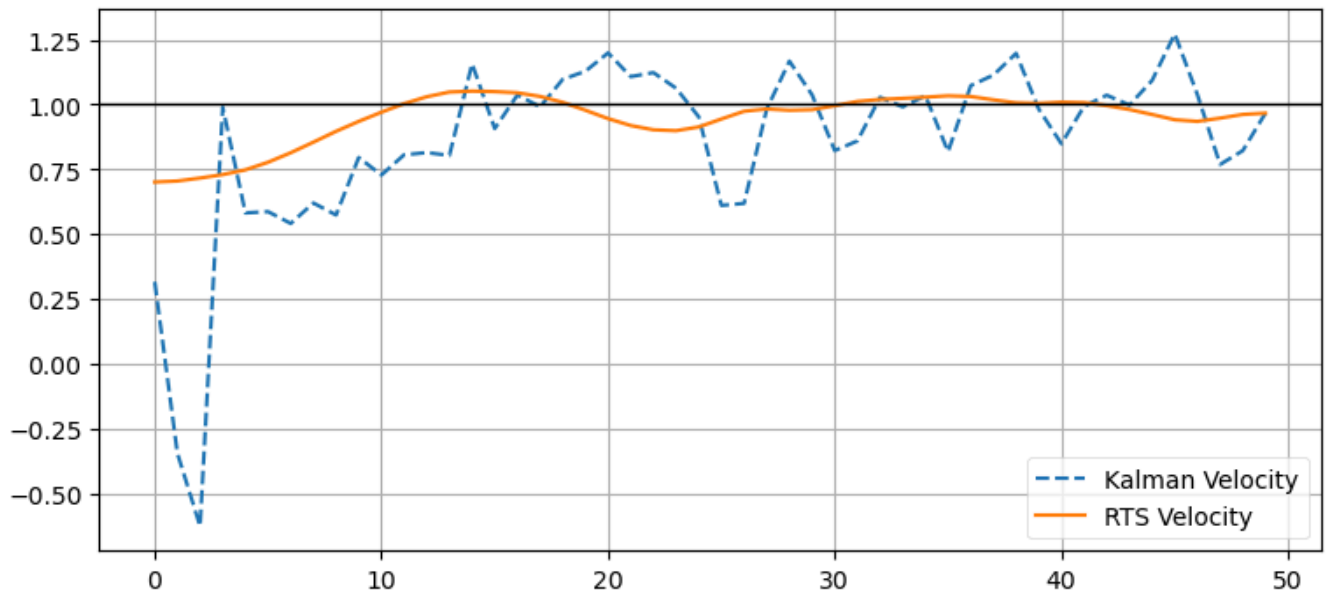
KalmanFilter 实现了 RTS 平滑器，其名称来自提出者 Rauch、Tung 和 Striebel。对应的方法是 rts_smoother()。将 batch_filter 计算得到的状态均值与协方差传入，即可得到平滑后的均值、协方差和平滑增益；平滑增益与前向滤波中的卡尔曼增益是不同的量。

```
from numpy.random import seed
count = 50
seed(8923)

P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
track, zs = compute_dog_data(3., .02, count)
Xs, Covs, _, _ = f.batch_filter(zs)
Ms, Ps, _, _ = f.rts_smoother(Xs, Covs)

book_plots.plot_measurements(zs)
plt.plot(Xs[:, 0], ls='--', label='Kalman Position')
plt.plot(Ms[:, 0], label='RTS Position')
plt.legend(loc=4);
```



这个输出效果真是棒极了！从这张图表中可以明显看出两点。首先，RTS 平滑处理后的输出结果比 KF 方法的输出结果要平滑得多。其次，RTS 方法的输出结果几乎总是比 KF 方法的更准确（我们将在“平滑处理”这一章节中详细探讨这一点）。至于作为“隐变量”的速度值，其改善程度则更为显著：

```
plt.plot(Xs[:, 1], ls='--', label='Kalman Velocity')
plt.plot(Ms[:, 1], label='RTS Velocity')
plt.legend(loc=4)
plt.gca().axhline(1, lw=1, c='k');
```



我们将在接下来的练习中探讨其中的原因。

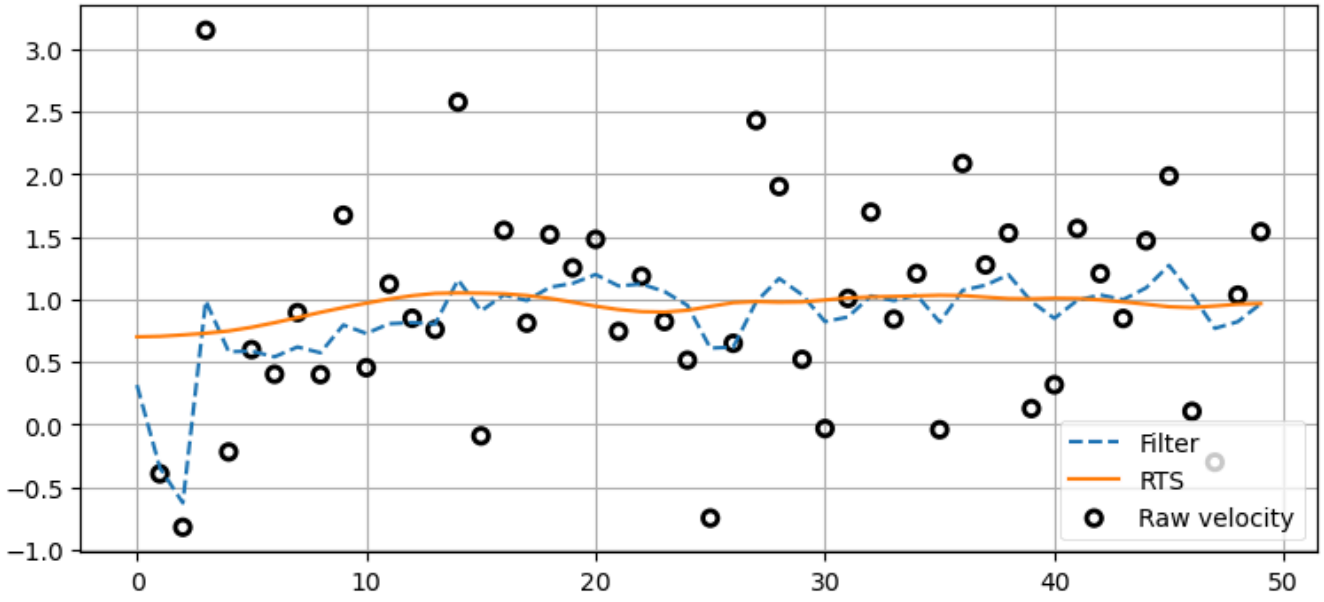
练习：比较速度大小

既然已经画出速度估计，下面也看看“原始”速度：用相邻位置测量的差分估计速度。本例采样间隔为一个时间单位，因此时刻 1 的速度可近似写为 xs[1] - xs[0]；一般情况下还应除以采样时间间隔。将原始速度与卡尔曼滤波和 RTS 平滑得到的速度画在一起，并讨论观察结果。

```
# your code here
```

解答

```
dx = np.diff(Xs[:, 0], axis=0)
plt.scatter(range(1, len(dx) + 1), dx, facecolor='none',
            edgecolor='k', lw=2, label='Raw velocity')
plt.plot(Xs[:, 1], ls='--', label='Filter')
plt.plot(Ms[:, 1], label='RTS')
plt.legend(loc=4);
```



可以看到，噪声几乎淹没信号，原始读数基本无法直接使用。滤波器另行维护速度估计。卡尔曼增益 $\mathbf{K}$ 是多维的，例如可能取 $\mathbf{K} = [0.1274, 0.843]^\mathsf{T}$。第一个分量将位置残差映射为位置修正量，第二个分量将同一个位置测量残差映射为速度修正量。协方差矩阵描述位置与速度的相关关系，使它们得到联合滤波。

这个例子再次说明，用卡尔曼滤波估计速度、加速度乃至更高阶导数量非常重要。即使位置测量已精确到可以直接使用而无需滤波，我仍会使用卡尔曼滤波器，因为它还能提供准确的速度与加速度估计。

讨论与总结

多元高斯模型使我们能够同时处理多个维度的数据，包括空间维度以及速度等其他维度。我们有一个重要的发现：隐变量有助于显著提高滤波器的准确性。之所以如此，是因为隐变量与那些可观测的变量之间存在相关性。

我对“可观测性”给出了一个直观的定义。可观测性这一概念是由卡尔曼博士为线性系统所提出的，其背后有着相当复杂的理论体系。可观测性用来判断：通过观察系统的输出，是否能够确定系统的状态。对于我们处理的问题来说，判断可观测性相对简单；但对于更复杂的系统，则需要复杂的分析才能得出结论。维基百科上的“可观测性”条目提供了相关概述；如果你想深入了解这个主题，[2]是一个不错的参考资料。

关于隐状态变量，有一个重要限制：写出估计它们的滤波器很容易，并不意味着它们真的可估。例如，形式上可以增加汽车颜色这个状态，但仅凭位置观测无法推断颜色，结果没有意义。设计者必须验证隐状态的估计是否可靠。没有速度传感器却要估计速度时，也必须测试，不能盲信。若速度含有正弦周期分量，采样频率必须满足适当的奈奎斯特条件，即高于所关心最高频率的两倍；若恰好每个信号周期只取同相位的一个样本，就可能误以为速度不变。

对于隐变量来说，初始化过程是个特别棘手的问题。如果初始化方式不当，虽然该滤波器通常能够恢复出可观测的变量，但在处理隐变量时则可能会遇到困难，甚至无法完成处理。估计隐变量虽然是一种强大的工具，但也可能带来风险。

我制定了一系列用于设计卡尔曼滤波器的步骤。这些步骤并非卡尔曼滤波器相关文献中常见的内容，仅供参考之用，而非固定不变的准则。针对复杂问题的设计过程是一个迭代的过程：先对状态向量进行初步估计，确定测量模型和状态模型，进行相关测试，然后再根据需要调整设计方案。

$\mathbf{R}$ 与 $\mathbf{Q}$ 的设计往往相当困难。前面的讲法似乎很科学：传感器具有 $\mathcal{N}(0, \sigma^2)$ 的高斯噪声，于是设 $\mathbf{R} = \sigma^2$，就完成了！但这是过度简化。真实传感器的噪声并不严格服从高斯分布。以本书开头的体重秤为例：若 $\sigma = 1$ kg，称量 0.5 kg 的物体，严格高斯模型就会给负读数分配非零概率，实际的秤却不会显示负重量。真实传感器噪声还可能呈现厚尾或偏斜；峰度可用来描述某些尾部特征。在体重秤这样的例子中，一侧或两侧尾部还可能被截断。

$\mathbf{Q}$ 的情况则更为糟糕。希望当我轻率地用“噪声矩阵”来描述对狗的行踪的预测时，你们能保持怀疑态度。谁又能知道狗接下来会做什么呢？我 GPS 中的卡尔曼滤波器根本不知道有山丘、外部风速，也不知道我糟糕的驾驶技术。然而，该滤波器却需要一个精确的数值来反映所有这些信息。无论是在沙漠中的越野驾驶，还是在一级方程式赛车赛道上，该滤波器都必须能够正常工作。

这些问题使一些研究者和工程师把卡尔曼滤波器贬称为“一团泥”，意思是它并不总能保持可靠表现。另一个值得了解的现象是滤波器可能变得“自满”：它根据你指定的噪声参数评估不确定性，这些参数可能导致过度自信。当 $\mathbf{P}$ 越来越小时，实际估计误差反而可能越来越大！最坏的情况就是滤波发散。学习非线性滤波时，我们还会多次遇到这一问题。

卡尔曼滤波器是一种描述世界的数学模型。其输出结果的准确性完全取决于该模型的精确度。为了使数学模型易于处理，我们必须做出一些假设。我们假设传感器和运动模型产生的噪声都符合高斯分布。同时，我们还假设所有过程都是线性的。如果这些假设成立，那么卡尔曼滤波器就达到了最小二乘意义上的最优解。这意味着，不可能有比卡尔曼滤波器更准确的估计结果了。不过，这些假设几乎从来都不成立，因此该模型的有效性必然有限，实际应用中的卡尔曼滤波器也很难达到最优效果。

后续章节会讨论非线性问题。现在要理解，设计线性滤波器矩阵往往既需要数学，也需要试验。先用数学给出初值，再通过试验调整。如果现实中存在大量未建模扰动，例如风，可能需要增大 $\mathbf{Q}$。在本章的常见运动学模型中，Q 太小会使滤波器过度信任预测、对变化反应迟缓；Q 太大则通常使它更偏向测量，估计更易受

到测量噪声影响。在自适应滤波一章，将学习根据输入和性能实时改变设计的方法；目前先寻找一组适合实际工况的数值。例如，特技飞行员是学员还是专家，会导致很不一样的运动特征，适用的噪声矩阵也可能不同。

参考文献

- [1] “卡尔曼滤波器”。维基百科：https://en.wikipedia.org/wiki/Kalman_filter#Details
- [2] Grewal, Mohinder S., Andrews, Angus P. 《卡尔曼滤波：基于 MATLAB 的理论与实践》。第三版。约翰·威利父子出版社。2008 年。

第 7 章：卡尔曼滤波的数学基础

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

读到这里，希望你已觉得卡尔曼滤波器令人畏惧的名声有些言过其实。当然，有些方程我暂时略去了推导，但希望实现过程对你而言还算清楚。基本思路很直接：取两个测量值，或者一个测量值与一个预测值，在二者之间选择融合结果。如果更信任测量，估计就更接近测量；如果更信任预测，估计就更接近预测。这可不是“火箭科学”——开个小玩笑，阿波罗飞船登月并返回，恰恰用的就是这些数学方法！

坦白说，我一直在精心挑选例题。对于一般问题，设计卡尔曼滤波器矩阵可能极其困难。不过，这也不算故意避重就轻：牛顿运动方程这类模型很容易用于卡尔曼滤波，而我们希望解决的大量问题正属于这一类。

我用代码和逻辑来阐释这些概念，而非数学公式。不过，也有一些主题需要用到比我所使用的更多的数学知识。本章将介绍你在阅读本书其余内容时所需要的数学知识。

动态系统的建模

动态系统是一种其状态（位置、温度等）会随时间而变化的物理系统。微积分正是研究数值变化的数学工具，因此我们用微分方程来描述动态系统。不过，有些系统无法用微分方程来描述，但在本书中我们不会遇到这类系统。

动态系统的建模其实是许多大学课程的内容。在某种程度上，没有任何东西可以替代几学期关于常微分方程和偏微分方程的学习，以及之后在控制系统理论方面的深入学习。如果你只是个业余爱好者，或者只是想解决工作中某个特定的滤波问题，那你很可能没有时间和精力去投入一年或更长时间来学习这些知识。

幸运的是，我能够阐述足够的理论内容，从而帮助我们为各种不同的卡尔曼滤波器建立相应的系统方程。我的目标是让你们能够阅读相关文献，并充分理解其内容，进而能够实际应用这些算法。虽然相关的数学原理相当复杂，但实际上我们只需要运用一些简单的技巧即可。

这是本书中涉及纯数学内容最长的部分。如果你想理解最常用的非线性滤波器——扩展卡尔曼滤波器，就必须掌握这一部分的所有内容。当然，我也介绍了些更现代的滤波器，这些滤波器不需要那么复杂的数学知识。你现在可以先跳过这一部分，如果以后决定要学习扩展卡尔曼滤波器，再回来阅读这部分内容。

首先，我们需要了解卡尔曼滤波所依据的数学方程和各种假设。既然我们试图用数学模型来描述现实世界中的现象，那么究竟有哪些因素需要考虑呢？

每个物理系统都有其演化过程。例如，汽车在给定时间内行驶的距离取决于速度，速度又会随加速度变化。可用高中学过的牛顿运动方程描述这些行为。下面的速度关系取初速度为零；初速度不为零时，还应加上初速度。

$$v = at$$
$$x = \frac{1}{2}at^2 + v_0t + x_0$$

一旦我们学会了微积分，就会以这种形式来理解它们：

$$\mathbf{v} = \frac{d\mathbf{x}}{dt}, \quad \mathbf{a} = \frac{d\mathbf{v}}{dt} = \frac{d^2\mathbf{x}}{dt^2}$$

在典型的汽车跟踪问题中，我们需要根据恒定的速度或加速度来计算汽车行驶的距离，就像我们在前面几章中所做的那样。不过，实际上情况并非如此简单。没有哪辆汽车能在完全平坦的道路上行驶。道路上会有各种凹凸不平之处，还有风阻以及起伏不平的路面，这些都会影响汽车的速度。此外，汽车的悬挂系统也属于机械装置，其中存在摩擦力，而且弹簧也并非完美无缺。

除了最简单的情形，完美建模系统是不可能的，因此必须简化。对于任意时刻 t，真实状态（例如汽车的位置）可表示为不完美模型的预测值，加上某种未知的过程噪声：

$$x(t) = x_{pred}(t) + noise(t)$$

这并不意味着 $noise(t)$ 是一个可以通过分析方法推导出来的函数。这仅仅是一个事实陈述而已：我们总可以把真实值表示为预测值加上各种过程噪声。所谓“噪声”并不一定指随机事件。如果我们正在追踪在大气中飞行的球，而模型却假设球处于真空中，那么空气阻力的影响就属于这种“过程噪声”。

在下一节中，我们将学习如何将一组高阶微分方程转化为一组一阶微分方程。转换完成后，无噪声系统的相关模型如下：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$$

$\mathbf{A}$ 称为系统动力学矩阵，因为它描述系统的动态行为。现在还需要对噪声建模，将它记为 $\mathbf{w}$ 并加入方程：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{w}$$

$\mathbf{w}$ 这个名字看起来可能不太合适，但你会很快明白：卡尔曼滤波器其实是以白噪声作为假设条件的。

最后，还要考虑系统输入。假设输入为 $\mathbf{u}$，且其对系统的影响可由线性模型描述。例如，踩油门使汽车加速，重力使球下落；这里都将它们视为已知输入。用矩阵 $\mathbf{B}$ 将 $\boldsymbol{u}$ 转换为系统所受的作用，再把这一作用项加入方程：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$$

就是这样。这就是卡尔曼博士试图解决的方程之一。如果我们假设 $\mathbf{w}$ 具有某些特定的性质，那么他就能找到一个最优的估算方法。

动态系统的状态空间表示

我们已经推导出了这个方程。

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu} + \mathbf{w}$$

不过，我们并不关心 $\mathbf{x}$ 的导数，而只关心 $\mathbf{x}$ 本身。暂且忽略那些干扰因素，我们希望找到一个方程，能够根据在时间 t_{k-1} 时的 $\mathbf{x}$ 值，递归地计算出在时间 t_k 时的 $\mathbf{x}$ 值。

$$\mathbf{x}(t_k) = \mathbf{F}(\Delta t)\mathbf{x}(t_{k-1}) + \mathbf{B}(t_k)\mathbf{u}(t_k)$$

按惯例，可将 $\mathbf{x}(t_k)$ 简写为 $\mathbf{x}_k$，表示 $\mathbf{x}$ 在时间 t 的第 k^{th} 个离散取值处的值。

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}_k\mathbf{u}_k$$

$\mathbf{F}$ 是一种常见的状态转移矩阵。它之所以被如此命名，是因为它能够使状态在离散的时间步长之间进行转换。它与系统动态矩阵 $\mathbf{A}$ 非常相似。两者的区别在于： $\mathbf{A}$ 描述的是一组线性微分方程，属于连续型模型；而 $\mathbf{F}$ 则是离散型模型，它描述的是一组线性方程（而非微分方程），这些方程使得状态在离散的时间步长 Δt 内从 $\mathbf{x}_{k-1}$ 状态转移为 $\mathbf{x}_k$ 状态。

找到这个矩阵通常相当困难。方程 $\dot{x} = v$ 是最简单的微分方程，我们很容易就能将其积分出来：

$$\int_{x_{k-1}}^{x_k} \mathrm{d}x = \int_0^{\Delta t} v\,\mathrm{d}t$$

$$x_k - x_{k-1} = v\Delta t$$

$$x_k = v\Delta t + x_{k-1}$$

这个方程具有递归性质：我们根据 $k-1$ 时刻 x 的值来计算 k 时刻 x 的值。这种递归结构使得我们能够以卡尔曼滤波器所要求的形式来表示该系统（即过程模型）。

$$\begin{aligned}\mathbf{x}_k &= \mathbf{F}\mathbf{x}_{k-1} \\ &= \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}\begin{bmatrix}x_{k-1} \\ \dot{x}_{k-1}\end{bmatrix}\end{aligned}$$

我们能够做到这一点，是因为 $\dot{x} = v$ 是最简单的微分方程。在物理系统中，几乎所有其他微分方程都更为复杂，因此无法用这种方法来求解。

状态空间方法大约在阿波罗任务时期普及，很大程度上得益于卡尔曼博士的工作。其思想很简单：先用一组 n^{th} 阶微分方程建立系统模型，再转换为等价的一阶微分方程组，然后写成上一节使用的向量—矩阵形式 $\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$。得到这一形式后，就可以用若干技术把这些线性微分方程转换为递归方程：

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}_k\mathbf{u}_k$$

有些书籍将状态转移矩阵称为“基本矩阵”。许多人使用 $\mathbf{\Phi}$ 来表示状态转移矩阵，而不是 $\mathbf{F}$ 。那些以控制理论为基础的资料，通常也使用这两种表示方式。

这些方法被称为“状态空间法”，因为我们是用系统状态来表示微分方程的解的。

从高阶方程推导出一阶方程

许多物理系统需要用带有控制输入 u 的二阶或更高阶微分方程描述：

$$a_n\frac{d^n y}{dt^n} + a_{n-1}\frac{d^{n-1} y}{dt^{n-1}} + \cdots + a_2\frac{d^2 y}{dt^2} + a_1\frac{dy}{dt} + a_0 y = u$$

状态空间方法需要一阶方程。任何更高阶的方程组都可以通过为导数引入额外的变量，进而将其转化为一阶方程来处理。

我们来看一个例子。给定系统 $\ddot{x} - 6\dot{x} + 9x = u$，请找出其等效的一阶方程。为便于理解，我使用了点号来表示时间导数。

第一步是把最高阶导数项单独移到等号一边。

$$\ddot{x} = 6\dot{x} - 9x + u$$

我们定义了两个新的变量：

$$\begin{aligned} x_1(t) &= x \\ x_2(t) &= \dot{x} \end{aligned}$$

现在将这些新变量代入原方程并整理，得到用新变量表示的一阶方程组。为简化记号，通常省略 (t)。

由定义可知 $\dot{x}_1 = x_2$，且 $\dot{x}_2 = \ddot{x}$。因此：

$$\begin{aligned} \dot{x}_2 &= \ddot{x} \\ &= 6\dot{x} - 9x + u \\ &= 6x_2 - 9x_1 + u \end{aligned}$$

因此，我们的一阶方程组为：

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= 6x_2 - 9x_1 + u \end{aligned}$$

稍加练习，就能熟练掌握：先将最高阶导数项单独移到等号一边，再为原变量及各阶导数定义新变量，最后代入整理。

状态空间形式下的一阶微分方程

将上一节中新定义的变量代入：

$$\frac{dx_1}{dt} = x_2,\ \frac{dx_2}{dt} = x_3,\ \ldots,\ \frac{dx_{n-1}}{dt} = x_n$$

把这些变量代入，并将低阶项移至等号右边，可将最高阶导数表示为：

$$\frac{dx_n}{dt} = -\frac{1}{a_n}\sum_{i=0}^{n-1} a_i x_{i+1} + \frac{1}{a_n}u$$

使用向量-矩阵表示法，我们有：

$$\begin{bmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \\ \vdots \\ \frac{dx_n}{dt} \end{bmatrix} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & -\frac{a_2}{a_n} & \cdots & -\frac{a_{n-1}}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \frac{1}{a_n} \end{bmatrix} u$$

我们将其记作 $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$。

求取时不变系统的基本矩阵

我们将系统方程表示为状态空间形式。

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$$

其中，$\mathbf{A}$ 为系统动力学矩阵。我们希望求得基本矩阵 $\mathbf{F}$，使状态 $\mathbf{x}$ 能按下式跨越时间间隔 Δt：

$$\mathbf{x}(t_k) = \mathbf{F}(\Delta t)\mathbf{x}(t_{k-1})$$

换言之，$\mathbf{A}$ 描述连续时间的线性微分方程组，我们需要找到 $\mathbf{F}$，用离散线性方程描述一个时间步之后的状态。在这里的时不变系统中，$\mathbf{A}$ 不随时间改变；随时间传播的是状态。

按照惯例，通常会省略 t_k 和 (Δt)，直接使用该符号来表示。

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$$

一般来说，有三种常见的方法可以用来确定卡尔曼滤波器中的该矩阵。最常用的方法是矩阵指数法。另一种方法是线性时不变系统理论。此外，还有数值计算方法。你可能还知道其他方法，但在卡尔曼滤波器的相关文献和实际应用中，你最常遇到的就是这三种方法。

矩阵指数

方程 $\frac{dx}{dt} = kx$ 的解可以通过以下方法求得：

$$\begin{aligned} \frac{dx}{dt} &= kx \\ \frac{dx}{x} &= k\,dt \\ \int \frac{1}{x}\,dx &= \int k\,dt \\ \log x &= kt + c \\ x &= e^{kt+c} \\ x &= e^c e^{kt} \\ x &= c_0 e^{kt} \end{aligned}$$

当 $t = 0$ 时，$x = x_0$。将这些值代入上述方程中。

$$x_0 = c_0 e^{k(0)}$$
$$x_0 = c_0 1$$
$$x_0 = c_0$$
$$x = x_0 e^{kt}$$

用类似方法，可得到下面这个一阶方程组的解：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x},\ \ \mathbf{x}(0) = \mathbf{x}_0$$

其中，$\mathbf{A}$ 为常数矩阵，其解为：

$$\mathbf{x} = e^{\mathbf{A}t}\mathbf{x}_0$$

将 $F = e^{\mathbf{A}t}$ 代入后，我们可以这样表示：

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$$

这就是我们正在寻找的形式！我们把求出该矩阵的问题，简化为求出 $e^{\mathbf{A}t}$ 的值的问题。

$e^{\mathbf{A}t}$ 被称为矩阵指数。它可以通过以下幂级数来计算：

$$e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \dots$$

这个幂级数来自 $e^{\mathbf{A}t}$ 的泰勒展开，具体推导不在这里展开。

让我们用这个方法来求解牛顿方程。用 v 来代替 $\dot{x}$，并假设速度为常数，那么就可以得到线性矩阵-向量形式。

$$\begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix}$$

这是一个一阶微分方程组。令 $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$，就可以求解下面的矩阵指数。这里用时间间隔 Δt 替换 t，强调我们关心的是一个离散时间步对应的状态转移：

$$\mathbf{F} = e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t + \frac{(\mathbf{A}\Delta t)^2}{2!} + \frac{(\mathbf{A}\Delta t)^3}{3!} + \dots$$

如果你进行乘法运算，就会发现 $\mathbf{A}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$。这意味着 $\mathbf{A}$ 的所有高次幂也都等于 $\mathbf{0}$。这样，我们就能得到一个确切的答案，而无需处理无穷多个项。

$$\begin{aligned} \mathbf{F} &= \mathbf{I} + \mathbf{A}\Delta t + 0 \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \Delta t \\ &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \end{aligned}$$

我们将其代入 $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$ 中，就能得到结果。

$$\boldsymbol{x}_k = \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}\boldsymbol{x}_{k-1}$$

你会认出，这就是我们在“多元卡尔曼滤波”一章中，通过分析方法推导出的、用于恒速卡尔曼滤波的矩阵。

SciPy 的 linalg 模块提供 expm() 计算矩阵指数。它采用 Padé 逼近，而不是直接使用泰勒级数。矩阵指数有多种计算方法，相关文献至少讨论了 19 种，而且都存在数值方面的困难 [1]。需要了解这些问题，尤其是在 $\mathbf{A}$ 的规模较大或数值条件不佳时。搜索“pade approximation matrix exponential”，可以找到大量有关研究。

在本章的简单模型中，你可能无须担心这些数值问题。例如，对前面的恒速模型，泰勒级数前两项便给出精确结果。但不要据此认为这里的讨论已经完整，就未经数值分析而把同样的截断方法用于其他问题。有趣的是，计算 $e^{\mathbf{A}t}$ 的一种方法是采用通用常微分方程求解器：把 $\mathbf{A}$ 对应的系统写成微分方程组，再用数值方法求解。除非高阶项为零，只取泰勒级数前两项通常只是近似，需要根据具体系统评估截断误差。

下面给出使用 expm() 计算 $e^{\mathbf{A}t}$ 的示例。

```
import numpy as np
from scipy.linalg import expm

dt = 0.1
A = np.array([[0, 1],
              [0, 0]])
expm(A*dt)
```

```
array([[1. , 0.1],
       [0. , 1. ]])
```

时间不变性

如果系统的行为随时间而变化，那么可以说，该动态系统可以用一阶微分方程来描述。

$$g(t) = \dot{x}$$

不过，如果该系统是时不变的，那么该方程的形式如下：

$$f(x) = \dot{x}$$

“时间不变”是什么意思呢？以家用音响为例。如果你在时间点 t 向音响输入信号 x，那么音响会输出信号 $f(x)$。而如果你在时间点 $t + \Delta t$ 进行输入操作，输出信号仍然会是 $f(x)$，只不过时间上有所延迟而已。

一个反例是：输入 $x(t) = \sin(t)$，系统定义为 $f(x) = t\,x(t) = t\sin(t)$。由于乘子 t 显式依赖时间，这个系统是时变的。飞机也可能具有时变特性：在更晚的时刻施加同样的控制输入时，燃油消耗已使飞机重量减轻，因此响应可能不同。

我们可以通过对等式的每一侧进行积分来求解这些方程。上面我已经演示了如何对时间不变系统 $v = \dot{x}$ 进行积分。不过，对时间不变方程 $\dot{x} = f(x)$ 进行积分则没那么简单。我们可以利用变量分离法，将等式两边同时除以 $f(x)$，然后把 dt 项移到等式的右边，这样就可以对等式的每一侧分别进行积分了。

$$\frac{dx}{dt} = f(x)$$
$$\int_{x_0}^{x} \frac{1}{f(x)} dx = \int_{t_0}^{t} dt$$

令 $F(x) = \int \frac{1}{f(x)} dx$，则有：

$$F(x) - F(x_0) = t - t_0$$

然后我们用该公式来求解 x 的值。

$$F(x) = t - t_0 + F(x_0)$$
$$x = F^{-1}[t - t_0 + F(x_0)]$$

换句话说，我们需要找到 F 的逆函数。这并非易事，因此在 STEM 教育中，有相当多的教学内容都是用来探讨如何解决这个棘手的分析问题的。

不过，这些其实都是些技巧而已。对于许多简单的 $f(x)$ 问题来说，要么没有明确的解析解，要么解决起来极其困难。因此，实际工作中的工程师们通常会采用状态空间法来寻求近似解。

矩阵指数可用来求解常系数线性微分方程组，即线性时不变系统。对于缓慢变化的系统，也常在短时间步内将系数近似视为常数。例如，飞机燃油消耗会使重量变化，但一秒钟内的变化可能足够小，因而可在这一时间步内近似采用不变的动力学参数。时间步足够短且近似条件合适时，结果仍可能相当准确。

示例：质量-弹簧-阻尼器模型

假设要跟踪连接弹簧和阻尼器的重物运动，例如汽车悬架。其中 m 为质量，k 为弹簧刚度，c 为阻尼系数，外部输入为 u，运动方程为：

$$m\frac{d^2x}{dt^2} + c\frac{dx}{dt} + kx = u$$

为了方便表示，我会将其写成这样。

$$m\ddot{x} + c\dot{x} + kx = u$$

令 $x_1(t) = x(t)$，并按下面的方式代入，就可以将它转化为一阶方程组：

$$x_1 = x$$
$$x_2 = \dot{x}_1$$
$$\dot{x}_2 = \ddot{x}_1 = \ddot{x}$$

和往常一样，为了方便表示，我省去了 (t) 符号。这样就能得到这个方程式了。

$$m\dot{x}_2 + cx_2 + kx_1 = u$$

求解 $\dot{x}_2$ 后，我们得到一个一阶方程：

$$\dot{x}_2 = -\frac{c}{m}x_2 - \frac{k}{m}x_1 + \frac{1}{m}u$$

我们将其转化为矩阵形式：

$$\begin{bmatrix}\dot{x}_1 \\ \dot{x}_2\end{bmatrix} = \begin{bmatrix}0 & 1 \\ -k/m & -c/m\end{bmatrix}\begin{bmatrix}x_1 \\ x_2\end{bmatrix} + \begin{bmatrix}0 \\ 1/m\end{bmatrix}u$$

现在，我们利用矩阵指数来求解状态转移矩阵：

$$\Phi(t) = e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \dots$$

前两项的结果是：

$$\mathbf{F} = \begin{bmatrix}1 & t \\ -(k/m)t & 1 - (c/m)t\end{bmatrix}$$

这样的精度可能足够，也可能不够。可将具体参数代入，计算下一项 $\frac{(\mathbf{A}t)^2}{2!}$，检查它对结果的影响。

线性时不变理论

线性时不变理论，也被称为 LTI 系统理论，提供了一种利用拉普拉斯逆变换来求解 $\mathbf{\Phi}$ 的方法。现在，你可能已经明白了，或者仍然完全搞不懂。在这本书中，我不会使用拉普拉斯变换。LTI 系统理论告诉我们：

$$\Phi(t) = \mathcal{L}^{-1}[(s\mathbf{I} - \mathbf{A})^{-1}]$$

这里不展开拉普拉斯变换的细节，只说明 $\mathcal{L}$ 把时间域信号变换到复频域 s，从而不再显式使用时间变量，但求解上面的方程并不简单。感兴趣可查阅维基百科关于线性时不变系统理论的介绍。之所以提到 LTI，是因为某些文献会用这种方法为较复杂的问题设计卡尔曼滤波矩阵。

数值解

最后，还有一些数值方法可以用来求解 $\mathbf{F}$。随着滤波器尺寸的增大，寻求解析解变得极为繁琐（不过像 SymPy 这样的工具可以简化这一过程）。C. F. van Loan[2]提出了一种数值方法，能够同时求解 $\mathbf{\Phi}$ 和 $\mathbf{Q}$。当然，这一切都是基于连续模型来进行的。

$$\dot{x} = Ax + Gw$$

其中，w 表示单位白噪声。根据 van Loan 的方法，可以同时计算出 $\mathbf{F}_k$ 和 $\mathbf{Q}_k$ 的值。

我已经在 `FilterPy` 中实现了 van Loan 的方法。你可以按照以下方式来使用它：

```
from filterpy.common import van_loan_discretization
A = np.array([[0., 1.], [-1., 0.]])
G = np.array([[0.], [2.]]) # white noise scaling
F, Q = van_loan_discretization(A, G, dt=0.1)
```

在“微分方程的数值积分”这一部分中，我介绍了几种在卡尔曼滤波中经常被使用的替代方法。

过程噪声矩阵的设计

一般而言，$\mathbf{Q}$ 矩阵是卡尔曼滤波设计中最难确定的部分之一。首先，相关数学需要信号理论基础；其次，我们试图为所知有限的扰动建模。例如，抛出的棒球可以近似为在空气中运动的球体，但仍存在许多未知因素：旋转及自旋衰减、缝线对阻力系数的影响、风以及空气密度等。即使能对给定过程模型求出精确的数学解，模型本身不完整，所得 $\mathbf{Q}$ 也就无法完全反映实际扰动。这会显著影响滤波行为：$\mathbf{Q}$ 太小，滤波器会过度信任预测模型，可能偏离真实状态；$\mathbf{Q}$ 太大，则会过度受测量噪声影响，性能也难以最优。实际中，需要大量仿真并分析采集数据，以选择合适的 $\mathbf{Q}$。下面先看数学推导。

假设我们有一个符合运动学规律的系统——也就是那种可以用牛顿运动定律来描述的系统。对于这个系统，我们可以做出几种不同的假设。

我们一直使用的是这样的过程模型。

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$$

其中，$\mathbf{w}$ 代表过程噪声。运动系统是连续的——其输入和输出可以在任意时间点发生变化。不过，我们所使用的卡尔曼滤波器是离散的（虽然卡尔曼滤波器也有连续形式，但本书不涉及这些内容）。我们需要以固定的时间间隔对系统进行采样。因此，我们必须为上述方程中的噪声项找到相应的离散表示形式。这取决于我们对噪声行为的假设。我们将考虑两种不同的噪声模型。

连续白噪声模型

我们用牛顿运动方程建立运动学模型。此前使用了位置与速度，或者位置、速度与加速度。也可以继续引入更高阶导数，如加加速度（jerk）及位置的四阶导数（常称 snap 或 jounce）等。不过，通常不会无限增加状态阶次，因为引入超过真实系统动力学所需的项，反而可能降低估计精度。

假设需要建模位置、速度与加速度。可以先假设每个离散时间步内加速度恒定。当然，系统存在过程噪声，加速度并不真正恒定；未建模的外力会让它随时间改变。本节假设加速度受到连续时间、零均值白噪声 $\boldsymbol{w}(t)$ 的扰动。也就是说，噪声引起的速度小增量的均值为零。

由于噪声是持续变化的，因此我们需要对其进行积分处理，从而得到所选离散时间间隔内的离散噪声值。这里就不详细证明这个过程了，不过，噪声离散化的公式如下：

$$\mathbf{Q} = \int_0^{\Delta t} \mathbf{F}(t)\mathbf{Q}_c\mathbf{F}^\mathsf{T}(t)dt$$

其中，$\mathbf{Q_c}$ 表示连续时间噪声的协方差强度（噪声谱密度矩阵）。直观地说，$\mathbf{F}(t)\mathbf{Q}_c\mathbf{F}^\mathsf{T}(t)$ 用时刻 t 的过程模型 $\mathbf{F}(t)$ 对该噪声进行传播。为求离散时间间隔 Δt 内累计引入的噪声协方差，需要在 $[0, \Delta t]$ 上对这一表达式积分。

我们知道，牛顿系统的基本矩阵为

$$F = \begin{bmatrix}1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1\end{bmatrix}$$

我们将连续噪声定义为：

$$\mathbf{Q_c} = \begin{bmatrix}0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1\end{bmatrix} \Phi_s$$

其中，$\mathbf{\Phi}_s$ 是白噪声的谱密度。这个矩阵形式可以从随机过程理论推导，细节超出本书范围，可参考相关标准教材。实际中往往不知道噪声谱密度，因此常把它作为工程参数，通过试验调整至滤波器表现合适。$\mathbf{\Phi}_s$ 所乘的矩阵将该噪声强度施加到加速度状态分量，使名义上的匀加速模型能够容纳噪声引起的加速度变化。

我们可以自己来进行这些计算，但我更愿意使用 SymPy 来解这些方程。

$$\mathbf{Q_c} = \begin{bmatrix}0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1\end{bmatrix} \Phi_s$$

```
import sympy
from sympy import (init_printing, Matrix, MatMul,
                   integrate, symbols)

init_printing(use_latex='mathjax')
dt, phi = symbols('\Delta{t} \Phi_s')
F_k = Matrix([[1, dt, dt**2/2],
              [0,  1,      dt],
              [0,  0,       1]])
Q_c = Matrix([[0, 0, 0],
              [0, 0, 0],
              [0, 0, 1]])*phi

Q = integrate(F_k * Q_c * F_k.T, (dt, 0, dt))

# factor phi out of the matrix to make it more readable
Q = Q / phi
MatMul(Q, phi)
```

$$\begin{bmatrix}\frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t\end{bmatrix} \Phi_s$$

为了完整性起见，让我们来计算一下 0 阶和 1 阶方程的解。

```
F_k = Matrix([[1]])
Q_c = Matrix([[phi]])

print('0th order discrete process noise')
integrate(F_k*Q_c*F_k.T,(dt, 0, dt))
```

```
0th order discrete process noise
```

$$\left[\Delta t \Phi_s\right]$$

```
F_k = Matrix([[1, dt],
              [0, 1]])
Q_c = Matrix([[0, 0],
              [0, 1]]) * phi

Q = integrate(F_k * Q_c * F_k.T, (dt, 0, dt))

print('1st order discrete process noise')
# factor phi out of the matrix to make it more readable
Q = Q / phi
MatMul(Q, phi)
```

```
1st order discrete process noise
```

$$\begin{bmatrix}\frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t\end{bmatrix} \Phi_s$$

分段白噪声模型

另一种模型假设：最高阶扰动，例如加速度扰动，在每个采样区间内恒定，而在不同区间取不同的随机值，且各区间的扰动互不相关。换言之，加速度扰动可在采样区间的交界处发生跳变。这与前面的连续时间白噪声驱动模型有所不同。

我们将把这个问题建模为……

$$f(x) = Fx + \Gamma w$$

其中，Γ 是系统的噪声增益，w 表示分段恒定的加速度扰动，也可以是速度、加加速度等其他阶次的扰动。

我们先来研究一阶系统。在这种情况下，我们有状态转移函数。

$$\mathbf{F} = \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}$$

在某个时间段内，速度的变化量为 $w(t)\Delta t$，位置的变化量为 $w(t)\Delta t^2/2$。

$$\Gamma = \begin{bmatrix}\frac{1}{2}\Delta t^2 \\ \Delta t\end{bmatrix}$$

该过程噪声的协方差则为

$$Q = \mathbb{E}[\Gamma w(t) w(t) \Gamma^{\mathsf{T}}] = \Gamma \sigma_v^2 \Gamma^{\mathsf{T}}$$

我们可以用 SymPy 来计算，具体步骤如下：

```
var = symbols('sigma^2_v')
v = Matrix([[dt**2 / 2], [dt]])

Q = v * var * v.T

# factor variance out of the matrix to make it more readable
Q = Q / var
MatMul(Q, var)
```

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix} \sigma_v^2$$

二阶系统也遵循同样的数学运算规则。

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

这里采用离散时间随机增量模型，得到下面的噪声增益。各时间段互不相关的驱动增量与这些增量累积形成的维纳过程是不同的对象。

$$\mathbf{\Gamma} = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 1 \end{bmatrix}$$

这个模型并不是唯一的“真相”，它只是计算方便、效果良好的一种选择。例如，也可以假设噪声作用于加加速度，但方程会更复杂。

该过程噪声的协方差则为

$$Q = \mathbb{E}[\mathbf{\Gamma} w(t) w(t) \mathbf{\Gamma}^\mathsf{T}] = \mathbf{\Gamma}\sigma_v^2\mathbf{\Gamma}^\mathsf{T}$$

我们可以用 SymPy 来计算，具体步骤如下：

```
var = symbols('sigma^2_v')
v = Matrix([[dt**2 / 2], [dt], [1]])

Q = v * var * v.T

# factor variance out of the matrix to make it more readable
Q = Q / var
MatMul(Q, var)
```

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 & \Delta t \\ \frac{\Delta t^2}{2} & \Delta t & 1 \end{bmatrix} \sigma_v^2$$

我们无法断言这种模型比连续模型更准确或更不准确——两者都只是对实际现象的近似描述而已。只有通过实践和实验，才能确定哪种模型更合适。在实际应用中，通常会发现两种模型都能给出合理的结果，但一般来说，其中一种模型的表现会更好一些。

第二种模型用 σ^2 表示噪声方差，可结合运动情况与预期误差来设定。第一种模型要求指定谱密度，不那么直观；不过，由于它在整个时间间隔内对噪声进行积分，因此更便于处理变化的采样间隔。这些都不是固定规则：应根据滤波性能试验以及对物理系统行为的了解，选择适用的模型，也可以自行建立模型。

一个不错的经验法则是：将 σ 的值设定在 $\frac{1}{2}\Delta a$ 到 Δa 之间。其中，Δa 表示加速度在各个采样周期之间的最大变化幅度。在实际操作中，我们会先选定一个数值，然后利用数据来进行模拟测试，最终选择那个效果最佳的数值。

使用 FilterPy 来计算 Q 值

FilterPy 提供了几种计算 $\mathbf{Q}$ 的函数。Q_continuous_white_noise() 根据给定的 Δt 和噪声谱密度计算 $\mathbf{Q}$。

```
from filterpy.common import Q_continuous_white_noise
from filterpy.common import Q_discrete_white_noise

Q = Q_continuous_white_noise(dim=2, dt=1, spectral_density=1)
print(Q)
```

```
[[0.333 0.5  ]
 [0.5   1.   ]]
```

```
Q = Q_continuous_white_noise(dim=3, dt=1, spectral_density=1)
print(Q)
```

```
[[0.05  0.125 0.167]
 [0.125 0.333 0.5  ]
 [0.167 0.5   1.   ]]
```

函数 `Q_discrete_white_noise()` 在假设噪声具有分段模型的前提下，计算出 $\mathbf{Q}$ 的值。

```
Q = Q_discrete_white_noise(2, var=1.)
print(Q)
```

```
[[0.25 0.5 ]
 [0.5  1.  ]]
```

```
Q = Q_discrete_white_noise(3, var=1.)
print(Q)
```

```
[[0.25 0.5  0.5 ]
 [0.5  1.   1.  ]
 [0.5  1.   1.  ]]
```

Q 的简化处理

许多资料使用更简单的 $\mathbf{Q}$：除右下角的噪声项外，其余元素均设为零。这是否合理？先看看较小时间间隔 Δt 对应的 $\mathbf{Q}$：

```
import numpy as np

np.set_printoptions(precision=8)
Q = Q_continuous_white_noise(
    dim=3, dt=0.05, spectral_density=1)
print(Q)
np.set_printoptions(precision=3)
```

```
[[0.00000002 0.00000078 0.00002083]
 [0.00000078 0.00004167 0.00125   ]
 [0.00002083 0.00125    0.05      ]]
```

我们可以看到，大多数项的数值都非常小。请记住，唯一使用这个矩阵的方程就是……

$$\mathbf{P} = \mathbf{FPF}^\mathsf{T} + \mathbf{Q}$$

如果 $\mathbf{Q}$ 的值相对于 $\mathbf{P}$ 来说很小，那么它对 $\mathbf{P}$ 的计算几乎没有任何贡献。可以将 $\mathbf{Q}$ 设置为零矩阵，右下角那个元素除外。

$$\mathbf{Q} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \sigma^2 \end{bmatrix}$$

虽然这种做法并不完全准确，但通常可以作为一种有效的近似方法。如果你要将这种方法应用于重要的场景中，那么就需要进行大量的测试，以确保该滤波器在各种情况下都能正常发挥作用。

采用这种简化时，“右下角项”应理解为每个运动维度中最高阶状态分量所对应的噪声方差。若状态为 $\boldsymbol{x} = [x \quad \dot{x} \quad \ddot{x} \quad y \quad \dot{y} \quad \ddot{y}]^\mathsf{T}$，$\mathbf{Q}$ 就是 6×6 矩阵；$\ddot{x}$ 与 $\ddot{y}$ 对应的两个对角元都应在 $\mathbf{Q}$ 中设为非零。

后验协方差的稳定计算

我已经给出了用于计算后验协方差的公式。

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$$

严格说，上式在使用最优增益时正确。但 FilterPy 采用的是下面更稳健的 Joseph（约瑟夫）形式：

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^\mathsf{T} + \mathbf{KRK}^\mathsf{T}$$

我经常收到邮件或 GitHub 问题报告，认为这一实现有错误，实际上并非如此。采用这种形式有几个原因：首先，涉及 $(\mathbf{I} - \mathbf{KH})$ 的减法在浮点运算中可能使最终协方差矩阵失去对称性。协方差应当对称，数值误差可能导致滤波发散，或在后续需要满足矩阵性质的运算中引发异常。

保持对称性的传统方法是使用以下公式：

$$\mathbf{P} = (\mathbf{P} + \mathbf{P}^\mathsf{T})/2$$

协方差矩阵理论上满足 $\sigma_{ij} = \sigma_{ji}$。如果浮点误差使这两个对称位置的数值不一致，上述操作会用它们的平均值替换二者，从而恢复矩阵的对称性。对称化不能保证消除全部数值误差，也不自动保证半正定性。

观察 Joseph 形式，可以发现两项都具有 $\mathbf{ABA}^\mathsf{T}$ 的结构，因而在理论上保持对称性。那么，这个方程从何而来？为什么采用它，而不是下面的简化更新加对称化处理？

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$$
$$\mathbf{P} = (\mathbf{P} + \mathbf{P}^\mathsf{T})/2$$

让我们从最基本的原则出发来推导这个方程吧。其实并不难。理解这个推导过程有助于我们明白该方程的用途。更重要的是，如果滤波器因为数值不稳定而出现异常，我们也能通过这个推导过程来找出问题所在。这个推导过程源自 Brown[4]的研究。

先说明记号：$\mathbf{x}$ 是系统的真实状态，$\hat{\mathbf{x}}$ 是后验状态估计，$\bar{\mathbf{x}}$ 是先验状态估计。

鉴于此，我们可以将模型定义为：

$$\mathbf{x}_{k+1} = \mathbf{F}_k\mathbf{x}_k + \mathbf{w}_k$$
$$\mathbf{z}_k = \mathbf{H}_k\mathbf{x}_k + \mathbf{v}_k$$

换句话说，下一状态 $\mathbf{x}_{k+1}$ 是第 k 步的当前状态经过模型 $\mathbf{F}_k$ 传播后，再加上过程噪声 $\mathbf{w}_k$ 得到的。

请注意，这些只是定义而已。没有任何系统能够完全遵循数学模型，因此我们用噪声项 $\mathbf{w}_k$ 来模拟这一情况。同样，由于传感器存在的误差，没有任何测量结果是完美的，所以我们用 $\mathbf{v}_k$ 来模拟这一误差。

我省略下标 k 吧，因为在后续的推导过程中，我们只考虑步骤 k 处的数值，而完全不考虑步骤 $k+1$ 处的数值。

现在，我们将估计误差定义为真实状态与估计状态之间的差异。

$$\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$$

这同样只是一种定义而已；我们不知道该如何计算 $\mathbf{e}$ 的值，它只不过是真实状态与估计状态之间的差异罢了。

由此定义估计误差的二阶矩，即 $\mathbf{ee}^\mathsf{T}$ 的期望。在无偏估计的假设下，它就是估计误差协方差：

$$\begin{aligned}P &= E[\mathbf{ee}^\mathsf{T}] \\ &= E[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\mathsf{T}]\end{aligned}$$

接下来，我们将后验估计定义为

$$\hat{\mathbf{x}} = \bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}})$$

这个形式与卡尔曼滤波方程相同，但这里首先把它作为估计的定义。我们尚未将 $\mathbf{K}$ 指定为最优卡尔曼增益，此处 $\mathbf{K}$ 只是一个待定的融合增益。在一般多维问题中，增益是矩阵，其元素不受介于 0 与 1 之间的限制。

现在我们已经有了相关的定义，接下来就来进行一些代入和代数运算吧。

将 $\hat{\mathbf{x}}$ 的定义代入表达式 $(\mathbf{x} - \hat{\mathbf{x}})$，即可展开为：

$$(\mathbf{x} - \hat{\mathbf{x}}) = \mathbf{x} - (\bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}}))$$

现在我们用 $\mathbf{Hx} + \mathbf{v}$ 来替换 $\mathbf{z}$：

$$\begin{aligned}(\mathbf{x} - \hat{\mathbf{x}}) &= \mathbf{x} - (\bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}})) \\ &= \mathbf{x} - (\bar{\mathbf{x}} + \mathbf{K}(\mathbf{Hx} + \mathbf{v} - \mathbf{H}\bar{\mathbf{x}})) \\ &= (\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}(\mathbf{Hx} + \mathbf{v} - \mathbf{H}\bar{\mathbf{x}}) \\ &= (\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{KH}(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{Kv} \\ &= (\mathbf{I} - \mathbf{KH})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{Kv}\end{aligned}$$

现在求 $\mathbf{P}$。利用先验误差 $(\mathbf{x} - \bar{\mathbf{x}})$ 的外积期望等于先验协方差 $\bar{\mathbf{P}}$，以及测量噪声 $\mathbf{v}$ 满足 $\boldsymbol{E}[\mathbf{v}\mathbf{v}^\mathsf{T}] = \mathbf{R}$，并假设先验误差与测量噪声互不相关，交叉项的期望便为零，从而得到：

$$\begin{aligned}\mathbf{P} &= \boldsymbol{E}\big[[(\mathbf{I} - \mathbf{KH})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{Kv})][(\mathbf{I} - \mathbf{KH})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{Kv}]^\mathsf{T}\big] \\ &= (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^\mathsf{T} + \mathbf{KRK}^\mathsf{T}\end{aligned}$$

这正是本节要证明的结果。

请注意，这个公式适用于所有的 $\mathbf{K}$ 值，而不仅仅是卡尔曼滤波器计算出的最优值 $\mathbf{K}$。这也是我使用这个公式的理由。在现实中，卡尔曼滤波器计算出的增益并非最优值：因为现实世界从来都不是完全线性的或符合高斯分布的；同时，计算过程中还会出现浮点数误差。相比而言，这个公式更不容易导致卡尔曼滤波器在现实条件下出现异常行为。

那么，$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$ 是从哪里来的呢？让我们来完成这个简单的推导过程吧。记得，卡尔曼滤波器的最优增益是由以下公式给出的：

$$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}$$

现在，我们将这个结果代入我们刚刚推导出的方程中：

$$\begin{aligned}&= (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^\mathsf{T} + \mathbf{KRK}^\mathsf{T} \\ &= \bar{\mathbf{P}} - \mathbf{KH}\bar{\mathbf{P}} - \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T} + \mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})\mathbf{K}^\mathsf{T} \\ &= \bar{\mathbf{P}} - \mathbf{KH}\bar{\mathbf{P}} - \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T} + \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})\mathbf{K}^\mathsf{T} \\ &= \bar{\mathbf{P}} - \mathbf{KH}\bar{\mathbf{P}} - \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T} + \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T} \\ &= \bar{\mathbf{P}} - \mathbf{KH}\bar{\mathbf{P}} \\ &= (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}\end{aligned}$$

因此，当增益处于最优值时，$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$ 在数学上是正确的；不过 $(\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^\mathsf{T} + \mathbf{KRK}^\mathsf{T}$ 同样也是正确的。正如我们之前所讨论的，即使增益不是最优值，$(\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^\mathsf{T} + \mathbf{KRK}^\mathsf{T}$ 依然正确，而且其数值稳定性也更高。因此，我在 FilterPy 中使用了这种计算方式。

很有可能，你的滤波器仍然会出现发散现象，尤其是当运行周期达到数百或数千次时。你需要仔细检查这些方程式。文献中还提供了其他多种计算方法，也许其中有一些更适用于你的问题。当然，如果你正在解决那些一旦出错就会导致设备损坏或人员伤亡的实际工程问题，那么你就需要参考工程领域的专业文献，而不仅仅是这本书的内容。如果你处理的只是些“简单”的问题，且出错不会造成严重后果，那么一旦发现发散现象，

你可以将 $\mathbf{P}$ 的值重置为某个“合理”的数值，然后继续计算。例如，你可以将矩阵中的非对角线元素设为零，这样矩阵就只包含方差了；之后，你可以乘以一个略大于一的常数，以弥补因操作而造成的信息损失。发挥你的想象力，多做些测试吧。

卡尔曼增益方程的推导

如果你已经读了上一节的内容，那不妨也把这一节读一读。通过这一节的讲解，我们就已经推导出了卡尔曼滤波器的方程式。

请注意，这种推导方式并没有使用贝叶斯定理。据我所知，至少有四种不同的方法可以用来推导卡尔曼滤波器的方程式。这种推导方式在文献中很常见，其逻辑其实源自上一节的论述。该推导方法的来源依然是 Brown[4]。

上一节用待定增益 $\mathbf{K}$ 推导了 Joseph 形式。要得到最优增益，还需用微积分最小化均方误差。熟悉的思路是：对函数 $\boldsymbol{f(x)}$ 求导，再令导数为零，即 $\frac{d}{dx}\boldsymbol{f(x)} = \mathbf{0}$。导数为零只给出驻点条件；本节目标函数的二次型结构提供了最小化依据。

这里用误差协方差矩阵 $\mathbf{P}$ 表示估计的不确定性，对角元是各状态分量的误差方差。最小化它们的总和，也就是最小化矩阵的迹，便能最小化各分量均方误差之和。因此，对矩阵的迹关于增益求导。

布朗提到了两个与迹的导数相关的公式：

$$\frac{d\,trace(\mathbf{AB})}{d\mathbf{A}} = \mathbf{B}^\mathsf{T}$$

$$\frac{d\,trace(\mathbf{ACA}^\mathsf{T})}{d\mathbf{A}} = 2\mathbf{AC}$$

其中，$\mathbf{AB}$ 是方阵，$\mathbf{C}$ 是对称矩阵。

我们将 Joseph 方程展开为：

$$\mathbf{P} = \bar{\mathbf{P}} - \mathbf{KH}\bar{\mathbf{P}} - \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T} + \mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})\mathbf{K}^\mathsf{T}$$

现在，我们需要求出 $\mathbf{P}$ 的迹关于 $\mathbf{K}$ 的导数： $\frac{d\,trace(\mathbf{P})}{d\mathbf{K}}$ 。

第一项关于 $\mathbf{K}$ 的迹的导数是 $\mathbf{0}$ ，因为在表达式中不包含 $\mathbf{K}$ 。

不计前面的负号时，第二项对应的迹的导数为 $(\mathbf{H}\bar{\mathbf{P}})^\mathsf{T}$。计入展开式中的负号后，该项的贡献应取其相反数。

注意，$\bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{K}^\mathsf{T}$ 是 $\mathbf{KH}\bar{\mathbf{P}}$ 的转置。矩阵与其转置具有相同的迹，因此第三项对迹的导数与第二项相同；两项都应计入展开式中的负号。

最后，第四项的迹的导数为 $2\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})$ 。

合并各项，得到最终的导数表达式：

$$\frac{d\,trace(\mathbf{P})}{d\mathbf{K}} = -2(\mathbf{H}\bar{\mathbf{P}})^\mathsf{T} + 2\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})$$

我们将该值设为零，然后求解方程，从而找到能够最小化误差的 $\mathbf{K}$ 值。

$$
\begin{aligned}
-2(\mathbf{H}\bar{\mathbf{P}})^\mathsf{T} + 2\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}) &= 0 \\
\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}) &= (\mathbf{H}\bar{\mathbf{P}})^\mathsf{T} \\
\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}) &= \bar{\mathbf{P}}\mathbf{H}^\mathsf{T} \\
\mathbf{K} &= \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}
\end{aligned}
$$

这个推导并不完全无懈可击，因为我省略了关于“为何最小化迹值就能最小化总误差”的解释。不过，我觉得对于本书而言，这样的推导已经足够了。如果需要更详细的解释，可以参考各种标准教材。

微分方程的数值积分

我们接触过多种用于求解线性微分方程的数值方法。这些方法包括状态空间法、拉普拉斯变换以及范洛恩法。

这些方法适合线性常微分方程，但处理非线性方程时效果未必好。例如预测快速转弯汽车的位置：汽车靠前轮转向，在前进时绕后轴改变朝向，因此路径持续变化，线性预测必然存在误差。如果相对于时间间隔 Δt，系统在一步内的变化足够小，结果仍可能可用；但在后续研究的非线性卡尔曼滤波问题中，不能总指望这一点。

基于这些原因，我们必须掌握如何对常微分方程进行数值积分。这其实是一个相当复杂的主题，需要用好几本书来详细阐述。不过，我在这里会介绍几种简单的技巧，这些技巧足以应对你遇到的大多数问题。

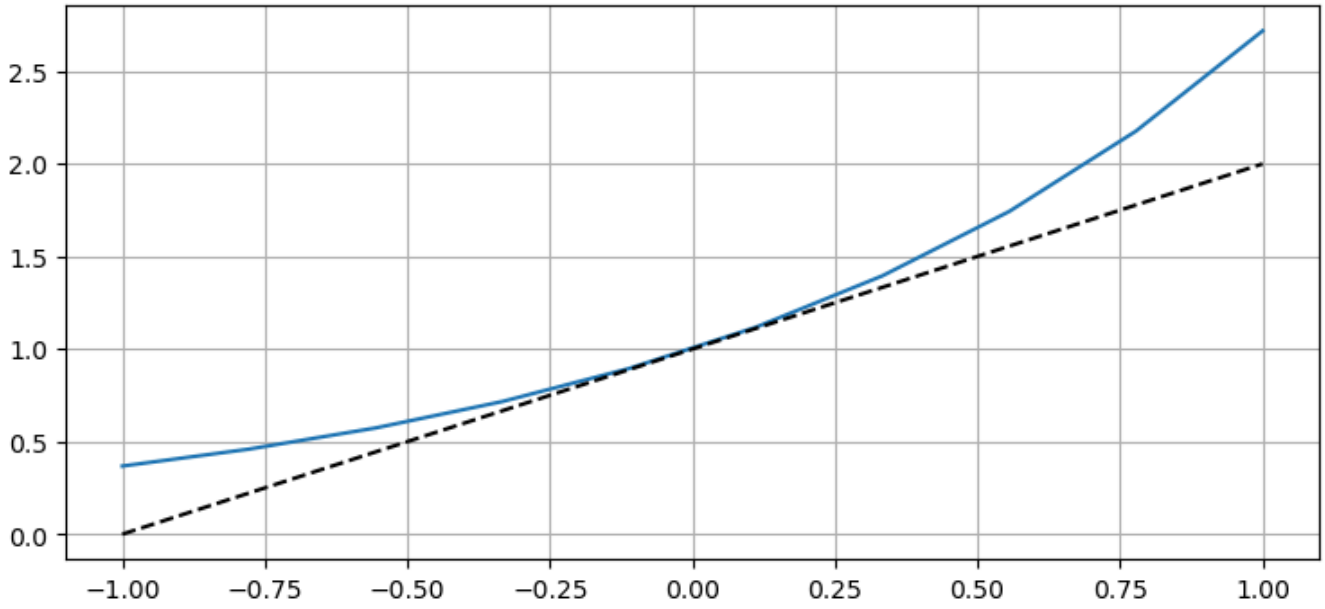
欧拉法

假设有如下初值问题：

$$
\begin{aligned}
y' &= y, \\
y(0) &= 1
\end{aligned}
$$

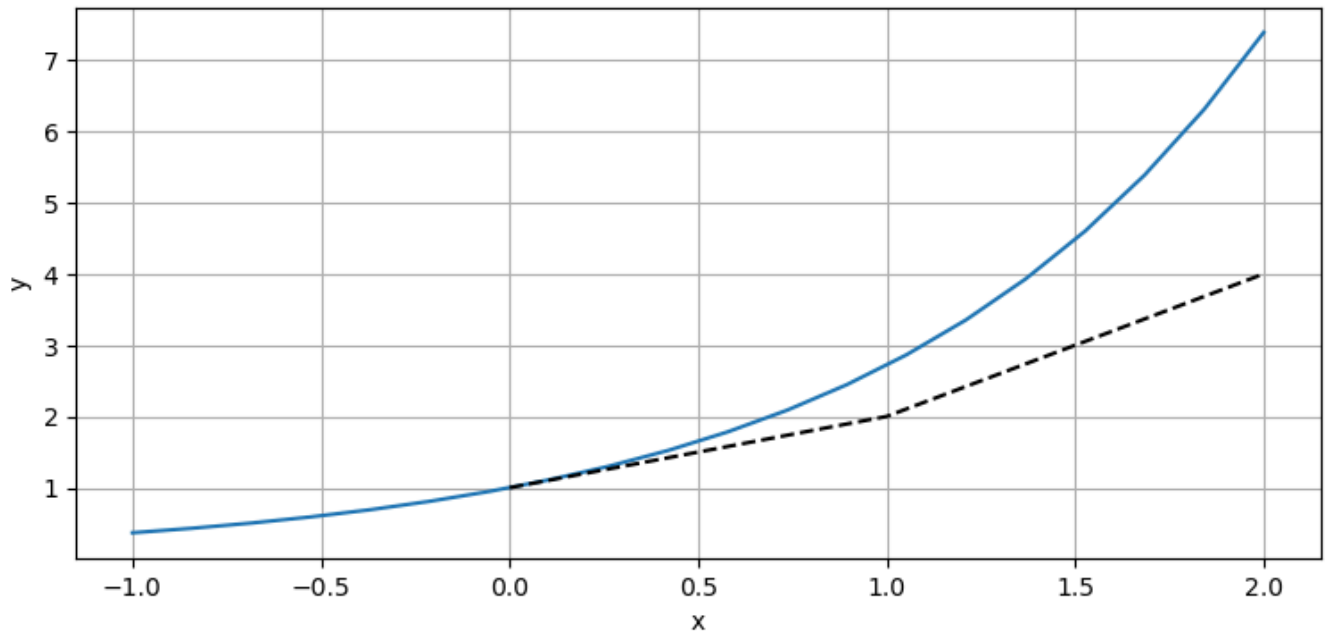
此前已经解过这个方程，知道精确解为 $y = e^t$。但对于一般常微分方程，我们未必知道精确解。通常已知的是导数，也就是解曲线的斜率，以及初始值：在 $t = 0$ 时有 $y = 1$。利用这两项信息，就能根据 $t = 0$ 处的斜率与 $y(0)$ 的值，近似预测 $y(t = 1)$。下图展示了这一思路。

```
import matplotlib.pyplot as plt

t = np.linspace(-1, 1, 10)
plt.plot(t, np.exp(t))
t = np.linspace(-1, 1, 2)
plt.plot(t,t+1, ls='--', c='k');
```



在 $t = 0.1$ 附近，沿初始斜率作出的直线近似很接近真实曲线；到 $t = 1$ 时，偏差已很明显。不过，先继续使用步长 1。在 $t = 1$ 时，y 的估计为 2。接着，用 $t = 1$ 时刻当前估计对应的斜率继续前进，得到 $t = 2$ 处的近似值。根据 $y' = y$，该近似斜率也等于 2。

```
import kf_book.book_plots as book_plots

t = np.linspace(-1, 2, 20)
plt.plot(t, np.exp(t))
t = np.linspace(0, 1, 2)
plt.plot([1, 2, 4], ls='--', c='k')
book_plots.set_labels(x='x', y='y');
```



在这里，我们看到 y 的下一个估计值为 4。不过，误差值增长得相当快，这可能会让人觉得不太理想。不过，1 这个步长其实相当大。让我们把这一算法用代码实现出来，然后通过使用较小的步长来验证其是否能够正常工作。

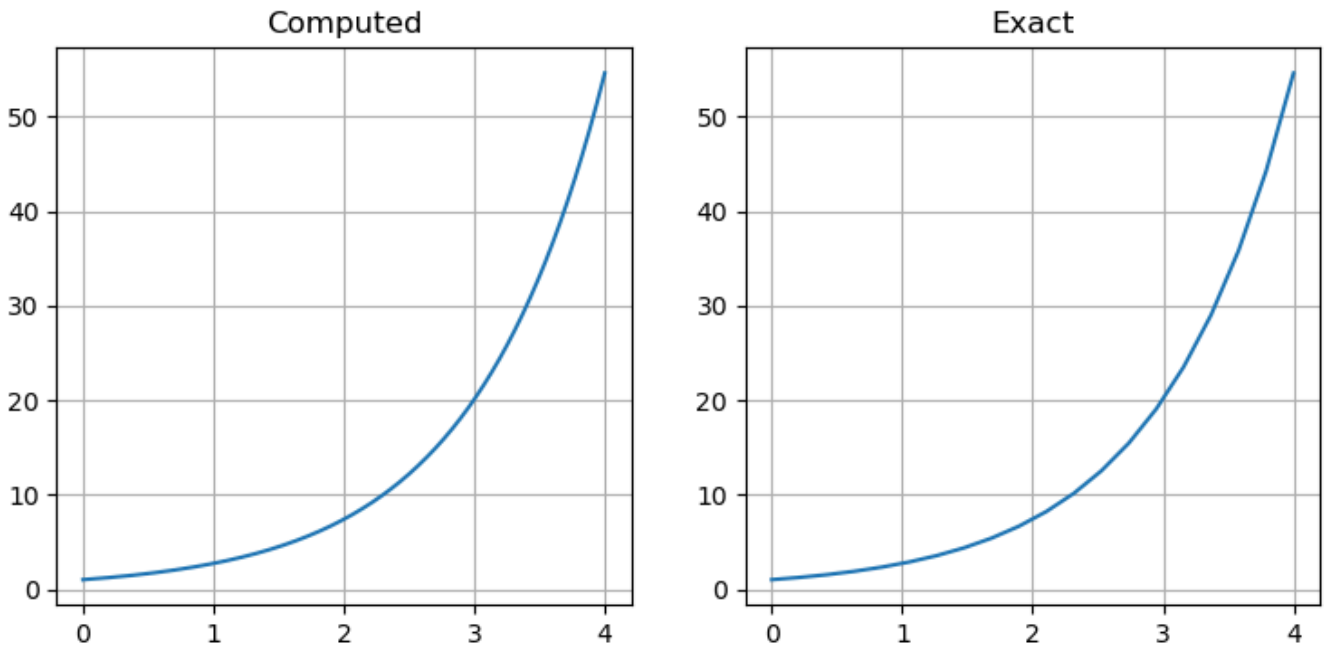
```
def euler(t, tmax, y, dx, step=1.):
    ys = []
    while t < tmax:
        y = y + step*dx(t, y)
        ys.append(y)
        t +=step
    return ys
```

```
def dx(t, y): return y

print(euler(0, 1, 1, dx, step=1.)[-1])
print(euler(0, 2, 1, dx, step=1.)[-1])
```

```
2.0
4.0
```

看起来没问题。那么现在，让我们试着用更小的步长来绘制结果吧。

```
ys = euler(0, 4, 1, dx, step=0.00001)
plt.subplot(1,2,1)
plt.title('Computed')
plt.plot(np.linspace(0, 4, len(ys)),ys)
plt.subplot(1,2,2)
t = np.linspace(0, 4, 20)
plt.title('Exact')
plt.plot(t, np.exp(t));
```




```
print('exact answer=', np.exp(4))
print('euler answer=', ys[-1])
print('difference =', np.exp(4) - ys[-1])
print('iterations =', len(ys))
```

```
exact answer= 54.598150033144236
euler answer= 54.59705808834125
difference = 0.0010919448029866885
iterations = 400000
```

这里的误差已相当小，但为了获得约三位有效数字，需要大量迭代。对于许多实际问题，欧拉法若要达到所需精度，就必须使用很小的步长，计算效率因而偏低，所以通常改用更高阶的方法。

在继续之前，让我们先正式推导一下欧拉方法。因为欧拉方法是后文将要介绍的更复杂的龙格-库塔方法的基础。实际上，欧拉方法就是龙格-库塔方法中最简单的形式。

下面给出 y 在初始时刻的泰勒展开，保留到步长的三次项。$O(h^4)$ 表示所略高阶项的局部余项阶数：

$$y(t_0+h) = y(t_0) + hy'(t_0) + \frac{1}{2!}h^2y''(t_0) + \frac{1}{3!}h^3y'''(t_0) + O(h^4)$$

由此可见，欧拉法只保留泰勒展开的前两项。在解足够光滑且步长足够小时，忽略的高阶项较小，因而可得到局部近似。误差与数值稳定性都依赖具体方程和步长，不能假定高阶项总会逐项减小。

龙格-库塔法

龙格—库塔法是数值积分中广泛使用的一类方法，文献中有许多不同形式。下面介绍的经典四阶方法在速度、精度与稳定性之间取得良好平衡，常作为一般非刚性常微分方程的起点。对于刚性方程等问题，可能需要其他算法。

让我们开始吧。首先从一些微分方程入手。

$$\ddot{y} = \frac{d}{dt}\dot{y}$$

我们可以用函数 f 来表示 y 的导数，具体如下：

$$\ddot{y} = \frac{d}{dt}f(y,t)$$

具体推导超出本书范围，下面直接给出经典四阶龙格—库塔方法（RK4）。在通常光滑性条件下，RK4 的单步局部截断误差为步长的五次阶，累计的全局误差为四次阶：

$$y(t+\Delta t) = y(t) + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) + O(\Delta t^5)$$

$$\begin{aligned}
k_1 &= f(y,t)\Delta t \\
k_2 &= f(y + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)\Delta t \\
k_3 &= f(y + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)\Delta t \\
k_4 &= f(y + k_3, t + \Delta t)\Delta t
\end{aligned}$$

以下是相应的代码：

```
def runge_kutta4(y, x, dx, f):
    """computes 4th order Runge-Kutta for dy/dx.
    y is the initial value for y
    x is the initial value for x
    dx is the difference in x (e.g. the time step)
    f is a callable function (y, x) that you supply
    to compute dy/dx for the specified values.
    """

    k1 = dx * f(y, x)
    k2 = dx * f(y + 0.5*k1, x + 0.5*dx)
    k3 = dx * f(y + 0.5*k2, x + 0.5*dx)
    k4 = dx * f(y + k3, x + dx)

    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.
```

我们用这个来举个简单的例子吧。假设……

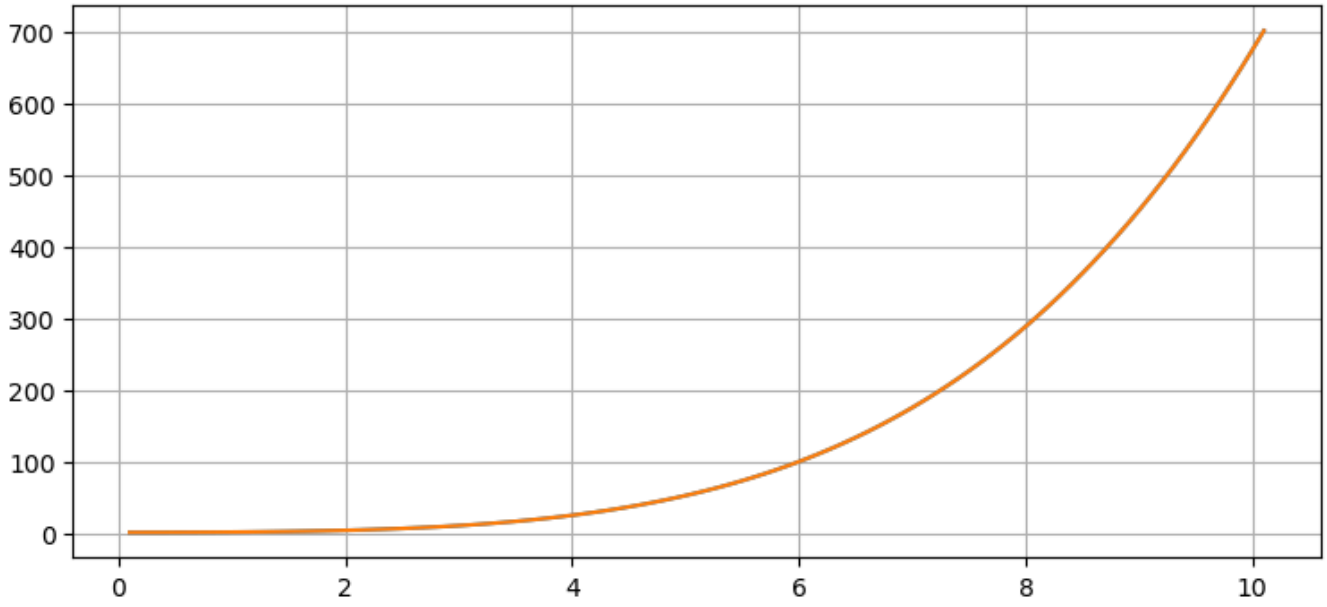
$$\dot{y} = t\sqrt{y(t)}$$

初始条件为：

$$t_0 = 0$$
$$y_0 = y(t_0) = 1$$

```
import math
import numpy as np
t = 0.
y = 1.
dt = .1

ys, ts = [], []

def func(y,t):
    return t*math.sqrt(y)

while t <= 10:
    y = runge_kutta4(y, t, dt, func)
    t += dt
    ys.append(y)
    ts.append(t)

exact = [(t**2 + 4)**2 / 16. for t in ts]
plt.plot(ts, ys)
plt.plot(ts, exact)

error = np.array(exact) - np.array(ys)
print(f"max error {max(error):.5f}")
```

```
max error 0.00005
```



贝叶斯滤波

从“离散贝叶斯”这一章开始，我采用了贝叶斯方法来进行数据处理。假设我们正在追踪某个物体。我们可以将该物体在某一特定时刻的状态定义为它的位置、速度等信息。例如，我们可以将时间点 t 时的物体状态表示为 $\mathbf{x}_t = \begin{bmatrix} x_t & \dot{x}_t \end{bmatrix}^\mathsf{T}$。

测量物体时，传感器测得的是其状态或状态的一部分，并叠加测量噪声。在这里采用的模型中，状态决定无噪声测量：状态变化可能改变测量，而仅仅改变测量读数并不会反过来改变物体的状态。

在滤波处理过程中，我们的目标是计算从时间 0 到时间 t 期间，各状态 $\mathbf{x}_{0:t}$ 的最优估计值。如果我们知道 $\mathbf{x}_{0:t}$ 的值，那么计算出与这些状态相对应的测量值 $\mathbf{z}_{0:t}$ 就很简单了。但实际上，我们只能得到一组测量值 $\mathbf{z}_{0:t}$，而需要计算出与之对应的状态 $\mathbf{x}_{0:t}$。这一过程被称为“统计反演”，因为我们试图从输出结果来反推输入值。

反演通常没有唯一解，因此很困难。给定状态序列 $\mathbf{x}_{0:t}$，测量模型确定了相应的无噪声输出，再叠加随机噪声；但给定一组带噪测量，却可能有许多不同的状态序列与之相容。

回顾贝叶斯定理：

$$P(x \mid z) = \frac{P(z \mid x)P(x)}{P(z)}$$

其中，$P(z \mid x)$ 是给定状态时测量 z 的似然，$P(x)$ 是由过程模型得到的先验分布，$P(z)$ 是归一化常数。$P(x \mid z)$ 是后验分布，即纳入测量 z 这一证据之后的状态分布。

这是统计反演：从给定状态时测量的条件分布 $P(z \mid x)$，转向给定测量时状态的后验分布 $P(x \mid z)$。完整的状态序列估计问题可写成：

$$P(\mathbf{x}_{0:t} \mid \mathbf{z}_{0:t}) = \frac{P(\mathbf{z}_{0:t} \mid \mathbf{x}_{0:t})P(\mathbf{x}_{0:t})}{P(\mathbf{z}_{0:t})}$$

一切都很好，直到下一次测量结果 $\mathbf{z}_{t+1}$ 出来为止。一旦有了新的测量数据，我们就需要重新计算整个区间 $\mathbf{0:}t+1$ 内的数值。

实际中，这样做难以计算，因为我们试图获得整个时间范围内的状态后验分布 $P(\mathbf{x}_{0:t} \mid \mathbf{z}_{0:t})$。但刚收到第十次测量时，通常真的还需要第三步的概率分布吗？一般不需要。因此，放宽要求，只计算当前时间步的分布。

第一个简化步骤是：我们将所描述的过程（比如，运动物体的运动模型）视为一个马尔可夫链。也就是说，当前状态完全取决于上一个状态以及一个转移概率 $P(\mathbf{x}_k \mid \mathbf{x}_{k-1})$，该转移概率指的是从上一个状态转移到当前状态的概率。我们将其表示为：

$$\mathbf{x}_k \sim P(\mathbf{x}_k \mid \mathbf{x}_{k-1})$$

对于许多问题，马尔可夫假设相当合理。例如，在停车场驾车时，若状态已充分包含当前位置、速度以及影响下一步运动的必要信息，那么预测下一秒通常不必再追溯一分钟前是在高速公路还是土路上。马尔可夫性取决于状态定义和模型假设；若遗漏了相关内部状态，历史仍可能提供额外信息。

我们接下来的简化步骤是：将测量模型定义为取决于当前状态 $\mathbf{x}_k$，同时考虑在给定当前状态的情况下进行测量的条件概率 $P(\mathbf{z}_k \mid \mathbf{x}_k)$。我们表示为：

$$\mathbf{z}_k \sim P(\mathbf{z}_k \mid \mathbf{x}_k)$$

现在已经得到递推关系，还需要一个初始条件作为递推起点。因此，将初始分布定义为初始状态 $\mathbf{x_0}$ 的概率分布：

$$\mathbf{x}_0 \sim P(\mathbf{x}_0)$$

将这些模型与贝叶斯公式结合，就能构造递推滤波。由初始状态 $\mathbf{x_0}$ 的分布出发，结合第一次测量得到后验 $P(\mathbf{x_1}|\mathbf{z_1})$。下一步使用状态转移模型 $P(\mathbf{x_2} \mid \mathbf{x_1})$，对上一后验传播并边缘化，得到新的先验，再结合新测量得到后验 $P(\mathbf{x_2}|\mathbf{z}_{1:2})$。要计算第 t 步的状态分布，只需第 $t-1$ 步的后验和第 t 步的新测量：历史测量所含的信息已通过上一后验传递过来。预测先验必须对上一状态分布积分或求和，不能直接以状态转移条件分布替代。

这些计算的具体数学形式取决于问题。“离散贝叶斯滤波”和“单变量卡尔曼滤波”两章给出了两种不同的形式，你应已能理解其推理。单变量卡尔曼滤波针对标量状态，假设过程模型和测量模型为线性模型，过程噪声与测量噪声都是零均值、彼此不相关的高斯噪声。

多元卡尔曼滤波器也基于同样的假设，只不过这里的“状态”和“测量值”都是向量，而非标量。卡尔曼博士证明了：只要这些假设成立，卡尔曼滤波器就具有最小二乘意义上的最优性。通俗来说，这意味着我们无法从这些有噪声的测量数据中获取更多信息。在本书的后续部分，我将介绍那些不那么严格限制线性条件和高斯噪声假设的滤波器。

在继续之前，我想再谈谈关于统计反演的问题。正如 Calvetti 和 Somersalo 在《贝叶斯科学计算导论》中所写的那样：“我们采用贝叶斯视角：所谓‘随机性’，其实只是意味着信息的缺失而已”[3]。状态参数用来描述那些原则上可以被测量或计算出来的物理现象：比如速度、空气阻力等等。由于我们缺乏足够的信息来

精确计算或测量这些数值，因此我们把它们视为随机变量。严格来说，它们并非真正的随机变量，所以这种看法其实是一种主观判断。

他们用一整章讨论这个主题，我这里只能用一段概述。贝叶斯滤波之所以可行，是因为我们用随机变量表达信息的不确定性，以概率分布描述这些变量，再用贝叶斯定理进行统计推断。线性、高斯条件下，卡尔曼滤波可以得到最优估计的闭式解；在给定离散状态模型中，精确的贝叶斯递推也可得到该模型下的正确后验。后面将介绍的粒子滤波则通常用有限样本近似后验。

将卡尔曼滤波器转换为 g-h 滤波器

我之前说过，卡尔曼滤波其实是一种 g-h 滤波的特例。只需通过一些代数运算就能证明这一点。在一维情况下，这种证明更为简单，所以我将用一维情况来举例说明。请回忆一下。

$$\mu_x = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}$$

为便于阅读，先用更简洁的符号重写：

$$\mu_x = \frac{ya + xb}{a + b}$$

我们可以通过以下代数运算，轻松将其转化为 g-h 形式。

$$\begin{aligned}
\mu_x &= (x - x) + \frac{ya + xb}{a + b} \\
\mu_x &= x - \frac{a + b}{a + b}x + \frac{ya + xb}{a + b} \\
\mu_x &= x + \frac{-x(a + b) + xb + ya}{a + b} \\
\mu_x &= x + \frac{-xa + ya}{a + b} \\
\mu_x &= x + \frac{a}{a + b}(y - x)
\end{aligned}$$

我们差不多完成了，但请记住，估计值的方差是由以下公式给出的。

$$\begin{aligned}
\sigma_x^2 &= \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}} \\
&= \frac{1}{\frac{1}{a} + \frac{1}{b}}
\end{aligned}$$

注意到下面的恒等关系，就可以把这个方差项代入上面的均值更新式：

$$
\begin{aligned}
\frac{a}{a+b} &= \frac{a/a}{(a+b)/a} = \frac{1}{(a+b)/a} \\
&= \frac{1}{1+\frac{b}{a}} = \frac{1}{\frac{b}{b}+\frac{b}{a}} \\
&= \frac{1}{b}\frac{1}{\frac{1}{b}+\frac{1}{a}} \\
&= \frac{\sigma_x^2}{b}
\end{aligned}
$$

我们可以把这一切联系起来。

$$
\begin{aligned}
\mu_x &= x + \frac{a}{a+b}(y-x) \\
&= x + \frac{\sigma_x^2}{b}(y-x) \\
&= x + g_n(y-x)
\end{aligned}
$$

其中

$$
g_n = \frac{\sigma_x^2}{\sigma_y^2}
$$

最终，我们将两个输入之间的残差乘以增益，再加到原先的值上，这就是 g-h 滤波器中 g 的更新形式。g 等于新估计方差与测量方差之比。当然，这里的 g 并不是固定常数，因为方差会随时间步变化。也可以用类似方法推导 h；该推导对理解并无太大帮助，这里略去。最终结果为：

$$
h_n = \frac{COV(x, \dot{x})}{\sigma_y^2}
$$

关键在于： g 和 h 的值完全由在时间点 n 时的测量值和预测值的方差及协方差来决定。换句话说，我们是通过一个由这两个输入数据的质量所决定的比例因子，来在测量值和预测值之间选择一个点。

参考文献

- [1] C. B. Moler and C. F. Van Loan. “Nineteen Dubious Ways to Compute the Exponential of a Matrix, Twenty-Five Years Later.” SIAM Review, 45: 3–49, 2003.
- [2] C.F. van Loan,《涉及矩阵指数的积分计算》,《IEEE 自动控制汇刊》，1978 年 6 月。
- [3] Calvetti, D.和 Somersalo, E.,《贝叶斯科学计算导论：关于主观计算的十讲》，施普林格出版社，2007 年。
- [4] Brown, R. G.和 Hwang, P. Y.C.著,《随机信号与卡尔曼滤波应用导论》，Wiley and Sons 出版社，第四版，第 143-147 页，2012 年。

第 8 章：卡尔曼滤波器的设计

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

上一章处理的是教科书式的问题：容易描述，只需几行代码即可实现，也便于讲解。现实问题很少如此简单。本章将研究更贴近实际的例子，并学习如何评价滤波性能。

先跟踪二维空间中的机器人，例如田野或仓库里的机器人。最初只使用一个带噪传感器，它输出有噪声的 (x, y) 坐标；经滤波得到二维轨迹。掌握这一概念后，再扩展问题，增加传感器并引入控制输入。

接下来将转向非线性问题。现实世界具有非线性，而这里的卡尔曼滤波器是线性的。对轻微非线性的问题，有时仍能使用它，有时却不行。我会分别举例，为本书后续讨论非线性问题的处理方法作准备。

追踪机器人

第一次尝试跟踪机器人时，与前面的一维狗跟踪问题非常相似。不同的是，传感器不再给出走廊中的一维位置，而是给出二维空间中的带噪位置测量。每个时刻 t 都提供一个 (x, y) 坐标对，表示机器人在场地中的带噪位置读数。

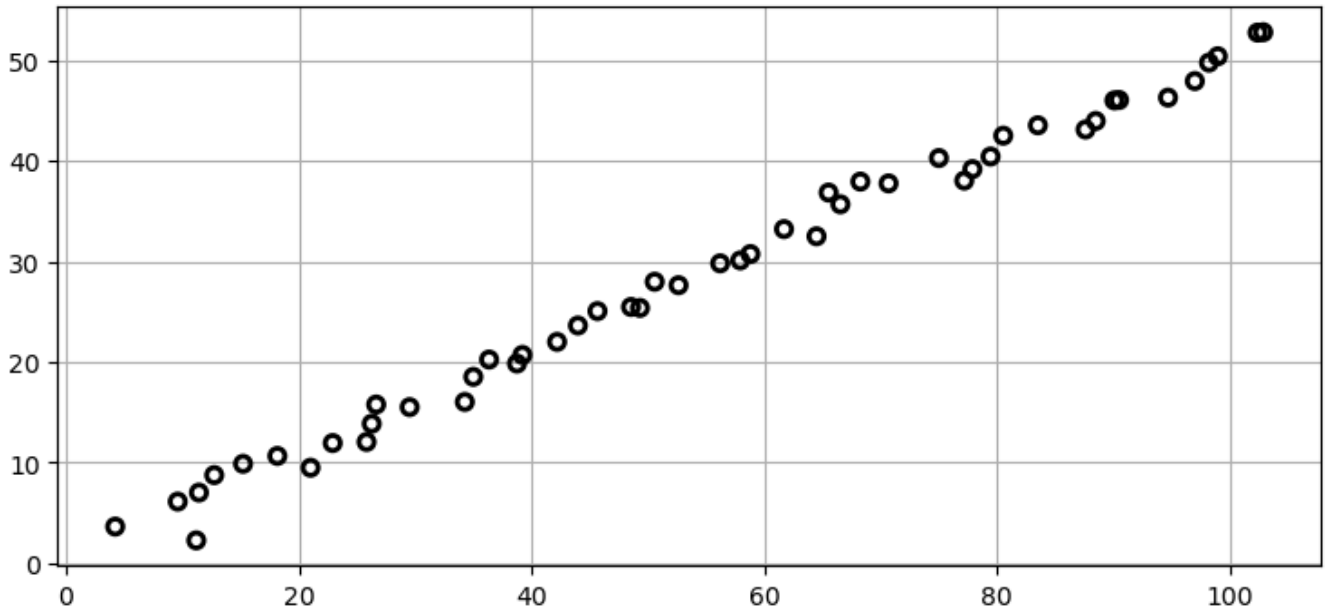
编写与真实传感器进行交互的代码超出了本书的讨论范围。因此，我们仍然会像之前那样，对传感器进行简单的模拟。在开发过程中，我们会逐步创建出更复杂的传感器模型。在编写代码时，我会在函数名称后加上相应的数字来区分不同的传感器。

那么，让我们从一种非常简单的传感器开始吧。这种传感器用于模拟对沿直线运动的物体的跟踪。在初始化时，需要指定物体的初始位置、速度以及噪声的标准差。每次调用 `read()` 时，该传感器都会将物体的位置更新一个时间步长，并返回新的测量结果。

```
from numpy.random import randn

class PosSensor(object):
    def __init__(self, pos=(0, 0), vel=(0, 0), noise_std=1.):
        self.vel = vel
        self.noise_std = noise_std
        self.pos = [pos[0], pos[1]]

    def read(self):
        self.pos[0] += self.vel[0]
        self.pos[1] += self.vel[1]

        return [self.pos[0] + randn() * self.noise_std,
                self.pos[1] + randn() * self.noise_std]
```

进行一次快速测试，以确认其功能符合我们的预期。

```
import matplotlib.pyplot as plt
import numpy as np
from kf_book.book_plots import plot_measurements

pos, vel = (4, 3), (2, 1)
sensor = PosSensor(pos, vel, noise_std=1)
ps = np.array([sensor.read() for _ in range(50)])
plot_measurements(ps[:, 0], ps[:, 1]);
```



看起来没错。斜率确实是 1/2，这与速度为(2, 1)的情况相符。数据似乎是从接近(6, 4)的位置开始的。不过，这种描述看起来不太符合现实情况。这仍然只是“教科书式”的描述而已。在后续的分析中，我们会加入一些复杂的因素，以更真实地反映实际情况。

选择状态变量

和往常一样，第一步是选择状态变量。我们在二维空间中跟踪目标，传感器能够同时给出两个维度的读数，因此有两个观测变量 $\boldsymbol{x}$ 和 $\boldsymbol{y}$。如果只用它们构建滤波器，就忽略了速度所能提供的信息，性能不会很好。因此，也要把速度纳入状态，写作：

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} \end{bmatrix}^\mathsf{T}$$

这种排列方式并无特殊之处，也可以使用 $\begin{bmatrix} x & y & \dot{x} & \dot{y} \end{bmatrix}^\mathsf{T}$，甚至采用不那么直观的顺序；只要其余矩阵都采用一致的状态排列即可。我喜欢把每个方向的位置与速度放在一起，这样二者的协方差就集中在协方差矩阵的同一个子块中。在这里的排列下，P[1, 0] 是 x 与 $\dot{x}$ 的协方差；若采用另一种排列，这个协方差就位于 P[2, 0]。维度越多，排列带来的差异越明显。

先讨论怎样识别需要估计的隐状态。本例已有一维跟踪的经验，比较明显；其他问题则未必有简单答案。首先问自己：传感器数据的一阶、二阶导数分别有什么物理意义？固定时间步长下，可以用相邻读数的差除以时间间隔，近似一阶导数，再对一阶导数作同样运算，近似二阶导数。在跟踪问题中，相邻位置之差是位移，除以时间间隔才是平均速度。

除此之外，你还可以考虑如何将来自两个或更多不同传感器的数据结合起来，从而获得更多有用的信息。这就引出了“传感器融合”的概念，我们将在后面的章节中详细介绍相关案例。目前需要明白的是，选择合适的状态变量是确保滤波器能够发挥最佳性能的关键。一旦确定了隐变量，就必须进行大量测试，以确保所获得的结果是可靠的。卡尔曼滤波器会按照你指定的模型来运行；如果你的模型无法为隐变量提供准确的信息，那么卡尔曼滤波器的输出结果将毫无意义。

设计状态转移函数

我们的下一步是设计状态转移函数。请记住，状态转移函数实际上是一个矩阵 $\mathbf{F}$。我们将该矩阵与系统的当前状态相乘，从而得到下一个状态。

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$$

我就不再详细阐述了，因为这与我们在前一章中讨论的一维情况非常相似。状态转移方程如下：

$$\begin{aligned}
x &= 1x + \Delta t\dot{x} + 0y + 0\dot{y} \\
v_x &= 0x + 1\dot{x} + 0y + 0\dot{y} \\
y &= 0x + 0\dot{x} + 1y + \Delta t\dot{y} \\
v_y &= 0x + 0\dot{x} + 0y + 1\dot{y}
\end{aligned}$$

这样的表述方式让我们清楚地了解了 $\mathbf{F}$ 所要求的数值结构以及行与列的排列方式。我们可以将其转化为矩阵-向量形式来表示：

$$\begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix}$$

下面用 Python 实现。代码很简单，新增的一点是将 dim_z 设为 2；为什么这样设置，会在第 4 步说明。

```
from filterpy.kalman import KalmanFilter

tracker = KalmanFilter(dim_x=4, dim_z=2)
dt = 1.   # time step 1 second

tracker.F = np.array([[1, dt, 0,  0],
                      [0,  1, 0,  0],
                      [0,  0, 1, dt],
                      [0,  0, 0,  1]])
```

设计过程噪声矩阵

FilterPy 可以计算 $\mathbf{Q}$。为简化问题，这里采用分段恒定、不同时间段之间互不相关的白噪声扰动模型，用方差描述相邻时间步之间可能出现的运动变化。如有疑问，可回顾“卡尔曼滤波的数学原理”一章。

```
from scipy.linalg import block_diag
from filterpy.common import Q_discrete_white_noise

q = Q_discrete_white_noise(dim=2, dt=dt, var=0.001)
tracker.Q = block_diag(q, q)
print(tracker.Q)
```

```
[[0.    0.001 0.    0.   ]
 [0.001 0.001 0.    0.   ]
 [0.    0.    0.    0.001]
 [0.    0.    0.001 0.001]]
```

在这里，我假设 x 和 y 中的噪声是相互独立的。因此，任意 x 和 y 变量之间的协方差都应该为零。这样一来，我就可以先计算出一维情况下的 $\mathbf{Q}$ 值，然后再利用 `block_diag` 将该值分别应用到 x 轴和 y 轴上。

设计控制函数

尚未给机器人引入控制输入，因此这一步无需额外设置。KalmanFilter 类的 B 默认是 None；未提供控制输入时，不会添加控制作用，可以保持 tracker.B 未设置的状态。

```
tracker.B
```

设计测量函数

测量函数 $\mathbf{H}$ 通过方程 $\mathbf{z} = \mathbf{Hx}$，定义如何把状态映射为测量。本例测量平面位置，因此将 $\mathbf{z}$ 定义为 $[x \quad y]^\mathsf{T}$，维数为 2×1；状态向量的维数为 4×1。由矩阵乘法的维数规则：M×N 矩阵乘以 N×P 矩阵，结果为 M×P 矩阵，可以确定 $\mathbf{H}$ 的维数：

$$(2 \times 1) = (a \times b)(4 \times 1) = (2 \times 4)(4 \times 1)$$

因此，$\mathbf{H}$ 的维数为 2×4。

确定 $\mathbf{H}$ 的元素很容易：测量量是机器人的位置，即状态 $\mathbf{x}$ 中的 x 和 y。下面增加一个单位转换：传感器以英尺报告位置，而状态希望使用米。$\mathbf{H}$ 将状态映射到测量空间，因此应按 $\mathbf{feet} = \mathbf{meters}/0.3048$ 将米转换为英尺。于是得到：

$$\mathbf{H} = \begin{bmatrix} \frac{1}{0.3048} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{0.3048} & 0 \end{bmatrix}$$

这对应于这些线性方程。

$$z_x = (\frac{x}{0.3048}) + (0 * v_x) + (0 * y) + (0 * v_y) = \frac{x}{0.3048}$$
$$z_y = (0 * x) + (0 * v_x) + (\frac{y}{0.3048}) + (0 * v_y) = \frac{y}{0.3048}$$

这是个简单的问题，我们本可以直接得出方程式，而不必进行我上面所进行的那些维度分析。不过，记住卡尔曼滤波器的方程式决定了所有矩阵都必须具有特定的维度这一点是很重要的。当我在设计某个系统时遇到困难时，查看一下各矩阵的维度往往能提供解题的线索。

以下是我的实现方式：

```
tracker.H = np.array([[1/0.3048, 0, 0,        0],
                      [0,        0, 1/0.3048, 0]])
```

设计测量噪声矩阵

假设 x 和 y 方向的测量噪声是相互独立的零均值高斯白噪声。下面统一采用米作为位置测量的单位，将 x 和 y 方向的噪声方差都设为 5 m^2。两方向噪声独立，因此互协方差及非对角元均为零。若传感器以英尺报告测量，则须同步换算测量矩阵和噪声协方差，使 R 的单位与测量向量一致。因此有：

$$\mathbf{R} = \begin{bmatrix} \sigma_x^2 & \sigma_y\sigma_x \\ \sigma_x\sigma_y & \sigma_y^2 \end{bmatrix} = \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$$

这是一个 $2{\times}2$ 矩阵，因为我们有两个传感器输入。对于 n 个变量来说，协方差矩阵的大小始终是 $n{\times}n$。在 Python 中，我们这样表示：

```
tracker.R = np.array([[5., 0],
                      [0, 5]])
tracker.R
```

```
array([[5., 0.],
       [0., 5.]])
```

初始条件

对于这个简单的问题，我们将把初始位置设定为(0,0)，初始速度也设为(0,0)。由于这只是一个初步的估计，我们会将协方差矩阵 $\mathbf{P}$ 的值设得很大。

$$\mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 500 & 0 & 0 & 0 \\ 0 & 500 & 0 & 0 \\ 0 & 0 & 500 & 0 \\ 0 & 0 & 0 & 500 \end{bmatrix}$$

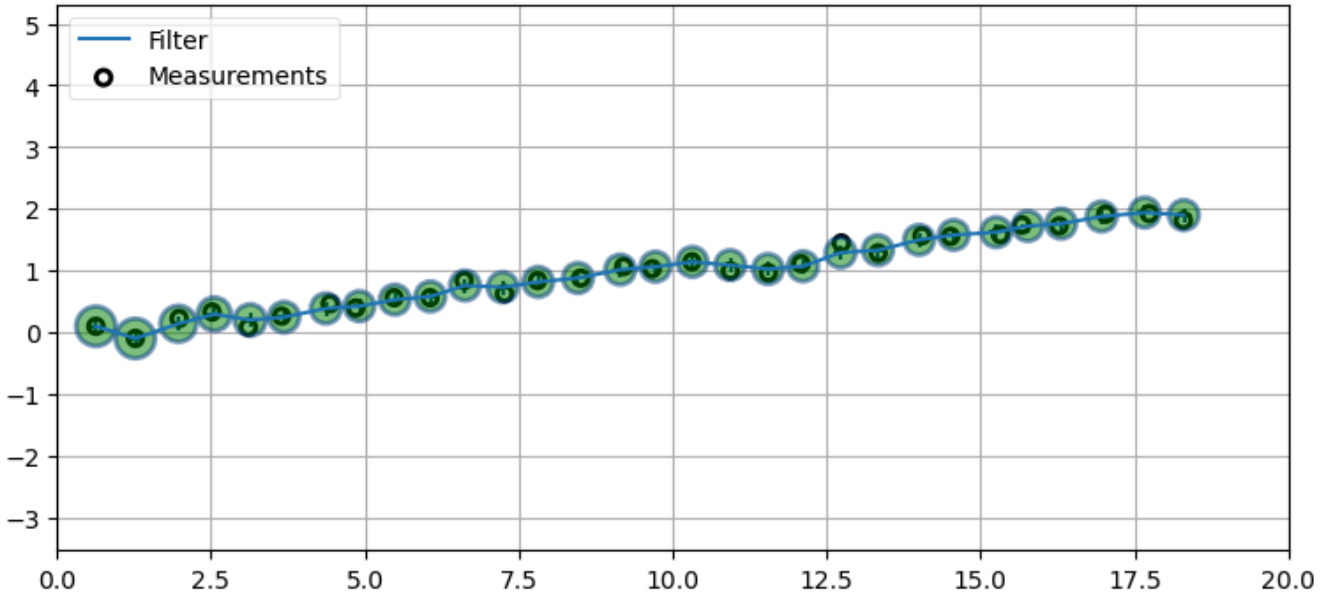
Python 版本的实现方式是

```
tracker.x = np.array([[0, 0, 0, 0]]).T
tracker.P = np.eye(4) * 500.
```

实现滤波功能

设计已经完成，现在只需编写代码来运行该滤波器，并将处理后的数据以我们希望的格式输出出来。我们将让代码运行 30 次。

```
from filterpy.stats import plot_covariance_ellipse
from kf_book.book_plots import plot_filter

R_std = 0.35
Q_std = 0.04

def tracker1():
    tracker = KalmanFilter(dim_x=4, dim_z=2)
    dt = 1.0   # time step

    tracker.F = np.array([[1, dt, 0,  0],
                          [0,  1, 0,  0],
                          [0,  0, 1, dt],
                          [0,  0, 0,  1]])
    tracker.u = 0.
    tracker.H = np.array([[1/0.3048, 0, 0, 0],
                          [0, 0, 1/0.3048, 0]])

    tracker.R = np.eye(2) * R_std**2
    q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_std**2)
    tracker.Q = block_diag(q, q)
    tracker.x = np.array([[0, 0, 0, 0]]).T
    tracker.P = np.eye(4) * 500.
    return tracker

# simulate robot movement
N = 30
sensor = PosSensor((0, 0), (2, .2), noise_std=R_std)

zs = np.array([sensor.read() for _ in range(N)])

# run filter
robot_tracker = tracker1()
mu, cov, _, _ = robot_tracker.batch_filter(zs)

for x, P in zip(mu, cov):
    # covariance of x and y
    cov = np.array([[P[0, 0], P[2, 0]],
                    [P[0, 2], P[2, 2]]])
    mean = (x[0, 0], x[2, 0])
    plot_covariance_ellipse(mean, cov=cov, fc='g', std=3, alpha=0.5)

#plot results
zs *= .3048 # convert to meters
plot_filter(mu[:, 0], mu[:, 2])
plot_measurements(zs[:, 0], zs[:, 1])
plt.legend(loc=2)
plt.xlim(0, 20);
```



建议你尝试为 $\mathbf{Q}$ 和 $\mathbf{R}$ 设置不同的值。前几章已经做过不少这类练习，而待讲内容还很多，因此下面转向更复杂的例子，届时仍有机会观察调参的效果。

图中用绿色绘制了 $\boldsymbol{x}$ 与 $\boldsymbol{y}$ 的 $3\boldsymbol{\sigma}$ 协方差椭圆。能解释它们的形状吗？你或许期待像前几章那样看到倾斜的椭圆。不过，前面画的不是 $\boldsymbol{x}$ 与 $\boldsymbol{y}$，而是 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$。$\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 相关，但本例假设 $\boldsymbol{x}$ 与 $\boldsymbol{y}$ 相互独立，因此椭圆的主轴与坐标轴平行。又因为 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 方向的噪声标准差设置相同，所以此处椭圆呈圆形。若将 R 改为下式：

$$\mathbf{R} = \begin{bmatrix} 1 & 0 \\ 0 & .5 \end{bmatrix}$$

我们会告诉卡尔曼滤波器：在 $\boldsymbol{x}$ 处，噪声比在 $\boldsymbol{y}$ 处要大；同时，我们所绘制的椭圆形状也会更长，高度则相对较低。

$\mathbf{P}$ 的最终数值能告诉我们关于各状态变量之间相关性的所有必要信息。如果我们只看对角线上的数值，就能得知每个变量的方差。换句话说，$\mathbf{P}_{0,0}$ 表示变量 x 的方差，$\mathbf{P}_{1,1}$ 表示 $\dot{\boldsymbol{x}}$ 的方差，$\mathbf{P}_{2,2}$ 表示变量 y 的方差，而 $\mathbf{P}_{3,3}$ 则表示 $\dot{\boldsymbol{y}}$ 的方差。我们可以使用 `numpy.diag()` 来提取矩阵的对角线数值。

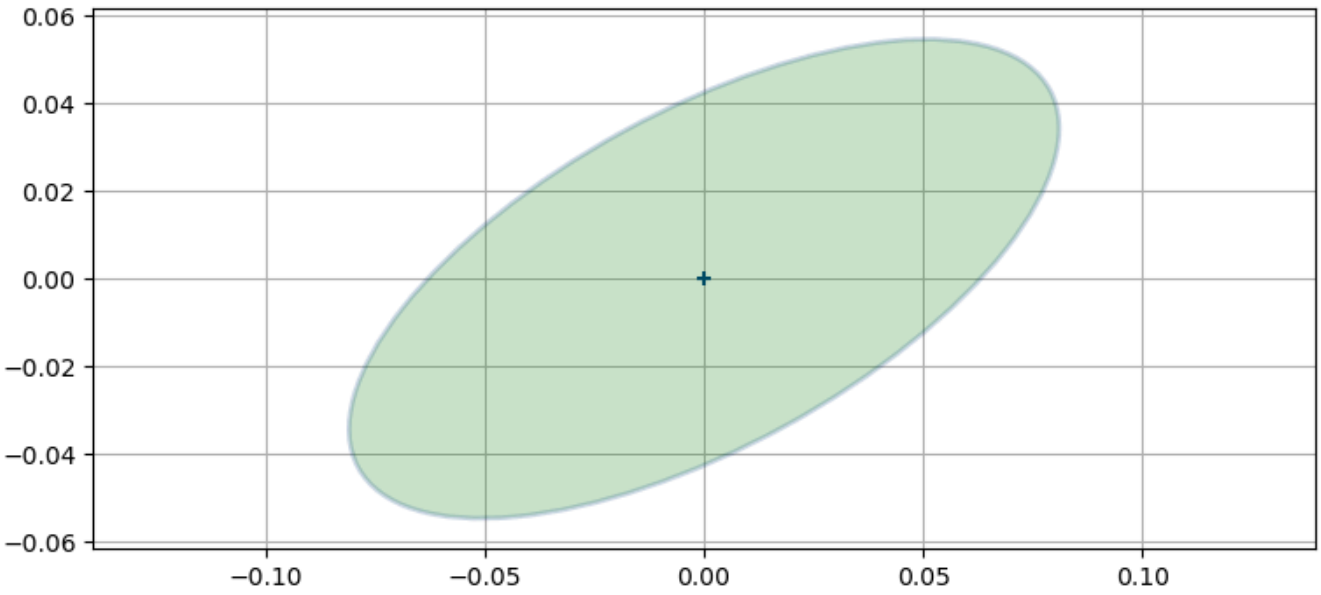
```
print(np.diag(robot_tracker.P))
```

```
[0.007 0.003 0.007 0.003]
```

该协方差矩阵包含四个 $\mathbf{2{\times}2}$ 矩阵，你应该能够轻松识别出它们。这是因为 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 之间存在相关性，同时 $\boldsymbol{y}$ 与 $\dot{\boldsymbol{y}}$ 之间也存在相关性。矩阵的左上角部分表示的是 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 之间的协方差。

```
c = robot_tracker.P[0:2, 0:2]
print(c)
plot_covariance_ellipse((0, 0), cov=c, fc='g', alpha=0.2)
```

```
[[0.007 0.003]
 [0.003 0.003]]
```



按照这里的状态排列，协方差矩阵左上角包含 $\boldsymbol{x}$ 与 $\dot{\boldsymbol{x}}$ 对应的子块。$\mathbf{P}_{i,j}$ 与 $\mathbf{P}_{j,i}$ 是相应状态分量的协方差，二者相等，均可写为 $\rho_{ij}\sigma_i\sigma_j$，即相关系数与两个标准差的乘积。

最后查看 $\mathbf{P}$ 的左下子块，其元素均为零。以 $\mathbf{P}_{3,0}$ 为例，它可写为 $\rho_{3,0}\sigma_3\sigma_0$，表示 $\dot{\boldsymbol{y}}$ 与 $\boldsymbol{x}$ 的协方差。本例假设这两个方向的运动独立，因此相关系数及协方差为零。其他元素同样对应独立变量之间的协方差。

```
robot_tracker.P[2:4, 0:2]
```

```
array([[0., 0.],
       [0., 0.]])
```

滤波器阶次

此前只研究了位置与速度的跟踪，效果很好，但那是因为选择的问题适合这样的模型。现在你已具备足够的卡尔曼滤波经验，可以从更一般的角度考虑这个问题。

这里所说的阶次，是指运动模型中需要保留到哪一阶导数。例如建筑物高度不变，不必在状态中加入导数，因此本书称其为零阶模型，可写作 $\boldsymbol{x} = 312.5$。

本书所称的一阶运动模型保留到一阶导数。例如，位置对时间的一阶导数是速度，可写作：

$$\boldsymbol{v} = \frac{\boldsymbol{dx}}{\boldsymbol{dt}}$$

积分后得到牛顿运动方程：

$$\boldsymbol{x} = \boldsymbol{vt} + \boldsymbol{x_0}.$$

这也被称为“恒速模型”，因为该模型假设速度是恒定的。

二阶系统具有二阶导数。位置的二阶导数就是加速度，其计算公式为：

$$a = \frac{d^2x}{dt^2}$$

积分后得到：

$$x = \frac{1}{2}at^2 + v_0t + x_0.$$

这也被称为恒定加速度模型。

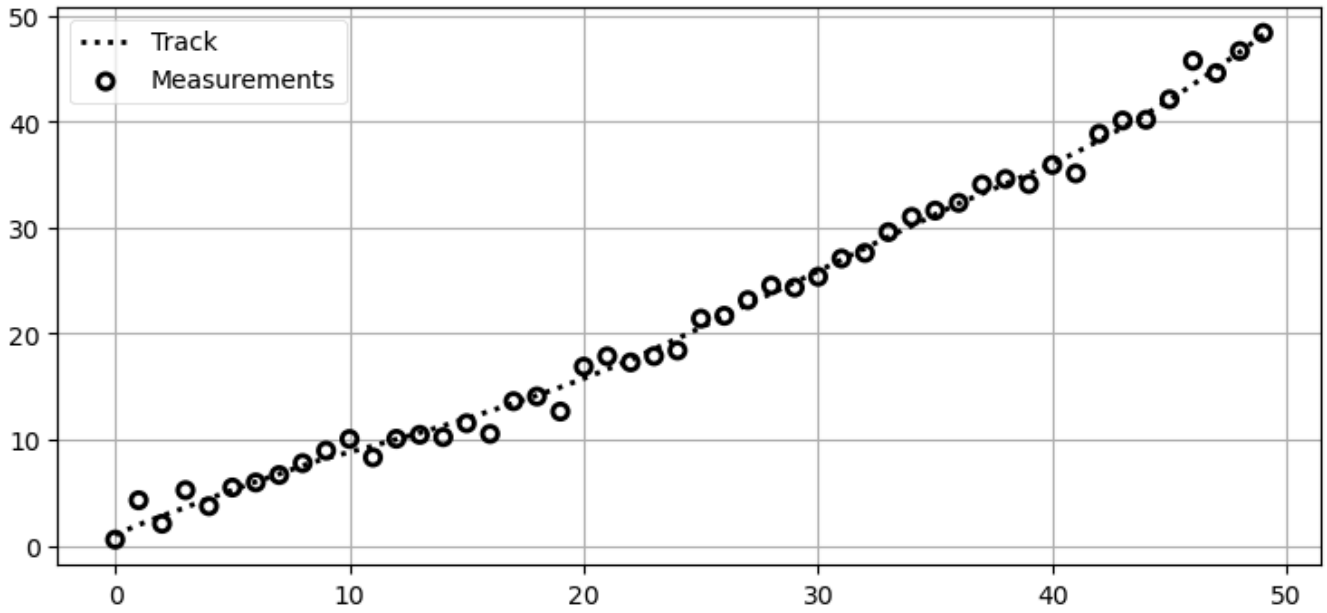
也可以从轨迹多项式的次数理解本章的阶次约定。恒加速度模型保留位置的二阶导数，因此称为二阶模型；相应的轨迹多项式 $x = \frac{1}{2}at^2 + v_0t + x_0$ 也是二次多项式。

设计状态变量和过程模型时，必须选择模型的阶次。假设目标大体以恒定速度运动。真实过程并不完美，速度在短时间内仍会轻微变化。你可能因此推想：最好使用二阶滤波器，让加速度项处理这些速度变化。

实际中，这样做未必好。为深入理解这一点，下面观察过程模型阶次与被滤波系统不匹配时的效果。

首先，我们需要一个用于滤波数据的系统。我会编写一个类来模拟那些具有恒定速度的物体。实际上，没有任何物理系统能够真正保持恒定速度，因此在每次更新时，我们都会让物体的速度发生微小的变化。此外，我还编写了一个“传感器”类，用来模拟传感器产生的高斯噪声。代码如下所示。我还绘制了测试结果，以确认该系统的运行是否正常。

```
from kf_book.book_plots import plot_track

class ConstantVelocityObject(object):
    def __init__(self, x0=0, vel=1., noise_scale=0.06):
        self.x = x0
        self.vel = vel
        self.noise_scale = noise_scale

    def update(self):
        self.vel += randn() * self.noise_scale
        self.x += self.vel
        return (self.x, self.vel)

def sense(x, noise_scale=1.):
    return x[0] + randn()*noise_scale

np.random.seed(124)
obj = ConstantVelocityObject()

xs, zs = [], []
for i in range(50):
    x = obj.update()
    z = sense(x)
    xs.append(x)
    zs.append(z)

xs = np.asarray(xs)

plot_track(xs[:, 0])
plot_measurements(range(len(zs)), zs)
plt.legend(loc='best');
```



我对这个模型很满意。由于我们在系统中加入了随机噪声，因此这条轨迹并不完全呈直线。这可能相当于一个人在街道上行走的轨迹，或者是一架飞机在多变的风力作用下飞行的轨迹。这里不存在有意的加速度，因此我们可以将其视为一个匀速运动系统。你可能会问：既然实际上存在微小的加速度，那为什么不用二阶卡尔曼滤波器来处理这些变化呢？让我们来了解一下吧。

怎样设计零阶、一阶或二阶卡尔曼滤波器？其实前面一直在做这些事，只是没有使用这些名称。下面逐一完整说明，过程可能略显繁琐；若概念已很清楚，可以略读。

零阶卡尔曼滤波器

零阶卡尔曼滤波器的状态中不包含位置的导数。本例跟踪位置，因此状态只有位置分量，没有速度或加速度；状态转移也只描述位置。矩阵形式为：

$$\mathbf{x} = [x]$$

状态转移函数非常简单。由于位置不会发生变化，因此我们只需对 $\boldsymbol{x} = \boldsymbol{x}$ 进行建模即可；换句话说，时间 t+1 时的 x 值与时间 t 时的 x 值相同。用矩阵形式表示，状态转移函数为：

$$\mathbf{F} = [1]$$

测量函数也很简单。它需要定义如何将状态变量 $\mathbf{x}$ 转换为测量值。假设传感器测量位置，而状态中也只有位置，就得到：

$$\mathbf{H} = [1]$$

让我们编写一个函数，用于构建并返回一个零阶卡尔曼滤波器。

```
def ZeroOrderKF(R, Q, P=20):
    """ Create zero order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=1, dim_z=1)
    kf.x = np.array([0.])
    kf.R *= R
    kf.Q *= Q
    kf.P *= P
    kf.F = np.eye(1)
    kf.H = np.eye(1)
    return kf
```

一阶卡尔曼滤波器

一阶卡尔曼滤波器用于追踪一阶系统，比如位置和速度等参数。在上面关于狗的追踪问题中，我们已经应用了这一方法，所以应该很容易理解。不过，我们还是再演示一遍吧。

一阶系统包含位置和速度这两个参数，因此状态变量必须同时包含这两个量。其矩阵表示形式可以如下：

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

现在，我们必须来设计状态转移的规则了。在某个时间步长内，牛顿方程如下：

$$\begin{aligned} x_t &= x_{t-1} + v\Delta t \\ v_t &= v_{t-1} \end{aligned}$$

请记住，我们需要将其转化为线性方程。

$$\begin{bmatrix} x \\ \dot{x} \end{bmatrix} = \mathbf{F} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

设置

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

从而得到了上述方程式。

最后设计测量函数，使它实现下面的映射：

$$\mathbf{z} = \mathbf{H}\mathbf{x}$$

传感器仍然只测量位置，因此测量函数只提取状态中的位置分量，将速度分量的系数设为零，如下所示。这里并不把速度状态本身置零。

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

该函数用于构建并返回一阶卡尔曼滤波器。

```python
def FirstOrderKF(R, Q, dt):
    """ Create first order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.x = np.zeros(2)
    kf.P *= np.array([[100, 0], [0, 1]])
    kf.R *= R
    kf.Q = Q_discrete_white_noise(2, dt, Q)
    kf.F = np.array([[1., dt],
                     [0., 1]])
    kf.H = np.array([[1., 0]])
    return kf
```

二阶卡尔曼滤波器

二阶卡尔曼滤波器用于跟踪二阶系统，比如位置、速度和加速度等参数。其状态变量即为上述参数。

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix}$$

现在，我们必须来设计状态转移的规则了。在某个时间步长内，牛顿方程如下：

$$\begin{aligned} x_t &= x_{t-1} + v_{t-1}\Delta t + 0.5a_{t-1}\Delta t^2 \\ v_t &= v_{t-1} + a_{t-1}\Delta t \\ a_t &= a_{t-1} \end{aligned}$$

请记住，我们需要将其转化为线性方程。

$$\begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} = \mathbf{F} \begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix}$$

设置

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & .5\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

从而得到了上述方程式。

最后设计测量函数，使它实现下面的映射：

$$z = \mathbf{H}\mathbf{x}$$

传感器仍然只测量位置，因此测量函数只提取位置分量，速度与加速度分量的系数都设为零，如下所示。状态中的速度与加速度估计本身仍保留。

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

该函数用于构建并返回一个二阶卡尔曼滤波器。

```
def SecondOrderKF(R_std, Q, dt, P=100):
    """ Create second order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=3, dim_z=1)
    kf.x = np.zeros(3)
    kf.P[0, 0] = P
    kf.P[1, 1] = 1
    kf.P[2, 2] = 1
    kf.R *= R_std**2
    kf.Q = Q_discrete_white_noise(3, dt, Q)
    kf.F = np.array([[1., dt, .5*dt*dt],
                     [0., 1.,       dt],
                     [0., 0.,       1.]])
    kf.H = np.array([[1., 0., 0.]])
    return kf
```

评估滤波器的阶数

现在，我们可以将每个卡尔曼滤波器应用于模拟结果中，然后对得到的结果进行评估。

如何评价结果？可以画出真实轨迹与卡尔曼滤波输出，目测作定性判断；严格评价则要借助数学。状态估计协方差矩阵 $\mathbf{P}$ 保存各状态分量的估计方差与协方差，其中对角元是方差。在模型假设成立、估计误差为零均值高斯分布且协方差正确描述误差的条件下，每个分量约有 99.7% 的误差落在正负三个标准差，即 $\mathbf{3\sigma}$ 范围内。若对此还不清楚，请先回顾高斯分布一章。

因此，可以用估计状态与真实状态之间的误差评价滤波器，再将这些误差与由 $\mathbf{P}$ 得到的标准差比较。在高斯假设成立、估计无偏且协方差正确描述误差的情况下，每个状态分量约有 99.7% 的误差应落在 $\mathbf{3\sigma}$ 范围内。这适用于各个状态分量，并不限于位置。这是逐分量的覆盖率，不是所有状态分量同时位于相应区间内的联合概率。

必须指出的是，这一结论仅适用于模拟系统。真实的传感器并不遵循完美的高斯分布，因此你可能需要将评估标准调整为适用于真实传感器数据的参数值，比如 $\mathbf{5\sigma}$。

那么，让我们用一阶卡尔曼滤波器来处理这个一阶系统，并评估其性能如何。你或许能猜到，它的表现应该不错。不过，我们还是用标准差来具体分析一下吧。

首先，让我们编写一个程序来生成这些带有噪声的测量数据吧。

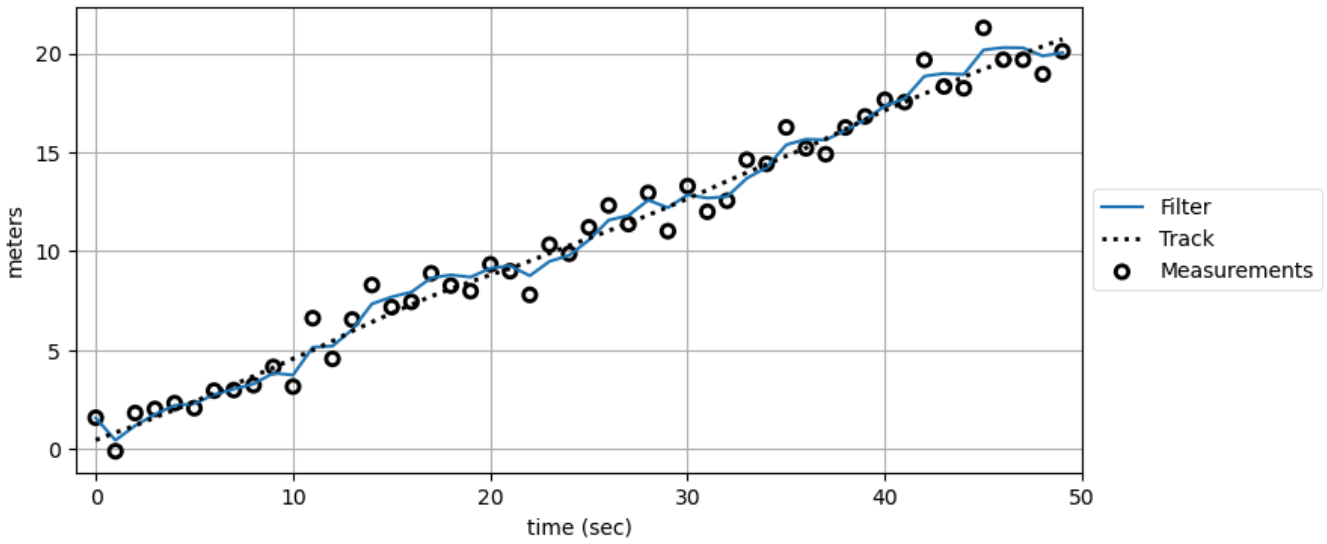
```
def simulate_system(Q, count):
    obj = ConstantVelocityObject(x0=.0, vel=0.5, noise_scale=Q)
    xs, zs = [], []
    for i in range(count):
        x = obj.update()
        z = sense(x)
        xs.append(x)
        zs.append(z)
    return np.array(xs), np.array(zs)
```

现在，我们来编写一个用于执行滤波操作并将结果保存到 `Saver` 对象中的程序。

```
from filterpy.common import Saver

def filter_data(kf, zs):
    s = Saver(kf)
    kf.batch_filter(zs, saver=s)
    s.to_array()
    return s
```

现在，我们已经准备好运行该滤波器并查看结果了。

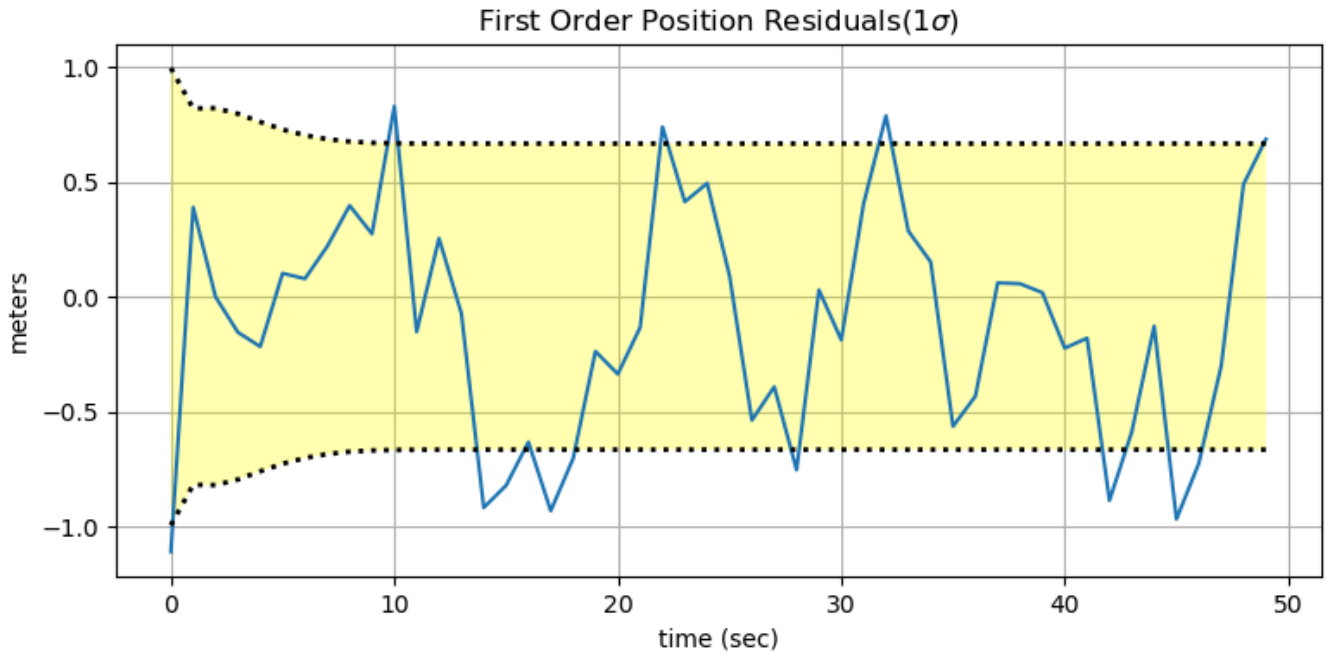
```
from kf_book.book_plots import plot_kf_output

R, Q = 1, 0.03
xs, zs = simulate_system(Q=Q, count=50)

kf = FirstOrderKF(R, Q, dt=1)
data1 = filter_data(kf, zs)

plot_kf_output(xs, data1.x, data1.z)
```



滤波器看起来表现不错，但具体好到什么程度还不容易判断。下面绘制估计状态与真实状态之间的误差。因为会多次用到，先写一个绘图函数。

```
from kf_book.book_plots import plot_residual_limits, set_labels

def plot_residuals(xs, data, col, title, y_label, stds=1):
    res = xs - data.x[:, col]
    plt.plot(res)
    plot_residual_limits(data.P[:, col, col], stds)
    set_labels(title, 'time (sec)', y_label)
```

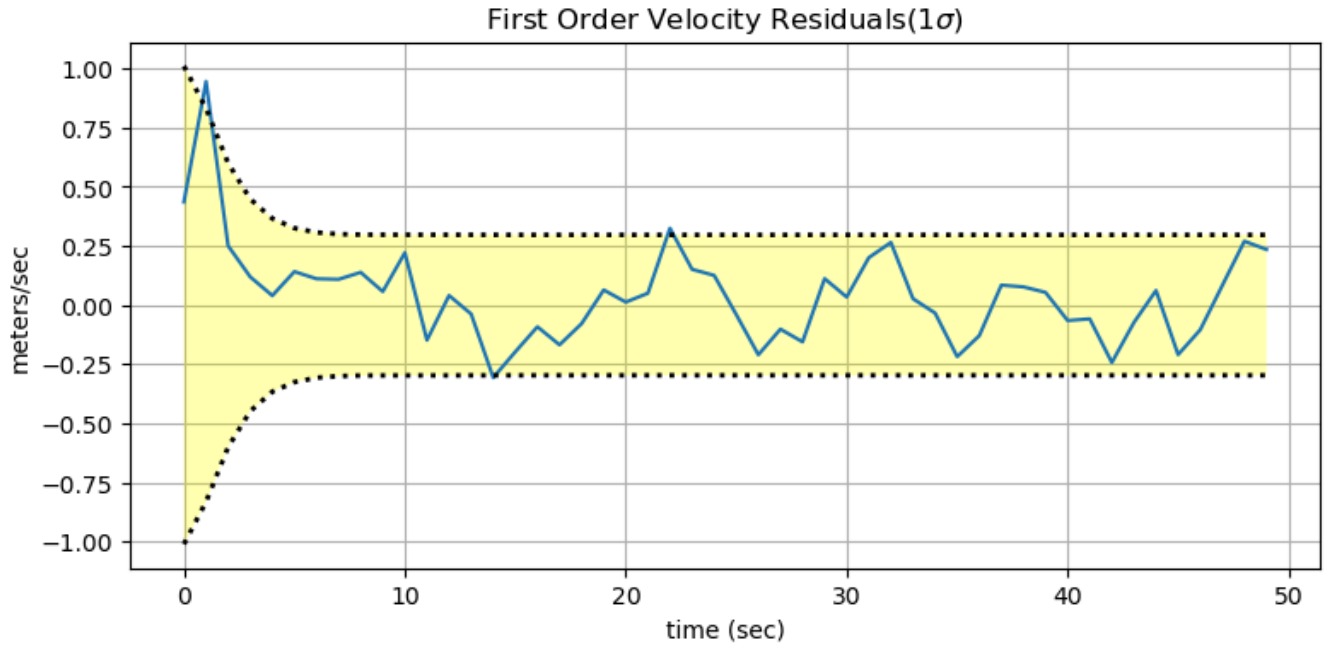
```
plot_residuals(xs[:, 0], data1, 0,
               title='First Order Position Residuals(1$\sigma$)',
               y_label='meters')
```



如何解释这幅图？锯齿线表示估计误差，即真实位置与滤波估计位置之差。如果没有测量噪声且估计始终完全准确，误差便恒为零，图中将是一条零水平线。目前误差大致围绕零上下波动。虚线之间的黄色区域是由滤波器协方差得到的正负一个标准差区间；在高斯假设及协方差估计正确的条件下，约 68% 的误差应落在其中。图中误差与该范围大体相符，暂未见明显发散。此处绘图代码比较的是真实状态与估计状态，并非测量新息；此外，围绕零对称并不能单独证明误差服从高斯分布。

再看速度估计误差。

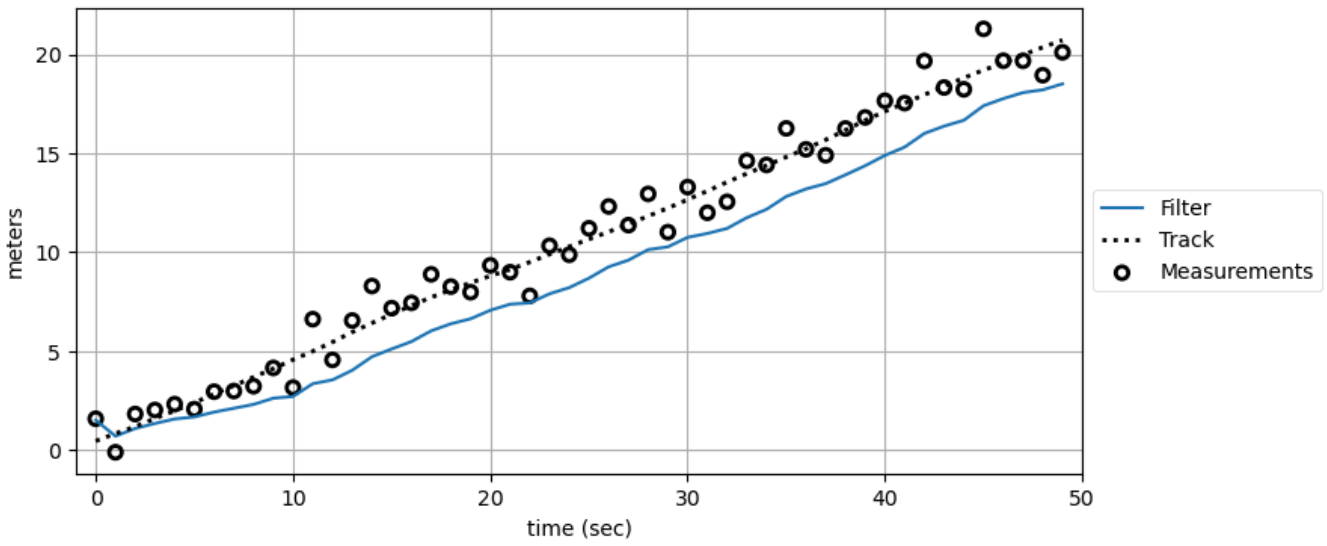
```
plot_residuals(xs[:, 1], data1, 1,
               title='First Order Velocity Residuals(1$\sigma$)',
               y_label='meters/sec')
```



同样，估计误差与滤波器给出的理论不确定性范围大体相符，这支持了滤波器设计适用于本系统的判断。

现在，让我们用零阶卡尔曼滤波器来做同样的事情。所有的代码和数学计算过程都差不多，所以我们只需查看结果即可，无需过多讨论实现细节。

```
kf0 = ZeroOrderKF(R, Q)
data0 = filter_data(kf0, zs)
plot_kf_output(xs, data0.x, data0.z)
```

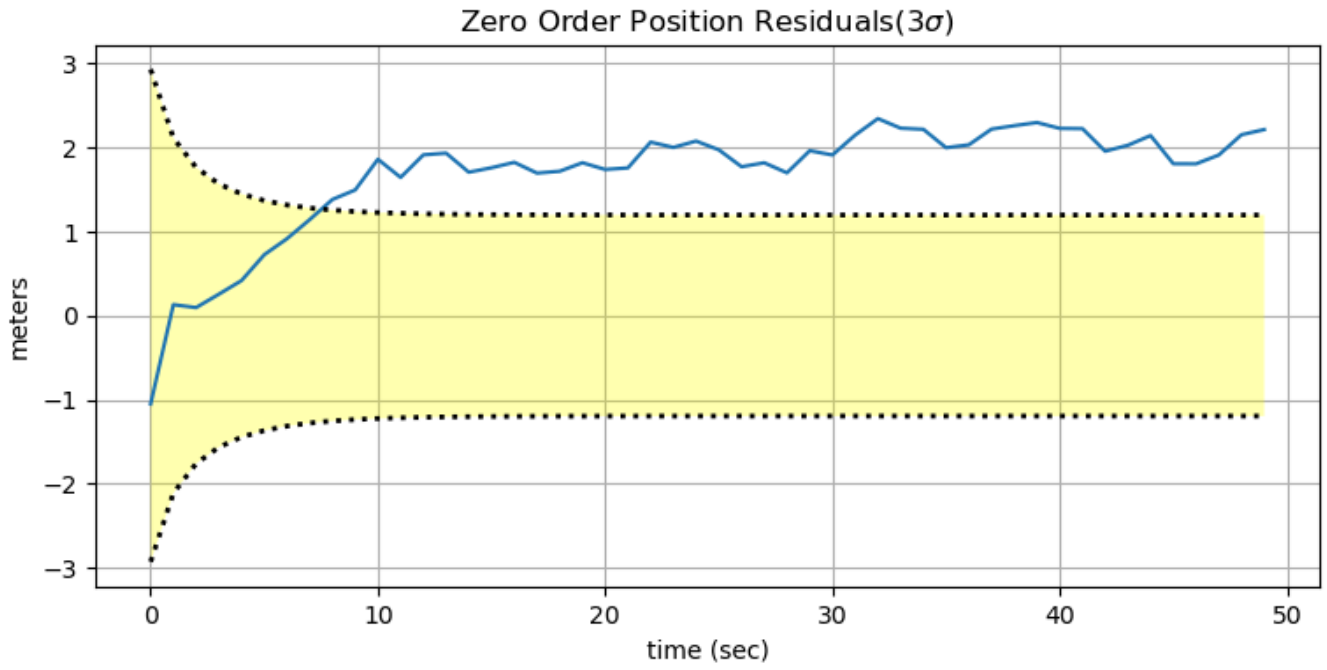


正如预期，滤波器出现了问题。回想 g-h 滤波一章中目标存在加速度的情形：由于模型阶次不够，滤波器无法及时适应速度变化，估计始终滞后于输入。这里，每次 predict() 都假设位置不变：若当前位置为 4.3，就预测下一时刻仍为 4.3，而实际位置已接近 5.3。带噪测量可能为 5.4，于是滤波器在 4.3 与 5.4 之间取融合估计，明显落后于真实位置 5.3。这个过程不断重复，滤波器始终追不上目标。

这引出了一个非常重要的问题。“恒定”这一假设，实际上只适用于离散样本之间的情况。滤波器的输出则可能会随着时间的推移而发生变化。

下面看估计误差。由于此模型没有把速度纳入状态，只能检查位置估计误差。

```
plot_residuals(xs[:, 0], data0, 0,
               title='Zero Order Position Residuals(3$\sigma$)',
               y_label='meters',
               stds=3)
```

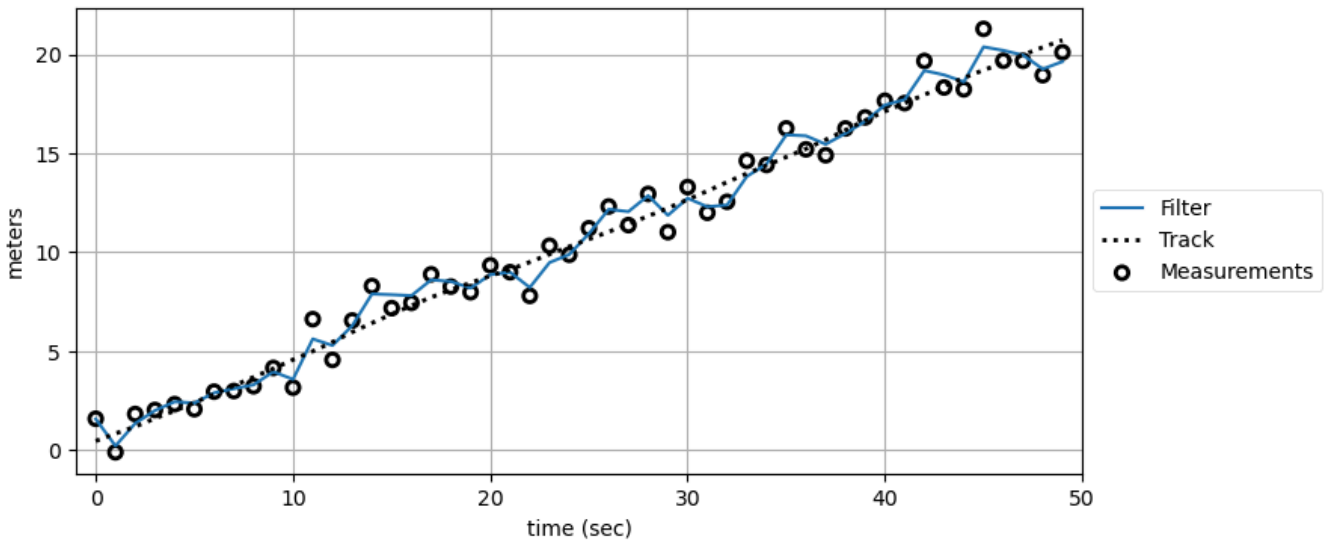


滤波器几乎立即出现不一致：几秒钟后，估计误差便越出正负三个标准差的范围。必须理解，$\mathbf{P}$ 只反映模型和噪声假设成立时的理论性能。这里估计已明显偏离真实轨迹，但 $\mathbf{P}$ 的方差却越来越小，仿佛估计正在不断改善。滤波器无法自行发现所提供的系统模型有误。这有时被称为“自满”的滤波器，即对自身性能过度自信。

本例中的偏离很快发生，而且十分明显。在其他系统中，它可能逐渐出现，幅度也可能很小。因此，应检查这类误差图，确认实际误差与理论不确定性是否相符。

现在，让我们来试试二阶模型。你可能会觉得这么做很合理。毕竟，我们知道，模拟物体的运动过程中存在一定的噪声，这意味着物体肯定有加速度。那为何不用二阶模型来描述这种加速度呢？如果没有加速度的话，那么加速度就应该被估计为 0，对吧？但实际情况真的如此吗？在继续之前，请先好好思考一下。

```
kf2 = SecondOrderKF(R, Q, dt=1)
data2 = filter_data(kf2, zs)
plot_kf_output(xs, data2.x, data2.z)
```

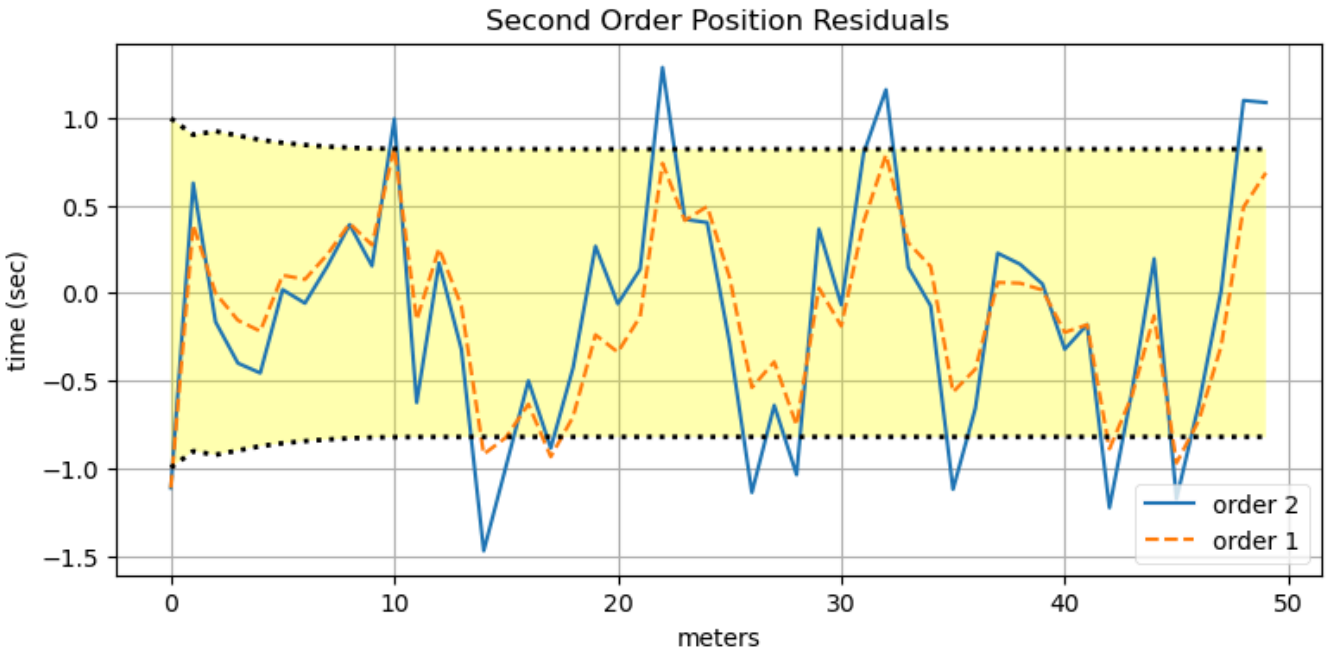


效果是否如你所预期?

可以看出，与一阶滤波器相比，二阶滤波器的性能要差得多。为什么呢？这种滤波器用于模拟加速度的变化，因此，当测量值出现较大波动时，系统会将其视为加速度而非噪声。这样一来，滤波器的输出结果就会与噪声的波动十分接近。更糟糕的是，如果噪声持续高于或低于某个水平，滤波器会过度响应这些噪声。因为滤波器错误地认为存在加速度，所以其预测结果会越来越偏离实际数值。这种情况显然不可取。

轨迹看起来还不算太糟。下面检查估计误差。二阶模型的误差既没有明显发散，也没有越出正负三个标准差的范围；但与一阶滤波器对比，更能说明问题，因此把两者画在同一幅图中。

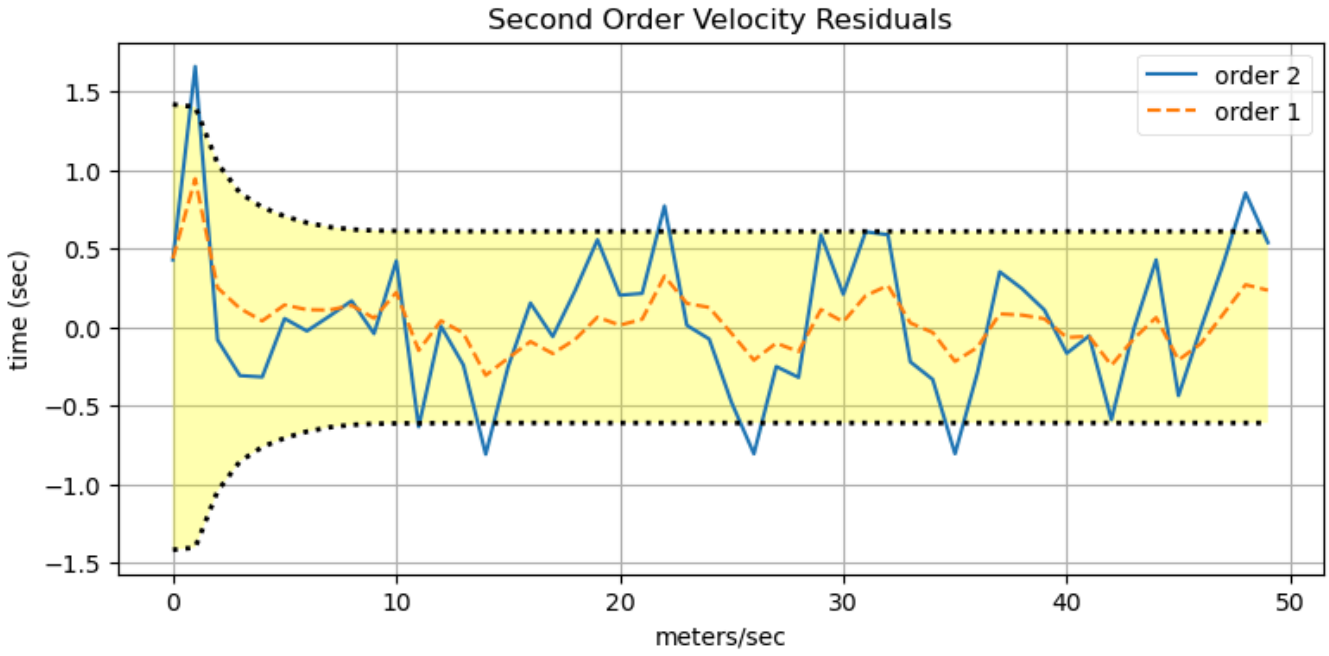
```
res2 = xs[:, 0] - data2.x[:, 0]
res1 = xs[:, 0] - data1.x[:, 0]

plt.plot(res2, label='order 2')
plt.plot(res1, ls="--", label='order 1')
plot_residual_limits(data2.P[:, 0, 0])
set_labels('Second Order Position Residuals',
           'meters', 'time (sec)')
plt.legend();
```



二阶滤波器的位置估计误差略大于一阶滤波器，但仍在理论范围内，暂时没有特别明显的异常。

再看速度估计误差。

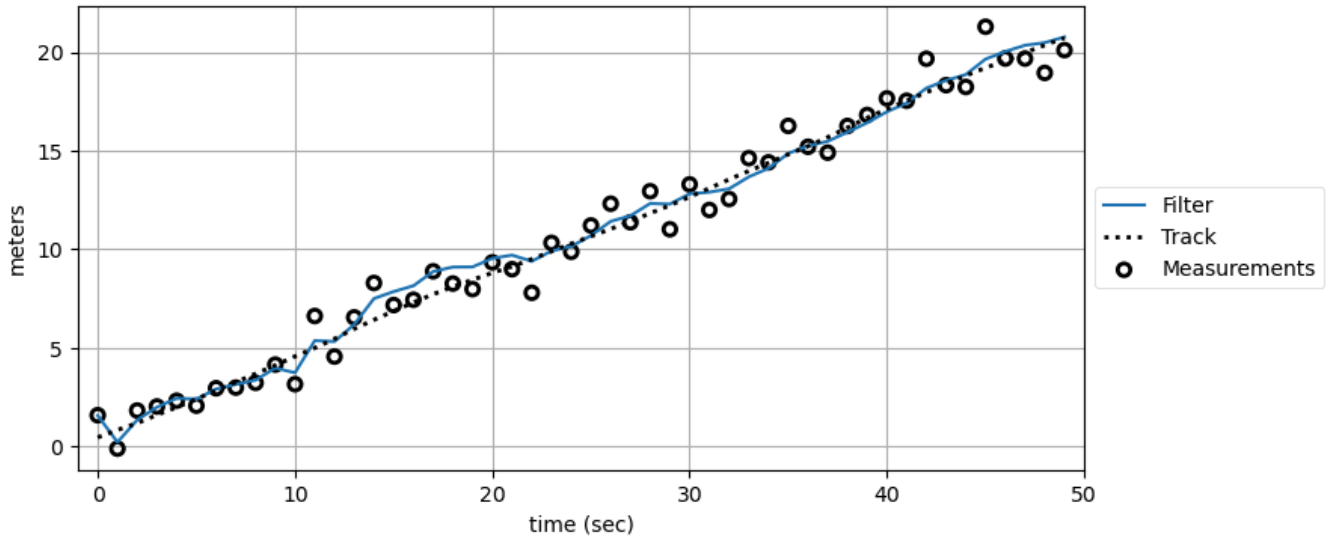
```
res2 = xs[:, 1] - data2.x[:, 1]
res1 = xs[:, 1] - data1.x[:, 1]

plt.plot(res2, label='order 2')
plt.plot(res1, ls='--', label='order 1')
plot_residual_limits(data2.P[:, 1, 1])
set_labels('Second Order Velocity Residuals',
                     'meters/sec', 'time (sec)')
plt.legend();
```



速度的情况很不一样。二阶滤波器的估计误差虽然仍在理论范围内，却远大于一阶滤波器。本例中，滤波器把测量噪声解释成加速度，并在每次预测时把这个加速度估计加入速度传播。由于目标并没有这样的持续加速度，速度估计误差便明显增大。

再考虑一种看似可行的办法。系统大体为恒速运动，但实际速度在各时间段之间仍会略有变化。一阶滤波器用过程噪声描述这些变化，矩阵 $\mathbf{Q}$ 正是为此构造的。如果改用二阶滤波器，既然模型已经包含速度变化，也许就不再需要过程噪声，可以令 $\mathbf{Q}$ 为零？下面检验这个想法。

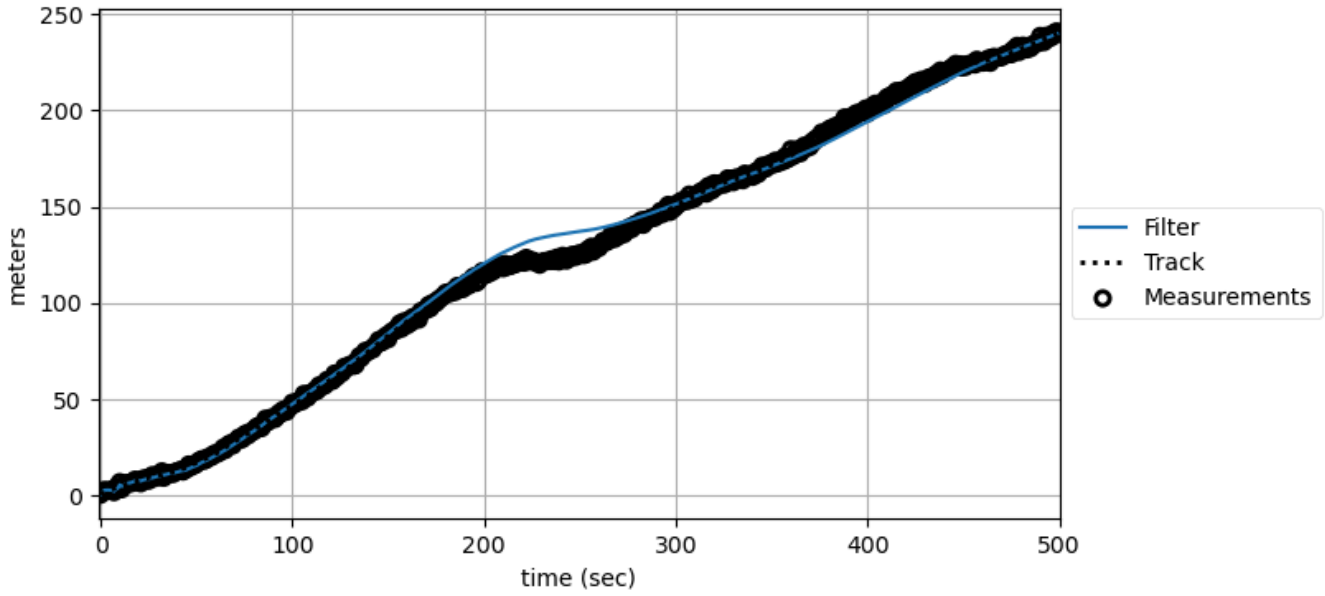
```
kf2 = SecondOrderKF(R, 0, dt=1)
data2 = filter_data(kf2, zs)
plot_kf_output(xs, data2.x, data2.z)
```

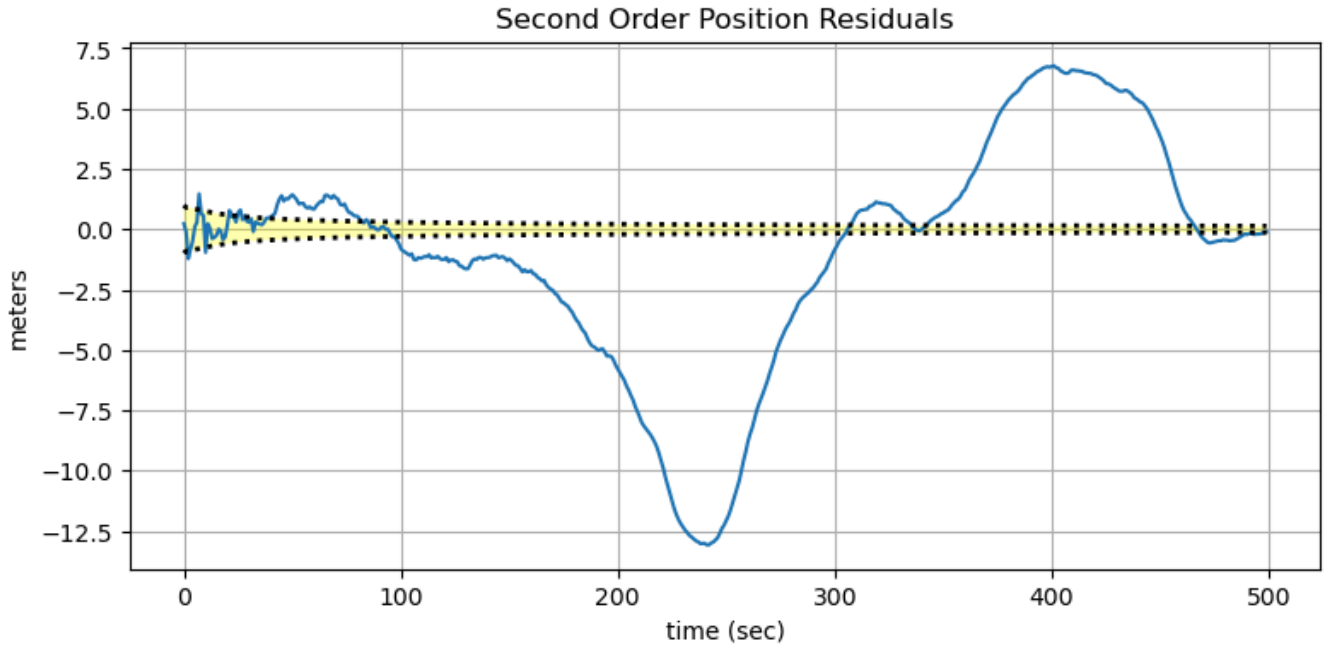


在我看来，该滤波器能够很快地与实际轨迹相吻合。成功了！

或者，也未必如此。将过程噪声设置为 0，就相当于告诉滤波器：该过程模型是完美的。让我们来观察一下该滤波器在更长时间内的表现吧。

```
np.random.seed(25944)
xs500, zs500 = simulate_system(Q=Q, count=500)

kf2 = SecondOrderKF(R, 0, dt=1)
data500 = filter_data(kf2, zs500)

plot_kf_output(xs500, data500.x, data500.z)
plot_residuals(xs500[:, 0], data500, 0,
               'Second Order Position Residuals',
               'meters')
```



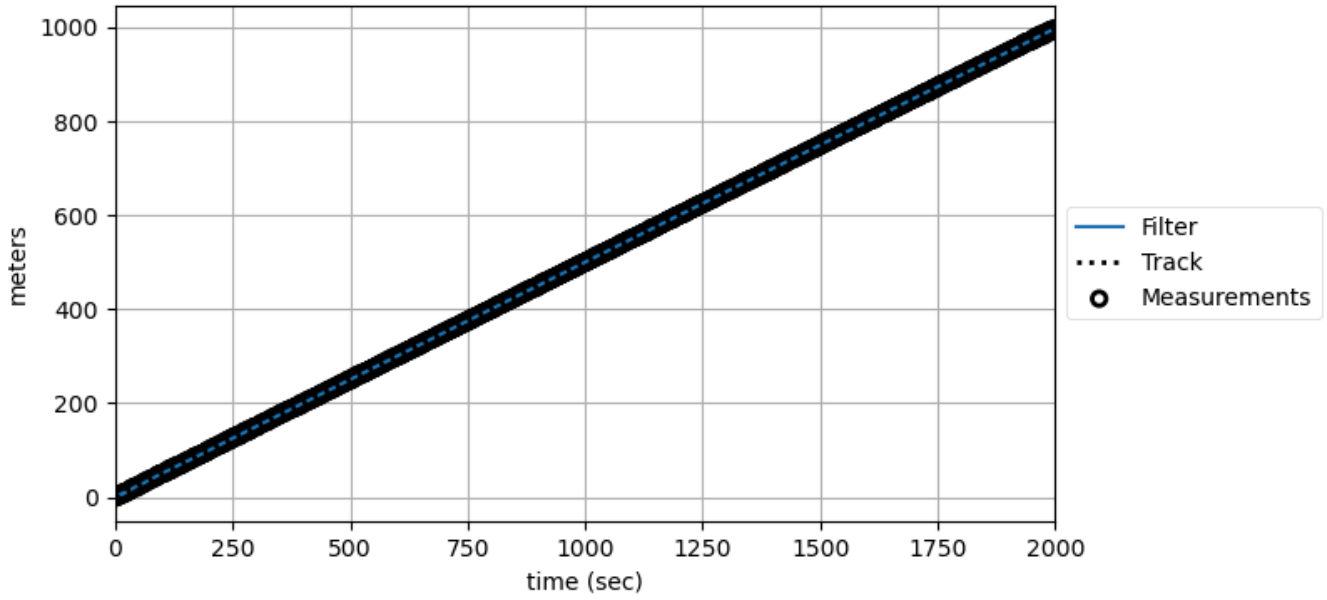


滤波器的表现很差：轨迹图中，估计长时间偏离真实轨迹；误差图则更清楚地揭示了问题。接近第 100 次更新时，估计误差明显越出滤波器给出的理论范围。末尾似乎有所收敛，但我对此持怀疑态度。与此同时，滤波器却始终报告越来越小的方差。因此，不能仅凭滤波器自身的协方差矩阵判定它表现良好。

为什么会这样？将过程噪声设为零，意味着假定过程模型完全准确。在本例中，随着协方差不断缩小，滤波器逐渐过度信任模型，测量的影响变得很弱，因而无法适应真实系统的未建模变化。Q 为零本身并不意味着立即忽略全部测量；测量权重还取决于状态协方差、测量模型和测量噪声。

那么，把过程噪声设得非常小呢？试试看。

```
np.random.seed(32594)
xs2000, zs2000 = simulate_system(Q=0.0001, count=2000)

kf2 = SecondOrderKF(R, 0, dt=1)
data2000 = filter_data(kf2, zs2000)

plot_kf_output(xs2000, data2000.x, data2000.z)
plot_residuals(xs2000[:, 0], data2000, 0,
               'Second Order Position Residuals',
               'meters')
```

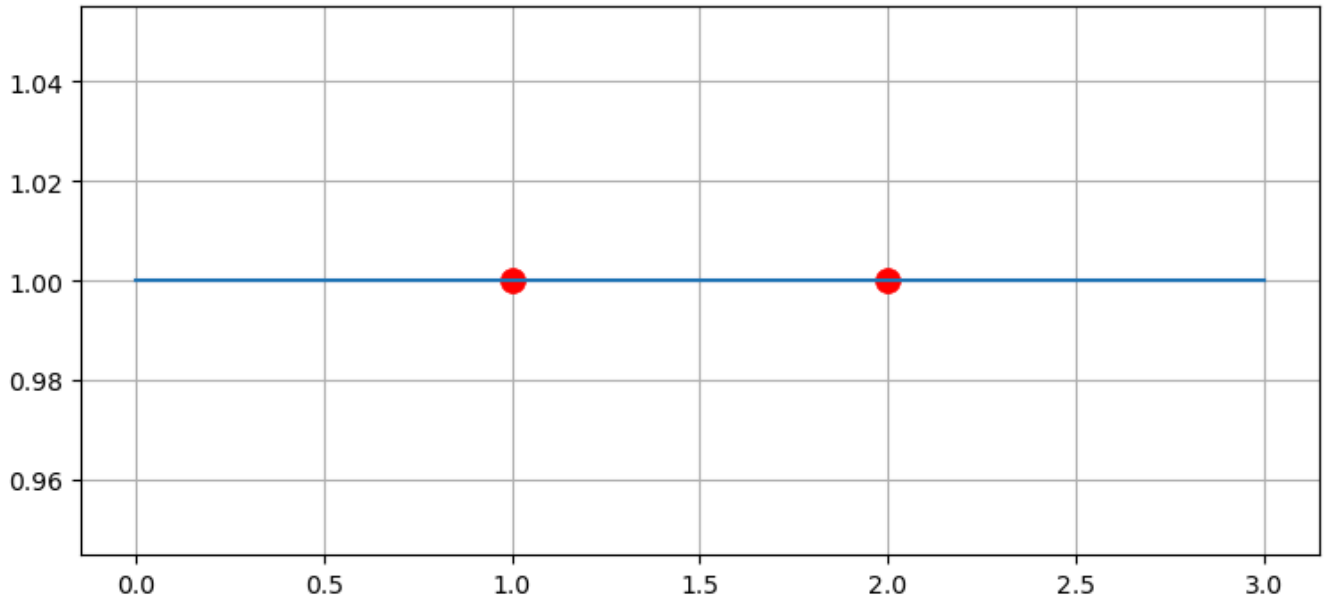


估计误差图再次揭示了问题：轨迹看起来不错，实际误差却在相当长的时间内超出理论不确定性范围。

你该如何看待这一切呢？你可能会认为，最后的那个滤波器效果已经“足够好了”，也许确实如此。不过我得提醒你：发散型的滤波器并不一定会收敛。如果使用的是不同的数据集，或者面对的是行为表现不同的物理系统，那么最终得到的滤波器可能会越来越偏离实际的测量结果。

另外，让我们从数据拟合的角度来思考这个问题。假设我给你两个点，要求你用一条直线来拟合这两个点。

```
plt.scatter([1, 2], [1, 1], s=100, c='r')
plt.plot([0, 3], [1, 1]);
```



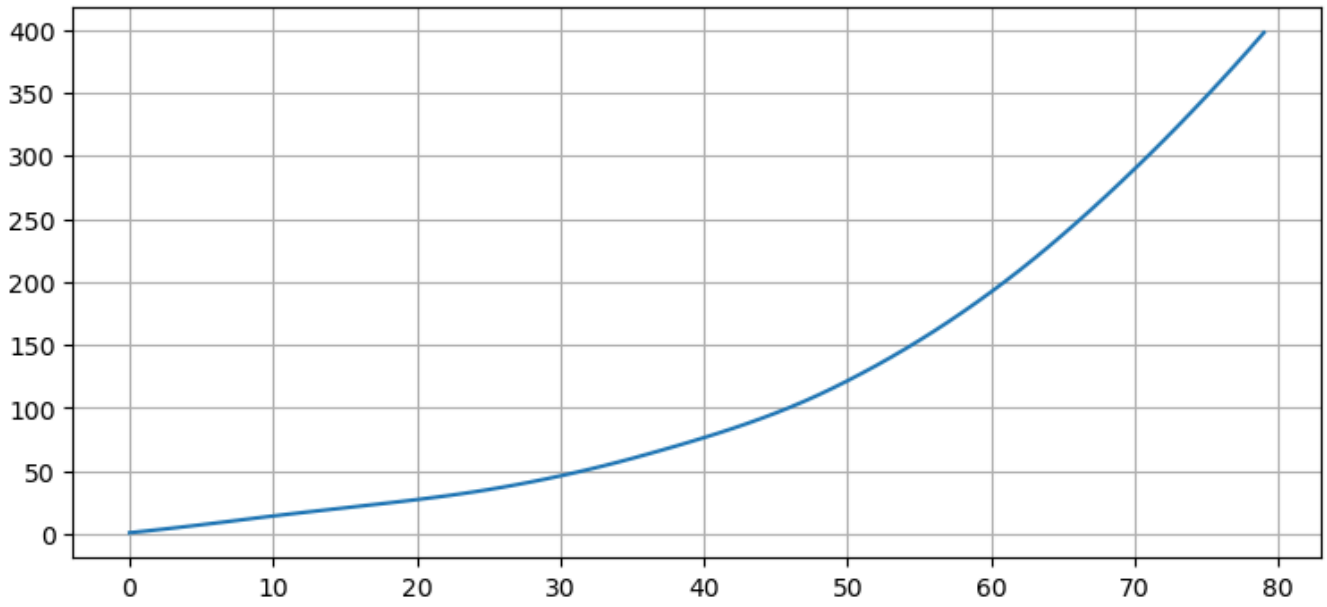
通过两个不同的点可以确定唯一一条直线。若给出更多数据点，可以用最小二乘法求出最佳拟合直线；它在最小二乘意义上仍是最优解。

若要求用更高次多项式拟合这两个点，解便不唯一，例如有无数条二次抛物线经过它们。这可以类比模型阶次过高带来的风险：滤波器可能把噪声解释为额外的运动变化，产生次优估计，甚至发散。不过，这只是拟合类比；给定模型、初始分布和测量序列后，卡尔曼递推仍有确定结果，较高阶次也并不必然导致发散。

为了获得最佳性能，所使用的滤波器的阶数必须与被监测对象的动态特性相匹配。在很多情况下，这并不难实现。例如，如果要用卡尔曼滤波器来检测冷冻柜内的温度，那么显然，零阶滤波器就是最佳选择。但是，如果我们想要监测汽车的运行情况呢？当汽车以恒定速度直线行驶时，一阶滤波器就足够了。不过，汽车还会转弯、加速或减速，这种情况下，二阶滤波器的性能会更好。这就是“自适应滤波器”这一章节要探讨的问题。在这一章中，我们将学习如何设计出能够适应被监测对象行为变化的滤波器。

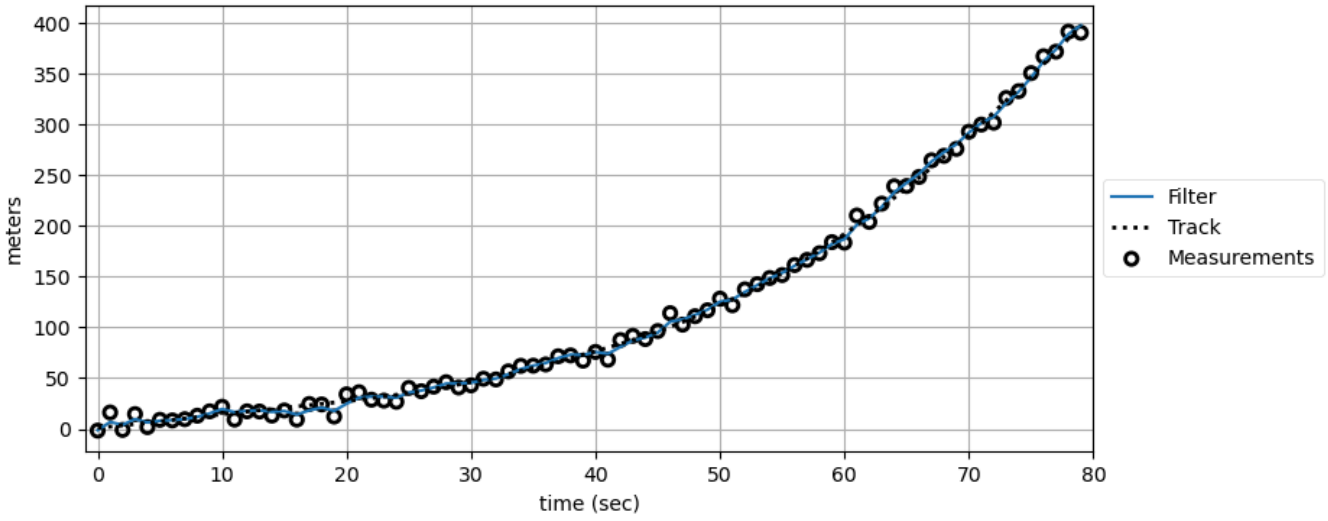
只要适当增大过程噪声，并使离散时间间隔足够短，低阶滤波器仍可跟踪较高阶的运动过程。结果未必最优，却可能很好。在尝试自适应滤波器之前，我通常先尝试这种办法。足够快的采样频率取决于系统动态；例如某个问题可能适合每秒 100 次采样，这并非所有问题通用的数值。下面看一个有加速度的例子，先建立仿真。

```
class ConstantAccelerationObject(object):
    def __init__(self, x0=0, vel=1., acc=0.1, acc_noise=.1):
        self.x = x0
        self.vel = vel
        self.acc = acc
        self.acc_noise_scale = acc_noise

    def update(self):
        self.acc += randn() * self.acc_noise_scale
        self.vel += self.acc
        self.x += self.vel
        return (self.x, self.vel, self.acc)

R, Q = 6., 0.02
def simulate_acc_system(R, Q, count):
    obj = ConstantAccelerationObject(acc_noise=Q)
    zs = []
    xs = []
    for i in range(count):
        x = obj.update()
        z = sense(x, R)
        xs.append(x)
        zs.append(z)
    return np.asarray(xs), zs

np.random.seed(124)
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)
plt.plot(xs[:, 0]);
```



现在，我们将使用二阶滤波器来对数据进行滤波。

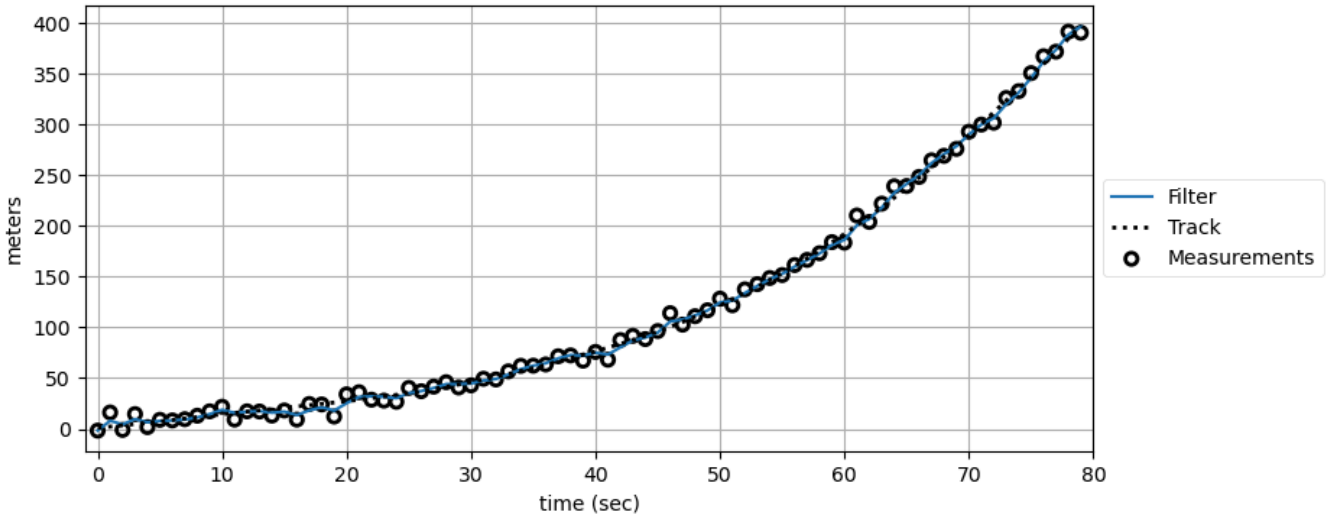
```
np.random.seed(124)
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)

kf2 = SecondOrderKF(R, Q, dt=1)
data2 = filter_data(kf2, zs)

plot_kf_output(xs, data2.x, data2.z, aspect_equal=False)
plot_residuals(xs[:, 0], data2, 0,
               'Second Order Position Residuals',
               'meters')
```



可以看出，该滤波器的性能符合其理论上的极限值。

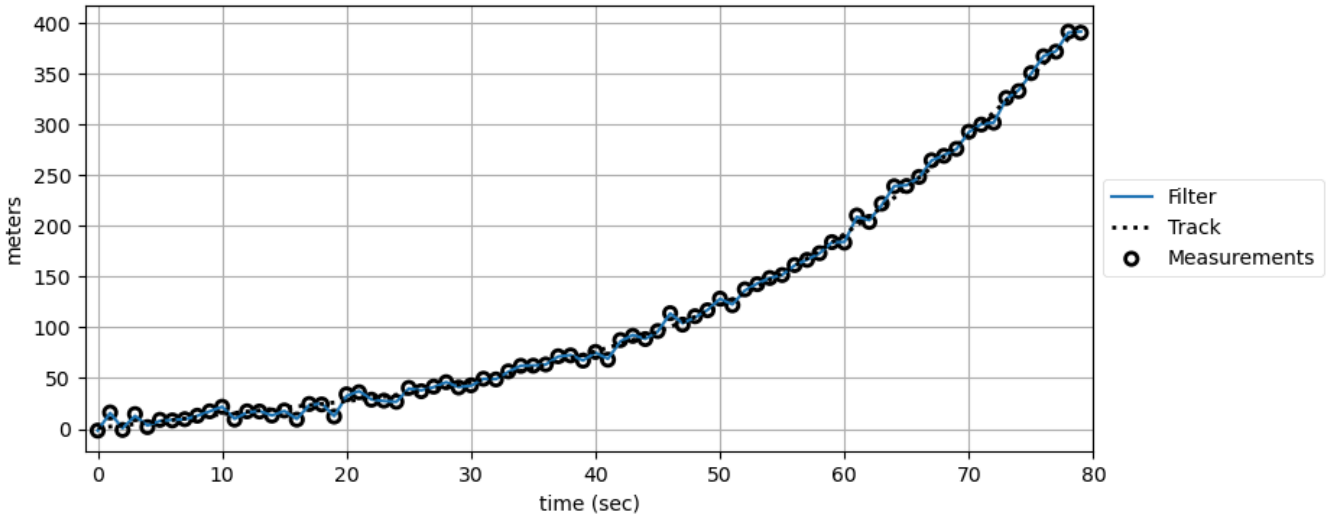
现在，让我们使用低阶滤波器。正如之前所演示的，低阶滤波器会导致信号出现延迟，因为它无法准确模拟加速度的变化。不过，我们可以通过增加“过程噪声”的大小来在一定程度上弥补这一缺陷。滤波器会将加速度视为过程模型中的噪声。虽然这样处理得到的结果并非最优，但如果设计得当，系统也不会出现异常行为。确定“过程噪声”的具体数值并非易事，需要通过实验来找到合适的数值。在这里，我将“过程噪声”增加了 10 倍，效果相当不错。

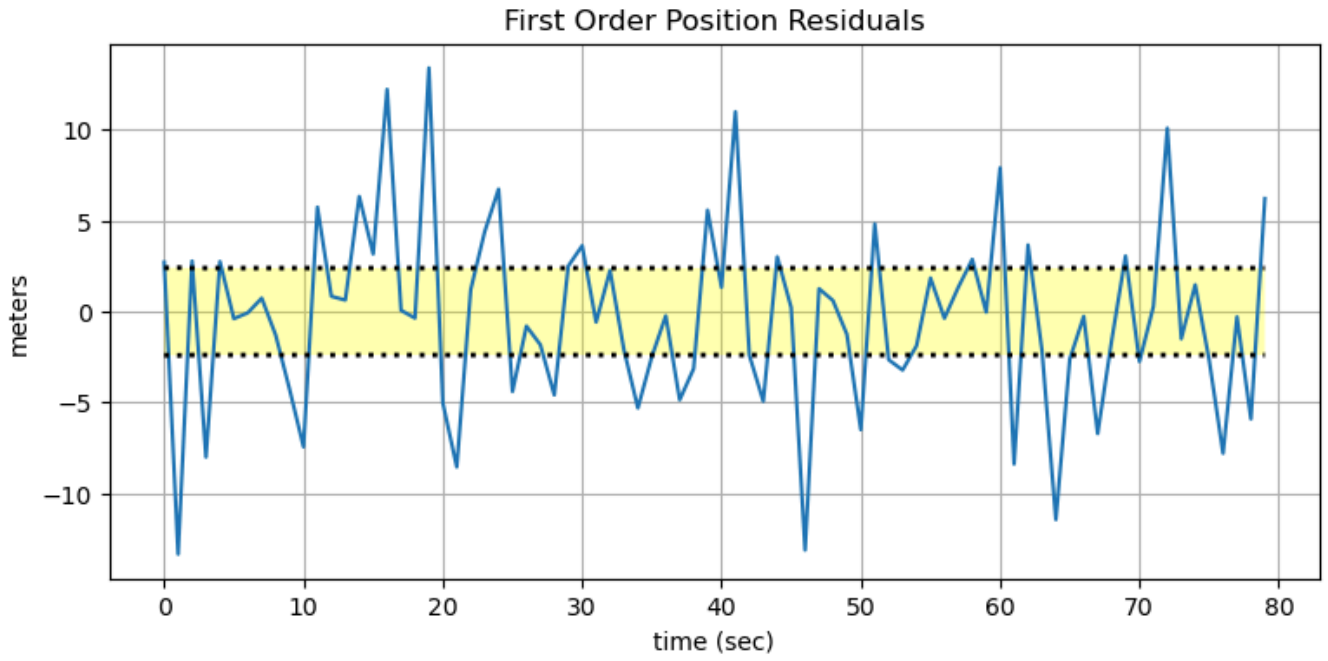
```
kf3 = FirstOrderKF(R, Q * 10, dt=1)
data3= filter_data(kf3, zs)

plot_kf_output(xs, data3.x, data3.z, aspect_equal=False)
plot_residuals(xs[:, 0], data3, 0,
               'First Order Position Residuals',
               'meters')
```



如果把过程噪声设得远大于实际所需，会怎样？较大的过程噪声会降低滤波器对预测的信任，使其更依赖测量，因此预计输出也会更紧地跟随测量中的噪声波动。下面验证。

```
kf4 = FirstOrderKF(R, Q * 10000, dt=1)
data4 = filter_data(kf4, zs)

plot_kf_output(xs, data4.x, data4.z, aspect_equal=False)
plot_residuals(xs[:, 0], data4, 0,
               'First Order Position Residuals',
               'meters')
```





练习：状态变量设计

如前所述，可以自行选择 $\mathbf{x}$ 中各状态分量的顺序。例如，一维恒加速度模型也能写成 $\mathbf{x} = \begin{bmatrix}\ddot{x} & x & \dot{x}\end{bmatrix}^\mathsf{T}$。这种排列并不直观，但数学上完全可行。

下面为二维运动机器人换一种状态排列：采用 $\mathbf{x} = \begin{bmatrix}x & y & \dot{x} & \dot{y}\end{bmatrix}^\mathsf{T}$，而不是此前的 $\mathbf{x} = \begin{bmatrix}x & \dot{x} & y & \dot{y}\end{bmatrix}^\mathsf{T}$。两种状态都只包含位置与速度，仍属于本章的一阶恒速模型；这里练习的是状态分量顺序的改变。

为什么要选择不同的排序方式呢？正如你马上会看到的，改变 $\mathbf{x}$ 的顺序，会影响到滤波器中其他大部分矩阵的顺序。根据你想要分析的数据类型，比如 $\mathbf{P}$ 中的相关性，不同的 $\mathbf{x}$ 排序方式可能会让分析变得更简单或更复杂。

仔细想想该怎么做。需要做出哪些改变呢？显然，你只需要调整卡尔曼滤波器中的矩阵，以反映这一新的设计要求即可。

请尝试使用下面的模板代码来操作：

```
N = 30 # number of iterations
dt = 1.0 # time step
R_std = 0.35
Q_std = 0.04
sensor = PosSensor((0, 0), (2, .5), noise_std=R_std)
zs = np.array([sensor.read() for _ in range(N)])
tracker = KalmanFilter(dim_x=4, dim_z=2)
# assign state variables here
xs, ys = [], []
for z in zs:
tracker.predict()
tracker.update(z)
xs.append(tracker.x[0])
ys.append(tracker.x[1])
plt.plot(xs, ys);
```

解答

先构造 $\mathbf{F}$。稍加练习后，就能直接写出矩阵；若觉得困难，可先按状态顺序列出 $\mathbf{F}$ 对应的各个方程，再提取系数。恒速模型中，速度在相邻时刻保持不变，其状态转移系数为 1：

$$
\begin{aligned}
x &= 1x + 0y + 1\dot{x}\Delta t + 0\dot{y}\Delta t \\
y &= 0x + 1y + 0\dot{x}\Delta t + 1\dot{y}\Delta t \\
\dot{x} &= 0x + 0y + 1\dot{x} + 0\dot{y} \\
\dot{y} &= 0x + 0y + 0\dot{x} + 1\dot{y}
\end{aligned}
$$

我们可以把系数抄录下来。

$$
\mathbf{F} = \begin{bmatrix}
1 & 0 & \Delta t & 0 \\
0 & 1 & 0 & \Delta t \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1
\end{bmatrix}
$$

过程噪声协方差也需要重新排列。先根据新的状态顺序确定各元素的位置。可以在矩阵的行、列两侧分别写出状态分量名称，便于看出每个元素对应哪一对变量。Jupyter 中不便这样标注，因此这里省略这一辅助图示。

$$
\mathbf{Q} = \begin{bmatrix}
\sigma_x^2 & \sigma_{xy} & \sigma_{x\dot{x}} & \sigma_{x\dot{y}} \\
\sigma_{yx} & \sigma_y^2 & \sigma_{y\dot{x}} & \sigma_{y\dot{y}} \\
\sigma_{\dot{x}x} & \sigma_{\dot{x}y} & \sigma_{\dot{x}}^2 & \sigma_{\dot{x}\dot{y}} \\
\sigma_{\dot{y}x} & \sigma_{\dot{y}y} & \sigma_{\dot{y}\dot{x}} & \sigma_{\dot{y}}^2
\end{bmatrix}
$$

x 与 y 之间没有关联，因此我们可以将涉及这两者的各项都设为零。

$$
\mathbf{Q} = \begin{bmatrix}
\sigma_x^2 & 0 & \sigma_{x\dot{x}} & 0 \\
0 & \sigma_y^2 & 0 & \sigma_{y\dot{y}} \\
\sigma_{\dot{x}x} & 0 & \sigma_{\dot{x}}^2 & 0 \\
0 & \sigma_{\dot{y}y} & 0 & \sigma_{\dot{y}}^2
\end{bmatrix}
$$

看清这种排列后，就能识别规律，更快地构造 $\mathbf{Q}$。

`Q_discrete_white_noise` 生成的矩阵中的元素排列方式不同，但我们只需复制其中的元素即可，具体操作会在下面的代码中展示。

现在让我们来设计 $\mathbf{H}$。它能够将状态 $\begin{bmatrix}x & y & \dot{x} & \dot{y}\end{bmatrix}^\mathsf{T}$ 转换为测量结果 $\mathbf{z} = \begin{bmatrix}z_x & z_y\end{bmatrix}^\mathsf{T}$。

$$\mathbf{Hx} = \mathbf{z}$$

$$\begin{bmatrix}? & ? & ? & ?\\ ? & ? & ? & ?\end{bmatrix}\begin{bmatrix}x\\y\\\dot{x}\\\dot{y}\end{bmatrix} = \begin{bmatrix}z_x\\z_y\end{bmatrix}$$

现在，填写这个矩阵应该很容易了：

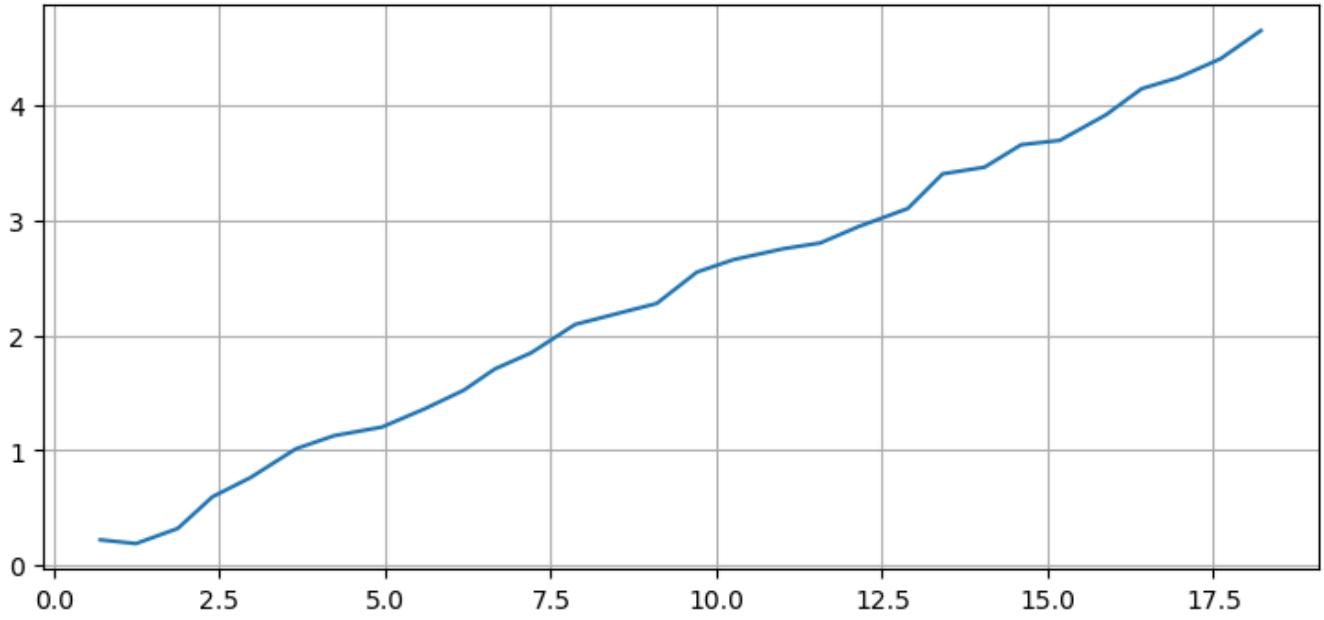
$$\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\end{bmatrix}\begin{bmatrix}x\\y\\\dot{x}\\\dot{y}\end{bmatrix} = \begin{bmatrix}z_x\\z_y\end{bmatrix}$$

测量向量 $\mathbf{z} = \begin{bmatrix}z_x & z_y\end{bmatrix}^\mathsf{T}$ 的分量及其排列没有改变，因此 $\mathbf{R}$ 也不需要改变。

最后是 $\mathbf{P}$。它的行列采用与 $\mathbf{Q}$ 相同的状态顺序，因此可按同样的排列规则构造。

```
N = 30 # number of iterations
dt = 1.0 # time step
R_std = 0.35
Q_std = 0.04

M_TO_FT = 1 / 0.3048

sensor = PosSensor((0, 0), (2, .5), noise_std=R_std)
zs = np.array([sensor.read() for _ in range(N)])

tracker = KalmanFilter(dim_x=4, dim_z=2)

tracker.F = np.array([[1, 0, dt,  0],
                      [0, 1,  0, dt],
                      [0, 0,  1,  0],
                      [0, 0,  0,  1]])

tracker.H = np.array([[M_TO_FT, 0, 0, 0],
                      [0, M_TO_FT, 0, 0]])

tracker.R = np.eye(2) * R_std**2
q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_std**2)
tracker.Q[0,0] = q[0,0]
tracker.Q[1,1] = q[0,0]
tracker.Q[2,2] = q[1,1]
tracker.Q[3,3] = q[1,1]
tracker.Q[0,2] = q[0,1]
tracker.Q[2,0] = q[0,1]
tracker.Q[1,3] = q[0,1]
tracker.Q[3,1] = q[0,1]

tracker.x = np.array([[0, 0, 0, 0]]).T
tracker.P = np.eye(4) * 500.

xs, ys = [], []
for z in zs:
    tracker.predict()
    tracker.update(z)
```

```
    xs.append(tracker.x[0])
    ys.append(tracker.x[1])
plt.plot(xs, ys);
```



检测并剔除不良测量结果

卡尔曼滤波器无法识别和剔除那些错误的测量数据。假设你正在追踪某架飞机，但你收到的测量数据实际上来自距离该飞机当前位置 100 公里以外的地方。如果你用这个错误的测量值来更新位置估计，那么新的估计值将会严重偏离实际位置。

下面用仿真给出具体例子。经过 100 个时间步后，故意用一个等于当前位置两倍的测量值进行更新。

```
from filterpy.common import kinematic_kf

kf = kinematic_kf(dim=2, order=1, dt=1.0, order_by_dim=False)
kf.Q = np.diag([0, 0, .003, .003])
kf.x = np.array([[1., 1., 0., 0.]]).T
kf.R = np.diag([0.03, 0.21]) # use different errors

for i in range(101):
    kf.predict()
    kf.update(np.array([[i*.05, i*.05]])) # around 200 kph

p0 = kf.x[0:2]

kf.predict()
prior = kf.x
z = kf.x[0:2]*2
kf.update(z)
p1 = kf.x[0:2]

# compute error of measurement from prior
y = np.abs(z - kf.H @ prior)
dist = np.linalg.norm(y)

np.set_printoptions(precision=2, suppress=True)

print(f'bad measurement       : {z.T} km')
print(f'before bad measurement: {p0.T} km')
print(f'after bad measurement : {p1.T} km')
```

```
print(f'estimate shift        : {np.linalg.norm(p1 - prior[:2]):.1f} km')
print(f'distance from prior   : {dist:.1f} km')
```

```
bad measurement       : [[10.1 10.1]] km
before bad measurement: [[5. 5.]] km
after bad measurement : [[7.84 7.01]] km
estimate shift        : 3.4 km
distance from prior   : 7.1 km
```

`kinematic_kf`？那是什么？`filterpy.common` 提供了 `kinematic_kf` 功能，使得用户能够创建任意维度和阶数的线性运动学滤波器。在本书中，我没有使用这个功能，因为我希望你们能通过实践来积累创建卡尔曼滤波器的经验。我在这里使用它，只是为了简化示例内容，同时让你们了解该库的这一功能而已。

回到正题。如你所见，估算值上升了 3.4 公里，而预测值与实际测量值之间的误差则超过了 7 公里。

我们能做些什么来避免这种情况呢？首先，我们可以检查一下：先前的估计值与实际测量值之间的差距是否过大。为什么要检查先前的估计值，而不是当前的估计值呢？因为经过更新后，当前的估计值可能会非常接近那个有误的测量值，不过在这个例子中并非如此。

按照新息的定义，使用 $\mathbf{z} - \mathbf{Hx}$，即实际测量减去预测测量。KalmanFilter 类已将更新时计算的新息保存在 KalmanFilter.y 中，下面改用该属性说明。使用残差时，必须保持差值符号的约定一致。

```
print(f'error = {np.linalg.norm(kf.y):.1f} km, at a speed of {dist*3600:.0f} kph')
```

```
error = 7.1 km, at a speed of 25710 kph
```

本例的测量值距离预测位置约 7 km，听起来很远。但判断差值大小必须结合单位与采样间隔：若数值差 7 的单位为千米、间隔为 1 秒，便意味着超过 25000 km/h 的速度，显然不符合本例飞机的运动能力；若同样的数值差以厘米计、间隔为 1 分钟，则非常小。

我们可以添加一项检查机制，该机制会考虑到飞机的性能限制因素。

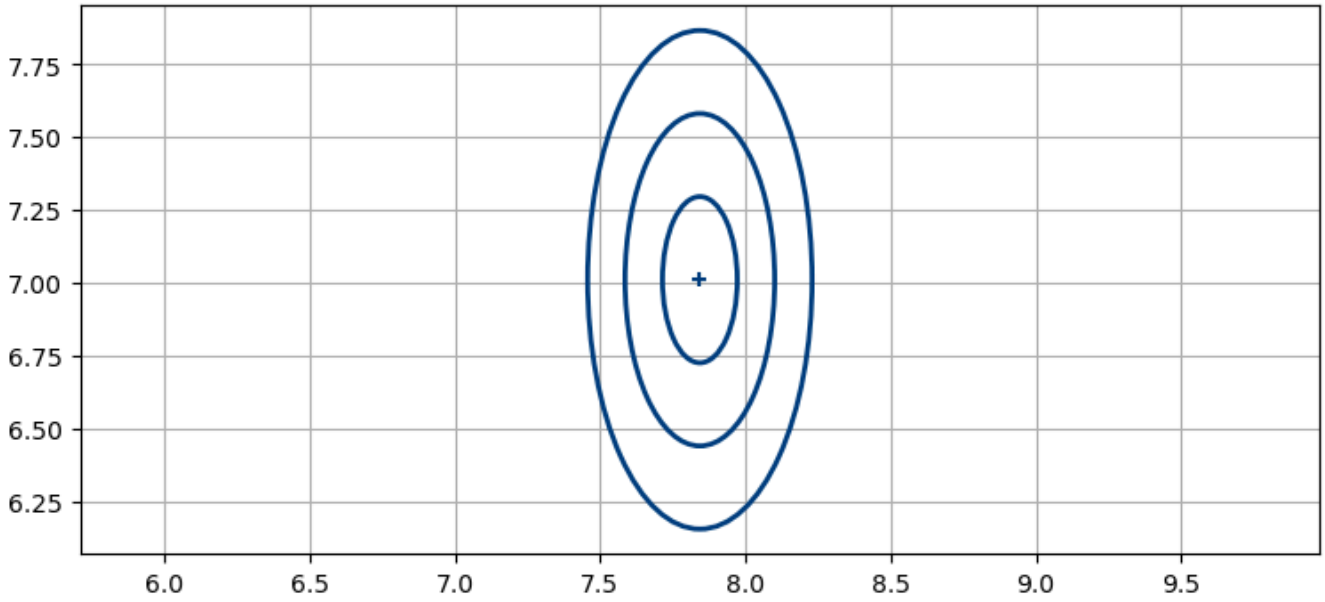
```
vel = y / dt
if vel >= MIN_AC_VELOCITY and vel <= MAX_AC_VELOCITY:
kf.update()
```

你认为这是一个合理且可行的解决方案吗？在继续阅读之前，请尽可能多地想出各种反对意见吧。

我对这种办法并不满意。假如初始化时的位置只是猜测，就可能把正常测量全部丢弃，使滤波永远无法正常开始。其次，这忽视了我们对传感器和过程误差的了解。卡尔曼滤波用 $\mathbf{P}$ 表示当前估计的不确定性。若 $\mathbf{P}$ 意味着位置标准差为 $\sigma_x = 10$ 米，而测量与先验相差 1 千米，就相差了 100 个标准差；在相应模型成立时，这显然是可疑的测量。

下面将 $\mathbf{P}$ 所描述的不确定性画出来，分别画出一倍、两倍和三倍标准差的协方差椭圆。

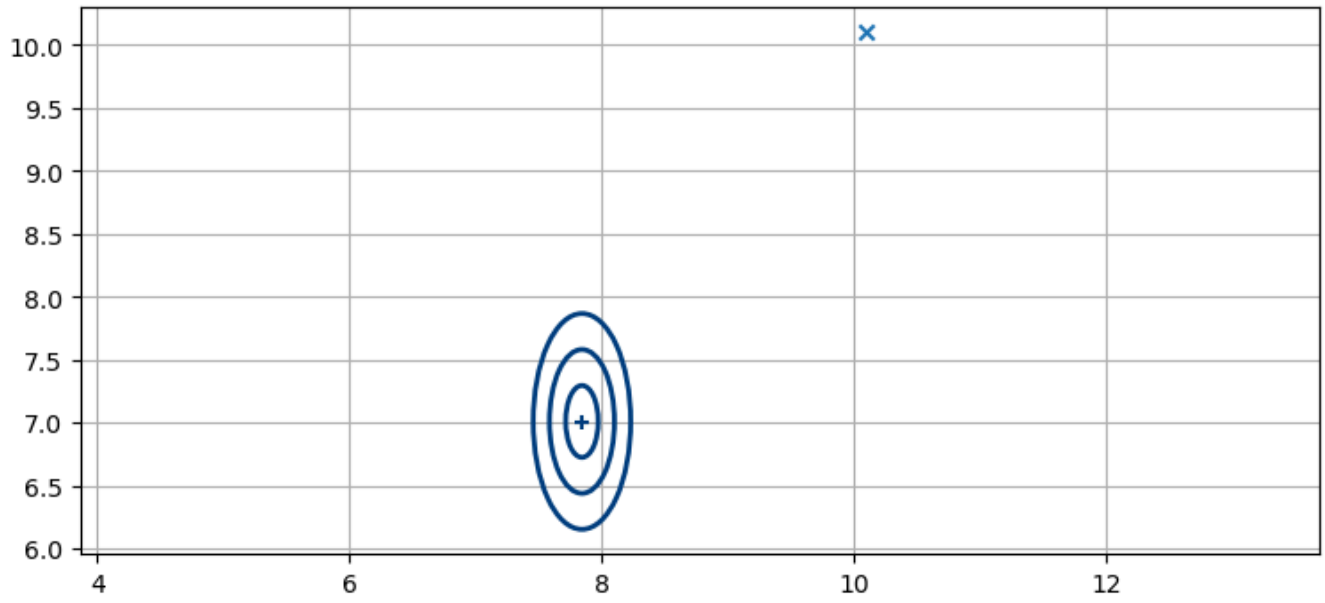
```
x, P = kf.x[0:2], kf.P[0:2, 0:2]
plot_covariance_ellipse(x, P, std=[1,2,3])
```



前面的位置协方差等高线呈圆形。这里在代码中设 $\mathbf{R} = \begin{bmatrix}0.03 & 0\\0 & 0.15\end{bmatrix}$，使 $\boldsymbol{y}$ 方向的测量噪声方差为 $\boldsymbol{x}$ 方向的 5 倍，因此 $\mathbf{P}$ 对应的等高线呈椭圆形。这样的设置略显人为，但后续章节中的椭圆形等高线通常都有具体的物理原因。

对于二维高斯分布，马氏距离不超过 3 的椭圆包含约 98.89% 的概率质量。一维均值两侧三倍标准差区间的覆盖率约为 99.73%，不能直接用于二维联合区域。下面把测量值与椭圆画在一起。

```
plot_covariance_ellipse(x, P, std=[1,2,3])
plt.scatter(z[0], z[1], marker='x');
```



显然，这个测量结果远远超出了先验值所对应的协方差范围。我们或许应该将这个测量结果视为无效数据，不予使用。那我们该怎么做呢?

一个办法是分别提取 x、y 的标准差，编写 if 条件。这里改用 KalmanFilter 的 residual_of 方法，它计算候选测量相对于当前先验的残差。本例已经调用 update()，kf.y 已有新息，所以不必另算；但若要先判定是否拒绝测量，update() 尚未执行，kf.y 保存的就还是上一时刻的新息。

先介绍术语：这里讨论的是波门筛选（gating）。波门由某个公式或算法定义，用于判定候选测量是否可以接受；只有符合条件的测量才能通过。

实际上，测量结果并不完全符合高斯分布。因此，如果以 3 个标准差作为判断标准，那么一些有效的测量结果可能会被忽略。我稍后会详细解释这一点。目前，我们暂且使用 4 个标准差作为判断标准。

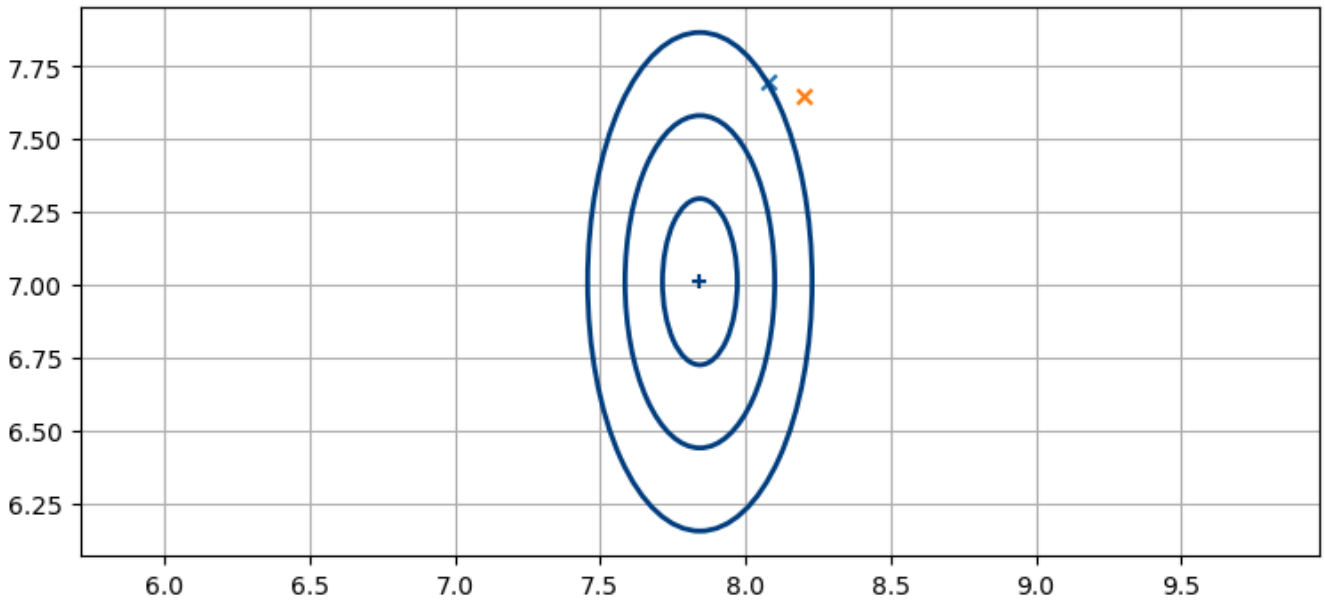
```
GATE_LIMIT = 4.
std_x = np.sqrt(P[0,0])
std_y = np.sqrt(P[1,1])
y = kf.residual_of(z)[:,0]

if y[0] > GATE_LIMIT * std_x or y[1] > GATE_LIMIT * std_y:
    print(f'discarding measurement, error is {y[0]/std_x:.0f} std, {y[1]/std_y:.0f} std')

print('y   is', y)
print(f'std is {std_x:.2f} {std_y:.2f}')
```

```
discarding measurement, error is 39 std, 18 std
y   is [5.05 5.05]
std is 0.13 0.29
```

我们看到，这些误差分别相当于 39 个和 18 个标准差。这样的结果够好吗?

也许吧。不过，请注意：if 语句所构成的区域实际上是一个围绕椭圆的矩形区域。在下图中，我画出了两个数值：其中一个明显超出了 3 个标准差的椭圆范围，但仍然会被系统接受；另一个数值则正好位于 3 个标准差的边界上。

```
plot_covariance_ellipse(x, P, std=[1,2,3])
plt.scatter(8.08, 7.7, marker='x')
plt.scatter(8.2, 7.65, marker='x');
```



定义这个“门”的方法其实还有很多种。马氏距离是一种用于衡量某一点与某个概率分布之间距离的统计量。在详细介绍定义和数学公式之前，我们先来计算几个点的马氏距离吧。 `filterpy.stats` 实现了 `mahalanobis()` 的功能。

```
from filterpy.stats import mahalanobis
m = mahalanobis(x=z, mean=x, cov=P)
print(f'mahalanobis distance = {m:.1f}')
```

```
mahalanobis distance = 20.6
```

即使不依赖原始物理单位，也可以将这个数值与 $\boldsymbol{x}$、$\boldsymbol{y}$ 方向分别计算的标准化距离 39 和 18 比较，它们处于相近量级。下面再计算前面图中所标两个点的马氏距离。

```
print(f'mahalanobis distance = {mahalanobis(x=[8.08, 7.7], mean=x, cov=P):.1f}')
print(f'mahalanobis distance = {mahalanobis(x=[8.2, 7.65], mean=x, cov=P):.1f}')
```

```
mahalanobis distance = 3.0
mahalanobis distance = 3.6
```

正如我们将看到的，马氏距离实际上是一种衡量某一点与整个分布之间的“标量标准差距离”的方法，这与欧几里得距离类似——欧几里得距离则是用来衡量两点之间的标量距离的。

上面的表格正好证明了这一点。位于 3 个标准差边界上的那个点的马氏距离为 3.0，而位于椭圆之外的那个点的马氏距离则达到了 3.6 个标准差。

我们该如何计算马氏距离呢？其定义如下：

$$D_m = \sqrt{(\mathbf{x} - \mu)^\mathsf{T}\mathbf{S}^{-1}(\mathbf{x} - \mu)}$$

请注意，这与欧几里得距离非常相似。欧几里得距离可以表示为：

$$D_e = \sqrt{(\mathbf{x} - \mathbf{y})^\mathsf{T}(\mathbf{x} - \mathbf{y})}$$

实际上，当协方差矩阵 $\mathbf{S}$ 为单位矩阵时，马氏距离就等于欧氏距离。线性代数可以直接说明：单位矩阵的逆仍是单位矩阵，相当于有关项乘以 1。从直觉看，如果各维标准差均为 1，且维间不相关，那么以均值为中心、半径为 1 的圆周上，各点的马氏距离和欧氏距离都是 1。

协方差矩阵为对角矩阵时，马氏距离就是将各坐标差分别除以相应标准差后得到的欧氏距离。因此，展开后的平方距离中，每个平方差应除以相应的方差：

$$D_m = \sqrt{\sum_{i=1}^{N} \frac{(x_i - \mu_i)^2}{\sigma_i^2}}$$

在二维空间中，情况就是如此。

$$D_m = \sqrt{\frac{1}{\sigma_x^2}(x_0 - x_1)^2 + \frac{1}{\sigma_y^2}(y_0 - y_1)^2}$$

由此可以理解马氏距离公式。令差向量为 $\mathbf{y} = \mathbf{x} - \boldsymbol{\mu}$，按协方差标准化后的平方距离就是二次型 $\mathbf{y}^{\mathsf{T}}\mathbf{S}^{-1}\mathbf{y}$。矩阵不能直接相除，而逆协方差矩阵在这里起到与标量情形除以方差相对应的作用。最后开平方，便得到无量纲的马氏距离。

波门筛选与数据关联策略

文献通常按形状将上面两种波门称为矩形波门和椭球波门。还有其他方式，这里不一一展开。例如，机动波门根据目标当前速度和机动能力，限定它下一步可能到达的区域。战斗机的波门可能像沿当前航向向前延伸的锥体；汽车的则可能是较小的二维扇形；船舶难以快速转向或加速，其扇形范围就更窄。

该选哪种波门？没有统一答案，主要取决于问题维数和计算资源。矩形波门计算量很小，机动波门也不算大；高维椭球波门的计算成本则可能较高。不过，维数越高，椭球与外接矩形区域的相对体积差通常越大。

这一差异很重要：杂波点可能落入波门，被误当成有效测量。矩形超出椭球的区域越大，误接收杂波的机会通常越高。假设杂波均匀分布，使用同半径球的外接超立方体时，五维体积比约为 6.08；实际误接收率则取决于波门阈值、形状与杂波分布，需要按具体模型计算。

如果计算时间是个问题，且存在大量无效的测量结果，可以采用两阶段处理方式。第一阶段使用一个较大的矩形结构来滤除那些明显错误的测量结果。只有通过第一阶段滤波的测量结果，才会被进一步处理，即进行复杂的马氏距离计算。如果在现代台式处理器上运行，这种矩阵运算所花费的时间可以忽略不计；但如果使用的是浮点运算能力有限的嵌入式芯片，那么这一过程可能会影响整体性能。

数据关联是一个足以独立成书的广泛主题。典型例子是雷达跟踪：每次扫描检测到多个回波，我们要据此建立飞机航迹，并排除杂波和噪声。这非常困难。假设第一次扫描得到 5 个测量，可建立 5 条候选航迹；第二次得到 6 个测量，与此前每一个测量配对，就有 30 种候选延伸。还须考虑新出现的飞机，另外建立 6 条新航迹。若保留所有组合，经过少数几个时间步，候选数量就可能增长到数百万乃至数十亿。

有了这数十亿条轨迹数据后，我们就可以为每条轨迹计算出一个得分。具体的计算方法我会在后面详细说明。不过，可以想象一下：如果某条轨迹在三个时间点上呈现出“Z”形，那这种轨迹是不可能存在的，因为没有任何飞机能够以这种方式移动。另一条轨迹虽然呈直线形状，但其速度却高达 10,000 公里/小时，这也显然不合常理。而另一条轨迹以 200 公里/小时的速度移动，这种可能性则相对较高。

因此，跟踪包含波门筛选、数据关联和剪枝。例如第二次雷达扫描刚完成，是否应把所有点都组合成候选航迹？不必。如果第一次扫描的点 1 与第二次扫描的点 3 之间，对应速度为 200 千米/小时，就可以构成候选航迹；若对应速度高达 5000 千米/小时，在本例的目标模型下就可以排除。随着航迹积累，可以建立明确的椭圆波门或机动波门，更有选择地关联测量。

对于各种轨迹，我们有不同的处理方式。我们可以选择将某次测量结果仅与某一条轨迹关联起来。或者，我们也可以选择将同一测量结果与多条轨迹关联起来，以此来反映我们无法确定该测量结果究竟属于哪条轨迹的事实。例如，从雷达的角度来看，飞机的轨迹可能会相互交叉。当飞机靠近时，就很难确定某次测量结果究竟属于哪架飞机。这时，我们可以暂时将该测量结果与两条轨迹都关联起来。随着更多测量数据的收集，我们可以根据新获得的情报来重新确定该测量结果应归属于哪条轨迹。

“数十亿”这个数字远远无法体现其中所涉及的复杂组合情况。只需几秒钟，计算机的内存就将被耗尽；再过一会儿，就需要利用宇宙中的所有原子来存储所有可能的路径了。因此，实用的算法必须对各种路径进行有效的筛选。而这种筛选过程本身又会消耗额外的计算资源。

本书的后面章节提出了解决这一问题的现代方法——粒子滤波法。该方法通过统计抽样来克服组合爆炸问题。这是我处理这类问题的首选方法，因此我就不再详细阐述本节中提到的各种技术了。由于我并不完全了解该领域的最新研究成果，如果你需要解决涉及多个物体的跟踪或处理大量错误测量数据的问题，建议你自行进行研究。当然，粒子滤波法也有其自身的局限性和难点。

下面推荐几位研究者及其著作。Samuel S. Blackman 的《Multiple-Target Tracking with Radar Applications》是我读过的同主题著作中讲解最清楚的一本，虽然出版较早（1986 年）。Yaakov Bar-Shalom 也有大量相关论著。Subhash Challa 等人的《Fundamentals of Object Tracking》较为新近，涵盖多种方法，数学表述也较严谨：各种滤波器常被写成体现贝叶斯形式的积分，读者须自行把这些表达式转化为可运行的算法。掌握本书数学内容后可以阅读，但并不轻松。Lawrence D. Stone 的《Bayesian Multiple Target Tracking》也将问题视为贝叶斯推断，不过同样偏重理论，有时只要求求出某个复杂积分表达式的最大值；实际计算可能需要粒子滤波等方法。

回到我们最初的问题——如何追踪一个物体，同时应对那些偶尔出现的错误测量结果。其实解决方法很简单：如果某次测量结果有误，就直接忽略它，不要进行更新操作。这样一来，`predict()` 会被调用两次，但这没什么问题。虽然这样会导致不确定性增加，但偶尔的几次错误测量通常不会造成什么麻烦。

波门阈值该取多少？没有统一答案。三倍标准差是常见起点，但未必适合实际数据，且多维波门还要考虑维数。应收集数据，以不同阈值运行滤波器，再比较效果。下一节介绍性能评价方法。也许你的问题适合接受马氏距离小于 4.5 的测量；我看过的 NASA 视频则提到约 5～6 的阈值。最终取值取决于问题、数据及允许的误拒绝率。

评估滤波器的性能

为仿真情形设计卡尔曼滤波器相对容易：你知道过程模型中注入了多少噪声，因此可以构造相应的 $\mathbf{Q}$；也知道测量仿真中的噪声，因此容易确定测量噪声协方差矩阵 $\mathbf{R}$。

在实践中，设计往往更为复杂。实际的传感器很少能完全符合技术规格要求，其性能也很难符合高斯分布规律。此外，传感器还很容易受到环境噪声的干扰。例如，电路中的噪声会导致电压波动，从而影响传感器的输出信号。要建立准确的系统模型并考虑噪声因素则更加困难。以汽车为例，其运行过程极其复杂：转向操作会导致非线性行为，轮胎容易打滑，驾驶员的急刹车或加速也会导致轮胎打滑，风力则可能使汽车偏离行驶轨迹。最终，卡尔曼滤波器实际上只是对系统的一种近似描述而已。这种不准确性会导致系统性能不佳，最糟糕的情况下，甚至会导致滤波器完全失效。

由于存在未知因素，通常无法完全通过解析计算得到滤波器各矩阵的正确参数。先依据已有知识作尽可能合理的估计，再用多种仿真数据和真实数据测试，根据性能评价调整矩阵。此前已经展示过 $\mathbf{Q}$ 过大或过小的影响。

现在，让我们来探讨一些更有效的性能分析方法。如果卡尔曼滤波器的运行效果理想，那么其估计误差（即真实状态与估计状态之间的差异）应具备以下特性：

1. 估计误差的均值为零。
2. 估计误差的协方差由卡尔曼滤波器的协方差矩阵描述。

归一化估计误差平方（NEES）

第一种方法最有力，但只有在知道真值时才可用，通常是在仿真中。仿真时我们知道系统的真实状态，也称 ground truth。于是，任意时刻的估计误差都可由真值 $\mathbf{x}$ 与滤波状态估计 $\hat{\mathbf{x}}$ 的差直接求出：

$$\tilde{\mathbf{x}} = \mathbf{x} - \hat{\mathbf{x}}$$

接着定义归一化估计误差平方（NEES）：

$$\epsilon = \tilde{\mathbf{x}}^{\mathsf{T}} \mathbf{P}^{-1} \tilde{\mathbf{x}}$$

为了理解这个方程，我们先假设该状态的维度为 1。在这种情况下，x 和 P 都是标量，因此……

$$\epsilon = \frac{x^2}{P}$$

如果还不清楚，请回忆：对于标量 a，有 $a^{\mathsf{T}} = a$，并且在非零时有 $a^{-1} = \frac{1}{a}$。

对同样大小的估计误差，如果滤波器给出的协方差更小，NEES 就会更大。协方差表示滤波器所报告的不确定性；若它相对于实际误差过小，就说明滤波器过度自信。不能仅因协方差较大便认定滤波性能更好，还须考察估计精度及不确定性的一致性。

这个计算结果是一个标量值。如果 $\mathbf{x}$ 的维度为 $(n \times 1)$，那么该计算的维度就是 $(1 \times n) \times (n \times n) \times (n \times 1) = (1 \times 1)$。那我们该如何处理这个数值呢？

相关数学超出本书范围。若估计误差为零均值高斯向量，且 P 正确描述其非奇异协方差，那么 $\tilde{\mathbf{x}}^{\mathsf{T}} \mathbf{P}^{-1} \tilde{\mathbf{x}}$ 服从自由度为状态维数的卡方分布，其期望为 n。Bar-Shalom [1] 对此有很好的讨论。

多次独立仿真得到的 NEES 平均值应与状态向量 x 的维数相符，应按卡方分布构造置信区间进行检验，而不是认为数值越小越好。明显偏高可能表示滤波器过度自信，明显偏低则可能表示协方差估计过于保守。下面用本章前面的例子说明。

```
from scipy.linalg import inv

def NEES(xs, est_xs, Ps):
    est_err = xs - est_xs
    err = []
    for x, p in zip(est_err, Ps):
        err.append(x.T @ inv(p) @ x)
    return err
```

```
R, Q = 6., 0.02
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)
kf2 = SecondOrderKF(R, Q, dt=1)
est_xs, ps, _, _ = kf2.batch_filter(zs)

nees = NEES (xs, est_xs, ps)
eps = np.mean(nees)

print(f'mean NEES is: {eps:.4f}')
if eps < kf2.dim_x:
    print('passed')
else:
    print('failed')
```

```
mean NEES is: 0.8893
passed
```

NEES 是通过 FilterPy 来实现的；可以通过相应的方式来访问它。

```
from filterpy.stats import NEES
```

NEES 是评价滤波器一致性的有力工具，在能够取得真实状态时应尽量使用。设计滤波器时，我仍喜欢绘制估计误差图，因为更直观。实际在线运行时若没有可靠真值，就无法直接计算 NEES；不能仅凭滤波器自身输出得到它。

不过，如果你的模拟精度有限，那就需要采用另一种方法了。

似然函数

统计学中的似然与概率密切相关，但侧重点不同。概率问题可以是：假定骰子公平，投掷五次恰有三次出现 6 点的概率是多少？似然则固定已经观察到的数据，比较不同骰子模型生成这些数据的可能程度。例如，观察到五次中三次为 6 后，可以比较公平骰子模型与其他模型的似然。似然不是“骰子公平的概率”；若要得到模型的后验概率，还需要先验等信息。

我们最早在“离散贝叶斯”这一章节中讨论了似然函数。在这些滤波器的背景下，似然函数用来衡量在当前状态下，获得某一测量结果的概率有多大。

我们已有滤波器输出，希望判断测量数据与当前线性、高斯模型及其参数是否相容。若似然很低，说明该测量在当前预测分布下很少见，可能需要检查模型、参数或测量质量。“自适应滤波”一章将讨论如何利用这些信息改进滤波器；这里只介绍这一指标的计算。

滤波器的新息为实际测量减去预测测量；新息协方差则为投影到测量空间的先验协方差加上测量噪声协方差：

$$\begin{aligned}\mathbf{y} &= \mathbf{z} - \mathbf{H}\bar{\mathbf{x}} \\ \mathbf{S} &= \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}\end{aligned}$$

据此可计算测量似然。对 m 维测量，它是均值为零、协方差为 S 的多元高斯密度在新息处的值：

$$\mathcal{L} = \frac{1}{\sqrt{(2\pi)^m \det(\mathbf{S})}} \exp\left[-\frac{1}{2}\mathbf{y}^\mathsf{T}\mathbf{S}^{-1}\mathbf{y}\right]$$

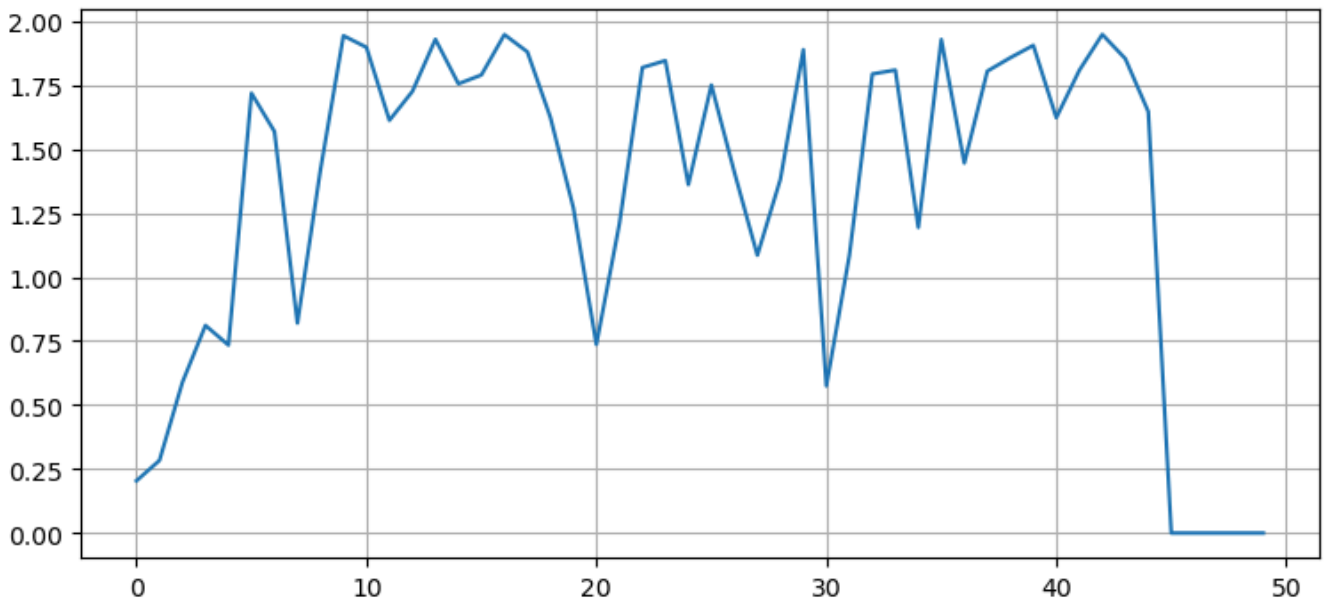
虽然看起来很复杂，但请注意，其中的指数函数实际上就是高斯分布的数学表达式。这就为该算法的实现提供了依据。

```
from scipy.stats import multivariate_normal
hx = np.dot(H, x).flatten()
S = np.dot(H, P).dot(H.T) + R
likelihood = multivariate_normal.pdf(z.flatten(), mean=hx, cov=S)
```

实际中通常采用稍有不同的做法。似然在数学上有时不好直接处理，因此常计算它的自然对数，即对数似然。这有几个好处。首先，对数函数严格递增，所以正似然函数与它的对数在同一点取得最大值。求极值通常需要求导；对数变换有时能使复杂乘积的求导更容易。注意，$\frac{d}{dx}log(f(x))$ 并不等于 $\frac{d}{dx}f(x)$；前者实际是 f'(x)/f(x)。在 f(x)>0 且可微的范围内，它们的驻点条件等价。本书不使用这一求导性质，但它在滤波器分析中很重要。

调用 update() 后，可以通过 likelihood 与 log_likelihood 属性获取测量似然及对数似然。下面先使用若干预期范围内的测量运行滤波器，再加入远离预测值的测量，观察结果。

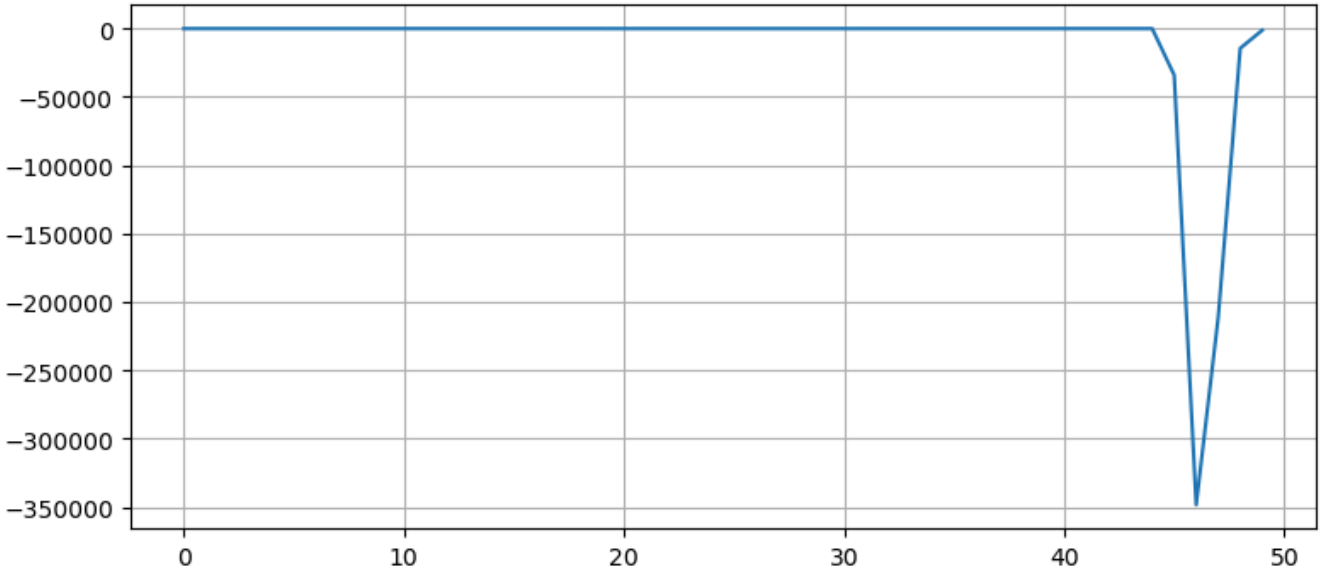
```
R, Q = .05, 0.02
xs, zs = simulate_acc_system(R=R, Q=Q, count=50)
zs[-5:-1] = [100, 200, 200, 200] # bad measurements, bad!

kf = SecondOrderKF(R, Q, dt=1, P=1)
s = Saver(kf)
kf.batch_filter(zs, saver=s)
plt.plot(s.likelihood);
```



最初几次迭代中，滤波器逐渐收敛，测量似然随之增大。之后似然上下波动，直到遇到异常测量时变得极小，数值上可能下溢为零。如果这些测量本身可信，就说明它们在当前滤波模型下很难出现，应检查模型与参数。似然低本身并不能证明某一个具体假设必然错误。

可以看出，对数似然值清楚地表明了滤波器在哪些地方出现了“异常行为”。

```
plt.plot(s.log_likelihood);
```



为什么曲线末尾又接近零？先想一想再看解释。滤波器开始适应新的测量，使状态估计逐渐靠近这些测量；新息因而减小，预测与测量重新趋于一致。对数似然取决于新息及其协方差，并不必然回到零；这里描述的是示例曲线的表现。

控制输入

在“离散贝叶斯”这一章中，我介绍了利用控制信号来提升滤波器性能的方法。我们不再假设物体会像之前那样持续运动，而是利用对控制输入的了解来预测物体的位置。在“单变量卡尔曼滤波器”这一章中，我们也采用了同样的思路。卡尔曼滤波器的预测功能就是基于这一原理来实现的。

```
def predict(pos, movement):
return (pos[0] + movement[0], pos[1] + movement[1])
```

在上一章中，我们了解到状态预测的方程式为：

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$$

我们的状态是一个向量，因此我们需要将控制输入也表示为向量形式。这里，$\mathbf{u}$ 代表控制输入，而 $\mathbf{B}$ 则是一个矩阵，该矩阵的作用是将控制输入转换为 $\mathbf{x}$ 的变化。我们来看一个简单的例子。假设我们所控制的机器人的状态为 $\boldsymbol{x} = \begin{bmatrix}\boldsymbol{x} & \dot{\boldsymbol{x}}\end{bmatrix}$，而控制输入则是指令速度。这样一来，我们的控制输入就为

$$\mathbf{u} = \begin{bmatrix}\dot{\boldsymbol{x}}_{\mathbf{cmd}}\end{bmatrix}$$

为简化问题，我们假设机器人能够立即对输入参数的变动做出反应。也就是说，经过 $\boldsymbol{\Delta t}$ 秒后，机器人的新位置和速度将会是……

$$\begin{aligned}\boldsymbol{x} &= \boldsymbol{x} + \dot{\boldsymbol{x}}_{\mathbf{cmd}}\boldsymbol{\Delta t} \\ \dot{\boldsymbol{x}} &= \dot{\boldsymbol{x}}_{\mathbf{cmd}}\end{aligned}$$

我们需要将这组方程表示为 $\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$ 的形式。

我将使用 $\mathbf{Fx}$ 这个项来提取出最上方那个方程中的 $\boldsymbol{x}$；而对于其他方程，则使用 $\mathbf{Bu}$ 这个项。具体如下：

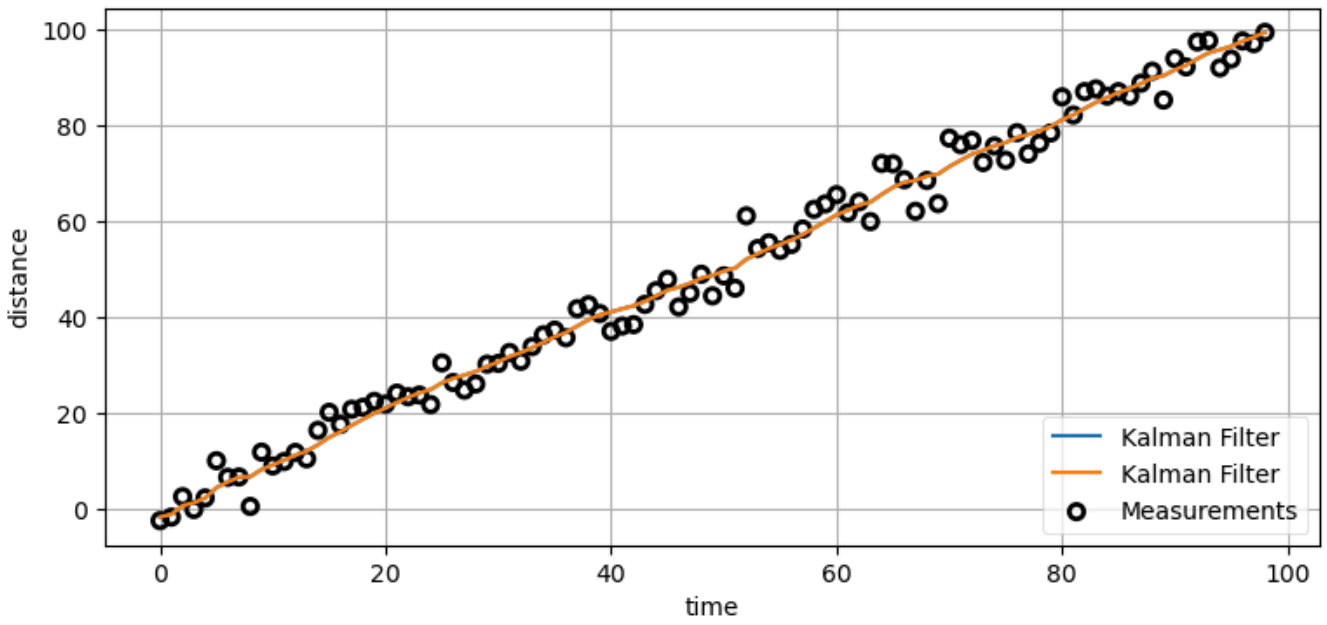
$$\begin{bmatrix}\boldsymbol{x} \\ \dot{\boldsymbol{x}}\end{bmatrix} = \begin{bmatrix}1 & 0 \\ 0 & 0\end{bmatrix}\begin{bmatrix}\boldsymbol{x} \\ \dot{\boldsymbol{x}}\end{bmatrix} + \begin{bmatrix}\Delta t \\ 1\end{bmatrix}\begin{bmatrix}\dot{\boldsymbol{x}}_{\mathbf{cmd}}\end{bmatrix}$$

这是一种简化的描述；实际上，典型的控制输入包括转向角的调整以及加速度的变化。这些因素都会引入非线性效应，我们将在后面的章节中学习如何处理这些非线性问题。

卡尔曼滤波器的其余部分将按照常规方式来设计。你们已经多次见过这样的设计了，所以这里就不赘述了。下面是一个示例。

```
dt = 1.
R = 3.
kf = KalmanFilter(dim_x=2, dim_z=1, dim_u = 1)
kf.P *= 10
kf.R *= R
kf.Q = Q_discrete_white_noise(2, dt, 0.1)
kf.F = np.array([[1., 0], [0., 0.]])
kf.B = np.array([[dt], [ 1.]])
kf.H = np.array([[1., 0]])
print(kf.P)

zs = [i + randn()*R for i in range(1, 100)]
xs = []
cmd_velocity = np.array([1.])
for z in zs:
    kf.predict(u=cmd_velocity)
    kf.update(z)
    xs.append(kf.x[0])

plt.plot(xs, label='Kalman Filter')
plot_measurements(zs)
plt.xlabel('time')
plt.legend(loc=4)
plt.ylabel('distance');
```

```
[[10.  0.]
 [ 0. 10.]]
```



传感器融合

在 g-h 滤波一章开头，我们讨论过同时使用两台秤：一台准确，另一台不太准确。结论是，即使较不准确的秤也包含有用信息，不应随意丢弃。现在假设有两个传感器测量同一个系统，该怎样将它们纳入卡尔曼滤波器?

假设有一列火车或一辆车厢在铁轨上行驶。火车的轮子上装有传感器，可以记录轮子的转动次数，进而换算出火车在铁轨上的行驶距离。此外，火车上还装有类似 GPS 的传感器，我将其称为“位置传感器”，该传感器能够反馈火车的位置信息。我会在下一节中解释为什么不直接使用 GPS。这样，我们就有了两种能够反映火车在铁轨上位置的测量数据。假设轮子传感器的精度为 1 米，而位置传感器的精度为 10 米。那么，我们该如何将这两种测量数据结合起来呢？虽然这看起来有些复杂，但实际上，飞机也是通过融合来自 GPS、惯性导航系统、多普勒雷达、VOR、空速计等多种传感器的信息来获取精确位置的。

对于惯性系统而言，使用卡尔曼滤波器来处理数据相当困难。不过，将两个或更多传感器所测得的同一状态变量（比如位置）的数据进行融合则相对容易。相关的矩阵就是测量矩阵 $\mathbf{H}$。需要记住的是，该矩阵告诉我们如何将卡尔曼滤波器所计算出的状态 $\mathbf{x}$ 转换为可测量的数值 $\mathbf{z}$。假设我们决定让卡尔曼滤波器的状态包含火车的位置和速度信息，那么……

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

我们有两种用于确定位置的测量方式，因此，我们将把测量向量定义为来自车轮和位置传感器的各项测量数值所构成的向量。

$$\mathbf{z} = \begin{bmatrix} z_{wheel} \\ z_{ps} \end{bmatrix}$$

因此，我们必须设计矩阵 $\mathbf{H}$，以便将 $\mathbf{x}$ 转换为 $\mathbf{z}$。由于两者都属于位置信息，所以这种转换实际上就是乘以 1 而已。

$$\begin{bmatrix} z_{wheel} \\ z_{ps} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

为了更清楚地说明，假设轮子不报告其具体位置，而是报告其旋转的圈数。其中，每旋转一圈，轮子就会前进 2 米。在这种情况下，我们可以这样表示：

$$\begin{bmatrix} z_{rot} \\ z_{ps} \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

接下来设计测量噪声协方差 $\mathbf{R}$。车轮传感器折算为距离后的标准差为 1.5 m，位置传感器的标准差取为其两倍，即 3 m，因此对应方差分别为 2.25 和 9 平方米。这里采用以位置为测量量的模型；若采用前一段的车轮转数读数，还须相应换算噪声方差的单位。

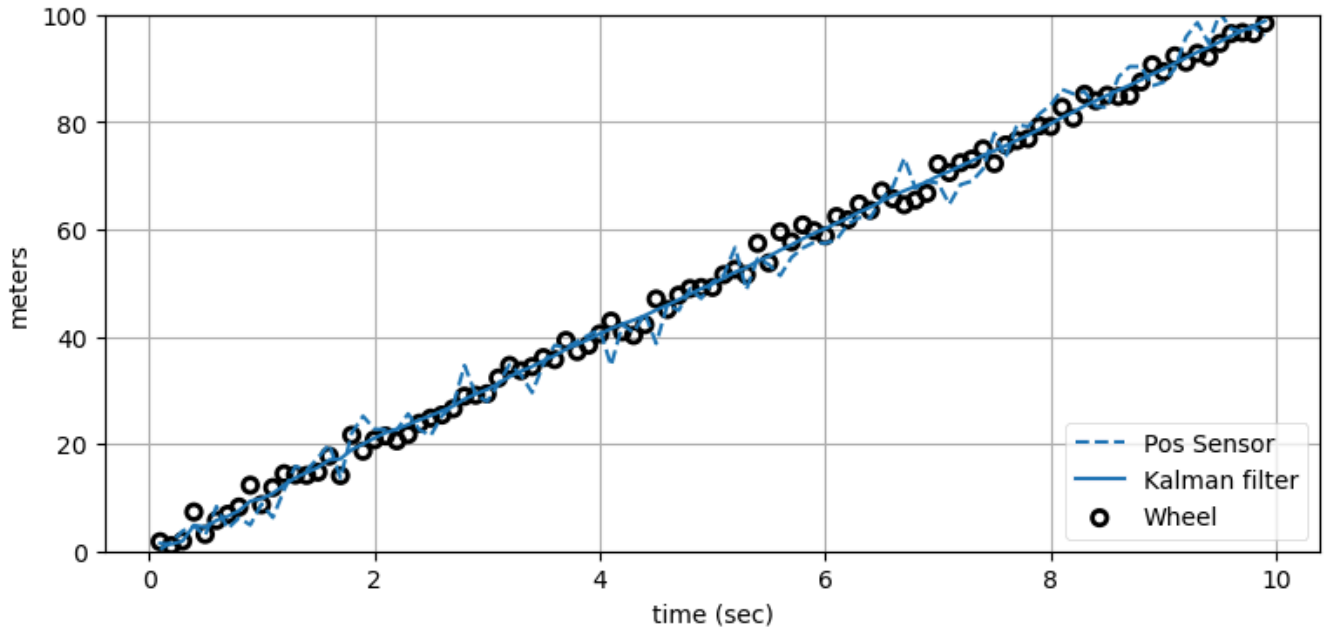
$$\begin{aligned}
\sigma_{wheel} &= 1.5 \\
\sigma^2_{wheel} &= 2.25 \\
\sigma_{ps} &= 1.5 * 2 = 3 \\
\sigma^2_{ps} &= 9.
\end{aligned}$$

滤波器的主体设计至此完成。还需确定 **Q**，但其构造不因是否融合多个传感器而改变，因此这里选择一个示例值。

下面仿真这一设计，假设速度为 10 m/s，更新间隔为 0.1 秒。

```
from numpy import array, asarray
import numpy.random as random

def fusion_test(wheel_sigma, ps_sigma, do_plot=True):
    dt = 0.1
    kf = KalmanFilter(dim_x=2, dim_z=2)

    kf.F = array([[1., dt], [0., 1.]])
    kf.H = array([[1., 0.], [1., 0.]])
    kf.x = array([[0.], [1.]])
    kf.Q *= array([[(dt**3)/3, (dt**2)/2],
                   [(dt**2)/2,  dt      ]]) * 0.02
    kf.P *= 100
    kf.R[0, 0] = wheel_sigma**2
    kf.R[1, 1] = ps_sigma**2
    s = Saver(kf)

    random.seed(1123)
    for i in range(1, 100):
        m0 = i + randn()*wheel_sigma
        m1 = i + randn()*ps_sigma
        kf.predict()
        kf.update(array([[m0], [m1]]))
        s.save()
    s.to_array()
    print(f'fusion std: {np.std(s.y[:, 0]):.3f}')
    if do_plot:
        ts = np.arange(0.1, 10, .1)
        plot_measurements(ts, s.z[:, 0], label='Wheel')
        plt.plot(ts, s.z[:, 1], ls='--', label='Pos Sensor')
        plot_filter(ts, s.x[:, 0], label='Kalman filter')
        plt.legend(loc=4)
        plt.ylim(0, 100)
        set_labels(x='time (sec)', y='meters')

fusion_test(1.5, 3.0)
```

```
fusion std: 1.647
```



我们可以用蓝色来表示卡尔曼滤波的处理结果。

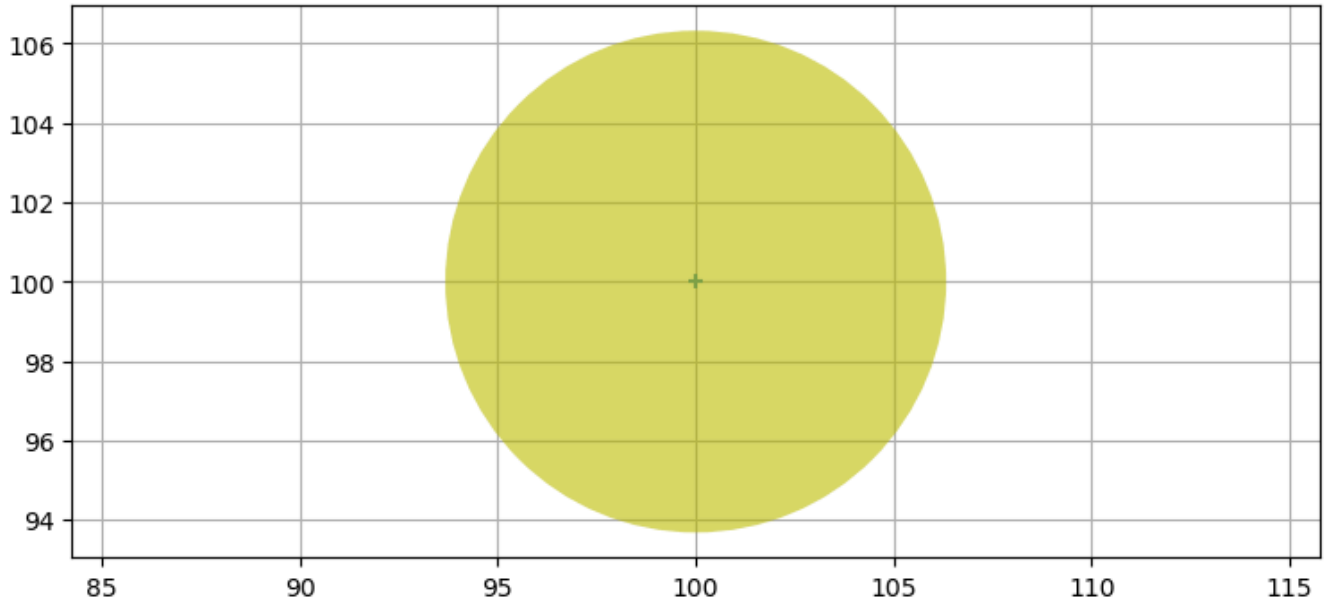
从直观的角度来看，可能有点难以理解上面的例子。让我们来看另一个例子吧。假设我们正在二维空间中追踪某个物体，同时有两组雷达系统分别位于不同的位置。每组雷达系统都能提供关于该目标的距离和方位信息。那么，来自这两组数据的信息会如何影响最终的测量结果呢?

这是一个非线性问题，因为我们需要利用三角函数来根据距离和方位角来计算坐标。而我们目前还不知道如何使用卡尔曼滤波器来求解非线性问题。因此，在解决这个问题时，请先忽略我所使用的代码，只需关注代码输出的结果即可。我们将在后面的章节中再回到这个问题上，学习如何编写这样的代码。

我会将目标定位在(100, 100)处。第一个雷达将位于(50, 50)处，第二个雷达则位于(150, 50)处。这样一来，第一个雷达测得的方位角为 45 度，而第二个雷达测得的方位角则为 135 度。

我先会构建卡尔曼滤波器，然后再绘制其初始协方差矩阵。我使用的是无迹卡尔曼滤波器，这一内容将在后面的章节中详细讲解。

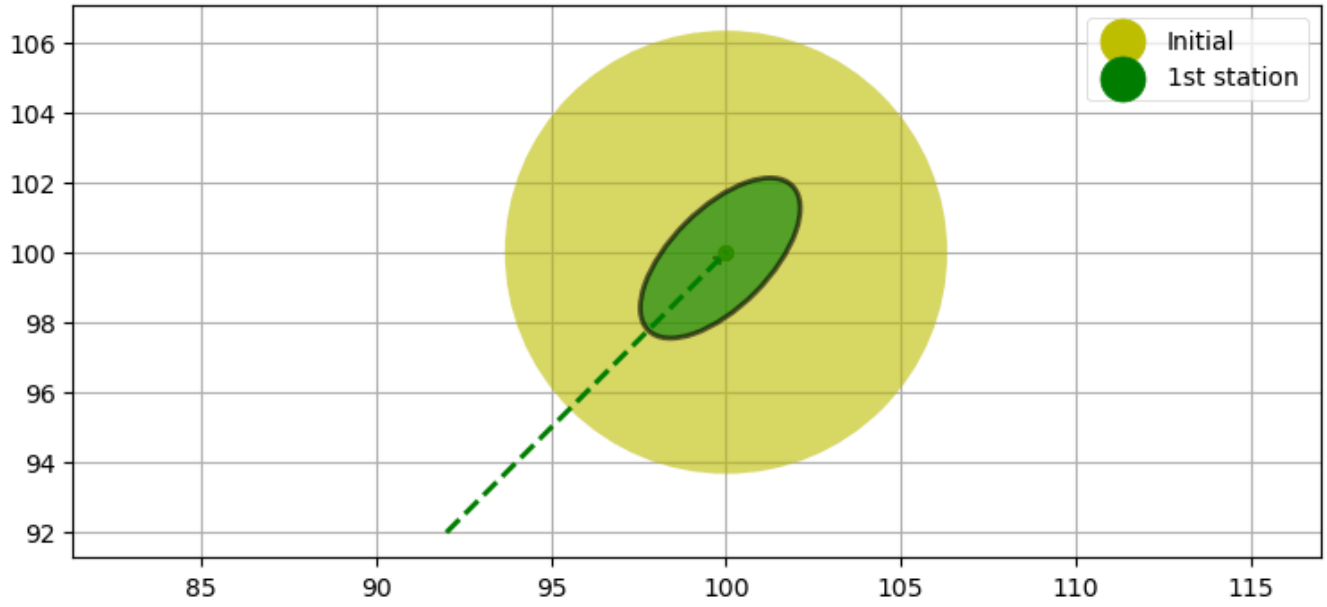
```
from kf_book.kf_design_internal import sensor_fusion_kf

kf = sensor_fusion_kf()
x0, p0 = kf.x.copy(), kf.P.copy()
plot_covariance_ellipse(x0, p0, fc='y', ec=None, alpha=0.6)
```



我们对物体在 x 轴和 y 轴上的位置同样不确定，因此协方差呈圆形分布。

现在使用第一部雷达的一次读数更新卡尔曼滤波。将方位角误差的标准差设为 0.5°，距离误差的标准差设为 3。

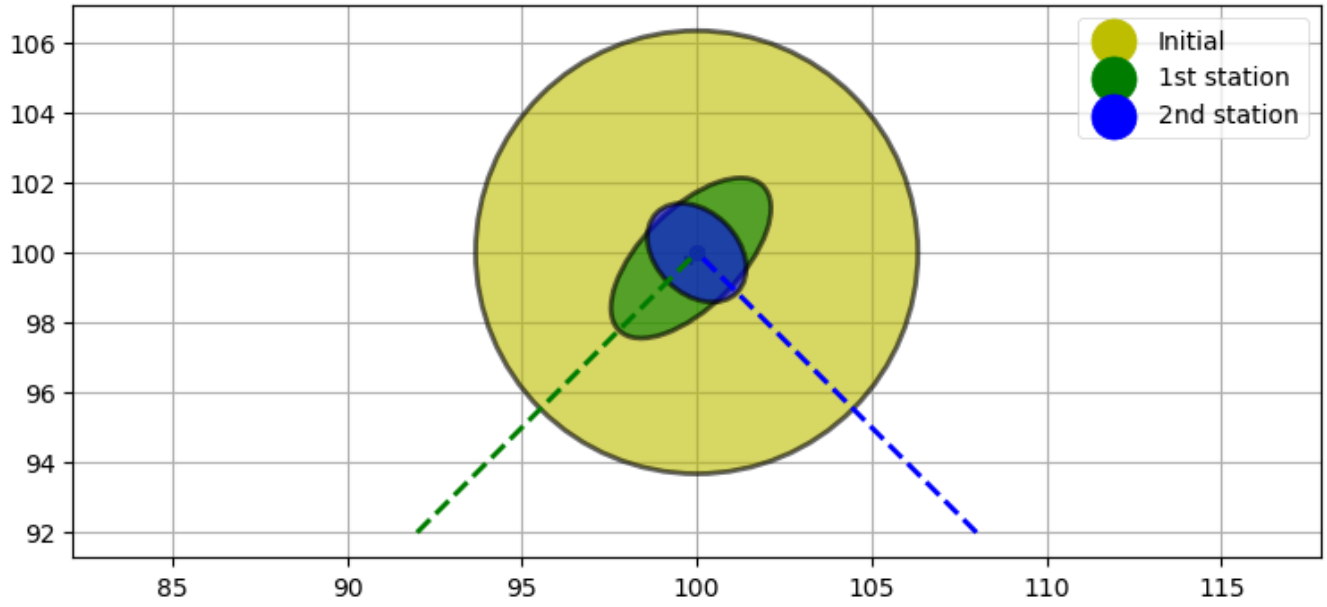
```
from math import radians
from kf_book.kf_design_internal import sensor_fusion_kf, set_radar_pos

# set the error of the radar's bearing and distance
kf.R[0, 0] = radians (.5)**2
kf.R[1, 1] = 3.**2

# compute position and covariance from first radar station
set_radar_pos((50, 50))
dist = (50**2 + 50**2) ** 0.5
kf.predict()
kf.update([radians(45), dist])

# plot the results
x1, p1 = kf.x.copy(), kf.P.copy()

plot_covariance_ellipse(x0, p0, fc='y', ec=None, alpha=0.6)
plot_covariance_ellipse(x1, p1, fc='g', ec='k', alpha=0.6)

plt.scatter([100], [100], c='y', label='Initial')
plt.scatter([100], [100], c='g', label='1st station')
plt.legend(scatterpoints=1, markerscale=3)
plt.plot([92, 100], [92, 100], c='g', lw=2, ls='--');
```



误差对问题几何形状的影响很明显。雷达位于目标左下方，方位角测量很精确，标准差为 $\sigma = 0.5°$；距离测量相对较粗，标准差为 $\sigma = 3$。绿色虚线表示雷达读数。协方差椭圆的形状清楚地反映了方位角较准、距离较不准的特点。

现在纳入第二部雷达的测量。第二部雷达位于 (150,50)，也就是目标的右下方。在继续阅读前，请先思考：加入这一新测量后，协方差会怎样变化？

```
# compute position and covariance from second radar station
set_radar_pos((150, 50))
kf.predict()
kf.update([radians(135), dist])

plot_covariance_ellipse(x0, p0, fc='y', ec='k', alpha=0.6)
plot_covariance_ellipse(x1, p1, fc='g', ec='k', alpha=0.6)
plot_covariance_ellipse(kf.x, kf.P, fc='b', ec='k', alpha=0.6)

plt.scatter([100], [100], c='y', label='Initial')
plt.scatter([100], [100], c='g', label='1st station')
plt.scatter([100], [100], c='b', label='2nd station')
plt.legend(scatterpoints=1, markerscale=3)
plt.plot([92, 100], [92, 100], c='g', lw=2, ls='--')
plt.plot([108, 100], [92, 100], c='b', lw=2, ls='--');
```



第二部雷达的测量改变了协方差。两部雷达到目标的视线彼此垂直，因此距离与方位角不确定性在位置空间中对应的方向互换。更新后，协方差椭圆的主轴方向也随之改变。更重要的是，椭圆所表示的不确定性范围显著缩小，并非只有朝向变化；协方差矩阵的维数并没有改变。

协方差始终会包含所有可用的信息，包括与问题所涉及的几何结构相关的各种因素。这种处理方式使得理解其运作原理变得十分容易。不过，即便是一个传感器提供位置信息，另一个传感器提供速度信息，或者两个传感器都提供位置测量数据，情况也是一样的。

继续之前还须说明：传感器融合是一个广泛主题，这里的讲解十分简化，甚至可能造成误解。例如，GPS 接收机可以对卫星伪距读数进行迭代最小二乘求解，直接确定位置，无须使用卡尔曼滤波。配套笔记本《Iterative Least Squares for Sensor Fusion》介绍了这个问题。

这其实是 GPS 接收机进行此类计算的常用方法，但并非唯一的方法。如果你只是个业余爱好者，我的介绍应该能帮你入门。而商用级别的滤波器则需要对各种处理流程进行精心设计。这一主题已经有很多相关书籍进行了探讨，你需要找一本与你的领域相关的书来进一步学习。

练习：能否对 GPS 输出再次滤波？

在上面的例子中，我让你将卡尔曼滤波器应用于一种“类似 GPS 的”传感器。那么，你能否将卡尔曼滤波器应用于商用卡尔曼滤波器的输出结果呢？换句话说，你设计的滤波器所输出的数值，会优于、劣于还是等于 GPS 的输出结果呢？

解答

许多商用 GPS 接收机内置卡尔曼滤波器，报告的是其滤波后的估计。假设接收机持续输出位置和位置不确定性，能否将这两项直接作为自己滤波器的输入？

那么，该数据流具有哪些特征呢？更重要的是，卡尔曼滤波器的输入数据应满足哪些基本要求呢？

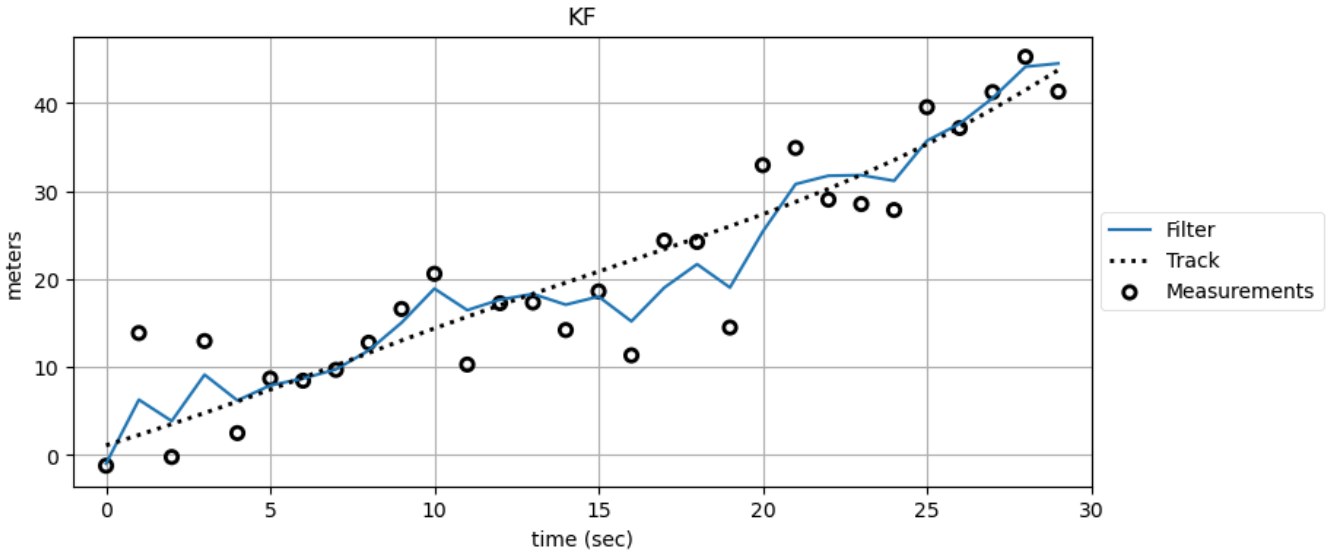
关键在于测量误差的统计特性。标准卡尔曼滤波模型通常对过程噪声和测量噪声作白噪声等假设，而不要求真实运动信号在时间上独立。GPS 接收机的滤波估计融合了过去观测，其输出误差可能在时间上相关。若将其直接当成独立原始测量，并把接收机报告的协方差用作新的 R，可能重复使用信息，破坏统计模型。若要再次滤波或融合，应正确建模时间相关性及交叉相关性，或者加入新的独立信息。

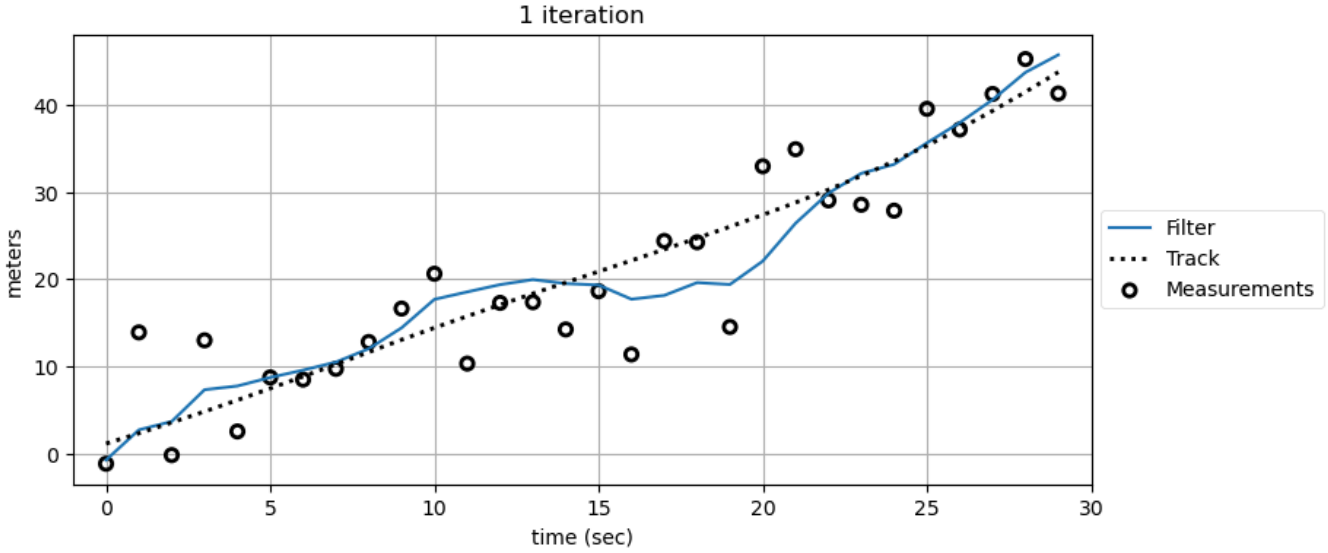
从最优性的角度看，在相同模型、相同可用信息和相同误差准则下，已经最优的估计不能仅靠重复处理而变得更优。不过，增加新信息、采用更准确的模型，或利用未来观测进行平滑，都会改变问题条件，不能用这一论点排除这些改进。

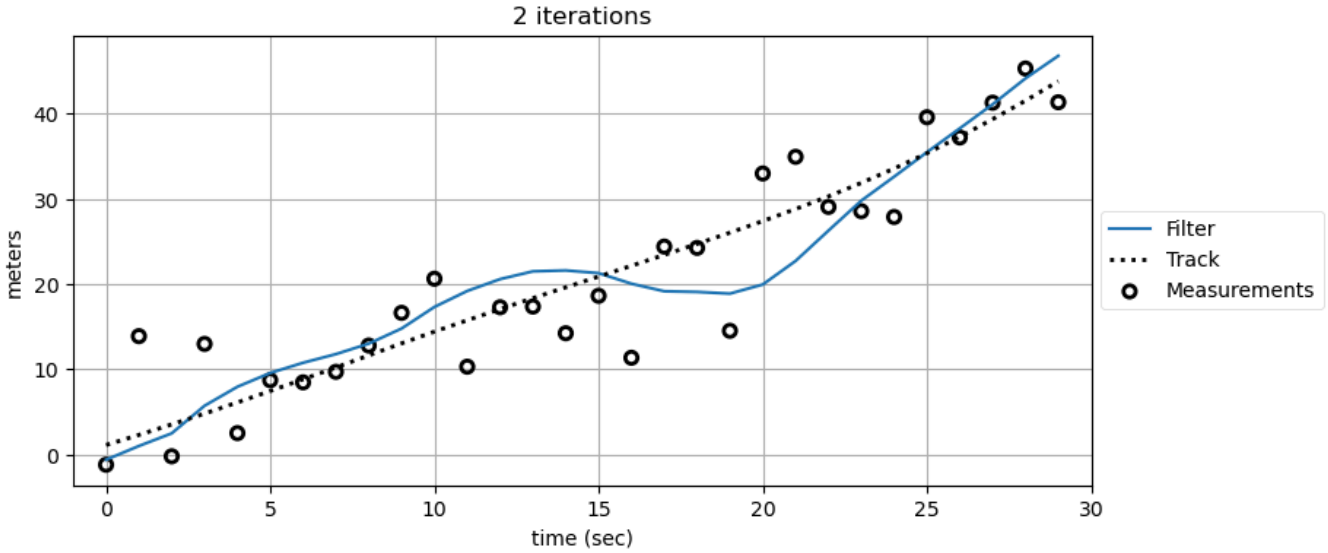
这是业余爱好者在尝试将 GPS、惯性测量单元以及其他现成传感器整合在一起时所面临的难题。

看看这样做的影响。商用 GPS 会报告位置和估计误差范围，这个范围可由卡尔曼滤波的 $\mathbf{P}$ 矩阵推算。下面先对带噪数据滤波，再把滤波输出当作新的带噪输入送回滤波器，观察结果。也就是说，以 $\mathbf{x}$ 提供 $\mathbf{z}$ 输入，以 $\mathbf{P}$ 提供测量协方差 $\mathbf{R}$。为放大效果，我会画出这样做一次、两次的结果。第二次重复并没有实际意义，也不是建议使用的方法，只是为了说明问题。下面先给出代码与图。

```
np.random.seed(124)
R = 5.
xs, zs = simulate_acc_system(R=R, Q=Q, count=30)

kf0 = SecondOrderKF(R, Q, dt=1)
kf1 = SecondOrderKF(R, Q, dt=1)
kf2 = SecondOrderKF(R, Q, dt=1)

# Filter measurements
fxs0, ps0, _, _ = kf0.batch_filter(zs)

# filter twice more, using the state as the input
fxs1, ps1, _, _ = kf1.batch_filter(fxs0[:, 0])
fxs2, _, _, _ = kf2.batch_filter(fxs1[:, 0])

plot_kf_output(xs, fxs0, zs, 'KF', False)
plot_kf_output(xs, fxs1, zs, '1 iteration', False)
plot_kf_output(xs, fxs2, zs, '2 iterations', False)
R,Q
```





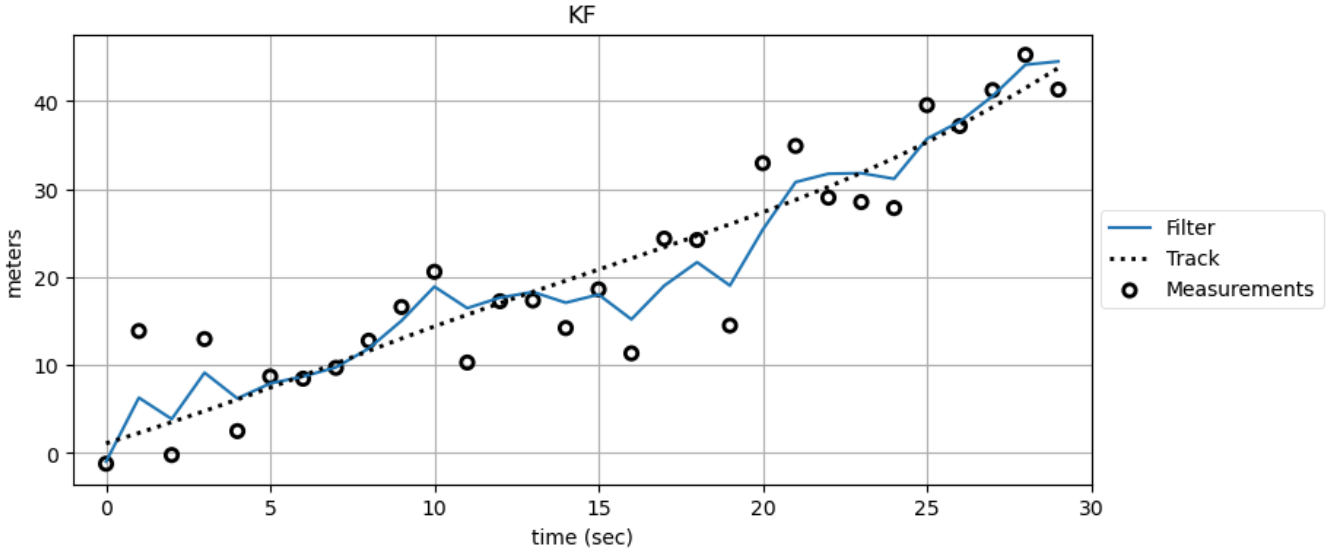


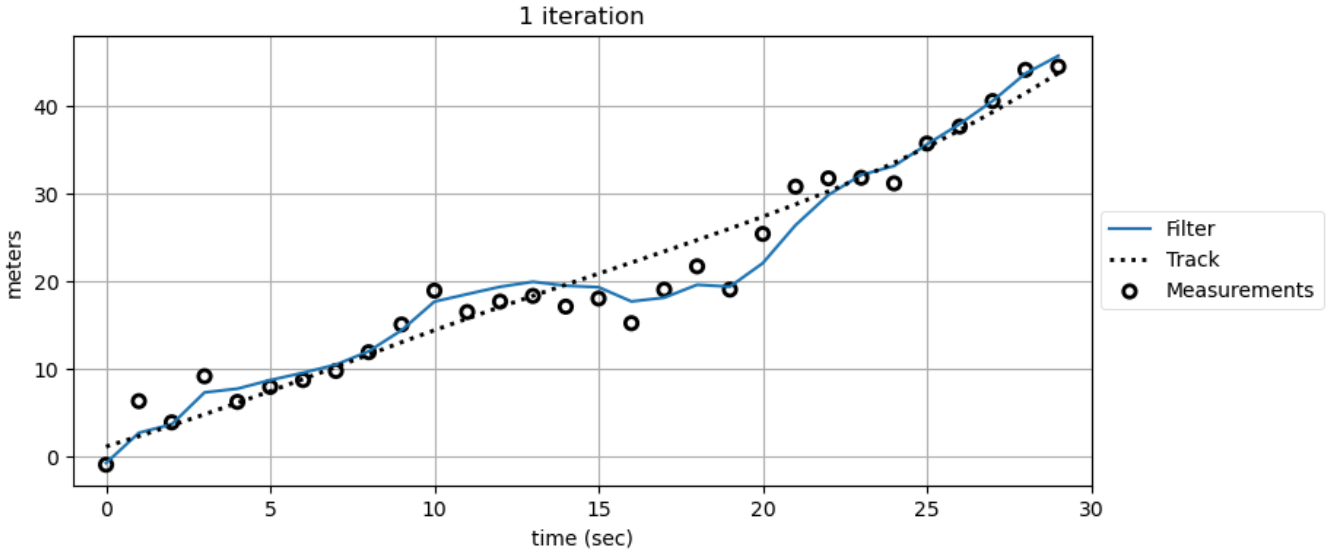

```
(5.0, 0.02)
```

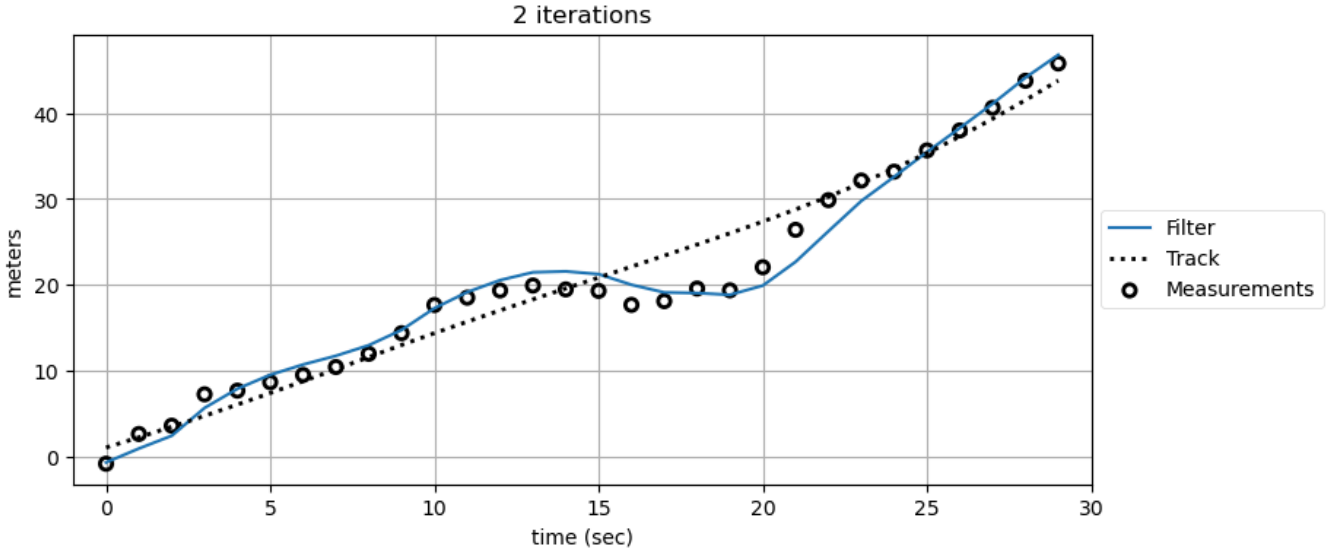
重复处理后，滤波输出虽然更平滑，却偏离了真实轨迹。原因在于标准模型通常假设有关过程噪声与测量噪声在时间上满足白噪声等条件，而不是要求真实状态信号本身没有时间相关性。卡尔曼滤波输出整合了过去的测量，其误差通常具有时间相关性，不能直接当成新的独立原始测量。最后一幅图中，起初几个读数高于轨迹，这一影响被滤波器“记住”；约 13 秒时，一段测量又恰好低于轨迹，这些相关影响再次累积，使重复滤波结果偏得更低。

再换一个角度：重复滤波并没有把原始 $\mathbf{z}$ 当作测量，而是把上一个滤波器的估计当作测量。因此，下面将本次滤波输出与上一级滤波输出画在一起比较。

```
plot_kf_output(xs, fxs0, zs, title='KF', aspect_equal=False)
plot_kf_output(xs, fxs1, fxs0[:, 0], '1 iteration', False)
plot_kf_output(xs, fxs2, fxs1[:, 0], '2 iterations', False)
```


KF
meters
time (sec)
Filter
Track
Measurements


1 iteration
meters
time (sec)
Filter
Track
Measurements


2 iterations
meters
time (sec)
Filter
Track
Measurements

希望现在大家都能明白这种方法的缺陷了。从下图可以看出，KF 算法实际上是在追踪上一个滤波器给出的不完美估计值。同时，由于上一次的测量结果也被纳入了当前的计算中，因此该算法还会将延迟效应也体现在信号中。

练习：证明位置传感器能够提升滤波效果。

想办法证明：将位置传感器的测量数据与车轮的测量数据相结合，所得到的结果会比单独使用车轮的测量数据更准确。

解决方案 1

通过将位置传感器的测量噪声设置为接近无穷大的值，迫使卡尔曼滤波器忽略该传感器的测量结果。重新运行滤波器后，观察残差的标准差。

```
fusion_test(1.5, 3.0, do_plot=False)
fusion_test(1.5, 1.e40, do_plot=False)
```

```
fusion std: 1.647
fusion std: 1.647
```

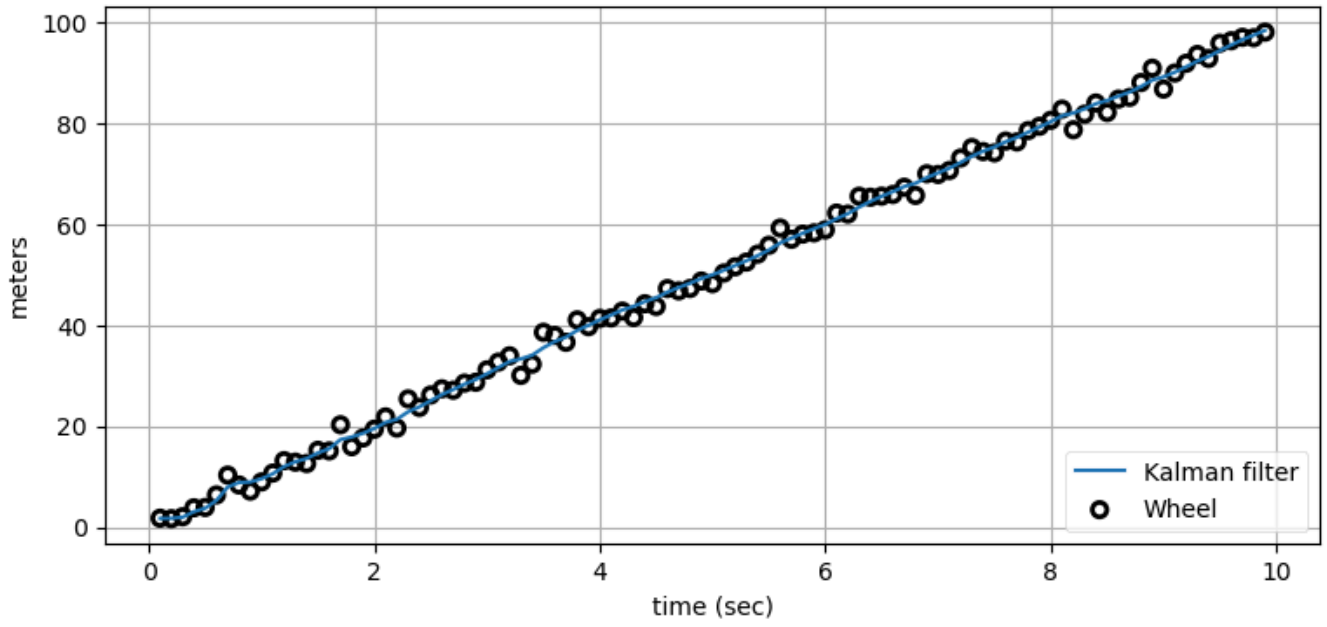
几乎完全忽略位置传感器时，滤波器的估计误差大于同时融合两种测量时的估计误差。

解决方案 2

另一种办法需要多写一些代码：构造一个每次只接受车轮传感器这一种测量的卡尔曼滤波器。

```
dt = 0.1
wheel_sigma = 1.5
kf = KalmanFilter(dim_x=2, dim_z=1)
kf.F = array([[1., dt], [0., 1.]])
kf.H = array([[1., 0.]])
kf.x = array([[0.], [1.]])
kf.Q *= 0.01
kf.P *= 100
kf.R[0, 0] = wheel_sigma**2

random.seed(1123)
nom = range(1, 100)
zs = np.array([i + randn()*wheel_sigma for i in nom])
xs, _, _, _ = kf.batch_filter(zs)
ts = np.arange(0.1, 10, .1)

res = nom - xs[:, 0, 0]
print(f'std: {np.std(res):.3f}')

plot_filter(ts, xs[:, 0], label='Kalman filter')
plot_measurements(ts, zs, label='Wheel')
set_labels(x='time (sec)', y='meters')
plt.legend(loc=4);
```

```
std: 0.523
```



在这次测量中，我的标准差为 0.523，而合并后的测量结果的标准差则为 0.391。

非平稳过程

到目前为止，我们一直假设卡尔曼滤波器中的各个矩阵都是静态的——即不会随时间而发生变化。例如，在机器人跟踪的相关内容中，我们假设在 $\Delta t = 1.0$ 秒的时间段内，各矩阵保持不变，并据此设计了状态转移矩阵。

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

但如果我们的数据传输速率以某种不可预测的方式发生变化呢？或者，如果我们有两个传感器，而它们的工作速率各不相同，那又会怎样呢？再者，如果测量的误差也发生了变化，那又会如何呢？

处理起来很简单：只需调整卡尔曼滤波器中的矩阵，以反映当前的情况即可。让我们回到狗狗跟踪的例子上来。假设输入的数据比较零散不规律。针对这个问题，我们设计了相应的解决方案。

$$\bar{\mathbf{x}} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}^-$$

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

并在初始化过程中按如下方式设置卡尔曼滤波器的变量 `F`：

```
dt = 0.1
kf.F = np.array([[1, dt],
[0, 1]])
```

若每次测量对应的时间间隔 Δt 都不同，怎么办？只需相应调整矩阵。F 随时间间隔变化，应在每次预测和更新循环中重算；Q 同样依赖时间间隔，也必须重新计算。下面是一种实现方式：

```
kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = array([0., 1.])
kf.H = array([[1, 0]])
kf.P = np.eye(2) * 50
kf.R = np.eye(1)
q_var = 0.02

# measurement tuple: (value, time)
zs = [(1., 1.),  (2., 1.1), (3., 0.9), (4.1, 1.23), (5.01, 0.97)]
for z, dt in zs:
    kf.F = array([[1, dt],
                  [0, 1]])
    kf.Q = Q_discrete_white_noise(dim=2, dt=dt, var=q_var)
    kf.predict()
    kf.update(z)
    print(kf.x)
```

```
[1. 1.]
[2.   0.92]
[2.96 1.  ]
[4.12 0.97]
[5.03 0.96]
```

传感器融合：不同采样频率

很少有两种不同的传感器能以相同的频率输出数据。假设位置传感器的更新频率为 3 赫兹，而车轮传感器的更新频率为 7 赫兹。另外，假设这些传感器的计时并不精确，会存在一定的误差，导致测量结果可能比预期时间稍早或稍晚一些。再复杂一点来说，假设车轮传感器提供的是速度估计值，而非位置估计值。

我们可以通过等待任一传感器发送的数据包来实现这一点。收到数据包后，我们可以计算出自上次更新以来经过的时间。之后，就需要对相关的矩阵进行相应的调整。 $\mathbf{F}$ 和 $\mathbf{Q}$ 中都包含时间相关的参数 Δt ，因此我们需要在每次数据更新时都对这些参数进行调整。

每次收到的测量可能来自不同传感器，因此还须选择相应的 $\mathbf{H}$ 和 $\mathbf{R}$。位置传感器测量的是 $\mathbf{x}$ 中的位置分量，对应设置如下：

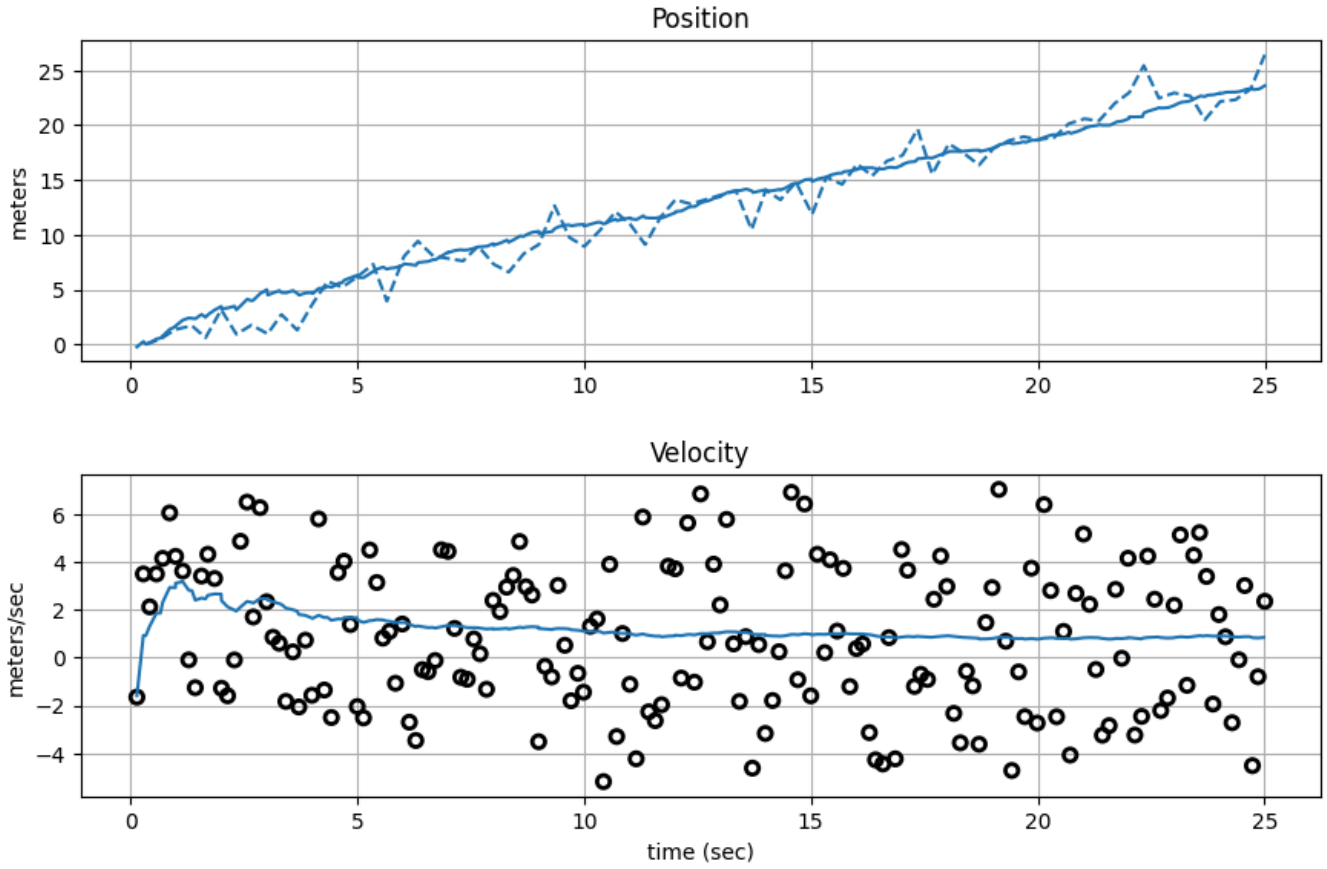
$$\mathbf{H} = \begin{bmatrix}1 & 0\end{bmatrix}$$
$$\mathbf{R} = \sigma_{ps}^2$$

轮速传感器测量的是 $\mathbf{x}$ 中的速度分量，因此将测量矩阵设置为：

$$\mathbf{H} = \begin{bmatrix}0 & 1\end{bmatrix}$$
$$\mathbf{R} = \sigma_{wheel}^2$$

```
def gen_sensor_data(t, ps_std, wheel_std):
    # generate simulated sensor data
    pos_data, vel_data = [], []
    dt = 0.
    for i in range(t*3):
        dt += 1/3.
        t_i = dt + randn() * .01 # time jitter
        pos_data.append([t_i, t_i + randn()*ps_std])

    dt = 0.
```

```
    for i in range(t*7):
        dt += 1/7.
        t_i = dt + randn() * .006 # time jitter
        vel_data.append([t_i, 1. + randn()*wheel_std])
    return pos_data, vel_data


def plot_fusion(xs, ts, zs_ps, zs_wheel):
    xs = np.array(xs)
    plt.subplot(211)
    plt.plot(zs_ps[:, 0], zs_ps[:, 1], ls='--', label='Pos Sensor')
    plot_filter(xs=ts, ys=xs[:, 0], label='Kalman filter')
    set_labels(title='Position', y='meters',)

    plt.subplot(212)
    plot_measurements(zs_wheel[:, 0], zs_wheel[:, 1],  label='Wheel')
    plot_filter(xs=ts, ys=xs[:, 1], label='Kalman filter')
    set_labels('Velocity', 'time (sec)', 'meters/sec')
    plt.gcf().set_size_inches(9, 6)
    plt.tight_layout(pad=1.5, h_pad=2.0)


def fusion_test(pos_data, vel_data, wheel_std, ps_std):
    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.F = array([[1., 1.], [0., 1.]])
    kf.H = array([[1., 0.], [1., 0.]])
    kf.x = array([[0.], [1.]])
    kf.P *= 100

    xs, ts = [],  []

    # copy data for plotting
    zs_wheel = np.array(vel_data)
    zs_ps = np.array(pos_data)

    last_t = 0
    while len(pos_data) > 0 and len(vel_data) > 0:
        if pos_data[0][0] < vel_data[0][0]:
            t, z = pos_data.pop(0)
            dt = t - last_t
            last_t = t

            kf.H = np.array([[1., 0.]])
            kf.R[0,0] = ps_std**2
        else:
            t, z = vel_data.pop(0)
            dt = t - last_t
            last_t = t

            kf.H = np.array([[0., 1.]])
            kf.R[0,0] = wheel_std**2

        kf.F[0,1] = dt
        kf.Q = Q_discrete_white_noise(2, dt=dt, var=.02)
        kf.predict()
        kf.update(np.array([z]))

        xs.append(kf.x.T[0])
        ts.append(t)
    plot_fusion(xs, ts, zs_ps, zs_wheel)

random.seed(1123)
pos_data, vel_data = gen_sensor_data(25, 1.5, 3.0)
fusion_test(pos_data, vel_data, 1.5, 3.0);
```



追踪球体运动轨迹

下面考虑运动规律受到已知物理约束的情形。在真空和恒定重力场中，抛出的球沿抛物线运动。取竖直向上为正，g 表示重力加速度的大小，则位置方程为：

$$y = -\frac{g}{2}t^2 + v_{y0}t + y_0$$
$$x = v_{x0}t + x_0$$

其中，g 表示重力加速度，t 表示时间，v_{x0} 与 v_{y0} 分别为 x、y 方向的初速度。若球以速率 v、高于水平面 θ 的角度抛出，则 v_{x0} 与 v_{y0} 为：

$$v_{x0} = v\cos\theta$$
$$v_{y0} = v\sin\theta$$

由于没有真实数据，先编写球的运动仿真。与前面一样，加入各时刻之间相互独立的噪声，以模拟带噪传感器。

```
from math import radians, sin, cos
import math

def rk4(y, x, dx, f):
    """computes 4th order Runge-Kutta for dy/dx.
    y is the initial value for y
    x is the initial value for x
    dx is the difference in x (e.g. the time step)
    f is a callable function (y, x) that you supply to
      compute dy/dx for the specified values.
```

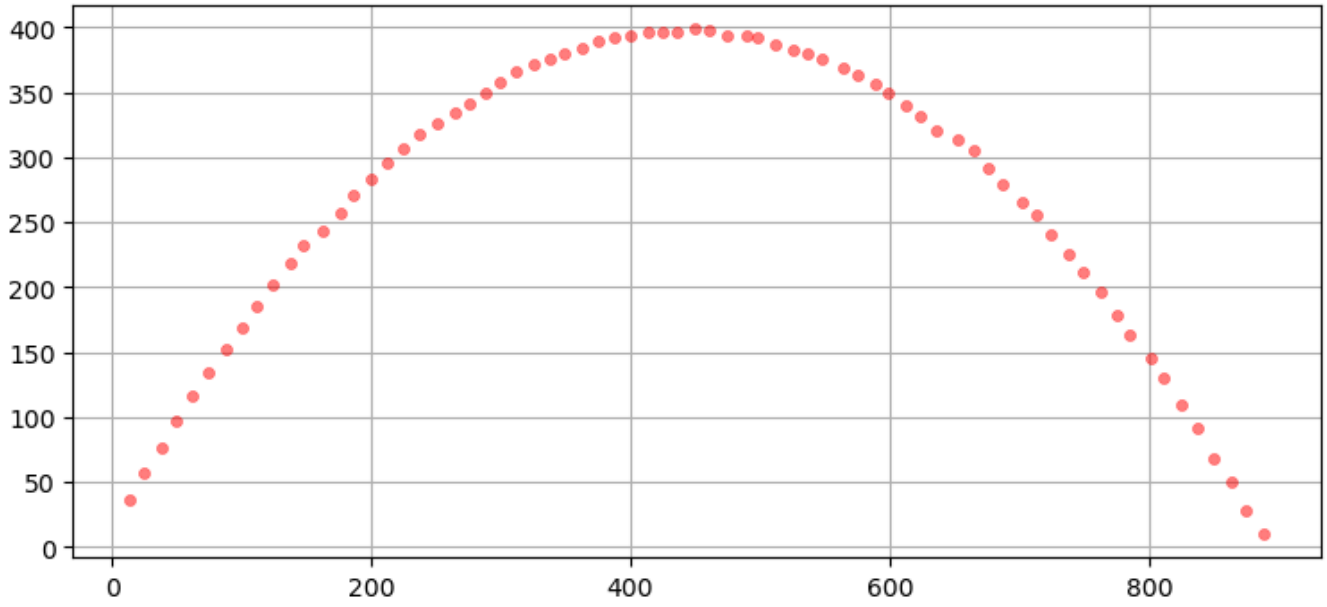
```
    """

    k1 = dx * f(y, x)
    k2 = dx * f(y + 0.5*k1, x + 0.5*dx)
    k3 = dx * f(y + 0.5*k2, x + 0.5*dx)
    k4 = dx * f(y + k3, x + dx)

    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.

def fx(x,t):
    return fx.vel

def fy(y,t):
    return fy.vel - 9.8*t


class BallTrajectory2D(object):
    def __init__(self, x0, y0, velocity,
                 theta_deg=0.,
                 g=9.8,
                 noise=[0.0, 0.0]):
        self.x = x0
        self.y = y0
        self.t = 0
        theta = math.radians(theta_deg)
        fx.vel = math.cos(theta) * velocity
        fy.vel = math.sin(theta) * velocity
        self.g = g
        self.noise = noise


    def step(self, dt):
        self.x = rk4(self.x, self.t, dt, fx)
        self.y = rk4(self.y, self.t, dt, fy)
        self.t += dt
        return (self.x + randn()*self.noise[0],
                self.y + randn()*self.noise[1])
```

因此，要创建一条以点(0, 15)为起点、速度为 100 米/秒、角度为 60°的轨迹，我们可以这样表示：

```
traj = BallTrajectory2D(x0=0, y0=15, velocity=100, theta_deg=60)
```

然后在每个时间步长上调用 `traj.step(t)`。我们来测试一下吧。

```
def test_ball_vacuum(noise):
    y = 15
    x = 0
    ball = BallTrajectory2D(x0=x, y0=y,
                            theta_deg=60., velocity=100.,
                            noise=noise)
    t = 0
    dt = 0.25
    while y >= 0:
        x, y = ball.step(dt)
        t += dt
        if y >= 0:
            plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)

    plt.axis('equal');

#test_ball_vacuum([0, 0]) # plot ideal ball position
test_ball_vacuum([1, 1]) # plot with noise
```



这看起来挺合理的，那我们就继续吧（给读者的练习：请更全面地验证这个模拟结果）。

选择状态变量

可能会先考虑沿用狗的位置跟踪模型，但仅使用恒速预测不足以描述球的运动。状态转移仍须写成 $\bar{\mathbf{x}} = \mathbf{Fx} + \mathbf{Bu}$，从上一时刻传播到下一时刻。在真空中，x 方向速度恒定，y 方向加速度由重力加速度 g 决定。以时间步长 Δt 作欧拉离散化，可得：

$$
\begin{aligned}
x_t &= x_{t-1} + v_{x(t-1)}\Delta t \\
v_{xt} &= v_{x(t-1)} \\
y_t &= y_{t-1} + v_{y(t-1)}\Delta t \\
v_{yt} &= -g\Delta t + v_{y(t-1)}
\end{aligned}
$$

> 旁注：欧拉法通过假定时刻 t 的斜率，也就是导数，在一个时间步内不变，逐步积分微分方程。这里位置的导数就是速度。在每个 Δt 时间步内，先假定速度恒定来计算新位置，再为下一步更新速度。还可以采用龙格—库塔等精度更高的方法。本例每一步都有测量更新，因此欧拉法通常已足够准确。如果确实需要龙格—库塔法，就要自己编写 predict()：先计算状态 $\mathbf{x}$ 的非线性时间演化，再使用通常的卡尔曼协方差预测式 $\bar{\mathbf{P}} = \mathbf{FPF}^\mathsf{T} + \mathbf{Q}$，其中 F 应与采用的状态转移模型一致。

这意味着，我们需要在卡尔曼滤波器中考虑 y 的加速度因素，而无需考虑 x 的加速度。由此可确定以下状态变量。

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} & \ddot{y} \end{bmatrix}^\mathsf{T}$$

不过，这种加速度实际上是由重力产生的，而重力是一个恒定不变的物理量。我们不必让卡尔曼滤波器去追踪这个恒定值，而可以将重力视为一种“控制输入”。换句话说，重力是一种以固定方式影响系统行为的力，它在球的整个飞行过程中都持续发挥作用。

状态预测的方程为 $\bar{\mathbf{x}} = \mathbf{Fx} + \mathbf{Bu}$。 $\mathbf{Fx}$ 就是我们常用的状态转移函数，我们将利用它来模拟球的位置和速度。向量 $\mathbf{u}$ 用于指定输入到滤波器中的控制信号。对于汽车而言，这些控制信号包括油门和刹车的踩踏程度、方向盘的转动角度等等。而对于我们的球来说，控制信号就是重力。矩阵 $\mathbf{B}$ 则描述了这些控制信号如何影响系统的行为。同样地，对于汽车而言， $\mathbf{B}$ 会将油门和刹车的输入转化为速度的变化，将方向盘的输入转化为车辆的位置和行驶方向的变化。对于我们的球跟踪问题，该矩阵则用于计算由于重力而产生的速度变化。我们稍后会详细讨论这一点。目前，我们先确定状态变量的具体形式吧。

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} \end{bmatrix}^\mathsf{T}$$

设计状态转移函数

下一步设计状态转移函数。在线性模型中，它由矩阵 $\mathbf{F}$ 实现：用它乘以上一时刻状态，得到下一时刻的先验状态 $\bar{\mathbf{x}} = \mathbf{Fx}$。

我就不再详细阐述了，因为这与我们在前一章中讨论的一维情况非常相似。关于位置和速度的状态方程如下：

$$\begin{aligned}
\bar{x} &= (1 * x) + (\Delta t * v_x) + (0 * y) + (0 * v_y) \\
\bar{v}_x &= (0 * x) + (1 * v_x) + (0 * y) + (0 * v_y) \\
\bar{y} &= (0 * x) + (0 * v_x) + (1 * y) + (\Delta t * v_y) \\
\bar{v}_y &= (0 * x) + (0 * v_x) + (0 * y) + (1 * v_y)
\end{aligned}$$

注意，这些状态转移项不含重力加速度 g。如前所述，重力影响将通过卡尔曼滤波器的控制输入计入。矩阵形式为：

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

设计控制输入函数

下面用控制输入计入重力影响。$\mathbf{Bu}$ 加到 $\bar{\mathbf{x}}$ 上，表示重力使先验状态 $\bar{\mathbf{x}}$ 发生的变化。也就是说，$\mathbf{Bu}$ 包含 $\begin{bmatrix} \Delta x_g & \Delta \dot{x}_g & \Delta y_g & \Delta \dot{y}_g \end{bmatrix}^\mathsf{T}$。

如果我们观察这些离散化的方程，就会发现：重力只会对 y 的速度产生影响。

$$\begin{aligned}
x_t &= x_{t-1} + v_{x(t-1)}\Delta t \\
v_{xt} &= v_{x(t-1)} \\
y_t &= y_{t-1} + v_{y(t-1)}\Delta t \\
v_{yt} &= -g\Delta t + v_{y(t-1)}
\end{aligned}$$

因此，我们希望 $\mathbf{Bu}$ 的乘积等于 $\begin{bmatrix} 0 & 0 & 0 & -g\Delta t \end{bmatrix}^\mathsf{T}$。从某种意义上说，如何定义 $\mathbf{B}$ 和 $\mathbf{u}$ 并不重要，只要将它们相乘后能得到这个结果即可。例如，我们可以这样定义 $\mathbf{B} = 1$ 和 $\mathbf{u} = \begin{bmatrix} 0 & 0 & 0 & -g\Delta t \end{bmatrix}^\mathsf{T}$。不过，这种定义方式与我们对 $\mathbf{B}$ 和 $\mathbf{u}$ 的定义并不相符。在那些定义中， $\mathbf{u}$ 是控制输入，而 $\mathbf{B}$ 则是控制函数。对于 y 的速度来说，控制输入就是 $-g$。所以，这只是其中一种可能的定义方式而已。

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \Delta t \end{bmatrix}, \mathbf{u} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -g \end{bmatrix}$$

在我看来，这似乎有些过头了。我建议，或许可以让 $\mathbf{u}$ 来负责控制 x 和 y 这两个维度。

$$\mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & \Delta t \end{bmatrix}, \mathbf{u} = \begin{bmatrix} 0 \\ -g \end{bmatrix}$$

也可以只保留实际存在的控制输入。本例 x 方向没有外部加速度输入，因此还可简化为：

$$\mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \Delta t \end{bmatrix}, \mathbf{u} = [-g]$$

也见过下面这种写法：

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \mathbf{u} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -g\Delta t \end{bmatrix}$$

这种写法虽然能得到正确结果，但我不赞成把时间间隔放进 $\mathbf{u}$：时间并不是控制输入。它用于把控制作用转换为状态变化，而这应由 $\mathbf{B}$ 负责。

设计测量函数

该测量函数定义了如何利用方程 $\mathbf{z} = \mathbf{Hx}$，将状态变量转化为可测量的数值。我们假设有一个传感器能够提供球在(x,y)坐标系中的位置信息，但无法测量球的速度或加速度。因此，我们的测量函数必须满足以下条件：

$$\begin{bmatrix} z_x \\ z_y \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix}$$

其中

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

设计测量噪声矩阵

与机器人例子一样，假设 x、y 方向的测量误差相互独立，并将两方向的测量噪声方差都设为 0.5 平方米。因此：

$$\mathbf{R} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

设计过程噪声矩阵

我们假设球体在真空中运动，因此不存在任何过程噪声。我们有 4 个状态变量，因此需要一个 $4{\times}4$ 协方差矩阵来描述这些状态变量之间的关系。

$$\mathbf{Q} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

确定初始条件

在测试状态转移函数时，我们已经执行过这一步骤。请回想一下，我们是利用三角函数来计算 x 和 y 的初始速度的，并且确定了 $\mathbf{x}$ 的值。

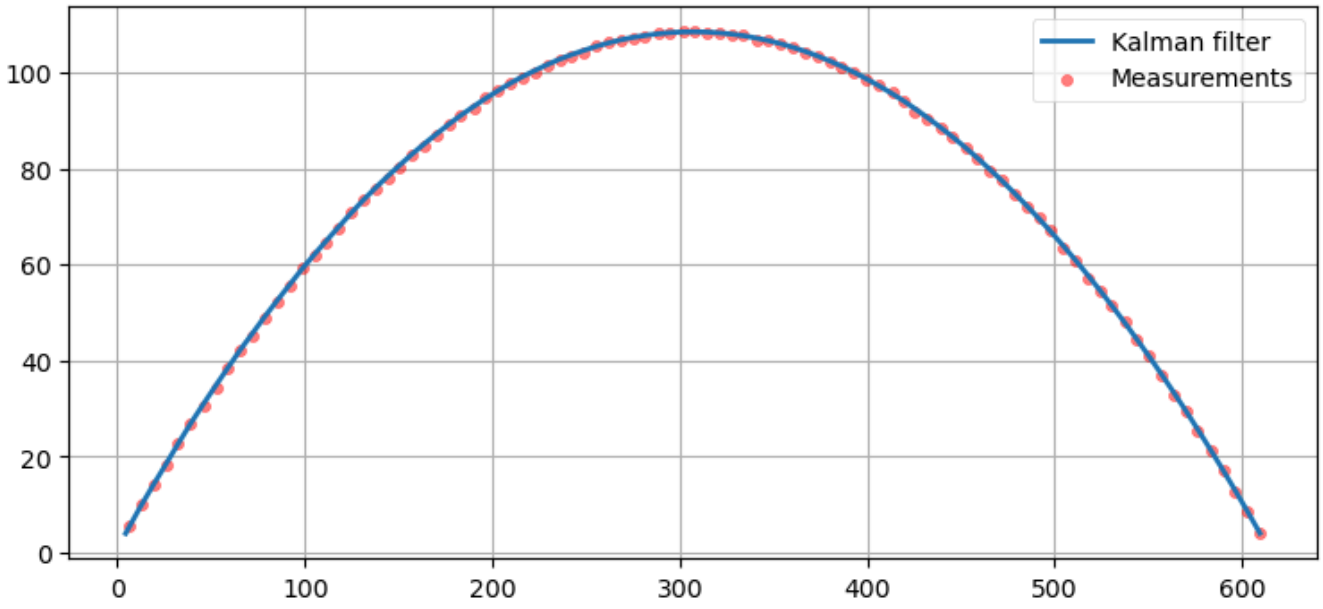
```
omega = radians(omega)
vx = cos(omega) * v0
vy = sin(omega) * v0
f1.x = np.array([[x, vx, y, vy]]).T
```

所有设计步骤都已完成，现在可以实现并测试滤波器。首先给出实现代码：

```
from math import sin, cos, radians

def ball_kf(x, y, omega, v0, dt, r=0.5, q=0.):
    kf = KalmanFilter(dim_x=4, dim_z=2, dim_u=1)

    kf.F = np.array([[1., dt, 0., 0.],   # x   = x0 + dx*dt
                     [0., 1., 0., 0.],   # dx  = dx0
                     [0., 0., 1., dt],   # y   = y0 + dy*dt
                     [0., 0., 0., 1.]])  # dy  = dy0

    kf.H = np.array([[1., 0., 0., 0.],
                     [0., 0., 1., 0.]])

    kf.B = np.array([[0., 0., 0., dt]]).T
    kf.R *= r
    kf.Q *= q

    omega = radians(omega)
    vx = cos(omega) * v0
    vy = sin(omega) * v0
    kf.x = np.array([[x, vx, y, vy]]).T
    return kf
```

现在，我们将通过使用球体模拟类来生成球的各项测量数据，从而测试该滤波器的性能。

```
def track_ball_vacuum(dt):
    global kf
    x, y = 0., 1.
    theta = 35.  # launch angle
    v0 = 80.
    g = np.array([[-9.8]])  # gravitational constant
    ball = BallTrajectory2D(x0=x, y0=y, theta_deg=theta, velocity=v0,
                            noise=[.2, .2])
    kf = ball_kf(x, y, theta, v0, dt)

    t = 0
    xs, ys = [], []
    while kf.x[2] > 0:
        t += dt
        x, y = ball.step(dt)
        z = np.array([[x, y]]).T

        kf.update(z)
        xs.append(kf.x[0])
        ys.append(kf.x[2])
        kf.predict(u=g)
        p1 = plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)
    p2, = plt.plot(xs, ys, lw=2)
    plt.legend([p2, p1], ['Kalman filter', 'Measurements'],
               scatterpoints=1)

track_ball_vacuum(dt=1./10)
```



我们可以看到，卡尔曼滤波器在追踪球的位置方面表现得相当不错。不过，正如之前所解释的，这其实是个过于简单的例子——因为我们不存在任何过程噪声。在真空中，我们可以以任意精确度来预测物体的轨迹；在这个例子中，使用卡尔曼滤波器其实毫无必要。采用最小二乘法进行曲线拟合，也会得到完全相同的结果。

追踪空中的球体

本例跟踪在地球大气中运动的球，其轨迹受到风、阻力和自旋影响。假设传感器是一台相机，图像处理程序负责用斑点检测（blob detection）定位球，我们不实现这部分代码。有些帧可能漏检，有些帧可能出现不属于球的伪斑点。我们也不假设已知起始位置、抛射角度或自旋，跟踪程序必须根据测量建立航迹。这里把运

动限制在二维平面内，直接仿真二维位置测量，不讨论从单相机恢复一般三维位置。

建立空气阻力模型

首先建立球在空气中运动的数学模型。较完整的模型需要考虑球面粗糙度及其随速度变化的阻力、自旋引起的马格努斯效应、升力、湿度和空气密度等因素。这里不深入讨论球体物理学，只考虑空气阻力对不旋转棒球的影响，采用 Nicholas Giordano 与 Hisao Nakanishi 所著《Computational Physics》（1997）中的模型。它没有包含全部因素。更详细的讨论可见 Alan Nathan 的网站：http://baseball.physics.illinois.edu/index.html。我在计算机视觉工作中也使用他的模型，但这里不希望让复杂物理细节分散注意力。

提示：不必掌握接下来的物理细节，也可以继续学习卡尔曼滤波。这里的目标是较真实地模拟棒球运动，以测试简化滤波模型的表现。实际工程中通常无法完整建模所有物理因素，只能先描述主要动态，再调整测量噪声和过程噪声参数。但这也有风险：滤波器可能在测试数据上调得很好，面对稍有不同的数据却表现很差。这或许是设计卡尔曼滤波器最难的部分，也是人们称其为一种难以言传的技艺的原因。

我不喜欢那些不加解释就直接实施某种机制的书籍。因此，现在我来详细解释一下球在空气中运动的物理原理。如果你对具体的模拟过程不感兴趣，可以跳过这部分内容。

在空气中运动的球会遇到风阻。这种风阻会对球施加一种力，这种力被称为“阻力”，它会影响球的运动轨迹。在 Giordano 的理论中，这一现象被如此描述：

$$F_{drag} = -B_2 v^2$$

其中，B_2 是通过实验确定的系数，而 v 则代表物体的速度。F_{drag} 可以分解为 x 和 y 这两个部分。

$$F_{drag,x} = -B_2 v v_x$$
$$F_{drag,y} = -B_2 v v_y$$

如果 m 表示球的质量，那么我们可以利用 $F = ma$ 来计算球的加速度。

$$a_x = -\frac{B_2}{m} v v_x$$
$$a_y = -\frac{B_2}{m} v v_y$$

Giordano 为 $\frac{B_2}{m}$ 提供了以下计算公式。该公式考虑了空气密度、棒球的横截面尺寸以及其表面粗糙度等因素。需要明白，这是一个基于风洞测试和各种简化假设得出的近似值。所有数值均以国际单位制表示：速度以米/秒为单位，时间以秒为单位。

$$\frac{B_2}{m} = 0.0039 + \frac{0.0058}{1 + \exp\left[(v - 35)/5\right]}$$

先看真空中球的欧拉离散更新。每一步以当前速度计算位移，再加到原来的位置上；重力使竖直速度减小：

$$
\begin{aligned}
x &= x + v_x \Delta t \\
y &= y + v_y \Delta t \\
v_x &= v_x \\
v_y &= v_y - 9.8 \Delta t
\end{aligned}
$$

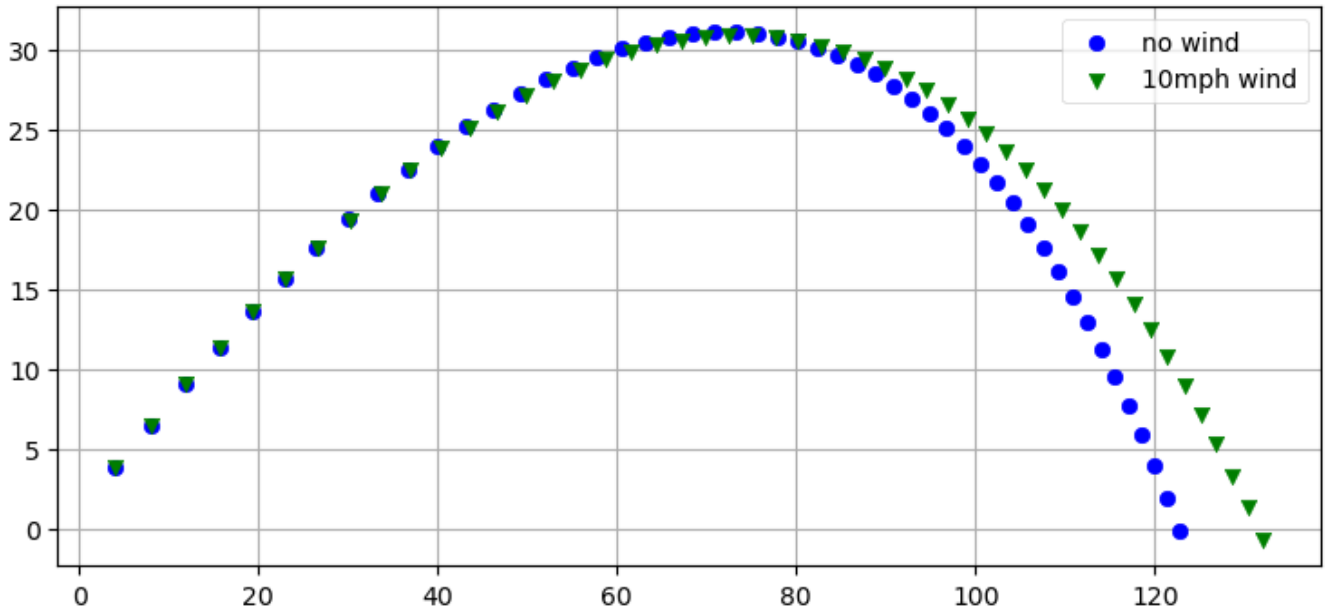
把 $accel * \Delta t$ 纳入速度更新，就能计入阻力造成的加速度。由于阻力使球减速，这一项应从速度中减去。实现很简单，只需把阻力分解为 x 和 y 两个方向的分量。

我不想再详细讨论这个问题了，因为计算物理学超出了本书的讨论范围。需要指出的是，要想获得更精确的模拟结果，就必须考虑高度、温度、球的旋转速度等众多因素。如果你感兴趣的话，艾伦·内森的相关研究可以为你提供参考。我的目的是让模拟结果更贴近现实世界的情况，从而测试卡尔曼滤波器所使用的简单预测模型在这种现实情况下的表现如何。你的模型永远无法完全再现现实世界中的实际情况。因此，设计出优秀的卡尔曼滤波器的关键在于仔细测试其在真实数据下的表现。

下面的代码计算海平面高度、有风条件下的棒球轨迹，分别画出相同初始击球条件下的无风轨迹，以及顺风 10 mph 时的轨迹。棒球统计通常采用美制单位，这里也沿用这一习惯（http://en.wikipedia.org/wiki/United_States_customary_units）。110 mph 是一次本垒打击球可能具有的典型初始离棒速率。

```
from math import sqrt, exp

def mph_to_mps(x):
    return x * .447

def drag_force(velocity):
    """ Returns the force on a baseball due to air drag at
    the specified velocity. Units are SI"""

    return velocity * (0.0039 + 0.0058 /
            (1. + exp((velocity-35.)/5.)))

v = mph_to_mps(110.)
x, y = 0., 1.
dt = .1
theta = radians(35)

def solve(x, y, vel, v_wind, launch_angle):
    xs = []
    ys = []
    v_x = vel*cos(launch_angle)
    v_y = vel*sin(launch_angle)
    while y >= 0:
        # Euler equations for x and y
        x += v_x*dt
        y += v_y*dt

        # force due to air drag
        velocity = sqrt((v_x-v_wind)**2 + v_y**2)
        F = drag_force(velocity)

        # euler's equations for vx and vy
        v_x = v_x - F*(v_x-v_wind)*dt
        v_y = v_y - 9.8*dt - F*v_y*dt

        xs.append(x)
        ys.append(y)

    return xs, ys

x, y = solve(x=0, y=1, vel=v, v_wind=0, launch_angle=theta)
p1 = plt.scatter(x, y, color='blue', label='no wind')
```

```
wind = mph_to_mps(10)
x, y = solve(x=0, y=1, vel=v, v_wind=wind, launch_angle=theta)
p2 = plt.scatter(x, y, color='green', marker="v",
                 label='10mph wind')
plt.legend(scatterpoints=1);
```



我们可以很容易地看出，球在真空中和空气中的运动轨迹是有所不同的。在上面的分析中，我对球在真空中的初始速度和发射角度都做了相同的设定。计算结果显示，球在真空中可以飞行 240 多米（接近 800 英尺）。而在空气中，球的飞行距离则略高于 120 米，大约 400 英尺。400 英尺这个距离对于一记击球精准的本垒打来说是很合理的，因此我们可以相信，我们的模拟结果相当准确。

不用多说，我们现在就来创建一个球体运动模拟。该模拟会利用上述数学公式来生成更符合实际情况的球体运动轨迹。需要指出的是，由于空气阻力的非线性特性，我们无法用解析方法来确定球体在任意时刻的位置。因此，我们必须通过逐步计算的方式来得到球体的位置。我在这里使用了欧拉方法来求解这个问题；至于使用更精确的算法，比如龙格-库塔法，则留给读者作为练习吧。对于我们的目的来说，这种稍微复杂的处理方式其实没必要，因为我们所使用的时间步长下，各种算法之间的精度差异其实很小。

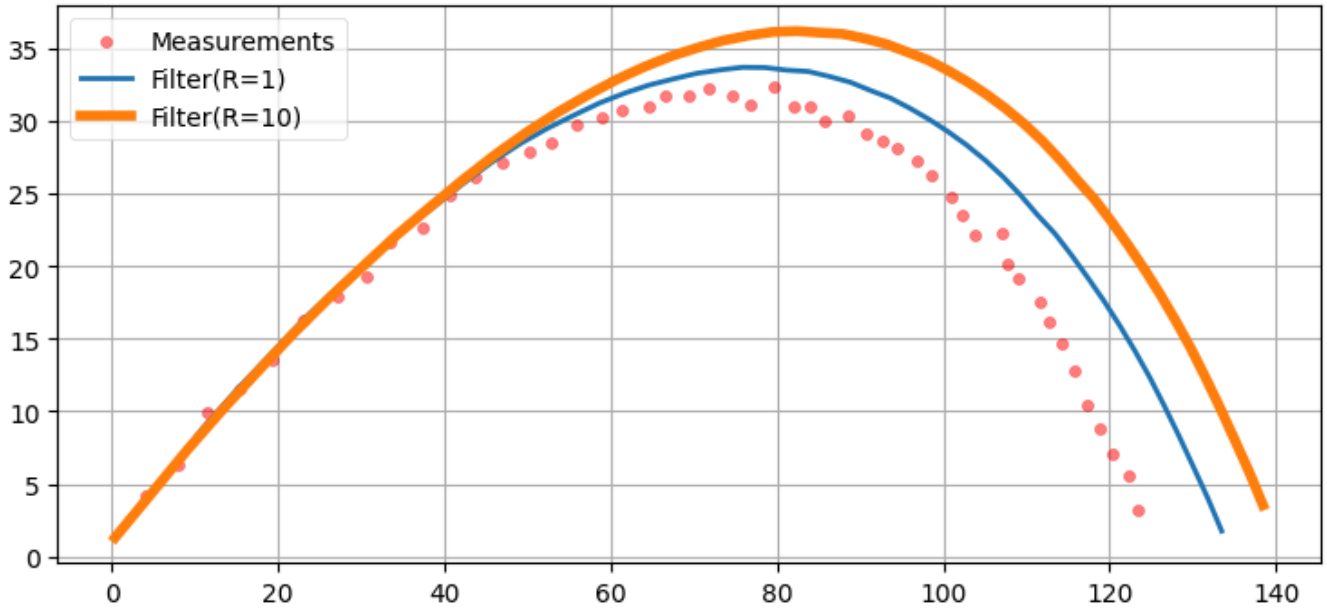
```
class BaseballPath:
    def __init__(self, x0, y0, launch_angle_deg, velocity_ms,
                 noise=(1.0, 1.0)):
        """ Create 2D baseball path object
           (x = distance from start point in ground plane,
            y=height above ground)

        x0,y0            initial position
        launch_angle_deg angle ball is travelling respective to
                         ground plane
        velocity_ms      speeed of ball in meters/second
        noise            amount of noise to add to each position
                         in (x, y)
        """

        omega = radians(launch_angle_deg)
        self.v_x = velocity_ms * cos(omega)
        self.v_y = velocity_ms * sin(omega)

        self.x = x0
        self.y = y0
```

```
        self.noise = noise


    def drag_force(self, velocity):
        """ Returns the force on a baseball due to air drag at
        the specified velocity. Units are SI
        """
        B_m = 0.0039 + 0.0058 / (1. + exp((velocity-35.)/5.))
        return B_m * velocity


    def update(self, dt, vel_wind=0.):
        """ compute the ball position based on the specified time
        step and wind velocity. Returns (x, y) position tuple.
        """

        # Euler equations for x and y
        self.x += self.v_x*dt
        self.y += self.v_y*dt

        # force due to air drag
        v_x_wind = self.v_x - vel_wind
        v = sqrt(v_x_wind**2 + self.v_y**2)
        F = self.drag_force(v)

        # Euler's equations for velocity
        self.v_x = self.v_x - F*v_x_wind*dt
        self.v_y = self.v_y - 9.81*dt - F*self.v_y*dt

        return (self.x + randn()*self.noise[0],
                self.y + randn()*self.noise[1])
```

现在，我们可以用该模型生成的测量数据来测试卡尔曼滤波器的性能了。

```
x, y = 0, 1.

theta = 35. # launch angle
v0 = 50.
dt = 1/10.   # time step
g = np.array([[-9.8]])

plt.figure()
ball = BaseballPath(x0=x, y0=y, launch_angle_deg=theta,
                    velocity_ms=v0, noise=[.3,.3])
f1 = ball_kf(x, y, theta, v0, dt, r=1.)
f2 = ball_kf(x, y, theta, v0, dt, r=10.)
t = 0
xs, ys = [], []
xs2, ys2 = [], []

while f1.x[2] > 0:
    t += dt
    x, y = ball.update(dt)
    z = np.array([[x, y]]).T

    f1.update(z)
    f2.update(z)
    xs.append(f1.x[0])
    ys.append(f1.x[2])
    xs2.append(f2.x[0])
    ys2.append(f2.x[2])
    f1.predict(u=g)
    f2.predict(u=g)

    p1 = plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)

p2, = plt.plot(xs, ys, lw=2)
p3, = plt.plot(xs2, ys2, lw=4)
plt.legend([p1, p2, p3],
```

```
            ['Measurements', 'Filter(R=1)', 'Filter(R=10)'],
            loc='best', scatterpoints=1);
```



图中展示了两种测量噪声设置下的卡尔曼滤波结果。红色圆点是测量值，较细的蓝线对应 R=1，较粗的橙线对应 R=10。这些参数只是为了展示测量噪声设定对输出的影响，不代表最优设计。

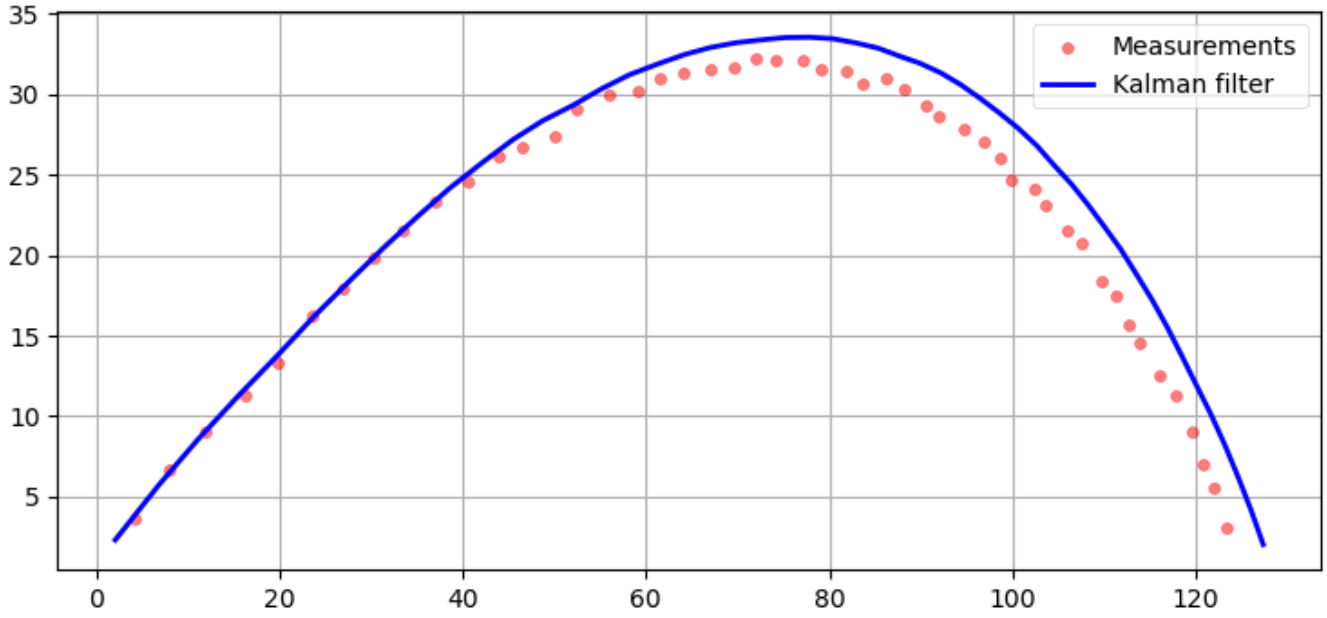
可以看出，这两种滤波器的表现都不理想。起初，两者都能很好地跟踪测量数据，但随着时间的推移，两者开始出现偏差。出现这种情况的原因是，空气阻力的状态模型是非线性的，而卡尔曼滤波器则假设该模型是线性的。回想一下我们在关于 g-h 滤波器的章节中讨论过的非线性问题，我们会明白为什么 g-h 滤波器总是会落后于系统的加速度。在这里也是如此：由于加速度为负值，卡尔曼滤波器总是会高估球的位置。只要加速度持续存在，滤波器就永远无法赶上实际情况，因此其偏差会持续扩大。

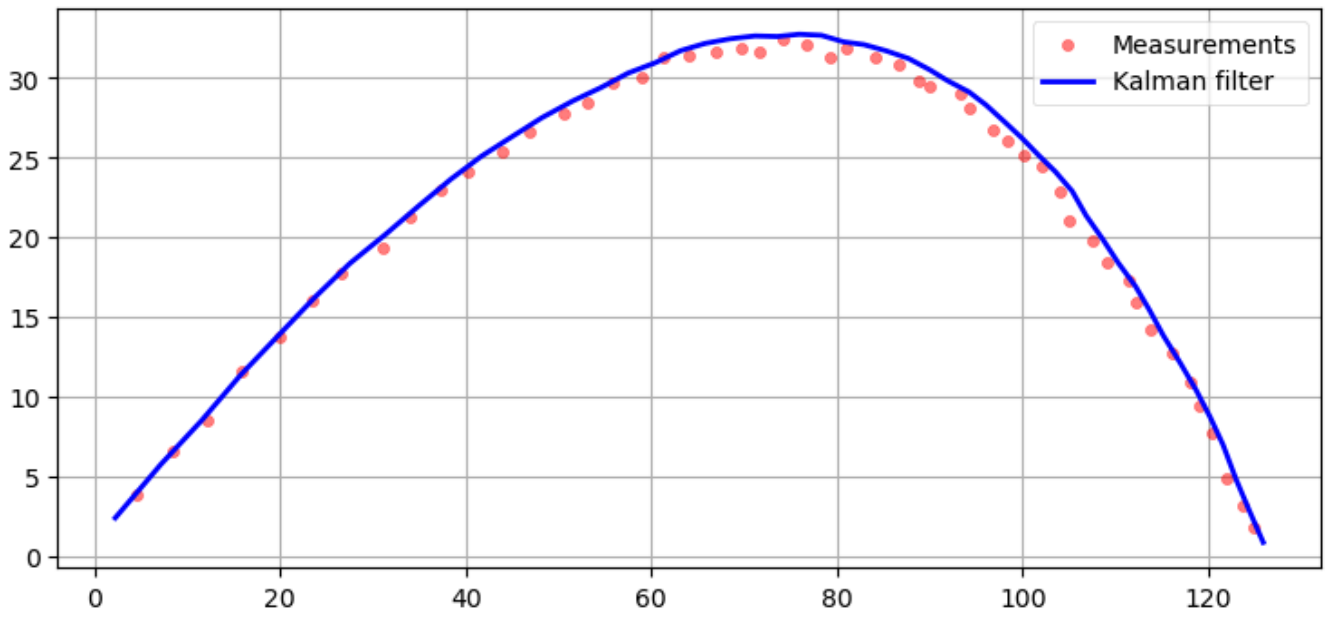
我们能做些什么来改善这个问题呢？最好的方法是使用非线性卡尔曼滤波器来进行处理，我们将在后面的章节中详细介绍这一方法。不过，对于这个问题，还有另一种所谓的“工程学上的解决方案”。卡尔曼滤波器假设球体处于真空环境中，因此不存在任何干扰因素。但实际上，球体处于空气中，大气会对球体施加力。我们可以把这种力视为一种干扰因素。不过，这种分析并不十分严谨：首先，这种力的分布并非高斯分布；其次，我们其实可以计算出这种力的大小，所以不能简单地说“这是随机的”就能解决问题。不过，让我们看看按照这种思路会得到怎样的结果吧。

以下代码实现了与之前相同的卡尔曼滤波器，只不过此时存在非零的过程噪声。我绘制了两个示例：一个是使用 `Q=.1` 的情况，另一个是使用 `Q=0.01` 的情况。

```
def plot_ball_with_q(q, r=1., noise=0.3):
    x, y = 0., 1.
    theta = 35. # launch angle
    v0 = 50.
    dt = 1/10.   # time step
    g = np.array([[-9.8]])

    ball = BaseballPath(x0=x,
                        y0=y,
                        launch_angle_deg=theta,
                        velocity_ms=v0,
                        noise=[noise,noise])
    f1 = ball_kf(x, y, theta, v0, dt, r=r, q=q)
```

```
    t = 0
    xs, ys = [], []

    while f1.x[2] > 0:
        t += dt
        x, y = ball.update(dt)
        z = np.array([[x, y]]).T

        f1.update(z)
        xs.append(f1.x[0])
        ys.append(f1.x[2])
        f1.predict(u=g)

        p1 = plt.scatter(x, y, c='r', marker='.', s=75, alpha=0.5)

    p2, = plt.plot(xs, ys, lw=2, color='b')
    plt.legend([p1, p2], ['Measurements', 'Kalman filter'])
    plt.show()

plot_ball_with_q(0.01)
plot_ball_with_q(0.1)
```

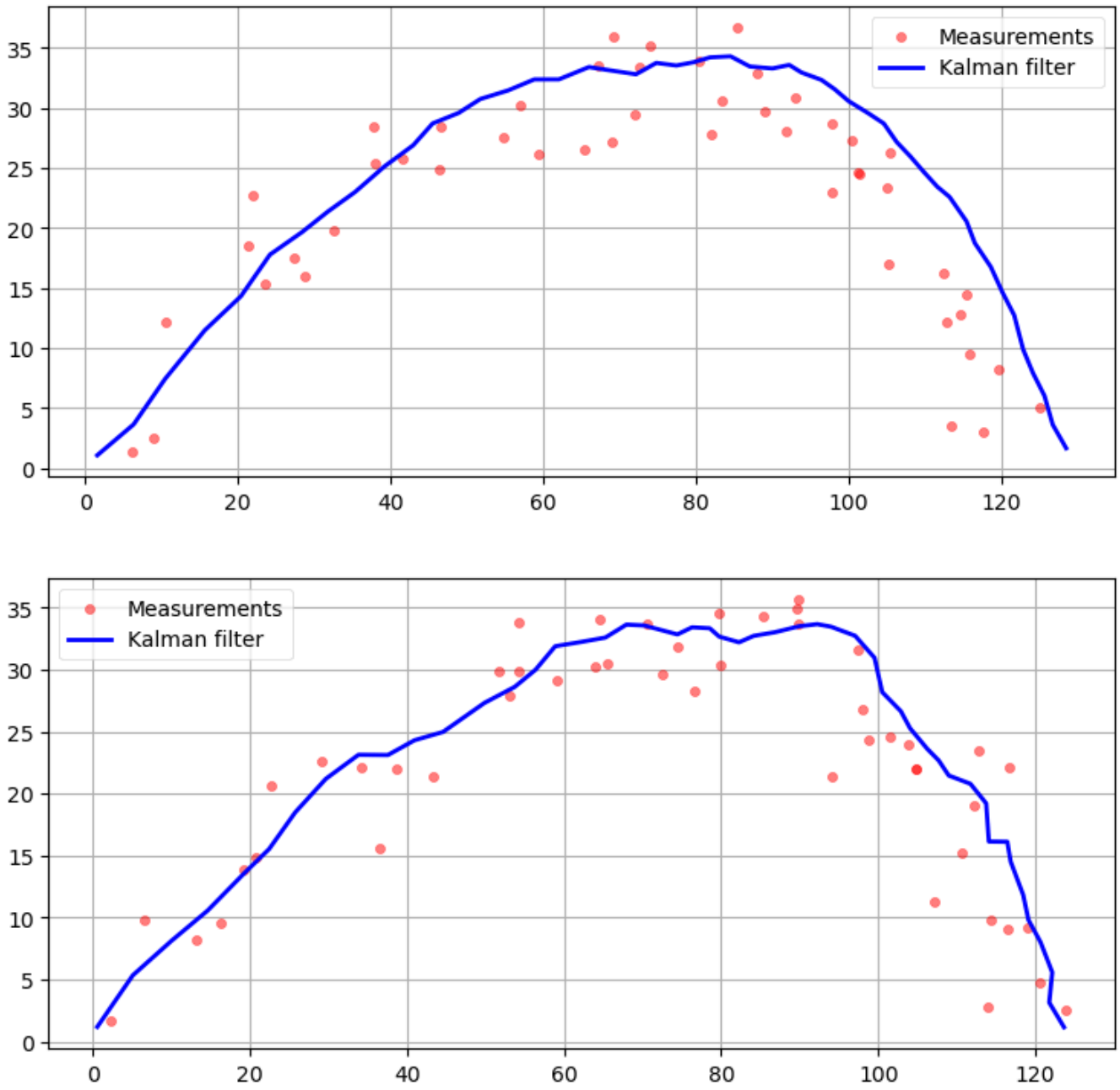




第二个滤波器能较好地跟随测量，仅有轻微滞后。

这是否是好办法？通常不是，但也取决于具体情形。本例中的非线性阻力变化相当规则；若跟踪汽车，加速度会随加减速与转弯改变。将过程噪声设得较大，会让滤波器更依赖测量。如果测量噪声很小，这种折中或许可用。但下面增加测量噪声，再看看结果。

```
plot_ball_with_q(0.01, r=3, noise=3.)
plot_ball_with_q(0.1, r=3, noise=3.)
```



这种输出效果糟糕透顶。线性卡尔曼滤波器别无选择，只能给予测量数据比预测结果更高的权重。但当测量数据存在噪声时，滤波器的输出就会完全受噪声的影响。正是线性卡尔曼滤波器的这一固有缺陷，促使人们开发出了非线性版本的滤波器。

也就是说，利用过程噪声来处理系统中的微小非线性误差是完全可行的。这也是卡尔曼滤波器所体现的“巧妙技巧”之一。我们对传感器和系统的建模从来都不可能做到完美。传感器的工作特性并非高斯分布，我们的过程模型也永远不可能完美无缺。我们可以通过将测量误差和过程误差设定得高于其理论上的正确值来弥补这些缺陷，但这样做的代价就是得到的解决方案并非最优解。当然，与其让卡尔曼滤波器无法正常工作，不如接受这种非最优的解决方案。不过，正如上面的图表所示，滤波器的输出结果很容易变得很糟糕。通常情况下，经过多次模拟和测试后，我们得到的滤波器在某些条件下表现不错。但当把该滤波器应用于实际数据时，由于条件略有不同，滤波器的性能就会变得很差。

目前，我们先暂且不讨论这个问题，因为在这个例子中，我们显然没有正确使用卡尔曼滤波器。我们将在后面的章节中重新探讨这个问题，看看使用各种非线性技术会带来怎样的效果。在某些领域，对于非线性问题，使用线性卡尔曼滤波器也是可行的。但通常情况下，你还是需要运用本书后面介绍的各种非线性技术。

参考文献

[1] Bar-Shalom, Yaakov 等人。《估计方法及其在跟踪与导航中的应用》。约翰·威利父子出版社，2001年。

第 9 章：非线性滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

我们所开发的卡尔曼滤波器依赖于线性方程，因此它只能处理线性问题。然而，现实世界是非线性的，所以迄今为止我们所研究的这种经典滤波器的作用其实非常有限。

在过程模型中可能存在非线性现象。比如，如果我们想追踪一个在大气中下落的物体，该物体的加速度取决于其所受到的阻力。而阻力又与空气密度有关，空气密度会随着高度的增加而降低。在一维情况下，这种现象可以用非线性微分方程来描述。

$$\ddot{x} = \frac{0.0034ge^{-x/22000}\dot{x}^2}{2\beta} - g$$

非线性的第二个来源在于测量过程本身。例如，雷达测量的是到目标的斜距，而我们实际上关心的是飞机在地面上的位置。根据勾股定理，我们可以得到以下非线性方程：

$$x = \sqrt{\mathbf{slant}^2 - \mathbf{altitude}^2}$$

卡尔曼滤波器的早期采用者们当然注意到了这些事实。在卡尔曼博士发表论文后不久，人们就开始研究如何将卡尔曼滤波器应用于非线性问题。

几乎可以打趣地说，我们真正熟练求解的就是 $\mathbf{Ax} = \mathbf{b}$：也就是线性代数问题。给定一个线性方程组，可以求解，或者判断它没有解。

接受过数学或物理正规教育的人，往往花过多年学习求积分、解微分方程等解析方法。然而，即使简单的物理系统，也可能产生无法用所需解析形式求解的方程。在一个原本能积分的方程中加入某个 $\log$ 项，就可能让求解变得困难。这也是物理学中“假设真空中的无摩擦平面上有一头球形奶牛……”这一笑话的由来。如果不作大幅简化，大多数物理问题都难以得到可用的解析解。

那么，如何在计算机上模拟飞机表面的气流、预测天气，或使用卡尔曼滤波跟踪导弹？一种重要办法是回到熟悉的 $\mathbf{Ax} = \mathbf{b}$：设法将问题局部线性化，转为一组线性方程，再利用线性代数软件求近似解。

将非线性问题线性化后得到的结果往往不够精确。在卡尔曼滤波器或天气预报系统这类递归算法中，这些微小的误差可能会在每一步中相互叠加，从而迅速导致算法输出无意义的结果。

接下来面对的问题并不简单，不再有一种显而易见、普遍适用的最优闭式解。我们将采用近似方法，并在计算中引入误差，还必须警惕滤波发散，即误差不断增长、估计逐渐失去可信度。

在本章的剩余部分，我将详细阐述非线性卡尔曼滤波器所面临的具体问题。只有先了解非线性因素所带来的各种问题，才能设计出有效的滤波器。在后面的章节中，我将教你如何设计和实现各种类型的非线性滤波器。

非线性的问题所在

卡尔曼滤波数学的优美之处，部分来自高斯分布的特殊性质。高斯密度本身是非线性函数，但联合高斯变量的和仍为高斯变量；两个高斯密度的乘积经归一化后也仍为高斯密度。一般函数并不具有类似的封闭性，例如 $\sin x * \sin y$ 通常不能写成一个简单的 $\sin$ 函数。这里必须区分随机变量相加与概率密度相乘：任意两个高斯密度直接相加，并不一定还是单个高斯密度。

我所说的“线性”概念可能很容易理解，但实际上其中有一些细微之处需要留意。从数学角度来看，有两个关键要求：

- 可加性：$f(x+y) = f(x) + f(y)$
- 齐次性：$f(ax) = af(x)$

因此，线性系统要求输入的叠加对应输出的叠加，并且满足比例缩放关系。一个直接推论是零输入必须产生零输出。以音频放大器为例：我对着麦克风唱歌，你开始说话，输出应为两人声音的和，再乘以放大器增益。若零输入时仍输出嗡声等非零信号，就不满足这种加法关系。因为加入零输入不应改变输出，即 $amp(voice) = amp(voice + 0)$；如果 amp(0) 非零，那么：

$$\begin{aligned} amp(voice) &= amp(voice + 0) \\ &= amp(voice) + amp(0) \\ &= amp(voice) + non_zero_value \end{aligned}$$

这显然是毫无道理的。因此，像这样的看似线性的方程……

$$L(f(t)) = f(t) + 1$$

并不满足线性要求，因为 $L(0) = 1$。应注意区分仿射映射与严格意义上的线性映射。

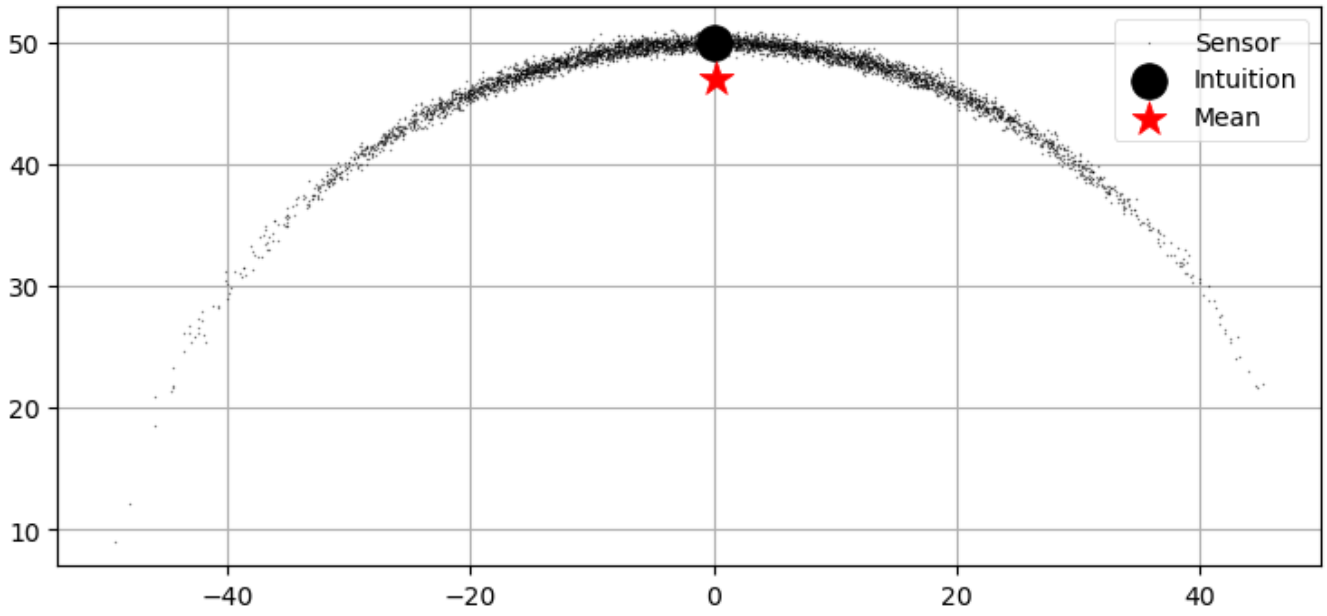
直观理解这个问题

我很喜欢 Dan Simon 在《Optimal State Estimation》中介绍的一种看法。考虑一个跟踪问题：测得目标距离和方位角，希望估计其位置。距离均值为 50 km，方位角均值为 90°。假设距离误差和角度误差均服从高斯分布。进行无限次测量时，转换得到的位置均值是多少？

前面一直建议用直觉帮助理解，现在看看它在这里是否有效。也许我们会认为：距离均值是 50 千米，角度均值是 90°，所以位置均值就应该是 x=0 千米、y=50 千米。

画图检验：生成 3000 个样本，距离服从均值为 50 km、标准差为 0.4 km 的正态分布，角度服从均值为 90°、标准差为 0.35 rad 的正态分布。将样本转换为位置坐标，计算位置的平均值，并用星号标出；直觉猜测的位置则用大圆圈标出。

```
import numpy as np
from numpy.random import randn
import matplotlib.pyplot as plt

N = 5000
a = np.pi/2. + (randn(N) * 0.35)
r = 50.0     + (randn(N) * 0.4)
xs = r * np.cos(a)
ys = r * np.sin(a)

plt.scatter(xs, ys, label='Sensor', color='k',
            alpha=0.4, marker='.', s=1)
xmean, ymean = sum(xs) / N, sum(ys) / N
plt.scatter(0, 50, c='k', marker='o', s=200, label='Intuition')
plt.scatter(xmean, ymean, c='r', marker='*', s=200, label='Mean')
plt.axis('equal')
plt.legend();
```

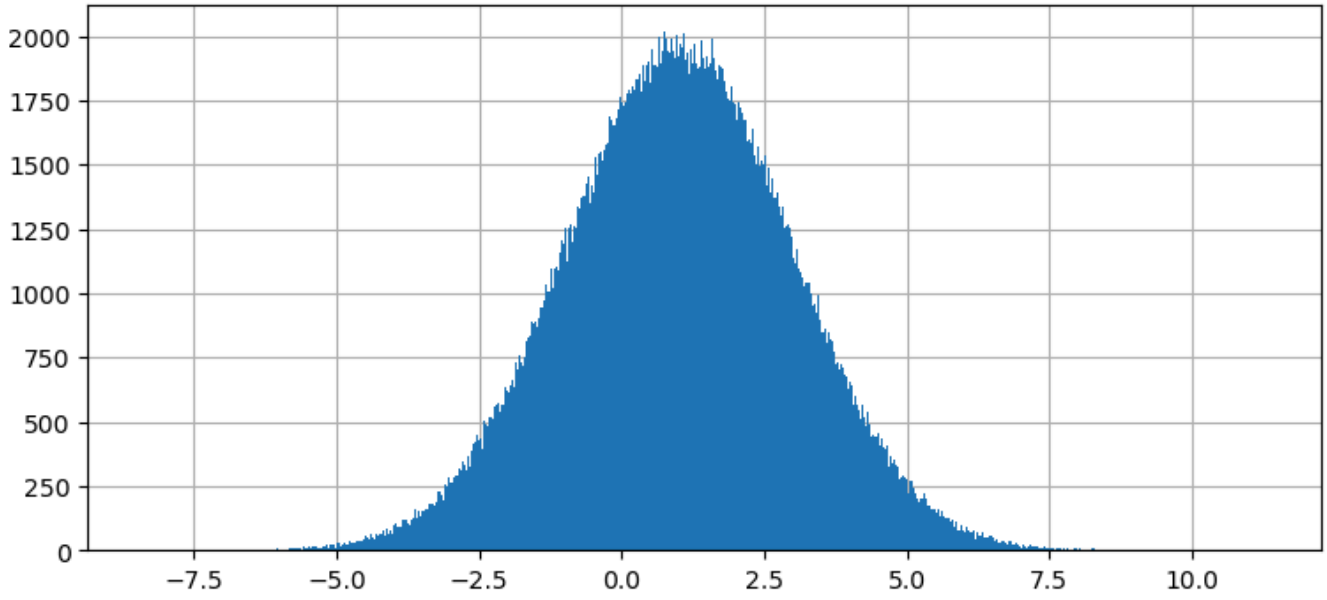


直觉在这里失效了：非线性坐标变换使位置均值相对于“把测量均值直接代入函数”的结果出现系统偏移。这种偏差若在多次迭代中累积，可能导致卡尔曼滤波发散；即使没有发散，也可能失去最优性。线性近似用于非线性问题时，必须评估它引入的误差。

非线性函数对高斯分布的影响

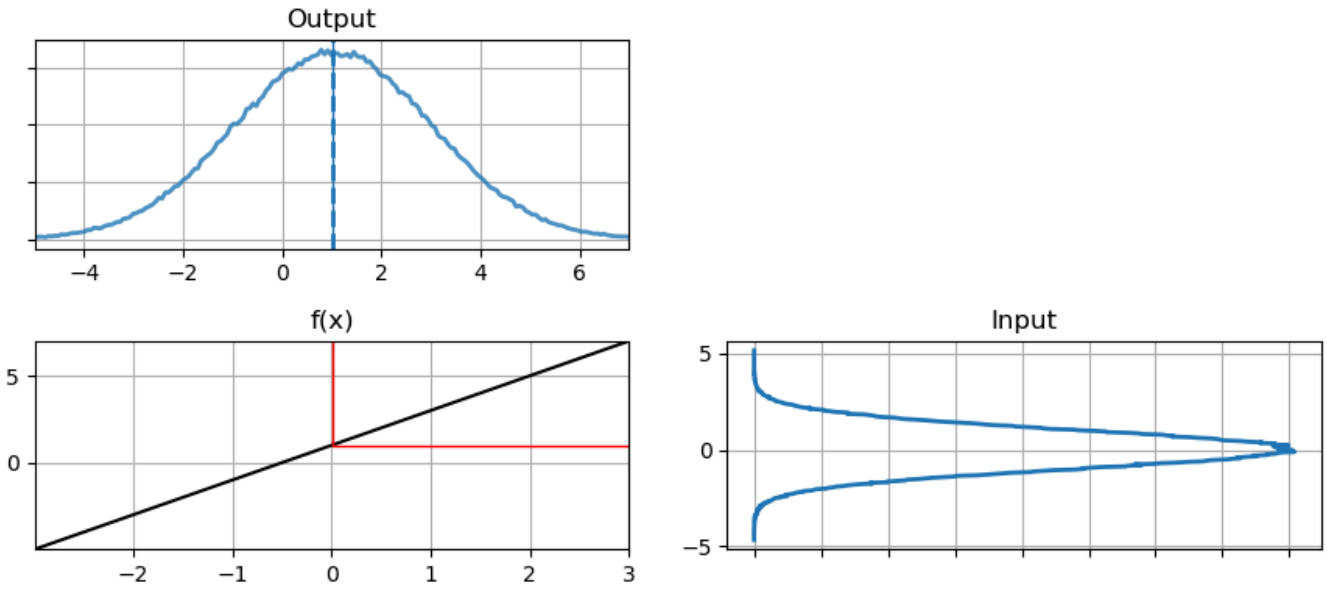
高斯分布对一般非线性变换并不封闭。回忆卡尔曼滤波：每次时间演化都通过过程函数传播状态分布，得到时刻 k 的分布。此前使用线性或仿射过程函数，所以输出仍为高斯分布。下面取一个高斯分布，经过 $f(x) = 2x + 1$ 变换并画图。虽然可以解析计算，这里采用采样：生成 500000 个正态样本，通过 $f(x)$ 后画出结果。这样也便于与下一个不直接采用解析计算的非线性例子比较。

```
from numpy.random import normal

data = normal(loc=0., scale=1., size=500000)
plt.hist(2*data + 1, 1000);
```



结果并不意外：均值为零的高斯变量经过仿射变换 $f(x) = 2x + 1$ 后，仍是高斯变量，均值变为 1。下面把输入、变换函数和输出放在同一幅图中比较。

```
from kf_book.book_plots import set_figsize, figsize
from kf_book.nonlinear_plots import plot_nonlinear_func

def g1(x):
    return 2*x+1

plot_nonlinear_func(data, g1)
```

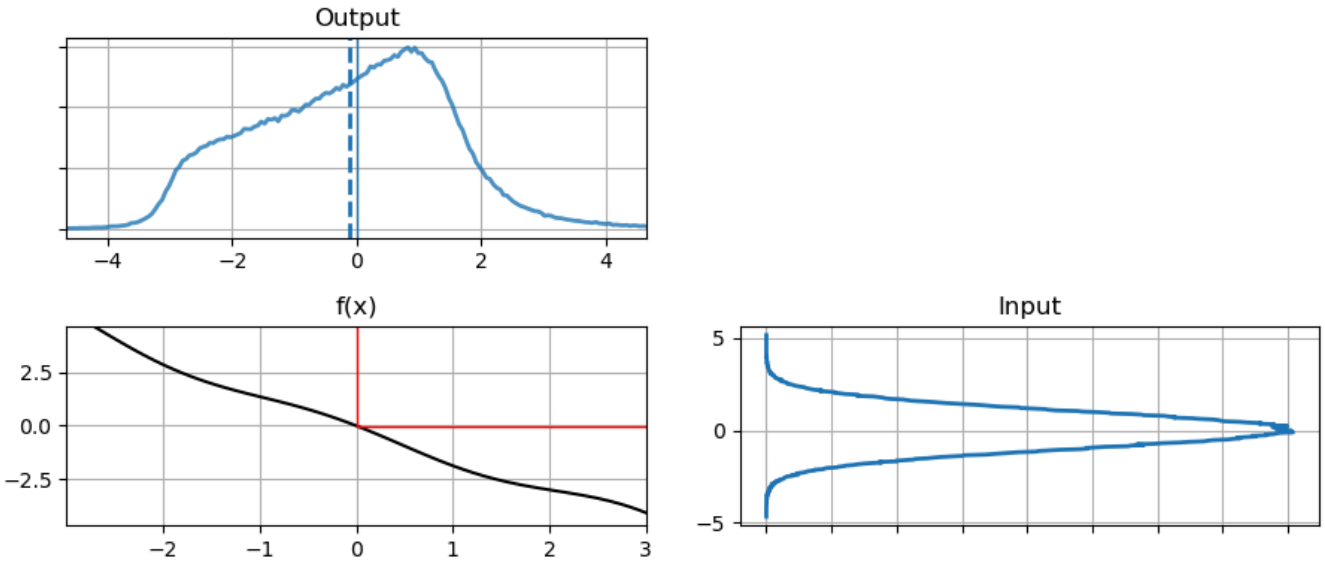


> 我在“Supporting_Notebooks”文件夹中的“Computing_and_Plotting_PDFs”文档里详细介绍了如何绘制高斯曲线图以及更多相关内容。你也可以在这里在线阅读该文档[1]。

“Input”图是原始样本的直方图。样本经过左下图所示函数 $f(x) = 2x + 1$ 变换后，形成右侧的输出分布。红线演示了 $x = 0$ 这个输入值如何通过函数；其余输入样本也按同样方式变换。输出图中的蓝色虚线是样本输出的平均值，蓝色实线则表示输入均值 $x = 0$ 经变换后的值；对本例的仿射变换，该值也正是输出的理论均值。输出确实仍为高斯分布，方差变大，均值由 0 移至 1，符合 $f(x) = 2x + 1$ 的性质：$2x$ 使方差扩大为原来的 4 倍，$+1$ 使均值平移。样本均值与理论均值很接近，增加样本数通常可进一步减小采样误差。

现在，让我们来看看一个非线性函数，看看它如何影响概率分布。

```
def g2(x):
    return (np.cos(3*(x/2 + 0.7))) * np.sin(0.3*x) - 1.6*x

plot_nonlinear_func(data, g2)
```



这个结果可能令人意外：函数看起来相当接近直线，但输出概率分布却明显不同于高斯分布。回忆两个单变量高斯密度相乘并归一化后的均值和方差公式：

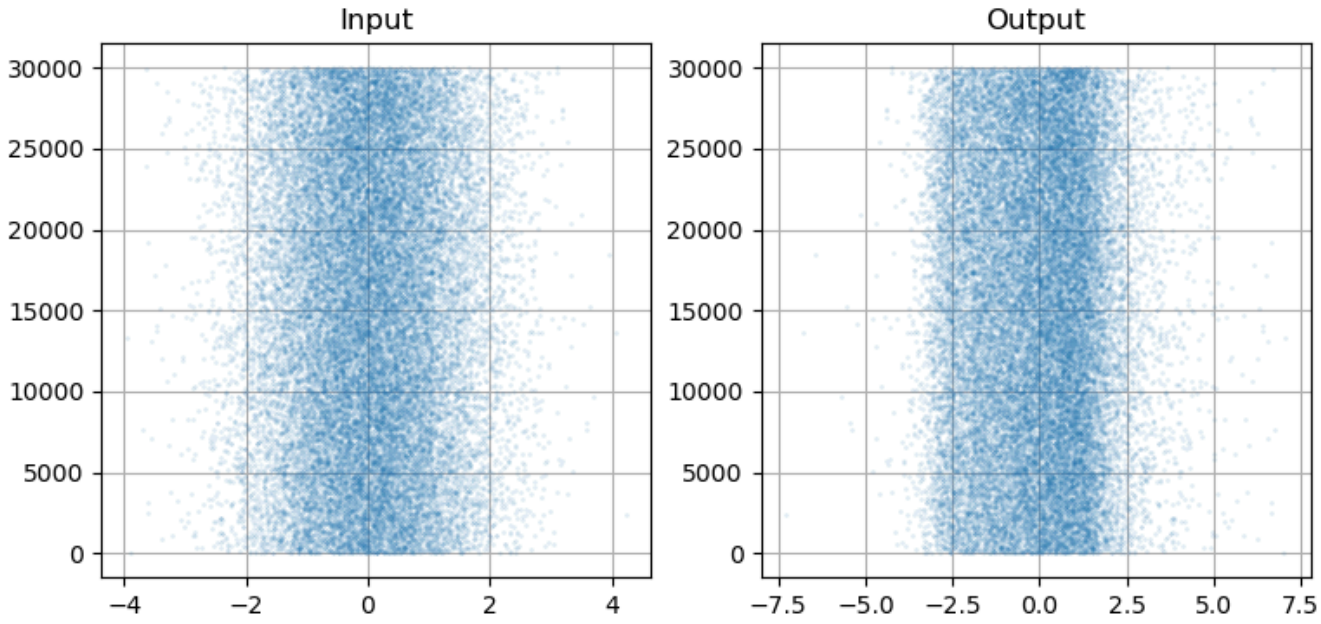
$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}$$

$$\sigma^2 = \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}$$

这些方程不适用于非高斯分布的情况，当然也不适用于上图“输出”中所显示的概率分布。

另一种查看相同数据的方式就是用散点图来表示。

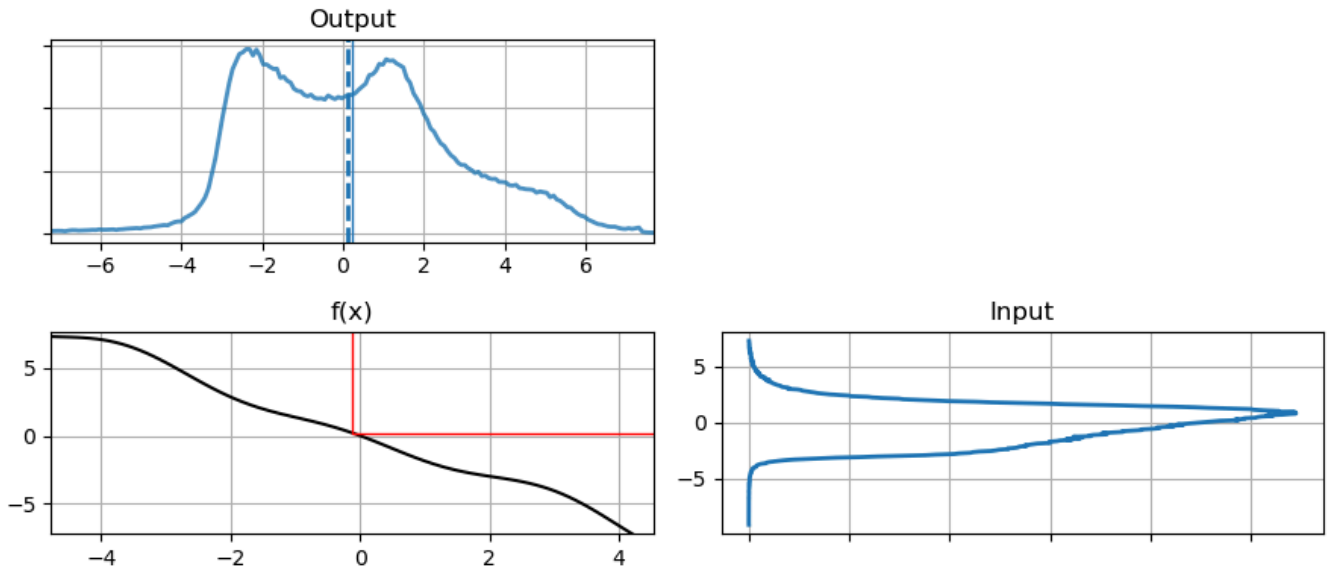
```
N = 30000
plt.subplot(121)
plt.scatter(data[:N], range(N), alpha=.1, s=1.5)
plt.title('Input')
plt.subplot(122)
plt.title('Output')
plt.scatter(g2(data[:N]), range(N), alpha=.1, s=1.5);
```



原始数据显然呈高斯分布，但经过 g2(x) 处理后的数据不再服从正态分布。在 -3 附近有一条密集的带状区域，其两侧的数据点分布并不均匀。如果将这个散点图与上图中标记为“Output”的概率密度图比较，就能看出概率密度曲线的形状与 g(data) 的数据分布是如何对应的。

试想一下，这对上一章中提到的卡尔曼滤波算法意味着什么。所有的方程都假设：经过该处理函数后，高斯分布仍然保持为高斯分布。如果这一假设不成立，那么卡尔曼滤波的所有前提和保证条件就都不复存在了。让我们来看看，当再次将处理后的结果输入该函数时会发生什么，这实际上模拟了卡尔曼滤波的下一个时间步的处理过程。

```
y = g2(data)
plot_nonlinear_func(y, g2)
```



如图所示，概率密度函数进一步偏离了原始的高斯形态。不过，图形仍大致关于 x=0 对称。让我们计算一下均值。

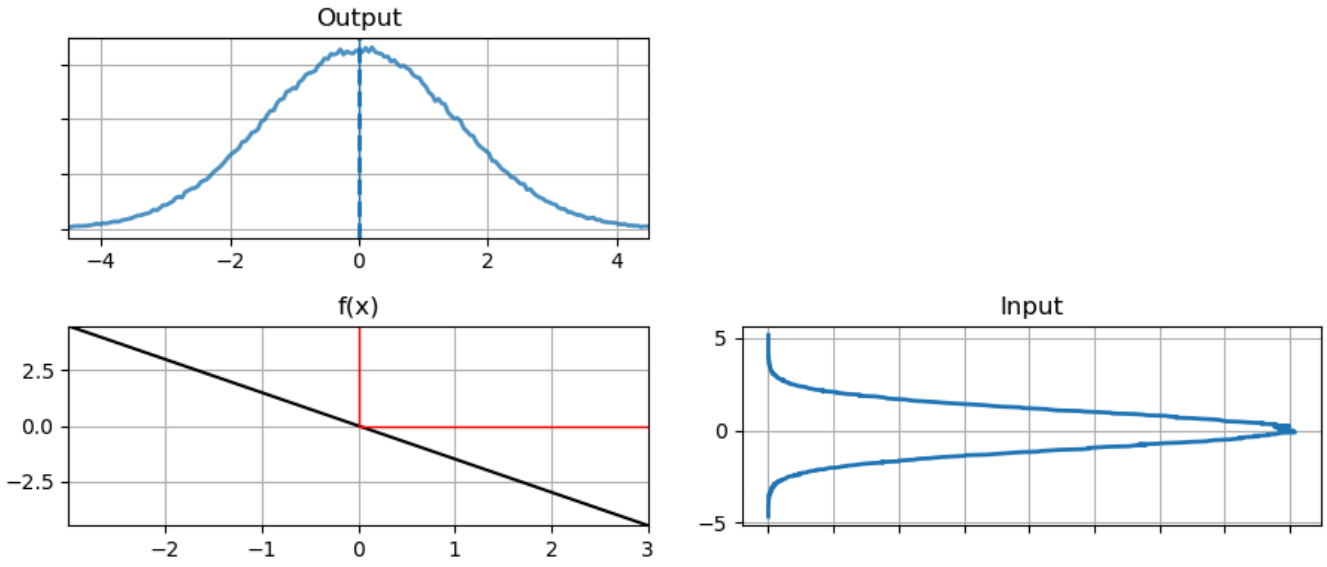
```
print('input  mean, variance: %.4f, %.4f' %
      (np.mean(data), np.var(data)))
print('output mean, variance: %.4f, %.4f' %
      (np.mean(y), np.var(y)))
```

```
input  mean, variance: 0.0022, 1.0000
output mean, variance: -0.1282, 2.4029
```

让我们将其与经过点(-2,3)和(2,-3)的直线函数进行比较。这条直线函数与我们所绘制的非线性函数非常接近。根据直线方程，我们有：

$$m = \frac{-3-3}{2-(-2)} = -1.5$$

```
def g3(x):
    return -1.5 * x

plot_nonlinear_func(data, g3)
out = g3(data)
print('output mean, variance: %.4f, %.4f' %
      (np.mean(out), np.var(out)))
```




```
output mean, variance: -0.0033, 2.2500
```

虽然各输出结果的形态差异很大，但它们的均值和方差却几乎相同。这让我们觉得，如果非线性方程“接近”线性方程的话，或许可以忽略这个问题。为了验证这一点，我们可以重复多次计算，然后比较结果。

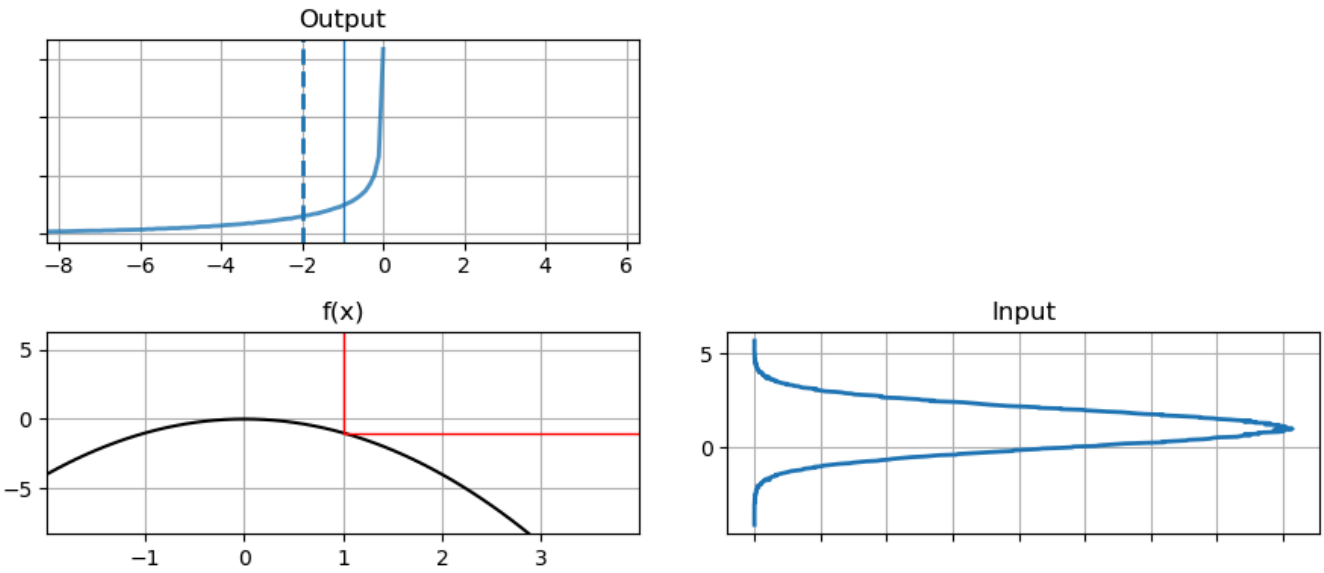
```
out = g3(data)
out2 = g2(data)

for i in range(10):
    out = g3(out)
    out2 = g2(out2)
print('linear    output mean, variance: %.4f, %.4f' %
      (np.average(out), np.std(out)**2))
print('nonlinear output mean, variance: %.4f, %.4f' %
      (np.average(out2), np.std(out2)**2))
```

```
linear    output mean, variance: -0.1915, 7481.9567
nonlinear output mean, variance: -9.5578, 30512.3474
```

不幸的是，非线性模型并不稳定。其数值偏离了 0 这个平均值，偏差幅度相当大；同时，其方差也大了半个数量级。

前面故意选取了非常接近直线的函数，以减弱问题。若函数改为 $y(x) = -x^2$，结果又如何？

```
def g3(x):
    return -x*x

data = normal(loc=1, scale=1, size=500000)
plot_nonlinear_func(data, g3)
```

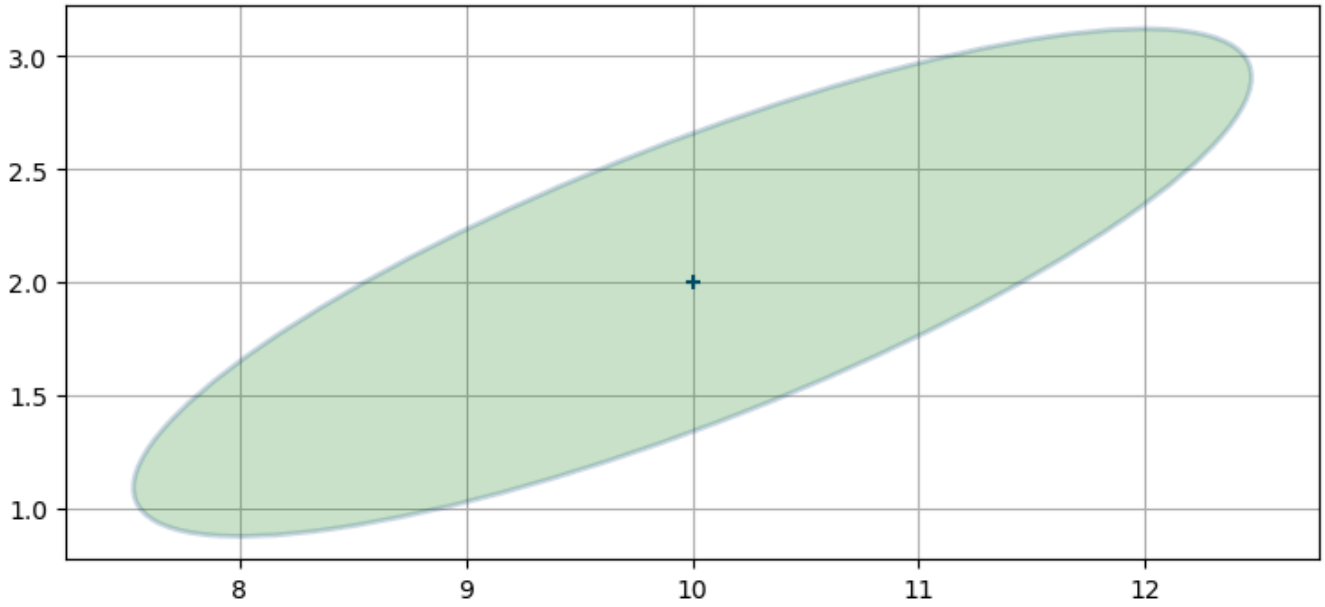


尽管曲线在 $x = 1$ 附近平滑且局部接近直线，输出分布仍明显不是高斯分布，而且输出均值与直接把输入均值代入函数所得的值差别很大。这种二次函数并不罕见，例如抛体位置与时间的关系就是二次函数。不过，恒定重力下，包含位置与速度的完整状态仍可通过线性状态转移加已知输入来精确传播；空气阻力等因素才会引入额外的非线性。前面跟踪球的例子提醒我们，建模时必须辨清非线性实际出现在哪个环节，不能只凭轨迹形状判断。

二维示例

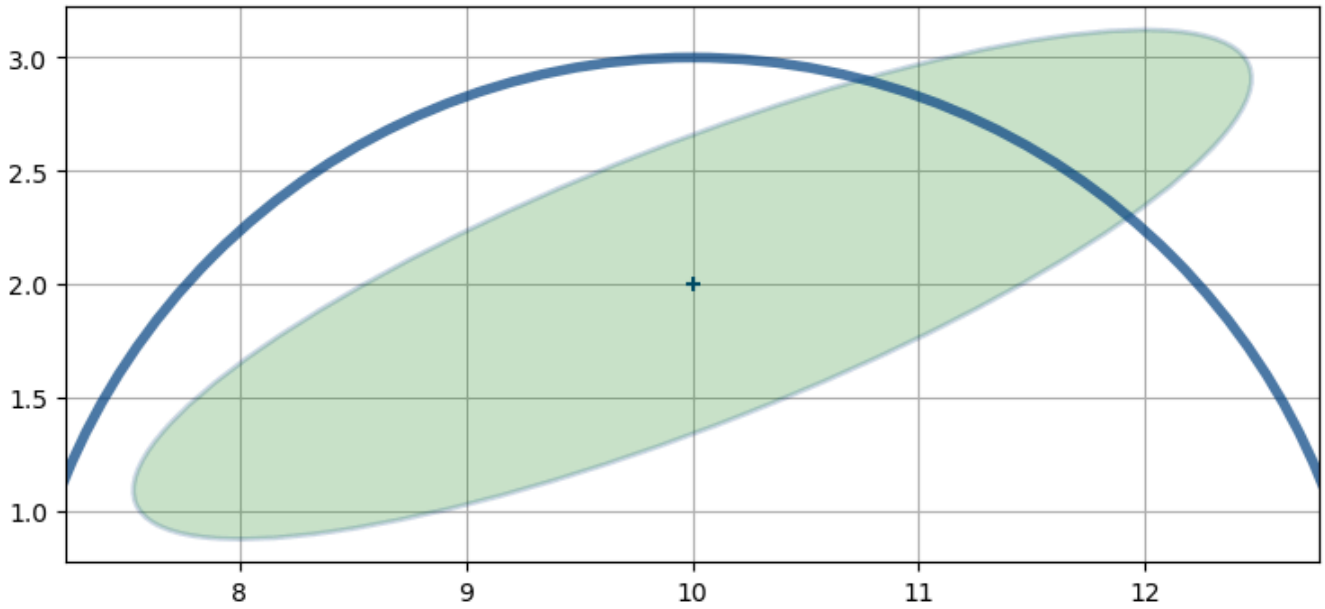
仅凭概率分布图，可能很难推想滤波器会怎样工作。换成用雷达跟踪飞机的例子：状态估计的协方差椭圆可能如下所示。

```
import kf_book.nonlinear_internal as nonlinear_internal

nonlinear_internal.plot1()
```



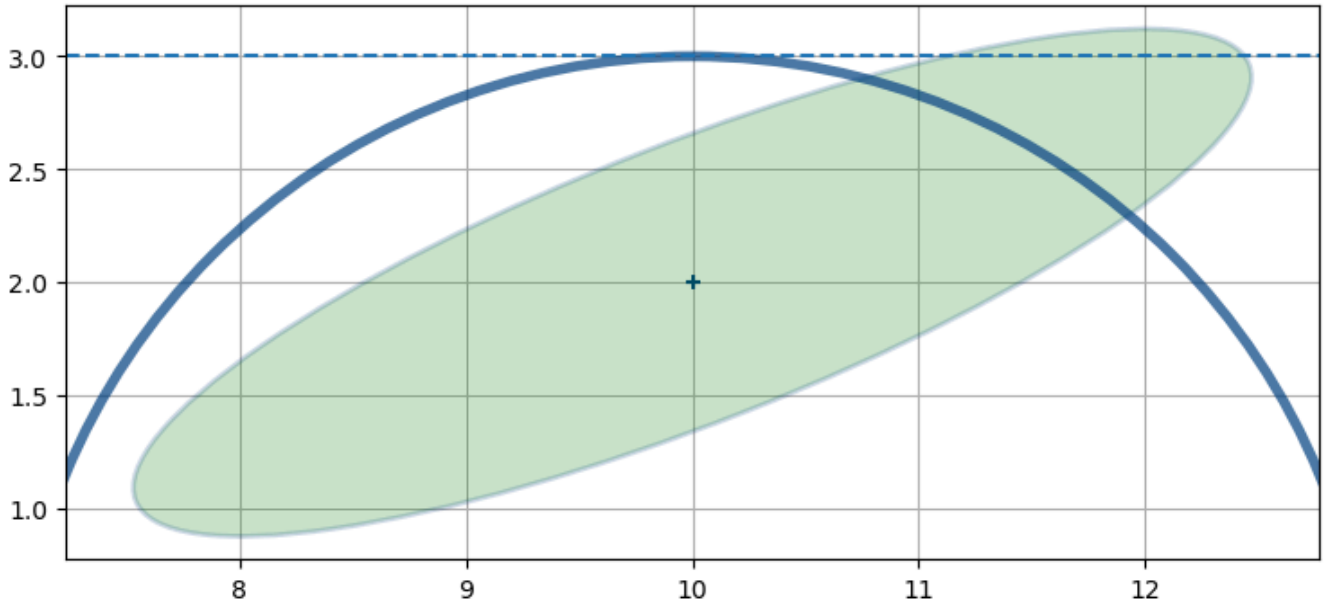
尝试将这个问题线性化，会发生什么？雷达给出的是到飞机的距离。假设雷达位于 x=10 处，恰好在飞机的正下方，而下一次测量表明飞机距离雷达 3 英里。所有与这一测距结果相符的位置，构成一个以雷达为圆心、半径为 3 英里的圆，如下图所示。

```
nonlinear_internal.plot2()
```



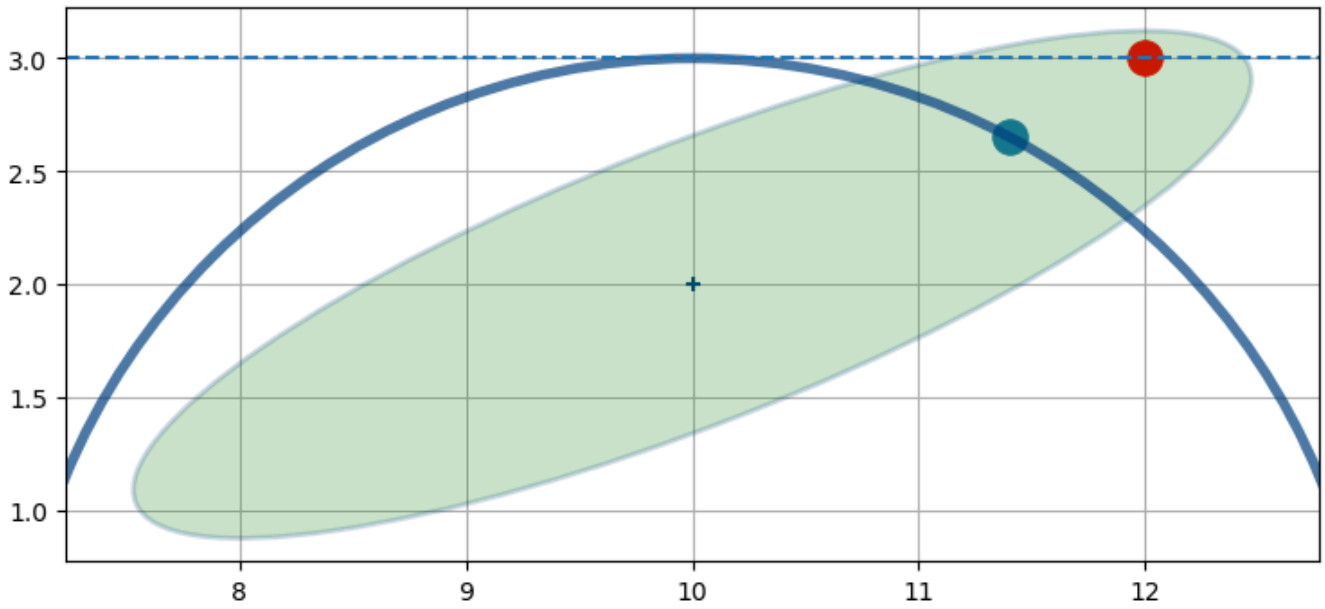
从图上看，飞机可能位于 x=11.4、y=2.7 附近，因为协方差椭圆与测距圆在那里重叠。距离测量模型是非线性的，因此需要线性化。后面将介绍扩展卡尔曼滤波器如何在当前先验位置附近计算测量函数的雅可比矩阵。本例先验位置为 (10,2)，相应的线性化测距约束在 x=10 时经过 y=3，所以下面用该处的切线展示近似效果。

```
nonlinear_internal.plot3()
```



现在，我们已经得到了该问题的线性表示形式（实际上就是一条直线），因此可以对其进行求解。不过，可以看到，这条直线与协方差椭圆的交点，距离飞机的实际位置还有相当大的距离。

```
nonlinear_internal.plot4()
```



这种错误往往会导致灾难性的后果。这次的估算误差相当大。但在滤波器的后续处理中，这个错误的估算值会被用来对下一个雷达测量结果进行线性化处理。这样一来，下一个估算结果很可能会比现在这个更糟糕。只要经过几次迭代，卡尔曼滤波器就会失去稳定性，产生的结果将完全不符合现实情况。

这个协方差椭圆横跨数英里，我有意放大了它，以说明在较大不确定性范围内使用局部线性近似的困难。实际雷达跟踪中的非线性通常没有这么严重，但误差仍可能累积。其他系统确实可能具有这样的非线性程度，因此这不只是夸张示例。处理非线性系统时，应始终检查滤波发散风险。

主要算法

你可能急于解决某个具体问题，不知道该使用哪种滤波方式。我会简要介绍一下各种选项。后面的章节彼此之间有一定的独立性，你可以随意跳读。不过，如果你真的想掌握所有内容，我建议按顺序阅读。

在非线性滤波器中，最常用的两种方法是线性化卡尔曼滤波器和扩展卡尔曼滤波器。这两种技术都是在卡尔曼发表其论文后不久被发明的，从那以后，它们就一直被广泛使用。飞机上的飞行控制系统、汽车或手机中的 GPS 系统，几乎都采用了这两种技术中的某一种。

不过，这些技术对建模和实现的要求较高。EKF 在当前估计附近线性化状态或测量函数，需要计算偏导数组成的雅可比矩阵。解析求导有时很困难；此时可用数值微分等方法，但会增加计算量并引入近似误差。若非线性很强，每一步线性化都可能产生较大误差，甚至导致滤波发散。不能简单地把方程交给任意求解器，就期待可靠结果；这对专业人员也是有挑战的工作。尽管 EKF 在实际应用中十分常见，许多教材仍只简略介绍它。

近年来，计算能力的增长使一些过去连超级计算机也难以承担的方法变得可行。其中一类采用蒙特卡洛技术：生成数千到数万个随机状态样本，用测量评价每个样本的相容程度，再按权重随机淘汰或复制样本。与测量不相容的样本通常较难保留，相容样本则更可能被复制。多次迭代后，粒子可能集中在目标状态附近，其他区域则较为稀疏。

这种方法有两方面优势：可以处理很强的非线性，也可通过适当设计扩展到多目标跟踪，用于交通和人群等场景。多目标应用仍需定义联合状态或相应的多目标模型、处理测量关联及目标的出生与消失，并配置足够的粒子。普通单目标粒子滤波器不会自动具备这些能力。

代价也很明显：每个时间步都要计算大量粒子，成本较高。现代 CPU 很快，算法中可并行的部分也适合 GPU。粒子滤波用加权样本近似后验，可以据此计算均值、协方差及其他不确定性指标，但通常不会直接给出解析的协方差递推。如果后验存在多个峰，或者涉及多目标，还需要决定如何解释这些粒子，例如先聚类，再从不同粒子群提取状态估计。这些处理有严格的概率理论基础，但会增加计算量。

最后，还有无迹卡尔曼滤波器（UKF）。它不要求显式计算非线性模型的雅可比矩阵，设计和实现也比较方便。在我的实践中，它通常优于 EKF，也能很好地处理让 EKF 遇到较大困难的非线性问题。因此，我建议从 UKF 开始尝试，尤其是没有接受过控制理论研究生训练的读者。UKF 有时会比 EKF 慢数倍，但这取决于模型维数，以及 EKF 的雅可比采用解析还是数值方式计算；数值求导成本较高时，UKF 也可能更快。具体精度、稳定性和速度仍取决于模型、参数与实现方式，并不存在 UKF 在所有问题上都优于 EKF 的保证。

本章小结

现实问题往往具有非线性，而我们最熟悉的求解工具是线性方法，这给卡尔曼滤波带来了挑战。本章从三种相互联系的角度展示了非线性对滤波的影响，并简要介绍主要处理方法：线性化卡尔曼滤波、扩展卡尔曼滤波、无迹卡尔曼滤波和粒子滤波。

在本书写作时，线性化卡尔曼滤波器和 EKF 长期是处理这些问题的标准方法。它们可能较难理解和使用，线性化不当时还会出现不稳定。

较新的方法提供了更方便的选择。UKF 无须显式计算偏导数，在不少问题上也能取得较好的精度；使用 FilterPy 可以很快搭建基本原型。粒子滤波通过大量随机样本近似后验分布，可处理一些解析方法难以解决的问题，但通常计算较慢。这些方法减轻了某些解析推导的负担，仍然需要建立状态转移、测量及噪声等概率模型。

我收到的关于 EKF 的邮件比关于其他任何方法的邮件都要多。我觉得，这是因为在各种书籍、论文和网络资料中，EKF 都是最常用的方法。如果你想真正掌握这一领域的相关知识，那么当然有必要了解 EKF。不过，如果你只是想获得不错的处理结果，我建议你先学习 UKF 和粒子滤波器。这两种方法的实现、理解和使用都相对简单，而且通常比 EKF 更稳定。

有些人会不同意上述建议。很多研究都比较 EKF、UKF 及其他方法在特定问题中的表现；你的问题是否也需要这样的比较？如果要把火箭送往火星，当然需要。必须权衡精度、舍入误差、发散风险、正确性论证以及计算量。在这样的应用中，很难想象设计者可以不深入了解 EKF。

另一方面，UKF 的表现实在出色！我在工作中实际应用了它。对于这些应用场景，我甚至没有尝试过使用 EKF，因为我已经确认 UKF 能够很好地发挥作用。当然，在某些情况下，或许还能从 EKF 中再提升 0.2% 的性能吧？不过，我并不在乎这点。我完全理解 UKF 的实现原理，它很容易测试和验证。我可以把相关代码交给别人，而且有信心他们能够理解并修改代码。反正，既然已经有有效的解决方案了，我可不想再去处理那些复杂的方程式。如果 UKF 或粒子滤波器在处理某些问题时表现不佳，我才会考虑使用其他技术。不过，通常情况下，UKF 在各种问题和条件下的表现都要优于 EKF。如果“相当不错”就已经足够好了，那我就把时间花在解决其他问题上吧。

我之所以反复强调这一点，是因为在大多数教科书中，EKF 都是被重点介绍的内容，而 UKF 要么根本不被提及，要么只是被简单介绍两页而已，这显然无法让人真正掌握如何使用 UKF。UKF 毕竟还是相对较新的算法，因此需要时间来更新相关教材。在许多教材编写时，UKF 要么还未被发明，要么只是被认为是一种有潜力但尚未得到验证的算法。不过，现在 UKF 已经取得了巨大的成功，因此它绝对应该成为你工具箱中的重要工具。这也是我接下来要重点教授你的内容。

参考文献

[1] https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/Supporting_Notebooks/Computing_and_plotting_PDFs.ipynb

第 10 章：无迹卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

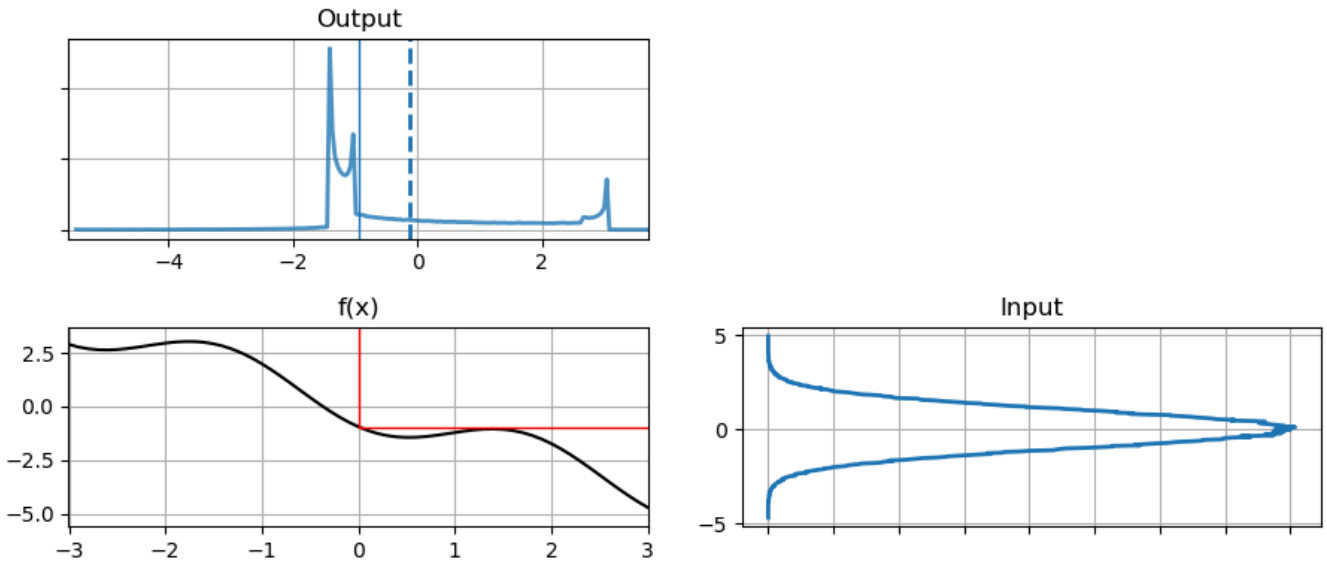
上一章讨论了非线性系统带来的困难。非线性可能出现在两个环节。一个是测量模型，例如雷达测量目标的斜距。斜距与平面位置坐标之间的关系包含平方根运算：

$$x = \sqrt{\text{slant}^2 - \text{altitude}^2}$$

这种非线性现象也可能出现在过程模型中——比如，当我们追踪一个在空气中飞行的球体时，空气阻力的影响会导致非线性行为。对于这类问题，标准的卡尔曼滤波器要么无法有效处理，要么效果很差。

在上一章中，我向你们展示过类似的图表。我对公式做了一些调整，以便更清楚地体现非线性效应。

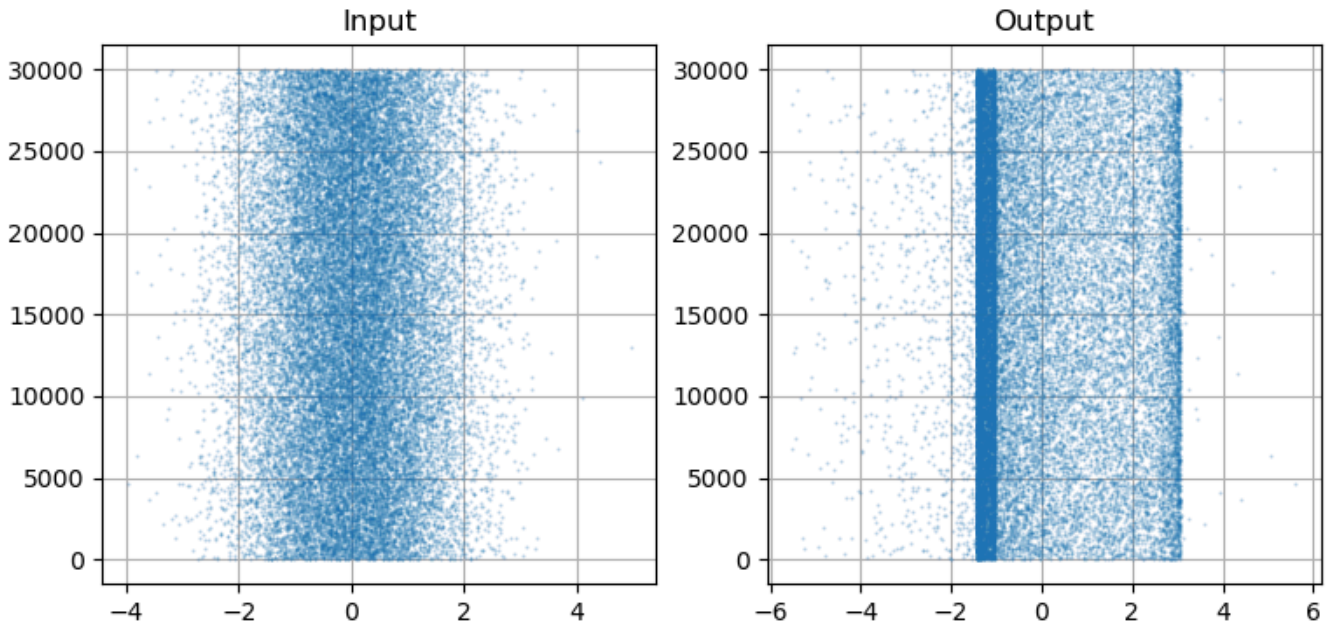
```
from kf_book.book_plots import set_figsize, figsize
import matplotlib.pyplot as plt
from kf_book.nonlinear_plots import plot_nonlinear_func
from numpy.random import normal
import numpy as np

# create 500,000 samples with mean 0, std 1
gaussian = (0., 1.)
data = normal(loc=gaussian[0], scale=gaussian[1], size=500000)

def f(x):
    return (np.cos(4*(x/2 + 0.7))) - 1.3*x

plot_nonlinear_func(data, f)
```



这里从输入分布随机抽取 500,000 个样本，将它们通过非线性变换，再绘制输出直方图。根据变换后的样本，可以计算输出均值与标准差，并用相应的高斯分布近似输出分布。这里采用的是随机采样，后面无迹变换使用的 sigma 点则是按确定性规则选出的少量代表点。

让我来向你展示一下数据在经过 `f(x)` 处理前后的散点图。

```
N = 30000
plt.subplot(121)
plt.scatter(data[:N], range(N), alpha=.2, s=1)
plt.title('Input')
plt.subplot(122)
plt.title('Output')
plt.scatter(f(data[:N]), range(N), alpha=.2, s=1);
```



输入数据看起来服从高斯分布，事实上也确实如此：这些点像白噪声一样散布在零均值附近。相比之下，g(data) 具有明显的结构，形成两条带状区域，二者之间还有相当多的点。带状区域之外也有零散的点，其中负值一侧的点明显更多。

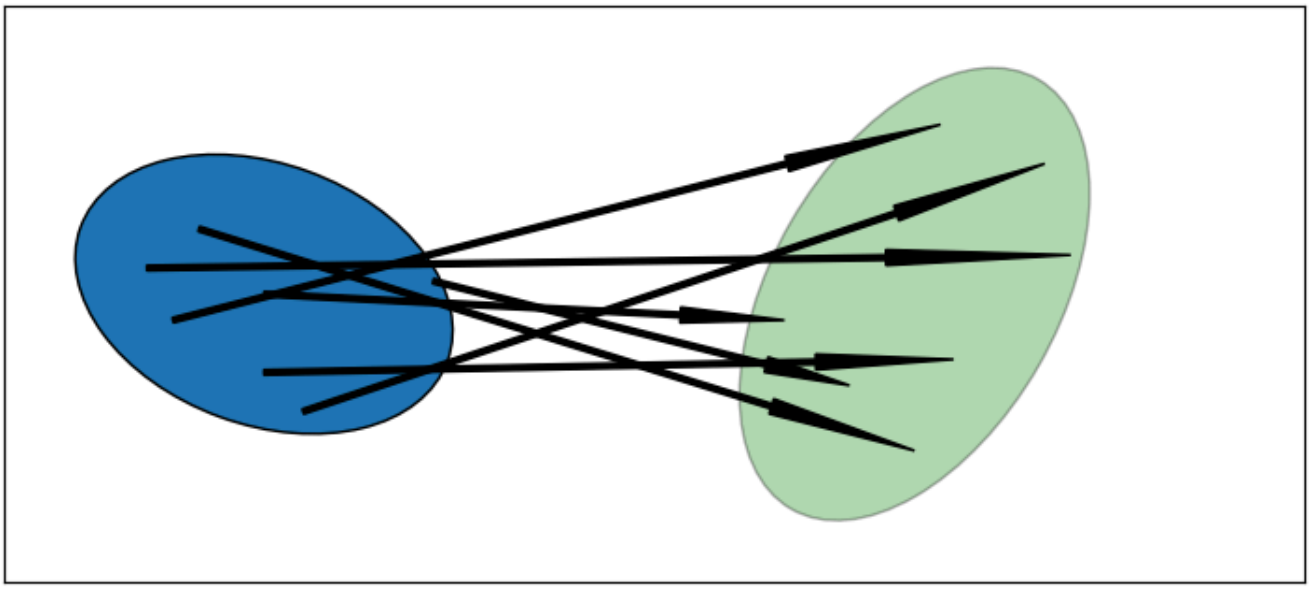
你也许已经想到，这种采样本身就提供了一种解法：每次更新时生成 500,000 个样本，通过函数变换，再计算输出的均值和方差。这就是蒙特卡洛方法，集合滤波器和粒子滤波器等方法会利用它。采样不要求推导专门的解析公式，也不要求有闭式解。只要满足适当的可积性条件，并使用足够多的样本，就能近似输出分布，即使变换函数具有很强的非线性。

难点在于究竟需要多少样本。上图使用 500,000 个随机样本，输出直方图仍不十分平滑，而且这还只是一维问题。如果要求在每个维度维持同样的网格分辨率，点数会随维度指数增长：每维 500 个点，二维需要 250,000 个，三维需要 125,000,000 个。这说明高维空间的密集覆盖代价很高，但不意味着所有蒙特卡洛积分都必然需要如此增长的样本数。集合滤波与粒子滤波通过更巧妙的采样减少所需点数，仍可能有较大计算负担；UKF 则按确定性规则选取少量 sigma 点，大幅降低计算量。

sigma 点——从概率分布中抽取样本

从二维协方差椭圆来理解这个问题会更直观。选择二维只是因为便于绘图，方法本身可推广到更高维。对于给定的非线性函数，先从第一个椭圆所代表的高斯分布中随机采样，将样本通过该函数，再画出变换后的点。然后计算其均值与协方差，用它们来估计输出分布，并构造相应的高斯近似。

```
import kf_book.ukf_internal as ukf_internal
ukf_internal.show_2d_transform()
```



左侧椭圆表示两个状态变量的 1σ 协方差轮廓。箭头展示若干随机样本经过某个非线性函数后如何改变位置。右侧半透明椭圆表示用变换后样本估计的均值与协方差。

下面编写一个函数：从下述高斯分布中随机抽取 10,000 个样本点，

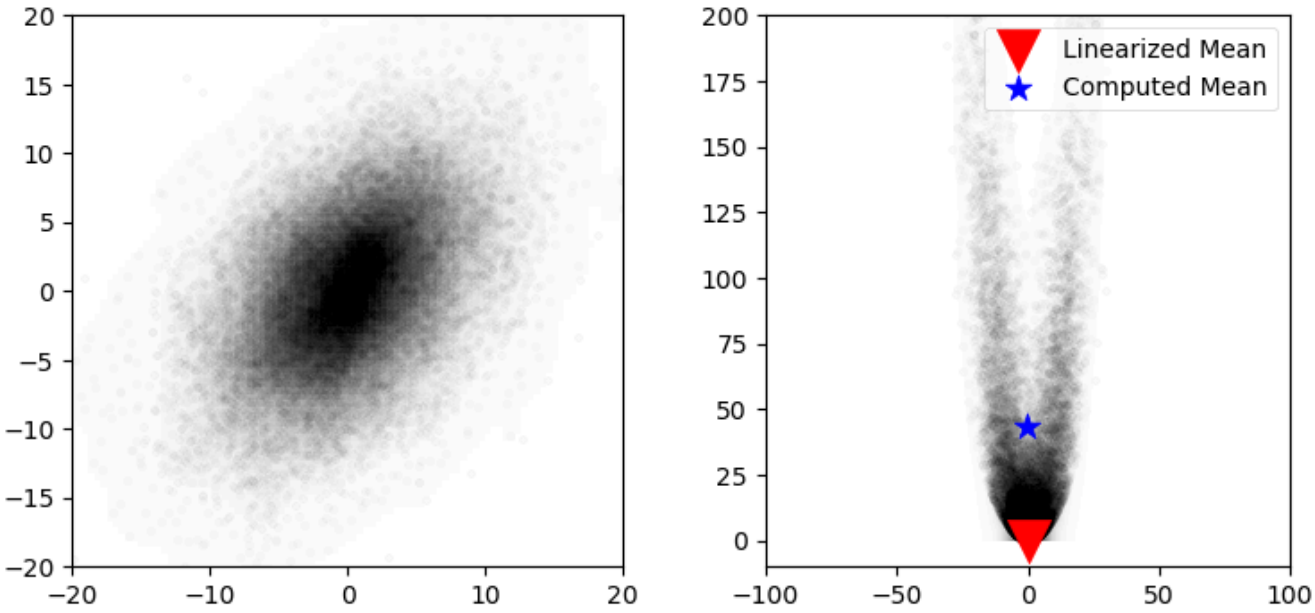
$$\mu = \begin{bmatrix}0\\0\end{bmatrix}, \Sigma = \begin{bmatrix}32 & 15\\15 & 40\end{bmatrix}$$

再将这些点通过以下非线性系统：

$$\begin{cases}\bar{x} = x + y\\ \bar{y} = 0.1x^2 + y^2\end{cases}$$

```
import numpy as np
from numpy.random import multivariate_normal
from kf_book.nonlinear_plots import plot_monte_carlo_mean

def f_nonlinear_xy(x, y):
    return np.array([x + y, .1*x**2 + y*y])

mean = (0., 0.)
p = np.array([[32., 15.], [15., 40.]])
# Compute linearized mean
mean_fx = f_nonlinear_xy(*mean)

#generate random points
xs, ys = multivariate_normal(mean=mean, cov=p, size=10000).T
plot_monte_carlo_mean(xs, ys, f_nonlinear_xy, mean_fx, 'Linearized Mean');
```

```
Difference in mean x=-0.055, y=43.117
```



该图表清楚地显示了该函数所具有的强烈非线性特征。如果我们按照扩展卡尔曼滤波的方法来对其实行线性化处理，将会产生很大的误差（我们将在下一章中详细学习这一内容）。

一个简单的例子

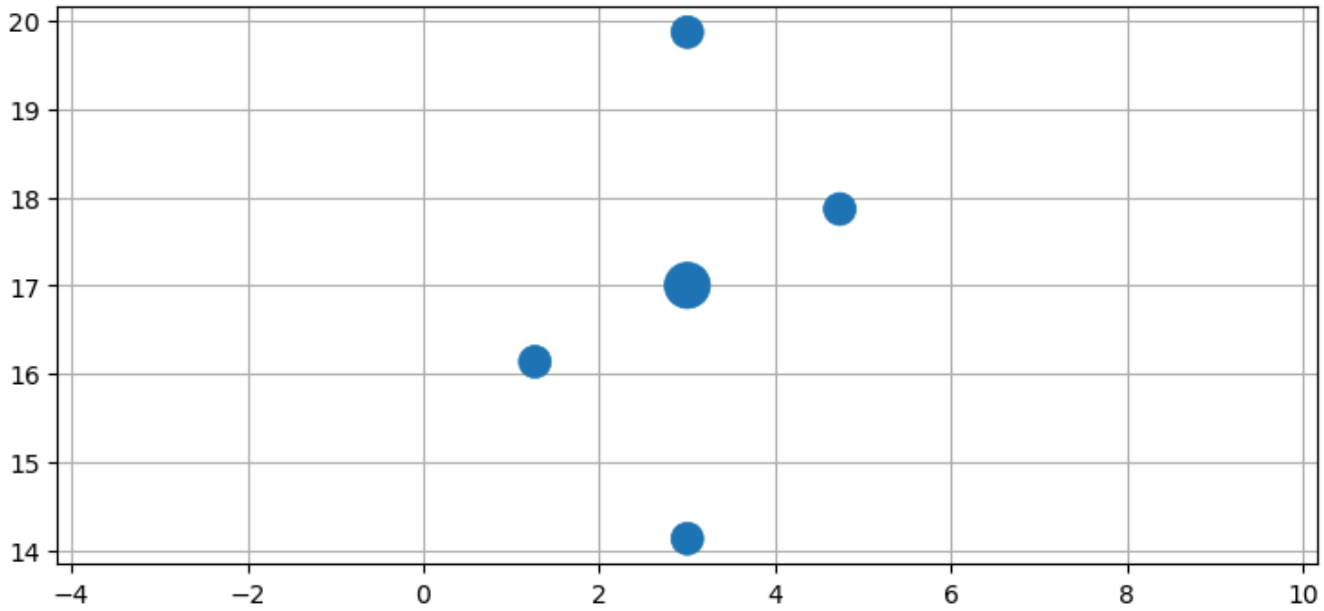
我马上就要开始讲解无迹卡尔曼滤波器在确定sigma 点以及进行后续计算时所使用的具体方法了。不过，我们先从一个例子开始吧，这样你就能明白整个过程的运作方式了。

我们将了解到，UKF 可以使用多种不同的算法来生成sigma 点。FilterPy 提供了几种这样的算法。以下是其中一种算法：

```
from filterpy.kalman import JulierSigmaPoints

sigmas = JulierSigmaPoints(n=2, kappa=1)
```

稍后就会更清楚了。不过，该对象会为任意给定的均值和协方差生成相应的加权sigma 点。我们来看一个例子吧：点的大小代表了该点的权重大小。

```
from kf_book.ukf_internal import plot_sigmas
plot_sigmas(sigmas, x=[3, 17], cov=[[1, .5], [.5, 3]])
```



这 5 个点以均值 (3, 17) 为中心，采用一种特殊的排列。用如此少的点，竟可能得到与 500,000 个随机样本相当、甚至更好的均值和协方差估计，看起来令人难以置信；但在合适的问题中确实可以做到。这里比较的是这些统计量的近似，并非完整输出分布的重建。

现在实现滤波器，先从一维线性跟踪问题开始，再进入非线性问题。总体设计与前面相似，主要区别是：KalmanFilter 用矩阵 $\mathbf{F}$ 表示状态转移，但固定矩阵只适用于线性映射，所以 UKF 改为接收一个函数。类似地，线性滤波器用矩阵 $\mathbf{H}$ 将状态映射为预测测量，UKF 则接收一个测量函数。这也说明了为什么 $\mathbf{H}$ 常被称为“测量函数”：在线性情形中，矩阵乘法只是计算该线性函数的一种简便方式。

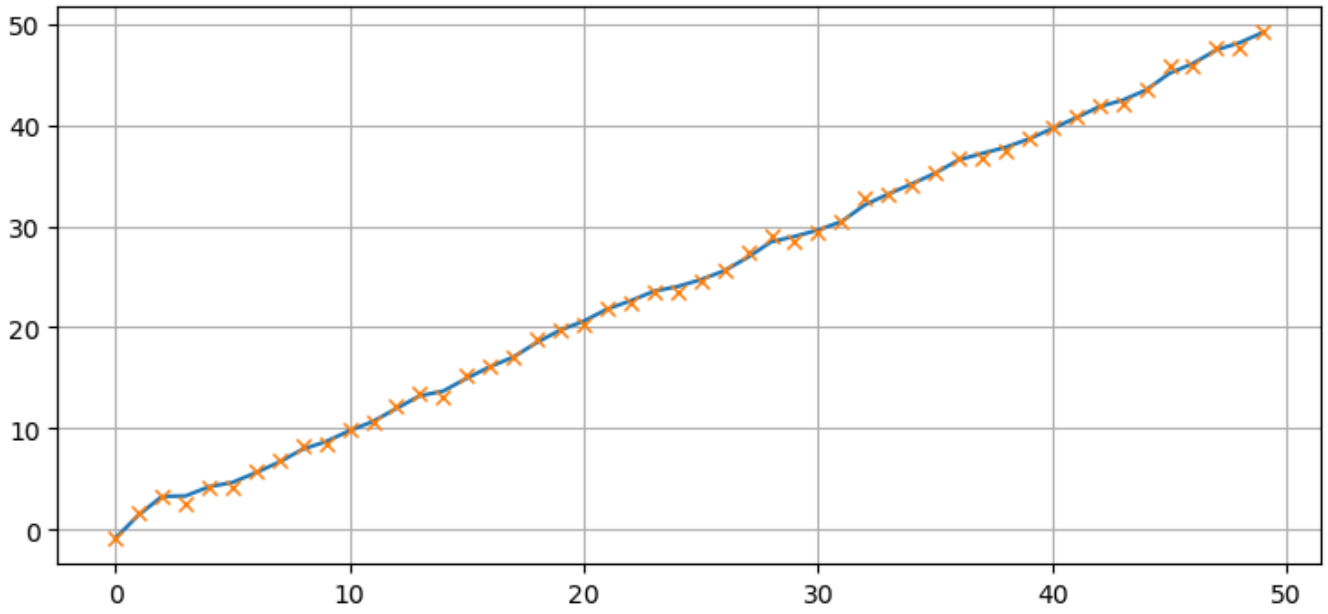
无需多言，以下是一维跟踪问题中的状态转移函数和测量函数。其中，状态为 $\mathbf{x} = [x\ \dot{x}]^\mathsf{T}$：

```
def fx(x, dt):
    xout = np.empty_like(x)
    xout[0] = x[1] * dt + x[0]
    xout[1] = x[1]
    return xout

def hx(x):
    return x[:1] # return position [x]
```

需要明确的是，这只是一个线性示例而已。对于线性问题来说，其实没必要使用 UKF 算法。不过，我还是从最简单的例子开始讲解。请注意，我使用 `fx()` 来计算 $\bar{\mathbf{x}}$，是通过一组方程来实现的，而不是通过矩阵乘法。这样做是为了说明：在这里，我可以任意定义非线性函数；我们并不一定非得使用线性方程来表示数学关系。

其余设计与前面相同：确定 $\mathbf{P}$、$\mathbf{R}$ 和 $\mathbf{Q}$。这些步骤已经熟悉，下面完成滤波器并运行。

```
from numpy.random import randn
from filterpy.kalman import UnscentedKalmanFilter
from filterpy.common import Q_discrete_white_noise

ukf = UnscentedKalmanFilter(dim_x=2, dim_z=1, dt=1., hx=hx, fx=fx, points=sigmas)
ukf.P *= 10
ukf.R *= .5
ukf.Q = Q_discrete_white_noise(2, dt=1., var=0.03)

zs, xs = [], []
for i in range(50):
    z = i + randn()*.5
    ukf.predict()
    ukf.update(z)
    xs.append(ukf.x[0])
    zs.append(z)

plt.plot(xs);
plt.plot(zs, marker='x', ls='');
```



新增的内容并不多：创建一个生成 sigma 点的对象，用函数替代 $\mathbf{F}$ 和 $\mathbf{H}$ 所表示的线性映射，其余设计思路与此前相同。希望这个例子能让你有信心继续学习下面的数学与算法，理解 UKF 的工作原理。

选择 sigma 点

本章开头，用 500,000 个随机样本估计了高斯输入通过非线性函数后的分布。虽然均值估计很准确，但每次更新都处理如此多的样本，会使滤波器非常慢。那么，最少可以使用多少个代表点？这些点又必须满足什么约束？我们不假定掌握非线性函数的特殊性质，因为希望构造具有普遍适用性的算法。

让我们先考虑最简单的情况，看看是否能从中得到一些启示。最简单的系统就是恒等函数：$\boldsymbol{f(x) = x}$。如果我们的算法不适用于恒等函数，那么该滤波器就无法实现收敛。换句话说，如果输入为 1（对于一维系统而言），那么输出也必须为 1。如果输出不是 1，比如是 1.1，那么在下一个时间步中，当我们将 1.1 输入到该滤波器时，得到的输出又会是一个不同的数值，比如 1.23。这样的滤波器是无法实现收敛的。

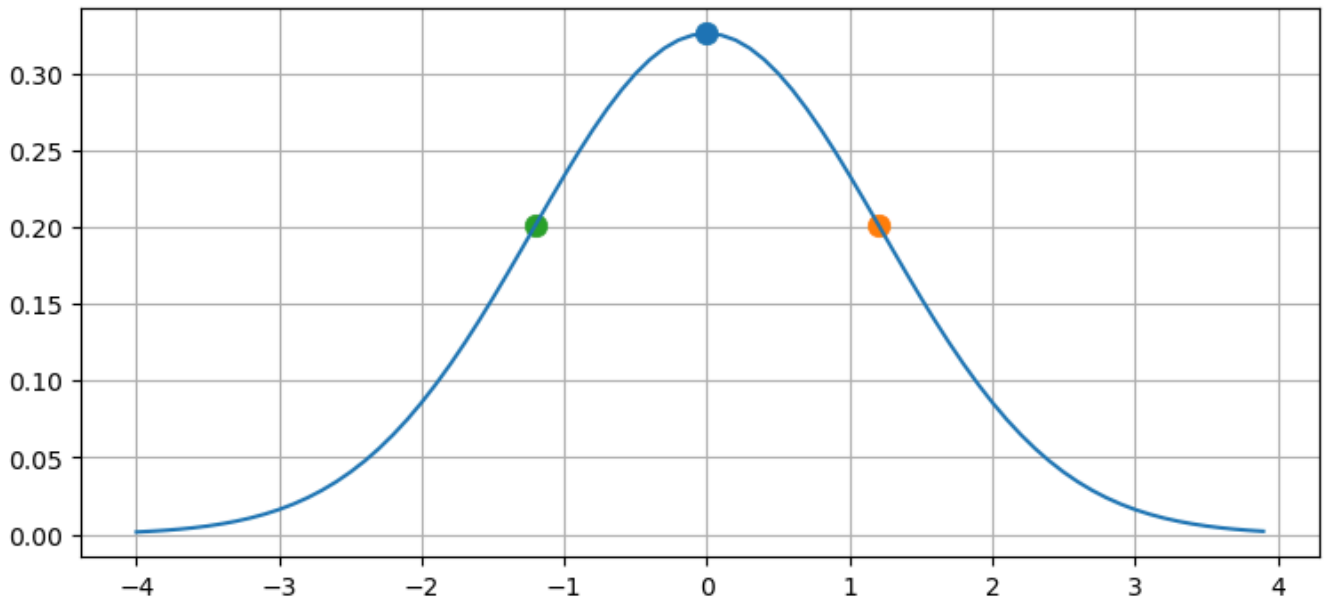
最简单的尝试是只在均值处计算函数值。线性卡尔曼滤波器可以这样传播均值：对于分布 $\mathcal{N}(\mu, \sigma^2)$，直接使用 μ。但它还会另行传播协方差；仅用一个点本身无法表示非零方差。因此，这种对均值的处理在线性情形下可行，对一般非线性情形则不够。

也许可以只用一个点，但对它作某种调整。不过，若将 $\mu + \Delta$ 代入恒等函数 $f(x) = x$，就无法保持原来的均值；而若不改变 μ，又回到了只在均值处计算的标准线性做法。因此，一个代表点不足以完成这里需要的非线性统计量近似。

比一个采样点更多、同时尽量少的选择是两个。两个关于均值对称的点可以表示均值和方差；不过，如果还要求点集中包含输入均值，并在恒等函数 $f(x) = x$ 下保持对称性，那么两个点就不够了：一个点位于均值时，另一个偏离均值的点会破坏对称性。因此，下面采用三个点。

再增加一个点，就得到 3 个点：一个位于均值，另两个分别位于均值两侧，如下图所示。

```
ukf_internal.show_3_sigma_points()
```



我们可以将这些点代入非线性函数 f(x)中，然后计算得到的平均值和方差。平均值可以是这三个点的算术平均值，但这种做法并不具有普遍性。例如，在处理高度非线性的问题时，我们可能希望给中心点的权重更大，而给边缘点的权重更小；或者反过来，给边缘点的权重更大。

一种更通用的方法是计算加权平均值 $\mu = \sum_i w_i f(\mathcal{X}_i)$，其中，那些被标记为 $\mathcal{X}$ 的点就是所谓的“sigma 点”。我们需要确保所有权重的总和等于 1。在满足这一条件的情况下，我们的任务就是选择合适的 $\mathcal{X}$ 及其对应的权重，使得计算出的结果能够反映经过转换后的sigma 点的平均值和方差。

既然均值可以加权，协方差也可以加权。均值权重 w^m 与协方差权重 w^c 可以不同。下面用上标区分这两类权重，把下标位置留给点的索引。于是可写为：

Constraints :

$$1 = \sum_i w_i^m$$
$$1 = \sum_i w_i^c$$
$$\mu = \sum_i w_i^m f(\mathcal{X}_i)$$
$$\Sigma = \sum_i w_i^c (f(\mathcal{X})_i - \mu)(f(\mathcal{X})_i - \mu)^\mathsf{T}$$

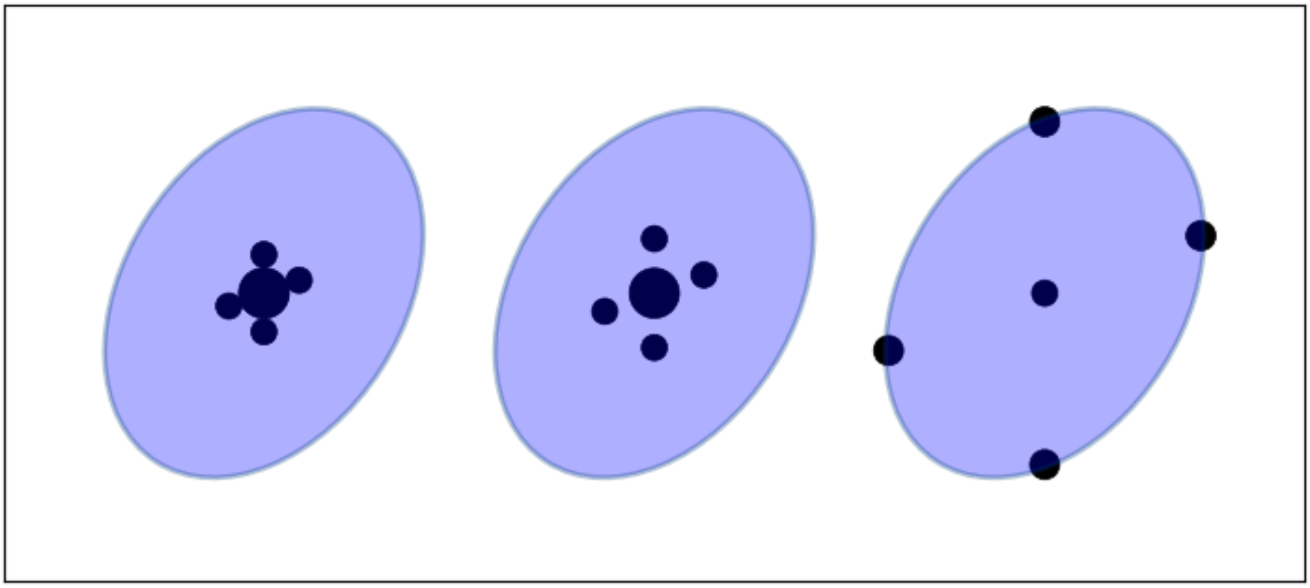
前两个方程给出两组权重之和为 1 的约束，第三个方程计算加权均值。第四个方程可能稍显陌生，不过可以回忆两个随机变量的协方差公式：

$$COV(x, y) = \frac{\sum(x - \bar{x})(y - \bar{y})}{n}$$

这些约束条件并不唯一确定一个解。例如，如果你减小 w_0^m 的值，就可以通过增大 w_1^m 和 w_2^m 的值来加以弥补。你可以为均值和协方差赋予不同的权重，也可以使用相同的权重。实际上，这些方程并不要求其中的任何一个点必须等于输入数据的均值，不过，从某种意义上说，让某个点成为均值会更好一些。

我们希望找到满足这些约束的算法，并尽量少用代表点。在继续之前，先明确这一思路。下图对同一个协方差椭圆给出了三种不同的 sigma 点选取方式；点的大小与相应权重成正比。

```
ukf_internal.show_sigma_selections()
```



这些点并不位于椭圆的长轴和短轴上；各项约束条件也并不要求我让点们位于这些轴上。虽然我把这些点标示为等间距分布的，但约束条件其实并不要求如此。

sigma 点的排列和权重设置会影响到我们对概率分布的采样方式。那些彼此靠近的点能够捕捉到局部效应，因此更适合用于处理高度非线性的问题。而那些相距较远、或位于椭圆轴线之外的点，则能捕捉到非局部效应以及非高斯分布特性。不过，通过调整每个点的权重，我们可以缓解这一问题。如果这些点虽然远离平均

值，但其权重很小，那么我们仍然可以获取到有关该分布的一些信息，同时避免因问题的非线性而导致估计结果出现偏差。

sigma 点的选择方式有无穷多种，上述约束只是其中一种设计方式。例如，并不是所有算法都要求均值权重与协方差权重分别都加起来等于 1。本书采用的缩放 sigma 点算法仍要求均值权重之和为 1，但协方差权重之和一般不等于 1。

无迹变换

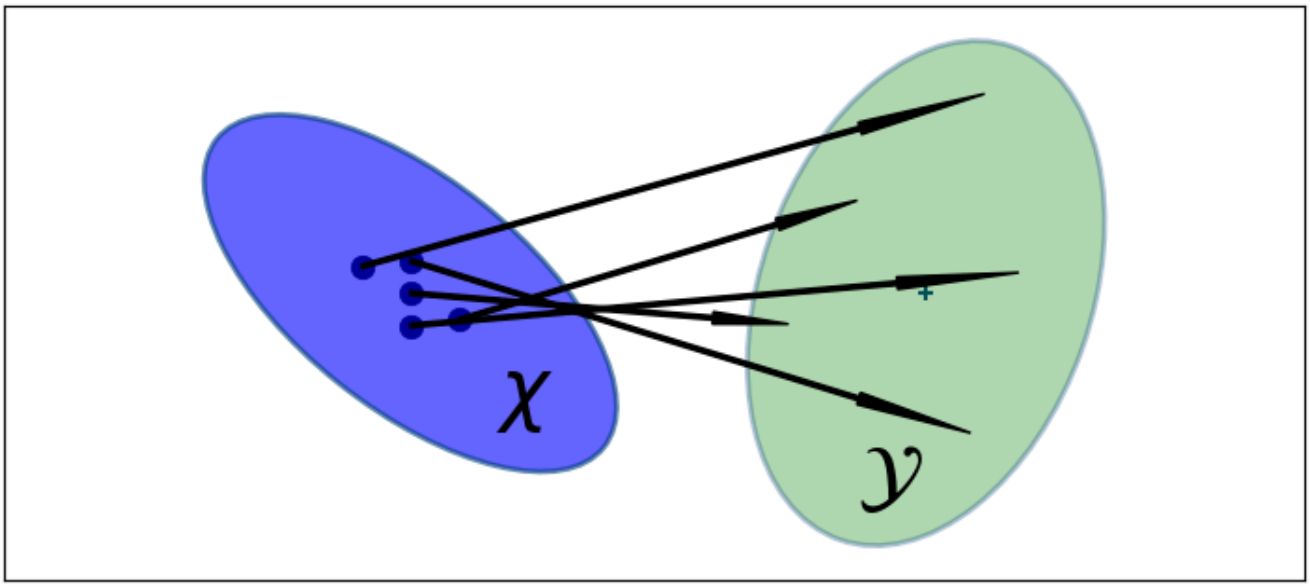
暂且假设，存在一种用于选择sigma 点与权重的算法。那么，这些sigma 点是如何被用来实现滤波功能的呢?

无迹变换是算法的核心，却十分简单：将 sigma 点 $\boldsymbol{\chi}$ 逐个通过非线性函数，得到变换后的点集。

$$\mathcal{Y} = f(\chi)$$

随后计算变换后点集的均值与协方差，用作新的统计量估计。下图展示无迹变换的过程：右侧绿色椭圆表示由变换后的 sigma 点计算出的均值与协方差。

```
ukf_internal.show_sigma_transform(with_text=True)
```



sigma 点的均值和协方差计算如下：

$$\mu = \sum_{i=0}^{2n} w_i^m \mathcal{Y}_i$$

$$\Sigma = \sum_{i=0}^{2n} w_i^c (\mathcal{Y}_i - \mu)(\mathcal{Y}_i - \mu)^\mathsf{T}$$

这些方程应该很熟悉——它们就是我们上面所推导出的约束方程。

无迹变换先将表示输入分布的一组 sigma 点通过非线性函数，再计算变换后整组点的加权均值与协方差，用一个高斯分布近似输出分布。注意，这不是为每一个点分别生成一个高斯分布。有了这种高斯近似，就可以继续使用前面建立的数学工具，构造非线性卡尔曼滤波器。

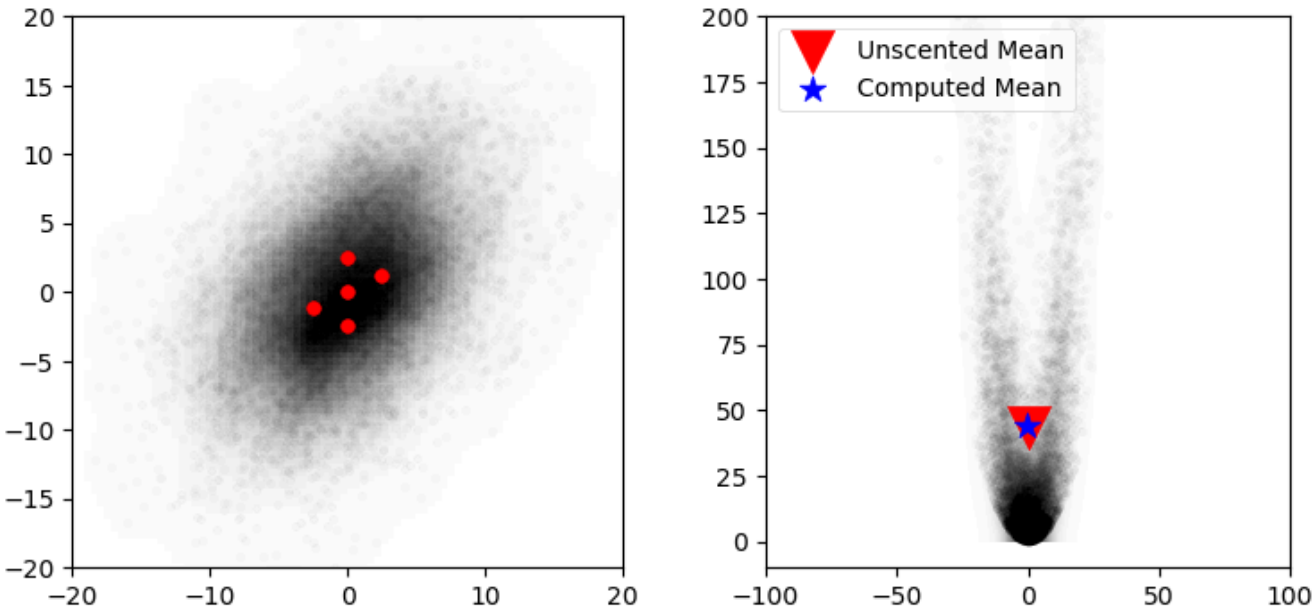
“unscented”这个名称可能令人困惑。它字面意为“无香味的”，据作者介绍，名称源于发明者的玩笑：自己的算法并不“难闻”。这个名字后来沿用下来，本身没有数学含义。本书采用通行译名“无迹”。

无迹变换的精度

前面曾通过将大量样本送入非线性函数，估计输出分布的均值。现在只用 5 个 sigma 点通过同一个函数，再用无迹变换计算均值。这里使用 FilterPy 的 MerweScaledSigmaPoints() 生成 sigma 点，使用 unscented_transform 计算变换后的统计量；后面会详细介绍。本章最初的例子使用 JulierSigmaPoints，两者都是选取 sigma 点的算法，但选点方式不同。

```
from filterpy.kalman import unscented_transform, MerweScaledSigmaPoints
import scipy.stats as stats


#initial mean and covariance
mean = (0., 0.)
p = np.array([[32., 15], [15., 40.]])

# create sigma points and weights
points = MerweScaledSigmaPoints(n=2, alpha=.3, beta=2., kappa=.1)
sigmas = points.sigma_points(mean, p)

### pass through nonlinear function
sigmas_f = np.empty((5, 2))
for i in range(5):
    sigmas_f[i] = f_nonlinear_xy(sigmas[i, 0], sigmas[i ,1])

### use unscented transform to get new mean and covariance
ukf_mean, ukf_cov = unscented_transform(sigmas_f, points.Wm, points.Wc)

#generate random points
np.random.seed(100)
xs, ys = multivariate_normal(mean=mean, cov=p, size=5000).T

plot_monte_carlo_mean(xs, ys, f_nonlinear_xy, ukf_mean, 'Unscented Mean')
ax = plt.gcf().axes[0]
ax.scatter(sigmas[:,0], sigmas[:,1], c='r', s=30);
```

```
Difference in mean x=-0.097, y=0.549
```



我觉得这个结果相当了不起。仅使用 5 个点，我们就能够以极高的精度计算出平均值。x 方向的误差仅为-0.097，y 方向的误差为 0.549。相比之下，线性化方法（即 EKF 所使用的方法，我们将在下一章中详细介绍）在 y 方向的误差则超过了 43。如果你仔细查看生成这些sigma 点的代码，就会发现：该代码并不了解非线性函数的细节，它只知道初始分布的均值和协方差。即使我们面对的是完全不同的非线性函数，生成的 5 个 sigma 点也是一样的。

我承认，我选择了一个非线性函数，这使得无迹变换的效果远远优于 EKF。不过，现实世界中到处都是非线性现象，而 UKF 能够很好地应对这些情况。我并没有刻意去寻找某个能让无迹变换效果良好的函数。在下一章中，你会看到，面对强烈的非线性现象时，传统技术会遇到很大困难。这张图表正好说明了为什么我建议尽可能使用 UKF 或类似的现代技术。

无迹卡尔曼滤波器

现在，我们可以来介绍 UKF 算法了。

预测步骤

UKF 的预测步骤利用过程模型 $\boldsymbol{f}()$ 计算先验状态分布。$\boldsymbol{f}()$ 可以是非线性函数，因此先按选点算法生成 sigma 点 $\boldsymbol{\mathcal{X}}$ 及相应的均值权重和协方差权重 $\boldsymbol{W^m}, \boldsymbol{W^c}$:

$$\begin{aligned}\chi &= \textbf{sigma-function}(\mathbf{x}, \mathbf{P}) \\ \boldsymbol{W^m}, \boldsymbol{W^c} &= \textbf{weight-function}(\mathbf{n}, \textbf{parameters})\end{aligned}$$

我们将每个sigma 点都通过 $\boldsymbol{f(x, \Delta t)}$ 进行处理。这样，根据该过程模型，这些sigma 点会被“向前推进”到未来的某个时间点，从而形成新的先验分布。这套新的sigma 点被我们称为 $\boldsymbol{\mathcal{Y}}$。

$$\mathcal{Y} = f(\chi, \Delta t)$$

我们利用无迹变换来处理经过变换后的sigma 点，从而计算出先验分布的均值和协方差。

$$\bar{\mathbf{x}}, \bar{\mathbf{P}} = UT(\mathcal{Y}, w_m, w_c, \mathbf{Q})$$

这些就是无迹变换所对应的方程式：

$$\bar{\mathbf{x}} = \sum_{i=0}^{2n} w_i^m \mathcal{Y}_i$$

$$\bar{\mathbf{P}} = \sum_{i=0}^{2n} w_i^c (\mathcal{Y}_i - \bar{\mathbf{x}})(\mathcal{Y}_i - \bar{\mathbf{x}})^\mathsf{T} + \mathbf{Q}$$

该表格对比了线性卡尔曼滤波器与无迹卡尔曼滤波器的算法公式。为便于阅读，我省略了下标 $\boldsymbol{i}$。

Kalman	Unscented
	$\mathcal{Y} = f(\chi)$
$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$	$\bar{\mathbf{x}} = \sum w^m \mathcal{Y}$
$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$	$\bar{\mathbf{P}} = \sum w^c (\mathcal{Y} - \bar{\mathbf{x}})(\mathcal{Y} - \bar{\mathbf{x}})^\mathsf{T} + \mathbf{Q}$

更新步骤

卡尔曼滤波器在测量空间中完成更新。因此，需要利用自定义的测量函数 $\boldsymbol{h(x)}$，将先验状态的 sigma 点映射为预测测量。

$$\mathcal{Z} = h(\mathcal{Y})$$

用无迹变换计算这些点的均值与协方差。下标 $\boldsymbol{z}$ 表示它们对应于映射到测量空间的 sigma 点。

$$\mu_z, \mathbf{P}_z = UT(\mathcal{Z}, w_m, w_c, \mathbf{R})$$

$$\mu_z = \sum_{i=0}^{2n} w_i^m \mathcal{Z}_i$$

$$\mathbf{P}_z = \sum_{i=0}^{2n} w_i^c (\mathcal{Z}_i - \mu_z)(\mathcal{Z}_i - \mu_z)^\mathsf{T} + \mathbf{R}$$

接下来，我们计算残差和卡尔曼增益。测量值 $\mathbf{z}$ 的残差很容易计算：

$$\mathbf{y} = \mathbf{z} - \mu_z$$

为了计算卡尔曼增益，我们首先需要计算状态量与测量值之间的互协方差。其定义如下：

$$\mathbf{P}_{xz} = \sum_{i=0}^{2n} w_i^c (\mathcal{Y}_i - \bar{\mathbf{x}})(\mathcal{Z}_i - \mu_z)^\mathsf{T}$$

然后，卡尔曼增益的定义如下：

$$\mathbf{K} = \mathbf{P}_{xz}\mathbf{P}_z^{-1}$$

如果你把“逆矩阵”视为某种矩阵的倒数，就会发现，卡尔曼增益其实是一个简单的比率，其计算方式如下：

$$\mathbf{K} \approx \frac{\mathbf{P}_{xz}}{\mathbf{P}_z} \approx \frac{\textbf{belief in state}}{\textbf{belief in measurement}}$$

最后，我们利用残差和卡尔曼增益来计算新的状态估计值：

$$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$$

新的协方差则按照以下方式计算：

$$\mathbf{P} = \bar{\mathbf{P}} - \mathbf{K}\mathbf{P}_z\mathbf{K}^\mathsf{T}$$

这一步中包含了一些需要直接接受的方程式，不过你应该能够理解这些方程式与线性卡尔曼滤波器方程式之间的联系。虽然线性代数的处理方式与线性卡尔曼滤波器略有不同，但所使用的算法其实还是我们在本书中一直使用的那种贝叶斯算法。

该表对比了线性 KF 算法和 UKF 算法的方程式。

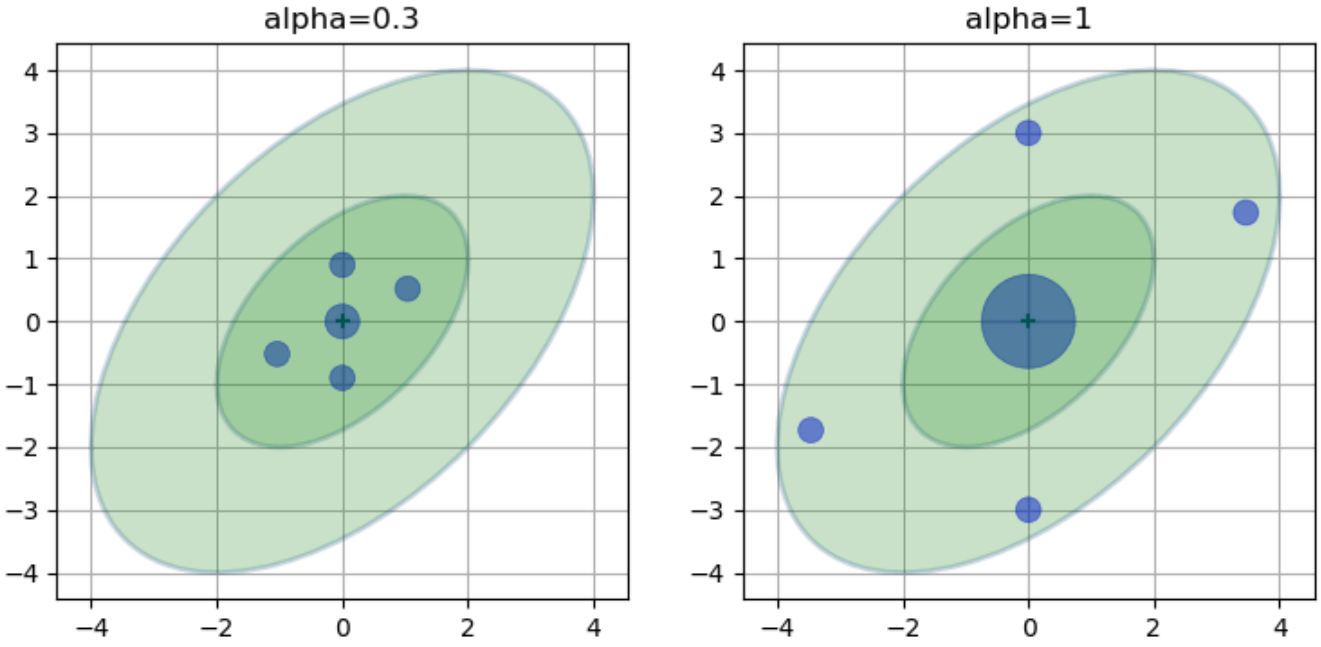
Kalman Filter	Unscented Kalman Filter
	$\mathcal{Y} = f(\chi)$
$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$	$\bar{\mathbf{x}} = \sum w^m \mathcal{Y}$
$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$	$\bar{\mathbf{P}} = \sum w^c (\mathcal{Y} - \bar{\mathbf{x}})(\mathcal{Y} - \bar{\mathbf{x}})^\mathsf{T} + \mathbf{Q}$
	$\mathcal{Z} = h(\mathcal{Y})$
	$\mu_z = \sum w^m \mathcal{Z}$
$\mathbf{y} = \mathbf{z} - \mathbf{H}\mathbf{x}$	$\mathbf{y} = \mathbf{z} - \mu_z$
$\mathbf{S} = \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R}$	$\mathbf{P}_z = \sum w^c (\mathcal{Z} - \mu_z)(\mathcal{Z} - \mu_z)^\mathsf{T} + \mathbf{R}$
$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}$	$\mathbf{K} = \left[\sum w^c (\mathcal{Y} - \bar{\mathbf{x}})(\mathcal{Z} - \mu_z)^\mathsf{T}\right]\mathbf{P}_z^{-1}$
$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$	$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$
$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}$	$\mathbf{P} = \bar{\mathbf{P}} - \mathbf{K}\mathbf{P}_z\mathbf{K}^\mathsf{T}$

Van der Merwe 缩放 sigma 点算法

用于选择sigma 点的算法有很多种。自 2005 年左右以来，无论是学术界还是工业界，都普遍采用 Rudolph Van der Merwe 在 2004 年的博士论文中提出的算法[1]。该算法在处理各种问题时都能表现出色，而且在性能与精度之间取得了良好的平衡。实际上，该算法是对 Simon J. Julier 提出的“缩放无迹变换”算法的略微改进而已[2]。

这一形式使用三个参数控制 sigma 点的位置与权重：α、β 和 κ。在研究公式之前，先看一个例子：将 sigma 点画在一倍与两倍标准差的协方差轮廓上，并根据均值权重调整点的大小。

```
ukf_internal.plot_sigma_points()
```



可以看到，外围 sigma 点位于一倍与两倍标准差轮廓之间。α 越大，点离均值越远。同时，较大的 α 比较小的 α 赋予中心点更高的均值权重，而赋予其余点更低的权重。这符合这里的直观想法：点离均值更远时，应降低它的权重。虽然还未介绍这些位置和权重的计算方法，但这样的选择看起来是合理的。

sigma 点计算

第一个sigma 点就是输入数据的平均值。这就是上图中椭圆中心所对应的sigma 点。我们将其记为 χ_0 。

$$\mathcal{X}_0 = \mu$$

为便于表示，我们定义 $\lambda = \alpha^2(n + \kappa) - n$，其中 n 表示 $\mathbf{x}$ 的维度。其余的sigma 点则通过相应公式计算得出。

下标约定如下：

$$\chi_i = \begin{cases} \mu + \left[\sqrt{(n+\lambda)\Sigma}\right]_i & i = 1..n \\ \mu - \left[\sqrt{(n+\lambda)\Sigma}\right]_{i-n} & i = (n+1)..2n \end{cases}$$

中的 i 选取第 i^{th} 个行向量，即第 i 行。这里采用平方根矩阵的行向量，因此计算矩阵平方根时，必须使用与这一约定相配套的因子。

换言之，先用一个系数缩放协方差矩阵，再求其矩阵平方根；将平方根矩阵的相应行向量分别加到均值上、再从均值中减去，就得到关于均值对称的 sigma 点。这里保证的是点集的对称性。稍后会讨论矩阵平方根的计算。

权重计算

这一算法分别使用均值权重与协方差权重。中心点 $\mathcal{X}_0$ 的均值权重为：

$$W_0^m = \frac{\lambda}{n+\lambda}$$

中心点 $\mathcal{X}_0$ 对协方差计算的权重为：

$$W_0^c = \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta$$

其余 sigma 点 $\chi_1 \ldots \chi_{2n}$ 的均值权重与协方差权重相同，均为：

$$W_i^m = W_i^c = \frac{1}{2(n+\lambda)} \quad i = 1..2n$$

这种选点方式为何合理并非一目了然，也不能证明它对所有非线性问题都是最优的。但可以看出，各 sigma 点相对于均值的偏移由协方差矩阵的平方根决定；方差的平方根是标准差，因此可将其理解为在 $\pm 1\sigma$ 的尺度上进一步缩放。权重公式的分母中含有 n，不过维数改变时点的间距和权重如何变化，还取决于其他参数，尤其是 κ 的选择。

注意：均值权重之和为 1，但协方差权重之和通常不为 1。关于这一点，我经常收到疑问。某些权重为负，或协方差权重之和大于 1，都可能是算法的正常结果。后面会进一步讨论。

参数的合理选择

对于高斯分布，$\beta = 2$ 是常用选择，也可取 $\kappa = 3 - n$，其中 n 是状态 $\mathbf{x}$ 的维数。选择 κ 时，应保证 n+λ 为正。参数通常取在 $0 < \alpha \leq 1$ 范围内；α 越大，sigma 点距均值越远，因此可按问题需要调整 α。

使用 UKF 算法

让我们来解几道题目，这样你就能明白使用 UKF 其实有多简单了。我们先从一个线性问题开始，你肯定已经知道如何用线性卡尔曼滤波器来解这个问题。虽然 UKF 最初是为了解决非线性问题而设计的，但对于线性问题来说，它得到的最优解与线性卡尔曼滤波器是一样的。我们将编写一个滤波器，利用恒定速度模型来追踪二维空间中的物体。这样，我们就能清楚地了解 UKF 与线性卡尔曼滤波器之间的相同之处（其实大部分都是相同的！），以及不同之处。

设计卡尔曼滤波器时，需要指定 $\mathbf{x}$、$\mathbf{F}$、$\mathbf{H}$、$\mathbf{R}$ 和 $\mathbf{Q}$ 这些矩阵。我们已经做过很多次这样的设计了，所以这里就不再详细解释了。由于我们希望模型中的物体保持恒定速度运动，因此我们将 $\mathbf{x}$ 定义为……

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} \end{bmatrix}^\mathsf{T}$$

按照这种状态变量的排列方式，状态转移矩阵为：

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

该矩阵实现以下牛顿运动方程：

$$x_k = x_{k-1} + \dot{x}_{k-1}\Delta t$$
$$y_k = y_{k-1} + \dot{y}_{k-1}\Delta t$$

传感器只提供位置，不提供速度，因此测量函数为：

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

传感器以米为单位报告位置，x 与 y 两个方向的测量噪声标准差均为 $\sigma = 0.3$ 米。因此测量噪声协方差矩阵为：

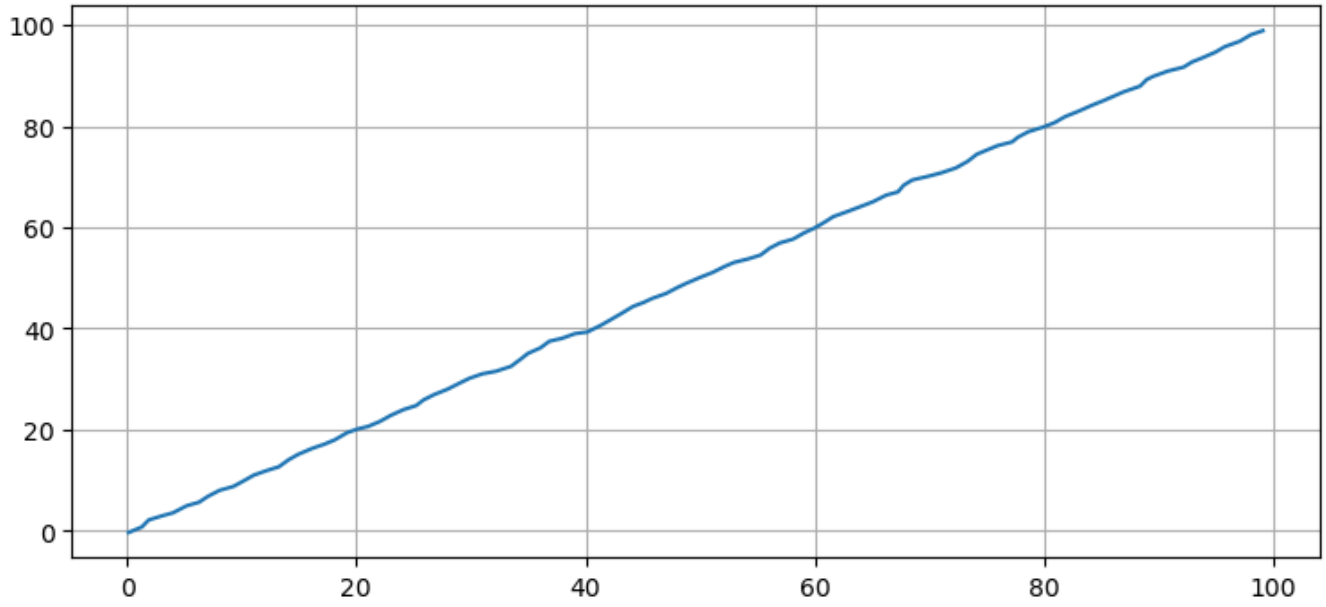
$$\mathbf{R} = \begin{bmatrix} 0.3^2 & 0 \\ 0 & 0.3^2 \end{bmatrix}$$

最后，假设过程噪声采用离散白噪声模型，即加速度扰动在每个采样时间段内恒定，不同时间段之间互不相关。可以使用 FilterPy 的 Q_discrete_white_noise() 构造该矩阵。为便于复习，矩阵写作：

$$\mathbf{Q} = \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix} \sigma^2$$

我对这个滤波器的实现方式如下：

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise
from numpy.random import randn

std_x, std_y = .3, .3
dt = 1.0

np.random.seed(1234)
kf = KalmanFilter(4, 2)
kf.x = np.array([0., 0., 0., 0.])
kf.R = np.diag([std_x**2, std_y**2])
kf.F = np.array([[1, dt, 0, 0],
                 [0, 1, 0, 0],
                 [0, 0, 1, dt],
                 [0, 0, 0, 1]])
kf.H = np.array([[1, 0, 0, 0],
                 [0, 0, 1, 0]])

kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=1, var=0.02)
kf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=1, var=0.02)

zs = [np.array([i + randn()*std_x,
                i + randn()*std_y]) for i in range(100)]
xs, _, _, _ = kf.batch_filter(zs)
plt.plot(xs[:, 0], xs[:, 2]);
```



这对你来说应该不算什么意外吧。现在，让我们来实现 UKF 算法。需要再次强调的是，这样做纯粹是出于教学目的而已；对于线性问题来说，使用 UKF 算法并没有任何好处。 `FilterPy` 通过 `UnscentedKalmanFilter` 这个类来实现 UKF 算法。

首先要实现 `f(x, dt)` 和 `h(x)` 这两个函数。 `f(x, dt)` 负责实现状态转移功能，而 `h(x)` 则负责实现测量功能。这些功能分别对应于线性滤波器中的 $\mathbf{F}$ 和 $\mathbf{H}$ 矩阵。

以下是这两个函数的合理实现方式。每个函数都应该返回一个包含结果的 1D NumPy 数组或列表。你可以给这些函数起一些更易理解的名称，而不是使用 `f` 和 `h` 这样的符号。

```
def f_cv(x, dt):
    """ state transition function for a
    constant velocity aircraft"""

    F = np.array([[1, dt, 0,  0],
                  [0,  1, 0,  0],
                  [0,  0, 1, dt],
                  [0,  0, 0,  1]])
    return F @ x

def h_cv(x):
    return x[[0, 2]]
```

接下来指定 sigma 点及权重的计算方法。前面介绍了 Van der Merwe 的版本，也可以采用其他算法。在 FilterPy 中，选点对象应提供下面的方法：

```
def sigma_points(self, x, P)
```

并且包含属性 `Wm` 和 `Wc` ，它们分别用于存储用于计算均值和协方差的权重值。

FilterPy 的 MerweScaledSigmaPoints 类提供了上述选点方法和权重属性。

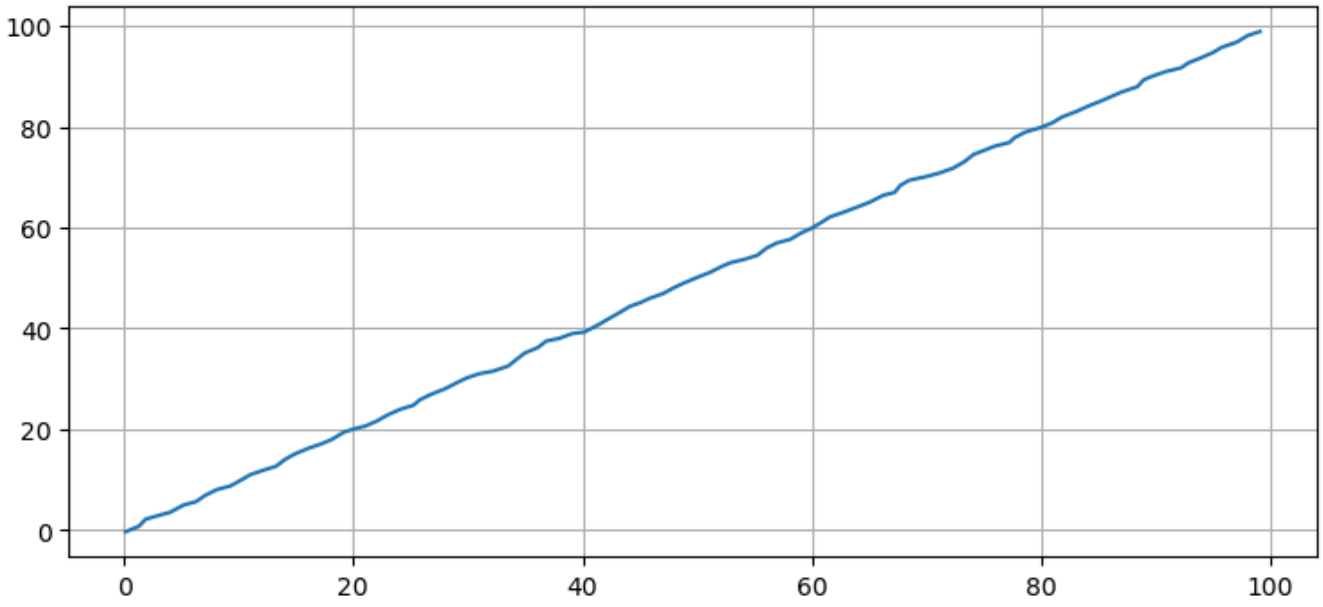
在创建 UKF 时，需要传入 $\boldsymbol{f}()$ 和 $\boldsymbol{h}()$ 函数以及 sigma 点对象，就像这个例子所示：

```
from filterpy.kalman import MerweScaledSigmaPoints
from filterpy.kalman import UnscentedKalmanFilter as UKF
points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=-1)
ukf = UKF(dim_x=4, dim_z=2, fx=f_cv, hx=h_cv, dt=dt, points=points)
```

其余的代码与线性卡尔曼滤波器的代码相同。我会使用相同的测量数据，然后计算两种算法所得结果之间差异的标准差。

```
from filterpy.kalman import UnscentedKalmanFilter as UKF

import numpy as np

sigmas = MerweScaledSigmaPoints(4, alpha=.1, beta=2., kappa=1.)
ukf = UKF(dim_x=4, dim_z=2, fx=f_cv,
          hx=h_cv, dt=dt, points=sigmas)
ukf.x = np.array([0., 0., 0., 0.])
ukf.R = np.diag([0.09, 0.09])
ukf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=1, var=0.02)
ukf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=1, var=0.02)

uxs = []
for z in zs:
    ukf.predict()
    ukf.update(z)
    uxs.append(ukf.x.copy())
uxs = np.array(uxs)

plt.plot(uxs[:, 0], uxs[:, 2])
print(f'UKF standard deviation {np.std(uxs - xs):.3f} meters')
```

```
UKF standard deviation 0.013 meters
```



由此得出的标准差为 0.013 米，这个数值相当小。

UKF 的实现与线性卡尔曼滤波并没有根本不同。线性版本用矩阵 $\mathbf{F}$ 和 $\mathbf{H}$ 表示状态转移与测量关系；UKF 则由用户提供非线性函数 f() 和 h()。预测和更新的具体计算会改变，但从设计者的角度看，问题建模及滤波器设计的整体思路很相似。

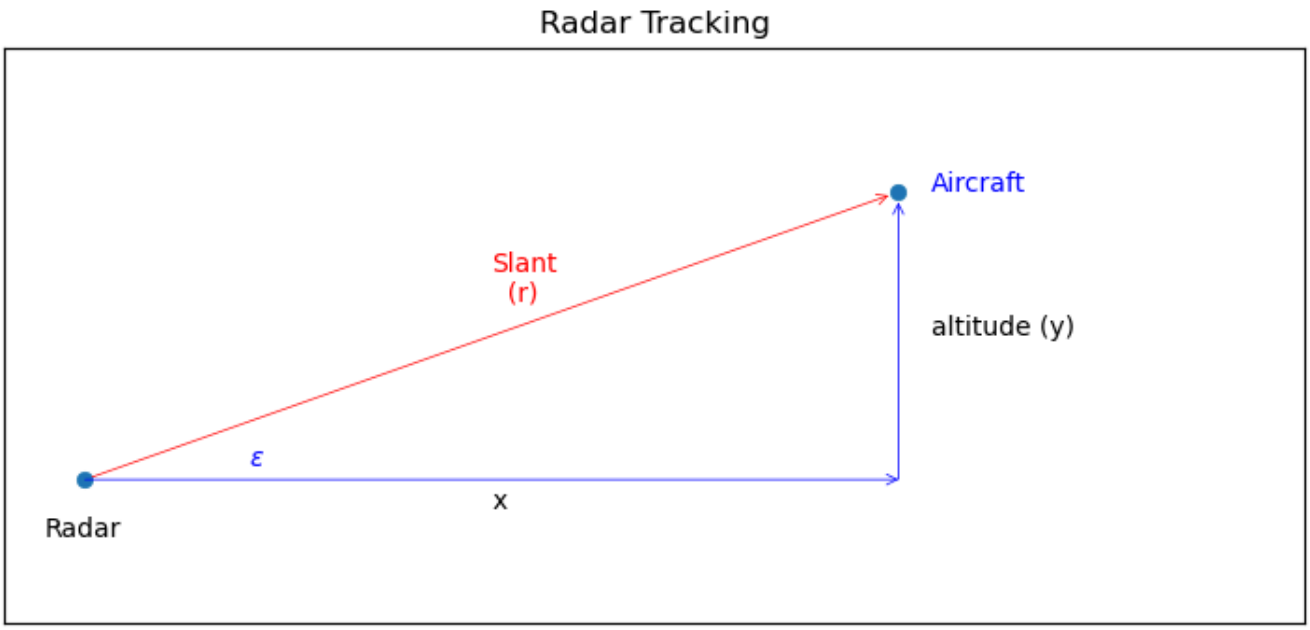
追踪飞机位置

下面处理第一个非线性问题：使用雷达跟踪飞机。为尽量接近前面的例子，先在二维平面中建模，状态包括一个水平方向的位置以及飞机高度。这里把两个方向的过程扰动视为独立，以简化建模；雷达测量仍会同时依赖它们。

雷达的工作原理是发射无线电波或微波。只要位于波束路径上的任何物体，都会将部分信号反射回雷达。通过测量反射信号返回雷达所需的时间，就可以计算出该物体到雷达的斜距。斜距指的是从雷达到该物体的直线距离。而方位角则是通过天线的定向增益来确定的。

斜距与仰角同飞机 (x, y) 位置的几何关系如下图所示：

```
import kf_book.ekf_internal as ekf_internal
ekf_internal.show_radar_chart()
```



仰角 ϵ 是从雷达所在的水平参考线向上量到目标视线的夹角。

我们假设飞机以恒定高度飞行。因此，我们需要一个包含三个变量的状态向量来描述飞机的状态。

$$\mathbf{x} = \begin{bmatrix}\textbf{distance}\\ \textbf{velocity}\\ \textbf{altitude}\end{bmatrix} = \begin{bmatrix}x\\ \dot{x}\\ y\end{bmatrix}$$

状态转移函数是线性的。

$$\bar{\mathbf{x}} = \begin{bmatrix}1 & \Delta t & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}x\\ \dot{x}\\ y\end{bmatrix}$$

其计算方式如下：

```
def f_radar(x, dt):
    """ state transition function for a constant velocity
    aircraft with state vector [x, velocity, altitude]'"""

    F = np.array([[1, dt, 0],
                  [0,  1, 0],
                  [0,  0, 1]], dtype=float)
    return F @ x
```

接下来设计测量函数。与线性卡尔曼滤波一样，它将先验状态映射为预测测量。本例需要把飞机状态中的位置信息转换为相对于雷达站的仰角与斜距；这两个测量量不直接依赖速度分量。

距离是通过勾股定理计算得出的：

$$\mathbf{range} = \sqrt{(x_{\mathbf{ac}} - x_{\mathbf{radar}})^2 + (y_{\mathbf{ac}} - y_{\mathbf{radar}})^2}$$

仰角 ϵ 实际上就是 y/x 的反正切值：

$$\epsilon = \tan^{-1} \frac{y_{\mathbf{ac}} - y_{\mathbf{radar}}}{x_{\mathbf{ac}} - x_{\mathbf{radar}}}$$

我们需要定义一个 Python 函数来计算这个值。我会利用函数对象可以附带属性这一特性，来存储雷达的位置。虽然本例并不一定需要这样做——也可以把数值直接写入代码，或者使用全局变量——但这种写法能使函数更加灵活。

```
def h_radar(x):
    dx = x[0] - h_radar.radar_pos[0]
    dy = x[2] - h_radar.radar_pos[1]
    slant_range = math.sqrt(dx**2 + dy**2)
    elevation_angle = math.atan2(dy, dx)
    return [slant_range, elevation_angle]

h_radar.radar_pos = (0, 0)
```

还有一个尚未处理的问题：角度具有模 360° 或模 2π 的周期性。残差是实际测量减去先验状态对应的预测测量；359° 与 1° 的最短角差为 2°，直接相减却得到 358°。无迹变换还需要计算加权均值，因此 UKF 尤其需要正确处理这种角度环绕。暂时将传感器与目标放在避开角度跳变的位置，后面再介绍处理方法。

我们需要对雷达和飞机进行模拟。对此你应该已经很熟悉了，所以我直接把代码提供给你，无需多言。

```
from numpy.linalg import norm
from math import atan2

class RadarStation:

    def __init__(self, pos, range_std, elev_angle_std):
        self.pos = np.asarray(pos)
        self.range_std = range_std
        self.elev_angle_std = elev_angle_std


    def reading_of(self, ac_pos):
        """ Returns (range, elevation angle) to aircraft.
        Elevation angle is in radians.
        """

        diff = np.subtract(ac_pos, self.pos)
        rng = norm(diff)
```

```
        brg = atan2(diff[1], diff[0])
        return rng, brg


    def noisy_reading(self, ac_pos):
        """ Compute range and elevation angle to aircraft with
        simulated noise"""

        rng, brg = self.reading_of(ac_pos)
        rng += randn() * self.range_std
        brg += randn() * self.elev_angle_std
        return rng, brg

class ACSim:
    def __init__(self, pos, vel, vel_std):
        self.pos = np.asarray(pos, dtype=float)
        self.vel = np.asarray(vel, dtype=float)
        self.vel_std = vel_std

    def update(self, dt):
        """ Compute and returns next position. Incorporates
        random variation in velocity. """

        dx = self.vel*dt + (randn() * self.vel_std) * dt
        self.pos += dx
        return self.pos
```

军用级雷达的测距均方根误差可达 1 米，仰角测量的均方根误差可达 1 毫弧度[3]。这里采用较低的精度，假设测距误差为 5 米、仰角误差为 0.5 度，以给滤波器更具挑战性的数据。

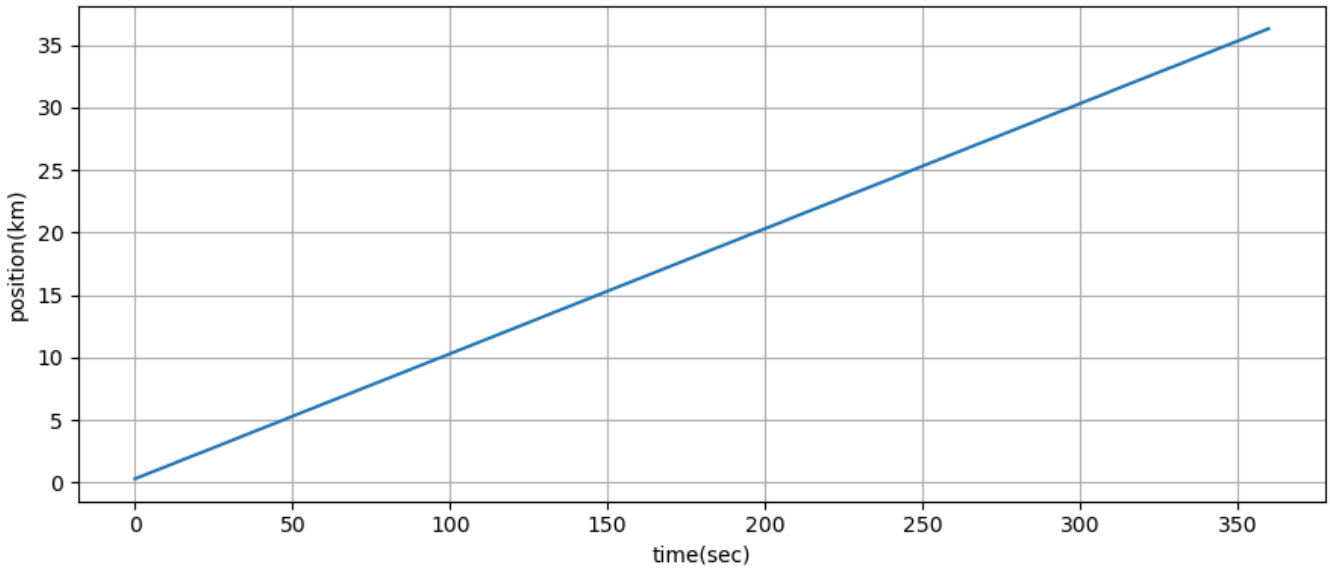
$\mathbf{Q}$ 的设计需要稍作讨论。状态为 $[x \quad \dot{x} \quad y]^{\mathrm{T}}$，前两个分量分别是水平航程与水平速度，因此可用 Q_discrete_white_noise 计算左上角的子块。第三个分量是高度，这里假设其过程噪声与 x 方向独立，因此 $\mathbf{Q}$ 采用分块形式：

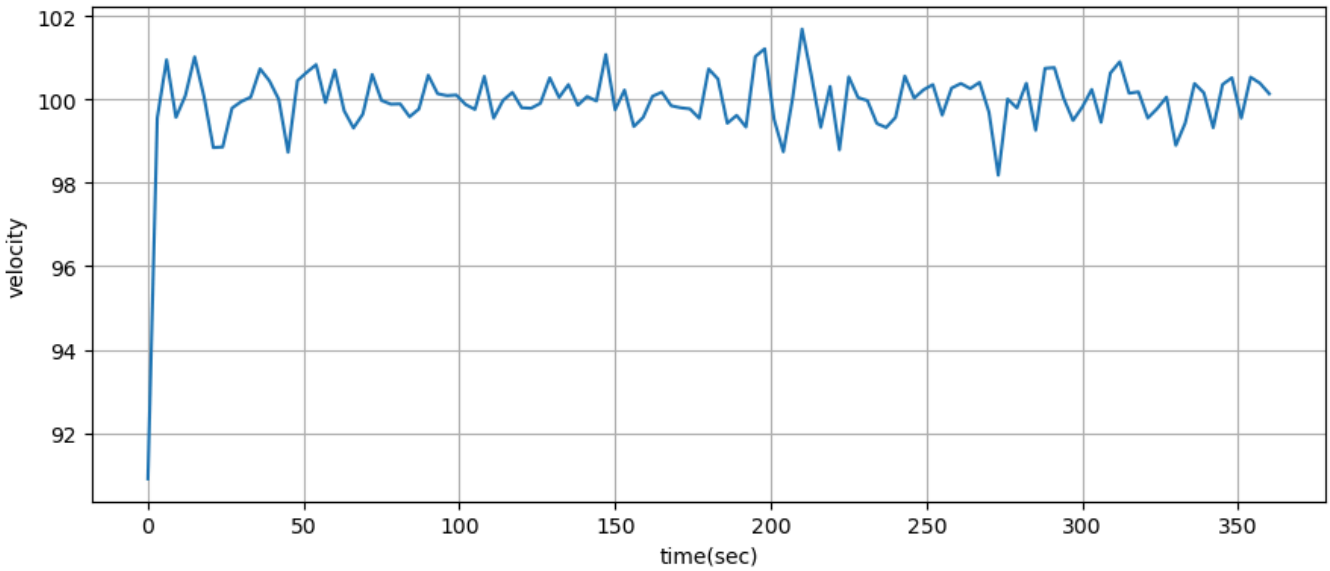
$$\mathbf{Q} = \begin{bmatrix} \mathbf{Q_x} & 0 \\ 0 & Q_y \end{bmatrix}$$

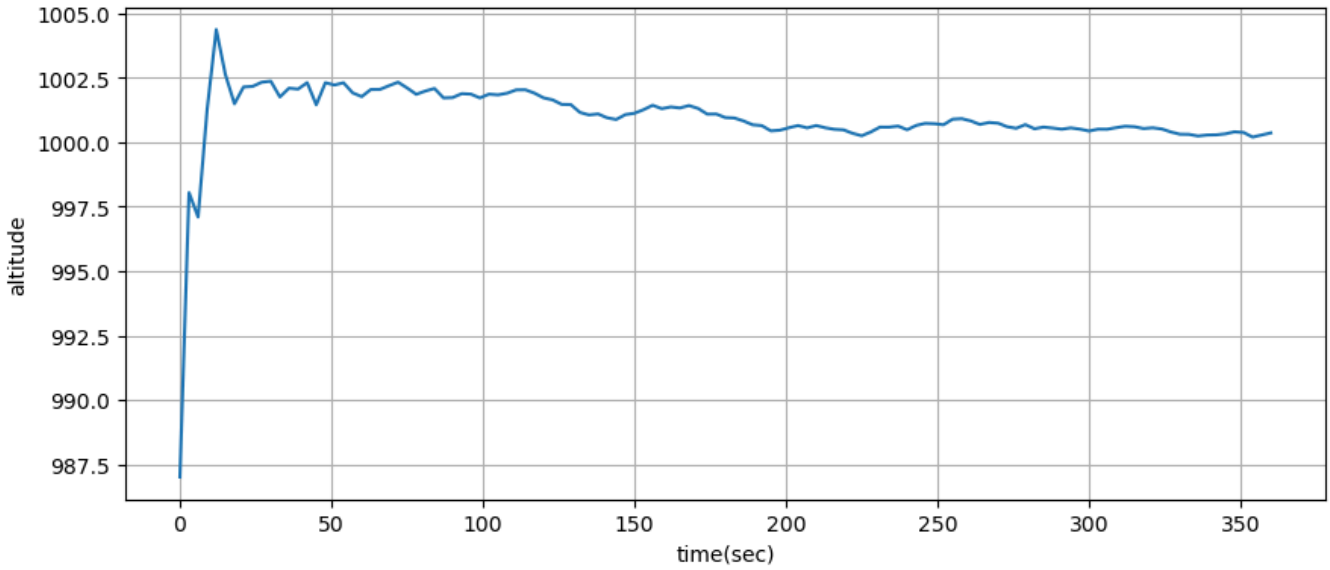
初始时，飞机位于雷达站正上方，以 100 m/s 的速度飞行。典型的测高雷达可能每 3 秒才更新一次，因此这里采用 3 秒的采样间隔。

```
import math
from kf_book.ukf_internal import plot_radar

dt = 3. # 12 seconds between readings
range_std = 5 # meters
elevation_angle_std = math.radians(0.5)
ac_pos = (0., 1000.)
ac_vel = (100., 0.)
radar_pos = (0., 0.)
h_radar.radar_pos = radar_pos

points = MerweScaledSigmaPoints(n=3, alpha=.1, beta=2., kappa=0.)
kf = UKF(3, 2, dt, fx=f_radar, hx=h_radar, points=points)

kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
kf.Q[2,2] = 0.1

kf.R = np.diag([range_std**2, elevation_angle_std**2])
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])

np.random.seed(200)
pos = (0, 0)
radar = RadarStation(pos, range_std, elevation_angle_std)
```

```
ac = ACSim(ac_pos, (100, 0), 0.02)

time = np.arange(0, 360 + dt, dt)
xs = []
for _ in time:
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    kf.predict()
    kf.update([r[0], r[1]])
    xs.append(kf.x)
plot_radar(xs, time)
```







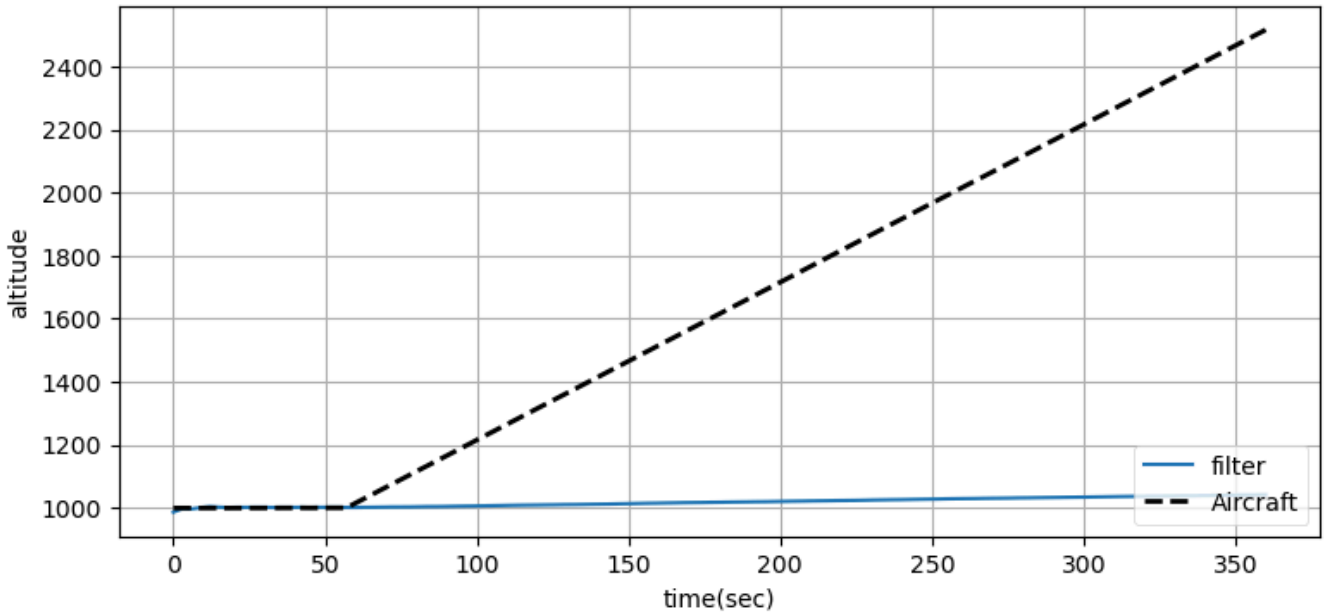
这或许不会让你觉得有什么了不起，但我却觉得相当了不起！在“扩展卡尔曼滤波”这一章中，我们将解决同样的问题，不过需要用到大量的数学公式。

对机动飞机的跟踪监测

之前的示例取得了不错的效果，但该示例假设飞机的飞行高度保持不变。以下是当飞机在 1 分钟后开始上升时的滤波结果。

```
from kf_book.ukf_internal import plot_altitude

# reset aircraft position
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])
ac = ACSim(ac_pos, (100, 0), 0.02)

np.random.seed(200)
time = np.arange(0, 360 + dt, dt)
xs, ys = [], []
for t in time:
    if t >= 60:
        ac.vel[1] = 300/60 # 300 meters/minute climb
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    ys.append(ac.pos[1])
    kf.predict()
    kf.update([r[0], r[1]])
    xs.append(kf.x)

plot_altitude(xs, time, ys)
print(f'Actual altitude: {ac.pos[1]:.1f}')
print(f'UKF altitude   : {xs[-1][2]:.1f}')
```

```
Actual altitude: 2515.6
UKF altitude   : 1042.1
```



该滤波器无法追踪高度的变化。我们在设计上需要做哪些调整呢?

我希望你的回答是“将爬升率添加到状态参数中”，也就是像这样:

$$\mathbf{x} = \begin{bmatrix}\textbf{distance}\\ \textbf{velocity}\\ \textbf{altitude}\\ \textbf{climb rate}\end{bmatrix} = \begin{bmatrix}x\\ \dot{x}\\ y\\ \dot{y}\end{bmatrix}$$

状态中增加爬升率后，需要相应修改状态转移函数。其形式仍然是线性的，预测状态等于下面的状态转移矩阵与当前状态向量的乘积:

$$\bar{\mathbf{x}} = \begin{bmatrix}1 & \Delta t & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & \Delta t\\ 0 & 0 & 0 & 1\end{bmatrix}\begin{bmatrix}x\\ \dot{x}\\ y\\ \dot{y}\end{bmatrix}$$

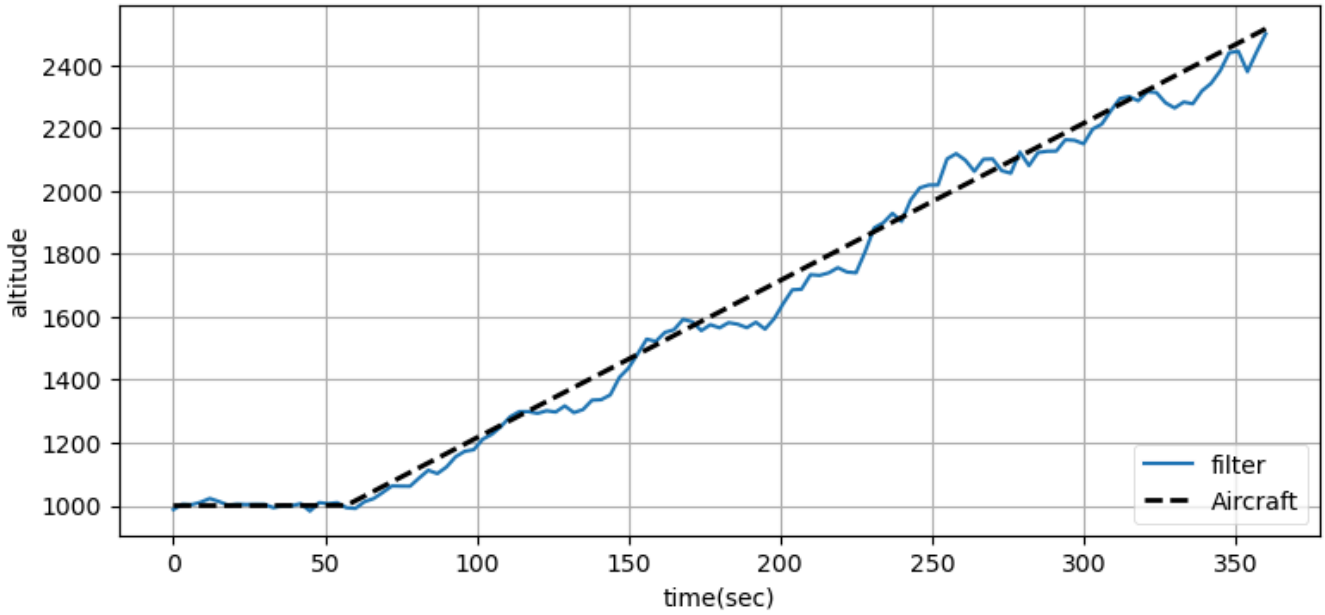
测量函数保持不变，但需要调整 $\mathbf{Q}$，以适应状态 $\mathbf{x}$ 的维数变化。

```
def f_cv_radar(x, dt):
    """ state transition function for a constant velocity
    aircraft"""
    F = np.array([[1, dt, 0, 0],
                  [0,  1, 0, 0],
                  [0,  0, 1, dt],
                  [0,  0, 0, 1]], dtype=float)
    return F @ x

def cv_UKF(fx, hx, R_std):
    points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=-1.)
    kf = UKF(4, len(R_std), dt, fx=fx, hx=hx, points=points)

    kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
    kf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=dt, var=0.1)
    kf.R = np.diag(R_std)
    kf.R = kf.R @ kf.R  # square to get variance
    kf.x = np.array([0., 90., 1100., 0.])
```

```
    kf.P = np.diag([300**2, 3**2, 150**2, 3**2])
    return kf
```

```
np.random.seed(200)
ac = ACSim(ac_pos, (100, 0), 0.02)

kf_cv = cv_UKF(f_cv_radar, h_radar, R_std=[range_std, elevation_angle_std])
time = np.arange(0, 360 + dt, dt)
xs, ys = [], []
for t in time:
    if t >= 60:
        ac.vel[1] = 300/60 # 300 meters/minute climb
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    ys.append(ac.pos[1])
    kf_cv.predict()
    kf_cv.update([r[0], r[1]])
    xs.append(kf_cv.x)

plot_altitude(xs, time, ys)
print(f'Actual altitude: {ac.pos[1]:.1f}')
print(f'UKF altitude   : {xs[-1][2]:.1f}')
```

```
Actual altitude: 2515.6
UKF altitude   : 2500.1
```



高度估计中的噪声虽然明显增大，但现在已经能准确跟踪高度变化。

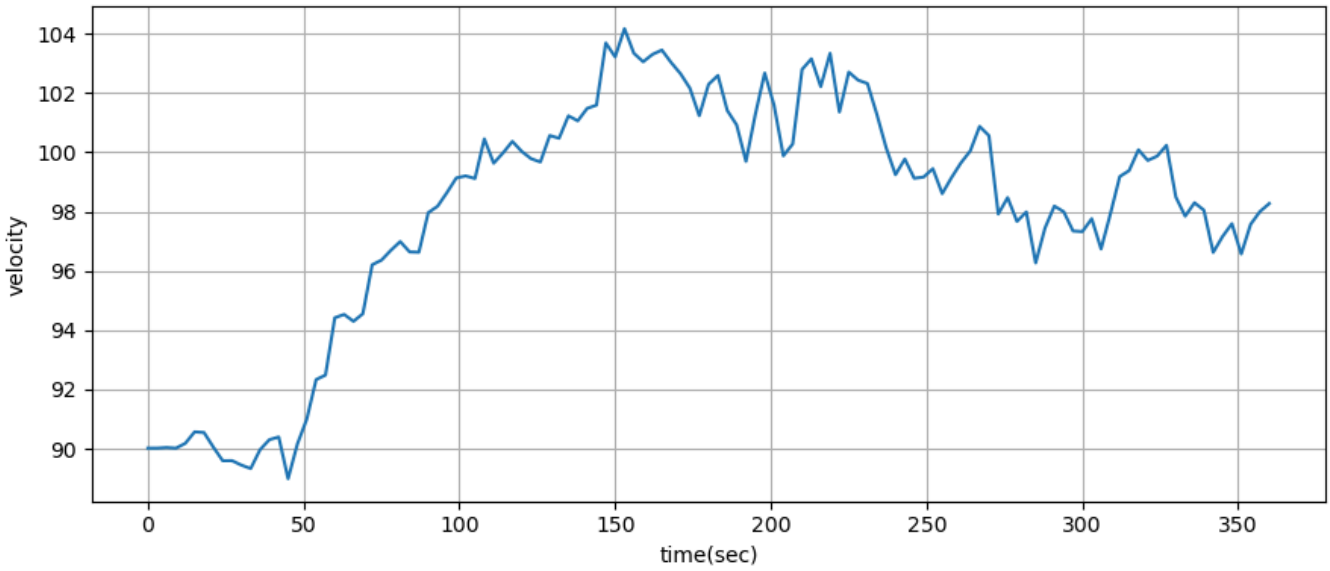
传感器融合

现在，让我们来看一个传感器融合的例子。我们有一种多普勒系统，它能够以 2 米/秒的均方根误差来估算物体的速度。我之所以说“某种多普勒系统”，是因为和雷达的情况类似，我并不打算在这里教你如何为多普勒系统设计出精确的滤波器。在实际应用中，必须考虑到信噪比、大气效应、系统的几何结构等因素。

前几个例子中，雷达精度足以使速度估计误差约为 1 m/s。下面降低测距精度，以展示传感器融合的作用：将距离测量噪声标准差设为 $\sigma = 500$ 米，再计算速度估计的标准差。滤波器最初尚在收敛，偏差会较大，因此统计时略去最初的若干测量。

不加入多普勒速度测量时，标准差为：

```
range_std = 500.
elevation_angle_std = math.degrees(0.5)
np.random.seed(200)
pos = (0, 0)
radar = RadarStation(pos, range_std, elevation_angle_std)
ac = ACSim(ac_pos, (100, 0), 0.02)

kf_sf = cv_UKF(f_cv_radar, h_radar, R_std=[range_std, elevation_angle_std])
time = np.arange(0, 360 + dt, dt)
xs = []
for _ in time:
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    kf_sf.predict()
    kf_sf.update([r[0], r[1]])
    xs.append(kf_sf.x)

xs = np.asarray(xs)
plot_radar(xs, time, plot_x=False, plot_vel=True, plot_alt=False)
print(f'Velocity std {np.std(xs[10:, 1]):.1f} m/s')
```




```
Velocity std 3.4 m/s
```

为加入多普勒测量，需要把 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 方向的速度也纳入测量向量。ACSim 类将速度存储在成员 vel 中。调用 update 时，传入一个包含斜距、仰角以及 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 方向速度的列表即可：

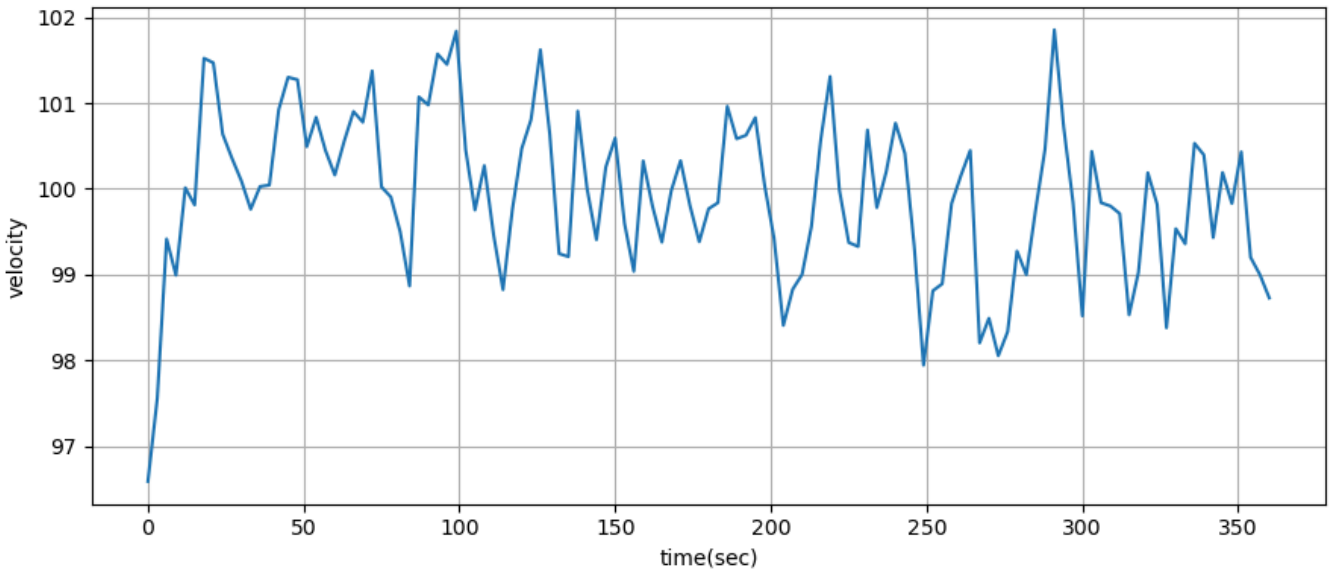
$$\boldsymbol{z = [\mathtt{slant_range},\ \text{elevation angle},\ \dot{x},\ \dot{y}]}$$

测量向量包含四个分量，因此测量函数也应返回四个分量。斜距与仰角仍按前面的方式计算；$\boldsymbol{x}$ 和 $\boldsymbol{y}$ 方向的速度则直接取自状态估计，无须另作几何转换。

```
def h_vel(x):
    dx = x[0] - h_vel.radar_pos[0]
    dz = x[2] - h_vel.radar_pos[1]
    slant_range = math.sqrt(dx**2 + dz**2)
    elevation_angle = math.atan2(dz, dx)
    return slant_range, elevation_angle, x[1], x[3]
```

现在实现滤波器：

```
h_radar.radar_pos = (0, 0)
h_vel.radar_pos = (0, 0)

range_std = 500.
elevation_angle_std = math.degrees(0.5)
vel_std = 2.

np.random.seed(200)
ac = ACSim(ac_pos, (100, 0), 0.02)
radar = RadarStation((0, 0), range_std, elevation_angle_std)

kf_sf2 = cv_UKF(f_cv_radar, h_vel,
            R_std=[range_std, elevation_angle_std, vel_std, vel_std])

time = np.arange(0, 360 + dt, dt)
xs = []
for t in time:
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    # simulate the doppler velocity reading
    vx = ac.vel[0] + randn()*vel_std
    vz = ac.vel[1] + randn()*vel_std
    kf_sf2.predict()
    kf_sf2.update([r[0], r[1], vx, vz])
    xs.append(kf_sf2.x)
xs = np.asarray(xs)
plot_radar(xs, time, plot_x=False, plot_vel=True, plot_alt=False)
print(f'Velocity std {np.std(xs[10:,1]):.1f} m/s')
```




```
Velocity std 0.9 m/s
```

通过引入速度传感器，我们成功将标准差从 3.5 米/秒降低到了 1.3 米/秒。

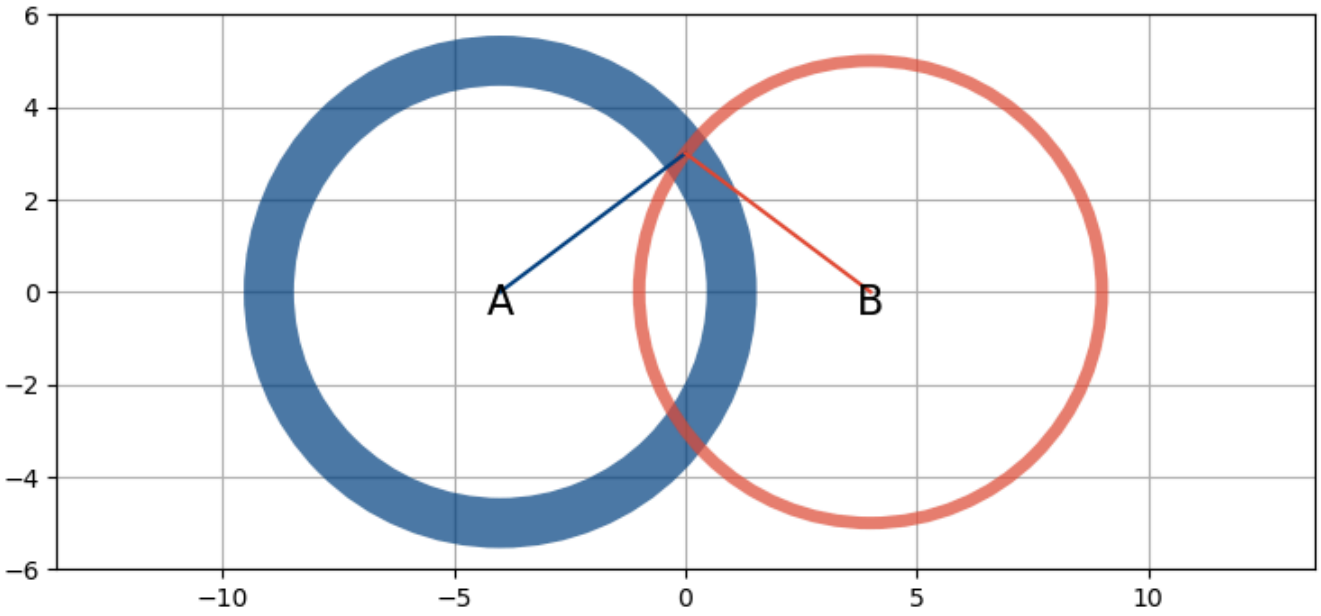
传感器融合是一个很大的主题，这里只是简单示例。导航系统中的传感器往往提供互补信息。例如，GPS 可能每秒提供一次较准确的位置更新，但速度估计较差；惯性导航系统可能以 50 Hz 提供较准确的短时速度更新，但位置估计较差。二者的优势互补，由此可构造互补滤波器，将惯性系统高更新率的速度信息与 GPS 准确但低更新率的位置结合，得到更新频率高且准确的位置与速度估计。在相邻 GPS 更新之间，对高频速度估计积分，即可持续更新位置。

多个位置传感器

上一个传感器融合问题比较简单，下面看一个更接近实际的例子。GPS 普及之前，船舶和飞机使用 VOR、LORAN、TACAN、DME 等无线电导航系统。这些系统发射无线电信标，接收设备据此求取相对于信标的距离和／或方位。例如，飞机可装有两个 VOR 接收机，分别调谐到不同的 VOR 台。每个接收机指示飞机相对于地面 VOR 台的径向线方向，即从台站指向飞机的方位。飞行员在航图上求两条径向线的交点，即可确定飞机位置。

那是一种精度不高的手工方法。卡尔曼滤波可以给出准确得多的位置估计。假设有两个传感器，每个只测量目标方位角，如下图所示。两个方向带的宽度按传感器噪声的 3σ 范围绘制，因此飞机应以很高概率位于两个方向带的交叠区域。

```
ukf_internal.show_two_sensor_bearing()
```



我们按照以下方式来计算传感器与目标之间的方位角：

```
def bearing(sensor, target_pos):
    return math.atan2(target_pos[1] - sensor[1],
                      target_pos[0] - sensor[0])
```

该滤波器以向量形式接收来自两个传感器的测量数据。该代码可以接受任何可迭代的数据结构，为了提高效率，我使用了 Python 列表来存储数据。具体实现方式如下：

```
def measurement(A_pos, B_pos, pos):
    angle_a = bearing(A_pos, pos)
    angle_b = bearing(B_pos, pos)
    return [angle_a, angle_b]
```

假设飞机的运动速度是恒定的。为了换个计算方式，我选择直接计算出飞机的新位置，而不是使用矩阵-向量乘法来计算。

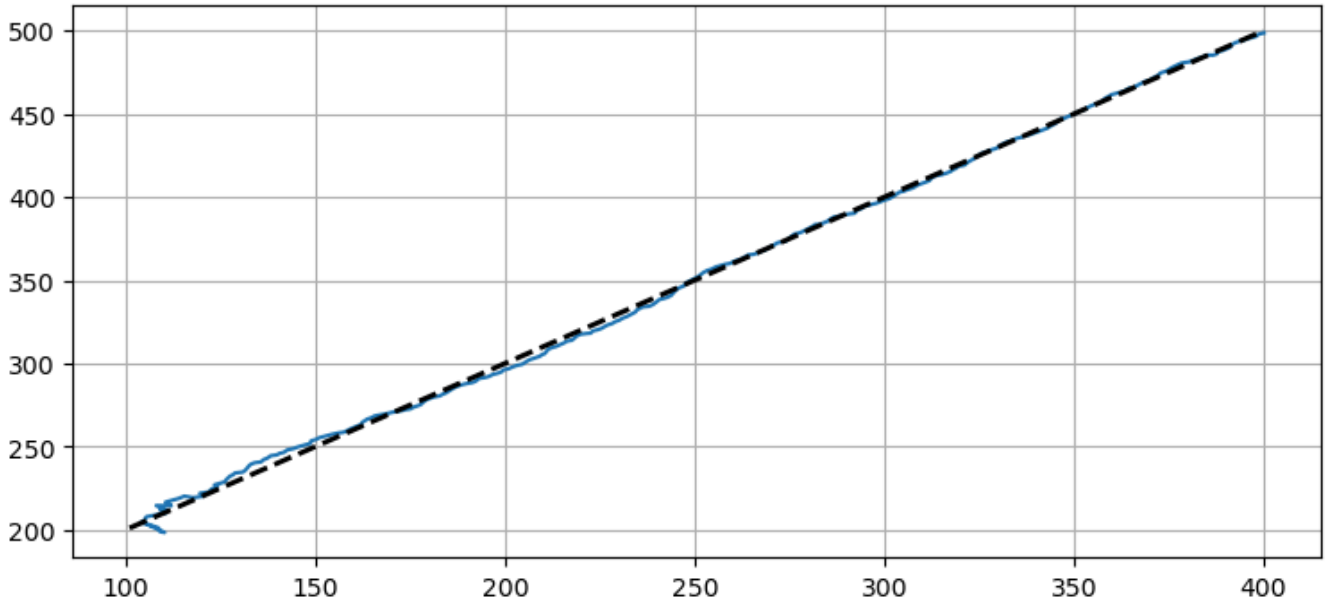
```
def fx_VOR(x, dt):
    x[0] += x[1] * dt
    x[2] += x[3] * dt
    return x
```

接下来实现测量函数，将先验状态映射为包含两个台站预测测量的数组。虽然我不喜欢全局变量，这里仍把台站位置存入 sa_pos 和 sb_pos，用以演示向 $\boldsymbol{h}()$ 共享额外数据的一种方式：

```
sa_pos = [-400, 0]
sb_pos = [400, 0]

def hx_VOR(x):
    # measurement to A
    pos = (x[0], x[2])
    return measurement(sa_pos, sb_pos, pos)
```

现在我们来编写一些通用代码，用于构建滤波器、运行该滤波器并绘制结果图表：

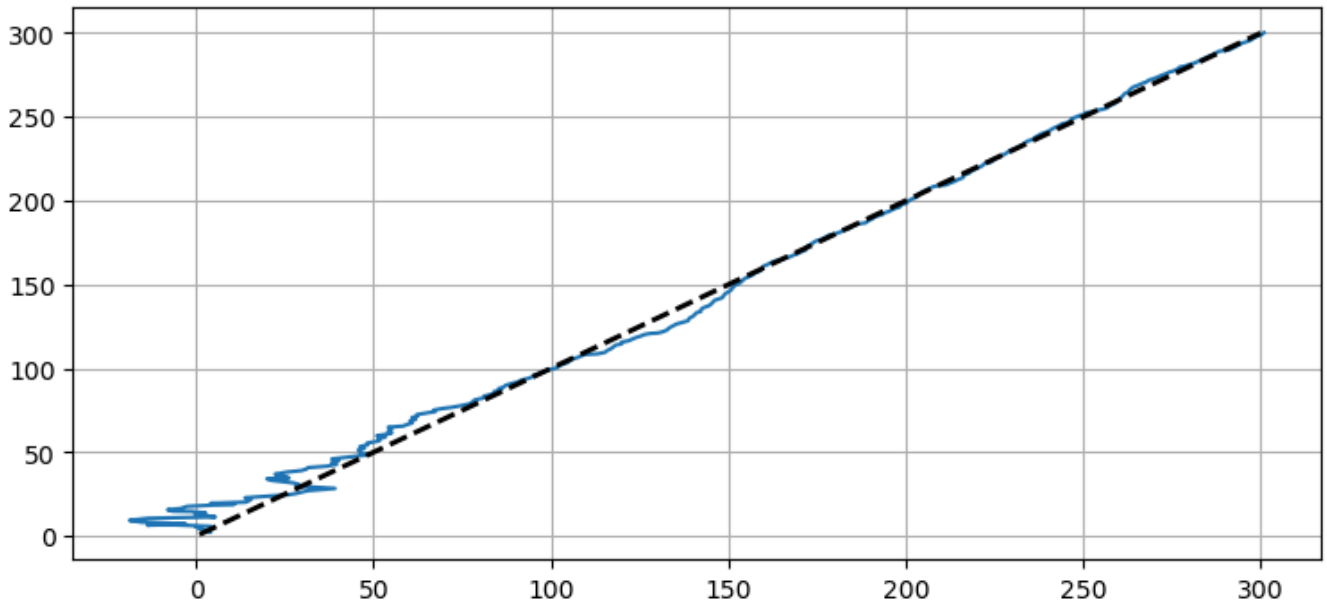
```
def moving_target_filter(pos, std_noise, Q, dt=0.1, kappa=0.0):
    points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=kappa)
    f = UKF(dim_x=4, dim_z=2, dt=dt,
            hx=hx_VOR, fx=fx_VOR, points=points)
    f.x = np.array([pos[0], 1., pos[1], 1.])

    q = Q_discrete_white_noise(2, dt, Q)
    f.Q[0:2, 0:2] = q
    f.Q[2:4, 2:4] = q
    f.R *= std_noise**2
    f.P *= 1000
    return f

def plot_straight_line_target(f, std_noise):
    xs, txs = [], []
    for i in range(300):
        target_pos[0] += 1 + randn()*0.0001
        target_pos[1] += 1 + randn()*0.0001
        txs.append((target_pos[0], target_pos[1]))

        z = measurement(sa_pos, sb_pos, target_pos)
        z[0] += randn() * std_noise
        z[1] += randn() * std_noise

        f.predict()
        f.update(z)
        xs.append(f.x)

    xs = np.asarray(xs)
    txs = np.asarray(txs)

    plt.plot(xs[:, 0], xs[:, 2])
    plt.plot(txs[:, 0], txs[:, 1], ls='--', lw=2, c='k')
    plt.show()

np.random.seed(123)
target_pos = [100, 200]

std_noise = math.radians(0.5)
f = moving_target_filter(target_pos, std_noise, Q=1.0)
plot_straight_line_target(f, std_noise)
```



在我看来，效果相当不错。在处理的初期，误差较大，但经过滤波处理后，得到的结果相当准确。

让我们再来看看角度的非线性问题。我会把目标点置于两个传感器之间的(0,0)位置。这样一来，残差的计算结果就会出现非线性：因为平均角度会接近零。当角度小于 0 时，测量结果会显示出接近 2π 的巨大正数值。因此，预测值与测量值之间的残差会非常大，接近 2π，而不是接近 0。这就使得滤波器无法准确地进行计算，如下例所示。

```
target_pos = [0, 0]
f = moving_target_filter(target_pos, std_noise, Q=1.0)
plot_straight_line_target(f, std_noise)
```

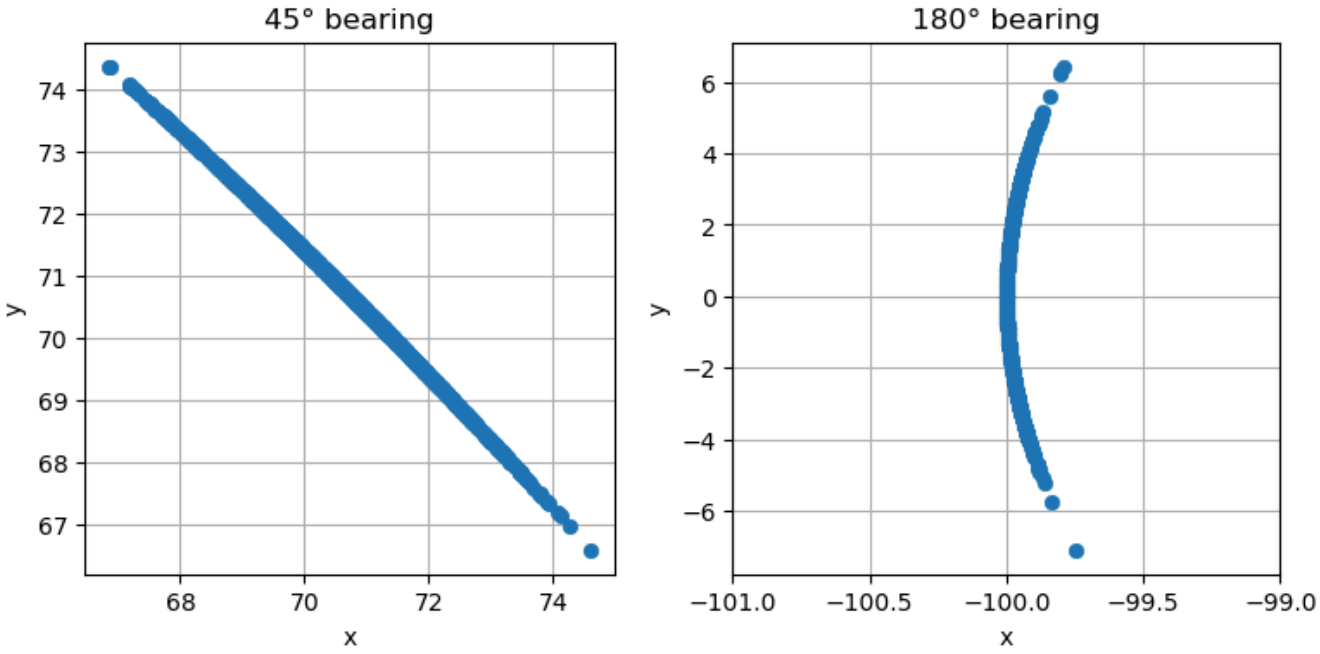


这种性能表现是不可接受的。 `FilterPy` 的 UKF 算法允许用户指定一个函数，用于计算在出现此类非线性行为时的残差值。本章的最后一个示例就展示了该算法的用法。

传感器误差与几何结构的影响

传感器与目标之间的几何关系，会对可获得的定位精度形成物理限制，给滤波器设计带来困难。如果两个 VOR 台对应的方位射线几乎平行，很小的角度误差就会转换成很大的距离误差。更麻烦的是，这种关系是非线性的：x、y 两个方向上的误差会随实际方位角而变。下面的散点图展示两种不同方位下，角度误差标准差 σ 为 1° 时的位置误差分布。

```
ukf_internal.plot_scatter_of_bearing_error()
```



练习：解释滤波器的性能表现

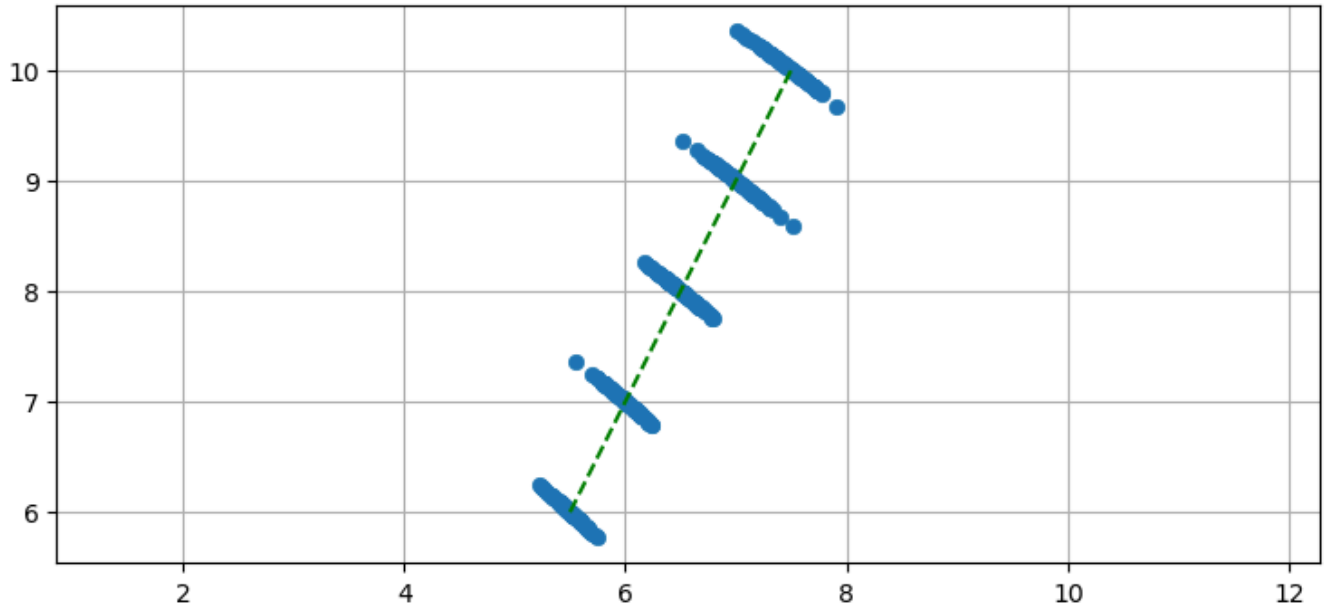
我们可以看到，当角度误差较小时，位置误差会非常大。请解释一下：在上述目标跟踪任务中，为什么 UKF 算法能够取得相对不错的性能。请同时针对单传感器和多传感器的情况进行说明。

解答

理解这一点十分重要。阅读下面的答案前，请认真尝试作答。如果仍无法回答，可以回顾“多元卡尔曼滤波器”一章的相关内容。

我们的成功得益于多个因素。首先，以仅使用一个传感器的情况为例。单次测量所得的结果范围可能非常大。不过，我们的目标物是在移动的，而 UKF 算法正好考虑到了这一点。让我们把对移动目标进行连续多次测量所得的结果绘制出来吧。

```
ukf_internal.plot_scatter_moving_target()
```



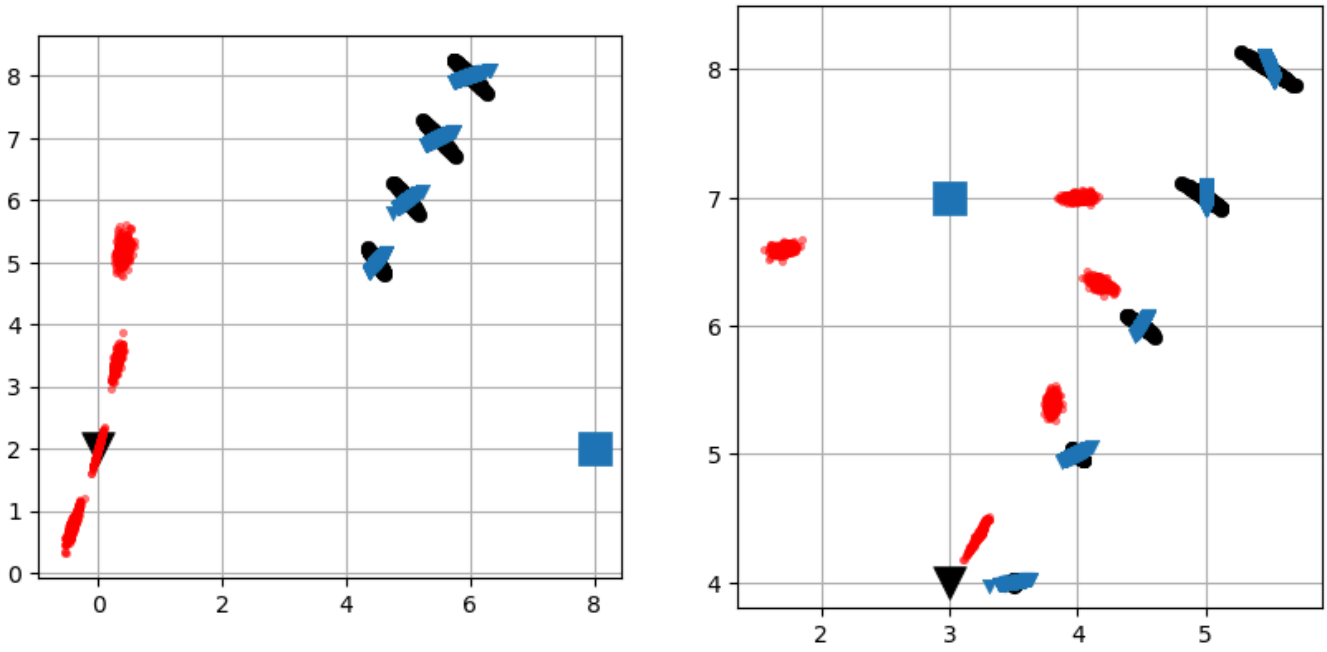
每次单独测量的位置误差都很大，但连续测量显示出清晰趋势：目标正朝右上方移动。卡尔曼滤波器通过测量函数，在计算增益时计入这种误差分布。本例的误差主要沿约 45° 的方向展开，因此滤波器会降低该方向上测量信息的权重；与其垂直的方向上误差很小，增益也会相应体现这一点。

从这张图表来看，一切似乎都很简单：因为我们为每个位置更新都记录了 100 次测量数据。飞机的运动轨迹一目了然。相比之下，卡尔曼滤波器每次只进行一次测量。因此，其预测结果自然无法像那条绿色的虚线所显示的那样精确。

再考虑仅有方位测量、没有直接距离信息的情况。假设实际目标距传感器 7.07 km，却将初始估计放在 1,000 km 处，并把 $\mathbf{P}$ 设得很小。在这一距离上，1° 的角误差相当于约 17.5 km 的横向位置误差。滤波器一方面错误地确信自己的初始位置，另一方面又缺少直接距离测量，因此在本例中难以收敛到真实目标。

现在考虑加入第二个传感器。下面两幅图展示不同布站方式的效果。用方形和三角形分别表示两个传感器，并用相同形状和颜色画出各自造成的误差分布；再根据两个带噪方位测量计算对应的 (x, y) 坐标，用红点表示在 x、y 平面中的位置误差分布。

```
with figsize(10,5):
    ukf_internal.plot_iscts_two_sensors()
```



第一幅图中，从目标初始位置看向两个传感器的方向近乎正交，因此形成清晰的“X”形交叠区域。红色散点的形状展示了目标运动时 $\boldsymbol{x}$ 与 $\boldsymbol{y}$ 方向的误差如何变化：目标离传感器越来越远，同时其高度坐标逐渐接近传感器 B 的 $\boldsymbol{y}$ 坐标时，点云变得明显呈椭圆状。

第二幅图中，飞机最初很接近一个传感器，随后飞过另一个。两个方向误差带的交角偏离直角很大，因此交点所对应的位置误差十分分散。

UKF 算法的实现

FilterPy 实现了 UKF 算法，但学习如何将数学公式转化为代码的过程也很有意义。实现 UKF 算法其实相当简单。首先，我们来编写代码来计算各 sigma 点的均值和协方差吧。

将 sigma 点及其权重存入矩阵，如下所示：

$$\mathbf{weights} = \begin{bmatrix} w_0 & w_1 & \dots & w_{2n} \end{bmatrix}$$

$$\mathbf{sigmas} = \begin{bmatrix} \mathcal{X}_{0,0} & \mathcal{X}_{0,1} & \dots & \mathcal{X}_{0,n-1} \\ \mathcal{X}_{1,0} & \mathcal{X}_{1,1} & \dots & \mathcal{X}_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{X}_{2n,0} & \mathcal{X}_{2n,1} & \dots & \mathcal{X}_{2n,n-1} \end{bmatrix}$$

为了描述这么简单的事情，竟然需要使用这么多下标。下面以二维问题的例子来说明吧（ $\boldsymbol{n}$ =2）：

```
points = MerweScaledSigmaPoints(n=2, alpha=.1, beta=2., kappa=1.)
points.sigma_points(x=[0.,0], P=[[1.,.1],[.1, 1]])
```

```
array([[ 0.   ,  0.   ],
       [ 0.173,  0.017],
       [ 0.   ,  0.172],
       [-0.173, -0.017],
       [ 0.   , -0.172]])
```

表示均值的 sigma 点位于第一行，位置为 (0,0)，与均值相同；第二个点位于 (0.173,0.017)，依此类推。共 $2n+1=5$ 行，每行对应一个 sigma 点。若状态维数为 $n=3$，则应有 3 列、7 行。n 表示状态维数，不是 sigma 点的数量。

每个 sigma 点按行存储还是按列存储，本来可以自行选择。我采用逐行存储，使代码更清楚：第 i^{th} 个 sigma 点可写为 sigmas[i]，而不必写成 sigmas[:, i]。

权重

用 NumPy 计算权重很容易。回顾 Van der Merwe 缩放 sigma 点算法的权重公式：

$$\begin{aligned}\lambda &= \alpha^2(n+\kappa) - n \\ W_0^m &= \frac{\lambda}{n+\lambda} \\ W_0^c &= \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta \\ W_i^m = W_i^c &= \frac{1}{2(n+\lambda)} \quad i = 1..2n\end{aligned}$$

相应代码为：

```
lambda_ = alpha**2 * (n + kappa) - n
Wc = np.full(2*n + 1, 1. / (2*(n + lambda_))
Wm = np.full(2*n + 1, 1. / (2*(n + lambda_))
Wc[0] = lambda_ / (n + lambda_) + (1. - alpha**2 + beta)
Wm[0] = lambda_ / (n + lambda_)
```

变量名 lambda_ 末尾带下划线，是因为 lambda 是 Python 的保留字。添加尾随下划线是 Python 中避开保留字的惯常写法。

sigma 点

sigma 点的计算公式如下：

$$\begin{cases}\mathcal{X}_0 = \mu \\ \mathcal{X}_i = \mu + \left[\sqrt{(n+\lambda)\Sigma}\right]_i, & i = 1..n \\ \mathcal{X}_i = \mu - \left[\sqrt{(n+\lambda)\Sigma}\right]_{i-n} & i = (n+1)..2n\end{cases}$$

只要我们理解了 $\left[\sqrt{(n+\lambda)\Sigma}\right]_i$ 这个概念，使用 Python 就并不难了。

由于 Σ 是矩阵，$\sqrt{(n+\lambda)\Sigma}$ 也是矩阵。$[\sqrt{(n+\lambda)\Sigma}]_i$ 中的下标 i 表示选取平方根矩阵的第 i 行。那么矩阵的平方根是什么？需要区分不同约定：通常说 Σ 的矩阵平方根 S，是指它与自身相乘得到 Σ；也就是说，若 $\Sigma = SS$，则 $S = \sqrt{\Sigma}$。

这里采用另一种便于数值计算的“协方差平方根”因子：矩阵 S 与其转置相乘，得到 Σ：

$$\Sigma = \mathrm{SS}^\mathsf{T}$$

这样的 $\mathbf{S}$ 可以通过 Cholesky 分解计算 [5]。该分解把厄米正定矩阵表示为三角矩阵及其共轭转置的乘积。使用下三角因子或上三角因子时，乘法次序分别如下：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^*$$
$$\boldsymbol{A} = \boldsymbol{U}^*\boldsymbol{U}$$

星号表示共轭转置；由于我们处理的都是实数，因此可以这样表示：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$$
$$\boldsymbol{A} = \boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}$$

当协方差矩阵 $\mathbf{P}$ 正定时，可以把 $\mathbf{S} = \mathbf{cholesky}(\mathbf{P})$ 用作 $\mathbf{P}$ 的协方差平方根因子。一般协方差只保证半正定，普通 Cholesky 分解还要求正定。上三角因子按“转置乘自身”的次序重构协方差，下三角因子按“自身乘转置”的次序重构，因此计算平方根的方式必须与选取行或列向量的约定配套。

SciPy 在 `scipy.linalg` 中提供了 `cholesky()` 方法。如果你使用的是 Fortran、C 或 C++等语言，那么 LAPACK 等库就提供了该功能。Matlab 则提供了 `chol()` 功能。

scipy.linalg.cholesky() 默认返回上三角因子，所以这里的代码也采用这一约定，并逐行取偏移向量。这样，第一行中各个非零元素会共同影响第一对偏离均值的 sigma 点；中心点本身仍等于均值。如果自行提供平方根算法，必须保持约定一致。文献中也有逐列取向量的实现：它可与下三角 Cholesky 因子配套使用。若平方根矩阵是对称的，则按行或按列选取没有区别。

```
import scipy
a = np.array([[2., .1], [.1, 3]])
s = scipy.linalg.cholesky(a)
print("cholesky:")
print(s)
print("\nsquare of cholesky:")
print(s @ s.T)
```

```
cholesky:
[[1.414 0.071]
 [0.    1.731]]

square of cholesky:
[[2.005 0.122]
 [0.122 2.995]]
```

因此，我们可以用这段代码来实现sigma 点算法：

```
sigmas = np.zeros((2*n+1, n))
U = scipy.linalg.cholesky((n+lambda_)*P) # sqrt
sigmas[0] = X
for k in range (n):
sigmas[k+1] = X + U[k]
sigmas[n+k+1] = X - U[k]
```

现在让我们来实现无迹变换吧。请回顾一下相关的公式。

$$\mu = \sum_i w_i^m \mathcal{X}_i$$
$$\Sigma = \sum_i w_i^c (\mathcal{X}_i - \mu)(\mathcal{X}_i - \mu)^{\mathsf{T}}$$

加权均值的求和可以这样实现：

```
x = np.dot(Wm, sigmas)
```

不常使用 NumPy 的读者可能对这种写法不熟悉。NumPy 的许多数值运算由底层 C、Fortran 等编译代码完成，通常比对应的 Python 循环快得多。要发挥这一优势，应尽量使用 NumPy 的数组运算，而不是逐元素写 for 循环。例如，两个向量的点积就是对应元素乘积之和，可直接调用 numpy.dot(x, y)。当一个输入是一维数组、另一个是二维数组时，dot 会将一维数组与二维数组的每一列作点积，得到如下结果：

```
a = np.array([10, 100])
b = np.array([[1, 2, 3],
              [4, 5, 6]])
np.dot(a, b)
```

```
array([410, 520, 630])
```

现在只需计算 $\mathbf{P} = \sum_i w_i(\mathcal{X}_i - \mu)(\mathcal{X}_i - \mu)^\mathsf{T} + \mathbf{Q}$ 即可：

```
kmax, n = sigmas.shape
P = zeros((n, n))
for k in range(kmax):
y = sigmas[k] - x
P += Wc[k] * np.outer(y, y)
P += Q
```

这里还涉及 NumPy 的一个特性。状态均值与 sigmas[k] 都是一维数组，因此 sigmas[k]-X 也仍是一维数组。一维数组的转置不会将它变成列向量：[1, 2, 3] 的转置仍是一维的 [1, 2, 3]。因此，调用 np.outer(y, y) 来计算外积 $\mathbf{yy}^\mathsf{T}$，其中 $\mathbf{y}$ 是一维数组。另一种实现方式是：

```
y = (sigmas[k] - x).reshape(n, 1) # convert into 2D array
P += Wc[k] * np.dot(y, y.T)
```

这种列向量写法需要将差值数组按状态维数重塑，并使用当前循环索引 k 对应的协方差权重。它较慢，也不符合 NumPy 的惯用风格，因此这里采用前面的 np.outer 写法。

预测步骤

在预测阶段，我们将按照上述方式来生成权重和sigma 点。然后，我们将每个sigma 点代入函数 f 中进行计算。

$$\mathcal{Y} = f(\mathcal{X})$$

再用无迹变换计算预测均值与协方差。下面假设这段代码是类的方法，类中保存滤波器所需的矩阵与向量。预测从当前后验均值 self.x 和后验协方差 self.P 生成 sigma 点，输出的 self.xp 和 self.Pp 分别保存先验均值与先验协方差。

```
def predict(self, sigma_points_fn):
 """ Performs the predict step of the UKF. On return,
    self.xp and self.Pp contain the predicted state (xp)
    and covariance (Pp). 'p' stands for prediction.
    """
# calculate sigma points for given mean and covariance
sigmas = sigma_points_fn(self.x, self.P)
for i in range(self._num_sigmas):
self.sigmas_f[i] = self.fx(sigmas[i], self._dt)
self.xp, self.Pp = unscented_transform(
self.sigmas_f, self.Wm, self.Wc, self.Q)
```

更新步骤

更新步骤通过测量函数 h(x) 将 sigma 点映射到测量空间。

$$\mathcal{Z} = h(\mathcal{Y})$$

用无迹变换计算这些点的均值与协方差，再计算残差和卡尔曼增益。其中，互协方差按下式计算：

$$\mathbf{P}_{xz} = \sum_{i=0}^{2n} w_i^c (\mathcal{Y}_i - \mu)(\mathcal{Z}_i - \mu_z)^\mathsf{T}$$

最后，我们利用残差和卡尔曼增益来计算新的状态估计值：

$$\begin{aligned} K &= \mathbf{P}_{xz}\mathbf{P}_z^{-1} \\ \mathbf{x} &= \bar{\mathbf{x}} + \mathbf{K}\mathbf{y} \end{aligned}$$

新的协方差则按照以下方式计算：

$$\mathbf{P} = \bar{\mathbf{P}} - \mathbf{K}\mathbf{P}_z\mathbf{K}^\mathsf{T}$$

假设这是某个类的方法，而该类保存了所需的矩阵和数据，那么可以这样实现：

```
def update(self, z):
# rename for readability
sigmas_f = self.sigmas_f
sigmas_h = self.sigmas_h
# transform sigma points into measurement space
for i in range(self._num_sigmas):
sigmas_h[i] = self.hx(sigmas_f[i])
# mean and covariance of prediction passed through UT
zp, Pz = unscented_transform(sigmas_h, self.Wm, self.Wc, self.R)
# compute cross variance of the state and the measurements
Pxz = np.zeros((self._dim_x, self._dim_z))
for i in range(self._num_sigmas):
Pxz += self.Wc[i] * np.outer(sigmas_f[i] - self.xp,
sigmas_h[i] - zp)
K = np.dot(Pxz, inv(Pz)) # Kalman gain
self.x = self.xp + np.dot(K, z - zp)
self.P = self.Pp - np.dot(K, Pz).dot(K.T)
```

FilterPy 的实现方式

FilterPy 对上述代码作了进一步推广：可以选择不同的 sigma 点算法，自定义状态残差的计算方式，以处理角度的周期性环绕；还可以提供矩阵平方根函数等。具体请参阅帮助文档。

https://filterpy.readthedocs.org/#unscented-kalman-filter

批量处理

卡尔曼滤波是递归算法：当前估计由当前测量与先验估计融合而来。不过，经常需要处理一批已经采集完毕的数据。这时可以使用批量接口，对整组测量依次执行滤波。批量接口并不改变每个时间步内部递归处理的性质。

把测量得到的数值收集到一个数组或列表中。

```
zs = read_altitude_from_csv()
```

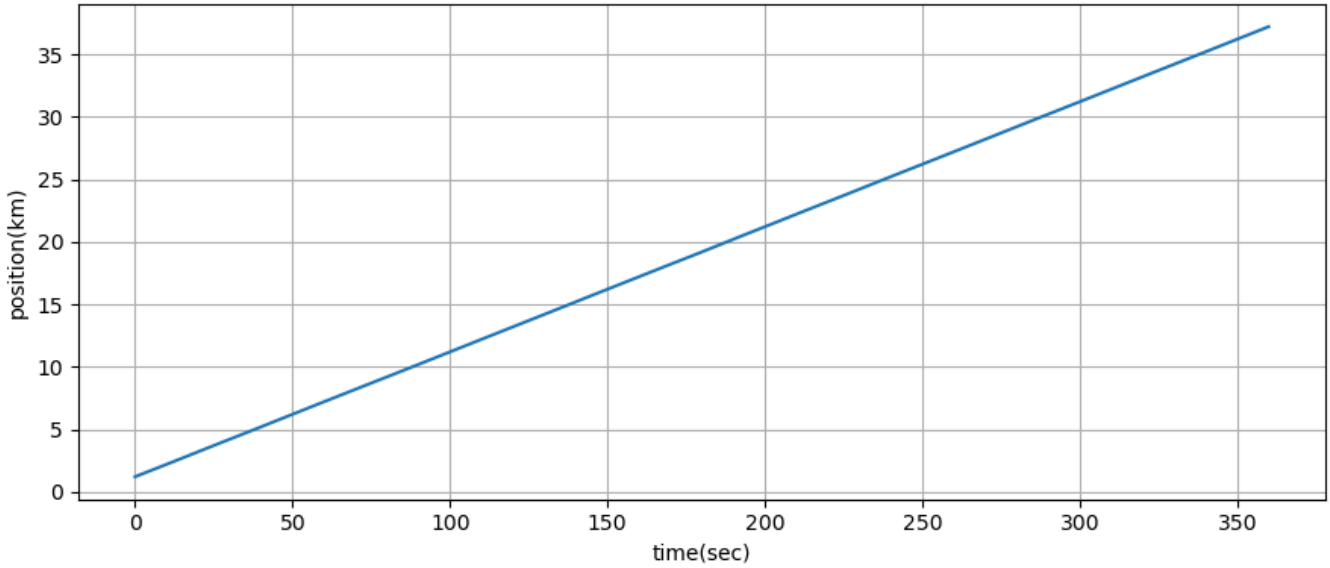
然后调用 `batch_filter()` 方法。

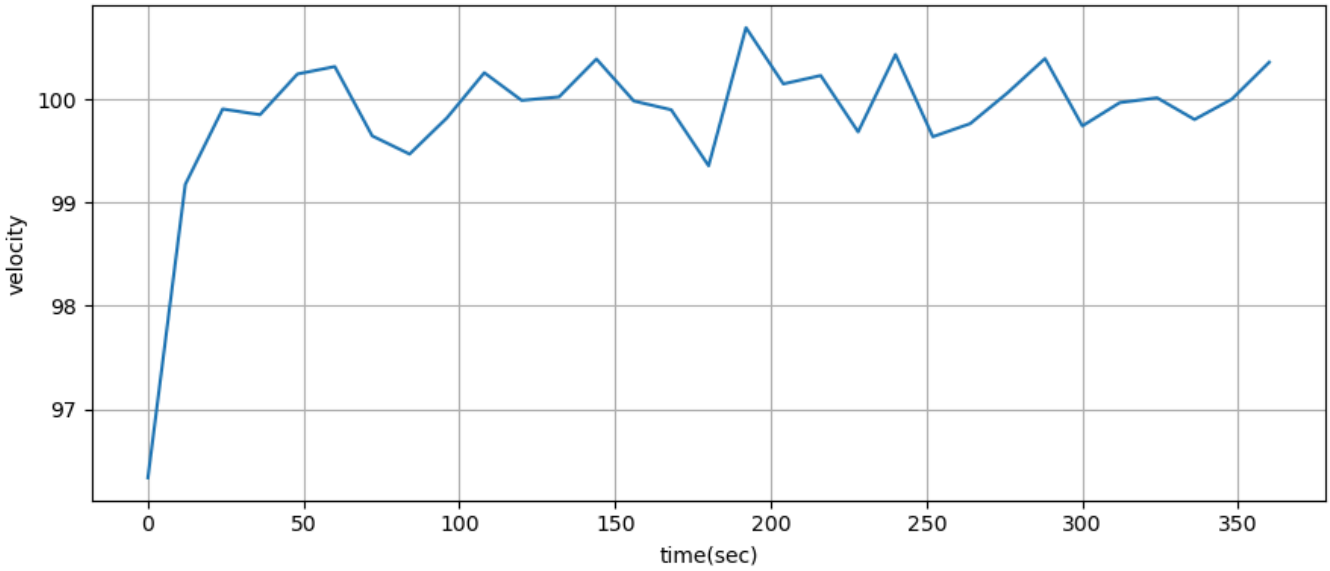
```
Xs, Ps = ukf.batch_filter(zs)
```

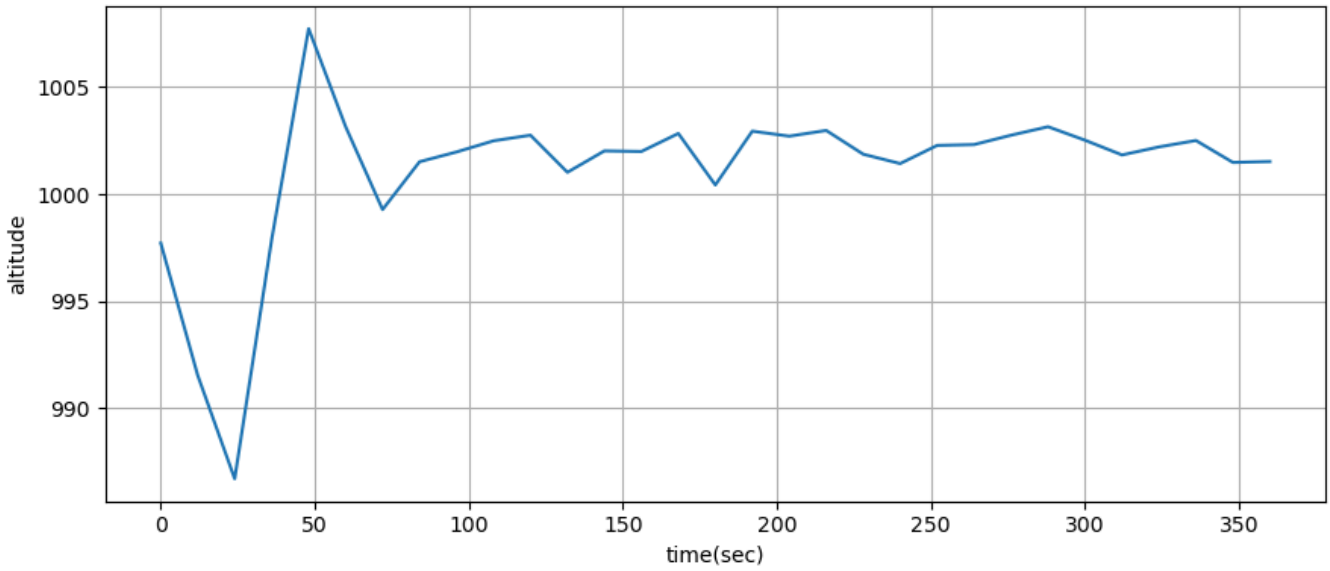
该函数接收测量值列表/数组，对其进行处理后，返回整个数据集的状态估计值数组（ `Xs` ）以及协方差矩阵数组（ `Ps` ）。

以下是针对上述雷达跟踪问题所给出的完整示例。

```
dt = 12. # 12 seconds between readings
range_std = 5 # meters
bearing_std = math.radians(0.5)

ac_pos = (0., 1000.)
ac_vel = (100., 0.)
radar_pos = (0., 0.)
h_radar.radar_pos = radar_pos

points = MerweScaledSigmaPoints(n=3, alpha=.1, beta=2., kappa=0.)
kf = UKF(3, 2, dt, fx=f_radar, hx=h_radar, points=points)

kf.Q[0:2 ,0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
kf.Q[2, 2] = 0.1

kf.R = np.diag([range_std**2, bearing_std**2])
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])

radar = RadarStation((0, 0), range_std, bearing_std)
ac = ACSim(ac_pos, (100, 0), 0.02)

np.random.seed(200)

t = np.arange(0, 360 + dt, dt)
n = len(t)

zs = []
for i in range(len(t)):
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    zs.append([r[0], r[1]])

xs, covs = kf.batch_filter(zs)
ukf_internal.plot_radar(xs, t)
```


position(km)
time(sec)


velocity
time(sec)


altitude
time(sec)

对结果进行平滑处理

假设我们正在追踪一辆汽车。如果我们得到的测量数据有较大误差，表明汽车似乎要向左转弯，但状态函数却预测汽车会继续直线行驶。在这种情况下，卡尔曼滤波器别无选择，只能将状态估计值稍微向那个有误差的测量结果方向调整。因为它无法判断：这究竟只是因为测量数据有误差，还是汽车真的开始转弯了。

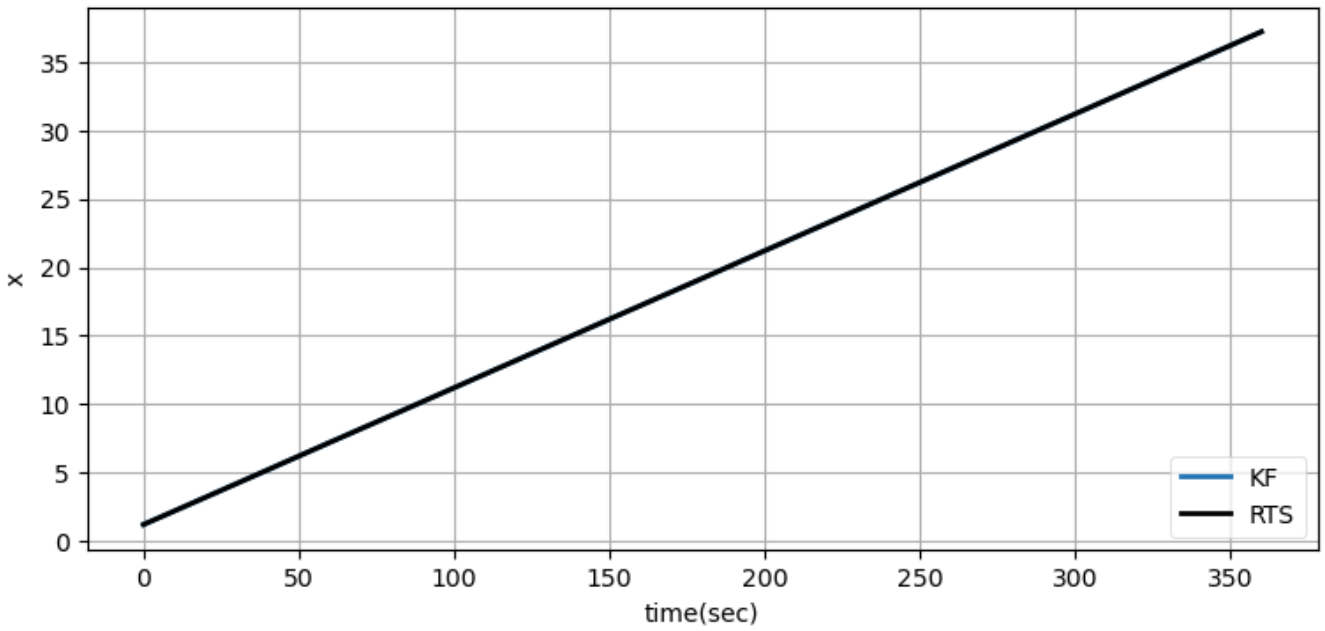
如果先收集数据再作后处理，就能利用有疑问的测量之后的观测，判断是否真的发生了转向。若后续观测持续显示向左转，就有更充分的依据认为先前那次测量并非严重受噪声污染，而是车辆确实开始转弯。

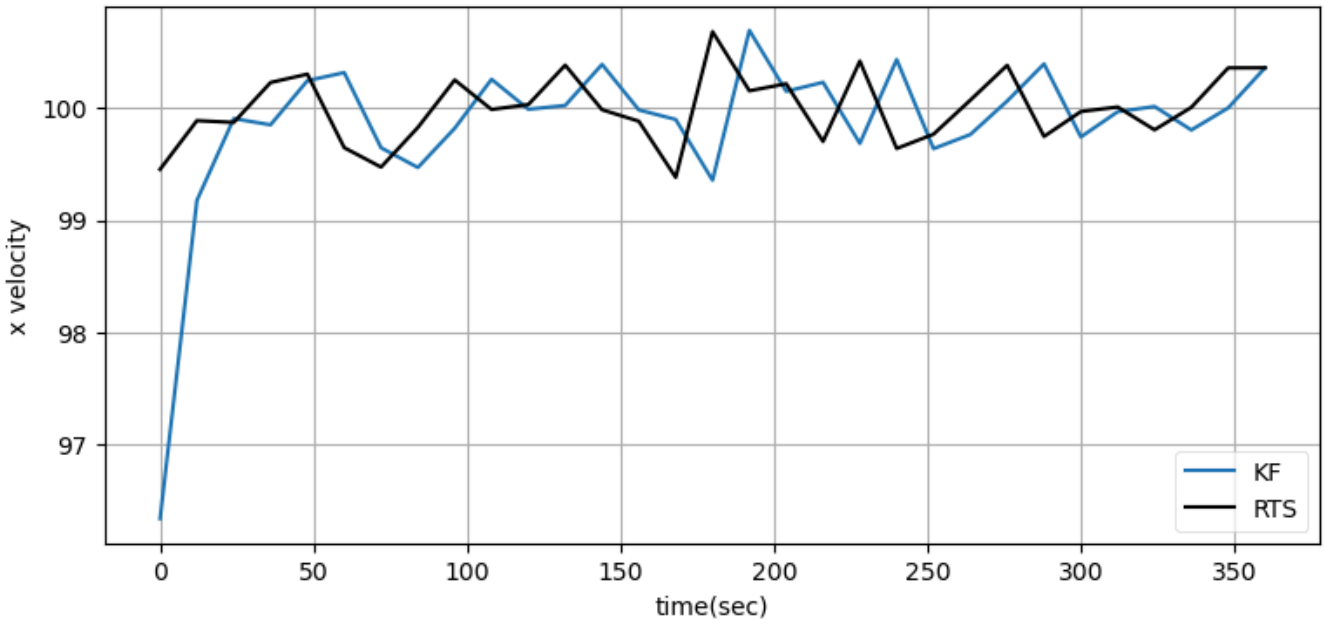
我们不会在这里详细阐述该数学公式或算法的实现细节。我只会向你展示如何在 `FilterPy` 中调用该算法。我们所实现的这个算法被称为“RTS 平滑器”，其名称源自该算法的三位发明者：Rauch、Tung 和 Striebel。

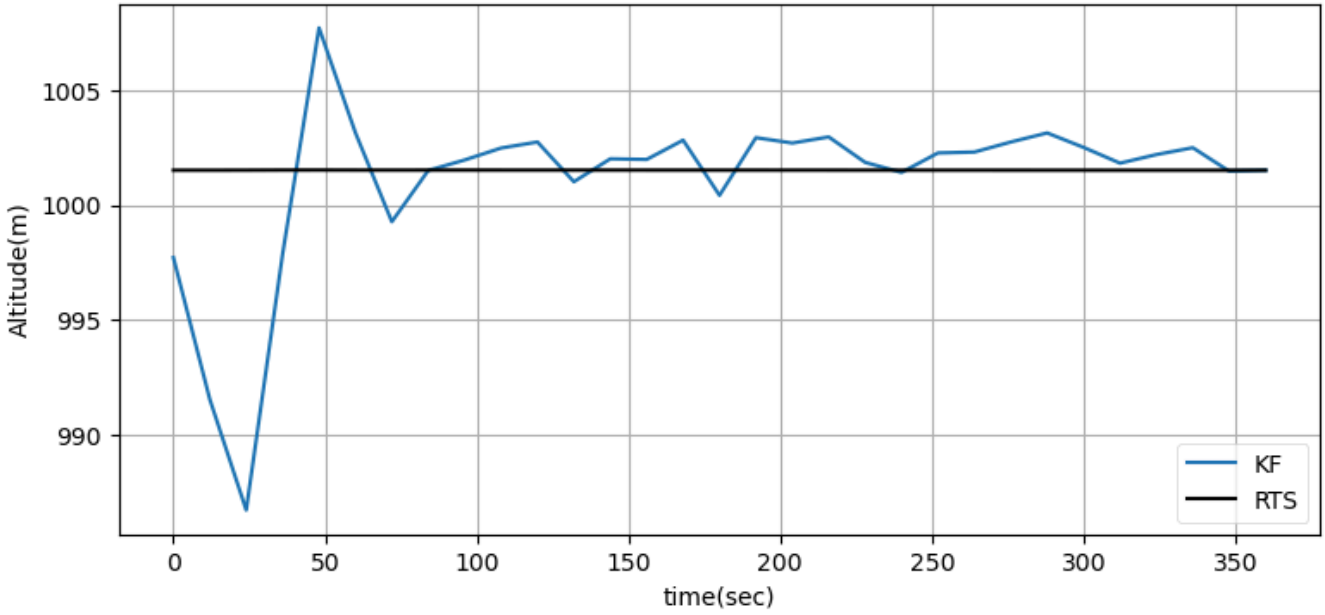
对应的方法是 UnscentedKalmanFilter.rts_smoother()。将 batch_filter 计算的均值与协方差传入，即可得到平滑后的均值、协方差和平滑增益。这里的平滑增益应与前向滤波中的卡尔曼增益区分。

```
Ms, P, K = kf.rts_smoother(xs, covs)
ukf_internal.plot_rts_output(xs, Ms, t)
```

```
Difference in position in meters:
     [-1.4166 -0.2815  1.2679 -1.2405 -2.1863]
```







从图中可以看到，水平位置的改善较小，速度的改善很明显，而高度的改善尤其显著。水平位置的差异很小，因此我打印了最后 5 个点中 UKF 结果与平滑结果之差。如果允许对数据作后处理，我建议使用 RTS 平滑器。

选择 sigma 点参数

我发现，关于如何为 α、β 和 κ 确定合适的数值，现有的文献相当有限。范德梅尔韦的论文中包含了最多的相关信息，但仍然不够全面。那么，让我们来仔细研究一下这些数值应该怎么确定吧。

Van der Merwe 建议高斯问题采用 $\beta = 2$，并取 $\kappa = 3 - n$。下面先采用这组设置，再改变 α。为减小需要观察的数组，并避免计算矩阵平方根，令 $n = 1$。

```
from kf_book.ukf_internal import print_sigmas
print_sigmas(mean=0, cov=3, alpha=1)
```

```
sigmas:  [ 0.  3. -3.]
mean weights: [0.6667 0.1667 0.1667]
cov weights: [2.6667 0.1667 0.1667]
lambda: 2
sum cov 2.9999999999999996
```

那么，这里到底发生了什么？我们可以看到，当平均值为 0 时，该算法选择的sigma 点分别为 0、3 和-3。但为什么会这样呢？请回想一下计算sigma 点的公式：

$$\begin{aligned}\mathcal{X}_0 &= \mu \\ \mathcal{X}_i &= \mu \pm \sqrt{(n+\lambda)\Sigma}\end{aligned}$$

我选择使用 $n=1$ 的方式可以将所有数值都转化为标量形式，从而避免了对矩阵求平方根的计算。因此，对于我们所处理的数值来说，该方程可以简化为：

$$\begin{aligned}\mathcal{X}_0 &= 0 \\ \mathcal{X}_i &= 0 \pm \sqrt{(1+2)\times 3} \\ &= \pm 3\end{aligned}$$

因此，当 α 的数值越大时，那些“sigma 点”就会分布得越广。我们不妨把它设置为一个极大的数值吧。

```
print_sigmas(mean=0, cov=3, alpha=200)
```

```
sigmas:  [   0.  600. -600.]
mean weights: [1. 0. 0.]
cov weights: [-39996.      0.      0.]
lambda: 119999
sum cov -39996.00000000001
```

可以看到，sigma 点分布的跨度已超过 100 个标准差。若输入服从高斯分布，这意味着要在距均值极远的位置计算非线性函数，结果很可能不好。若输入分布具有很厚的尾部，为估计其影响，可能需要选取更远的代表点。这里通过增大 α 扩展点集，但 200 只是为使变化明显而选取的极端值，实际使用时应谨慎选择。

用类似的逻辑来推理，假设我们的概率分布几乎没有“尾部”——也就是说，其形状更像是一个倒置的抛物线。在这种情况下，我们可能会希望将那些“sigma 点”调整得更靠近均值，从而避免在那些永远不会有真实数据出现的区域进行采样。

再看权重的变化。当 $\kappa + n = 3$ 时，中心点的均值权重为 0.6667，两个外围点的权重各为 0.1667。当 $\alpha = 200$ 时，中心点权重接近 0.99999，外围点权重各约为 0.000004。回顾权重公式：

$$\begin{aligned}W_0 &= \frac{\lambda}{n+\lambda} \\ W_i &= \frac{1}{2(n+\lambda)}\end{aligned}$$

当 λ 增大时，中心点的均值权重 $\lambda/(n+\lambda)$ 趋近于 1，其余点的权重趋近于 0。这些权重并不依赖协方差的尺度。因此，扩展点集时，外围点通常获得较小权重。不过，不能据此认为把点移近均值会使各权重近似相等：使用很小的 α 时，中心权重可能是绝对值很大的负数，外围权重则为较大的正数。

Van der Merwe 建议将 α 限制在 $0 < \alpha \leq 1$ 范围内，并将 10^{-3} 作为一个可以尝试的取值。下面看看效果。

```
print_sigmas(mean=0, cov=13, alpha=.001, kappa=0)
```

```
sigmas:  [ 0.      0.0036 -0.0036]
mean weights: [-999999.  500000.  500000.]
cov weights: [-999996.  500000.  500000.]
lambda: -0.999999
sum cov 3.9999989999923855
```

机器人定位——一个完整的实现案例

是时候来处理一些棘手的问题了。大多数书籍都选择那些简单易懂、有标准答案的例子来讲解。这样一来，读者就会不知道该如何应对现实世界中的复杂问题。这个例子虽然无法教会你如何解决各种问题，但可以让你了解在设计和实现滤波器时需要考虑的各种因素。

我们将考虑机器人定位的问题。在这种情况下，有一个机器人正在环境中移动，它通过传感器来检测地标。这可能是一辆使用计算机视觉来识别树木、建筑物和其他地标的自动驾驶汽车。也可能是那种可以打扫你房子的小型机器人，或者是在仓库中工作的机器人。

这款机器人的结构与汽车类似，也有四个轮子。它通过转动前轮来改变方向。这样一来，机器人在前进的同时，也会以后轴为中心进行旋转。这是一种非线性运动方式，我们需要对这种行为进行建模。

机器人配有传感器，能够粗略测量已知地标相对于它的距离与方位。该测量模型是非线性的：由相对位置计算距离会用到平方根，计算方位会用到反三角函数；反过来由距离和方位计算坐标则会用到三角函数。

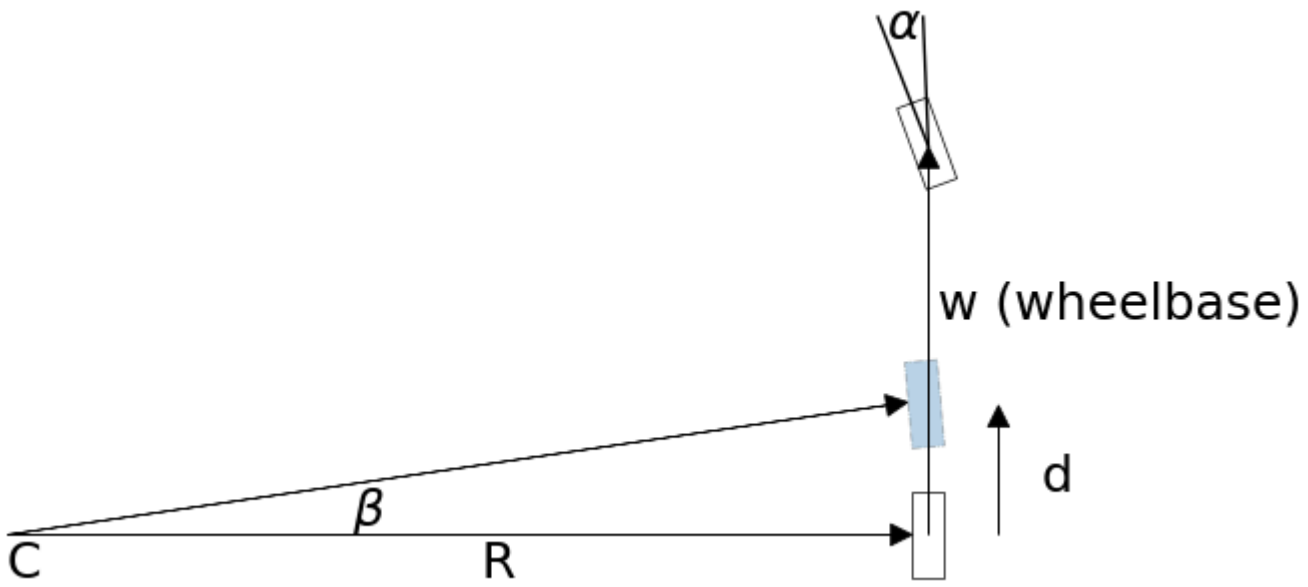
无论是过程模型还是测量模型，都属于非线性模型。UKF 算法能够同时处理这两种非线性情况。因此，我们可以初步得出结论：UKF 算法是解决该问题的合适选择。

机器人运动模型

简单来说，汽车在行驶过程中是通过转动前轮来实现转向的。汽车的前部会朝着车轮所指的方向移动，而整个车身则围绕后轮进行转动。不过，这一简单的机制实际上相当复杂：摩擦力会导致打滑现象；橡胶轮胎在不同速度下的表现也有所不同；此外，外侧轮胎和内侧轮胎的转动半径也必须有所不同。要准确描述汽车的转向过程，就需要用到复杂的微分方程。

在卡尔曼滤波方面，尤其是对于低速运行的机器人应用而言，一种较为简单的自行车模型被证明具有出色的性能。以下是该模型的示意图：

```
ekf_internal.plot_bicycle()
```



图中前轮相对于车身纵向的转角为 α。在很短的时间内，车辆前进，后轮到达蓝色阴影所示的位置，同时车身朝向略有改变。可将这一运动近似为沿半径 R 的圆弧转动，转过的角度 β 为：

$$\beta = \frac{d}{w} \tan(\alpha)$$

而转弯半径 R 则由以下公式给出：

$$R = \frac{d}{\beta}$$

当前进速度为 v 时，后轮行驶的距离为 $d = v\Delta t$。

设机器人朝向为 θ，转弯开始时的圆心 C 可计算为：

$$\begin{aligned} C_x &= x - R\sin(\theta) \\ C_y &= y + R\cos(\theta) \end{aligned}$$

在时间向前推进 Δt 之后，机器人的新位置和方向如下。

$$\begin{aligned} \bar{x} &= C_x + R\sin(\theta + \beta) \\ \bar{y} &= C_y - R\cos(\theta + \beta) \\ \bar{\theta} &= \theta + \beta \end{aligned}$$

一旦我们将 C 代入其中，就能得到

$$\begin{aligned} \bar{x} &= x - R\sin(\theta) + R\sin(\theta + \beta) \\ \bar{y} &= y + R\cos(\theta) - R\cos(\theta + \beta) \\ \bar{\theta} &= \theta + \beta \end{aligned}$$

如果你对转向模型不感兴趣，不必详细了解这些数学推导。需要记住的是，运动模型具有非线性，因此必须在卡尔曼滤波中恰当地处理这种非线性。

设计状态变量

机器人状态选取位置与朝向：

$$\mathbf{x} = \begin{bmatrix} x & y & \theta \end{bmatrix}^\mathsf{T}$$

我可以将速度因素纳入这个模型中，但正如你所看到的，这样一来，数学计算会变得相当复杂。

控制输入 $\mathbf{u}$ 代表所需的行驶速度和转向角度。

$$\mathbf{u} = \begin{bmatrix} v & \alpha \end{bmatrix}^\mathsf{T}$$

设计系统模型

我们将该系统建模为非线性运动模型加上白噪声。

$$\bar{x} = x + f(x, u) + \mathcal{N}(0, Q)$$

利用我们上面所创建的机器人运动模型，我们可以写出如下公式：

```
from math import tan, sin, cos, sqrt

def move(x, dt, u, wheelbase):
    hdg = x[2]
    vel = u[0]
    steering_angle = u[1]
    dist = vel * dt

    if abs(steering_angle) > 0.001: # is robot turning?
        beta = (dist / wheelbase) * tan(steering_angle)
        r = wheelbase / tan(steering_angle) # radius

        sinh, sinhb = sin(hdg), sin(hdg + beta)
        cosh, coshb = cos(hdg), cos(hdg + beta)
        return x + np.array([-r*sinh + r*sinhb,
                             r*cosh - r*coshb, beta])
    else: # moving in straight line
        return x + np.array([dist*cos(hdg), dist*sin(hdg), 0])
```

用这个函数实现状态转移函数 f(x)。move 直接返回预测状态，而不是状态增量，因此调用时不应再额外加一次原状态。

这里选择较小的采样时间间隔 Δt。如果机器人运动较慢，这个函数应能给出合理准确的预测。当 Δt 较大，或者实际动力学比这里的简化模型更复杂时，预测可能不够准确；此时可考虑更精细的模型以及龙格—库塔法等数值积分方法。“卡尔曼滤波的数学原理”一章已作简要介绍。

设计测量模型

传感器对环境中多个已知位置的地标提供带噪距离和方位角。测量模型必须把状态 $\begin{bmatrix} x & y & \theta \end{bmatrix}^\mathsf{T}$ 转换为到地标的距离与方位角。若 $\boldsymbol{p}$ 是地标位置，距离 $\boldsymbol{r}$ 为：

$$r = \sqrt{(p_x - x)^2 + (p_y - y)^2}$$

假设传感器报告的是相对于机器人自身朝向的方位角。因此，需要从世界坐标系下指向地标的方位角中减去机器人的朝向，得到预测传感器读数：

$$\phi = \tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta$$

因此，我们的测量函数为

$$\begin{aligned}\mathbf{z} = h(\mathbf{x}, \mathbf{P}) &&+ \mathcal{N}(0, R)\\ = \begin{bmatrix}\sqrt{(p_x - x)^2 + (p_y - y)^2} \\ \tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta\end{bmatrix} &&+ \mathcal{N}(0, R)\end{aligned}$$

暂不实现这个函数，因为还有一个问题要在下面的“实现”一节处理。

设计测量噪声

这里假设距离测量噪声与方位角测量噪声相互独立，因此：

$$\mathbf{R} = \begin{bmatrix}\sigma_{range}^2 & 0 \\ 0 & \sigma_{bearing}^2\end{bmatrix}$$

实现

编写代码之前，还要处理角度残差。残差为 $\boldsymbol{y = z - h(x)}$；如果 z 中的方位角为 $\mathbf{1°}$，而 $\boldsymbol{h(x)}$ 对应的角为 $\mathbf{359°}$，直接相减得到 $\mathbf{-358°}$，但正确的有符号最短角差应为 $\mathbf{2°}$。错误的残差会使滤波更新失真，因此必须单独编写方位角残差函数。

```
def normalize_angle(x):
    x = x % (2 * np.pi)    # force in range [0, 2 pi)
    if x > np.pi:          # move to [-pi, pi)
        x -= 2 * np.pi
    return x
```

```
print(np.degrees(normalize_angle(np.radians(1-359))))
```

```
1.9999999999999774
```

状态向量中的航向角位于索引 2，而每个地标测量向量中的方位角位于索引 1，因此要分别编写相应的角差处理函数。机器人运动时，可见地标也会改变，必须支持可变数量的测量。测量残差函数接收包含多个地标测量的数组，每个地标对应一组测量。

```
def residual_h(a, b):
    y = a - b
    # data in format [dist_1, bearing_1, dist_2, bearing_2,...]
    for i in range(0, len(y), 2):
        y[i + 1] = normalize_angle(y[i + 1])
    return y

def residual_x(a, b):
    y = a - b
    y[2] = normalize_angle(y[2])
    return y
```

现在可以实现测量模型，其表达式为

$$h(\mathbf{x}, \mathbf{P}) = \begin{bmatrix} \sqrt{(p_x - x)^2 + (p_y - y)^2} \\ \tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta \end{bmatrix}$$

。函数的第二个参数表示地标位置，与通常用 P 表示的状态协方差矩阵含义不同。

表达式 $\tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta$ 可能产生超出范围 $[-\pi, \pi)$ 的结果，因此我们需要将角度值调整到该范围内。

该函数会接收一个包含各地标信息的数组作为输入，需要输出一个格式为 `[dist_to_1, bearing_to_1, dist_to_2, bearing_to_2, ...]` 的测量结果数组。

```
def Hx(x, landmarks):
    """ takes a state variable and returns the measurement
    that would correspond to that state. """
    hx = []
    for lmark in landmarks:
        px, py = lmark
        dist = sqrt((px - x[0])**2 + (py - x[1])**2)
        angle = atan2(py - x[1], px - x[0])
        hx.extend([dist, normalize_angle(angle - x[2])])
    return np.array(hx)
```

困难还没有结束。无迹变换要计算状态向量和测量向量的加权均值，而二者都可能含有角度。角度具有周期性，不能直接沿用普通实数的平均方法。例如，359° 与 3° 的平均角是多少？直觉上应为 1°，但直接使用 $\frac{1}{n}\sum \boldsymbol{x}$ 会得到 181°。

一种常用方法是分别求角度正弦和余弦的平均，再用双参数反正切 atan2 确定平均方向：

$$\bar{\theta} = atan2\left(\frac{\sum_{i=1}^{n} \sin\theta_i}{n}, \frac{\sum_{i=1}^{n} \cos\theta_i}{n}\right)$$

`UnscentedKalmanFilter.__init__()` 为用于计算状态均值的函数提供了参数 `x_mean_fn`，同时为用于计算测量值均值的函数提供了参数 `z_mean_fn`。我们将把这些函数分别编码为：

```
def state_mean(sigmas, Wm):
    x = np.zeros(3)

    sum_sin = np.sum(np.dot(np.sin(sigmas[:, 2]), Wm))
    sum_cos = np.sum(np.dot(np.cos(sigmas[:, 2]), Wm))
    x[0] = np.sum(np.dot(sigmas[:, 0], Wm))
    x[1] = np.sum(np.dot(sigmas[:, 1], Wm))
    x[2] = atan2(sum_sin, sum_cos)
    return x
```

```
def z_mean(sigmas, Wm):
    z_count = sigmas.shape[1]
    x = np.zeros(z_count)

    for z in range(0, z_count, 2):
        sum_sin = np.sum(np.dot(np.sin(sigmas[:, z+1]), Wm))
        sum_cos = np.sum(np.dot(np.cos(sigmas[:, z+1]), Wm))

        x[z] = np.sum(np.dot(sigmas[:,z], Wm))
        x[z+1] = atan2(sum_sin, sum_cos)
    return x
```

这些函数利用 NumPy 的三角函数可直接作用于数组的特性，再用 dot 计算加权和。对于这里的一维数组，dot 将对应元素相乘后求和，并不是仅执行逐元素乘法。NumPy 的底层数值运算由编译代码实现，因此这种写法通常比等价的 Python 循环快得多。

至此便可实现 UKF。不过，设计时我并不是一次就从零写出了上述全部函数。我先用预先设定的地标构建基本 UKF，确认有效后，再逐步补齐细节。“如果看到不同的地标怎么办？”促使我让测量函数接受地标数组。“角度残差怎样计算？”促使我编写角度归一化代码。“一组角度怎样求均值？”则让我查阅维基百科并实现相应算法。不必被问题吓倒：先完成能够完成的部分，再逐个提出问题、解决问题。

你已经看过 UKF 的实现方式了，所以我就不详细介绍了。这里有两个新的要点需要说明。在构建sigma点和滤波器时，我们必须提供自己编写的用于计算残差和均值的函数。

```
points = SigmaPoints(n=3, alpha=.00001, beta=2, kappa=0,
subtract=residual_x)
ukf = UKF(dim_x=3, dim_z=2, fx=fx, hx=Hx, dt=dt, points=points,
x_mean_fn=state_mean, z_mean_fn=z_mean,
residual_x=residual_x, residual_z=residual_h)
```

接下来，我们需要将额外的数据传递给 `f(x, dt)` 和 `h(x)` 函数。我们希望使用 `move(x, dt, u, wheelbase)` 来处理 `f(x, dt)`，用 `Hx(x, landmarks)` 来处理 `h(x)`。要做到这一点，只需将额外的参数作为关键字参数传递给 `predict()` 和 `update()` 即可：

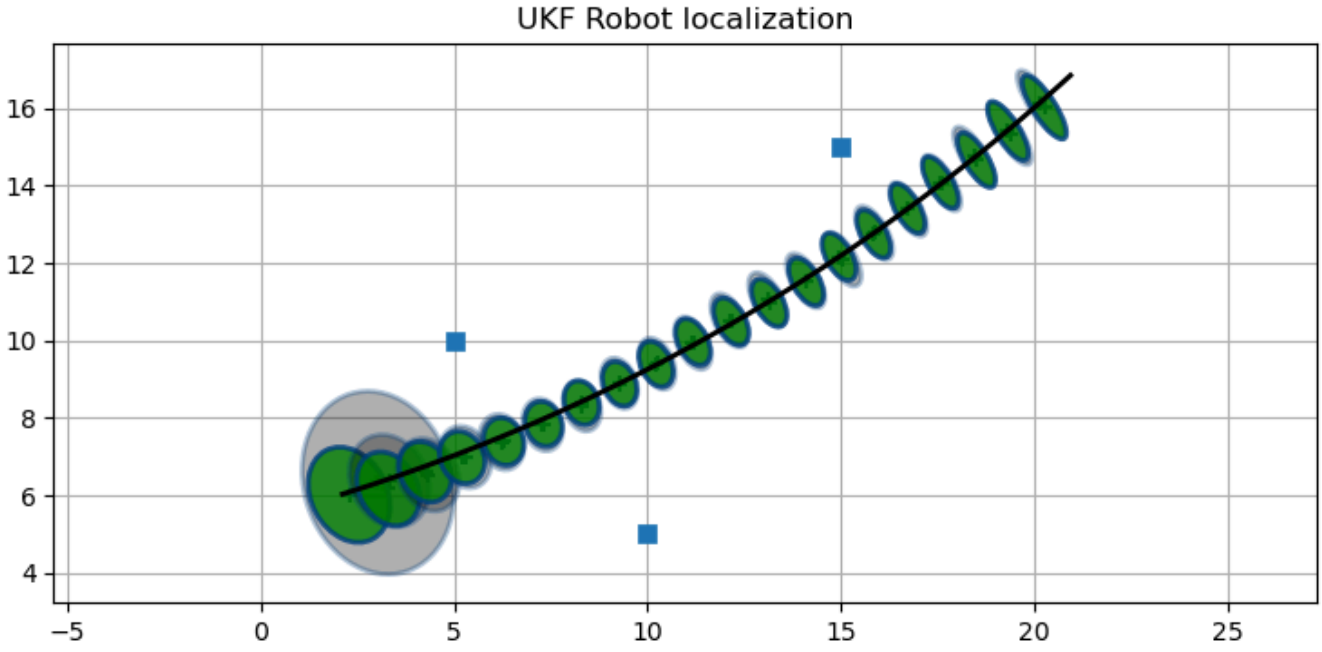
```
ukf.predict(u=u, wheelbase=wheelbase)
ukf.update(z, landmarks=landmarks)
```

其余代码运行仿真并绘图。变量 landmarks 存放地标坐标。仿真每秒更新机器人位置 10 次，UKF 则每秒运行一次。这里没有用龙格—库塔法积分运动微分方程，采用较小的仿真步长有助于更准确地描述运动。

```
from filterpy.stats import plot_covariance_ellipse

dt = 1.0
wheelbase = 0.5

def run_localization(
    cmds, landmarks, sigma_vel, sigma_steer, sigma_range,
    sigma_bearing, ellipse_step=1, step=10):

    plt.figure()
    points = MerweScaledSigmaPoints(n=3, alpha=.00001, beta=2, kappa=0,
                                    subtract=residual_x)
    ukf = UKF(dim_x=3, dim_z=2*len(landmarks), fx=move, hx=Hx,
              dt=dt, points=points, x_mean_fn=state_mean,
              z_mean_fn=z_mean, residual_x=residual_x,
              residual_z=residual_h)
```

```
    ukf.x = np.array([2, 6, .3])
    ukf.P = np.diag([.1, .1, .05])
    ukf.R = np.diag([sigma_range**2,
                     sigma_bearing**2]*len(landmarks))
    ukf.Q = np.eye(3)*0.0001

    sim_pos = ukf.x.copy()

    # plot landmarks
    if len(landmarks) > 0:
        plt.scatter(landmarks[:, 0], landmarks[:, 1],
                    marker='s', s=60)

    track = []
    for i, u in enumerate(cmds):
        sim_pos = move(sim_pos, dt/step, u, wheelbase)
        track.append(sim_pos)

        if i % step == 0:
            ukf.predict(u=u, wheelbase=wheelbase)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ukf.x[0], ukf.x[1]), ukf.P[0:2, 0:2], std=6,
                     facecolor='k', alpha=0.3)

            x, y = sim_pos[0], sim_pos[1]
            z = []
            for lmark in landmarks:
                dx, dy = lmark[0] - x, lmark[1] - y
                d = sqrt(dx**2 + dy**2) + randn()*sigma_range
                bearing = atan2(lmark[1] - y, lmark[0] - x)
                a = (normalize_angle(bearing - sim_pos[2] +
                     randn()*sigma_bearing))
                z.extend([d, a])
            ukf.update(z, landmarks=landmarks)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ukf.x[0], ukf.x[1]), ukf.P[0:2, 0:2], std=6,
                     facecolor='g', alpha=0.8)
    track = np.array(track)
    plt.plot(track[:, 0], track[:,1], color='k', lw=2)
    plt.axis('equal')
    plt.title("UKF Robot localization")
    plt.show()
    return ukf
```

```
landmarks = np.array([[5, 10], [10, 5], [15, 15]])
cmds = [np.array([1.1, .01])] * 200
ukf = run_localization(
    cmds, landmarks, sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1)
print('Final P:', ukf.P.diagonal())
```




```
Final P: [0.0092 0.0187 0.0007]
```

其余代码运行仿真并绘图，landmarks 保存地标坐标。机器人仿真位置每秒更新 10 次，UKF 则每秒运行一次。这样安排有两个原因：首先，较小仿真步长有助于提高运动模拟的精度；其次，嵌入式系统通常算力有限，因此应根据实际需要选择滤波器更新频率。

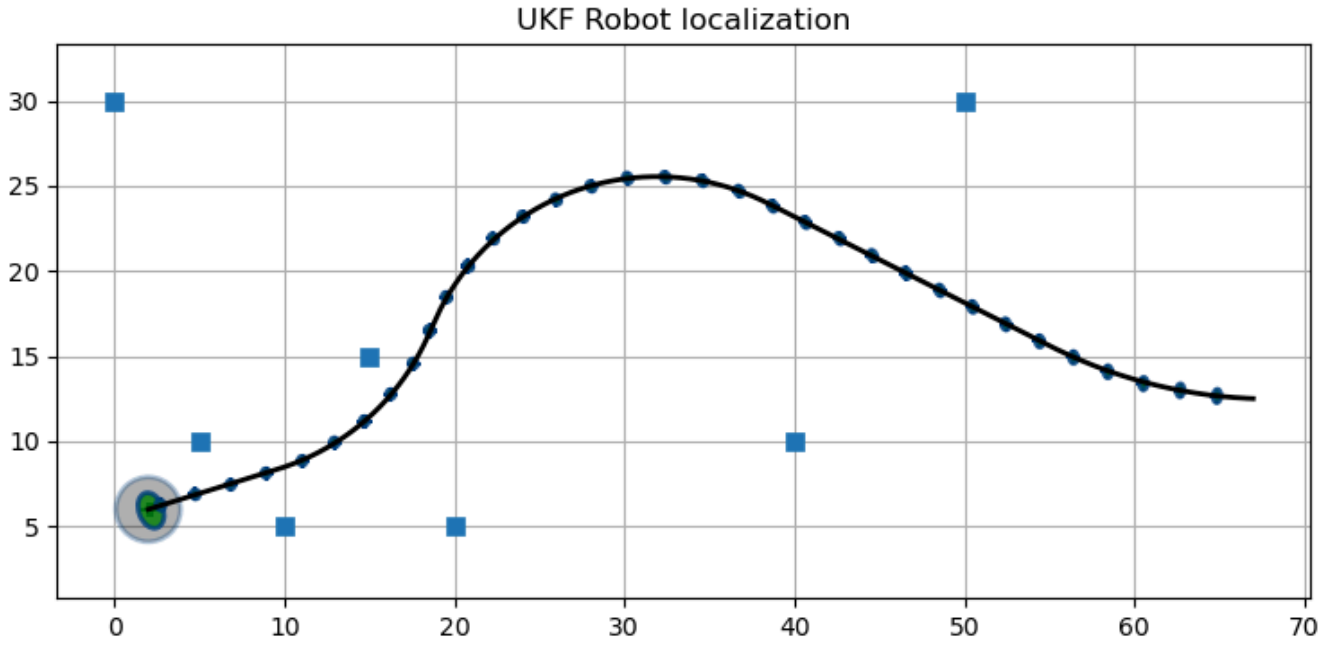
操控机器人

前面的转向仿真不够真实：速度与转向角始终不变，对滤波器而言难度不大。可以构建带 PID 控制的复杂机器人仿真，不过这里仅用 NumPy 的 linspace 生成变化的转向指令。由于机器人比第一个例子走得更远，还要增加一些地标。

```
landmarks = np.array([[5, 10], [10, 5], [15, 15], [20, 5],
                      [0, 30], [50, 30], [40, 10]])
dt = 0.1
wheelbase = 0.5
sigma_range=0.3
sigma_bearing=0.1

def turn(v, t0, t1, steps):
  return [[v, a] for a in np.linspace(
                np.radians(t0), np.radians(t1), steps)]

# accelerate from a stop
cmds = [[v, .0] for v in np.linspace(0.001, 1.1, 30)]
cmds.extend([cmds[-1]]*50)

# turn left
v = cmds[-1][0]
cmds.extend(turn(v, 0, 2, 15))
cmds.extend([cmds[-1]]*100)

#turn right
cmds.extend(turn(v, 2, -2, 15))
cmds.extend([cmds[-1]]*200)

cmds.extend(turn(v, -2, 0, 15))
cmds.extend([cmds[-1]]*150)
```

```
cmds.extend(turn(v, 0, 1, 25))
cmds.extend([cmds[-1]]*100)
```

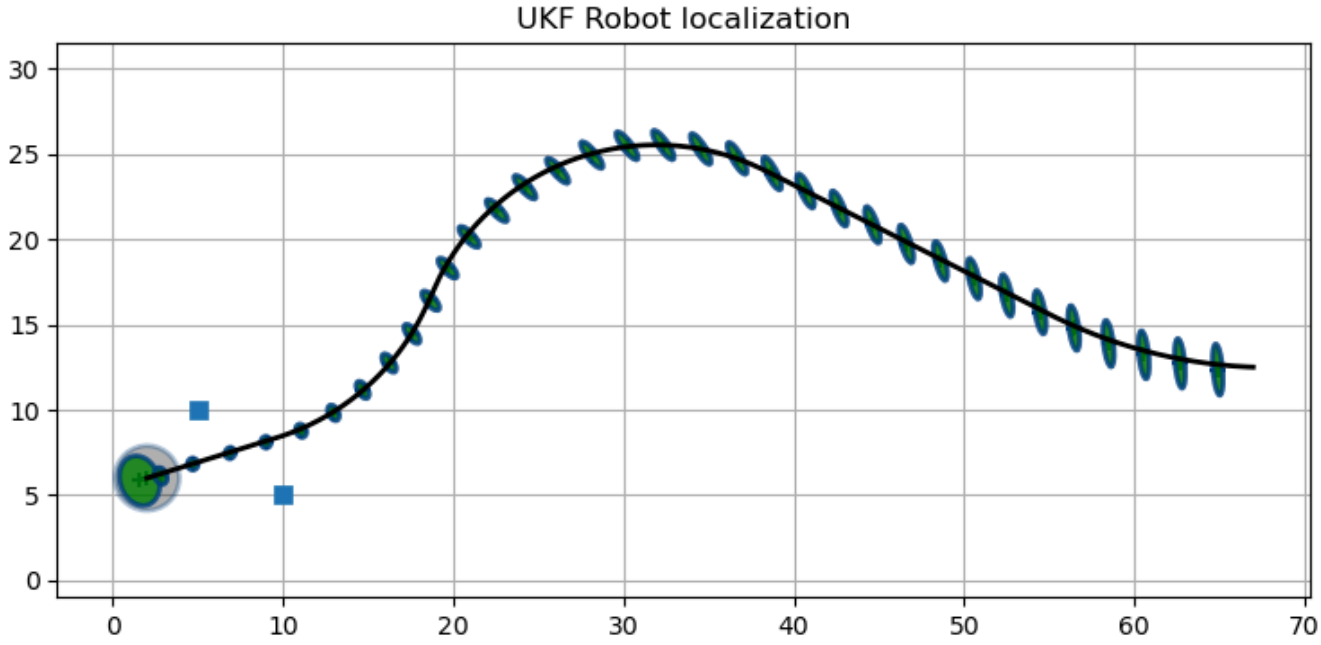
```
ukf = run_localization(
    cmds, landmarks, sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1, step=1,
    ellipse_step=20)
print('final covariance', ukf.P.diagonal())
```




```
final covariance [0.0013 0.0043 0.0004]
```

估计不确定性很快变小。图中即使用 6σ 的协方差轮廓绘制椭圆，椭圆仍小得难以看清。若只保留起点附近的两个地标，定位误差就会更大；随着机器人远离这些地标，误差还会逐渐增加。

```
ukf = run_localization(
    cmds, landmarks[0:2], sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1, step=1,
    ellipse_step=20)
print('final covariance', ukf.P.diagonal())
```




```
final covariance [0.0026 0.0657 0.0008]
```

讨论

你对本章的感受，可能取决于过去实现非线性滤波器的经验。初次接触时，计算 $2n+1$ 个 sigma 点，再编写 $f(x)$ 与 $h(x)$，或许显得繁琐。我确实花了不少时间才正确处理角度的周期性运算。不过，熟悉扩展卡尔曼滤波器（EKF）的读者可能会觉得轻松得多。UKF 的函数实现虽有些细节要处理，基本概念却比较直接；对同一问题，EKF 往往还需要更复杂的数学推导。有些方程没有闭式解，必须采用数值或迭代方法。

UKF 相比 EKF 的优势不仅是实现相对方便。虽然尚未学习 EKF，这里可以先作直观比较：EKF 在当前估计附近进行局部线性化，UKF 则用 $2n+1$ 个 sigma 点近似传播均值与协方差。因此，在一些非线性较强的问题中，UKF 往往更准确。不过，这不是普遍保证；具体性能仍取决于模型、噪声、参数、非线性程度与数值实现，需要针对问题验证。

因此，我建议先尝试实现 UKF。如果滤波器发散会造成严重后果，例如人员伤亡、重大经济损失或发电厂爆炸，当然需要深入分析并开展实验，以选择最合适的滤波器。这些内容超出了本书的范围，相关理论需要在研究生阶段进一步学习。

最后，本章介绍的 UKF 并不是 sigma 点滤波器的唯一形式。这里采用的是 Julier 的缩放无迹滤波，并使用 Van der Merwe 在 2004 年博士论文中给出的参数化方式。查阅 Julier、Van der Merwe、Uhlmann 和 Wan 的研究，还会看到一系列选点与赋权方式不同的 sigma 点滤波器。其他选择也不止于此，例如某些 SVD 卡尔曼滤波方法利用奇异值分解近似计算分布的均值与协方差。因此，应将本章视为 sigma 点滤波的入门，而非对所有方法的完整论述。

参考文献

- [1] 鲁道夫·范德梅尔韦。《用于动态状态空间模型中概率推断的sigma 点卡尔曼滤波器》，博士论文（2004年）。
- [2] Simon J. Julier. “The Scaled Unscented Transformation”（缩放无迹变换）. Proceedings of the American Control Conference, Vol. 6. IEEE, 2002.
- [3] http://www.esdradar.com/brochures/Compact%20Tracking%2037250X.pdf
- [4] Simon J. Julier and Jeffrey Uhlmann. “A New Extension of the Kalman Filter to Nonlinear Systems”（卡尔曼滤波面向非线性系统的一种新扩展）. Proc. SPIE 3068, Signal Processing, Sensor Fusion, and Target Recognition VI, p. 182, July 28, 1997.
- [5] 乔列斯基分解。维基百科。http://en.wikipedia.org/wiki/Cholesky_decomposition

第 11 章：扩展卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

我们已经建立了线性卡尔曼滤波的理论，并在前两章开始讨论如何将卡尔曼滤波用于非线性问题。本章学习扩展卡尔曼滤波器（EKF）。EKF 在当前估计点附近对系统作局部线性化，再利用线性卡尔曼滤波的形式进行计算。它是最早用于非线性滤波的方法之一，至今仍是最常用的方法之一。

EKF 为滤波器的设计者带来了巨大的数学挑战；这是本书中最具有挑战性的章节。我尽一切努力避免使用 EKF，而倾向于采用其他用于解决非线性问题的技术。然而，这个主题却是不可避免的；所有经典的论文以及该领域当前的大部分研究都使用了 EKF。即使你在自己的工作中不使用 EKF，你也必须熟悉这个主题，才能理解相关文献。

线性化卡尔曼滤波器

卡尔曼滤波器使用线性方程来进行计算，因此无法处理非线性问题。非线性问题可能源于两个方面：首先，系统模型本身可能是非线性的。例如，物体在大气中下落时，会受到空气阻力的影响，从而降低其加速度。而空气阻力系数又取决于物体的运动速度。这种复杂的动态过程无法用线性方程来描述。其次，测量结果本身也可能具有非线性特征。例如，雷达可以提供目标的距离和方位信息。我们需要利用三角函数来计算目标的位置，而三角函数本身就是非线性的函数。

对于线性滤波器而言，其过程模型和测量模型对应的方程如下：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \boldsymbol{w}_x$$
$$\mathbf{z} = \mathbf{H}\mathbf{x} + \boldsymbol{w}_z$$

其中，$\mathbf{A}$ 是系统动力学矩阵。利用“卡尔曼滤波的数学原理”一章介绍的状态空间方法，可将这些方程转换为

$$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x}$$
$$\mathbf{z} = \mathbf{H}\mathbf{x}$$

。

其中 $\mathbf{F}$ 是状态转移基本矩阵。测量噪声 $\boldsymbol{w}_z$ 与过程噪声 $\boldsymbol{w}_x$ 的统计特性分别并入 $\mathbf{R}$ 和 $\mathbf{Q}$。这一组方程利用第 $\boldsymbol{k}$ 步的测量和第 $\boldsymbol{k-1}$ 步的状态估计，计算第 $\boldsymbol{k}$ 步的状态。前面的章节使用牛顿运动学能够描述的问题，以建立直觉并减少数学负担；依据高中物理，我们已经知道怎样设计 $\mathbf{F}$。

对于非线性模型，用非线性函数 $\boldsymbol{f}(\mathbf{x}, \mathbf{u})$ 代替线性表达式 $\mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$，用非线性函数 $\boldsymbol{h}(\mathbf{x})$ 代替 $\mathbf{H}\mathbf{x}$：

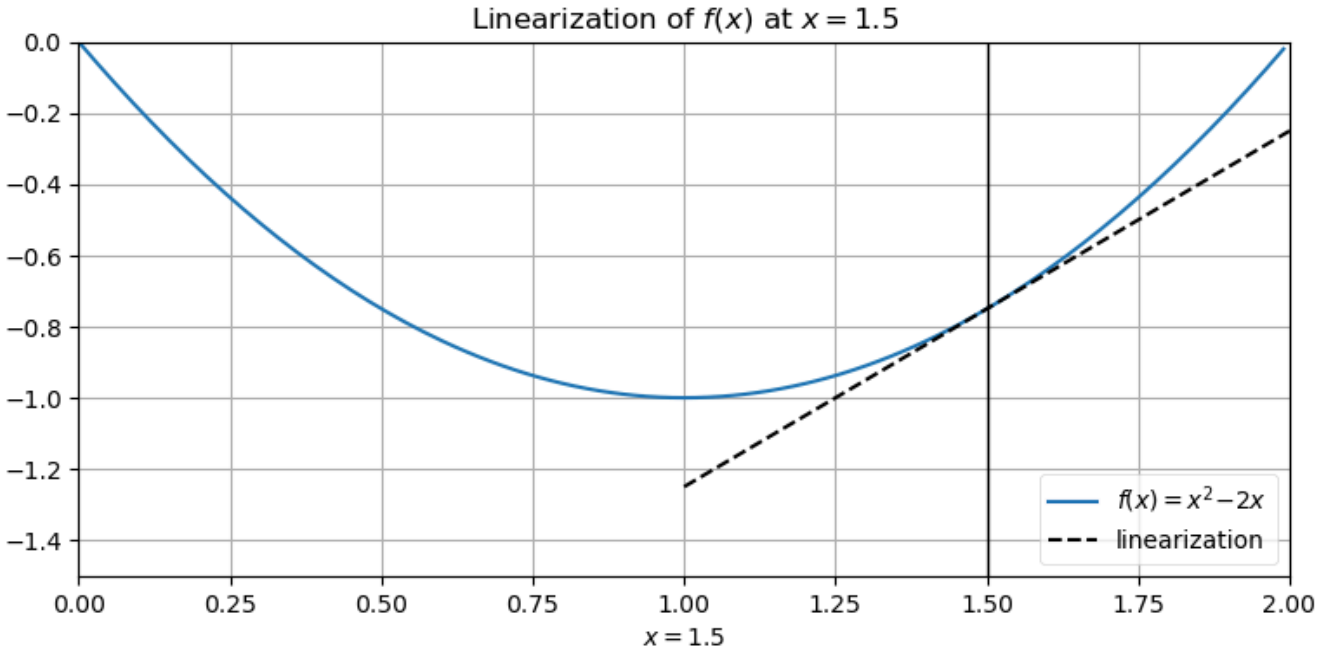
$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) + w_x$$
$$\mathbf{z} = h(\mathbf{x}) + w_z$$

你可以想象，我们可能会尝试找到一组新的卡尔曼滤波方程，以最优地解决这些问题。但如果你还记得“非线性滤波”章节中的图表，就会明白：当将一个高斯分布通过非线性函数处理后，得到的概率分布就不再具有高斯特性了。因此，这种方法是行不通的。

EKF 并没有改变卡尔曼滤波器的线性方程。相反，它是在当前估计值处将非线性方程线性化，并将这种线性化应用于线性卡尔曼滤波器中。

线性化就是在指定点附近，寻找与曲线局部最接近的直线，也就是切线。下图在 $x = 1.5$ 处对抛物线 $f(x) = x^2 - 2x$ 进行线性化。

```
import kf_book.ekf_internal as ekf_internal
ekf_internal.show_linearization()
```



如果上面的曲线代表了该过程模型，那么虚线则代表了该曲线在 $x = 1.5$ 这一点的线性化表达。

我们通过对系统求导来使其线性化，求导的过程实际上就是确定曲线的斜率。

$$f(x) = x^2 - 2x$$
$$\frac{df}{dx} = 2x - 2$$

然后在 x 处求值：

$$\begin{aligned} m &= f'(x = 1.5) \\ &= 2(1.5) - 2 \\ &= 1 \end{aligned}$$

微分方程组的线性化也类似。分别对 $\boldsymbol{f(\mathbf{x}, \mathbf{u})}$ 和 $\boldsymbol{h(\mathbf{x})}$ 求偏导，得到 $\mathbf{F}$ 和 $\mathbf{H}$，并在 $\mathbf{x}_t$ 和 $\mathbf{u}_t$ 处求值。向量值函数各分量关于各输入变量的一阶偏导数组成的矩阵，称为雅可比矩阵。由此得到离散状态转移矩阵与测量模型矩阵：

$$\mathbf{F} = \frac{\partial f(\mathbf{x}_t, \mathbf{u}_t)}{\partial \mathbf{x}}\Big|_{\mathbf{x}_t, \mathbf{u}_t}$$

$$\mathbf{H} = \frac{\partial h(\bar{\mathbf{x}}_t)}{\partial \bar{\mathbf{x}}}\Big|_{\bar{\mathbf{x}}_t}$$

这就得到了 EKF 的相关方程式。我把那些与线性滤波器不同的部分用方框标了出来：

linear Kalman filter	EKF
	$\boxed{\mathbf{F} = \frac{\partial f(\mathbf{x}_t, \mathbf{u}_t)}{\partial \mathbf{x}}\Big|_{\mathbf{x}_t, \mathbf{u}_t}}$
$\bar{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u}$	$\boxed{\bar{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})}$
$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$	$\bar{\mathbf{P}} = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}$
	$\boxed{\mathbf{H} = \frac{\partial h(\bar{\mathbf{x}}_t)}{\partial \bar{\mathbf{x}}}\Big|_{\bar{\mathbf{x}}_t}}$
$\mathbf{y} = \mathbf{z} - \mathbf{H}\bar{\mathbf{x}}$	$\mathbf{y} = \mathbf{z} - \boxed{h(\bar{x})}$
$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}$	$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^\mathsf{T}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}$
$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$	$\mathbf{x} = \bar{\mathbf{x}} + \mathbf{K}\mathbf{y}$
$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}$	$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}$

通常不直接用 $\mathbf{F}\mathbf{x}$ 传播 EKF 的状态均值，因为局部线性近似可能引入误差，而是用非线性模型计算 $\bar{\mathbf{x}}$。若模型以连续时间微分方程给出，可采用欧拉法、龙格－库塔法等数值积分方法；这里将该状态传播过程简写为 $\bar{\mathbf{x}} = \boldsymbol{f(\mathbf{x}, \mathbf{u})}$。同样，计算残差时也不用 $\mathbf{H}\bar{\mathbf{x}}$，而使用非线性测量函数 $\boldsymbol{h(\bar{\mathbf{x}})}$。

我认为，理解 EKF 最简单的方法就是先通过一个例子来了解。之后，你可能还想再回来阅读这一部分内容。

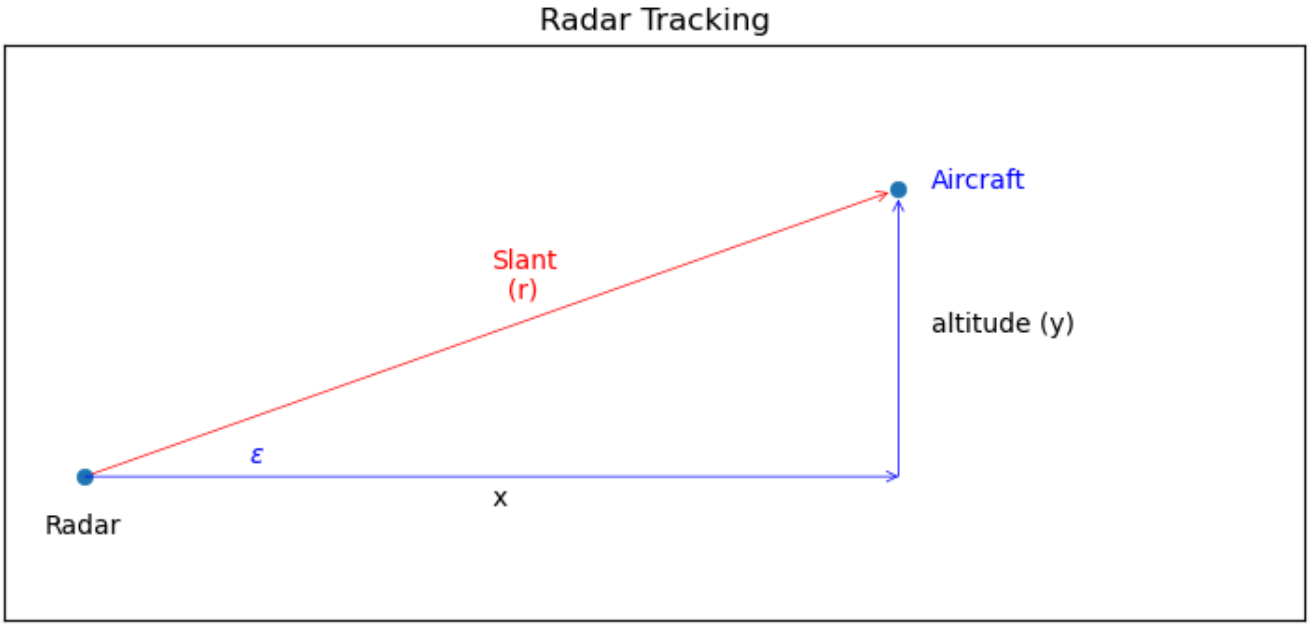
示例：追踪一架飞机

这个例子使用了基于地面的雷达来跟踪一架飞机。在上一章中，我们已经为这个问题实现了一个 UKF 滤波器。现在，我们将为同一个问题实现一个 EKF 滤波器，这样就能比较两种滤波器的性能以及实现它们所需的努力程度了。

雷达发射无线电波束，并接收目标返回的回波。波束路径上的物体会将部分信号反射回雷达。测量发射到回波返回的往返时间，就能计算斜距，即雷达站到目标的直线距离。

下图展示了雷达的斜距 $\boldsymbol{r}$、仰角 $\boldsymbol{\epsilon}$ 与飞机的水平位置 $\boldsymbol{x}$ 及高度 $\boldsymbol{y}$ 之间的关系：

```
ekf_internal.show_radar_chart()
```



这就得到了以下等式：

$$\epsilon = \tan^{-1} \frac{y}{x}$$
$$r^2 = x^2 + y^2$$

设计状态变量

我们希望追踪在恒定速度和恒定高度下飞机的位置，同时还需要测量到该飞机的斜距。这意味着我们需要三个状态变量：水平距离、水平速度和高度。

$$\mathbf{x} = \begin{bmatrix} \mathbf{distance} \\ \mathbf{velocity} \\ \mathbf{altitude} \end{bmatrix} = \begin{bmatrix} x \\ \dot{x} \\ y \end{bmatrix}$$

设计过程模型

假设飞机是一个服从牛顿运动方程的运动学系统。前几章已经使用过这一模型，因此也许可以直接看出，我们需要下面的状态转移矩阵：

$$\mathbf{F} = \left[\begin{array}{cc|c} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ \hline 0 & 0 & 1 \end{array}\right]$$

我将该矩阵划分成了几个区块。其中，左上角的区块对应的是 $\boldsymbol{x}$ 的恒速运动模型；右下角的区块则对应的是 $\boldsymbol{y}$ 的恒定位置模型。

不过，让我们来练习如何找到这些矩阵吧。我们用一组微分方程来描述各种系统。我们需要的是形如这样的方程：

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{w}$$

，其中 $\mathbf{w}$ 为系统噪声。

变量 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 是相互独立的，因此我们可以分别对它们进行计算。一维运动中的微分方程如下：

$$\begin{aligned}\boldsymbol{v} &= \dot{\boldsymbol{x}} \\ \boldsymbol{a} &= \ddot{\boldsymbol{x}} = 0\end{aligned}$$

现在，我们将这些微分方程转化为状态空间形式。如果这是一个二阶或更高阶的微分系统，那么我们必须先将其转化为等价的一阶方程组。由于这些方程都是一阶的，因此我们可以直接将它们表示为状态空间矩阵形式。

$$\begin{aligned}\begin{bmatrix}\dot{\boldsymbol{x}} \\ \ddot{\boldsymbol{x}}\end{bmatrix} &= \begin{bmatrix}0 & 1 \\ 0 & 0\end{bmatrix}\begin{bmatrix}\boldsymbol{x} \\ \dot{\boldsymbol{x}}\end{bmatrix} \\ \dot{\mathbf{x}} &= \mathbf{A}\mathbf{x}\end{aligned}$$

，其中 $\mathbf{A} = \begin{bmatrix}0 & 1 \\ 0 & 0\end{bmatrix}$。

$\mathbf{A}$ 是系统动力学矩阵，描述一组连续时间线性微分方程。由它计算状态转移矩阵 $\mathbf{F}$。$\mathbf{F}$ 描述相应的离散线性关系，用来计算经过时间间隔 $\boldsymbol{\Delta t}$ 后的状态 $\mathbf{x}$。

计算 $\mathbf{F}$ 的常用方法是利用矩阵指数的幂级数展开式：

$$\mathbf{F}(\Delta t) = e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t + \frac{(\mathbf{A}\Delta t)^2}{2!} + \frac{(\mathbf{A}\Delta t)^3}{3!} + \dots$$

$\mathbf{A}^2 = \begin{bmatrix}0 & 0 \\ 0 & 0\end{bmatrix}$，因此 $\mathbf{A}$ 的所有高次幂也都是 $\mathbf{0}$。所以，其幂级数展开形式为：

$$\begin{aligned}\mathbf{F} &= \mathbf{I} + \mathbf{A}t + 0 \\ &= \begin{bmatrix}1 & 0 \\ 0 & 1\end{bmatrix} + \begin{bmatrix}0 & 1 \\ 0 & 0\end{bmatrix}\Delta t \\ \mathbf{F} &= \begin{bmatrix}1 & \Delta t \\ 0 & 1\end{bmatrix}\end{aligned}$$

这与运动学方程得出的结果是一样的！除了用来演示如何从线性微分方程中求出状态转移矩阵之外，这个练习其实没有必要。在本章的最后，我们将通过一个例子来进一步应用这一技巧。

设计测量模型

测量函数将先验状态估计 $\bar{\mathbf{x}}$ 映射为预测斜距。由勾股定理可得：

$$h(\bar{\mathbf{x}}) = \sqrt{x^2 + y^2}$$

由于含有平方根，斜距与地面位置之间的关系是非线性的。我们在 $\mathbf{x}_t$ 处计算有关偏导数，对测量函数作局部线性化：

$$\mathbf{H} = \frac{\partial h(\bar{\mathbf{x}})}{\partial \bar{\mathbf{x}}}\Big|_{\bar{\mathbf{x}}_t}$$

向量值函数各分量关于各输入变量的一阶偏导数组成雅可比矩阵，其形式为：

$$\frac{\partial h}{\partial \bar{\mathbf{x}}} = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial x_2} & \cdots \\ \frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

也就是说，矩阵的各元素分别是函数 $\boldsymbol{h}$ 的各分量关于状态变量 $\boldsymbol{x}$ 的偏导数。本例中有：

$$\mathbf{H} = [\partial h/\partial x \quad \partial h/\partial \dot{x} \quad \partial h/\partial y]$$

逐项计算这些偏导数：

$$\begin{aligned} \frac{\partial h}{\partial x} &= \frac{\partial}{\partial x}\sqrt{x^2 + y^2} \\ &= \frac{x}{\sqrt{x^2 + y^2}} \end{aligned}$$

以及

$$\begin{aligned} \frac{\partial h}{\partial \dot{x}} &= \frac{\partial}{\partial \dot{x}}\sqrt{x^2 + y^2} \\ &= 0 \end{aligned}$$

还有

$$\begin{aligned} \frac{\partial h}{\partial y} &= \frac{\partial}{\partial y}\sqrt{x^2 + y^2} \\ &= \frac{y}{\sqrt{x^2 + y^2}} \end{aligned}$$

于是得到下面的测量雅可比矩阵。它有三个分量，依次对应水平位置、水平速度和高度：

$$\mathbf{H} = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & 0 & \frac{y}{\sqrt{x^2+y^2}} \end{bmatrix}$$

这些数学看起来可能令人望而却步，但退后一步就会发现，它们在做一件简单的事：飞机斜距的测量方程是非线性的，而线性卡尔曼滤波要求线性关系，因此需要用局部线性测量矩阵 $\mathbf{H}$ 来近似非线性测量函数。前面已说明，指定点处的切线斜率可以给出局部近似。对滤波器而言，该点由状态变量 $\mathbf{x}$ 决定，所以需要对斜距关于 $\mathbf{x}$ 求偏导。线性卡尔曼滤波中的 $\mathbf{H}$ 是运行前算好的常量；EKF 中的 $\mathbf{H}$ 则随着每一步的线性化点 $\bar{\mathbf{x}}$ 改变而重新计算。

为了更具体地说明这一点，现在让我们编写一个 Python 函数，用来计算该问题中 $\boldsymbol{h}$ 的雅可比矩阵。

```
from math import sqrt
def HJacobian_at(x):
    """ compute Jacobian of H matrix at x """

    horiz_dist = x[0]
    altitude   = x[2]
    denom = sqrt(horiz_dist**2 + altitude**2)
    return array ([[horiz_dist/denom, 0., altitude/denom]])
```

最后，让我们来看看 $h(\bar{\mathbf{x}})$ 的代码实现：

```
def hx(x):
    """ compute measurement for slant range that
    would correspond to state x.
    """

    return (x[0]**2 + x[2]**2) ** 0.5
```

现在，让我们来编写一个用于我们的雷达的模拟程序吧。

```
from numpy.random import randn
import math

class RadarSim:
    """ Simulates the radar signal returns from an object
    flying at a constant altityude and velocity in 1D.
    """

    def __init__(self, dt, pos, vel, alt):
        self.pos = pos
        self.vel = vel
        self.alt = alt
        self.dt = dt

    def get_range(self):
        """ Returns slant range to the object. Call once
        for each new measurement at dt time from last call.
        """

        # add some process noise to the system
        self.vel = self.vel  + .1*randn()
        self.alt = self.alt + .1*randn()
        self.pos = self.pos + self.vel*self.dt

        # add measurement noise
        err = self.pos * 0.05*randn()
        slant_dist = math.sqrt(self.pos**2 + self.alt**2)

        return slant_dist + err
```

设计过程与测量噪声

雷达测量目标距离。令测量噪声的标准差为 $\sigma_{range} = 5$ 米，则测量噪声协方差为：

$$\mathbf{R} = \begin{bmatrix}\sigma^2_{range}\end{bmatrix} = [25]$$

$\mathbf{Q}$ 的设计需要进一步说明。状态为 $\mathbf{x} = \begin{bmatrix}x & \dot{x} & y\end{bmatrix}^\mathsf{T}$，前两个分量是位置（水平距离）和水平速度，因此可用 Q_discrete_white_noise 计算 $\mathbf{Q}$ 左上角的子块。$\mathbf{x}$ 的第三个分量是高度，假设其过程噪声与水平运动的过程噪声独立，因此可将 $\mathbf{Q}$ 设计为以下分块形式：

$$\mathbf{Q} = \begin{bmatrix}\mathbf{Q_x} & 0 \\ 0 & \mathbf{Q_y}\end{bmatrix}$$

实现

FilterPy 提供 ExtendedKalmanFilter 类。它与前面使用的 KalmanFilter 类相似，但允许传入计算测量雅可比矩阵 $\mathbf{H}$ 的函数，以及非线性测量函数 $h(\mathbf{x})$。

我们首先导入该滤波器并对其进行创建。 x 的维度为 3，而 z 的维度为 1。

```
from filterpy.kalman import ExtendedKalmanFilter
rk = ExtendedKalmanFilter(dim_x=3, dim_z=1)
```

创建雷达模拟器：

```
radar = RadarSim(dt, pos=0., vel=100., alt=1000.)
```

我们将在飞机实际所在位置附近初始化该滤波器：

```
rk.x = array([radar.pos, radar.vel-10, radar.alt+100])
```

用前面得到的矩阵指数展开式设置状态转移矩阵；这里保留单位矩阵项及一阶项，后续项均为零：

```
dt = 0.05
rk.F = eye(3) + array([[0, 1, 0],
[0, 0, 0],
[0, 0, 0]])*dt
```

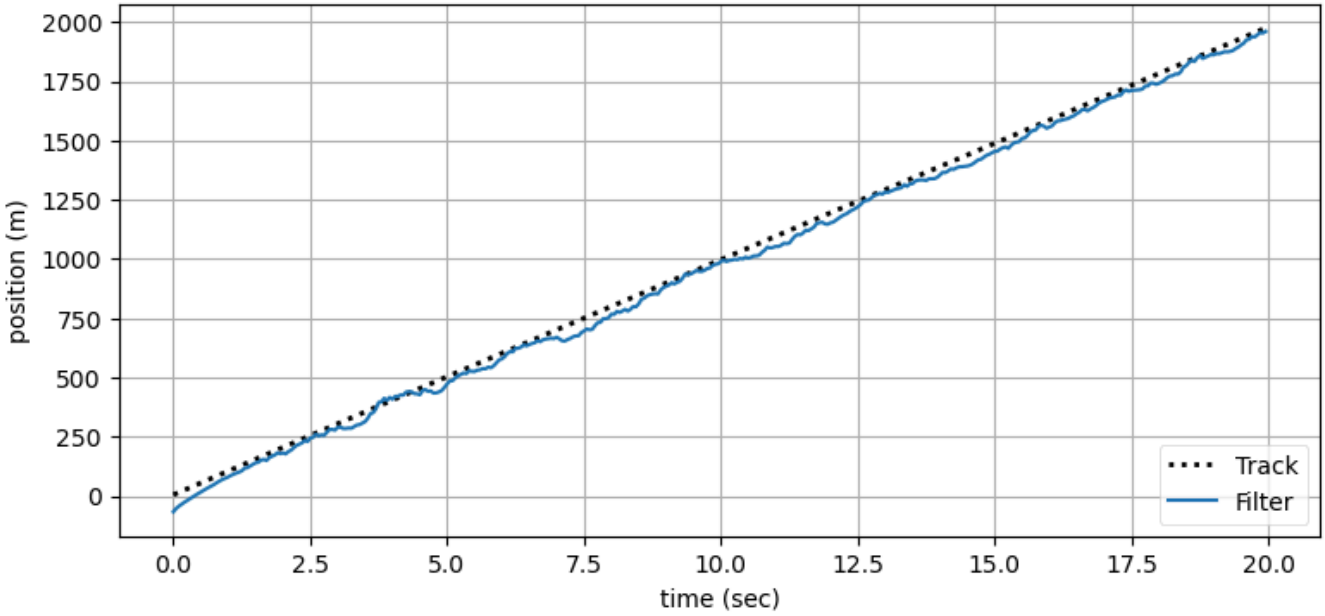
为 $\mathbf{R}$、$\mathbf{Q}$ 和 $\mathbf{P}$ 赋予合理数值后，就可以用一个简单循环运行滤波器。将计算测量雅可比矩阵 $\mathbf{H}$ 的函数与非线性测量函数 $\boldsymbol{h}(\boldsymbol{x})$ 一同传入 update 方法。

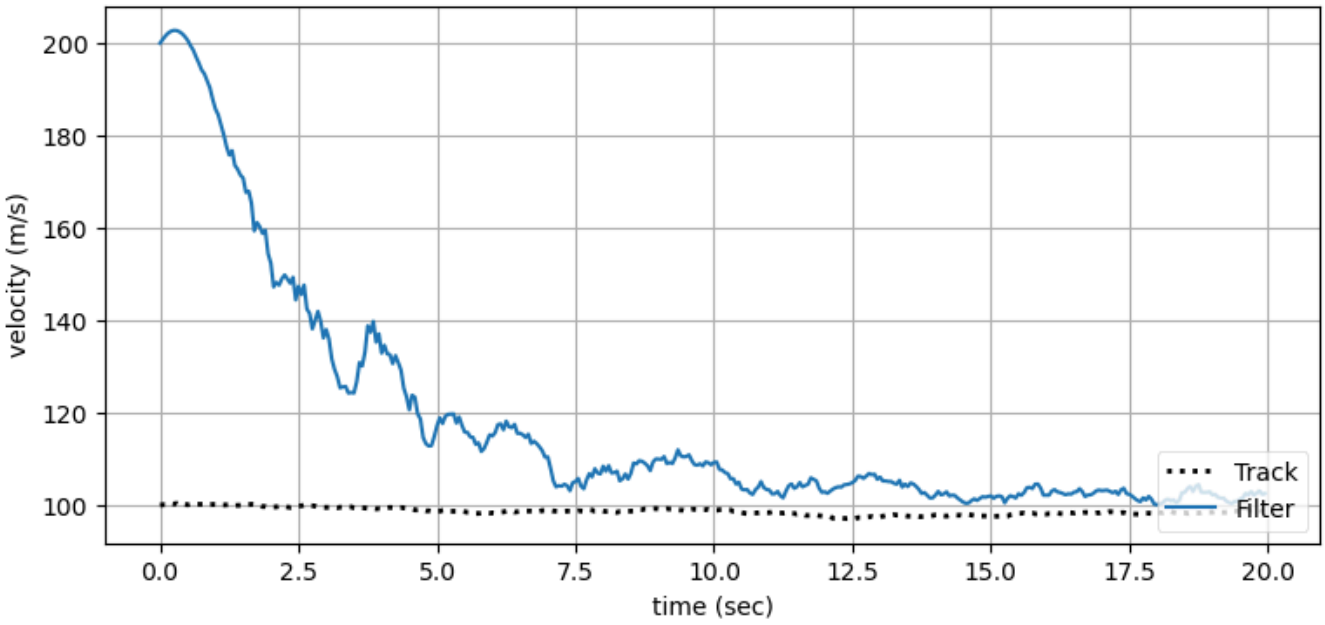
```
for i in range(int(20/dt)):
z = radar.get_range()
rk.update(array([z]), HJacobian_at, hx)
rk.predict()
```

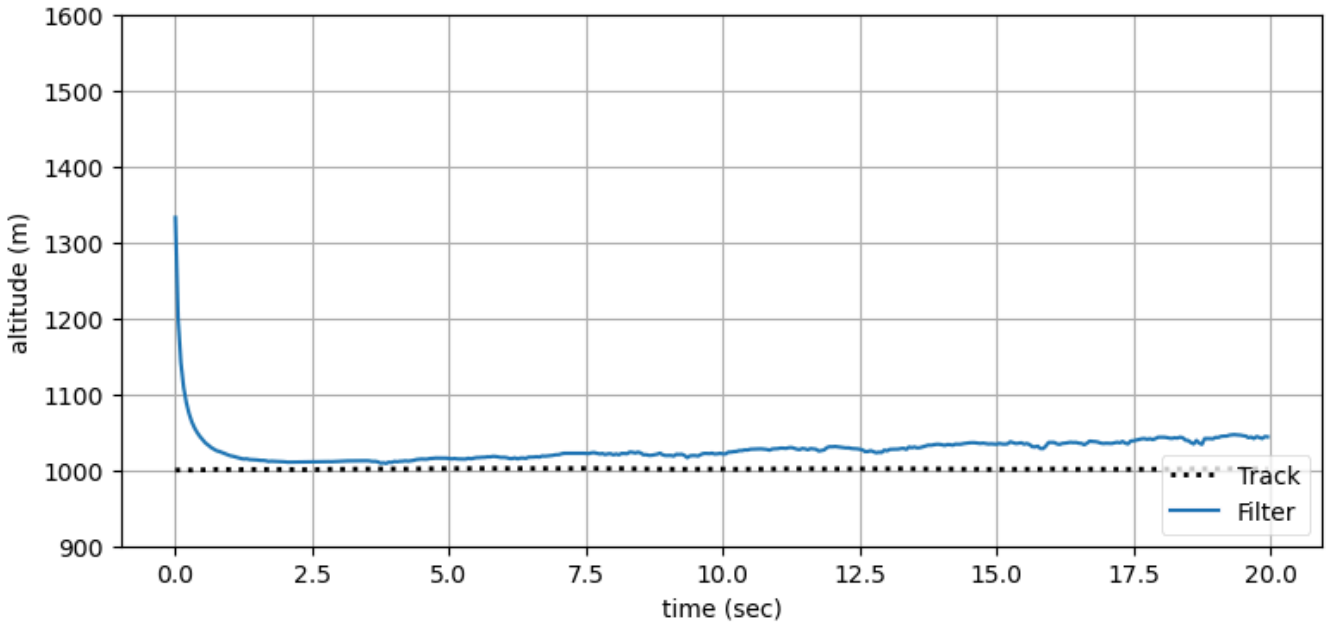
添加一些用于保存和绘制所得结果的样板代码：

```
from filterpy.common import Q_discrete_white_noise
from filterpy.kalman import ExtendedKalmanFilter
from numpy import eye, array, asarray
import numpy as np

dt = 0.05
rk = ExtendedKalmanFilter(dim_x=3, dim_z=1)
radar = RadarSim(dt, pos=0., vel=100., alt=1000.)

# make an imperfect starting guess
rk.x = array([radar.pos-100, radar.vel+100, radar.alt+1000])

rk.F = eye(3) + array([[0, 1, 0],
                       [0, 0, 0],
                       [0, 0, 0]]) * dt

range_std = 5. # meters
rk.R = np.diag([range_std**2])
rk.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
rk.Q[2,2] = 0.1
rk.P *= 50

xs, track = [], []
for i in range(int(20/dt)):
    z = radar.get_range()
    track.append((radar.pos, radar.vel, radar.alt))

    rk.update(array([z]), HJacobian_at, hx)
    xs.append(rk.x)
```

```
    rk.predict()

xs = asarray(xs)
track = asarray(track)
time = np.arange(0, len(xs)*dt, dt)
ekf_internal.plot_radar(xs, track, time)
```







使用 SymPy 计算雅可比矩阵

根据你对求导的熟悉程度，计算雅可比矩阵可能并不容易。即使这个例子对你而言很简单，问题稍复杂一些，求导计算也可能迅速变得繁琐。

如附录 A 中所解释的，我们可以使用 SymPy 软件包来计算雅可比矩阵。

```
import sympy
from IPython.display import display
sympy.init_printing(use_latex='mathjax')

x, x_vel, y = sympy.symbols('x, x_vel y')

H = sympy.Matrix([sympy.sqrt(x**2 + y**2)])

state = sympy.Matrix([x, x_vel, y])
J = H.jacobian(state)

display(state)
display(J)
```

$$\begin{bmatrix} x \\ x_{vel} \\ y \end{bmatrix}$$

$$\begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & 0 & \frac{y}{\sqrt{x^2+y^2}} \end{bmatrix}$$

这个结果与我们上面计算出的结果相同，而且我们付出的努力要少得多！

机器人定位

现在尝试一个实际问题。先提醒你，这一节比较难。许多书只介绍容易求解的教科书例题，读者读完后却仍不知道如何处理实际问题，因此这里采用一个更贴近实际的例子。

我们将探讨机器人的定位问题。我们已经在“无迹卡尔曼滤波”那一章中实现了这一功能。如果你还没有阅读那一章，我建议你现在就去看看。在这种情况下，机器人通过传感器来识别周围的地标，从而确定自己的位置。这可以是利用计算机视觉来识别树木、建筑物等地标的自动驾驶汽车；也可以是那些用于打扫房子的微型机器人，或是仓库里的作业机器人。

这款机器人的结构与汽车类似，也有四个轮子。它通过转动前轮来改变方向。这样一来，机器人在前进的同时，也会以后轴为中心进行旋转。这是一种非线性运动方式，我们需要对这种行为进行建模。

机器人配有传感器，能够测量已知地标相对于它的距离和方位角。该测量模型是非线性的：由相对位置计算距离会用到平方根，计算方位角会用到反三角函数；反过来，由距离和方位角计算坐标则会用到三角函数。

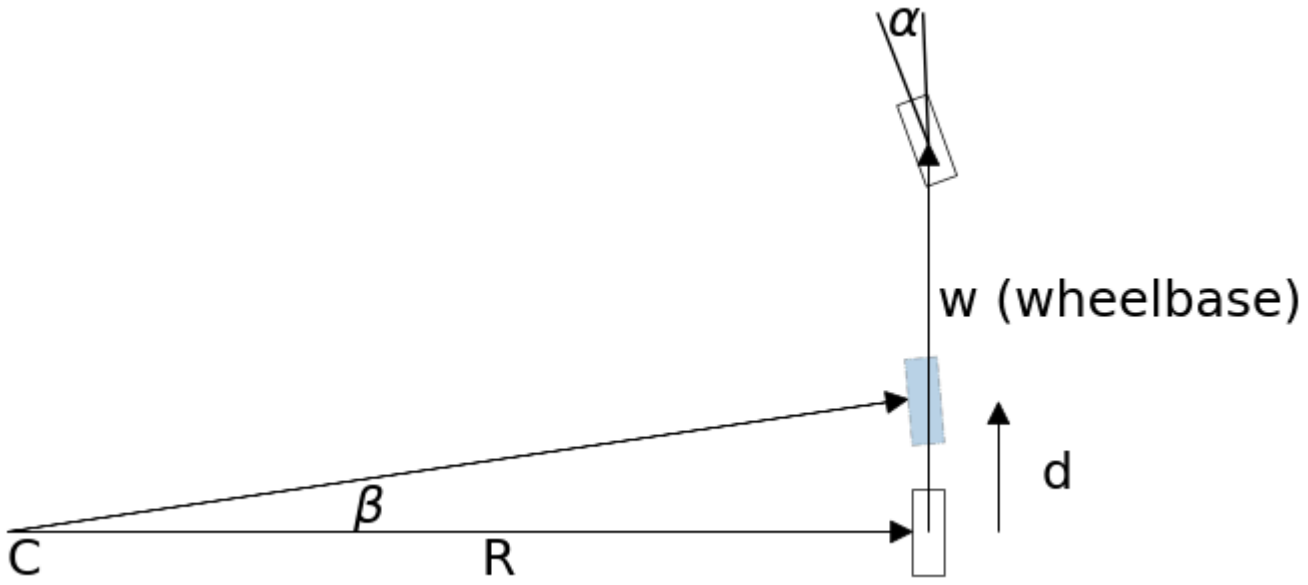
无论是过程模型还是测量模型，都属于非线性模型。而 EKF 能够同时处理这两种非线性问题，因此我们初步认为 EKF 是解决这个问题的一个可行方案。

机器人运动模型

简单来说，汽车在行驶过程中是通过转动前轮来实现转向的。汽车的前部会朝着车轮所指的方向移动，而整个车身则围绕后轮进行转动。不过，这一简单的机制实际上相当复杂：摩擦力会导致打滑现象；橡胶轮胎在不同速度下的表现也有所不同；此外，外侧轮胎和内侧轮胎的转动半径也必须有所不同。要准确描述汽车的转向过程，就需要用到复杂的微分方程。

对于速度较低的机器人应用来说，采用较为简单的自行车模型能够取得良好的效果。以下是该模型的示意图：

```
ekf_internal.plot_bicycle()
```



在无迹卡尔曼滤波这一章中，我们推导出了以下方程：

$$
\begin{aligned}
\beta &= \frac{d}{w}\tan(\alpha) \\
x &= x - R\sin(\theta) + R\sin(\theta+\beta) \\
y &= y + R\cos(\theta) - R\cos(\theta+\beta) \\
\theta &= \theta + \beta
\end{aligned}
$$

其中，θ 为机器人的航向角。

如果你对转向模型不感兴趣，不必详细了解这个模型。需要记住的是，运动模型具有非线性，因此必须在卡尔曼滤波中恰当地处理这种非线性。

设计状态变量

滤波器的状态包含机器人的位置 x, y 和航向角 θ:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

我们的控制输入 $\mathbf{u}$ 包括速度 v 和转向角 α。

$$\mathbf{u} = \begin{bmatrix} v \\ \alpha \end{bmatrix}$$

设计系统模型

我们将该系统建模为一个非线性运动模型，同时考虑了噪声的影响。

$$\bar{x} = f(x, u) + \mathcal{N}(0, Q)$$

将前面建立的机器人运动模型代入，可将状态预测展开为:

$$\begin{bmatrix} \bar{x} \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} -R\sin(\theta) + R\sin(\theta+\beta) \\ R\cos(\theta) - R\cos(\theta+\beta) \\ \beta \end{bmatrix}$$

我们通过计算 $f(x, u)$ 的雅可比矩阵来找到 $\mathbf{F}$ 的值。

$$\mathbf{F} = \frac{\partial f(x, u)}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial \theta} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial \theta} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} & \frac{\partial f_3}{\partial \theta} \end{bmatrix}$$

当我们进行这些计算后，我们得到的结果是

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & -R\cos(\theta) + R\cos(\theta+\beta) \\ 0 & 1 & -R\sin(\theta) + R\sin(\theta+\beta) \\ 0 & 0 & 1 \end{bmatrix}$$

我们可以用 SymPy 来再次检查我们的计算结果。

```
import sympy
from sympy.abc import alpha, x, y, v, w, R, theta
from sympy import symbols, Matrix
sympy.init_printing(use_latex="mathjax", fontsize='16pt')
time = symbols('t')
d = v*time
beta = (d/w)*sympy.tan(alpha)
r = w/sympy.tan(alpha)

fxu = Matrix([[x-r*sympy.sin(theta) + r*sympy.sin(theta+beta)],
              [y+r*sympy.cos(theta)- r*sympy.cos(theta+beta)],
              [theta+beta]])
F = fxu.jacobian(Matrix([x, y, theta]))
F
```

$$\begin{bmatrix}1 & 0 & -\frac{w\cos{(\theta)}}{\tan{(\alpha)}} + \frac{w\cos{\left(\frac{tv\tan{(\alpha)}}{w}+\theta\right)}}{\tan{(\alpha)}} \\ 0 & 1 & -\frac{w\sin{(\theta)}}{\tan{(\alpha)}} + \frac{w\sin{\left(\frac{tv\tan{(\alpha)}}{w}+\theta\right)}}{\tan{(\alpha)}} \\ 0 & 0 & 1\end{bmatrix}$$

那看起来有点复杂。我们可以使用 SymPy 来替换其中的各项：

```
# reduce common expressions
B, R = symbols('beta, R')
F = F.subs((d/w)*sympy.tan(alpha), B)
F.subs(w/sympy.tan(alpha), R)
```

$$\begin{bmatrix}1 & 0 & -R\cos{(\theta)} + R\cos{(\beta+\theta)} \\ 0 & 1 & -R\sin{(\theta)} + R\sin{(\beta+\theta)} \\ 0 & 0 & 1\end{bmatrix}$$

这一表达形式验证了前面雅可比矩阵的计算。

现在讨论噪声。这里噪声存在于控制输入中，因此首先是在控制空间里描述的。也就是说，我们下达指定速度和转向角，但必须把它们的不确定性转换为 x, y, θ 中的误差。实际系统中，这种转换可能随速度变化，所以每次预测都要重新计算。这里采用下面的噪声模型；真实机器人则需要选择准确描述其自身误差的模型。

$$\mathbf{M} = \begin{bmatrix}\sigma_{vel}^2 & 0 \\ 0 & \sigma_\alpha^2\end{bmatrix}$$

在线性情形下，可以使用熟悉的协方差传播形式 $\mathbf{FMF}^\mathsf{T}$，将控制输入的不确定性映射到状态空间。这里的运动模型是非线性的，因此采用关于控制输入的雅可比矩阵作一阶近似，记为 $\mathbf{V}$。下面给出其偏导形式及逐项展开的矩阵形式：

$$\mathbf{V} = \frac{\partial f(x, u)}{\partial u} = \begin{bmatrix}\frac{\partial f_1}{\partial v} & \frac{\partial f_1}{\partial \alpha} \\ \frac{\partial f_2}{\partial v} & \frac{\partial f_2}{\partial \alpha} \\ \frac{\partial f_3}{\partial v} & \frac{\partial f_3}{\partial \alpha}\end{bmatrix}$$

这些偏导数很难处理。让我们用 SymPy 来计算它们吧。

```
V = fxu.jacobian(Matrix([v, alpha]))
V = V.subs(sympy.tan(alpha)/w, 1/R)
V = V.subs(time*v/R, B)
V = V.subs(time*v, 'd')
V
```

$$\begin{bmatrix} t\cos(\beta+\theta) & \frac{d(\tan^2(\alpha)+1)\cos(\beta+\theta)}{\tan(\alpha)} - \frac{w(-\tan^2(\alpha)-1)\sin(\theta)}{\tan^2(\alpha)} + \frac{w(-\tan^2(\alpha)-1)\sin(\beta+\theta)}{\tan^2(\alpha)} \\ t\sin(\beta+\theta) & \frac{d(\tan^2(\alpha)+1)\sin(\beta+\theta)}{\tan(\alpha)} + \frac{w(-\tan^2(\alpha)-1)\cos(\theta)}{\tan^2(\alpha)} - \frac{w(-\tan^2(\alpha)-1)\cos(\beta+\theta)}{\tan^2(\alpha)} \\ \frac{t}{R} & \frac{d(\tan^2(\alpha)+1)}{w} \end{bmatrix}$$

这应该能让你明白，EKF 在数学上变得难以处理的速度有多快。

这样，我们就得到了预测方程的最终形式：

$$\begin{aligned} \bar{\mathbf{x}} &= \mathbf{x} + \begin{bmatrix} -R\sin(\theta) + R\sin(\theta+\beta) \\ R\cos(\theta) - R\cos(\theta+\beta) \\ \beta \end{bmatrix} \\ \bar{\mathbf{P}} &= \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{V}\mathbf{M}\mathbf{V}^\mathsf{T} \end{aligned}$$

这种线性化处理方式并非预测 $\mathbf{x}$ 的唯一方法。例如，我们可以使用龙格-库塔之类的数值积分技术来计算机器人的运动轨迹。当时间步长较大时，就需要采用这种做法。与卡尔曼滤波器相比，EKF 的处理方式则没有那么简单明了。在解决实际问题时，必须先用微分方程来精确描述系统，然后再确定最合适的求解方法。究竟采用哪种方法，取决于所需的精度、方程的非线性程度、处理器的性能限制以及数值稳定性方面的要求。

设计测量模型

机器人传感器对环境中多个已知地标提供带噪方位角和距离测量。测量模型必须把状态 $\begin{bmatrix} x & y & \theta \end{bmatrix}^\mathsf{T}$ 转换为到地标的距离与方位角。若 $\mathbf{p}$ 是地标的位置，距离 r 为：

$$r = \sqrt{(p_x - x)^2 + (p_y - y)^2}$$

传感器报告的是相对于机器人朝向的方位角，因此要从地标的全局方位角中减去机器人的航向角，才能得到预测传感器读数：

$$\phi = \arctan(\frac{p_y - y}{p_x - x}) - \theta$$

因此，测量模型 h 为：

$$\begin{aligned} \mathbf{z} &= h(\bar{\mathbf{x}}, \mathbf{p}) &&+ \mathcal{N}(0, R) \\ &= \begin{bmatrix} \sqrt{(p_x - x)^2 + (p_y - y)^2} \\ \arctan(\frac{p_y - y}{p_x - x}) - \theta \end{bmatrix} &&+ \mathcal{N}(0, R) \end{aligned}$$

该模型显然非线性，因此需要求出 h 的雅可比矩阵，在 $\mathbf{x}$ 处作局部线性化。下面用 SymPy 计算。

```
px, py = symbols('p_x, p_y')
z = Matrix([[sympy.sqrt((px-x)**2 + (py-y)**2)],
            [sympy.atan2(py-y, px-x) - theta]])
z.jacobian(Matrix([x, y, theta]))
```

$$\begin{bmatrix} \frac{-p_x+x}{\sqrt{(p_x-x)^2+(p_y-y)^2}} & \frac{-p_y+y}{\sqrt{(p_x-x)^2+(p_y-y)^2}} & 0 \\ -\frac{-p_y+y}{(p_x-x)^2+(p_y-y)^2} & -\frac{p_x-x}{(p_x-x)^2+(p_y-y)^2} & -1 \end{bmatrix}$$

现在，我们需要把这段代码写成一个 Python 函数。例如，我们可以这样写：

```
from math import sqrt

def H_of(x, landmark_pos):
    """ compute Jacobian of H matrix where h(x) computes
    the range and bearing to a landmark for state x """

    px = landmark_pos[0]
    py = landmark_pos[1]
    hyp = (px - x[0, 0])**2 + (py - x[1, 0])**2
    dist = sqrt(hyp)

    H = array(
        [[-(px - x[0, 0]) / dist, -(py - x[1, 0]) / dist, 0],
         [ (py - x[1, 0]) / hyp,  -(px - x[0, 0]) / hyp, -1]])
    return H
```

还需要定义一个函数，将系统状态映射为预测测量。

```
from math import atan2

def Hx(x, landmark_pos):
    """ takes a state variable and returns the measurement
    that would correspond to that state.
    """
    px = landmark_pos[0]
    py = landmark_pos[1]
    dist = sqrt((px - x[0, 0])**2 + (py - x[1, 0])**2)

    Hx = array([[dist],
                [atan2(py - x[1, 0], px - x[0, 0]) - x[2, 0]]])
    return Hx
```

设计测量噪声

可以合理地假设，距离和方位角的测量误差是相互独立的。因此，

$$\mathbf{R} = \begin{bmatrix} \sigma^2_{range} & 0 \\ 0 & \sigma^2_{bearing} \end{bmatrix}$$

实现

我们将使用 `FilterPy` 的 `ExtendedKalmanFilter` 类来实现该滤波器。该类的 `predict()` 方法采用了标准的线性方程来描述处理过程。不过，我们的系统属于非线性系统，因此我们必须用自己的实现来覆盖 `predict()` 方法。我还想利用这个类来模拟机器人的行为，因此会添加一个 `move()` 方法，用于计算机器人的位置。 `predict()` 以及我的模拟程序都可以调用这个方法。

在预测步骤中，所涉及的矩阵规模相当大。在编写这段代码时，我犯了几个错误，最终才成功实现了功能。我是通过使用 SymPy 的 `evalf` 函数来发现这些错误的。 `evalf` 函数会用特定的变量值来计算 SymPy 表达式 `Matrix` 的结果。我决定向你展示这种技巧，因此在卡尔曼滤波器的代码中使用了 `evalf` 函数。你需要理解其中的几个要点。

首先， `evalf` 使用字典来指定各个数值。例如，如果你的矩阵中包含 `x` 和 `y` ，你可以这样表示：

```
M.evalf(subs={x:3, y:17})
```

这样就能在 x = 3、y = 17 时对矩阵求值。

其次， `evalf` 返回一个 `sympy.Matrix` 类型的对象。需要使用 `numpy.array(M).astype(float)` 将其转换为 NumPy 数组。而 `numpy.array(M)` 创建的是一个 `object` 类型的数组，这并不是我们想要的格式。

以下是 EKF 的代码：

```
from filterpy.kalman import ExtendedKalmanFilter as EKF
from numpy import array, sqrt
class RobotEKF(EKF):
    def __init__(self, dt, wheelbase, std_vel, std_steer):
        EKF.__init__(self, 3, 2, 2)
        self.dt = dt
        self.wheelbase = wheelbase
        self.std_vel = std_vel
        self.std_steer = std_steer

        a, x, y, v, w, theta, time = symbols(
            'a, x, y, v, w, theta, t')
        d = v*time
        beta = (d/w)*sympy.tan(a)
        r = w/sympy.tan(a)

        self.fxu = Matrix(
            [[x-r*sympy.sin(theta)+r*sympy.sin(theta+beta)],
             [y+r*sympy.cos(theta)-r*sympy.cos(theta+beta)],
             [theta+beta]])

        self.F_j = self.fxu.jacobian(Matrix([x, y, theta]))
        self.V_j = self.fxu.jacobian(Matrix([v, a]))

        # save dictionary and it's variables for later use
        self.subs = {x: 0, y: 0, v:0, a:0,
                     time:dt, w:wheelbase, theta:0}
        self.x_x, self.x_y, = x, y
        self.v, self.a, self.theta = v, a, theta

    def predict(self, u):
        self.x = self.move(self.x, u, self.dt)
        self.subs[self.x_x] = self.x[0, 0]
        self.subs[self.x_y] = self.x[1, 0]

        self.subs[self.theta] = self.x[2, 0]
        self.subs[self.v] = u[0]
        self.subs[self.a] = u[1]

        F = array(self.F_j.evalf(subs=self.subs)).astype(float)
        V = array(self.V_j.evalf(subs=self.subs)).astype(float)

        # covariance of motion noise in control space
        M = array([[self.std_vel**2, 0],
                   [0, self.std_steer**2]])

        self.P = F @ self.P @ F.T + V @ M @ V.T
```

```
    def move(self, x, u, dt):
        hdg = x[2, 0]
        vel = u[0]
        steering_angle = u[1]
        dist = vel * dt

        if abs(steering_angle) > 0.001: # is robot turning?
            beta = (dist / self.wheelbase) * tan(steering_angle)
            r = self.wheelbase / tan(steering_angle) # radius

            dx = np.array([[-r*sin(hdg) + r*sin(hdg + beta)],
                           [r*cos(hdg) - r*cos(hdg + beta)],
                           [beta]])
        else: # moving in straight line
            dx = np.array([[dist*cos(hdg)],
                           [dist*sin(hdg)],
                           [0]])
        return x + dx
```

现在我们还有另一个问题需要处理。从理论上讲，残差应该按照 $\boldsymbol{y = z - h(x)}$ 的方式来计算。但实际上，由于我们的测量数据中包含了角度信息，这种计算方式并不适用。假设 z 的方位角为 $\mathbf{1°}$，而 $\boldsymbol{h(x)}$ 的方位角为 $\mathbf{359°}$。如果直接将这两个数值相减，得到的角度差为 $\mathbf{-358°}$，但实际上正确的数值应该是 $\mathbf{2°}$。因此，我们必须编写相应的代码来正确计算出残差值。

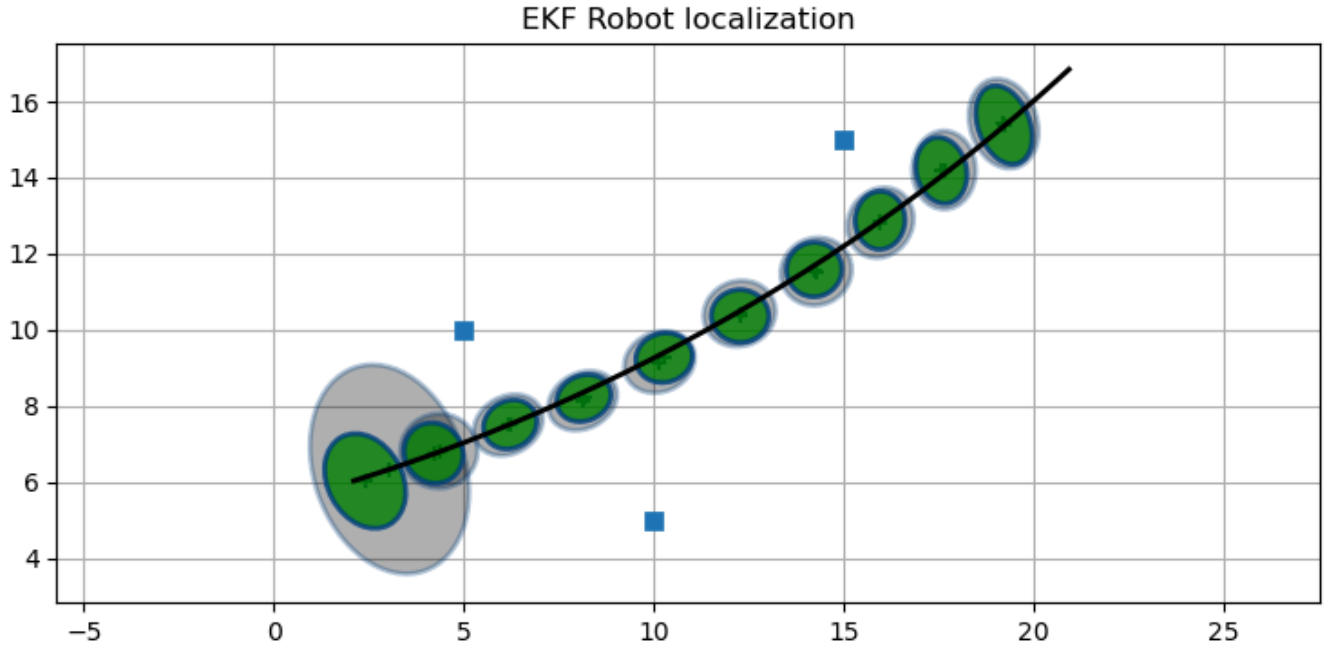
```
def residual(a, b):
    """ compute residual (a-b) between measurements containing
    [range, bearing]. Bearing is normalized to [-pi, pi)"""
    y = a - b
    y[1] = y[1] % (2 * np.pi)    # force in range [0, 2 pi)
    if y[1] > np.pi:             # move to [-pi, pi)
        y[1] -= 2 * np.pi
    return y
```

其余的代码负责执行模拟运算并绘制结果图，这些部分应该不需要太多解释。我创建了一个名为 `landmarks` 的变量，用于存储各个参考点的坐标。模拟中的机器人位置每秒更新 10 次，而卡尔曼滤波则每秒只计算一次。这样做有两个原因：首先，我们没有使用龙格-库塔法来求解运动方程，因此较小的时间步长有助于提高模拟的准确性。其次，在嵌入式系统中，处理速度往往有限。因此，必须仅在确实必要时才运行卡尔曼滤波器。

```
from filterpy.stats import plot_covariance_ellipse
from math import sqrt, tan, cos, sin, atan2
import matplotlib.pyplot as plt

dt = 1.0

def z_landmark(lmark, sim_pos, std_rng, std_brg):
    x, y = sim_pos[0, 0], sim_pos[1, 0]
    d = np.sqrt((lmark[0] - x)**2 + (lmark[1] - y)**2)
    a = atan2(lmark[1] - y, lmark[0] - x) - sim_pos[2, 0]
    z = np.array([[d + randn()*std_rng],
                  [a + randn()*std_brg]])
    return z

def ekf_update(ekf, z, landmark):
    ekf.update(z, HJacobian=H_of, Hx=Hx,
               residual=residual,
               args=(landmark), hx_args=(landmark))


def run_localization(landmarks, std_vel, std_steer,
                     std_range, std_bearing,
```

```
                     step=10, ellipse_step=20, ylim=None):
    ekf = RobotEKF(dt, wheelbase=0.5, std_vel=std_vel,
                   std_steer=std_steer)
    ekf.x = array([[2, 6, .3]]).T # x, y, steer angle
    ekf.P = np.diag([.1, .1, .1])
    ekf.R = np.diag([std_range**2, std_bearing**2])

    sim_pos = ekf.x.copy() # simulated position
    # steering command (vel, steering angle radians)
    u = array([1.1, .01])

    plt.figure()
    plt.scatter(landmarks[:, 0], landmarks[:, 1],
                marker='s', s=60)

    track = []
    for i in range(200):
        sim_pos = ekf.move(sim_pos, u, dt/10.) # simulate robot
        track.append(sim_pos)

        if i % step == 0:
            ekf.predict(u=u)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ekf.x[0,0], ekf.x[1,0]), ekf.P[0:2, 0:2],
                     std=6, facecolor='k', alpha=0.3)

            x, y = sim_pos[0, 0], sim_pos[1, 0]
            for lmark in landmarks:
                z = z_landmark(lmark, sim_pos,
                               std_range, std_bearing)
                ekf_update(ekf, z, lmark)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ekf.x[0,0], ekf.x[1,0]), ekf.P[0:2, 0:2],
                    std=6, facecolor='g', alpha=0.8)
    track = np.array(track)
    plt.plot(track[:, 0], track[:,1], color='k', lw=2)
    plt.axis('equal')
    plt.title("EKF Robot localization")
    if ylim is not None: plt.ylim(*ylim)
    plt.show()
    return ekf
```

```
landmarks = array([[5, 10], [10, 5], [15, 15]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.017 0.035 0.002]
```

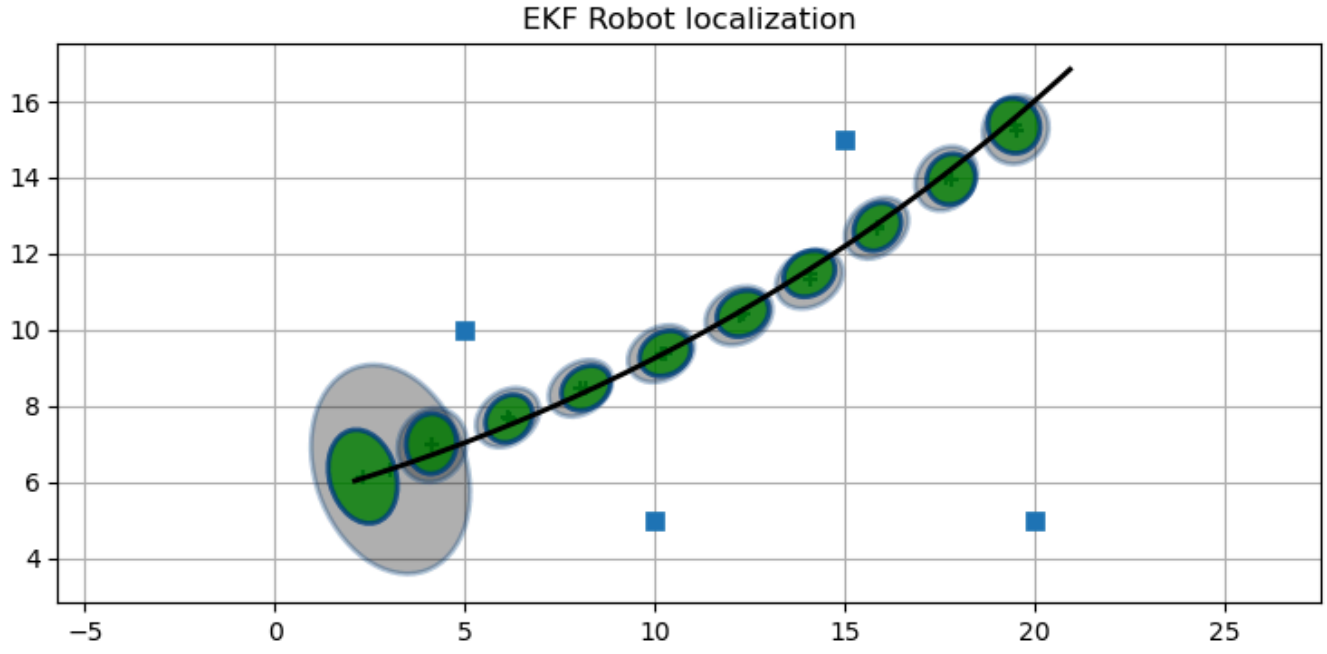
图中用实心方块表示地标，黑线表示机器人的实际路径。预测步骤的协方差椭圆为浅灰色，更新后的椭圆为绿色。为使它们在当前比例下清晰可见，将椭圆边界设为 6 σ。

运动模型引入了相当大的不确定性，而且主要沿运动方向展开，从预测协方差椭圆的形状可以看出这一点。经过几步后，滤波器逐渐融合地标测量，不确定性随之减小。

这里采用了与 UKF 一章相同的初始条件和地标位置。就误差椭圆而言，UKF 的结果明显更好；而就状态 $\mathbf{x}$ 的估计而言，两者表现大致相当。

现在再增加一个地标。

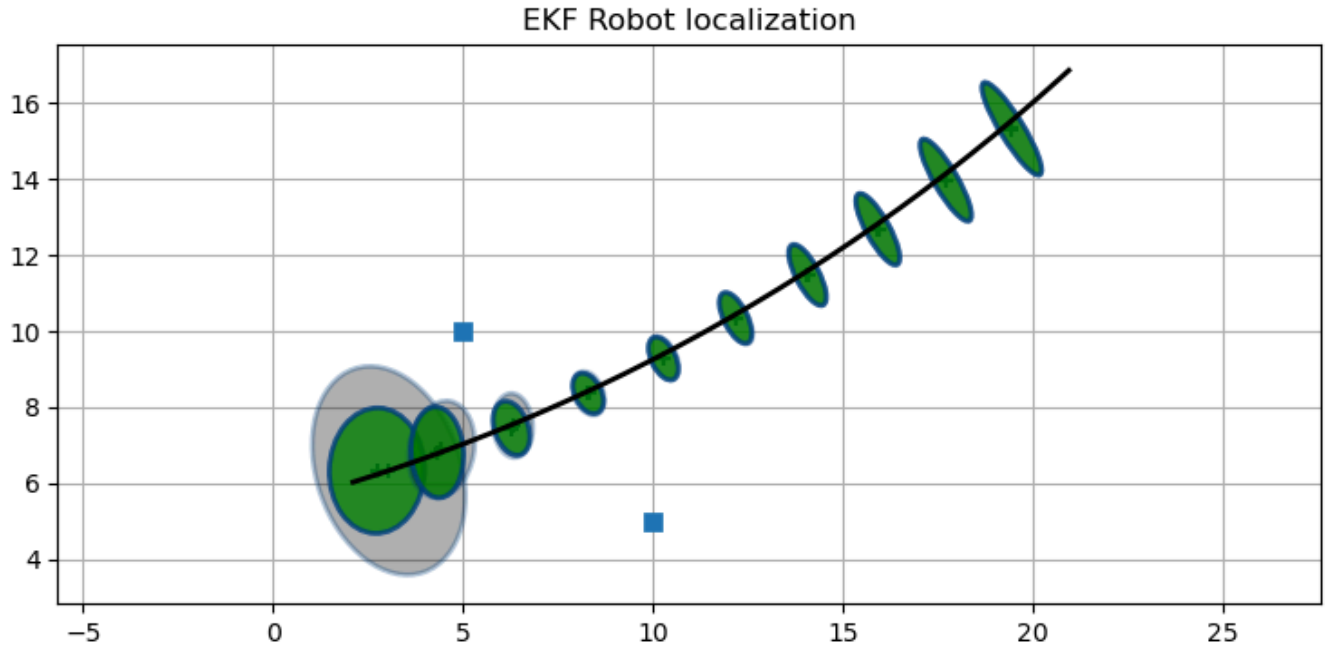
```
landmarks = array([[5, 10], [10, 5], [15, 15], [20, 5]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1)
plt.show()
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.014 0.016 0.002]
```

轨迹末端的估计不确定性减小了。下面只使用前两个地标，对比地标数量变化对不确定性的影响。

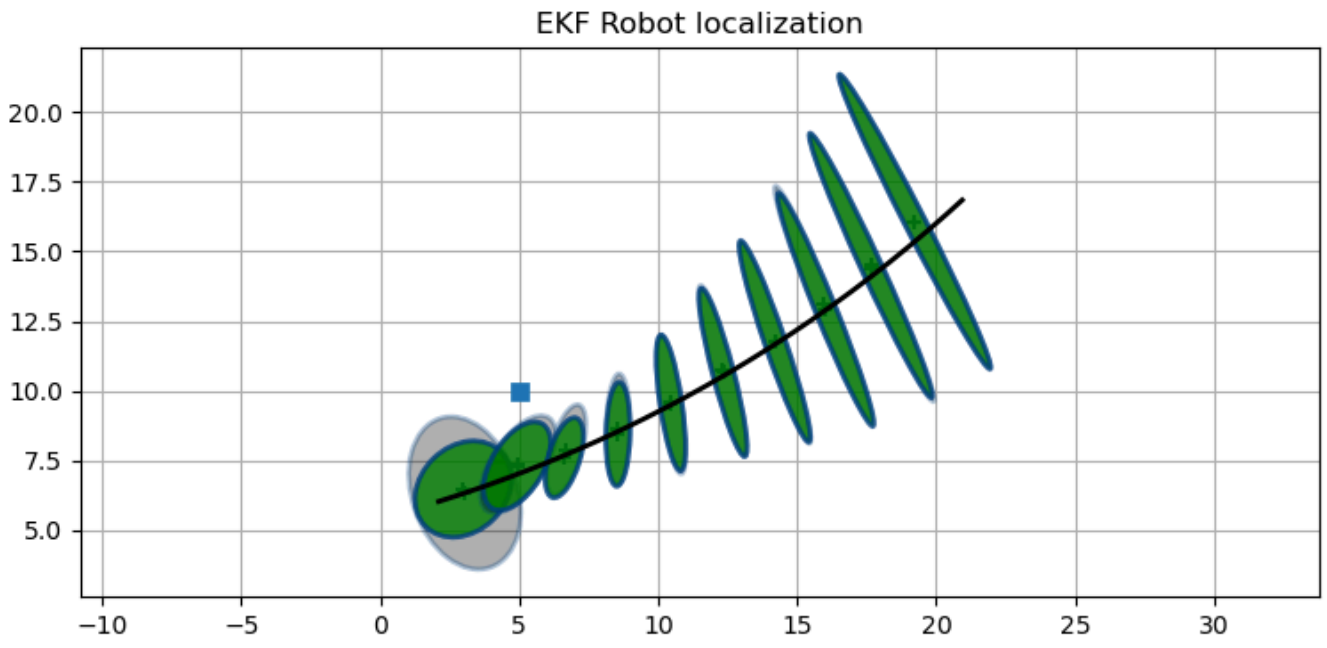
```
ekf = run_localization(
    landmarks[0:2], std_vel=1.e-10, std_steer=1.e-10,
    std_range=1.4, std_bearing=.05)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.02  0.046 0.   ]
```

在经过各个地标之后，该估计值很快就会与机器人的实际路径产生偏差。同时，协方差也会迅速增大。让我们来看看，当只有一个地标时会发生什么：

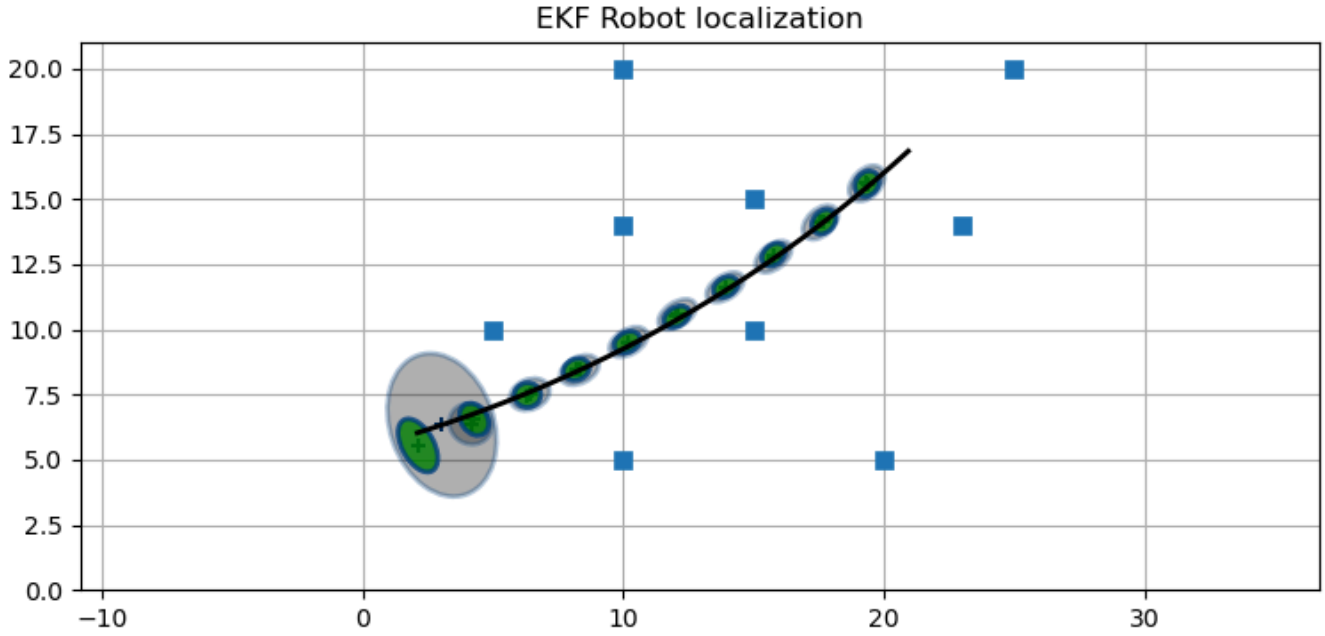
```
ekf = run_localization(
    landmarks[0:1], std_vel=1.e-10, std_steer=1.e-10,
    std_range=1.4, std_bearing=.05)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.249 0.849 0.004]
```

正如你可能已经猜到的那样，如果只使用一个地标点，那么得到的结果会非常不准确。相反，如果使用大量的地标点，就能得到非常精确的估算结果。

```
landmarks = array([[5, 10], [10,  5], [15, 15], [20,  5], [15, 10],
                   [10,14], [23, 14], [25, 20], [10, 20]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1, ylim=(0, 21))
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.006 0.007 0.001]
```

讨论

我称它为一个实际问题，在某些方面确实如此。也有其他讲解采用更简单的机器人运动模型，从而得到较简单的雅可比矩阵。不过，本例的运动模型也作了多项简化。首先使用自行车模型，而真实汽车的不同车轮沿不同半径的轨迹运动，轮胎与地面之间也并非完全无滑动。此外，还假设机器人能瞬时响应控制输入。Sebastian Thrun 等人在《概率机器人》中指出，这种简化模型有其合理性，因为它用于跟踪真实车辆时仍能取得良好效果。这说明，非线性模型需要足够准确，却不必完美才能有效工作。设计者必须在模型保真度、数学复杂性与线性代数计算所需的 CPU 时间之间作出权衡。

这个问题的另一个简化处理方式是：我们假设自己知道各个地标与相应测量数据之间的对应关系。但是，如果我们使用的是雷达技术，又该如何确定某个特定的信号回波对应于现场中的哪栋建筑物呢？这个问题其实涉及到 SLAM 算法——即同时定位与地图构建。不过，SLAM 并非本书的重点，因此我就不详细讨论这个话题了。

UKF 与 EKF 的比较

在上一章中，我使用了 UKF 方法来解决这个问题。两种方法的实现方式有着明显的区别。尽管采用了较为简单的运动模型，但计算状态模型和测量模型的雅可比矩阵并非易事。对于某些问题来说，雅可比矩阵可能难以或无法通过解析方法来求得。而 UKF 则只需要用户提供两个函数：一个用于计算系统运动模型，另一个用于计算测量模型。

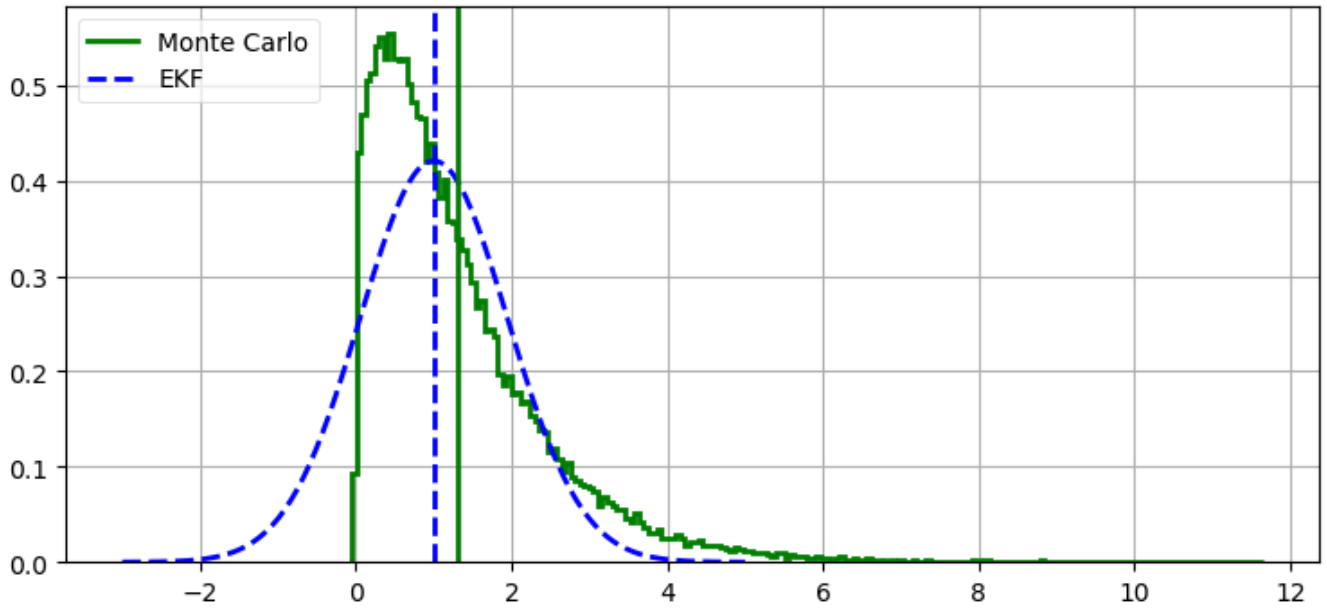
在很多情况下，雅可比矩阵无法通过解析方法来求出。这些细节超出了本书的讨论范围，你不得不借助数值方法来计算雅可比矩阵。这项工作并非易事，因此在攻读 STEM 专业的硕士学位期间，你需要花费大量时间来学习处理这类问题的方法。即便如此，你通常也只能解决与自己所在领域相关的问题——比如，航空工程师会深入学习纳维-斯托克斯方程，但几乎不会涉及化学反应速率的建模。

UKF 用起来比较方便，那么准确吗？实际中，它常能比 EKF 表现更好，许多研究论文也在具体问题中得到这一结果。EKF 在单个点处线性化系统模型和测量模型，而这里的 UKF 使用 $2n+1$ 个点描述分布。不过，具体性能仍取决于模型、参数与数值实现，不能据此保证 UKF 对所有问题都优于 EKF。

来看一个具体例子。取非线性函数 $f(x) = x^3$，考察高斯随机变量经过它变换后的分布。这里用蒙特卡洛模拟获得高精度近似：随机生成 50,000 个服从该高斯分布的样本，将每个样本代入 $f(x)$，再计算变换结果的均值与方差。

EKF 通过求导，得到函数在求值点 x 处的斜率，并据此作局部线性化。所得线性近似用于传播高斯分布。下图展示了这一过程。

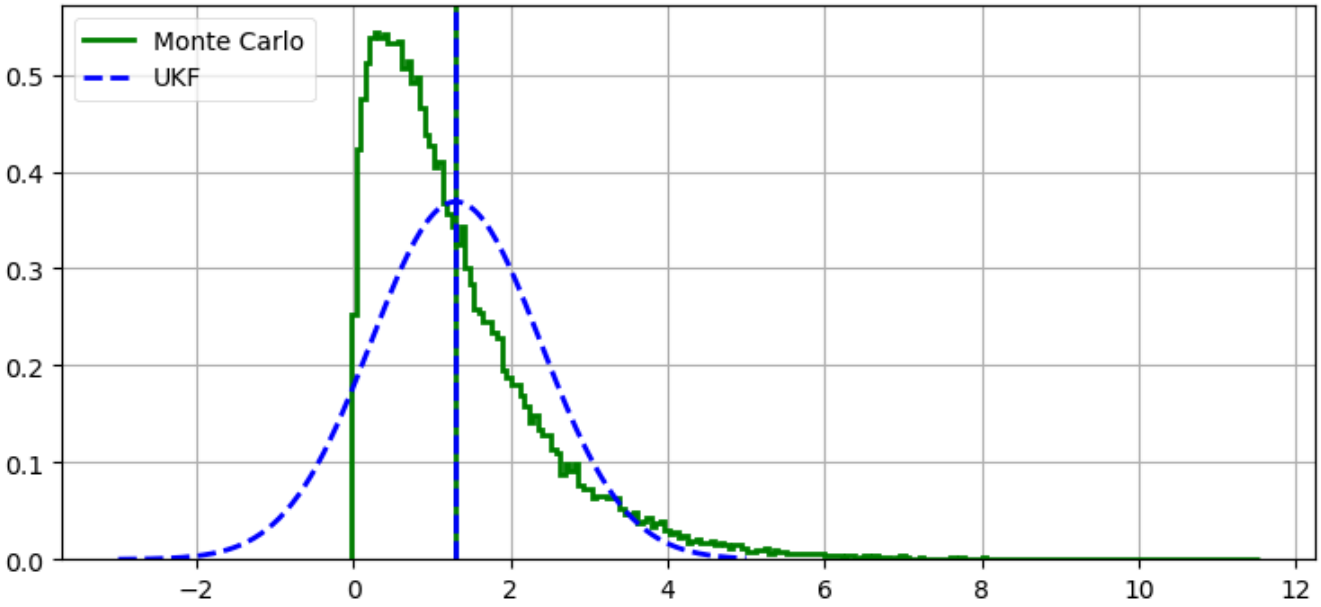
```
import kf_book.nonlinear_plots as nonlinear_plots
nonlinear_plots.plot_ekf_vs_mc()
```




```
actual mean=1.30, std=1.12
EKF    mean=1.00, std=0.95
```

EKF 算法的计算结果相当不准确。相比之下，UKF 算法的性能则要好得多：

```
nonlinear_plots.plot_ukf_vs_mc(alpha=0.001, beta=3., kappa=1.)
```




```
actual mean=1.30, std=1.13
UKF    mean=1.30, std=1.08
```

这里 UKF 算得的均值与蒙特卡洛参考值相符到小数点后两位，标准差仍略有偏差。可以通过生成 sigma 点的参数 α、β 和 κ 调整结果；这里采用 $\alpha = 0.001$、$\beta = 3$ 和 $\kappa = 1$。可以自行改变参数观察效果，但应避免针对某个测试过度调参，以免在该例上表现更好，在一般情形下反而更差。

第 12 章：粒子滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引入背景

这就是我们面临的问题。我们需要追踪一些正在移动的物体。这些物体可能是战斗机或导弹，也可能是正在球场上打板球的人。其实并不重要。那么，我们所学过的各种滤波方法中，有哪种能够解决这个问题呢？遗憾的是，没有一种方法是完美的。让我们来仔细分析一下这个问题的特点吧。

- 多峰性：一个目标的状态存在歧义时，后验分布可能出现多个峰。多目标跟踪还要求同时处理零个、一个或多个目标；目标个数与分布峰数不能简单等同。
- 遮挡：一个目标可能遮住另一个目标，使多个目标只对应一个测量结果。
- 非线性行为：飞机会受到风的冲击而颠簸不定，球会沿抛物线轨迹运动，人们也会相互碰撞。
- 非线性测量：例如，雷达测距与笛卡尔坐标 (x, y, z) 之间的关系涉及非线性运算。仅凭一个距离不能唯一恢复三维位置，还需要角度等额外信息。
- 非高斯噪声：当物体在背景上移动时，计算机视觉系统可能会将背景的某些部分误认为是物体本身。
- 连续性：物体的位置和速度（即状态空间）可以随时间平滑地变化。
- 多变量：需要跟踪位置、速度、转弯角速度等多个属性。
- 未知过程模型：可能不了解系统的过程模型。

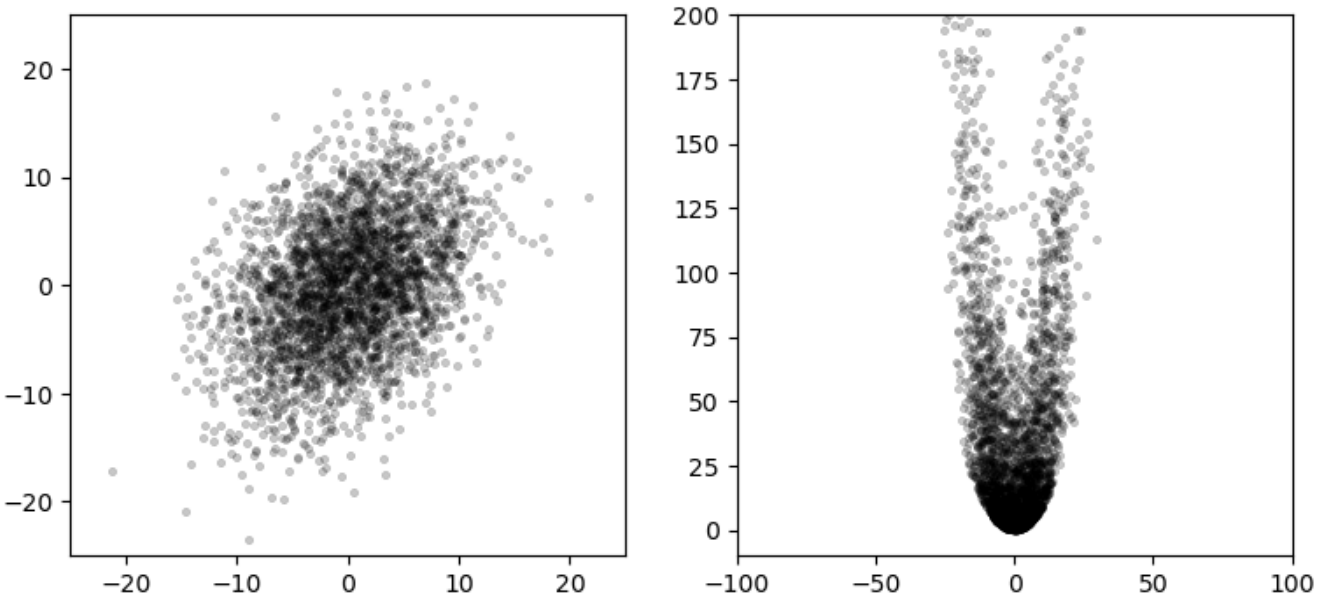
我们所了解的各类滤波方法，没有一种能在所有这些限制条件下都发挥良好的效果。

- 离散贝叶斯滤波器：它可以表示多峰分布，处理非线性测量，并扩展到非线性运动。本书前面介绍的版本采用离散的一维状态，但离散贝叶斯滤波本身也可推广到多维状态。
- 卡尔曼滤波器：在线性高斯等假设成立时，卡尔曼滤波能够得到最优估计，并用单峰高斯分布表示状态不确定性。本例所考虑的一般情形不满足这些假设。
- 无迹卡尔曼滤波器：UKF 能够处理连续状态空间中的非线性多变量问题，但单个高斯状态近似不能表示多峰后验，也不能独自解决目标遮挡带来的关联歧义。对于适度偏离高斯的噪声，它可能仍然有效；面对明显非高斯的分布或强非线性问题时，效果则可能不佳。
- 扩展卡尔曼滤波器：与 UKF 相比，EKF 具有相同的优点和局限性。只不过，EKF 对强烈的非线性因素和非高斯噪声更为敏感。

蒙特卡洛抽样法

在 UKF 相关的内容中，我绘制了类似的图表，用以说明非线性系统对高斯分布的影响。

```
import kf_book.pf_internal as pf_internal
pf_internal.plot_monte_carlo_ukf()
```



左边的图表显示了 3,000 个遵循高斯分布的数值。

$$\mu = \begin{bmatrix}0\\0\end{bmatrix},\ \Sigma = \begin{bmatrix}32 & 15\\15 & 40\end{bmatrix}$$

右图展示这些样本点经过下面这组方程变换后的结果：

$$\begin{aligned}x &= x + y\\ y &= 0.1x^2 + y^2\end{aligned}$$

利用有限数量的随机采样点来计算结果的方法，被称为蒙特卡洛方法。其原理很简单：先生成足够多的样本点，以构成该问题的代表性样本；然后让这些样本点在所建模的系统中运行；最后根据这些样本点的运算结果来得出最终结果。

简而言之，这就是粒子滤波的运作原理。我们在本书中一直使用的贝叶斯滤波算法，实际上是应用于成千上万个“粒子”上的。每个粒子都代表着系统的一种可能状态。我们通过对这些粒子的加权统计结果，来得出系统的估计状态。

通用粒子滤波算法

1. **随机生成一群粒子**

粒子可以包含位置、航向角，以及其他需要估计的状态变量。每个粒子各有一个权重，表示它与系统真实状态的相容程度。初始化时，为所有粒子赋予相同权重。

2. **预测粒子的下一个状态**

根据你对真实系统行为的预测来移动这些粒子。

3. **更新**

根据测量结果更新各粒子的权重。与测量结果最匹配的粒子，其权重会高于那些与测量结果不太符合的粒子。

4. **重采样**

抛弃那些出现概率极低的粒子，用出现概率较高的粒子的“副本”来替代它们。

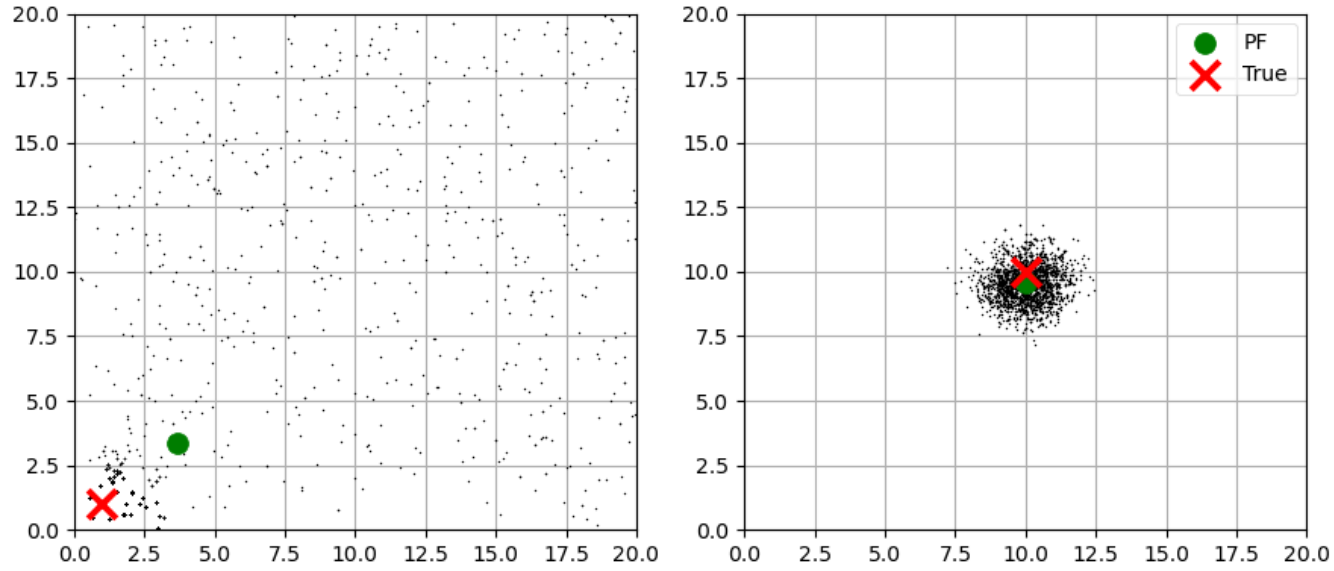
5. **计算状态估计值**

可选地，可以计算这些粒子的加权平均值和协方差，从而得到状态估计值。

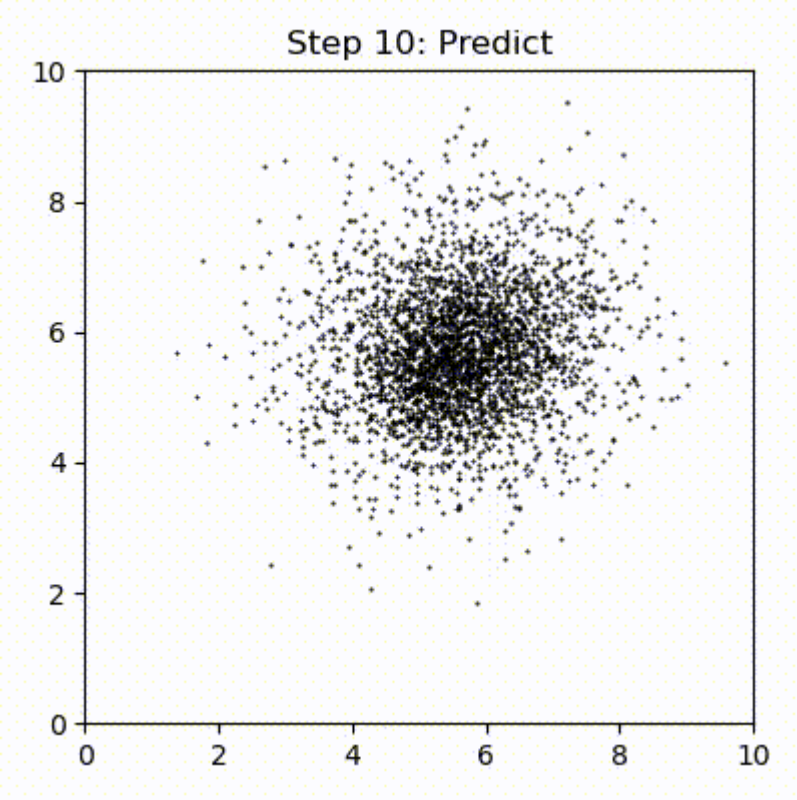
这种简单的算法在实际应用中存在诸多难题，我们需要设法克服这些困难。不过，这毕竟是一个基本的思路。我们来看个例子吧。我根据 UKF 和 EKF 的相关理论，为机器人定位问题编写了一个粒子滤波算法。该机器人具有转向和速度控制功能。它还配备了能够测量到可见地标距离的传感器。不过，无论是传感器还是控制机制都会产生噪声，而我们需要准确确定机器人的位置。

在这里，我运用了粒子滤波算法，并绘制出了各粒子的位置。左边的图显示的是经过一次迭代后的结果，右边的图则显示了经过 10 次迭代后的结果。红色的“X”标记代表了机器人的实际位置，而大圆圈则代表了经过计算得出的加权平均位置。

```
pf_internal.show_two_pf_plots()
```



如果你是通过浏览器查看此内容，那么该动画会完整地展示整个过程：



在第一次迭代之后，这些粒子仍然大致随机地分布在地图上。不过可以看出，有些粒子已经聚集在了机器人的附近。计算出的平均值与机器人的位置非常接近。这是因为每个粒子的权重取决于它与实际测量值的匹配程度。机器人位于(1,1)附近，因此那些位于(1,1)附近的粒子会因为与测量值高度匹配而拥有较高的权重。而那些远离机器人的粒子则无法与测量值匹配，因此其权重很低。最终的位置估计值就是各粒子位置的加权平均值。由于靠近机器人的粒子对计算结果的贡献更大，因此估计值相当准确。

经过几次迭代后，可以看到所有粒子都聚集在了机器人的周围。这是由于重采样步骤的作用。重采样会丢弃那些出现概率极低的粒子，然后用出现概率较高的粒子来替代它们。

我还没有完全解释清楚这一机制的运作原理，也没有详细阐述粒子权重分配和重采样的具体算法。不过，从直觉上来看，这个方法应该是可行的。我们可以生成大量随机粒子，让它们“大致”跟随机器人的移动轨迹；根据这些粒子与实际测量数据的匹配程度来为它们分配权重，只保留那些匹配度较高的粒子。看起来这种方法确实有效。

通过蒙特卡洛方法得到的概率分布

假设我们想知道曲线 $y = \mathrm{e}^{\sin(x)}$ 在区间$[0, \pi]$内的面积。这个面积可以通过定积分 $\int_0^{\pi} \mathrm{e}^{\sin(x)}\,\mathrm{d}x$ 来计算。作为练习，请你自己试着算出答案吧；我会等待你的结果。

如果你很明智，也许没有接受这个挑战：$\mathrm{e}^{\sin(x)}$ 的原函数不能用初等函数表示。现实中有许多积分无法求得方便的解析表达式。例如，计算物体的亮度：物体反射照射到表面的部分光，其中一些反射光经过其他物体反射后又照回原物体，从而增加它的亮度。这就引入了递归的积分问题，求解起来并不容易。

不过，使用蒙特卡洛方法来计算积分其实相当简单。要计算曲线下的面积，可以先确定一个包含该曲线的矩形区域。然后在矩形区域内随机生成一些点，计算其中落在曲线下的点的比例。例如，如果 40%的点落在曲线下方，而矩形的面积为 1，那么曲线下的面积大约就是 0.4。当生成的点数趋于无穷大时，就能得到任意精确的结果。实际上，只需几千个点就能得到相当准确的结果。

蒙特卡洛技术可对复杂函数作数值积分，包括没有初等原函数的函数，以及许多不连续但可积的函数。这类方法由洛斯阿拉莫斯国家实验室的 Stanisław Ulam 等人发展起来，用于计算难以手工求解的核反应问题。使用时仍须满足可积性等适用条件。

让我们通过计算圆的面积来求出 π 的值。我们先定义一个半径为 1 的圆，并将其置于一个边长为 2 的正方形内。那么，这个正方形的面积为 4。接下来，我们在这个正方形内随机生成一些点，然后统计其中有多少点落在圆内。圆的面积就可以通过正方形的面积乘以圆内点数与总点数之比来计算。最后，既然我们已经知道了 $A = \pi r^2$ 的值，那么就可以计算出 $\pi = A/r^2$ 了。

我们首先创建这些点。

```
N = 20000
pts = uniform(-1, 1, (N, 2))
```

如果一个点到圆心的距离小于或等于圆的半径，那么这个点就位于圆内。我们使用 `numpy.linalg.norm` 来计算距离，该函数用于计算向量的模长。由于向量的起点都是(0, 0)，因此调用 norm 函数就能得到该点到原点的距离。

```
dist = np.linalg.norm(pts, axis=1)
```

接下来，我们判断这些距离中哪些符合标准。该代码会返回一个布尔值数组：如果某个距离符合条件 `dist <= 1`，则该数组在该位置的值为 `True`。

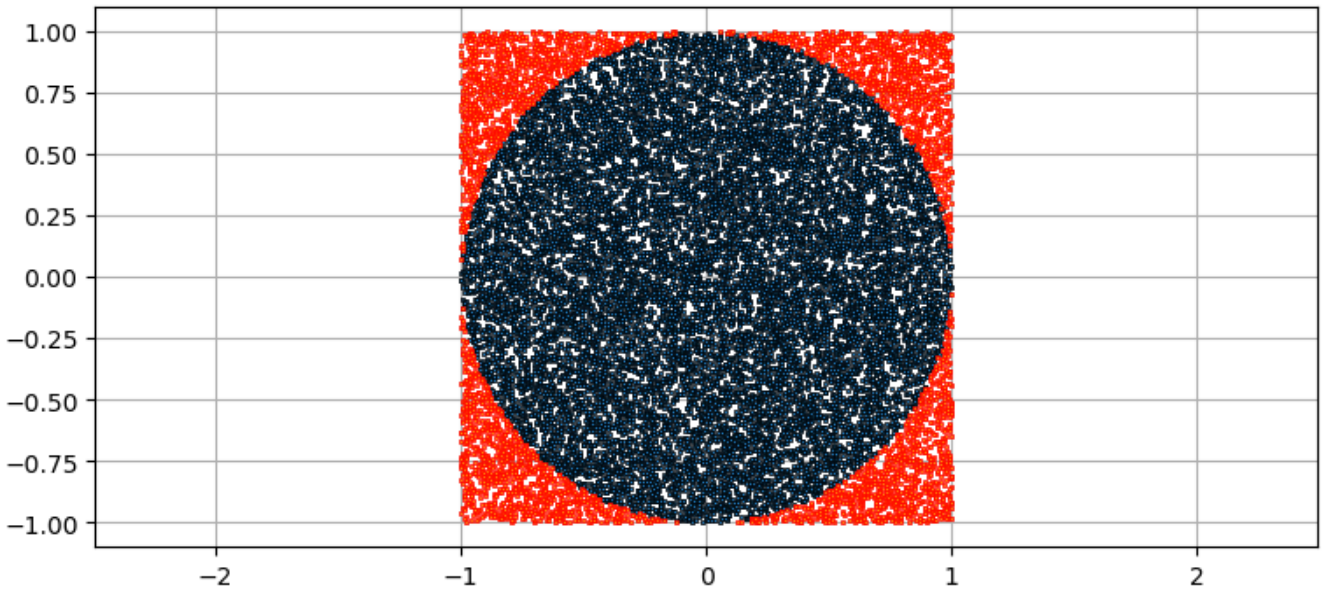
```
in_circle = dist <= 1
```

现在要做的就是计算圆内的点数、求出π的值，然后绘制出相应的图表。我把所有步骤都放在了一个单元格中，这样你就可以尝试使用不同的点数 `N` 来测试结果了。

```
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import uniform

N = 20000  # number of points
radius = 1.
area = (2*radius)**2

pts = uniform(-1, 1, (N, 2))

# distance from (0,0)
dist = np.linalg.norm(pts, axis=1)
in_circle = dist <= 1

pts_in_circle = np.count_nonzero(in_circle)
pi = 4 * (pts_in_circle / N)

# plot results
plt.scatter(pts[in_circle,0], pts[in_circle,1],
            marker=',', edgecolor='k', s=1)
```

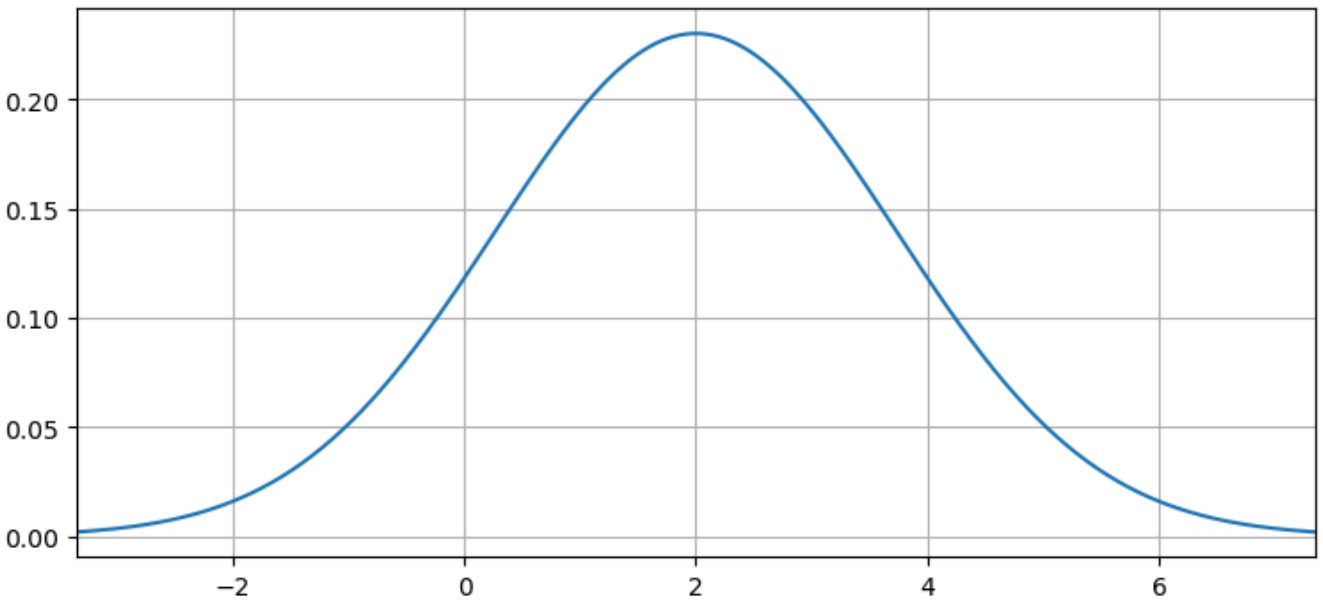
```
plt.scatter(pts[~in_circle,0], pts[~in_circle,1],
            marker=',', edgecolor='r', s=1)
plt.axis('equal')

print(f'mean pi(N={N})= {pi:.4f}')
print(f'err  pi(N={N})= {np.pi-pi:.4f}')
```

```
mean pi(N=20000)= 3.1684
err  pi(N=20000)= -0.0268
```



这说明，蒙特卡洛样本可以用来近似表示概率分布，并估计相应区间的概率。例如，考虑下面的高斯分布：

```
from filterpy.stats import plot_gaussian_pdf
plot_gaussian_pdf(mean=2, variance=3);
```

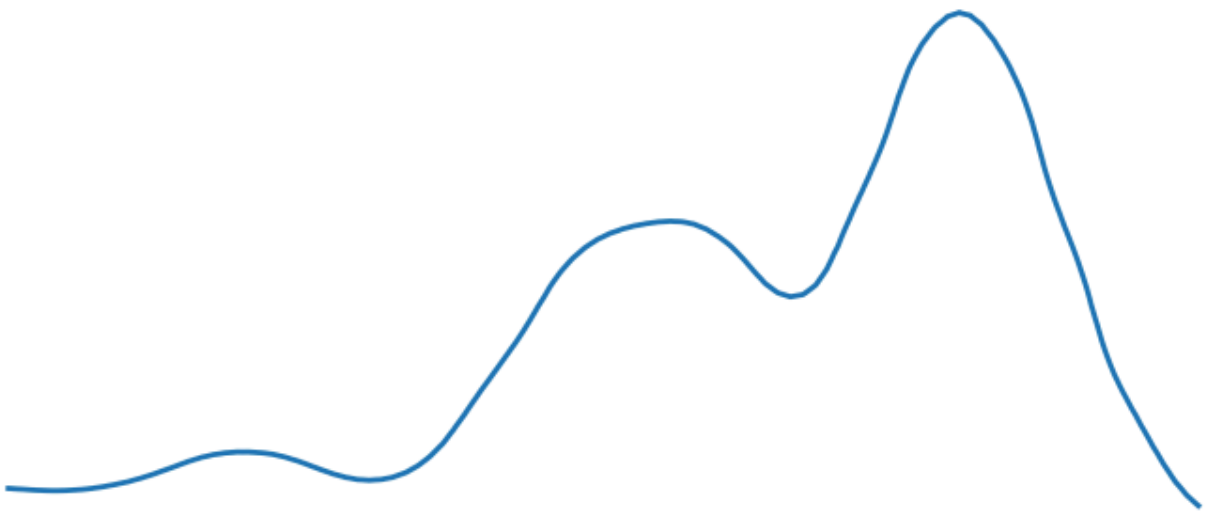


概率密度函数在一个区间上的积分，给出随机变量落在该区间内的概率。例如，我们可能想知道上图中 x 位于 0 与 2 之间的概率。对于连续分布，需要对密度函数积分，求出曲线下相应部分的面积；该面积就是区间概率。

$$P[a \leq X \leq b] = \int_a^b f_X(x)\, dx$$

对于高斯函数来说，计算这个积分相当简单。但在现实生活中，情况并非如此简单。例如，下图展示了一个概率分布。我们无法用解析方法来描述任意的曲线，更不用说对其进行积分了。

```
pf_internal.plot_random_pd()
```



蒙特卡洛方法可用来近似计算概率密度曲线下的积分，由此估计随机变量落在相应区间内的概率。

粒子滤波器

这些内容引出了粒子滤波器。以城市环境中的机器人或汽车定位为例，为保持前后连贯，这里继续采用 EKF 和 UKF 章节中的机器人定位问题。在这个问题中，传感器测量机器人到已知地标的距离和方位角。

粒子滤波是一类算法。我在这里介绍的是一种易于理解的粒子滤波实现方式，这种实现方式与我们在这本书中研究的问题密切相关。不过，其中有一些步骤可能看起来有点“神奇”，因为我还没有给出完整的解释。这些解释会在本章的后面部分给出。

借鉴上一节的讨论，我们首先创建了数千个粒子。每个粒子的位置代表了机器人可能所处的位置；此外，每个粒子还可能具有方向和速度信息。假设我们完全不知道机器人的具体位置，那么我们应该将这些粒子均匀地分布在整个场景中。可以把所有的粒子看作是一种概率分布：粒子数量较多的地方，意味着我们对机器人位于该位置的置信度较高；粒子数量较少的地方，则意味着置信度较低。如果某个特定位置附近有大量的粒子，那就说明我们更确定机器人就位于那里。

每个粒子都需要权重，理想上反映它代表机器人真实位置的可能性。相关的已归一化后验通常难以直接计算，因此先计算与目标后验成比例的未归一化权重，再归一化。初始化时，没有理由偏爱某个粒子，所以对 N 个粒子，每个都赋权 $1/N$。使用 $1/N$ 可保证全部权重之和为 1。

粒子及其权重共同构成对概率分布的近似。回想“离散贝叶斯滤波”一章：当时把走廊位置表示为等间距的离散网格。这里很相似，只是粒子分布在连续空间中随机选择的位置，不局限于固定网格。本例让机器人在一个大小任意的二维平面区域中运动，并将图中的一角设为 (0, 0)。

跟踪机器人需要记录 x、y 和航向角。将 N 个粒子存放在形状为 (N, 3) 的数组中，三列依次保存 x、y 和航向角。

若进行无控制输入的被动跟踪，就需要把速度纳入状态，并用速度估计进行预测。为在更高维空间中保持足够好的覆盖，所需粒子数可能随维数急剧增加，常表现出维数灾难。因此，应尽量减少不必要的状态变量。

下面的代码分别在指定区域内创建均匀分布和高斯分布的粒子群：

```
from numpy.random import uniform

def create_uniform_particles(x_range, y_range, hdg_range, N):
    particles = np.empty((N, 3))
    particles[:, 0] = uniform(x_range[0], x_range[1], size=N)
    particles[:, 1] = uniform(y_range[0], y_range[1], size=N)
    particles[:, 2] = uniform(hdg_range[0], hdg_range[1], size=N)
    particles[:, 2] %= 2 * np.pi
    return particles

def create_gaussian_particles(mean, std, N):
    particles = np.empty((N, 3))
    particles[:, 0] = mean[0] + (randn(N) * std[0])
    particles[:, 1] = mean[1] + (randn(N) * std[1])
    particles[:, 2] = mean[2] + (randn(N) * std[2])
    particles[:, 2] %= 2 * np.pi
    return particles
```

例如：

```
create_uniform_particles((0,1), (0,1), (0, np.pi*2), 4)
```

```
array([[0.772, 0.336, 4.171],
       [0.333, 0.34 , 4.319],
       [0.6  , 0.274, 5.02 ],
       [0.054, 0.022, 5.034]])
```

预测步骤

在贝叶斯算法中，预测步骤是利用系统模型来更新我们对系统状态的置信度。那么，我们该如何用粒子来实现这一过程呢？每个粒子都代表着机器人可能所处的位置。假设我们命令机器人移动 0.1 米，同时旋转 0.007 弧度。我们可以让每个粒子都按照这个量进行移动。不过，这样做很快就会遇到问题：机器人的控制并不完美，因此它的实际移动不会完全符合指令。因此，我们需要在粒子的移动过程中加入“噪声”，这样才能更准确地反映机器人的实际运动情况。如果不对系统中的不确定性进行建模，粒子滤波器就无法正确地描述我们对机器人位置的置信度分布。

```
def predict(particles, u, std, dt=1.):
    """ move according to control input u (heading change, velocity)
    with noise Q (std heading change, std velocity)`"""

    N = len(particles)
    # update heading
    particles[:, 2] += u[0] + (randn(N) * std[0])
    particles[:, 2] %= 2 * np.pi

    # move in the (noisy) commanded direction
    dist = (u[1] * dt) + (randn(N) * std[1])
    particles[:, 0] += np.cos(particles[:, 2]) * dist
    particles[:, 1] += np.sin(particles[:, 2]) * dist
```

更新步骤

接下来会得到一组测量数据，每个当前可见的地标对应一组测量值。应该如何利用这些测量数据，调整由粒子近似表示的概率分布?

回想“离散贝叶斯滤波”一章：当时把走廊位置表示为等间距离散点，并为每个位置指定先验概率。收到新测量后，将该位置的先验概率乘以在该位置产生当前测量的似然：

```
def update(likelihood, prior):
posterior = prior * likelihood
return normalize(posterior)
```

这是该方程式的具体实现方式。

$$x = \|\mathcal{L}\bar{x}\|$$

这其实体现了贝叶斯定理的原理：

$$\begin{aligned} P(x \mid z) &= \frac{P(z \mid x)\, P(x)}{P(z)} \\ &= \frac{\textbf{likelihood} \times \textbf{prior}}{\textbf{normalization}} \end{aligned}$$

我们对粒子也采用同样的处理方式。每个粒子都有一个位置值和一个权重值，该权重值用来衡量该粒子与实际测量结果的匹配程度。通过将所有粒子的权重值归一化，使其总和为 1，就可以将其转化为一个概率分布。通常来说，那些距离机器人最近的粒子的权重会更高，而那些距离机器人较远的粒子的权重则较低。

```
def update(particles, weights, z, R, landmarks):
    for i, landmark in enumerate(landmarks):
        distance = np.linalg.norm(particles[:, 0:2] - landmark, axis=1)
        weights *= scipy.stats.norm(distance, R).pdf(z[i])

    weights += 1.e-300      # avoid round-off to zero
    weights /= sum(weights) # normalize
```

文献将这一部分称为序贯重要性采样（SIS）。后文会介绍它的理论基础。直觉上，按粒子与测量的相容程度更新权重，就可用加权粒子近似纳入测量后的后验分布。在这里采用的自举粒子滤波条件下，新权重与旧权重乘以测量似然成正比。重要性密度是用于提出样本的分布，一般的 SIS 权重还需要包含目标密度与提议密度之比。

计算状态估计值

在多数应用中，每次更新后都希望得到状态估计，而粒子滤波器内部保存的是一组加权粒子。后验为单峰时，可用加权均值概括其位置。如果权重已归一化为总和 1，加权均值就是各粒子状态与相应权重乘积之和。需要注意，单个目标的后验也可能具有多个峰，此时只用均值未必能准确概括其状态。

$$\mu = \sum_{i=1}^{N} w^i x^i$$

这里用 x^i 表示第 i^{th} 个粒子。采用上标是因为下标常用于时间步。例如，第 $k+1^{th}$ 个时间步的第 i 个粒子可记为 x^i_{k+1}，虽然这种写法略显繁复。

该函数同时计算粒子位置的加权均值和方差：

```
def estimate(particles, weights):
    """returns mean and variance of the weighted particles"""

    pos = particles[:, 0:2]
    mean = np.average(pos, weights=weights, axis=0)
    var  = np.average((pos - mean)**2, weights=weights, axis=0)
    return mean, var
```

如果我们在 1x1 的正方形区域内以相等的权重来随机分布这些点，那么其平均位置会非常接近正方形的中心，即(0.5, 0.5)点。同时，该分布的方差也会很小。

```
particles = create_uniform_particles((0,1), (0,1), (0, 5), 1000)
weights = np.array([.25]*1000)
estimate(particles, weights)
```

```
(array([0.494, 0.514]), array([0.083, 0.085]))
```

粒子重采样

SIS 算法存在“退化问题”。该算法最初让所有粒子以相同的权重均匀分布。在机器人附近，可能只有少数几个粒子。随着算法的运行，那些与测量结果不匹配的粒子，其权重会变得极低。只有那些位于机器人附近的粒子，才能保持较高的权重。比如，可能有 5,000 个粒子，但其中只有 3 个粒子对状态估计有实际贡献！这种情况就被称为“算法退化”。为了解决这个问题，通常需要对粒子进行重新采样处理。

那些权重极小的粒子无法有效描述机器人的概率分布。重采样算法会丢弃那些概率极低的粒子，然后用概率较高的新粒子来替代它们。具体来说，就是复制那些概率较高的粒子。在预测过程中，这些被复制的粒子会因为噪声的影响而稍微发生偏移。这样一来，最终得到的粒子集合中，绝大多数粒子都能准确反映概率分布的情况。

重采样算法有很多。先看最简单的一类：简单随机重采样，也称多项分布重采样。按权重从当前粒子集合中有放回地抽取 N 次，构成新的粒子集合；每个粒子被选中的概率与其权重成正比。

用 NumPy 的 cumsum 可以计算权重数组的累积和。采用从 0 开始的索引时，索引 1 处保存原数组索引 0 与 1 的和，索引 2 处保存索引 0、1、2 的和，依此类推。接着在 0 到 1 之间生成均匀随机数，通过二分查找，确定每个数落入哪个累计权重区间，从而选出对应粒子。

```
def simple_resample(particles, weights):
    N = len(particles)
    cumulative_sum = np.cumsum(weights)
    cumulative_sum[-1] = 1. # avoid round-off error
    indexes = np.searchsorted(cumulative_sum, random(N))

    # resample according to indexes
    particles[:] = particles[indexes]
    weights.fill(1.0 / N)
```

不必在每个时间步都重采样。例如，没有新测量时，就没有新的观测信息可供重采样利用。可用有效样本量（effective sample size）决定是否重采样，它近似反映权重集中后仍对估计有实质贡献的粒子数量。公式为：

$$\hat{N}_{\text{eff}} = \frac{1}{\sum w^2}$$

我们可以在 Python 中用这种方式来实现它。

```
def neff(weights):
    return 1. / np.sum(np.square(weights))
```

当 $\hat{N}_{\text{eff}}$ 低于阈值时，可以进行重采样。常用起点为 $N/2$，但应按具体问题调整。$\hat{N}_{\text{eff}} = N$ 表示所有粒子权重相等。有效样本量只衡量权重集中程度，不能判断粒子的空间多样性：即使所有粒子位置相同，重采样后权重也可能相等。因此，还需要单独检查样本贫化，必要时可增加粒子数、改善提议分布，或采用适当的多样性恢复方法。

SIR 滤波器——一个完整的示例

虽然还有许多需要学习的内容，但就目前所掌握的知识而言，我们已经足以实现完整的粒子滤波算法了。我们将实现采样重要性重采样滤波器，也就是 SIR 算法。

我需要使用比上面所提到的更复杂的重采样方法。FilterPy 提供了多种重采样方式。我稍后会详细介绍这些方法。这些方法接收一个权重数组作为输入，然后返回那些被选中用于重抽样的粒子的索引。我们只需要编写一个函数，根据这些索引来执行重采样操作即可。

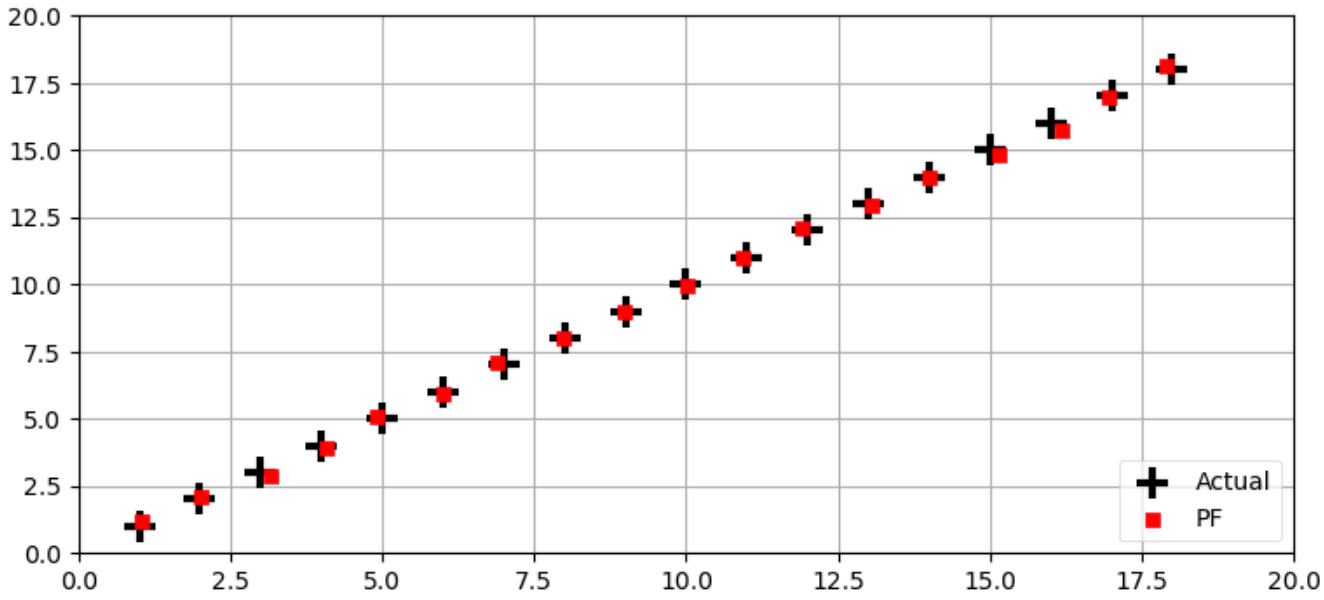
```
def resample_from_index(particles, weights, indexes):
    particles[:] = particles[indexes]
    weights.resize(len(particles))
    weights.fill (1.0 / len(weights))
```

为了实现该滤波器，我们需要先创建粒子以及各个参考点。接着，我们进入循环处理：依次调用 `predict`、`update`、重新采样函数，最后再利用 `estimate` 来计算新的状态估计值。

```
from filterpy.monte_carlo import systematic_resample
from numpy.linalg import norm
from numpy.random import randn
import scipy.stats

def run_pf1(N, iters=18, sensor_std_err=.1,
            do_plot=True, plot_particles=False,
            xlim=(0, 20), ylim=(0, 20),
```

```
            initial_x=None):
    landmarks = np.array([[-1, 2], [5, 10], [12,14], [18,21]])
    NL = len(landmarks)

    plt.figure()

    # create particles and weights
    if initial_x is not None:
        particles = create_gaussian_particles(
            mean=initial_x, std=(5, 5, np.pi/4), N=N)
    else:
        particles = create_uniform_particles((0,20), (0,20), (0, 6.28), N)
    weights = np.ones(N) / N

    if plot_particles:
        alpha = .20
        if N > 5000:
            alpha *= np.sqrt(5000)/np.sqrt(N)
        plt.scatter(particles[:, 0], particles[:, 1],
                    alpha=alpha, color='g')

    xs = []
    robot_pos = np.array([0., 0.])
    for x in range(iters):
        robot_pos += (1, 1)

        # distance from robot to each landmark
        zs = (norm(landmarks - robot_pos, axis=1) +
              (randn(NL) * sensor_std_err))

        # move diagonally forward to (x+1, x+1)
        predict(particles, u=(0.00, 1.414), std=(.2, .05))

        # incorporate measurements
        update(particles, weights, z=zs, R=sensor_std_err,
               landmarks=landmarks)

        # resample if too few effective particles
        if neff(weights) < N/2:
            indexes = systematic_resample(weights)
            resample_from_index(particles, weights, indexes)
            assert np.allclose(weights, 1/N)
        mu, var = estimate(particles, weights)
        xs.append(mu)

        if plot_particles:
            plt.scatter(particles[:, 0], particles[:, 1],
                        color='k', marker=',', s=1)
        p1 = plt.scatter(robot_pos[0], robot_pos[1], marker='+',
                         color='k', s=180, lw=3)
        p2 = plt.scatter(mu[0], mu[1], marker='s', color='r')

    xs = np.array(xs)
    #plt.plot(xs[:, 0], xs[:, 1])
    plt.legend([p1, p2], ['Actual', 'PF'], loc=4, numpoints=1)
    plt.xlim(*xlim)
    plt.ylim(*ylim)
    print('final position error, variance:\n\t', mu - np.array([iters, iters]), var)
    plt.show()

from numpy.random import seed
seed(2)
run_pf1(N=5000, plot_particles=False)
```

```
final position error, variance:
     [-0.106  0.106] [0.009 0.008]
```



这段代码的大部分内容都用于初始化和数据绘图。整个粒子滤波处理过程实际上就由这几行代码构成而已。

```
# move diagonally forward to (x+1, x+1)
predict(particles, u=(0.00, 1.414), std=(.2, .05))
# incorporate measurements
update(particles, weights, z=zs, R=sensor_std_err,
landmarks=landmarks)
# resample if too few effective particles
if neff(weights) < N/2:
indexes = systematic_resample(weights)
resample_from_index(particles, weights, indexes)
mu, var = estimate(particles, weights)
```

第一行调用预测函数。这里机器人沿直线运动，u[0] == 0；期望每步在 x、y 方向各前进 1 个单位，所以总行程为 u[1] == 1.414。转向误差的标准差为 0.2，运动距离误差的标准差为 0.05。调用后，粒子已按带噪运动模型传播，其权重随后还需根据当前测量更新。

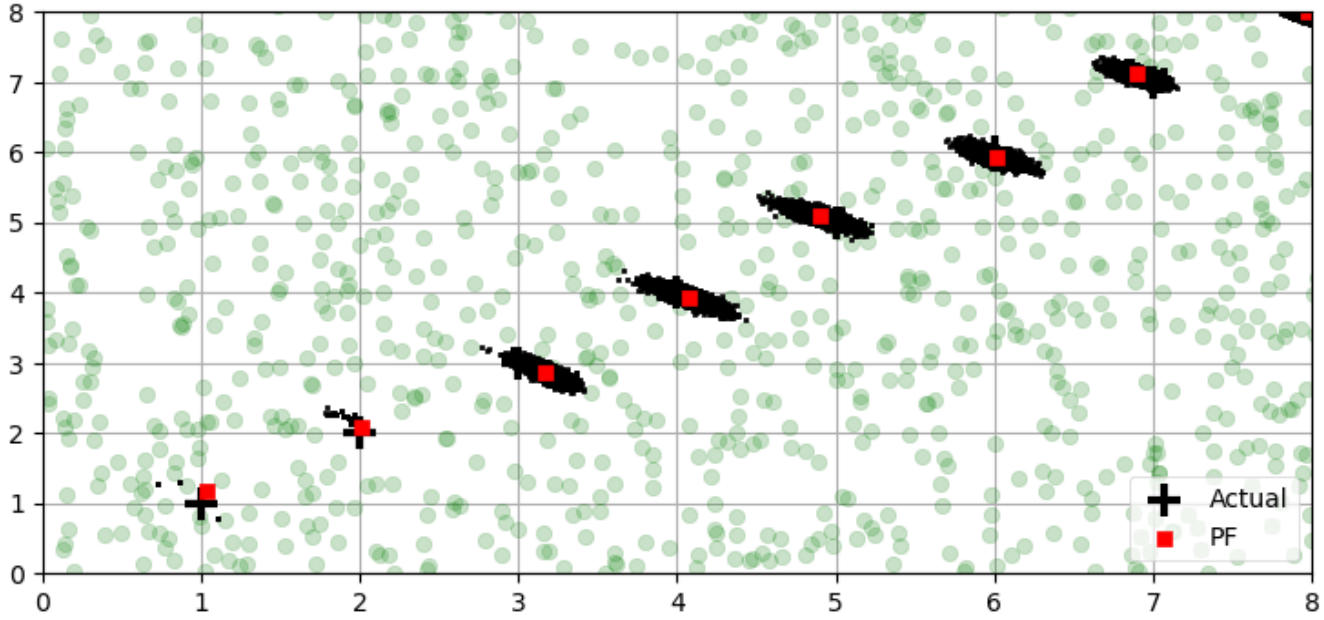
下一行代码将测量结果纳入了滤波过程中。这一操作不会改变粒子的位置，只会调整它们的权重。如果你还记得的话，粒子的权重实际上代表了该粒子与传感器误差模型所描述的高斯分布的匹配程度。粒子与实际测量值相差越远，它作为有效候选值的概率就越低。

最后两行检查有效样本量 ($\hat{N}_{\text{eff}}$)。若低于 $N/2$，便进行重采样，以改善粒子集合对后验分布的表示。

现在，让我们在绘制出所有粒子之后来观察这一现象。通过交互式的方式查看这一过程会更加直观明了，不过这种形式仍然能为我们提供有用的信息。我使用了非常浅绿色的圆圈来表示原始的随机分布点，以便与后续迭代中用黑色像素表示的粒子区分开来。由于粒子数量较多，很难看清细节，因此我将迭代次数限制在了 8 次，这样我们就可以放大视角，更仔细地观察了。

```
seed(2)
run_pf1(N=5000, iters=8, plot_particles=True,
        xlim=(0,8), ylim=(0,8))
```

```
final position error, variance:
     [-0.019 -0.005] [0.005 0.006]
```



从图上看，前两个机器人位置处似乎只有少数几个粒子。其实那里有 5,000 个粒子，只是重采样后大多相互重合。原因在于传感器噪声的高斯分布很窄。这种现象称为“样本贫化”，可能导致滤波发散，后面会详细讨论。先看第二步，即 x=2 时，粒子已经有所分散。这来自运动模型中的噪声：所有粒子都根据控制输入向前移动，同时分别加入与机器人控制误差相应的噪声。到第三步时，粒子已经充分分散，在机器人周围形成清晰的粒子云。

图中的粒子云近似呈椭圆形。在这个具体例子中，传感器噪声与运动扰动采用高斯模型，局部后验粒子云也近似高斯，因此呈现出这样的形状。不过，高斯噪声并不能保证非线性系统的后验一定为高斯分布。

需要认识到的是，粒子滤波算法并不要求传感器或系统的行为符合高斯分布或线性规律。由于我们用粒子云来表示概率分布，因此该算法能够处理各种概率分布以及高度非线性的问题。在概率模型中，完全可以存在不连续性或硬性限制。

传感器误差对滤波器的影响

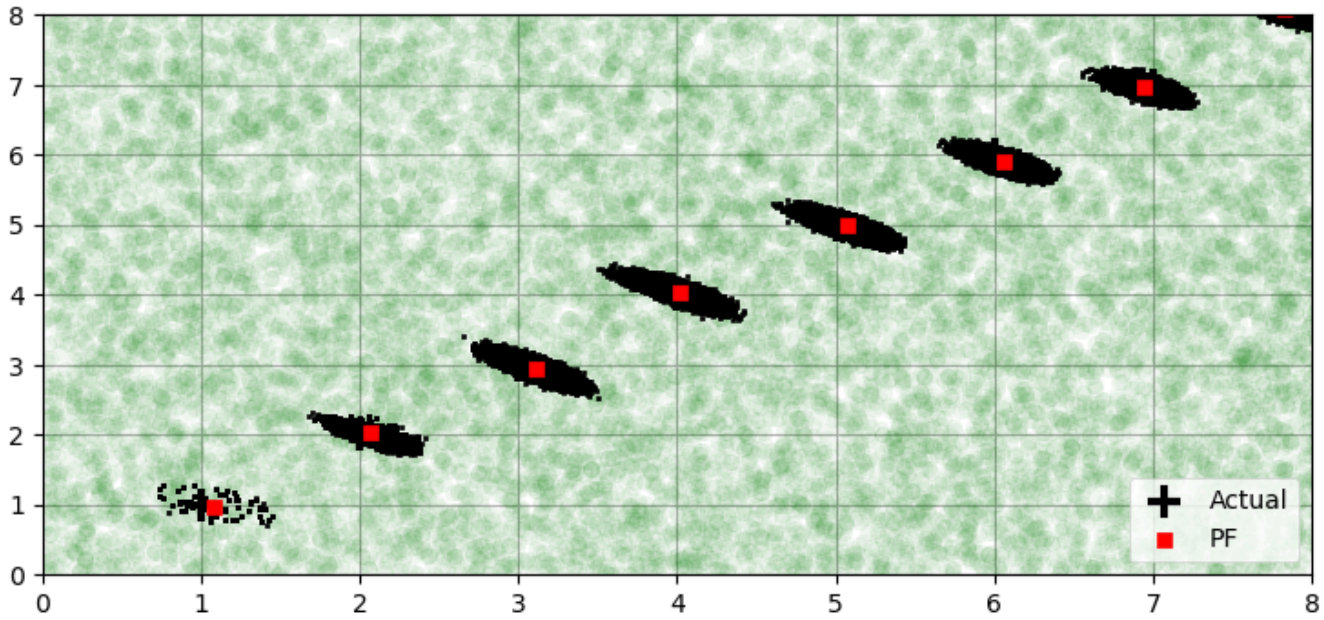
在滤波器的最初几次迭代过程中，产生了大量重复的粒子。出现这种情况的原因是：传感器的模型属于高斯模型，而我们给定了较小的标准差 $\boldsymbol{\sigma = 0.1}$。乍看之下，这种设定似乎不合常理。通常情况下，当噪声较小时，卡尔曼滤波器的性能会更好；但粒子滤波器的性能反而可能更差。

我们可以推理出为什么会出现这种情况。如果 $\boldsymbol{\sigma = 0.1}$，那么机器人位于(1, 1)点，而粒子位于(2, 2)点。此时，该粒子距离机器人有 14 个标准差之远。因此，该粒子对平均值估计几乎没有影响，而且在重新采样后，该粒子几乎不可能被保留下来。如果 $\boldsymbol{\sigma = 1.4}$，那么该粒子与机器人的距离仅为 $\boldsymbol{1\sigma}$，因此它会对平均值估计有所贡献。在重新采样过程中，该粒子很可能会被复制一次或多次。

这一点很重要：在粒子数有限、提议分布覆盖不足时，非常精确的传感器反而可能使滤波器表现不佳，因为只有极少数粒子与狭窄的测量似然相容。一种权宜办法是人为增大传感器噪声的标准差，让更多粒子获得可观的权重，但这改变了噪声模型，并非最优处理。根本问题是有效区域中的粒子不足，增加 N 通常有帮助，代价则是更多计算。下面看看采用 100,000 个粒子的结果。

```
seed(2)
run_pf1(N=100000, iters=8, plot_particles=True,
        xlim=(0,8), ylim=(0,8))
```

```
final position error, variance:
     [-0.17   0.084] [0.005 0.005]
```



在 x=1 的位置，有更多的粒子存在；而在 x=2 的位置，则形成了较为明显的“云状结构”。显然，这种滤波方式的效果更好，但代价是巨大的内存消耗和较长的处理时间。

另一种方法是更巧妙地生成初始粒子云。假设我们判断机器人位于(0, 0)附近。虽然这一判断并不完全准确，因为实际上机器人位于(1, 1)，但已经很接近了。如果在(0, 0)附近生成一个正态分布的粒子云，那么这些粒子就更有可能与机器人的实际位置相符。

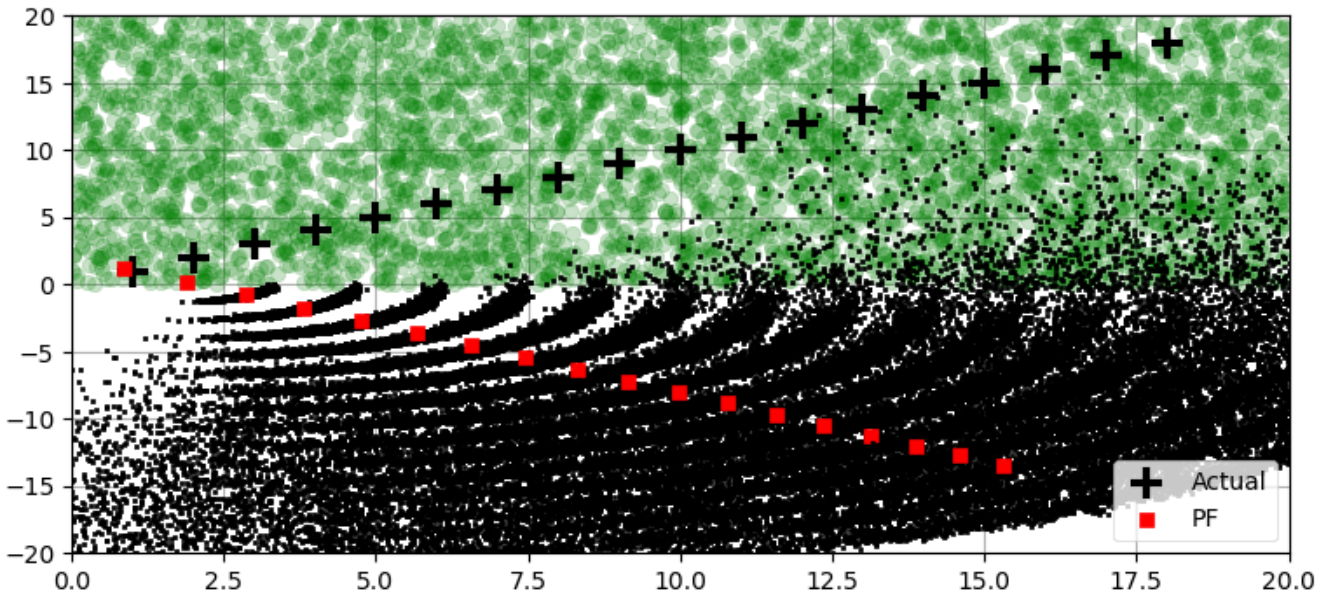
`run_pf1()` 具有一个可选参数 `initial_x` 。该参数用于指定机器人的初始位置估计值。之后，代码会利用 `create_gaussian_particles(mean, std, N)` 来在初始位置周围生成呈正态分布的粒子。我们将在下一节中使用这个功能。

样本不足导致的滤波退化

按照当前的实现方式，这个滤波器的效果远称不上完美。下面是使用不同随机种子时的测试结果。

```
seed(6)
run_pf1(N=5000, plot_particles=True, ylim=(-20, 20))
```

```
final position error, variance:
     [ -2.688 -31.47 ] [47.065 47.03 ]
```

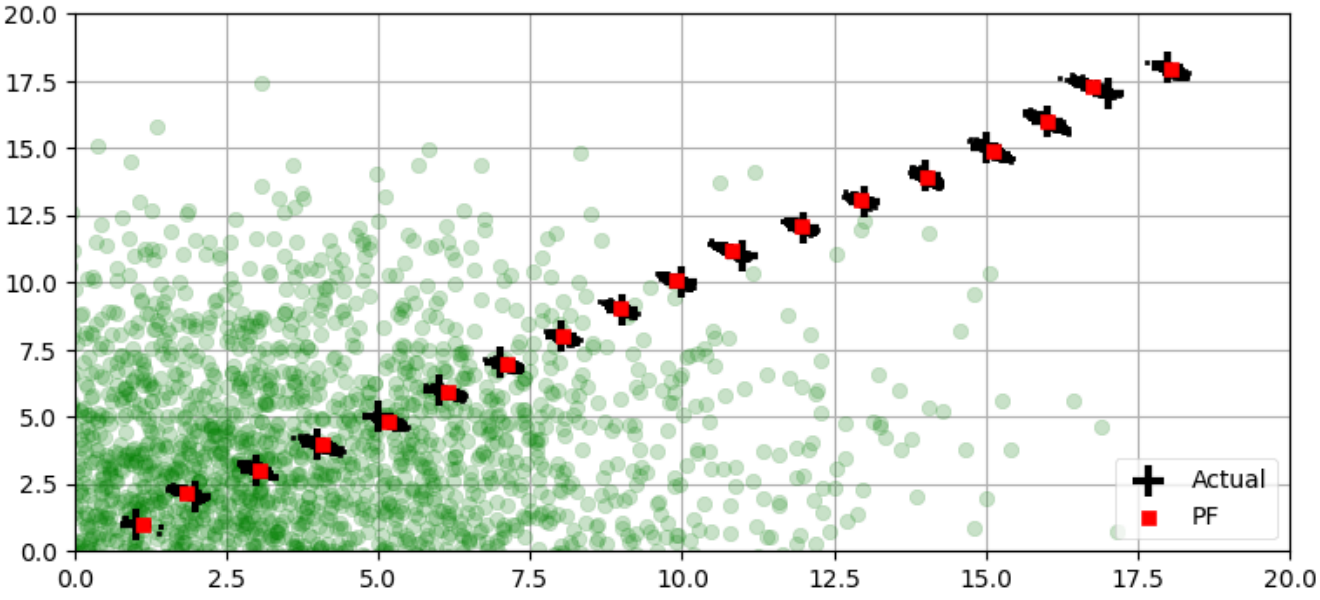


本例的初始采样没有在机器人附近放置足够粒子。重采样本身只复制现有粒子，不会创造新的状态位置，因此可能反复复制缺乏代表性的样本，造成样本贫化。随后，粒子无法良好匹配地标测量，形成弯曲的非线性分布，估计逐渐偏离实际。若预测扰动也无法让粒子到达真实状态附近，滤波器就难以重新收敛。

让我们利用 `create_gaussian_particles()` 方法来尝试在机器人附近生成更多的点。我们可以通过设置 `initial_x` 参数来指定生成粒子的位置。

```
seed(6)
run_pf1(N=5000, plot_particles=True, initial_x=(1,1, np.pi/4))
```

```
final position error, variance:
     [ 0.035 -0.077] [0.007 0.009]
```



这次效果很好。如果能够粗略估计初始位置，应尽量在该位置附近生成粒子，但不能将分布收得过窄，否则可能无法覆盖真实状态及系统的非线性变化。本例相对接近线性，因此可以采用较小的初始方差；具体取值仍取决于问题。增加粒子数也是改善样本覆盖的方法，但其计算成本可能难以接受。

重要性抽样

我试图用简单的方法来回避这个我们必须面对的难题。实际上，存在某种概率分布来描述机器人的位置和运动情况。我们希望从该概率分布中抽取一些样本点，然后利用蒙特卡洛方法来计算相关的积分值。

我们的难题在于：在很多情况下，我们并不知道数据的分布情况。例如，被追踪物体的运动方式可能与我们根据状态模型所预测的完全不同。既然不知道概率分布，那我们该如何从中抽取样本呢?

在统计学中，有一个被称为“重要性抽样”的定理[1]。有趣的是，这个定理为我们提供了一种从另一个已知概率分布中抽取样本的方法，进而利用这些样本来推断出那个未知概率分布的各个特性。这真是一个了不起的定理，让我感到非常高兴。

这个想法很简单，我们之前就已经用过这种方法了。我们从已知的概率分布中抽取样本，但会根据我们感兴趣的分布来给这些样本赋予不同的权重。这样，我们就可以通过计算这些样本的加权平均值和加权方差，从而得到诸如均值和方差之类的统计量。

机器人定位例子从状态模型给出的预测分布中生成样本，直觉是：“机器人先前在那里，按这个方向和速度运动，现在可能来到这里。”但实际运动可能与模型不同，例如发生跌落或碰撞。随后，利用测量似然更新粒子权重，使它们更好地表示后验分布。这种近似需要可计算的目标密度比和足够的支持集覆盖，不能自动纠正任意错误的运动模型，也不能恢复提议分布完全没有覆盖的区域。有限样本得到的均值与方差仍是近似值。

这怎么可能呢？我来给你详细解释一下数学原理。如果你不想深入研究机器人定位问题，可以跳过这部分内容。不过，对于其他涉及粒子滤波的问题，就需要采用不同的重要性抽样方法，此时数学知识就非常重要了。此外，大多数文献和网络上的资料都采用数学公式来阐述这些概念，而不是我那种不太精确的“想象一下……”的说法。如果你想真正理解这些文献，就必须掌握以下公式。

设目标分布为 $\pi(x)$，我们希望从中获取信息，但难以直接从 $\pi(x)$ 采样。于是引入容易采样的提议分布 $q(x)$。在机器人定位中，$\pi(x)$ 是希望估计的状态后验，$q(x)$ 可以选为运动预测分布。重要性采样通常要求能够计算目标密度，或与之成比例的量，而不要求能够直接从目标分布采样。

若随机变量服从分布 $\pi(x)$，函数 $f(x)$ 的期望为：

$$\mathbb{E}\big[f(x)\big] = \int f(x)\pi(x)\,dx$$

直接从 $\pi(x)$ 采样或计算相应积分可能很困难。若有一个便于采样的分布 $q(x)$，可以在被积函数中乘以再除以该密度，不改变积分值：

$$\mathbb{E}\big[f(x)\big] = \int f(x)\pi(x)\frac{q(x)}{q(x)}\,dx$$

现在我们来重新排列并整理各项。

$$\mathbb{E}[f(x)] = \int f(x)q(x) \cdot \frac{\pi(x)}{q(x)}\, dx$$

已知提议分布 $q(x)$，便能从它采样，并用蒙特卡洛方法估计形如 $\int f(x)q(x)$ 的积分。重写目标积分后出现密度比 $\pi(x)/q(x)$，即未归一化的重要性权重。对从 q 抽取的样本，将函数值与这个权重相乘，再取样本平均，就得到下式：

$$\mathbb{E}[f(x)] \approx \frac{1}{N}\sum_{i=1}^{N} f(x^i)w(x^i)$$

也许这听起来有点抽象。如果我们想计算这些粒子的平均值，那么我们需要进行相应的计算。

$$\mu = \sum_{i=1}^{N} x^i w^i$$

这与本章前面讨论的加权均值一致。下面使用总和为 1 的归一化权重，对各粒子的位置直接作加权求和。

要求各权重与比例 $\pi(x)/q(x)$ 成正比。由于我们通常无法知道该比例的具体数值，因此在实际操作中，我们会通过将各权重除以 $\sum w(x^i)$ 来进行归一化处理。

构建粒子滤波器时，需要针对具体模型实现这一步。对这里的机器人定位算法，一种方便的 $q(x)$ 选择是 predict() 步骤得到的预测分布。再看代码：

```
def update(particles, weights, z, R, landmarks):
for i, landmark in enumerate(landmarks):
dist = np.linalg.norm(particles[:, 0:2] - landmark, axis=1)
weights *= scipy.stats.norm(dist, R).pdf(z[i])
weights += 1.e-300 # avoid round-off to zero
weights /= sum(weights) # normalize
```

在这里，我们根据贝叶斯计算方法来确定权重值。 $\|\mathbf{likelihood} \times \mathbf{prior}\|$

若能够直接计算后验分布，当然可以直接计算；做不到时，重要性采样提供了一种近似办法。实际问题的后验计算往往很困难。卡尔曼滤波利用线性高斯模型的性质得到解析递推，因此取得了巨大成功。放宽线性与高斯等限制后，重要性采样和蒙特卡洛方法仍可提供可计算的近似解。本章的标准粒子滤波递推仍采用马尔可夫状态模型，需要针对系统建立适当的转移与测量概率模型。

重采样方法

重采样算法会影响滤波性能。例如，如果忽略权重，以相等概率从所有粒子中抽取，就可能选入许多权重很低的粒子，使新粒子集合无法良好表示目标概率分布。

这方面的研究仍在继续，已有几种算法在多种实际问题中表现良好。理想的算法应优先选择高权重粒子，同时保留具有代表性的样本，减少样本贫化；还应保持适当的多样性，使滤波器有机会跟踪强非线性行为。

FilterPy 实现了几种常用重采样算法。由于它不了解你的粒子数据结构，这些函数不直接构造新粒子，而是返回一个 NumPy 数组，保存选中粒子的索引。你需要据此完成复制与权重重置。例如，机器人例子中采用：

```
def resample_from_index(particles, weights, indexes):
    particles[:] = particles[indexes]
    weights.resize(len(particles))
    weights.fill(1.0 / len(weights))
```

多项分布重采样

多项分布重采样是前面开发机器人定位例子时介绍的算法。先计算归一化权重的累积和，得到非递减、最终达到 1 的数组。下图展示各权重在单位区间内所占的长度；颜色仅用于区分这些区间。

```
from kf_book.pf_internal import plot_cumsum
print('cumulative sume is', np.cumsum([.1, .2, .1, .6]))
plot_cumsum([.1, .2, .1, .6])
```

```
cumulative sume is [0.1 0.3 0.4 1. ]
```

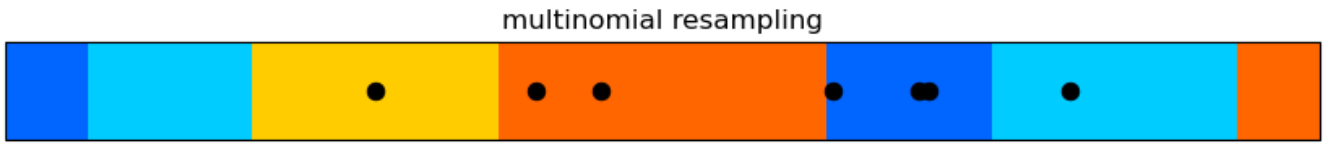
为选取粒子，先在 0 与 1 之间生成均匀随机数，再通过二分查找确定它在累计权重数组中的插入位置。权重大的粒子对应更长的区间，因此更容易被选中。

利用 NumPy 的数组操作，实现起来很简单。searchsorted 使用二分查找确定待查值在有序数组中的插入位置；传入一组待查值，就会返回对应的一组索引。这里用它把均匀随机采样位置映射为粒子索引。

```
def multinomal_resample(weights):
    cumulative_sum = np.cumsum(weights)
    cumulative_sum[-1] = 1.  # avoid round-off errors
    return np.searchsorted(cumulative_sum, random(len(weights)))
```

以下是一个例子：

```
from kf_book.pf_internal import plot_multinomial_resample
plot_multinomial_resample([.1, .2, .3, .4, .2, .3, .1])
```



这一实现的复杂度为 $O(n\log(n))$。这并不算太差，但还有复杂度为 $O(n)$、样本分布性质也更好的重采样方法。这里先介绍它，是为了帮助理解其他方法如何由此变化而来。多项分布重采样也有更快的实现，可从离散累积分布的广义逆采样角度理解；感兴趣可查阅有关资料。这里的“逆”不是数值倒数。

可从 FilterPy 导入 multinomial_resample 函数：

```
from filterpy.monte_carlo import multinomial_resample
```

残差重采样

残差重采样先作确定性复制，再对剩余份额随机采样。把归一化权重乘以粒子总数 N，再取整数部分，就得到每个粒子的确定性复制次数。例如，权重为 0.0012、N = 3000 时，缩放值为 3.6，先复制该粒子 3 次。因此，缩放后权重至少为 1 的粒子都会被保留。确定性复制部分的开销为 $O(N)$；总体开销还取决于剩余份额采用的采样实现。

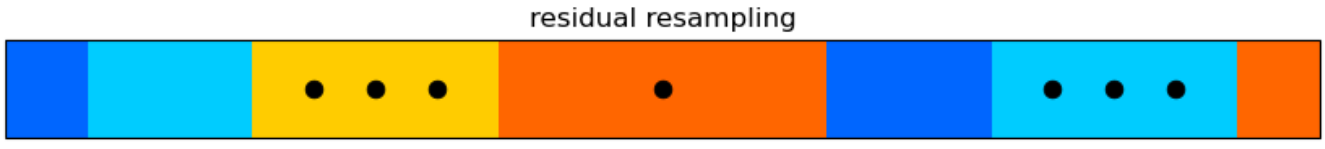
确定性复制通常不能填满全部 N 个名额。对剩余名额，先用缩放权重减去整数部分，得到 residual = N * np.asarray(weights) - num_copies，再归一化这些残差，并按残差随机采样。例如，上例的残差为 3.6 - int(3.6) = 0.6，因此该粒子还有机会再次被复制。残差越大，说明确定性截断留下的未分配份额越大，后续采样时就应获得更大权重。如果确定性复制已经填满 N 个名额，应直接返回，无须再对全为零的残差归一化。

```
def residual_resample(weights):
    N = len(weights)
    indexes = np.zeros(N, 'i')

    # take int(N*w) copies of each weight
    num_copies = (N*np.asarray(weights)).astype(int)
    k = 0
    for i in range(N):
        for _ in range(num_copies[i]): # make n copies
            indexes[k] = i
            k += 1

    if k == N:
        return indexes

    # use multinomial resampling on the residual to fill up the rest.
    residual = N * np.asarray(weights) - num_copies  # fractional copies
    residual /= sum(residual)     # normalize
    cumulative_sum = np.cumsum(residual)
    cumulative_sum[-1] = 1. # ensures sum is exactly one
    indexes[k:N] = np.searchsorted(cumulative_sum, random(N-k))

    return indexes
```

你可能会想用切片 `indexes[k:k + num_copies[i]] = i` 来替代内部的 for 循环，但实际上，过短的切片处理起来比较慢，而 for 循环的运行速度通常更快。

我们来看一个例子：

```
from kf_book.pf_internal import plot_residual_resample
plot_residual_resample([.1, .2, .3, .4, .2, .3, .1])
```



你可以使用 FilterPy 来导入这个模块。

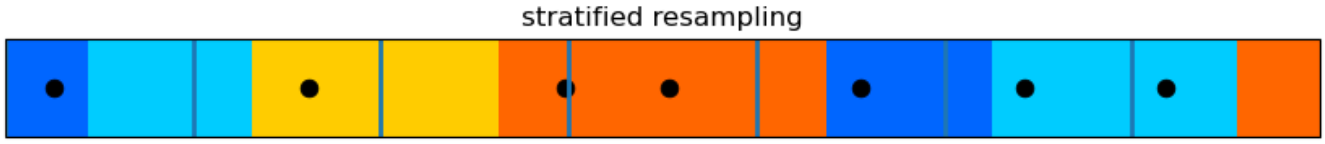
```
from filterpy.monte_carlo import residual_resample
```

分层重采样

分层重采样希望在累计概率区间上更均匀地布置采样位置。将单位区间分成 N 个等长分层，并在每层中独立均匀抽取一个位置，再映射到相应粒子。按顺序排列后，相邻采样位置的距离介于 0 与 $\frac{2}{N}$ 之间。这里的均匀性指累计概率区间中的采样位置，并非让所有粒子等概率入选。

下图说明这一过程：彩色条表示权重数组的累积和，黑线表示 N 个等宽分层。在每层中随机选取一个采样位置，图中以黑色圆点表示。

```
from kf_book.pf_internal import plot_stratified_resample
plot_stratified_resample([.1, .2, .3, .4, .2, .3, .1])
```



用于执行分层处理的代码相当简单易懂。

```
def stratified_resample(weights):
    N = len(weights)
    # make N subdivisions, chose a random position within each one
    positions = (random(N) + range(N)) / N

    indexes = np.zeros(N, 'i')
    cumulative_sum = np.cumsum(weights)
    i, j = 0, 0
    while i < N:
        if positions[i] < cumulative_sum[j]:
            indexes[i] = j
            i += 1
        else:
            j += 1
    return indexes
```

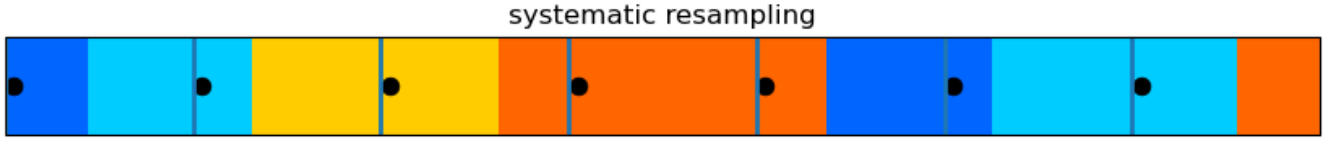
通过 FilterPy 将其导入。

```
from filterpy.monte_carlo import stratified_resample
```

系统重采样

最后介绍系统重采样（systematic resampling）。与分层重采样一样，先把单位区间分成 N 份；然后只生成一个随机偏移，供全部分区共同使用，从而使相邻采样位置恰好相距 $\frac{1}{N}$。如下图所示。

```
from kf_book.pf_internal import plot_systematic_resample
plot_systematic_resample([.1, .2, .3, .4, .2, .3, .1])
```



看了前面的例子后就会发现，这段代码简直再简单不过了。

```
def systematic_resample(weights):
    N = len(weights)

    # make N subdivisions, choose positions
    # with a consistent random offset
    positions = (np.arange(N) + random()) / N

    indexes = np.zeros(N, 'i')
    cumulative_sum = np.cumsum(weights)
    i, j = 0, 0
    while i < N:
        if positions[i] < cumulative_sum[j]:
            indexes[i] = j
            i += 1
        else:
            j += 1
    return indexes
```

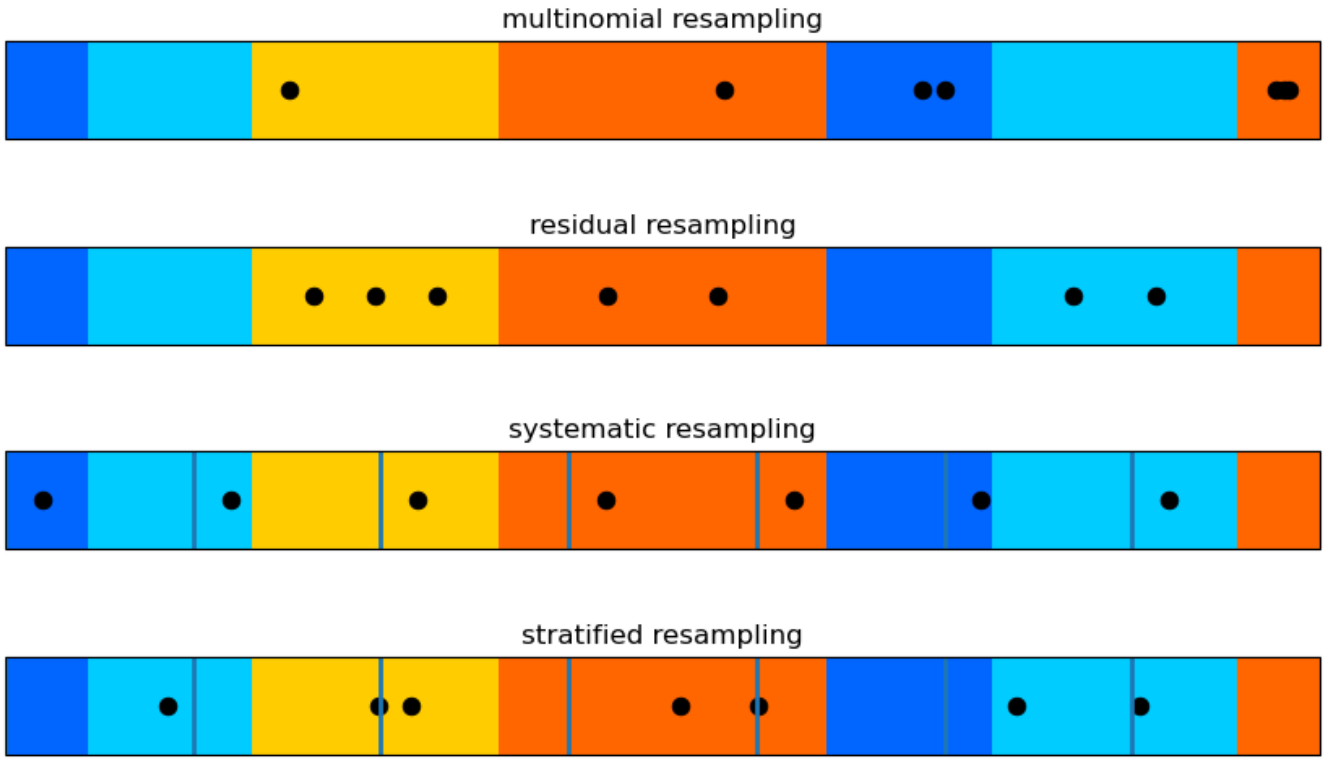
通过 FilterPy 进行导入

```
from filterpy.monte_carlo import systematic_resample
```

选择重采样算法

我们一起来看一下这四种算法吧，这样比较起来会更容易些。

```
a = [.1, .2, .3, .4, .2, .3, .1]
np.random.seed(4)
plot_multinomial_resample(a)
plot_residual_resample(a)
plot_systematic_resample(a)
plot_stratified_resample(a)
```



这一次试验中，多项分布重采样的结果不理想：有一个较大权重的粒子完全没有被选中，最大权重粒子只入选一次，最小权重粒子却入选两次。我读过的许多网络教程采用这种方法，但它的抽样方差较大。在实际应用中，我更倾向于选择下面介绍的替代算法，除非有充分理由采用它。

残差重采样很好地实现了预定目标：确保权重最大的粒子得到多个副本。不过，样本并没有均匀分布到各粒子上，一些权重相对较大的粒子在这次试验中也完全没有被选中。

在这次比较中，系统重采样与分层重采样都表现良好。系统重采样在累计概率区间中等间隔布置采样位置，使较大权重的粒子得到相应较多的副本。分层重采样的位置没有系统重采样那么规则，本图中较高权重粒子的复制次数也较为合适。不过，单次随机试验不能给出所有问题上的性能排序。

已有大量文献研究这些算法的理论性能，感兴趣可以查阅。实际中，我常把粒子滤波用于难以解析处理的问题，因此也会谨慎判断某项理论分析是否适用于当前情形。分层重采样和系统重采样在多种问题中都表现良好，效果往往相近。可以先尝试一种，若能满足要求就继续采用；若滤波性能至关重要，则应比较二者，并查阅针对具体问题的研究以获得更可靠的依据。

本章小结

本章只是对这个庞大主题的浅尝辄止。我的目的并非要教授你们整个领域的知识，而是让你们了解用于数据滤波的实际贝叶斯蒙特卡洛技术。

粒子滤波属于用样本集合表示分布的滤波方法。卡尔曼滤波则用高斯分布表示状态不确定性，利用贝叶斯定理纳入测量，并通过状态空间模型完成预测。这些运算直接作用于高斯分布的参数。

相比之下，样本集合方法用离散的点及其权重近似概率分布。测量信息用于更新这些样本的权重，系统模型则用于传播样本，而不是直接变换一个高斯分布的参数。随后再由所得样本集合计算所需统计量。

这些选择各有取舍。在线性、高斯等假设成立时，卡尔曼滤波效率很高，并能给出最优估计。处理非线性问题时，EKF 等方法需要作局部线性化，而单个高斯分布无法表示多峰后验。多峰分布与多个目标并非一回事；多目标也可用多个滤波器及数据关联方法处理。卡尔曼滤波还依赖所指定的状态模型，模型无法反映系统行为时，性能就可能很差。

粒子滤波则能近似表示形式复杂、没有方便解析表达式的概率分布。粒子数量充分且采样方案合适时，样本集合可准确逼近目标分布，也能用于强非线性问题。重要性采样避免了直接从目标后验采样的困难，但仍需要可计算的权重以及适当的提议分布；蒙特卡洛求和则替代难以直接计算的积分。

这些能力需要付出代价。最明显的是较高的计算量与内存需求；另一个难点是性能对实现与参数较敏感，需要谨慎避免权重退化、样本贫化及滤波发散。证明滤波器的正确性也可能很困难。处理多峰分布时，可能还需对粒子聚类，区分不同的状态假设；在多目标跟踪中，还需识别各目标轨迹。当目标相互接近时，这些任务尤其困难。

粒子滤波器有很多不同的类型；我在这里只介绍了最简单的 SIS 算法，随后又介绍了性能较好的 SIR 算法。实际上，粒子滤波器有很多种类，每种类型下又有许多具体的实现方式。要详细描述所有这些算法，恐怕需要写一本完整的书了。

在阅读有关粒子滤波的文献时，你会发现其中充斥着各种积分运算。我们在处理概率分布时也会使用积分，因此，积分这种表示方式让作者能够更简洁、高效地表达数学概念。你需要明白的是，当把这些数学公式转化为代码时，实际上是用粒子来表示各种概率分布，而积分则被替换为对粒子的求和操作。只要记住本章的核心思想，这些内容其实并不难理解。

参考文献

[1] 重要性抽样，维基百科。https://en.wikipedia.org/wiki/Importance_sampling

第 13 章：平滑

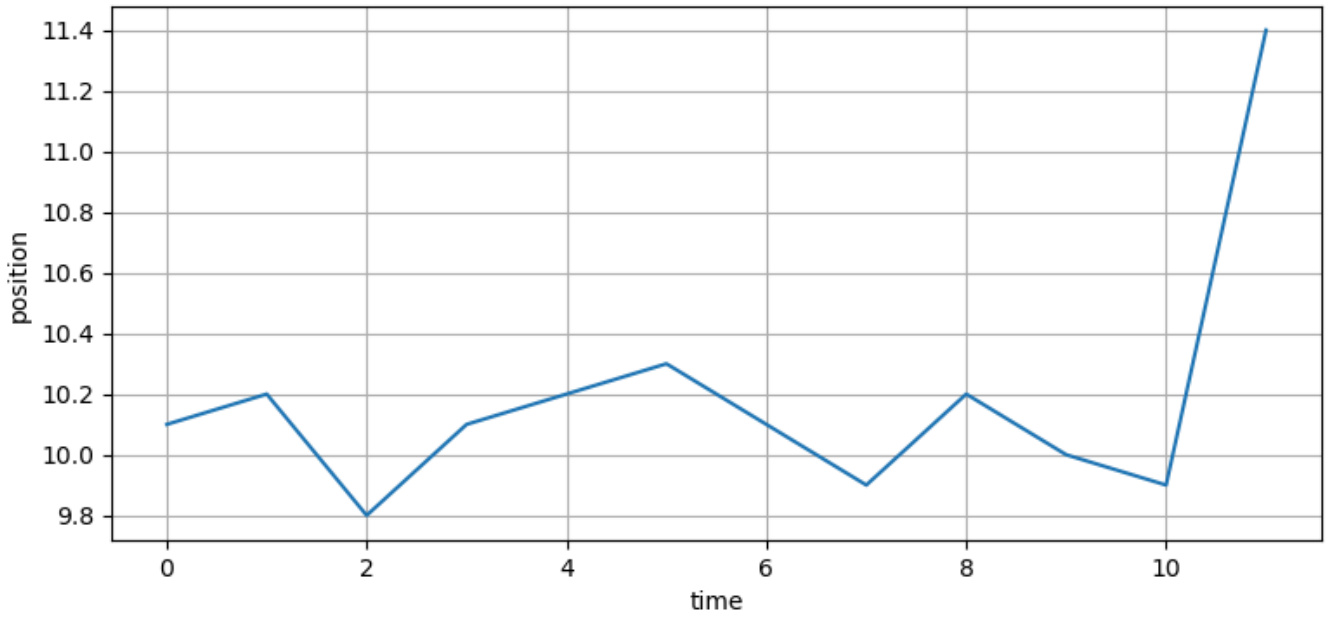
```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

如果还可以利用后续时刻的数据，仅依赖当前及历史数据的卡尔曼滤波估计就不再是最优的。例如，假设正在跟踪一架飞机，而最新测量值明显偏离当前轨迹，如下图所示。为简化讨论，这里只考虑一维情况。

```
import matplotlib.pyplot as plt

data = [10.1, 10.2, 9.8, 10.1, 10.2, 10.3,
        10.1, 9.9, 10.2, 10.0, 9.9, 11.4]
plt.plot(data)
plt.xlabel('time')
plt.ylabel('position');
```



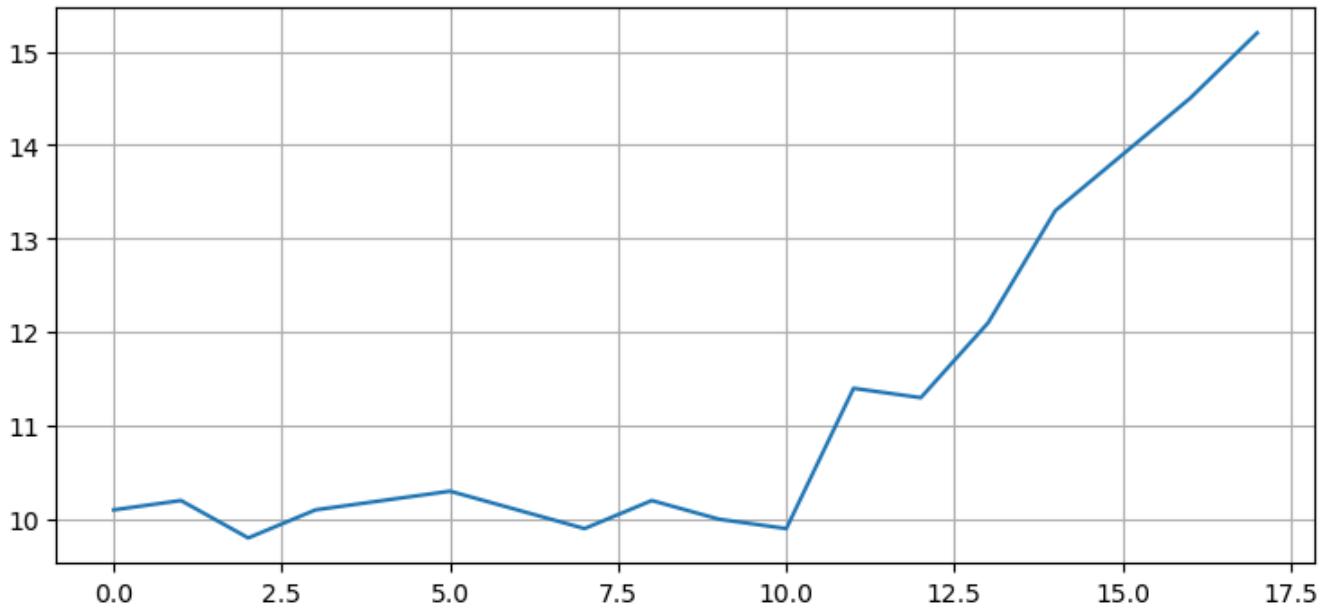
经过一段接近稳态的运动后，测量值突然发生了很大变化。假设按照这一读数推算的变化已超出飞机的飞行包线，普通卡尔曼滤波器仍会根据当前卡尔曼增益将它纳入更新。滤波器不能仅凭这一读数断定它是噪声，因为它也可能反映转弯的开始。飞机突然如此急转固然不太可能，但仅凭现有信息，还无法区分以下情况：

- 飞机其实早已开始转弯，但前几次测量受噪声影响，没有显现这一变化。
- 飞机正在转弯，而这次测量同时含有很大的噪声。
- 飞机没有转弯，这次测量只是含有很大的噪声。
- 飞机正在朝相反方向转弯，而这次测量的噪声极大。

现在，假设随后取得的测量值如下：

11.3 12.1 13.3 13.9 14.5 15.2

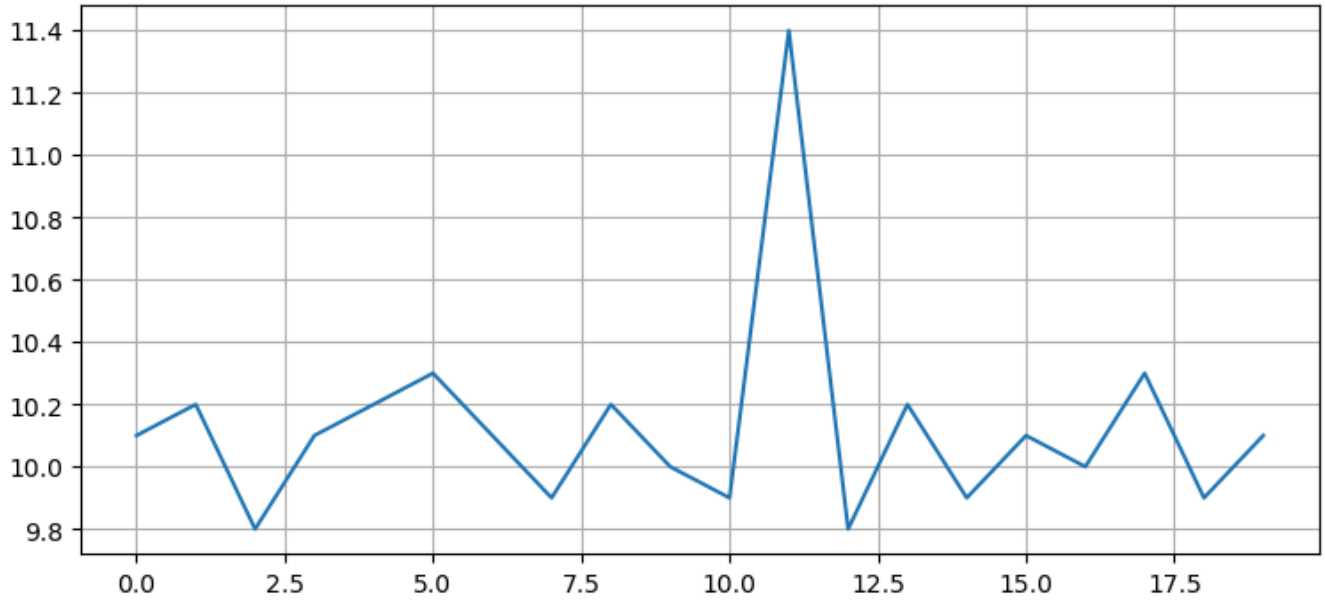
```
data2 = [11.3, 12.1, 13.3, 13.9, 14.5, 15.2]
plt.plot(data + data2);
```



利用这些后续测量，可以推断飞机确实开始了转弯。

反过来，假设后续测量呈现下面的情况。

```
data3 = [9.8, 10.2, 9.9, 10.1, 10.0, 10.3, 9.9, 10.1]
plt.plot(data + data3);
```



此时，更合理的结论是飞机没有转弯，先前那个偏离轨迹的测量值只是噪声很大。

平滑器的工作原理概述

卡尔曼滤波器是递推滤波器。在马尔可夫模型下，第 k 步的估计只需由第 k−1 步的估计和第 k 步的测量递推得到。不过，第 k−1 步的估计又包含第 k−2 步的信息，如此逐步追溯到最初时刻。因此，第 k 步的估计实际上已包含所有历史测量的信息，只是各次测量的影响不同。在这里讨论的典型跟踪问题中，较近的历史测量通常影响更大。

平滑器还将后续测量纳入第 k 步的估计。在这里讨论的典型跟踪问题中，第 k+1 步的测量通常影响较大，第 k+2 步的影响较小，第 k+3 步更小，依此类推。

这个主题称为“平滑”，不过这个名称可能引起误解。也可以用低通滤波器将上面的数据处理得平滑，但结果未必准确，因为低通滤波器在抑制噪声的同时，也可能抑制真实变化。相比之下，在线性、高斯及相应模型假设成立时，卡尔曼平滑器利用全部可用信息，给出均方误差意义下的最优状态估计。

平滑器的类型

下面三类卡尔曼平滑器，都可以改善上述情形中的跟踪结果。

- 固定区间平滑

这是一类批量处理方法：等待给定时间区间内的数据全部收集完毕，再计算平滑估计。例如，科学家收集实验数据，通常可以等实验结束后再分析结果。固定区间平滑器利用区间内全部过去与后续测量，估计各测量时刻的状态。如果应用允许批量处理，通常值得采用这类平滑器；在模型适当的条件下，它能比前几章只利用历史数据的滤波估计提供更准确的结果。

- 固定滞后平滑

固定滞后平滑器的输出带有延迟。例如，采用包含 4 个时刻的窗口：接收前 3 次测量时暂不输出最终的平滑结果，第 4 次测量到达后，利用第 1 至第 4 次测量给出第 1 个时刻的平滑估计；第 5 次测量到达后，再给出第 2 个时刻的平滑估计。这个窗口跨越 3 个采样间隔，早于窗口的信息仍可通过前向滤波结果保留。这适用于既需要较新数据，又能接受少量延迟的场景。例如，用机器视觉监控生产过程时，若允许估计延迟几秒，固定滞后平滑便可能给出准确而平稳的结果。

- 固定点平滑

固定点平滑器一方面像普通卡尔曼滤波器那样运行，另一方面持续改进某个固定时刻 j 的状态估计。在时间 k 尚未到达 j 时，先进行普通滤波。当 $k > j$ 时，除了估计当前状态 x_k，还利用 $j \dots k$ 之间新增的测量更新对 x_j 的估计。这可用于估计系统初始参数，或改进某个特定时刻的事件状态。例如，机器人在时刻 j 拍摄了一张照片；随着机器人继续运动，可以用固定点平滑不断改进相机在拍摄时刻 j 的位姿估计。

如何选择平滑器

平滑器的选择取决于应用需求，以及可用内存和计算时间。固定点平滑在估计当前状态的同时，持续更新某个指定历史时刻的估计，因此可以在线运行。标准递推实现不必保存全部原始测量，也不必在每次新测量到达时重新计算所有历史状态。

固定滞后平滑只需保留一个时间窗口所需的信息。每次新测量到达后，只更新这一窗口，因此计算量较为可控。它的缺点是输出始终滞后于输入，而且可利用的后续信息有限，通常不及固定区间平滑充分。

固定区间平滑能利用给定区间内全部数据，代价是必须等区间数据齐全后进行批量处理。许多算法采用一次前向滤波和一次后向递推，计算量通常约为单次递推卡尔曼滤波的两倍。

固定区间平滑

文献中有许多固定区间平滑方法。我选择实现 Rauch、Tung 和 Striebel 提出的平滑器，因为它容易实现、计算效率高，也是在实际应用中很常见的方法，通常简称 RTS 平滑器。

RTS 平滑器的推导需要好几页密集的数学公式，这里不展开，而是简要给出算法与方程，再直接讨论实现和示例。

RTS 平滑器先运行一次前向卡尔曼滤波，得到各时刻的状态估计及对应协方差，然后从最后时刻向前递推，将后续观测带来的信息纳入较早时刻的状态估计。回溯到第一个时刻后，整个过程完成。在适用的线性高斯模型下，每个平滑估计都充分利用了给定区间内的全部测量信息。

RTS 平滑方程简洁，也容易实现。下面针对线性卡尔曼滤波器给出方程；EKF 和 UKF 也有相应的平滑方法。将前向批量滤波结果从最新时刻向前处理，直到最早时刻。每次迭代都将后续信息纳入当前正在修正的状态估计；而前向估计已经包含过去测量，因此平滑结果同时利用了过去与后续的测量。这里必须区分各时刻，所以用下标明确时间关系。

预测步骤

$$\mathbf{P} = \mathbf{F}\mathbf{P}_k\mathbf{F}^\mathsf{T} + \mathbf{Q}$$

更新步骤

$$\begin{aligned}
\mathbf{K}_k &= \mathbf{P}_k\mathbf{F}^\mathsf{T}\mathbf{P}^{-1} \\
\mathbf{x}_k &= \mathbf{x}_k + \mathbf{K}_k(\mathbf{x}_{k+1} - \mathbf{F}\mathbf{x}_k) \\
\mathbf{P}_k &= \mathbf{P}_k + \mathbf{K}_k(\mathbf{P}_{k+1} - \mathbf{P})\mathbf{K}_k^\mathsf{T}
\end{aligned}$$

实现时，最容易出错的仍然是时间下标。省略注释和错误检查后的基本实现如下：

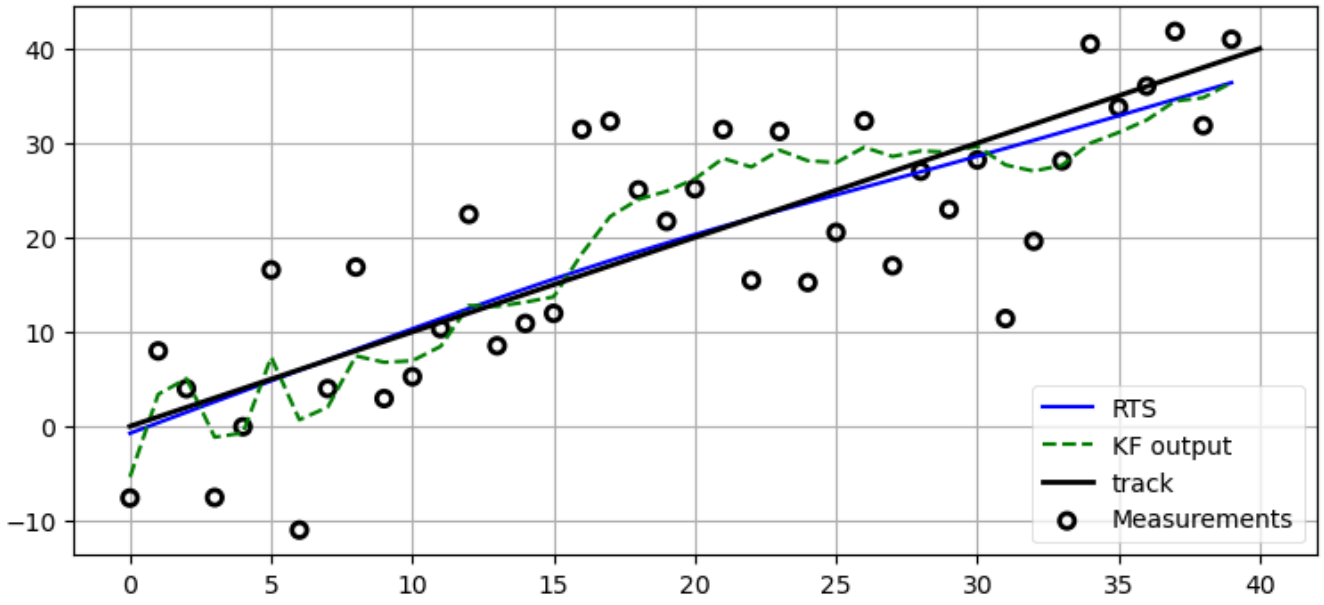
```
def rts_smoother(Xs, Ps, F, Q):
n, dim_x, _ = Xs.shape
# smoother gain
K = zeros((n,dim_x, dim_x))
x, P, Pp = Xs.copy(), Ps.copy(), Ps.copy()
for k in range(n-2,-1,-1):
Pp[k] = F @ P[k] @ F.T + Q # predicted covariance
K[k] = P[k] @ F.T @ inv(Pp[k])
x[k] += K[k] @ (x[k+1] - (F @ x[k]))
P[k] += K[k] @ (P[k+1] - Pp[k]) @ K[k].T
return (x, P, K, Pp)
```

这段代码对应 FilterPy 中的实现思路。它假定已在外部批量运行卡尔曼滤波，并通过 Xs 和 Ps 传入状态估计与协方差。copy() 创建数组副本，使平滑递推不会覆盖传入的滤波结果。

以下是一个例子。

```
import numpy as np
from numpy import random
from numpy.random import randn
import matplotlib.pyplot as plt
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise
import kf_book.book_plots as bp

def plot_rts(noise, Q=0.001, show_velocity=False):
    random.seed(123)
    fk = KalmanFilter(dim_x=2, dim_z=1)

    fk.x = np.array([0., 1.])      # state (x and dx)

    fk.F = np.array([[1., 1.],
                     [0., 1.]])    # state transition matrix

    fk.H = np.array([[1., 0.]])    # Measurement function
    fk.P*= 10.                     # covariance matrix
    fk.R = noise                   # state uncertainty
    fk.Q = Q_discrete_white_noise(dim=2, dt=1., var=Q)  # process uncertainty

    # create noisy data
    zs = np.asarray([t + randn()*noise for t in range (40)])

    # filter data with Kalman filter, than run smoother on it
    mu, cov, _, _ = fk.batch_filter(zs)
    M, P, C, _ = fk.rts_smoother(mu, cov)

    # plot data
    if show_velocity:
        index = 1
```

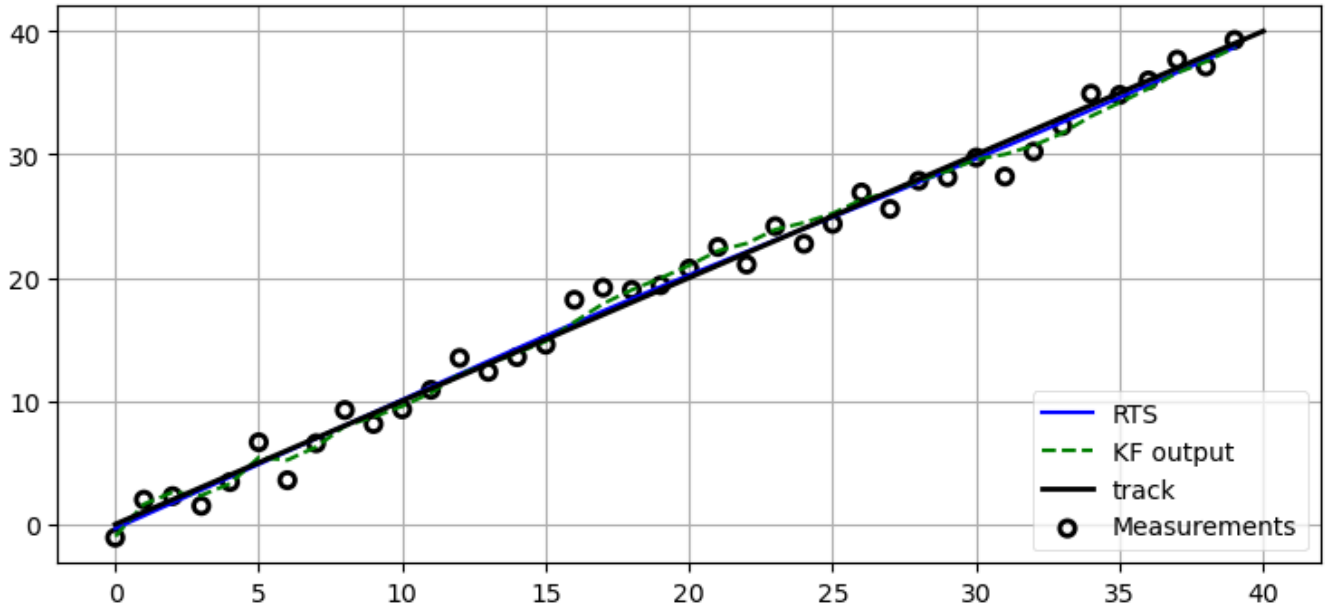
```
        print('gu')
    else:
        index = 0
    if not show_velocity:
        bp.plot_measurements(zs, lw=1)
    plt.plot(M[:, index], c='b', label='RTS')
    plt.plot(mu[:, index], c='g', ls='--', label='KF output')
    if not show_velocity:
        N = len(zs)
        plt.plot([0, N], [0, N], 'k', lw=2, label='track')
    plt.legend(loc=4)
    plt.show()

plot_rts(7.)
```



我在信号中加入了大量噪声，便于区分 RTS 输出与理想输出。上图中的绿色虚线表示卡尔曼滤波结果，相比输入已相当平滑；但当连续几次测量都偏向理想直线的一侧时，估计仍会随之偏离。相比之下，RTS 输出既平滑，又很接近理想输出。

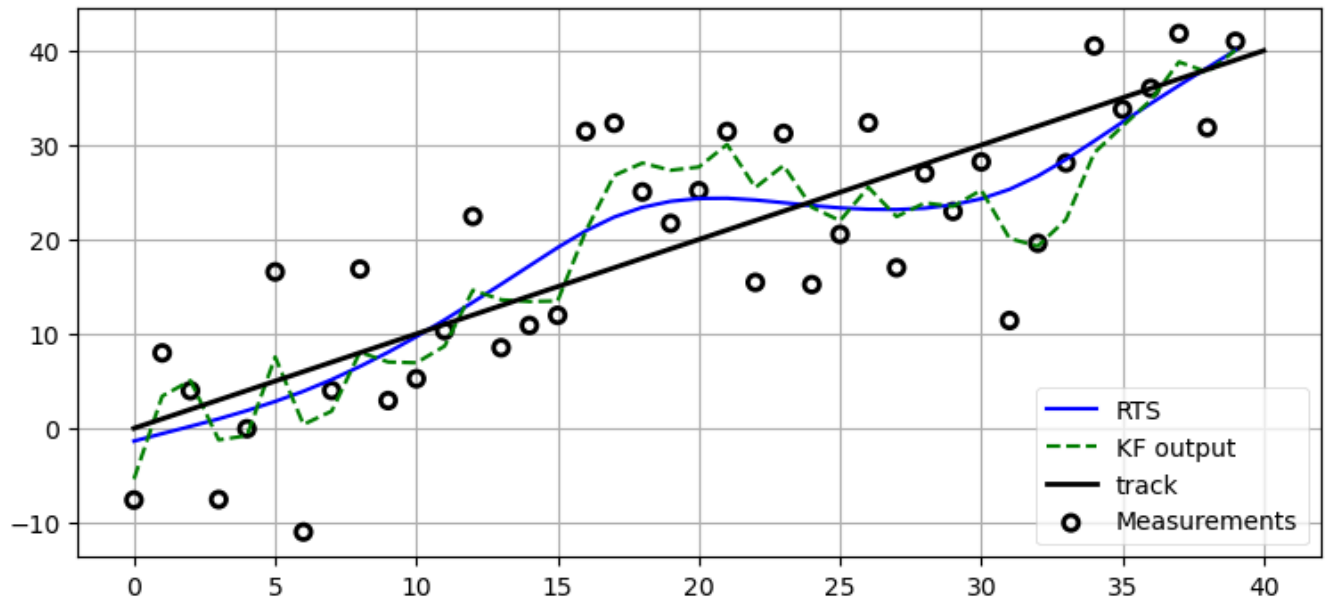
将噪声调整为更合理的水平后，RTS 输出几乎与理想输出重合。卡尔曼滤波结果虽也有所改善，但波动幅度仍大得多。

```
plot_rts(noise=1.)
```



必须理解，这种平滑效果以系统模型为基础。我们告诉滤波器：目标符合匀速模型，过程噪声很小。利用后续测量时，平滑器发现其行为与匀速模型十分吻合，因此能抑制信号中的大部分噪声。如果过程噪声很大，例如在阵风中跟踪轻型飞机，目标速度经常变化，就更难区分测量噪声与风引起的真实不规则运动。下一幅图展示了这种情况。

```
plot_rts(noise=7., Q=.1)
```

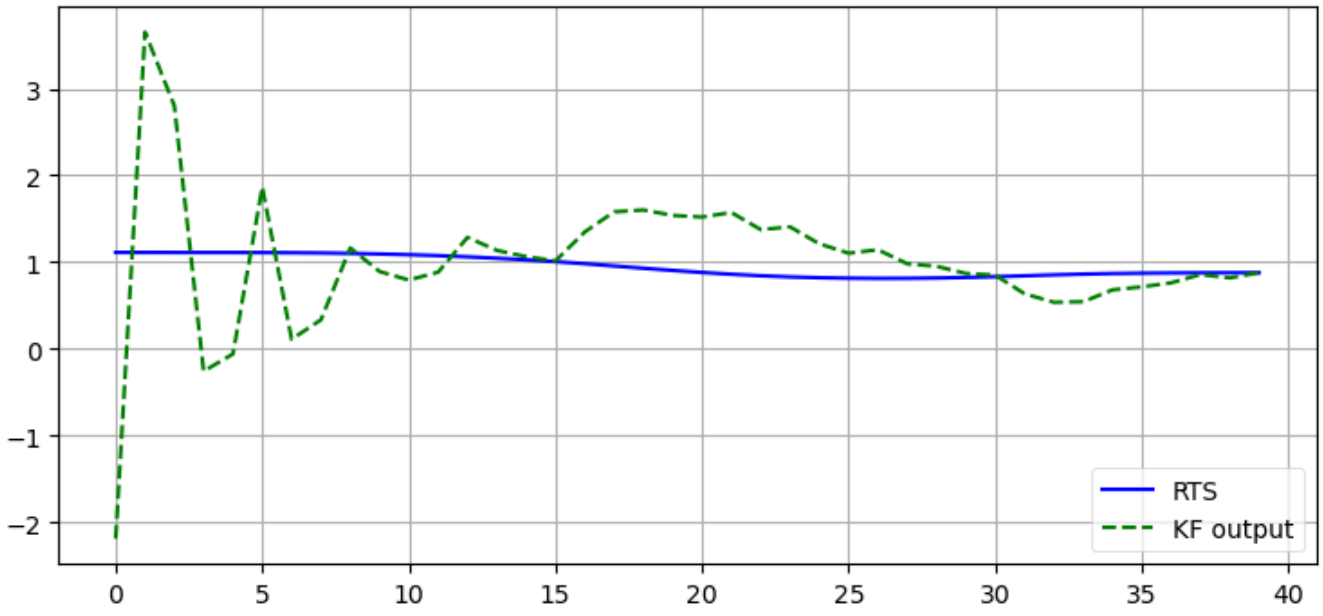


这再次表明，平滑器并不是单纯把曲线处理得平滑。它依据历史与后续测量，以及关于系统行为、过程噪声和测量噪声的模型，计算最优状态估计。

最后，比较卡尔曼滤波器与 RTS 平滑器给出的速度估计。

```
plot_rts(7.,show_velocity=True)
```

```
gu
```

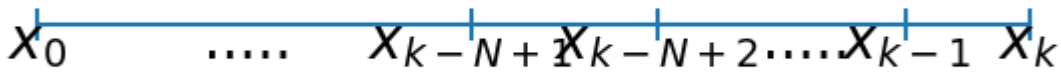


对于速度这个隐变量，估计效果的改善更为显著。

固定滞后平滑

如果允许批量处理，前面介绍的 RTS 平滑器通常是很好的选择，因为它将全部可用测量纳入各时刻的估计。但有些问题不允许等待所有数据到齐，同时仍希望改进近期历史状态的估计。下图说明这一思路。

```
from kf_book.book_plots import figsize
from kf_book.smoothing_internal import *

with figsize(y=2):
    show_fixed_lag_numberline()
```



在第 k 步，可以用普通卡尔曼滤波方程估计 x_k；还可以利用为状态 x_k 取得的新测量，改进对 x_{k-1} 的估计。同样，利用状态 x_{k-1} 和 x_k 对应的测量，可以改进对 x_{k-2} 的估计。这种计算可以向前回溯任意选定的 N 个时间步。

详细推导超出本书范围；Dan Simon 的《最优状态估计》[2] 有很好的介绍。其核心思想是将原状态向量 $\mathbf{x}$ 扩充为包含一段历史状态的增广状态：

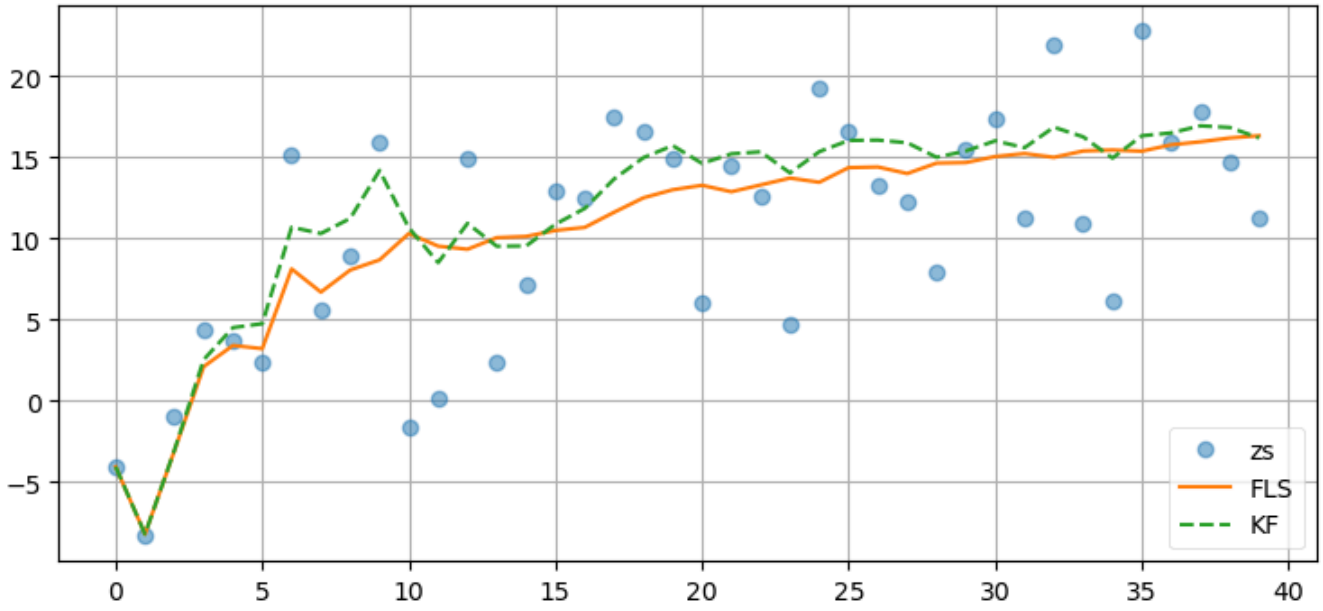
$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{x}_{k-1} \\ \vdots \\ \mathbf{x}_{k-N+1} \end{bmatrix}$$

这样得到的增广协方差矩阵包含不同时刻状态之间的协方差。FilterPy 的 FixedLagSmoother 类封装了相应计算。像配置 KalmanFilter 一样配置它，再调用 smooth()，即可完成该算法的预测、更新与历史状态平滑。

每次调用 smooth，不仅计算当前时刻的状态估计，还回溯修正近期历史估计。平滑结果保存在 FixedLagSmoother.xSmooth 列表中。FixedLagSmoother.x 则是当前最新估计，尚未利用后续测量进行平滑，因此与标准卡尔曼滤波输出相同。

```
from filterpy.kalman import FixedLagSmoother, KalmanFilter
import numpy.random as random

fls = FixedLagSmoother(dim_x=2, dim_z=1, N=8)

fls.x = np.array([0., .5])
fls.F = np.array([[1.,1.],
                  [0.,1.]])

fls.H = np.array([[1.,0.]])
fls.P *= 200
fls.R *= 5.
fls.Q *= 0.001

kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = np.array([0., .5])
kf.F = np.array([[1.,1.],
                 [0.,1.]])
kf.H = np.array([[1.,0.]])
kf.P *= 200
kf.R *= 5.
kf.Q = Q_discrete_white_noise(dim=2, dt=1., var=0.001)

N = 4 # size of lag

nom =  np.array([t/2. for t in range (0, 40)])
zs = np.array([t + random.randn()*5.1 for t in nom])

for z in zs:
    fls.smooth(z)

kf_x, _, _, _ = kf.batch_filter(zs)
x_smooth = np.array(fls.xSmooth)[:, 0]


fls_res = abs(x_smooth - nom)
kf_res = abs(kf_x[:, 0] - nom)

plt.plot(zs,'o', alpha=0.5, label='zs')
plt.plot(x_smooth, label='FLS')
plt.plot(kf_x[:, 0], label='KF', ls='--')
plt.legend(loc=4)

print(f'standard deviation fixed-lag: {np.mean(fls_res):.3f}')
print(f'standard deviation kalman: {np.mean(kf_res):.3f}')
```

```
standard deviation fixed-lag: 2.616
standard deviation kalman: 3.564
```



这里设置 N=8，让近期状态估计能够利用后续测量进行修正。此实现每次处理当前点及前 7 个历史点。滤波器收敛后，结果相当平滑，计算量则约为标准卡尔曼滤波的 8 倍。可以尝试更大或更小的 N；选择 8 主要是为了演示，并无特殊的理论依据。

参考文献

- [1] H. Rauch、F. Tung 和 C. Striebel。“线性动态系统的最大似然估计”，《AIAA 杂志》，第 3 卷第 8 期，第 1445-1450 页（1965 年 8 月）。http://arc.aiaa.org/doi/abs/10.2514/3.3166
- [2] 丹·西蒙著。《最优状态估计》，约翰·威利父子出版社，2006 年。

第 14 章：自适应滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

此前讨论的跟踪对象，其行为都与过程模型比较吻合。例如，用匀速滤波器跟踪沿直线运动的目标，只要速度大致恒定，或轨迹和速度变化很慢，效果就很好。但若跟踪的是机动目标，例如沿道路行驶的汽车、飞行中的飞机，原先的滤波器可能表现很差。再如跟踪海上帆船，即使建立了控制输入模型，也难以准确建模风与洋流。

一种初步办法是增大过程噪声协方差 $\mathbf{Q}$，以容纳系统动力学中不可预测的变化。这可能有助于避免滤波发散，但结果通常远非最优。更大的 $\mathbf{Q}$ 使滤波器更依赖测量，因而也更容易受测量噪声影响。稍后将给出例子。

在本章中，我们将探讨自适应滤波器的概念。当系统检测到过程模型无法解释的动态变化时，自适应滤波器会自动调整自身的参数以适应这些变化。我会先通过一个具体的例子来说明这个问题，然后再详细介绍各种自适应滤波器的实现方式。

机动目标

首先需要模拟机动目标。下面实现一个带转向输入的简单二维模型。给定新的速率和／或方向后，模型会相应调整其状态。

```
from math import sin, cos, radians

def angle_between(x, y):
  return min(y-x, y-x+360, y-x-360, key=abs)

class ManeuveringTarget(object):
    def __init__(self, x0, y0, v0, heading):
        self.x = x0
        self.y = y0
        self.vel = v0
        self.hdg = heading

        self.cmd_vel = v0
        self.cmd_hdg = heading
        self.vel_step = 0
        self.hdg_step = 0
        self.vel_delta = 0
        self.hdg_delta = 0

    def update(self):
        vx = self.vel * cos(radians(90-self.hdg))
```

```
        vy = self.vel * sin(radians(90-self.hdg))
        self.x += vx
        self.y += vy

        if self.hdg_step > 0:
            self.hdg_step -= 1
            self.hdg += self.hdg_delta

        if self.vel_step > 0:
            self.vel_step -= 1
            self.vel += self.vel_delta
        return (self.x, self.y)

    def set_commanded_heading(self, hdg_degrees, steps):
        self.cmd_hdg = hdg_degrees
        self.hdg_delta = angle_between(self.cmd_hdg,
                                       self.hdg) / steps
        if abs(self.hdg_delta) > 0:
            self.hdg_step = steps
        else:
            self.hdg_step = 0

    def set_commanded_speed(self, speed, steps):
        self.cmd_vel = speed
        self.vel_delta = (self.cmd_vel - self.vel) / steps
        if abs(self.vel_delta) > 0:
            self.vel_step = steps
        else:
            self.vel_step = 0
```

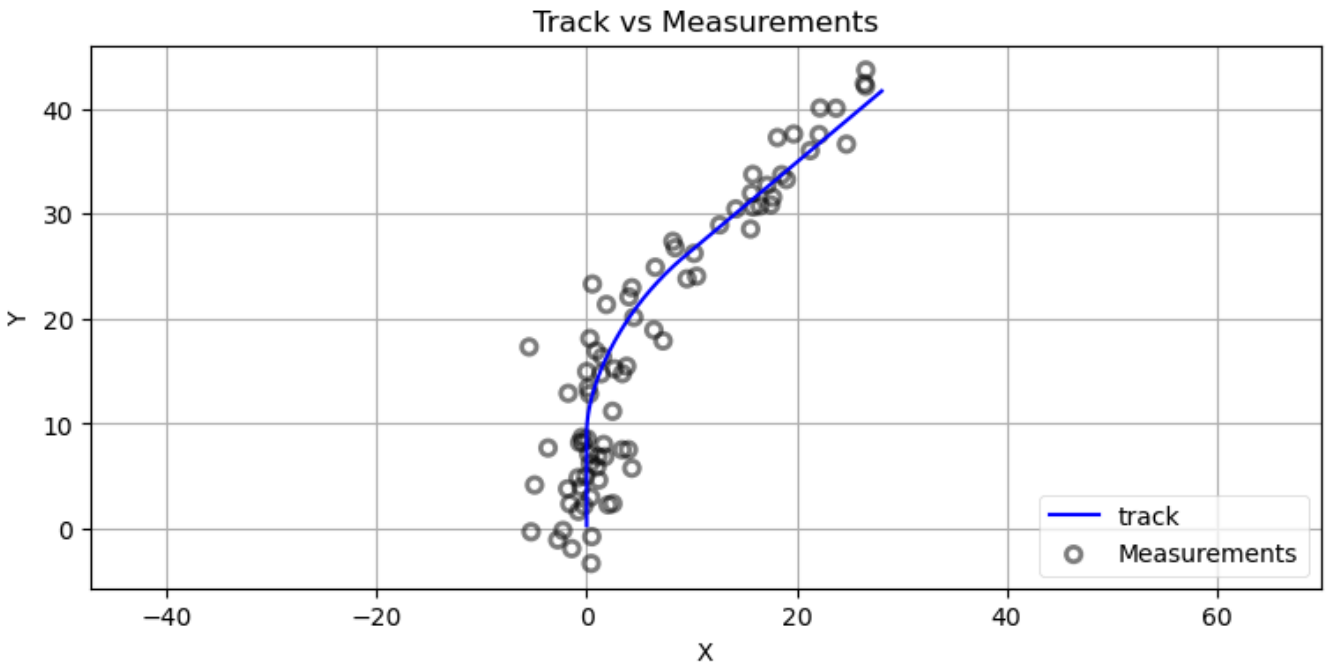
现在，让我们来实现一个带有噪声的模拟传感器吧。

```
from numpy.random import randn

class NoisySensor(object):
    def __init__(self, std_noise=1.):
        self.std = std_noise

    def sense(self, pos):
        """Pass in actual position as tuple (x, y).
        Returns position with noise added (x,y)"""

        return (pos[0] + (randn() * self.std),
                pos[1] + (randn() * self.std))
```

现在，让我们生成一条路径并将其绘制出来，以确认一切正常。我会把数据生成的过程封装在一个函数中，这样我们就可以创建出不同长度的路径了（具体原因稍后就会明白）。

```
import kf_book.book_plots as bp
import numpy as np
import matplotlib.pyplot as plt

def generate_data(steady_count, std):
    t = ManeuveringTarget(x0=0, y0=0, v0=0.3, heading=0)
    xs, ys = [], []

    for i in range(30):
        x, y = t.update()
        xs.append(x)
        ys.append(y)

    t.set_commanded_heading(310, 25)
    t.set_commanded_speed(1, 15)

    for i in range(steady_count):
        x, y = t.update()
        xs.append(x)
```

```
        ys.append(y)

    ns = NoisySensor(std)
    pos = np.array(list(zip(xs, ys)))
    zs = np.array([ns.sense(p) for p in pos])
    return pos, zs

sensor_std = 2.
track, zs = generate_data(50, sensor_std)
plt.figure()
bp.plot_measurements(*zip(*zs), alpha=0.5)
plt.plot(*zip(*track), color='b', label='track')
plt.axis('equal')
plt.legend(loc=4)
bp.set_labels(title='Track vs Measurements', x='X', y='Y')
```



较大的噪声便于观察不同设计选择的影响。

现在，我们可以利用卡尔曼滤波器来追踪这个物体了。不过，我们可以先进行简化处理：由于 x 坐标和 y 坐标是相互独立的，因此我们可以分别独立地追踪这两个坐标。在本章的后续内容中，我们将只追踪 x 坐标，这样可以尽量简化代码和相关的数学运算。

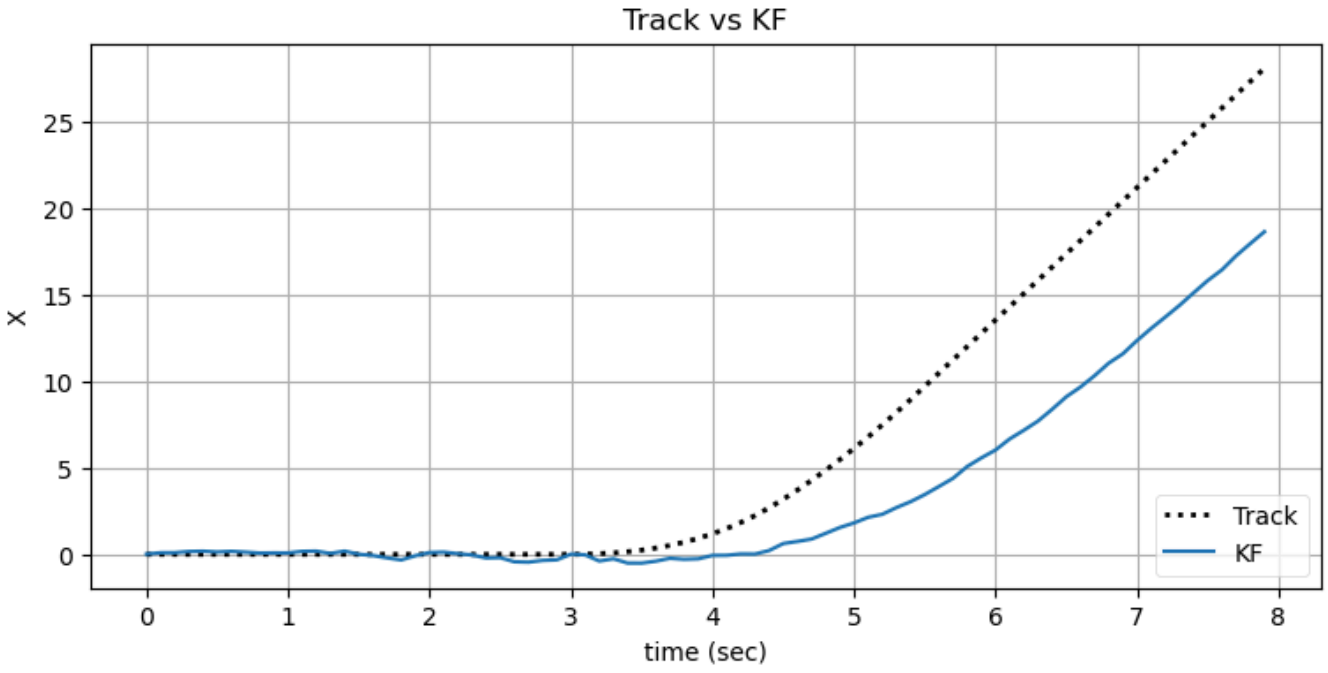
先从匀速滤波器开始。

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

def make_cv_filter(dt, std):
    cvfilter = KalmanFilter(dim_x = 2, dim_z=1)
    cvfilter.x = np.array([0., 0.])
    cvfilter.P *= 3
    cvfilter.R *= std**2
    cvfilter.F = np.array([[1, dt],
                           [0,  1]], dtype=float)
    cvfilter.H = np.array([[1, 0]], dtype=float)
    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
    return cvfilter

def initialize_filter(kf, std_R=None):
    """ helper function - we will be reinitialing the filter
```

```
    many times.
    """
    kf.x.fill(0.)
    kf.P = np.eye(kf.dim_x) * .1
    if std_R is not None:
        kf.R = np.eye(kf.dim_z) * std_R
```

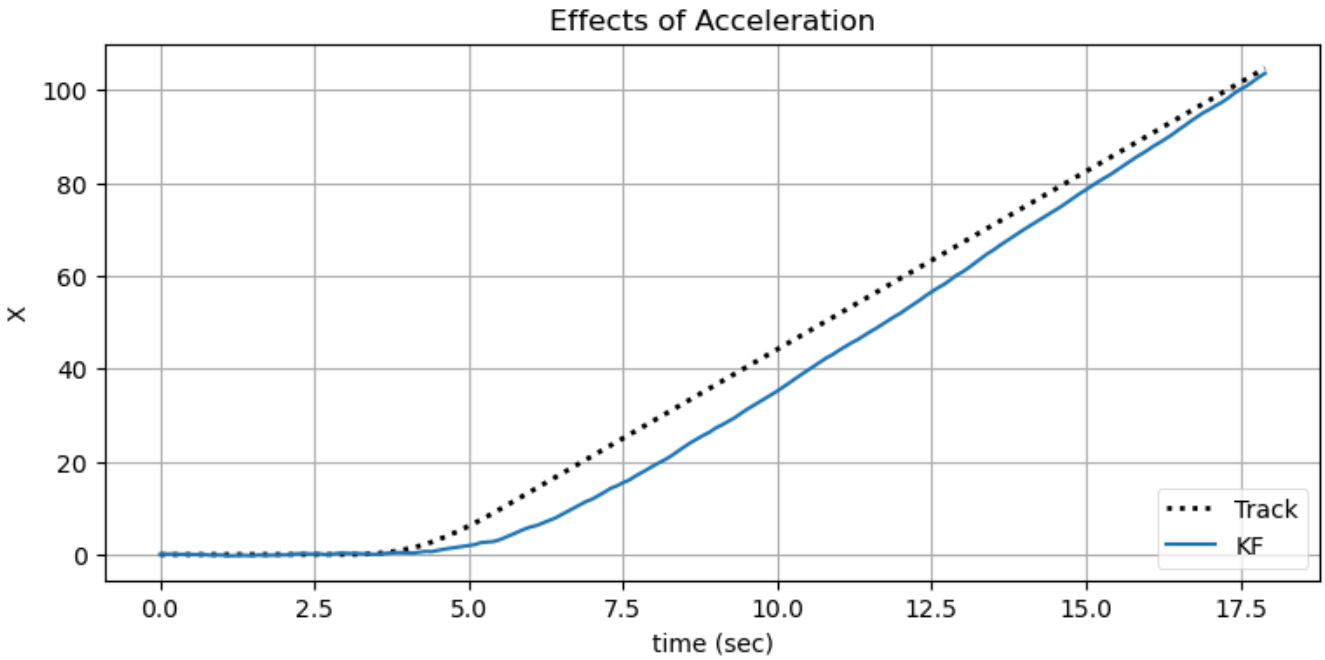
现在我们来运行它：

```
sensor_std = 2.
dt = 0.1

# initialize filter
cvfilter = make_cv_filter(dt, sensor_std)
initialize_filter(cvfilter)

track, zs = generate_data(50, sensor_std)

# run it
z_xs = zs[:, 0]
kxs, _, _, _ = cvfilter.batch_filter(z_xs)

# plot results
bp.plot_track(track[:, 0], dt=dt)
bp.plot_filter(kxs[:, 0], dt=dt, label='KF')
bp.set_labels(title='Track vs KF', x='time (sec)', y='X');
plt.legend(loc=4);
```



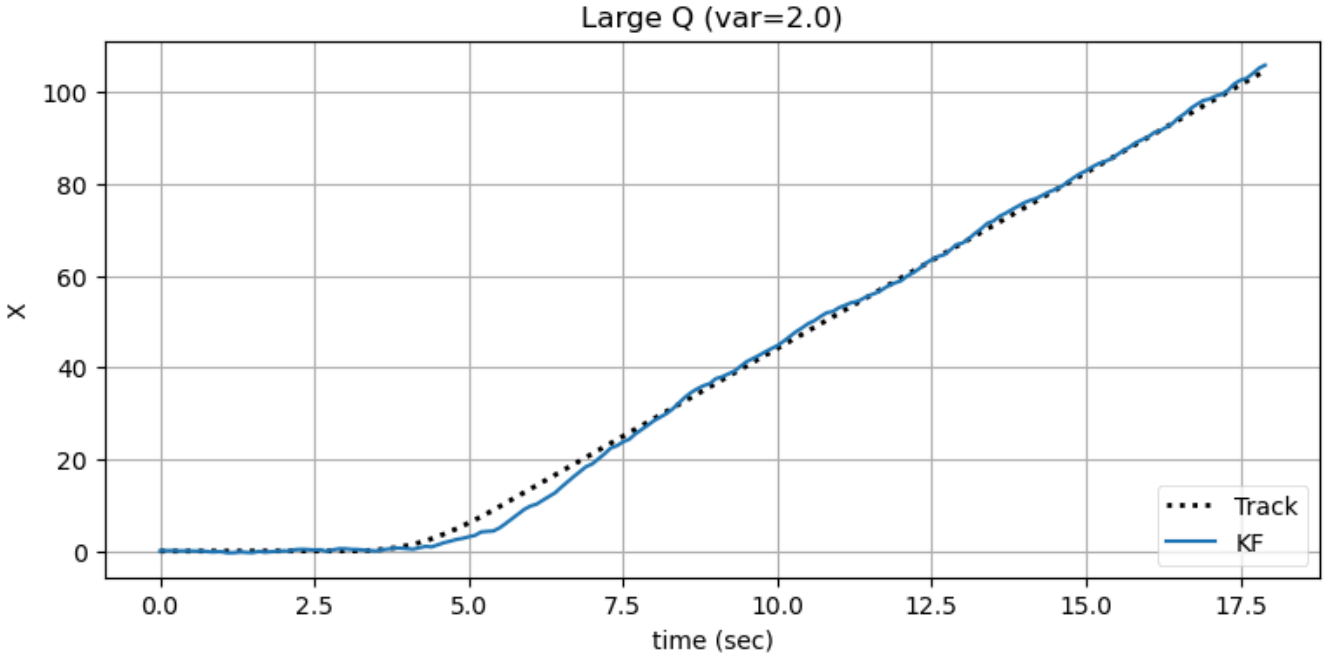
从图表中可以看出，卡尔曼滤波器无法准确追踪航向的变化。回想一下在 g-h 滤波器那一章的内容，这是因为该滤波器没有考虑加速度因素，因此它总是会滞后于输入信号。不过，如果信号处于稳定状态，滤波器最终会追上该信号。我们来看看具体情况吧。

```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)

track2, zs2 = generate_data(150, sensor_std)
xs2 = track2[:, 0]
z_xs2 = zs2[:, 0]

kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Effects of Acceleration',
              x='time (sec)', y='X')
```



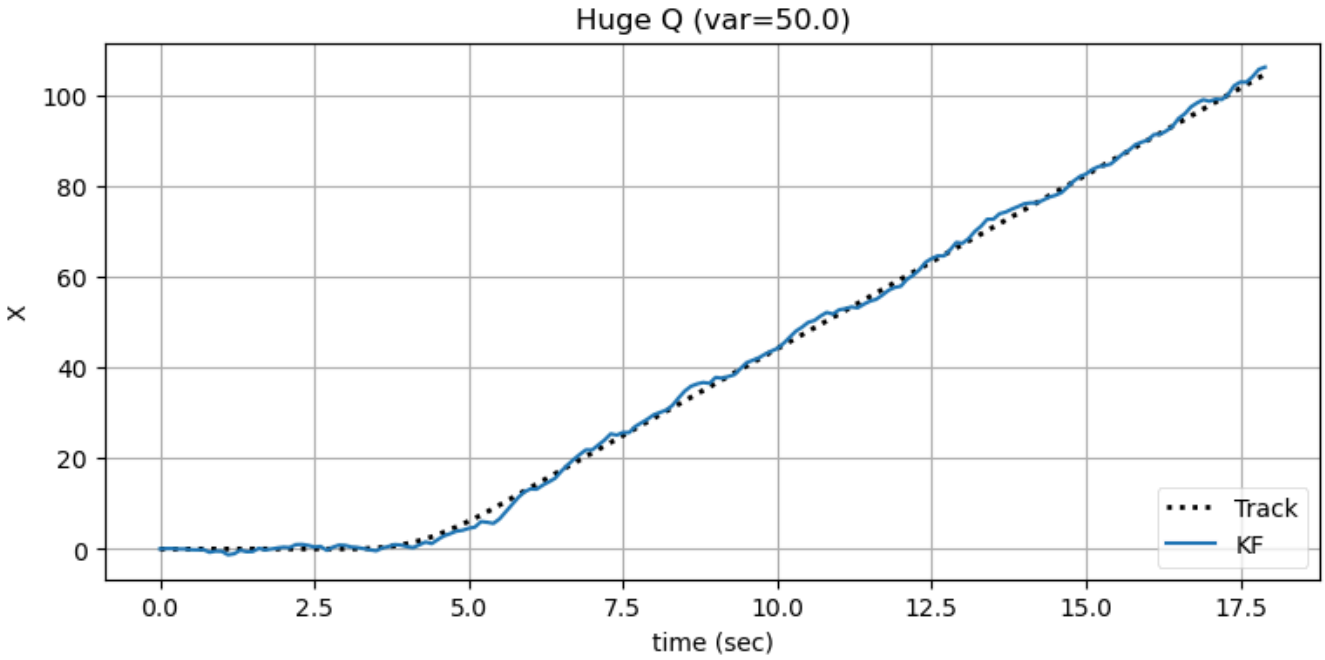
根本问题在于：过程模型适用于稳态运动，却不能准确描述目标机动时的运动。可以尝试增大 Q 来补偿这种模型失配，如下所示。

```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=2.0)
track, zs = generate_data(50, sensor_std)

# recompute track
kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)
bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Large Q (var=2.0)', x='time (sec)', y='X')
```



我们可以看到，该滤波器能够更快地重新捕捉到目标轨迹，但代价是输出信号中包含了大量噪声。此外，在第 4 秒到第 8 秒之间，存在较长的延迟时间，这种延迟在许多跟踪场景中是无法接受的。我们虽然可以进一步减少延迟时间，但代价是输出信号会变得极其嘈杂，如下所示：

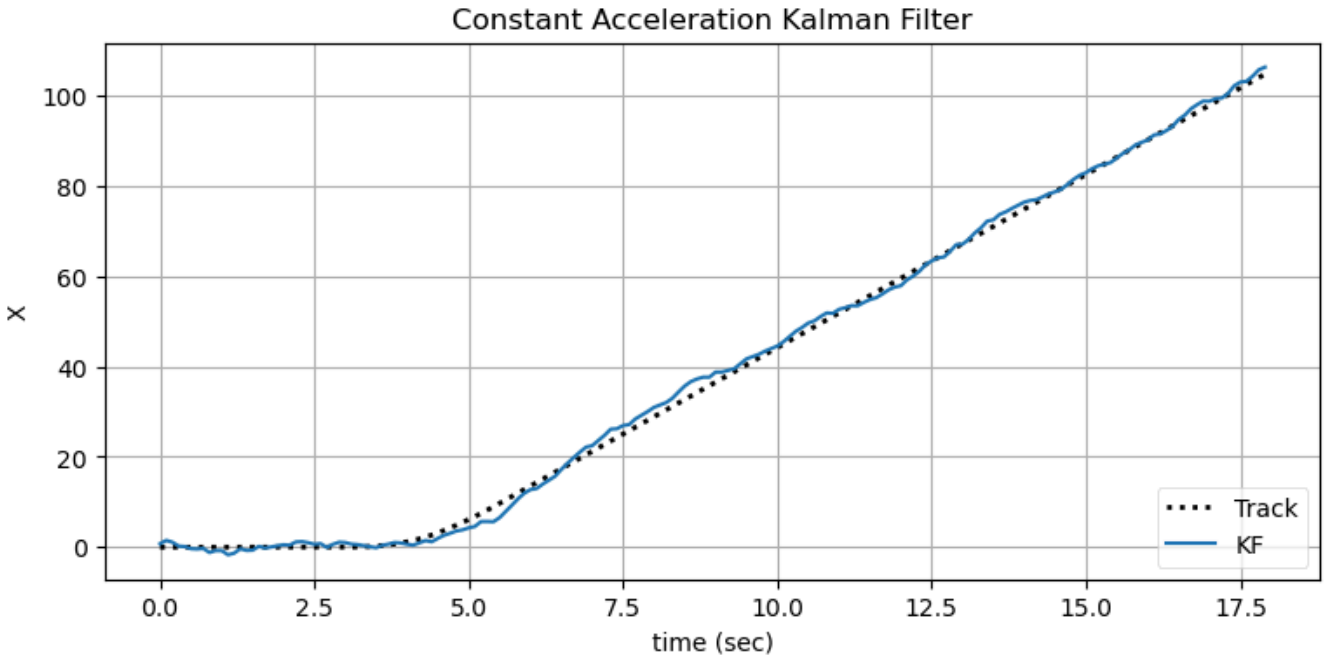
```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=50.0)
track, zs = generate_data(50, sensor_std)

# recompute track
cvfilter.x.fill(0.)
kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Huge Q (var=50.0)', x='time (sec)', y='X')
```



机动意味着存在加速度。下面实现匀加速卡尔曼滤波器，用同一组数据检验其表现。

```
def make_ca_filter(dt, std):
    cafilter = KalmanFilter(dim_x=3, dim_z=1)
    cafilter.x = np.array([0., 0., 0.])
    cafilter.P *= 3
    cafilter.R *= std
    cafilter.Q = Q_discrete_white_noise(dim=3, dt=dt, var=0.02)
    cafilter.F = np.array([[1, dt, 0.5*dt*dt],
                           [0, 1,         dt],
                           [0, 0,          1]])
    cafilter.H = np.array([[1., 0, 0]])
    return cafilter

def initialize_const_accel(f):
    f.x = np.array([0., 0., 0.])
    f.P = np.eye(3) * 3
```

```
dt = 0.1
cafilter = make_ca_filter(dt, sensor_std)
initialize_const_accel(cafilter)

kxs2, _, _, _ = cafilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Constant Acceleration Kalman Filter',
              x='time (sec)', y='X')
```



匀加速模型在这个例子中能够及时跟踪机动，但代价是稳态阶段的输出噪声很大。原因在于，滤波器难以区分机动刚开始时的变化与信号中的噪声。噪声也可能看起来像加速度，于是滤波器的加速度分量也会跟随噪声变化。

似乎两难：目标加速时，匀速滤波器响应不够快；目标处于零加速度的匀速阶段时，匀加速滤波器又会将噪声误认为加速度。

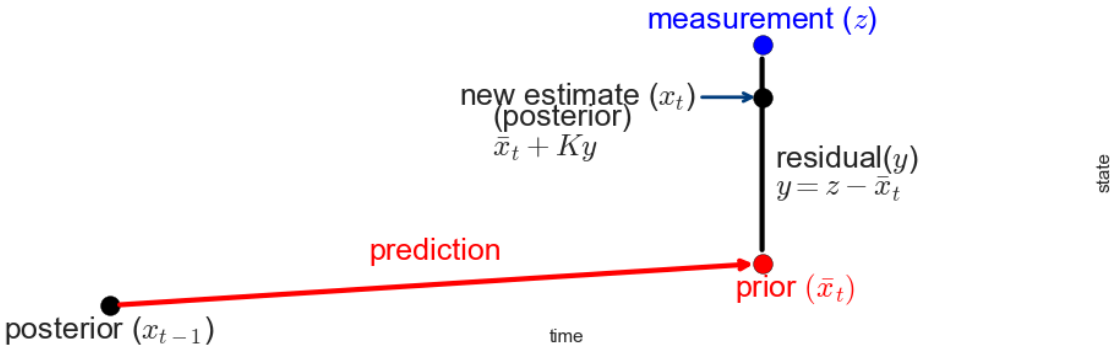
这里有一个重要观察，可以引导我们找到解决办法：目标不机动、加速度为零时，匀速滤波器表现最佳；目标机动时，匀加速滤波器表现较好，人为增大过程噪声 Q 的匀速滤波器也能较好跟踪。如果让滤波器自行适应被跟踪对象的行为，就可能兼顾两种情形的优势。

检测机动

讨论如何构建自适应滤波器之前，必须先回答：如何检测目标机动？若不知道机动何时发生，就无法合理地调整滤波器来应对。

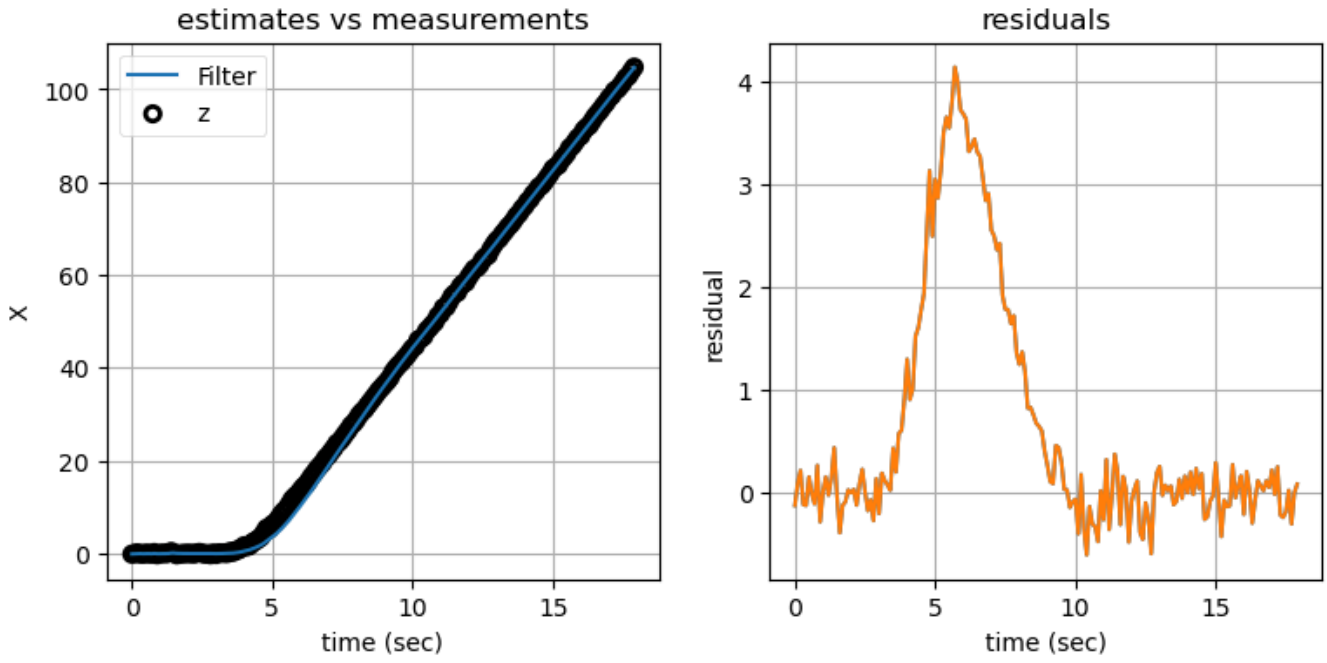
此前将“机动”理解为被跟踪目标具有非零加速度的阶段。更一般地，对某个卡尔曼滤波器而言，只要目标行为偏离它采用的过程模型，就可以视为相对于该模型发生了机动。

目标机动会给滤波器带来什么数学后果？由于真实行为偏离预测，残差往往增大。这里的残差是实际测量减去预测测量。



为验证这一点，下面画出机动过程中滤波器的残差。为便于观察，先减小数据中的噪声。

```
from kf_book.adaptive_internal import plot_track_and_residuals

def show_residual_chart():
    dt = 0.1
    sensor_std = 0.2

    # initialize filter
    cvfilter = make_cv_filter(dt, sensor_std)
    initialize_filter(cvfilter)
    pos2, zs2 = generate_data(150, sensor_std)
    xs2 = pos2[:, 0]
    z_xs2 = zs2[:, 0]

    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
    xs, res = [], []
    for z in z_xs2:
        cvfilter.predict()
        cvfilter.update([z])
        xs.append(cvfilter.x[0])
        res.append(cvfilter.y[0])

    xs = np.asarray(xs)
    plot_track_and_residuals(dt, xs, z_xs2, res)

show_residual_chart();
```



左图同时绘制了带噪测量与卡尔曼滤波输出，右图则绘制滤波器计算的残差，即测量减去预测测量。需要特别区分：右图不是左图两条曲线直接相减的结果。左图展示的是测量与更新后状态估计的差别，而右图展示的是测量与过程模型所给预测之间的差别。

这看似细微，图中的区别却很明显。机动开始时，左图的偏离还很小，右图的残差却已发生显著变化。若目标运动符合过程模型，残差通常应在 0.0 附近波动，因为测量满足：

$$\mathbf{measurement} = \mathbf{process_model}(t) + \mathbf{noise}(t)$$

目标开始机动后，过程模型的预测不再完全符合真实运动，测量关系变为：

$$\mathbf{measurement} = \mathbf{process_model}(t) + \mathbf{maneuver_delta}(t) + \mathbf{noise}(t)$$

因此，残差的均值若持续偏离 0.0，就提示可能发生了机动。

从残差图可见，问题并不简单。机动的影响很明显，但噪声掩盖了机动的起始时刻。这仍是从噪声中提取信号的老问题。

可调过程噪声

首先考虑使用较低阶模型，并根据是否发生机动来调整过程噪声。当残差按适当标准判定为“较大”时，增大过程噪声，使滤波器更依赖测量而不是过程预测，从而更紧密地跟踪信号。残差较小时，再减小过程噪声。

在相关文献中，有多种实现这种方式的方法。我会介绍其中的几种。

连续调整

第一种方法来自 Bar-Shalom [1]，用下面的二次型对残差进行归一化：

$$\epsilon = \mathbf{y}^\mathsf{T}\mathbf{S}^{-1}\mathbf{y}$$

其中，$\mathbf{y}$ 是残差，$\mathbf{S}$ 是新息协方差，也称系统不确定性，其表达式为：

$$\mathbf{S} = \mathbf{H}\mathbf{P}\mathbf{H}^\mathsf{T} + \mathbf{R}$$

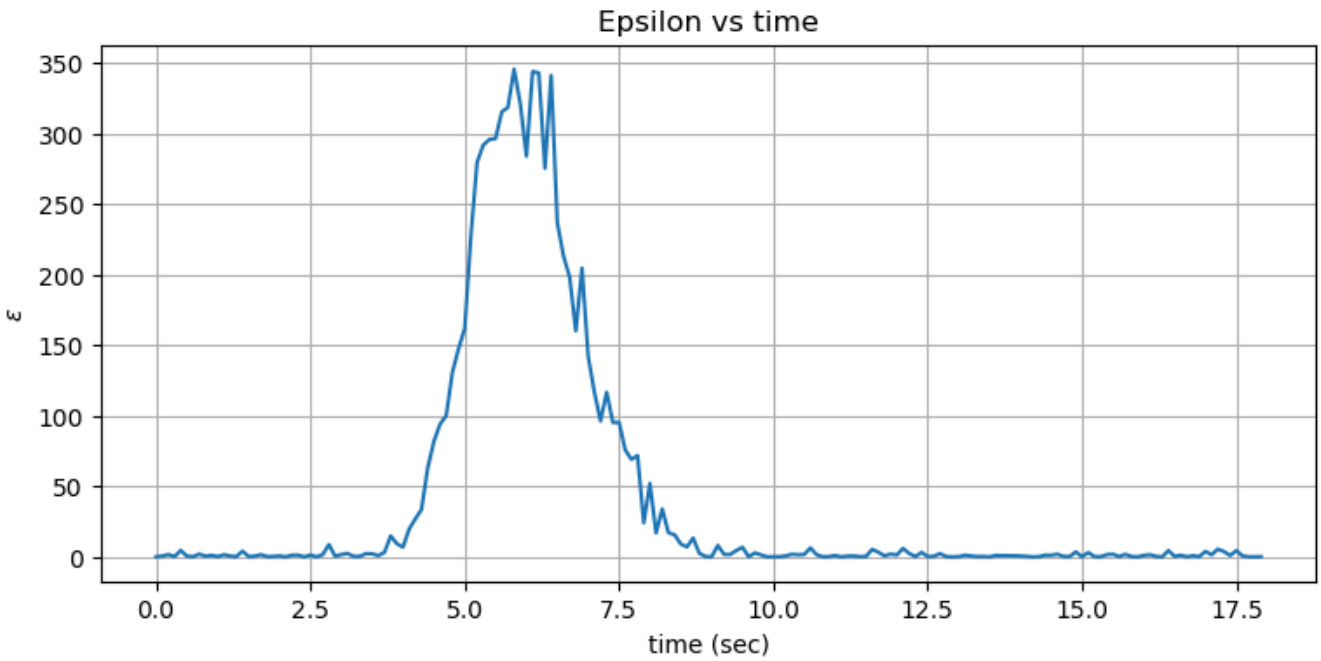
如果不熟悉这里的线性代数，可以把矩阵求逆类比为标量除法。因此，$\epsilon = \mathbf{y}^\mathsf{T}\mathbf{S}^{-1}\mathbf{y}$ 在标量情形中可理解为下面的计算：

$$\epsilon \approx \frac{\mathbf{y}^2}{\mathbf{S}}$$

$\mathbf{y}$ 和 $\mathbf{S}$ 都是 `filterpy.KalmanFilter` 的属性，因此实现起来相当简单。

让我们来看看 ϵ 随时间变化的图表吧。

```
from numpy.linalg import inv
dt = 0.1
sensor_std = 0.2
cvfilter= make_cv_filter(dt, sensor_std)
_, zs2 = generate_data(150, sensor_std)

epss = []
for z in zs2[:, 0]:
    cvfilter.predict()
    cvfilter.update([z])
    y, S = cvfilter.y, cvfilter.S
    eps = y.T @ inv(S) @ y
    epss.append(eps)

t = np.arange(0, len(epss) * dt, dt)
plt.plot(t, epss)
bp.set_labels(title='Epsilon vs time',
              x='time (sec)', y='$\epsilon$')
```

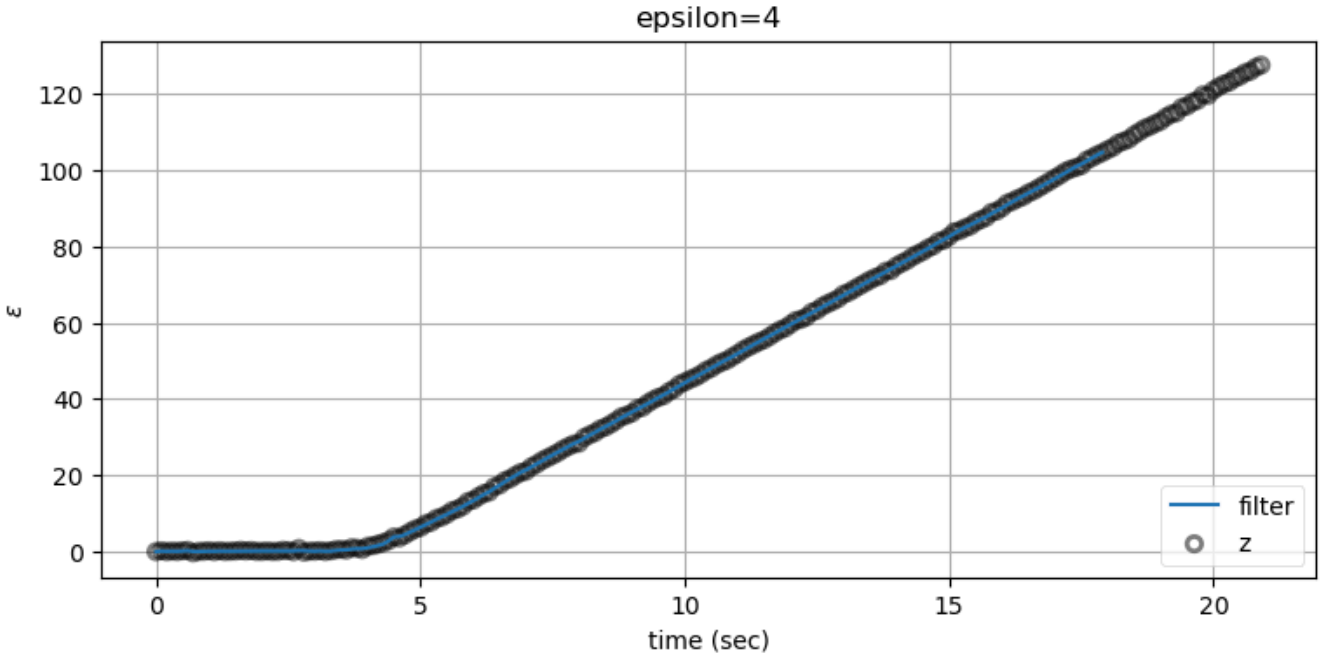


这幅图展示了残差归一化的作用。标量残差平方后为非负，再除以新息方差，就能衡量残差相对于预测不确定性是否显著增大。新息协方差 S 同时包含预测测量的不确定性和测量噪声。目标在 t=3 秒开始机动，不久后便可看到 ϵ 迅速增大。

使用乘法缩放因子增大过程噪声 Q：当统计量 ϵ 超过选定阈值时增大，重新降到阈值以下时，再减小 Q。缩放因子可通过试验选择。对于统计量 ϵ 的阈值 ϵ_{max}，可利用统计依据：该统计量是标准化残差的平方，因此标量情形的三个标准差对应阈值 9。实际传感器往往并非严格高斯，有时需采用更宽的门限。多维情况下，可依据新息维数选择卡方门限。超过门限只是目标可能机动的迹象，并非确定结论。

下面为 ϵ_{max} 和 Q 的缩放因子选择合适的演示值，实现这一算法。为便于观察，只绘出仿真的前 10 秒。

```
# reinitialize filter
dt = 0.1
sensor_std = 0.2
cvfilter = make_cv_filter(dt, sensor_std)
_, zs2 = generate_data(180, sensor_std)

Q_scale_factor = 1000.
eps_max = 4.

xs, epss = [], []

count = 0
for i, z in zip(t, zs2[:, 0]):
    cvfilter.predict()
    cvfilter.update([z])
    y, S = cvfilter.y, cvfilter.S
    eps = y.T @ inv(S) @ y
    epss.append(eps)
    xs.append(cvfilter.x[0])

    if eps > eps_max:
        cvfilter.Q *= Q_scale_factor
        count += 1
    elif count > 0:
        cvfilter.Q /= Q_scale_factor
        count -= 1

bp.plot_measurements(zs2[:,0], dt=dt, label='z', alpha=0.5)
bp.plot_filter(t, xs, label='filter')
plt.legend(loc=4)
bp.set_labels(title='epsilon=4', x='time (sec)', y='$\epsilon$')
```



这种滤波器的性能明显优于原先的匀速滤波器。机动开始后，原滤波器大约需要 10 秒才能重新跟上信号，自适应滤波器却只需不到 1 秒。

连续调整：标准差形式

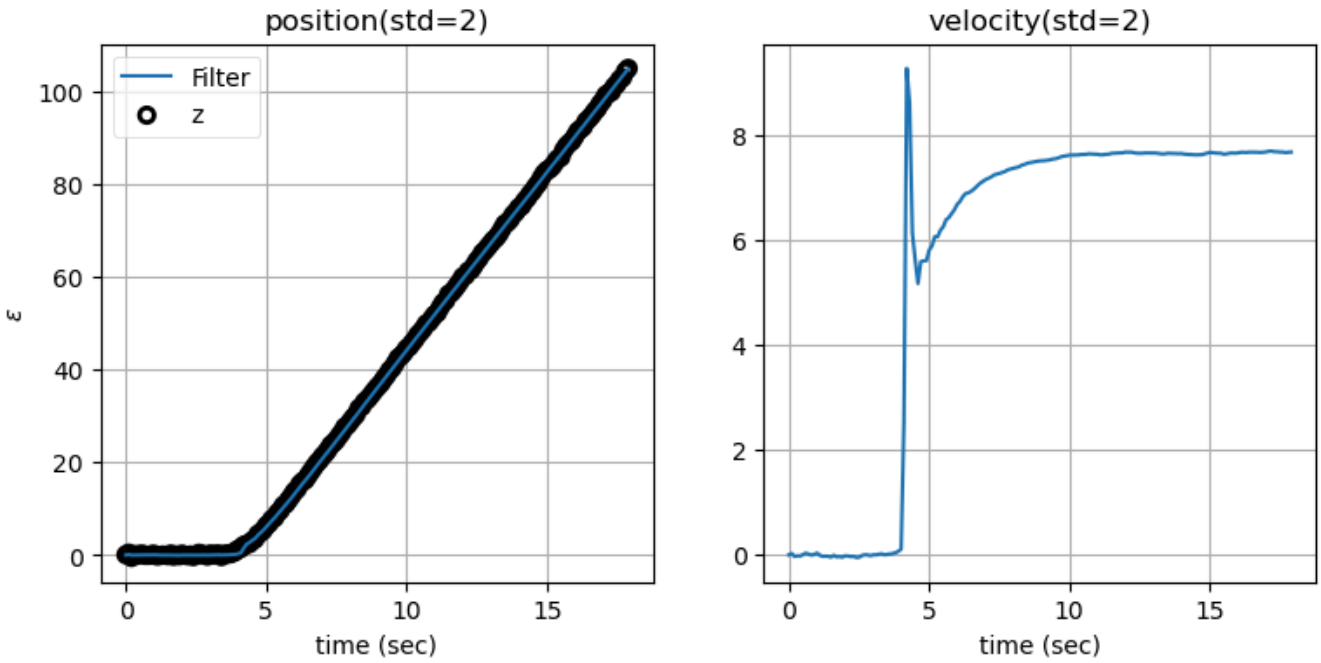
Zarchan [2] 给出了另一种很相近的方法：在这里的标量测量情形中，以新息协方差的平方根，即新息标准差，来设定门限。公式如下：

$$\begin{aligned} \boldsymbol{std} &= \sqrt{\mathbf{HPH}^\mathsf{T} + \mathbf{R}} \\ &= \sqrt{\mathbf{S}} \end{aligned}$$

如果残差的绝对值大于上述标准差的若干倍，我们就将过程噪声增加一个固定数值，然后重新计算 Q 值，再继续下一步操作。

```
from math import sqrt

def zarchan_adaptive_filter(Q_scale_factor, std_scale,
                            std_title=False,
                            Q_title=False):
    cvfilter = make_cv_filter(dt, std=0.2)
    pos2, zs2 = generate_data(180-30, std=0.2)
    xs2 = pos2[:,0]
    z_xs2 = zs2[:,0]

    # reinitialize filter
    initialize_filter(cvfilter)
    cvfilter.R = np.eye(1)*0.2

    phi = 0.02
    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=phi)
    xs, ys = [], []
    count = 0
    for z in z_xs2:
        cvfilter.predict()
        cvfilter.update([z])
        y = cvfilter.y
```

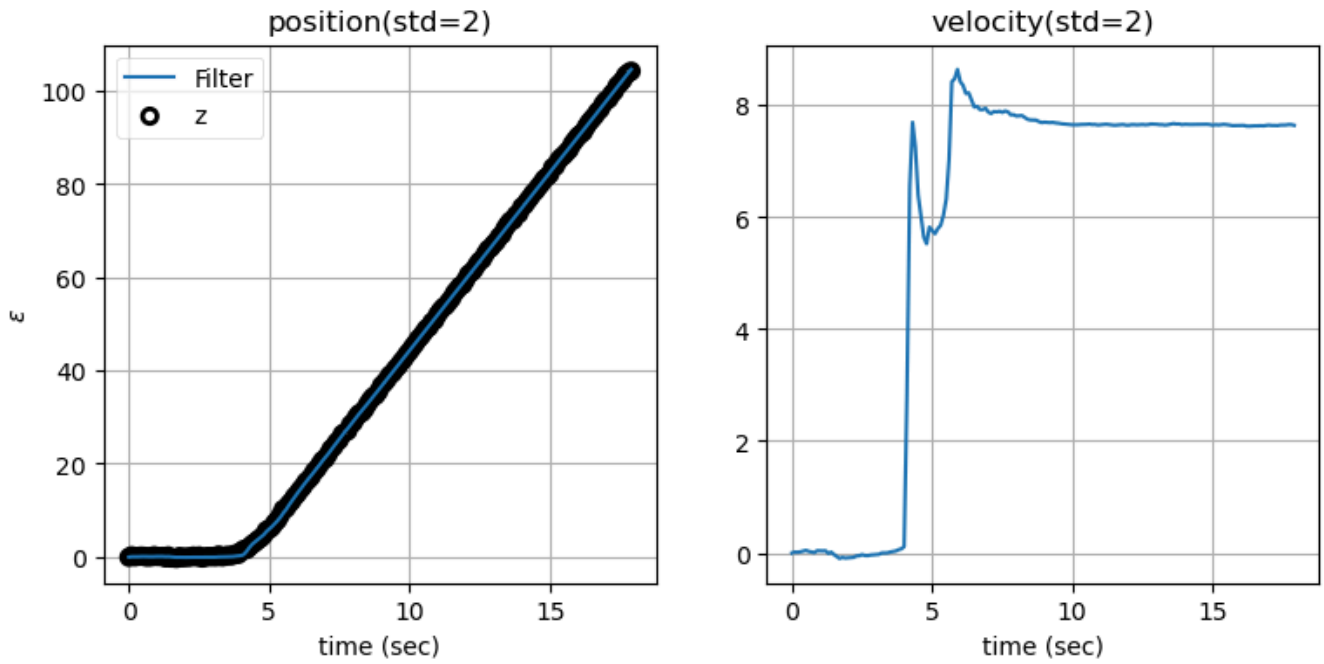
```
        S = cvfilter.S
        std = sqrt(S)

        xs.append(cvfilter.x)
        ys.append(y)

        if abs(y[0]) > std_scale*std:
            phi += Q_scale_factor
            cvfilter.Q = Q_discrete_white_noise(2, dt, phi)
            count += 1
        elif count > 0:
            phi -= Q_scale_factor
            cvfilter.Q = Q_discrete_white_noise(2, dt, phi)
            count -= 1

    xs = np.asarray(xs)
    plt.subplot(121)
    bp.plot_measurements(z_xs2, dt=dt, label='z')
    bp.plot_filter(xs[:, 0], dt=dt, lw=1.5)
    bp.set_labels(x='time (sec)', y='$\epsilon$')
    plt.legend(loc=2)
    if std_title:
        plt.title(f'position(std={std_scale})')
    elif Q_title:
        plt.title(f'position(Q scale={Q_scale_factor})')
    else:
        plt.title('position')

    plt.subplot(122)
    plt.plot(np.arange(0, len(xs)*dt, dt), xs[:, 1], lw=1.5)
    plt.xlabel('time (sec)')
    if std_title:
        plt.title(f'velocity(std={std_scale})')
    elif Q_title:
        plt.title(f'velocity(Q scale={Q_scale_factor})')
    else:
        plt.title('velocity')
    plt.show()

zarchan_adaptive_filter(1000, 2, std_title=True)
```



这里选择 1000 作为过程噪声的增大量，选择 2 倍标准差作为门限。为什么采用这些值？先比较 2 倍与 3 倍标准差门限的区别。

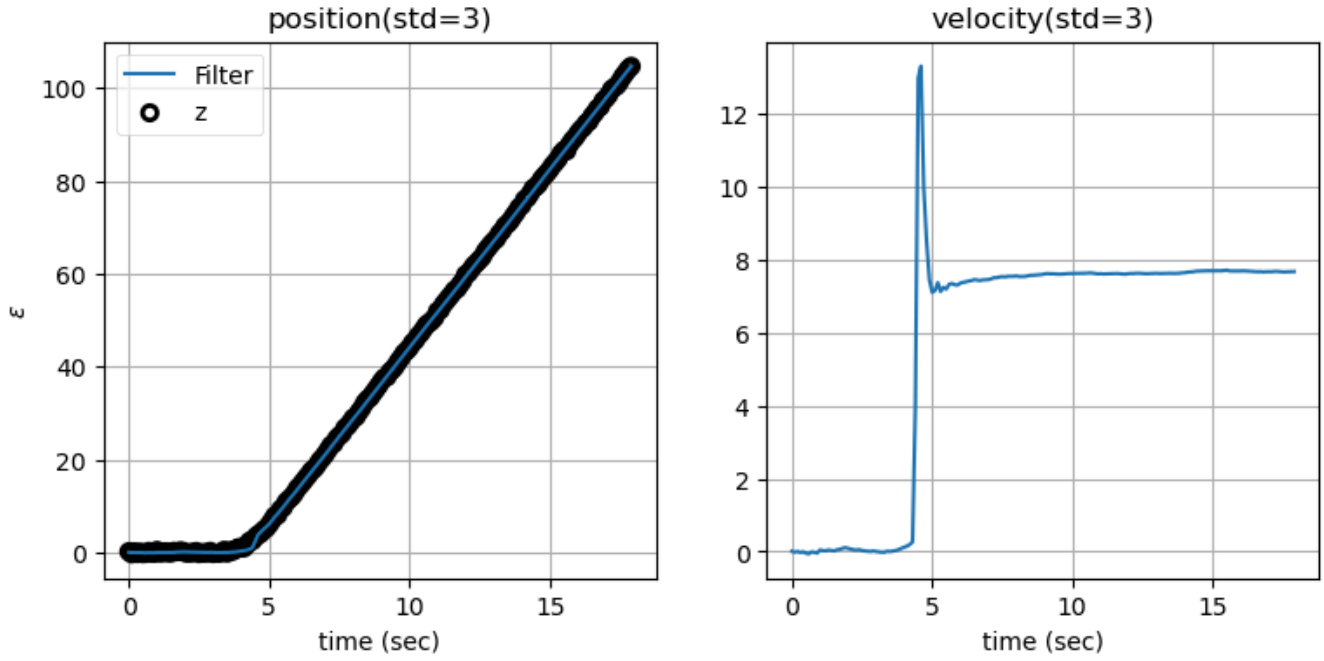
2 倍标准差

```
zarchan_adaptive_filter(1000, 2, std_title=True)
```



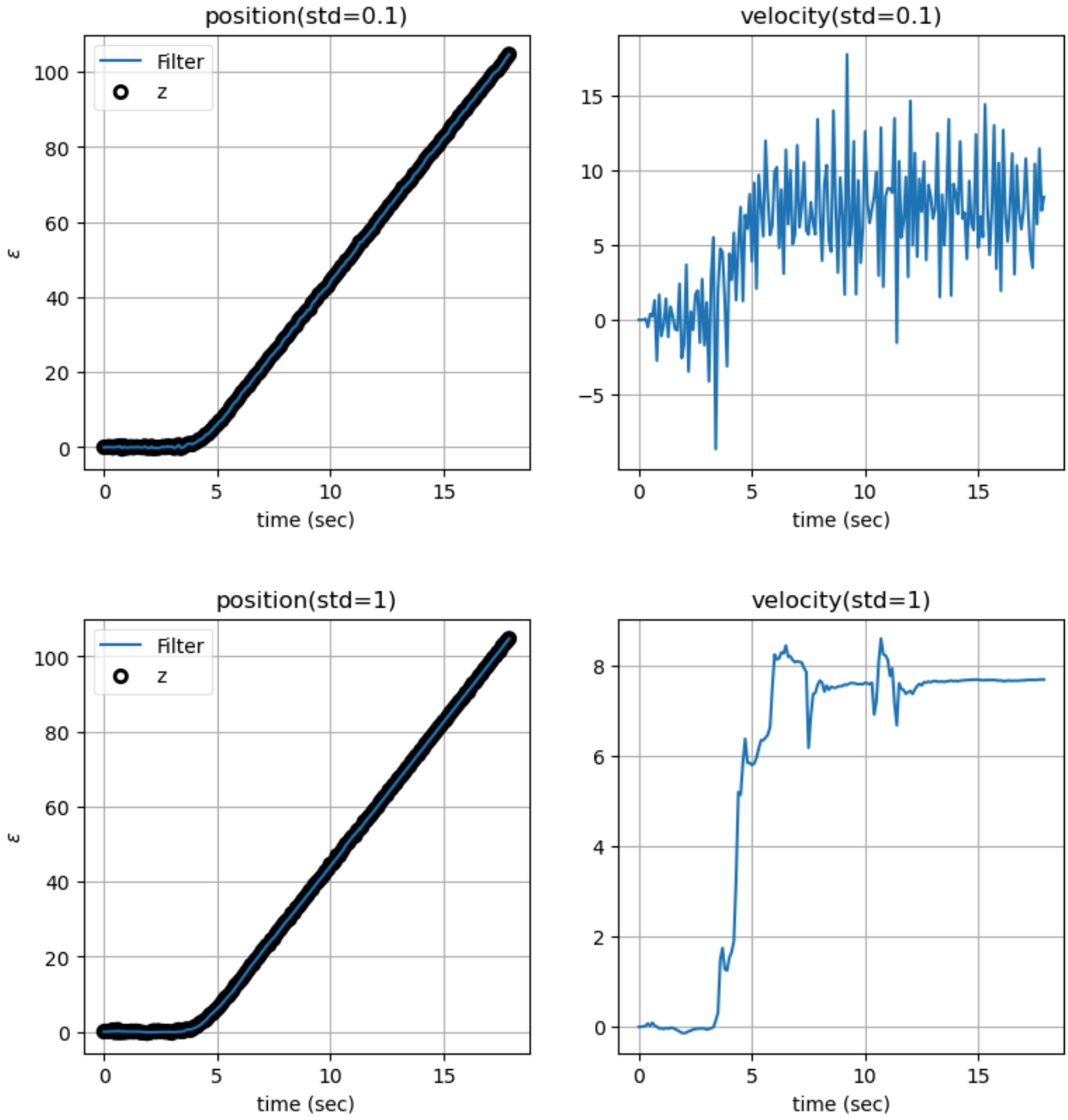
3 倍标准差

```
zarchan_adaptive_filter(1000, 3, std_title=True)
```



从图中可见，使用 2 倍或 3 倍标准差门限时，位置估计很相似，但速度估计有所不同。下面进一步观察，先把标准差门限的倍数设得很小。

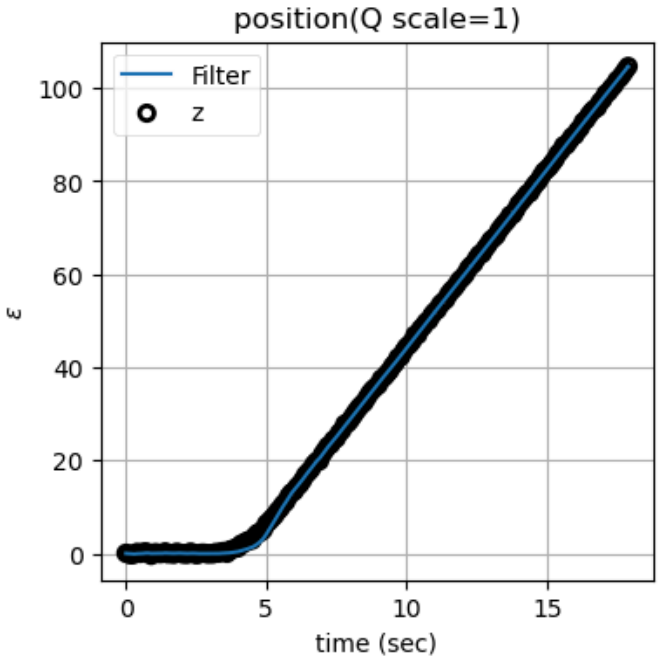
```
zarchan_adaptive_filter(1000, .1, std_title=True)
zarchan_adaptive_filter(1000, 1, std_title=True)
```

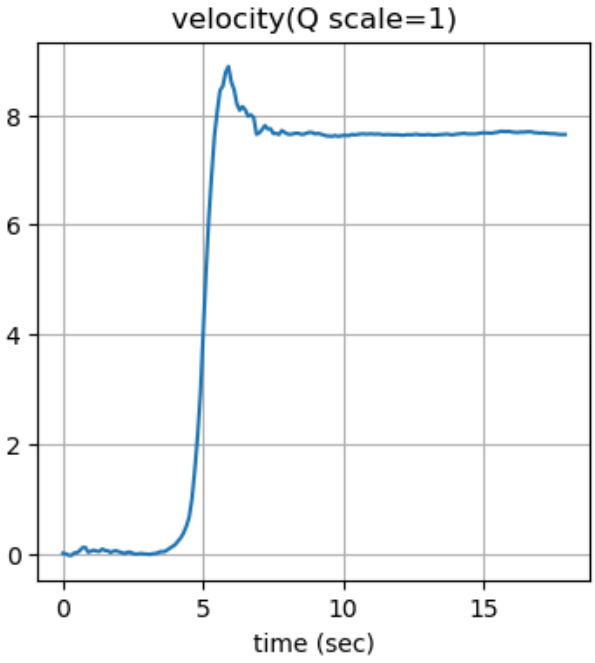


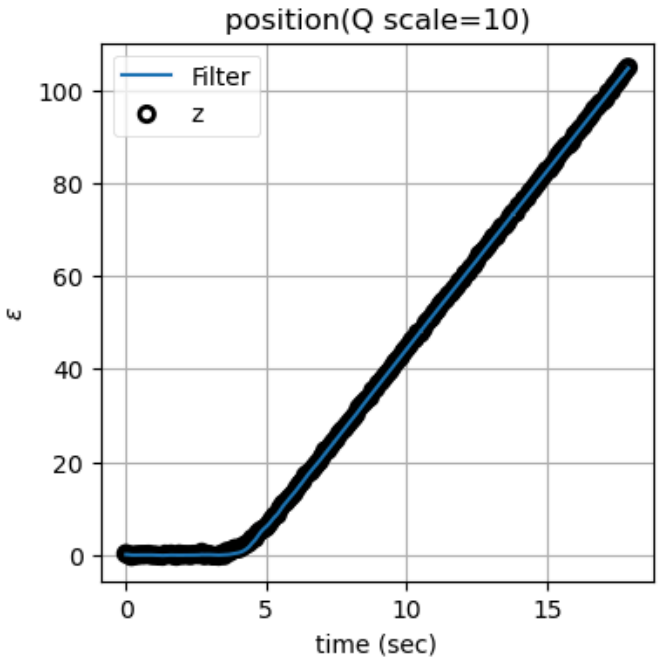
门限倍数越小，速度估计反而越差。想想原因：残差只要稍微偏离零就触发调整，会使滤波器很快给测量过大的权重。预测模型的作用过小时，就难以可靠地推断速度这类隐变量。因此，门限为 0.1 倍标准差时，速度估计几乎被测量噪声淹没。另一方面，位置估计由于紧随测量，能很快跟上机动。

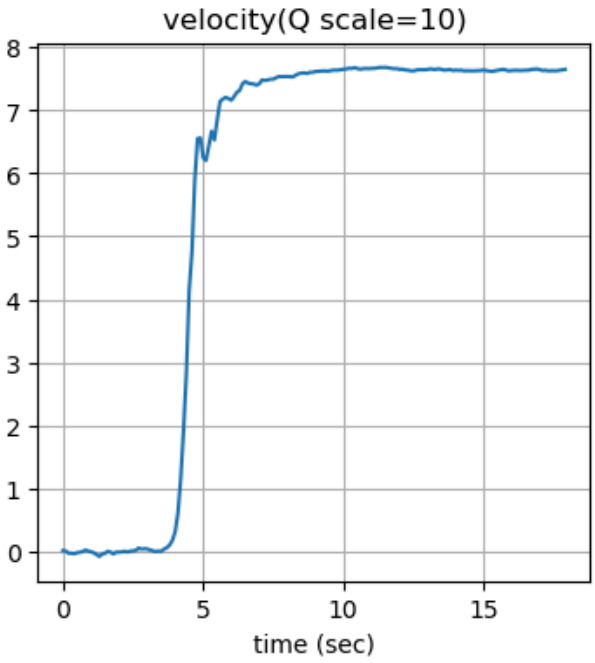
再看看过程噪声增大量的影响。这里将门限固定为 2 倍标准差，并将增大量从 1 调整到 10,000。

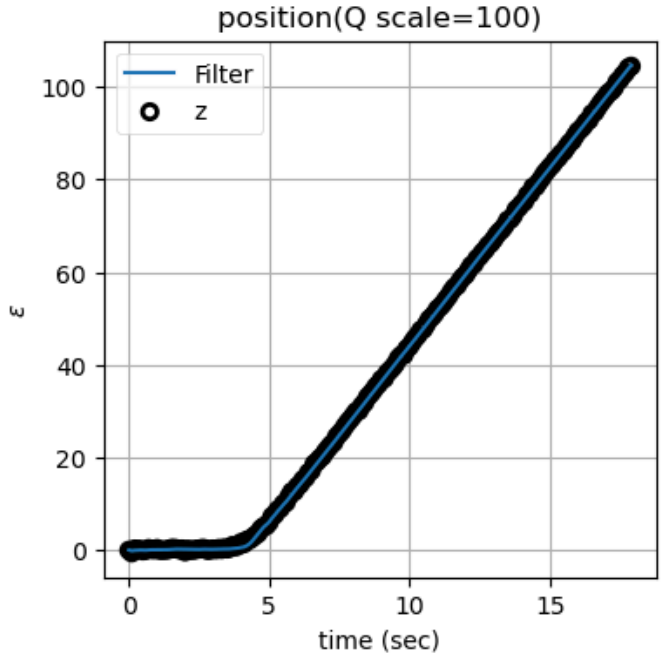
```
zarchan_adaptive_filter(1, 2, Q_title=True)
zarchan_adaptive_filter(10, 2, Q_title=True)
zarchan_adaptive_filter(100, 2, Q_title=True)
zarchan_adaptive_filter(1000, 2, Q_title=True)
zarchan_adaptive_filter(10000, 2, Q_title=True)
```

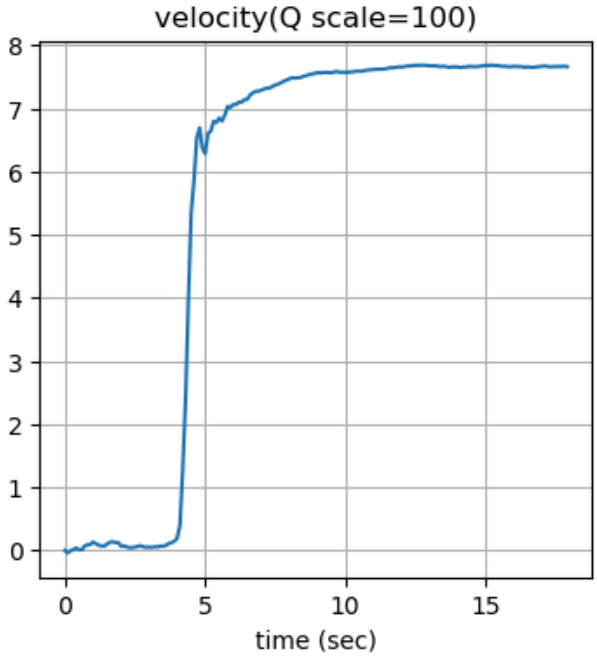


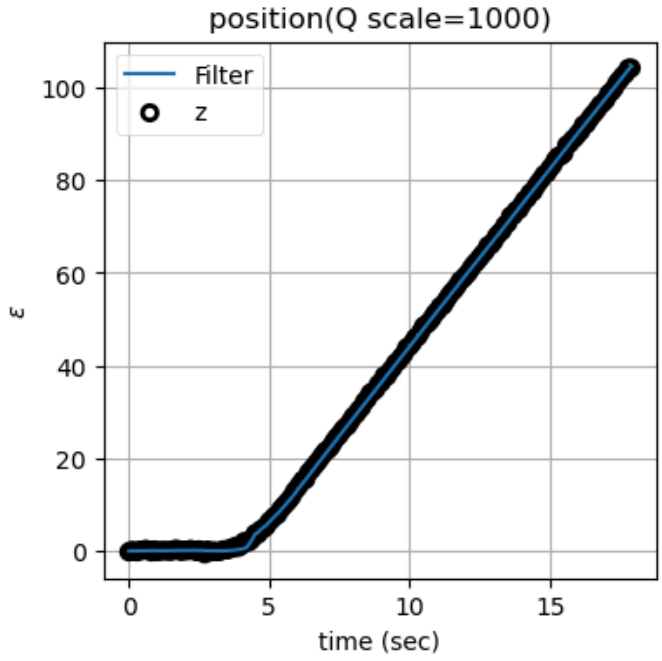


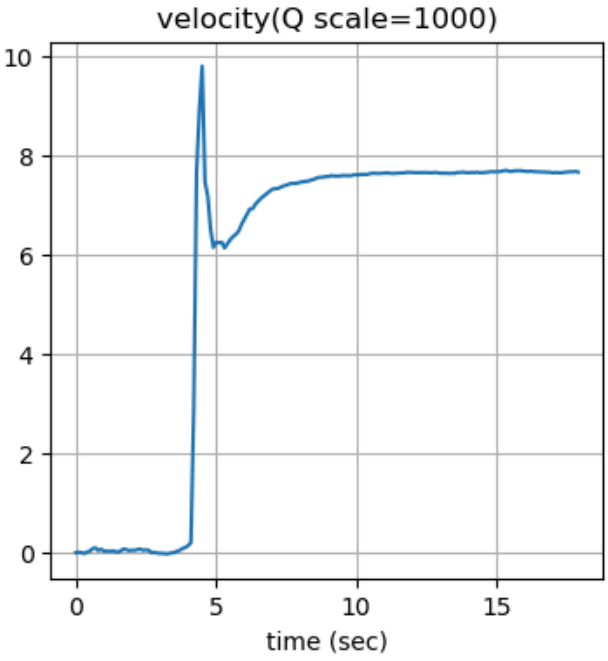


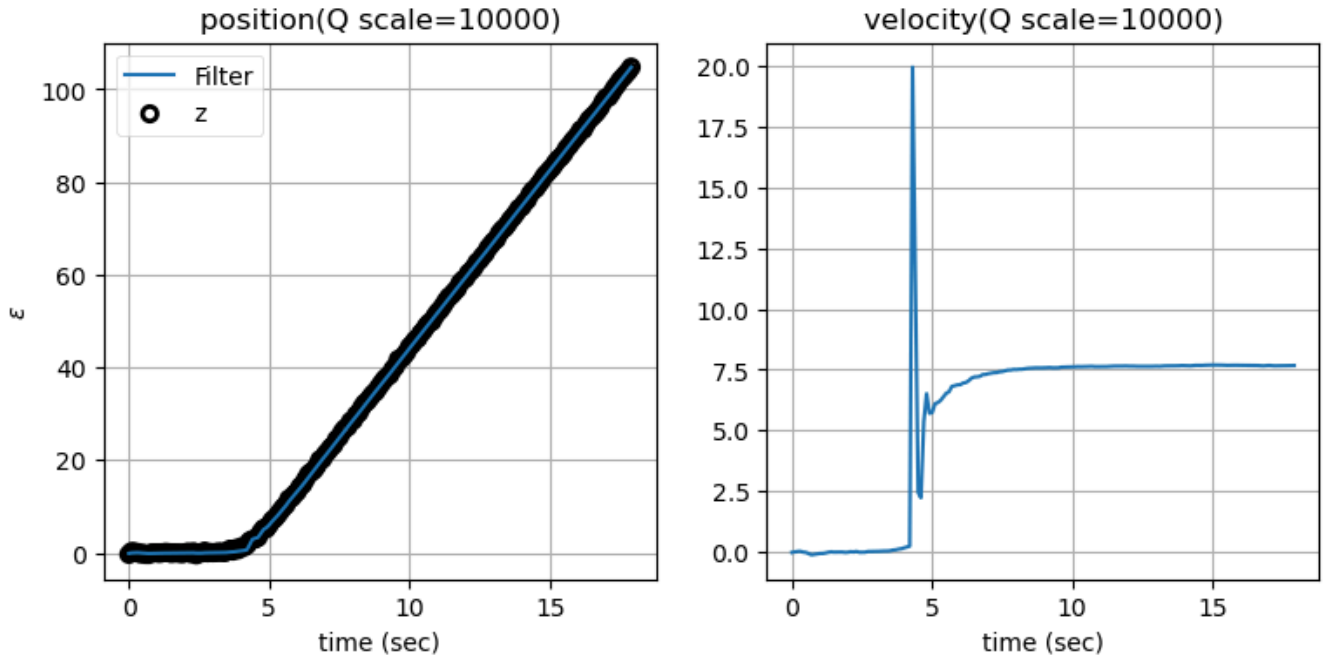



position(Q scale=100)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)


velocity(Q scale=100)
8
7
6
5
4
3
2
1
0
0
5
10
15
time (sec)


position(Q scale=1000)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)


velocity(Q scale=1000)
10
8
6
4
2
0
0
5
10
15
time (sec)



增大量越大，位置估计略有改善，但速度估计开始出现很大的超调。

我无法告诉你其中哪一种才是“正确的”方案。你需要用真实数据和模拟数据来测试各种滤波器的性能，然后选择那种最能满足你对各个状态变量所要求的性能指标的滤波器设计。

衰减记忆滤波器

衰减记忆滤波器通常不归入自适应滤波器，因为它不会根据输入自行改变参数，但在机动目标跟踪中也能有良好表现。对于一阶、二阶和三阶运动学模型，它有计算形式很简单的实现，无须递推求解 Riccati 方程来计算卡尔曼增益，因此能大幅降低计算量。另外，也有可直接用于标准卡尔曼滤波器的形式。本章主要关注自适应滤波，因此重点讨论后一种形式。FilterPy 实现了这两类衰减记忆滤波器。

卡尔曼滤波器虽然递推运行，当前状态估计仍汇总了所有历史测量的信息。在目标行为与过程模型相符时，这有助于得到最优估计。例如，估计飞行中球的位置时，利用过去全部测量通常比只保留部分测量更可靠；否则，不确定性较大，当前估计就更容易受测量噪声影响。极端地说，如果只保留最后一次测量和估计，而丢弃此前积累的信息，就只能对当前测量赋予较大权重。测量噪声很大时，估计结果也会受到很大的噪声影响。这与滤波器刚初始化时的现象相似：早期估计波动较大，随着信息积累才逐渐稳定。

但目标机动时，真实行为不再总符合过程模型，保留历史信息反而可能使滤波器反应迟缓。上面的图中，目标已经转弯，滤波器却仍预测直线运动：它根据历史运动建立了很强的信心，错误地相信目标还会沿既有航向和速度前进。

衰减记忆滤波器降低较早测量的影响，提高较近测量的相对权重，以缓解这个问题。

关于这种衰减记忆滤波器，有多种不同的实现方式。我采用的是 Dan Simon 在《最优状态估计》[3]一书中提出的方法。这里就不详细阐述他的推导过程了，直接给出结果即可。

卡尔曼滤波器中，用于描述估计误差协方差的方程为：

$$\bar{\mathbf{P}} = \mathbf{FPF}^{\mathsf{T}} + \mathbf{Q}$$

引入参数 $\boldsymbol{\alpha}$ 对传播的协方差进行放大，可以降低历史信息的相对影响：

$$\tilde{\mathbf{P}} = \alpha^2\mathbf{FPF}^{\mathsf{T}} + \mathbf{Q}$$

取 $\boldsymbol{\alpha} > \mathbf{1.0}$。当 $\boldsymbol{\alpha} == \mathbf{1}$ 时，就恢复为普通卡尔曼滤波。$\boldsymbol{\alpha}$ 是 KalmanFilter 类的属性，默认值为 1；因此，只有把 $\boldsymbol{\alpha}$ 设为大于 1 的值，才会启用这里的衰减记忆效果。$\boldsymbol{\alpha}$ 没有固定的最佳取值，通常很接近 1，例如 1.01。需要用仿真或真实数据反复试验，在快速响应机动和避免过度跟随测量噪声之间取得平衡。

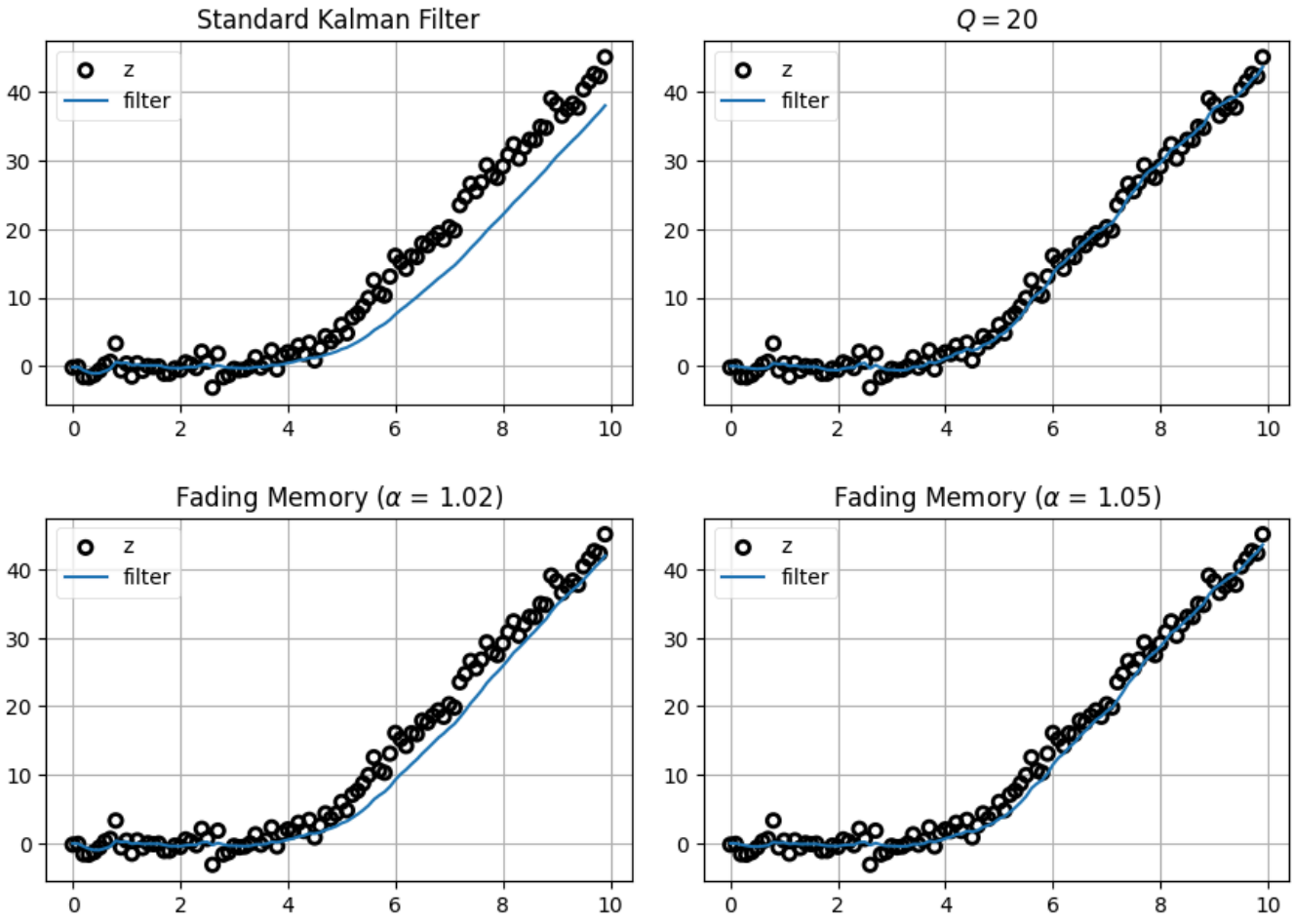
为什么有效？增大估计误差协方差，意味着滤波器对自身估计更不确定，因此更新时会给测量更大权重。

需要注意：启用大于 1 的 $\boldsymbol{\alpha}$ 后，传播得到的是人为放大的 $\tilde{\mathbf{P}}$，而非未放大情况下的 $\bar{\mathbf{P}}$。因此，KalmanFilter.P 不宜直接解释为真实估计误差协方差；它是滤波器用于调整权重的内部不确定性量。

让我们使用“衰减记忆滤波器”来处理数据，然后看看结果如何。我会在系统中引入大量误差，这样我们就可以比较不同的处理方法了。

```
pos2, zs2 = generate_data(70, std=1.2)
xs2 = pos2[:, 0]
z_xs2 = zs2[:, 0]

cvfilter = make_cv_filter(dt, std=1.2)
cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.00

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(221)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Standard Kalman Filter')

cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=20.)
cvfilter.alpha = 1.00

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])

xs = np.asarray(xs)

plt.subplot(222)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('$Q=20$')

cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.02
```

```
xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(223)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Fading Memory ($\\alpha$ = 1.02)')

cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.05

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(224)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Fading Memory ($\\alpha$ = 1.05)');
plt.gcf().set_size_inches(9, 6.5)
plt.tight_layout(pad=1.5, h_pad=2.0)
```



第一幅图是普通卡尔曼滤波：机动开始后，估计偏离目标，约到 10 秒才重新跟上。增大过程噪声后，响应明显加快，但由于对带噪测量赋予过大权重，输出也更噪。接着使用 $\alpha = 1.02$ 的衰减记忆滤波器，估计相当平滑；目标恢复稳态后虽仍需几秒收敛，但比普通卡尔曼滤波快得多，滞后也小得多，估计更接近真实轨迹。最后将 α 提高到 1.05，滤波器几乎立即响应机动，但由于对历史测量的记忆减弱，稳态阶段的估计也不再那么平直。

对于如此微小的代码改动来说，这样的性能已经相当不错了！需要注意的是，这里并没有所谓的“正确”选择。你需要根据自身的需求，以及测量噪声、过程噪声以及目标的运动特性来设计滤波器。

多模型估计

本章的例子中，目标先稳态运动，再执行机动，然后恢复稳态。可以用匀速和匀加速两种模型分别描述这些阶段。当系统的行为可以由有限组模型之一描述时，就可以使用多模型估计（MM）：设置一组滤波器，各自采用不同的过程模型，再根据目标的运动情况切换或融合它们的输出。

这个主题范围很广，多模型估计器也有多种实现方式。对本章目标而言，可以同时运行匀速与匀加速滤波器，检查残差以检测机动，再切换输出。不过还有更多选择。例如，汽车转弯时前轮转向，车身绕后轴改变朝向，这是非线性过程；若要更准确地建模转弯，可能需要 EKF、UKF 等非线性滤波器，而稳态直行阶段采用线性匀速滤波器即可。于是滤波器组可以由直行用的线性 KF 和转弯用的 EKF 组成。但它们未必擅长描述加速和制动。因此，高性能多模型估计器可能包含多个滤波器，分别适应目标不同的运动范围与行为。

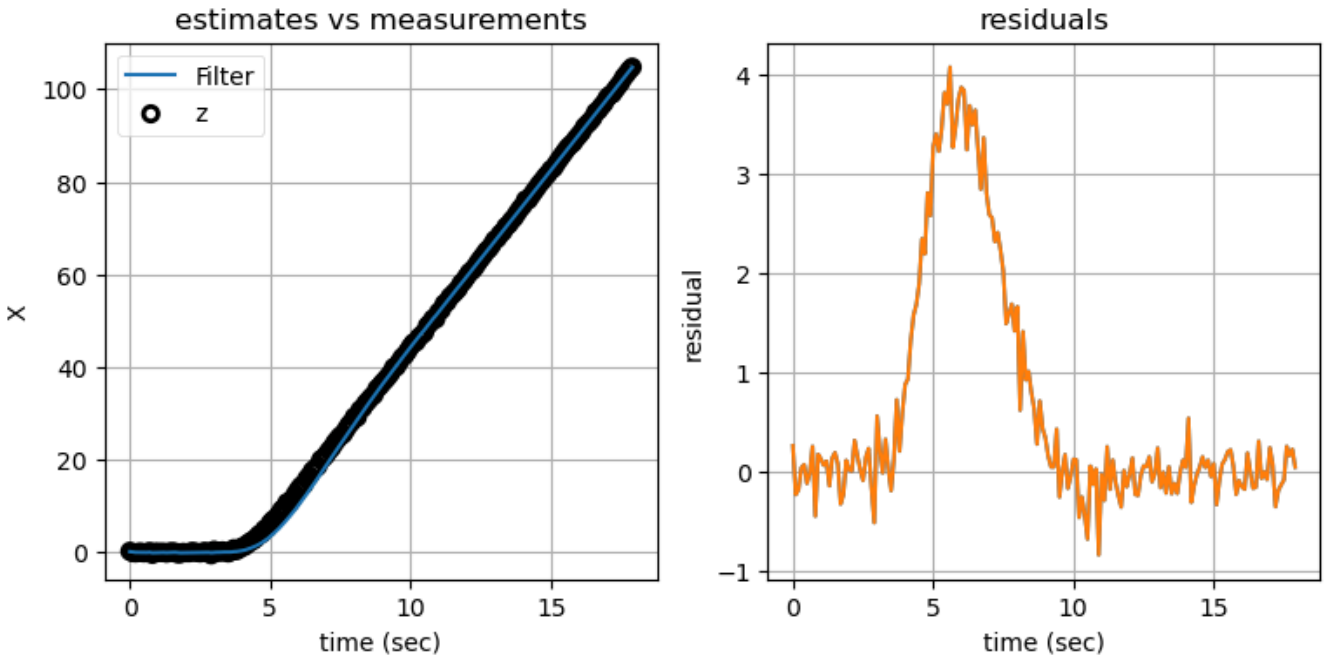
各滤波器也不一定只在模型阶次上有所不同，还可以采用不同噪声模型或不同的自适应调节方式。例如，前面某组参数可能使位置估计较好，另一组参数则使速度估计较好，可以把它们都纳入滤波器组，再分别利用各自更可靠的估计。

一种双滤波器自适应滤波器

我相信，通过切换不同的滤波器来获得最佳性能这一理念是清晰的。但是，要实现这一理念，我们应该依据哪些数学原理呢？我们面临的问题是：如何从充满噪声的测量数据中判断出，何时应该调整模型。在卡尔曼滤波器中，是哪个参数用来衡量测量结果与预测值之间的偏差呢？没错，就是残差。

假设使用一阶匀速卡尔曼滤波器。目标不机动且模型合适时，滤波器能紧密跟踪其运动。零均值高斯新息约有 68% 落在均值两侧一个标准差 σ 的范围内，正负残差也应大致对称。若残差持续增大并超出预测范围，就提示目标行为可能偏离过程模型。前图已经展示：机动开始后，残差从围绕零波动变为突然增大，并持续位于零的一侧。

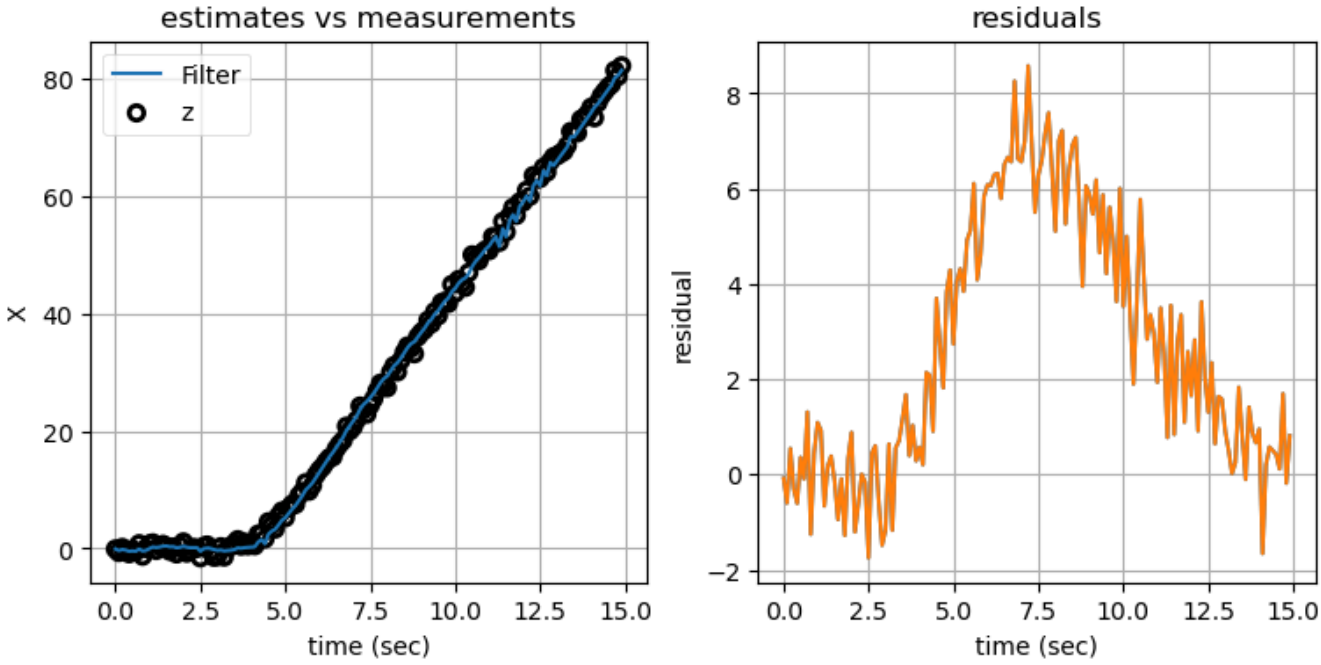
```
show_residual_chart()
```



在本例中，目标稳态运动时，匀速滤波器优于匀加速滤波器；目标机动时则相反。上图中的转变发生在第 4 秒。

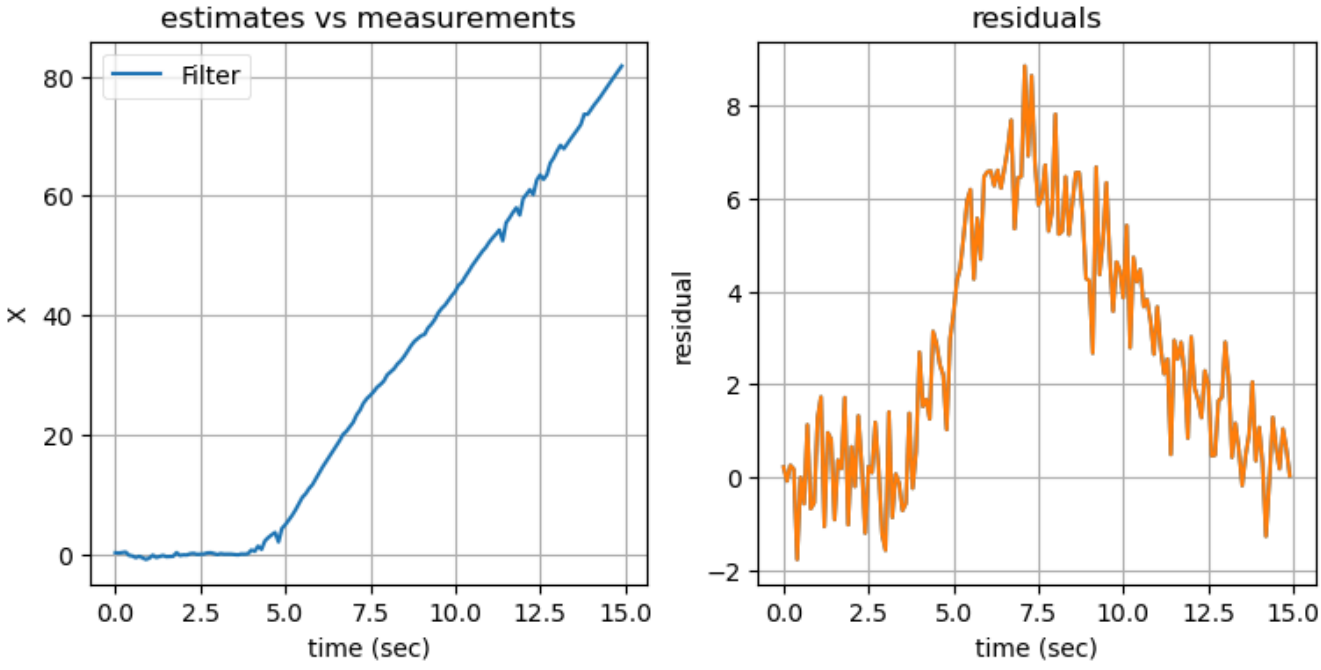
算法很简单：初始化匀速与匀加速两个滤波器，在同一预测—更新循环中同时运行。每次更新后检查匀速滤波器的残差；若残差位于设定的统计范围内，就采用匀速滤波器的估计，否则采用匀加速滤波器的估计。

```
def run_filter_bank(threshold, show_zs=True):
    dt = 0.1
    cvfilter= make_cv_filter(dt, std=0.8)
    cafilter = make_ca_filter(dt, std=0.8)
    pos, zs = generate_data(120, std=0.8)
    z_xs = zs[:, 0]
    xs, res = [], []

    for z in z_xs:
        cvfilter.predict()
        cafilter.predict()
        cvfilter.update([z])
        cafilter.update([z])

        std = np.sqrt(cvfilter.R[0,0])
        if abs(cvfilter.y[0]) < 2 * std:
            xs.append(cvfilter.x[0])
        else:
            xs.append(cafilter.x[0])
        res.append(cvfilter.y[0])
    xs = np.asarray(xs)
    if show_zs:
        plot_track_and_residuals(dt, xs, z_xs, res)
    else:
        plot_track_and_residuals(dt, xs, None, res)

run_filter_bank(threshold=1.4)
```



这个滤波器能紧密跟踪机动。目标不机动时，估计几乎不含噪声；一旦发生机动，就能很快检测到，并切换到匀加速滤波器。不过，结果仍不理想。下面单独画出输出：

```
run_filter_bank(threshold=1.4, show_zs=False)
```



可以看到，当滤波器组从一种滤波器切换到另一种滤波器时，估算结果会有较大的波动。我不会在生产系统中使用这种算法。下一节将介绍一种能够解决这个问题的先进滤波器组实现方式。

MMAE

使用多个滤波器检测机动的思路合理，但突然切换滤波器会使估计不连续。非此即彼地选择一个滤波器，也不符合本书强调的概率融合思想。此前，我们并不简单选择测量或预测，而是根据各自的不确定性将二者融合；这里也应依据模型概率融合各滤波器的估计。这种方法称为多模型自适应估计器，简称 MMAE。

在“卡尔曼滤波器的设计”这一章中，我们学习了似然函数。

$$\mathcal{L} = \frac{1}{\sqrt{2\pi S}}\exp[-\frac{1}{2}\mathbf{y}^\mathsf{T}\mathbf{S}^{-1}\mathbf{y}]$$

这个似然函数衡量测量与滤波模型的相容程度。$\mathbf{y}$ 是残差，$\mathbf{S}$ 是新息协方差，即测量空间中的预测不确定性。将残差代入相应的零均值高斯密度，即可得到测量似然。给定协方差时，马氏意义上的残差越大，测量似然越低；残差变大并不意味着新息协方差自动增大。上式适用于标量测量；多维测量应采用包含测量维数和新息协方差行列式的多元高斯密度。结合各模型的先验概率与测量似然，可以计算后验模式概率：

$$p_k^i = \frac{\mathcal{L}_k^i p_{k-1}^i}{\sum_{j=1}^{N} \mathcal{L}_k^j p_{k-1}^j}$$

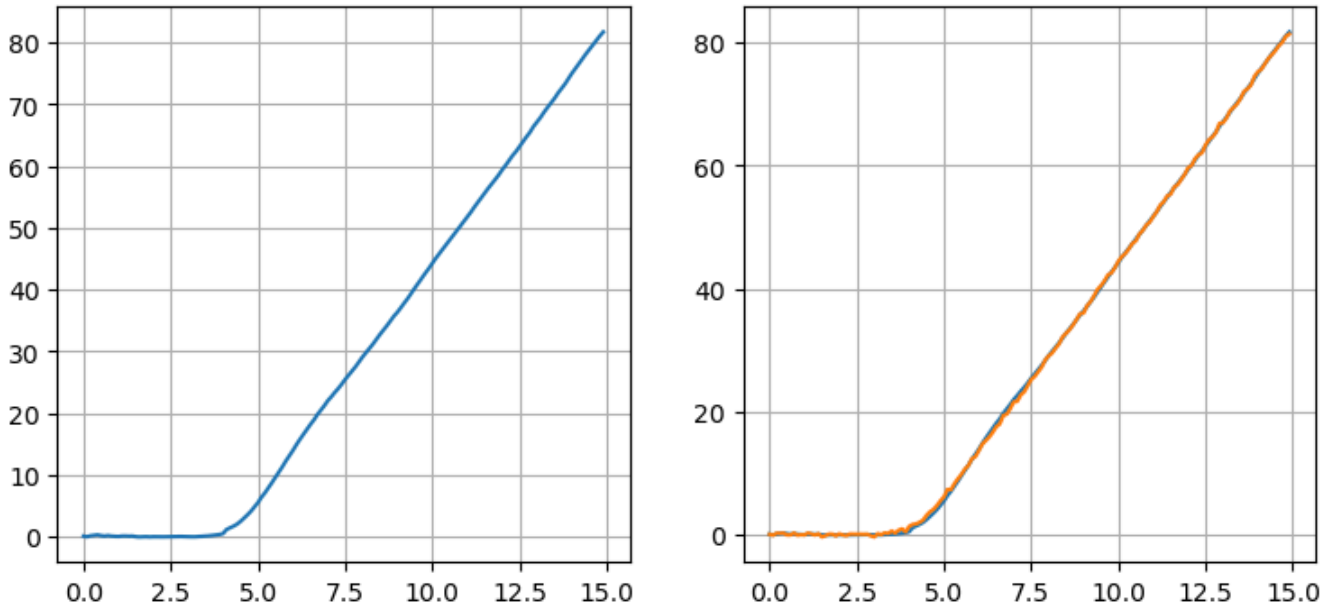
公式看起来复杂，其实很直接。分子是本时间步的测量似然，乘以上一时间步该滤波器对应模式的概率。再除以所有滤波器对应分子的总和进行归一化，使模式概率之和为 1。注意，分母包括当前模型，并非只包含“其他”滤波器。

这是递推关系，因此必须为各滤波器指定初始模式概率。没有更多信息时，可以都设为 $\frac{1}{N}$。然后将各滤波器的状态估计按其模式概率加权求和，得到融合状态估计。

以下是完整的实现代码：

```
def run_filter_bank():
    dt = 0.1
    cvfilter = make_cv_filter(dt, std=0.2)
    cafilter = make_ca_filter(dt, std=0.2)

    _, zs = generate_data(120, std=0.2)
    z_xs = zs[:, 0]
    xs, probs = [], []

    pv, pa = 0.8, 0.2
    pvsum, pasum = 0., 0.

    for z in z_xs:
        cvfilter.predict()
        cafilter.predict()
        cvfilter.update([z])
        cafilter.update([z])

        cv_likelihood = cvfilter.likelihood * pv
        ca_likelihood = cafilter.likelihood * pa

        pv = (cv_likelihood) / (cv_likelihood + ca_likelihood)
        pa = (ca_likelihood) / (cv_likelihood + ca_likelihood)

        x = (pv * cvfilter.x[0]) + (pa*cafilter.x[0])
        xs.append(x)
        probs.append(pv / pa)
```

```
    xs = np.asarray(xs)
    t = np.arange(0, len(xs) * dt, dt)
    plt.subplot(121)
    plt.plot(t, xs)
    plt.subplot(122)
    plt.plot(t, xs)
    plt.plot(t, z_xs)
    return xs, probs

xs, probs = run_filter_bank()
```



左图单独绘出融合估计，便于观察其平滑程度；右图同时画出估计与测量，展示滤波器对机动的跟踪情况。

我再次强调，这仍然是全书一直使用的贝叶斯算法。我们有两个或更多的测量值或估计值，每个值都对应一定的概率。最终估计是这些值的加权组合，各项权重与相应估计正确的概率成正比。每一步的概率都按下式计算：

$$\frac{\textbf{Prob(meas | state)} \times \textbf{prior}}{\textbf{normalization}}$$

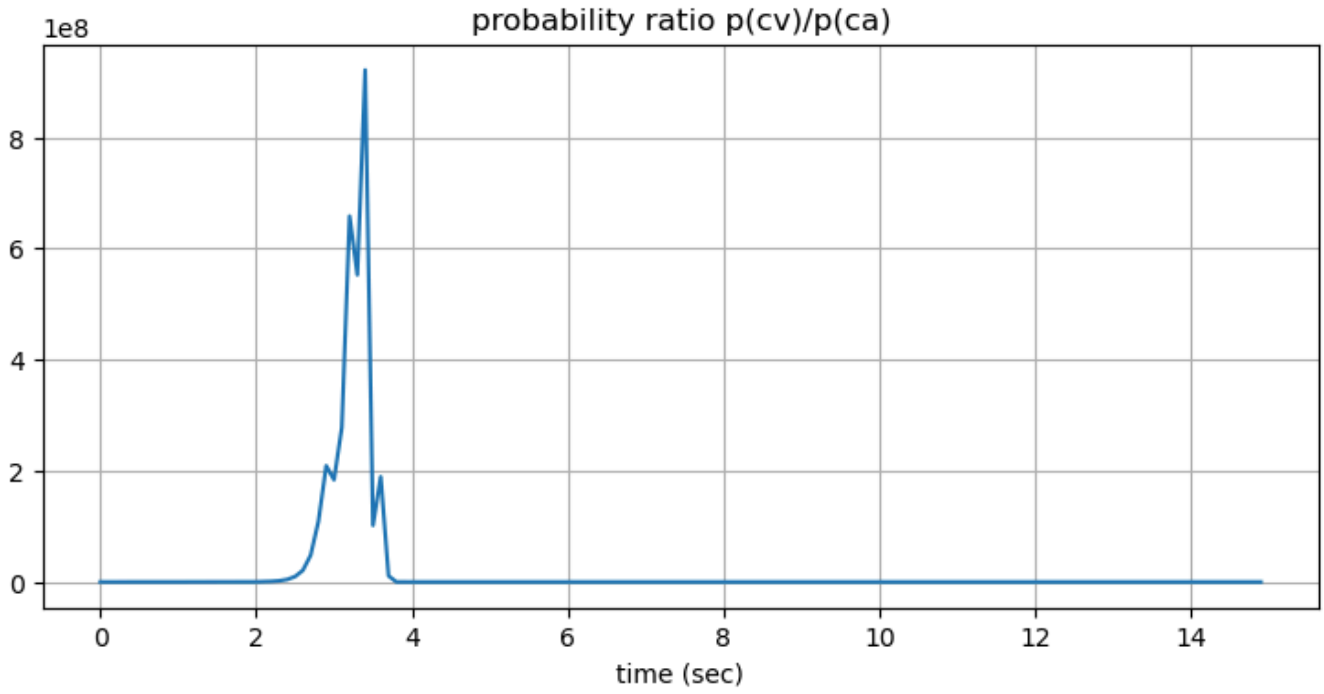
这就是贝叶斯定理。

实际问题往往需要两个以上的滤波器。例如，我在工作中用计算机视觉跟踪冰球：它会滑动、弹跳、跳跃滑行、滚动、反弹，也可能被拿起带走，或被球员快速控运。我还跟踪运动员，他们的非线性运动几乎变化无穷。在这些情况下，仅靠两个滤波器远远不够，还需考虑不同过程模型以及视觉检测噪声的不同假设。不过，基本思想是相同的。

MMAE 滤波器的局限性

正如我所阐述的，MMAE 存在一个严重的问题。请看这张图表，它展示了恒定速度滤波器和恒定加速度滤波器的概率比。

```
plt.plot(t[0:len(probs)], probs)
plt.title('probability ratio p(cv)/p(ca)')
plt.xlabel('time (sec)');
```



最初 3 秒，目标沿直线运动，匀速模型的概率远高于匀加速模型。机动开始后，概率迅速转而支持匀加速模型。但机动在第 6 秒已结束，预期匀速模型的概率会重新上升，图中却仍然接近零。

这是由模式概率的递推计算方式造成的：

$$p_k = \frac{\mathcal{L} p_{k-1}}{\sum \text{probabilities}}$$

当某个模式的概率变得极小时，在这种递推实现中可能很难恢复；若数值下溢到零，更会始终保持零。滤波器组于是迅速由少数高概率模型支配。稳健的实现需要监控模式概率，可淘汰低概率模型并用更合适的模型替换，也可从现有模型派生新模型，覆盖目标可能的行为特征。如果某个滤波器已经发散，还可以重新初始化，使其状态靠近当前测量。

交互多模型方法（IMM）

换个角度考虑多模型问题。仍以机动目标跟踪为例，可以设计一组采用不同假设的卡尔曼滤波器：它们的阶次可以不同，过程噪声大小也可以不同。每当新测量到达，都可以评估各模型与目标当前运动的相容程度。

这种简单的方法会导致组合爆炸的问题。在第 1 步中，我们会生成 N 个假设，也就是每个滤波器产生 1 个假设。第 2 步中，我们又会生成另外 N 个假设，这些假设需要与之前的 N 个假设结合起来，从而得到 N^2 个最终假设。人们尝试了多种方法来处理这个问题：要么淘汰那些不太可能的假设，要么将相似的假设合并起来。不过，这些算法仍然存在计算成本高和/或性能不佳的问题。在本书中，我不会详细讨论这些方法。不过，文献中比较著名的例子就是广义伪贝叶斯算法。

Blom [5] 提出交互多模型（IMM）算法，以解决模式序列导致的组合爆炸；Blom 与 Bar-Shalom 随后的论文 [6] 被广泛引用。基本思想是为系统的每种可能运动模式设置一个滤波器，并在每个时间步让它们交换信息：高概率模型对低概率模型的初始状态产生较大影响，使后者接近系统当前状态；低概率模型也会反过来影响高概率模型，但权重较小。整个混合过程依据概率进行。

例如，直行与转弯两种模式各对应一个滤波器，可以分别采用一阶与二阶模型。目标转弯时，二阶模型给出的估计较好，一阶模型则会滞后。各模型的测量似然可以反映与当前数据的相容程度。一阶模型的似然较低，因此其估计较大程度上受二阶模型修正；二阶模型更可信，因此只会受到一阶模型的小幅影响。

现在，假设目标物体不再转动了。由于我们不断用二阶滤波器的估算结果来修正一阶滤波器的估算结果，因此一阶滤波器对信号的滞后现象会大大减少。只需几个时间周期后，一阶滤波器就能给出非常准确的估算结果，从而成为最可靠的滤波器。之后，它将为二阶滤波器的估算结果做出重要贡献。需要记住的是，二阶滤波器容易将测量噪声误认为是加速度信号。这种调整有助于大幅减少这种错误。

各模式的概率

我们为该系统定义了若干种工作模式，记为 $\boldsymbol{m}$。我们假设目标始终处于这些模式中的某一种。在上面的讨论中，我们提到了“直线行驶”和“转弯”这两种模式，因此可以记为 $\boldsymbol{m} = \{\textbf{straight}, \textbf{turn}\}$。

为目标处于每种模式分别赋予概率，就得到模式概率向量，每个可能模式对应一个分量。$\boldsymbol{m}$ 包含两种模式，因此向量有两个概率。若认为目标以 70% 的概率直行，则可写成：

$$\boldsymbol{\mu} = [0.7 \quad 0.3]$$

由于两种模式的概率之和必须为 1，转弯模式的概率就是 0.3。这里采用文献中常用的 $\boldsymbol{\mu}$ 表示模式概率，注意不要将它与均值混淆。

在 Python 中，我们可以这样来实现。

```
mu = np.array([0.7, 0.3])
mu
```

```
array([0.7, 0.3])
```

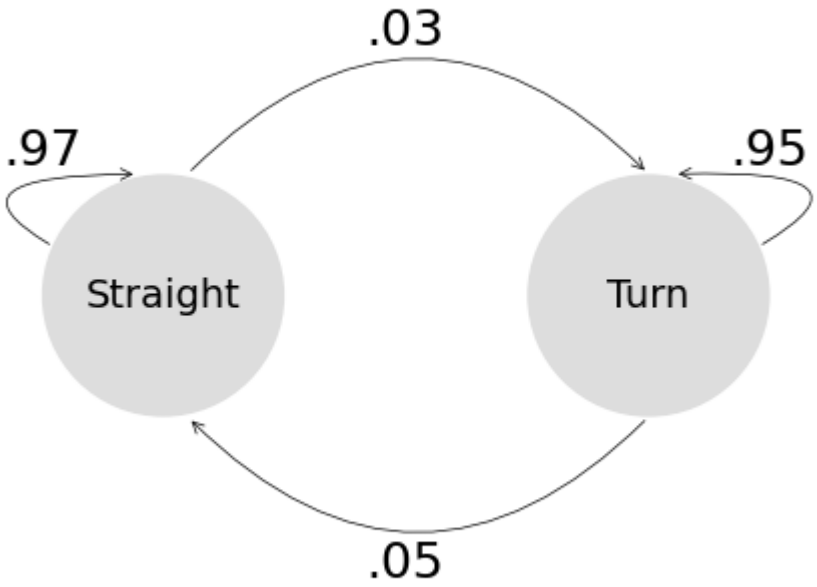
形式化地说，给定此前的测量信息 $\boldsymbol{Z}$，模型 $\boldsymbol{m_i}$ 为正确模式，即目标处于模式 $\boldsymbol{i}$ 的先验概率为：

$$\boldsymbol{\mu_i} = P(\boldsymbol{m_i}|\boldsymbol{Z})$$

模式转移

接下来，我们必须考虑到，这是一个会不断改变运动轨迹的目标。它会先直线前进，然后转弯，再继续直线前进。我们可以把这种状态转移过程建模为马尔可夫链，就像下图所示：

```
import kf_book.adaptive_internal as adaptive_internal
adaptive_internal.plot_markov_chain()
```



这里展示了一个目标的两种运动模式的示例：直线前进和转弯。如果目标当前处于直线前进状态，那么我们有 97%的概率认为它会继续直线前进，而只有 3%的概率认为它会开始转弯。一旦目标开始转弯，我们有 95%的概率认为它会继续沿转弯路径前进，而 5%的概率认为它会回到直线路径上。

该算法对具体数值并不敏感，通常需要通过模拟或试验来选择合适的数值。不过，这些数值还是具有相当的代表性。

我们用转移概率矩阵来表示马尔可夫链，我们将其记为 $\mathbf{M}$。对于图中的马尔可夫链，我们可以这样表示：

$$\mathbf{M} = \begin{bmatrix} .97 & .03 \\ .05 & .95 \end{bmatrix}$$

换句话说，$\mathbf{M}[i, j]$ 表示在上一模式为 i 的条件下，当前模式为 j 的概率。在这个例子中，如果上一模式是转弯 $(i = 1)$，那么当前模式为直线 $(j = 0)$ 的概率就是 $\mathbf{M}[1,\ 0] = 0.05$。在 Python 中，我们可以这样表示：

```
M = np.array([[.97, .03], [.05, .95]])
print(M)
print('From turn to straight probablility is', M[1, 0], 'percent')
```

```
[[0.97 0.03]
 [0.05 0.95]]
From turn to straight probablility is 0.05 percent
```

转移概率可以用来预测新的模式概率。转移后直行有两种可能：此前就直行并继续直行，或此前转弯而现在改为直行。前一种概率为 (0.7×0.97)，后一种为 (0.3×0.05)，即将先前模式概率乘以相应转移概率。两者相加得到直行的总概率 $(0.7)(0.97) + (0.3)(0.05) = 0.694$。

回顾第二章的全概率定理：若若干互斥且完备的事件构成划分，就可以按这些条件事件求和，计算所关心事件的概率：

$$P(A) = \sum P(A \mid B)\, P(B)$$

这里，$P(A \mid B)$ 对应转移概率矩阵 $\mathbf{M}$ 中的元素，$P(B)$ 对应模式概率向量 $\boldsymbol{\mu}$ 中的分量。利用向量与矩阵相乘即可完成各项乘积的求和：

$$\begin{bmatrix}\mu_1 & \mu_2\end{bmatrix} \begin{bmatrix}m_{11} & m_{12} \\ m_{21} & m_{22}\end{bmatrix} = \begin{bmatrix}\mu_1 m_{11} + \mu_2 m_{21} & \mu_1 m_{12} + \mu_2 m_{22}\end{bmatrix}$$

IMM 文献将这一点表述为：

$$\bar{c}_j = \sum_{i=1}^{N} \mu_i M_{ij}$$

我们使用 NumPy 中的 `dot` 函数来计算这个值。当然，我们也可以使用矩阵乘法运算符 `@`，不过我觉得，用点号来表示求和操作——也就是点积——更为直观。

```
cbar = np.dot(mu, M)
cbar
```

```
array([0.694, 0.306])
```

计算模式概率

我们将利用贝叶斯定理来计算新的模式概率。请记住，贝叶斯定理的内容如下：

$$\text{posterior} = \frac{\text{prior} \cdot \text{likelihood}}{\text{normalization factor}}$$

这里的“先验概率”指的是我们在上一节中计算出的总概率。卡尔曼滤波器则负责计算“似然值”，即在滤波器当前状态下，各项测量结果的合理性程度。为了便于理解，公式如下：

$$\mathcal{L} = \frac{1}{\sqrt{2\pi \mathrm{S}}} \exp\left[-\frac{1}{2}\mathrm{y}^{\mathsf{T}}\mathrm{S}^{-1}\mathrm{y}\right]$$

用数学符号表示，更新后的模式概率为：

$$\mu_i = \|\mathcal{L}_i \bar{c}_i\|$$

换句话说，对每个卡尔曼滤波器，也就是每种模式，先根据模式转移计算当前模式的预测概率，再乘以该模式下当前测量的似然；最后把所有结果归一化，使总和为 1。模式概率与测量似然是不同的量。

在 Python 中计算很简单。用变量 L 保存各模型的测量似然。KalmanFilter.update() 会计算这些似然；但下面尚未创建滤波器，因此先给 L 指定几个演示值：

```
# L = [kf0.L, kf1.L]  # get likelihoods from Kalman filters
L = [0.000134, 0.0000748]
mu = cbar * L
mu /= sum(mu) # normalize
mu
```

```
array([0.802, 0.198])
```

可以看到，直行模型下较大的测量似然，把直行模式的概率从 70% 推高到 80.2%。

混合概率

此时，可以利用模式转移计算所有可能路径的概率。假设 $\mu = [0.73 \quad 0.27]$，再用转移概率矩阵计算后续情况：当前直行模式的概率为 $(\mu = 0.73)$，下一步可以继续直行或转弯；对转弯模式 $(\mu = 0.27)$ 也进行同样计算。这样，2 条模式路径变成 4 条，下一步变成 8 条，依此类推。穷举计算虽然直接，实际却不可行；仅 30 步，用双精度保存路径概率就需要约 8 GiB 内存。

需要一种计算可行的近似方法。IMM 通过混合概率解决这个问题。假设直行模式当前概率较高，转弯模式概率较低；不是让各滤波器完全独立地演化，而是为每个目标模式融合滤波器组中各模型的状态估计。较可信的来源模型具有较大权重，因此它们的信息能帮助低概率模型保持合理的状态。这就是算法的关键。

基本思路是：各个卡尔曼滤波器先完成更新，得到新的均值与协方差；再依据称为 ω 的混合概率，为下一轮各模型计算混合初始状态 $\mathbf{x}_j^m$ 与混合初始协方差 $\mathbf{P}_j^m$。高概率模型只会受到低概率模型的小幅影响，低概率模型则会明显受到高概率模型的影响。这些量在文献中称为混合条件或混合初始条件。混合均值是加权均值，而混合协方差还必须包含各模型均值偏离混合均值所产生的项，不能只对原协方差简单加权。公式如下：

$$\mathbf{x}_j^m = \sum_{i=1}^{N} \omega_{ij} \mathbf{x}_i$$

$$\mathbf{P}_j^m = \sum_{i=1}^{N} \omega_{ij} \left[(\mathbf{x}_i - \mathbf{x}_j^m)(\mathbf{x}_i - \mathbf{x}_j^m)^\mathsf{T} + \mathbf{P}_i \right]$$

可将下标理解为数组索引，下面用伪 Python 代码表示。计算目标模式 j 的混合协方差时，各模型均值都应减去该模式的混合均值 x0[j]，再用同一个偏差向量与自身计算外积。

```
for j in N:
x0[j] = sum_over_i(w[i,j] * x[i])
P0[j] = sum_over_i(w[i, j] * (P[i] + np.outer(x[i] - x0[j], x[i] - x0[j])))
```

不要被记号掩盖了简单的思想：让高概率模型的估计帮助修正低概率模型，使各滤波器都尽量保持合理的状态估计。

我们该如何计算这些混合概率呢？请先仔细思考一下，试着给出一个合理的答案，然后再继续阅读。我们有描述各模式当前概率的“模式概率”，还有描述我们切换模式的可能性的“转移概率”。那么，我们该如何计算出新的概率呢？

仍然使用贝叶斯思想：将先验模式概率与模式转移概率相乘，再按目标模式归一化。这里的转移概率存储在矩阵 $\mathbf{M}$ 中；应注意它不同于前面根据残差计算的测量似然。

$$\omega_{ij} = \|\mu_i \cdot \mathbf{M}_{ij}\|$$

下面计算这些量。前面对 $\boldsymbol{\mu}$ 和 $\bar{\boldsymbol{c}}$ 的更新仅作分步演示，计算顺序尚未统一：必须先将转移概率矩阵与先前的 $\boldsymbol{\mu}$ 结合，得到 $\bar{\boldsymbol{c}}$。这里按正确顺序重新计算：

```
cbar = np.dot(mu, M) #compute total probability that target is in mode j

omega = np.zeros((2, 2))
for i in range(2):
    for j in range(2):
        omega[i, j] = (M[i, j] * mu[i]) / cbar[j]
omega
```

```
array([[0.987, 0.114],
       [0.013, 0.886]])
```

卡尔曼滤波器需要通过预测步骤来计算新的先验值。在计算过程中，它们会使用混合估计值。

$$\bar{\mathbf{x}}_j = \mathbf{F}_j\mathbf{x}_j^m$$
$$\bar{\mathbf{P}}_j = \mathbf{F}_j\mathbf{P}_j^m\mathbf{F}_j^\mathsf{T} + \mathbf{Q}_j$$

IMM 融合估计

最后，使用当前测量更新后的后验模式概率，对各滤波器更新后的状态估计进行加权融合，并计算包含模型间均值差异的混合协方差：

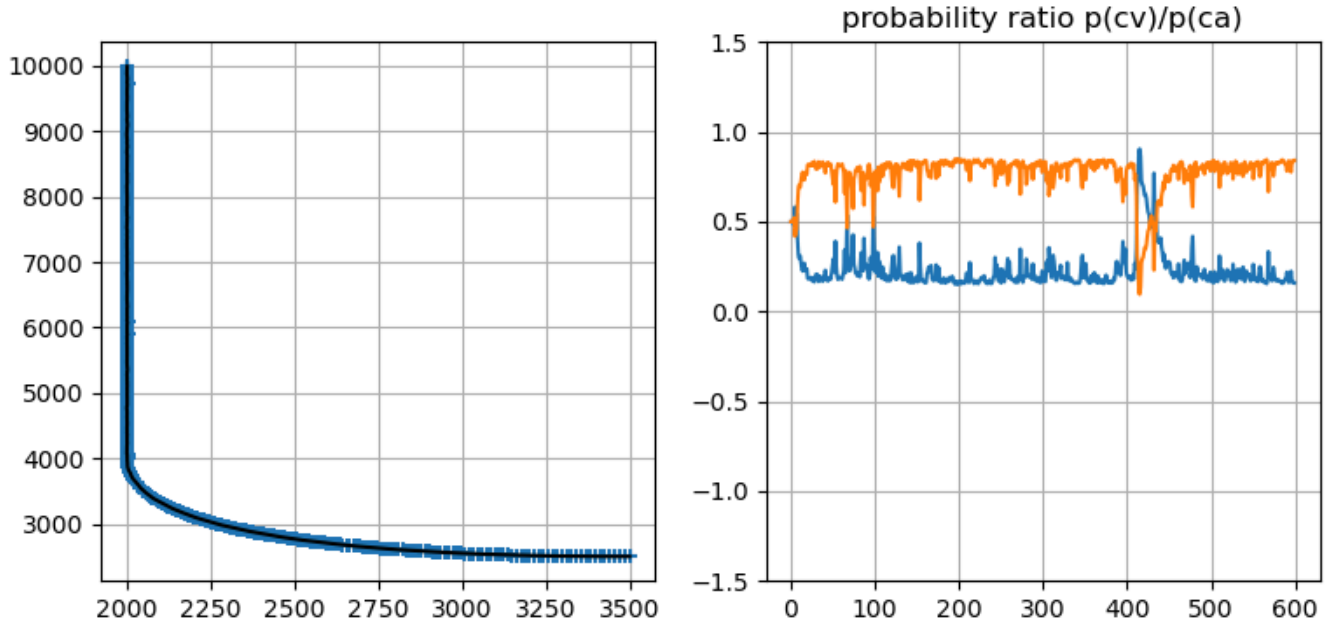
$$\mathbf{x} = \sum_{j=1}^N \mu_j\mathbf{x}_j$$
$$\mathbf{P} = \sum_{j=1}^N \mu_j \left[(\mathbf{x}_j - \mathbf{x})(\mathbf{x}_j - \mathbf{x})^\mathsf{T} + \mathbf{P}_j\right]$$

利用 IMM 技术追踪机动目标

来看一个具体例子。Crassidis 和 Junkins 的著作 [4] 是少数提供完整演算示例的教材之一，因此这里采用其中的例子：跟踪目标 600 秒。目标最初直行，从第 400 秒开始加入控制输入，使其转弯 90 度。使用两个匀加速卡尔曼滤波器，一个假设没有过程噪声，另一个假设过程噪声谱密度为 $10^{-3}\mathbf{I}$。还假设滤波器初始化非常准确，两个滤波器均取极小的初始协方差 $\mathbf{P} = 10^{-12}$。下面给出实现。

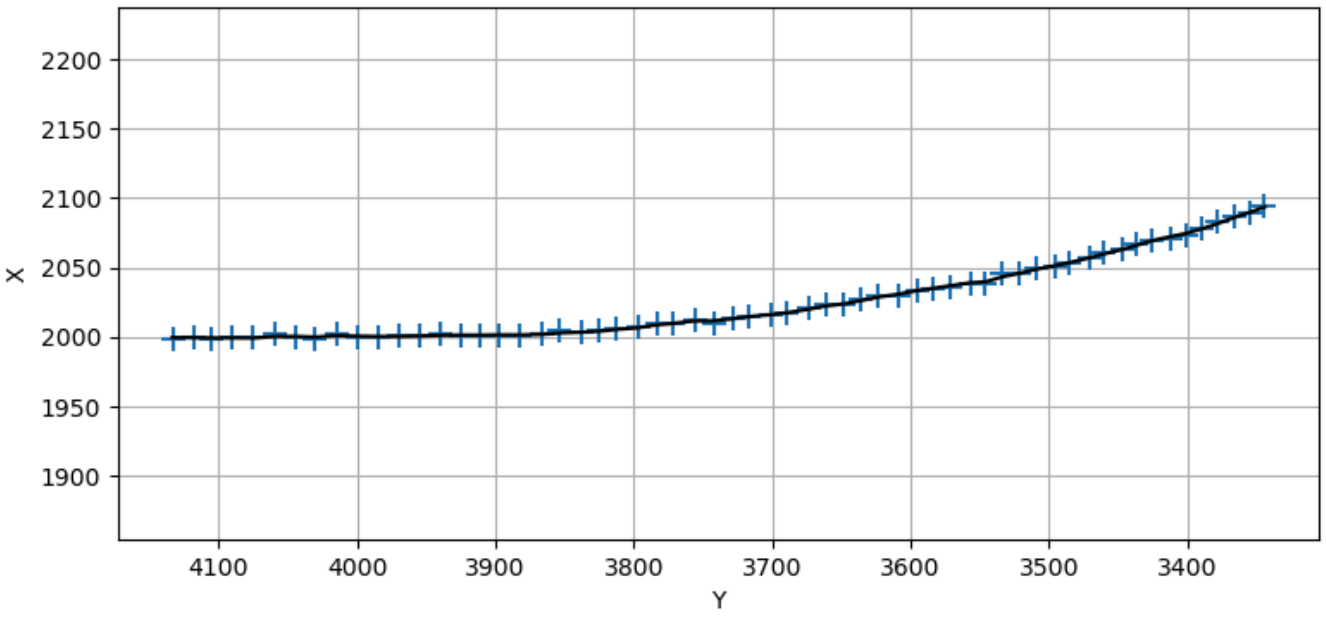
```
import copy
from scipy.linalg import block_diag
from filterpy.kalman import IMMEstimator
from filterpy.common import Saver
```

```
N = 600
dt = 1.
imm_track = adaptive_internal.turning_target(N)

# create noisy measurements
zs = np.zeros((N, 2))
r = 1
for i in range(N):
    zs[i, 0] = imm_track[i, 0] + randn()*r
    zs[i, 1] = imm_track[i, 2] + randn()*r

ca = KalmanFilter(6, 2)
dt2 = (dt**2)/2
F = np.array([[1, dt, dt2],
              [0,  1,  dt],
              [0,  0,   1]])

ca.F = block_diag(F, F)
ca.x = np.array([[2000., 0, 0, 10000, -15, 0]]).T
ca.P *= 1.e-12
ca.R *= r**2
q = np.array([[.05, .125, 1/6],
              [.125, 1/3, .5],
              [1/6, .5, 1]])*1.e-3
ca.Q = block_diag(q, q)
ca.H = np.array([[1, 0, 0, 0, 0, 0],
                 [0, 0, 0, 1, 0, 0]])

# create identical filter, but with no process error
cano = copy.deepcopy(ca)
cano.Q *= 0

filters = [ca, cano]

M = np.array([[0.97, 0.03],
              [0.03, 0.97]])
mu = np.array([0.5, 0.5])
bank = IMMEstimator(filters, mu, M)

xs, probs = [], []
for i, z in enumerate(zs):
    z = np.array([z]).T
    bank.predict()
    bank.update(z)

    xs.append(bank.x.copy())
    probs.append(bank.mu.copy())

xs = np.array(xs)
probs = np.array(probs)
plt.subplot(121)
plt.plot(xs[:, 0], xs[:, 3], 'k')
plt.scatter(zs[:, 0], zs[:, 1], marker='+')

plt.subplot(122)
plt.plot(probs[:, 0])
plt.plot(probs[:, 1])
plt.ylim(-1.5, 1.5)
plt.title('probability ratio p(cv)/p(ca)');
```



很难直接看出该滤波器的性能如何，那么让我们在转弯开始时来观察一下其表现吧。我调换了 x 和 y 轴的坐标，以便能更仔细地观察情况。下图中的转弯点位于 $Y = 4000$ 处。仔细观察可以发现，转弯开始后，估算值会有轻微的波动，但滤波器能够即时、准确地跟踪测量结果，很快就能恢复平稳的跟踪状态。

```
plt.plot(xs[390:450, 3], xs[390:450, 0], 'k')
plt.scatter(zs[390:450, 1], zs[390:450, 0], marker='+', s=100);
plt.xlabel('Y'); plt.ylabel('X')
plt.gca().invert_xaxis()
plt.axis('equal');
```



IMM 的局限性

我对 IMM 的实际使用经验不算丰富，因此无法充分讨论其局限。但 IMM 最初就是为航空交通管制中的机动飞机跟踪而提出的，据相关应用报道，它在这类任务中的表现很好。

这种实现有若干前提，其中最重要的是滤波器组中的状态向量必须具有相同维数。回顾数学形式就能理解原因：IMM 需要进行如下融合计算。

$$\mathbf{x} = \sum_{j=1}^{N} \mu_j \mathbf{x}_j$$

在本节直接加权相加的实现中，各滤波器的状态 $\boldsymbol{x}$ 必须具有相同维数；相同位置的分量 $\boldsymbol{x}[i]$ 还必须具有相同的物理含义和单位。

例如，若尝试把采用匀速模型和采用匀加速模型的滤波器直接放入同一个 IMM，而它们的状态 $\boldsymbol{x}$ 维数不同，就不能直接混合。FilterPy 对这种不同状态维数的组合会抛出 ValueError。

```
ca = KalmanFilter(3, 1)
cv = KalmanFilter(2, 1)

trans = np.array([[0.97, 0.03],
                  [0.03, 0.97]])

# Deliberately demonstrate the documented dimension mismatch.
try:
    imm = IMMEstimator([ca, cv], (0.5, 0.5), trans)
except ValueError as exc:
    print("Expected dimension error:", exc)
```

```
Expected dimension error: All filters must have the same state dimension
```

我偶尔会收到相关邮件或问题报告。过去曾建议将匀速滤波器的状态也扩展到三维，再让 F 在状态传播时忽略加速度，如下所示。

```
F = np.array([[1, dt, 0],
[0, 1, 0],
[0, 0, 0]])
```

回顾起来，我不确定这是否是个合理的建议。虽然这种方法能让 IMM 运行，但加速度的估计显然会有问题：其中一个滤波器能准确估计加速度，另一个给出的加速度估计却为 0。混合后得到的不准确加速度还会用于下一次预测。

更极端地说，若一个滤波器把 x[2] 解释为线加速度，另一个却解释为角速度，那么混合后的 x[2] 就没有物理意义，因为这两种量不能相加。

正如我所说，我对 IMM 并不太了解。也许相关的文献中会介绍如何处理这些情况。我只能说，FilterPy 所实现的 IMM 方式不适用于这些用例。

航空交通管制中的 IMM 使用不同的过程假设：飞机可能平飞、爬升或下降，也可能作协调转弯或非协调转弯。可以为每种情况设置不同的 F 和 Q，但所有滤波器的状态向量 x 应采用相同的分量定义和单位，而不是要求它们给出相同数值的估计。

本章小结

本章包含了本书中一些较为复杂的内容。不过，它却是实现实用型卡尔曼滤波器的关键所在。如果我们正在控制某个机器人，那么我们就知道该机器人的过程模型，此时构建卡尔曼滤波器就相当简单。但更常见的情况是，我们只能得到一组时间序列数据，需要从中提取有用的信息。在这种情况下，该过程模型对我们来说几乎是未知的。我们需要利用本章中的技术来学习如何对模型进行参数化处理。由于目标状态会随时间而变化，因此我们的滤波器也必须具备自适应能力。

精确求解所有可能模式序列会导致组合爆炸，实际中难以承担。IMM 因性能良好且计算量可控，已成为常用算法。

实际滤波器组通常不止两个模型，可能包含多个滤波器。随着目标运动状态改变，某些模型的后验概率会变得极低。自适应滤波器组可以采用淘汰与替换策略，以更适合当前运动的模型替换低概率模型。这类策略高度依赖具体问题，往往需要专门设计，并用仿真或真实数据检验。

尽管这些算法相当复杂，但我希望你们能明白，其背后的原理其实非常简单。我们仍然使用第二章中学到的两种工具：贝叶斯定理和全概率定理。我们利用贝叶斯定理来处理新信息，再利用全概率定理来分析各种过程模型所带来的影响。

对我来说，这一章充分体现了卡尔曼滤波器中贝叶斯方法的优点。我并不太在意你是否了解 IMM 算法的细节。但我希望你能明白：这些结果其实都是通过非常简单的概率推理得出的。卡尔曼博士所推导出的卡尔曼滤波器的线性代数方程，其实源自另一种称为“正交投影”的推理方式。这种推理方式本身也很美妙，我建议你阅读他的论文。不过，我觉得这些公式不太容易理解，也不清楚如何利用这些方法来设计出像 IMM 这样的非最优滤波器。相比之下，贝叶斯定理则能让我们轻松地解决这些问题。

参考文献

- [1] Bar-Shalom, Y., Xiao-Rong L., and Thiagalingam Kirubarajan.《估计方法及其在跟踪与导航中的应用》。纽约：Wiley 出版社，第 424 页，2001 年。
- [2] Zarchan, P.和 Musoff, H.,《卡尔曼滤波基础：实用方法》第四版。弗吉尼亚州雷斯顿：美国航空航天学会，第 584-595 页，2015 年。
- [3] Simon, D. Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches. Hoboken, NJ: Wiley-Interscience, pp. 208–212, 2006.
- [4] Crassidis, John L.和 John L. Junkins. 动态系统的最优估计. CRC 出版社, 2011.
- [5] Blom, H.A.P.,《一种适用于突发变化系统的有效滤波算法》，载于《第 23 届决策与控制会议论文集》，内华达州拉斯维加斯，1984 年 12 月。
- [6] Blom, H.A.P.和 Bar-Shalom, Y.,“具有马尔可夫切换系数的系统的交互多模型算法”,《IEEE 自动控制汇刊》，第 AC-8 卷，第 8 期，1998 年 8 月，第 780-783 页。

附录 A：安装

本书的 22 个笔记本、608 个代码单元已在 Python 3.12 的 Jupyter 环境中逐章执行通过。安装已验证的依赖并启动 Notebook，请在仓库根目录运行：

```
python -m pip install -r requirements-notebooks.txt
jupyter notebook
```

本书使用 Jupyter Notebook 编写。这是一个基于浏览器的交互式 Python 环境，可以混合 Python、文字和数学。我选择它，是因为交互功能非常重要：在开始使用交互环境之前，我觉得卡尔曼滤波几乎无法学会。如果不能改变参数并立即看到输出，就很难对许多参数形成直觉。交互环境也让你能够试验各种“如果”：例如，“如果把 Q 设为零会怎样？”在 Jupyter Notebook 中，找到答案轻而易举。

另一个原因是，大多数教材并不展示许多实现细节。例如，某段伪代码旁边有一幅漂亮的图。图是软件生成的，但读者拿不到软件。我希望制作这本书用到的一切都对你开放。协方差椭圆是怎么画出来的？大多数书不会告诉你；在 Jupyter Notebook 中，只要查看源代码就知道了。

即使选择在线阅读，你仍然需要安装 Python 和 SciPy 软件栈，以便编写自己的卡尔曼滤波器。安装方法很多，无法一一介绍，但我会说明几种典型情况。

安装 SciPy 软件栈

本书需要 IPython、Jupyter、NumPy、SciPy、SymPy 和 Matplotlib。由 NumPy、SciPy、Matplotlib 组成的 SciPy 软件栈依赖第三方 Fortran 和 C 代码，从源代码安装并不简单。SciPy 网站强烈建议使用预编译发行版，我同意这一建议。

Jupyter Notebook 让你在浏览器内运行 Python，本书就是一组 Jupyter 笔记本。IPython 为 Jupyter 和数据可视化提供基础。NumPy、SciPy 提供滤波器所用的线性代数实现。SymPy 负责符号运算，我会用它对代数方程求导。Matplotlib 则提供绘图功能。

我使用 Continuum Analytics 的 Anaconda 发行版。它把上述全部软件包及许多其他包整合在一起，是很好的发行版；IPython 也推荐这种安装方式。安装过程很直接，可以与计算机上已有的其他 Python 安装并存，而且可以免费使用。下载地址：http://continuum.io/downloads 。我强烈建议选择它提供的最新 Python 3 版本；本次代码验证使用 Python 3.12，具体依赖版本见本附录开头。

也有其他安装 SciPy 软件栈的方法，见 http://scipy.org/install.html 。过程可能很繁琐，我不为此提供支持或详细说明。

许多 Linux 发行版预装了这些包，但通常版本较旧；本书依赖较新的版本，可能需要升级。如何更新特定 Linux 系统不在本书范围内。Anaconda 的优点是不修改系统原有的 Python，因此通常不会破坏 Linux 发行版。有些读者安装了 Anaconda，却仍在使用系统默认 Python，导致书中程序运行异常，需要留意。

我不会为旧版库运行回归测试。事实上，我知道代码无法在例如 2014—2015 年的旧版库上运行。我不想把全部时间都花在一本书的技术支持上，因此请读者自行安装较新的 Python 和 SciPy 软件栈。

安装时请采用本附录开头已验证的 Python 与依赖组合。

如果安装了最新 Anaconda 仍有问题，请在本书的 GitHub 仓库提交错误报告；我会继续保证本书适配新的 Anaconda 版本。对于旧环境则无法提供同等支持，我没有时间兼顾每个人不同的配置。`jupyter notebook` 等软件发展很快，我无法跟进所有变化，同时还保持对所有旧版的兼容。

如果其他项目需要旧版软件，Anaconda 允许多个版本并行安装，文档见：

https://conda.io/docs/user-guide/tasks/manage-python.html

部分安装链接随软件版本演进可能发生变化；实际安装请优先采用本附录开头给出的命令和固定依赖组合。

安装 FilterPy

FilterPy 是一个 Python 库，实现了本书使用的全部滤波器及许多其他滤波器。可以使用 `pip` 安装，在命令提示符下运行：

```
pip install filterpy
```

FilterPy 由我编写，最新开发版本始终可以在 https://github.com/rlabbe/filterpy 获取。

下载和运行本书

本书存储在 GitHub 仓库中。在命令行输入：

```
git clone --depth=1 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python.git
```

这会创建名为 Kalman-and-Bayesian-Filters-in-Python 的目录。`depth` 参数只获取最新版本；除非需要完整提交历史，否则这样更快、更节省空间。

没有安装 Git 时，可以在浏览器访问 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python 并下载本书。

接着，在命令行进入刚创建的目录，再运行 Jupyter Notebook：

```
cd Kalman-and-Bayesian-Filters-in-Python
jupyter notebook
```

浏览器窗口应当打开，显示本书所有章节。点击第一章，再打开相应的笔记本链接。

有关运行 Notebook 的更多信息见：

http://jupyter-notebook-beginner-guide.readthedocs.org/en/latest/execute.html

配套软件

本书专用代码存放在 *./kf_book* 子目录中。这些代码仍在变化，所以暂时不在此详细记录。使用该目录代码时，我会在书中说明，因此不必觉得神秘。

kf_book 里有形如 `xxx_internal.py` 的 Python 文件，保存对某章有用的函数。这样可以隐藏不太值得逐行阅读的辅助代码。例如，生成图表时，我希望你关注图表表达的内容，而非用 Python 绘图的步骤；若感兴趣，可以直接查看源代码。

有些章节引入的函数会在后文继续使用。它们最初在 Notebook 中定义，同时也存入 Python 文件，供后续章节导入。我会在首次定义时说明，但相关整理仍未完成。我尽量避免这种重复，以免目录代码与书中代码不同步。不过，IPython Notebook 没有提供跨笔记本引用代码单元的功能，这是我所知道的共享这些代码的唯一机制。

另有一个没有文档的 **experiments** 目录，是我把代码放入书中之前进行编写和测试的地方。里面有有趣的东西，可以自由查看。随着本书发展，我计划创建示例和项目，许多材料会移入其中，小型试验最终可能删除。若只想阅读本书，可以放心忽略它。

./kf_book 中有定义本书外观的 CSS 文件。IPython Notebook 默认外观较朴素，这方面仍在改进。我借鉴了 Probabilistic Programming and Bayesian Methods for Hackers 等书，也深受 Lorena Barba 教授的优秀工作影响：CFDPython。本书的外观得益于这些项目。

使用 Jupyter Notebook

完整的 Jupyter Notebook 教程超出了本书范围，网上有很多可用资料。简而言之，Python 代码放在单元中，前面通常有 `In [1]:` 这样的提示，代码本身位于方框中。当焦点在代码框内时按 CTRL-ENTER，代码会运行，结果显示在下方。例如：

```
print(3+7.2)
```

```
10.2
```

如果现在用 Jupyter Notebook 打开了本书，请修改上面的代码，改变 print 语句里的表达式，再按 CTRL+ENTER。输出应随你输入的代码而变化。

SymPy

SymPy 是执行符号数学运算的 Python 包。它的完整能力超出本书范围，但可以进行代数运算、积分、求导、求解微分方程等等。例如，我们会用它计算向量值函数的雅可比矩阵，以及期望值的积分。

先看一个简单例子：导入 SymPy，初始化美化输出功能（使用 LaTeX 显示公式），再声明一个供 SymPy 使用的符号。

```
import sympy
sympy.init_printing(use_latex='mathjax')

phi, x = sympy.symbols('\phi, x')
phi
```

$$\phi$$

注意，它如何用 LaTeX 显示符号 `phi`。现在做一些数学运算：$\sqrt{\phi}$ 的导数是什么？

```
sympy.diff('sqrt(phi)')
```

$$\frac{1}{2\sqrt{\phi}}$$

可以对表达式进行因式分解：

```
sympy.factor(phi**3 -phi**2 + phi - 1)
```

$$(\phi - 1)\left(\phi^2 + 1\right)$$

也可以展开表达式。

```
((phi+1)*(phi-4)).expand()
```

$$\phi^2 - 3\phi - 4$$

可以给变量赋具体数值，计算表达式的值：

```
w =x**2 -3*x +4
print(w.subs(x, 4))
print(w.subs(x, 12))
```

```
8
112
```

也可以使用字符串，表示包含尚未定义符号的表达式：

```
x = sympy.expand('(t+1)*2')
x
```

$$2t + 2$$

现在用 SymPy 计算雅可比矩阵。给定函数

$$h = \sqrt{(x^2 + z^2)}$$

求它相对于 x、y、z 的雅可比矩阵。

```
x, y, z = sympy.symbols('x y z')

H = sympy.Matrix([sympy.sqrt(x**2 + z**2)])

state = sympy.Matrix([x, y, z])
H.jacobian(state)
```

$$\begin{bmatrix}\frac{x}{\sqrt{x^2+z^2}} & 0 & \frac{z}{\sqrt{x^2+z^2}}\end{bmatrix}$$

下面根据连续过程噪声矩阵，计算离散过程噪声矩阵 $\mathbf{Q}$。连续时间噪声的协方差强度矩阵为

$$\mathbf{Q} = \Phi_s \begin{bmatrix}0 & 0 & 0\\ 0 & 0 & 0\\ 0 & 0 & 1\end{bmatrix}$$

积分为

$$\mathbf{Q} = \int_0^{\Delta t} \mathbf{F}(t)\mathbf{Q}\mathbf{F}^T(t)\,dt$$

其中

$$\mathbf{F}(\Delta t) = \begin{bmatrix}1 & \Delta t & \Delta t^2/2\\ 0 & 1 & \Delta t\\ 0 & 0 & 1\end{bmatrix}$$

这里要区分连续时间噪声强度与离散过程噪声协方差。积分中的矩阵表示前者，等号左侧表示后者；虽然都使用 Q 作为记号，它们的含义和单位不同。

```
dt = sympy.symbols('\Delta{t}')
F_k = sympy.Matrix([[1, dt, dt**2/2],
                    [0,  1,      dt],
                    [0,  0,       1]])
Q = sympy.Matrix([[0,0,0],
                  [0,0,0],
                  [0,0,1]])

sympy.integrate(F_k*Q*F_k.T,(dt, 0, dt))
```

$$\begin{bmatrix}\frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6}\\ \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2}\\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t\end{bmatrix}$$

相关链接

https://ipython.org/

https://jupyter.org/

https://www.scipy.org/

附录 B：符号与记法

下面汇集了不同作者在表示线性卡尔曼滤波方程时所使用的记法。

Labbe

$$
\begin{aligned}
\bar{\mathbf{x}} &= \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u} \\
\bar{\mathbf{P}} &= \mathbf{F}\mathbf{P}\mathbf{F}^{\mathsf{T}} + \mathbf{Q}
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{y} &= \mathbf{z} - \mathbf{H}\bar{\mathbf{x}} \\
\mathbf{S} &= \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^{\mathsf{T}} + \mathbf{R} \\
\mathbf{K} &= \bar{\mathbf{P}}\mathbf{H}^{\mathsf{T}}\mathbf{S}^{-1} \\
\mathbf{x} &= \bar{\mathbf{x}} + \mathbf{K}\mathbf{y} \\
\mathbf{P} &= (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}
\end{aligned}
$$

维基百科的记法

$$
\begin{aligned}
\hat{\mathbf{x}}_{k|k-1} &= \mathbf{F}_k\hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k\mathbf{u}_k \\
\mathbf{P}_{k|k-1} &= \mathbf{F}_k\mathbf{P}_{k-1|k-1}\mathbf{F}_k^{\mathsf{T}} + \mathbf{Q}_k \\
\tilde{\mathbf{y}}_k &= \mathbf{z}_k - \mathbf{H}_k\hat{\mathbf{x}}_{k|k-1} \\
\mathbf{S}_k &= \mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^{\mathsf{T}} + \mathbf{R}_k \\
\mathbf{K}_k &= \mathbf{P}_{k|k-1}\mathbf{H}_k^{\mathsf{T}}\mathbf{S}_k^{-1} \\
\hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\tilde{\mathbf{y}}_k \\
\mathbf{P}_{k|k} &= (I - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1}
\end{aligned}
$$

Brookner

$$
\begin{aligned}
X^*_{n+1,n} &= \Phi X^*_{n,n} \\
X^*_{n,n} &= X^*_{n,n-1} + H_n(Y_n - MX^*_{n,n-1}) \\
H_n &= S^*_{n,n-1}M^{\mathsf{T}}[R_n + MS^*_{n,n-1}M^{\mathsf{T}}]^{-1} \\
S^*_{n,n-1} &= \Phi S^*_{n-1,n-1}\Phi^{\mathsf{T}} + Q_n \\
S^*_{n-1,n-1} &= (I - H_{n-1}M)S^*_{n-1,n-2}
\end{aligned}
$$

Gelb

$$
\begin{aligned}
\underline{\hat{x}}_k(-) &= \Phi_{k-1}\underline{\hat{x}}_{k-1}(+) \\
\underline{\hat{x}}_k(+) &= \underline{\hat{x}}_k(-) + K_k[Z_k - H_k\underline{\hat{x}}_k(-)] \\
K_k &= P_k(-)H_k^{\mathsf{T}}[H_kP_k(-)H_k^{\mathsf{T}} + R_k]^{-1} \\
P_k(-) &= \Phi_{k-1}P_{k-1}(+)\Phi_{k-1}^{\mathsf{T}} + Q_{k-1} \\
P_k(+) &= (I - K_kH_k)P_k(-)
\end{aligned}
$$

Brown

$$
\begin{aligned}
\hat{\mathbf{x}}_{k+1}^- &= \phi_k\hat{\mathbf{x}}_k \\
\hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_k^- + \mathbf{K}_k[\mathbf{z}_k - \mathbf{H}_k\hat{x}_k^-] \\
\mathbf{K}_k &= \mathbf{P}_k^-\mathbf{H}_k^{\mathsf{T}}[\mathbf{H}_k\mathbf{P}_k^-\mathbf{H}_k^T + \mathbf{R}_k]^{-1} \\
\mathbf{P}_{k+1}^- &= \phi_k\mathbf{P}_k\phi_k^{\mathsf{T}} + \mathbf{Q}_k \\
\mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_k^-
\end{aligned}
$$

Zarchan

$$
\begin{aligned}
\hat{x}_k &= \Phi_k\hat{x}_{k-1} + G_ku_{k-1} + K_k[z_k - H\Phi_k\hat{x}_{k-1} - HG_ku_{k-1}] \\
M_k &= \Phi_kP_{k-1}\Phi_k^{\mathsf{T}} + Q_k \\
K_k &= M_kH^{\mathsf{T}}[HM_kH^{\mathsf{T}} + R_k]^{-1} \\
P_k &= (I - K_kH)M_k
\end{aligned}
$$

这里的 (−) 表示预测得到的先验量，(+) 表示测量更新后的后验量。比较不同记法时，应同时核对时间下标、先后验标记及矩阵维数。

附录 D：H∞ 滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

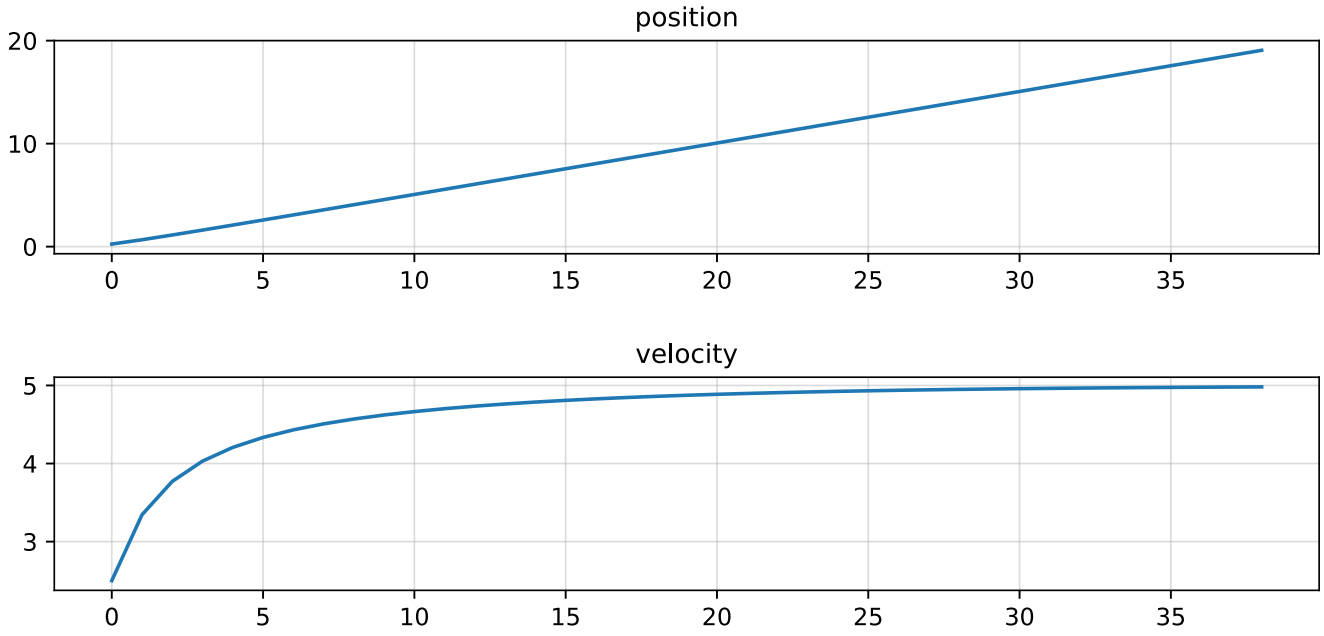
我仍在考虑该如何写这一章。在此期间，可以阅读克利夫兰州立大学 Dan Simon 教授的这篇易懂的介绍：

http://academic.csuohio.edu/simond/courses/eec641/hinfinity.pdf

用一句话说，$\boldsymbol{H}_\infty$（H 无穷）滤波器类似卡尔曼滤波器，但在面对非高斯、不可预测的输入时具有鲁棒性。

我的 FilterPy 库包含 H∞ 滤波器。下面粘贴了一些测试代码，实现了 Simon 在上述文章中设计的滤波器，希望对你有帮助。

```
import numpy as np
import matplotlib.pyplot as plt

from filterpy.hinfinity import HInfinityFilter

dt = 0.1
f = HInfinityFilter(2, 1, dim_u=1, gamma=.01)

f.F = np.array([[1., dt],
                [0., 1.]])

f.H = np.array([[0., 1.]])
f.G = np.array([[dt**2 / 2, dt]]).T

f.P = 0.01
f.W = np.array([[0.0003, 0.005],
                [0.0050, 0.100]])/ 1000  #process noise

f.V = 0.01
f.Q = 0.01
u = 1. #acceleration of 1 f/sec**2

xs = []
vs = []

for i in range(1,40):
    f.update (5)
    #print(f.x.T)
    xs.append(f.x[0,0])
    vs.append(f.x[1,0])
    f.predict(u=u)

plt.subplot(211)
plt.plot(xs)
plt.title('position')
plt.subplot(212)
plt.plot(vs)
plt.title('velocity');
plt.tight_layout(pad=1.5, h_pad=2.0)
```


position
20
10
0
0
5
10
15
20
25
30
35
velocity
5
4
3
0
5
10
15
20
25
30
35

附录 E：集合卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

> 我对集合滤波器还不够熟悉。为本书实现过一个，也让它运行起来了，但没有在实际工作中使用过。不同资料给出的方程形式略有差异；按这些资料直接实现时，我没有得到正常工作的滤波器，也可能是我做错了。不过，网上也有人提到采用与我类似的调整，才使滤波器正常运行。我对这个主题的理解尚不充分，但与其略过，不如坦诚呈现自己知道什么、还有哪些不懂。我希望以后能掌握它，写出更可靠、完整的介绍。本章末尾记录了目前的困惑与问题。既然这些资料让我感到困惑，其他读者也可能遇到同样的问题；把困惑写下来，也许有助于你避免重蹈覆辙。

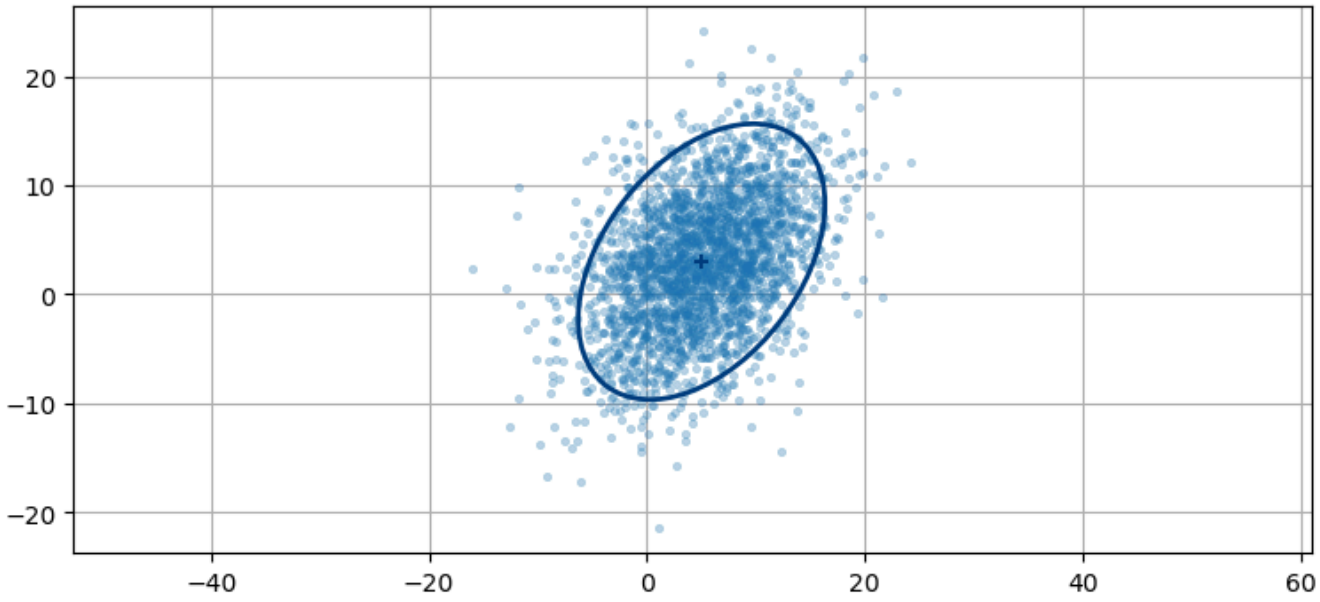
集合卡尔曼滤波器（EnKF）与前面介绍的无迹卡尔曼滤波器（UKF）十分相似。回忆一下，UKF 按确定性规则选取一组带权 sigma 点，再让它们通过非线性的状态转移函数和测量函数。随后计算这些点的均值与协方差，作为滤波器新的均值与协方差。这样得到的只是对真实结果的近似，因此通常并非最优，但实际中可以很准确。与扩展卡尔曼滤波器（EKF）相比，UKF 往往能给出更准确的估计，而且不必解析推导状态方程和测量方程的线性化形式。

集合卡尔曼滤波器的思路类似，但使用蒙特卡洛方法生成大量集合成员，而不是按确定规则选取少量 sigma 点。它起源于地球物理科学，用来处理海洋、大气等模型所需的超高维状态与系统。SIAM News [1] 有一篇介绍它在天气建模中发展历程的文章。开始时，在滤波器初始状态附近随机生成大量样本，其分布应具有滤波器设定的协方差 $\mathbf{P}$。在高斯初始假设下，对每个单独的标量分量，约 68% 的取值位于均值的一个标准差范围内，约 95% 位于两个标准差内。注意，这些比例不能直接用作任意维度椭球区域的联合覆盖率。下面考察二维情况，用 numpy.random.multivariate_normal() 从均值 (5,3)、协方差如下的多元正态分布中生成样本：

$$\begin{bmatrix}32 & 15\\15 & 40\end{bmatrix}$$

我画出了表示两个标准差的协方差椭圆，以此来展示这些点的分布情况。

```
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import multivariate_normal
from filterpy.stats import (covariance_ellipse,
                            plot_covariance_ellipse)

mean = (5, 3)
P = np.array([[32, 15],
              [15., 40.]])

x,y = multivariate_normal(mean=mean, cov=P, size=2500).T
plt.scatter(x, y, alpha=0.3, marker='.')
plt.axis('equal')

plot_covariance_ellipse(mean=mean, cov=P,
                        variance=2.**2,
                        facecolor='none')
```



算法

如前所述，初始化时，根据初始状态均值 $\mathbf{x}$ 与协方差 $\mathbf{P}$ 随机生成大量集合成员。此后的算法与 UKF 很相似：预测时，让各成员通过状态转移函数，并加入相应过程噪声；更新时，通过测量函数将成员映射到测量空间，并用随机扰动计入测量噪声。随后根据协方差计算卡尔曼增益，完成集合成员的更新。具体计算见下文方程。

UKF 与 EnKF 的主要区别已经提过：UKF 按确定性规则选择 sigma 点。上述算法还体现了另一点区别：UKF 在每次预测时重新生成 sigma 点，通过非线性函数后，再用无迹变换重建均值与协方差；EnKF 则持续传播最初生成的集合成员，不在每步按高斯近似重新生成它们。只需计算均值与协方差，作为滤波器的输出。

下面看滤波方程。与往常一样，这里省略通常使用的上下标，按算法执行顺序表达计算。其中，N 是集合成员数，χ 是集合成员组成的点集；代码沿用 sigmas 作为集合成员数组的变量名。

初始化步骤

$$\chi \sim \mathcal{N}(\mathbf{x}_0, \mathbf{P}_0)$$

按滤波器的初始均值与协方差，从相应分布抽取集合成员。代码可以写成：

```
N = 1000
sigmas = multivariate_normal(mean=x, cov=P, size=N)
```

预测步骤

$$\chi = f(\chi, \mathbf{u}) + v_Q$$
$$\mathbf{x} = \frac{1}{N}\sum_1^N \chi$$

第一行让所有集合成员通过用户提供的状态转移函数，再加入均值为零、协方差由 $\mathbf{Q}$ 指定的随机过程噪声。均值参数应使用零向量，避免额外平移集合。Python 代码可以写成：

```
for i, s in enumerate(sigmas):
sigmas[i] = fx(x=s, dt=0.1, u=0.)
sigmas += multivariate_normal(np.zeros_like(x), Q, N)
```

第二行计算集合成员的均值。使用 numpy.mean，可以简洁、高效地完成这一操作。

```
x = np.mean(sigmas, axis=0)
```

还可以选择计算集合成员相对于均值的样本协方差。算法的推进并不要求在此显式输出它，但它常有助于分析。方程为：

$$\mathbf{P} = \frac{1}{N-1}\sum_1^N [\chi - \mathbf{x}^-][\chi - \mathbf{x}^-]^\mathsf{T}$$

对每个集合成员，$\chi - \mathbf{x}^-$ 是一维数组表示的向量，因此可用 numpy.outer 计算外积 $[\chi - \mathbf{x}^-][\chi - \mathbf{x}^-]^\mathsf{T}$。Python 代码可以写成：

```
 P = 0
for s in sigmas:
P += outer(s-x, s-x)
P = P / (N-1)
```

更新步骤

更新时，先将集合成员代入测量函数，计算测量空间中的样本均值与协方差，以及状态与测量之间的样本互协方差；再计算卡尔曼增益，用它将测量残差映射为各成员的状态修正量。方程如下：

$$\chi_h = h(\chi, u)$$
$$\mathbf{z}_{mean} = \frac{1}{N}\sum_1^N \chi_h$$

$$\mathbf{P}_{zz} = \frac{1}{N-1}\sum_1^N [\chi_h - \mathbf{z}_{mean}][\chi_h - \mathbf{z}_{mean}]^\mathsf{T} + \mathbf{R}$$
$$\mathbf{P}_{xz} = \frac{1}{N-1}\sum_1^N [\chi - \mathbf{x}^-][\chi_h - \mathbf{z}_{mean}]^\mathsf{T}$$

$$\mathbf{K} = \mathbf{P}_{xz}\mathbf{P}_{zz}^{-1}$$
$$\chi = \chi + \mathbf{K}[\mathbf{z} - \chi_h + \mathbf{v}_R]$$

$$\mathbf{x} = \frac{1}{N}\sum_1^N \chi$$
$$\mathbf{P} = \mathbf{P} - \mathbf{K}\mathbf{P}_{zz}\mathbf{K}^\mathsf{T}$$

这与线性 KF 和 UKF 非常相似。我们逐行来分析吧。

第一行，

$$\chi_h = h(\chi, u),$$

将集合成员通过测量函数 h 映射到测量空间。将所得点记为 χ_h，以区别于原来的状态空间样本。Python 代码可以写成：

```
sigmas_h = h(sigmas, u)
```

下一行计算测量空间中集合成员的均值。

$$\mathbf{z}_{mean} = \frac{1}{N}\sum_1^N \chi_h$$

在 Python 中，我们这样编写：

```
z_mean = np.mean(sigmas_h, axis=0)
```

有了测量空间中的样本均值，就可以计算测量样本的协方差，以及状态样本与测量样本之间的互协方差。对应的两个方程为：

$$\mathbf{P}_{zz} = \frac{1}{N-1}\sum_1^N [\chi_h - \mathbf{z}_{mean}][\chi_h - \mathbf{z}_{mean}]^\mathsf{T} + \mathbf{R}$$
$$\mathbf{P}_{xz} = \frac{1}{N-1}\sum_1^N [\chi - \mathbf{x}^-][\chi_h - \mathbf{z}_{mean}]^\mathsf{T}$$

我们可以用 Python 来表达这个公式。

```
P_zz = 0
for sigma in sigmas_h:
s = sigma - z_mean
P_zz += outer(s, s)
P_zz = P_zz / (N-1) + R
P_xz = 0
for i in range(N):
P_xz += outer(self.sigmas[i] - self.x, sigmas_h[i] - z_mean)
P_xz /= N-1
```

卡尔曼增益可直接计算为 $\mathbf{K} = \mathbf{P}_{xz}\mathbf{P}_{zz}^{-1}$。

在 Python 中，是这样的。

```
K = P_xz @ inv(P_zz)
```

接下来，按下式更新每个集合成员：

$$\boldsymbol{\chi} = \boldsymbol{\chi} + \mathbf{K}[\mathbf{z} - \boldsymbol{\chi}_h + \mathbf{v}_R]$$

这里，$\mathbf{v}_R$ 是加入更新中的随机测量扰动。Python 中可按以下方式实现：

```
v_r = multivariate_normal([0]*dim_z, R, N)
for i in range(N):
sigmas[i] += K @ (z + v_r[i] - sigmas_h[i])
```

最后一步是重新计算该滤波器的均值和协方差。

```
 x = np.mean(sigmas, axis=0)
P = self.P - K @ P_zz @ K.T
```

实现与示例

我在 FilterPy 中实现了 EnKF，但从许多方面看，它只是教学演示。使用大量集合成员会使计算很慢，文献中也存在许多细节不同的算法变体。我主要是为了学习才编写它，没有将其用于实际问题，也无法就其擅长的大规模应用给出建议。因此，这里只讨论一个简单实现：跟踪一维运动目标，并与线性卡尔曼滤波器比较。本书已经多次设计过这种滤波器，下面不再详细解释每个步骤。状态向量为：

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

状态转移函数为

$$\mathbf{F} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

测量函数为：

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

EnKF 面向非线性问题，因此这个接口要求提供 Python 函数，分别实现状态转移与测量映射，而不是仅传入矩阵。本例是线性模型，相应函数可写为：

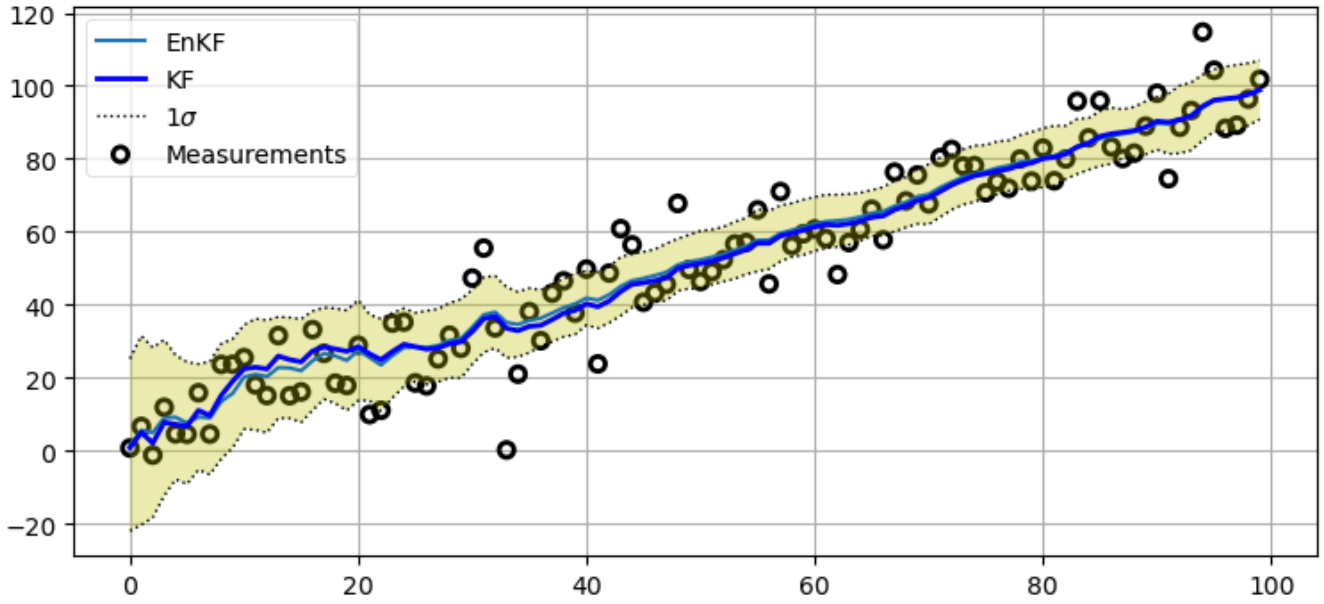
```
def hx(x):
return np.array([x[0]])
def fx(x, dt):
return F @ x
```

最后一点：EnKF 与 UKF 的代码都用一维数组表示 $\mathbf{x}$，而不是线性卡尔曼滤波器代码中使用的二维列矩阵。

不用多说了，以下就是代码。

```
import kf_book.book_plots as bp
from numpy.random import randn
from filterpy.kalman import EnsembleKalmanFilter as EnKF
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

np.random.seed(1234)

def hx(x):
    return np.array([x[0]])

def fx(x, dt):
    return F @ x

F = np.array([[1., 1.],[0., 1.]])

x = np.array([0., 1.])
P = np.eye(2) * 100.
enf = EnKF(x=x, P=P, dim_z=1, dt=1., N=20, hx=hx, fx=fx)

std_noise = 10.
enf.R *= std_noise**2
enf.Q = Q_discrete_white_noise(2, 1., .001)

kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = np.array([x]).T
kf.F = F.copy()
kf.P = P.copy()
kf.R = enf.R.copy()
kf.Q = enf.Q.copy()
kf.H = np.array([[1., 0.]])

measurements = []
results = []
ps = []
kf_results = []

zs = []
for t in range (0,100):
    # create measurement = t plus white noise
    z = t + randn()*std_noise
    zs.append(z)

    enf.predict()
    enf.update(np.asarray([z]))

    kf.predict()
    kf.update(np.asarray([[z]]))

    # save data
    results.append (enf.x[0])
    kf_results.append (kf.x[0,0])
    measurements.append(z)
```

```
    ps.append(3*(enf.P[0,0]**.5))

results = np.asarray(results)
ps = np.asarray(ps)

plt.plot(results, label='EnKF')
plt.plot(kf_results, label='KF', c='b', lw=2)
bp.plot_measurements(measurements)
plt.plot (results - ps, c='k',linestyle=':', lw=1, label='1$\sigma$')
plt.plot(results + ps, c='k', linestyle=':', lw=1)
plt.fill_between(range(100), results - ps, results + ps, facecolor='y', alpha=.3)
plt.legend(loc='best');
```



图中两种滤波器的初始结果略有不同，但很快便趋近于相同数值。有限集合的 EnKF 是近似滤波器，通常不能精确重现线性 KF 的最优解。这里故意将 N 设得很小，仅为 20，使近似误差容易显现。如果改用 2000 等更合理的成员数，在这幅图的尺度上就难以分辨两种输出。

作者尚未解决的问题

所有这些都应该被视为我自己的问题，而不是文献中已有的、悬而未决的问题。不过，我所引用的公式都是直接摘自该领域的权威来源的，而这些公式并没有解释那些不一致之处。

首先，在 Brown[2]中，所有的求和结果都乘以了 $\frac{1}{N}$，就像文中所述那样。

$$\hat{x} = \frac{1}{N}\sum_{i=1}^{N} \chi_k^{(i)}$$

在 Crassidis 的著作[3]中，同一个方程的表述如下（我会使用与 Brown 相同的符号表示法，不过 Crassidis 使用的符号有所不同）。

$$\hat{x} = \frac{1}{N-1}\sum_{i=1}^{N} \chi_k^{(i)}$$

协方差求和也需要归一化。Crassidis 采用 $N-1$ 作为分母，以得到无偏样本协方差估计。下式分别给出残差样本均值和样本方差：

$$\mu = \frac{1}{N}\sum_{i=1}^{N}[\tilde{z}(t_i) - \hat{z}(t_i)]$$

$$\sigma^2 = \frac{1}{N-1}\sum_{i=1}^{N}\{[\tilde{z}(t_i) - \hat{z}(t_i)] - \mu\}^2$$

然而，我看不出为什么计算均值时也要用 $N-1$ 作为分母。在我的实现中，均值采用 $N-1$ 作分母时滤波器不收敛，状态几乎只是跟随测量值，没有起到滤波作用。对于样本协方差，采用 N−1 作分母则具有无偏估计的依据。我的这一实现使用 $N-1$ 时能够工作，改用 N 时却未得到预期结果。这只是针对当前公式和实现的实验现象；不同算法变体的归一化约定与配套计算需要整体分析，不能据此断言其他写法均无法工作。

第二个问题涉及 $\mathbf{R}$ 矩阵。Brown 将 $\mathbf{R}$ 加到 $\mathbf{P}_{zz}$ 上，Crassidis 和其他一些资料则没有显式这样做。我在网上读到其他实现者的笔记，也认为加上 R 有帮助；这对我来说既合理又必要，因此这里采用了这种写法。

我的第三个问题与协方差 $\mathbf{P}$ 的计算方式有关。同样，Crassidis 和 Brown 给出的公式并不相同。我选择了 Brown 的公式，因为其计算结果符合我的预期（即 $\mathbf{P}$ 会随着时间的推移而趋于稳定），而且其形式与线性 KF 算法中的公式很相似。相比之下，Crassidis 的公式似乎很难让计算结果趋于稳定。

我的第四个问题与状态估计的更新有关。在 Brown 的模型中，我们有……

$$\chi = \chi + \mathbf{K}[\mathbf{z} - \mathbf{z}_{mean} + \mathbf{v}_R]$$

而在 Crassidis 中，情况则有所不同。

$$\chi = \chi + \mathbf{K}[\mathbf{z} - \chi_h + \mathbf{v}_R]$$

在我看来，Crassidis 方程相当合理。它所生成的滤波器，其性能与用于线性问题的线性 KF 滤波器类似。因此，我选择了这种公式。

我不愿断言哪本书有错，很可能只是我遗漏了使各组方程成立的条件。能确定的是，我按书面公式直接实现时，没有得到正常工作的滤波器。这里“正常工作”的标准，是在线性问题上基本重现线性卡尔曼滤波器的表现。参考网上的实现笔记并分析这些问题后，我选择了本章的实现，它看起来能够正常工作。我还没有深入阅读足够多的原始文献，而它们很可能能够解释这些差异。即使以后找到了协调各种写法的解释，我也希望以某种形式保留这段讨论：既然这些书让我困惑，其他读者也可能如此。

参考文献

- [1] Mackenzie, Dana. “Ensemble Kalman Filters Bring Weather Models Up to Date.” SIAM News, 36(8), October 2003. http://www.siam.org/pdf/news/362.pdf
- [2] Brown, Robert Grover, and Patrick Y. C. Hwang. Introduction to Random Signals and Applied Kalman Filtering, with MATLAB® Exercises and Solutions. Wiley, 2012.
- [3] Crassidis, John L., and John L. Junkins. Optimal Estimation of Dynamic Systems. CRC Press, 2011.

附录 G：非线性卡尔曼滤波器的设计

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

作者注：最初计划写一章非线性滤波器设计，比较不同方法，但尚不确定是否会完成。目前本章没有可供使用的完整内容，建议暂不阅读。

我们可以看到，卡尔曼滤波器在追踪球的位置方面表现得相当不错。不过，正如之前所解释的，这其实是个不恰当的例子：在真空中，我们可以以任意高的精度来预测物体的轨迹。因此，在这个例子中使用卡尔曼滤波器其实是没有必要的。

考虑空气阻力的卡尔曼滤波器

下面不再严格采用“第 1 步、第 2 步”的方式，而按解决实际工程问题时更自然的思路展开。此前的卡尔曼滤波器能够很好地跟踪真空中的球，但模型没有计入空气阻力。既然过程模型由 $\mathbf{F}$ 实现，就先从它入手。

从概念上说，$\mathbf{F}$ 完成的计算为：

$$x' = Fx$$

忽略空气阻力时，有：

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

这对应于相应的方程式。

$$\begin{aligned} x &= x + v_x \Delta t \\ v_x &= v_x \end{aligned}$$

$$\begin{aligned} y &= y + v_y \Delta t + \frac{a_y}{2}\Delta t^2 \\ v_y &= v_y + a_y \Delta t \\ a_y &= a_y \end{aligned}$$

这里先保留无空气阻力时的欧拉递推关系，加入阻力后的模型与离散化尚待完成：

$$
\begin{aligned}
x &= x + v_x \Delta t \\
v_x &= v_x
\end{aligned}
$$

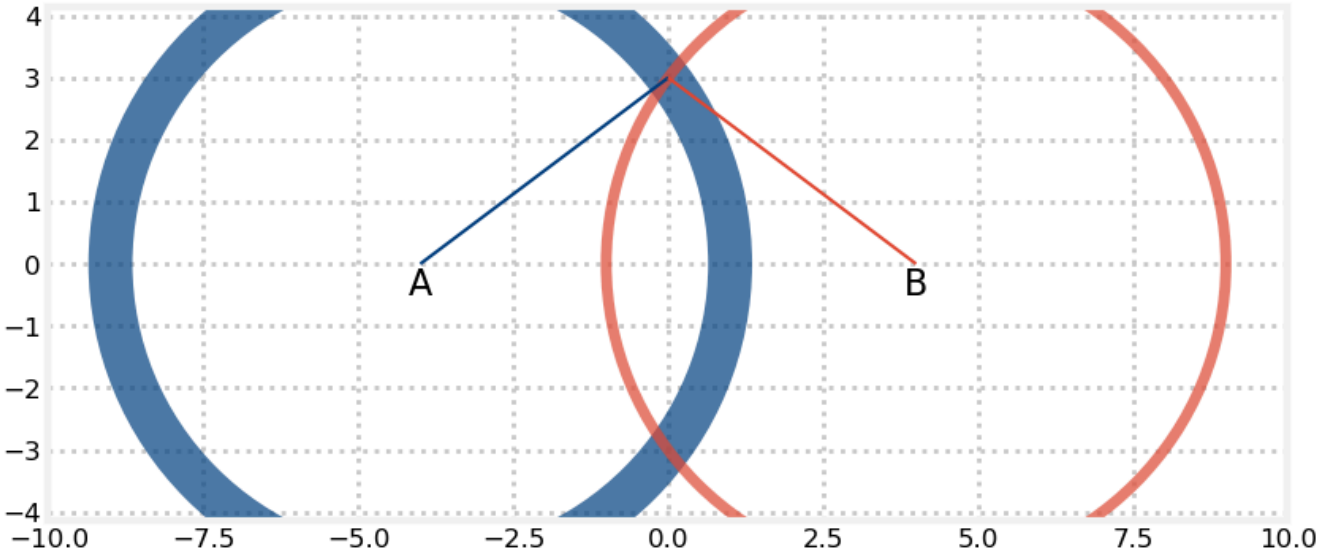
$$
\begin{aligned}
y &= y + v_y \Delta t + \frac{a_y}{2} \Delta t^2 \\
v_y &= v_y + a_y \Delta t \\
a_y &= a_y
\end{aligned}
$$

更贴近实际的二维位置传感器

前一个例子的位置传感器还不够贴近实际。通常，直接给出 (x,y) 坐标的设备并不是简单的原始传感器。例如，GPS 接收机可能已使用卡尔曼滤波生成输出。如果没有引入额外传感器或其他新信息，再串接一个卡尔曼滤波器未必能改善信号；效果还取决于接收机输出、所用模型及噪声的相关性。后面会讨论这一问题。

考虑如下布置：在开阔场地中，将两个发射器分别放在已知位置。每个发射器都发出可检测的信号，处理这些信号后，可得到含噪声的测距值，即自身到各发射器的距离。先看示意图。

```
import matplotlib.pyplot as plt

circle1=plt.Circle((-4, 0), 5, color='#004080',
                   fill=False, linewidth=20, alpha=.7)
circle2=plt.Circle((4, 0), 5, color='#E24A33',
                   fill=False, linewidth=5, alpha=.7)

fig = plt.gcf()
ax = fig.gca()

plt.axis('equal')
plt.xlim((-10, 10))
plt.ylim((-10, 10))

plt.plot ([-4, 0], [0, 3], c='#004080')
plt.plot ([4, 0], [0, 3], c='#E24A33')
plt.text(-4, -.5, "A", fontsize=16, horizontalalignment='center')
plt.text(4, -.5, "B", fontsize=16, horizontalalignment='center')

ax.add_artist(circle1)
ax.add_artist(circle2)
plt.show()
```



图中，红色发射机 A 位于 (-4,0)，蓝色发射机 B 位于 (4,0)。红、蓝圆环表示到各发射机的距离，环带以 1σ 的尺度示意测距不确定性；蓝色发射机的误差大于红色发射机。机器人最可能位于两圆环交叠处，图中用红、蓝线标出。两圆一般有两个交点，设计测量模型时还需处理这种位置歧义。

这是一种常见的传感器配置。飞机仍使用此类系统导航，称为 DME（测距设备）。如今 GPS 更为常见，但我曾参与过飞机项目，将这种测距传感器与 GPS、惯性导航系统（INS）、高度计等一同接入滤波器。后面会讨论多传感器融合；这里先研究这一简单配置。

第一步是选择状态变量。假设机器人沿直线以恒定速度运动。这个假设不太可能长期成立，但在短时间内可以接受。与前一个问题一样，我们希望跟踪位置与速度。因此：

$$\mathbf{x} = \begin{bmatrix} x \\ v_x \\ y \\ v_y \end{bmatrix}$$

下一步是设计状态转移函数。这与之前的问题类似，因此无需多言。

$$\mathbf{x}' = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}$$

下一步考虑控制输入。本例没有控制输入，因此设 $\mathbf{B} = \mathbf{0}$。

下一步设计测量关系。在线性情形中，它写作 $\mathbf{z} = \mathbf{H}\mathbf{x}$；本例的距离测量则需要用勾股定理构造非线性测量函数：

$$z_a = \sqrt{(x - x_A)^2 + (y - y_A)^2} + v_a$$

$$z_b = \sqrt{(x - x_B)^2 + (y - y_B)^2} + v_b$$

其中， $\boldsymbol{v_a}$ 和 $\boldsymbol{v_b}$ 属于白噪声。

我们面临着一个紧迫的问题：卡尔曼滤波器是为处理线性方程而设计的，而当前的问题显然属于非线性问题。在接下来的章节中，我们将探讨几种处理非线性问题的有效方法。不过目前，我们可以采取一种更简单的方法：如果我们知道机器人的大致位置，那么就可以以该点为基准，将相关方程线性化处理。虽然我现在可以详细阐述这种方法的数学原理，但在这里，我先通过一个具体的例子来说明这一方法的运用方式。

不能用一个恒定矩阵 $\mathbf{H}$ 在全局表示这个非线性测量关系；这里需要计算局部线性化的测量矩阵 $\mathbf{H}$。它是测量函数对机器人状态 $\mathbf{x}$ 的雅可比矩阵。$\mathbf{H}$ 描述状态发生小变化时测量值的一阶变化；每个时间步都要在当前估计位置重新计算 $\mathbf{H}$。其一般形式为：

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial x_2} & \cdots \\ \frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \end{bmatrix}$$

让我们先来计算第一个偏导数。我们需要找出的就是这个偏导数。

$$\frac{\partial}{\partial x_r}\sqrt{(x_r - x_A)^2 + (y_r - y_A)^2}$$

机器人与发射器距离非零时，按链式法则计算如下。这里用 x_r、y_r 表示机器人坐标，用 x_A、y_A 表示发射器 A 的坐标：

$$\begin{aligned} \frac{\partial h_1}{\partial x_r} &= \frac{\partial}{\partial x_r}\left[(x_r - x_A)^2 + (y_r - y_A)^2\right]^{1/2} \\ &= \frac{1}{2}\left[(x_r - x_A)^2 + (y_r - y_A)^2\right]^{-1/2} 2(x_r - x_A) \\ &= \frac{x_r - x_A}{\sqrt{(x_r - x_A)^2 + (y_r - y_A)^2}} \end{aligned}$$

我们继续对这两个距离方程关于 $\boldsymbol{x}$ 、 $\boldsymbol{y}$ 、 $\boldsymbol{dx}$ 和 $\boldsymbol{dy}$ 的偏导数进行计算，最终得到以下结果：

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{x_r - x_A}{\sqrt{(x_r - x_A)^2 + (y_r - y_A)^2}} & 0 & \frac{y_r - y_A}{\sqrt{(x_r - x_A)^2 + (y_r - y_A)^2}} & 0 \\ \frac{x_r - x_B}{\sqrt{(x_r - x_B)^2 + (y_r - y_B)^2}} & 0 & \frac{y_r - y_B}{\sqrt{(x_r - x_B)^2 + (y_r - y_B)^2}} & 0 \end{bmatrix}$$

那确实很麻烦，而这些其实都是非常简单的方程式而已。对于更复杂的系统来说，计算雅可比矩阵可能极其困难，甚至无法实现。不过，利用 SymPy 模块，我们可以轻松地让 Python 来完成这项工作[1]。SymPy 是一个用于符号数学的 Python 库。它的功能非常强大，超出了本书的讨论范围。不过，它能够进行代数运算、积分和微分计算、求解微分方程等等。我们将利用它来计算雅可比矩阵！

先看一个简单例子：导入 SymPy，初始化美化输出功能，以 LaTeX 显示数学表达式，然后声明供 SymPy 使用的符号。

```
import sympy
from sympy import init_printing
init_printing(use_latex='mathjax')

phi, x = sympy.symbols('\phi, x')
phi
```

$$\phi$$

注意，我们为符号 phi 使用了 LaTeX 表达式。这不是必需的，但这样可在输出中按 LaTeX 显示。现在做些运算：$\sqrt{\phi}$ 的导数是什么?

```
sympy.diff('sqrt(phi)')
```

$$\frac{1}{2\sqrt{\phi}}$$

我们可以对方程进行因式分解。

```
sympy.factor('phi**3 -phi**2 + phi - 1')
```

$$(\phi - 1)\left(\phi^2 + 1\right)$$

SymPy 拥有许多出色的功能。虽然我很乐意逐一试用这些功能，但在这里我们无法一一详述。不过，我们可以先来计算一下雅可比矩阵吧。

```
from sympy import symbols, Matrix
phi = symbols('\phi')
phi

x, y, xa, xb, ya, yb, dx, dy = symbols('x y x_a x_b y_a y_b dx dy')

H = Matrix([[sympy.sqrt((x-xa)**2 + (y-ya)**2)],
            [sympy.sqrt((x-xb)**2 + (y-yb)**2)]])

state = Matrix([x, dx, y, dy])
H.jacobian(state)
```

$$\begin{bmatrix} \frac{x-x_a}{\sqrt{(x-x_a)^2+(y-y_a)^2}} & 0 & \frac{y-y_a}{\sqrt{(x-x_a)^2+(y-y_a)^2}} & 0 \\ \frac{x-x_b}{\sqrt{(x-x_b)^2+(y-y_b)^2}} & 0 & \frac{y-y_b}{\sqrt{(x-x_b)^2+(y-y_b)^2}} & 0 \end{bmatrix}$$

具体而言，元素 (0,0) 等于机器人横坐标与发射器 A 横坐标之差，除以机器人到 A 的距离。元素 (0,2) 对纵坐标作同样计算；第二行则使用发射器 B 的坐标。速度分量对应的元素为零，因为瞬时距离测量不直接依赖速度。

这个矩阵的元素随机器人位置改变，因此它不再是常量，必须在滤波器的每个时间步重新计算。

这些元素是相对坐标分量与距离之比，也可以写成三角函数形式。将角度定义为由机器人指向发射器的方位角，则有：

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{bmatrix} -\cos\theta_A & 0 & -\sin\theta_A & 0 \\ -\cos\theta_B & 0 & -\sin\theta_B & 0 \end{bmatrix}$$

目前求得的不是能够在全局表示非线性测量关系的矩阵 $\mathbf{H}$，而是局部线性化矩阵。用 $\boldsymbol{\delta z}$ 表示测量的小变化时，它与雅可比矩阵 $\mathbf{H}$ 以及状态的小变化相关。在全局线性模型中，可以用 $\mathbf{Hx}$ 生成状态估计 $\mathbf{x}$ 对应的预测测量；本例中的 $\mathbf{H}$ 则应作用于状态小偏差，以近似测量变化，不能直接代替完整的非线性测量函数。

这里选择以 $\mathbf{x}$ 相对于标称状态的偏差作为滤波状态，因此需要重新设计状态定义。这是误差状态建模的一种选择；普通 EKF 也可以直接估计位置与速度，并分别使用非线性测量函数和它的雅可比矩阵。

> 请注意，这在卡尔曼滤波器的设计过程中是再正常不过的现象。教科书里通常把这类例子当作理所当然的事情来介绍，好像“状态变量应该是速度而非位置”这种结论是显而易见的。也许，当你做了足够多的这类题目后，这一点确实会变得显而易见。但到那时，你还要读教科书干什么呢？我自己经常反复阅读各种讲解资料，试图理解为什么作者会做出那样的选择。最终我发现，这些选择都是因为后文中的某些因素所导致的。我的讲解可能比较冗长，但它反映了实际设计滤波器时的真实情况：你会做出看似合理的决策，但随着设计的深入，你会发现一些新的情况，从而不得不重新考虑之前的决策。因此，我打算放弃那种按步骤来讲解的方法，因为现实中的问题往往没那么简单。

如果状态只保存速度而不保存位置，就无法仅凭该状态得到绝对位置。围绕标称轨迹线性化时，需要先设定位置和运动方向，再用估计的状态偏差修正轨迹。两个测距圆环通常有两个候选交叠区域；若发射器相距很近，含噪声的交叠区域还可能呈细长月牙形。因此，仅凭当前距离测量不能消除所有位置歧义，还需要初始位置、运动连续性或其他传感器信息。这一误差状态模型的完整设计尚未完成，下面仅保留其中的中间关系。

先考察速度恒定、加速度为零时的运动关系：

$$
\begin{aligned}
\dot{x}' &= \dot{x} \\
\ddot{x}' &= 0 \\
\dot{y}' &= \dot{y} \\
\ddot{y}' &= 0
\end{aligned}
$$

暂以横向速度、横向加速度、纵向速度、纵向加速度为状态时，上述关系对应下面的矩阵。这个临时状态不能直接与前面的位置测量雅可比配用；完整误差状态模型尚未整理。

$$
\mathbf{F} = \begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0
\end{bmatrix}
$$

最后还需调整输入的测量量。$\mathbf{Hx}$ 此时近似计算相对于标称位置的测量变化，因此输入不再是到 A、B 的原始距离，而应是实际测量距离与标称位置所预测距离之差。

这里有许多内容需要理解，所以我们逐段分析代码。首先定义一个函数，用来计算每个时间步的 $\frac{\partial \mathbf{h}}{\partial \mathbf{x}}$。

```
from math import sin, cos, atan2

def H_of(pos, pos_A, pos_B):
    """ Given the position of our object at 'pos' in 2D, and two
    transmitters A and B at positions 'pos_A' and 'pos_B', return
    the partial derivative of H
    """

    theta_a = atan2(pos_a[1] - pos[1], pos_a[0] - pos[0])
    theta_b = atan2(pos_b[1] - pos[1], pos_b[0] - pos[0])

    return np.array([[0, -cos(theta_a), 0, -sin(theta_a)],
                     [0, -cos(theta_b), 0, -sin(theta_b)]])
```

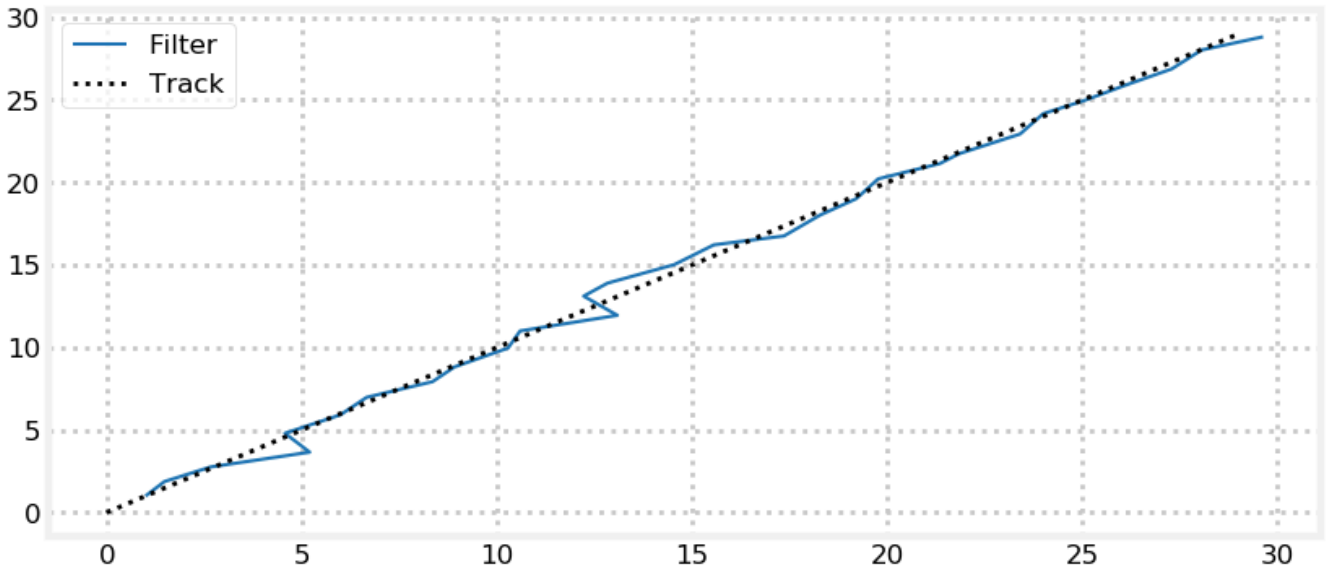
现在，我们需要创建这个模拟传感器。

```
from numpy.random import randn

class DMESensor(object):
    def __init__(self, pos_a, pos_b, noise_factor=1.0):
        self.A = pos_a
        self.B = pos_b
        self.noise_factor = noise_factor

    def range_of(self, pos):
        """ returns tuple containing noisy range data to A and B
        given a position 'pos'
        """

        ra = math.sqrt((self.A[0] - pos[0])**2 + (self.A[1] - pos[1])**2)
        rb = math.sqrt((self.B[0] - pos[0])**2 + (self.B[1] - pos[1])**2)

        return (ra + randn()*self.noise_factor,
                rb + randn()*self.noise_factor)
```

最后编写卡尔曼滤波代码。将两个发射器分别放在 x=−100 与 x=100，纵坐标均为 y=−20，以获得较好的测距定位几何构型。机器人从 (0,0) 出发，每个时间步位移为 (1,1)。

```
import kf_book.book_plots as bp
from filterpy.kalman import KalmanFilter
import math
import numpy as np

pos_a = (100, -20)
pos_b = (-100, -20)

f1 = KalmanFilter(dim_x=4, dim_z=2)

f1.F = np.array ([[0, 1, 0, 0],
                  [0, 0, 0, 0],
                  [0, 0, 0, 1],
                  [0, 0, 0, 0]], dtype=float)

f1.R *= 1.
f1.Q *= .1

f1.x = np.array([[1, 0, 1, 0]], dtype=float).T
f1.P = np.eye(4) * 5.

# initialize storage and other variables for the run
count = 30
xs, ys = [], []
pxs, pys = [], []

# create the simulated sensor
d = DMESensor(pos_a, pos_b, noise_factor=3.)
```

```
# pos will contain our nominal position since the filter does not
# maintain position.
pos = [0, 0]

for i in range(count):
    # move (1,1) each step, so just use i
    pos = [i, i]

    # compute the difference in range between the nominal track
    # and measured ranges
    ra,rb = d.range_of(pos)
    rx,ry = d.range_of((pos[0] + f1.x[0, 0], pos[1] + f1.x[2, 0]))
    z = np.array([[ra - rx], [rb - ry]])

    # compute linearized H for this time step
    f1.H = H_of (pos, pos_a, pos_b)

    # store stuff so we can plot it later
    xs.append(f1.x[0, 0]+i)
    ys.append(f1.x[2, 0]+i)
    pxs.append(pos[0])
    pys.append(pos[1])

    # perform the Kalman filter steps
    f1.predict()
    f1.update(z)

bp.plot_filter(xs, ys)
bp.plot_track(pxs, pys)
plt.legend(loc=2)
plt.show()
```



线性化卡尔曼滤波器

既然我们已经了解了将卡尔曼滤波器线性化的具体示例，现在我们就能更深入地理解其中的数学原理了。

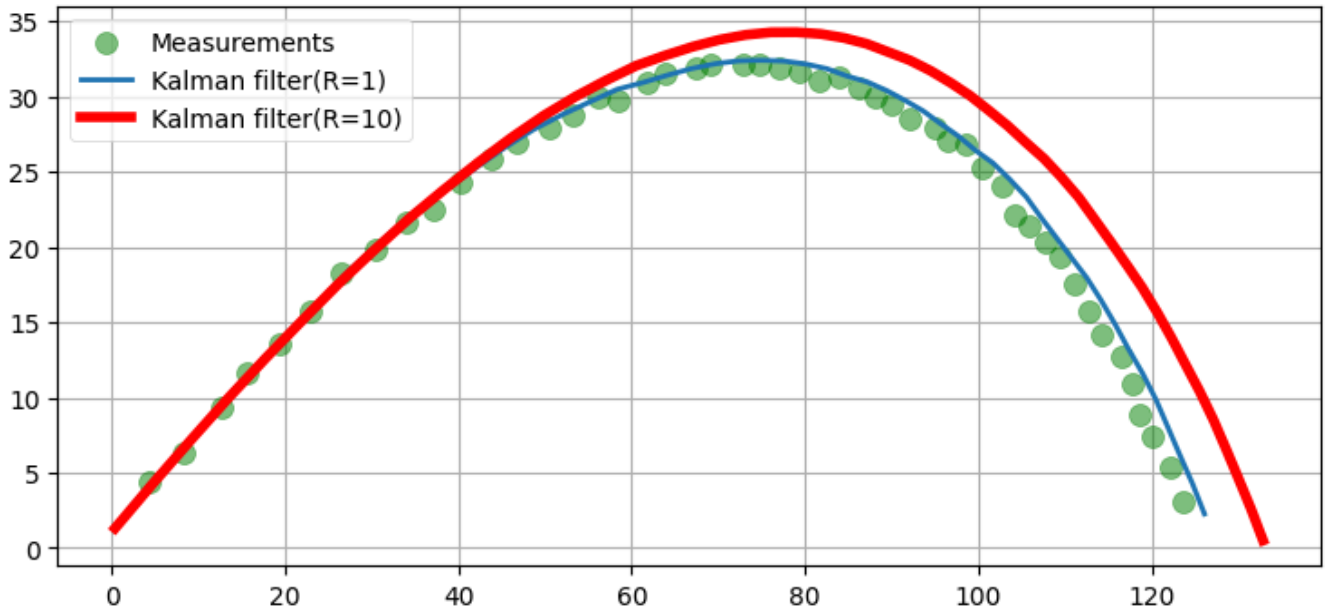
首先，设有函数 $\mathbf{f}$。

示例：下落的球体

作者注：暂时忽略此部分。

在“卡尔曼滤波器的设计”一章中，先研究了真空中球的跟踪，再考虑空气阻力。卡尔曼滤波器在真空中表现很好，但在大气模型中偏离了球的真实轨迹。下面看看输出；为避免重复那一章的大量代码，我将其放在 ekf_internal.py 文件中。

```
import kf_book.ekf_internal as ekf
ekf.plot_ball()
```



我们可以通过将 Q 的值设置得很大，从而强制卡尔曼滤波器去跟踪球的位置。这样一来，滤波器会怀疑自己的预测结果，进而调整卡尔曼增益 K 的数值，使其更倾向于信任实际的测量数据。不过，这种做法并不妥当。如果卡尔曼滤波器能够正确地预测过程状态，我们就没必要通过谎报“存在实际并不存在的误差”来“欺骗”滤波器。在某些情况下，这种做法或许可行，但总体而言，卡尔曼滤波器的性能将会受到影响。

回想一下在“卡尔曼滤波器的设计”这一章中提到的内容：加速度为

$$a_x = (0.0039 + \frac{0.0058}{1+\exp[(v-35)/5]}) * v * v_x$$
$$a_y = (0.0039 + \frac{0.0058}{1+\exp[(v-35)/5]}) * v * v_y - g$$

在研究这个课题的过程中，这些方程式会让人觉得非常难以处理。因此，目前我先采用一个更简单的一维模型来进行分析。在这个模型中，我使用了经过简化的加速度公式，该公式没有考虑阻力系数的非线性特性。

$$\ddot{y} = \frac{0.0034ge^{-y/20000}\dot{y}^2}{2\beta} - g$$
$$\ddot{x} = \frac{0.0034ge^{-x/20000}\dot{x}^2}{2\beta}$$

这里， β 表示弹道系数。数值越大，表示空气阻力越小。

这仍是非线性方程，需要在当前估计状态附近线性化。若状态按位置、速度的顺序排列，则状态 $\mathbf{x}$ 的微小变化与状态导数的微小变化通过雅可比矩阵联系如下：

$$\begin{bmatrix}\delta\dot{x}\\ \delta\ddot{x}\\ \delta\dot{y}\\ \delta\ddot{y}\end{bmatrix} = \begin{bmatrix}\frac{\partial\dot{x}}{\partial x} & \frac{\partial\dot{x}}{\partial\dot{x}} & \frac{\partial\dot{x}}{\partial y} & \frac{\partial\dot{x}}{\partial\dot{y}}\\ \frac{\partial\ddot{x}}{\partial x} & \frac{\partial\ddot{x}}{\partial\dot{x}} & \frac{\partial\ddot{x}}{\partial y} & \frac{\partial\ddot{x}}{\partial\dot{y}}\\ \frac{\partial\dot{y}}{\partial x} & \frac{\partial\dot{y}}{\partial\dot{x}} & \frac{\partial\dot{y}}{\partial y} & \frac{\partial\dot{y}}{\partial\dot{y}}\\ \frac{\partial\ddot{y}}{\partial x} & \frac{\partial\ddot{y}}{\partial\dot{x}} & \frac{\partial\ddot{y}}{\partial y} & \frac{\partial\ddot{y}}{\partial\dot{y}}\end{bmatrix}\begin{bmatrix}\delta x\\ \delta\dot{x}\\ \delta y\\ \delta\dot{y}\end{bmatrix}$$

这些方程没有同时依赖 x 与 y，因此相应的交叉偏导数为零。又有 $\frac{\partial\dot{x}}{\partial x}=0$ 与 $\frac{\partial\dot{x}}{\partial\dot{x}}=1$，于是连续时间雅可比矩阵中对应横向位置和速度的前两行为下式。完整的四维离散时间滤波模型尚待整理。

$$\mathbf{A}_x = \begin{bmatrix}0 & 1 & 0 & 0\\ -\frac{0.0034e^{-x/20000}\dot{x}^2 g}{40000\beta} & \frac{0.0034e^{-x/20000}\dot{x}g}{\beta} & 0 & 0\end{bmatrix}$$

$$\ddot{x} = -\frac{1}{2}C_d\rho A\dot{x}$$

$$\ddot{y} = -\frac{1}{2}C_d\rho A\dot{y} - g$$

```
from sympy.abc import *
from sympy import *

init_printing(pretty_print=True, use_latex='mathjax')

x1 = (0.0034*g*exp(-x/22000)*((x)**2))/(2*b) - g

x2 = (a*g*exp(-x/c)*(Derivative(x)**2))/(2*b) - g

#pprint(x1)
#pprint(Derivative(x)*Derivative(x,n=2))
#pprint(diff(x2, x))
```

这种方法存在多种问题。首先，线性化通常不是精确的。更重要的是，线性化所围绕的不是目标真实轨迹，而是滤波器估计的轨迹。我们期望估计接近真实值，却没有保证。因此，若当前估计较差，在错误的状态附近线性化可能使下一步估计更差，进而引起发散。这正是卡尔曼滤波设计中需要经验与分析的地方：围绕估计点线性化并不能保证稳定性，大量文献专门研究这一问题。另一个困难是，若采用解析方法推导线性化，对于某些系统，雅可比表达式可能难以求得；即便能够求出，计算代价也可能很高。

参考文献

[1] http://sympy.org

附录 H：最小二乘滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

作者注：这些内容从 g-h 滤波一章中移出，因为不适合放在那里。这一章还不适合阅读——我还没有把它写完。

在那一章开头附近，我用 `numpy.polyfit()` 对体重测量数据拟合了一条直线。它通过“最小二乘拟合”，用 n 次多项式拟合数据。这与 g-h 滤波器有什么不同?

这要视具体情况而定。我们最终会学到，在某些条件下，卡尔曼滤波从最小二乘拟合的角度看是最优的。不过，`polyfit()` 拟合的是多项式，而非任意曲线，方法是最小化下式：

$$E = \sum_{j=0}^{k} |p(x_j) - y_j|^2$$

我假定自己的体重每天恒定增加 1 磅。因此，使用 $\boldsymbol{n = 1}$ 的多项式，也就是直线，拟合结果与真实增重非常接近。但当然，没有人会一直只增重或只减重，我们的体重会波动。对较长的数据序列使用 `polyfit()`，结果会很差。相反，g-h 滤波器能应对变化率的改变，其中 $\boldsymbol{h}$ 项控制滤波器响应这些变化的速度。如果我们先增重，稳定一段时间，再减重，滤波器会自动跟踪变化。除非这种增减能很好地用多项式表示，否则 `polyfit()` 做不到这一点。

即便数据能用 n 次多项式拟合，这类滤波器还有一个优点：它是*递推*的。也就是说，只需知道上一时刻的估计和变化率，就能计算本时刻的估计。相反，查看 `polyfit()` 的实现会发现，它需要取得全部数据后才能给出答案。因此，`polyfit()` 这样的算法不适合实时数据滤波。20 世纪 60 年代卡尔曼滤波问世时，计算机速度很慢，内存极其有限，根本无法存储例如飞机惯性导航系统的数千次读数，也不能在为导航提供准确、及时信息所要求的短时间内处理这些数据。

直到 20 世纪中叶，这类滤波主要采用各种形式的最小二乘估计。例如，美国国家航空航天局的阿波罗计划使用地面网络跟踪指令与服务舱（CSM）和登月舱（LM）。地面系统用数分钟收集测量值，将数据成批汇总，再缓慢计算答案。1960 年，NASA 艾姆斯研究中心的 Stanley Schmidt 认识到鲁道夫·卡尔曼那篇开创性论文的价值，邀请他到艾姆斯。Schmidt 把卡尔曼的工作应用于 CSM 和 LM 的机载导航系统，并将其称为“卡尔曼滤波器”。[1] 此后，人们很快转向这种更快的递推滤波器。

卡尔曼滤波只需存储上一次估计和少量相关参数，便能用相对较少的计算生成下一次估计。今天的内存和处理能力非常充裕，这个优势不像当时那么关键。但在那个年代，卡尔曼滤波不仅因其数学性质而成为重大突破，还因为它能勉强在当时的硬件上运行。

这一主题远比这段简短讨论所呈现的深刻。全书后面还会多次讨论这些问题。

附录 I：性能的解析评估

本附录尚未撰写。

```
#format the book
%matplotlib inline
import book_format
book_format.set_style()
```